MACHINE LEARNING

AI · ML · DS / REVISION NOTES

COURSERA · DEEPLEARNING.AI & STANFORD ONLINE · ANDREW NG

Advanced Learning Algorithms

Machine Learning Specialization, Course 2 — neural networks, ML system design, decision trees

Volume 1 of 2 · Sections 1–47

47 SECTIONS

466 FIGURES

431 PAGES

Compiled October 2026

github.com/ankitgupta1729/AI-ML-DS

Contents

LECTURE 1 Neural Networks 4

LECTURE 2 Neurons in the Brain 6

LECTURE 3 Demand Prediction 17

LECTURE 4 Face Recognition 35

LECTURE 5 Neural Network Layer 45

LECTURE 6 More Complex Neural Network 58

LECTURE 7 Handwritten Digit Recognition 69

LECTURE 8 Coffee Roasting 75

LECTURE 9 Note About Numpy Arrays 81

LECTURE 10 Building a Neural Network Architecture 90

LECTURE 11 Implement Forward Prop 99

LECTURE 12 Forward Prop in NumPy 106

LECTURE 13 Neural Network and the Brain 113

LECTURE 14 Vectorized Implementation and Matrix Multiplication 122

LECTURE 15 Dot Products 126

LECTURE 16 Transpose 133

LECTURE 17 Dense Layer Vectorized 143

LECTURE 18 Train a Neural Network in TensorFlow 152

LECTURE 19 Model Training Steps 156

LECTURE 20 Demand Prediction Example 168

LECTURE 21 Hidden Layer 174

LECTURE 22 Linear Example 188

LECTURE 23 Multiclass Classification Example 198

LECTURE 24 Logistic Regression 201

LECTURE 25 MNIST with Softmax 212

LECTURE 26 Numerical Roundoff Errors 219

LECTURE 27 Multi-label Classification 230

LECTURE 28 Adam Algorithm Intuition 233

LECTURE 29 Convolutional Neural Network 246
LECTURE 30 More Derivative Examples 259
LECTURE 31 Forward Prop 278
LECTURE 32 Neural Network Example 292
LECTURE 33 Debugging a Learning Algorithm 300
LECTURE 34 Train/test Procedure for Linear Regression 304
LECTURE 35 Model Selection (Choosing a Model) 314
LECTURE 36 Diagnosing Bias and Variance 325
LECTURE 37 Lambda 331
LECTURE 38 Bias/variance Examples 341
LECTURE 39 High Bias 350
LECTURE 40 Debugging a Learning Algorithm 363
LECTURE 41 Neural Networks and Bias Variance 367
LECTURE 42 Iterative Loop of ML Development 379
LECTURE 43 Error Analysis 387
LECTURE 44 Data Augmentation by Introducing Distortions 393
LECTURE 45 Transfer Learning 408
LECTURE 46 Full Cycle of a Machine Learning Project 417
LECTURE 47 Adverse Use Cases 424

LECTURE 1

Neural Networks

Welcome to Course 2 of this machine learning specialization. In this course, you'll learn about neural networks, also called deep learning algorithms, as well as decision trees. These are some of the most powerful and widely used machine learning algorithms and you'd get to implement them and get them to work for yourself. One of the things you see also in this course is practical advice on how to build machine learning systems. This part of the material is quite unique to this course. When you're building a practical machine learning system, there are a lot of decisions you have to make, such as should you spend more time collecting data or should you buy a much bigger GPU to build a much bigger neural network?

Even today, when I visit a leading tech company and talk to the team working there on a machine learning application, unfortunately, sometimes I look at what they've been doing for the last six months and go, gee, someone could have told you maybe even six months ago that that approach wasn't going to work that well. With some of the tips that you learn in this course, I hope that you'll be one or the ones to not waste those six months, but instead, be able to make more systematic and better decisions about how to build practical working machine learning applications. With that, let's dive in. In detail, this is what you see in the four weeks of this course.

In Week 1, we'll go over neural networks and how to carry out inference or prediction. If you were to go to the Internet and download the parameters of a neural network that someone else had trained and whose parameters that posted on the Internet, then to use that neural network to make predictions would be called inference, and you learned how neural networks work, and how to do inference in this week. Next week, you'll learn how to train your own neural network. In particular, if you have a training set of labeled examples, X and Y, how do you train the parameters of a neural network for yourself? In the third week, we'll then go into practical advice for building machine learning systems and I'll share with you some tips that I think even highly paid engineers building machine learning systems very successfully today don't really always manage to consistently apply and I think that will help you build systems yourself efficiently and quickly.

Then in the final week of this course, you learn about decision trees.

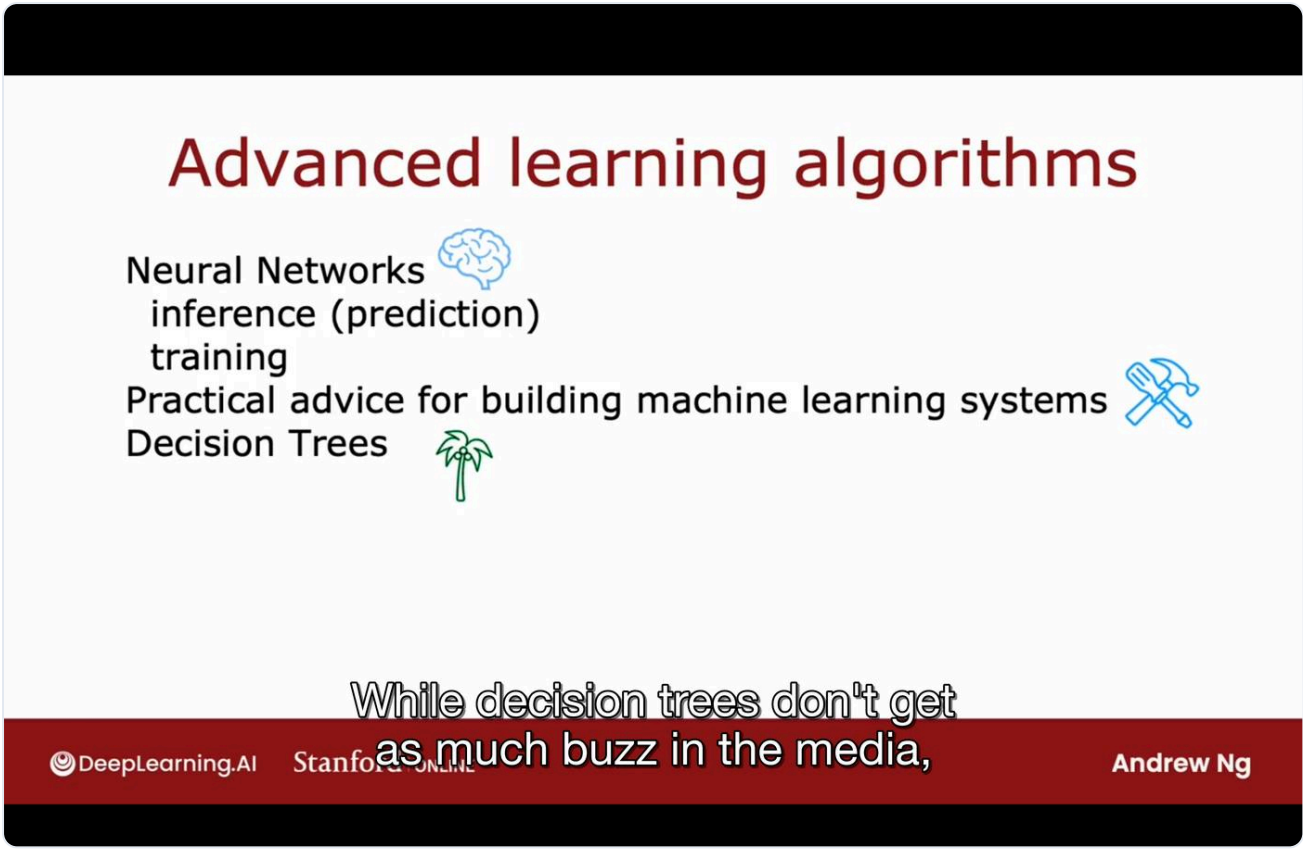


FIGURE 1.1 | Neural Networks

While decision trees don't get as much buzz in the media, there's local less hype about decision trees compared to neural networks. They are also one of the widely used and very powerful learning algorithms that I think there's a good chance you end up using yourself if you end up building an application. With that, let's jump into neural networks and we're going to start by taking a quick look at how the human brain, that is how the biological brain works. Let's go on to the next video.

LECTURE 2

Neurons in the Brain

When neural networks were first invented many decades ago, the original motivation was to write software that could mimic how the human brain or how the biological brain learns and thinks. Even though today, neural networks, sometimes also called artificial neural networks, have become very different than how any of us might think about how the brain actually works and learns. Some of the biological motivations still remain in the way we think about artificial neural networks or computer neural networks today. Let's start by taking a look at how the brain works and how that relates to neural networks. The human brain, or maybe more generally, the biological brain demonstrates a higher level or more capable level of intelligence and anything else would be on the bill so far.

So neural networks has started with the motivation of trying to build software to mimic the brain. Work in neural networks had started back in the 1950s, and then it fell out of favor for a while. Then in the 1980s and early 1990s, they gained in popularity again and showed tremendous traction in some applications like handwritten digit recognition, which were used even backed then to read postal codes for writing mail and for reading dollar figures in handwritten checks. But then it fell out of favor again in the late 1990s. It was from about 2005 that it enjoyed a resurgence and also became re-branded little bit with deep learning. One of the things that surprised me back then was deep learning and neural networks meant very similar things.

But maybe under-appreciated at the time that the term deep learning, just sounds much better because it's deep and this learning. So that turned out to be the brand that took off in the last decade or decade and a half.

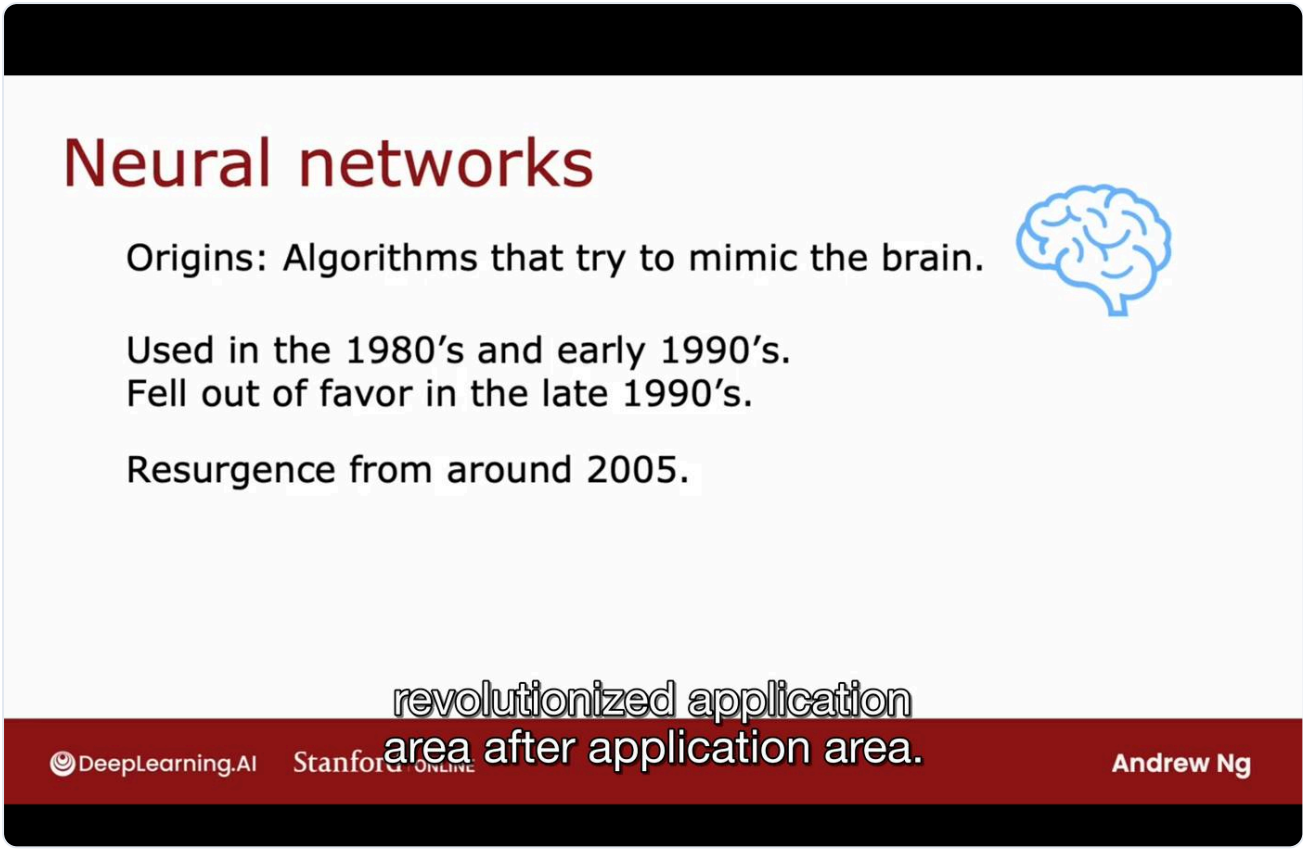


FIGURE 2.1 | Neural Networks

Since then, neural networks have revolutionized application area after application area.

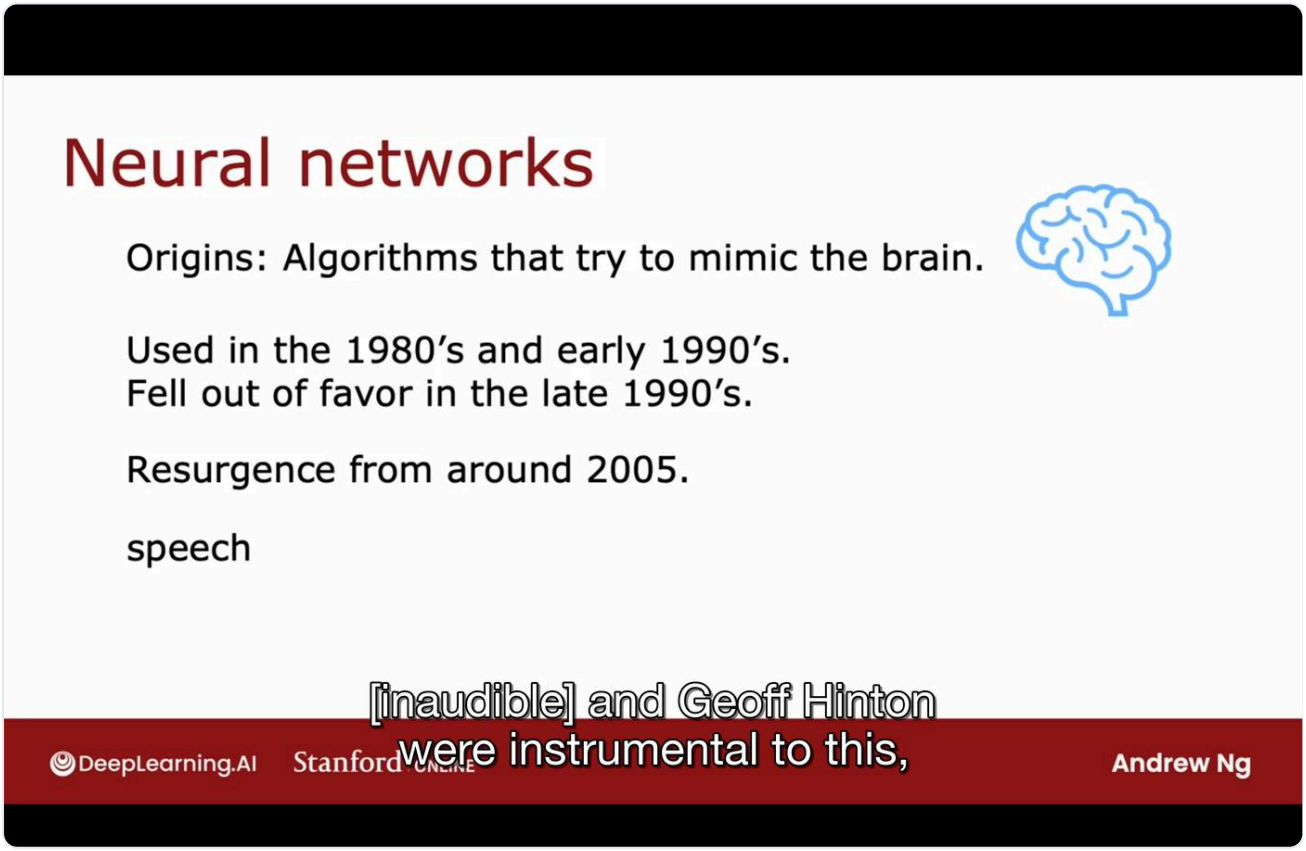


FIGURE 2.2 | Neural Networks

I think the first application area that modern neural networks or deep learning, had a huge impact on was probably speech recognition, where we started to see much better speech recognition systems due to modern deep learning and authors such as [inaudible] and Geoff Hinton were instrumental to this, and then it started to make inroads into computer vision. Sometimes people still speak of the ImageNet moments in 2012, and that was maybe a bigger splash where then [inaudible] draw their imagination and had a big impact on computer vision. Then the next few years, it made us inroads into texts or into natural language processing, and so on and so forth.

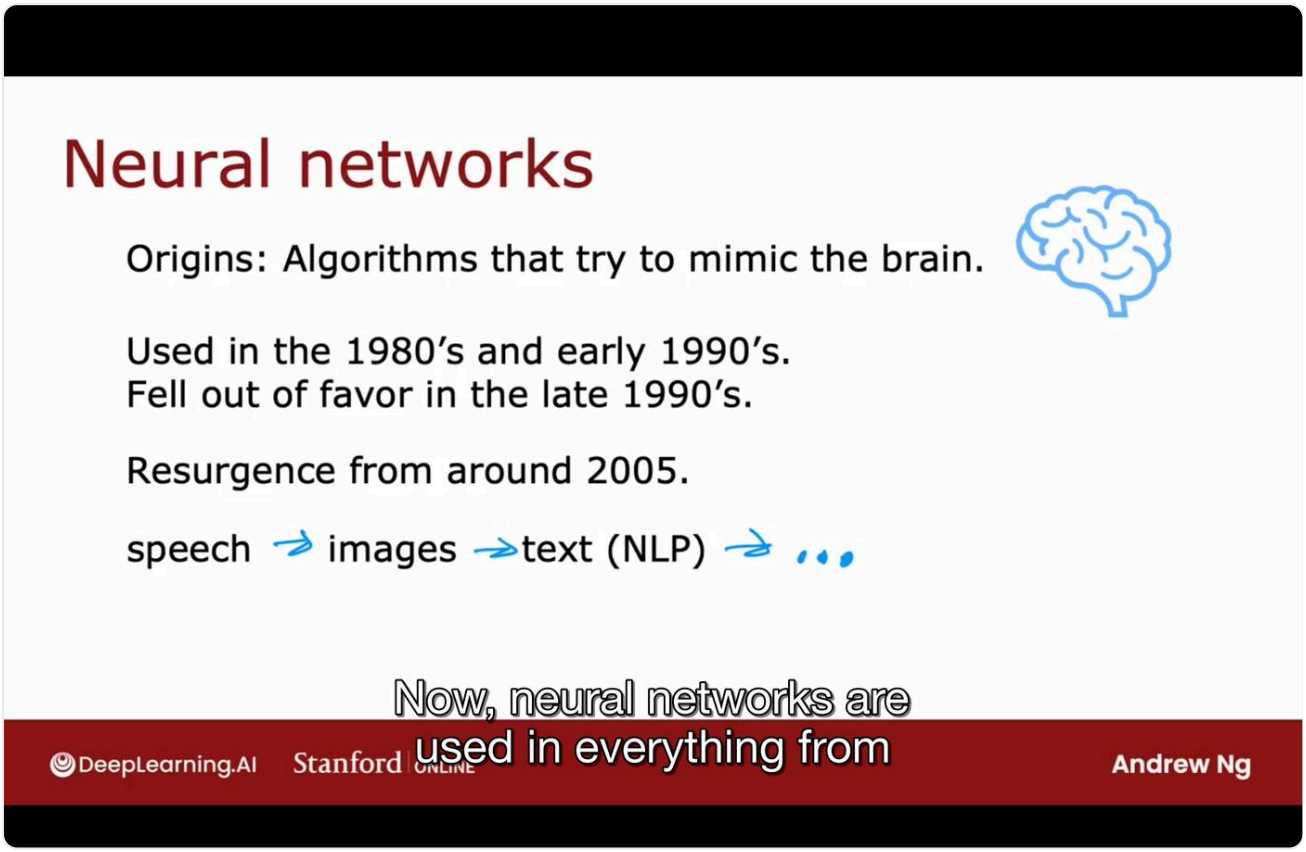


FIGURE 2.3 | Origins: Algorithms That Try to Mimic the Brain

Now, neural networks are used in everything from climate change to medical imaging to online advertising to product recommendations and really lots of application areas of machine learning now use neural networks. Even though today's neural networks have almost nothing to do with how the brain learns, there was the early motivation of trying to build software to mimic the brain. So how does the brain work? Here's a diagram illustrating what neurons in a brain look like.

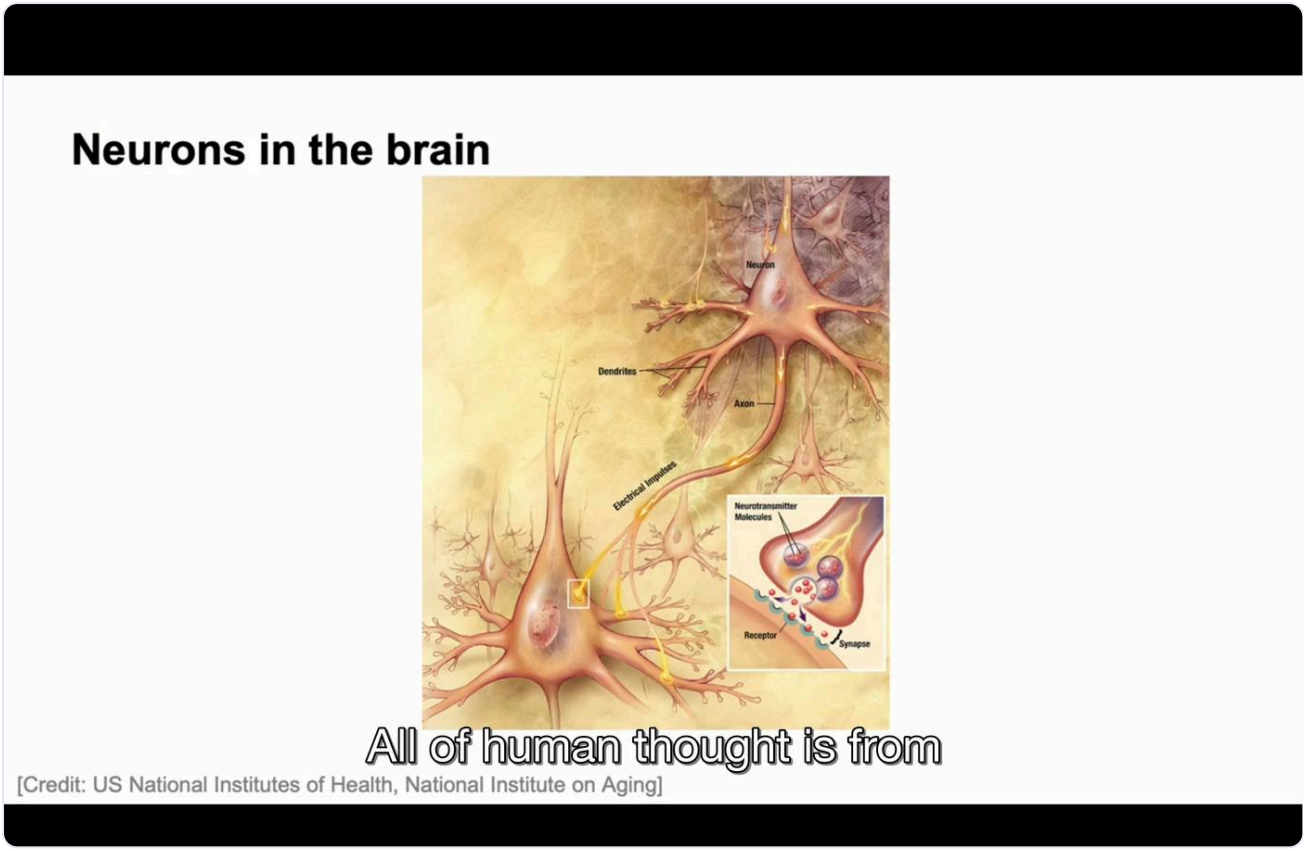


FIGURE 2.4 | Neurons in the Brain

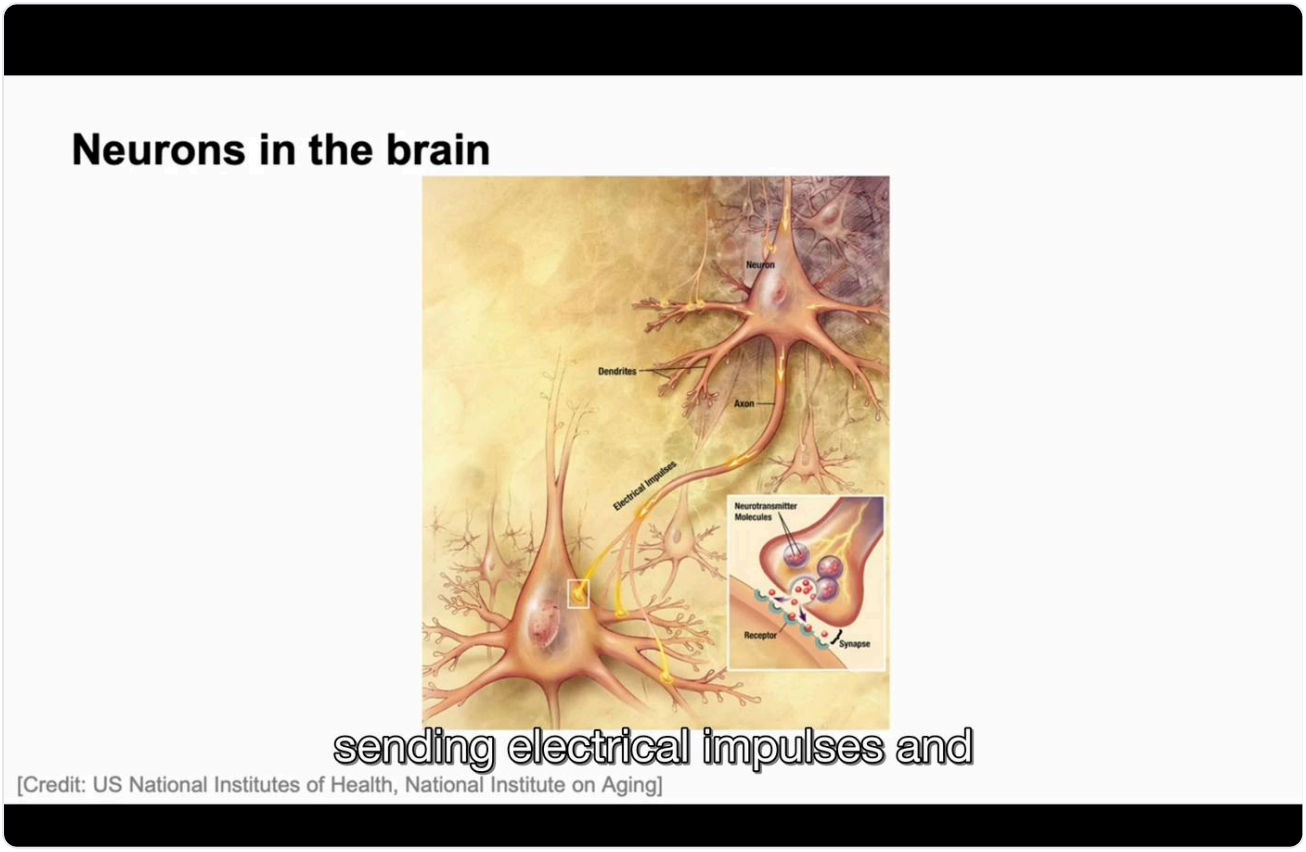


FIGURE 2.5 | Neurons in the Brain

All of human thought is from neurons like this in your brain and mine, sending electrical impulses and sometimes forming new connections of other neurons.

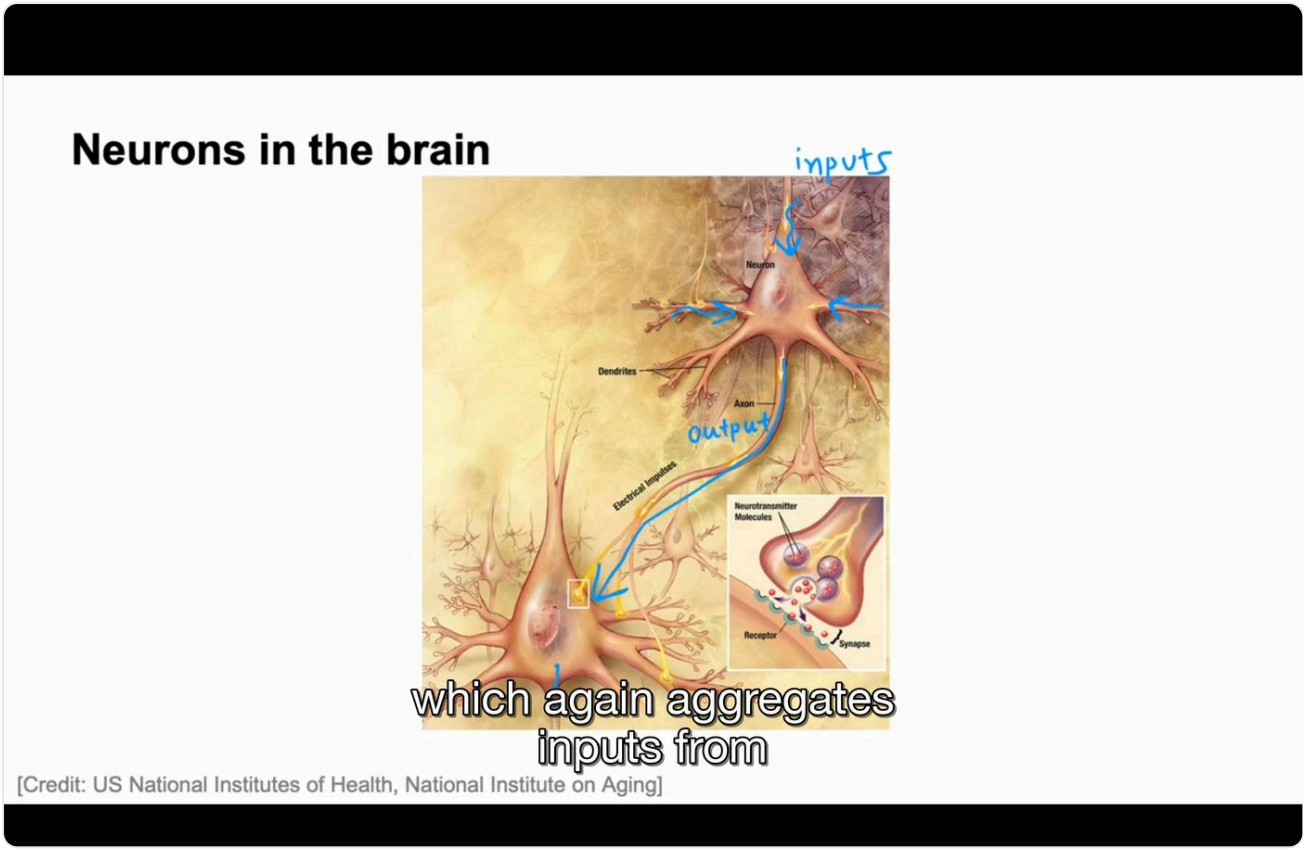


FIGURE 2.6 | Neurons in the Brain

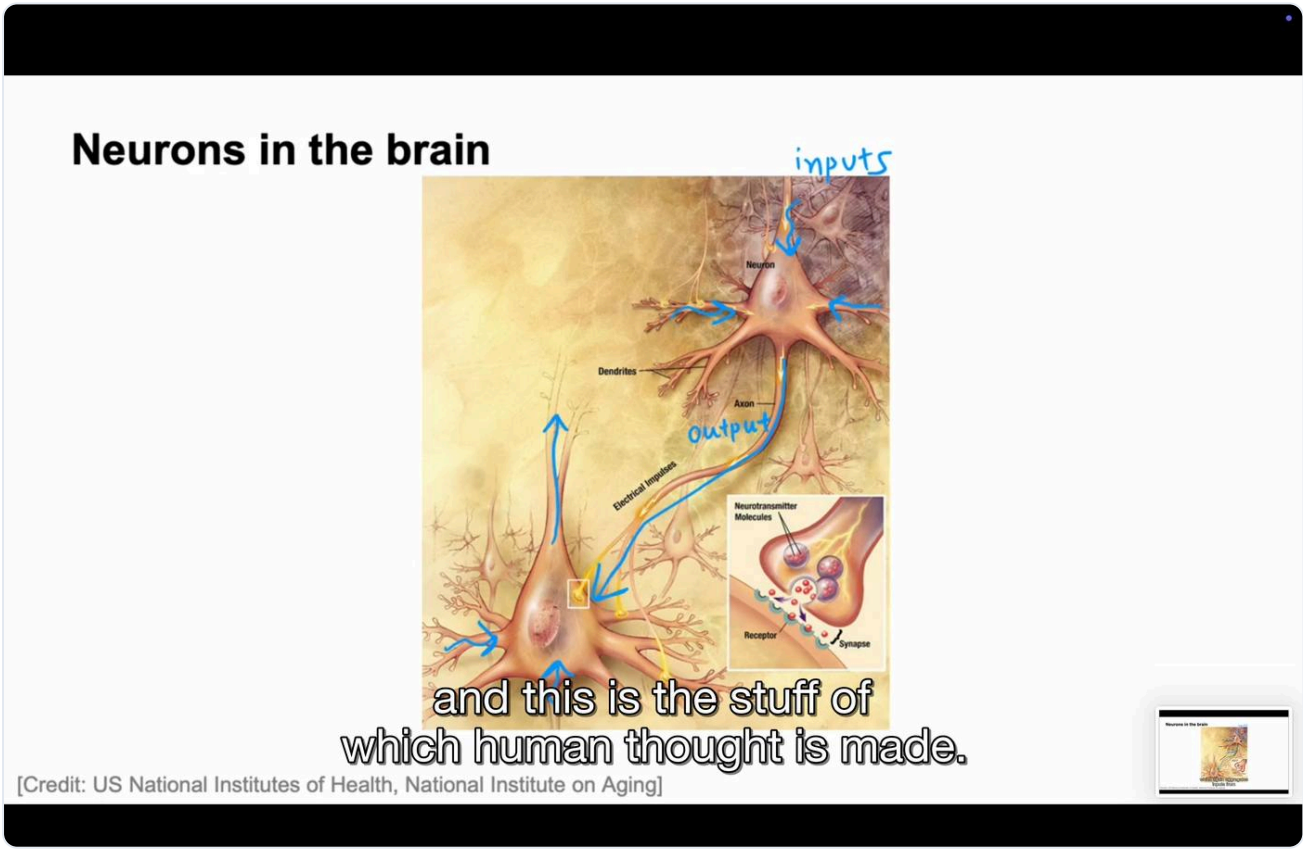


FIGURE 2.7 | Neurons in the Brain

Given a neuron like this one, it has a number of inputs where it receives electrical impulses from other neurons, and then this neuron that I've circled carries out some computations and will then send this outputs to other neurons by this electrical impulses, and this upper neuron's output in turn becomes the input to this neuron down below, which again aggregates inputs from multiple other neurons to then maybe send its own output, to yet other neurons, and this is the stuff of which human thought is made. Here's a simplified diagram of a biological neuron. A neuron comprises a cell body shown here on the left, and if you have taken a class in biology, you may recognize this to be the nucleus of the neuron.

As we saw on the previous slide, the neuron has different inputs. In a biological neuron, the input wires are called the dendrites, and it then occasionally sends electrical impulses to other neurons via the output wire, which is called the axon. Don't worry about these biological terms.

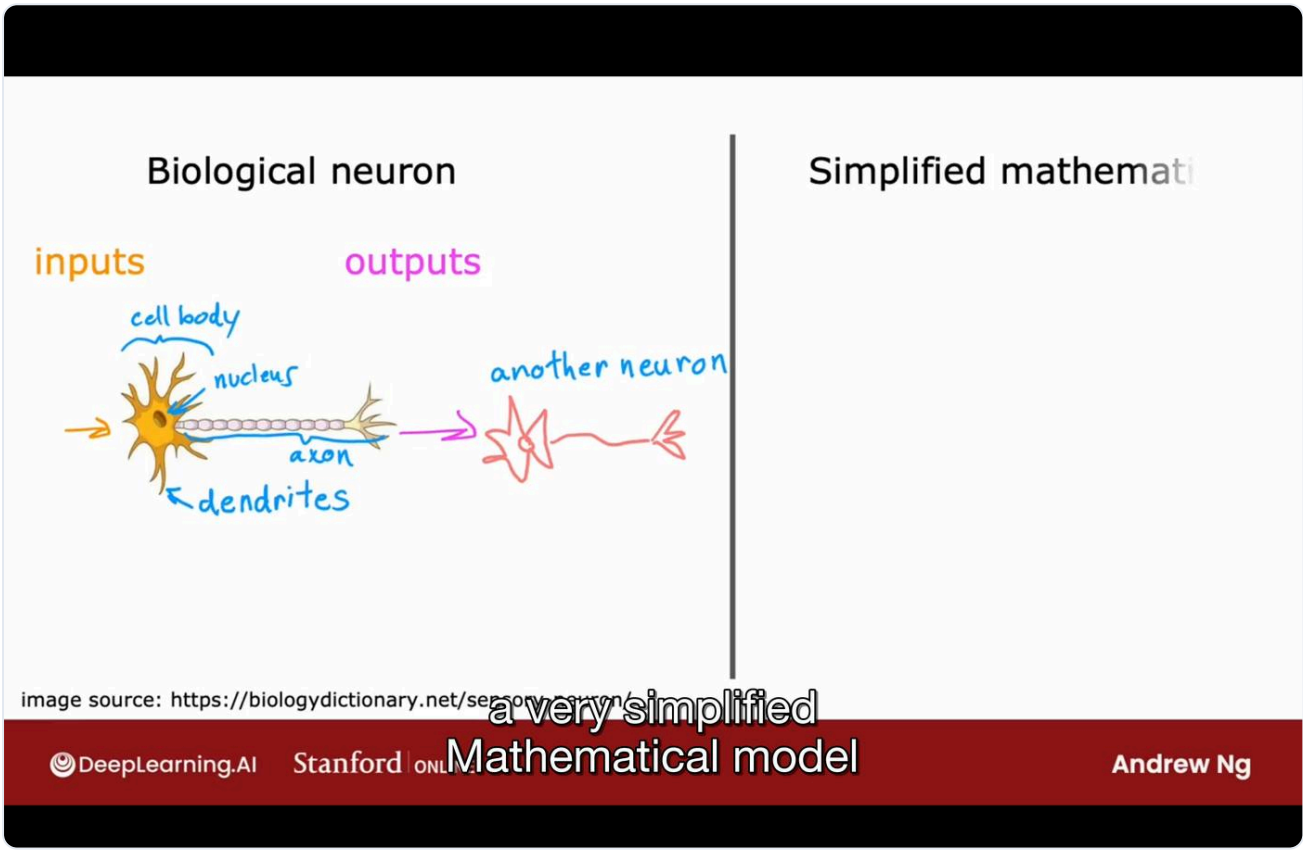


FIGURE 2.8 | Biological Neuron Simplified Mathema

If you saw them in a biology class, you may remember them, but you don't really need to memorize any of these terms for the purpose of building artificial neural networks. But this biological neuron may then send electrical impulses that become the input to another neuron. So the artificial neural network uses a very simplified Mathematical model of what a biological neuron does. I'm going to draw a little circle here to denote a single neuron. What a neuron does is it takes some inputs, one or more inputs, which are just numbers.

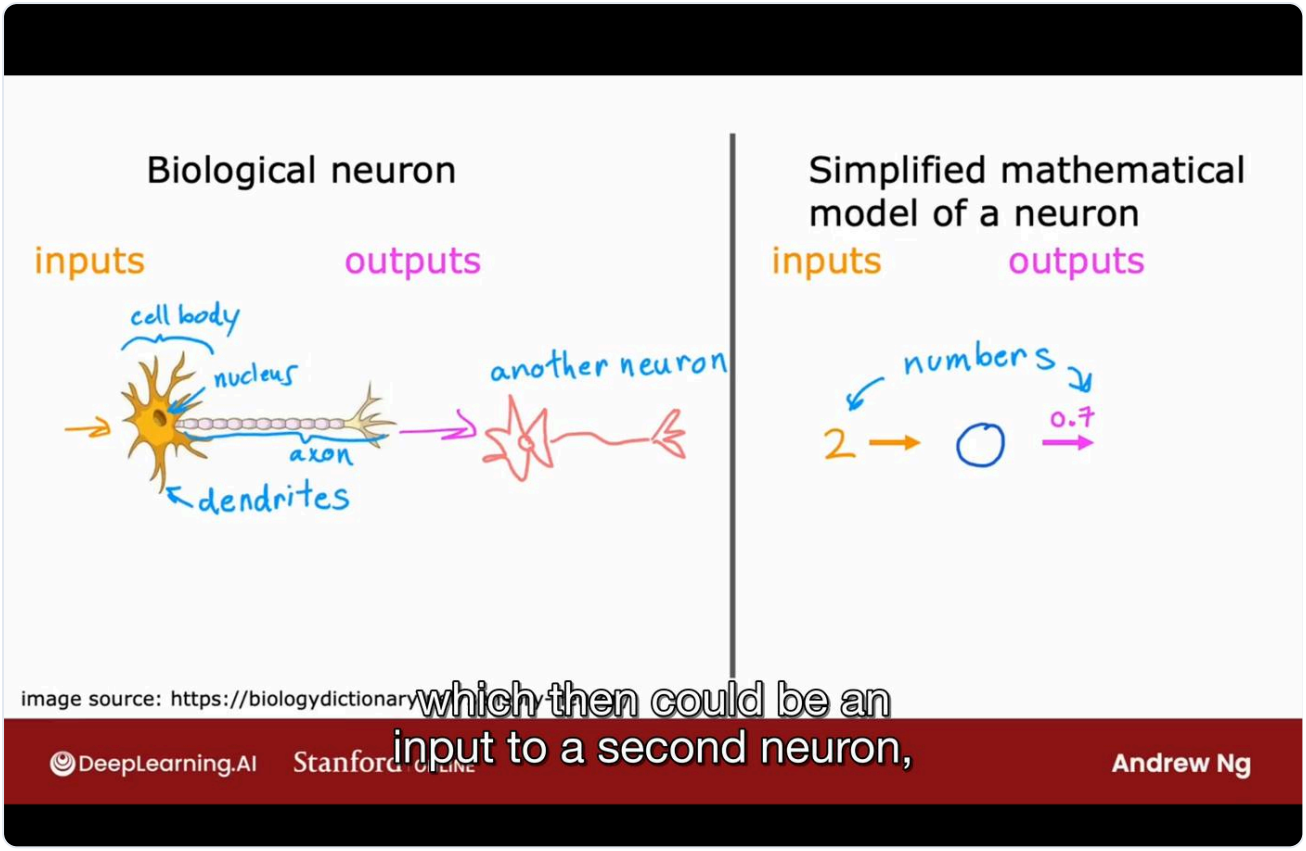


FIGURE 2.9 | Biological Neuron Simplified Mathematical

It does some computation and it outputs some other number, which then could be an input to a second neuron, shown here on the right. When you're building an artificial neural network or deep learning algorithm, rather than building one neuron at a time, you often want to simulate many such neurons at the same time. In this diagram, I'm drawing three neurons.

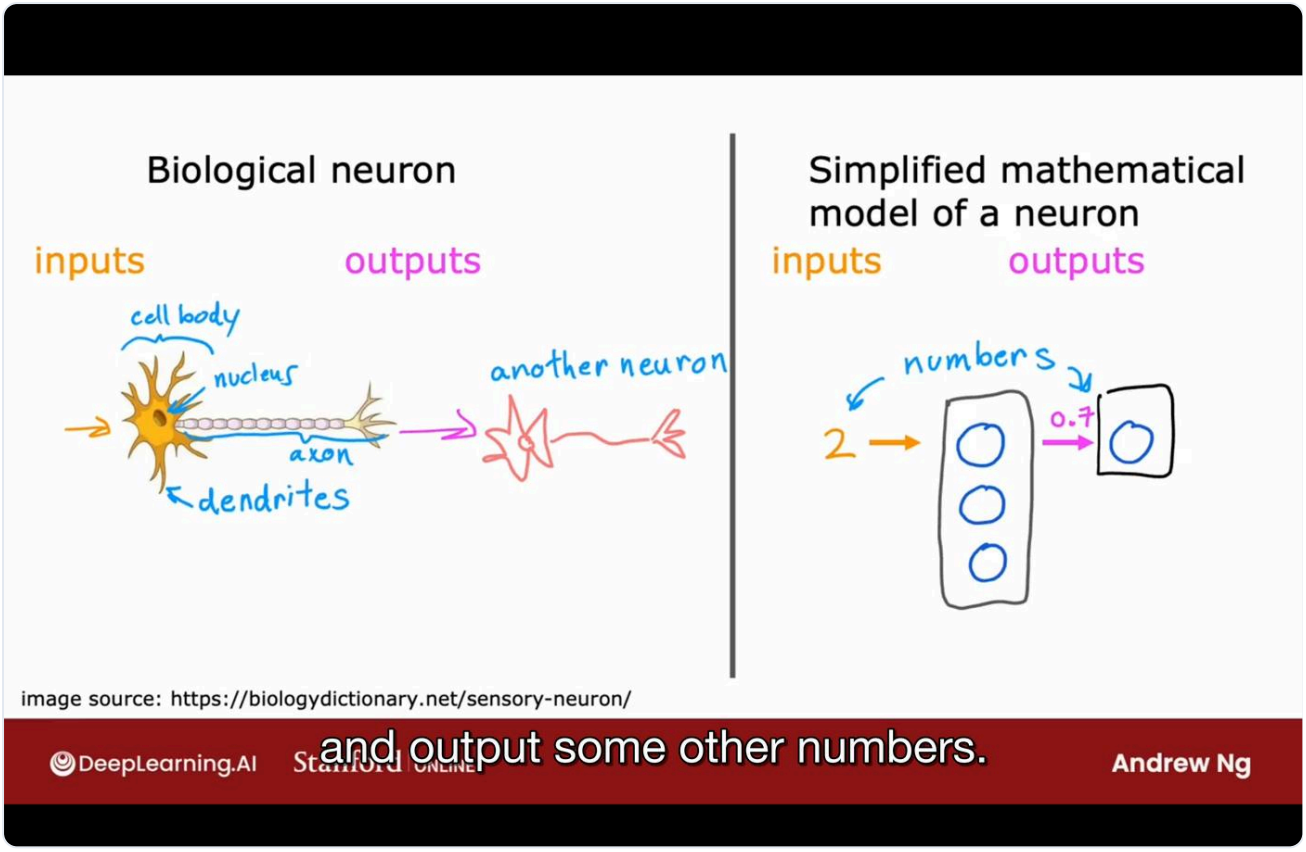


FIGURE 2.10 | Biological Neuron Simplified Mathematical

What these neurons do collectively is input a few numbers, carry out some computation, and output some other numbers. Now, at this point, I'd like to give one big caveat, which is that even though I made a loose analogy between biological neurons and artificial neurons, I think that today we have almost no idea how the human brain works. In fact, every few years, neuroscientists make some fundamental breakthrough about how the brain works. I think we'll continue to do so for the foreseeable future. That to me is a sign that there are many breakthroughs that are yet to be discovered about how the brain actually works, and thus attempts to blindly mimic what we know of the human brain today, which is frankly very little, probably won't get us that far toward building raw intelligence.

Certainly not with our current level of knowledge in neuroscience. Having said that, even with these extremely simplified models of a neuron, which we'll talk about, we'll be able to build really powerful deep learning algorithms. So as you go deeper into neural networks and into deep learning, even though the origins were biologically motivated, don't take the biological motivation too seriously. In fact, those of us that do research in deep learning have shifted away from looking to biological motivation that much. But instead, they're just using engineering principles to figure out how to build algorithms that are more effective. But I think it might still be fun to speculate and think about how biological neurons work every now and then.

The ideas of neural networks have been around for many decades. A few people have asked me, "Hey Andrew, why now? Why is it that only in the last handful of years that neural networks have really taken off? This is a picture I draw for them when I'm asked that question and that maybe you could draw for others as well if they ask you that question. Let me plot on the horizontal axis the amount of data you have for a problem, and on the vertical axis, the performance or the accu-

racy of a learning algorithm applied to that problem. Over the last couple of decades, with the rise of the Internet, the rise of mobile phones, the digitalization of our society, the amount of data we have for a lot of applications has steadily marched to the right.

Lot of records that use P on paper, such as if you order something rather than it being on a piece of paper, there's much more likely to be a digital record. Your health record, if you see a doctor, is much more likely to be digital now compared to on pieces of paper. So in many application areas, the amount of digital data has exploded. What we saw was with traditional machine-learning algorithms, such as logistic regression and linear regression, even as you fed those algorithms more data, it was very difficult to get the performance to keep on going up. So it was as if the traditional learning algorithms like linear regression and logistic regression, they just weren't able to scale with the amount of data we could now feed it and they weren't able to take effective advantage of all this data we had for different applications.

What AI researchers started to observe was that if you were to train a small neural network on this dataset, then the performance maybe looks like this. If you were to train a medium-sized neural network, meaning one with more neurons in it, its performance may look like that. If you were to train a very large neural network, meaning one with a lot of these artificial neurons, then for some applications the performance will just keep on going up.

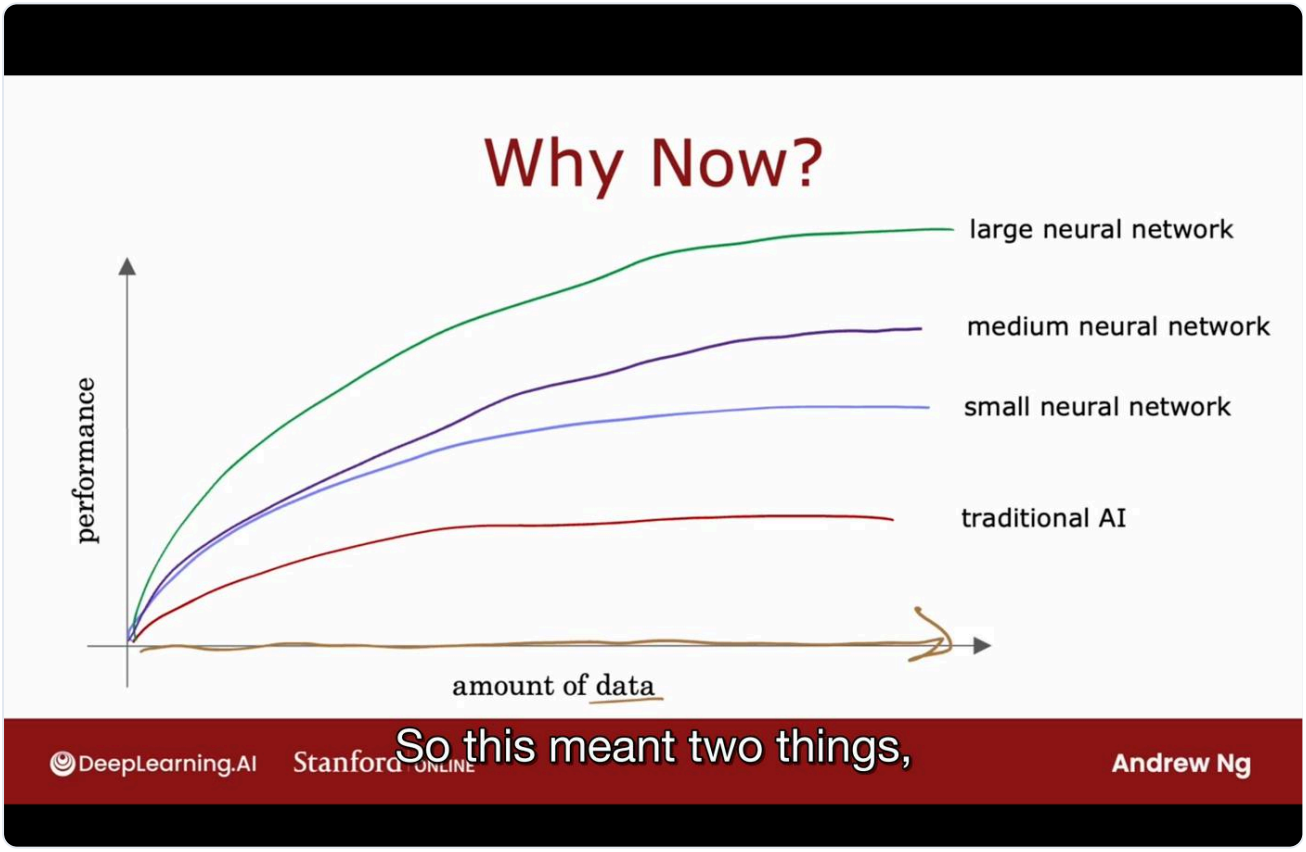


FIGURE 2.11

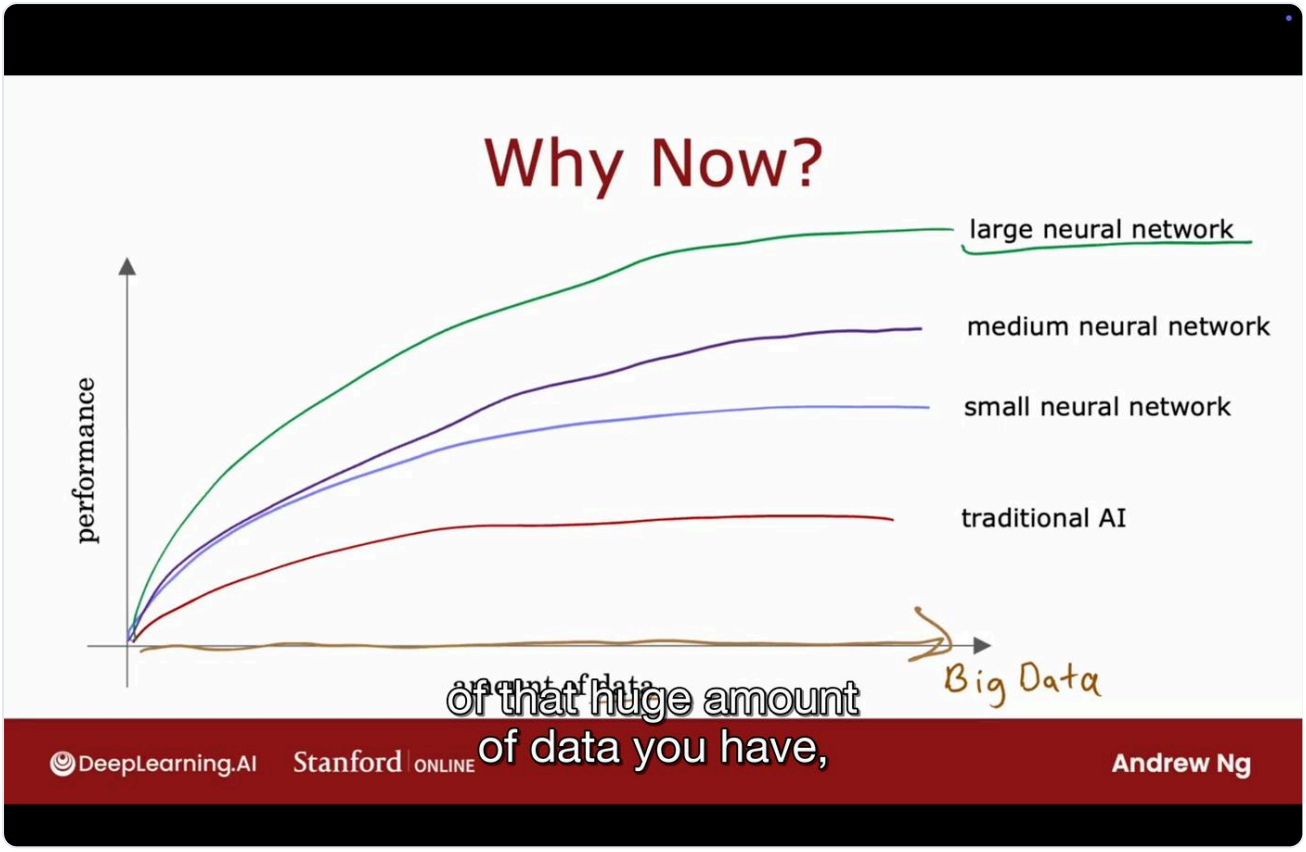


FIGURE 2.12

So this meant two things, it meant that for a certain class of applications where you do have a lot of data, sometimes you hear the term big data toss around, if you're able to train a very large neural network to take advantage of that huge amount of data you have, then you could attain performance on anything ranging from speech recognition, to image recognition, to natural language processing applications and many more, they just were not possible with earlier generations of learning algorithms. This caused deep learning algorithms to take off, and this too is why faster computer processors, including the rise of GPUs or graphics processor units. This is hardware originally designed to generate nice-looking computer graphics, but turned out to be really powerful for deep learning as well.

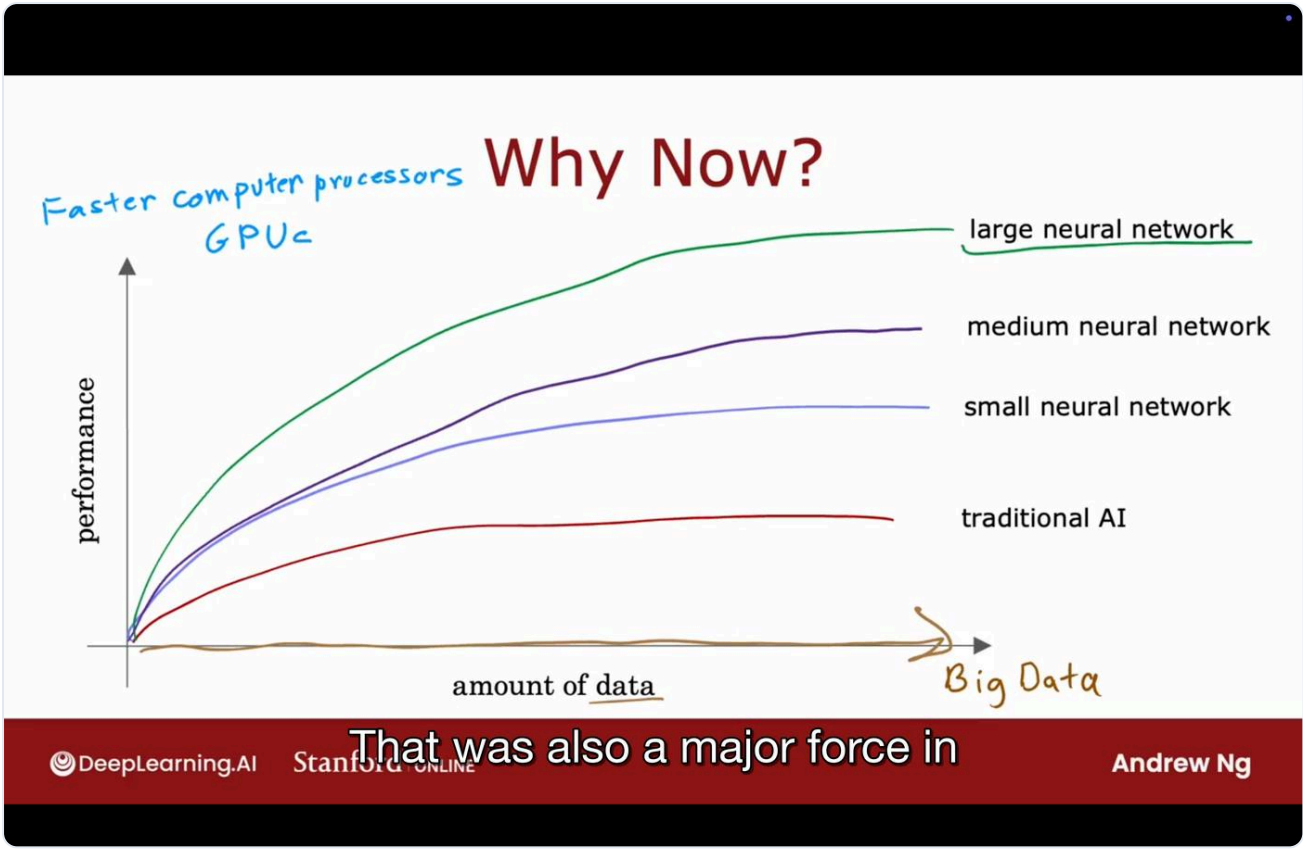


FIGURE 2.13

That was also a major force in allowing deep learning algorithms to become what it is today. That's how neural networks got started, as well as why they took off so quickly in the last several years. Let's now dive more deeply into the details of how neural network actually works. Please go on to the next video.

LECTURE 3

Demand Prediction

To illustrate how neural networks work, let's start with an example. We'll use an example from demand prediction in which you look at the product and try to predict, will this product be a top seller or not? Let's take a look. In this example, you're selling T-shirts and you would like to know if a particular T-shirt will be a top seller, yes or no, and you have collected data of different t-shirts that were sold at different prices, as well as which ones became a top seller. This type of application is used by retailers today in order to plan better inventory levels as well as marketing campaigns. If you know what's likely to be a top seller, you would plan, for example, to just purchase more of that stock in advance.

In this example, the input feature x is the price of the T-shirt, and so that's the input to the learning algorithm. If you apply logistic regression to fit a sigmoid function to the data that might look like that then the outputs of your prediction might look like this, 1/1 plus e to the negative wx plus b.

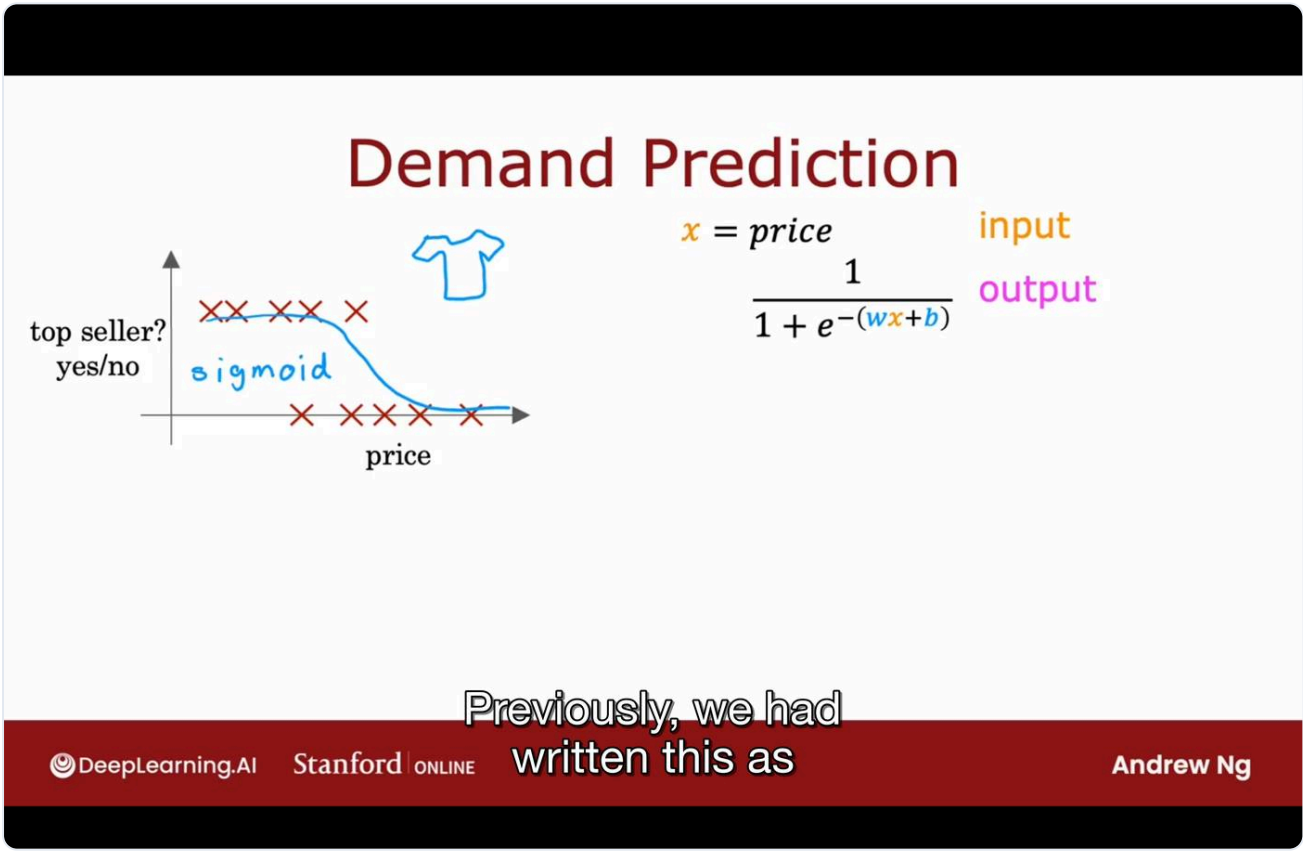


FIGURE 3.1 | Demand Prediction

Previously, we had written this as f of x as the output of the learning algorithm.

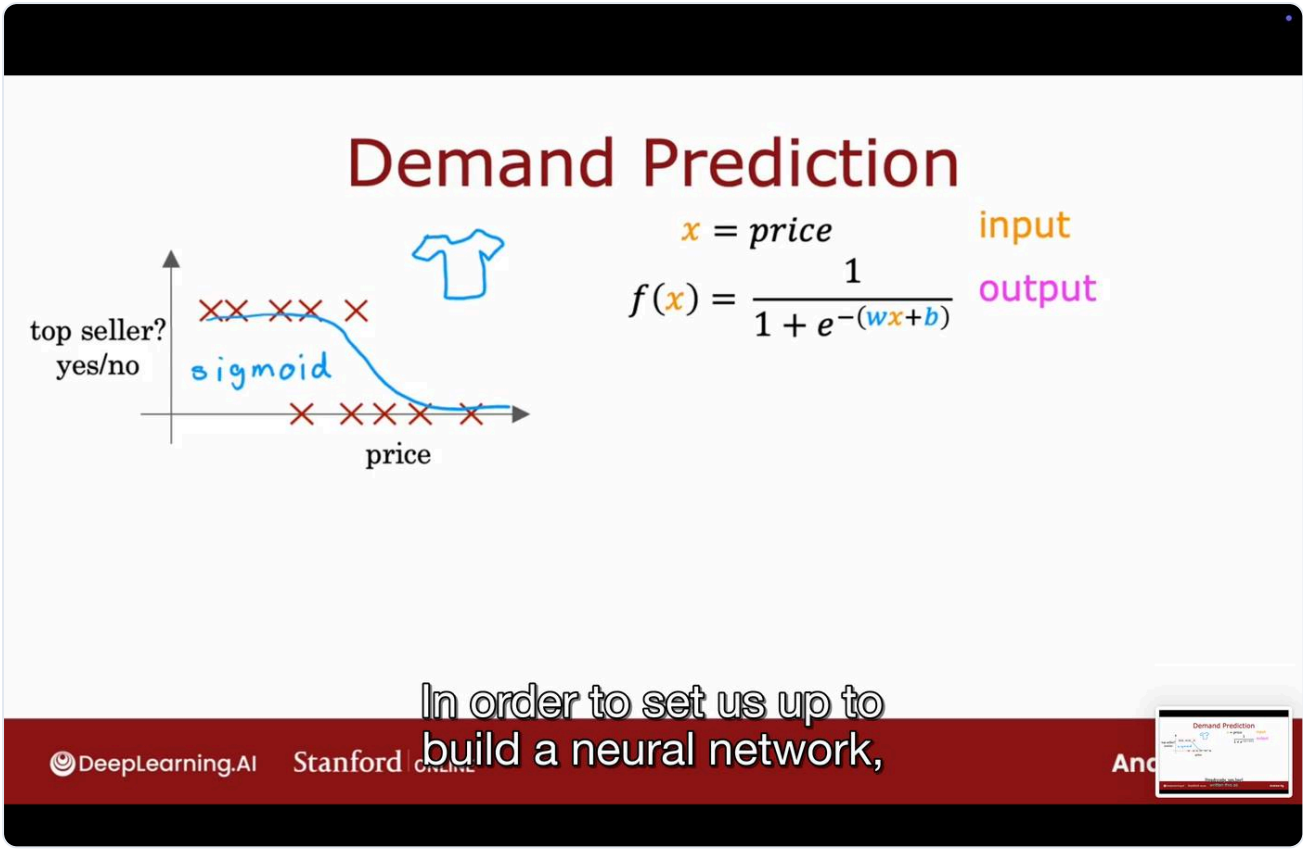


FIGURE 3.2 | Demand Prediction

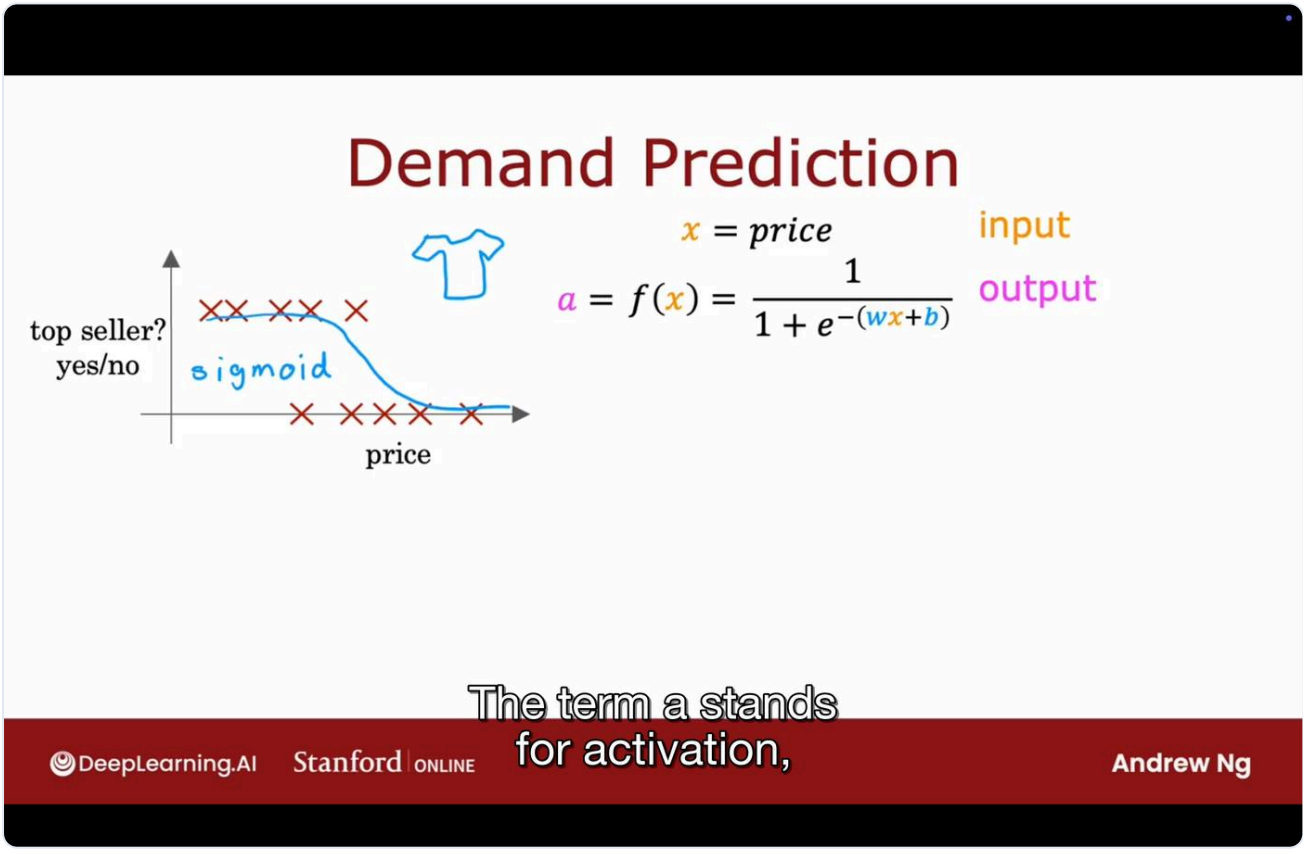


FIGURE 3.3 | Demand Prediction

In order to set us up to build a neural network, I'm going to switch the terminology a little bit and use the alphabet a to denote the output of this logistic regression algorithm.

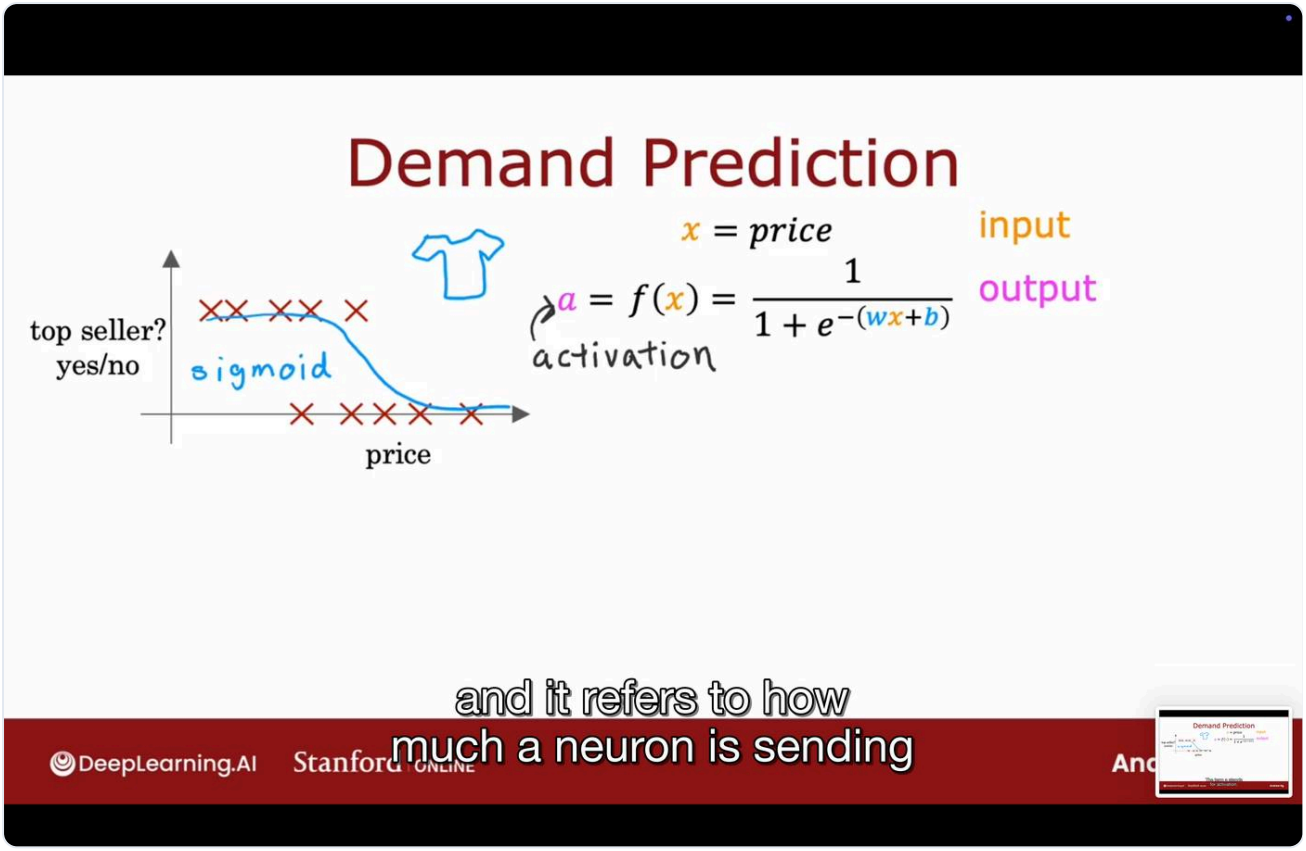


FIGURE 3.4 | Demand Prediction

The term a stands for activation, and it's actually a term from neuroscience, and it refers to how much a neuron is sending a high output to other neurons downstream from it. It turns out that this logistic regression units or this little logistic regression algorithm, can be thought of as a very simplified model of a single neuron in the brain. Where what the neuron does is it takes us input the price x, and then it computes this formula on top, and it outputs the number a, which is computed by this formula, and it outputs the probability of this T-shirt being a top seller.

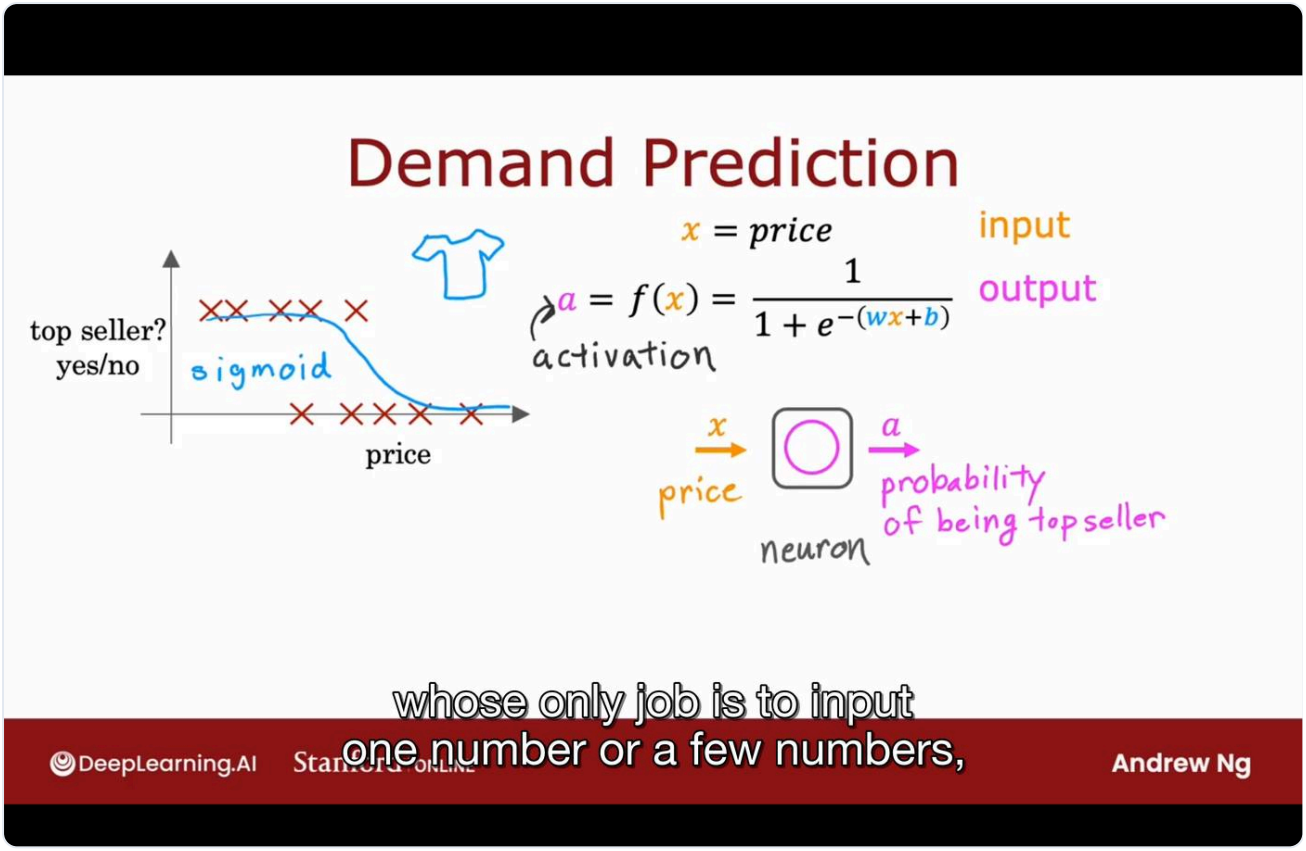


FIGURE 3.5 | Demand Prediction

Another way to think of a neuron is as a tiny little computer whose only job is to input one number or a few numbers, such as a price, and then to output one number or maybe a few other numbers which in this case is the probability of the T-shirt being a top seller. As I alluded in the previous video, a logistic regression algorithm is much simpler than what any biological neuron in your brain or mine does. Which is why the artificial neural network is such a vastly oversimplified model of the human brain. Even though in practice, as you know, deep learning algorithms do work very well. Given this description of a single neuron, building a neural network now it just requires taking a bunch of these neurons and wiring them together or putting them together.

Let's now look at a more complex example of demand prediction. In this example, we're going to have four features to predict whether or not a T-shirt is a top seller. The features are the price of the T-shirt, the shipping costs, the amounts of marketing of that particular T-shirt, as well as the material quality, is this a high-quality, thick cotton versus maybe a lower quality material? Now, you might suspect that whether or not a T-shirt becomes a top seller actually depends on a few factors. First, one is the affordability of this T-shirt. Second is, what's the degree of awareness of this T-shirt that potential buyers have? Third is perceived quality to bias or potential bias saying this is a high-quality T-shirt.

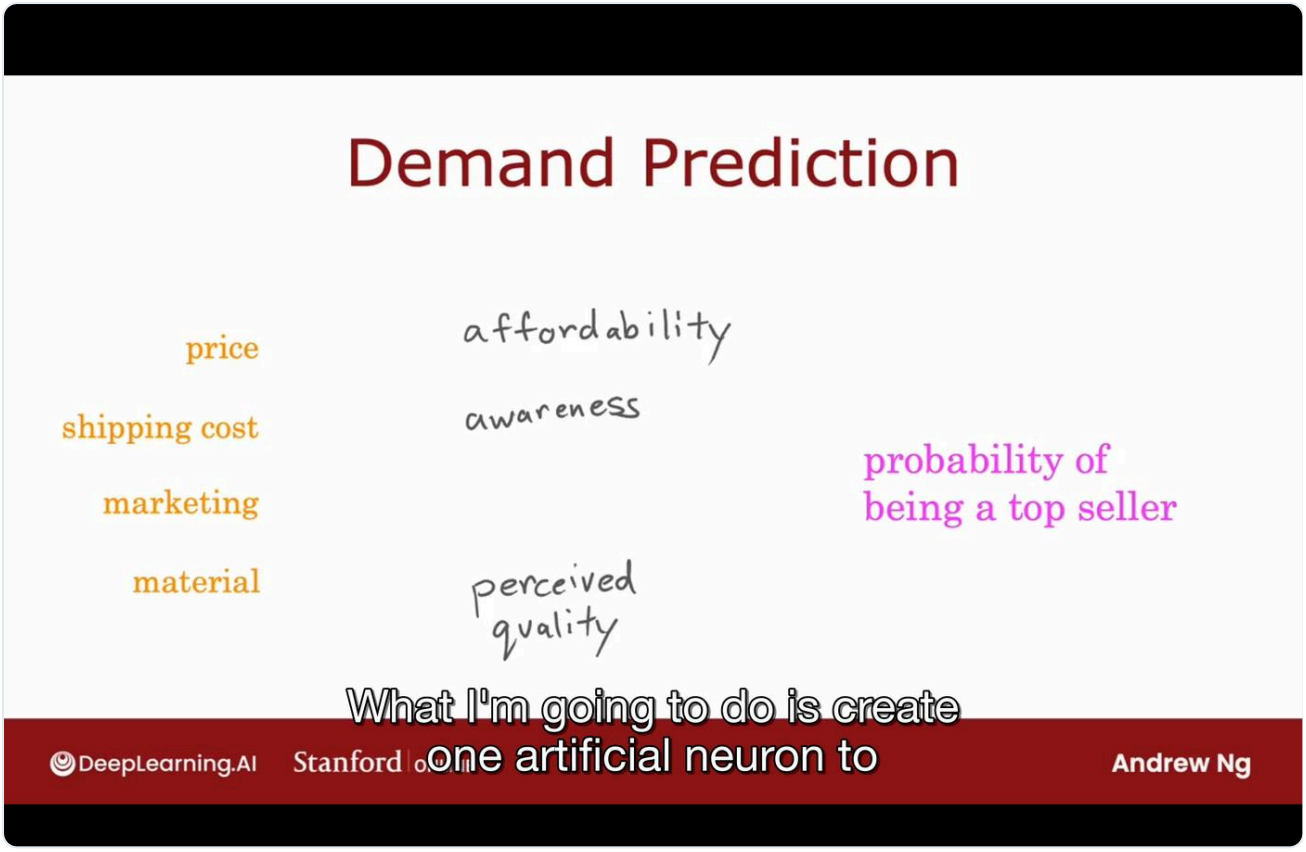


FIGURE 3.6 | Demand Prediction

What I'm going to do is create one artificial neuron to try to estimate the probability that this T-shirt is perceive as highly affordable. Affordability is mainly a function of price and shipping costs because the total amount of the pay is some of the price plus the shipping costs. We're going to use a little neuron here, a logistic regression unit to input price and shipping costs and predict do people think this is affordable? Second, I'm going to create another artificial neuron here to estimate, is there high awareness of this?

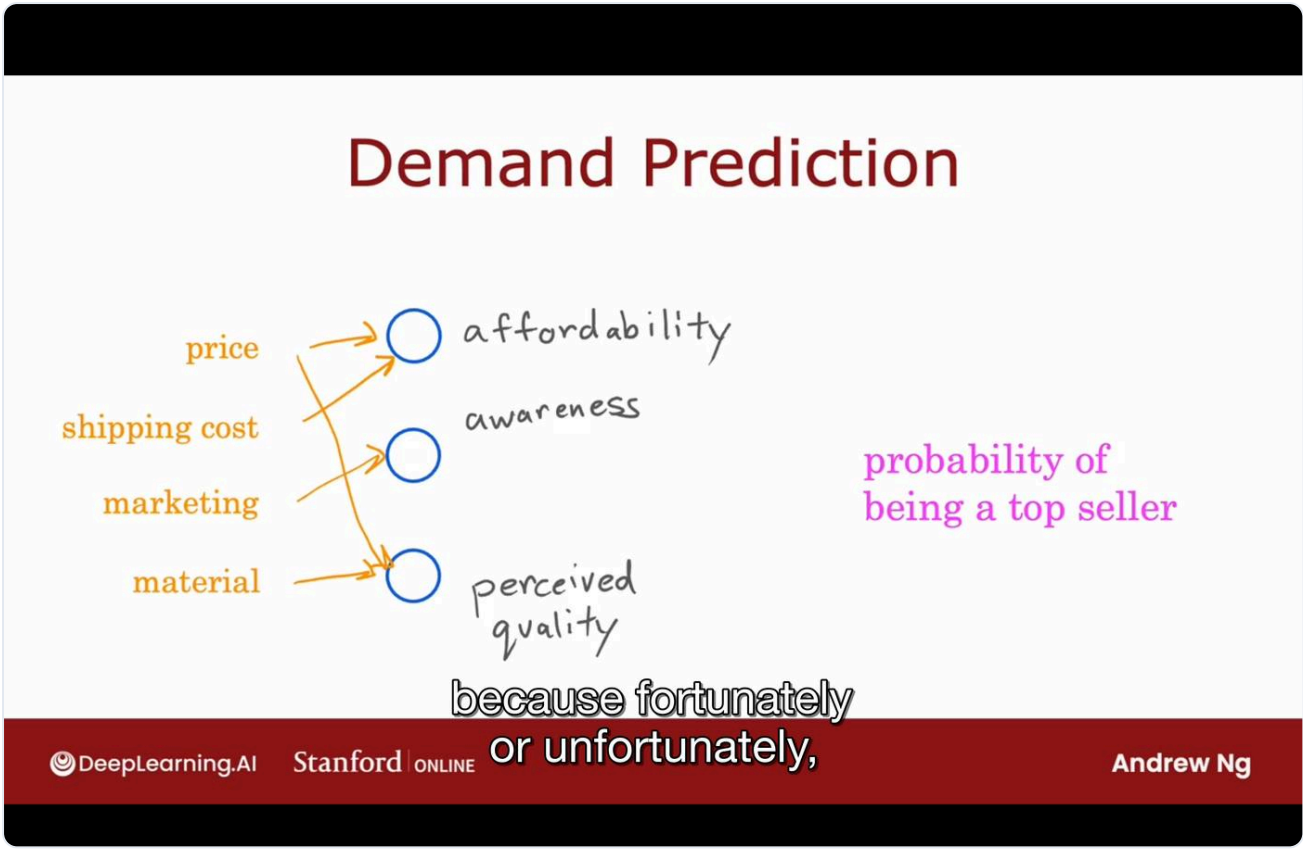


FIGURE 3.7 | Demand Prediction

Awareness in this case is mainly a function of the marketing of the T-shirt. Finally, going to create another neuron to estimate do people perceive this to be of high quality, and that may mainly be a function of the price of the T-shirt and of the material quality. Price is a factor here because fortunately or unfortunately, if there's a very high priced T-shirt, people will sometimes perceive that to be of high quality because it is very expensive than maybe people think it's going to be of high-quality. Given these estimates of affordability, awareness, and perceived quality we then wire the outputs of these three neurons to another neuron here on the right, that then there's another logistic regression unit.

That finally inputs those three numbers and outputs the probability of this t-shirt being a top seller.

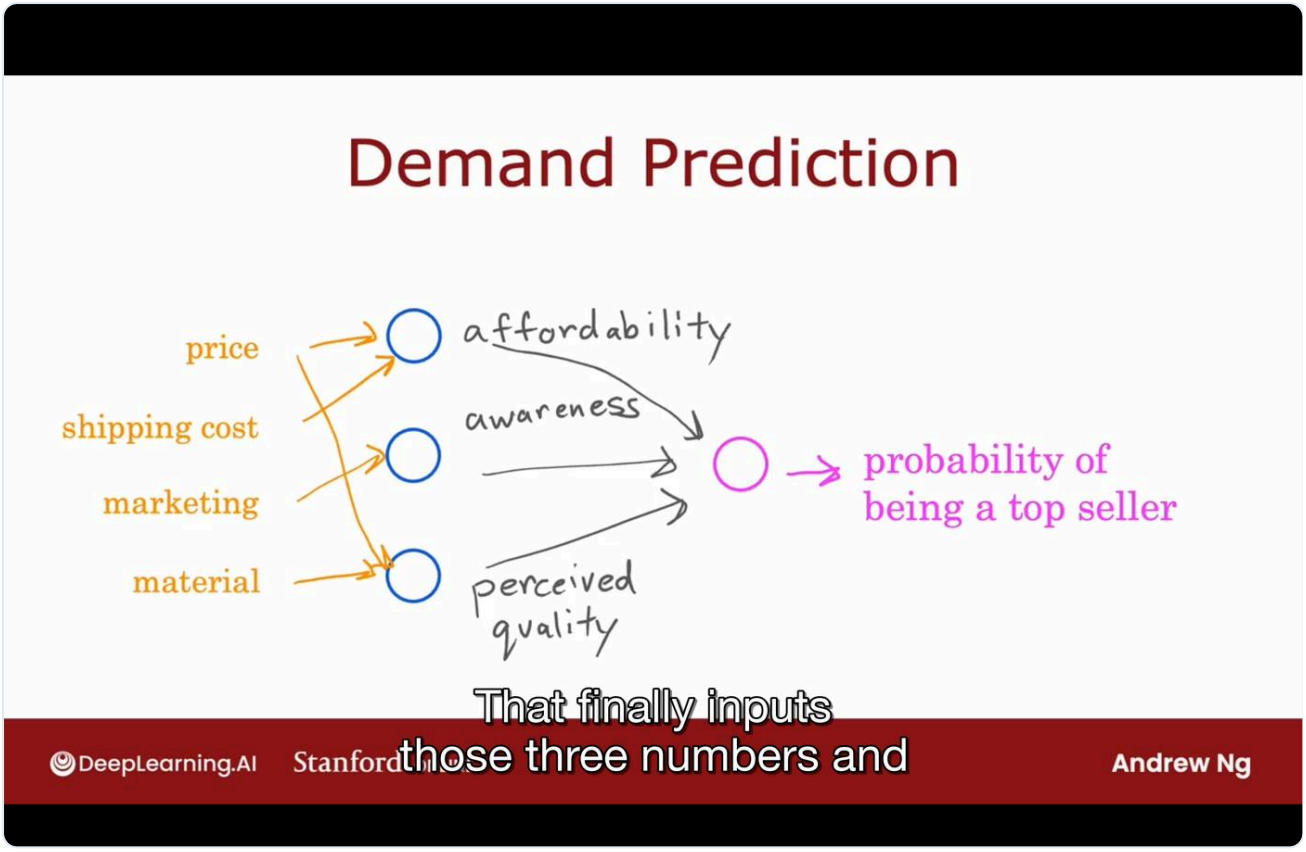


FIGURE 3.8 | Demand Prediction

In the terminology of neural networks, we're going to group these three neurons together into what's called a layer. A layer is a grouping of neurons which takes as input the same or similar features, and that in turn outputs a few numbers together. These three neurons on the left form one layer which is why I drew them on top of each other, and this single neuron on the right is also one layer. The layer on the left has three neurons, so a layer can have multiple neurons or it can also have a single neuron as in the case of this layer on the right.

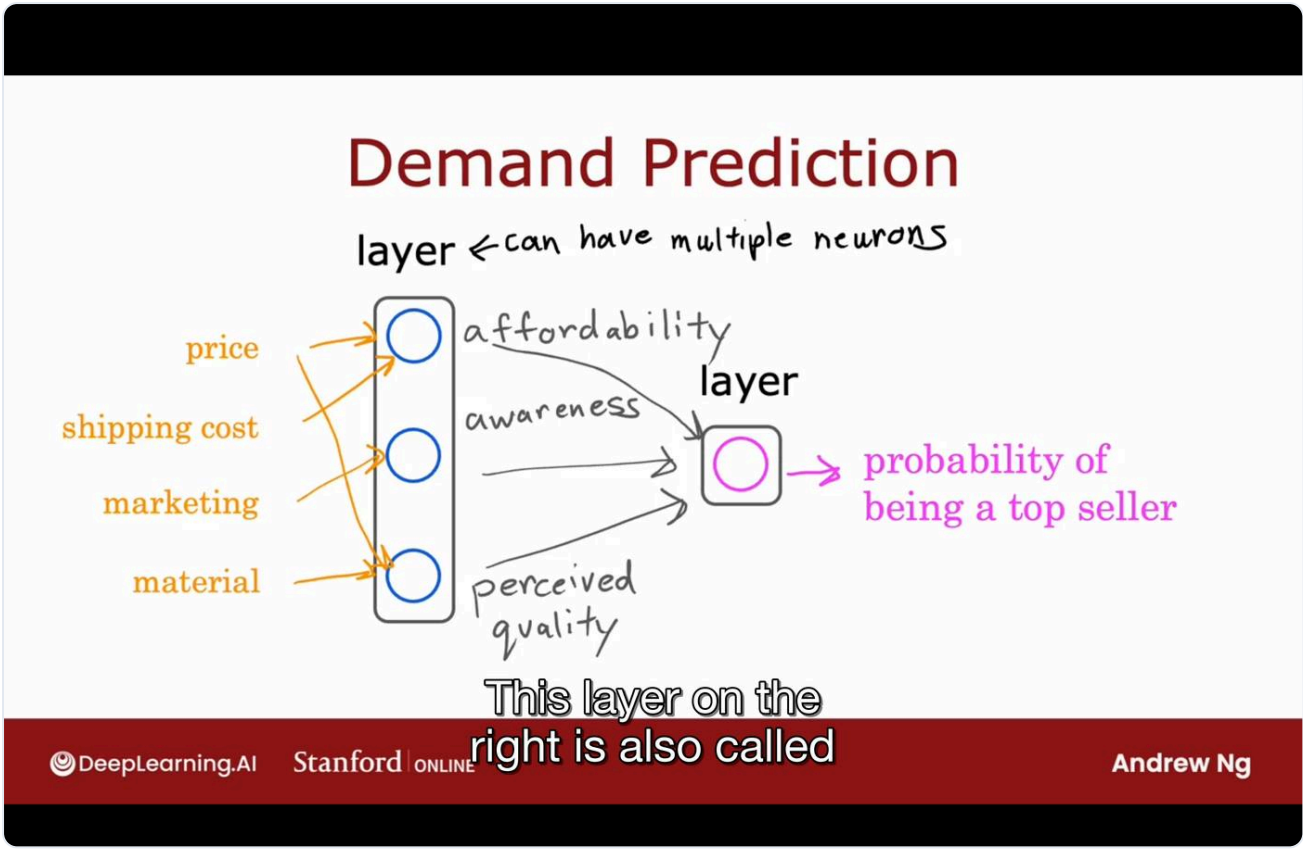


FIGURE 3.9 | Demand Prediction

This layer on the right is also called the output layer because the outputs of this final neuron is the output probability predicted by the neural network. In the terminology of neural networks we're also going to call affordability awareness and perceive quality to be activations. The term activations comes from biological neurons, and it refers to the degree that the biological neuron is sending a high output value or sending many electrical impulses to other neurons to the downstream from it.

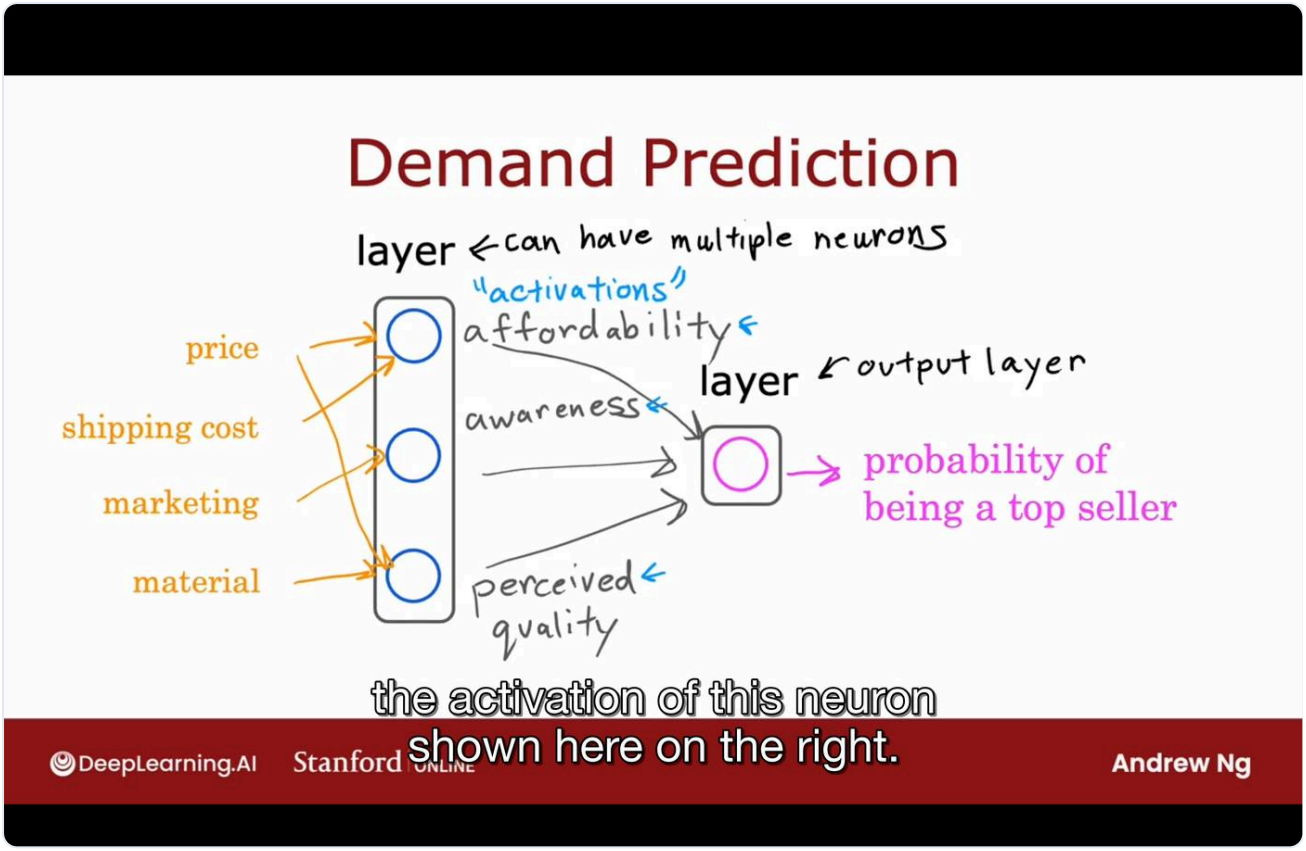


FIGURE 3.10 | Demand Prediction

These numbers on affordability, awareness, and perceived quality are the activations of these three neurons in this layer, and also this output probability is the activation of this neuron shown here on the right. This particular neural network therefore carries out computations as follows. It inputs four numbers then this layer of the neural network uses those four numbers to compute the new numbers also called activation values. Then the final layer, the output layer of the neural network used those three numbers to compute one number.

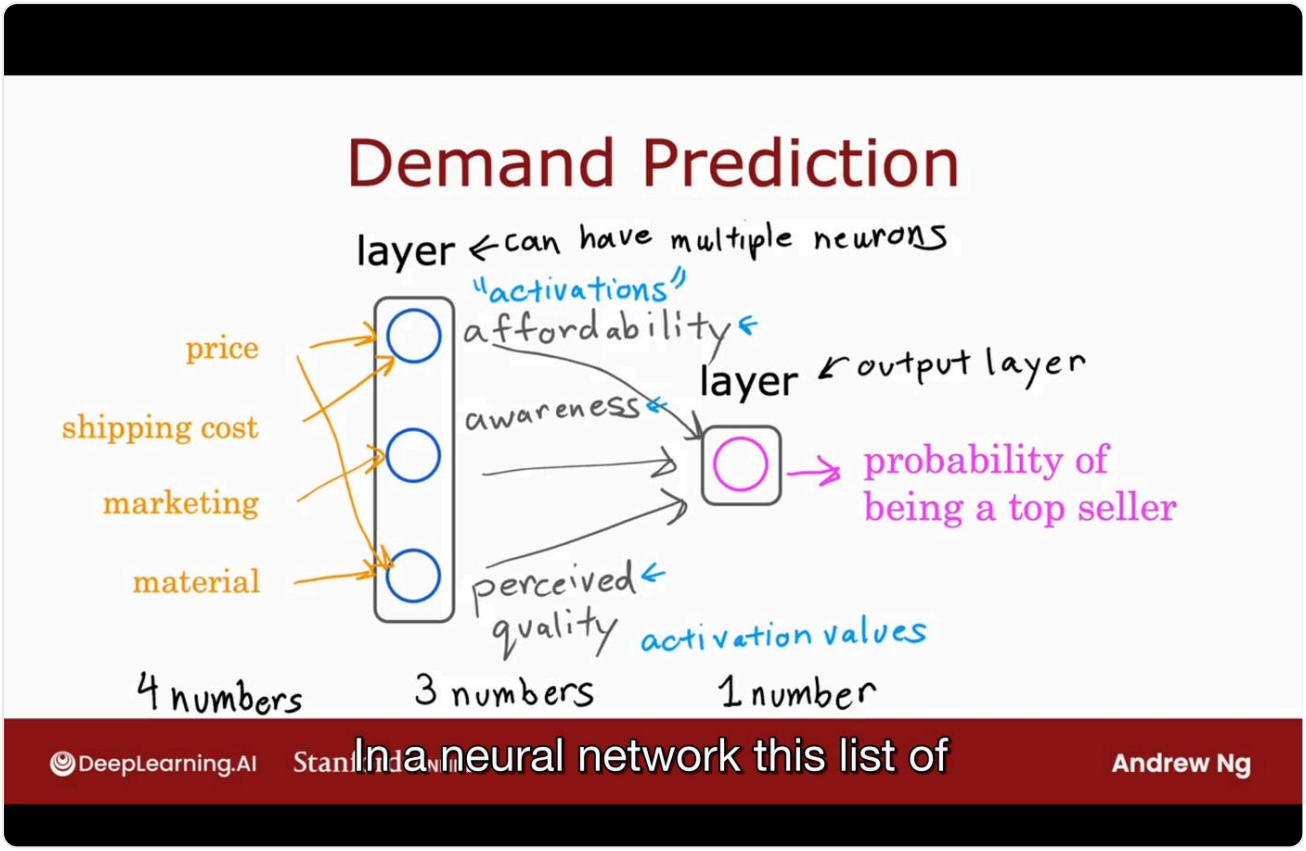


FIGURE 3.11 | Demand Prediction

In a neural network this list of four numbers is also called the input layer, and that's just a list of four numbers. Now, there's one simplification I'd like make to this neural network. The way I've described it so far, we had to go through the neurons one at a time and decide what inputs it would take from the previous layer. For example, we said affordability is a function of just price and shipping costs and awareness is a function of just marketing and so on, but if you're building a large neural network it'd be a lot of work to go through and manually decide which neurons should take which features as inputs. The way a neural network is implemented in practice each neuron in a certain layer; say this layer in the middle, will have access to every feature, to every value from the previous layer, from the input layer which is why I'm now drawing arrows from every input feature to every one of these neurons shown here in the middle.

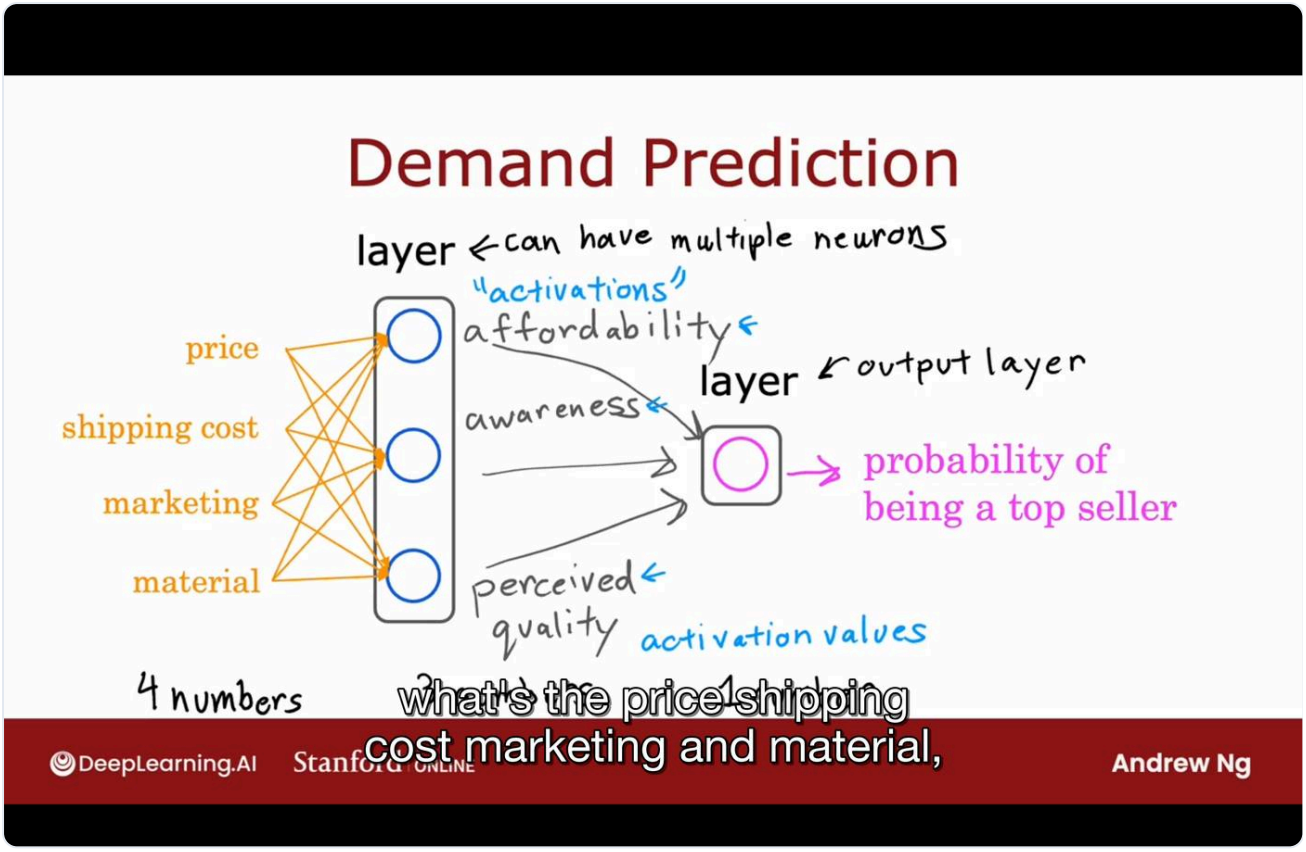


FIGURE 3.12 | Demand Prediction

You can imagine that if you're trying to predict affordability and it knows what's the price shipping cost marketing and material, may be you'll learn to ignore marketing and material and just figure out through setting the parameters appropriately to only focus on the subset of features that are most relevant to affordability.

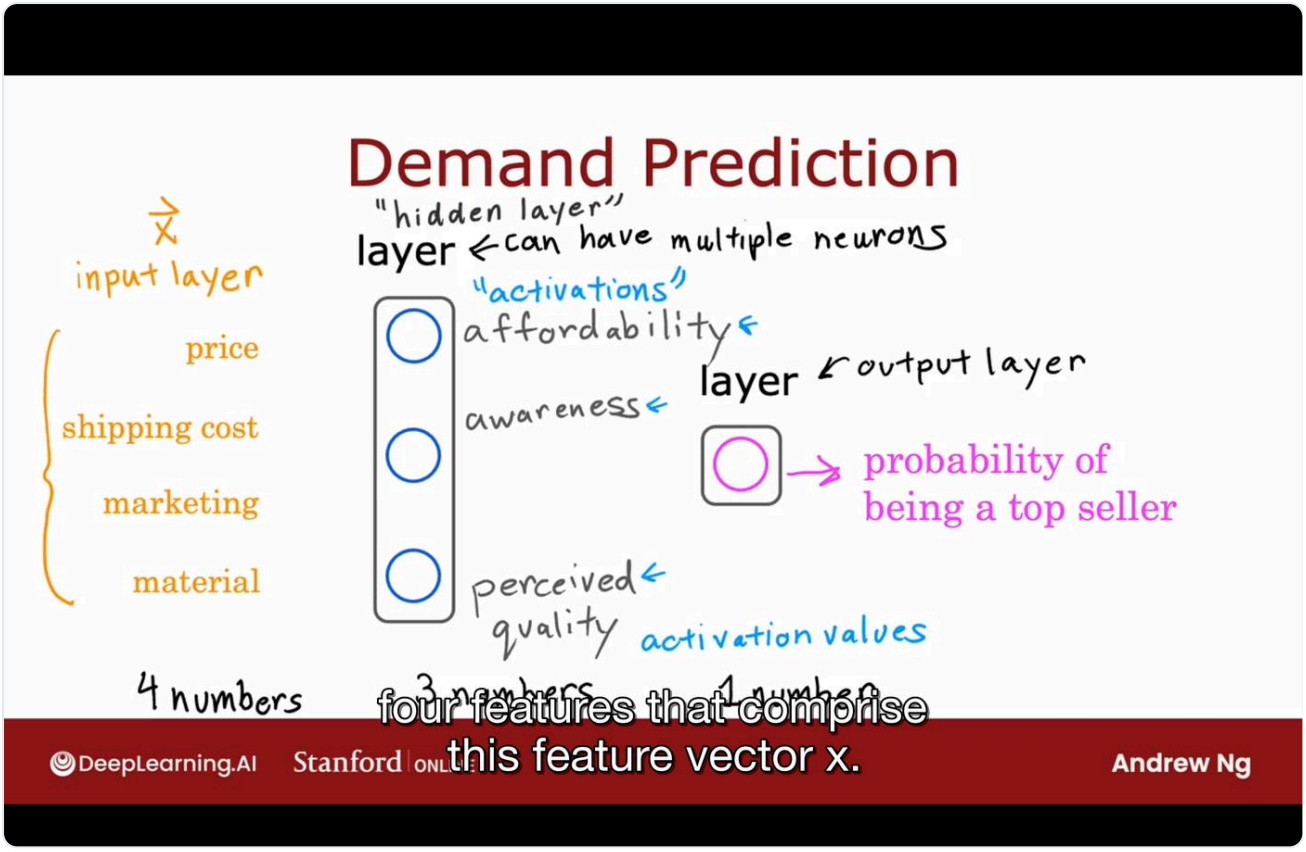


FIGURE 3.13 | Demand Prediction

To further simplify the notation and the description of this neural network I'm going to take these four input features and write them as a vector x, and we're going to view the neural network as having four features that comprise this feature vector x. This feature vector is fed to this layer in the middle which then computes three activation values. That is these numbers and these three activation values in turn becomes another vector which is fed to this final output layer that finally outputs the probability of this t-shirt to being a top seller. That's all a neural network is.

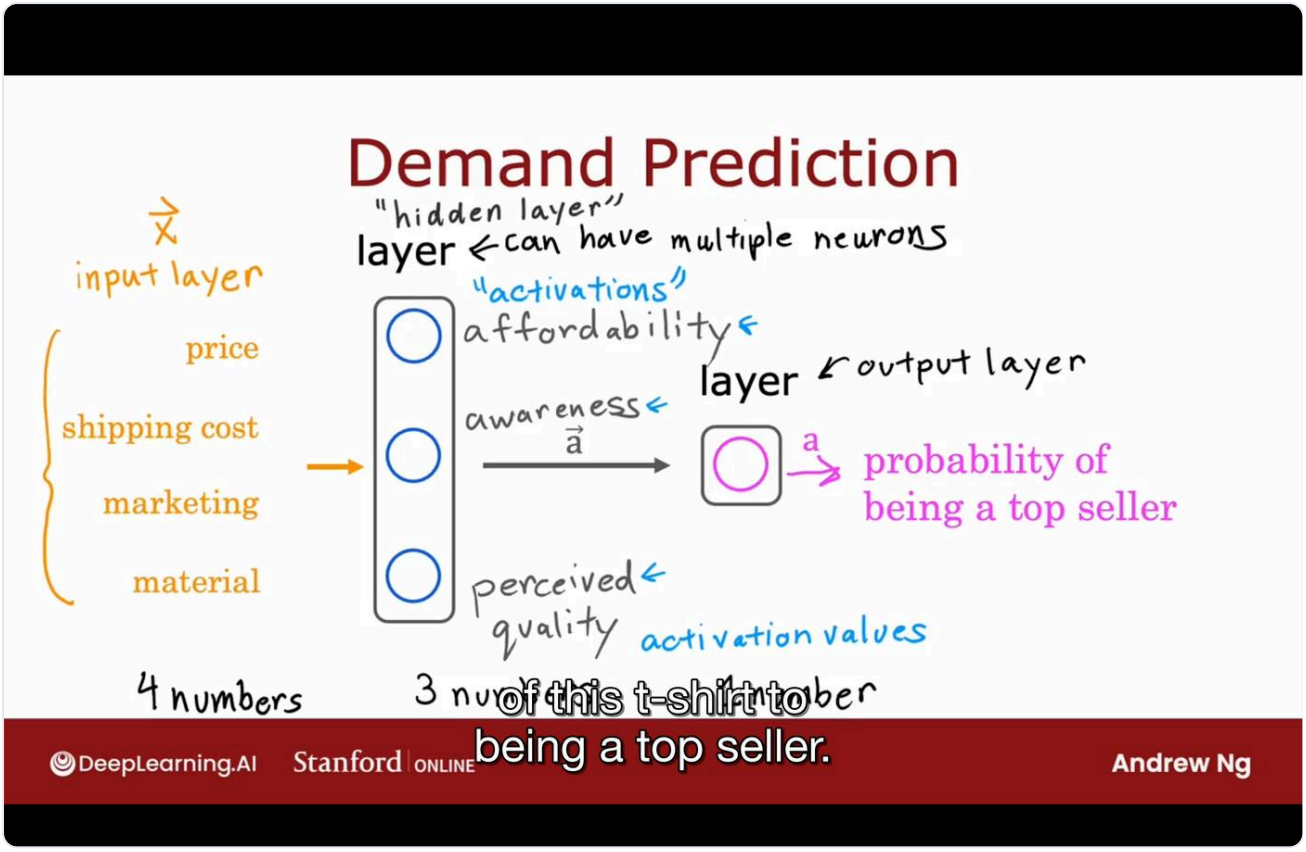


FIGURE 3.14 | Demand Prediction

It has a few layers where each layer inputs a vector and outputs another vector of numbers. For example, this layer in the middle inputs four numbers x and outputs three numbers corresponding to affordability, awareness, and perceived quality. To add a little bit more terminology, you've seen that this layer is called the output layer and this layer is called the input layer. To give the layer in the middle a name as well, this layer in the middle is called a hidden layer.

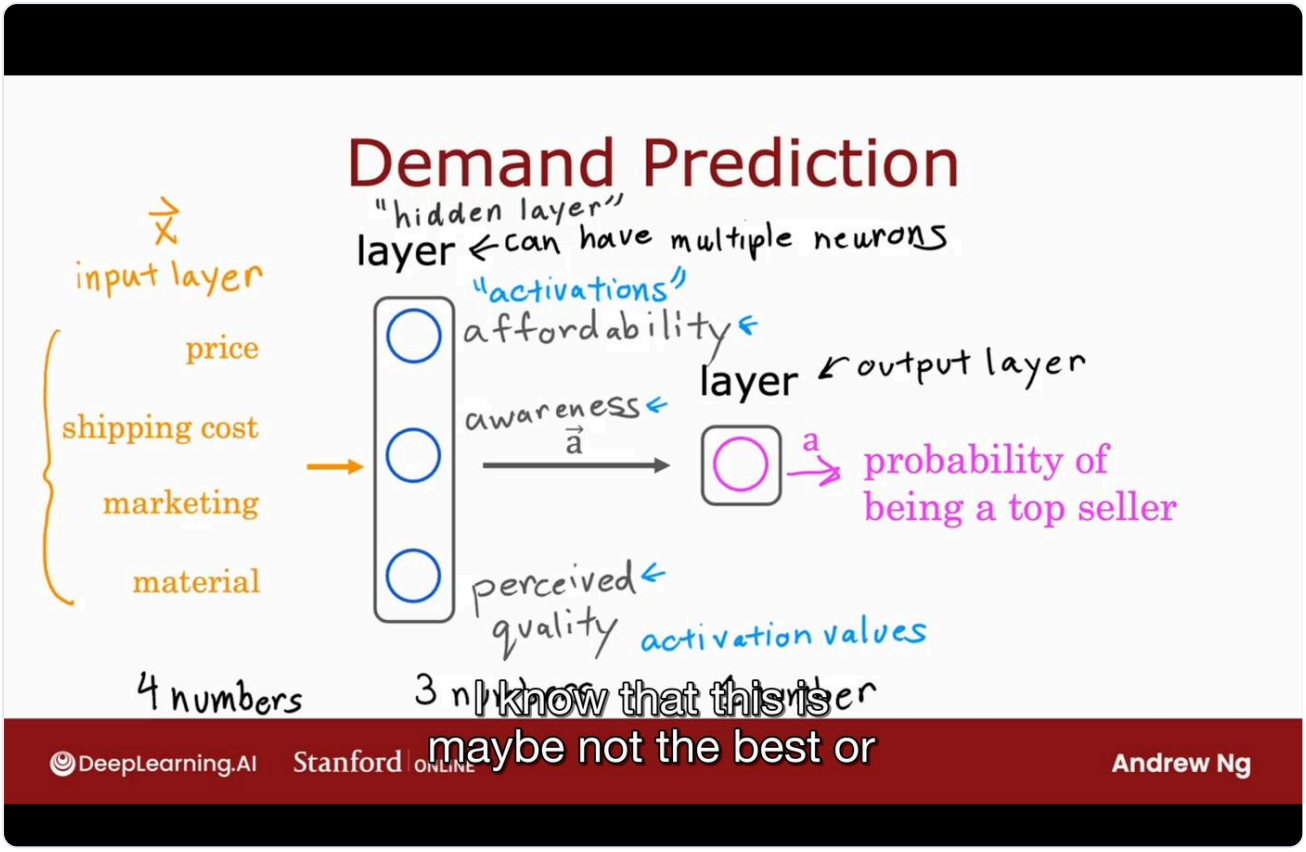


FIGURE 3.15 | Demand Prediction

I know that this is maybe not the best or the most intuitive name but that terminology comes from that's when you have a training set. In a training set, you get to observe both x and y. Your data set tells you what is x and what is y, and so you get data that tells you what are the correct inputs and the correct outputs. But your dataset doesn't tell you what are the correct values for affordability, awareness, and perceived quality. The correct values for those are hidden. You don't see them in the training set, which is why this layer in the middle is called a hidden layer. I'd like to share with you another way of thinking about neural networks that I've found useful for building my intuition about it.

Just let me cover up the left half of this diagram, and see what we're left with. What you see here is that there is a logistic regression algorithm or logistic regression unit that is taking as input, affordability, awareness, and perceived quality of a t-shirt, and using these three features to estimate the probability of the t-shirt being a top seller. This is just logistic regression.

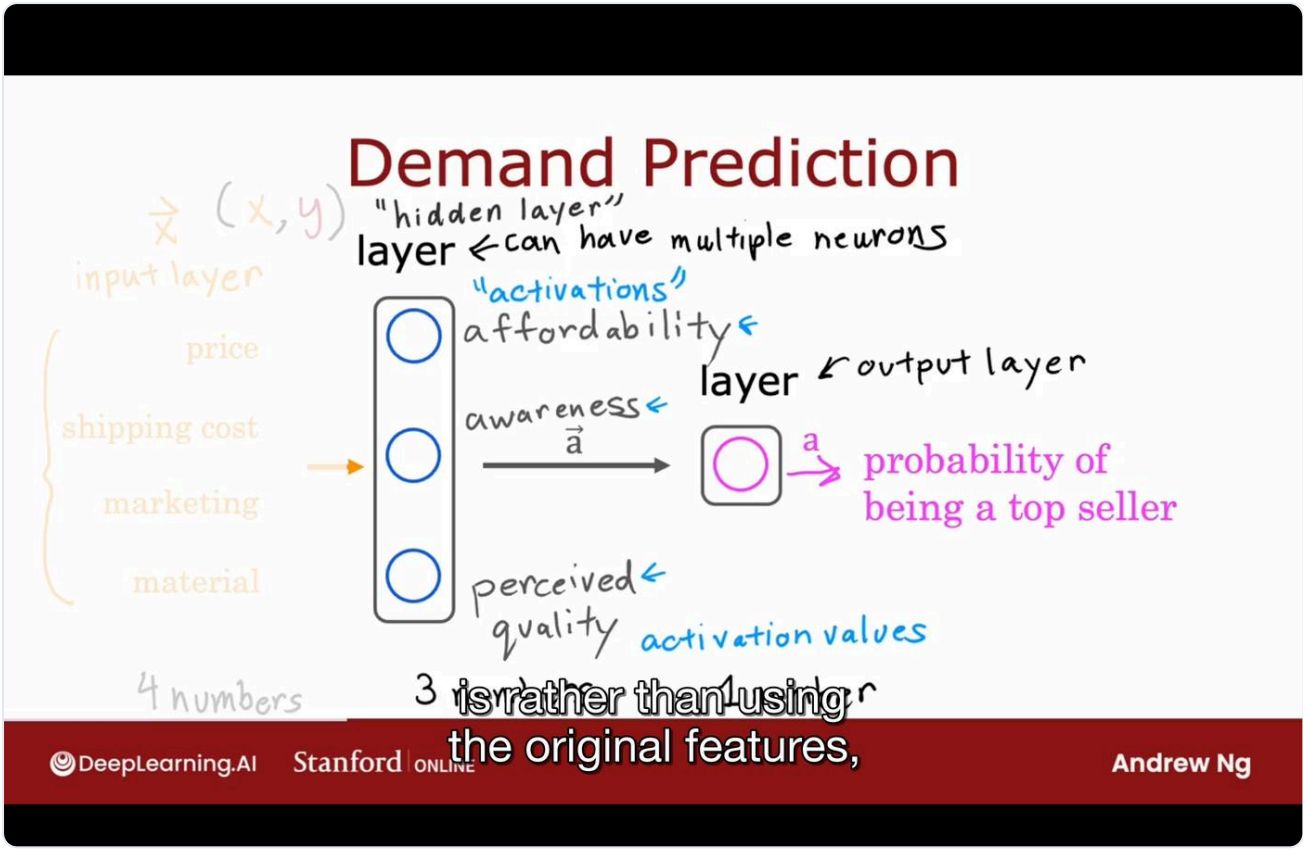


FIGURE 3.16 | Demand Prediction

But the cool thing about this is rather than using the original features, price, shipping cost, marketing, and so on, is using maybe better set of features, affordability, awareness, and perceived quality, that are hopefully more predictive of whether or not this t-shirt will be a top seller. One way to think of this neural network is, just logistic regression. But as a version of logistic regression, they can learn its own features that makes it easier to make accurate predictions. In fact, you might remember from the previous course, this housing example where we said that if you want to predict the price of the house, you might take the frontage or the width of lots and multiply that by the depth of a lot to construct a more complex feature, x_1 times x_2, which was the size of the lawn.

There we were doing manual feature engineering where we had to look at the features x_1 and x_2 and decide by hand how to combine them together to come up with better features. What the neural network does is instead of you needing to manually engineer the features, it can learn, as you'll see later, its own features to make the learning problem easier for itself. This is what makes neural networks one of the most powerful learning algorithms in the world today. To summarize, a neural network, does this, the input layer has a vector of features, four numbers in this example, it is input to the hidden layer, which outputs three numbers. I'm going to use a vector to denote this vector of activations that this hidden layer outputs.

Then the output layer takes its input to three numbers and outputs one number, which would be the final activation, or the final prediction of the neural network. One note, even though I previously described this neural network as computing affordability, awareness, and perceived quality, one of the really nice properties of a neural network is when you train it from data, you don't

need to go in to explicitly decide what other features, such as affordability and so on, that the neural network should compute instead or figure out all by itself what are the features it wants to use in this hidden layer. That's what makes it such a powerful learning algorithm.

You've seen here one example of a neural network and this neural network has a single layer that is a hidden layer. Let's take a look at some other examples of neural networks, specifically, examples with more than one hidden layer. Here's an example. This neural network has an input feature vector X that is fed to one hidden layer. I'm going to call this the first hidden layer. If this hidden layer has three neurons, it will then output a vector of three activation values. These three numbers can then be input to the second hidden layer.

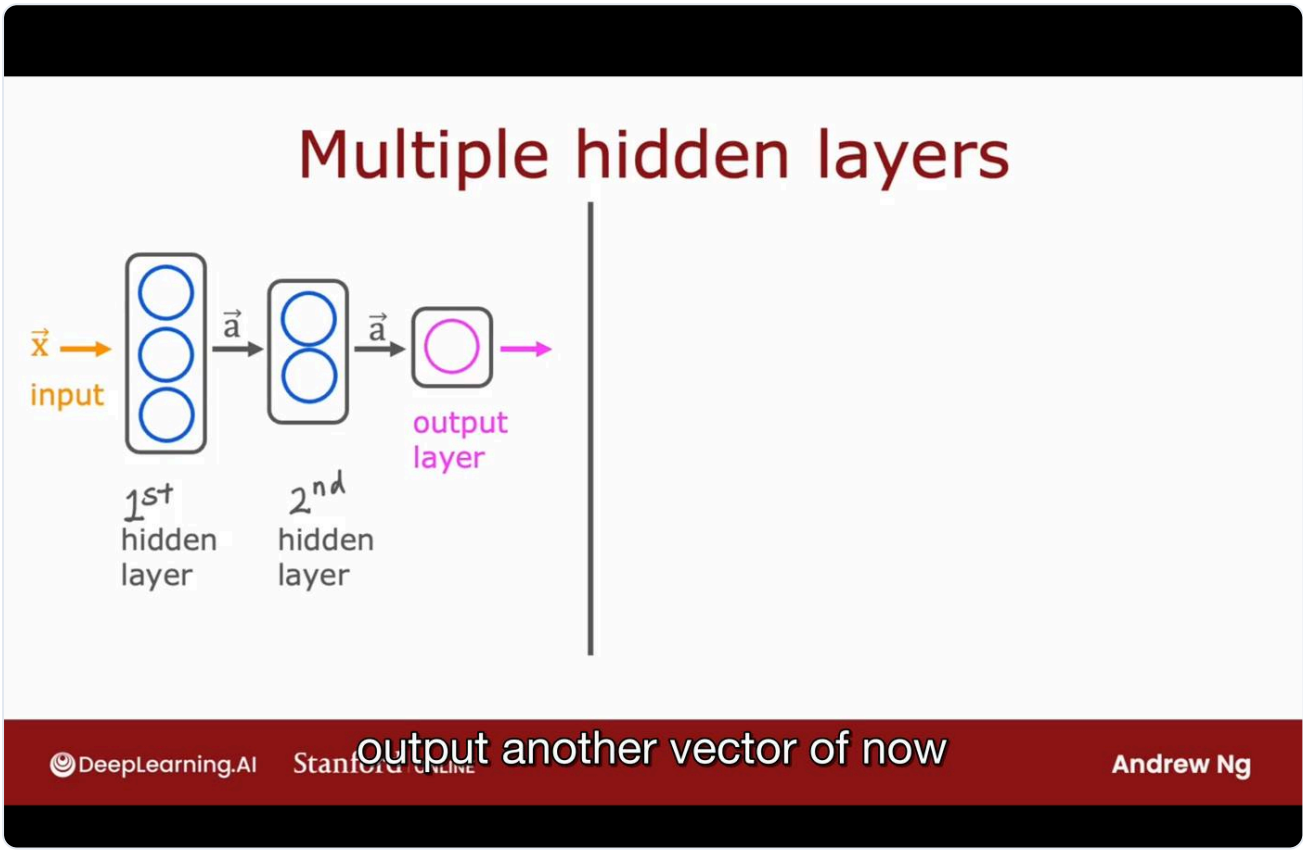


FIGURE 3.17 | Multiple Hidden Layers

If the second hidden layer has two neurons to logistic units, then this second hidden there will output another vector of now two activation values that maybe goes to the output layer that then outputs the neural network's final prediction. Here's another example. Here's a neural network that it's input goes to the first hidden layer, the output of the first hidden layer goes to the second hidden layer, goes to the third hidden layer, and then finally to the output layer. When you're building your own neural network, one of the decisions you need to make is how many hidden layers do you want and how many neurons do you want each hidden layer to have.

This question of how many hidden layers and how many neurons per hidden layer is a question of the architecture of the neural network.

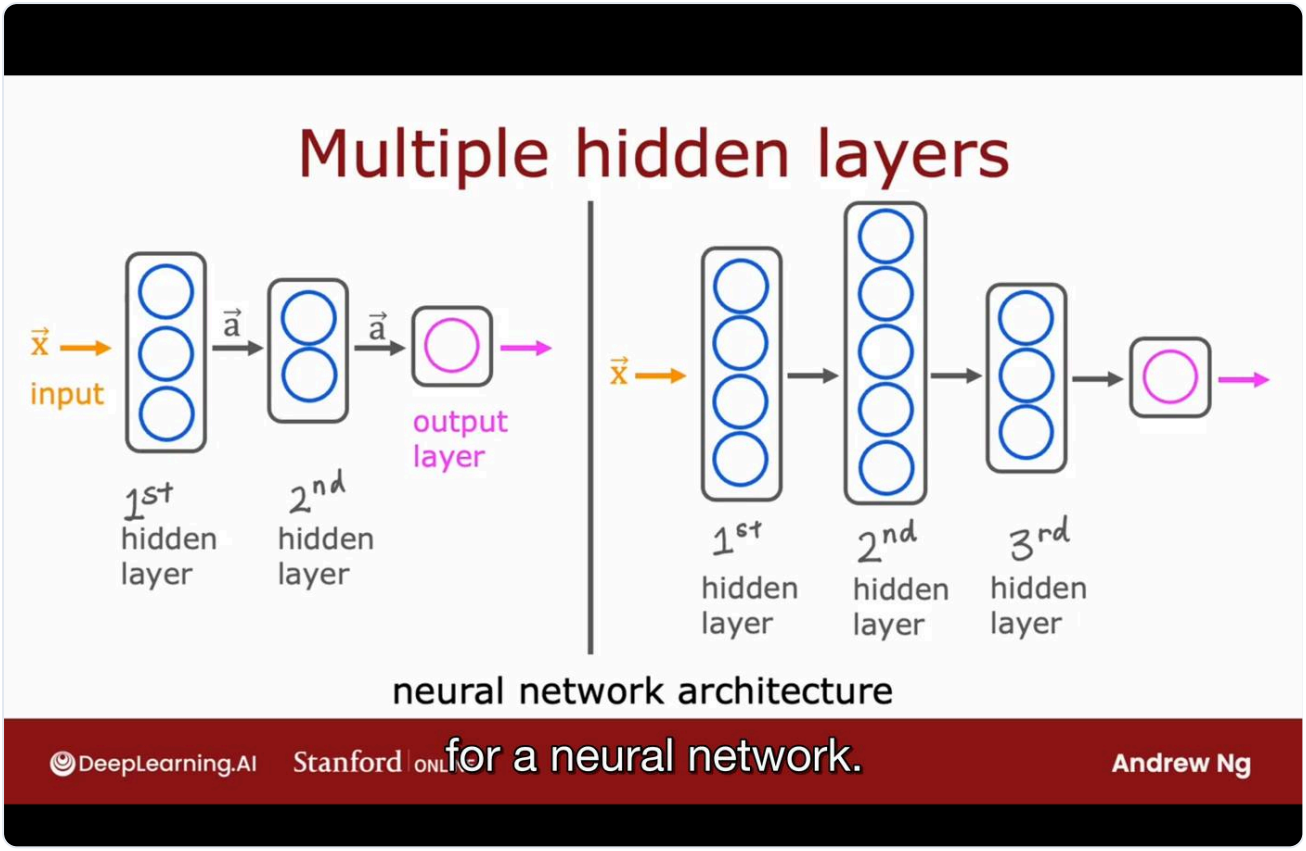


FIGURE 3.18 | Multiple Hidden on

You'll learn later in this course some tips for choosing an appropriate architecture for a neural network. But choosing the right number of hidden layers and number of hidden units per layer can have an impact on the performance of a learning algorithm as well. Later in this course, you'll learn how to choose a good architecture for your neural network as well. By the way, in some of the literature, you see this type of neural network with multiple layers like this called a multilayer perceptron.

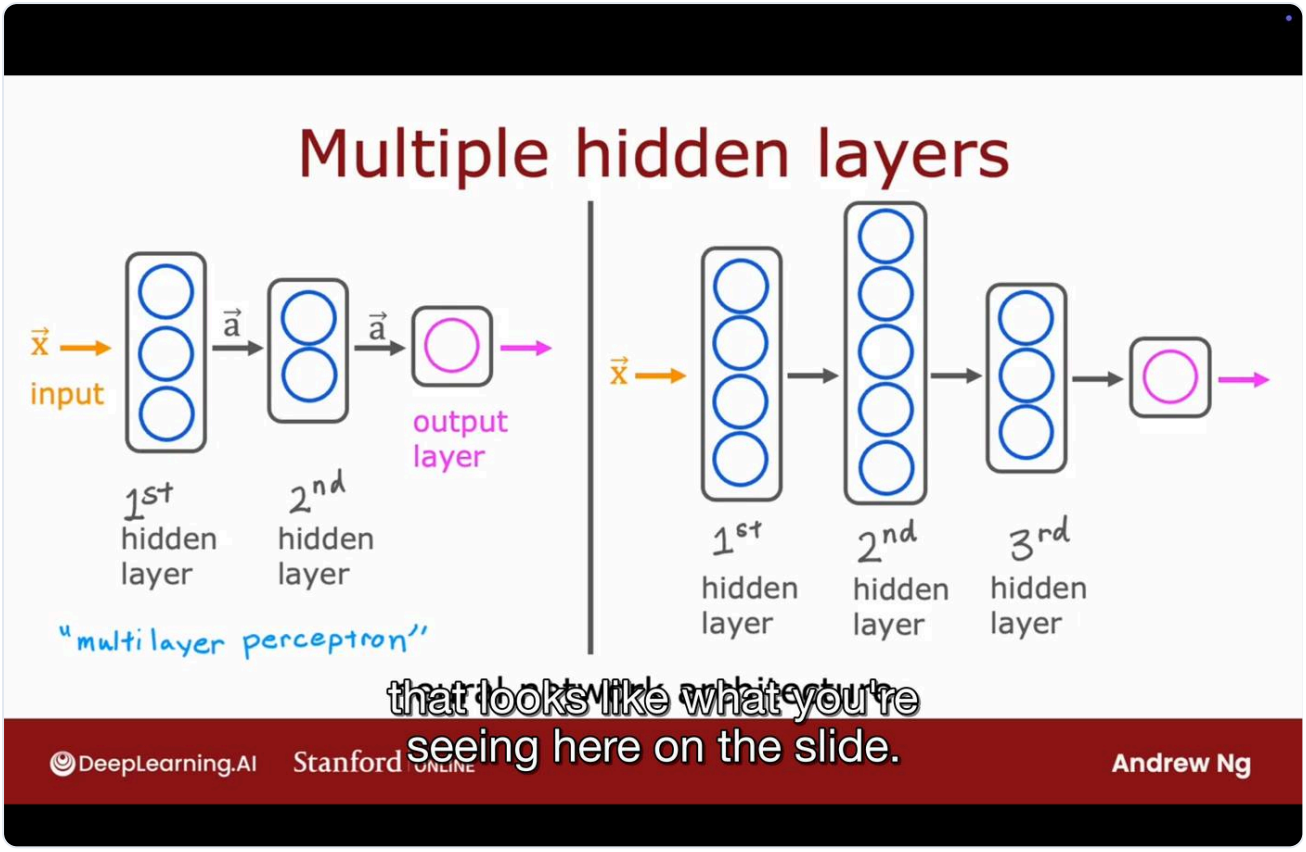


FIGURE 3.19 | Multiple Hidden on

If you see that, that just refers to a neural network that looks like what you're seeing here on the slide. That's a neural network. I know we went through a lot in this video. Thank you for sticking with me. But you now know how a neural network works. In the next video, let's take a look at how these ideas can be applied to other applications as well. In particular, we'll take a look at the computer vision application of face recognition. Let's go on to the next video.

LECTURE 4

Face Recognition

In the last video, you saw how a neural network works in a demand prediction example. Let's take a look at how you can apply a similar type of idea to computer vision application. Let's dive in. If you're building a face recognition application, you might want to train a neural network that takes as input a picture like this and outputs the identity of the person in the picture. This image is 1,000 by 1,000 pixels.

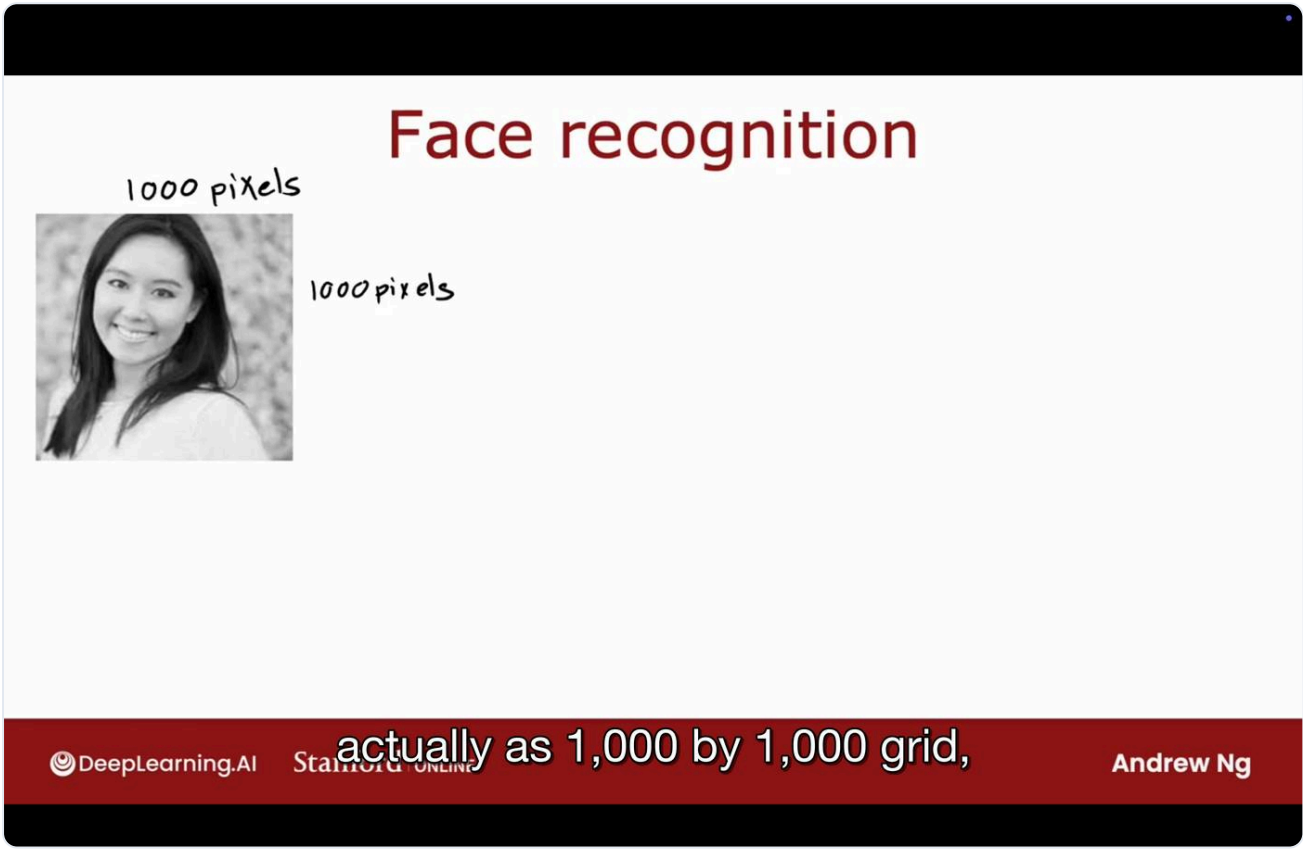


FIGURE 4.1 | Face Recognition

Its representation in the computer is actually as 1,000 by 1,000 grid, or also called 1,000 by 1,000 matrix of pixel intensity values. In this example, my pixel intensity values or pixel brightness values, goes from 0-255 and so 197 here would be the brightness of the pixel in the very upper left of the image, 185 is brightness of the pixel, one pixel over, and so on down to 214 would be the lower right corner of this image.

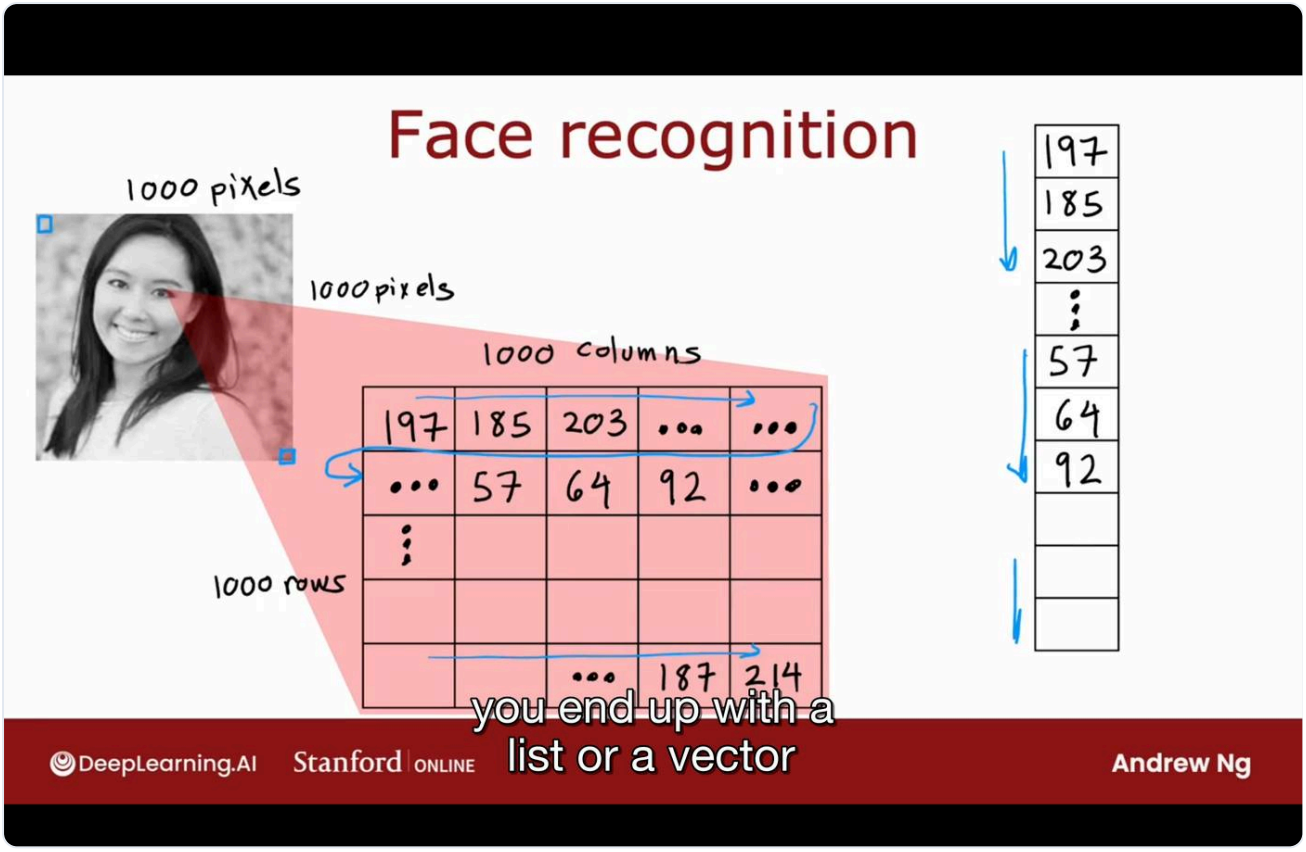


FIGURE 4.2

If you were to take these pixel intensity values and unroll them into a vector, you end up with a list or a vector of a million pixel intensity values. One million because 1,000 by 1,000 square gives you a million numbers. The face recognition problem is, can you train a neural network that takes as input a feature vector with a million pixel brightness values and outputs the identity of the person in the picture. This is how you might build a neural network to carry out this task. The input image X is fed to this layer of neurons. This is the first hidden layer, which then extract some features.

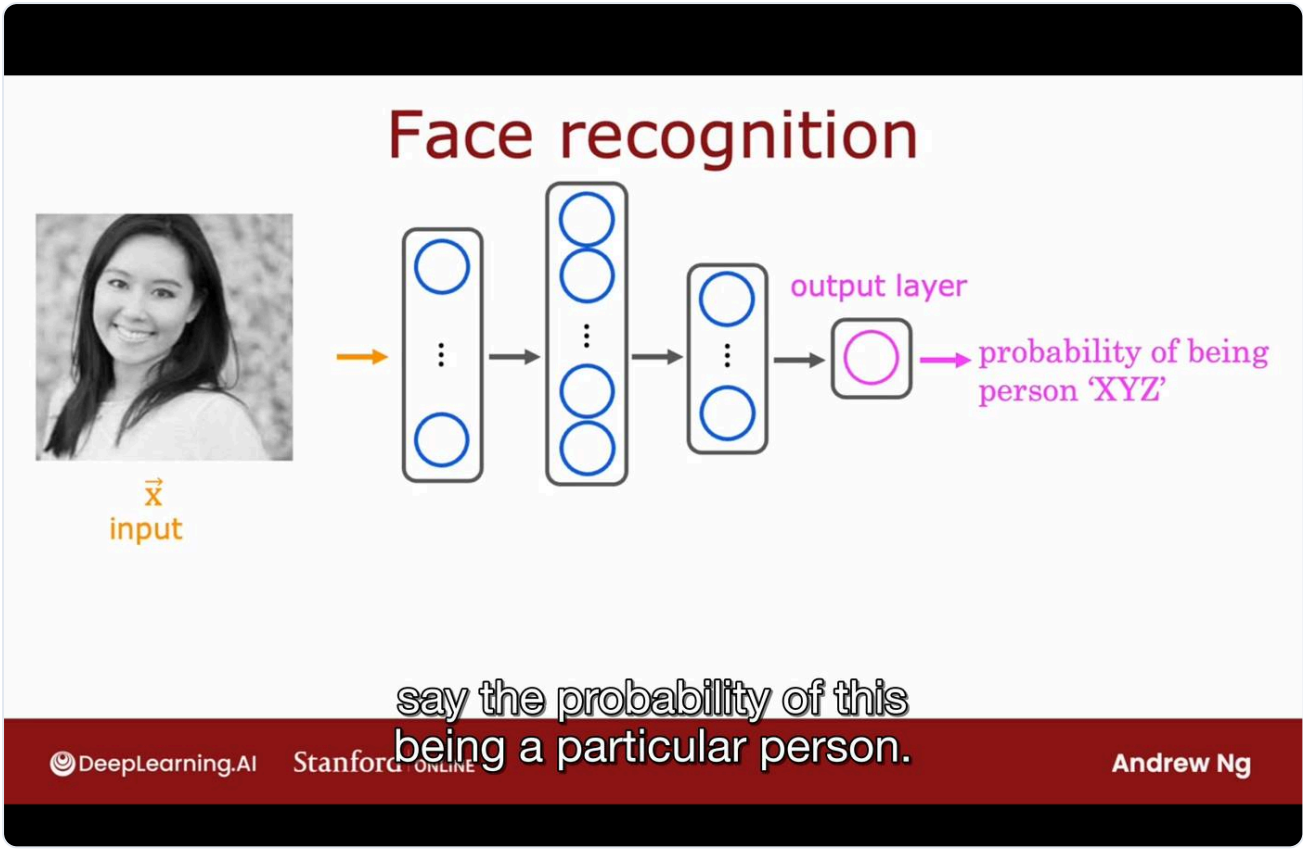


FIGURE 4.3

The output of this first hidden layer is fed to a second hidden layer and that output is fed to a third layer and then finally to the output layer, which then estimates, say the probability of this being a particular person. One interesting thing would be if you look at a neural network that's been trained on a lot of images of faces and to try to visualize what are these hidden layers, trying to compute. It turns out that when you train a system like this on a lot of pictures of faces and you peer at the different neurons in the hidden layers to figure out what they may be computing this is what you might find. In the first hidden layer, you might find one neuron that is looking for the low vertical line or a vertical edge like that.

A second neuron looking for a oriented line or oriented edge like that.

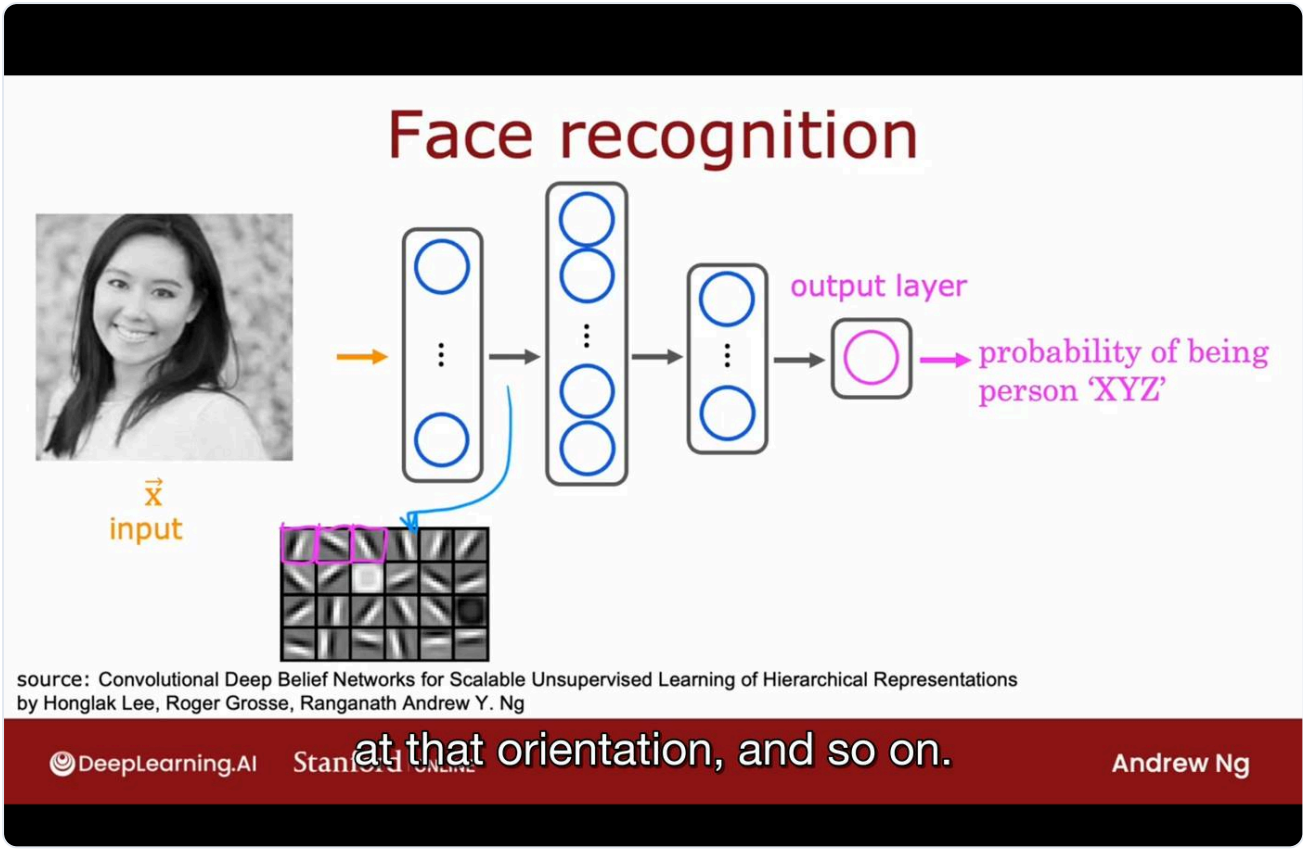


FIGURE 4.4 | Face Recognition

The third neuron looking for a line at that orientation, and so on. In the earliest layers of a neural network, you might find that the neurons are looking for very short lines or very short edges in the image.

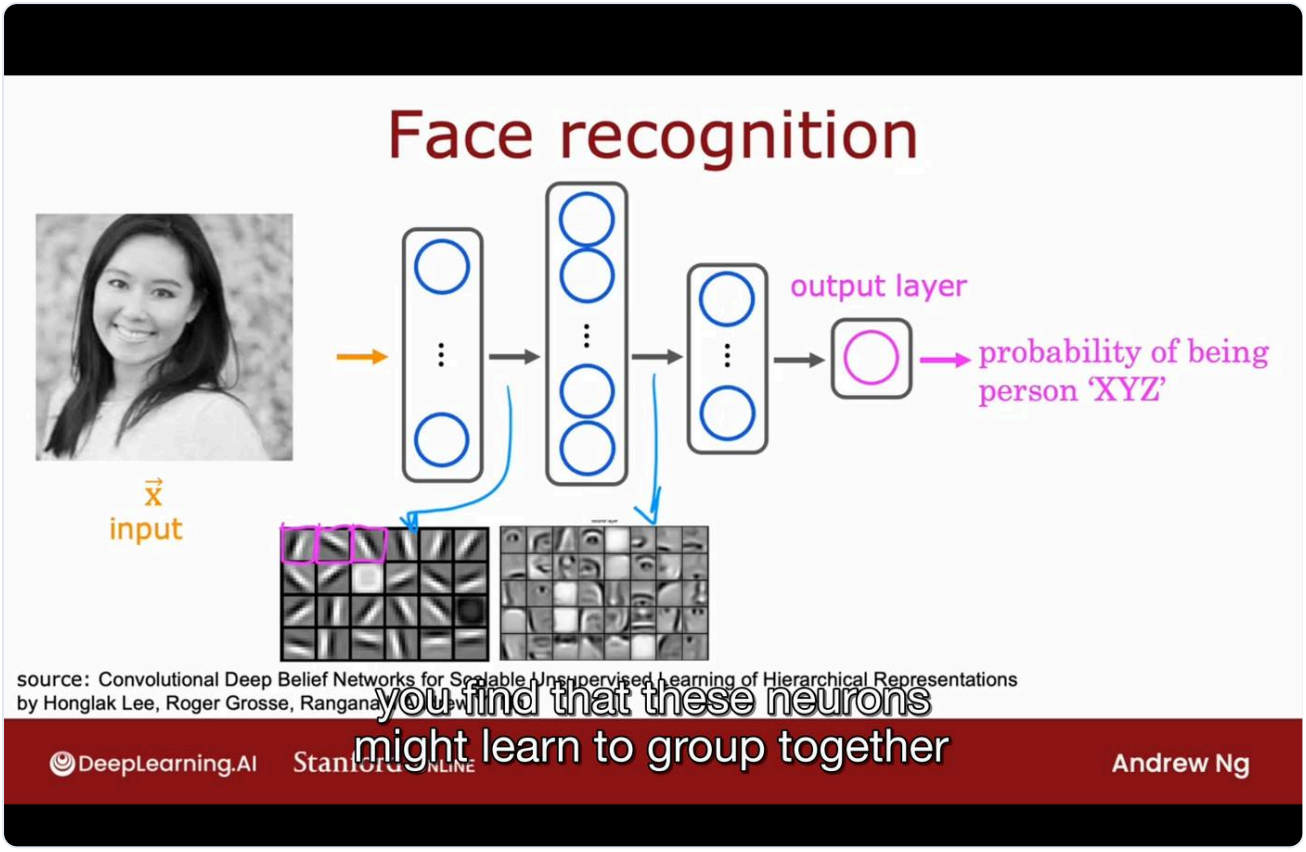


FIGURE 4.5

If you look at the next hidden layer, you find that these neurons might learn to group together lots of little short lines and little short edge segments in order to look for parts of faces. For example, each of these little square boxes is a visualization of what that neuron is trying to detect. This first neuron looks like it's trying to detect the presence or absence of an eye in a certain position of the image.

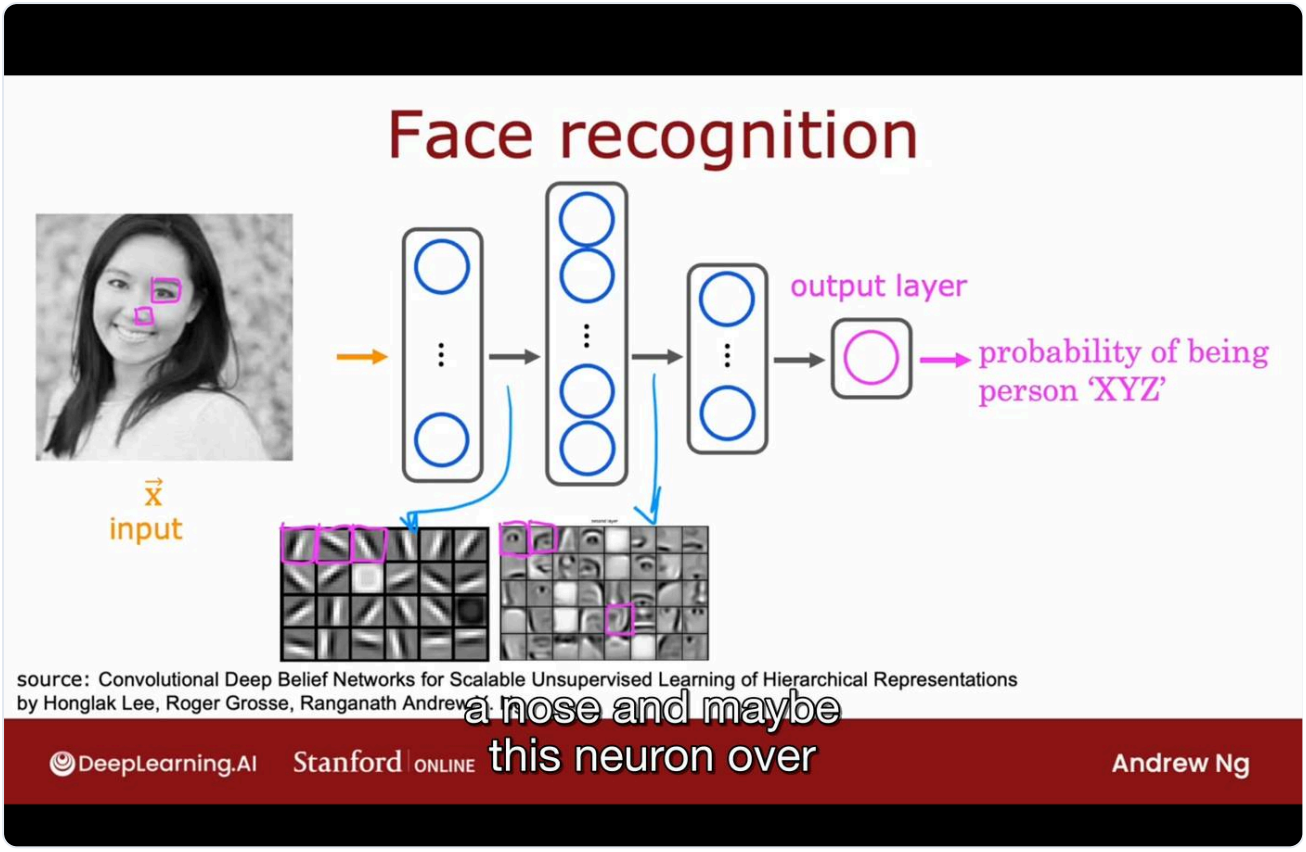


FIGURE 4.6 | Face Recognition

The second neuron, looks like it's trying to detect like a corner of a nose and maybe this neuron over here is trying to detect the bottom of an ear.

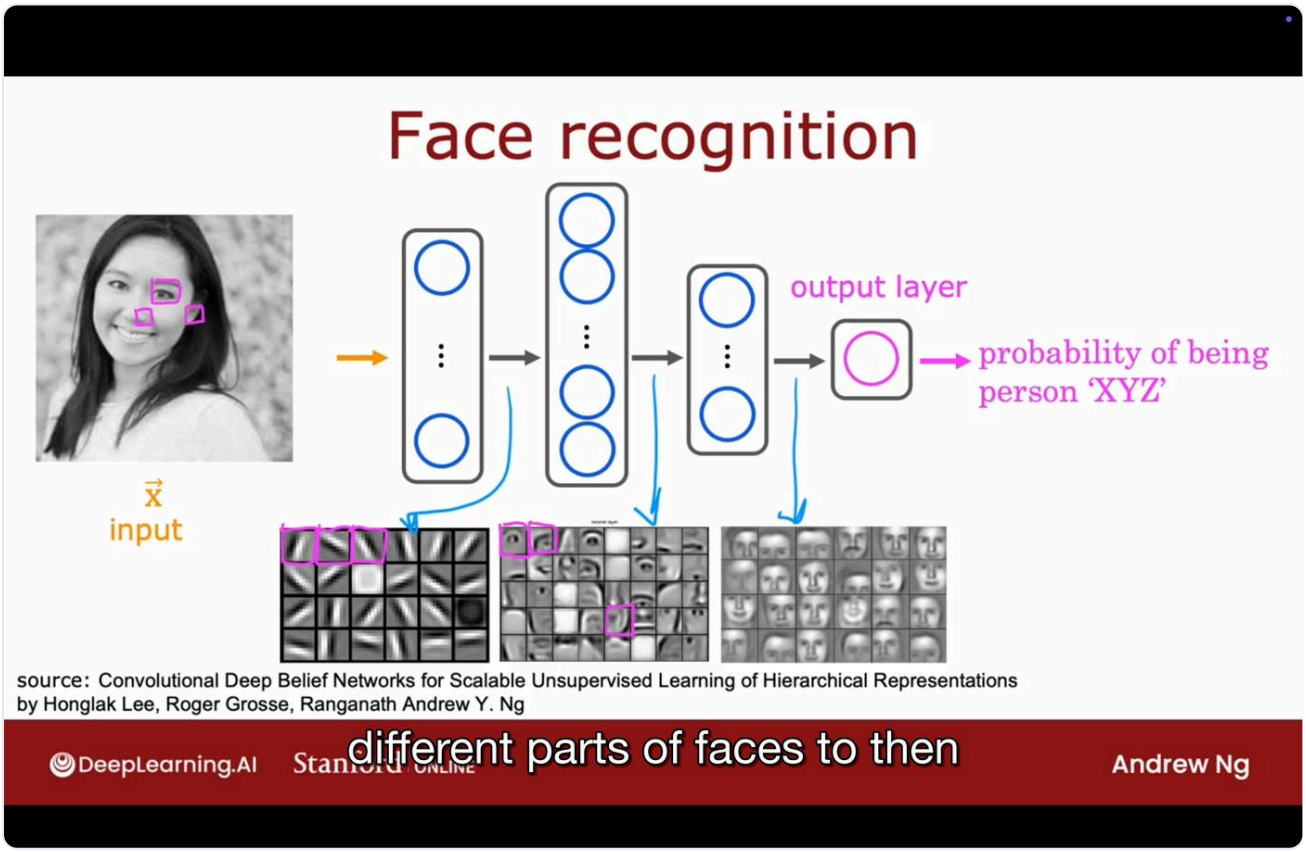


FIGURE 4.7

Then as you look at the next hidden layer in this example, the neural network is aggregating different parts of faces to then try to detect presence or absence of larger, coarser face shapes. Then finally, detecting how much the face corresponds to different face shapes creates a rich set of features that then helps the output layer try to determine the identity of the person picture. A remarkable thing about the neural network is you can learn these feature detectors at the different hidden layers all by itself. In this example, no one ever told it to look for short little edges in the first layer, and eyes and noses and face parts in the second layer and then more complete face shapes at the third layer.

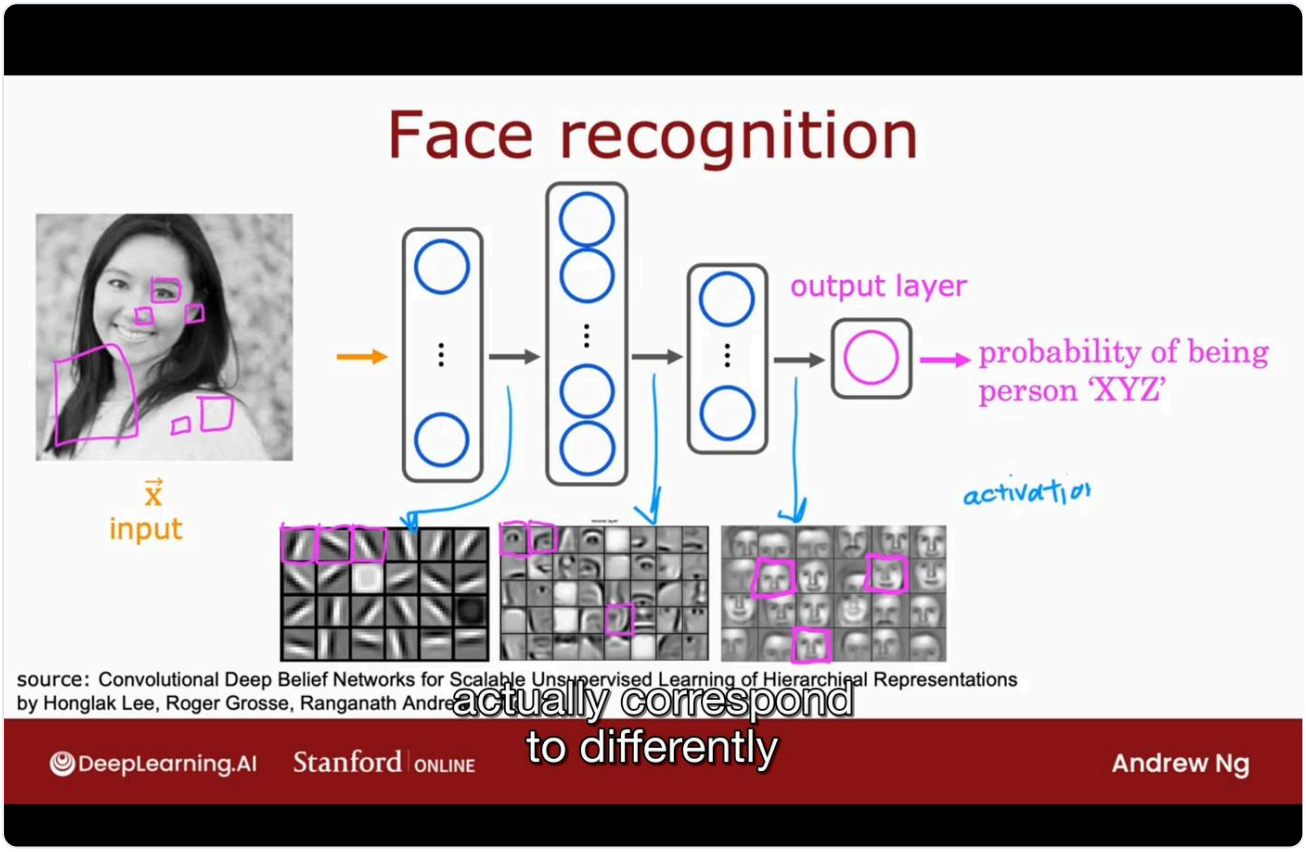


FIGURE 4.8

The neural network is able to figure out these things all by itself from data. Just one note, in this visualization, the neurons in the first hidden layer are shown looking at relatively small windows to look for these edges. In the second hidden layer is looking at bigger window, and the third hidden layer is looking at even bigger window.

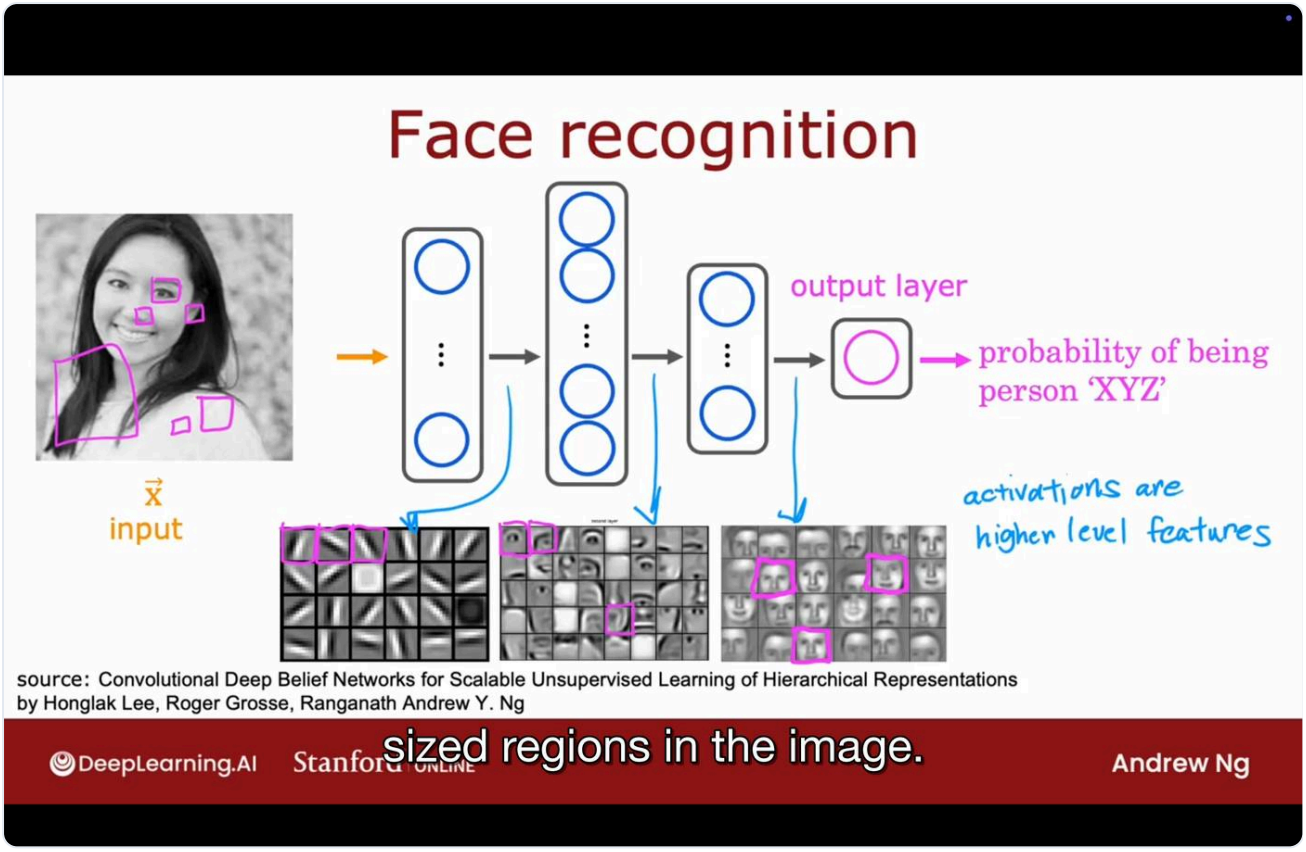


FIGURE 4.9 | Face Recognition

These little neurons visualizations actually correspond to differently sized regions in the image. Just for fun, let's see what happens if you were to train this neural network on a different dataset, say on lots of pictures of cars, picture on the side. The same learning algorithm is asked to detect cars, will then learn edges in the first layer.

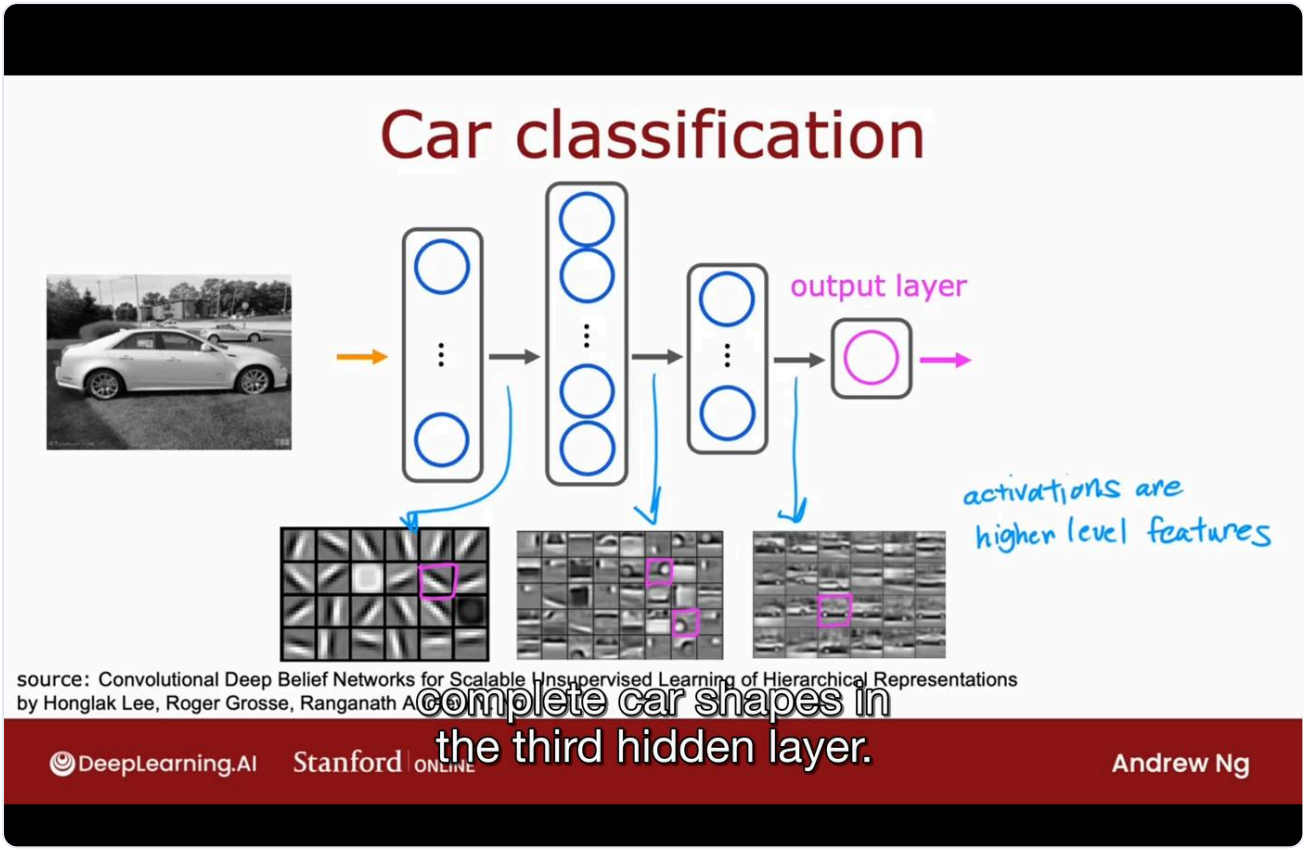


FIGURE 4.10 | Car a

Pretty similar but then they'll learn to detect parts of cars in the second hidden layer and then more complete car shapes in the third hidden layer.

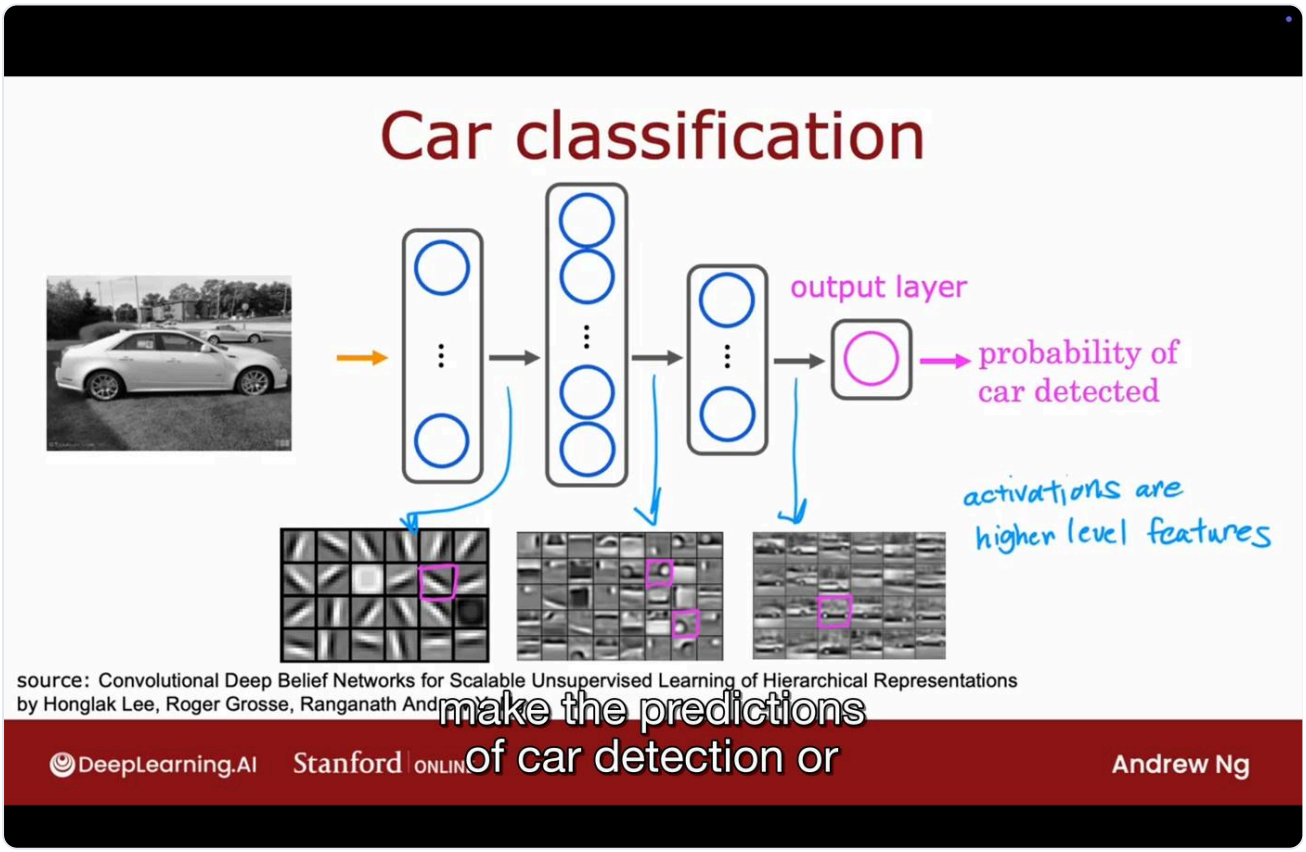


FIGURE 4.11

Just by feeding it different data, the neural network automatically learns to detect very different features so as to try to make the predictions of car detection or person recognition or whether there's a particular given task that is trained on. That's how a neural network works for computer vision application. In fact, later this week, you'll see how you can build a neural network yourself and apply it to a handwritten digit recognition application. So far we've been going over the description of intuitions of neural networks to give you a feel for how they work. In the next video, let's look more deeply into the concrete mathematics and a concrete implementation of details of how you actually build one or more layers of a neural network, and therefore how you can implement one of these things yourself.

Let's go on to the next video.

LECTURE 5

Neural Network Layer

The fundamental building block of most modern neural networks is a layer of neurons. In this video, you'll learn how to construct a layer of neurons and once you have that down, you'd be able to take those building blocks and put them together to form a large neural network. Let's take a look at how a layer of neurons works. Here's the example we had from the demand prediction example where we had four input features that were set to this layer of three neurons in the hidden layer that then sends its output to this output layer with just one neuron. Let's zoom in to the hidden layer to look at its computations.

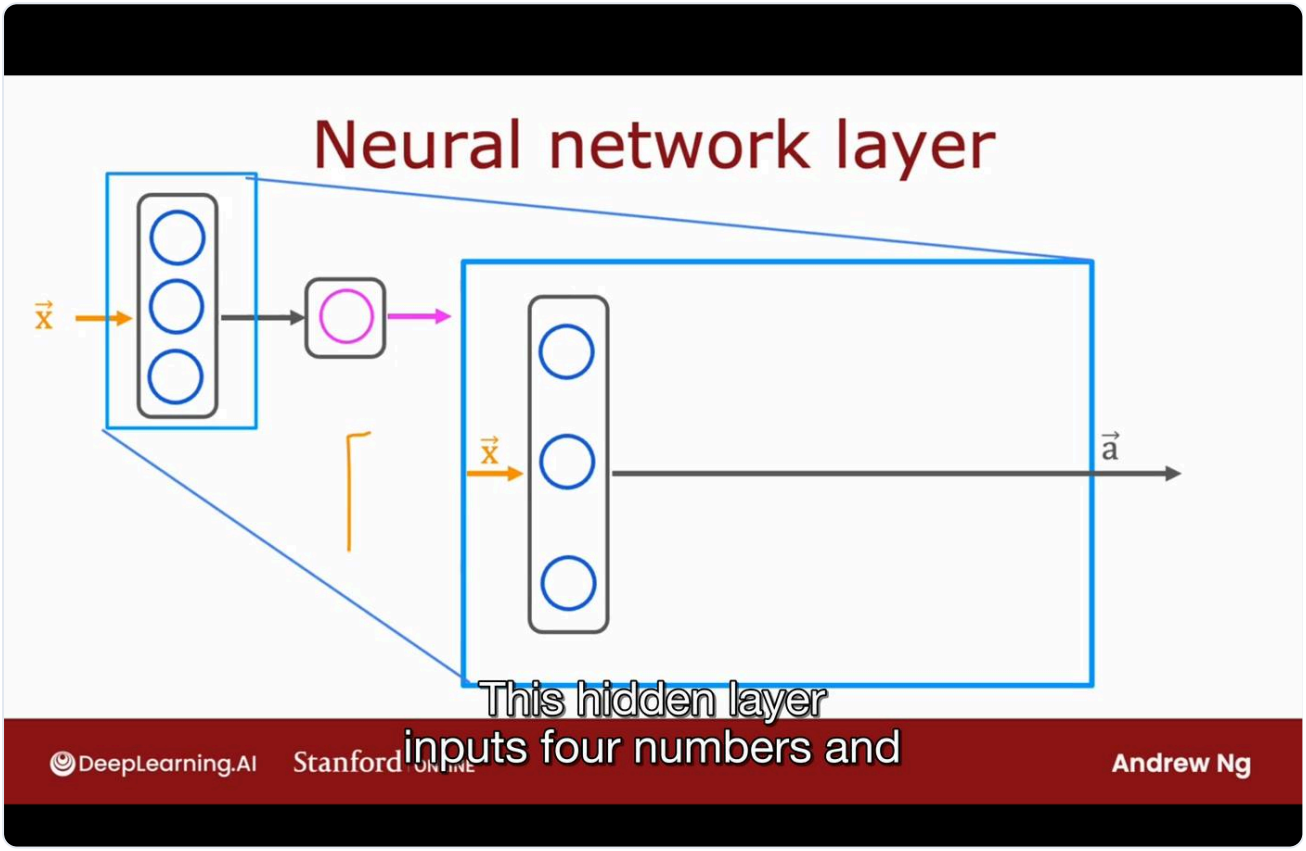


FIGURE 5.1 | Neural Network Layer

This hidden layer inputs four numbers and these four numbers are inputs to each of three neurons. Each of these three neurons is just implementing a little logistic regression unit or a little bit logistic regression function. Take this first neuron. It has two parameters, w and b. In fact, to denote that, this is the first hidden unit, I'm going to subscript this as w_1, b_1.

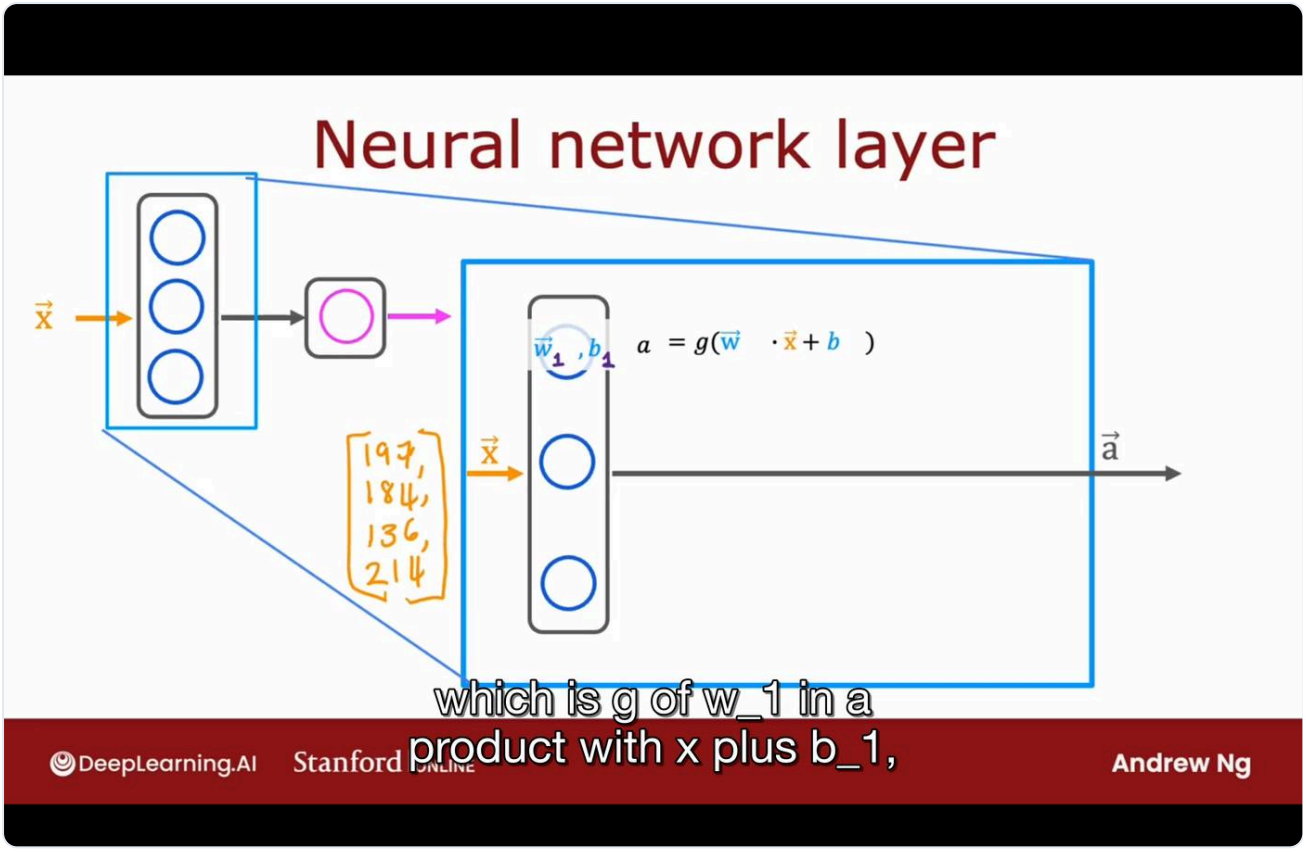


FIGURE 5.2 | Neural Network Layer

What it does is I'll output some activation value a, which is g of w_1 in a product with x plus b_1, where this is the familiar z value that you have learned about in logistic regression in the previous course, and g of z is the familiar logistic function, 1 over 1 plus e to the negative z. Maybe this ends up being a number 0.3 and that's the activation value a of the first neuron. To denote that this is the first neuron, I'm also going to add a subscript a_1 over here, and so a_1 may be a number like 0.3. There's a 0.3 chance of this being highly affordable based on the input features. Now let's look at the second neuron.

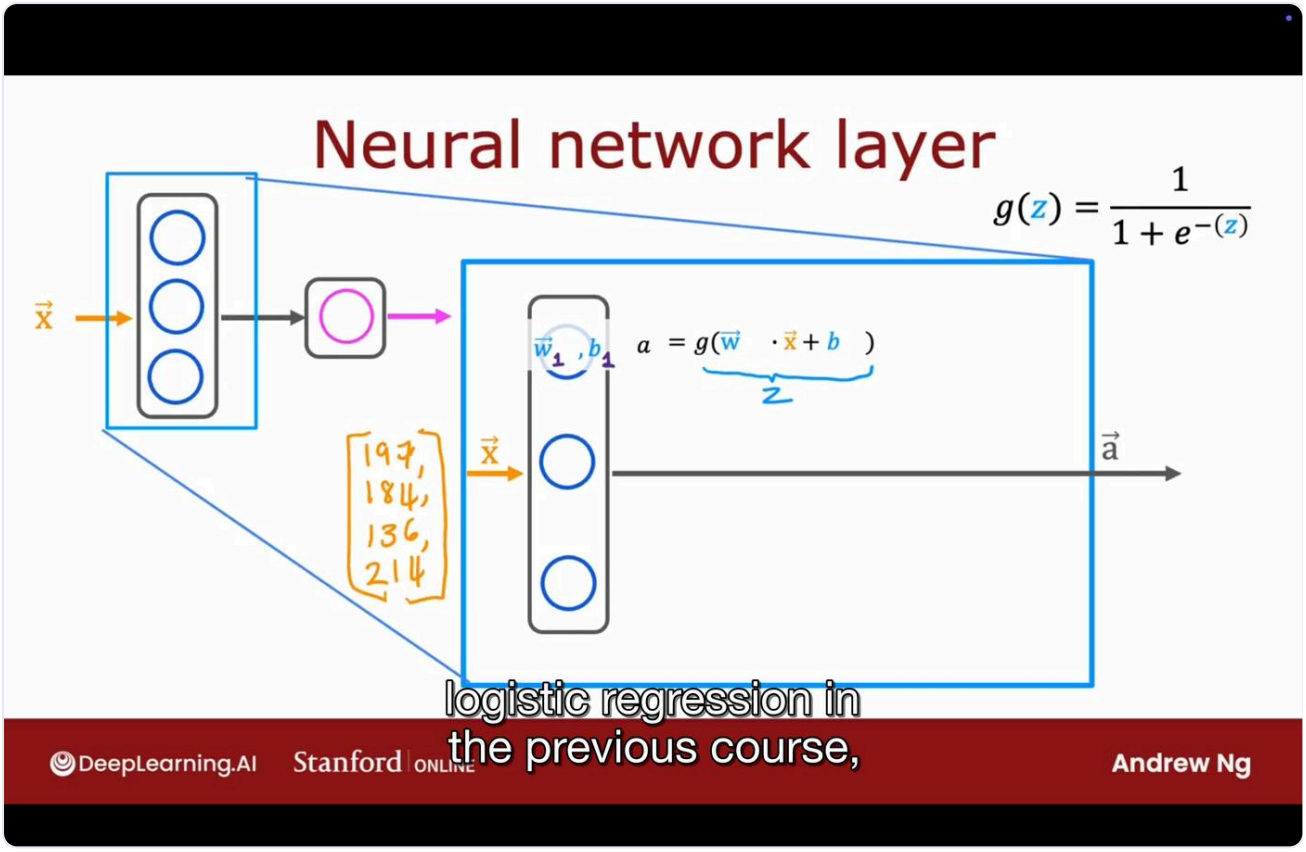


FIGURE 5.3 | Neural Network Layer

The second neuron has parameters w_2 and b_2, and these w, b or w_2, b_2 are the parameters of the second logistic unit. It computes a_2 equals the logistic function g applied to w_2 dot product x plus b_2 and this may be some other number, say 0.7. Because in this example, there's a 0.7 chance that we think the potential buyers will be aware of this t-shirt. Similarly, the third neuron has a third set of parameters w_3, b_3. Similarly, it computes an activation value a_3 equals g of w_3 dot product x plus b_3 and that may be say, 0.2.

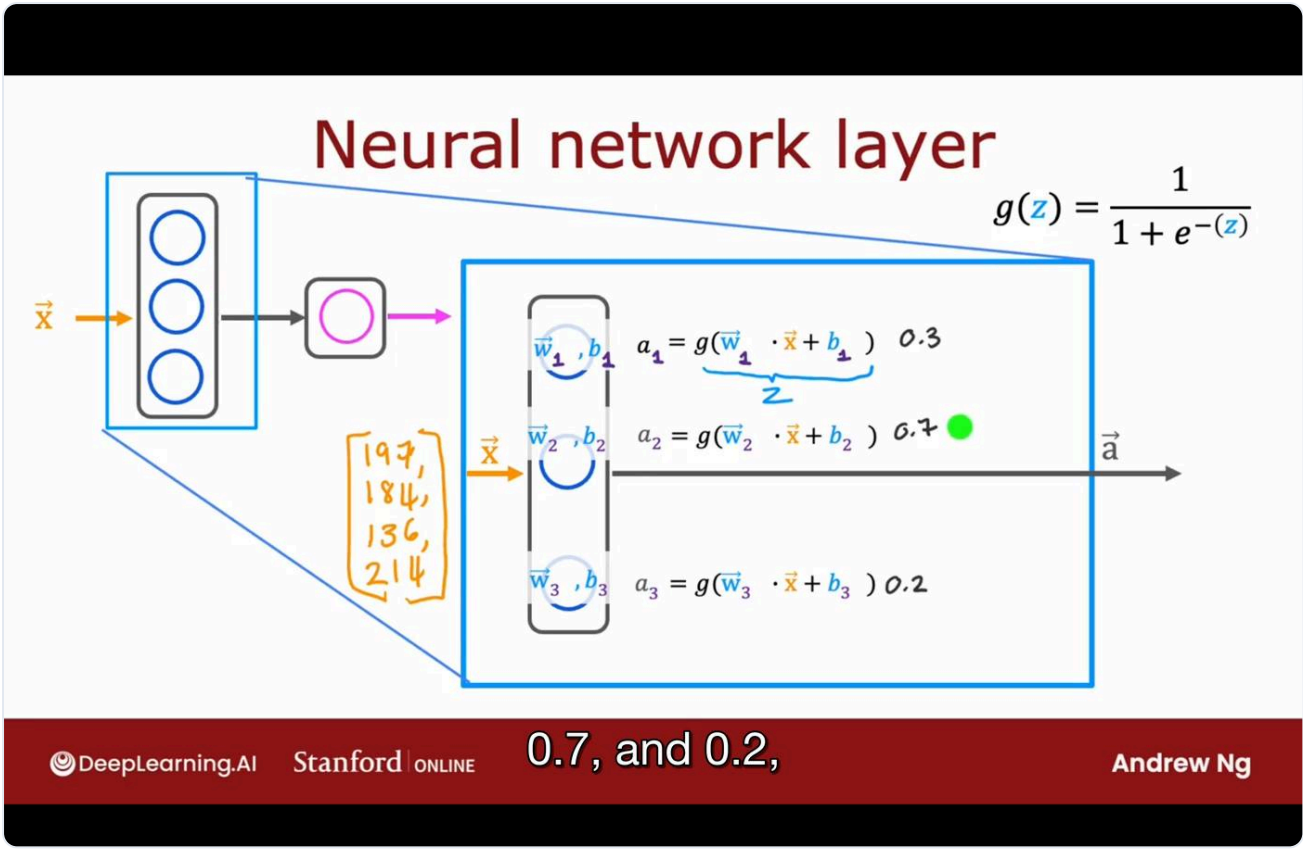


FIGURE 5.4 | Neural Network Layer

In this example, these three neurons output 0.3, 0.7, and 0.2, and this vector of three numbers becomes the vector of activation values a, that is then passed to the final output layer of this neural network. Now, when you build neural networks with multiple layers, it'll be useful to give the layers different numbers. By convention, this layer is called layer 1 of the neural network and this layer is called layer 2 of the neural network.

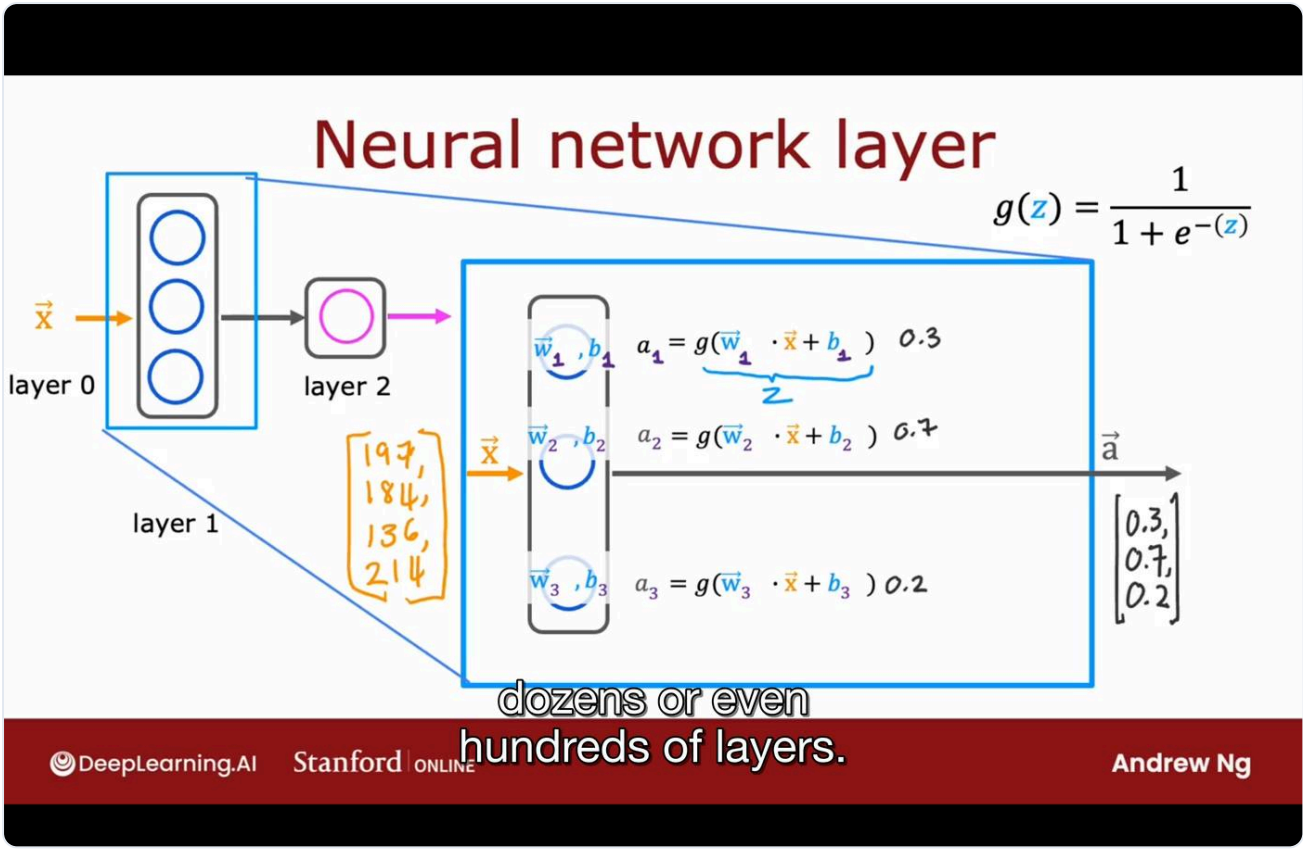


FIGURE 5.5 | Neural Network Layer

The input layer is also sometimes called layer 0 and today, there are neural networks that can have dozens or even hundreds of layers. But in order to introduce notation to help us distinguish between the different layers, I'm going to use superscript square bracket 1 to index into different layers. In particular, a superscript in square brackets 1, I'm going to use, that's a notation to denote the output of layer 1 of this hidden layer of this neural network, and similarly, w_1, b_1 here are the parameters of the first unit in layer 1 of the neural network, so I'm also going to add a superscript in square brackets 1 here, and w_2, b_2 are the parameters of the second hidden unit or the second hidden neuron in layer 1.

Its parameters are also denoted here w^[1] like so.

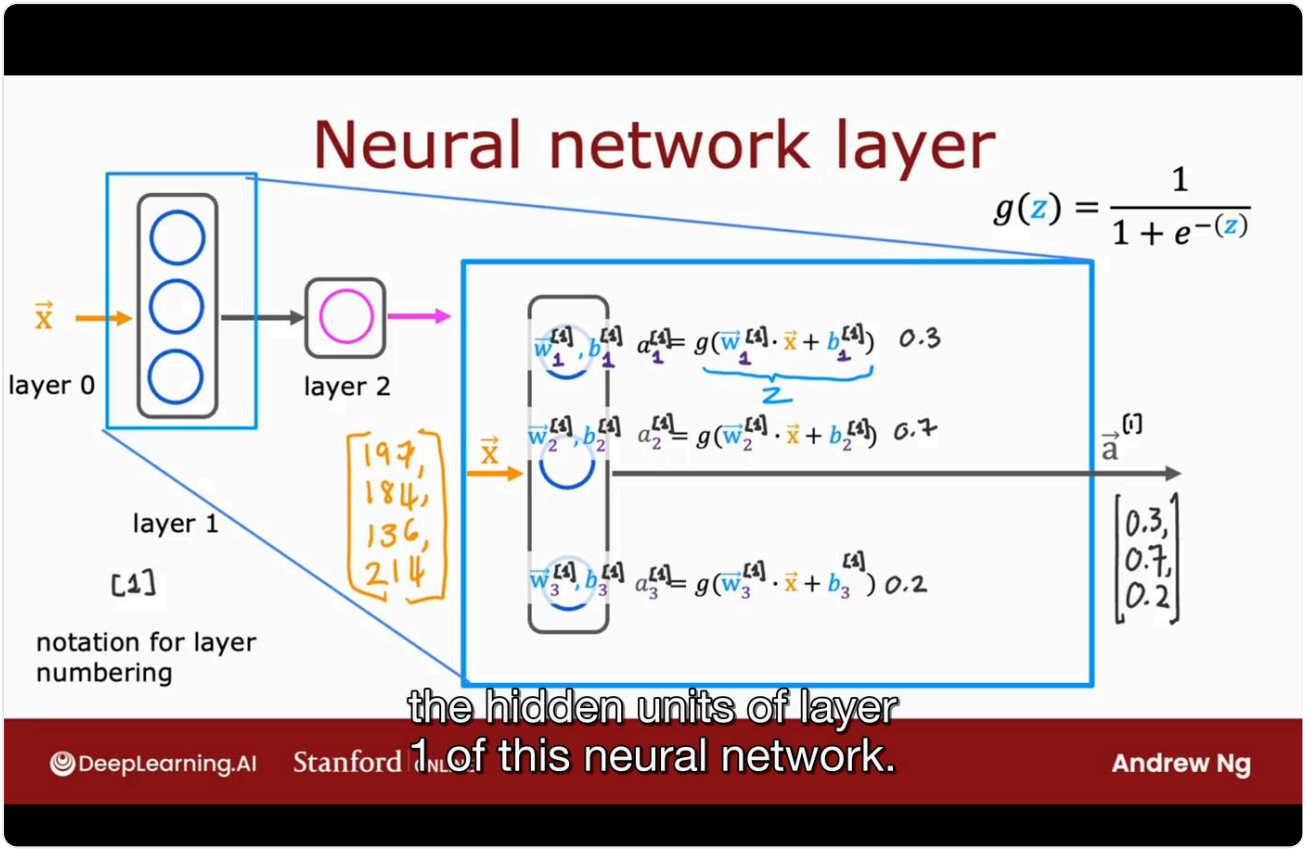


FIGURE 5.6

Similarly, I can add superscripts square brackets like so to denote that these are the activation values of the hidden units of layer 1 of this neural network. I know maybe this notation is getting a little bit cluttered. But the thing to remember is whenever you see this superscript square bracket 1, that just refers to a quantity that is associated with layer 1 of the neural network. If you see superscript square bracket 2, that refers to a quantity associated with layer 2 of the neural network and similarly for other layers as well, including layer 3, layer 4 and so on for neural networks with more layers. That's the computation of layer 1 of this neural network.

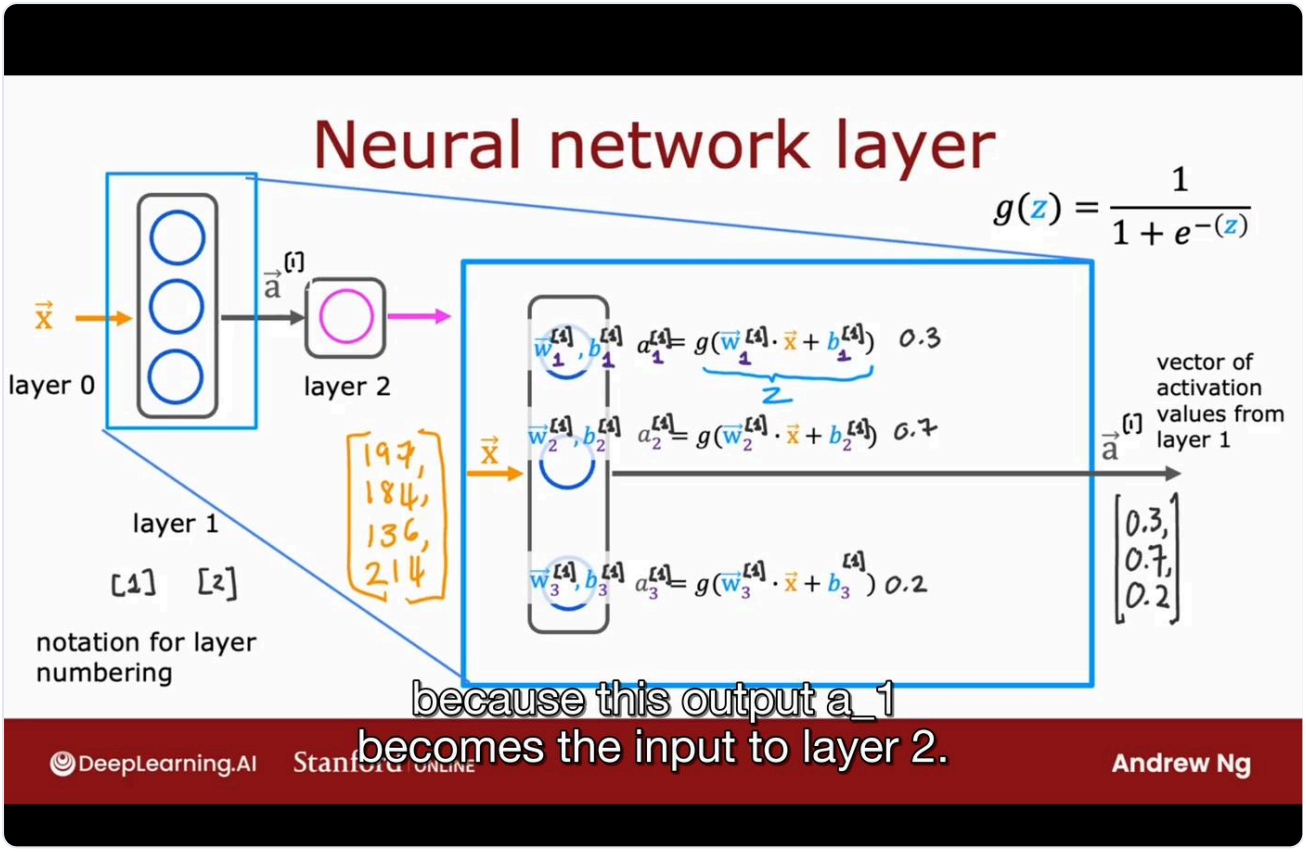


FIGURE 5.7 | Neural Network Layer

Its output is this activation vector, a^[1] and I'm going to copy this over here because this output a_1 becomes the input to layer 2. Now let's zoom into the computation of layer 2 of this neural network, which is also the output layer.

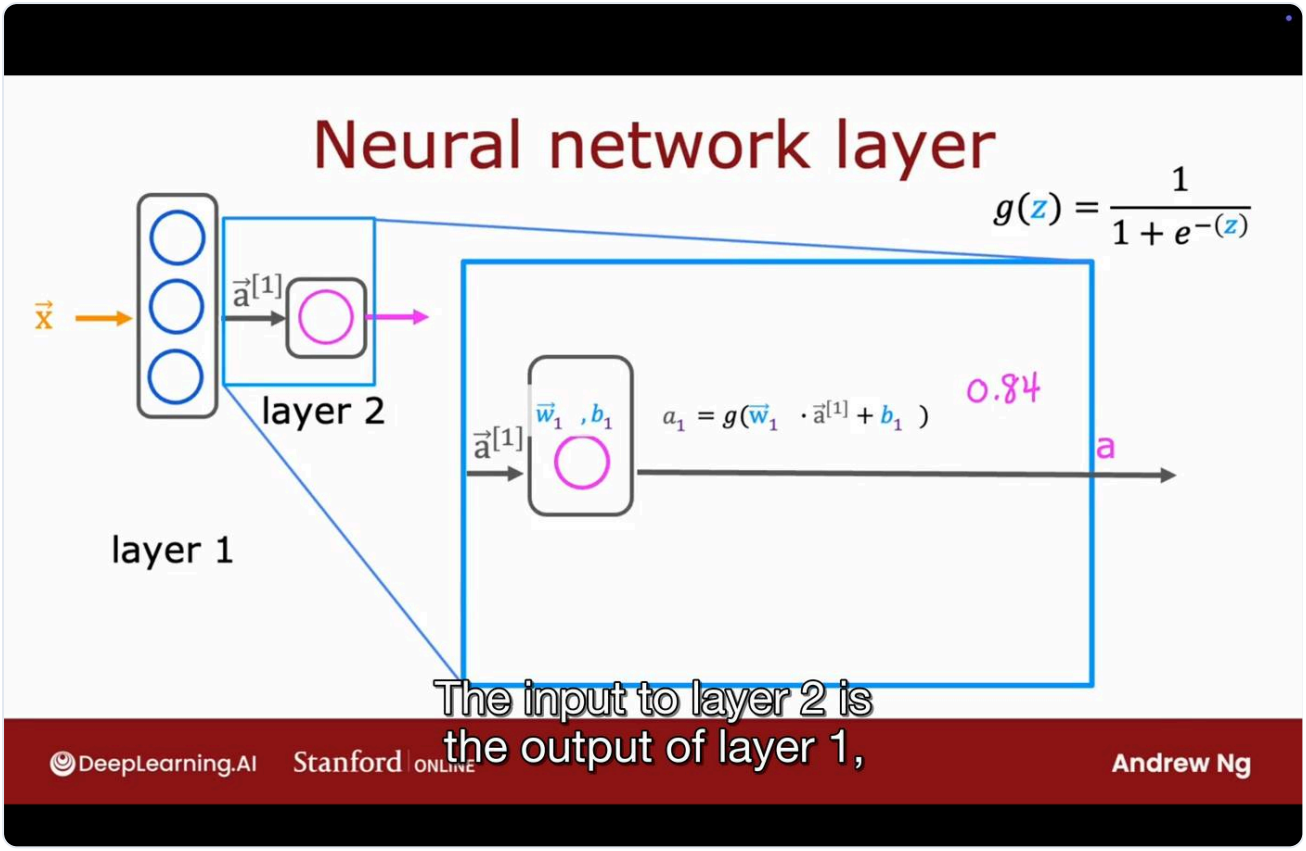


FIGURE 5.8 | Neural Network Layer

The input to layer 2 is the output of layer 1, so a_1 is this vector 0.3, 0.7, 0.2 that we just computed on the previous part of this slide. Because the output layer has just a single neuron, all it does is it computes a_1 that is the output of this first and only neuron, as g, the sigmoid function applied to w _1 in a product with a^[1], so this is the input into this layer, and then plus b_1.

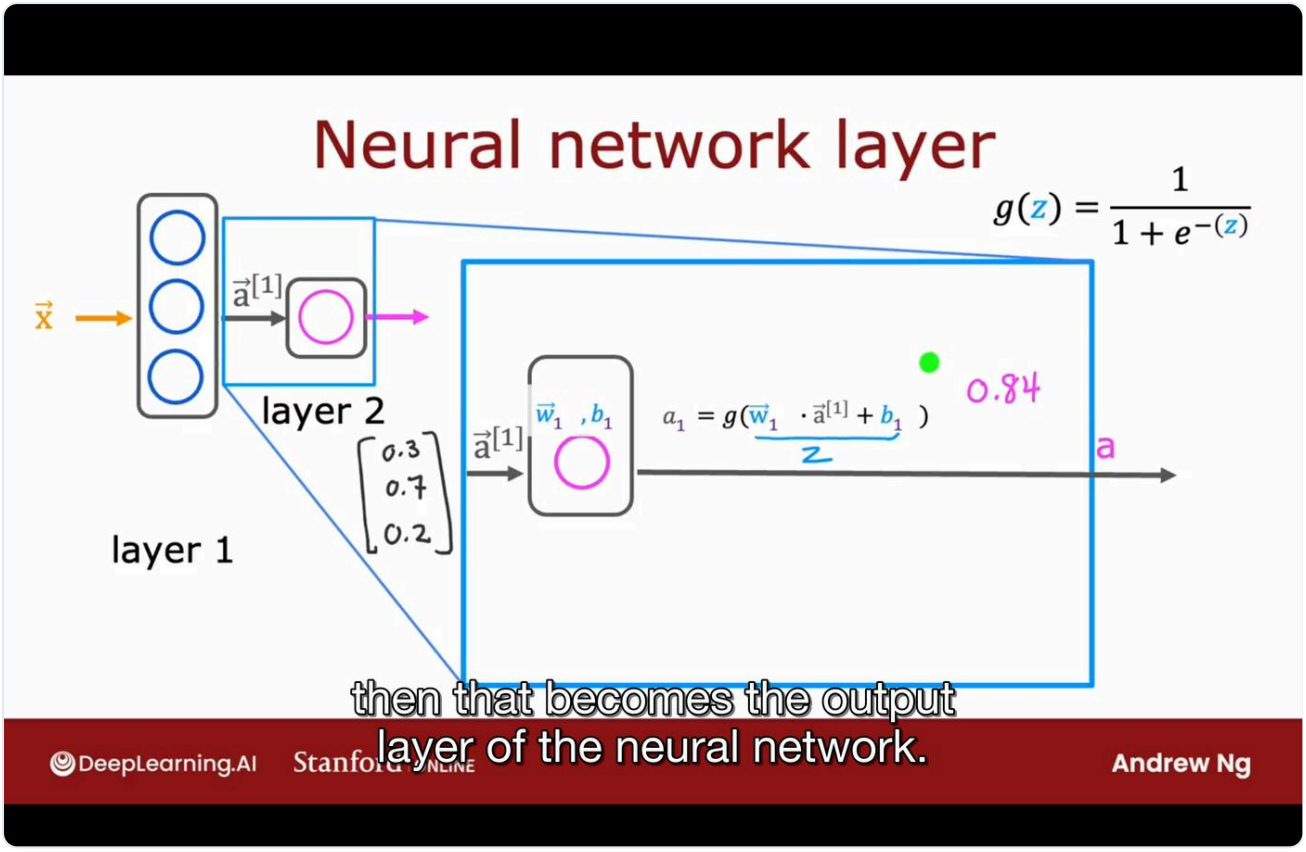


FIGURE 5.9

Here, this is the quantity z that you familiar with and g as before is the sigmoid function that you apply to this. If this results in a number, say 0.84, then that becomes the output layer of the neural network.

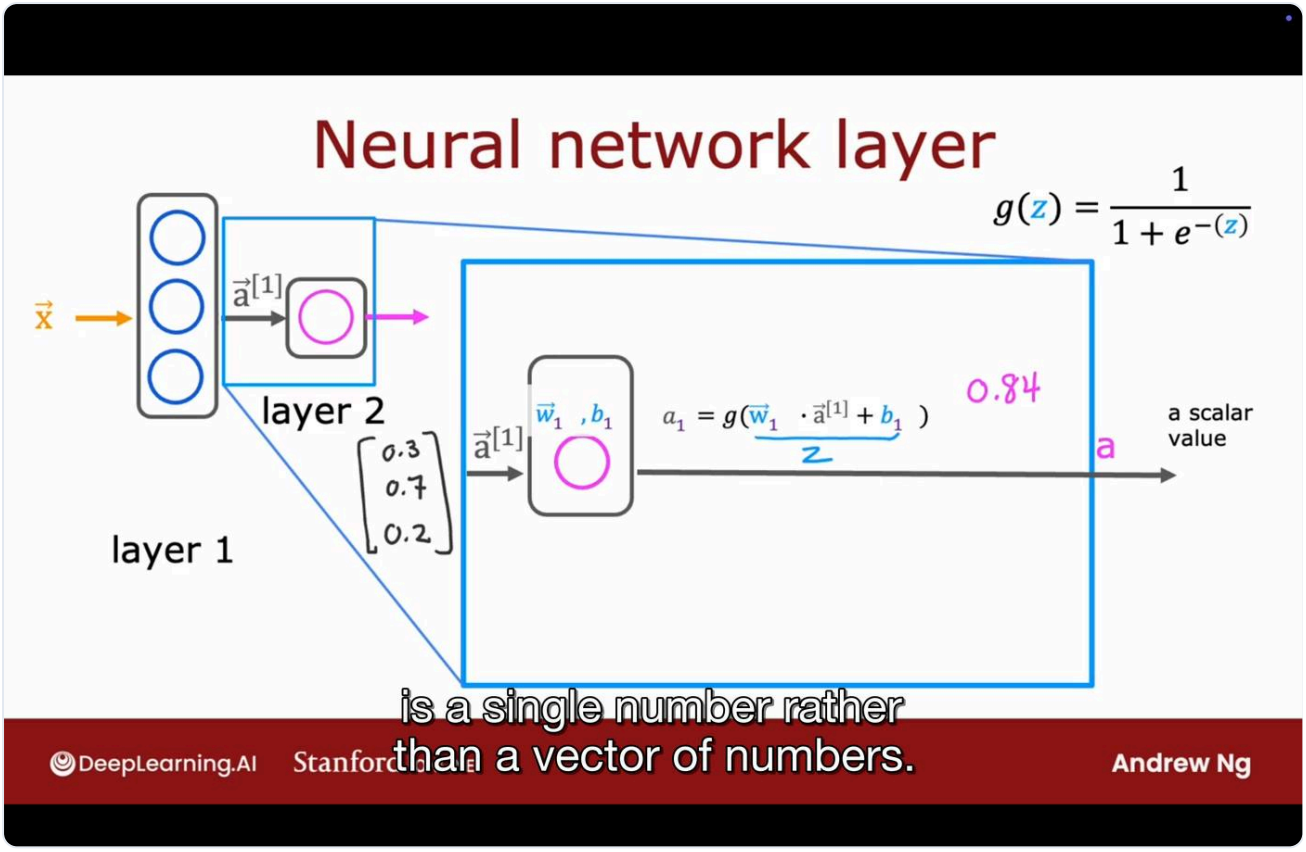


FIGURE 5.10 | Neural Network Layer

In this example, because the output layer has just a single neuron, this output is just a scalar, is a single number rather than a vector of numbers.

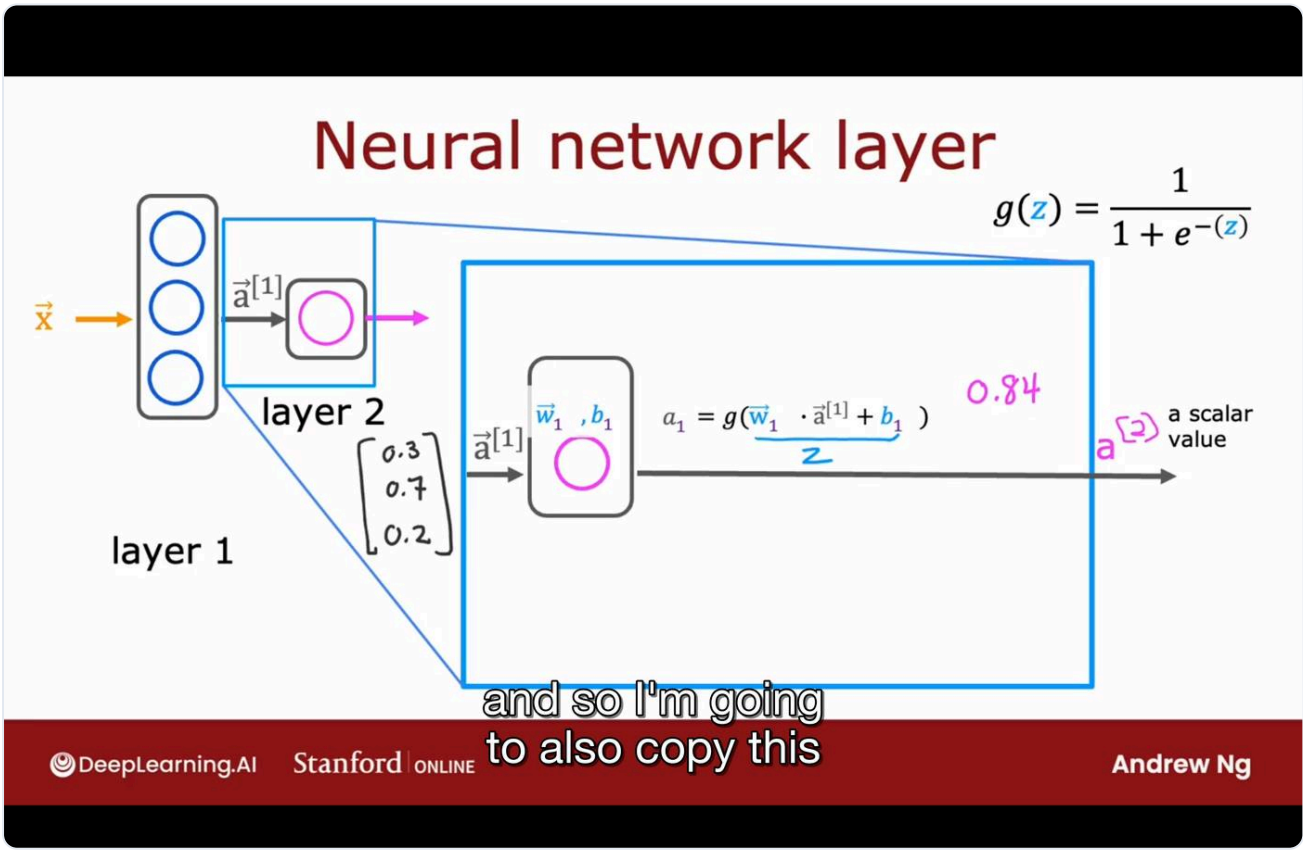


FIGURE 5.11 | Neural Network Layer

Sticking with our notational convention from before, we're going to use a superscript in square brackets 2, to denote the quantities associated with layer 2 of this neural network, so a^[2] is the output of this layer, and so I'm going to also copy this here as the final output of the neural network.

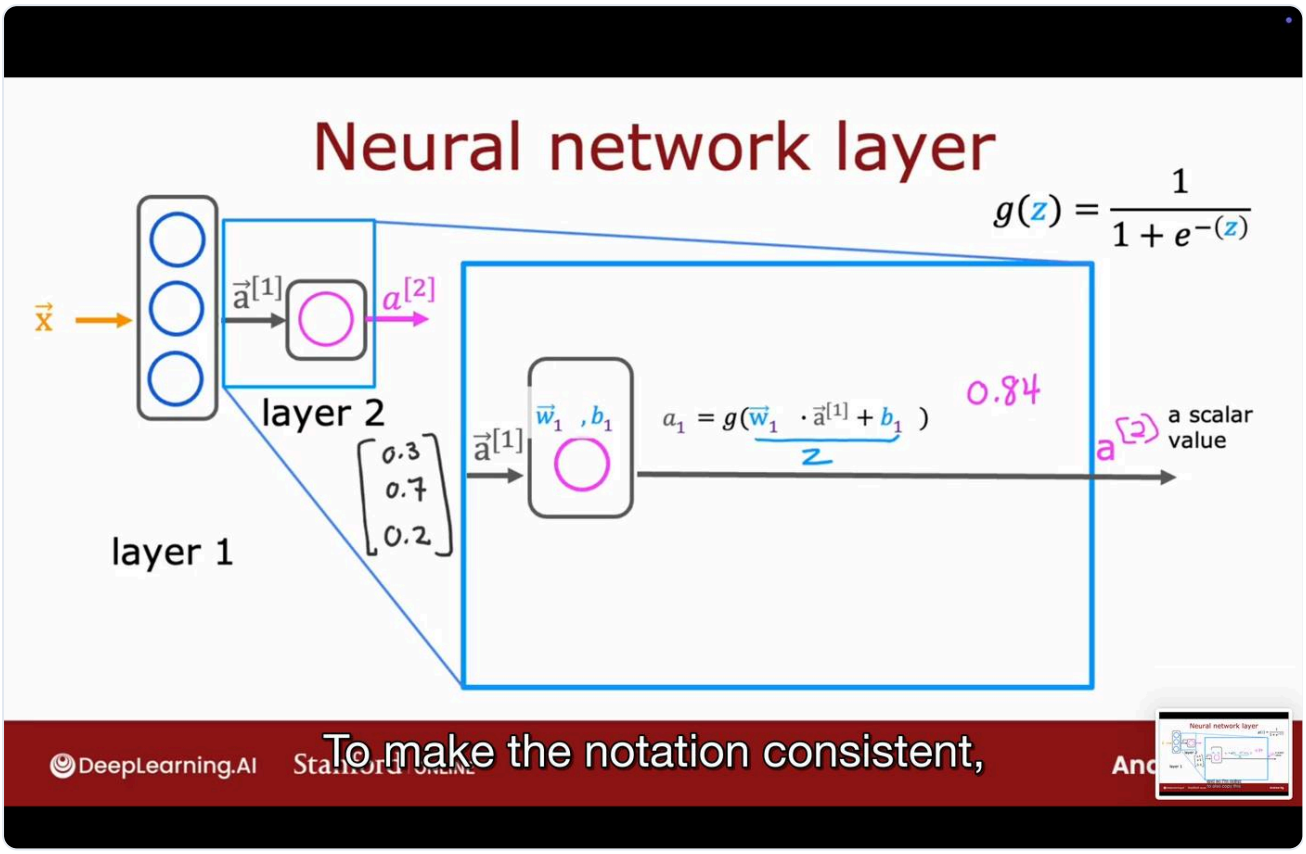


FIGURE 5.12 | Neural Network Layer

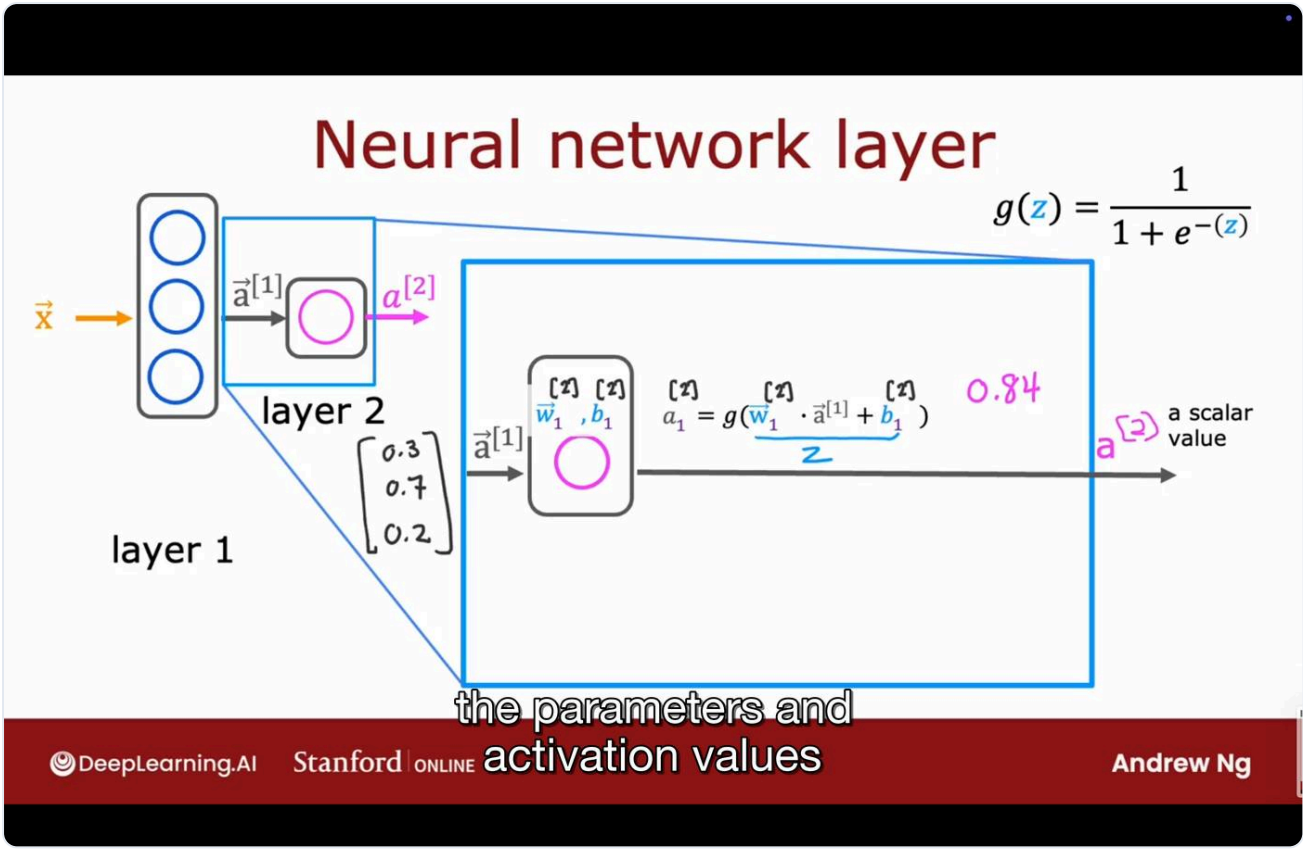


FIGURE 5.13 | Neural Network Layer

To make the notation consistent, you can also add these superscripts square bracket 2s to denote that these are the parameters and activation values associated with layer 2 of the neural network.

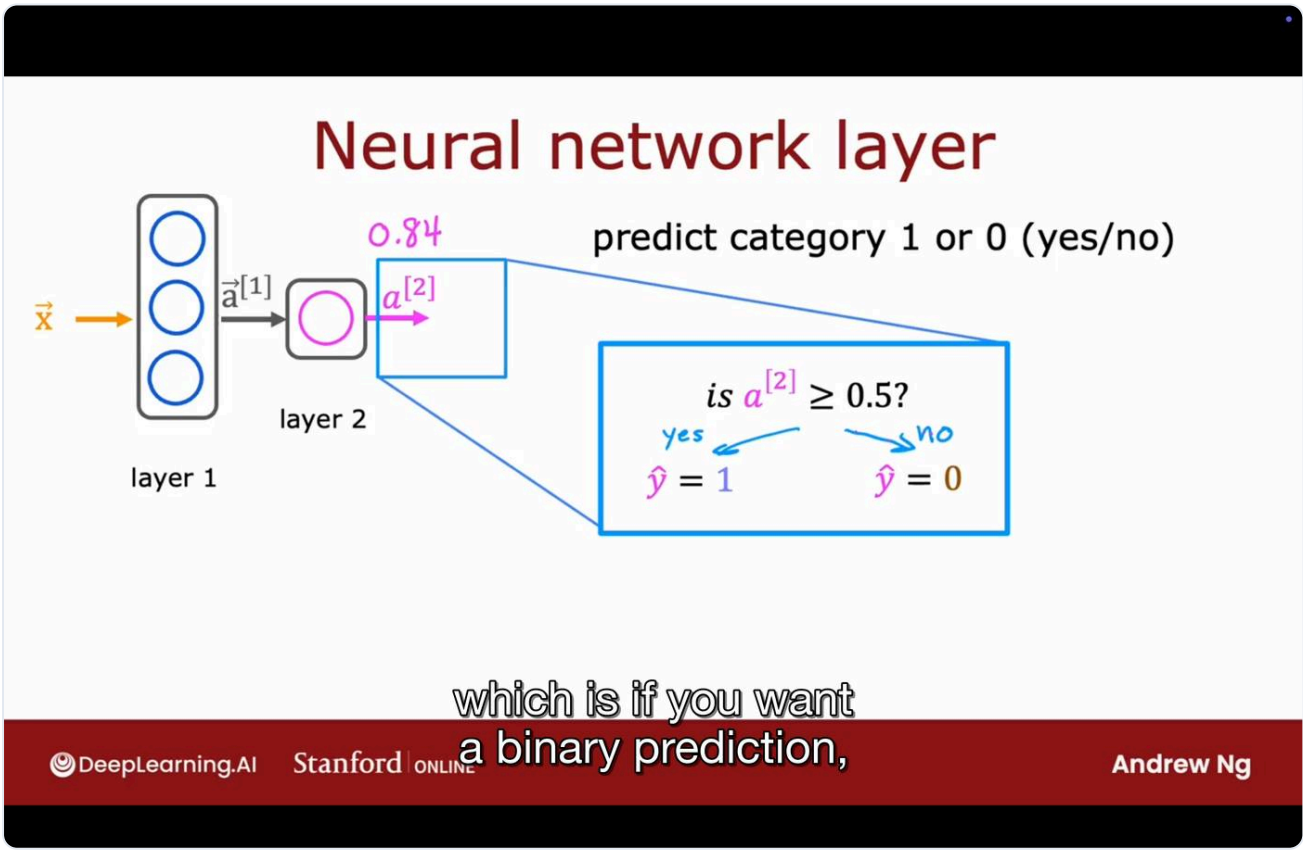


FIGURE 5.14 | Neural Network Layer

Once the neural network has computed a_2, there's one final optional step that you can choose to implement or not, which is if you want a binary prediction, 1 or 0, is this a top seller? Yes or no? As you can take the number a superscript square brackets 2 subscript 1, and this is the number 0.84 that we computed, and threshold this at 0.5. If it's greater than 0.5, you can predict y hat equals 1 and if it is less than 0.5, then predict your y hat equals 0. We saw this thresholding as well when you learned about logistic regression in the first course of the specialization.

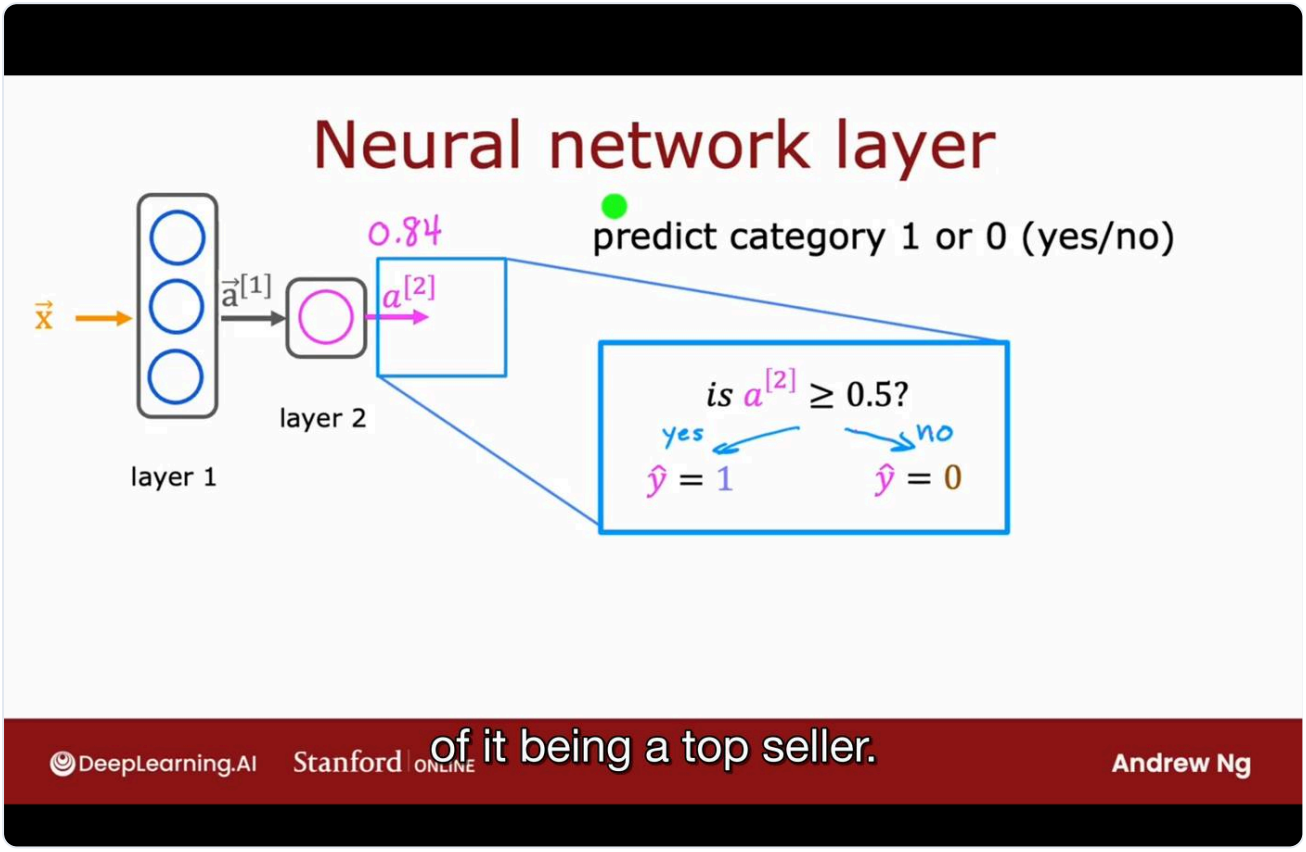


FIGURE 5.15 | Neural Network Layer

If you wish, this then gives you the final prediction y hat as either one or zero, if you don't want just the probability of it being a top seller. So that's how a neural network works. Every layer inputs a vector of numbers and applies a bunch of logistic regression units to it, and then computes another vector of numbers that then gets passed from layer to layer until you get to the final output layers computation, which is the prediction of the neural network. Then you can either threshold at 0.5 or not to come up with the final prediction. With that, let's go on to use this foundation we've built now to look at some even more complex, even larger neural network models.

I hope that by seeing more examples, this concept of layers and how to put them together to build a neural network will become even clearer. So let's go on to the next video.

LECTURE 6

More Complex Neural Network

In the last video, you learned about the neural network layer and how that takes this inputs a vector of numbers and in turn, outputs another vector of numbers. In this video, let's use that layer to build a more complex neural network. Through this, I hope that the notation that we're using for neural networks will become clearer and more concrete as well. Let's take a look. This is the running example that I'm going to use throughout this video as an example of a more complex neural network.

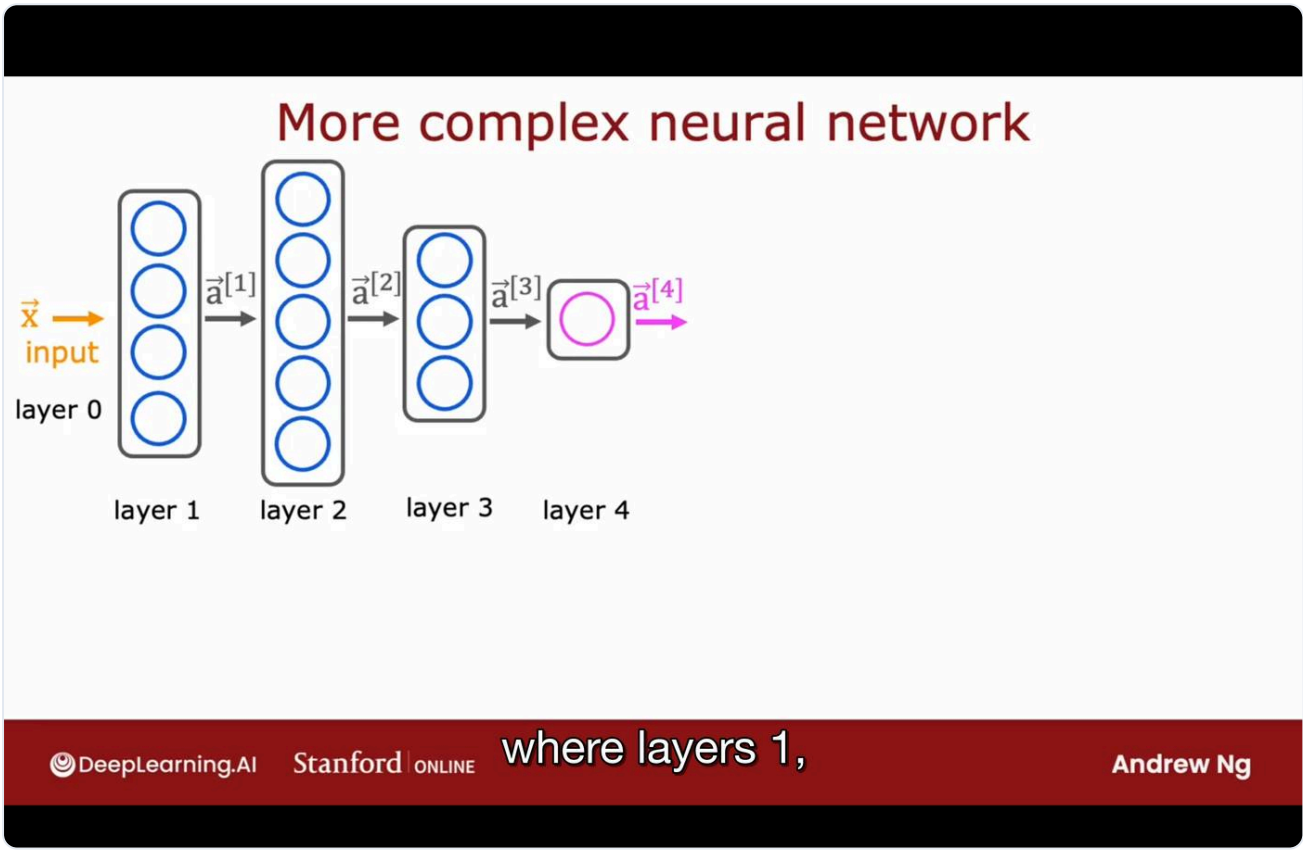


FIGURE 6.1

This network has four layers, not counting the input layer, which is also called Layer 0, where layers 1, 2, and 3 are hidden layers, and Layer 4 is the output layer, and Layer 0, as usual, is the input layer. By convention, when we say that a neural network has four layers, that includes all the hidden layers in the output layer, but we don't count the input layer. This is a neural network with four layers in the conventional way of counting layers in the network. Let's zoom in to Layer 3, which is the third and final hidden layer to look at the computations of that layer. Layer 3 inputs a vector, a superscript square bracket 2 that was computed by the previous layer, and it outputs a_3, which is another vector.

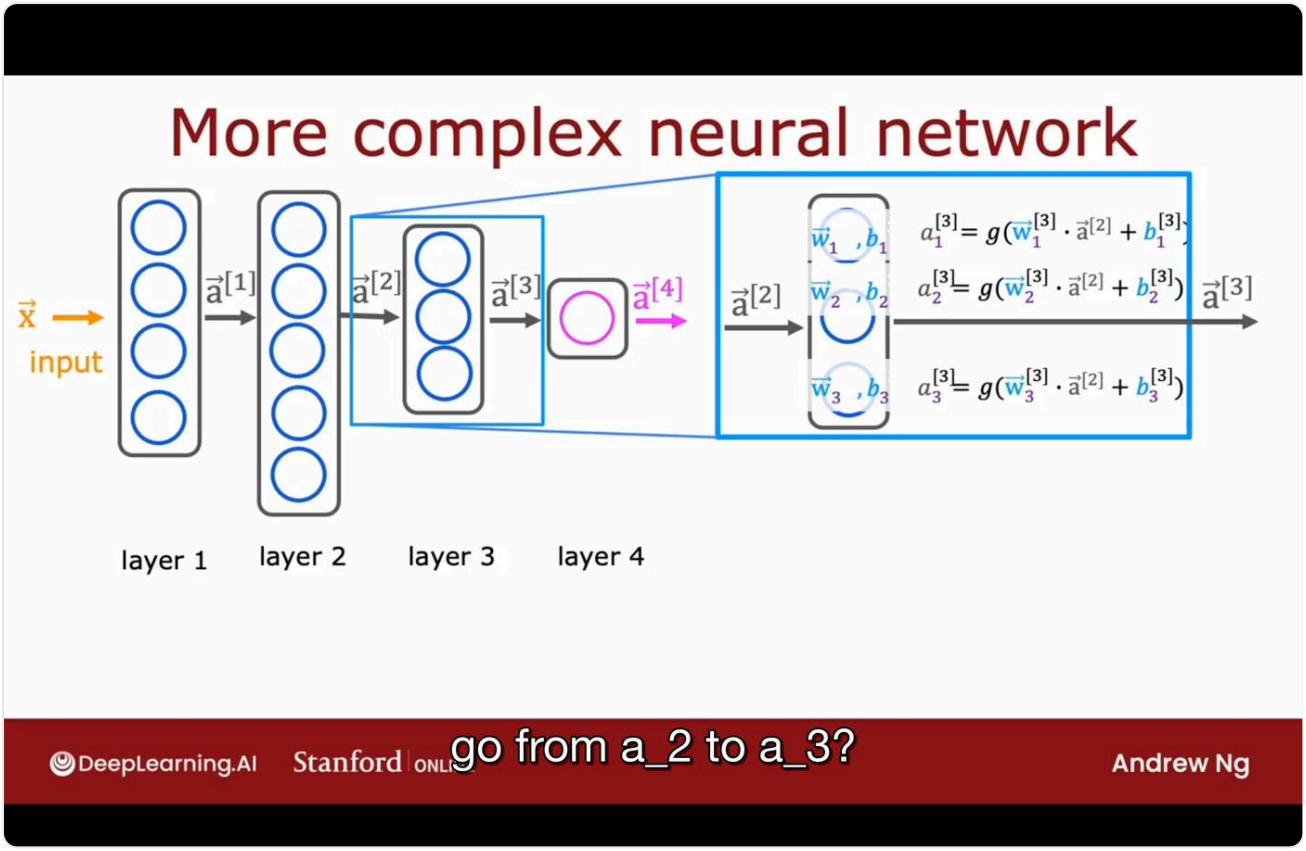


FIGURE 6.2 | More Complex Neural Network

What is the computation that Layer 3 does in order to go from a_2 to a_3? If it has three neurons or we call it three hidden units, then it has parameters w_1, b_1, w_2, b_2, and w_3, b_3 and it computes a_1 equals sigmoid of w_1. product with this input to the layer plus b_1, and it computes a_2 equals sigmoid of w_2. product with again a_2, the input to the layer plus b_2 and so on to get a_3. Then the output of this layer is a vector comprising a_1, a_2, and a_3.

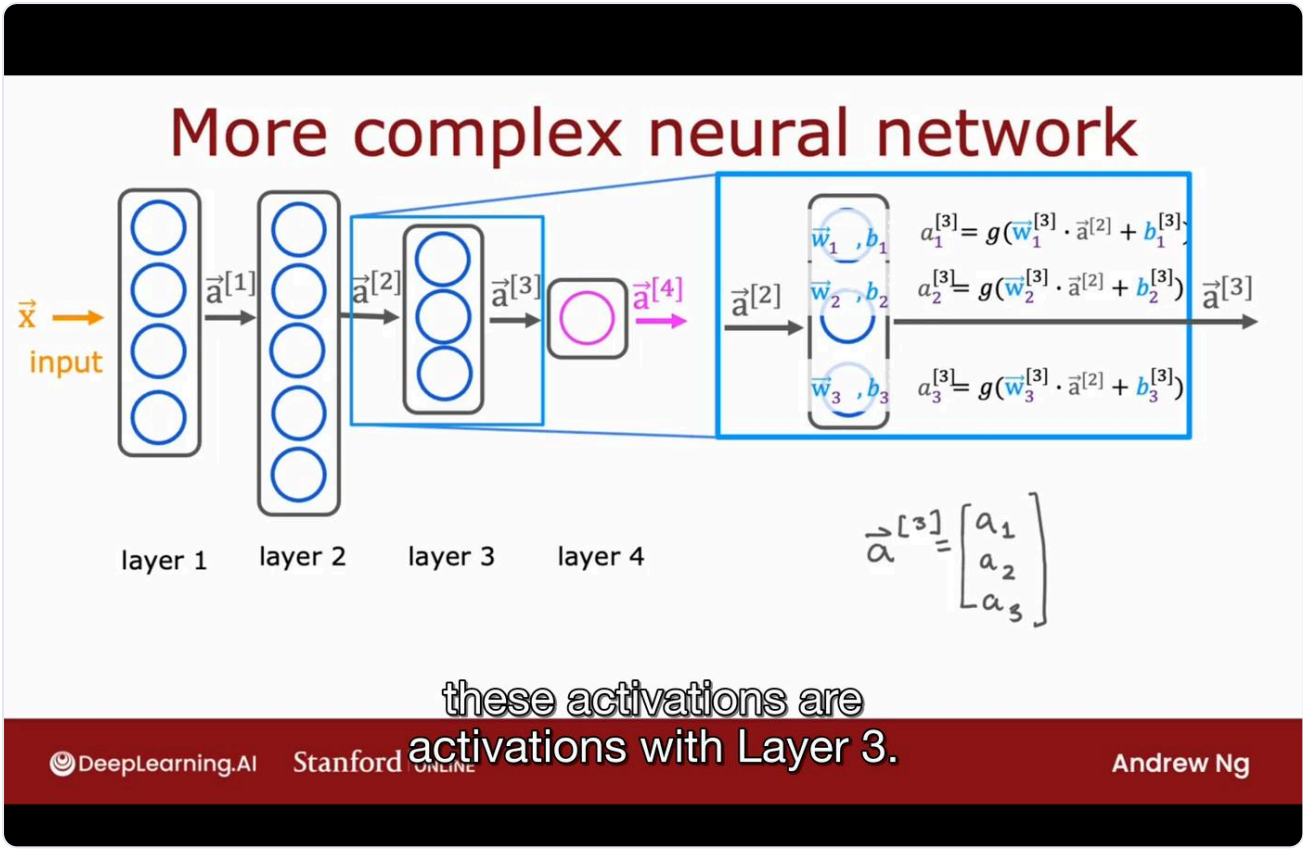


FIGURE 6.3 | More Complex Neural Network

Again, by convention, if we want to more explicitly denote that all of these are quantities associated with Layer 3 then we add in all of these superscript, square brackets 3 here, to denote that these parameters w and b are the parameters associated with neurons in Layer 3 and that these activations are activations with Layer 3. Notice that this term here is w_1 superscript square bracket 3, meaning the parameters associated with Layer 3. product with a superscript square bracket 2, which was the output of Layer 2, which became the input to Layer 3. That's why it has a_3 here because it's a parameter associator of Layer 3. product with, and there's a_2 there because is the output of Layer 2.

Now, let's just do a quick double check on our understanding of this. I'm going to hide the superscripts and subscripts associated with the second neuron and without rewinding this video, go ahead and rewind if you want, but prefer you not.

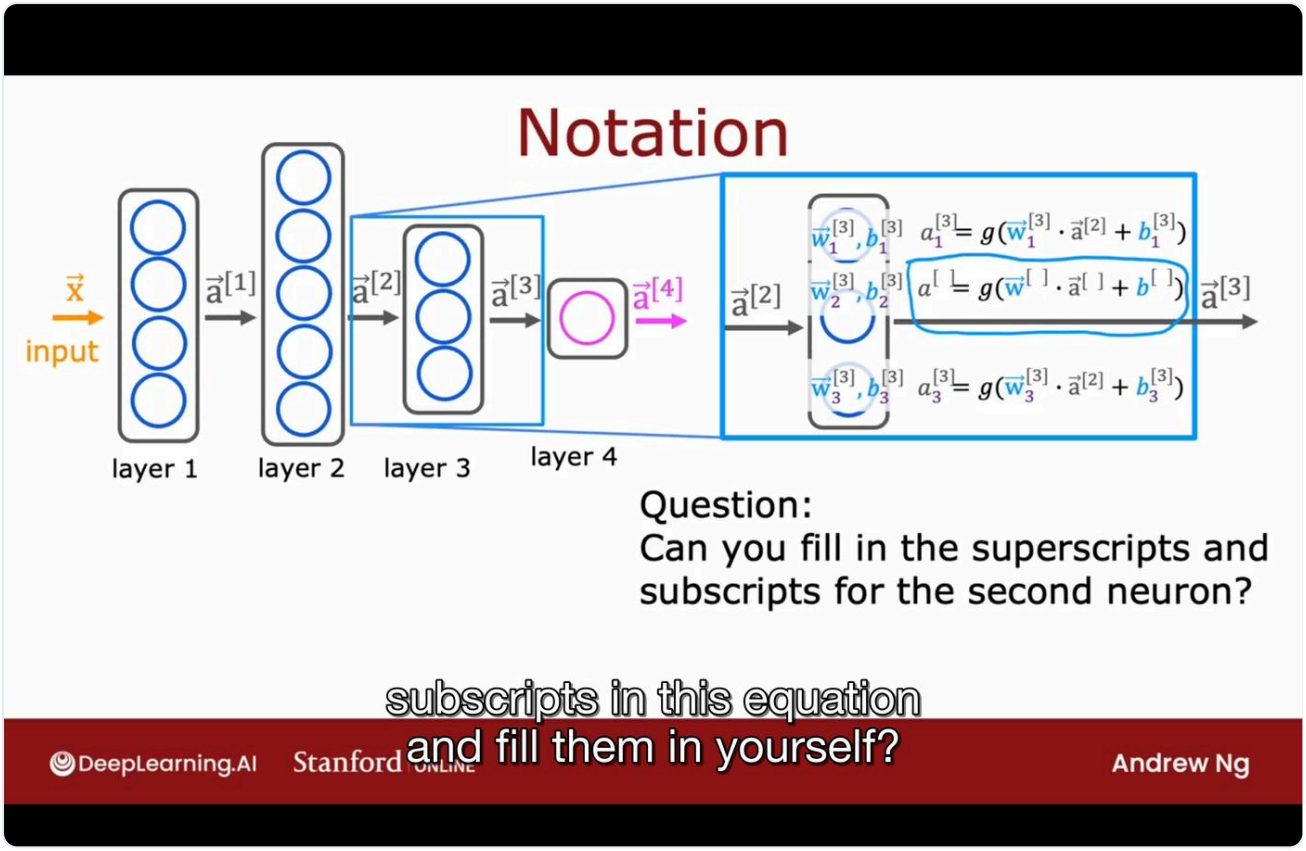


FIGURE 6.4

But without rewinding this video, are you able to think through what are the missing superscripts and subscripts in this equation and fill them in yourself? Once you take a look at the end video quiz and see if you can figure out what are the appropriate superscripts and subscripts for this equation over here. If you chose the 1st option, then you got it right!

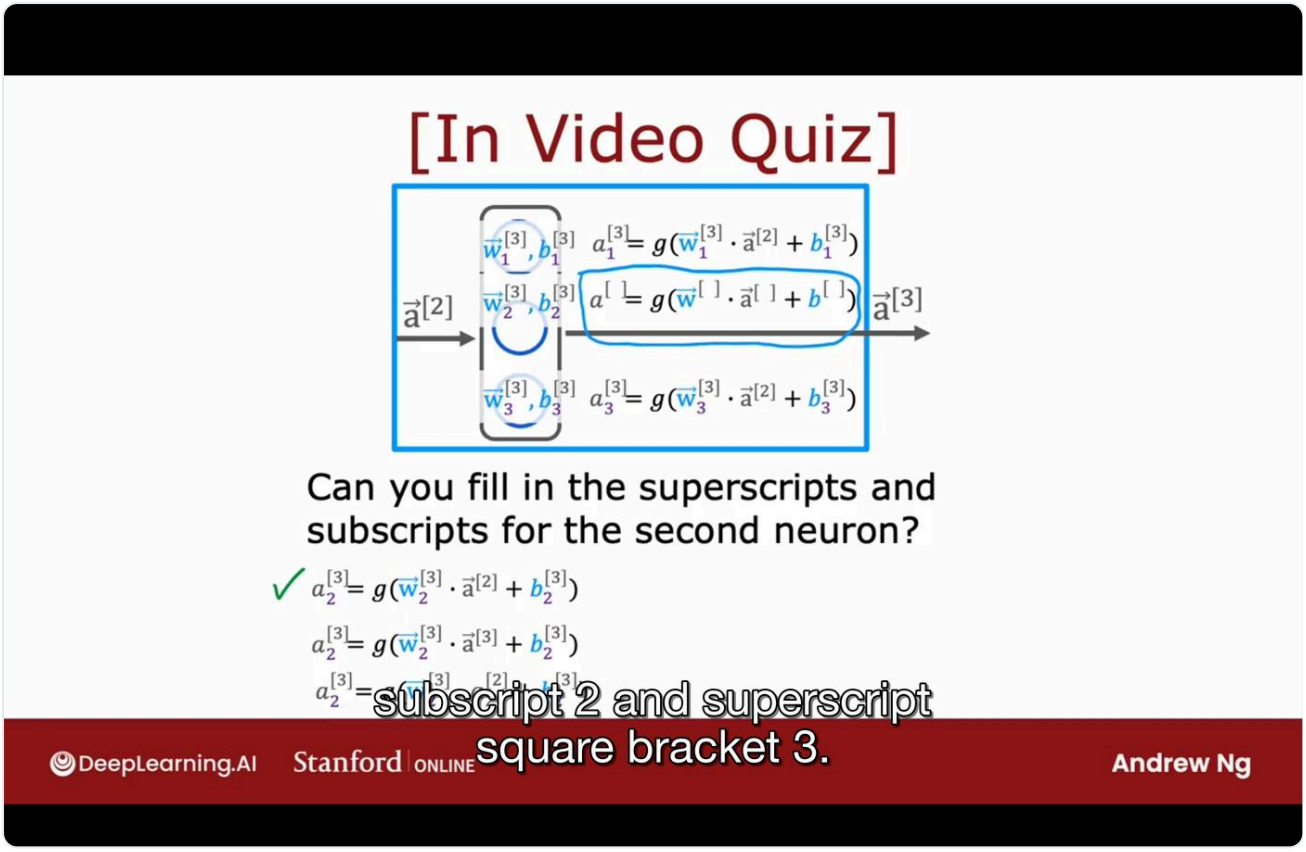


FIGURE 6.5

The activation of the 2nd neuron at layer 3 is denoted by 'a' three two. To apply the activation function, g, lets use the parameters of this same neuron. So w and b will have the same subscript 2 and superscript square bracket 3.

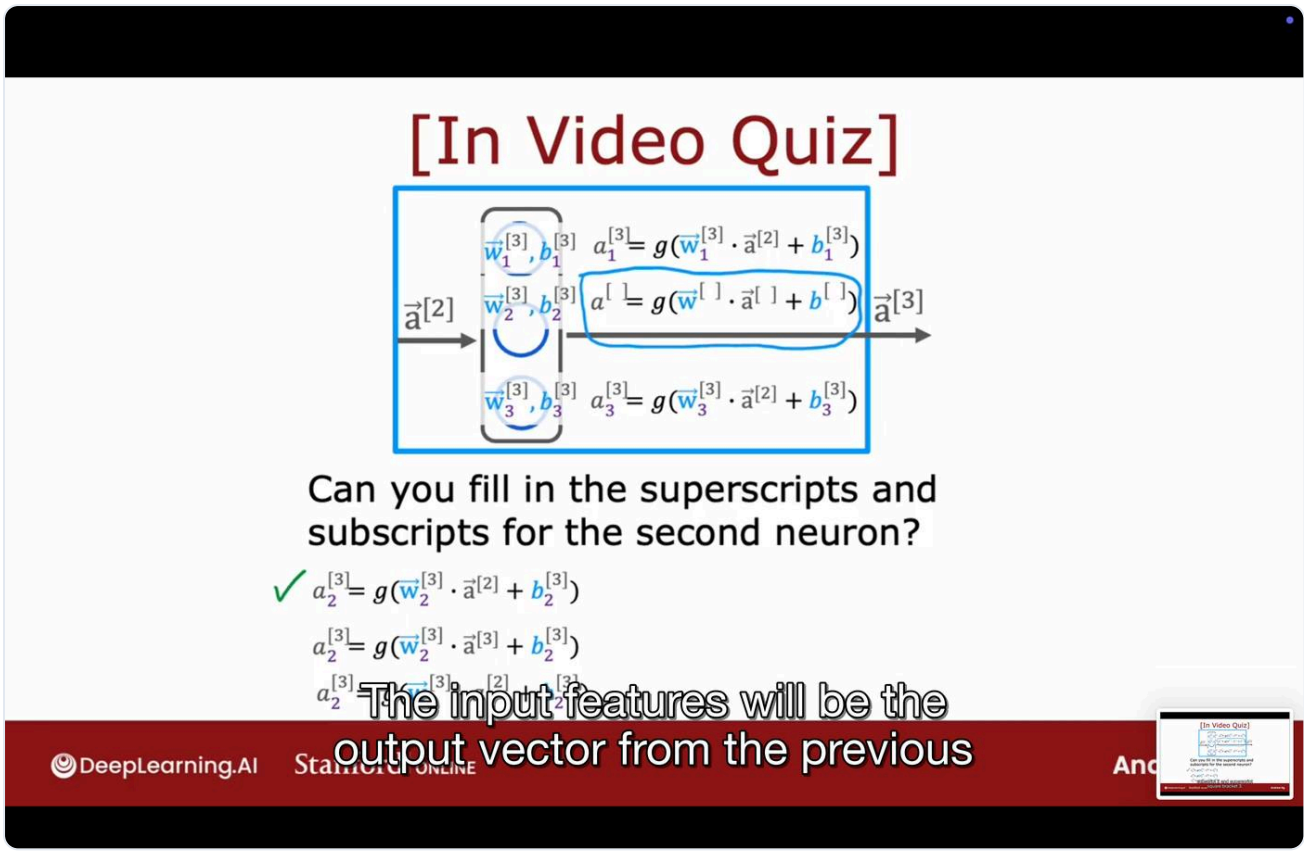


FIGURE 6.6

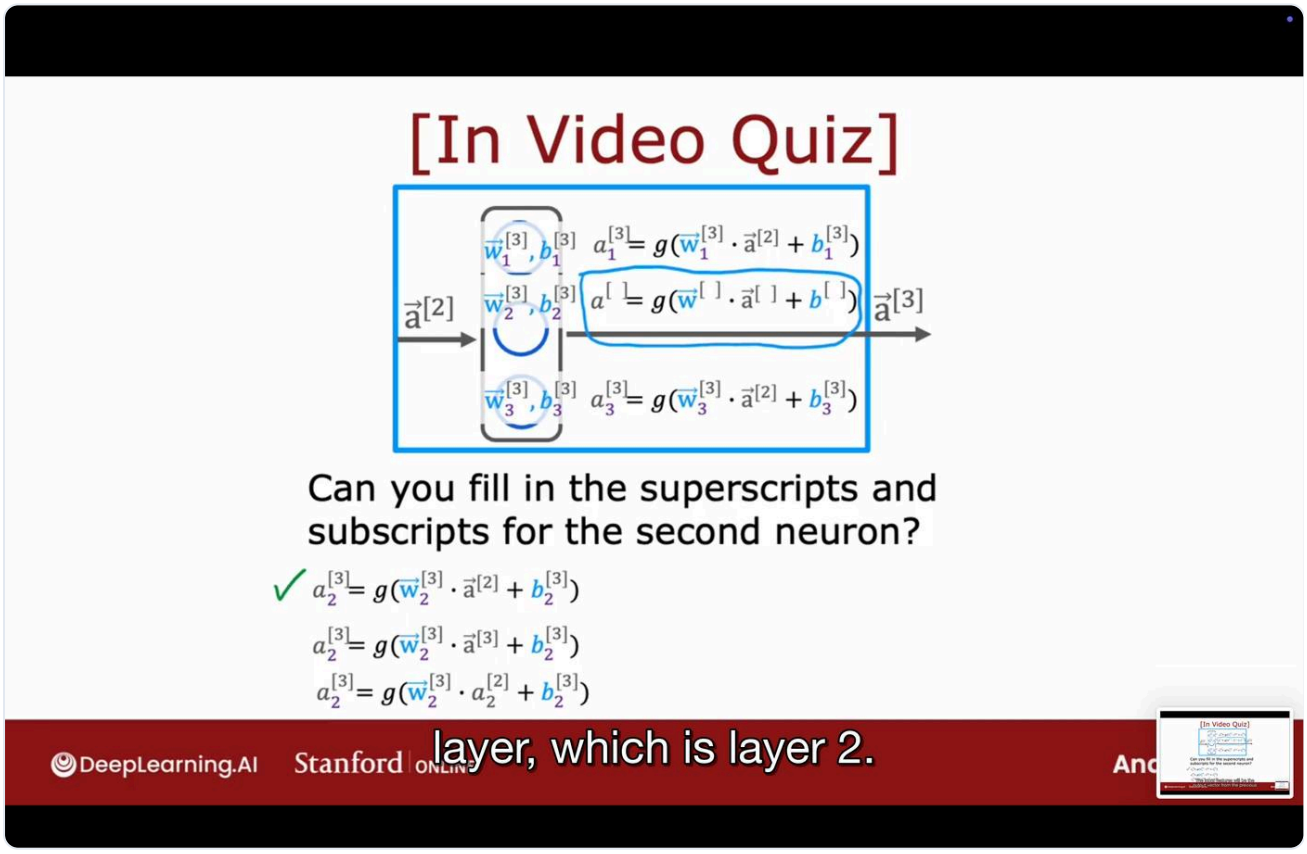


FIGURE 6.7

The input features will be the output vector from the previous layer, which is layer 2. So that will be the vector 'a' superscript 2. The second option is using vector ‘a’ 3 which is not the output vector from the previous layer. The input to this layer is 'a' two.

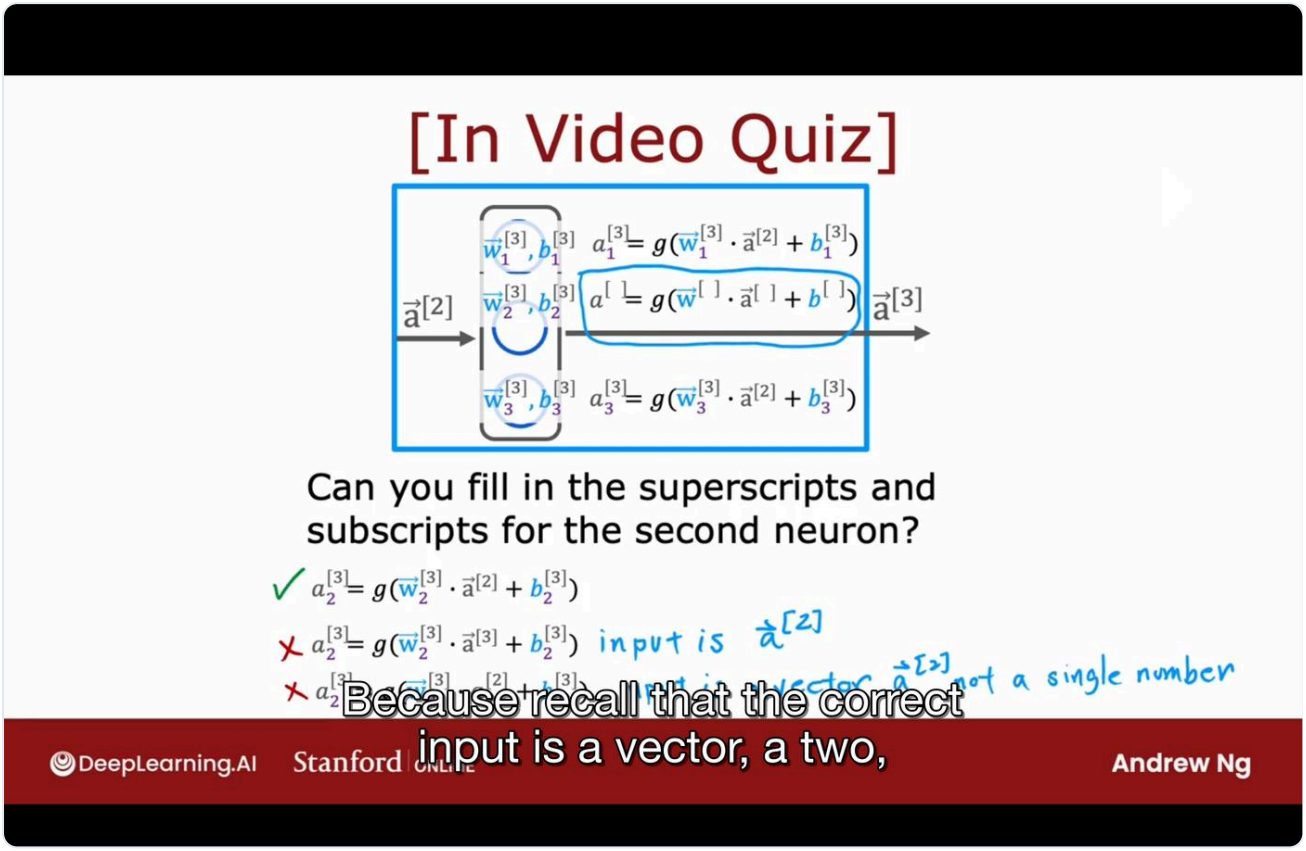


FIGURE 6.8

And the 3rd option has a two two as input, which is a single number rather than the vector Because recall that the correct input is a vector, a two, with the little arrow on top, and not a single number. To recap, a_3 is activation associated with Layer 3 for the second neuron hence, this a_2 is a parameter associated with the third layer. For the second neuron, this is a_2, same as above and then plus b_3 too.

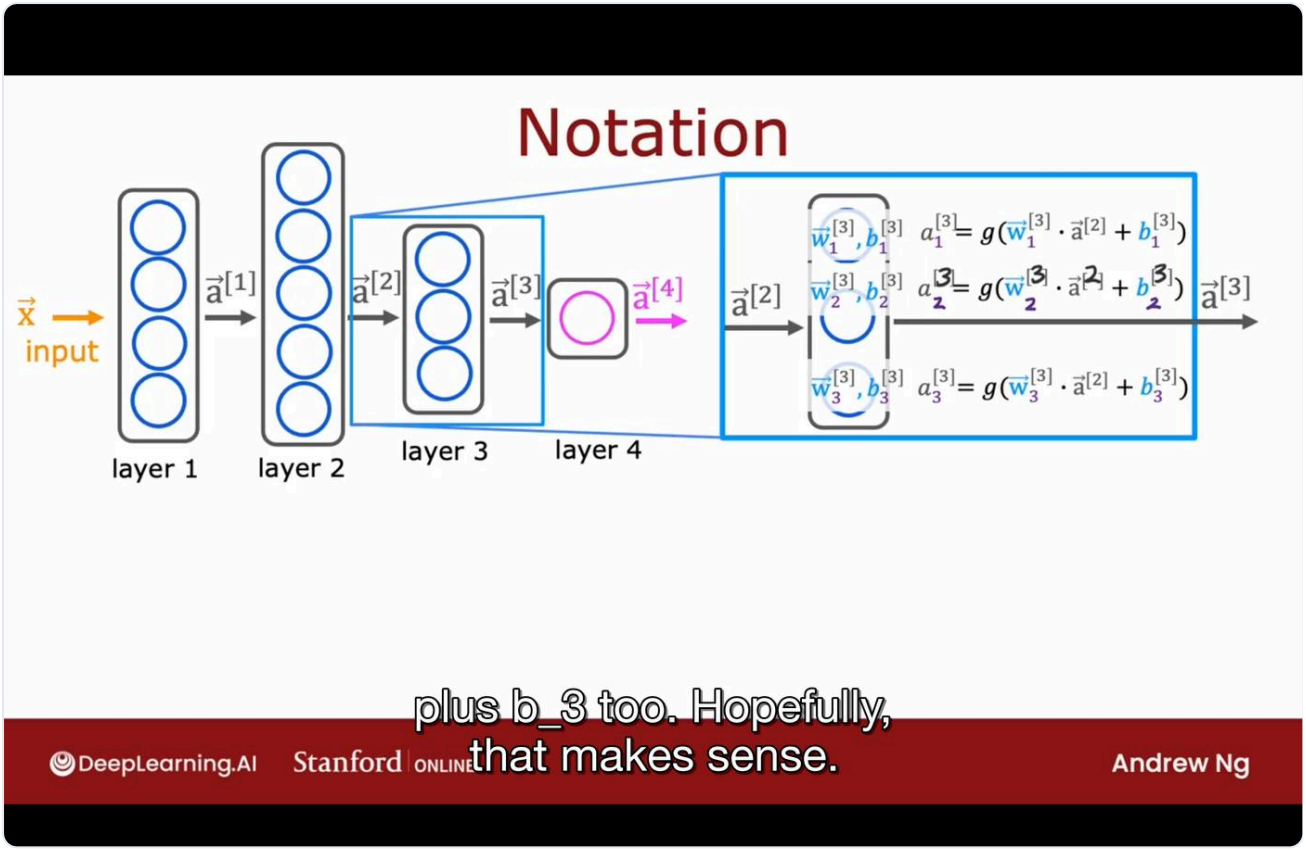


FIGURE 6.9

Hopefully, that makes sense. Just the more general form of this equation for an arbitrary Layer 0 and for an arbitrary unit j, which is that a deactivation outputs of layer l, unit j, like a32, that's going to be the sigmoid function applied to this term, which is the wave vector of layer l, such as Layer 3 for the jth unit so there's a_2 again, in the example above, and so that's dot-producted with a deactivation value. Notice, this is not l, this is l minus 1, like a_2 above here because you're dot-producting with the output from the previous layer and then plus b, the parameter for this layer for that unit j. This gives you the activation of layer l unit j, where the superscript in square brackets l denotes layer l and a subscript j denotes unit j.

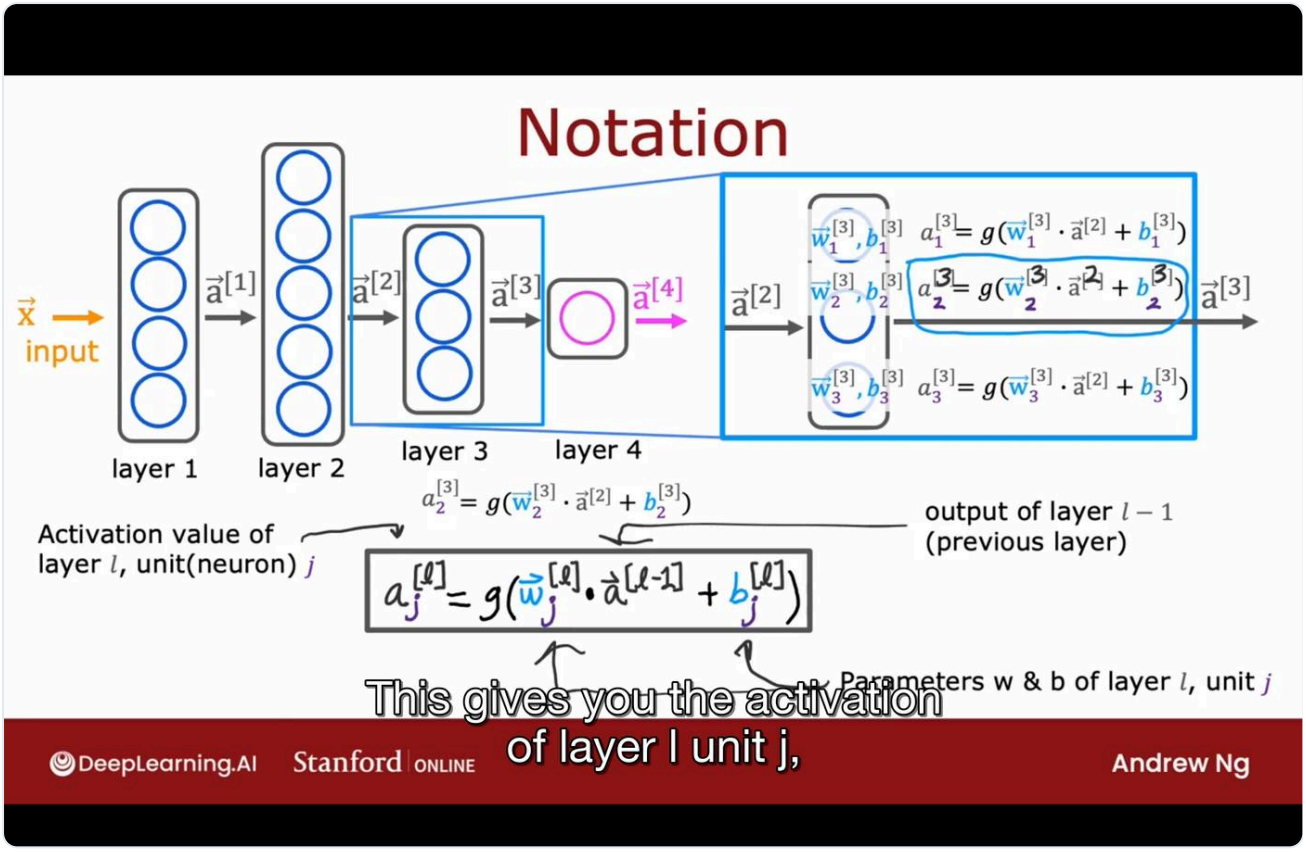


FIGURE 6.10

When building neural networks, unit j refers to the jth neuron, so we use those terms a little bit interchangeably where each unit is a single neuron in the layer. G here is the sigmoid function. In the context of a neural network, g has another name, which is also called the activation function, because g outputs this activation value.

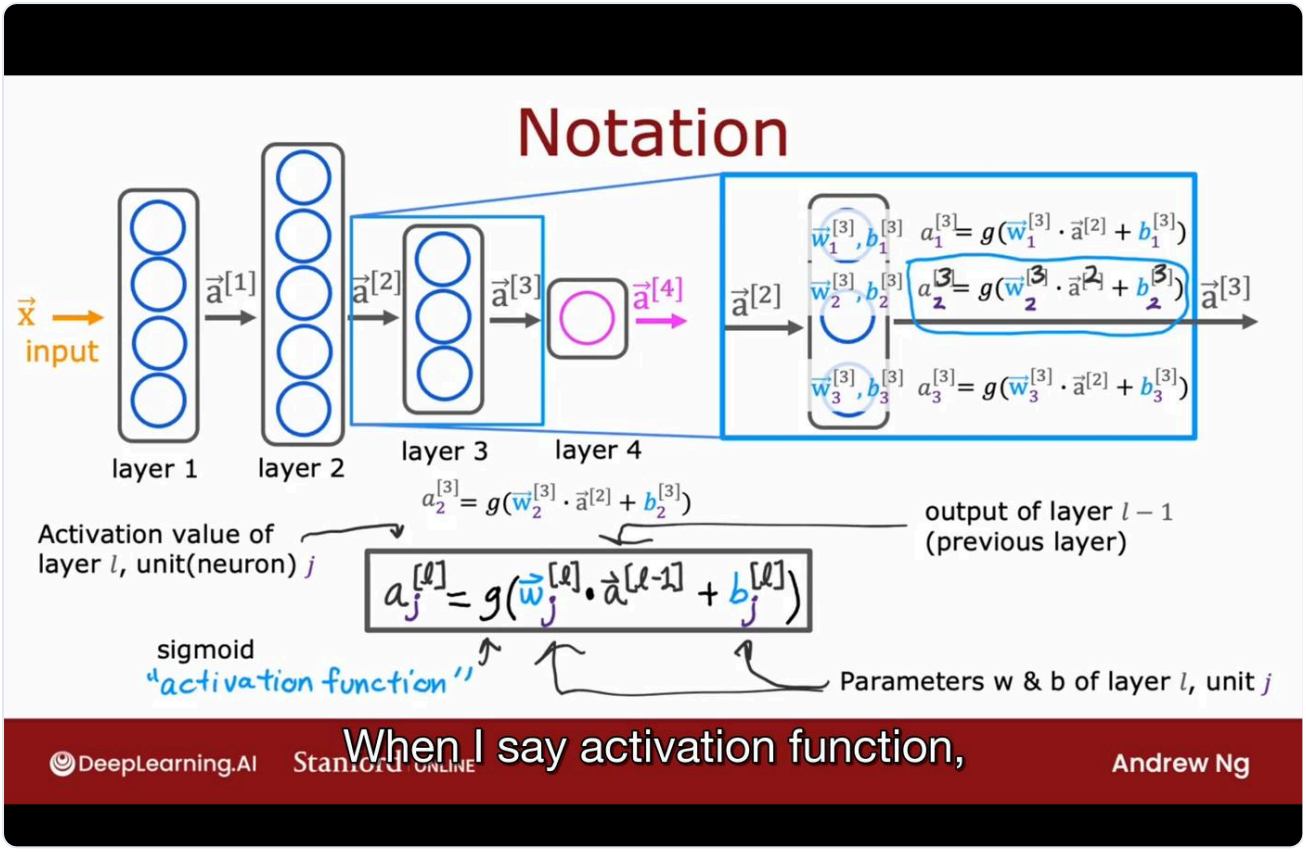


FIGURE 6.11

When I say activation function, I mean this function g here. So far, the only activation function you've seen, this is a sigmoid function but next week, we'll look at when other functions, then the sigmoid function can be plugged in place of g as well.. The activation function is just that function that outputs these activation values. Just one last piece of notation.

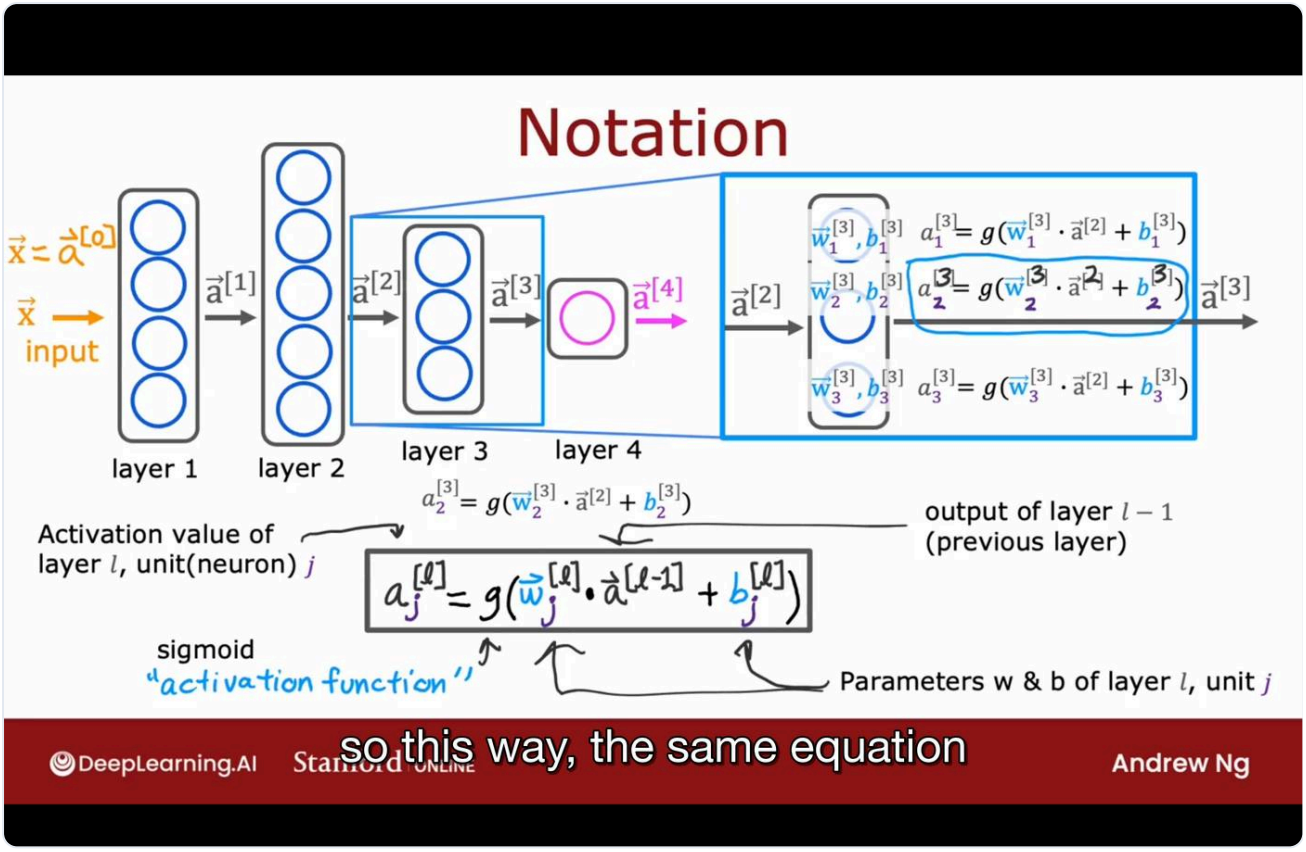


FIGURE 6.12

In order to make all this notation consistent, I'm also going to give the input vector X and another name which is a_0, so this way, the same equation also works for the first layer, where when l is equal to 1, the activations of the first layer, that is a_1, would be the sigmoid times the weights dot-product with a_0, which is just this input feature vector X. With this notation, you now know how to compute the activation values of any layer in a neural network as a function of the parameters as well as the activations of the previous layer. You now know how to compute the activations of any layer given the activations of the previous layer. Let's put this into an inference algorithm for a neural network.

In other words, how to get a neural network to make predictions. Let's go see that in the next video.

LECTURE 7

Handwritten Digit Recognition

Let's take what we've learned and put it together into an algorithm to let your neural network make inferences or make predictions. This will be an algorithm called forward propagation. Let's take a look. I'm going to use as a motivating example, handwritten digit recognition. And for simplicity we are just going to distinguish between the handwritten digits zero and one. So it's just a binary classification problem where we're going to input an image and classify, is this the digit zero or the digit one? And you get to play with this yourself later this week in the practice lab as well. For the example of the slide, I'm going to use an eight by eight image.

And so this image of a one is this grid or matrix of eight by eight or 64 pixel intensity values where 255 denotes a bright white pixel and zero would denote a black pixel.

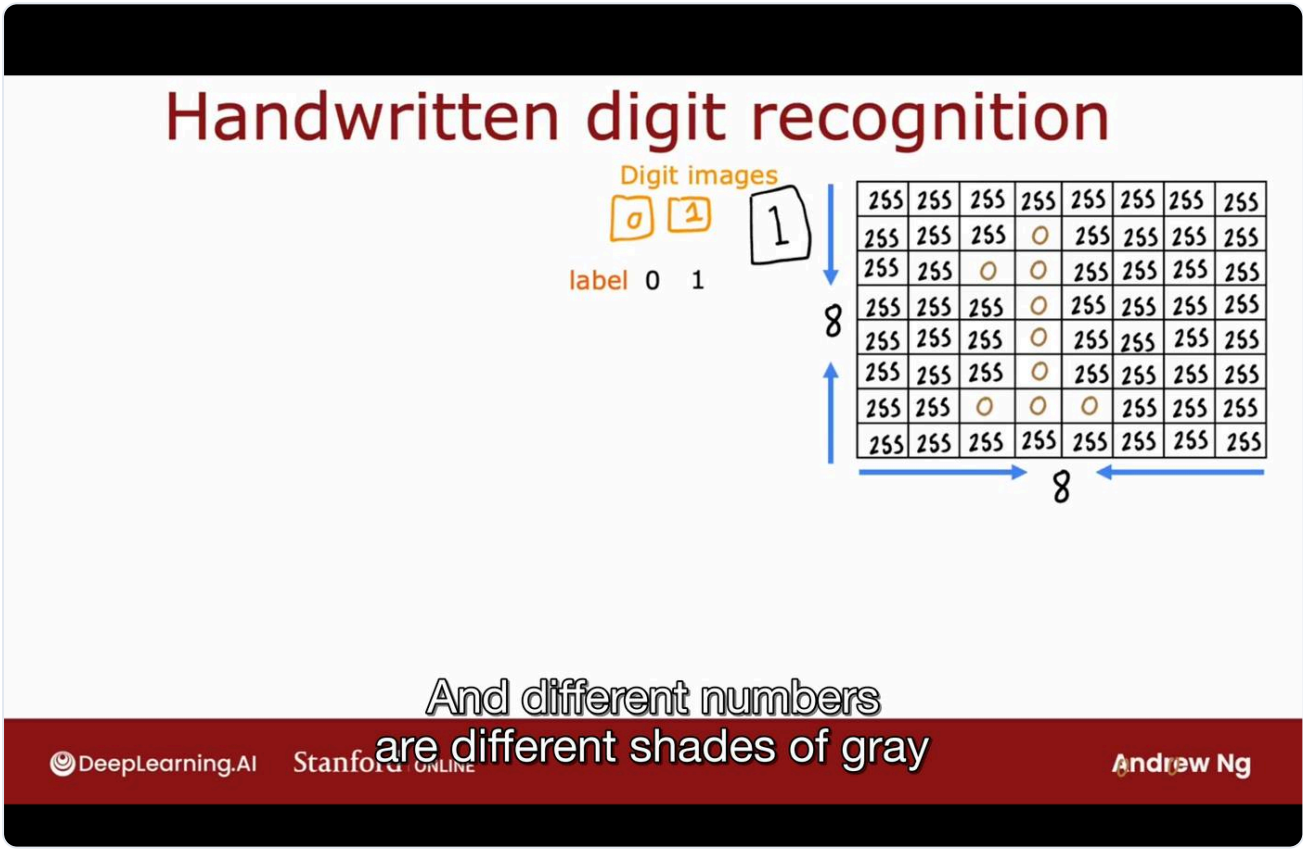


FIGURE 7.1 | Handwritten Digit Recognition

And different numbers are different shades of gray in between the shades of black and white. Given these 64 input features, we're going to use the neural network with two hidden layers. Where the first hidden layer has 25 neurons or 25 units. Second hidden layer has 15 neurons or 15 units. And then finally the output layer or outputs unit, what's the chance of this being 1 versus 0?.

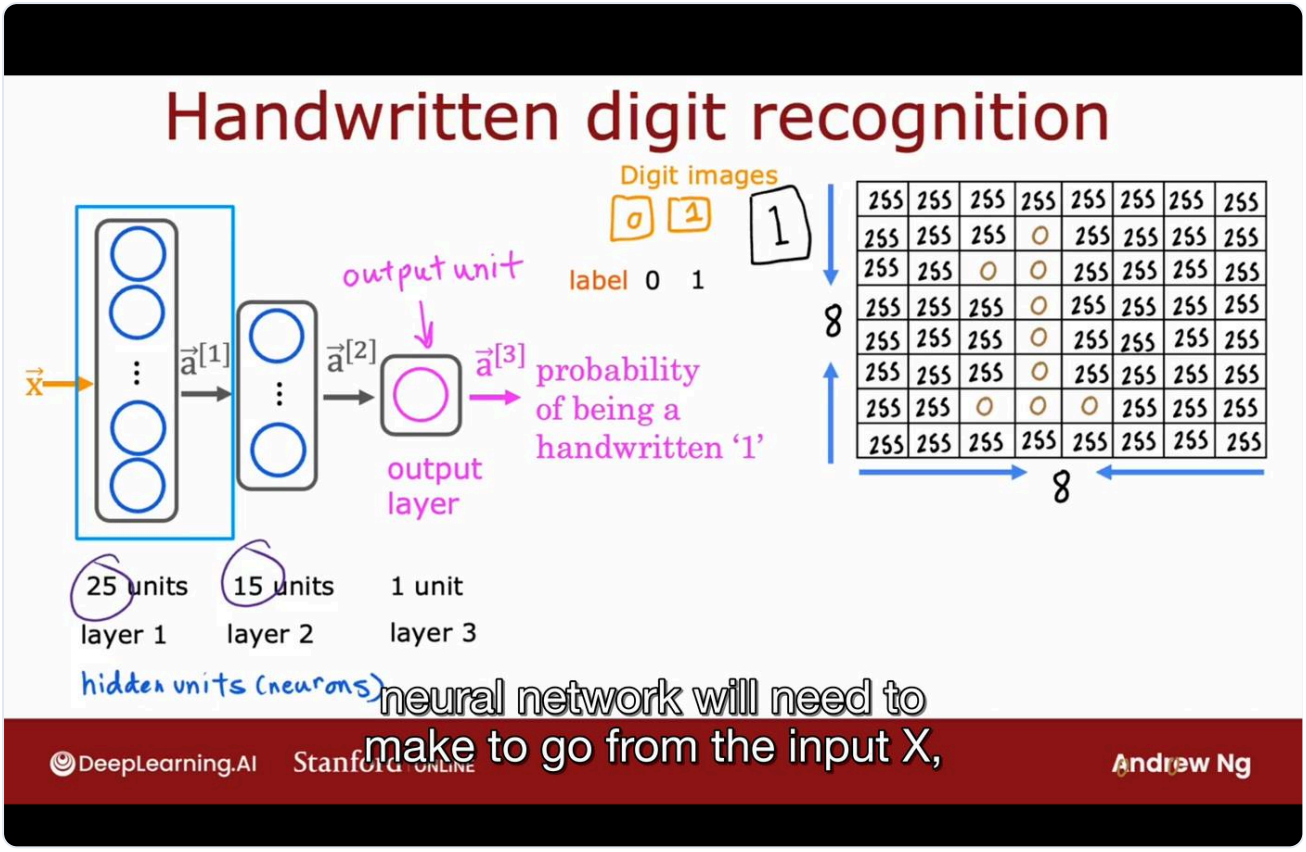


FIGURE 7.2 | Handwritten Digit Recognition

So let's step through the sequence of computations that in your neural network will need to make to go from the input X, this eight by eight or 64 numbers to the predicted probability a3. The first computation is to go from X to a1, and that's what the first layer of the first hidden layer does. It carries out a computation of a super strip square bracket 1 equals this formula on the right. Notice that a one has 25 numbers because this hidden layer has 25 units. Which is why the parameters go from w1 through w25 as well as b1 through b25. And I've written x here but I could also have written a0 here because by convention the activation of layer zero, that is a0 is equal to the input feature value x.

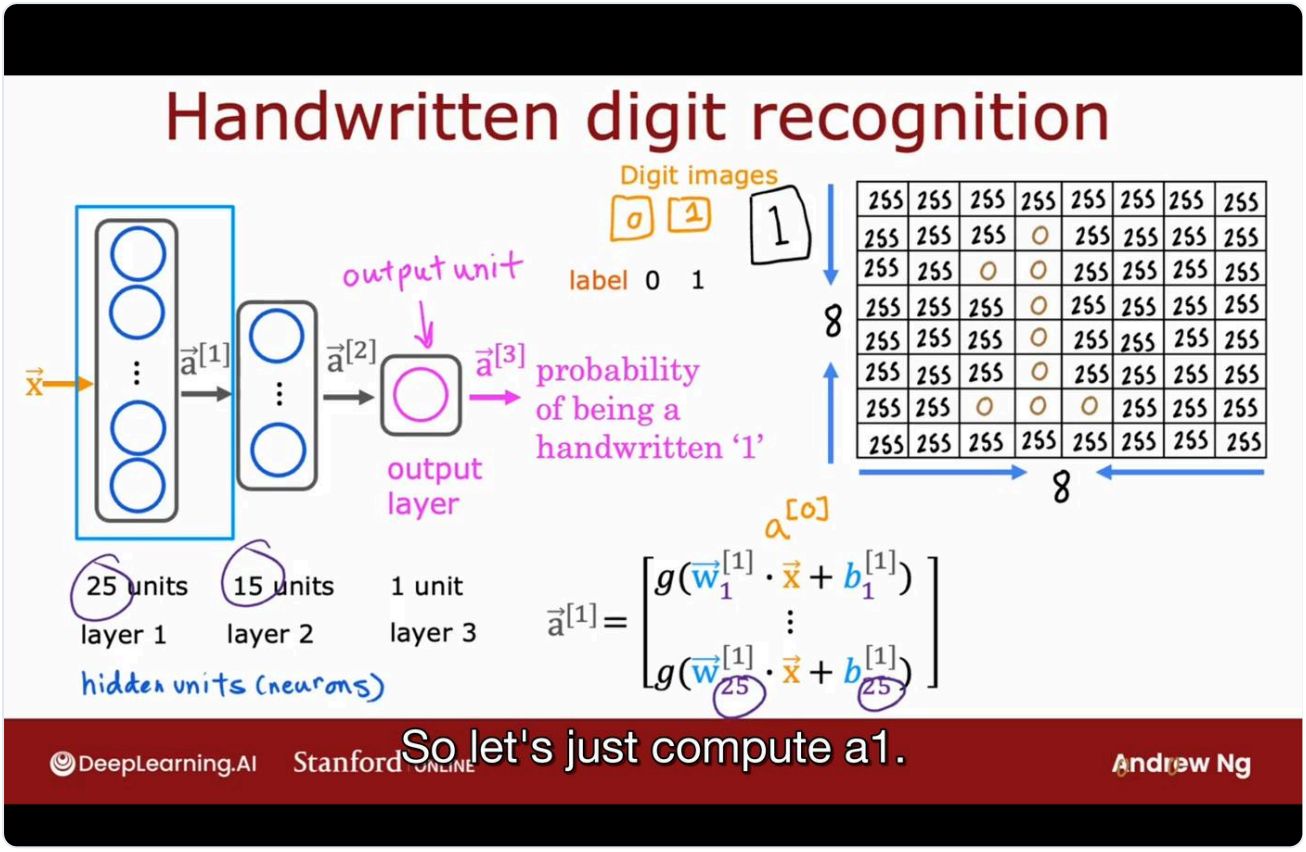


FIGURE 7.3 | Handwritten Digit Recognition

So let's just compute a1. The next step is to compute a2.

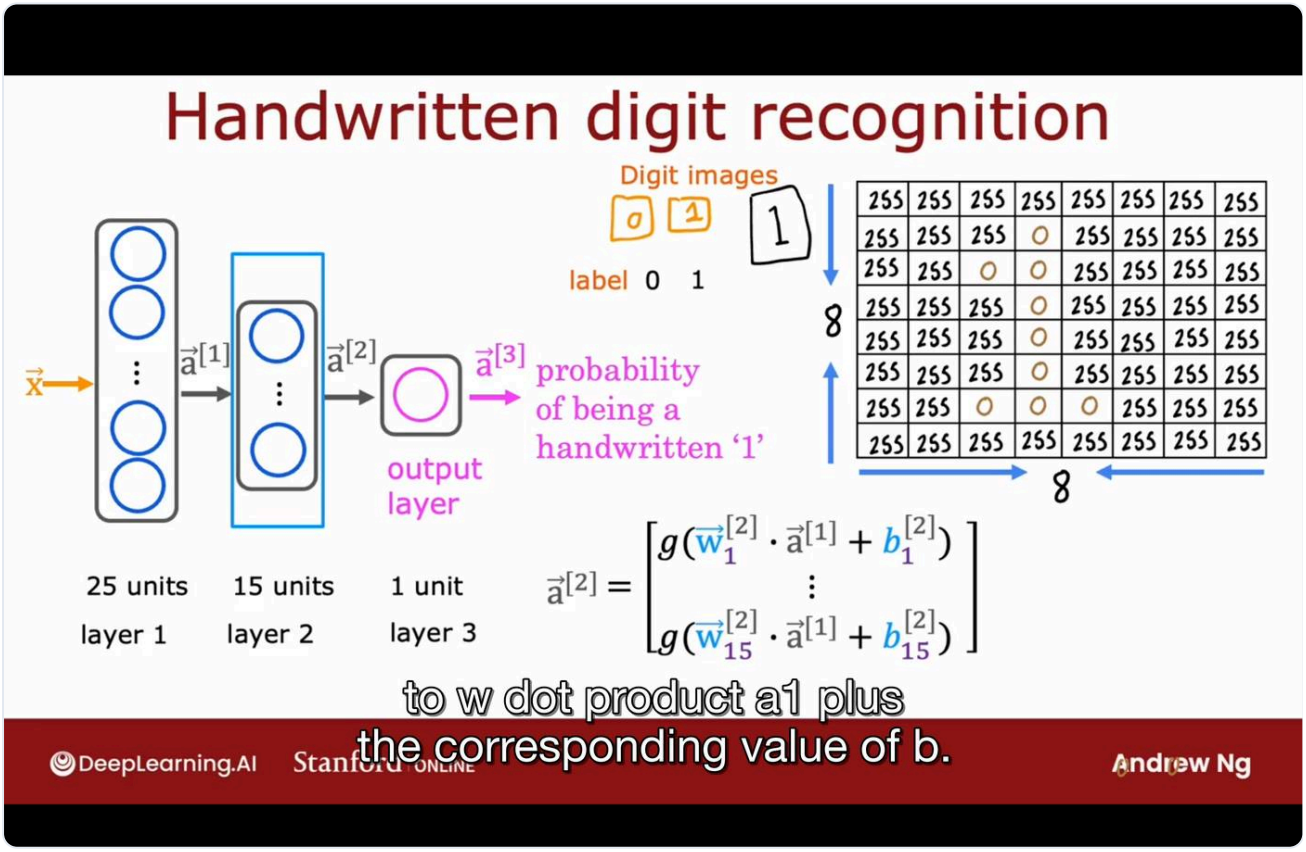


FIGURE 7.4 | Handwritten Digit Recognition

Looking at the second hidden layer, it then carries out this computation where a2 is a function of a1 and it's computed as the safe point activation function applied to w dot product a1 plus the corresponding value of b. Notice that layer two has 15 neurons or 15 units, which is why the parameters Here run from w1 through w15 and b1 through b15. Now we've computed a2. The Final step is then to compute a3 and we do so using a very similar computation.

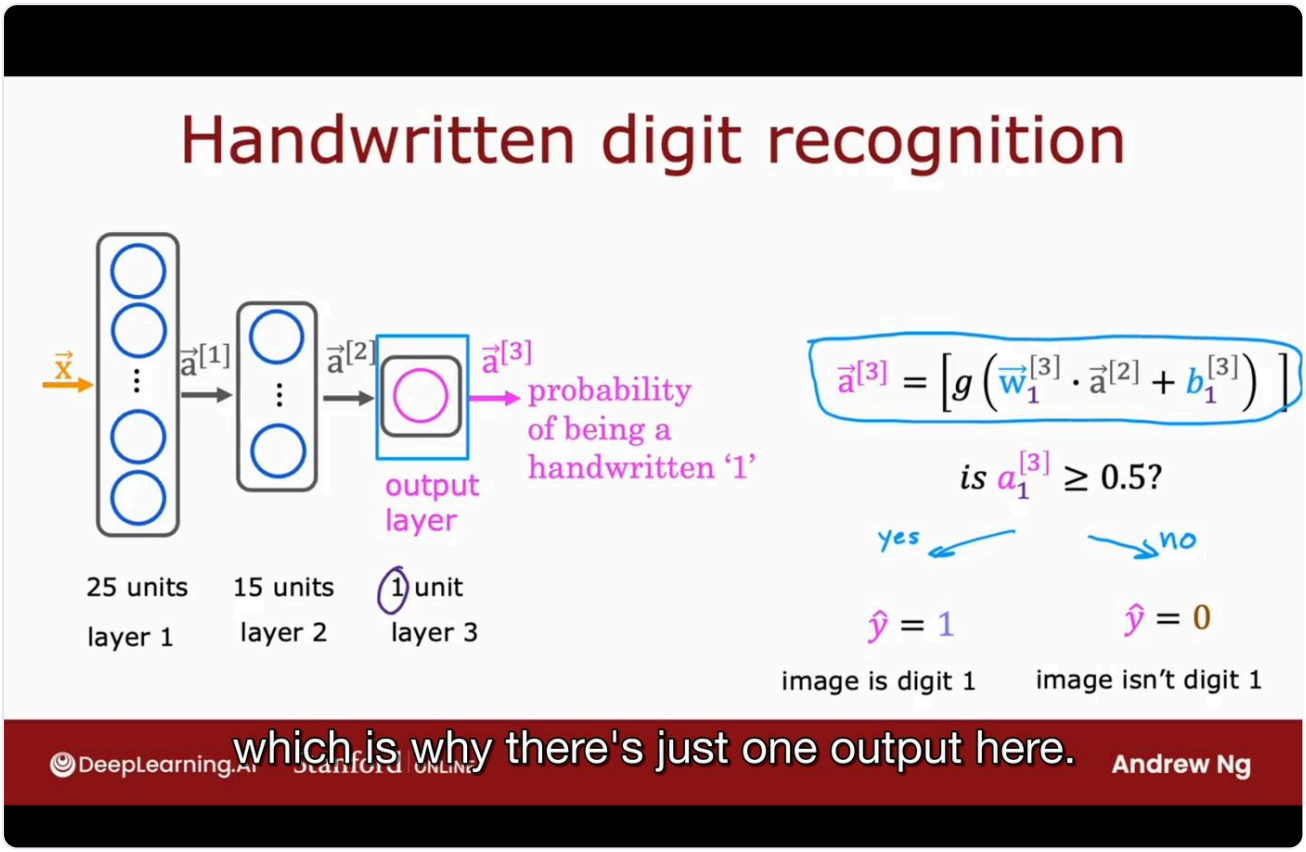


FIGURE 7.5 | Handwritten Digit Recognition

Only now, this third layer, the output layer has just one unit, which is why there's just one output here. So a3 is just a scalar. And finally you can optionally take a3 subscript one and threshold it at 4.5 to come up with a binary classification label. Is this the digit 1? Yes or no? So the sequence of computations first takes x and then computes a1, and then computes a2, and then computes a3, which is also the output of the neural networks. You can also write that as f(x). So remember when we learned about linear regression and logistic regression, we use f(x) to denote the output of linear regression or logistic regression. So we can also use f(x) to denote the function computed by the neural network as a function of x.

Because this computation goes from left to right, you start from x and compute a1, then a2, then a3.

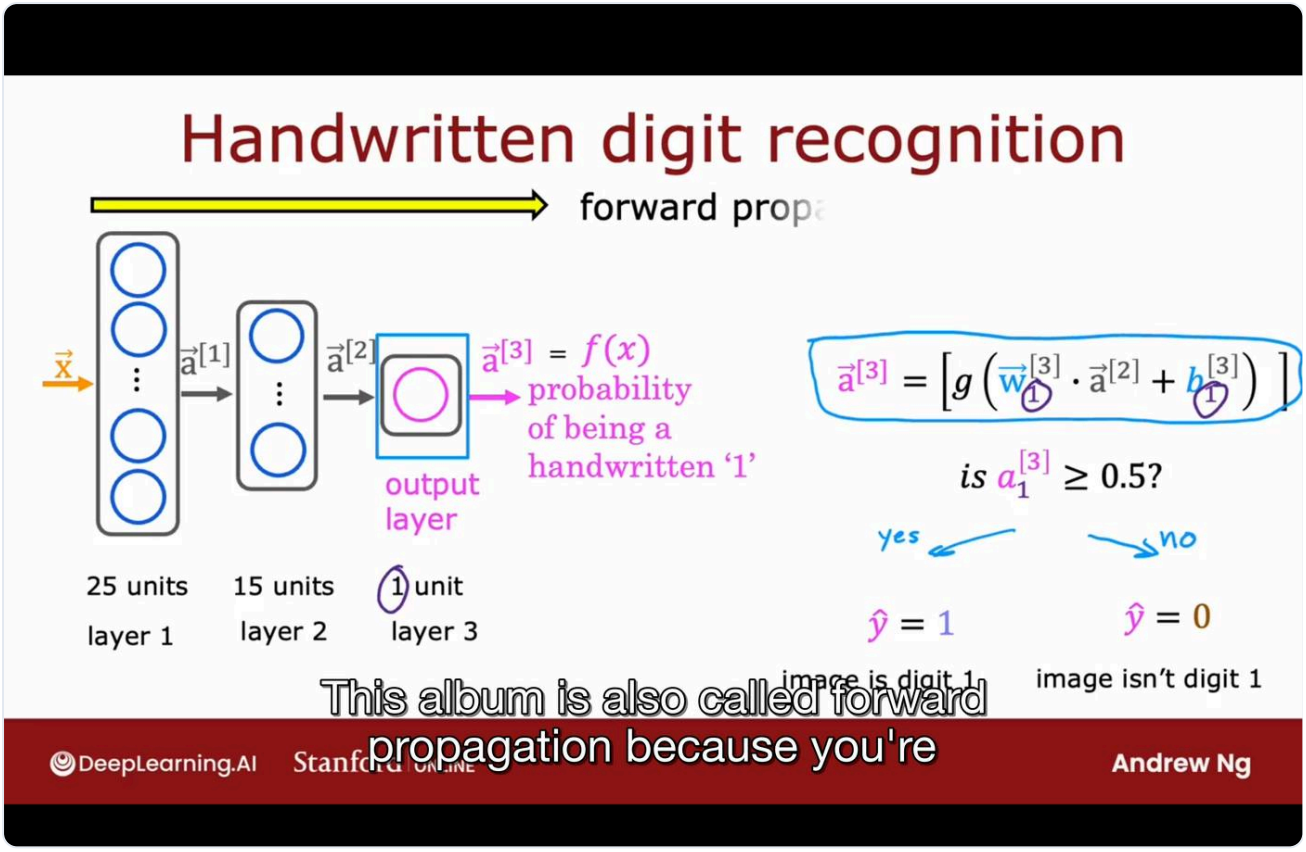


FIGURE 7.6 | Handwritten Digit Recognition

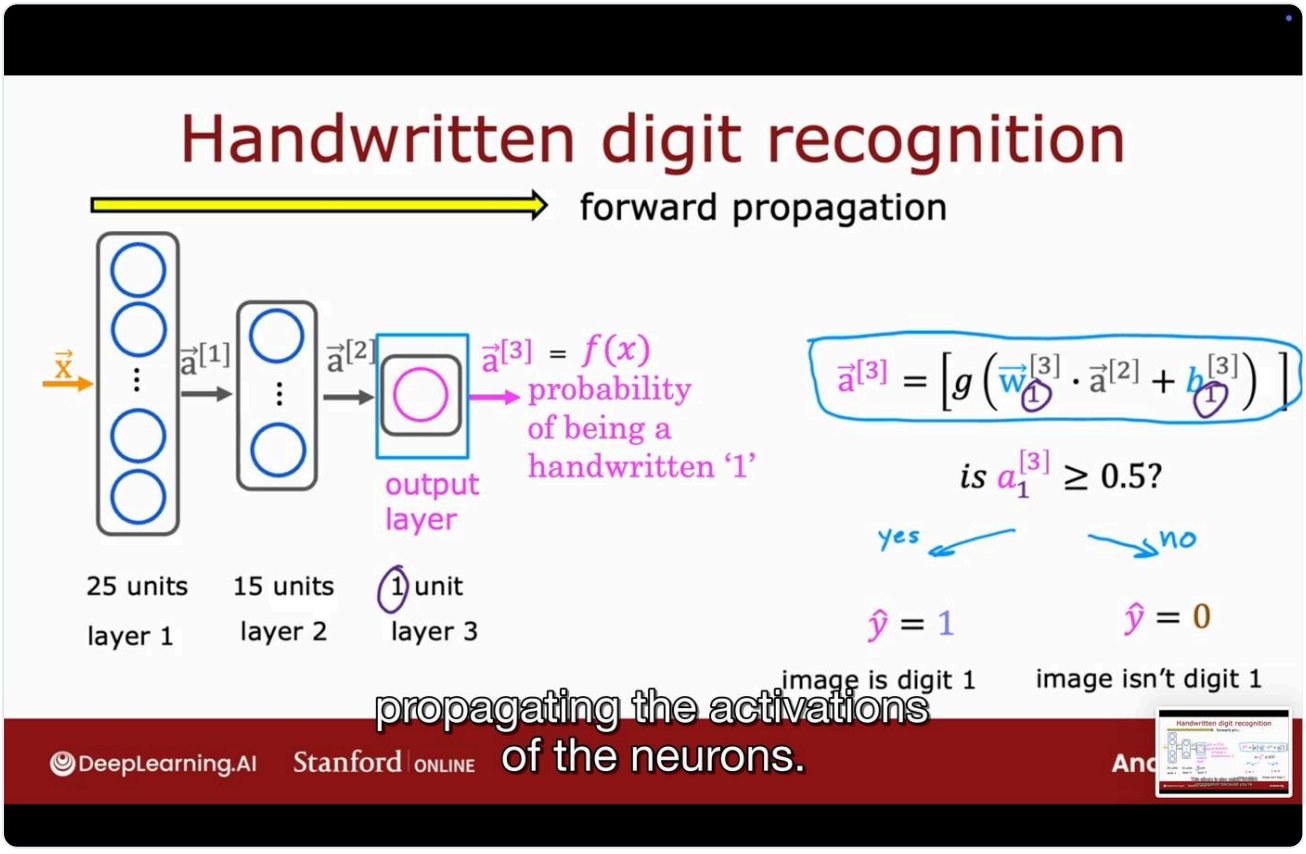


FIGURE 7.7 | Handwritten Digit Recognition

This album is also called forward propagation because you're propagating the activations of the neurons. So you're making these computations in the forward direction from left to right. And this is in contrast to a different algorithm called backward propagation or back propagation, which is used for learning. And that's something you learn about next week. And by the way, this type of neural network architecture where you have more hidden units initially and then the number of hidden units decreases as you get closer to the output layer. There's also a pretty typical choice when choosing neural network architectures. And you see more examples of this in the practice lab as well.

So that's neural network inference using the forward propagation algorithm. And with this, you'd be able to download the parameters of a neural network that someone else had trained and posted on the Internet. And you'd be able to carry out inference on your new data using their neural network. Now that you've seen the math and the algorithm, let's take a look at how you can actually implement this in tensorflow. Specifically, let's take a look at this in the next video.

LECTURE 8

Coffee Roasting

TensorFlow is one of the leading frameworks to implementing deep learning algorithms. When I'm building projects, TensorFlow is actually a tool that I use the most often. The other popular tool is PyTorch. But we're going to focus in this specialization on TensorFlow. In this video, let's take a look at how you can implement inferencing code using TensorFlow. Let's dive in. One of the remarkable things about neural networks is the same algorithm can be applied to so many different applications. For this video and in some of the labs for you to see what the neural network is doing, I'm going to use another example to illustrate inference. Sometimes I do like to roast coffee beans myself at home.

My favorite is actually Colombian coffee beans. Can the learning algorithm help optimize the quality of the beans you get from a roasting process like this?

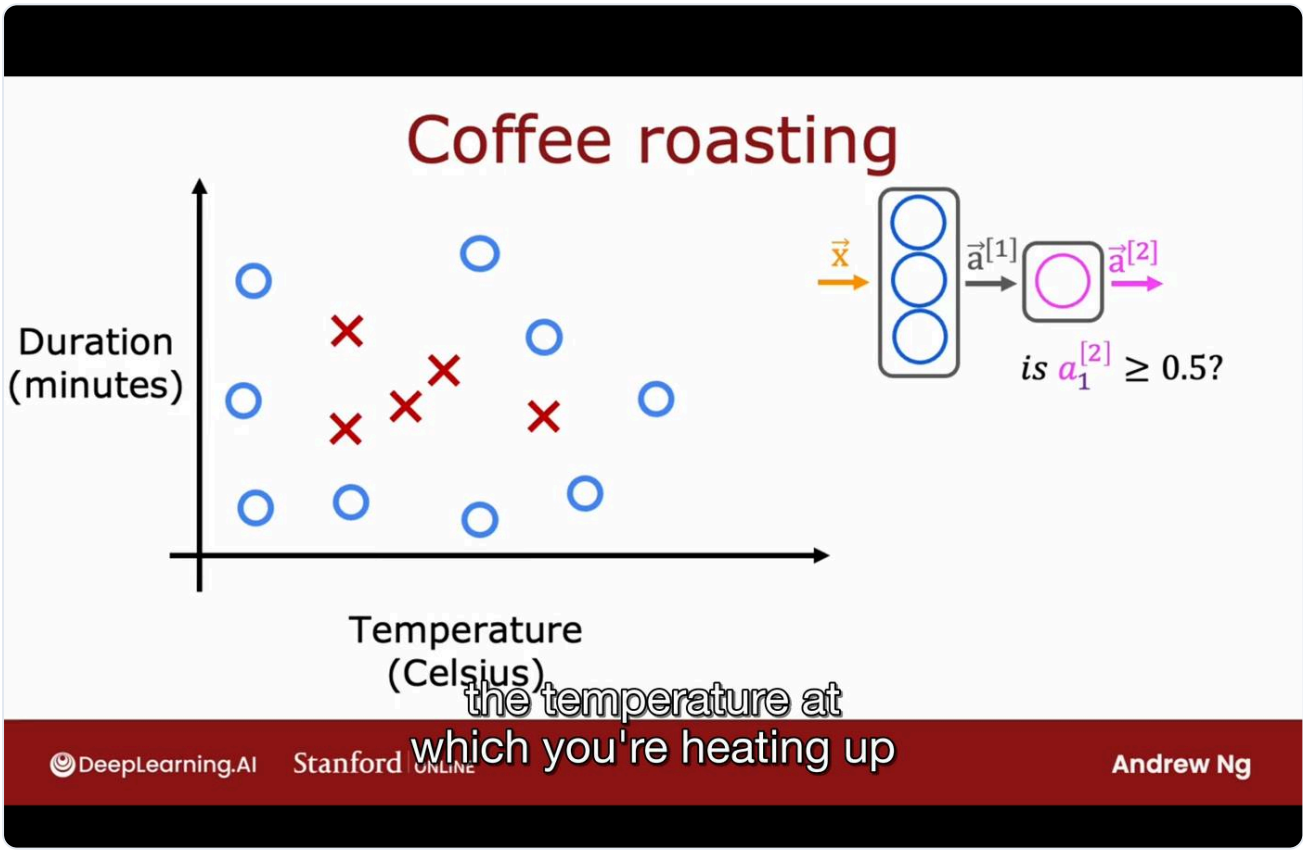


FIGURE 8.1 | Coffee Roasting

When you're roasting coffee, two parameters you get to control are the temperature at which you're heating up the raw coffee beans to turn them into nicely roasted coffee beans, as well as the duration or how long are you going to roast the beans. In this slightly simplified example, we've created the datasets of different temperatures and different durations, as well as labels showing whether the coffee you roasted is good-tasting coffee. Where cross here, the positive

cross y equals 1 corresponds to good coffee, and all the negative cross corresponds to bad coffee. It looks like a reasonable way to think of this dataset is if you cook it at too lower temperature, it doesn't get roasted and it ends up undercooked.

If you cook it, not for long enough, the duration is too short, it's also not a nicely roasted set of beans. Finally, if you were to cook it either for too long or for too higher temperature, then you end up with overcooked beans. They're a little bit burnt beans.

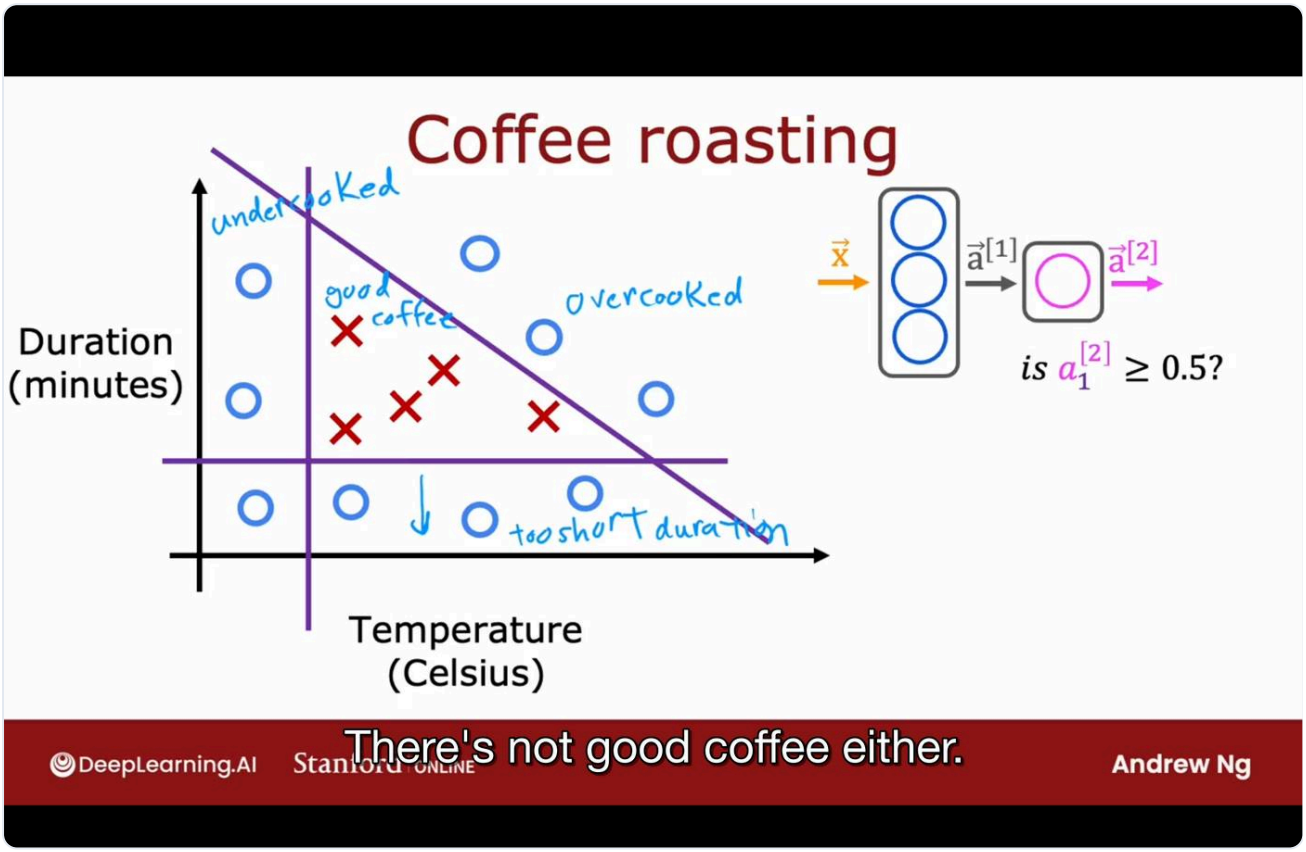


FIGURE 8.2 | Coffee Roasting

There's not good coffee either. It's only points within this little triangle here that corresponds to good coffee. This example is simplified a bit from actual coffee roasting.

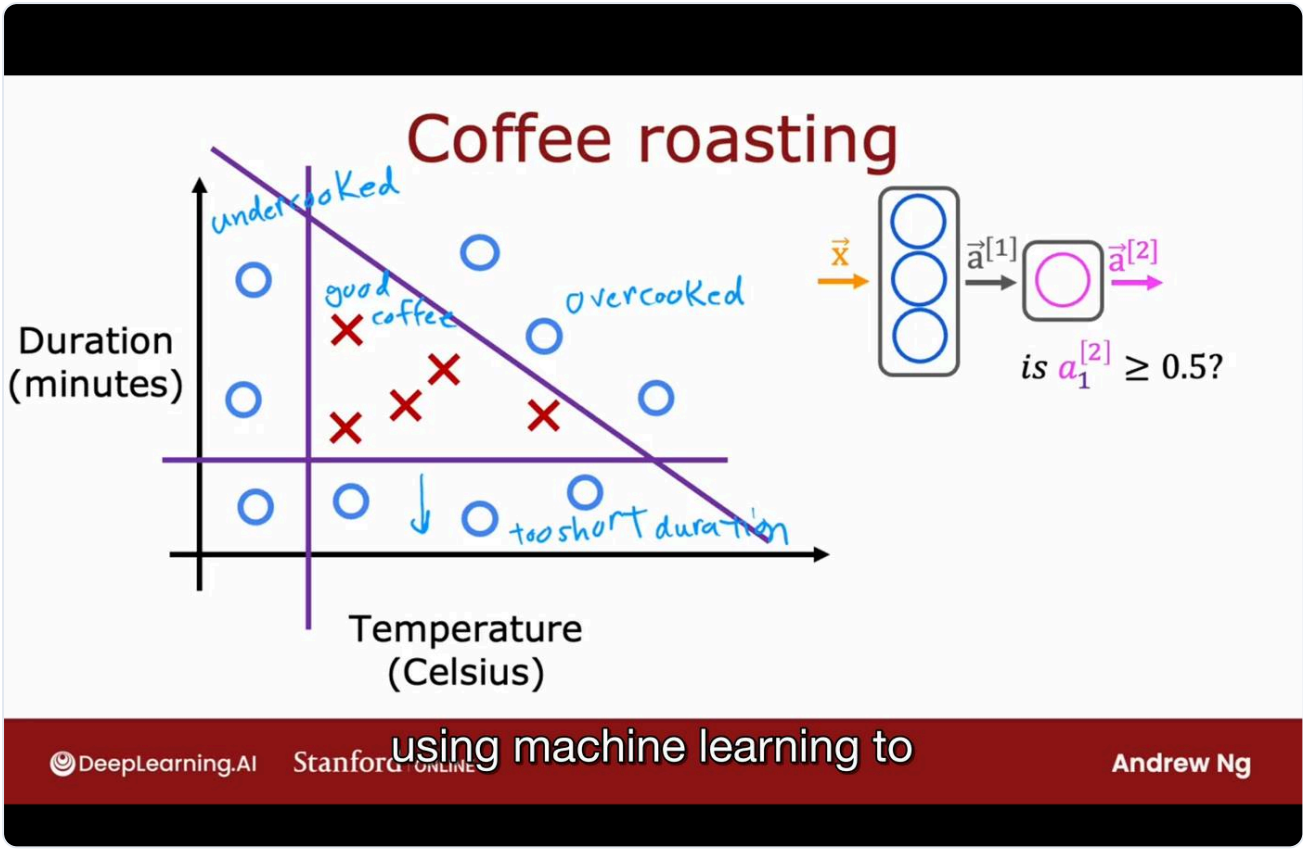


FIGURE 8.3 | Coffee Roasting

Even though this example is a simplified one for the purpose of illustration, there have actually been serious projects using machine learning to optimize coffee roasting as well.

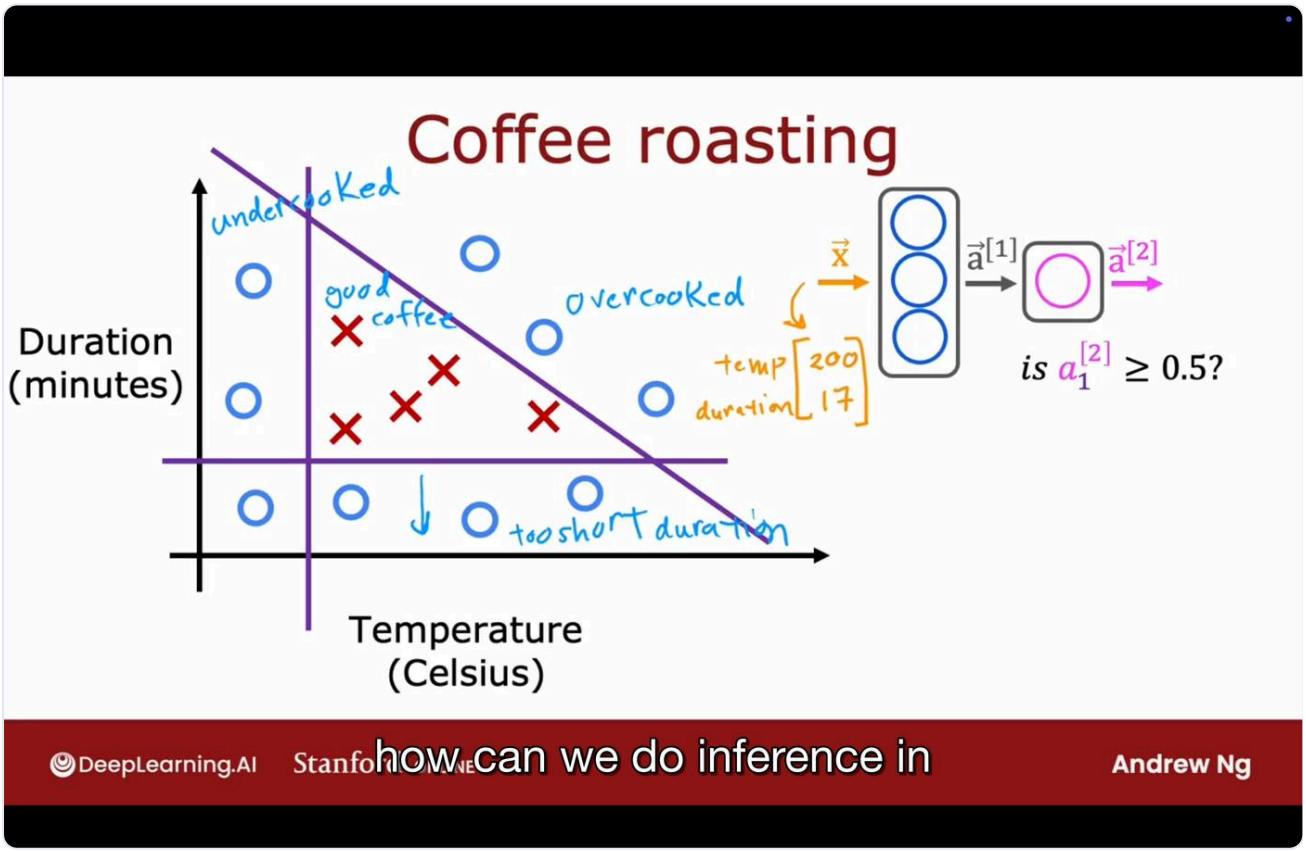


FIGURE 8.4 | Coffee Roasting

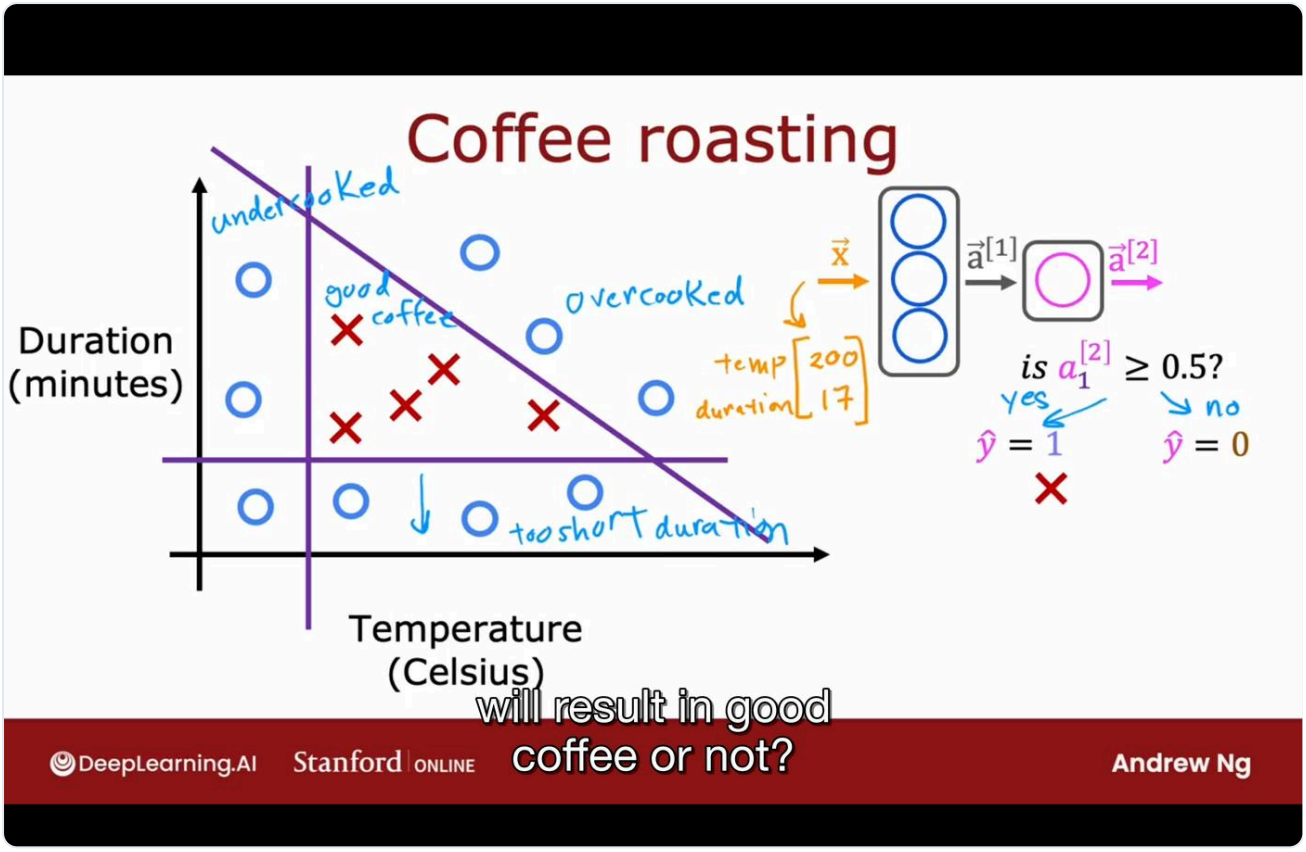


FIGURE 8.5 | Coffee Roasting

The task is given a feature vector x with both temperature and duration, say 200 degrees Celsius for 17 minutes, how can we do inference in a neural network to get it to tell us whether or not this temperature and duration setting will result in good coffee or not? It looks like this. We're going to set x to be an array of two numbers. The input features 200 degrees celsius and 17 minutes. Then you create Layer 1 as this first hidden layer, the neural network, as dense open parenthesis units 3, that means three units or three hidden units in this layer using as the activation function, the sigmoid function. Dense is another name for the layers of a neural network that we've learned about so far.

As you learn more about neural networks, you learn about other types of layers as well. But for now, we'll just use the dense layer, which is the layer type you've learned about in the last few videos for all of our examples. Next, you compute a1 by taking Layer 1, which is actually a function, and applying this function Layer 1 to the values of x. That's how you get a1, which is going to be a list of three numbers because Layer 1 had three units. So a1 here may, just for the sake of illustration, be 0.2, 0.7, 0.3. Next, for the second hidden layer, Layer 2, would be dense. Now this time it has one unit and again to sigmoid activation function, and you can then compute a2 by applying this Layer 2 function to the activation values from Layer 1 to a1.

That will give you the value of a2, which for the sake of illustration is maybe 0.8.

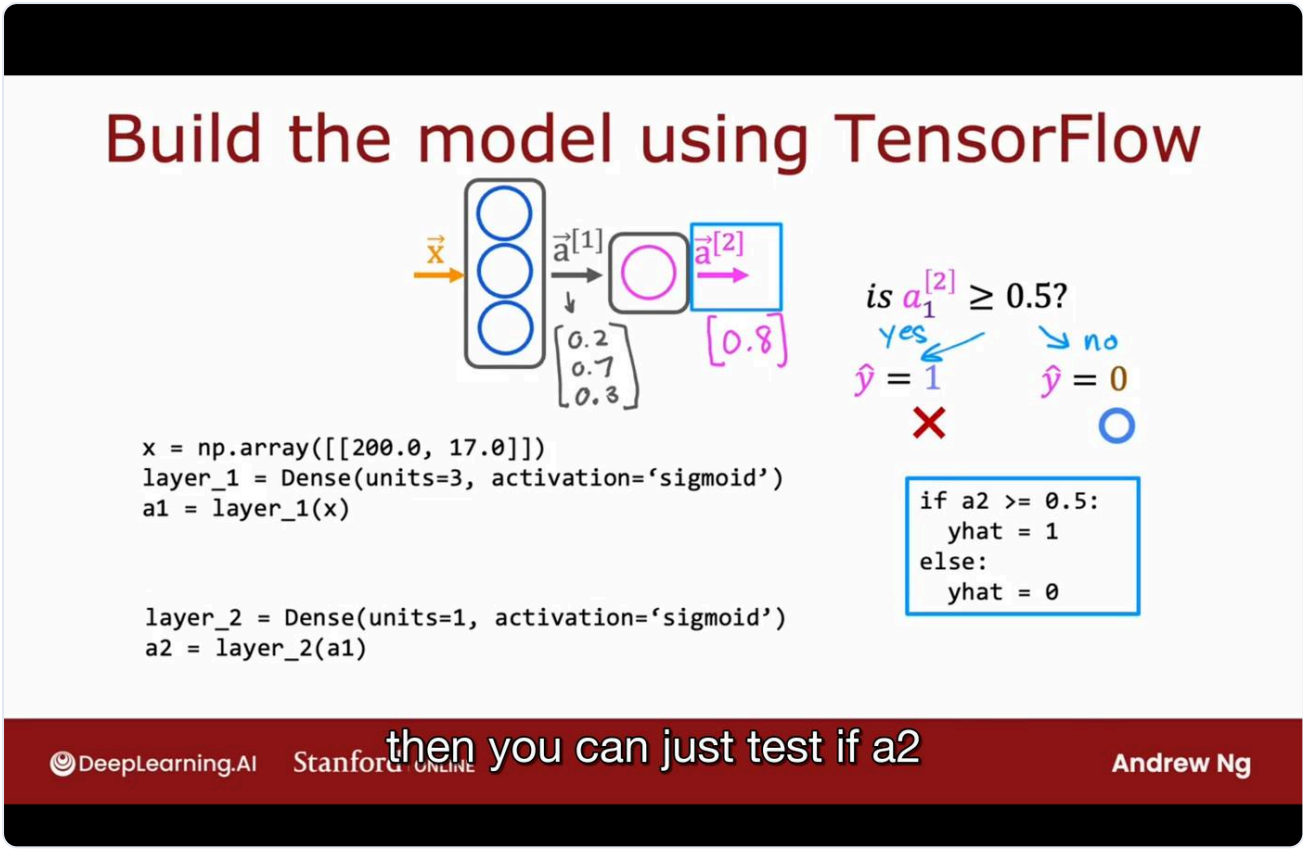


FIGURE 8.6 | Build the Model Using TensorFlow

Finally, if you wish to threshold it at 0.5, then you can just test if a2 is greater and equal to 0.5 and set y-hat equals to one or zero positive or negative cross accordingly. That's how you do inference in the neural network using TensorFlow. There are some additional details that I didn't go over here, such as how to load the TensorFlow library and how to also load the parameters w and b of the neural network. But we'll go over that in the lab. Please be sure to take a look at the lab. But these are the key steps for forward propagation in how you compute a1 and a2 and optionally threshold a2. Let's look at one more example and we're going to go back to the handwritten digit classification problem.

In this example, x is a list of the pixel intensity values. So x is equal to a numpy array of this list of pixel intensity values. Then to initialize and carry out one step of forward propagation, Layer 1 is a dense layer with 25 units and the sigmoid activation function. You then compute a1 equals the Layer 1 function applied to x. To build and carry out inference through the second layer, similarly, you set up Layer 2 as follows, and then computes a2 as Layer 2 applied to a1. Then finally, Layer 3 is the third and final dense layer.

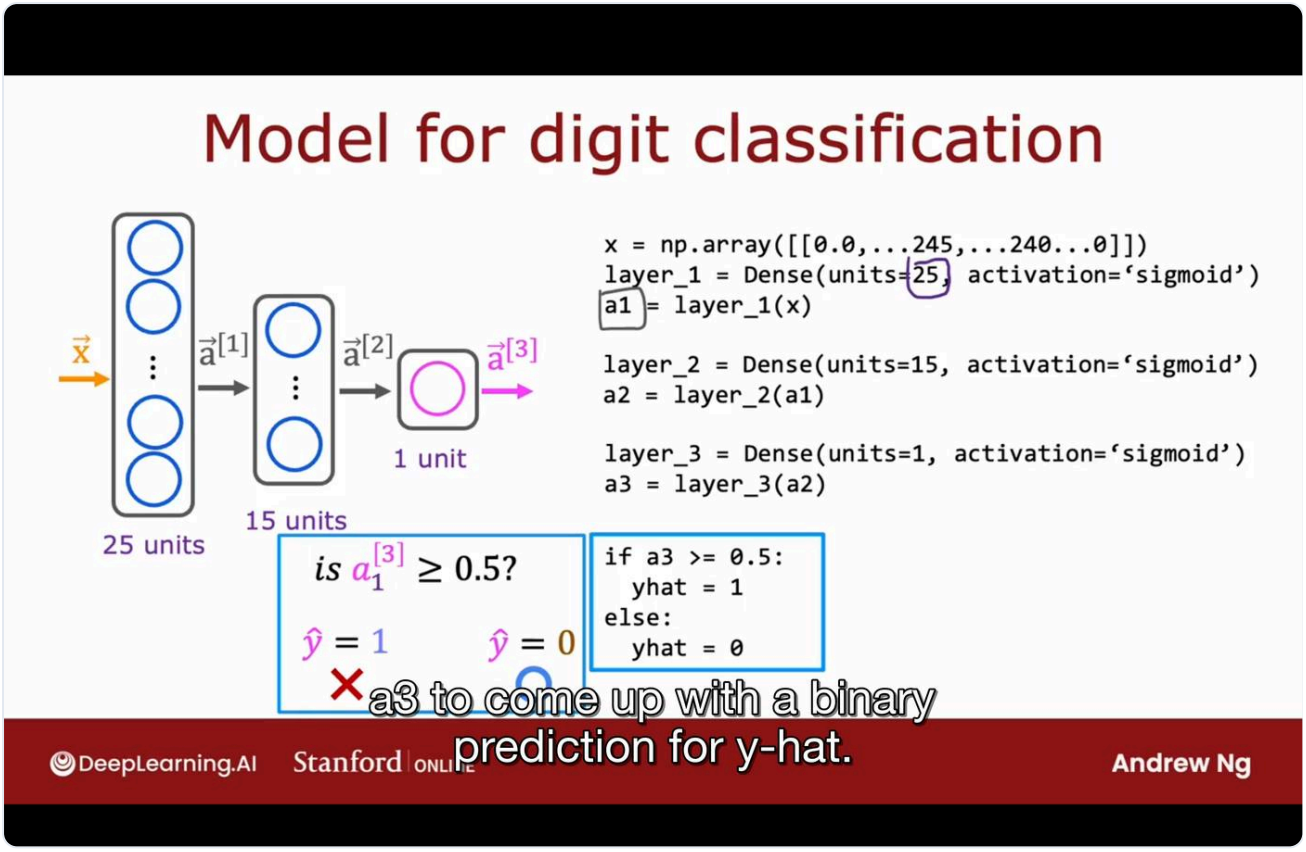


FIGURE 8.7 | Model for Digit Classification

Then finally, you can optionally threshold a3 to come up with a binary prediction for y-hat. That's the syntax for carrying out inference in TensorFlow. One thing I briefly alluded to is the structure of the numpy arrays. TensorFlow treats data in a certain way that is important to get right. In the next video, let's take a look at how TensorFlow handles data.

LECTURE 9

Note About Numpy Arrays

In this video, I want to step through with you how data is represented in NumPy and in TensorFlow. So that as you're implementing new neural networks, you can have a consistent framework to think about how to represent your data. One of the unfortunate things about the way things are done in code today is that many, many years ago NumPy was first created and became a standard library for linear algebra and Python. And then much later the Google brain team, the team that I had started and once led created TensorFlow. And so unfortunately there are some inconsistencies between how data is represented in NumPy and in TensorFlow. So it's good to be aware of these conventions so that you can implement correct code and hopefully get things running in your neural networks.

Let's start by taking a look at how TensorFlow represents data. Let's see you have a data set like this from the coffee example. I mentioned that you would write x as follows. So why do you have this double square bracket here? Let's take a look at how NumPy stores vectors and matrices. In case you think matrices and vectors are complicated mathematical concepts don't worry about it.

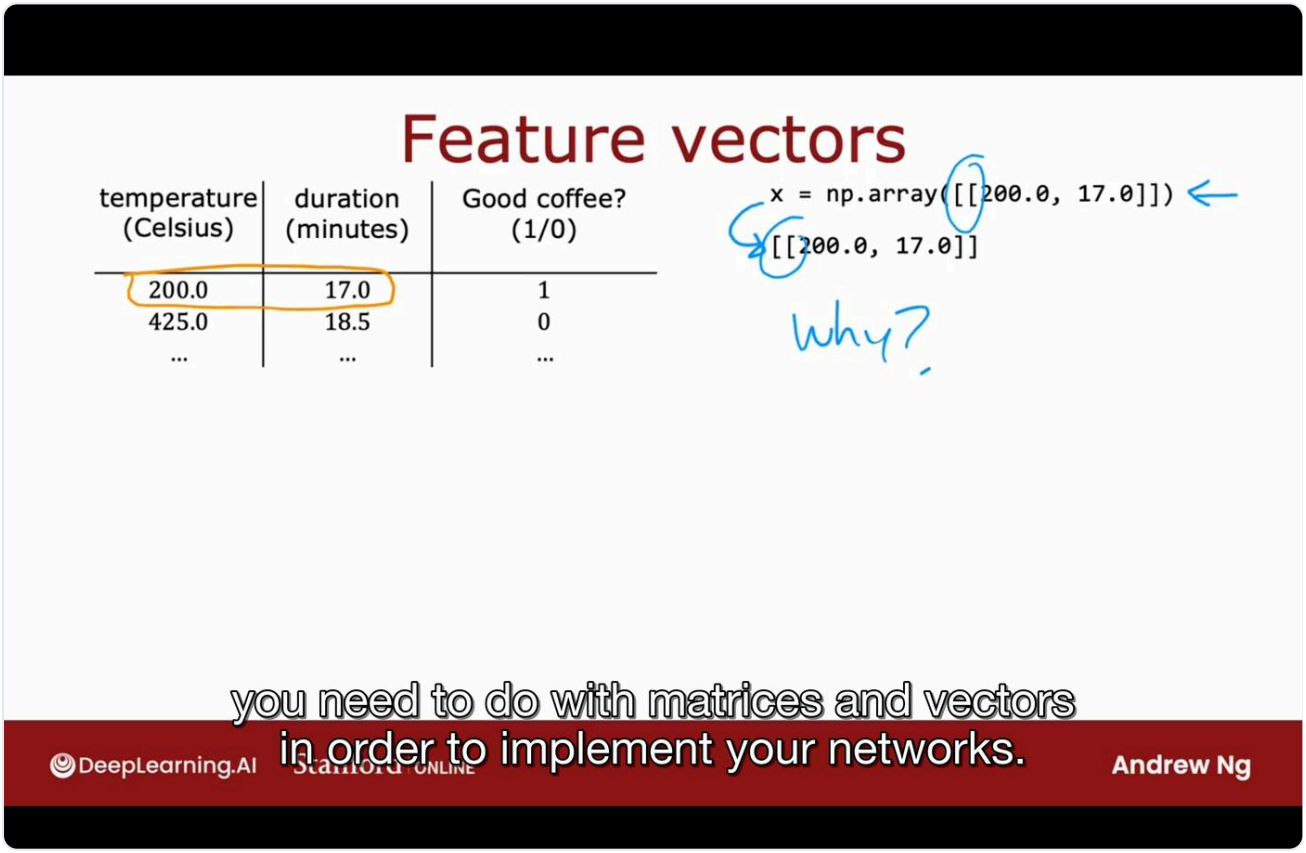


FIGURE 9.1 | Feature Vectors

We'll go through a few concrete examples and you'll be able to do everything you need to do with matrices and vectors in order to implement your networks. Let's start with an example of a matrix. Here is a matrix with 2 rows and 3 columns. Notice that there are one, two rows and 1, 2, 3

columns. So we call this a 2 x 3 matrix. And so the convention is the dimension of the matrix is written as the number of rows by the number of columns. So in code to store this matrix, this 2 x 3 matrix, you just write x = np.array of these numbers like these. Where you notice that the square bracket tells you that 1, 2, 3 is the first row of this matrix and 4, 5, 6 is the second row of this matrix.

And then this open square bracket groups the first and the second row together. So this sets x to be this to the array of numbers.

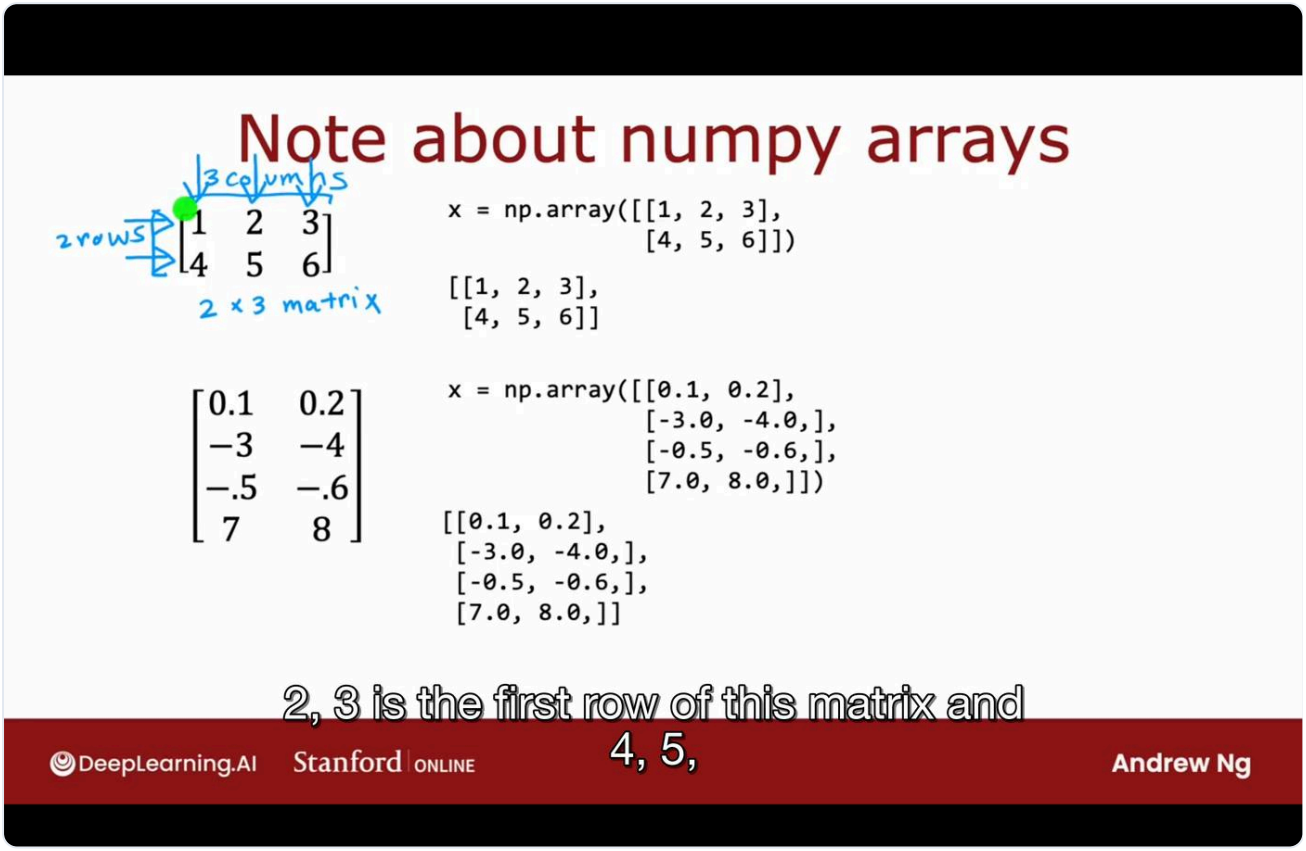


FIGURE 9.2 | Note About Numpy Arrays

So matrix is just a 2D array of numbers. Let's look at one more example, here I've written out another matrix. How many roles and how many columns does this have? Well, you can count this as one, two, three, four rows and it has one, two columns. So this is a number of rows by the number of columns matrix, so it's a 4 x 2 matrix. And so to store this in code, you will write x equals np.array and then this syntax over here to store these four rows of matrix in the variable x. So this creates a 2D array of these eight numbers. Matrices can have different dimensions. You saw an example of an 2 x 3 matrix and the 4 x 2 matrix. A matrix can also be other dimensions like 1 x 2 or 2 x 1.

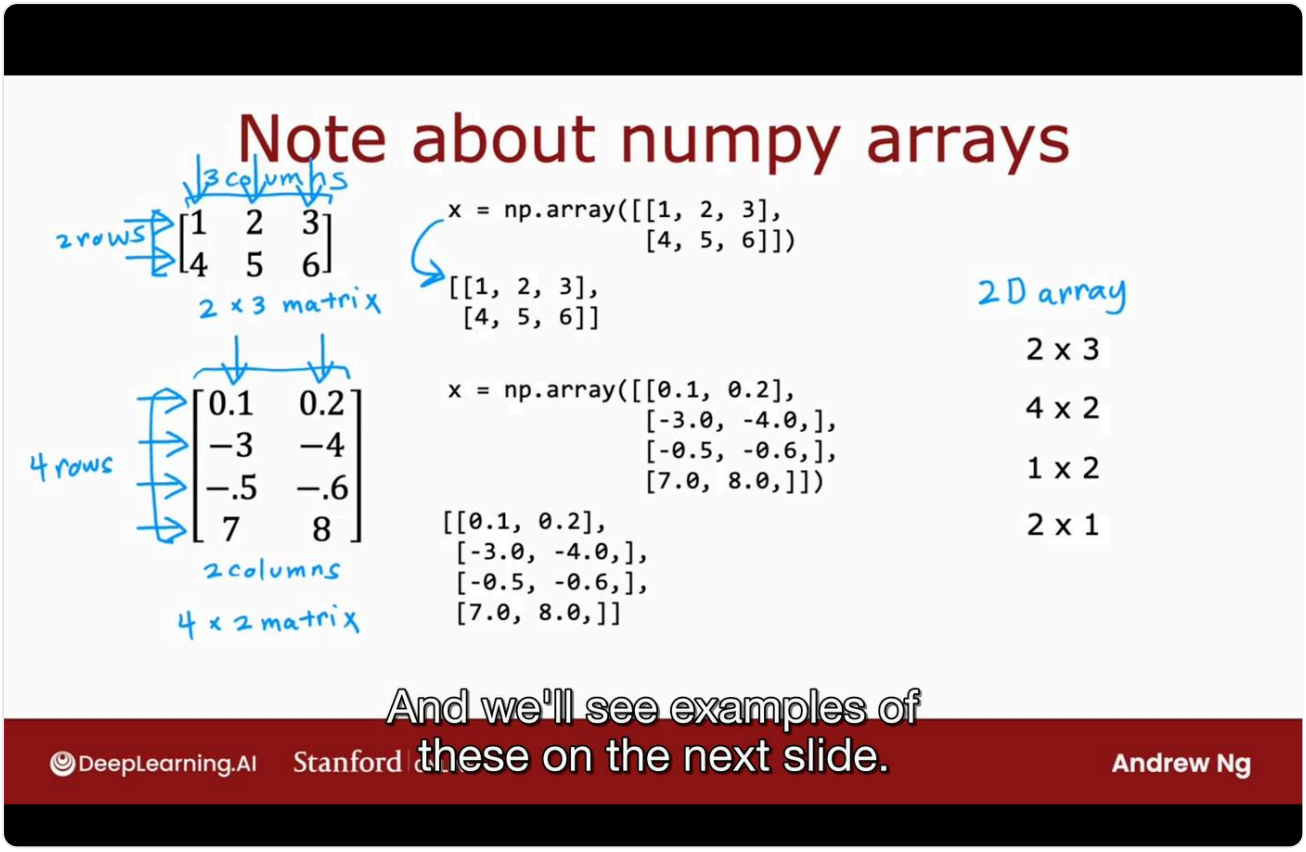


FIGURE 9.3 | Note About Numpy Arrays

And we'll see examples of these on the next slide. So what we did previously when setting x to be input feature vectors, was set x to be equal to np.array with two square brackets, 200, 17. And what that does is this creates a 1 x 2 matrix, that is just one row and two columns. Let's look at a different example, if you were to define x to be np.array but now written like this, this creates a 2 x 1 matrix that has two rows and one column. Because the first row is just the number 200 and the second row, is just the number 17. And so this has the same numbers but in a 2 x 1 instead of a 1 x 2 matrix. Enough this example on top is also called a row vector, is a vector that is just a single row.

And this example is also called a column vector because this vector that just has a single column.

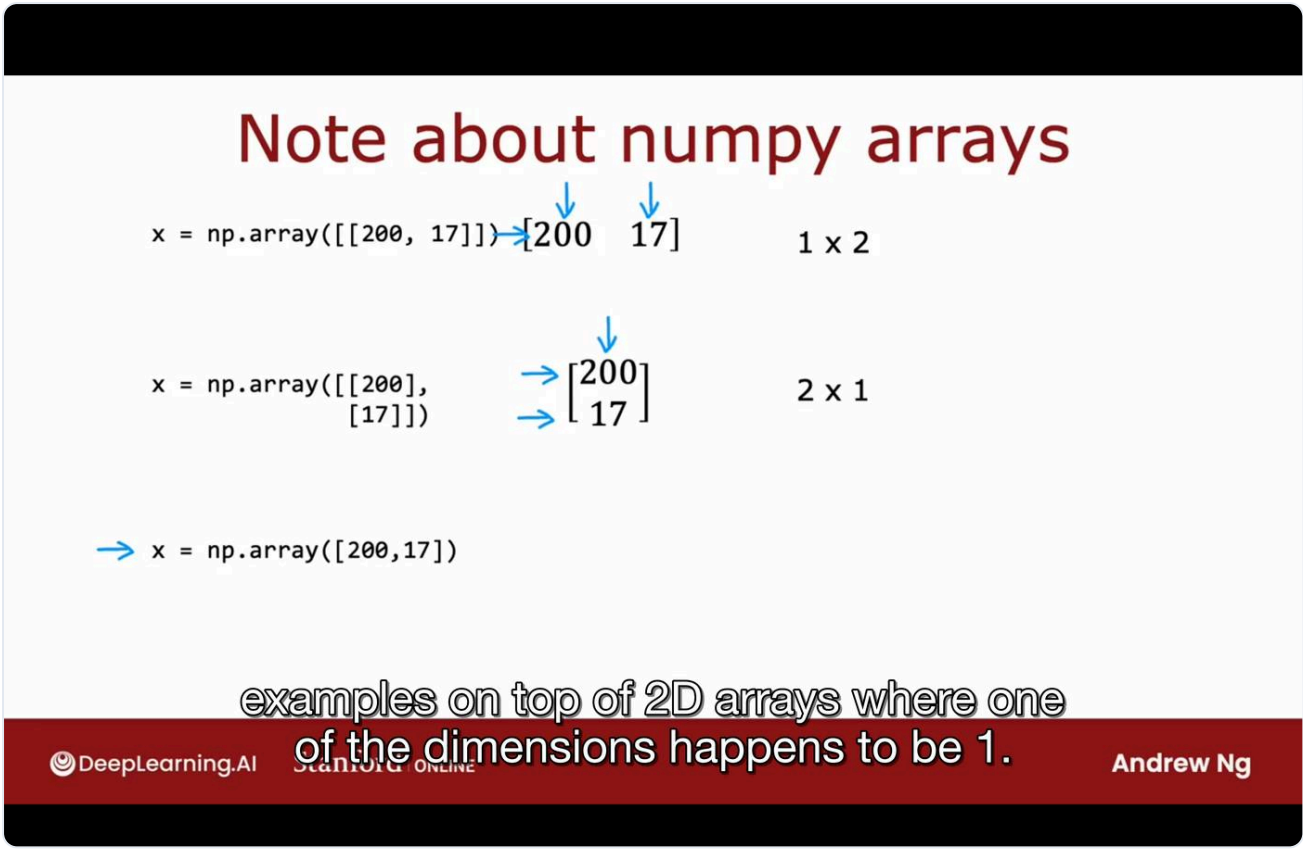


FIGURE 9.4 | Note About Numpy Arrays

And the difference between using double square brackets like this versus a single square bracket like this, is that whereas the two examples on top of 2D arrays where one of the dimensions happens to be 1. This example results in a 1D vector.

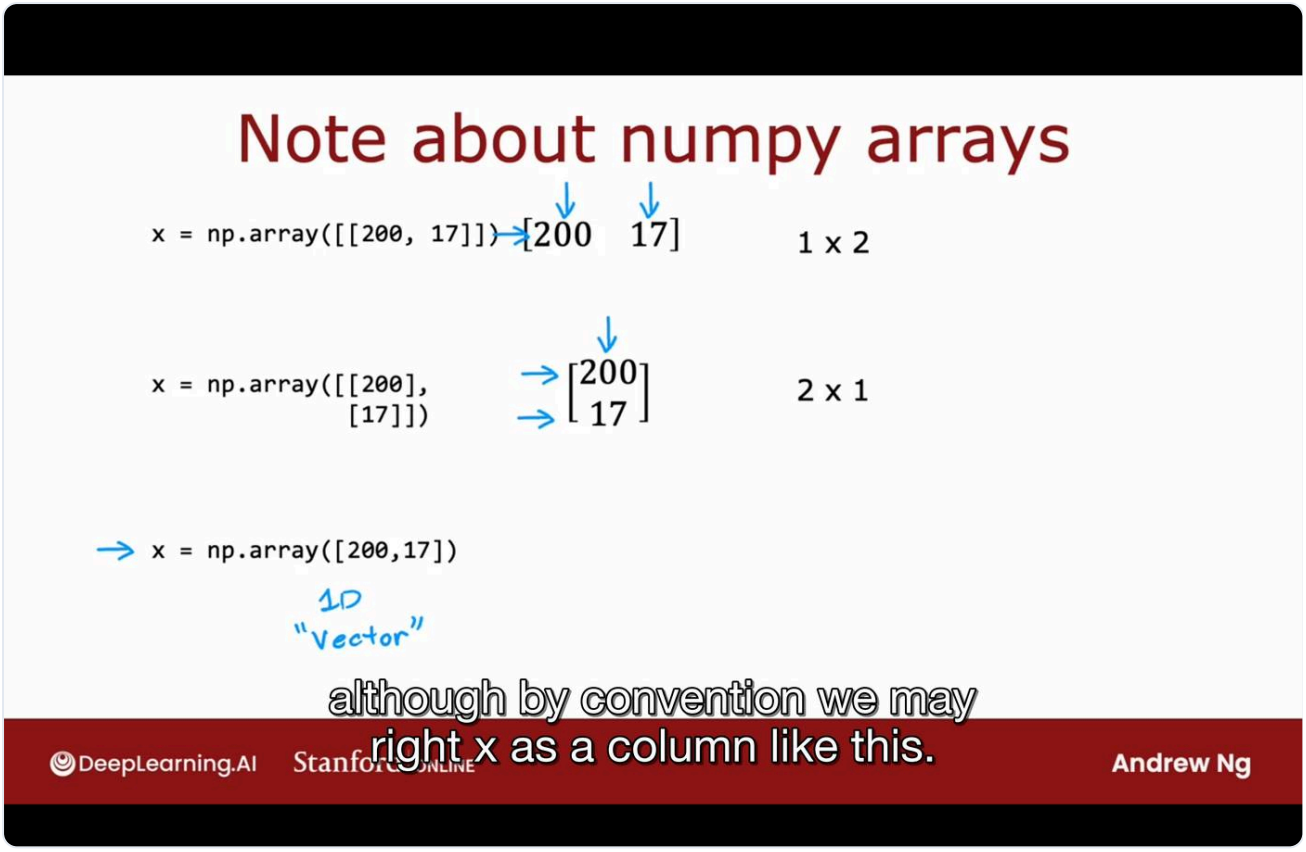


FIGURE 9.5 | Note About Numpy Arrays

So this is just a 1D array that has no rows or columns, although by convention we may right x as a column like this. So on a contrast this with what we had previously done in the first course, which was to write x like this with a single square bracket. And that resulted in what's called in Python, a 1D vector instead of a 2D matrix. And this technically is not 1 x 2 or 2 x 1, is just a linear array with no rows or no columns, but it's just a list of numbers. So whereas in course one when we're working with linear regression and logistic regression, we use these 1D vectors to represent the input features x. With TensorFlow the convention is to use matrices to represent the data.

And why is there this switching conventions? Well it turns out that TensorFlow was designed to handle very large datasets and by representing the data in matrices instead of 1D arrays, it lets TensorFlow be a bit more computationally efficient internally. So going back to our original example for the first training, example in this dataset with features 200°C in 17 minutes, we were represented like this. And so this is actually a 1 x 2 matrix that happens to have one row and two columns to store the numbers 217. And in case this seems like a lot of details and really complicated conventions, don't worry about it all of this will become clearer. And you get to see the concrete implementations of the code yourself in the optional labs and in the practice labs.

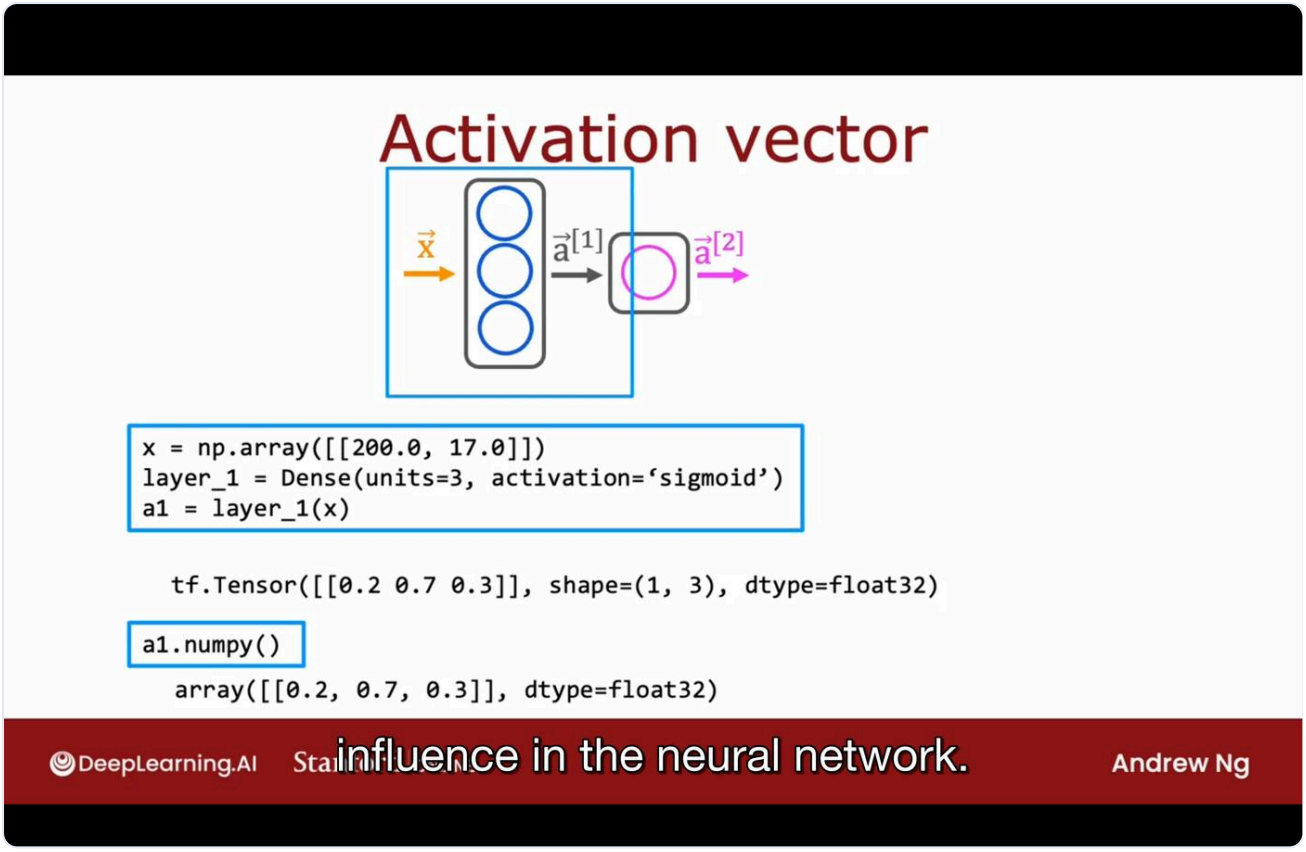


FIGURE 9.6 | Activation Vector

Going back to the code for carrying out for propagation or influence in the neural network. When you compute a1 equals layer 1 applied to x, what is a1? Well, a1 is actually going to be because the three numbers, is actually going to be a 1 x 3 matrix.

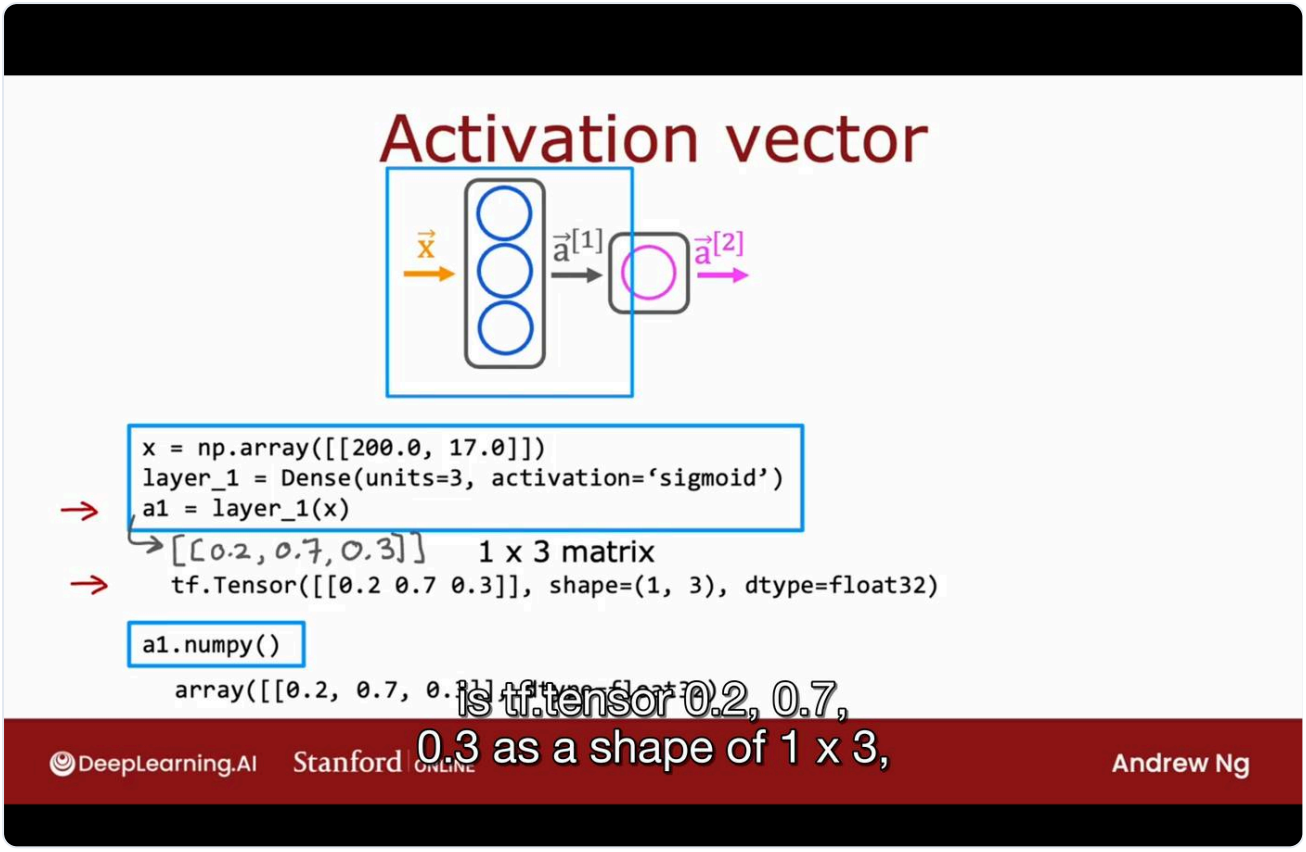


FIGURE 9.7 | Activation Vector

And if you print out a1 you will get something like this is tf.tensor 0.2, 0.7, 0.3 as a shape of 1 x 3, 1, 3 refers to that this is a 1 x 3 matrix. And this is TensorFlow's way of saying that this is a floating point number meaning that it's a number that can have a decimal point represented using 32 bits of memory in your computer, that's where the float 32 is.

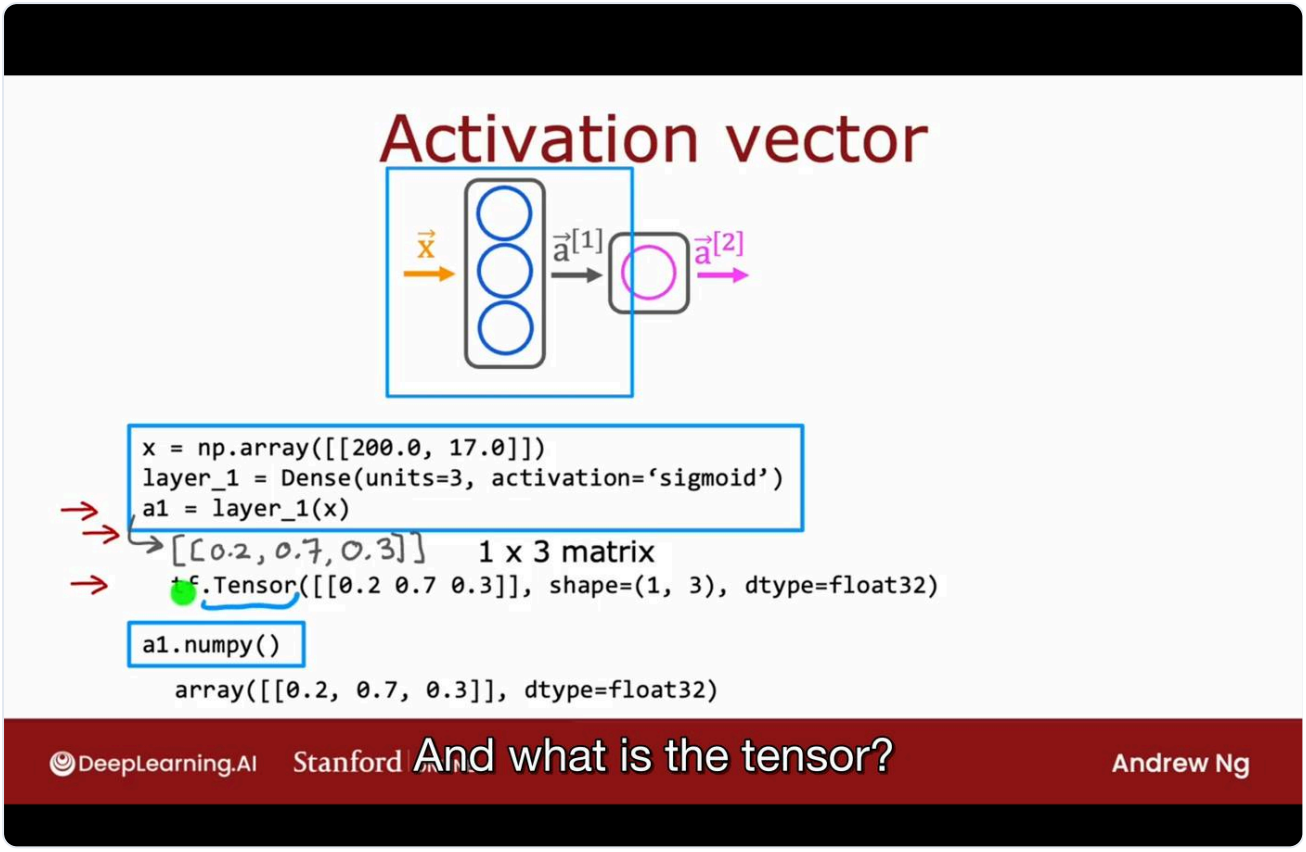


FIGURE 9.8 | Activation Vector

And what is the tensor? A tensor here is a data type that the TensorFlow team had created in order to store and carry out computations on matrices efficiently. So whenever you see tensor just think of that matrix on these few slides. Technically a tensor is a little bit more general than the matrix but for the purposes of this course, think of tensor as just a way of representing matrices. So remember I said at the start of this video that there's the TensorFlow way of representing the matrix and the NumPy way of representing matrix. This is an artifact of the history of how NumPy and TensorFlow were created and unfortunately there are two ways of representing a matrix that have been baked into these systems.

And in fact if you want to take a1 which is a tensor and want to convert it back to NumPy array, you can do so with this function a1.numpy. And it will take the same data and return it in the form of a NumPy array rather than in the form of a TensorFlow array or TensorFlow matrix. Now let's take a look at what the activations output the second layer would look like.

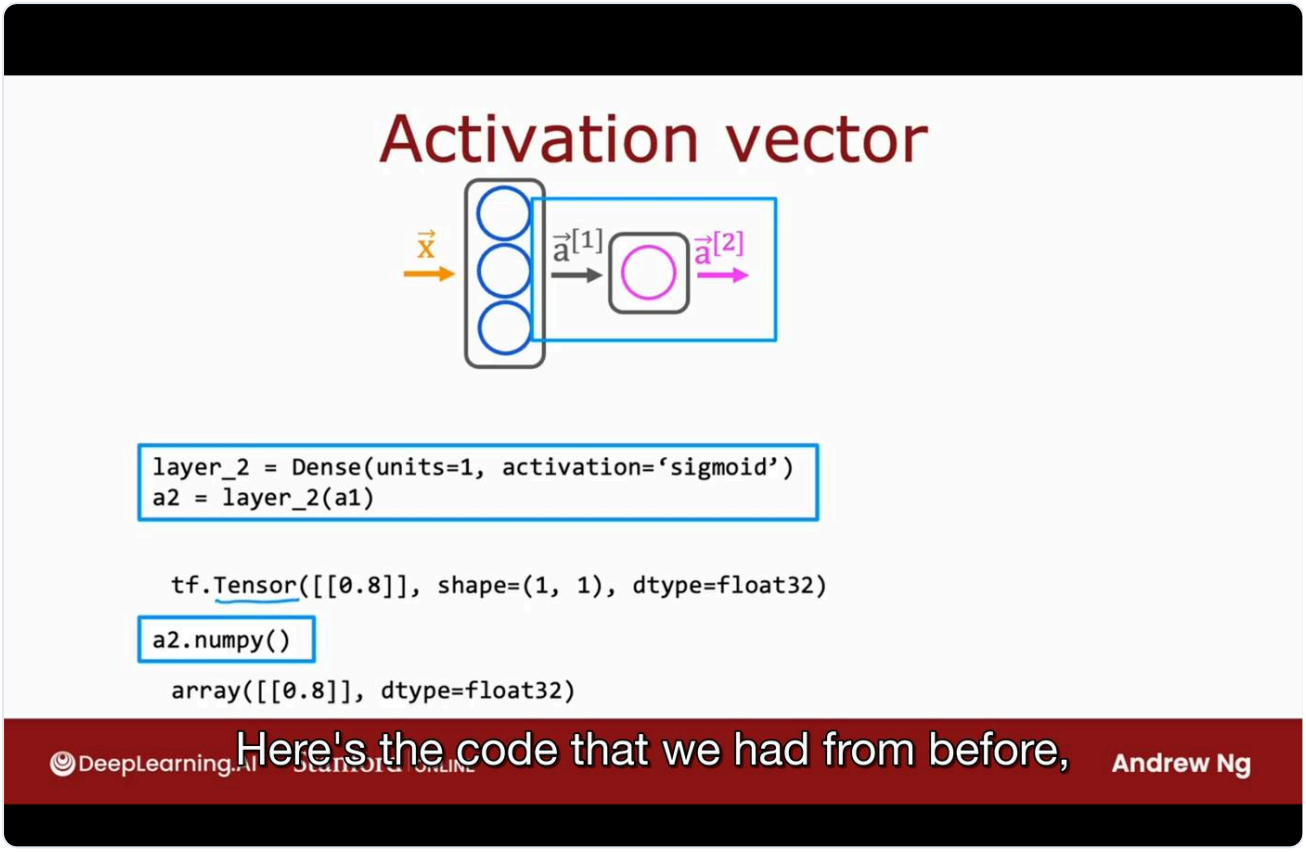


FIGURE 9.9 | Activation Vector

Here's the code that we had from before, layer 2 is a dense layer with one unit and sigmoid activation and a2 is computed by taking layer 2 and applying it to a1 so what is a2? A2, maybe a number like 0.8 and technically this is a 1 x 1 matrix is a 2D array with one row and one column and so it's equal to this number 0.8. And if you print out a2, you see that it is a TensorFlow tensor with just one element one number 0.8 and it is a 1 x 1 matrix. And again it is a float32, decimal points number taking up 32 bits in computer memory. Once again you can convert from a tensorflow tensor to a NumPy matrix using a2.numpy and that will turn this back into a NumPy array that looks like this.

So that hopefully gives you a sense of how data is represented in TensorFlow and in NumPy. I'm used to loading data and manipulating data in NumPy, but when you pass a NumPy array into TensorFlow, TensorFlow likes to convert it to its own internal format. The tensor and then operate efficiently using tensors. And when you read the data back out you can keep it as a tensor or convert it back to a NumPy array. I think it's a bit unfortunate that the history of how these library evolved has let us have to do this extra conversion work when actually the two libraries can work quite well together. But when you convert back and forth, whether you're using a NumPy array or a tensor, it's just something to be aware of when you're writing code.

Next let's take what we've learned and put it together to actually build a neural network. Let's go see that in the next video.

LECTURE 10

Building a Neural Network Architecture

So you've seen a bunch of TensorFlow code by now learned about how to build a layer in TensorFlow, how to do forward prop through a single layer in TensorFlow. And also learned about data in TensorFlow. Let's put it all together and talk about how to build a neural network in TensorFlow. This is also the last video on TensorFlow for this week. And in this video you also learn about a different way of building a neural network, that will be even a little bit simpler than what you've seen so far. So let's dive in what you saw previously was.

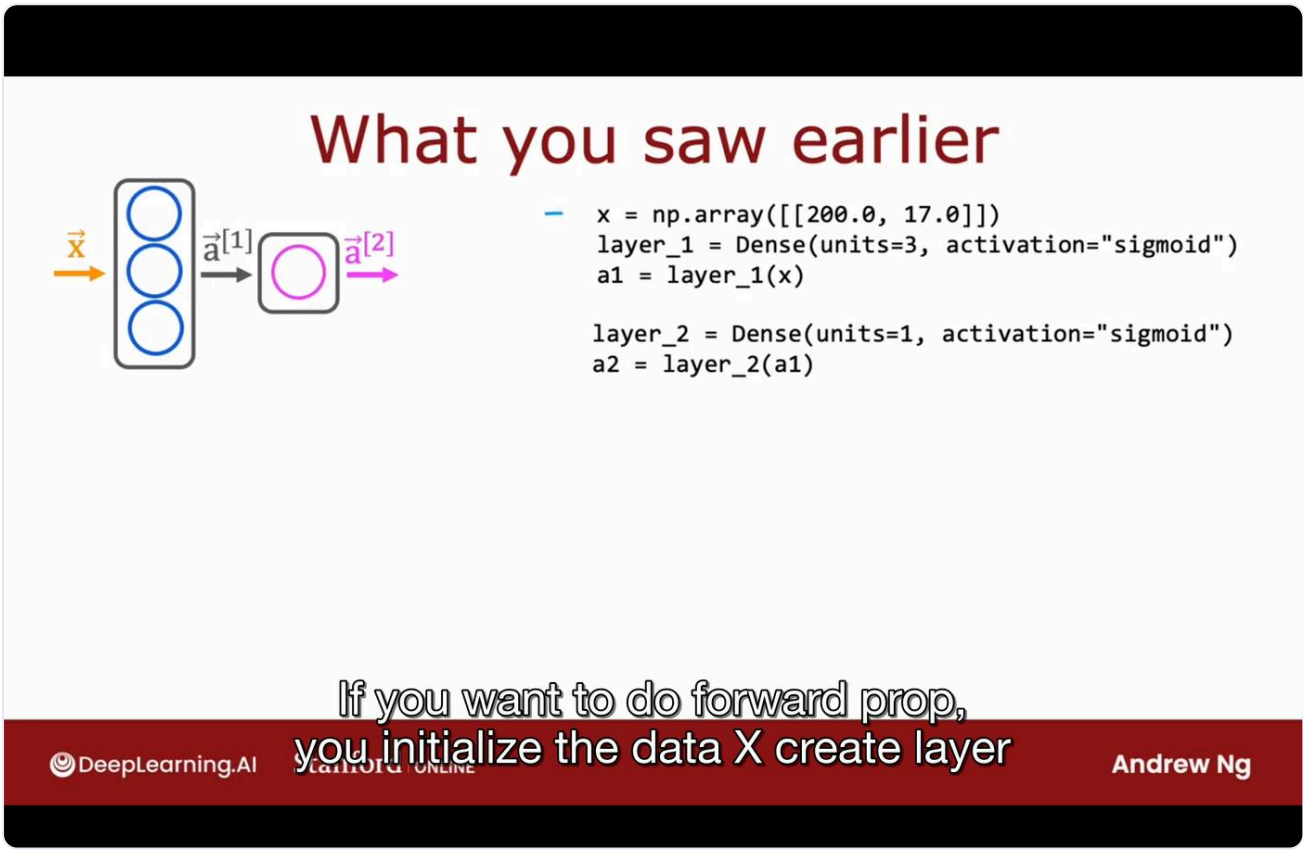


FIGURE 10.1 | What You Saw Earlier

If you want to do forward prop, you initialize the data X create layer one then compute a one, then create layer two and compute a two. So this was an explicit way of carrying out forward prop one layer of computation at the time. It turns out that TensorFlow has a different way of implementing forward prop as well as learning.

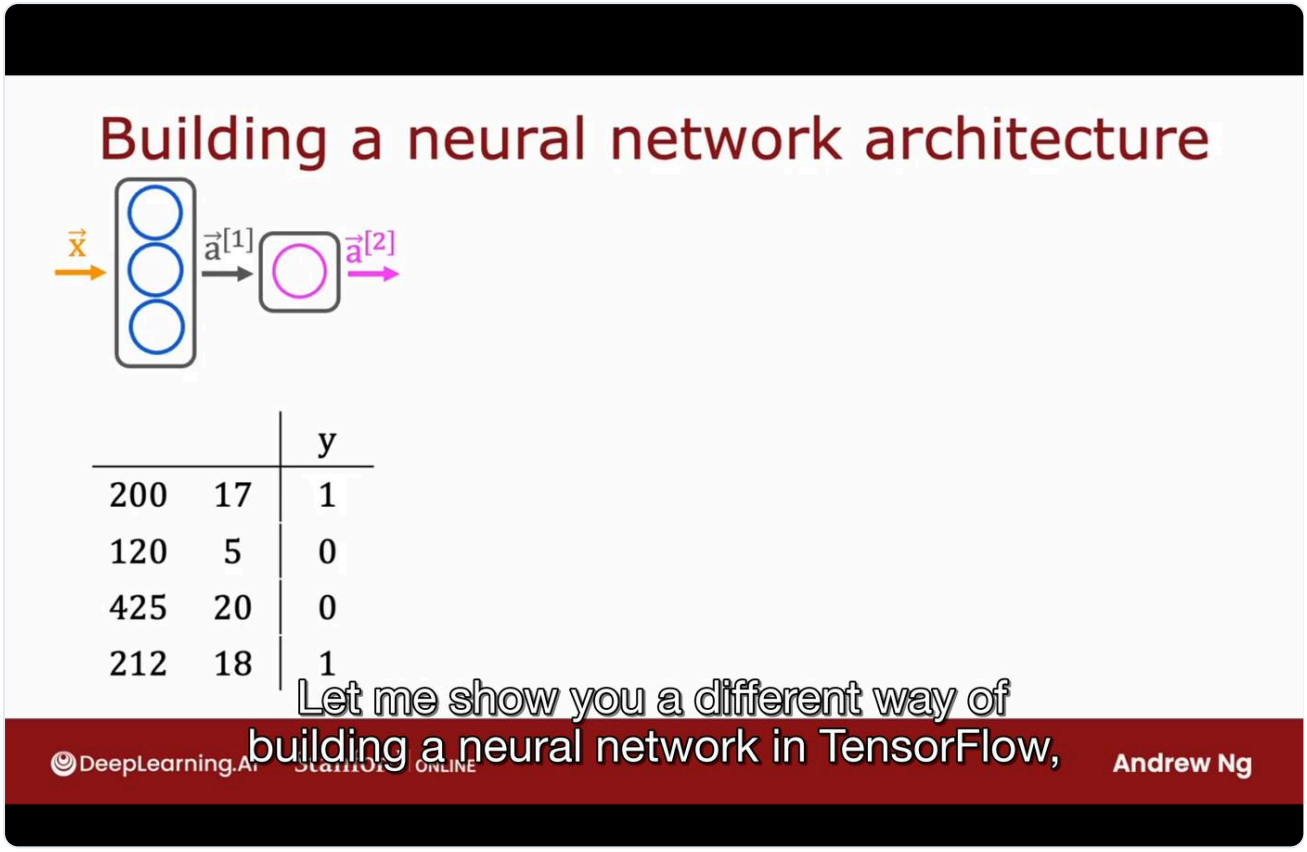


FIGURE 10.2

Let me show you a different way of building a neural network in TensorFlow, which is that same as before you're going to create layer one and create layer two. But now instead of you manually taking the data and passing it to layer one and then taking the activations from layer one and pass it to layer two. We can instead tell TensorFlow that we would like it to take layer one and layer two and string them together to form a neural network. That's what the sequential function in TensorFlow does which is it says, Dear TensorFlow, please create a neural network for me by sequentially string together these two layers that I just created. It turns out that with the sequential framework TensorFlow can do a lot of work for you.

Let's say you have a training set like this on the left.

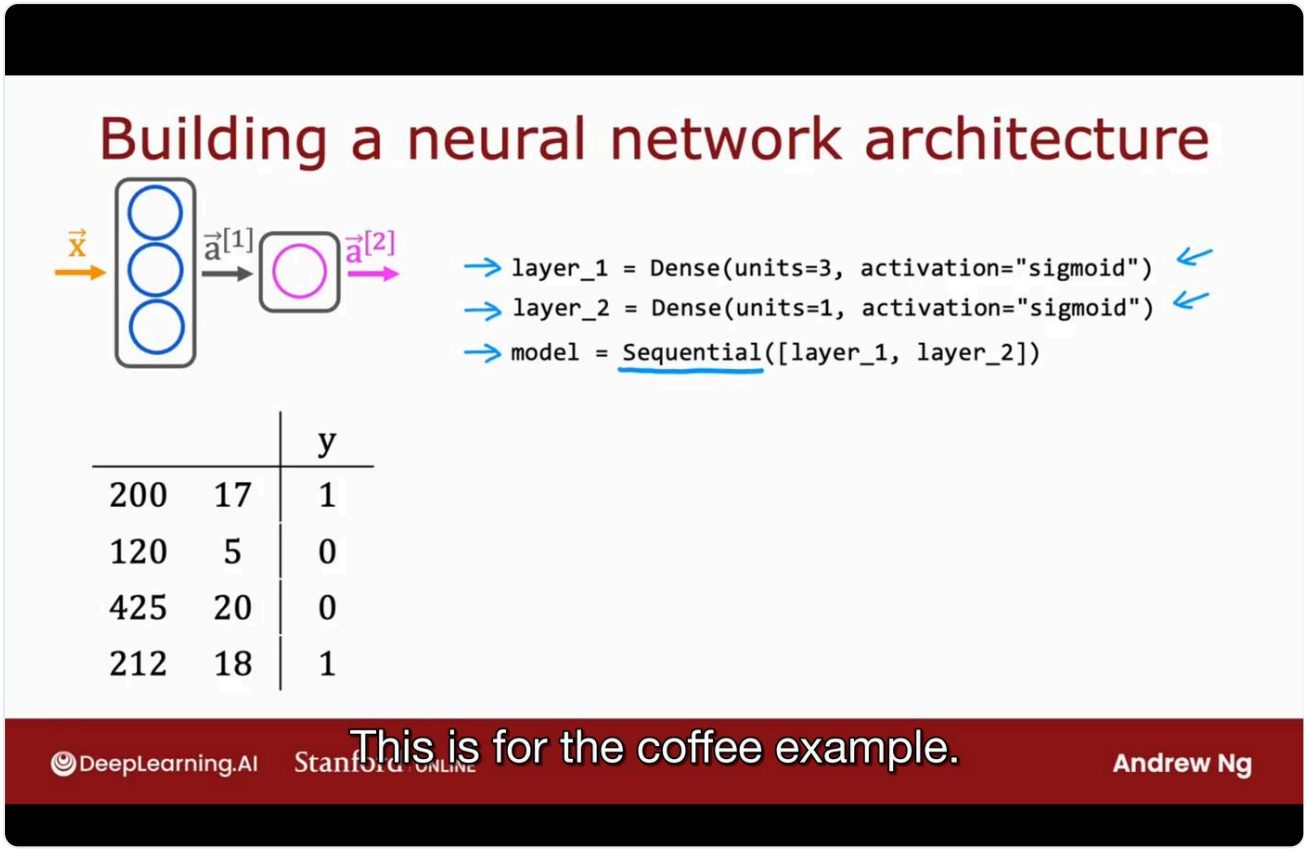


FIGURE 10.3 | Building a Neural Network Architecture

This is for the coffee example. You can then take the training data as inputs X and put them into a numpy array. This here is a four by two matrix and the target labels. Y can then be written as follows. And this is just a one dimensional array of length four Y this set of targets can then be stored as a 1-D array like this 1001 corresponding to four train examples. And it turns out that given the data, X and Y stored in this matrix X and this array, Y. If you want to train this neural network, all you need to do is call to functions you need to call model dot compile with some parameters.

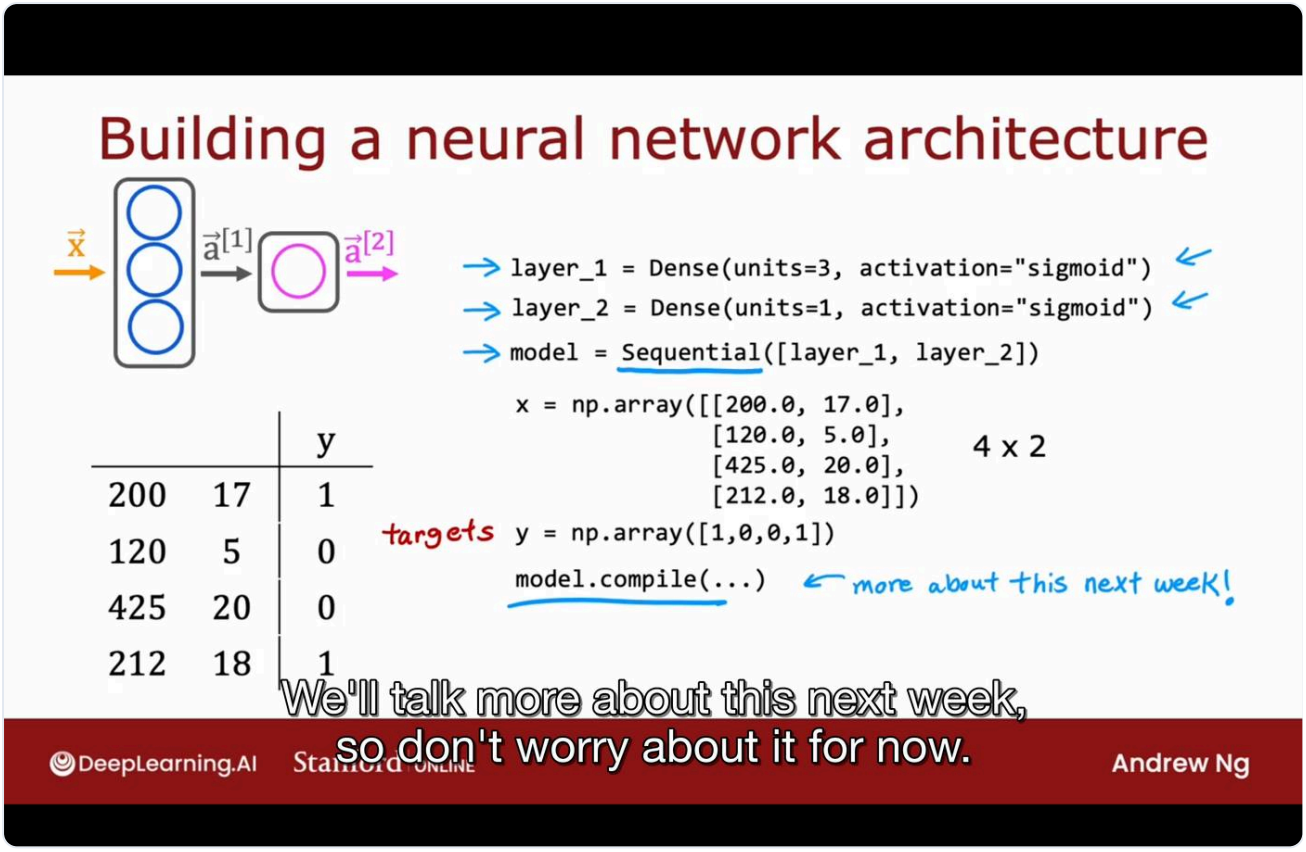


FIGURE 10.4 | Building a Neural Network Architecture

We'll talk more about this next week, so don't worry about it for now.

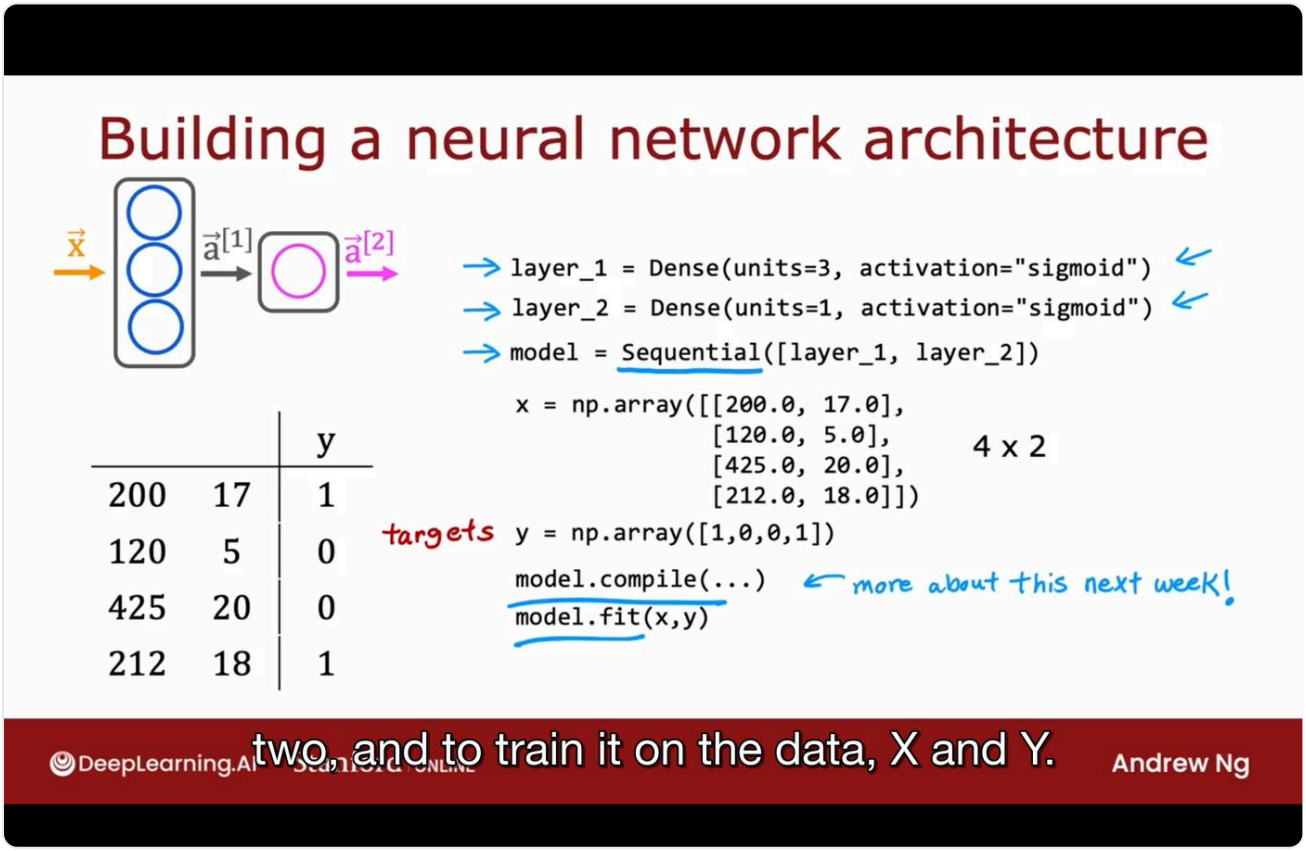


FIGURE 10.5 | Building a Neural Network Architecture

And then you need to call model dot fit X Y, which tells TensorFlow to take this neural network that are created by sequentially string together layers one and two, and to train it on the data, X and Y. But we'll learn how but we'll learn the details of how to do this next week and then finally how do you do inference on this neural network?

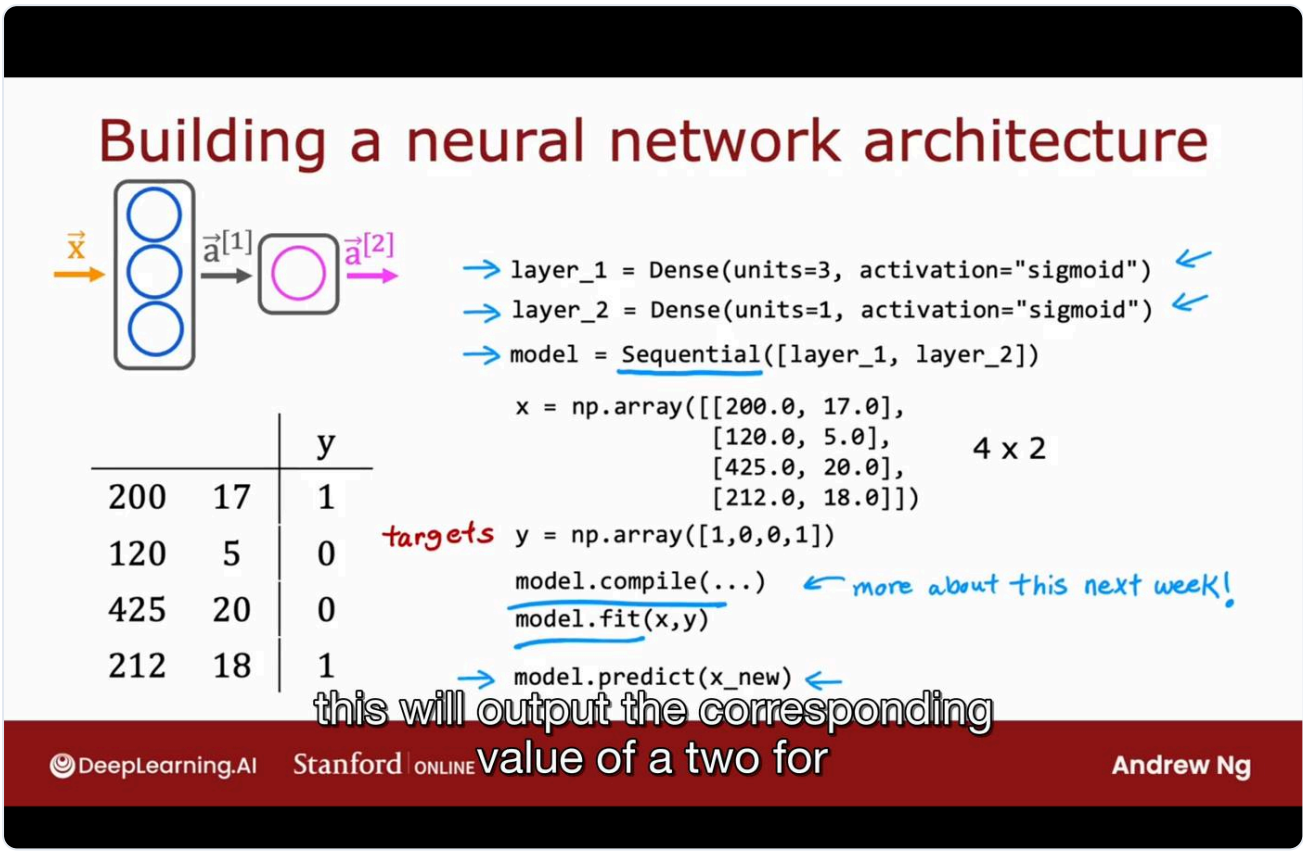


FIGURE 10.6 | Building a Neural Network Architecture

How do you do forward prop if you have a new example, say X new, which is NP array with these two features than to carry out forward prop instead of having to do it one layer at a time yourself, you just have to call model predict on X new and this will output the corresponding value of a two for you given this input value of X. So model predicts carries out forward propagation and carries an inference for you, using this neural network that you compiled using the sequential function. Now I want to take these three lines of code on top and just simplify it a little bit further, which is when coding in Tensorflow.

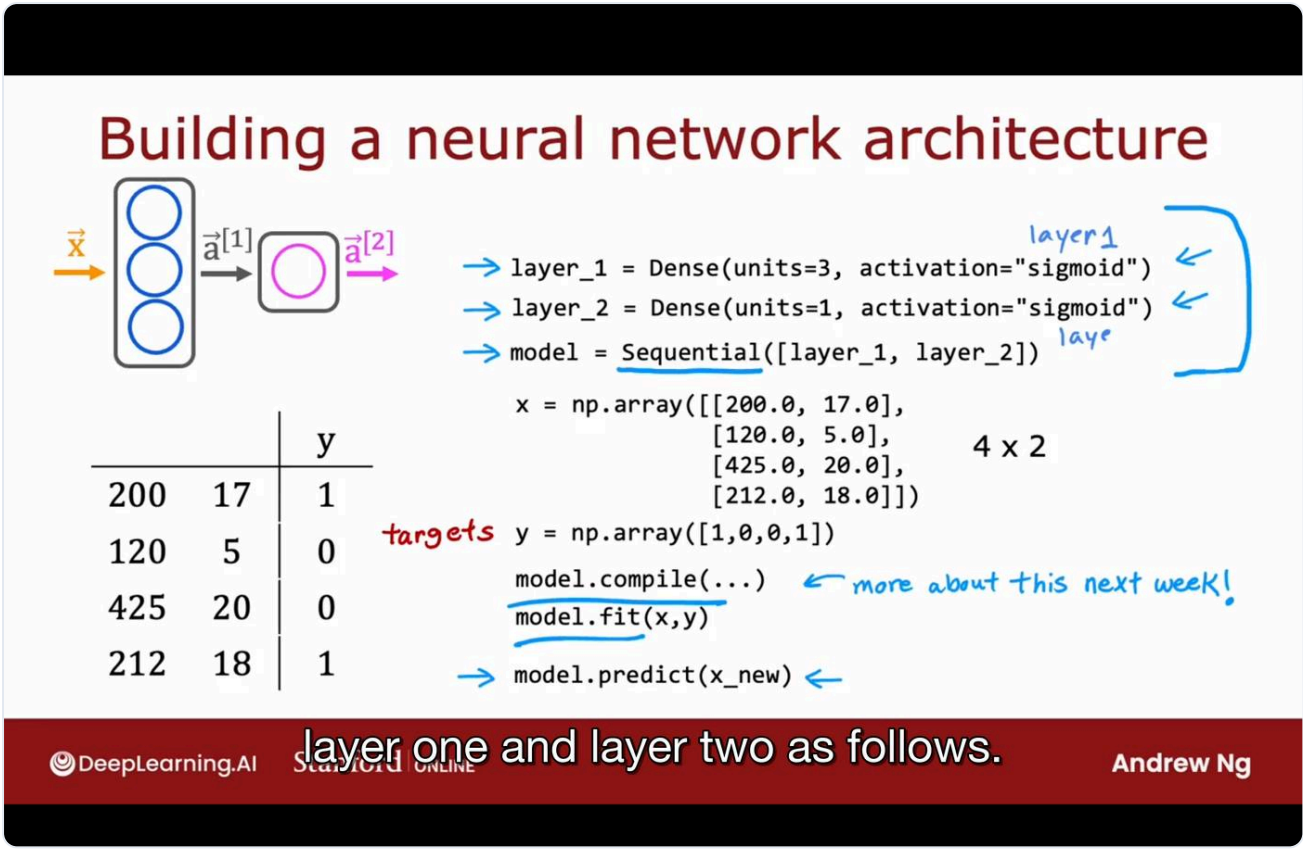


FIGURE 10.7 | Building a Neural Network Architecture

By convention we don't explicitly assign the two layers to two variables, layer one and layer two as follows. But by convention I would usually just write a code like this, when we say the model is a sequential model of a few layers strung together.

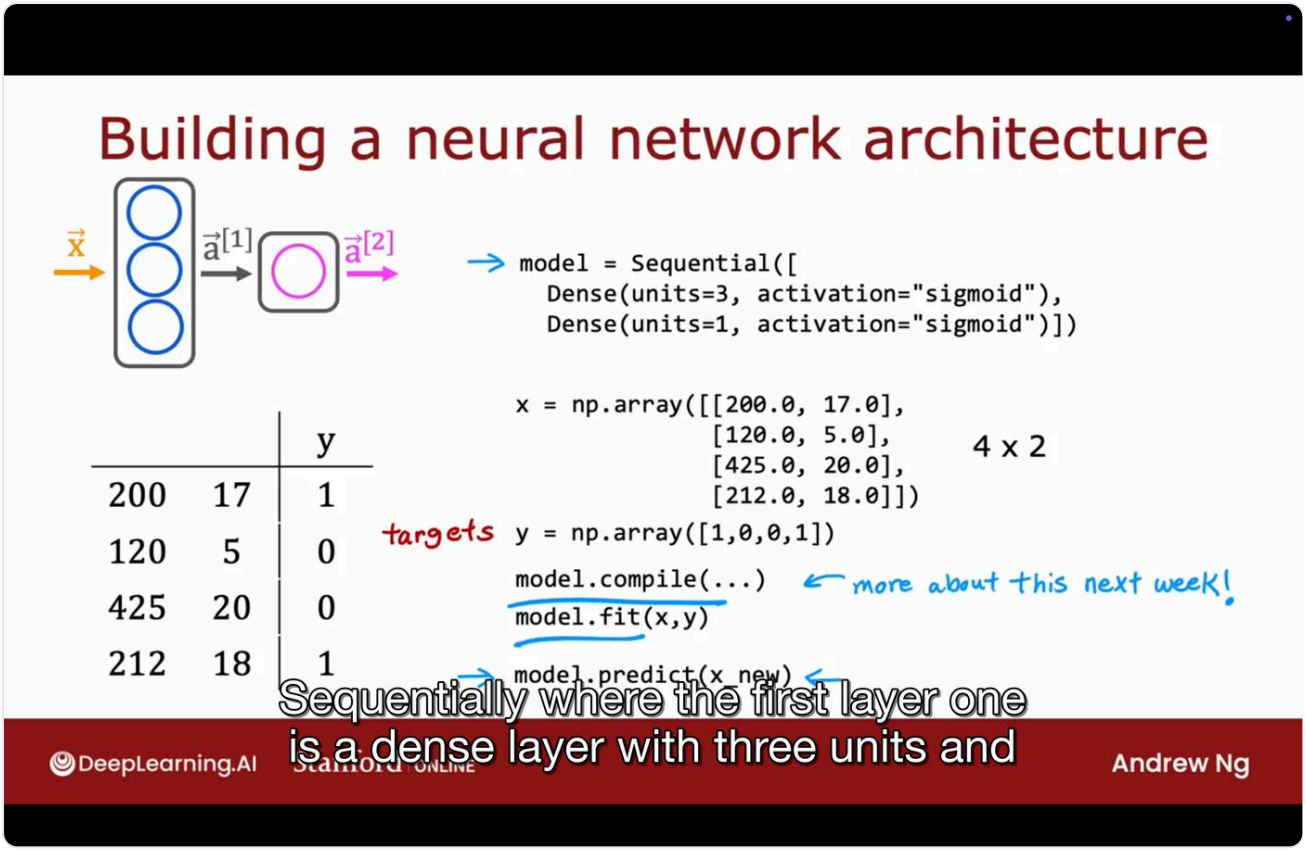


FIGURE 10.8 | Building a Neural Network Architecture

Sequentially where the first layer one is a dense layer with three units and activation of sigmoid and the second layer, is a dense layer with one unit and again a sigmoid activation function.

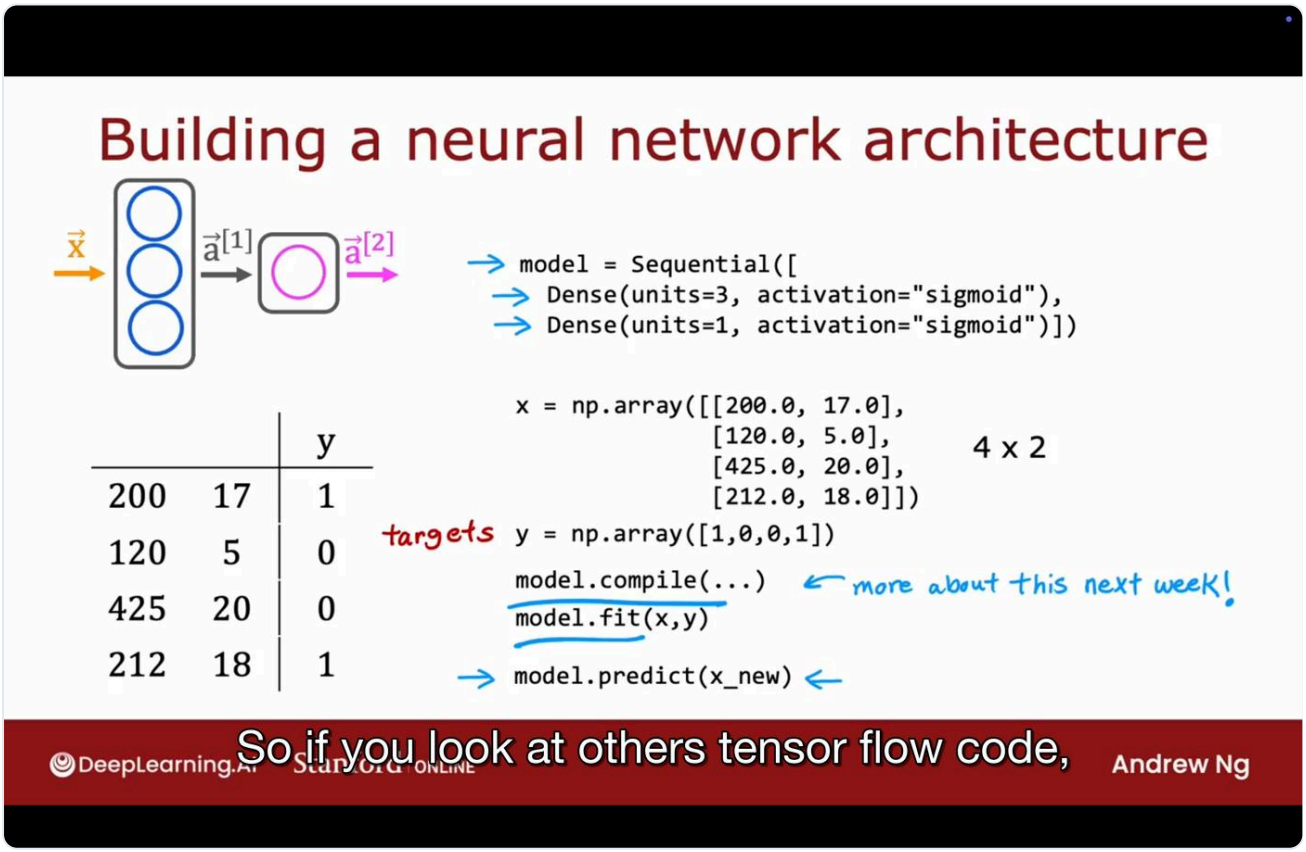


FIGURE 10.9 | Building a Neural Network Architecture

So if you look at others TensorFlow code, you often see it look more like this rather than having an explicit assignment to these layer one and layer two variables. And so that's it. This is pretty much the code you need in order to train as well as to inference on a neural network in TensorFlow. Where again we'll talk more about the training bits of this two combined the compiler and the fit function next week. Let's redo this for the digit classification example as well. So previously we had X, in this input layer one is a layer a one equals.

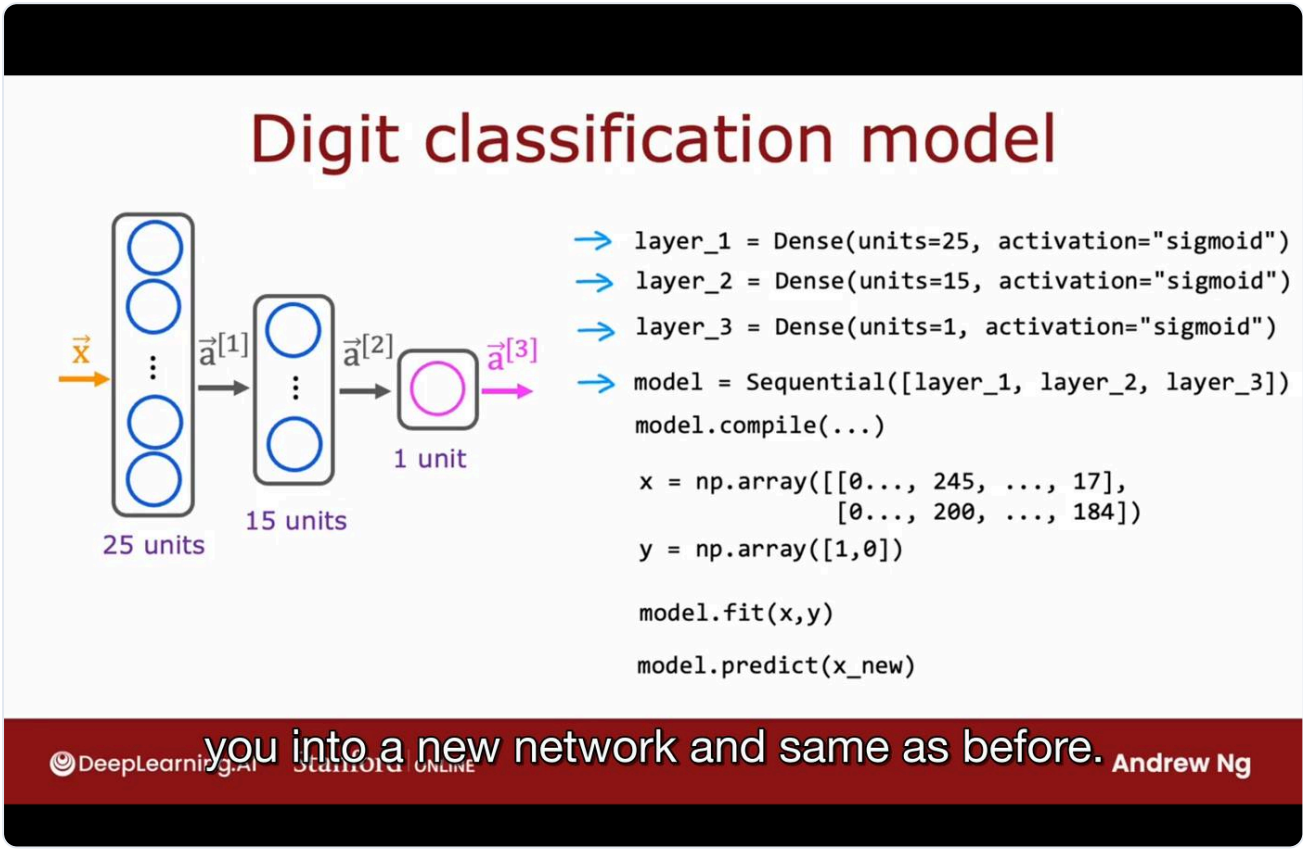


FIGURE 10.10 | Digit Classification Model

They want to apply to X and so on through layer two and layer three in order to try to classify a digit, with this new coding convention with using TensorFlow sequential function, you can instead specify what are layer one, layer two, layer three and tell TensorFlow to string the layers together for you into a new network and same as before. You can then store the data in the matrix and run the compile function and fit the model as follows. Again, more on this next week. Finally to do inference or to make predictions you can use model predict on X new and similar to what you saw before with the coffee classification network by convention, instead of assigning layer one, layer two, layer three, explicitly like this, we would more commonly just take these layers and put them directly into the sequential function.

So you end up with this more compact code which just tell TensorFlow, create a model for me that sequentially strings together these three layers and then the rest of the code works the same as before. So that's how you have built a neural network in TensorFlow. Now I know that when you're learning about these techniques, sometimes someone may ask you to implement these five lines of code and then you type five lines of code and then someone says congratulations with just five lines of code. You built this crazy complicated state of the art neural network and sometimes that makes you wonder, what exactly did I do with just these five lines of codes?

One thing I want you to take away from the machine learning specialization is the ability to use cutting edge libraries like TensorFlow to do your work efficiently. But I don't really want you to just call five lines of code and not really also know what the code is actually doing underneath the hood. So in the next video I'll let you go back and share with you how you can implement from scratch by yourself. forward propagation in python, so that you can understand the whole thing for yourself in practice. Most machine learning engineers don't actually implement forward propagation in python that often we just use libraries like tensorflow and pytorch, but because I want you to understand how these algorithms work yourself so that if something goes wrong, you can think through for yourself, what you might need to change, what's likely to work, what's less likely to work.

Let's also go through what it would take for you to implement for propagation from scratch because that way, even when you're calling a library and having it run efficiently and do great things in your application, I want you in the back of your mind to also have that deeper understanding of what your code is actually doing, so that let's go on to the next video.

LECTURE 11

Implement Forward Prop

if you had to implement forward propagation yourself from scratch in python, how would you go about doing so, in addition to gaining intuition about what's really going on in libraries like TensorFlow and PyTorch. If ever some day you decide you want to build something even better than TensorFlow and PyTorch, maybe now you have a better idea home, I don't really recommend doing this for most people. But maybe someday, someone will come up with an even better framework than TensorFlow and PyTorch and whoever does that may end up having to implement these things from scratch themselves. So let's take a look, on this slide I'm going to go through quite a bit of code and you see all this code again later in the optional lab as was in the practice lab.

So don't worry about having to take notes on every line of code or memorize every line of code. You see this code written down in the Jupyter notebook in the lab and the goal of this video is to just show you the code to make sure you can understand what it's doing. So that when you go to the optional lab and the practice lab and see the code there, you know what to do so don't worry about taking detailed notes on every line. If you can read through the code on this slide and understand what it's doing, that's all you need. So let's take a look at how you implement forward prop in a single layer, we're going to continue using the coffee roasting model shown here.

And let's look at how you would take an input feature vector x, and implement forward prop to get this output a2. In this python implementation, I'm going to use 1D arrays to represent all of these vectors and parameters, which is why there's only a single square bracket here. This is a 1D array in python rather than a 2D matrix, which is what we had when we had double square brackets. So the first value you need to compute is, a super strip square bracket 1 subscript 1, which is the first activation value of a1 and that's g of this expression over here.

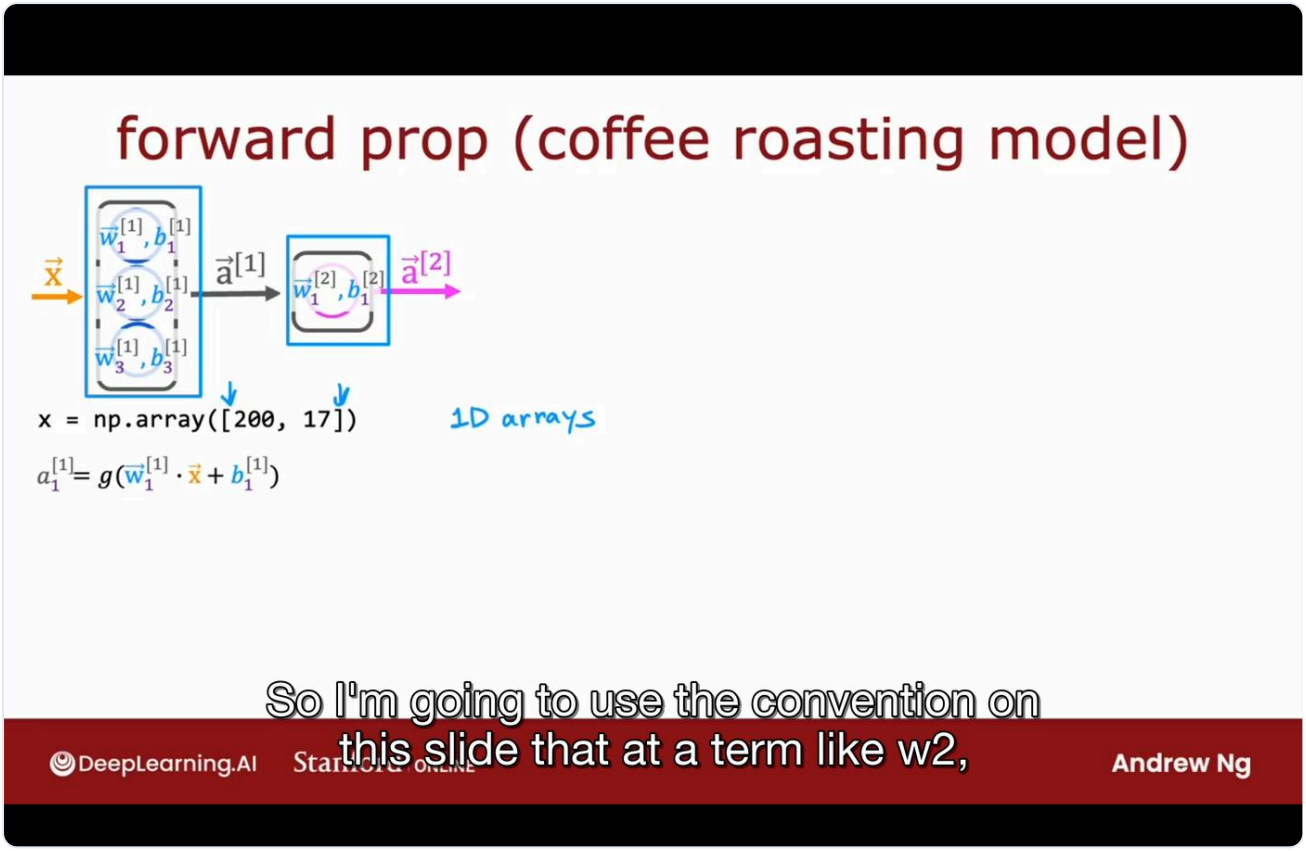


FIGURE 11.1

So I'm going to use the convention on this slide that at a term like w2, 1, I'm going to represent as a variable w2 and then subscript 1. This underscore one denotes subscript one, denotes subscript one so w2 means w superscript 2 in square brackets and then subscript 1.

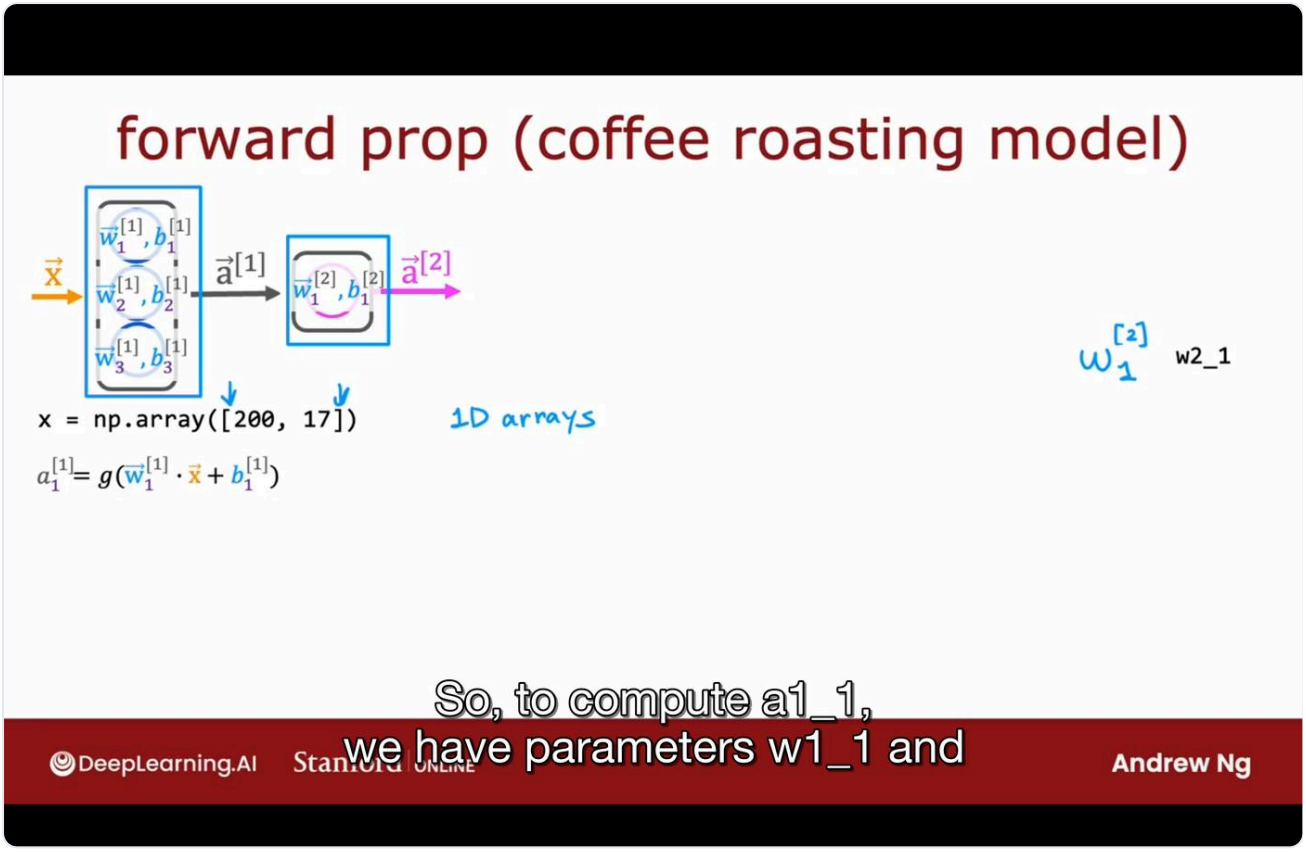


FIGURE 11.2

So, to compute a1_1, we have parameters w1_1 and b1_1, which are say 1_2 and -1. You would then compute z1_1 as the dot product between that parameter w1_1 and the input x, and added to b1_1 and then finally a1_1 is equal to g, the sigmoid function applied to z1_1.

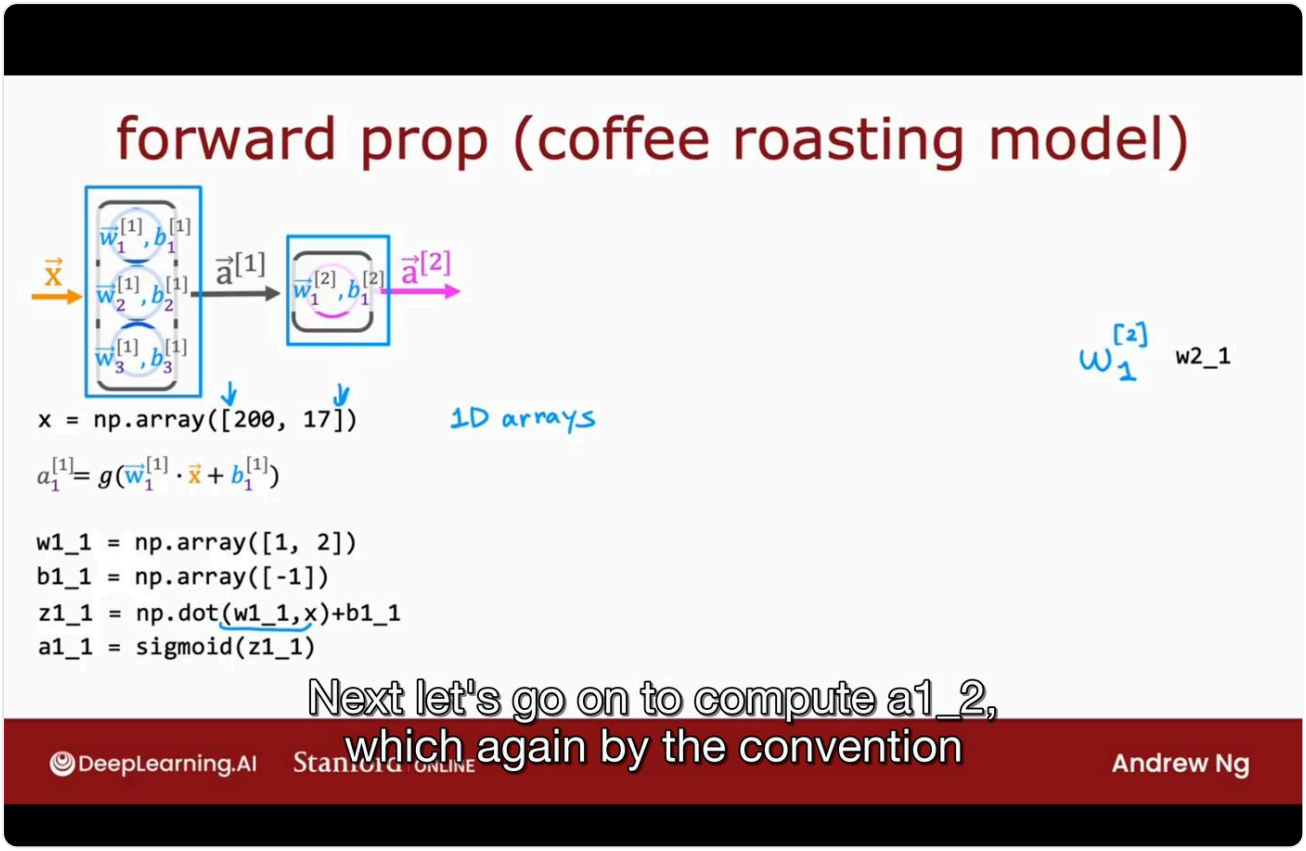


FIGURE 11.3

Next let's go on to compute a1_2, which again by the convention I described here is going to be a1_2, written like that.

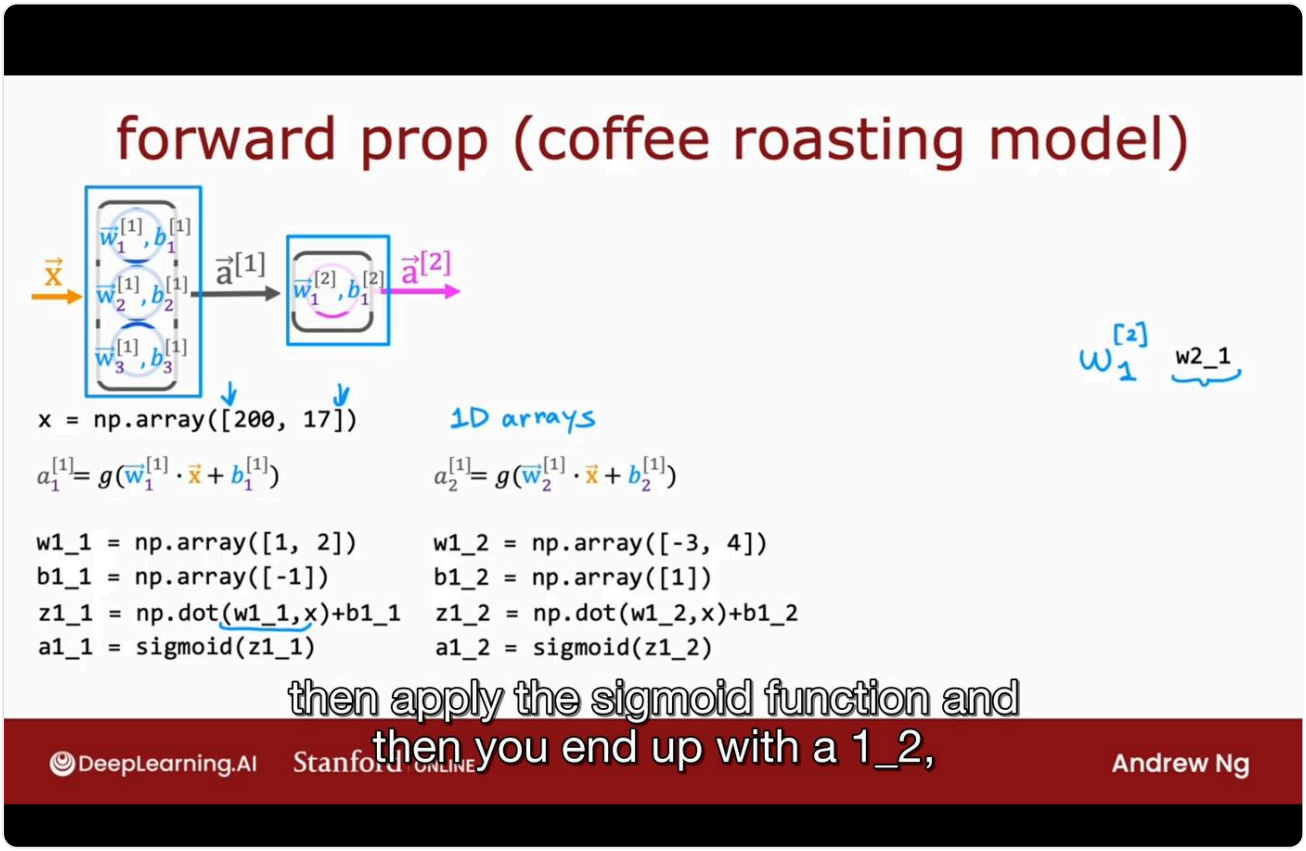


FIGURE 11.4

So similar as what we did on the left, w1_2 is two parameters -3, 4, b1_2 is the term, b 1, 2 over there, so you compute z as this term in the middle and then apply the sigmoid function and then you end up with a 1_2, and finally you do the same thing to compute a1_3.

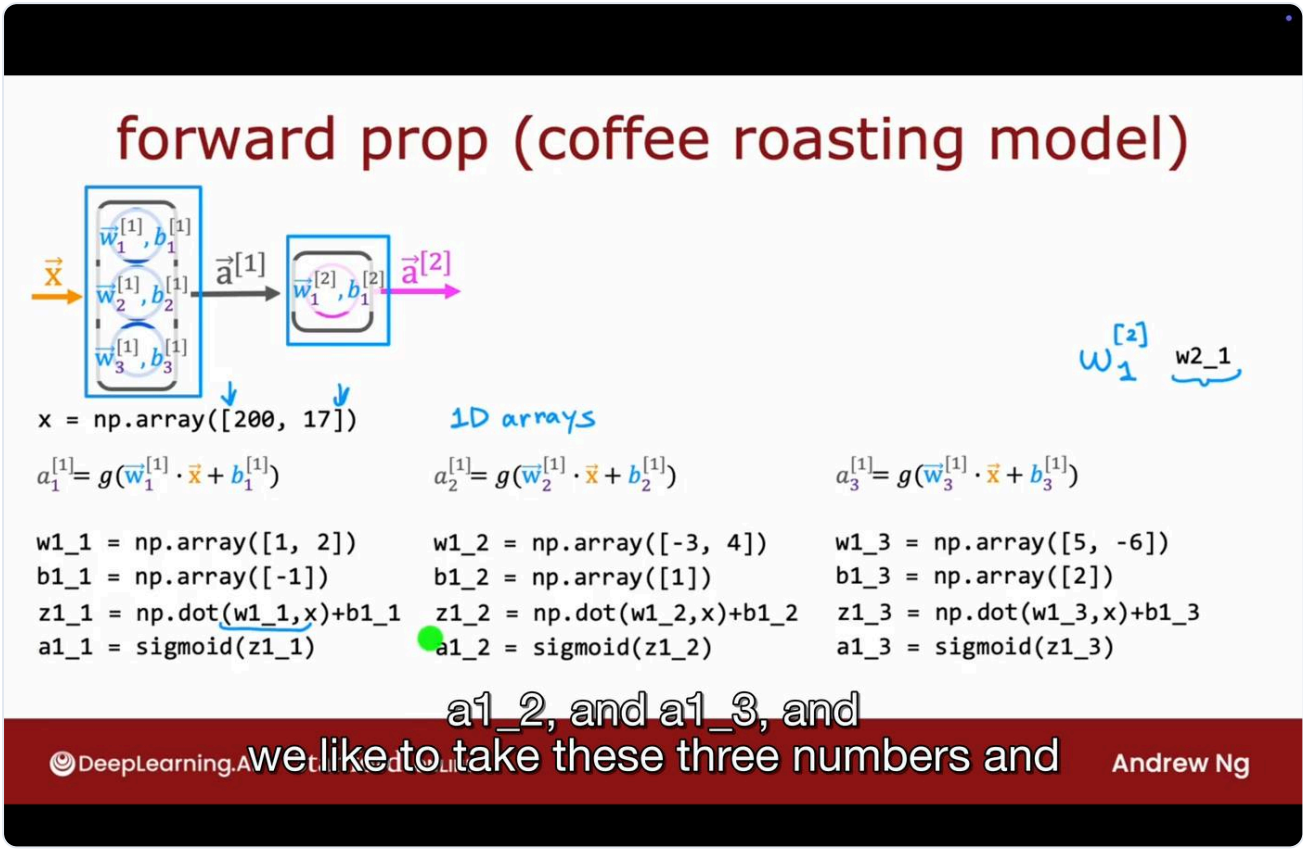


FIGURE 11.5

Now, you've computed these three values, a1_1, a1_2, and a1_3, and we like to take these three numbers and group them together into an array to give you a1 up here, which is the output of the first layer. And so you do that by grouping them together using a np array as follows, so now you've computed a_1, let's implement the second layer as well.

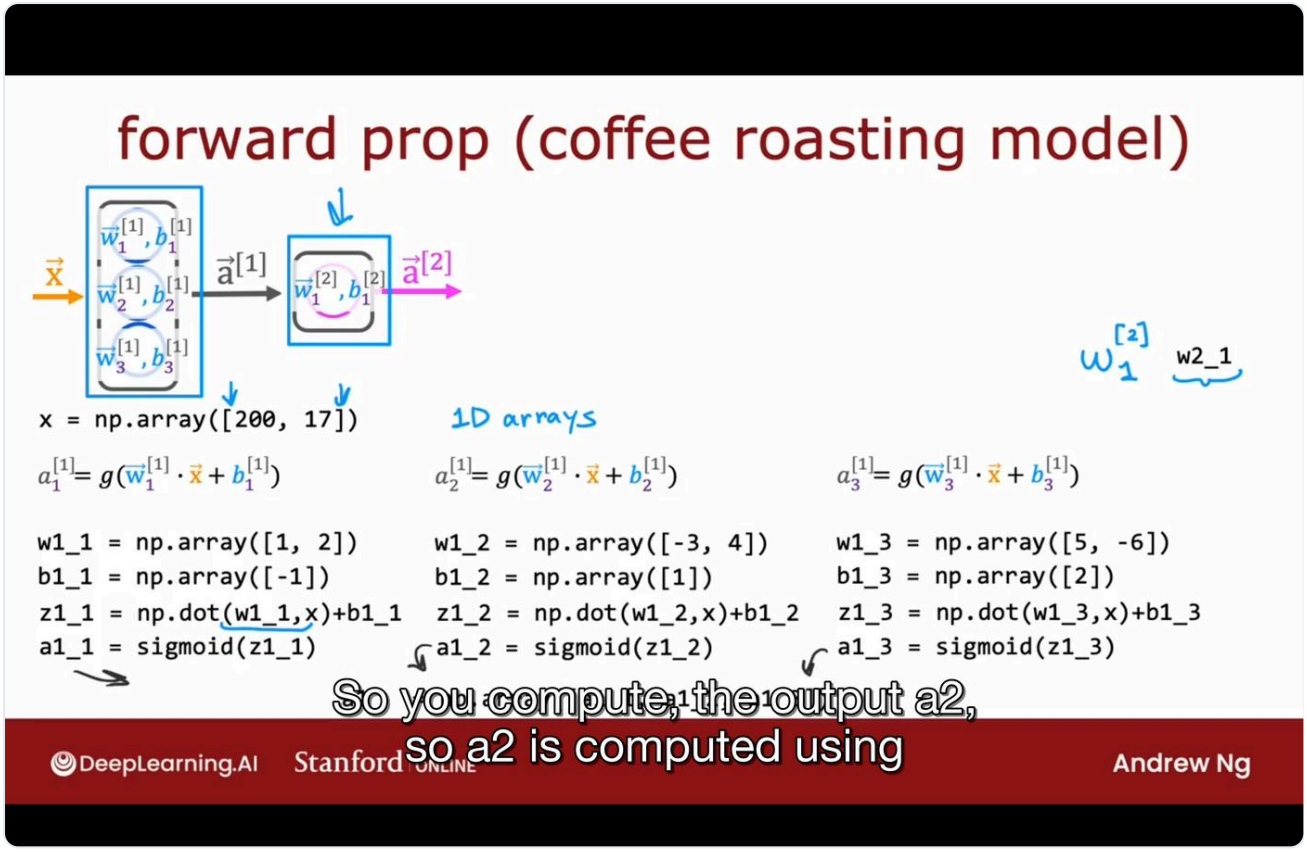


FIGURE 11.6

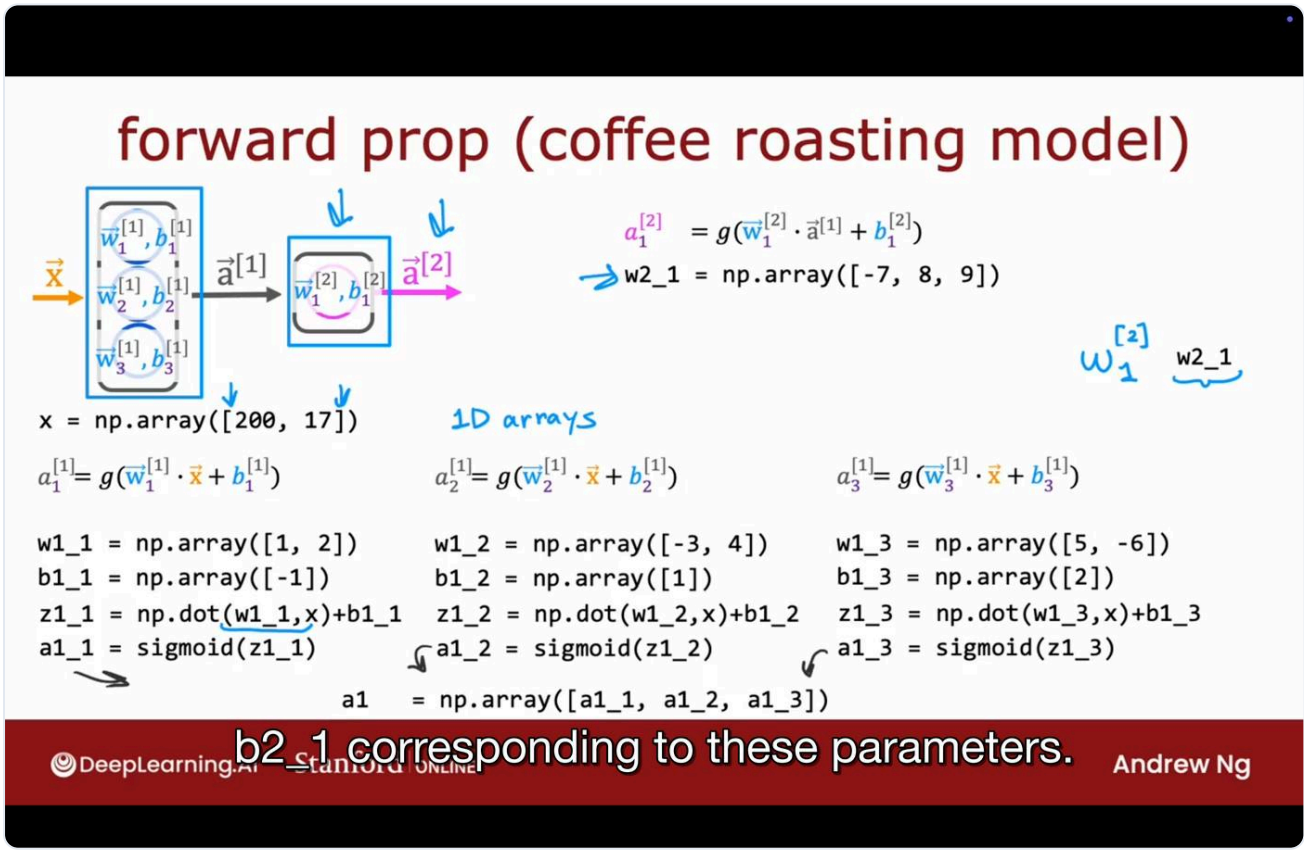


FIGURE 11.7

So you compute, the output a2, so a2 is computed using this expression and so we would have parameters w2_1 and b2_1 corresponding to these parameters.

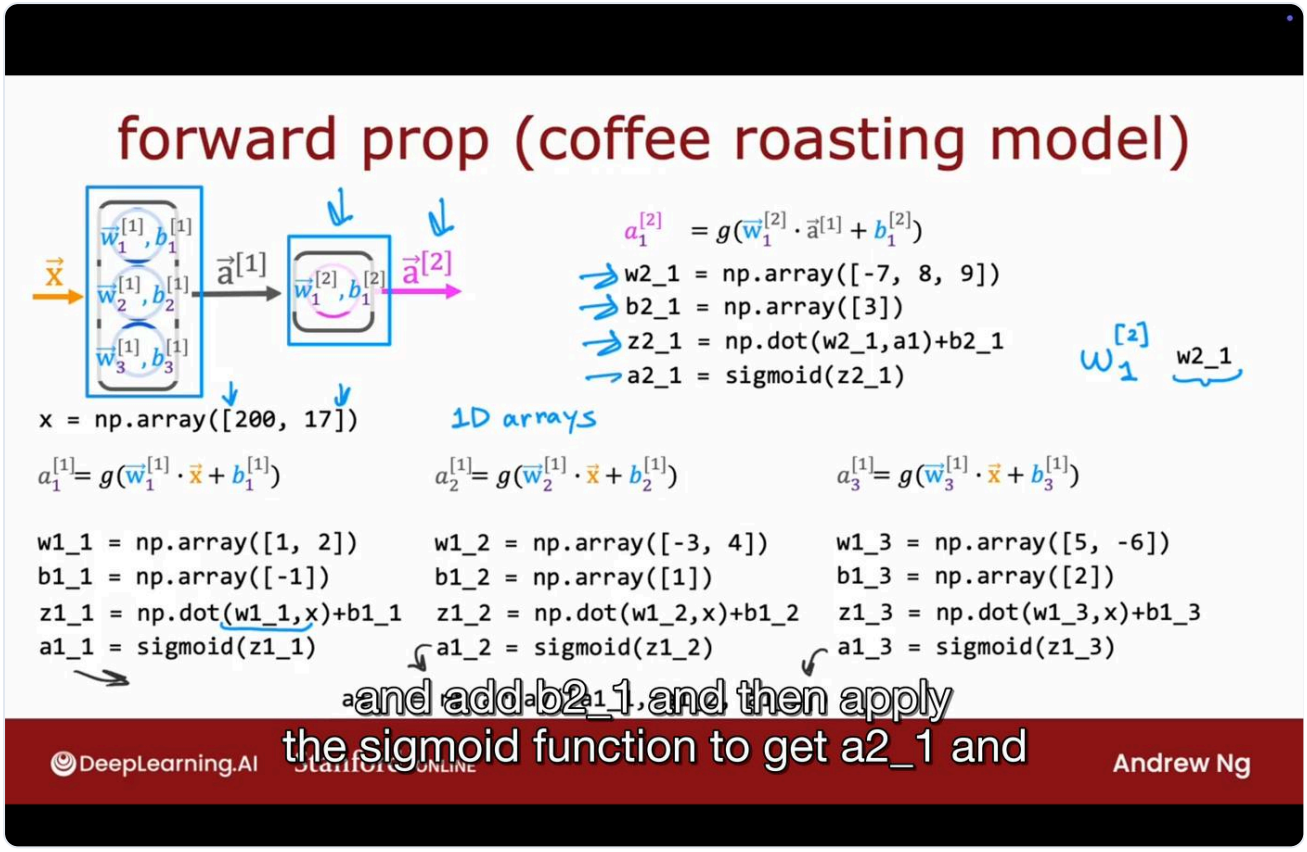


FIGURE 11.8

And then you would compute z as the dot product between w2_1 and a1, and add b2_1 and then apply the sigmoid function to get a2_1 and that's it, that's how you implement forward prop using just python and np. Now, there are a lot of expressions in this page of code that you just saw, let's in the next video look at how you can simplify this to implement forward prop for a more general neural network, rather than hard coding it for every single neuron like we just did. So let's go see that in the next video.

LECTURE 12

Forward Prop in NumPy

In the last video, you saw how to implement forward prop in Python, but by hard coding lines of code for every single neuron. Let's now take a look at the more general implementation of forward prop in Python. Similar to the previous video, my goal in this video is to show you the code so that when you see it again in their practice lab, in the optional labs, you know how to interpret it. As we walk through this example, don't worry about taking notes on every single line of code. If you can read through the code and understand it, that's definitely enough. What you can do is write a function to implement a dense layer, that is a single layer of a neural network.

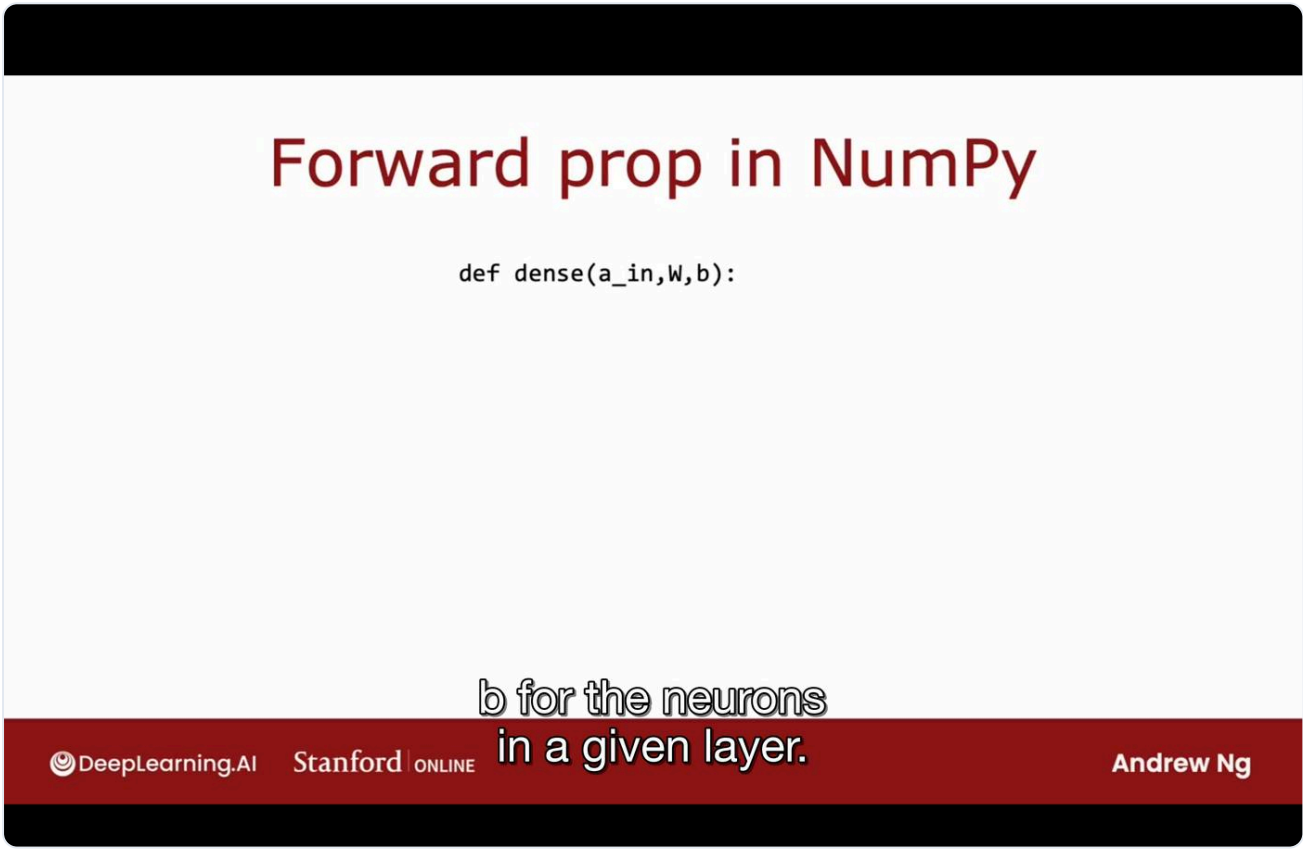


FIGURE 12.1 | Forward Prop in NumPy

I'm going to define the dense function, which takes as input the activation from the previous layer, as well as the parameters w and b for the neurons in a given layer. Using the example from the previous video, if layer 1 has three neurons, and if w_1 and w_2 and w_3 are these, then what we'll do is stack all of these wave vectors into a matrix. This is going to be a two by three matrix, where the first column is the parameter w_1,1 the second column is the parameter w_1, 2, and the third column is the parameter w_1,3. Then in a similar way, if you have parameters be, b_1,1 equals negative one, b_1,2 equals one, and so on, then we're going to stack these three numbers into a 1D array b as follows, negative one, one, two.

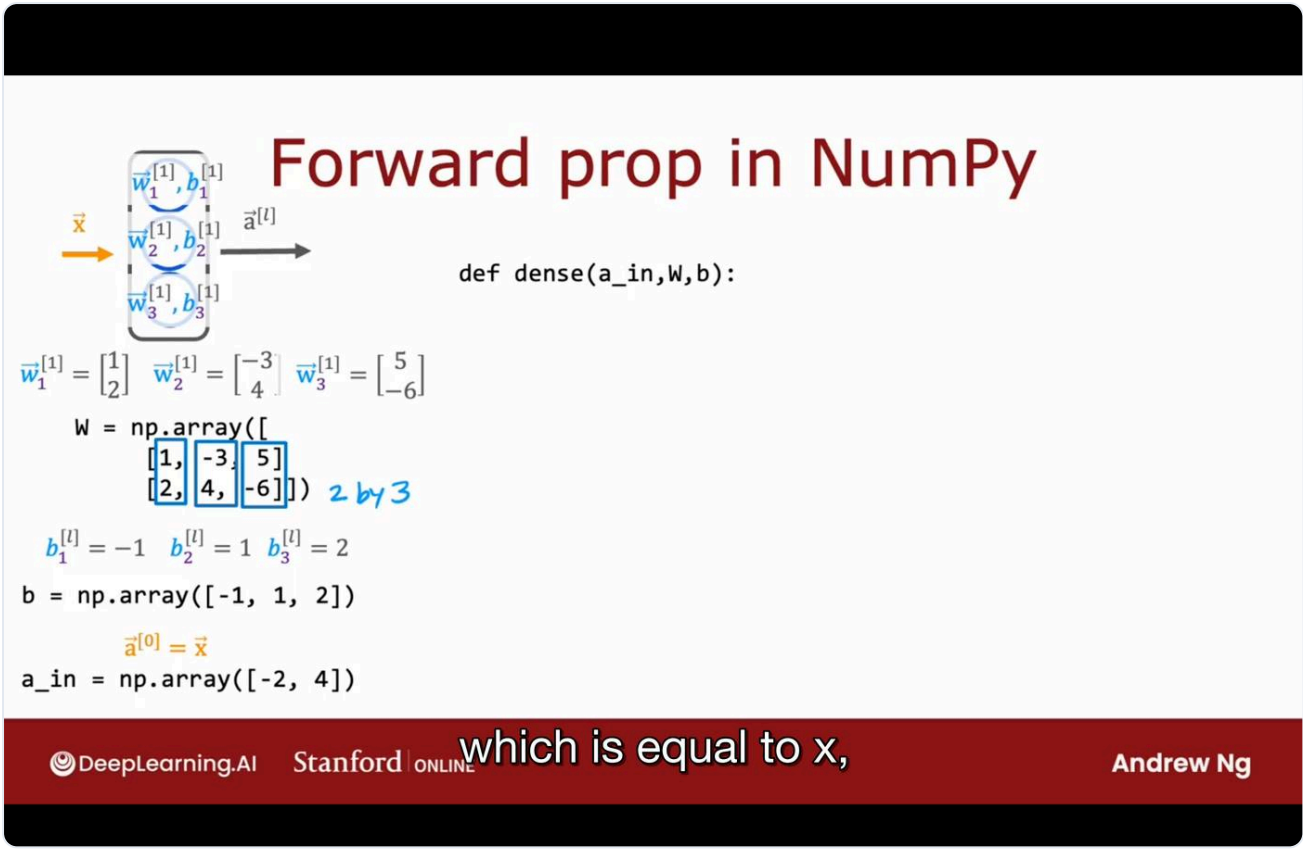


FIGURE 12.2

What the dense function will do is take as inputs the activation from the previous layer, and a here could be a_0, which is equal to x, or the activation from a later layer, as well as the w parameters stacked in columns, like shown on the right, as well as the b parameters also stacked into a 1D array, like shown to the left over there. What this function would do is input a to activation from the previous layer and will output the activations from the current layer. Let's step through the code for doing this. Here's the code. First, units equals W.shape,1. W here is a two-by-three matrix, and so the number of columns is three. That's equal to the number of units in this layer.

Here, units would be equal to three. Looking at the shape of w, is just a way of pulling out the number of hidden units or the number of units in this layer. Next, we set a to be an array of zeros with as many elements as there are units. In this example, we need to output three activation values, so this just initializes a to be zero, zero, zero, an array of three zeros. Next, we go through a for loop to compute the first, second, and third elements of a. For j in range units, so j goes from zero to units minus one. It goes from 0, 1, 2 indexing from zero and Python as usual.

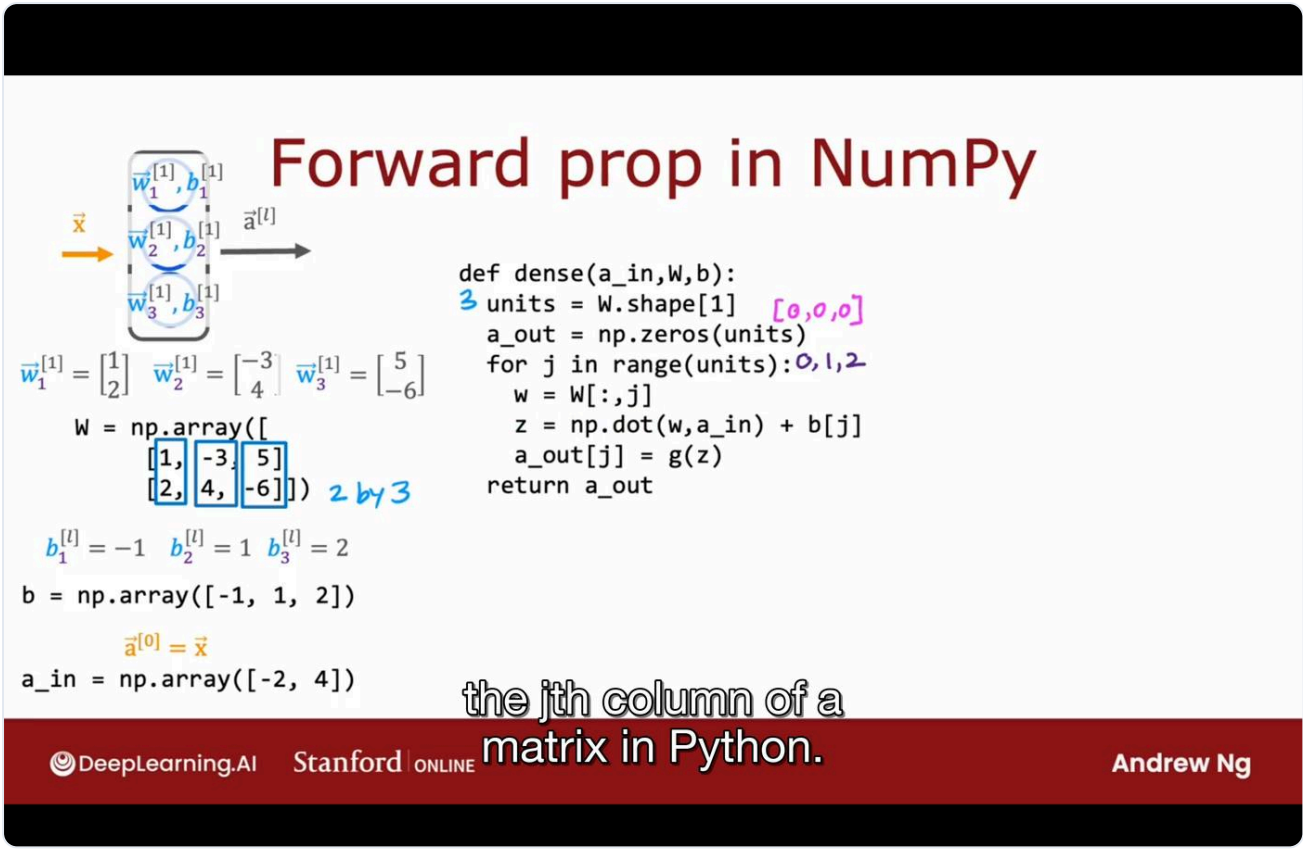


FIGURE 12.3

This command w equals W colon comma j, this is how you pull out the jth column of a matrix in Python. The first time through this loop, this will pull the first column of w, and so will pull out w_1,1. The second time through this loop, when you're computing the activation of the second unit, will pull out the second column corresponding to w_1, 2, and so on for the third time through this loop. Then you compute z using the usual formula, is a dot product between that parameter w and the activation that you have received, plus b, j. And then you compute the activation a, j, equals g sigmoid function applied to z.

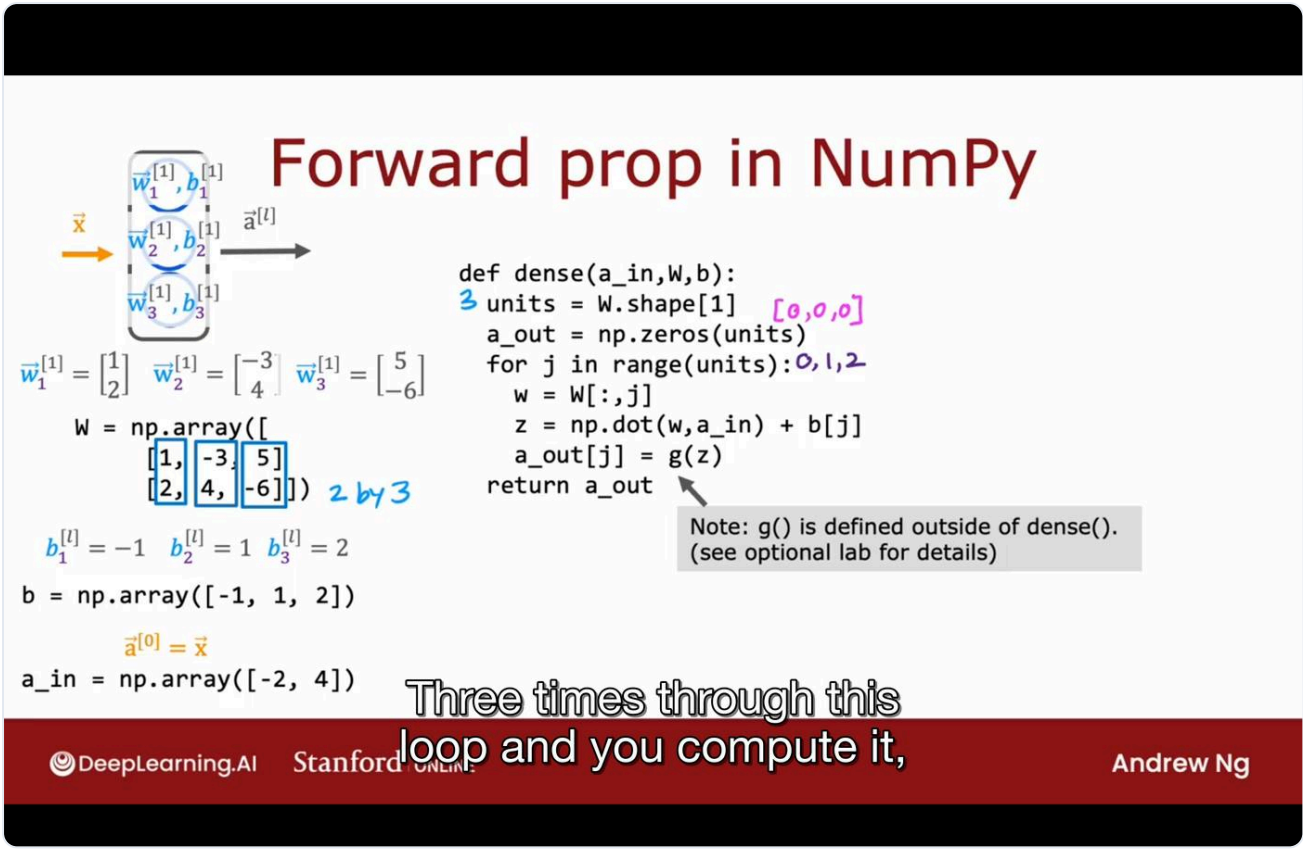


FIGURE 12.4

Three times through this loop and you compute it, the values for all three values of this vector of activation is a. Then finally you return a. What the dense function does is it inputs the activations from the previous layer, and given the parameters for the current layer, it returns the activations for the next layer.

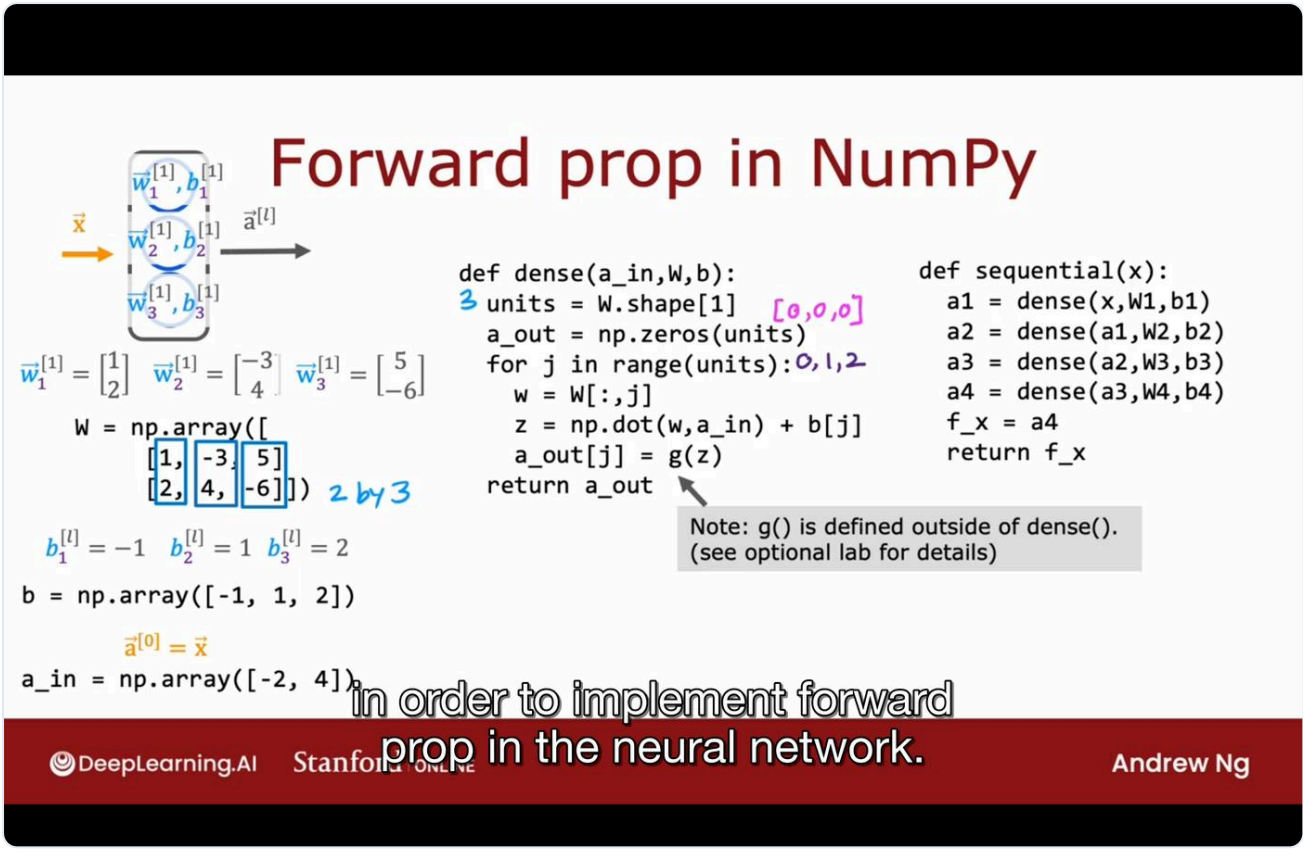


FIGURE 12.5

Given the dense function, here's how you can string together a few dense layers sequentially, in order to implement forward prop in the neural network. Given the input features x, you can then compute the activations a_1 to be a_1 equals dense of x, w_1, b_1, where here w_1, b_1 are the parameters, sometimes also called the weights of the first hidden layer. Then you can compute a_2 as dense of now a_1, which you just computed above. W_2, b-2 which are the parameters or weights of this second hidden layer. Then compute a_3 and a_4.

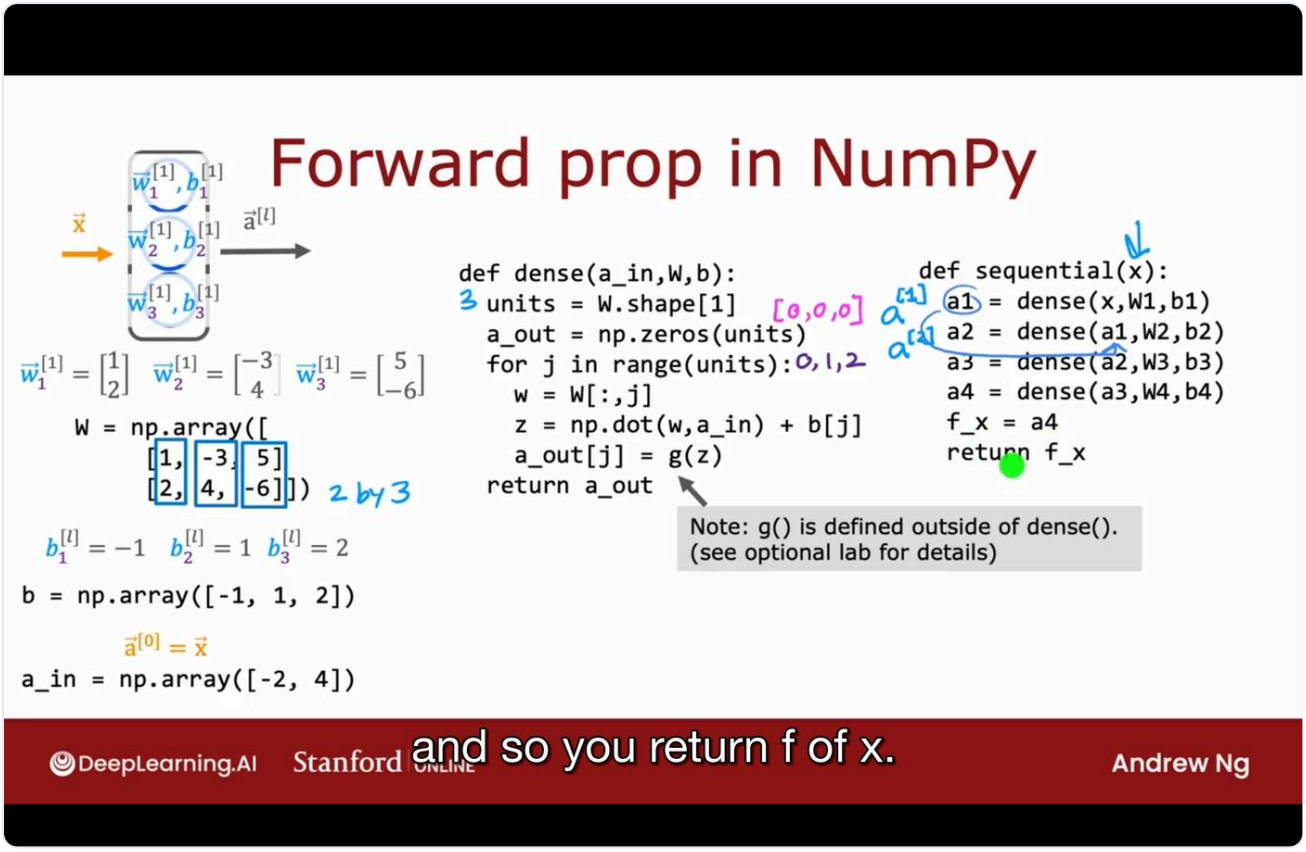


FIGURE 12.6

If this is a neural network with four layers, then define the output f of x is just equal to a_4, and so you return f of x.

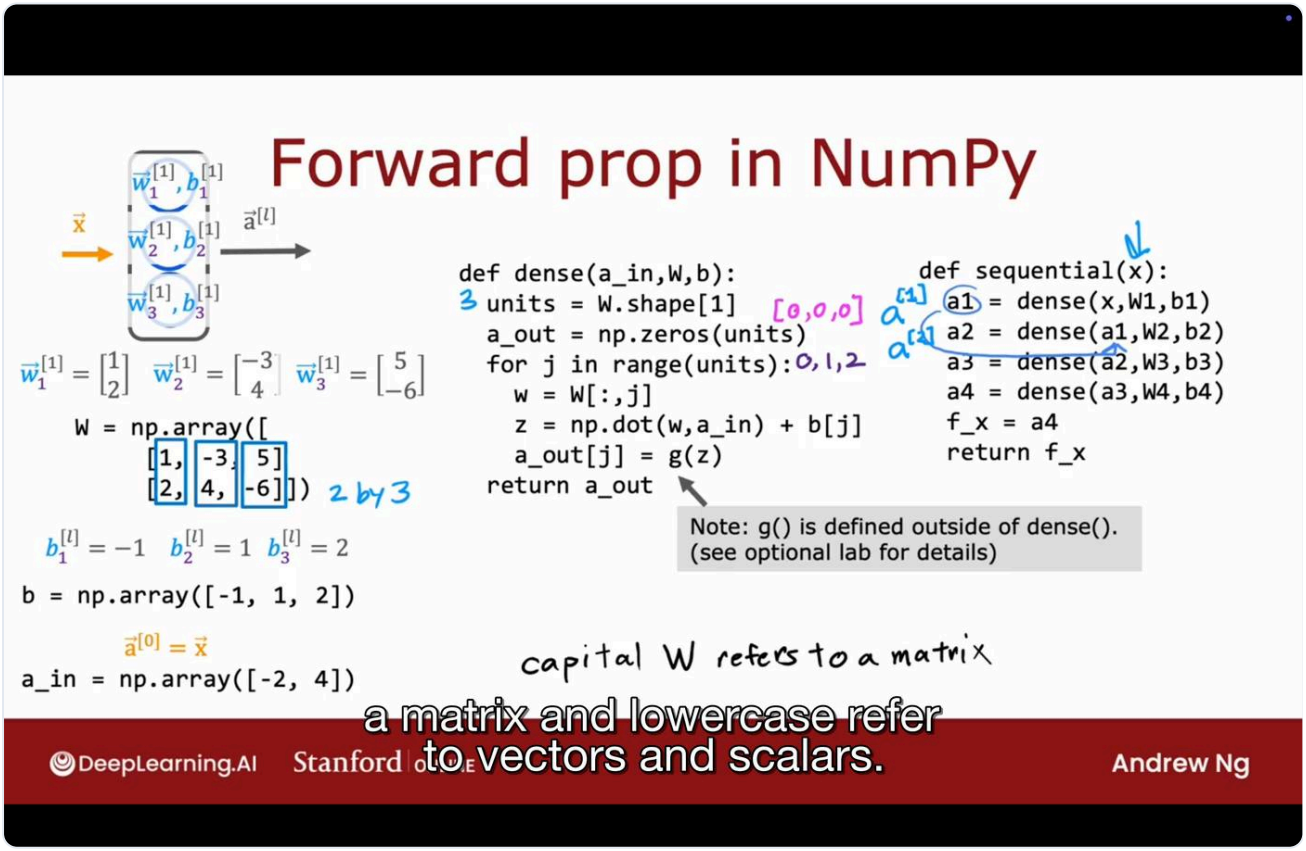


FIGURE 12.7

Notice that here I'm using W, because under the notational conventions from linear algebra is to use uppercase or a capital alphabet is when it's referring to a matrix and lowercase refer to vectors and scalars. So because it's a matrix, this is W. That's it. You now know how to implement forward prop yourself from scratch. You get to see all this code and run it and practice it yourself in the practice lab coming off to this as well. I think that even when you're using powerful libraries like TensorFlow, it's helpful to know how it works under the hood. Because in case something goes wrong, in case something runs really slowly, or you have a strange result, or it looks like there's a bug, your ability to understand what's actually going on will make you much more effective when debugging your code.

When I run machine learning algorithms a lot of the time, frankly, it doesn't work. Sophie, not the first time. I find that my ability to debug my code to be a TensorFlow code or something else, is really important to being an effective machine learning engineer. Even when you're using TensorFlow or some other framework, I hope that you find this deeper understanding useful for your own applications and for debugging your own machine learning algorithms as well. That's it. That's the last required video of this week with code in it. In the next video, I'd like to dive into what I think is a fun and fascinating topic, which is, what is the relationship between neural networks and AI or AGI, artificial general intelligence?

This is a controversial topic, but because it's been so widely discussed, I want to share with you some thoughts on this. When you are asked, are neural networks at all on the path to human level intelligence? You have a framework for thinking about that question. Let's go take a look at that fun topic, I think, in the next video.

LECTURE 13

Neural Network and the Brain

Ever since I was a teenager starting to play around with neural networks, I just felt that the dream of maybe someday building an AI system that's as intelligent as myself or as intelligent as a typical human, that that was one of the most inspiring dreams of AI. I still hold that dream alive today. But I think that the path to get there is not clear and could be very difficult. I don't know whether it would take us mere decades and whether we'll see breakthroughs within our lifetimes, or if it may take centuries or even longer to get there. Let's take a look at what this AGI, artificial general intelligence dream is like and speculate a bit on what might be possible paths, unclear paths, difficult paths to get there someday.

I think there's been a lot of unnecessary hype about AGI or artificial general intelligence. Maybe one reason for that is AI actually includes two very different things. One is ANI which stands for artificial narrow intelligence. This is an AI system that does one thing, a narrow task, sometimes really well and can be incredibly valuable, such as the smart speaker or self-driving car or web search, or AI applied to specific applications such as farming or factories. Over the last several years, ANI has made tremendous progress and it's creating, as you know, tremendous value in the world today. Because ANI is a subset of AI, the rapid progress in ANI makes it logically true that AI has also made tremendous progress in the last decade.

There's a different idea in AI, which is AGI, artificial general intelligence. There's hope of building AI systems that could do anything a typical human can do.

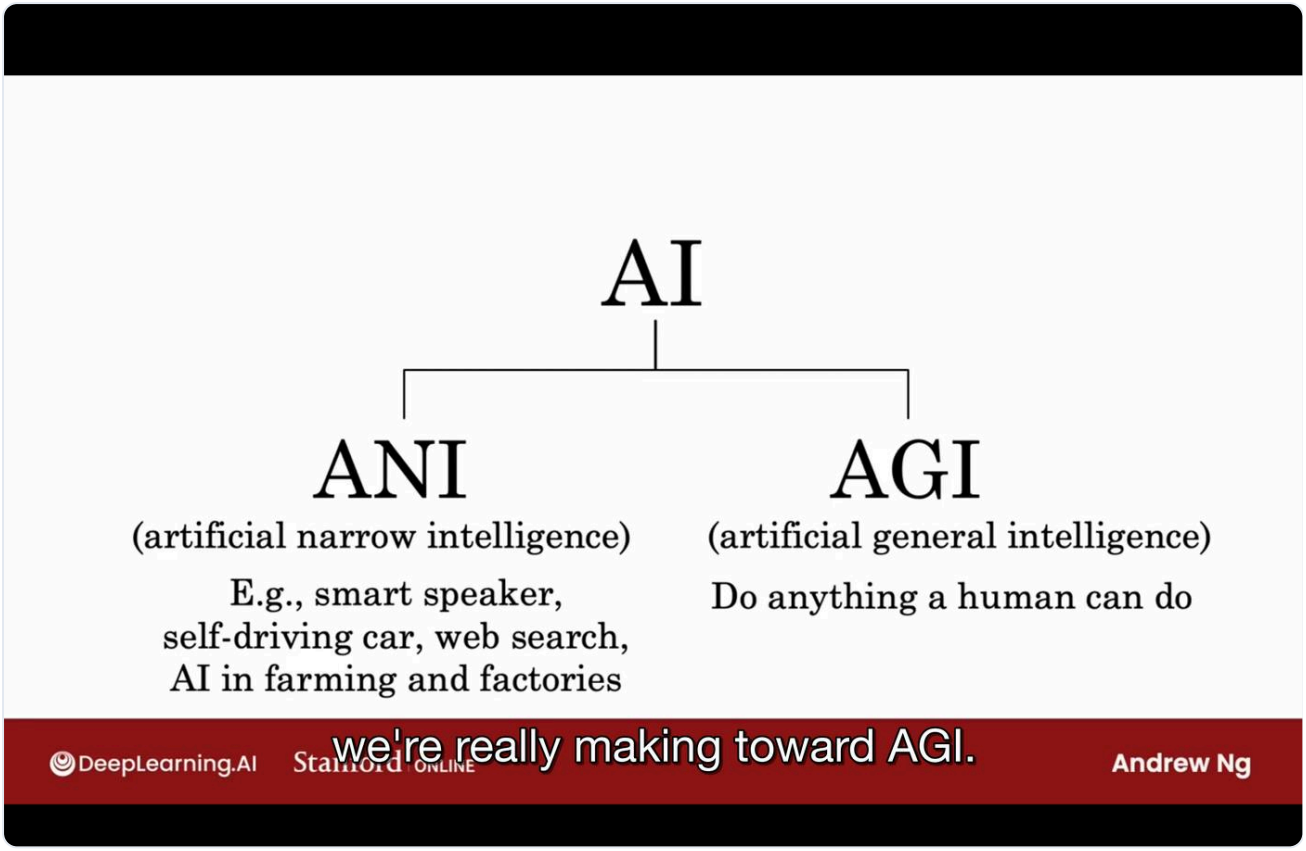


FIGURE 13.1

Despite all the progress in ANI and therefore tremendous progress in AI, I'm not sure how much progress, if any, we're really making toward AGI. I think all the progress in ANI has made people conclude correctly that there's tremendous progress in AI. But that has caused some people to conclude, I think incorrectly that a lot of progress in AI necessarily means that there's a lot of progress towards AGI. If you have else about AI and AGI, sometimes you might find drawing this picture useful for explaining some of the things going on in AI as well and some of the sources of unnecessary hype about AGI.

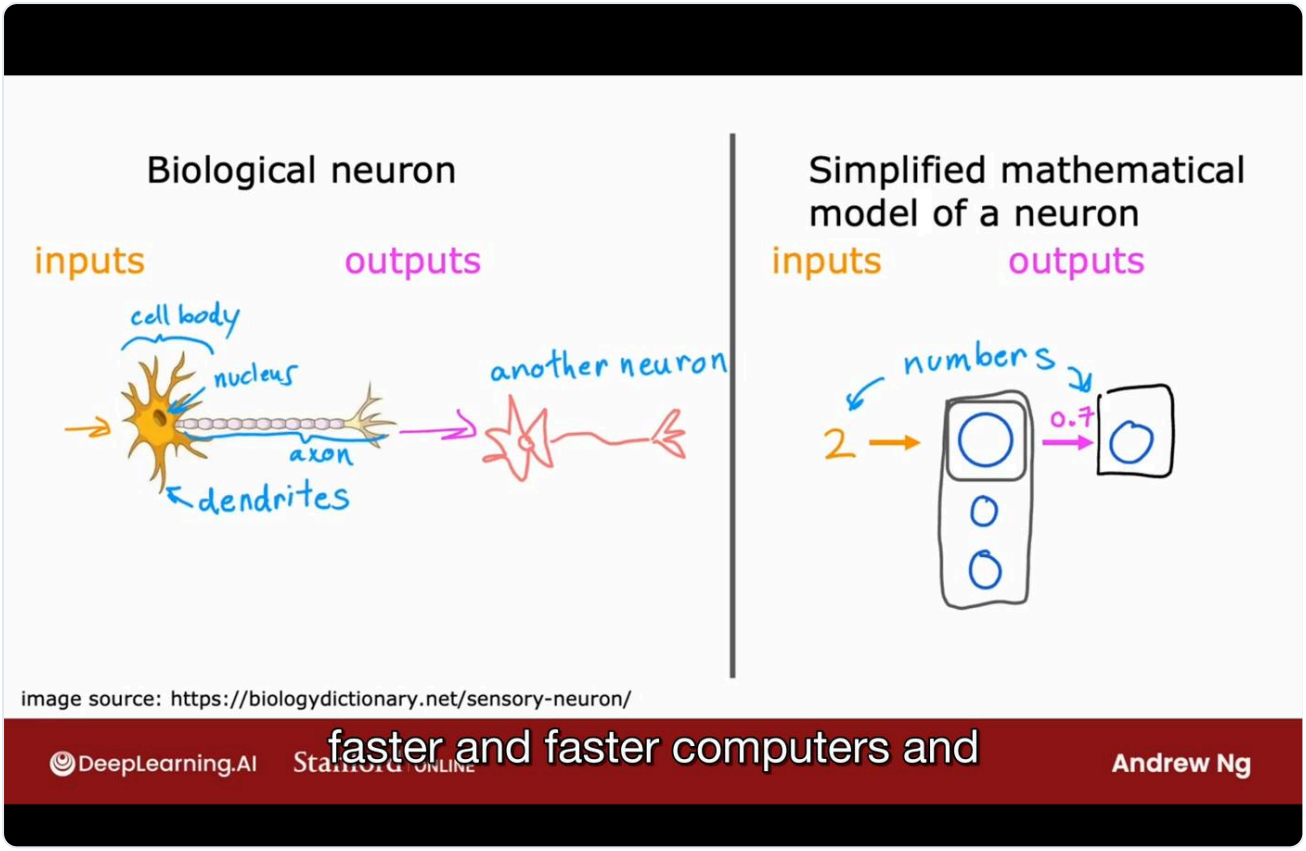


FIGURE 13.2

With the rise of modern deep learning, we started to simulate neurons and with faster and faster computers and even GPUs we can simulate even more neurons. I think there was this big hope many years ago that, boy, if only we could simulate a lot of neurons then we can simulate the human brain or something like a human brain and we've really intelligent systems. Sadly, it's turned out not to be quite as simple as that. I think two reasons for this is first, if you look at the artificial neural networks we're building, they are so simple that a logistic regression unit is really nothing like what any biological neuron is doing is so much simpler than what any neuron in your brain or mine is doing.

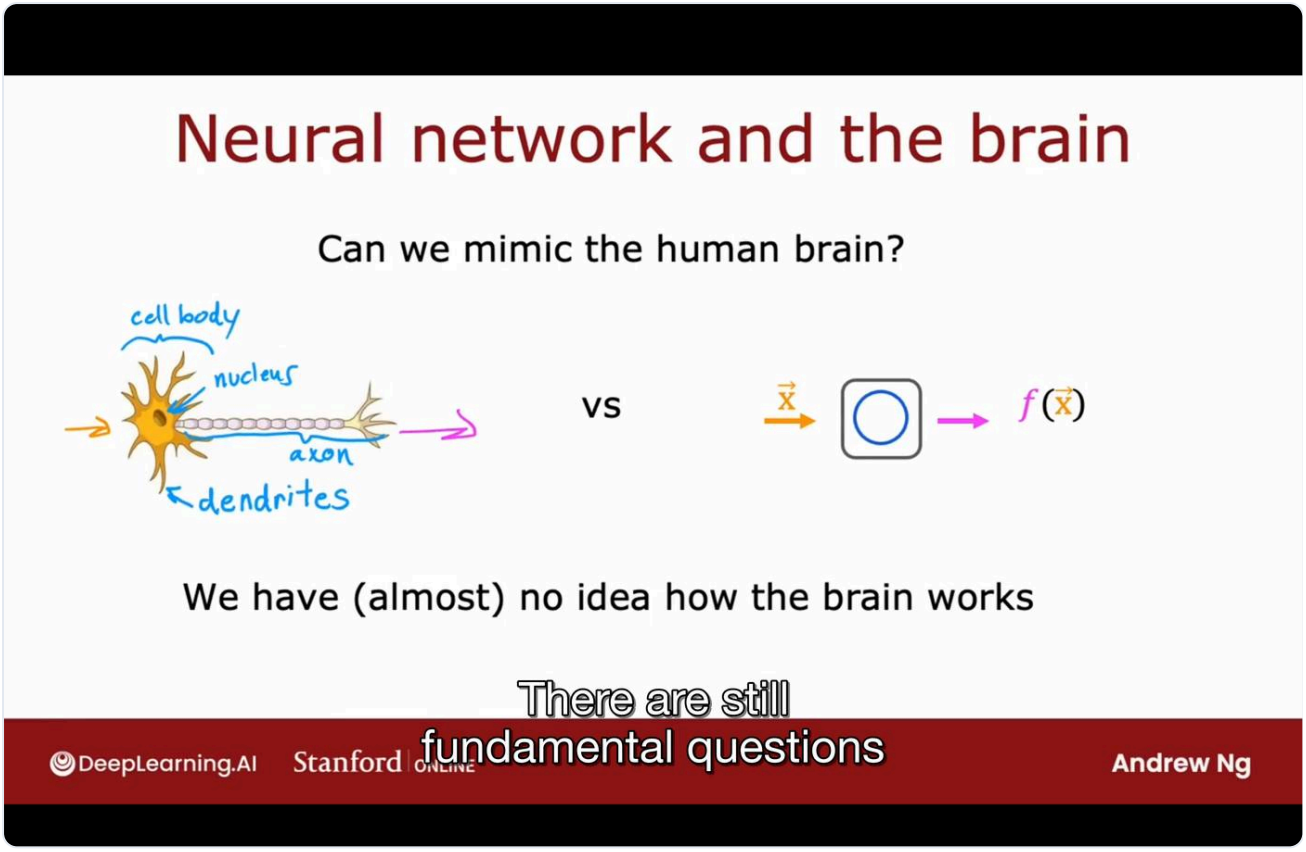


FIGURE 13.3 | Neural Network and the Brain

Second, even to this day, I think we have almost no idea how the brain works. There are still fundamental questions about how exactly does a neuron map from inputs to outputs that we just don't know today. Trying to simulate that in a computer, much less a single logistic function is just so far from an accurate model of what the human brain actually does. Given our very limited understanding both now and probably for the near future of how the human brain works, I think just trying to simulate the human brain as a path to AGI will be an incredibly difficult path. Having said that, is there any hope of within our lifetimes seeing breakthroughs in AGI?

Let me share with you some evidence that helps me keep that hope alive, at least for myself. There have been some fascinating experiments done on animals that shows or strongly suggests that the same piece of biological brain tissue can do a surprisingly wide range of tasks. This has led to the one learning algorithm hypothesis that maybe a lot of intelligence could be due to one or a small handful of learning algorithms. If only we could figure out what that one or small handful of algorithms are, we may be able to implement that in a computer someday. Let me share with you some details of those experiments. This is a result due to Roe et al. from many decades ago.

The part of your brain shown here is your auditory cortex, and your brain is wired to feed signals from your ears in the form of electrical impulses, depending on what sound your ear is detecting to that auditory cortex.

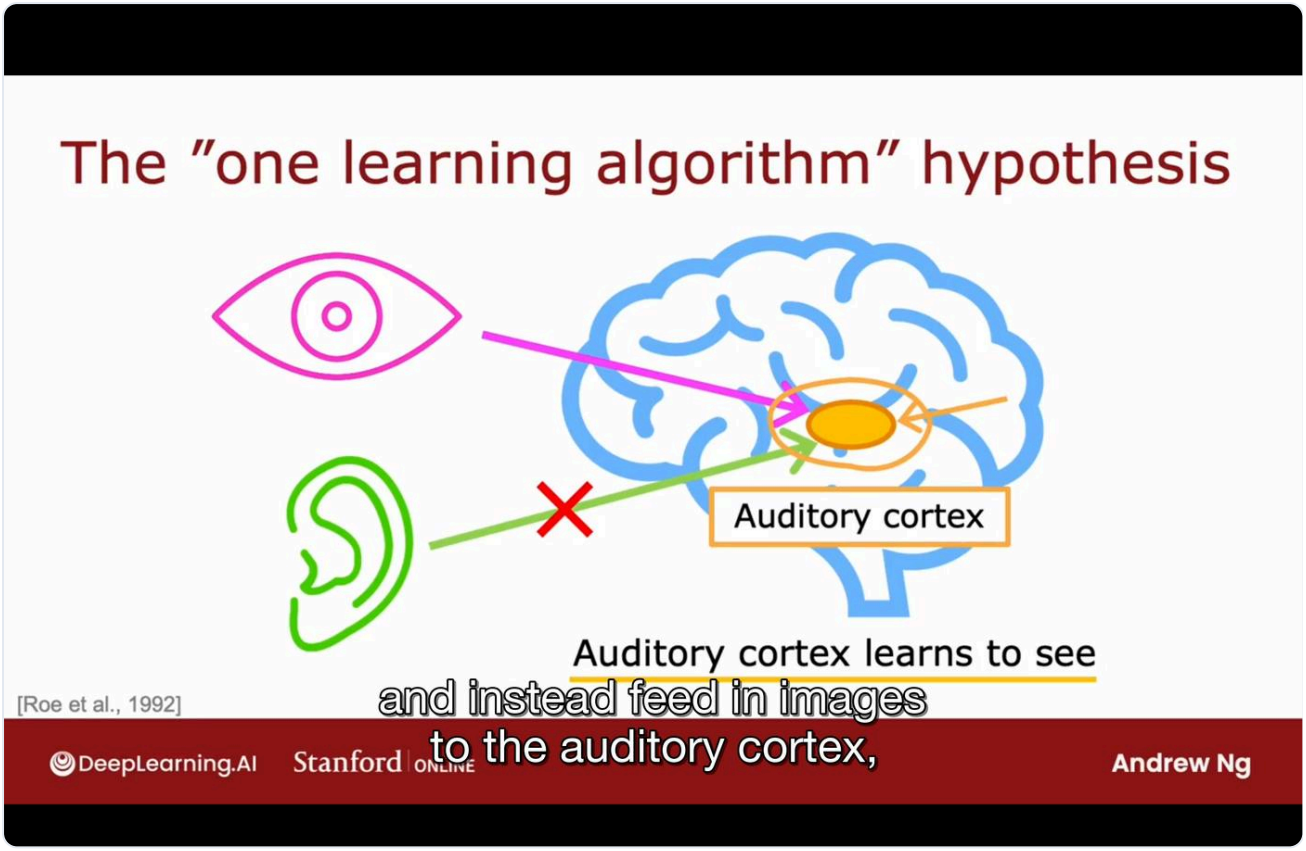


FIGURE 13.4

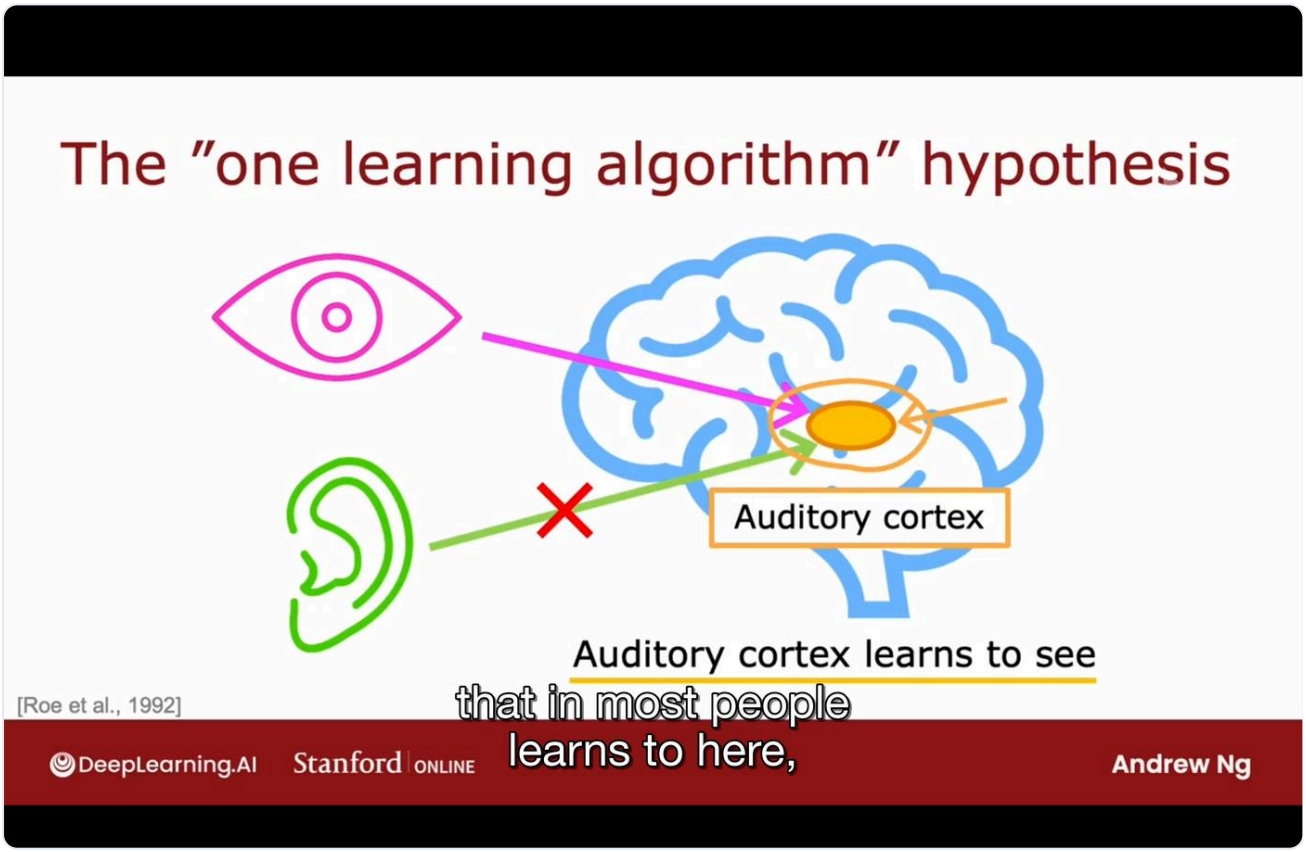


FIGURE 13.5

It turns out that if you were to rewire an animal brain,s to cut the wire between the ear and the auditory cortex, and instead feed in images to the auditory cortex, then the auditory cortex learns to see. Auditory refers to sound, and so this piece of the brain that in most people learns to here, when it is fed different data, it instead learns to see. Here's another example. This part of your brain is your somatosensory cortex, somatosensory refers to touch processing.

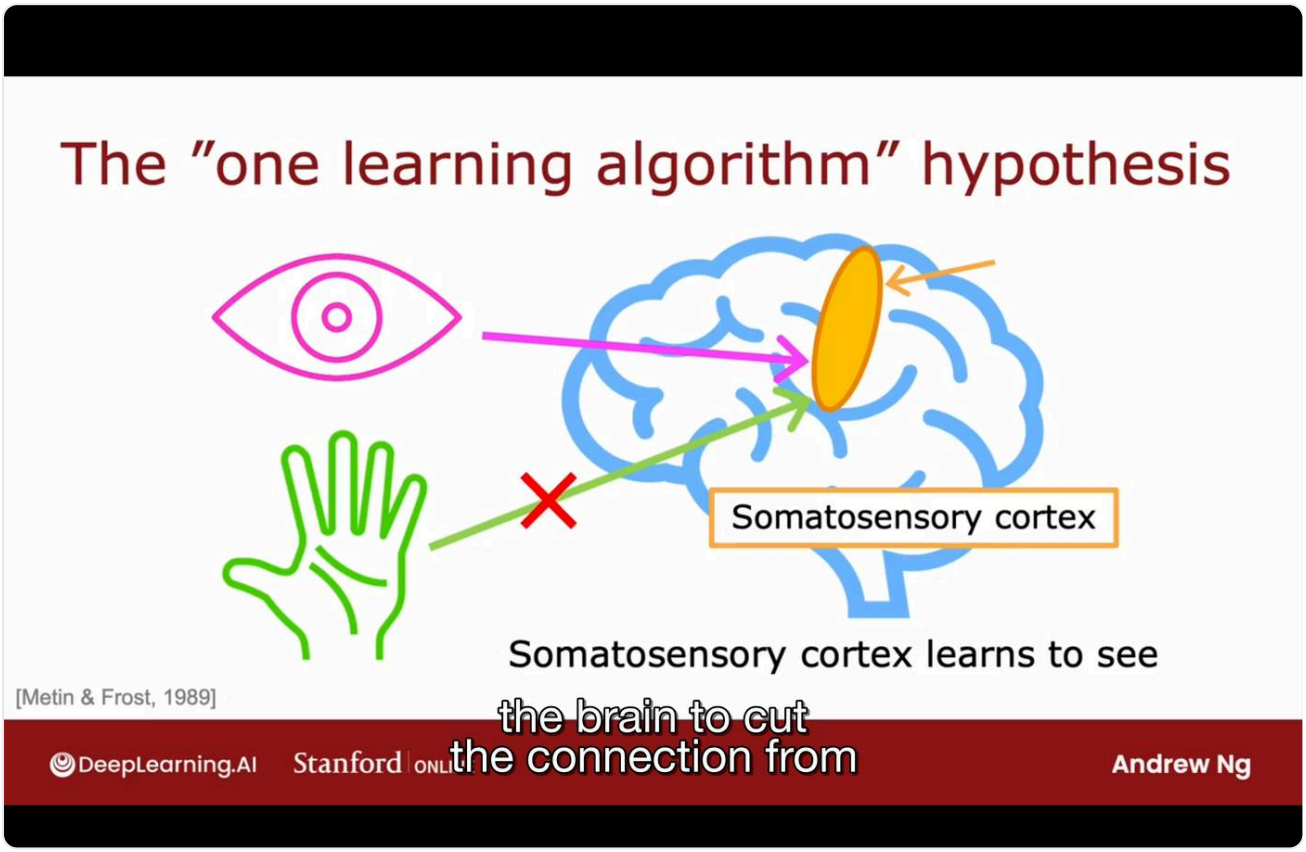


FIGURE 13.6

If you were to similarly rewire the brain to cut the connection from the touch sensors to that part of the brain and instead rewire the brain to feed in images, then the somatosensory cortex learns to see. There's been a sequence of experiments like this, showing that many different parts of the brain, just depending on what data is given can learn to see, or learn to feel, or learn to hear as if there was maybe one algorithm that just depending on what data or this given, learns to process that inputs accordingly. There happens systems built which take a camera may be mounted to someone's forehead and maps it to a pattern of voltages in a grid on someone's tongue.

By mapping a grayscale image to a pattern of voltages on your tongue, this can help people that are not cited line individuals learn to see with your tongue, or they've been fascinating experiments with human echolocation or humans sonar, so animals like dolphins and bats use sonar to see, and researchers have found that if you train humans to make clicking sounds, and listen to how that bounces off surroundings, humans can sometimes learn some degree of human echolocation.

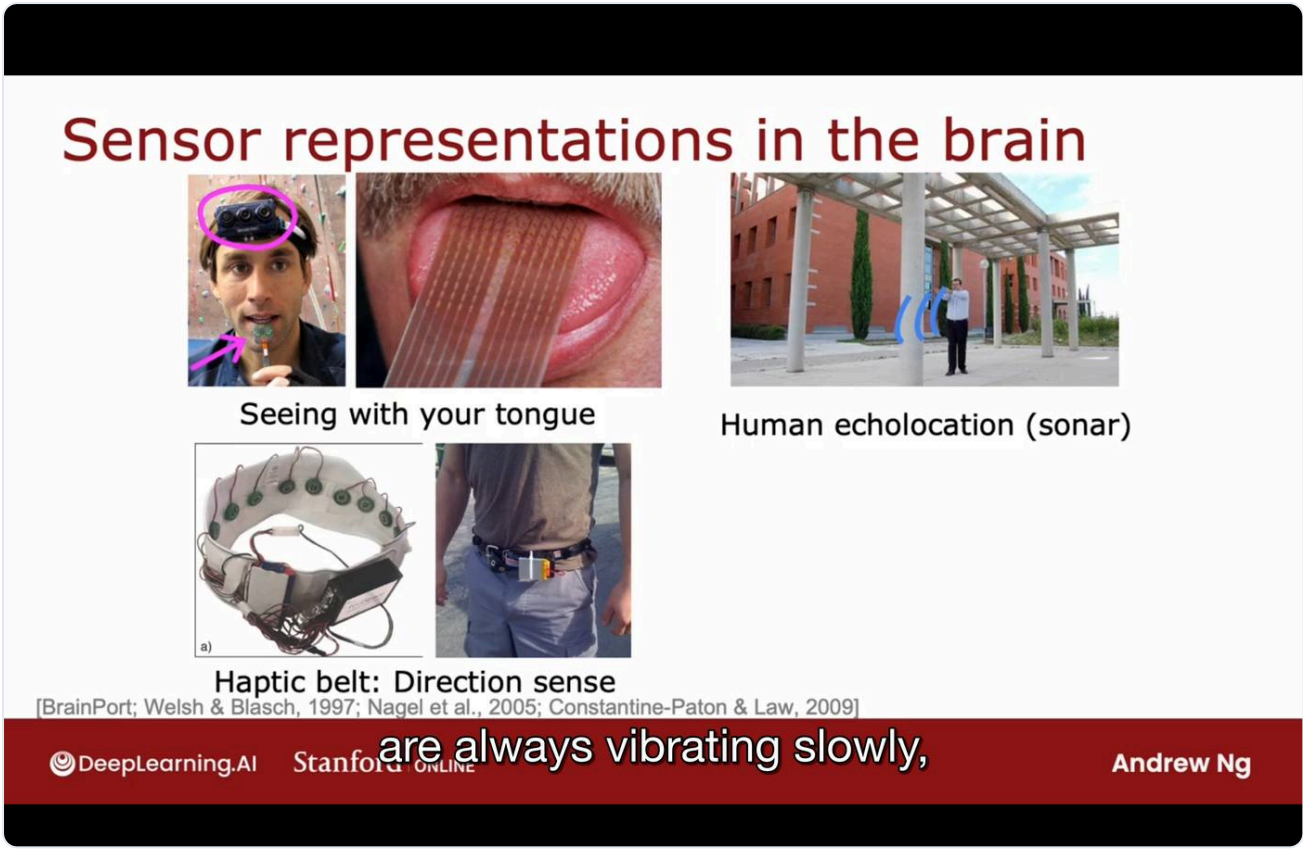


FIGURE 13.7 | Sensor Representations in the Brain

Or this is a haptic belt, and my research lab at Stanford once built something like this before as well, but if you mount a ring of buzzes around your waist and program it using a magnetic compass, so that say, the buzzers to the North most direction are always vibrating slowly, then you somehow gain a direction sense, which some animals have, but humans don't. Then it just feels like you're walking around and you just know where North is, it doesn't feel like that part of my waist is buzzing, it feels like, oh, I know where that north is.

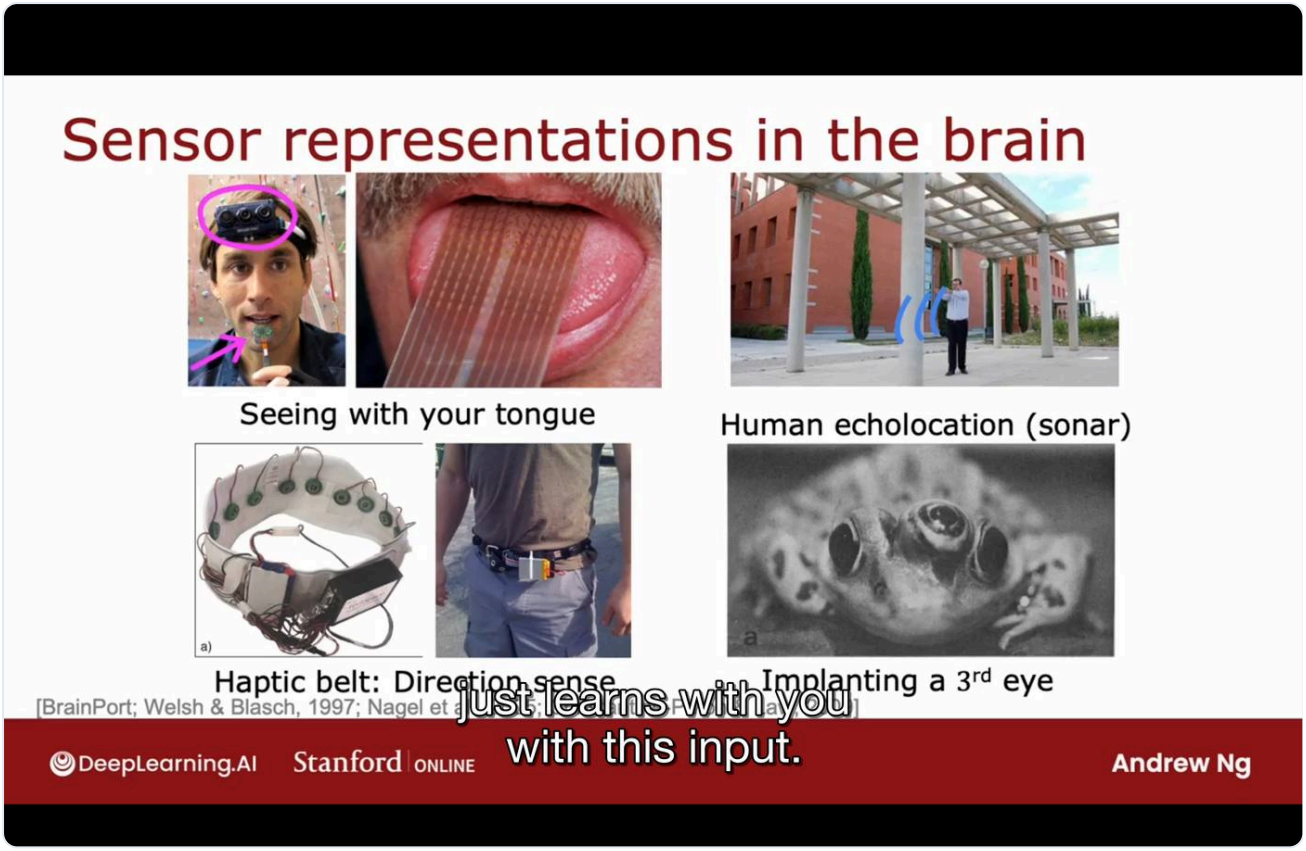


FIGURE 13.8 | Sensor Representations, in the Brain

Or surgeries implant a third eye onto frog and the brain just learns with you with this input. There have been a variety of experiments like these just showing that the human brain is amazingly adaptable, neuroscientists say is amazingly plastic, they just mean adaptable to bewildering range of sensor inputs, and so the question is, if the same piece of brain tissue can learn to see, or touch, or feel, or even other things, what is the average of users, and can we replicate this algorithm and implemented in a computer? I do feel bad for the frog and other animals, or which these experiments were done, although I think the conclusions are also quite fascinating.

Even to this day, I think working on AGI is one of the most fascinating science and engineering problems of all time, and maybe you will choose someday to do research on it. However, I think it's important to avoid over-hyping, I don't know if the brain is really one or a small handful of algorithms, and even if it were, I have no idea, and I don't think anyone knows what the algorithm is, but I still this hope alive, and maybe it is, and maybe we could, through a lot of hard work, someday discover an approximation to it. I still find this one of the most fascinating topics, I really think about it in my spare time and maybe someday, you be the one to make a contribution to this problem.

In the short term, I think even without pursuing AGI, machine learning and neural networks are very powerful tool, and even without trying to go all the way to build human-level intelligence, I think you find neural networks to be an incredibly powerful, and useful set of tools for applications that you might build. That's it for the required videos of this week, congratulations on getting to this point in the lessons. After this, we'll also have a few optional videos to dive a little bit

more deeply into efficient implementations of neural networks. In particular, in the optional videos to come, I'd like to share with you some details of how to vectorize implementations of neural networks.

I hope you also take a look at those videos.

LECTURE 14

Vectorized Implementation and Matrix Multiplication

One of the reasons that deep learning researchers have been able to scale up neural networks, and thought really large neural networks over the last decade, is because neural networks can be vectorized. They can be implemented very efficiently using matrix multiplications. It turns out that parallel computing hardware, including GPUs, but also some CPU functions are very good at doing very large matrix multiplications. In this video, we'll take a look at how these vectorized implementations of neural networks work. Without these ideas, I don't think deep learning would be anywhere near a success and scale today. Here on the left is the code that you had seen previously of how you would implement forward prop, or forward propagation, in a single layer.

X here is the input, W, the weights of the first, second, and third neurons, say, parameters B, and then this is the same code as which we saw before. This will output three numbers, say, like that.

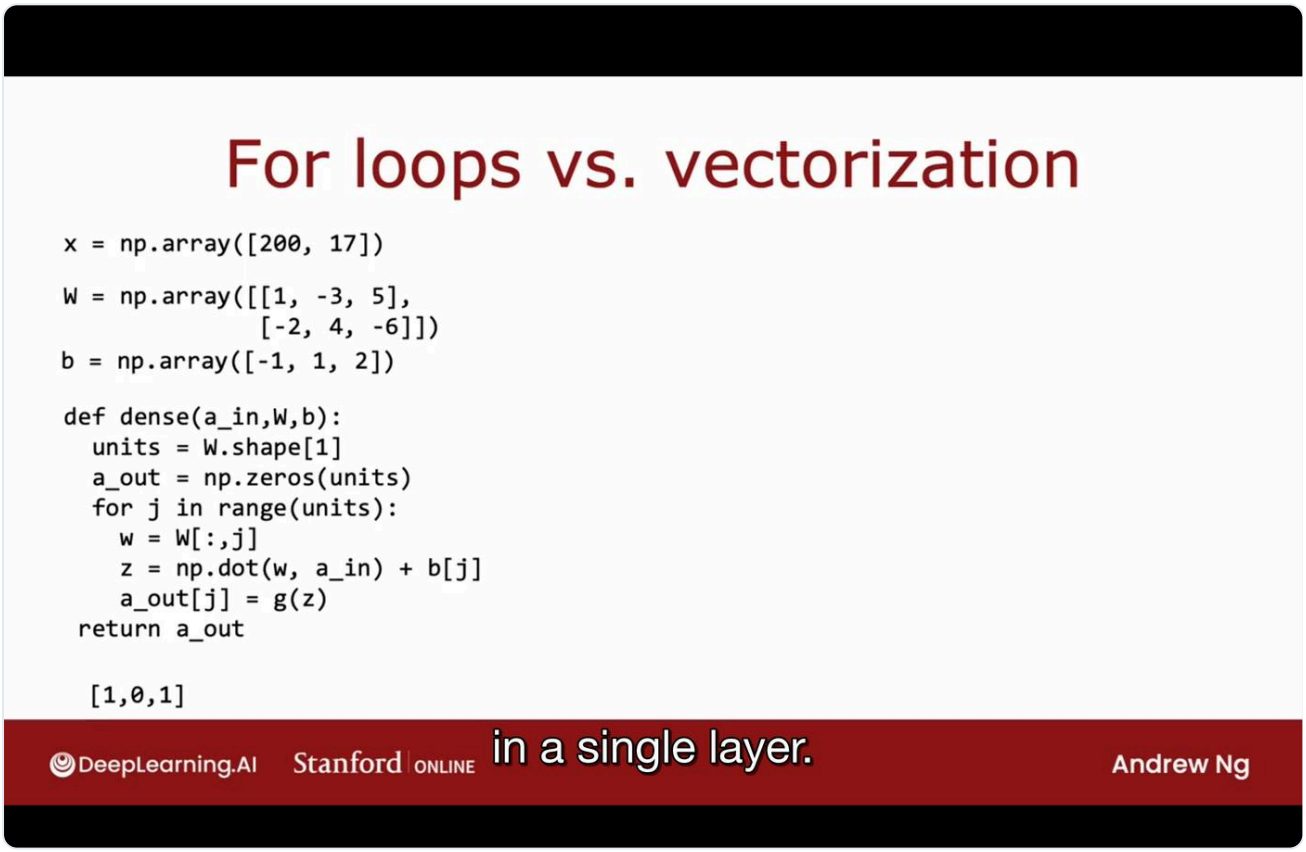


FIGURE 14.1

If you actually implement this computation, you get 1, 0, 1. It turns out you can develop a vectorized implementation of this function as follows. Set X to be equal to this. Notice the double square brackets. This is now a 2D array, like in TensorFlow. W is the same as before, and B, I'm now using B, is also a one by three 2D array.

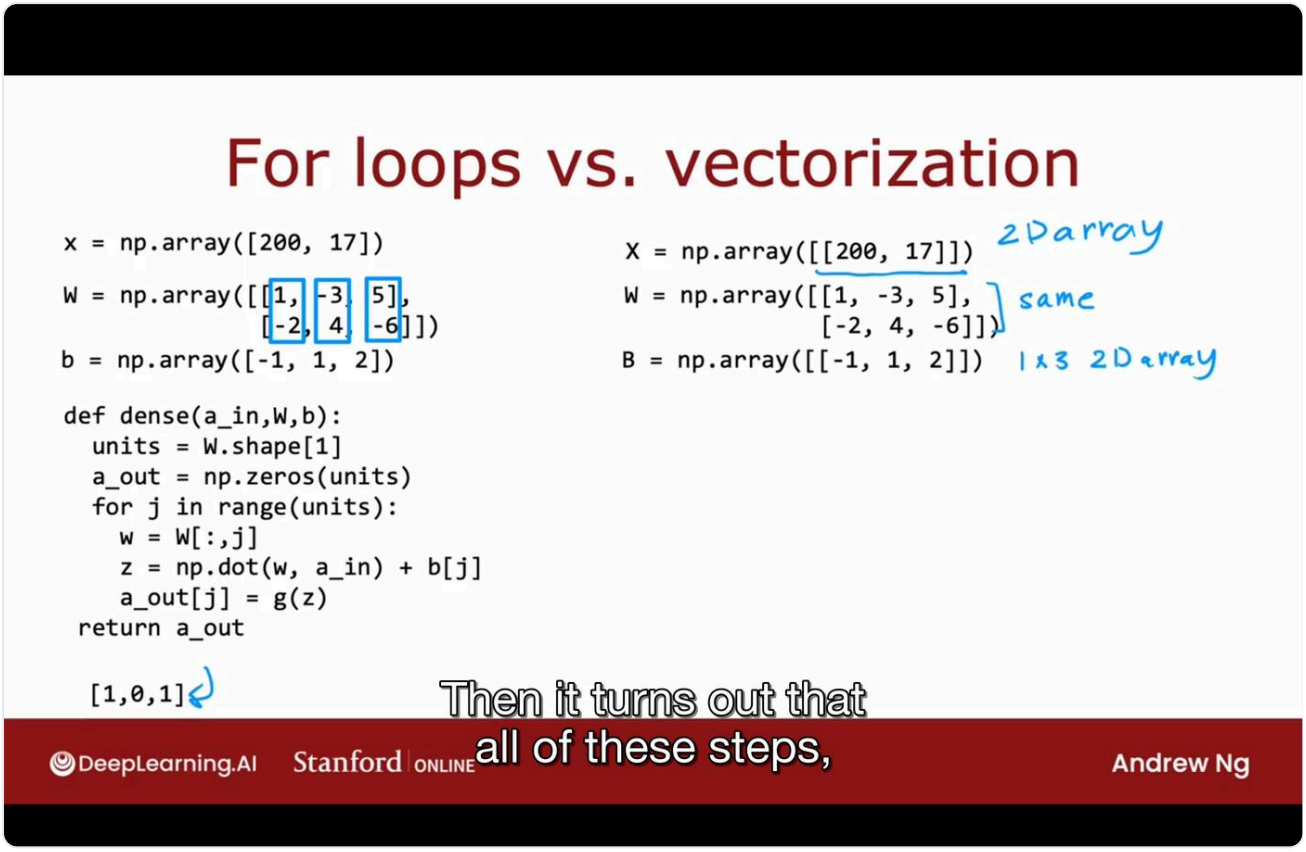


FIGURE 14.2

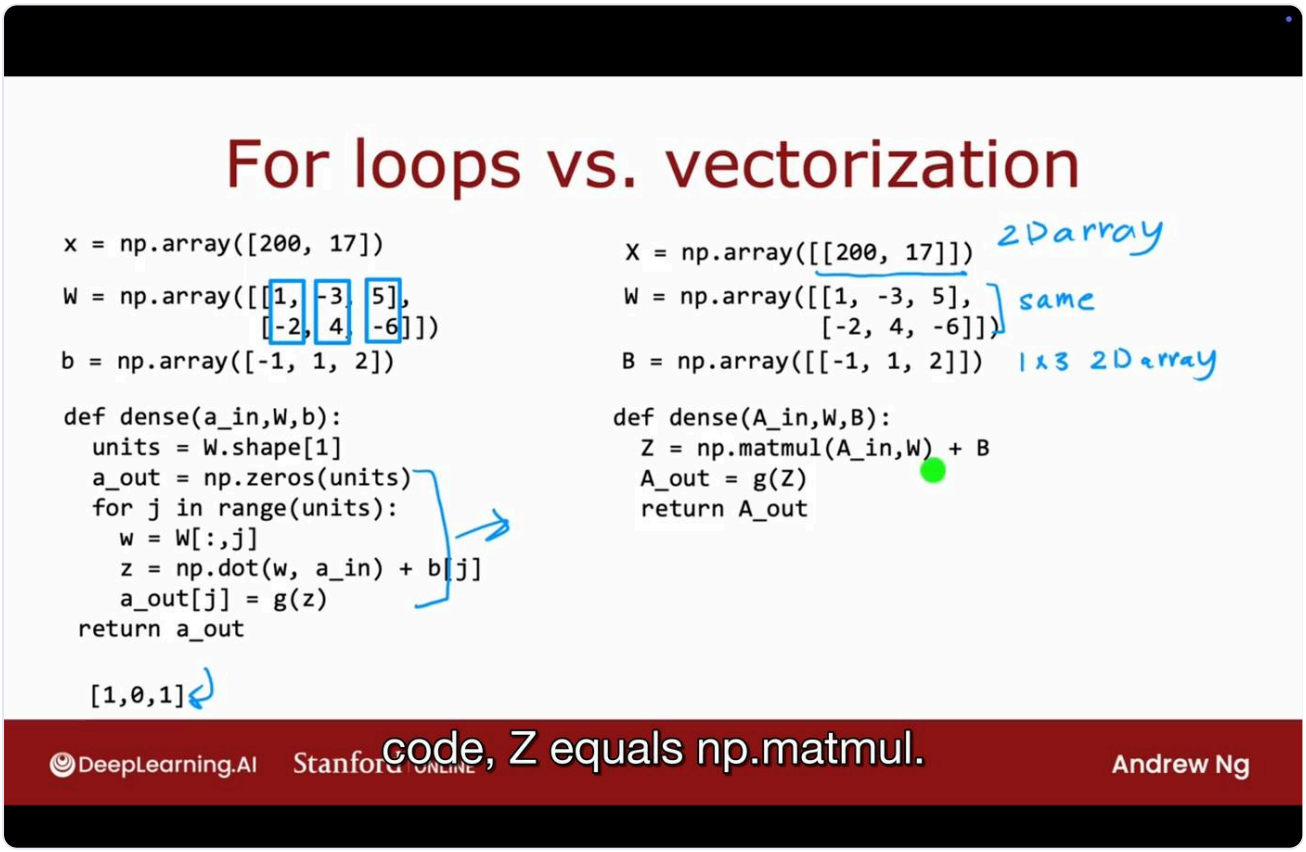


FIGURE 14.3

Then it turns out that all of these steps, this for loop inside, can be replaced with just a couple of lines of code, Z equals np.matmul. Matmul is how NumPy carries out matrix multiplication. Where now X and W are both matrices, and so you just multiply them together. It turns out that this for loop, all of these lines of code can be replaced with just a couple of lines of code, which gives a vectorized implementation of this function. You compute Z, which is now a matrix again, as numpy.matmul between A in and W, where here A in and W are both matrices, and matmul is how NumPy carries out a matrix multiplication. It multiplies two matrices together, and then adds the matrix B to it.

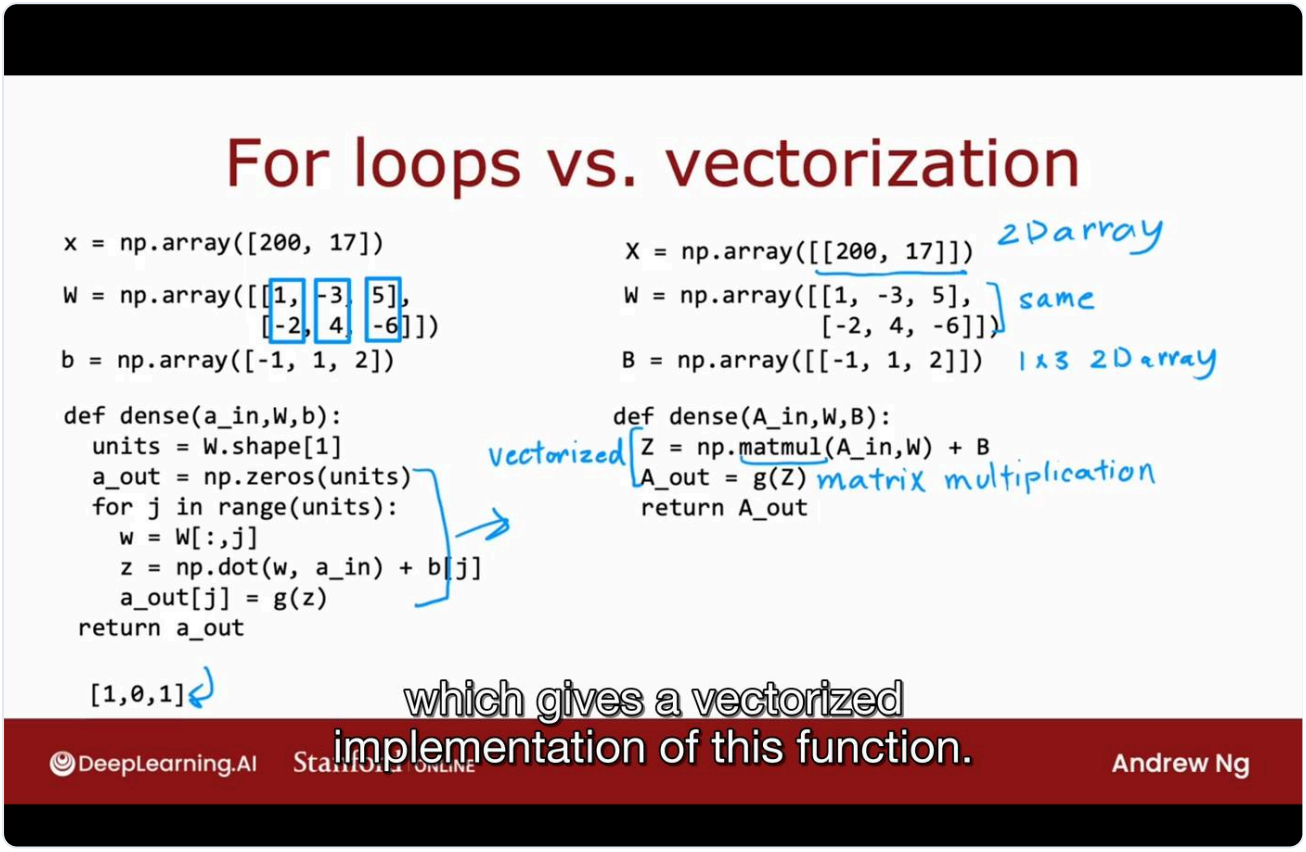


FIGURE 14.4

Then A out is equal to the activation function g, that is the sigmoid function, applied element-wise to this matrix Z, and then you finally return A out. This is what the code looks like. Notice that in the vectorized implementation, all of these quantities, x, which is fed into the value of A in as well as W, B, as well as Z and A out, all of these are now 2D arrays. All of these are matrices.

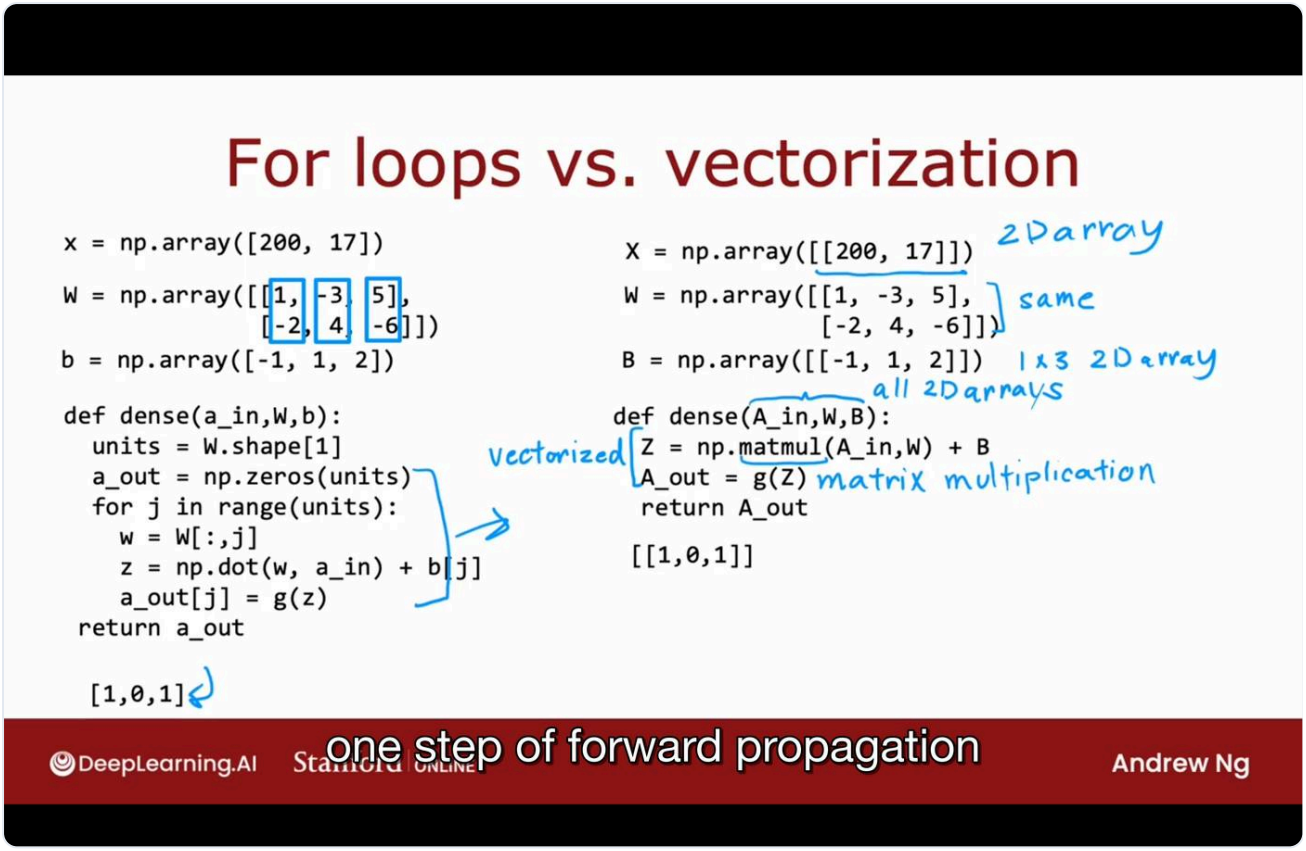


FIGURE 14.5

This turns out to be a very efficient implementation of one step of forward propagation through a dense layer in the neural network. This is code for a vectorized implementation of forward prop in a neural network. But what is this code doing and how does it actually work? What is this matmul actually doing? In the next two videos, both also optional, we'll go over matrix multiplication and how that works. If you're familiar with linear algebra, if you're familiar with vectors, matrices, transposes, and matrix multiplications, you can safely just quickly skim over these two videos and jump to the last video of this week. Then in the last video of this week, also optional, we'll dive into more detail to explain how matmul gives you this vectorized implementation.

Let's go onto the next video, where we'll take a look at what matrix multiplication is.

LECTURE 15

Dot Products

You know that a matrix is just a block or 2D array of numbers. What does it mean to multiply two matrices? Let's take a look.

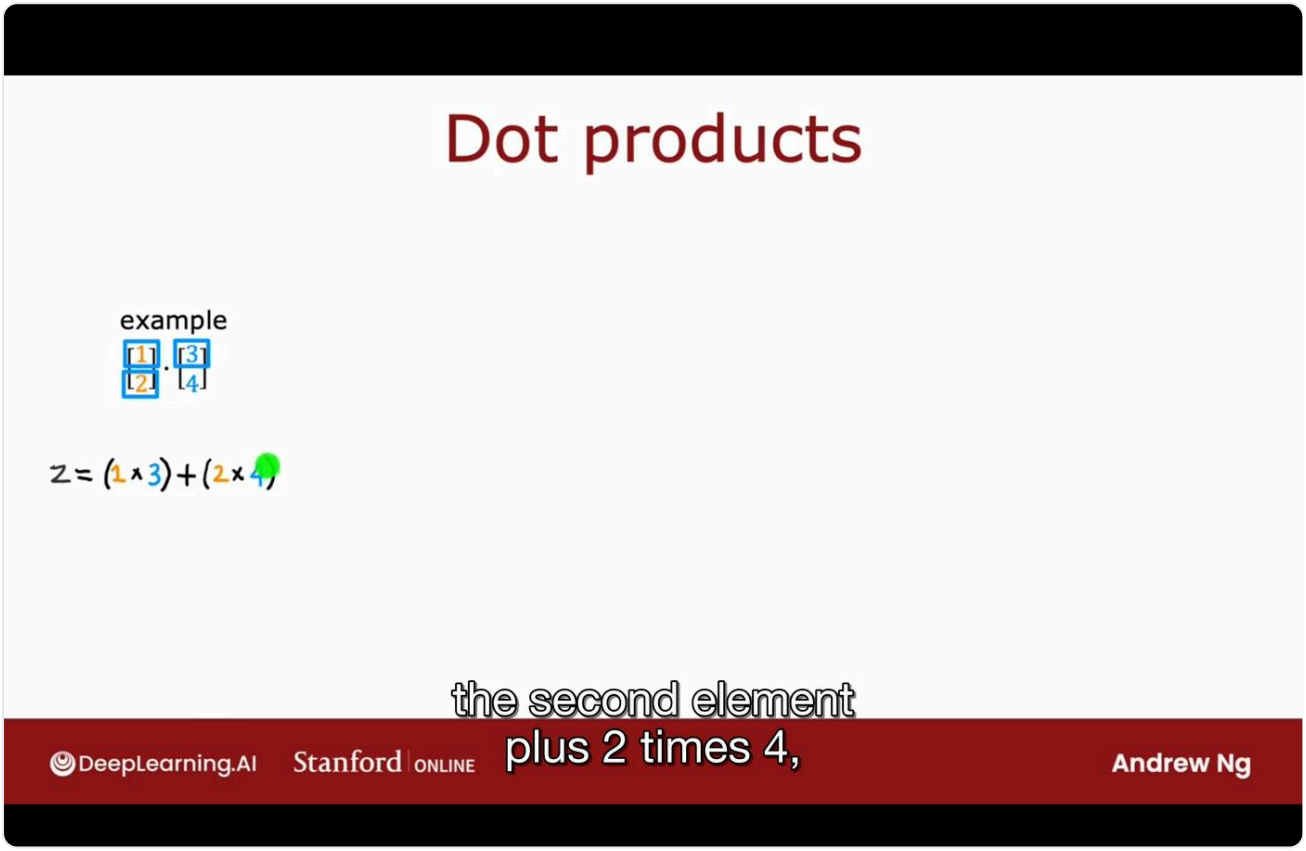


FIGURE 15.1 | Dot Products

In order to build up to multiplying matrices, let's start by looking at how we take dot products between vectors. Let's use the example of taking the dot product between this vector 1, 2 and this vector 3, 4. If z is the dot product between these two vectors, then you compute z by multiplying the first element by the first element here, it's 1 times 3, plus the second element times the second element plus 2 times 4, and so that's just 3 plus 8, which is equal to 11.

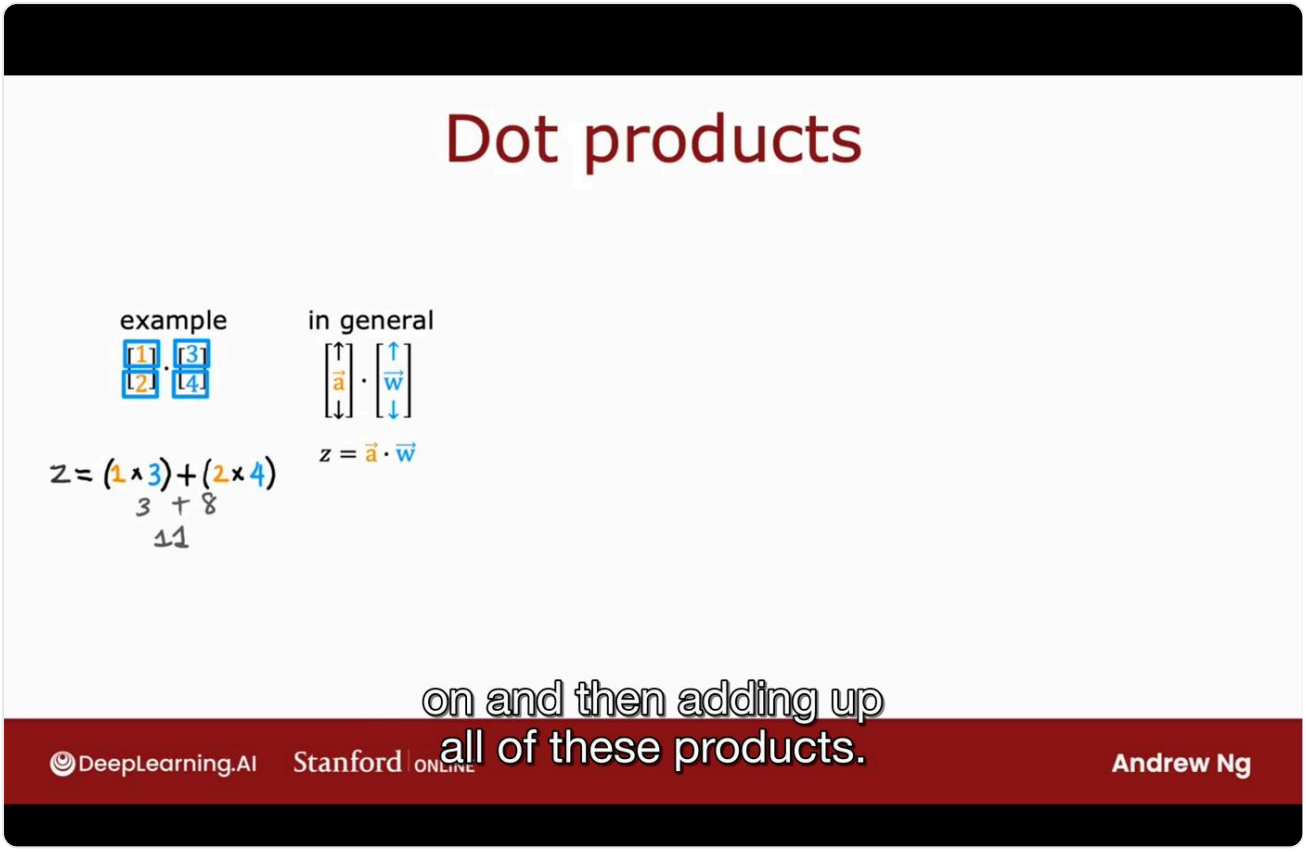


FIGURE 15.2 | Dot Products

In the more general case, if z is the dot product between a vector a and vector w, then you compute z by multiplying the first element together and then the second elements together and the third and so on and then adding up all of these products. That's the vector, vector dot product. It turns out there's another equivalent way of writing a dot product, which has given a vector a, that is, 1, 2 written as a column. You can turn this into a row. That is, you can turn it from what's called a column vector to a row vector by taking the transpose of a. The transpose of the vector a means you take this vector and lay its elements on the side like this.

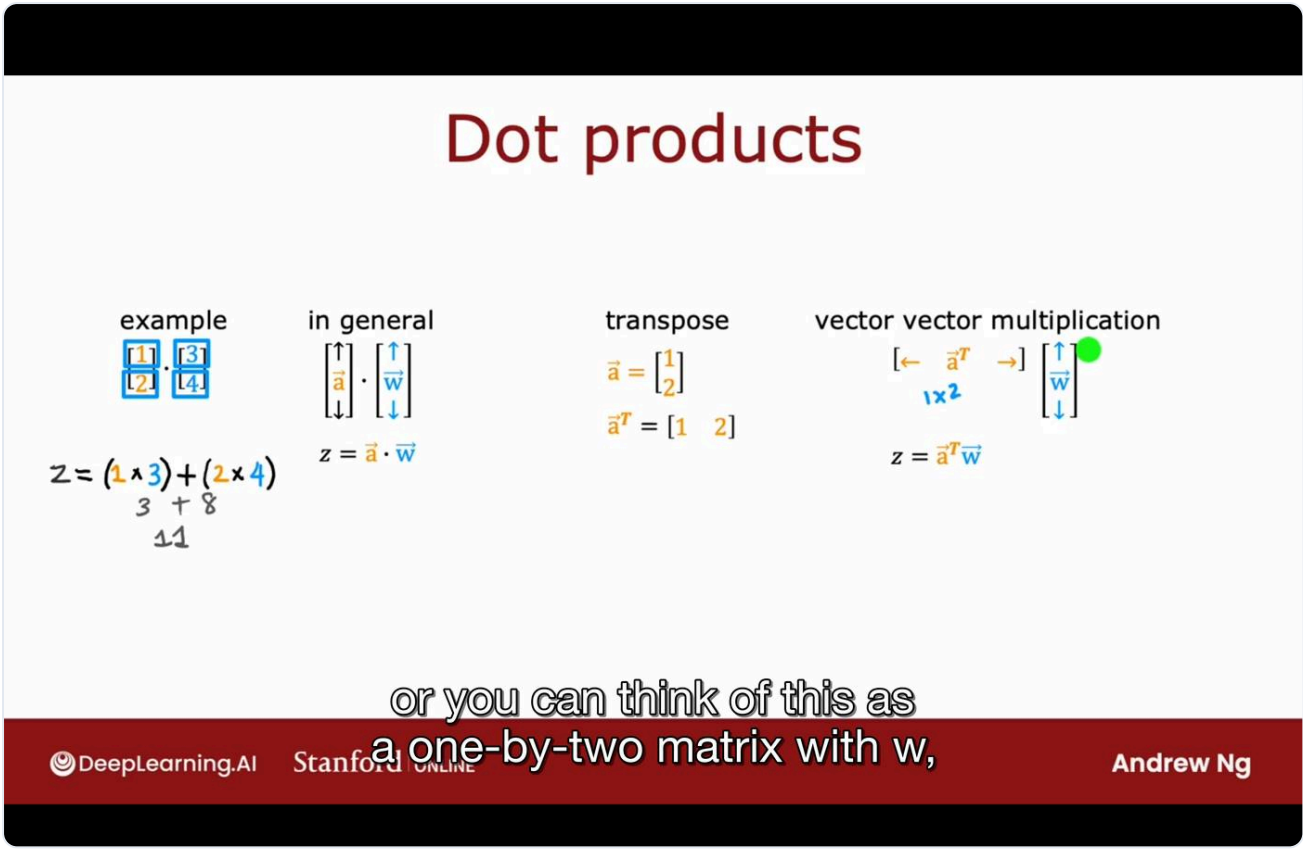


FIGURE 15.3 | Dot Products

It turns out that if you multiply a transpose, this is a row vector, or you can think of this as a one-by-two matrix with w, which you can now think of as a two-by-one matrix.

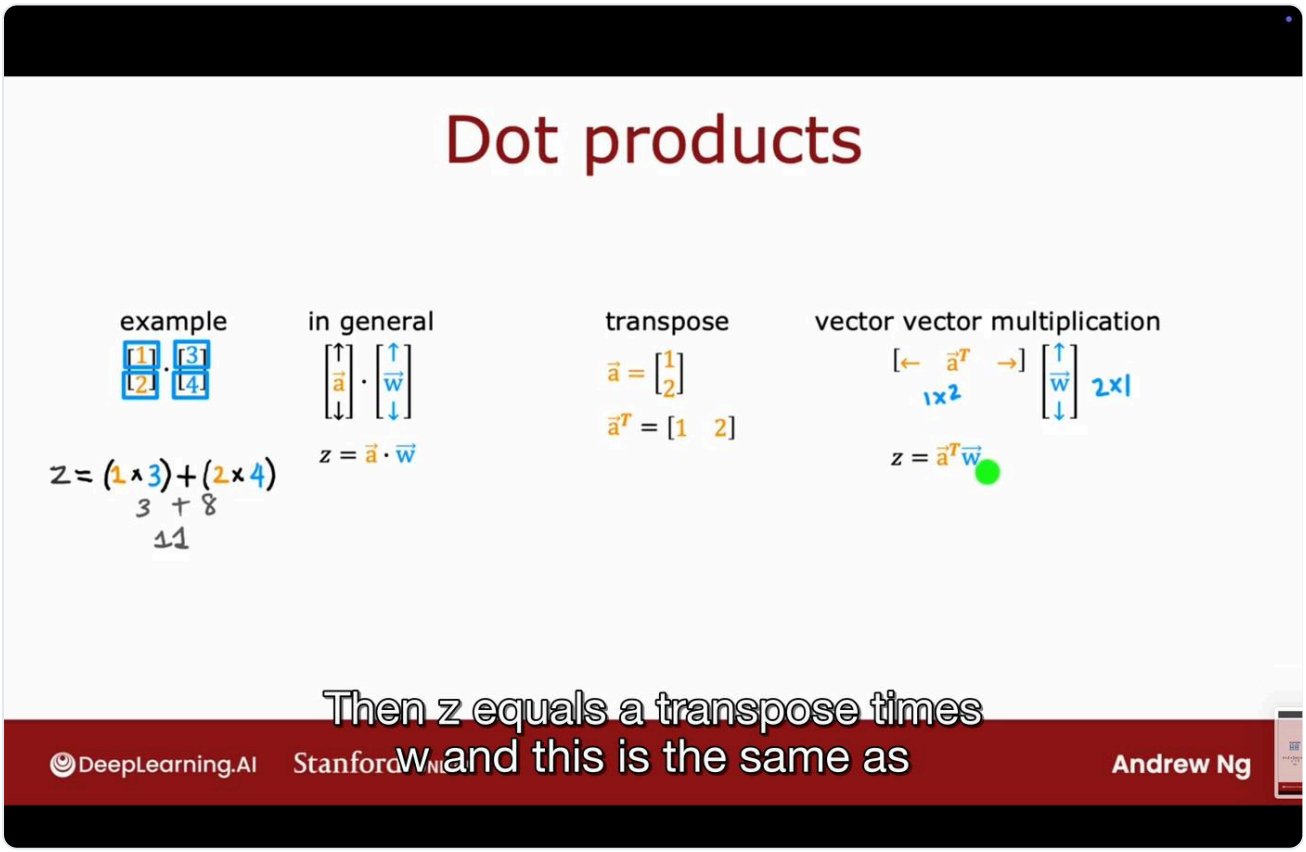


FIGURE 15.4 | Dot Products

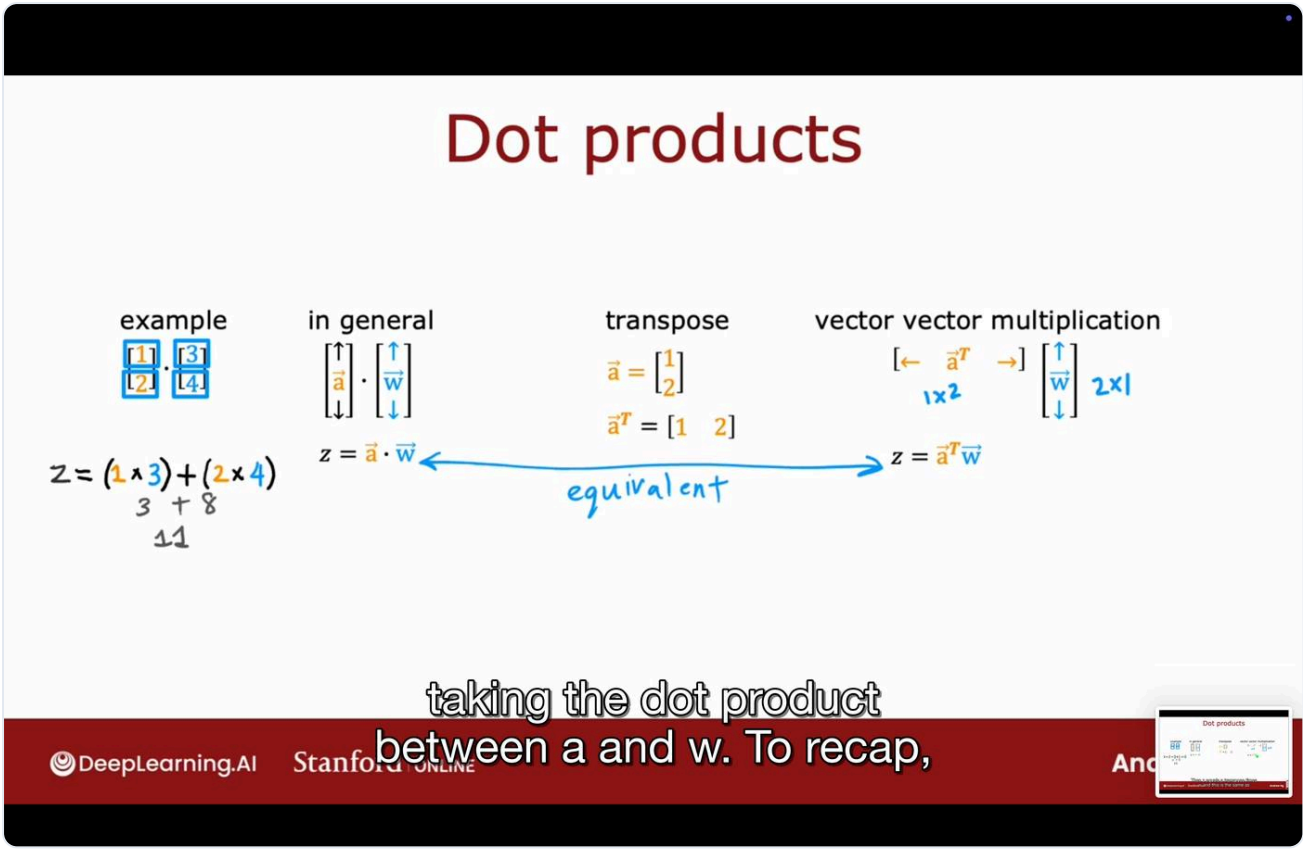


FIGURE 15.5 | Dot Products

Then z equals a transpose times w and this is the same as taking the dot product between a and w. To recap, z equals the dot product between a and w is the same as z equals a transpose, that is a laid on the side, multiplied by w and this will be useful for understanding matrix multiplication.

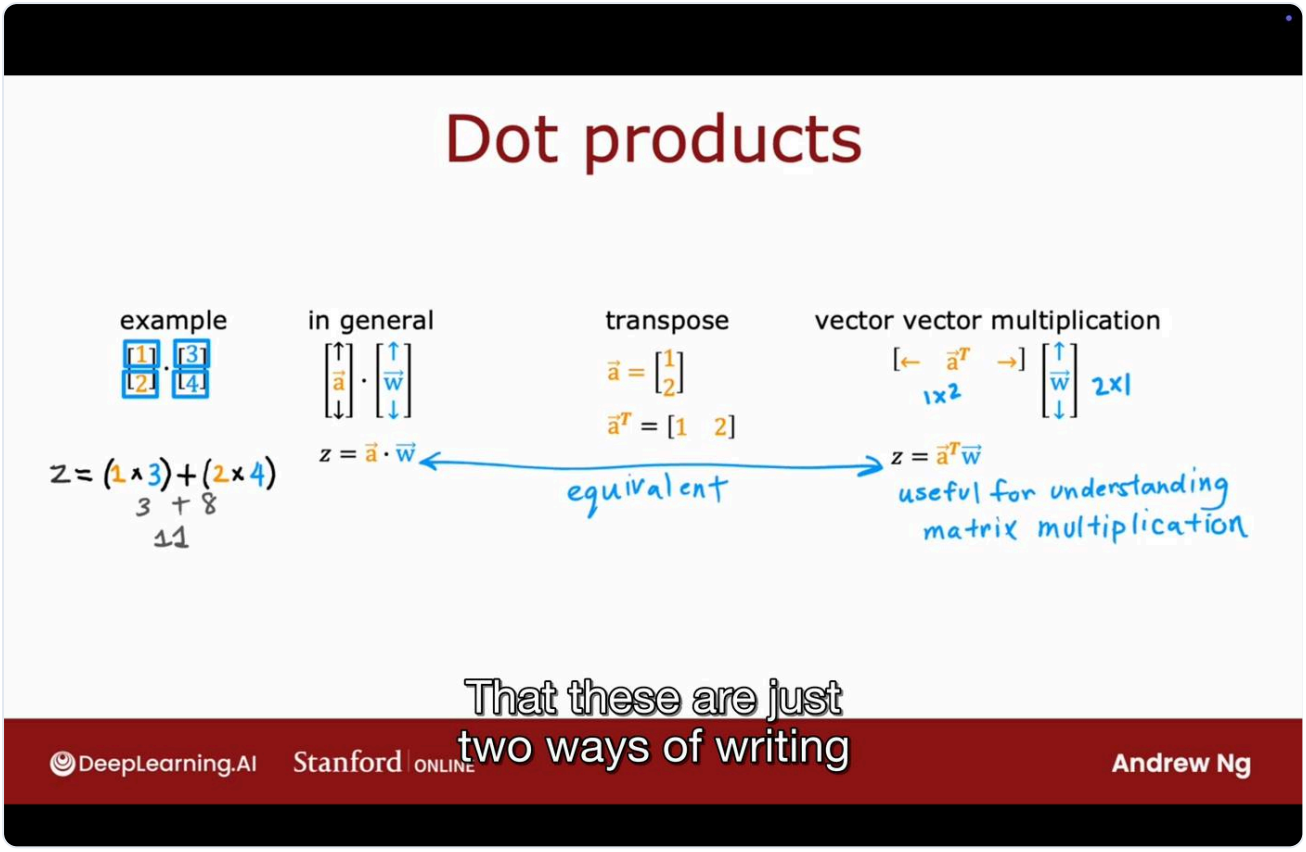


FIGURE 15.6 | Dot Products

That these are just two ways of writing the exact same computation to arrive at z. Now let's look at vector matrix multiplication, which is when you take a vector and you multiply a vector by a matrix. Here again is the vector a 1, 2 and a transpose is a laid on the side, so rather than this think of this as a two-by-one matrix it becomes a one-by-two matrix. Let me now create a two-by-two matrix w with these four elements, 3, 4, 5, 6. If you want to compute Z as a transpose times w. Let's see how you go about doing so. It turns out that Z is going to be a one-by-two matrix, and to compute the first value of Z we're going to take a transpose, 1, 2 here, and multiply that by the first column of w, that's 3, 4.

To compute the first element of Z, you end up with 1 times 3 plus 2 times 4, which we saw earlier is equal to 11, and so the first element of Z is 11. Let's figure out what's the second element of Z. It turns out you just repeat this process, but now multiplying a transpose by the second column of w. To do that computation, you have 1 times 5 plus 2 times 6, which is equal to 5 plus 12, which is 17.

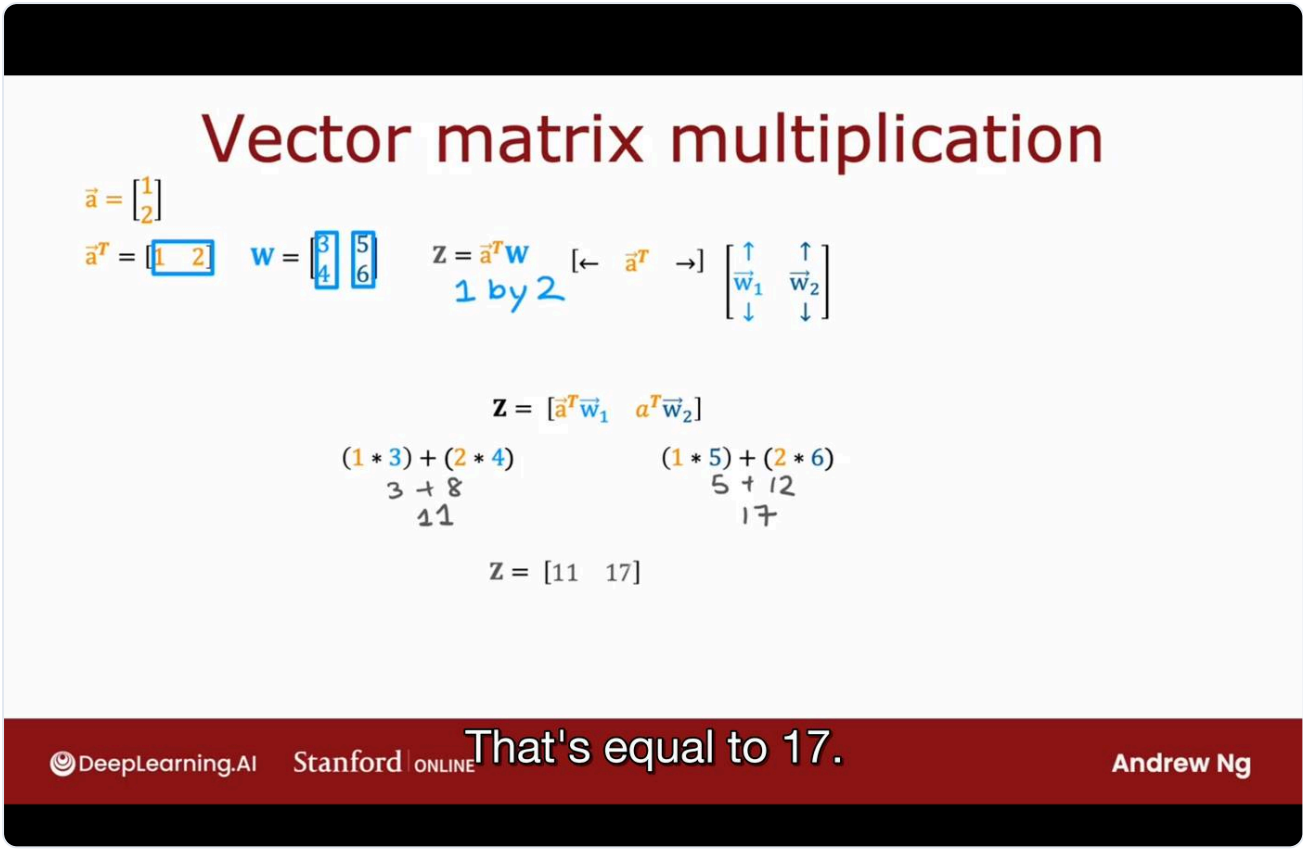


FIGURE 15.7 | Vector Matrix Multiplication

That's equal to 17. Z is equal to this one-by-two matrix, 11 and 17. Now, just one last thing, and then that'll take us to the end of this video, which is how to take vector matrix multiplication and generalize it to matrix matrix multiplication. I have a matrix A with these four elements, the first column is 1, 2 and the second column is negative 1, negative 2 and I want to know how to compute a transpose times w. Unlike the previous slide, A now is a matrix rather than just the vector or the matrix is just a set of different vectors stacked together in columns. First let's figure out what is A transpose. In order to compute A transpose, we're going to take the columns of A and similar to what happened when you transpose a vector, we're going to take the columns and lay them on the side, one column at a time.

The first column 1, 2 becomes the first row 1, 2, let's just laid on side, and this second column, negative 1, negative 2 becomes laid on the side negative 1, negative 2 like this. The way you transpose a matrix is you take the columns and you just lay the columns on the side, one column at a time, you end up with this being A transpose. Next we have this matrix W, which going to write as 3,4, 5,6. There's a column 3, 4 and the column 5, 6. One way I encourage you to think of matrices. At least there's useful for neural network implementations is if you see a matrix, think of the columns of the matrix and if you see the transpose of a matrix, think of the rows of that matrix as being grouped together as illustrated here, with A and A transpose as well as W.

Now, let me show you how to multiply A transpose and W. In order to carry out this computation let me call the columns of A, a_1 and a_2 and that means that a_1 transpose, this the first row of A transpose, and a_2 transpose is the second row of A transpose. Then same as before, let me call the columns of W to be w_1 and w_2. It turns out that to compute A transpose W, the first thing we need to do is let's just ignore the second row of A and let's just pay attention to the first row of A

and let's take this row 1, 2 that is a_1 transpose and multiply that with W. You already know how to do that from the previous slide. The first element is 1, 2, inner product or dot product we've 3, 4.

That ends up with 3 times 1 plus 2 times 4, which is 11. Then the second element is 1, 2 A transpose, inner product we've 5, 6. There's 5 times 1 plus 6 times 2, which is 5 plus 12, which is 17. That gives you the first row of Z equals A transpose W. All we've done is take a_1 transpose and multiply that by W. That's exactly what we did on the previous slide. Next, let's forget a_1 for now, and let's just look at a_2 and take a_2 transpose and multiply that by W. Now we have a_2 transpose times W. To compute that first we take negative 1 and negative 2 and dot product that with 3, 4. That's negative 1 times 3 plus negative 2 times 4 and that turns out to be negative 11.

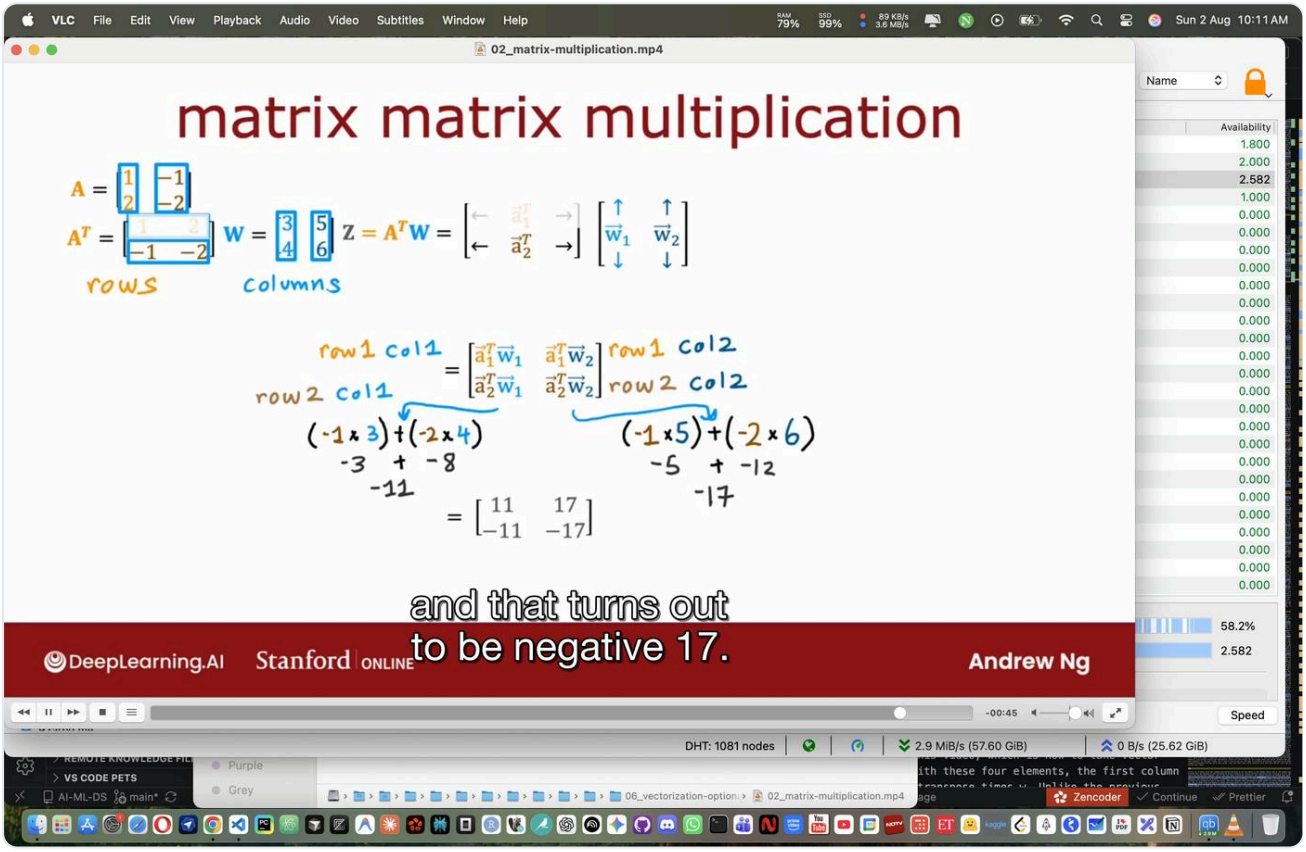


FIGURE 15.8

Then we have to compute a_2 transpose times the second column, and has negative 1 times 5 plus negative 2 times 6, and that turns out to be negative 17. You end up with A transpose times W is equal to this two-by-two matrix over here. Let's talk about the general form of matrix matrix multiplication. This was an example of how you multiply a vector with a matrix, or a matrix with a matrix is a lot of dot products between vectors but ordered in a certain way to construct the elements of the upper Z, one element at a time. I know this was a lot, but in the next video, let's look at the general form of how a matrix matrix multiplication is defined and I hope that will make all this clear as well.

Let's go on to the next video.

LECTURE 16

Transpose

So let's take a look at the general form of how you multiply two matrices together. And then in the last video after this one, we'll take this and apply it to the vectorized implementation of a neural network. Let's dive in. Here's the matrix A, which is a 2 by 3 matrix because it has two rows and three columns. As before I encourage you to think of the columns of this matrix as three vectors, vectors a1, a2 and a3.

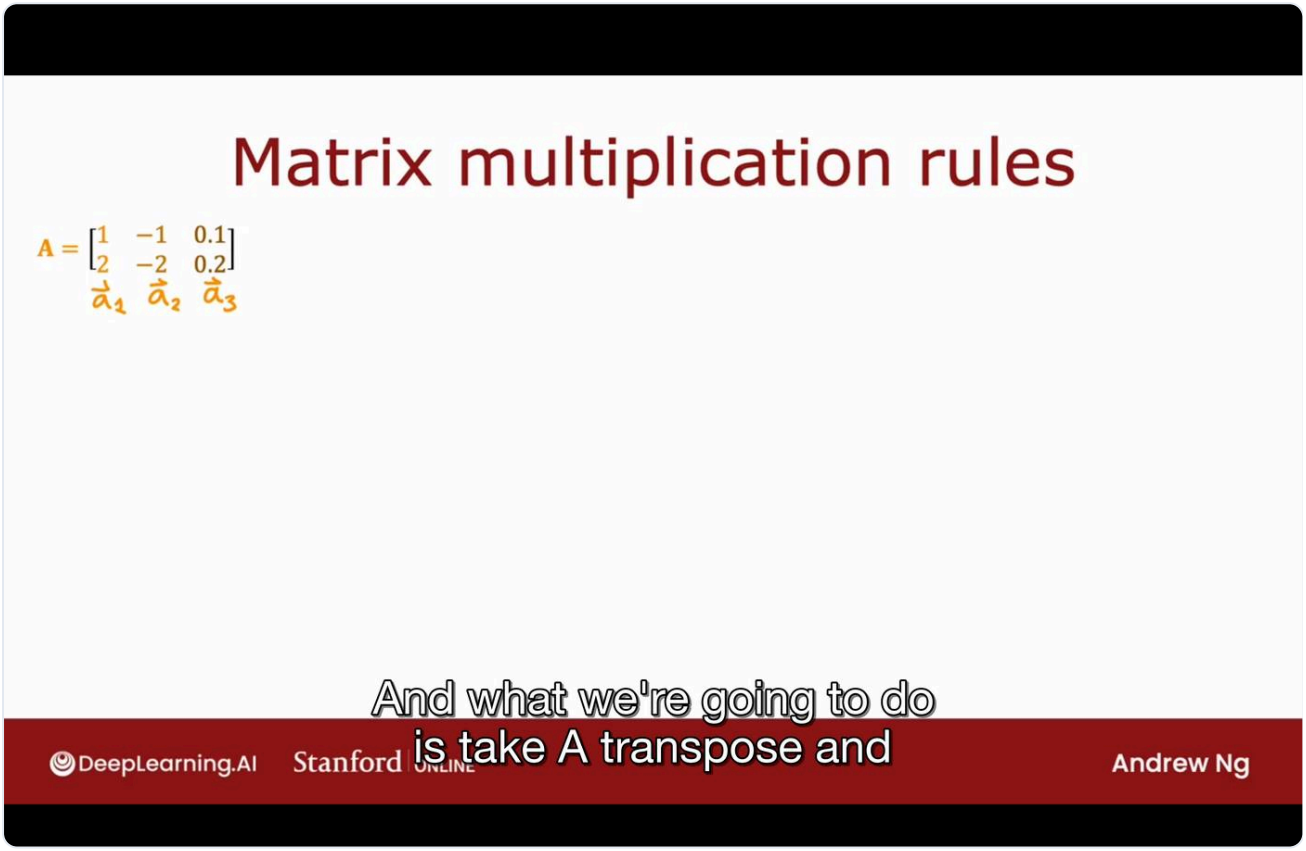


FIGURE 16.1

And what we're going to do is take A transpose and multiply that with the matrix W. The first, what is A transpose? Well, A transpose is obtained by taking the first column of A and laying it on the side like this and then taking the second column of A and laying on his side like this. And then the third column of A and laying on the side like that.

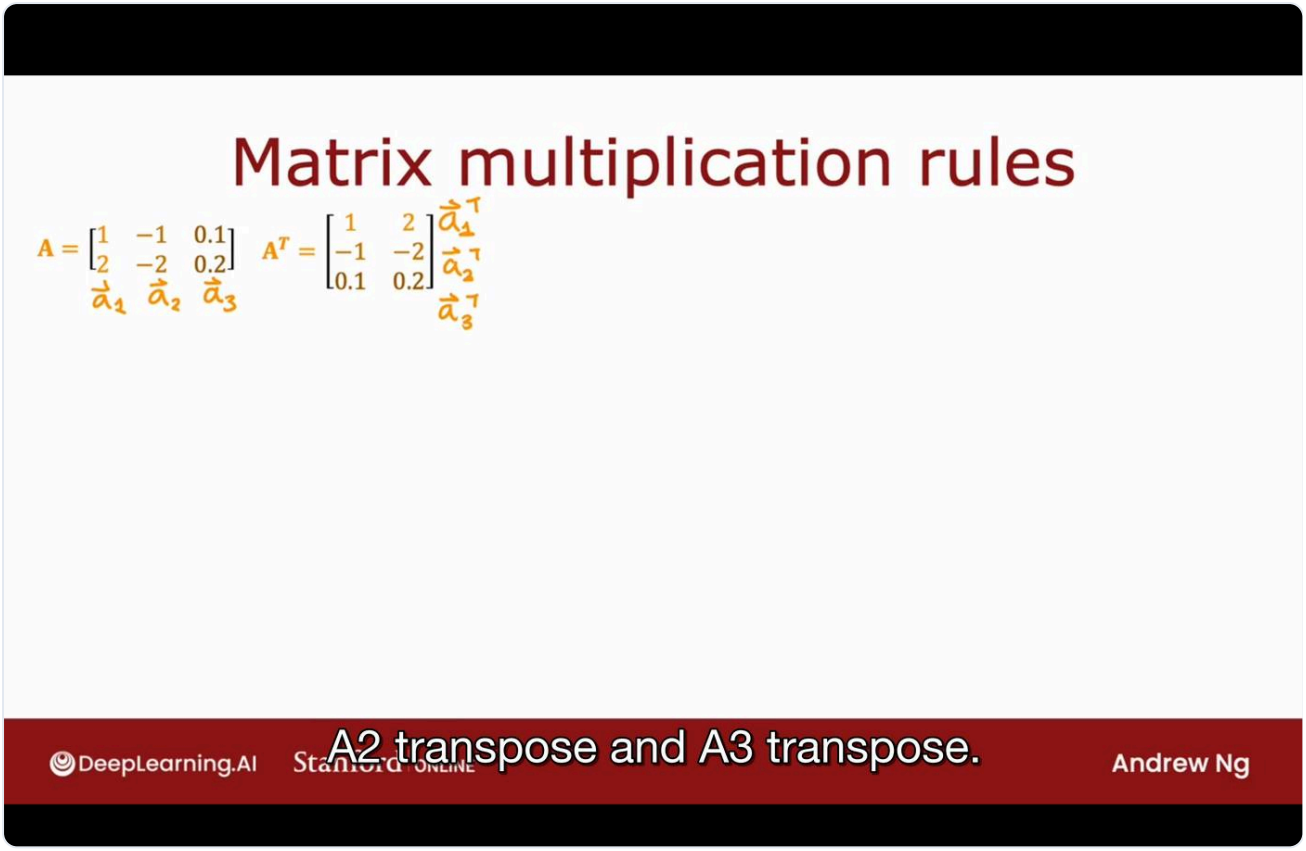


FIGURE 16.2

And so these rows are now A1 transpose, A2 transpose and A3 transpose. Next, here's the matrix W. I encourage you to think of W as factors w1, w2, w3, and w4 stacked together. As so let's look at how you then compute A transpose times W. Now, notice that I've also used slightly different shades of orange to denote the different columns of A, where the same shade corresponds to numbers that we think of as grouped together into a vector. And that same shade is used to indicate different roles of A transpose because the different roles of A transpose are A1 transpose, A2 transpose and A3 transpose. And in a similar way, I've used different shades to denote the different columns of W.

Because the numbers are the same shade of blue, are the ones that are grouped together to form the vectors w1, w 2, or w3 or w4. Now, let's look at how you can compute A transpose times W.

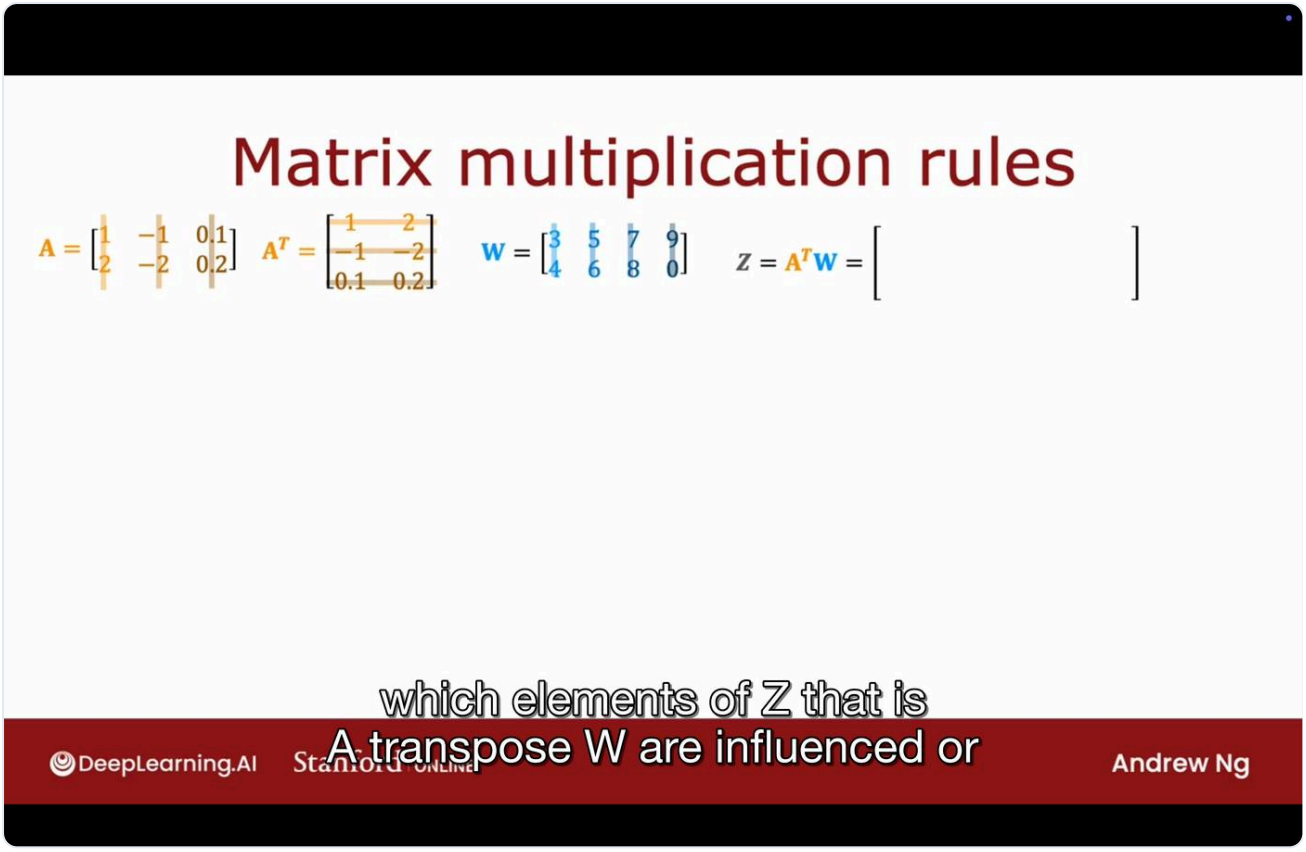


FIGURE 16.3

I'm going to draw vertical bows to the different shades of blue and horizontal bars with the different shades of orange to indicate which elements of Z that is A transpose W are influenced or affected by the different roles of A transpose and which are influenced or affected by the different columns of W. So for example, let's look at the first Column of W. So that's w1 as indicated by the lightest shade of blue here. So w1 will influence or will correspond to this first column of Z shown here by this lighter shade of blue. And the values of this second column of W that is w2 as indicated by this second lighter shade of blue will affect the values computed into second column of Z and so on for the third and fourth columns.

Correspondingly, let's look at A transpose. A1 transpose is the first row of A transpose as indicated by the lightest shade of orange and A1 transpose will effect or influence or correspond to the values in the first row of Z.

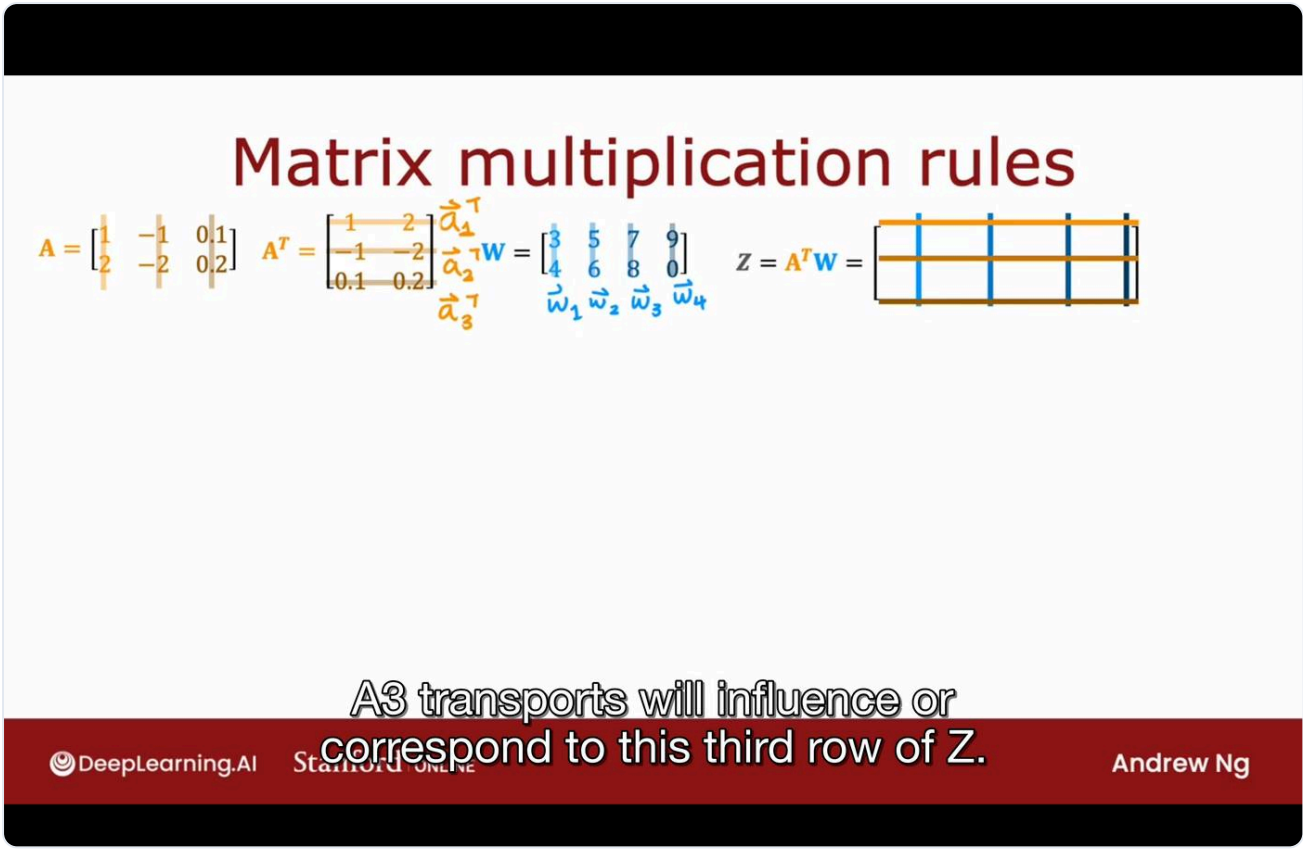


FIGURE 16.4

And A2 transpose will influence the second row of Z and A3 transports will influence or correspond to this third row of Z. So let's figure out how to compute the matrix Z, which is going to be a 3 by 4 matrix. So with 12 numbers altogether. Let's start off and figure out how to compute the number in the first row, in the first column of Z.

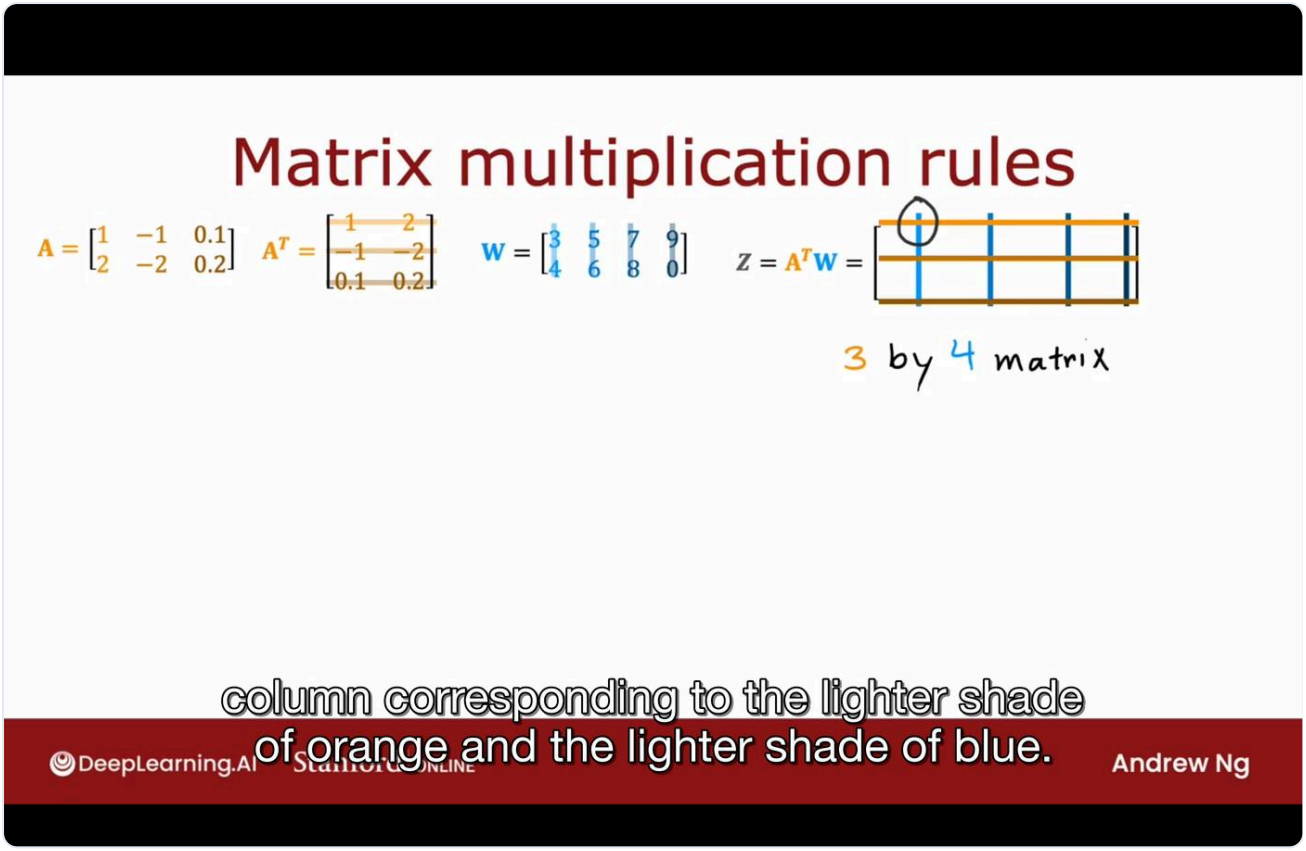


FIGURE 16.5

So this upper left most element here because this is the first row and first column corresponding to the lighter shade of orange and the lighter shade of blue. The way you compute that is to grab the first row of a transpose and the first column of W and take their inner product or the product. And so this number is going to be (1,2) dot product with (3,4) which is (1 * 3) + (2 * 4) = 11. Let's look at the second example. How would you compute this element of Z. So this is in the third row, row 1, row 2, row 3. So this is in row 3 and the second column, column 1, column 2. So to compute the number in row 3, column 2 of Z, you would now grab row 3 of A transpose and column 2 of W and dot product those together.

Notice that this corresponds to the darkest shade of orange and the second lightest shade of blue.

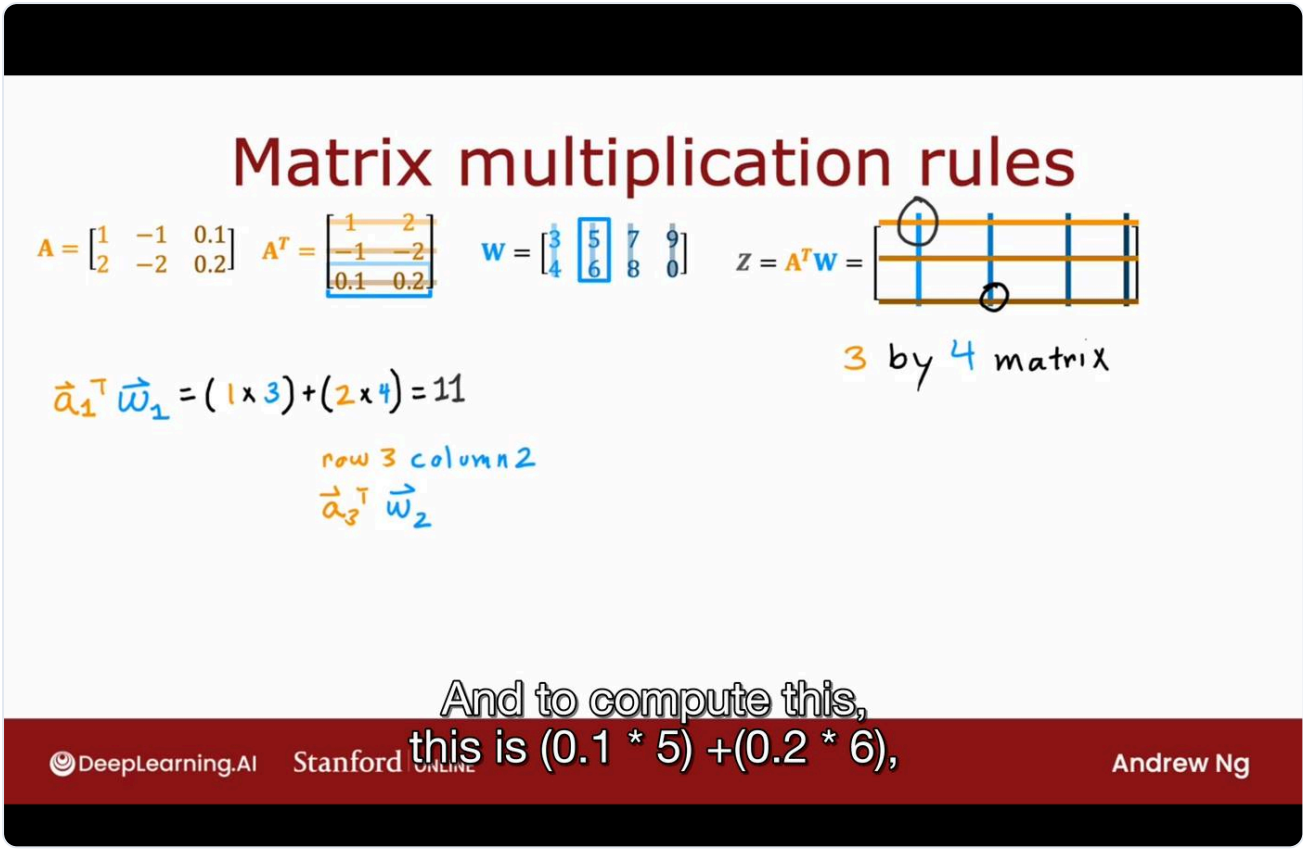


FIGURE 16.6

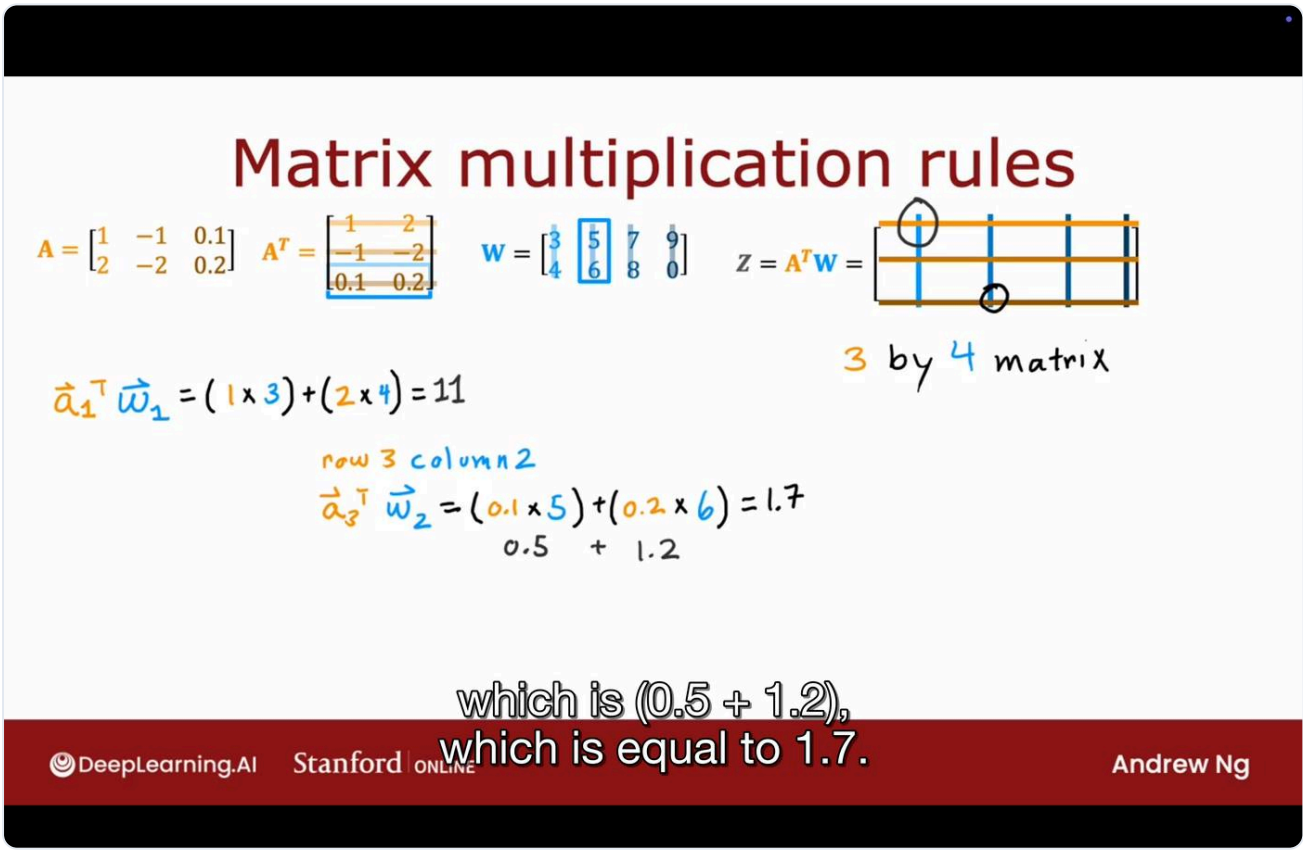


FIGURE 16.7

And to compute this, this is (0.1 * 5) +(0.2 * 6), which is (0.5 + 1.2), which is equal to 1.7. So to compute the number in row 3, column 2 of Z, you grab the third row, row 3 of a transpose and column 2 of W. Let's look at one more example and let's see if you can figure this one out. This is row 2, column 3 of the matrix Z. Why don't you take a look and see if you can figure out which row and which column to grab the dot product together and therefore what is the number that will go in this element of this matrix. Hopefully you got that. You should be grabbing row 2 of A transpose and column 3 of W.

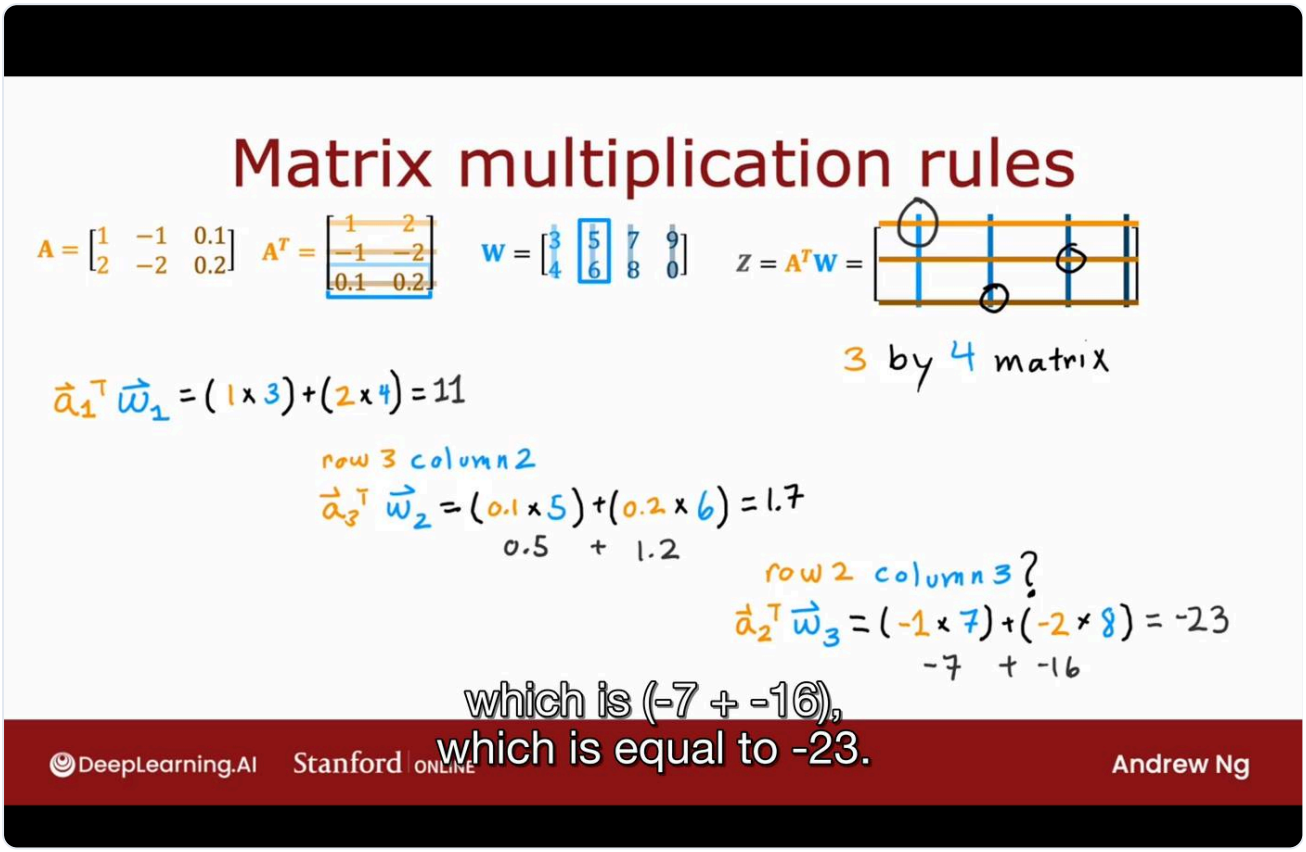


FIGURE 16.8

And when you dot product that together you have A2 transpose w3 is (-1 * 7) + (-2 * 8), which is (-7 + -16), which is equal to -23. And so that's how you compute this element of the matrix Z. And it turns out if you do this for every element of the matrix Z, then you can compute all of the numbers in this matrix which turns out to look like that. Feel free to pause the video if you want and picking the elements and double check that the formula we've been going through gives you the right value for Z. I just want to point out one last interesting requirement for multiplying matrices together, which is that X transpose here is a 3 by 2 matrix because it has 3 rows and 2 columns, and W here is a 2 by 4 matrix because it has 2 rows and 4 columns.

One requirement in order to multiply two matrices together is that this number must match that number. And that's because you can only take dot products between vectors that are the same length.

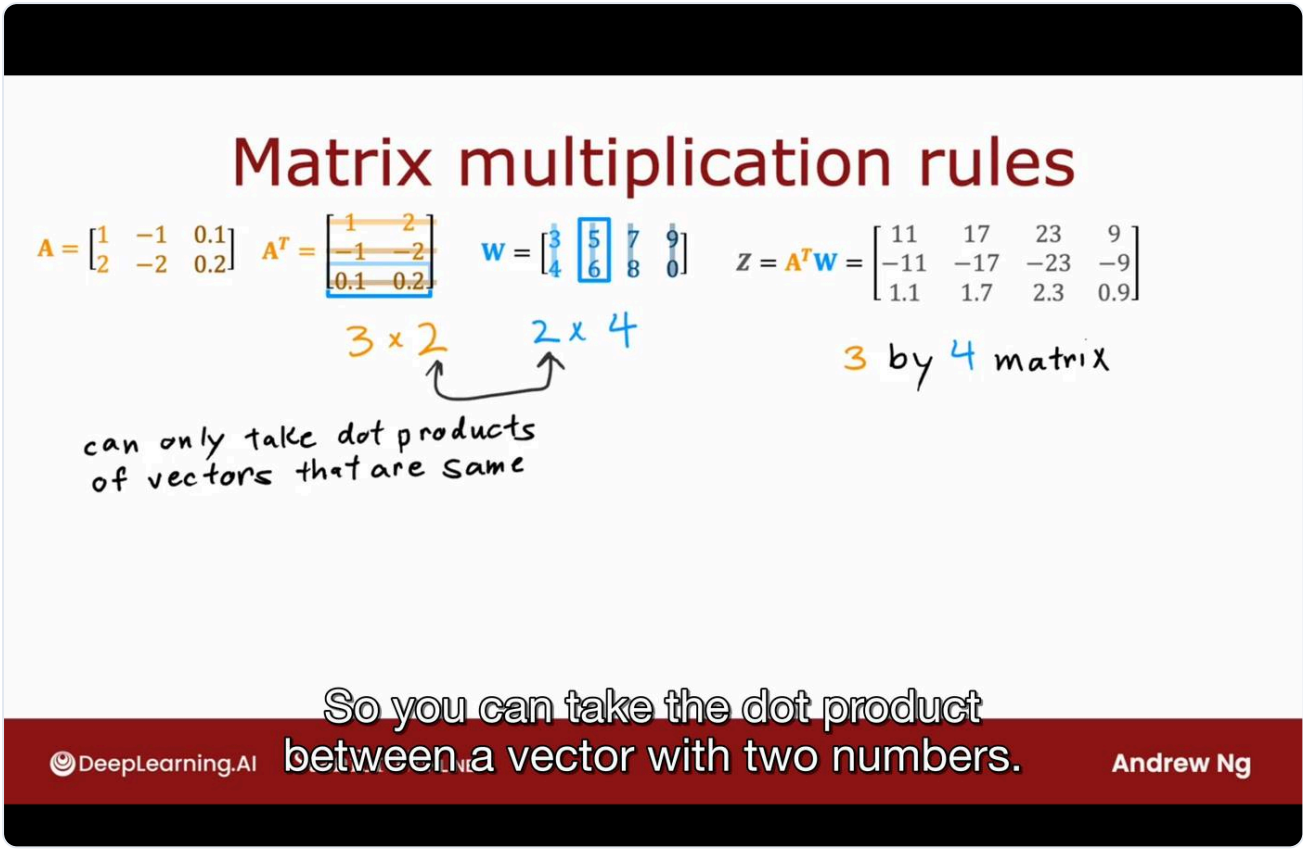


FIGURE 16.9

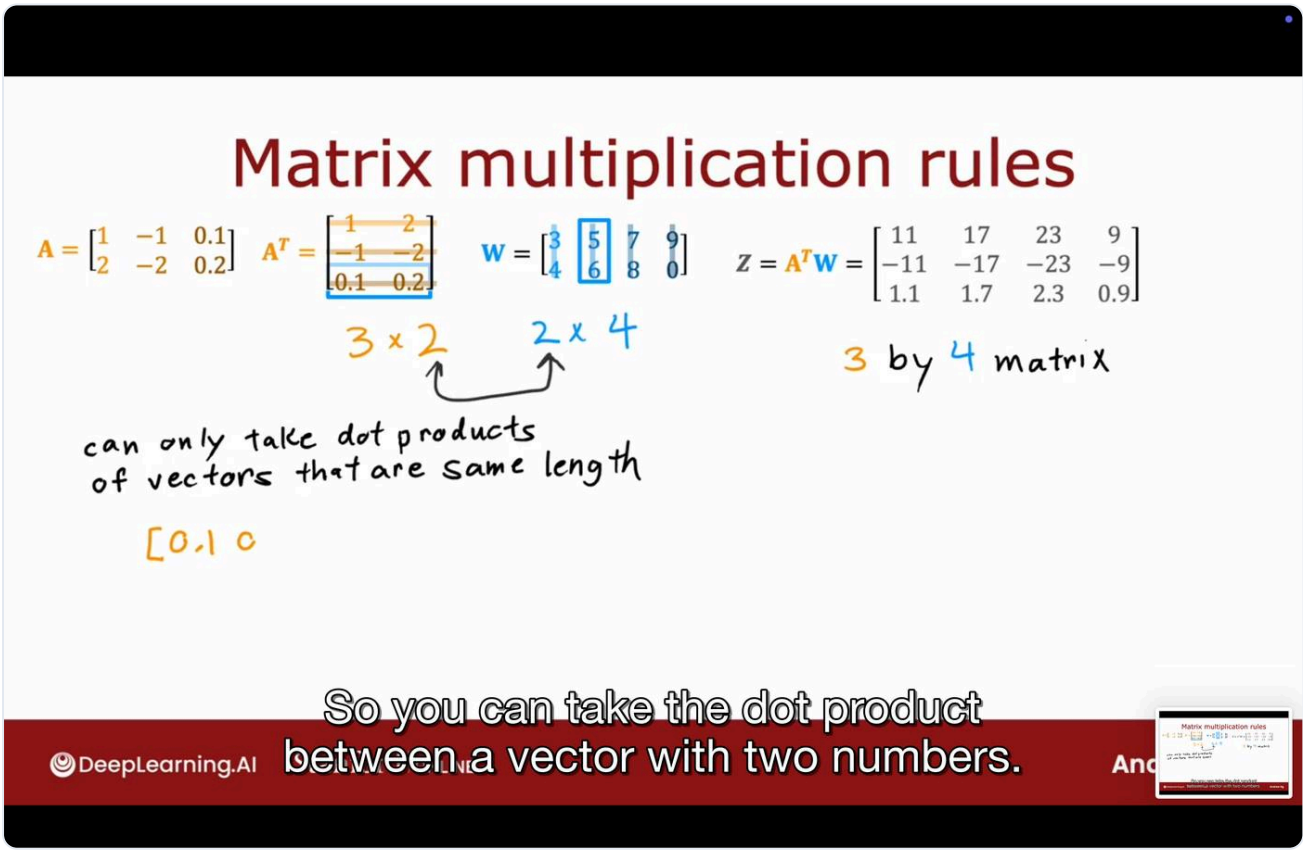


FIGURE 16.10

So you can take the dot product between a vector with two numbers. And that's because you can take the inner product between the vector of length 2 only with another vector of length 2.

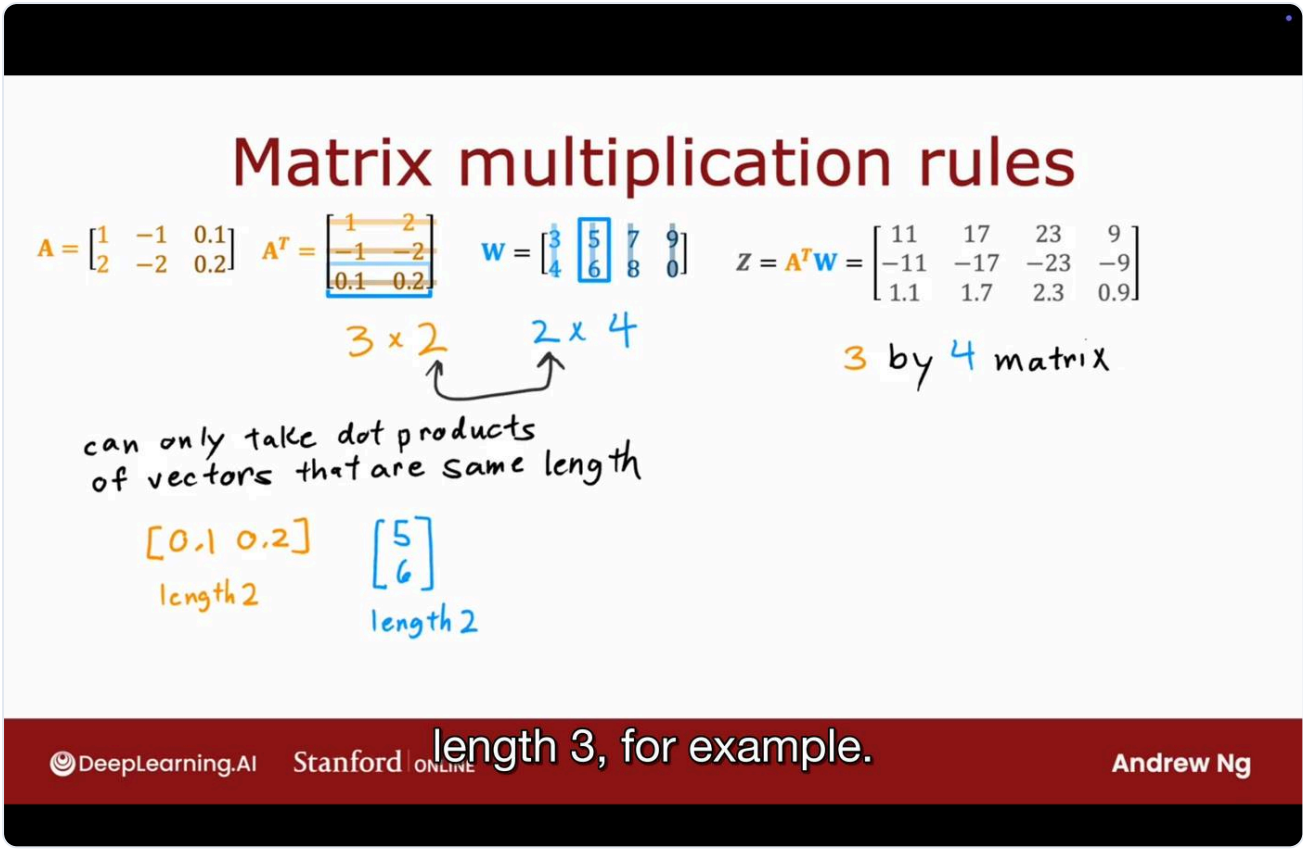


FIGURE 16.11

You can't take the inner product between vector of length 2 with a vector of length 3, for example. And that's why matrix multiplication is valid only if the number of columns of the first matrix, that is A transpose here is equal to the number of rolls of the second matrix, that is the number of rolls of W here. So that when you take dot products during this process, you're taking dot products of vectors of the same size. And then the other observation is that the output Z equals a transpose, W. The dimensions of Z is 3 by 4. And so the output of this multiplication will have the same number of rows as X transpose and the same number of columns as W.

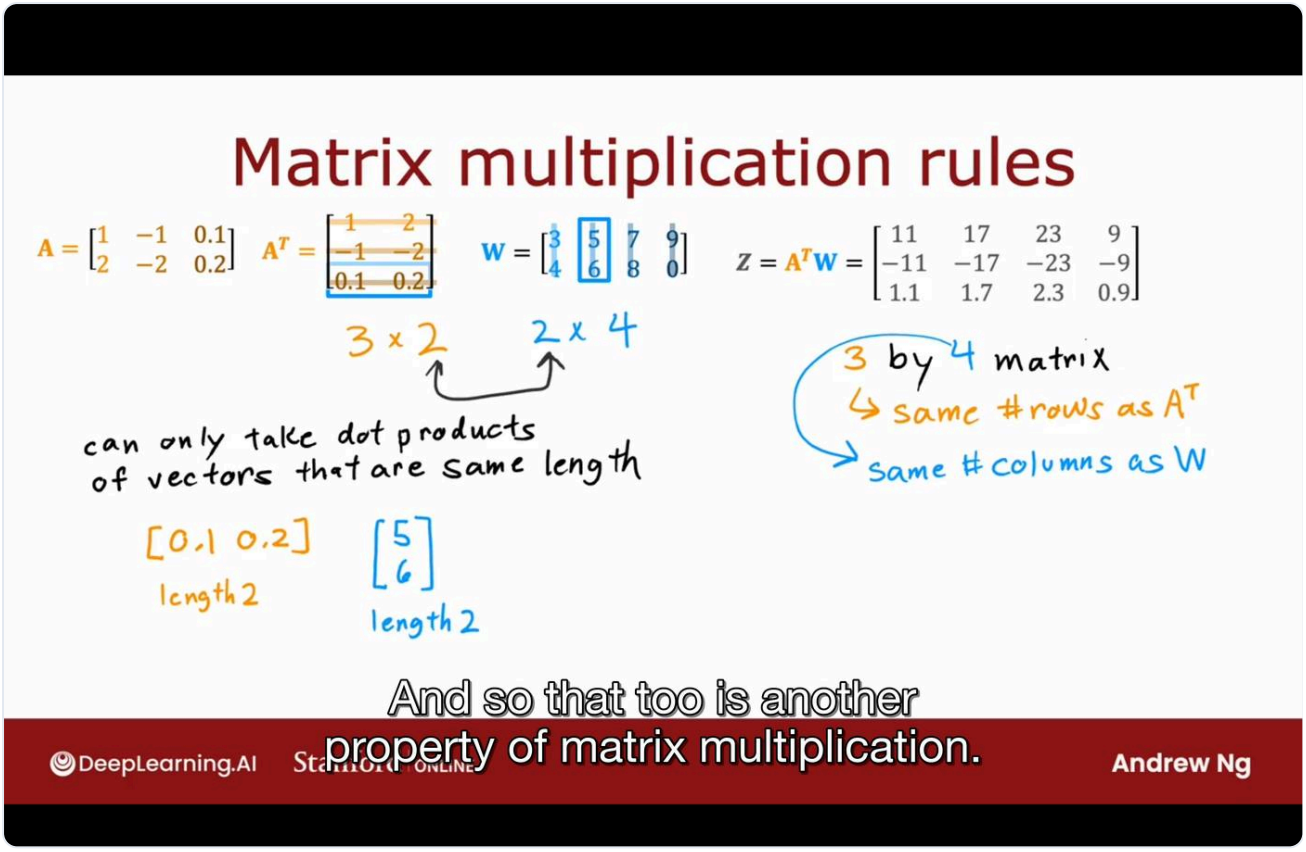


FIGURE 16.12

And so that too is another property of matrix multiplication. So that's matrix multiplication. All these videos are optional. So thank you for sticking with me through these. And if you're interested later in this week, there are also some purely optional quizzes to let you practice some more of these calculations yourself as well. Some of that, let's take what we've learned about matrix multiplication and applied back to the vectorized implementation of a Neural Network. I have to say the first time I understood the vectorized implementation, I thought that's actually really cool. I've been implementing Neural Networks for awhile myself without the vectorized implementation.

And when I finally understood the vectorized implementation and implemented it that way for the first time, it ran blazingly much faster than anything I've ever done before. And I thought, wow, I wish I had figured this out earlier. The vectorized implementation, it is a little bit complicated, but it makes your networks run much faster. So let's take a look at that in the next video

LECTURE 17

Dense Layer Vectorized

Without further ado, let's jump into the vectorize implementation of a neural network. We'll look at the code that you have seen in a earlier video, and hopefully, Matmul, that is that matrix multiplication calculation, will make more sense. Let's jump in. You saw previously how you can take the matrix A and compute A transpose times W resulting in this matrix here, Z. In code if this is the matrix A, this is a NumPy array with the elements corresponding to what I wrote on top, then A transpose, which I'm going to write as AT, is going to be this matrix here, with again the columns of A now laid out in rows instead. By the way, instead of setting up AT this way, another way to compute AT in NumPy, we will write AT equals A.T.

That's the transpose function that takes the columns of a matrix and lays them on the side. In code, here's how you initialize the matrix W as another 2D NumPy array. Then to compute Z equals A transpose times W, you will write Z equals np.matmul, AT, W, and that will compute this matrix Z over here, giving you this result down here.

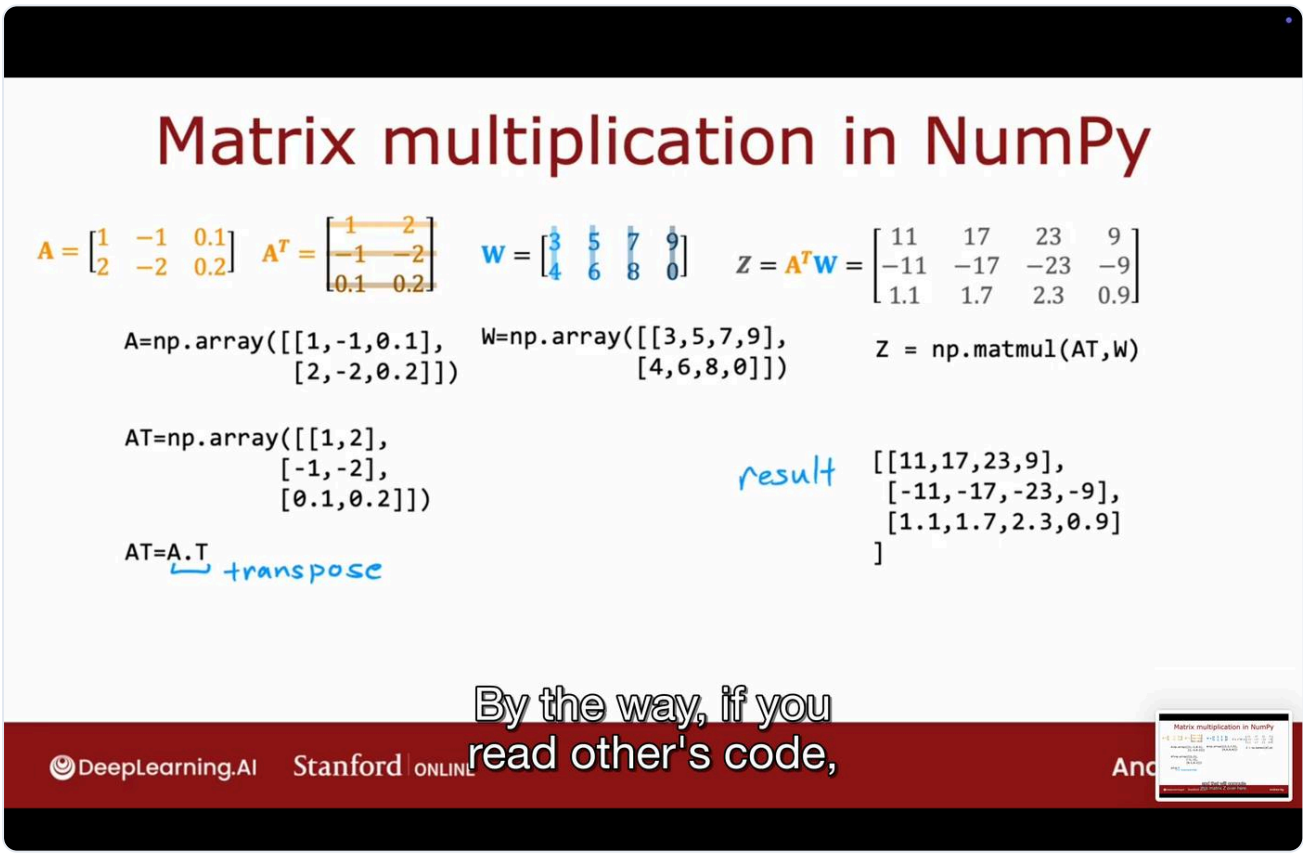


FIGURE 17.1 | Matrix Multiplication in NumPy

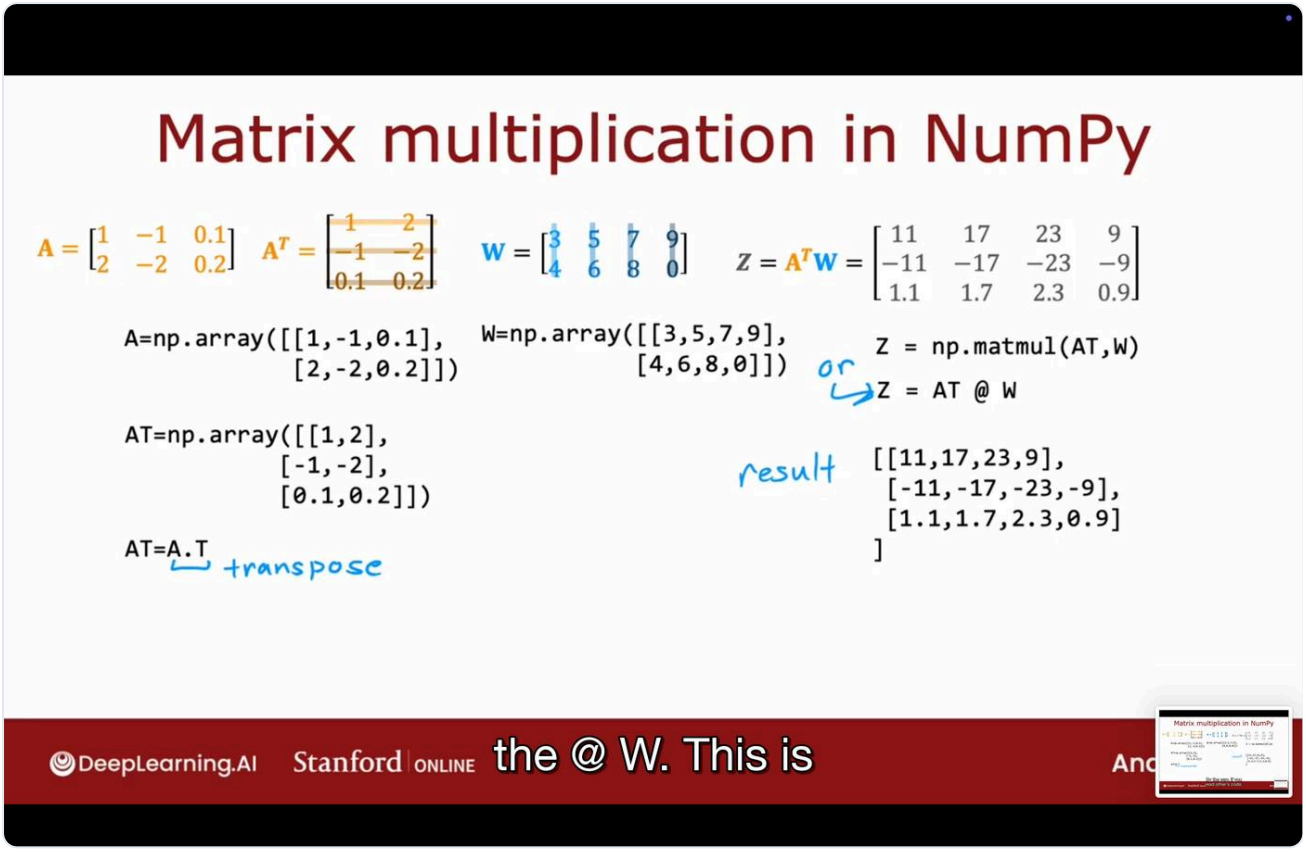


FIGURE 17.2 | Matrix Multiplication in NumPy

By the way, if you read other's code, sometimes you see Z equals AT and then the @ W. This is an alternative way of calling the matmul function. Although I find using np.matmul to be clearer. The call you see in this class, we just use the matmul function like this rather than this @. Let's look at what a vectorized implementation of forward prop looks like. I'm going to set A transpose to be equal to the input feature values 217. These are just the usual input feature values, 200 degrees roasting coffee for 17 minutes. This is a one by two matrix.

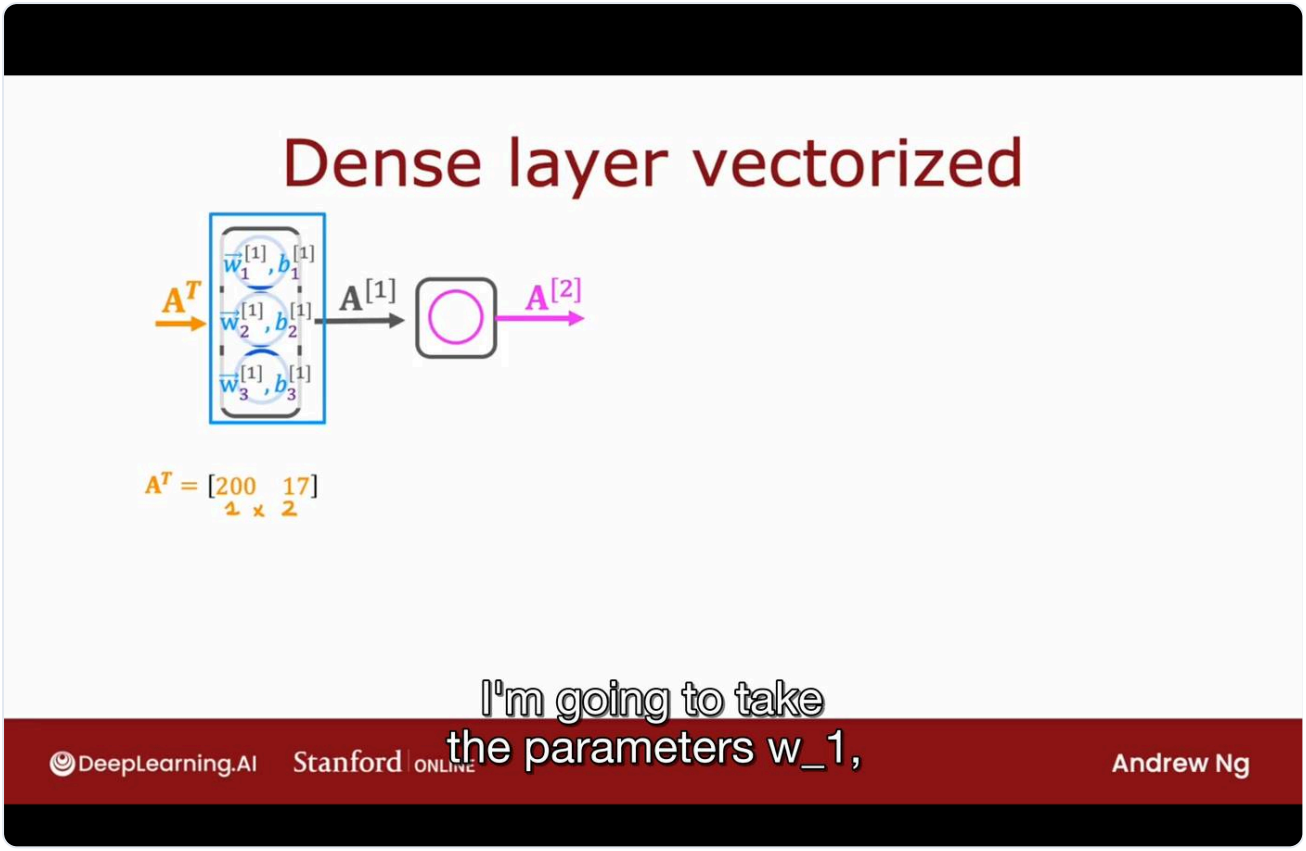


FIGURE 17.3 | Dense Layer Vectorized

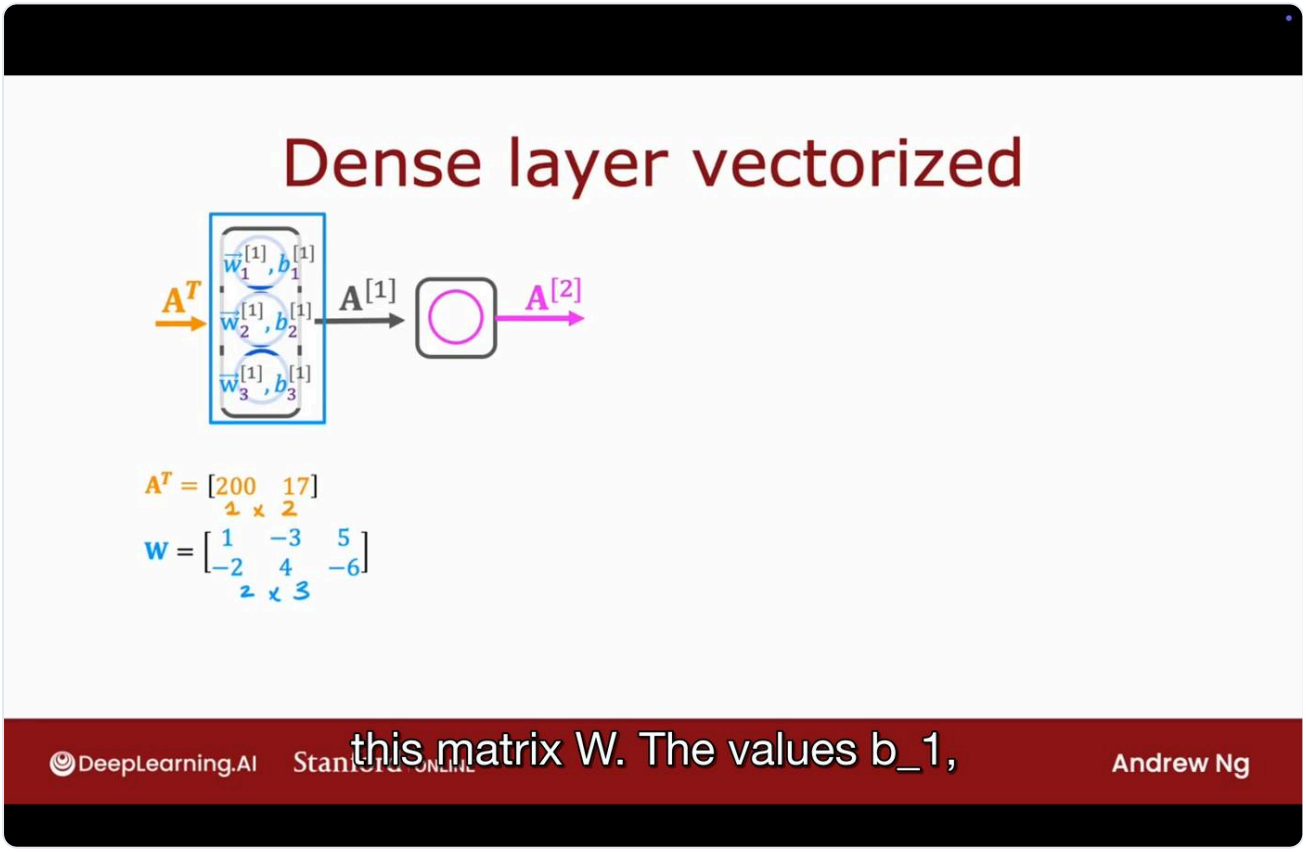


FIGURE 17.4 | Dense Layer Vectorized

I'm going to take the parameters w_1, w_2, and w_3, and stack them in columns like this to form this matrix W. The values b_1, b_2, b_3, I'm going to put it into a one by three matrix, that is this matrix B as follows. Then it turns out that if you were to compute Z equals A transpose W plus B, that will result in these three numbers and that's computed by taking the input feature values and multiplying that by the first column and then adding B to get 165.

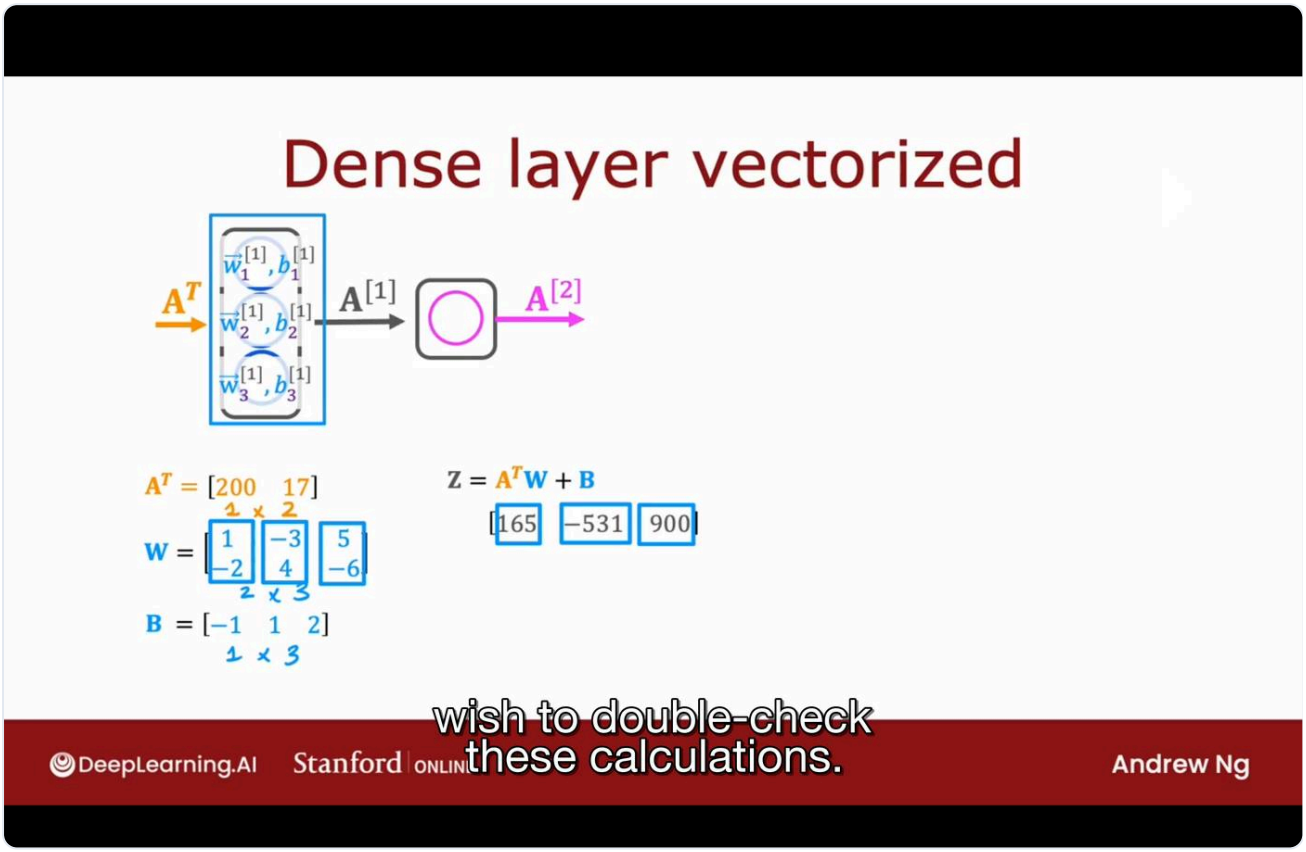


FIGURE 17.5 | Dense Layer Vectorized

Taking these feature values, dot-producting with the second column, that is a weight w_2 and adding b_2 to get negative 531. These feature values dot product with the weights w_3 plus b_3 to get 900. Feel free to pause the video if you wish to double-check these calculations.

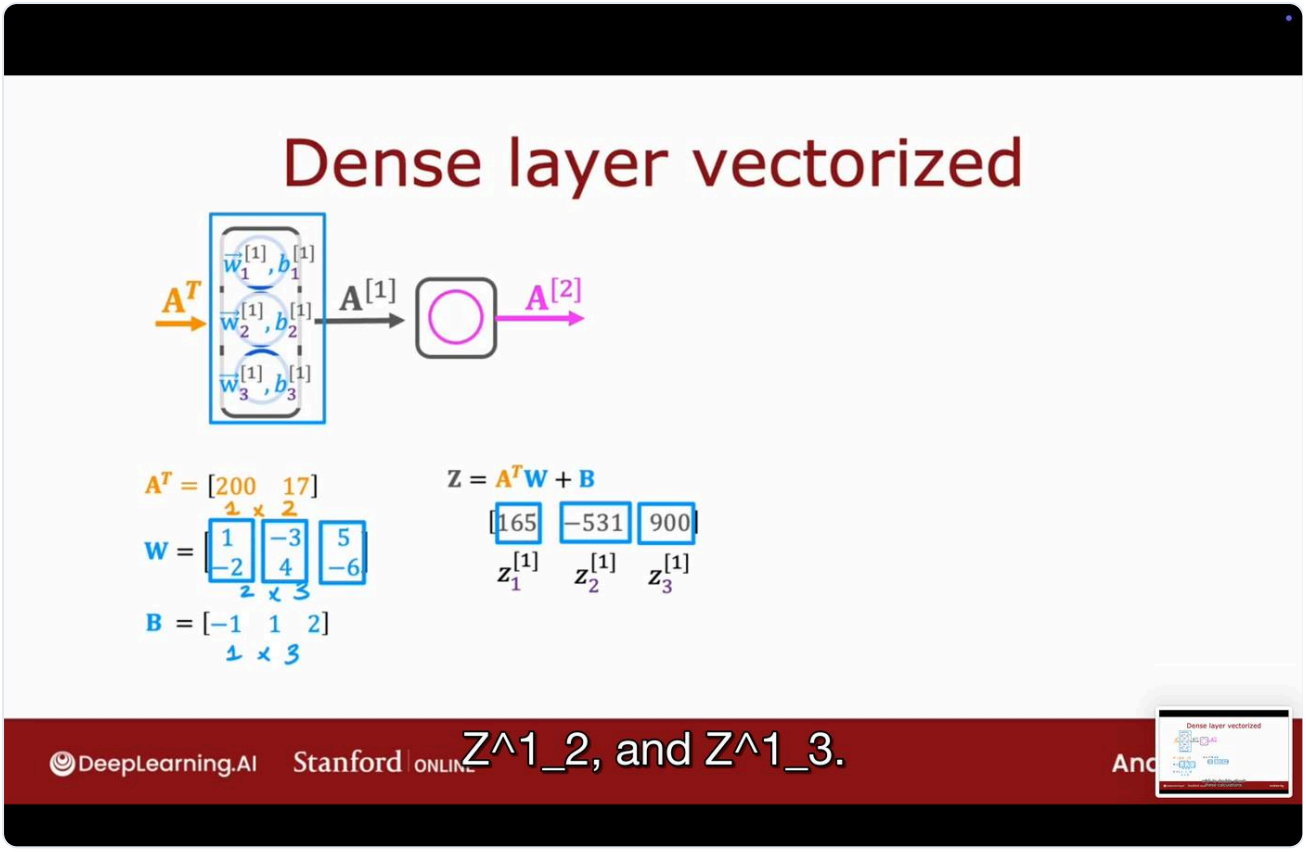


FIGURE 17.6 | Dense Layer Vectorized

But this gives you is the values of z^1_1, Z^1_2, and Z^1_3. Then finally, if the function g applies the sigmoid function to these three numbers element-wise, that is, applies the sigmoid function to 165, to negative 531, and to 900, then you end up with A equals g of this matrix Z ends up being 1,0,1.

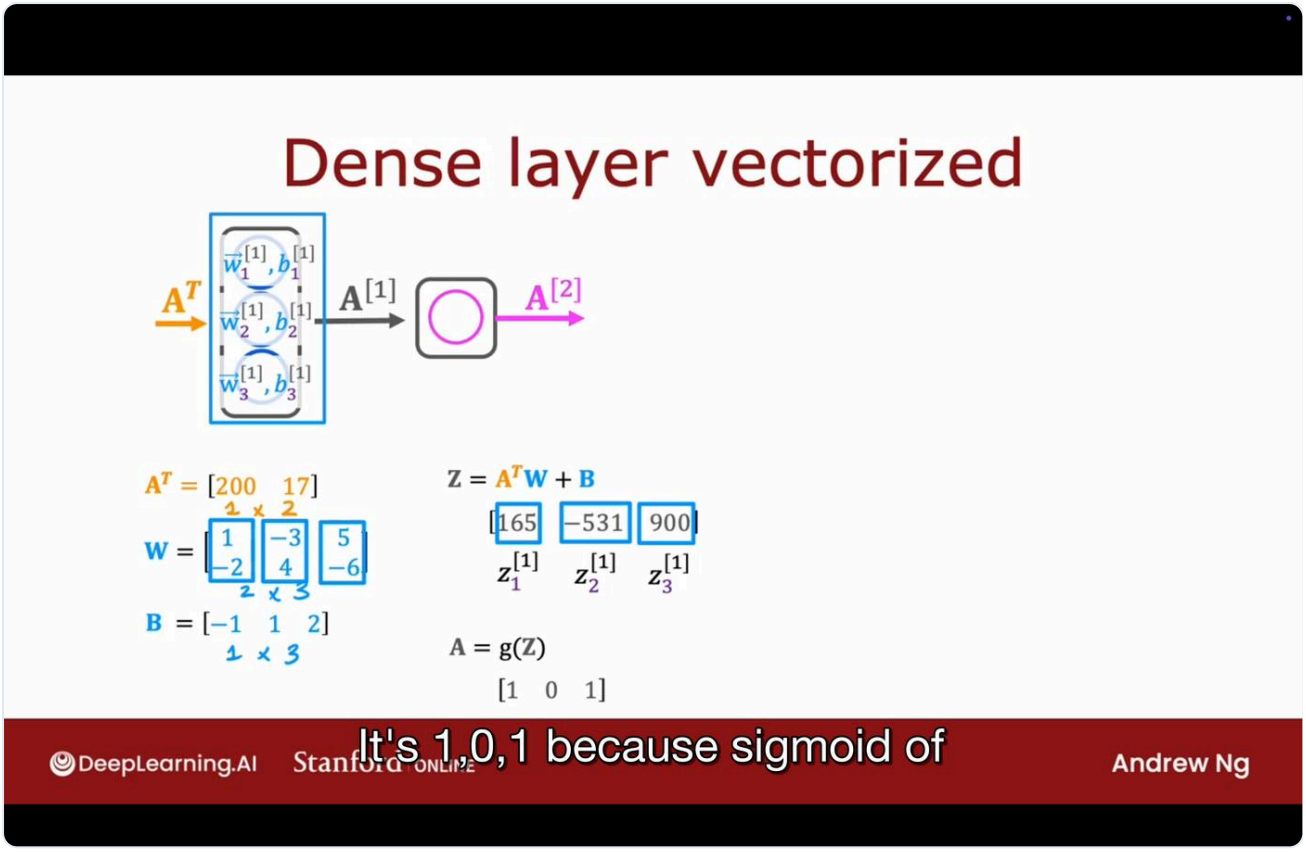


FIGURE 17.7 | Dense Layer Vectorized

It's 1,0,1 because sigmoid of 165 is so close to one that up to numerical round off is based to one and these are bases 0 and 1. Let's look at how you implement this in code. A transpose is equal to this, is this one by two array of 217. The matrix W is this two by three matrix, and B, this is one by three matrix.

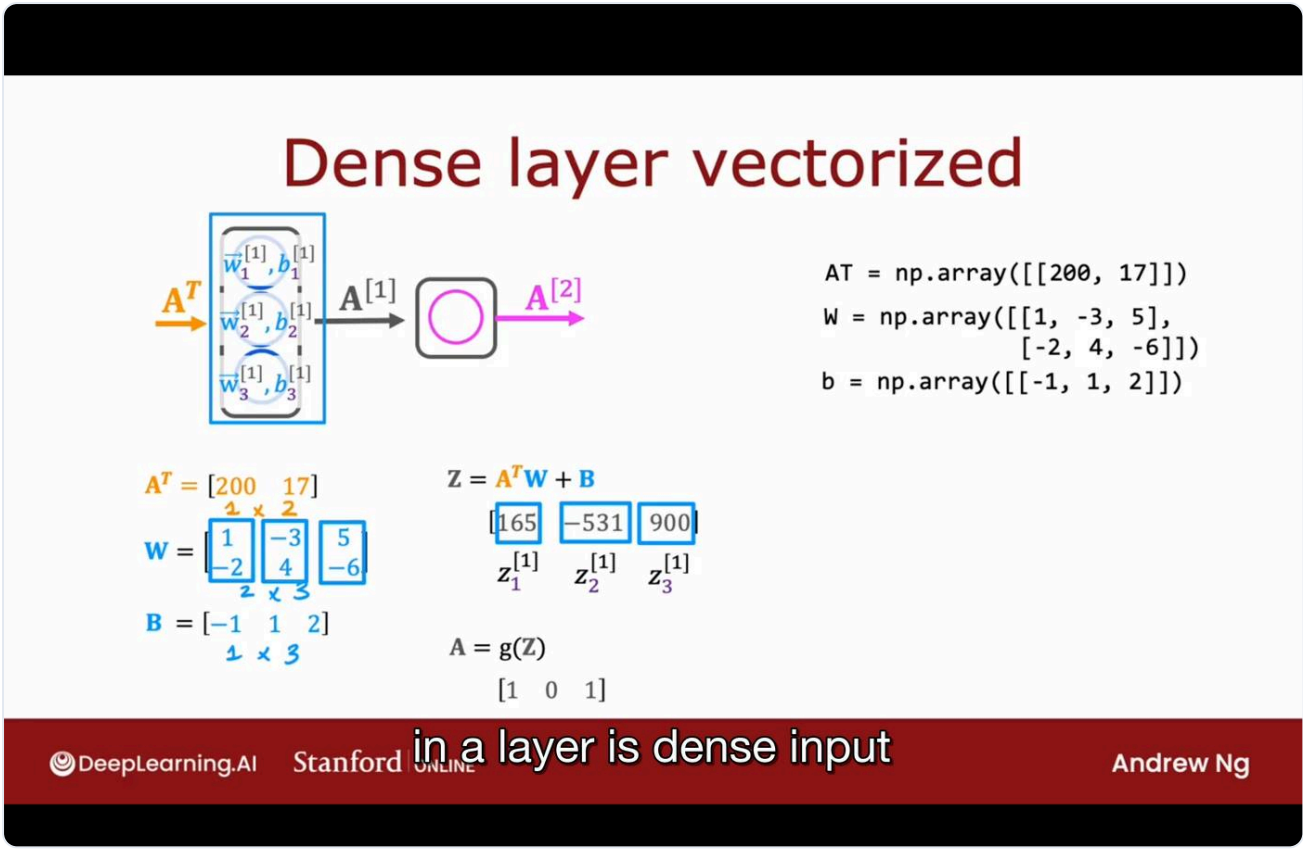


FIGURE 17.8

The way you can implement forward prop in a layer is dense input A transpose W b is equal to z equals matmul A transpose times W plus b. That just implements this line of code. Then a_out that is the output of this layer is equal to g, the activation function applied element-wise to this matrix Z.

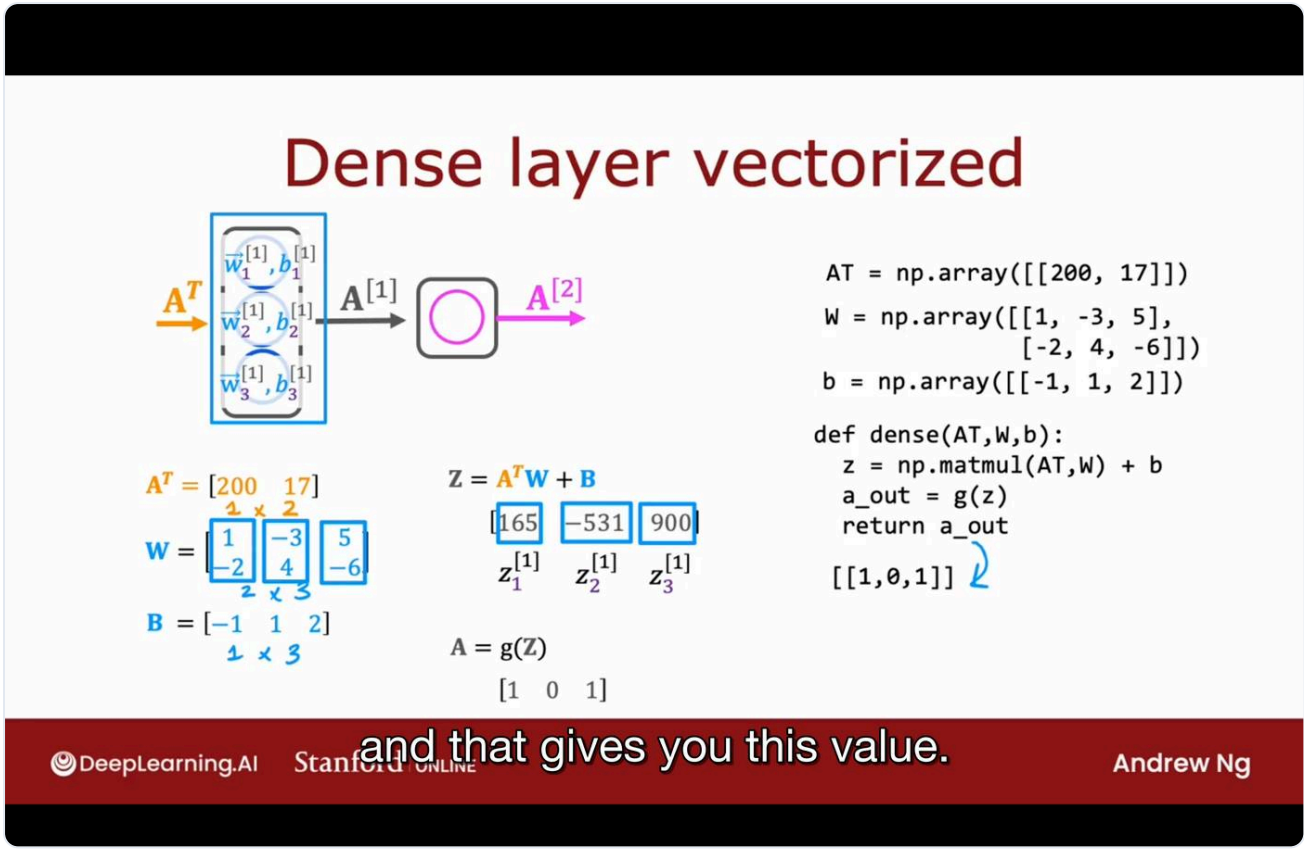


FIGURE 17.9 | Dense Layer Vectorized

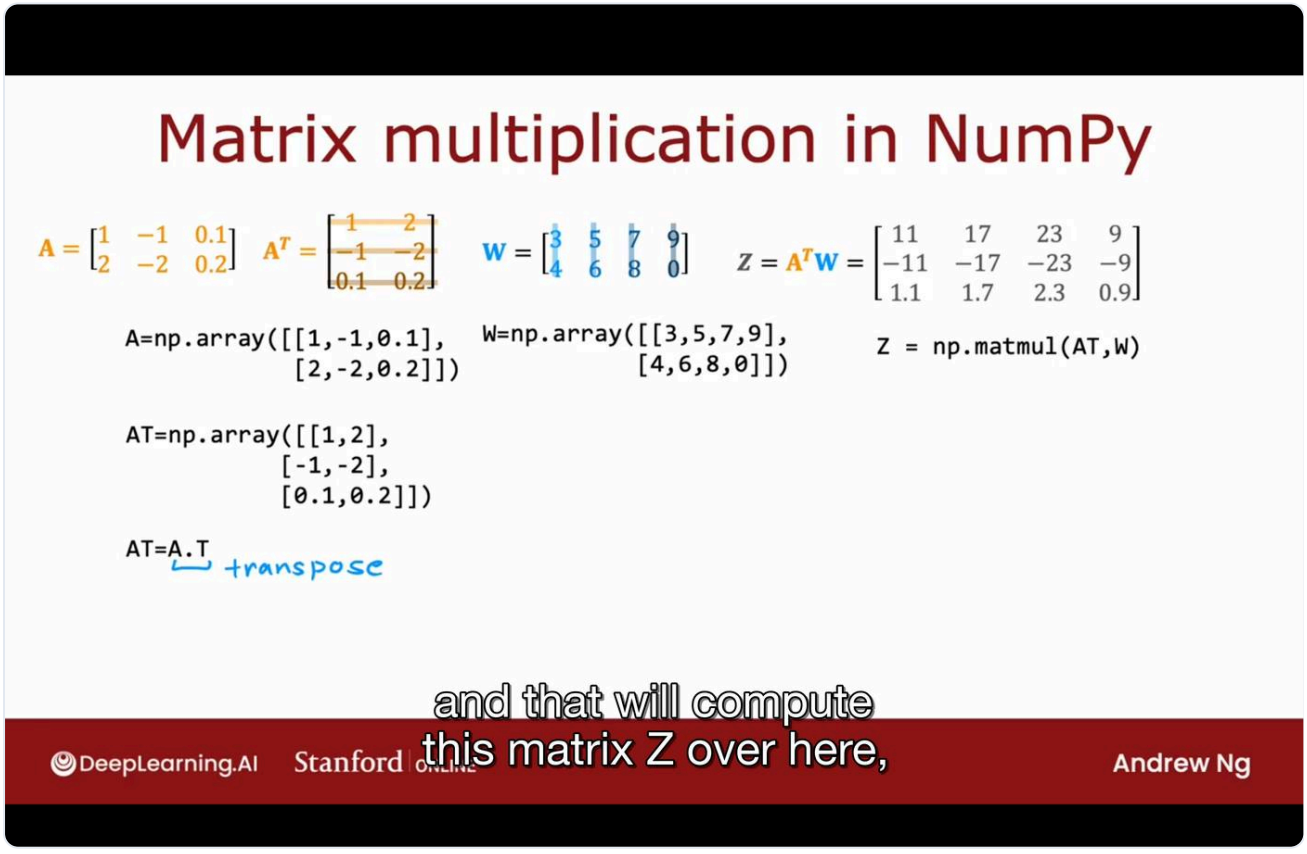


FIGURE 17.10 | Matrix Multiplication in NumPy

You return a_out, and that gives you this value. In case you're comparing this slide with the slide a few videos back, there was just one little difference, which was by convention, the way this is implemented in TensorFlow, rather than calling this variable A,T, we were calling it A_in, which is why this too is the correct implementation of the code. There is a convention in TensorFlow that individual examples are actually laid out in rows in the matrix X rather than in the matrix X transpose which is why the code implementation actually looks like this in TensorFlow. But this explains why with just a few lines of code you can implement forward prop in the neural network and moreover, get a huge bonus because modern computers are very good at implementing matrix multiplications such as matmul efficiently.

That's the last video this week. Thanks for sticking with me all the way through the end of these optional videos. For the rest of this week, I hope you also take a look at the quizzes and the practice labs and also the optional labs to exercise this material even more deeply. You now know how to do inference and forward prop in a neural network, which I think is really cool, so congratulations. After you have gone through the quizzes and the labs, please also come back and in the next week, we'll look at how to actually train a neural network. I look forward to seeing you next week.

LECTURE 18

Train a Neural Network in TensorFlow

Welcome back to the second week of this course on advanced learning algorithms. Last week you learned how to carry out inference in the neural network. This week, we're going to go over training of a neural network. I think being able to take your own data and train your own neural network on it is really fun. This week, we'll look at how you could do that. Let's dive in. Let's continue with our running example of handwritten digit recognition recognizing this image as zero or a one.

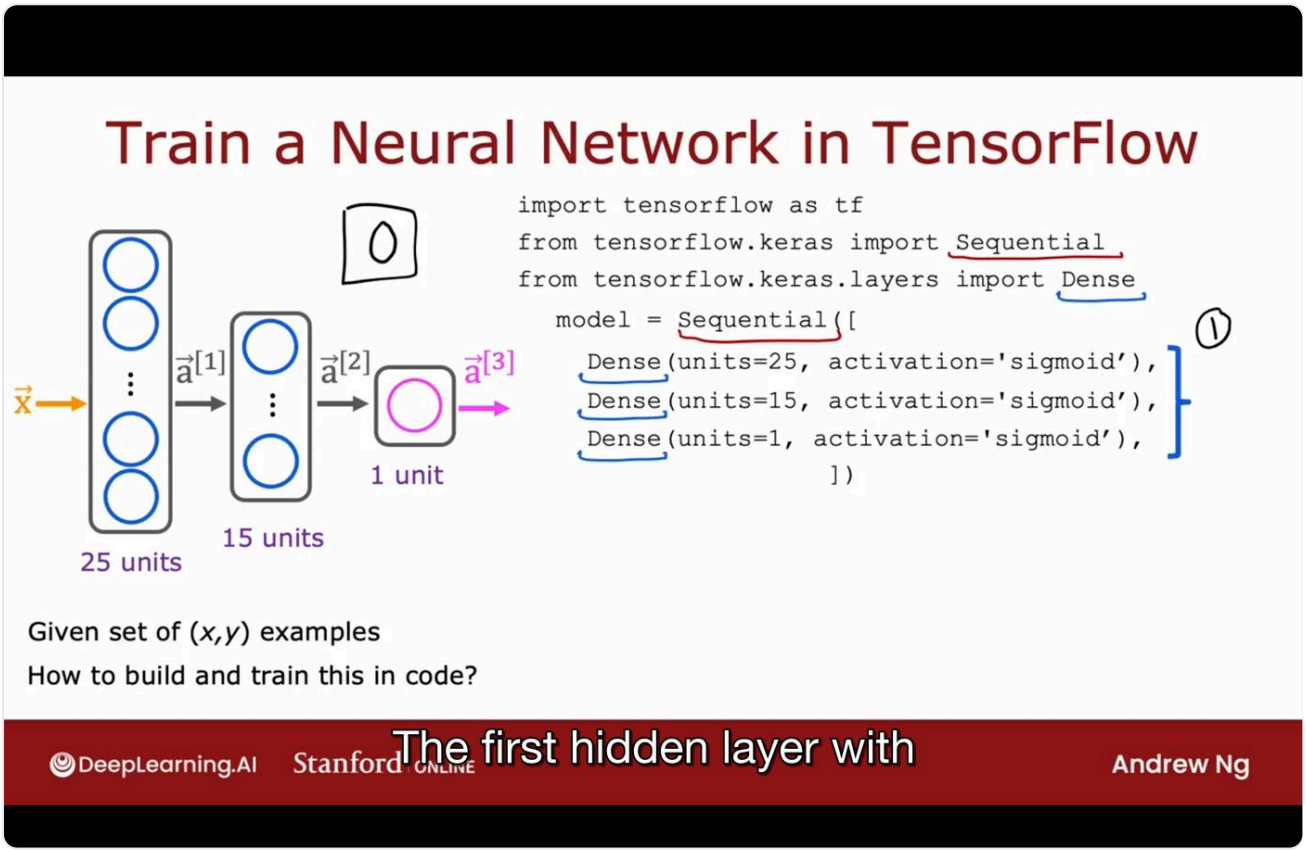


FIGURE 18.1 | Train a Neural Network in TensorFlow

Here we're using the neural network architecture that you saw last week, where you have an input X, that is the image, and then the first hidden layer was 25 units, second hidden layer with 15 units, and then one output unit. If you're given a training set of examples comprising images X, as was the ground truth label Y, how would you train the parameters of this neural network? Let me go ahead and show you the code that you can use in TensorFlow to train this network. Then in the next few videos after this, we'll dive into details to explain what the code is actually doing. This is a code you write. This first part may look familiar from the previous week where you are asking TensorFlow to sequentially string together these three layers of a neural network.

The first hidden layer with 25 units and sigmoid activation, the second hidden layer, and then finally the output layer. Nothing new here relative to what you saw last week. Second step is you're to ask TensorFlow to compile the model.

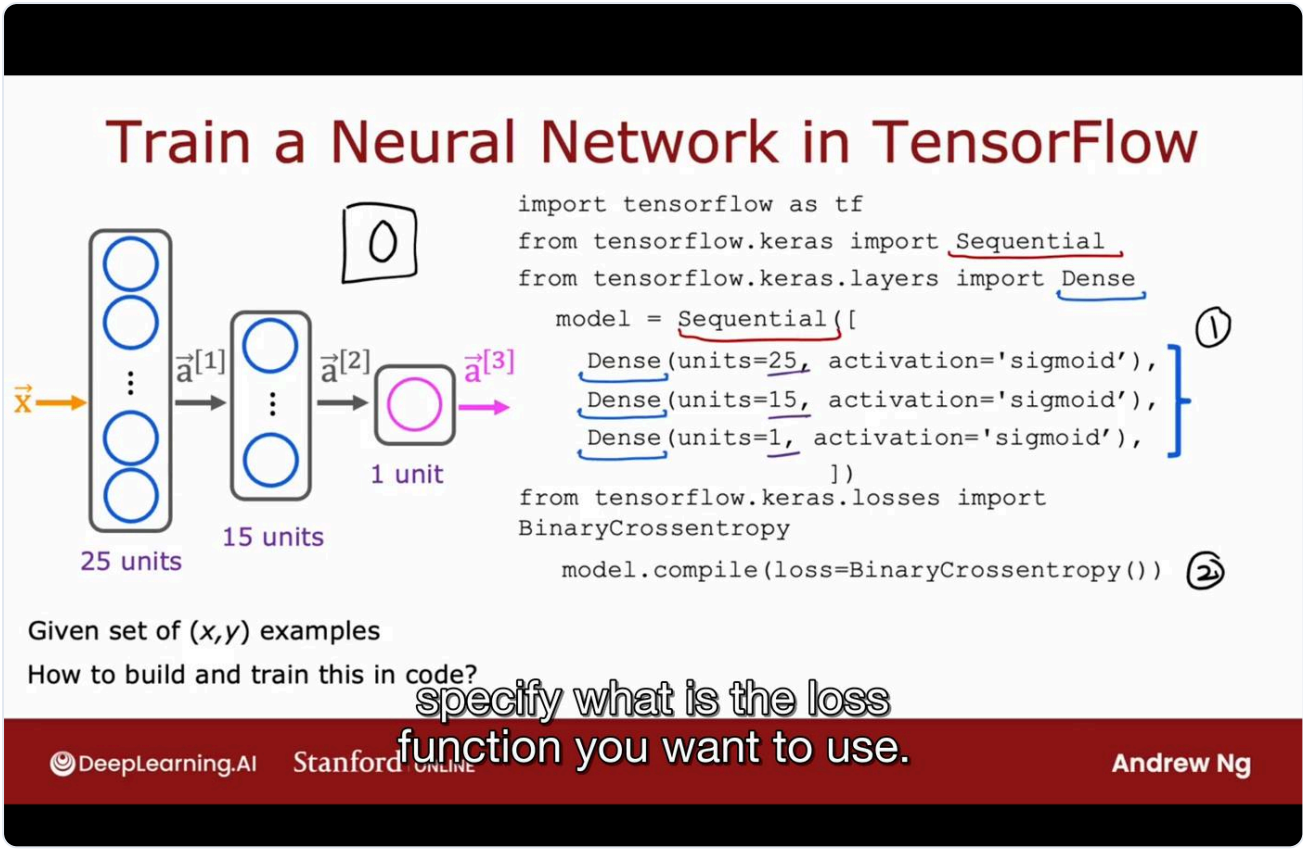


FIGURE 18.2 | Train a Neural Network in TensorFlow

The key step in asking TensorFlow to compile the model is to specify what is the loss function you want to use. In this case we'll use something that goes by the binary crossentropy loss function We'll see more in the next video what this really is.

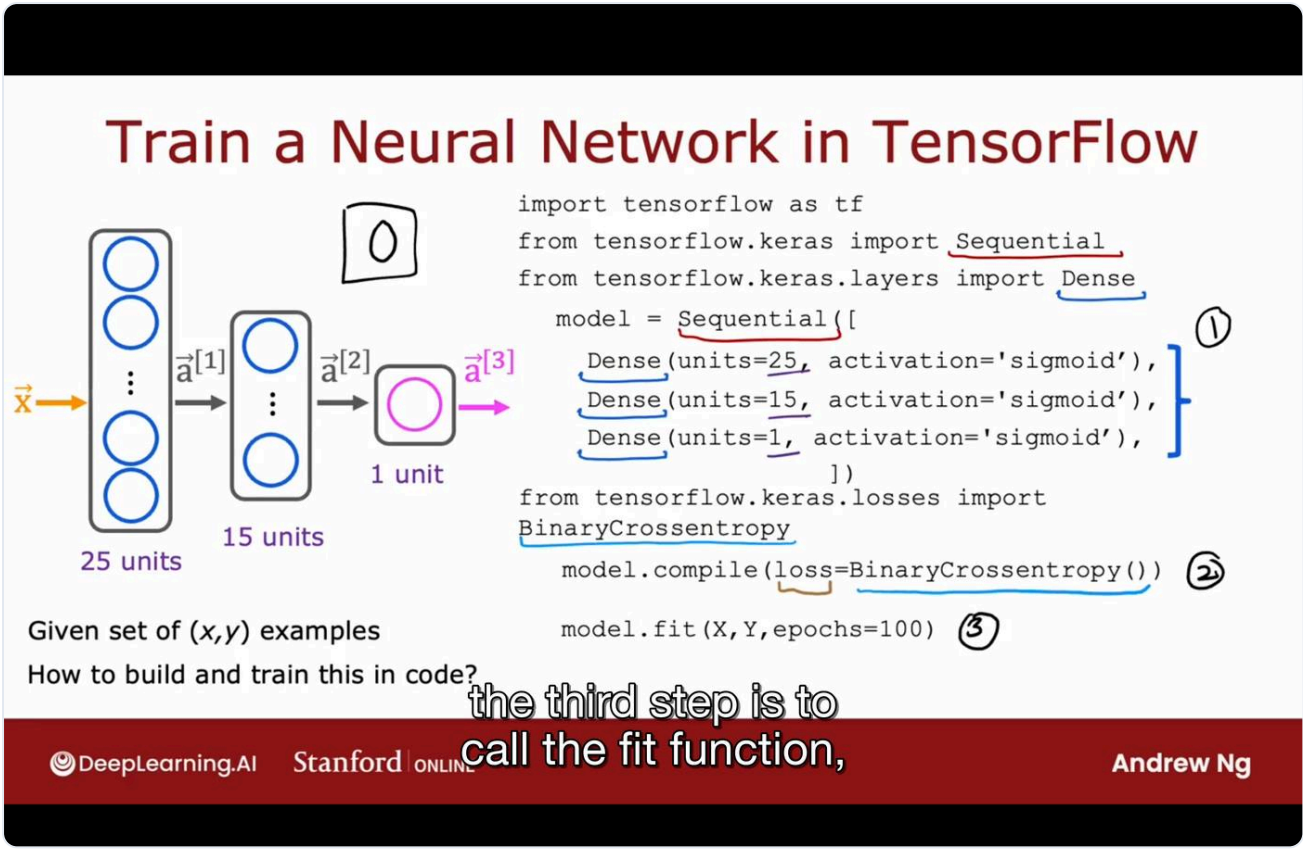


FIGURE 18.3 | Train a Neural Network in TensorFlow

Then having specified the loss function, the third step is to call the fit function, which tells TensorFlow to fit the model that you specified in step 1 using the loss of the cost function that you specified in step 2 to the dataset X, Y.

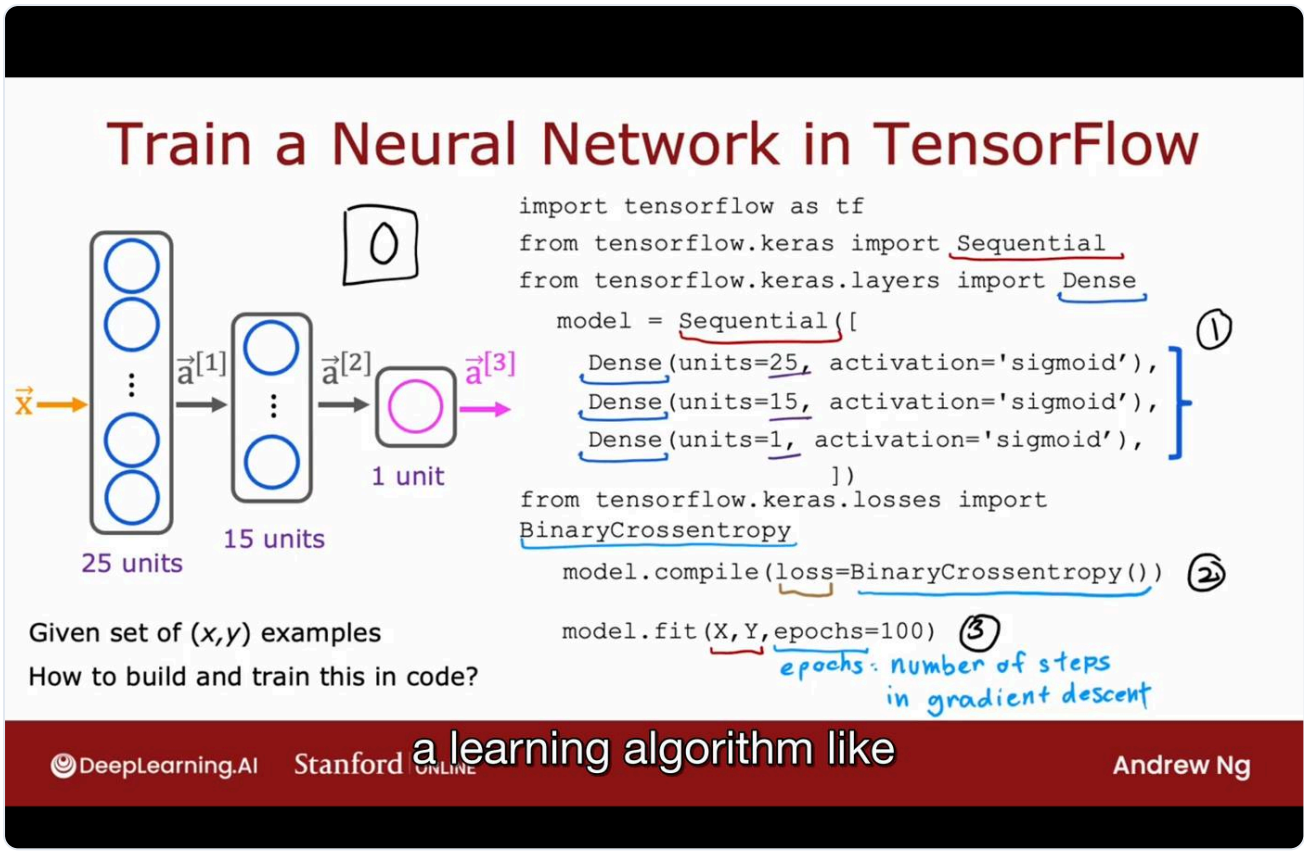


FIGURE 18.4 | Train a Neural Network in TensorFlow

Back in the first course when we talked about gradient descent, we had to decide how many steps to run gradient descent or how long to run gradient descent, so epochs is a technical term for how many steps of a learning algorithm like gradient descent you may want to run. That's it. Step 1 is to specify the model, which tells TensorFlow how to compute for the inference. Step 2 compiles the model using a specific loss function, and step 3 is to train the model. That's how you can train a neural network in TensorFlow. As usual, I hope that you people are not just call these lines of code to train the model, but that you also understand what's actually going on behind these lines of code, so you don't just call it without really understanding what's going on.

I think this is important because when you're running a learning algorithm, if it doesn't work initially, having that conceptual mental framework of what's really going on will help you debug whenever things don't work the way you expect. With that, let's go on to the next video where we'll dive more deeply into what these steps in the TensorFlow implementation are actually doing. I'll see you in the next video.

LECTURE 19

Model Training Steps

Let's take a look at the details of what the TensorFlow code for training a neural network is actually doing. Let's dive in. Before looking at the details of training in neural network, let's recall how you had trained a logistic regression model in the previous course. Step 1 of building a logistic regression model was you would specify how to compute the output given the input feature x and the parameters w and b. In the first course we said the logistic regression function predicts f of x is equal to G. The sigmoid function applied to W. product X plus B which was the sigmoid function applied to W.X plus B. If Z is the dot product of W of X plus B, then F of X is 1 over 1 plus e to the negative z, so those first step were to specify what is the input to output function of logistic regression, and that depends on both the input x and the parameters of the model.

The second step we had to do to train the literacy regression model was to specify the loss function and also the cost function, so you may recall that the loss function said, if religious regression opus f of x and the ground truth label, the actual label and a training set was y then the loss on that single training example was negative y log f of x minus one minus y times log of one minus f of x. This was a measure of how well is logistic regression doing on a single training example x comma y. Given this definition of a loss function, we then define the cost function, and the cost function was a function of the parameters W and B, and that was just the average that is taking an average overall M training examples of the loss function computed on the M training examples, X1, Y1 through XMYM, and remember that in the convention we're using the loss function is a function of the output of the learning algorithm and the ground truth label as computed over a single training example whereas the cost function J is an average of the loss function computed over your entire training set.

That was step two of what we did when building up logistic regression. Then the third and final step to train a logistic regression model was to use an algorithm specifically gradient descent to minimize that cost function J of WB to minimize it as a function of the parameters W and B. We minimize the cost J as a function of the parameters using gradient descent where W is updated as W minus the learning rate alpha times the derivative of J with respect to W. And B similarly is updated as B minus the learning rate alpha times the derivative of J with respect to B.

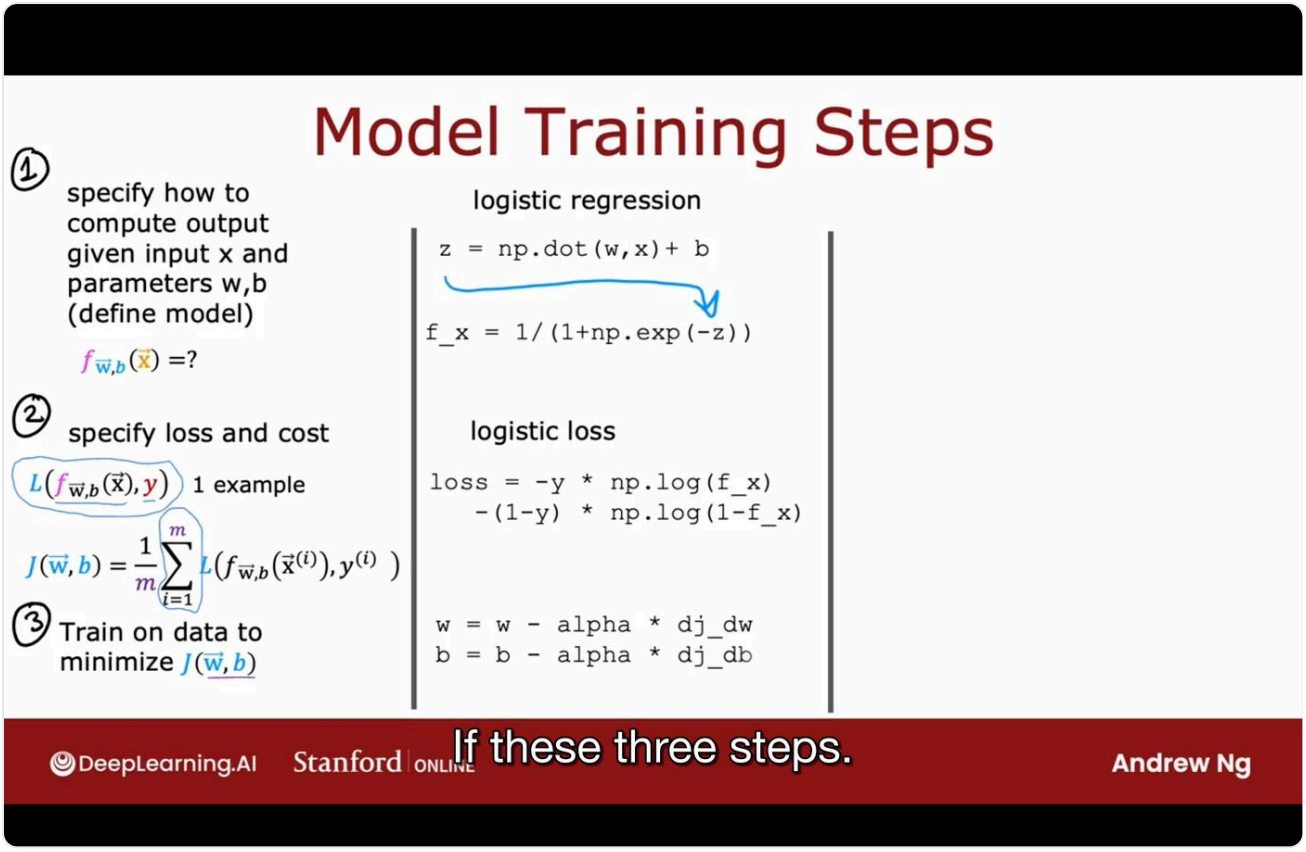


FIGURE 19.1 | Model Training Steps

If these three steps. Step one, specifying how to compute the outputs given the input X and parameters, step 2 specify loss and costs, and step three minimize the cost function we trained logistic regression. The same three steps is how we can train a neural network in TensorFlow. Now let's look at how these three steps map to training a neural network. We'll go over this in greater detail on the next three slides but really briefly. Step one is specify how to compute the output given the input x and parameters W and B that's done with this code snippet which should be familiar from last week of specifying the neural network and this was actually enough to specify the computations needed in forward propagation or for the inference algorithm for example.

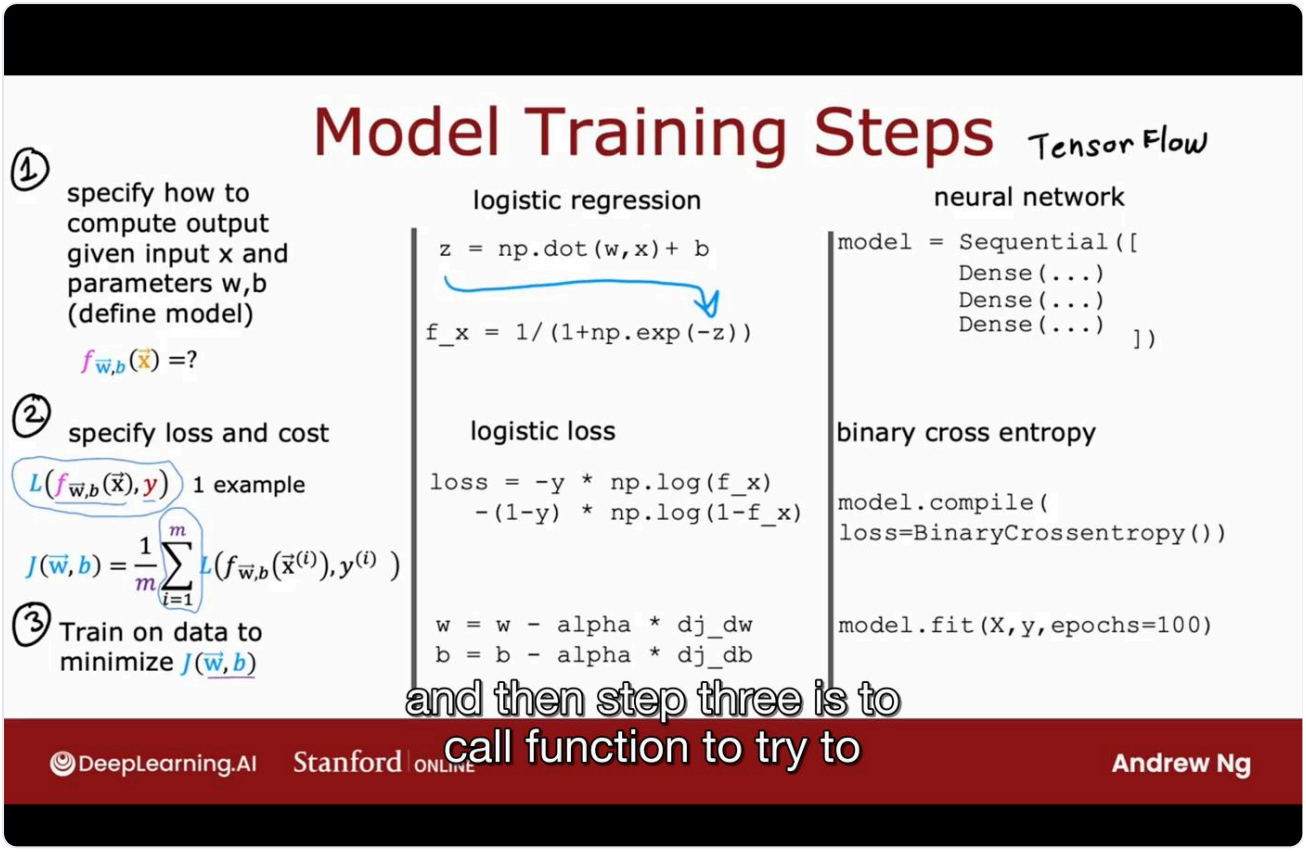


FIGURE 19.2

The second step is to compile the model and to tell it what loss you want to use, and here's the code that you use to specify this loss function which is the binary cross entropy loss function, and once you specify this loss taking an average over the entire training set also gives you the cost function for the neural network, and then step three is to call function to try to minimize the cost as a function of the parameters of the neural network. Let's look in greater detail in these three steps in the context of training a neural network. The first step, specify how to compute the output given the input x and parameters w and b. This code snippet specifies the entire architecture of the neural network.

It tells you that there are 25 hidden units in the first hidden layer, then the 15 in the next one, and then one output unit and that we're using the sigmoid activation value. Based on this code snippet, we know also what are the parameters w1, v1 though the first layer parameters of the second layer and parameters of the third layer.

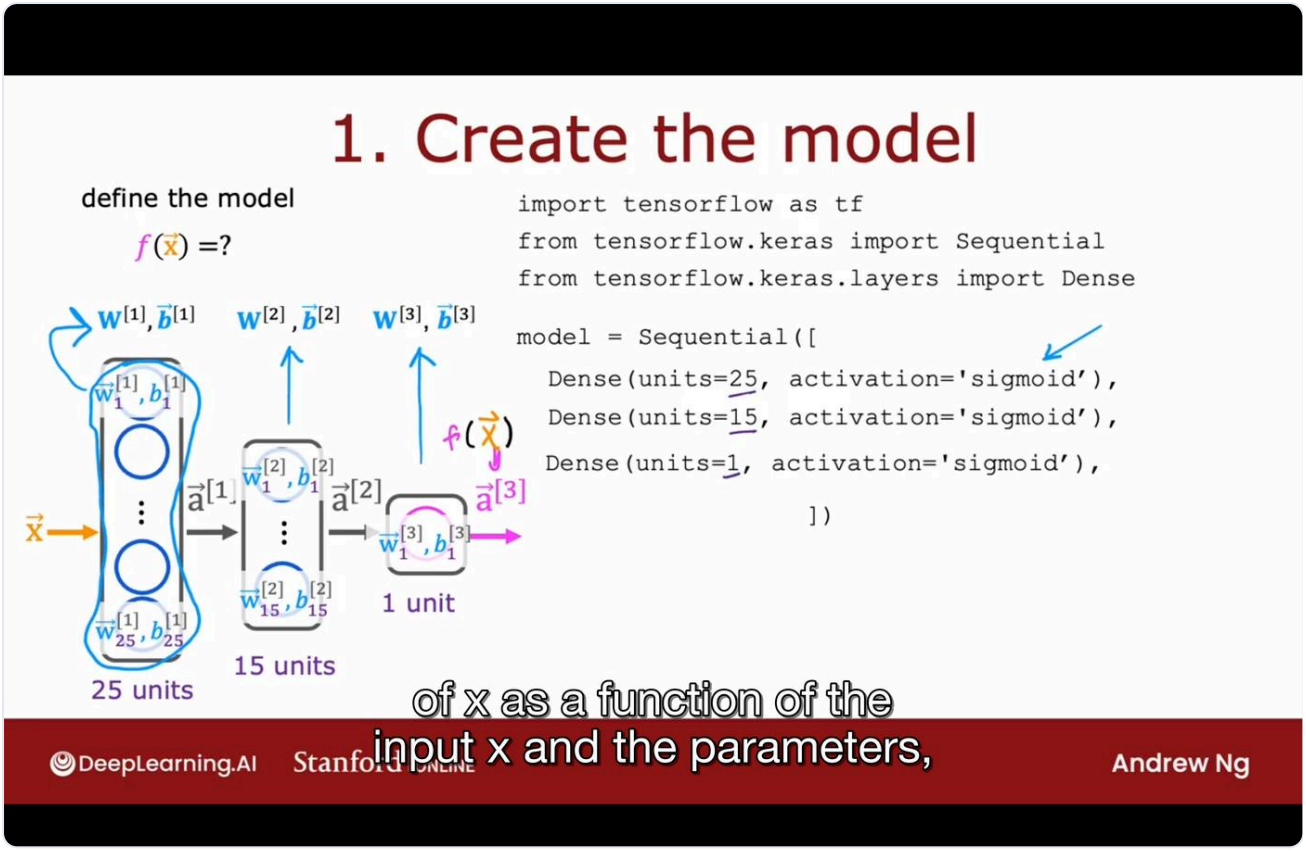


FIGURE 19.3

This code snippet specifies the entire architecture of the neural network and therefore tells TensorFlow everything it needs in In order to compute the output a 3 or f of x as a function of the input x and the parameters, here we have written w l and b l. Let's go on to step 2. In the second step, you have to specify what is the loss function. That will also define the cost function we use to train the neural network. For the handwritten digit classification problem where images are either of a zero or a one the most common by far, loss function to use is this one is actually the same loss function as what we had for logistic regression is negative y log f of x minus 1 minus y times log 1 minus f of x, where y is the ground truth label, sometimes also called the target label y, and f of x is now the output of the neural network.

In TensorFlow, this is called the binary cross-entropy loss function. Where does that name come from?

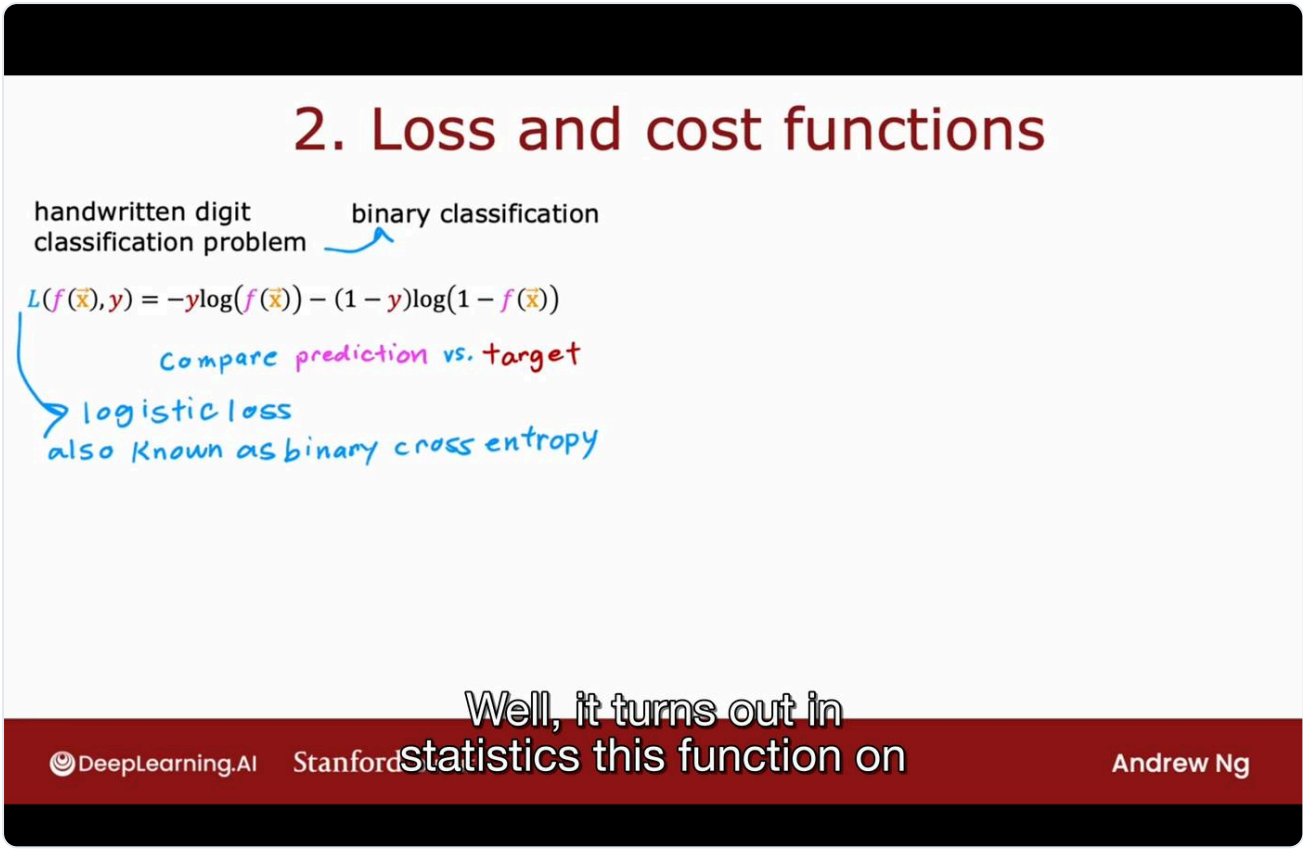


FIGURE 19.4 | 2. Loss and Cost Functions

Well, it turns out in statistics this function on top is called the cross-entropy loss function, so that's what cross-entropy means, and the word binary just reemphasizes or points out that this is a binary classification problem because each image is either a zero or a one. The syntax is to ask TensorFlow to compile the neural network using this loss function.

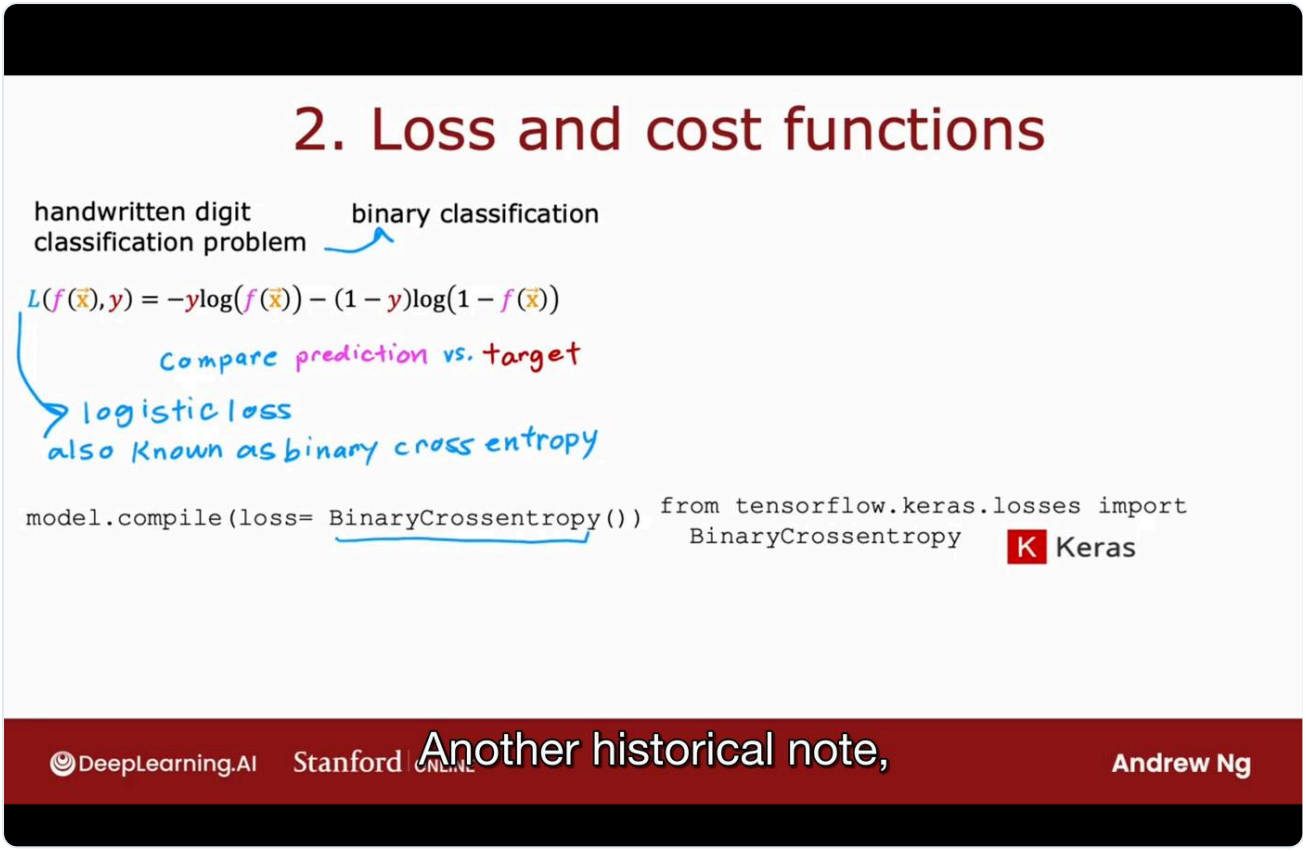


FIGURE 19.5 | 2. Loss and Cost Functions

Another historical note, carers was originally a library that had developed independently of TensorFlow is actually totally separate project from TensorFlow. But eventually it got merged into TensorFlow, which is why we have tf.Keras library.losses dot the name of this loss function. By the way, I don't always remember the names of all the loss functions and TensorFlow, but I just do a quick web search myself to find the right name and then I plug that into my code. Having specified the loss with respect to a single training example, TensorFlow knows that it costs you want to minimize is then the average, taking the average over all m training examples of the loss on all of the training examples.

Optimizing this cost function will result in fitting the neural network to your binary classification data. In case you want to solve a regression problem rather than a classification problem. You can also tell TensorFlow to compile your model using a different loss function. For example, if you have a regression problem and if you want to minimize the squared error loss. Here is the squared error loss.

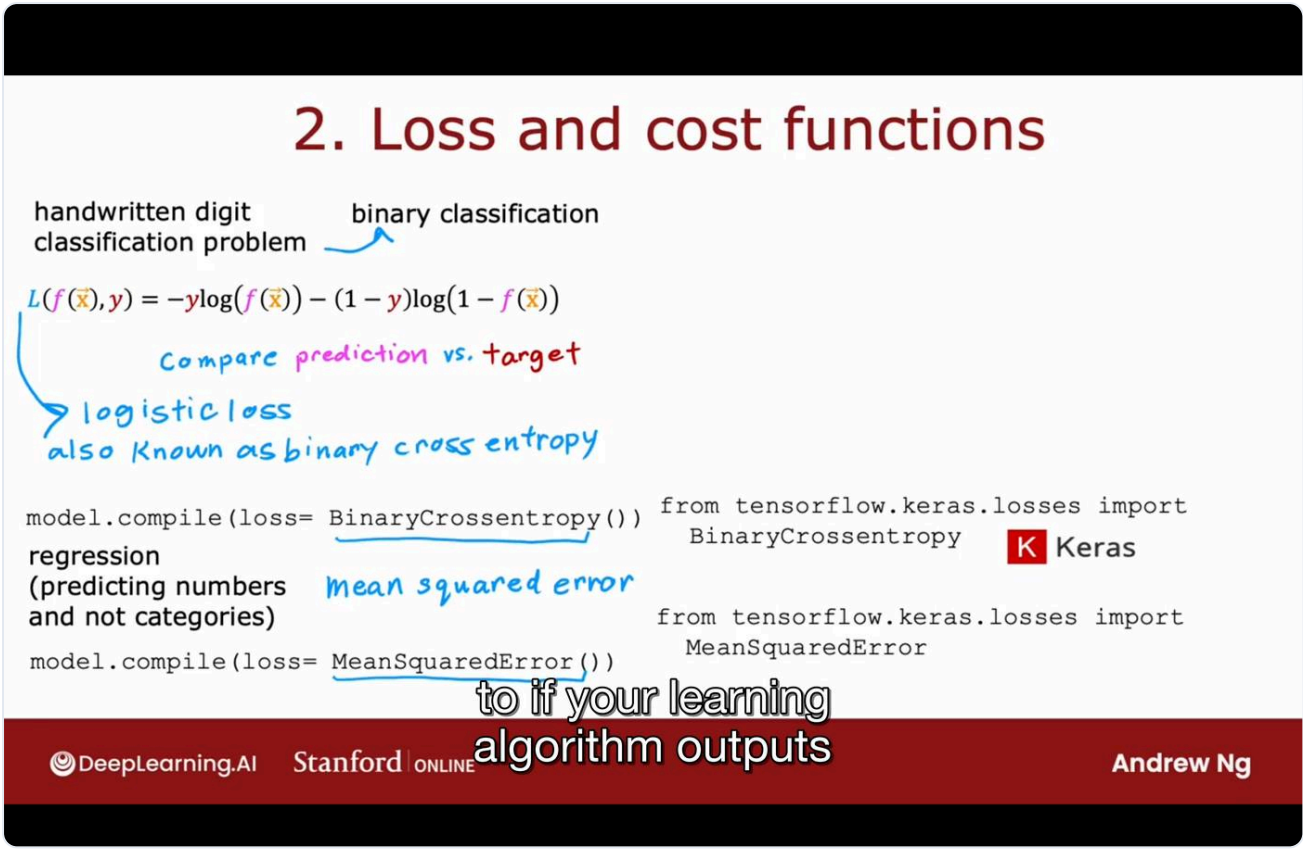


FIGURE 19.6 | 2. Loss and Cost Functions

The loss with respect to if your learning algorithm outputs f of x with a target or ground truth label of y, that's 1/2 of the squared error. Then you can use this loss function in TensorFlow, which is to use the maybe more intuitively named mean squared error loss function. Then TensorFlow will try to minimize the mean squared error. In this expression, I'm using j of capital w comma capital b to denote the cost function. The cost function is a function of all the parameters into neural network. You can think of capital W as including W1, W2, W3. All the W parameters and the entire new network and be as including b1, b2, and b3. If you are optimizing the cost function respect to w and b, if we tried to optimize it with respect to all of the parameters in the neural network.

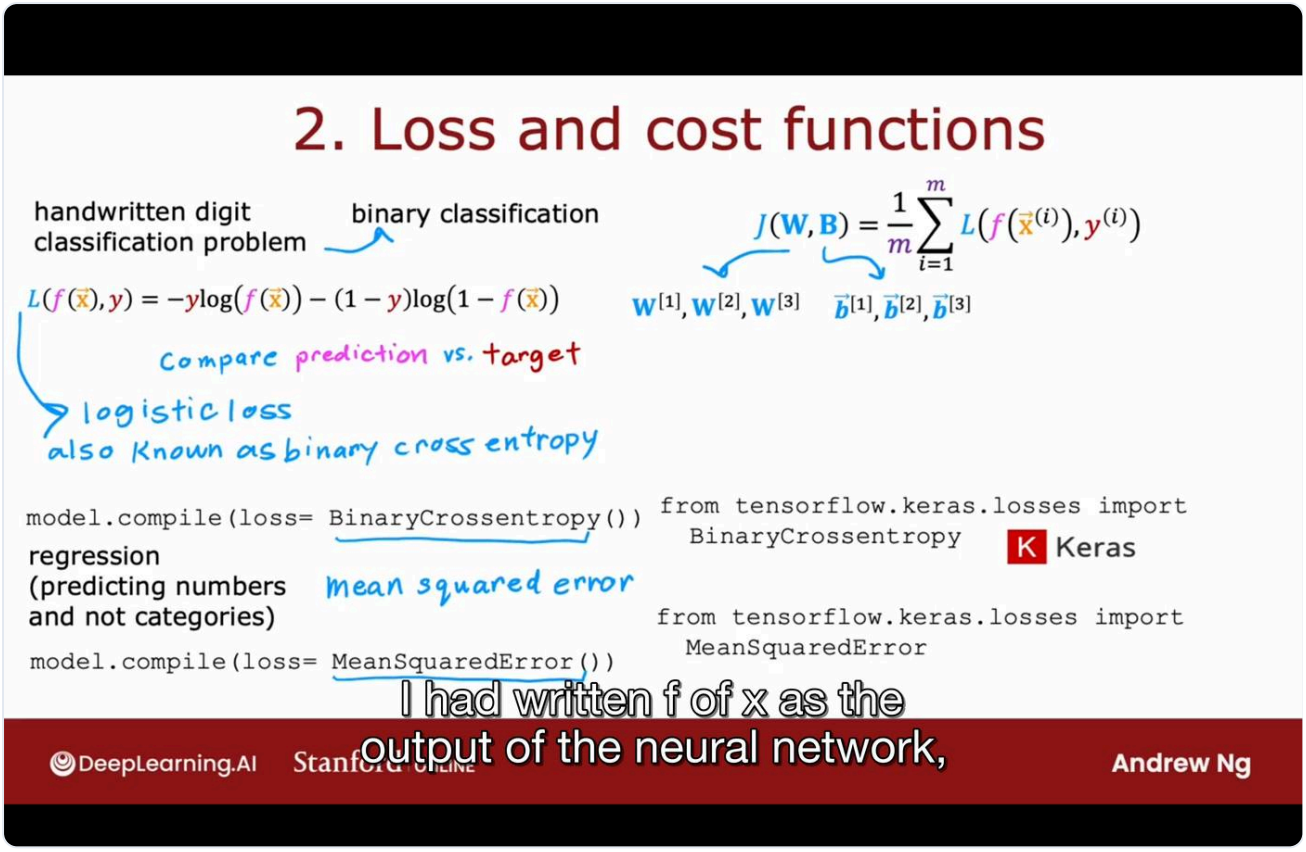


FIGURE 19.7 | 2. Loss and Cost Functions

Up on top as well, I had written f of x as the output of the neural network, but we can also write f of w b if we want to emphasize that the output of the neural network as a function of x depends on all the parameters in all the layers of the neural network. That's the loss function and the cost function. Finally, you will ask TensorFlow to minimize the cross-function. You might remember the gradient descent algorithm from the first course.

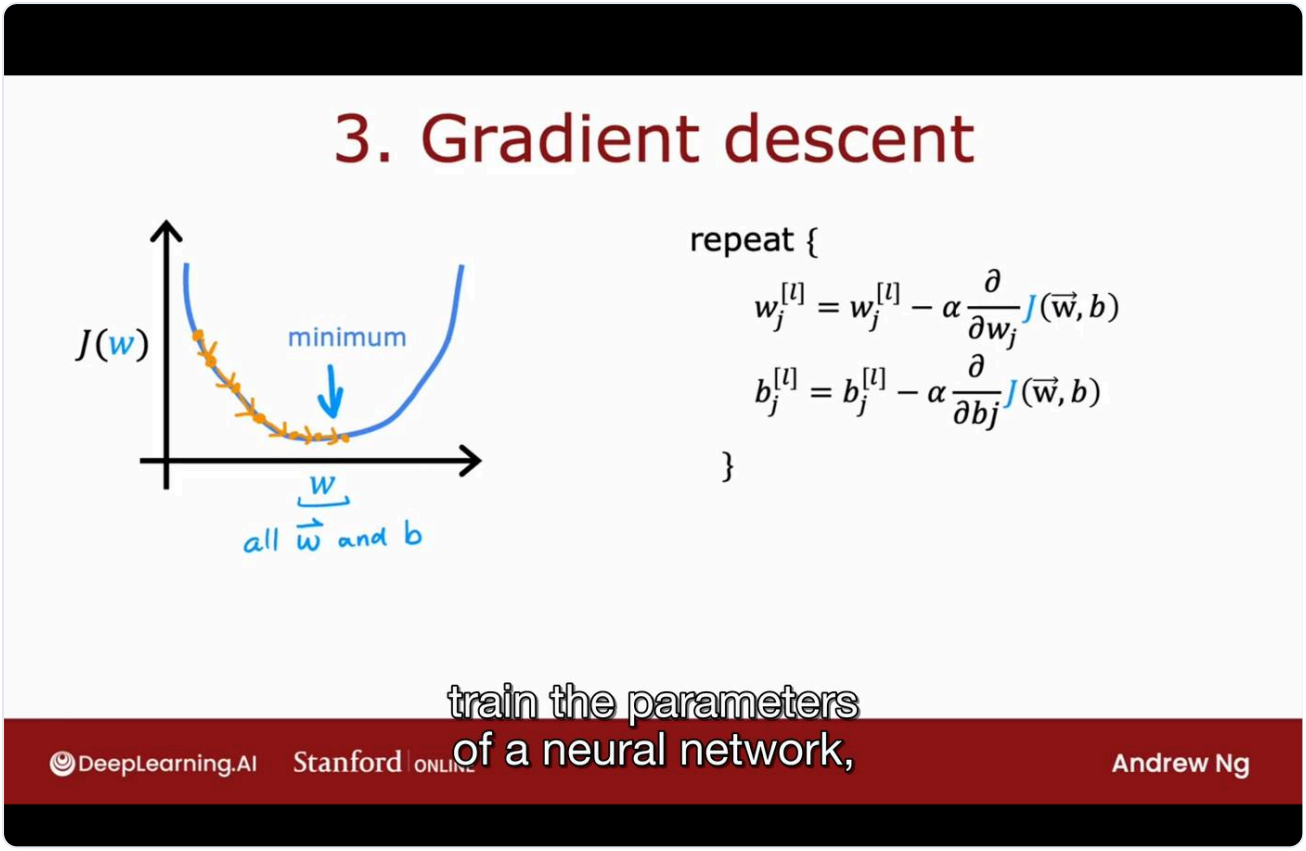


FIGURE 19.8 | 3. Gradient Descent

If you're using gradient descent to train the parameters of a neural network, then you are repeatedly, for every layer l and for every unit j, update wlj according to wlj minus the learning rate alpha times the partial derivative with respect to that parameter of the cost function j of wb and similarly for the parameters b as well. After doing, say, 100 iterations of gradient descent, hopefully, you get to a good value of the parameters. In order to use gradient descent, the key thing you need to compute is these partial derivative terms.

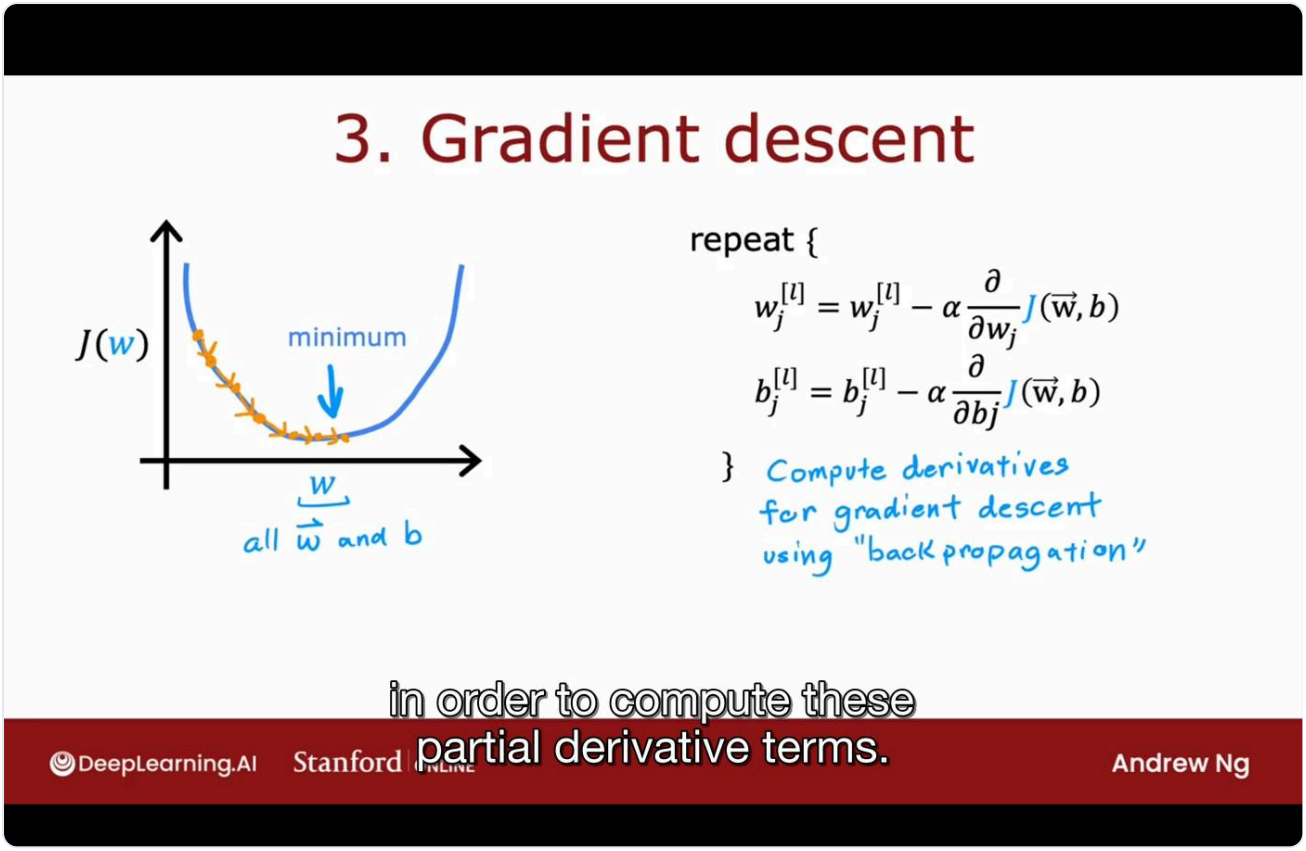


FIGURE 19.9 | 3. Gradient Descent

What TensorFlow does, and, in fact, what is standard in neural network training, is to use an algorithm called backpropagation in order to compute these partial derivative terms. TensorFlow can do all of these things for you. It implements backpropagation all within this function called fit.

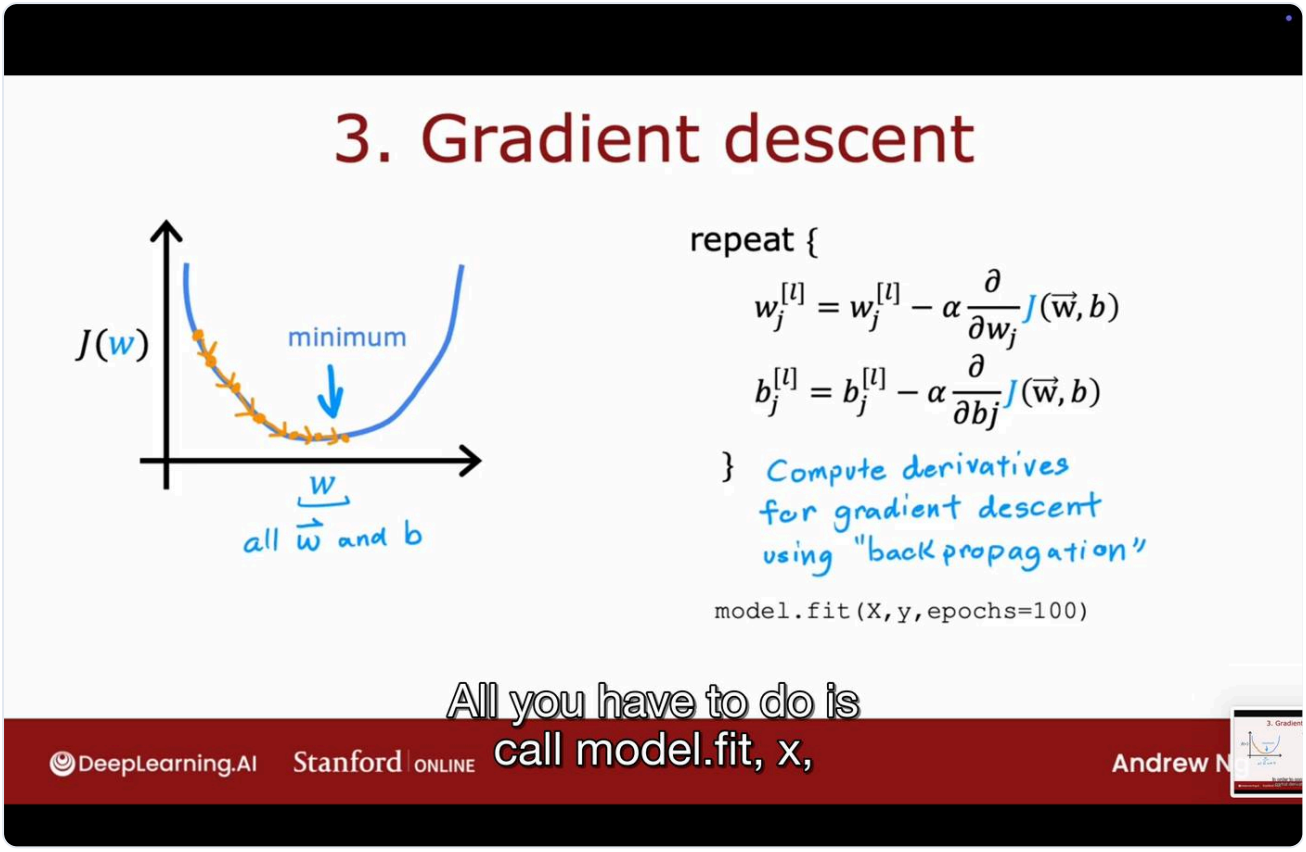


FIGURE 19.10 | 3. Gradient Descent

All you have to do is call model.fit, x, y as your training set, and tell it to do so for 100 iterations or 100 epochs. In fact, what you see later is that TensorFlow can use an algorithm that is even a little bit faster than gradient descent, and you'll see more about that later this week as well. Now, I know that we're relying heavily on the TensorFlow library in order to implement a neural network. One pattern I've seen across multiple ideas is as the technology evolves, libraries become more mature, and most engineers will use libraries rather than implement code from scratch. There have been many other examples of this in the history of computing.

Once, many decades ago, programmers had to implement their own sorting function from scratch, but now sorting libraries are quite mature that you probably call someone else's sorting function rather than implement it yourself, unless you're taking a computing class and I ask you to do it as an exercise. Today, if you want to compute the square root of a number, like what is the square root of seven, well, once programmers had to write their own code to compute this, but now pretty much everyone just calls a library to take square roots, or matrix operations, such as multiplying two matrices together. When deep learning was younger and less mature, many developers, including me, were implementing things from scratch using Python or C++ or some other library.

But today, deep learning libraries have matured enough that most developers will use these libraries, and, in fact, most commercial implementations of neural networks today use a library like TensorFlow or PyTorch.

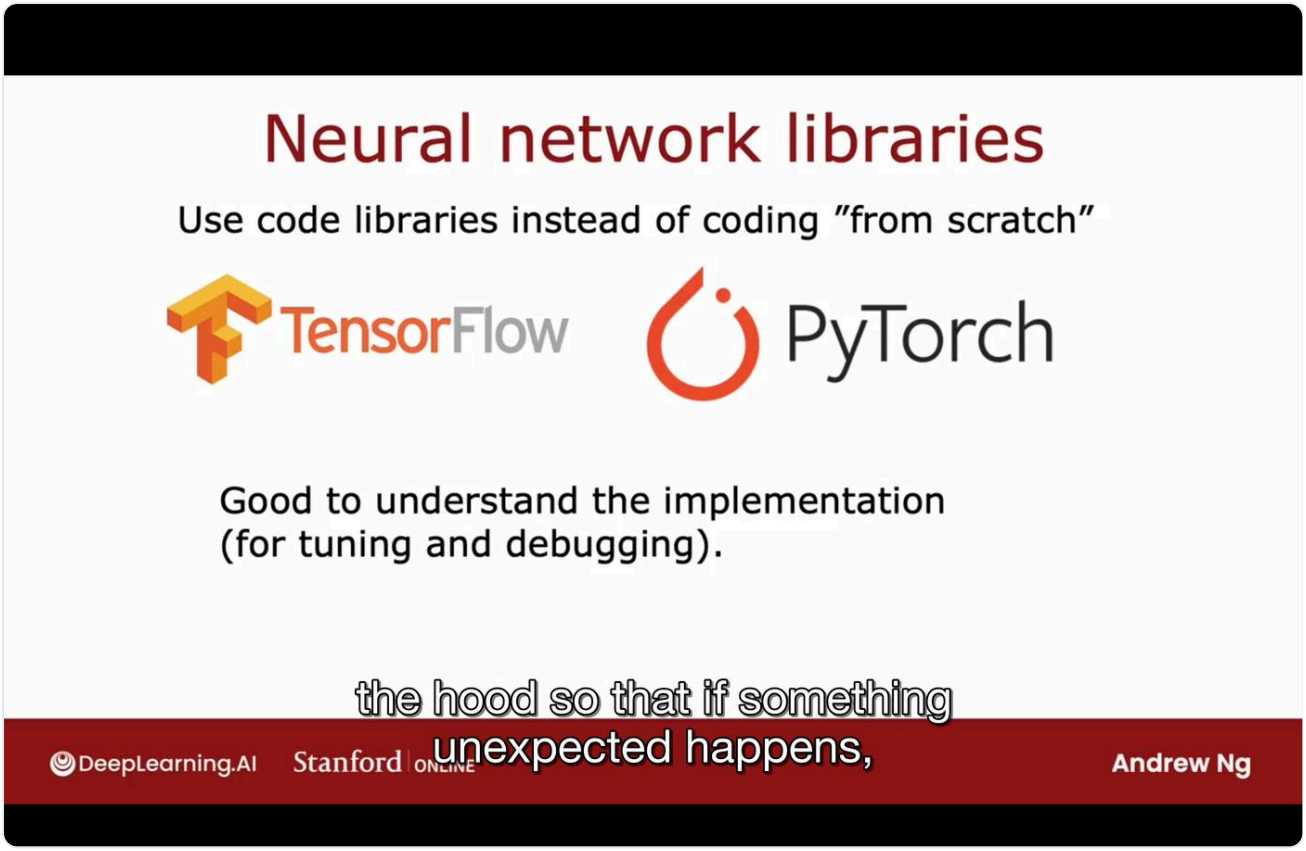


FIGURE 19.11 | Neural Network Libraries

But as I've mentioned, it's still useful to understand how they work under the hood so that if something unexpected happens, which still does with today's libraries, you have a better chance of knowing how to fix it. Now that you know how to train a basic neural network, also called a multilayer perceptron, there are some things you can change about the neural network that will make it even more powerful. In the next video, let's take a look at how you can swap in different activation functions as an alternative to the sigmoid activation function we've been using. This will make your neural networks work even much better. Let's go take a look at that in the next video.

LECTURE 20

Demand Prediction Example

So far, we've been using the sigmoid activation function in all the nodes in the hidden layers and in the output layer. And we have started that way because we were building up neural networks by taking logistic regression and creating a lot of logistic regression units and string them together. But if you use other activation functions, your neural network can become much more powerful. Let's take a look at how to do that. Recall the demand prediction example from last week where given price, shipping cost, marketing, material, you would try to predict if something is highly affordable. If there's good awareness and high perceived quality and based on that try to predict it was a top seller.

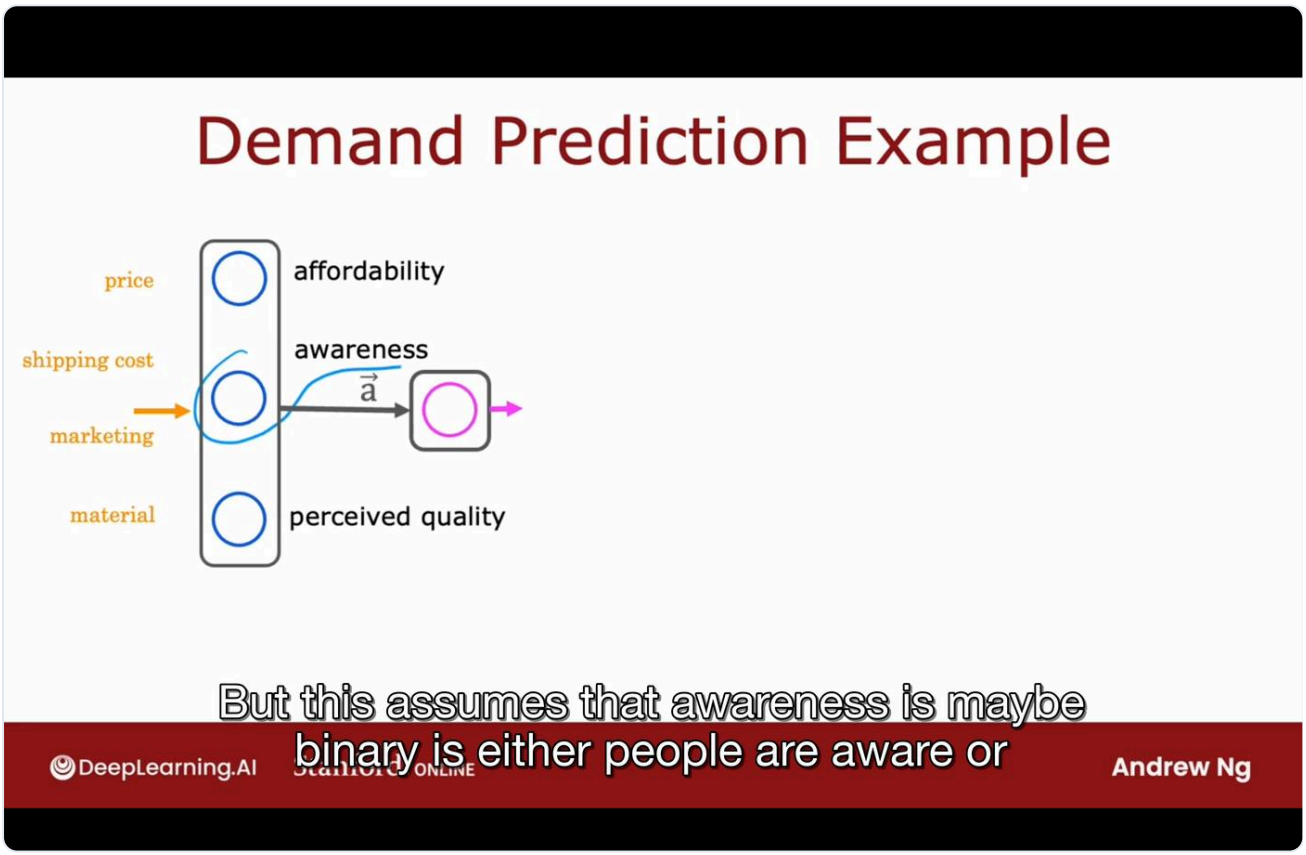


FIGURE 20.1 | Demand Prediction Example

But this assumes that awareness is maybe binary is either people are aware or they are not.

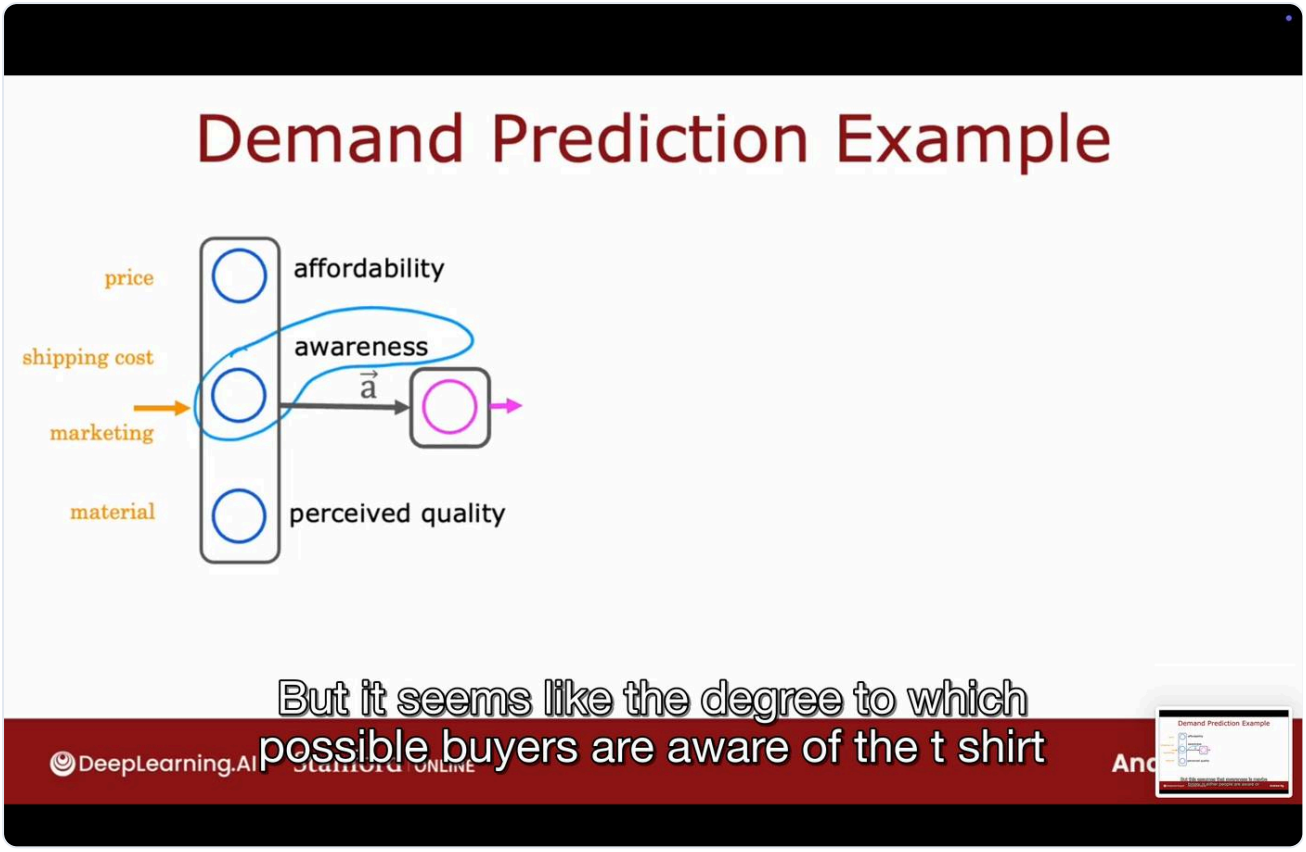


FIGURE 20.2 | Demand Prediction Example

But it seems like the degree to which possible buyers are aware of the t shirt you're selling may not be binary, they can be a little bit aware, somewhat aware, extremely aware or it could have gone completely viral. So rather than modeling awareness as a binary number 0, 1, that you try to estimate the probability of awareness or rather than modeling awareness is just a number between 0 and 1. Maybe awareness should be any non negative number because there can be any non negative value of awareness going from 0 up to very very large numbers. So whereas previously we had used this equation to calculate the activation of that second hidden unit estimating awareness where g was the sigmoid function and just goes between 0 and 1.

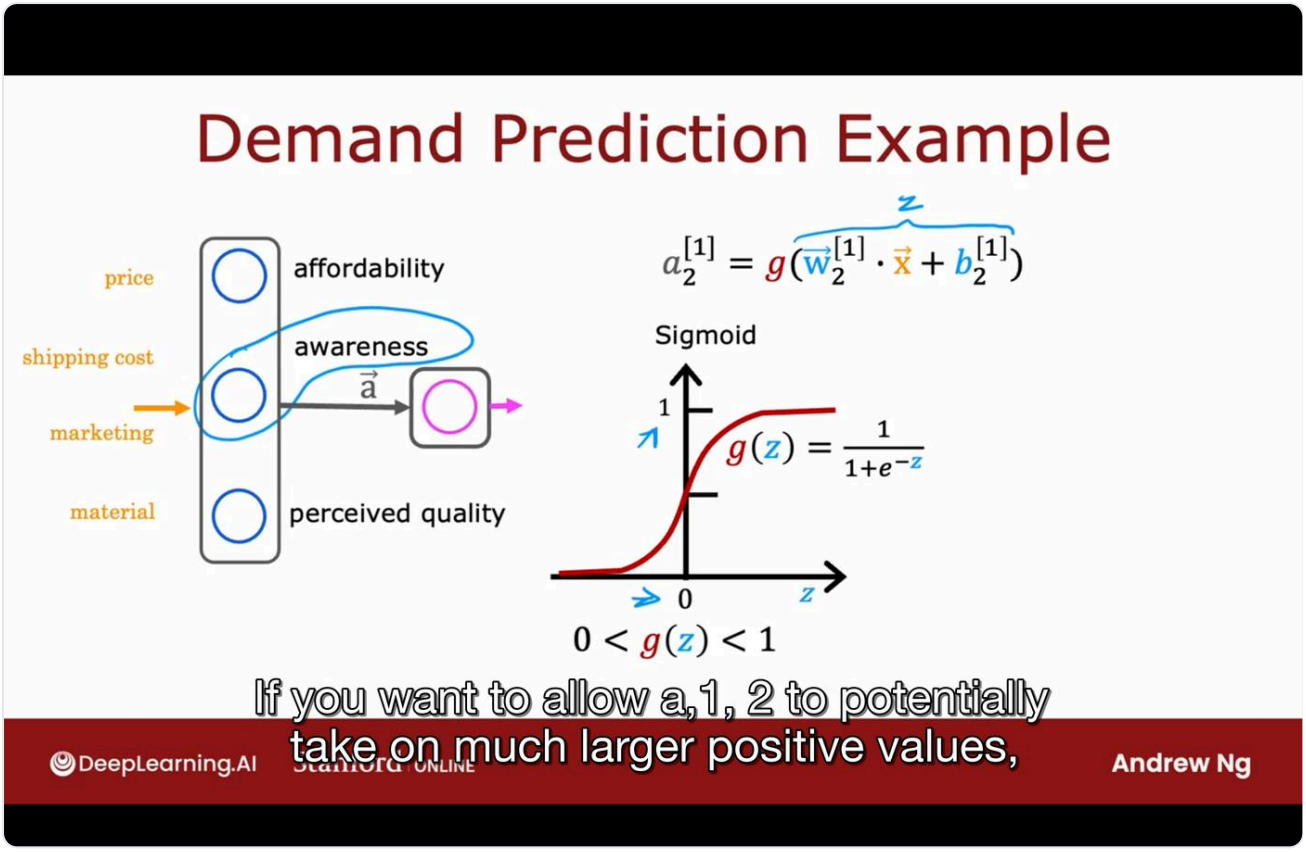


FIGURE 20.3 | Demand Prediction Example

If you want to allow a,1, 2 to potentially take on much larger positive values, we can instead swap in a different activation function. It turns out that a very common choice of activation function in neural networks is this function. It looks like this. It goes if z is this, then g(z) is 0 to the left and then there's this straight line 45° to the right of 0. And so when z is greater than or equal to 0, g(z) is just equal to z.

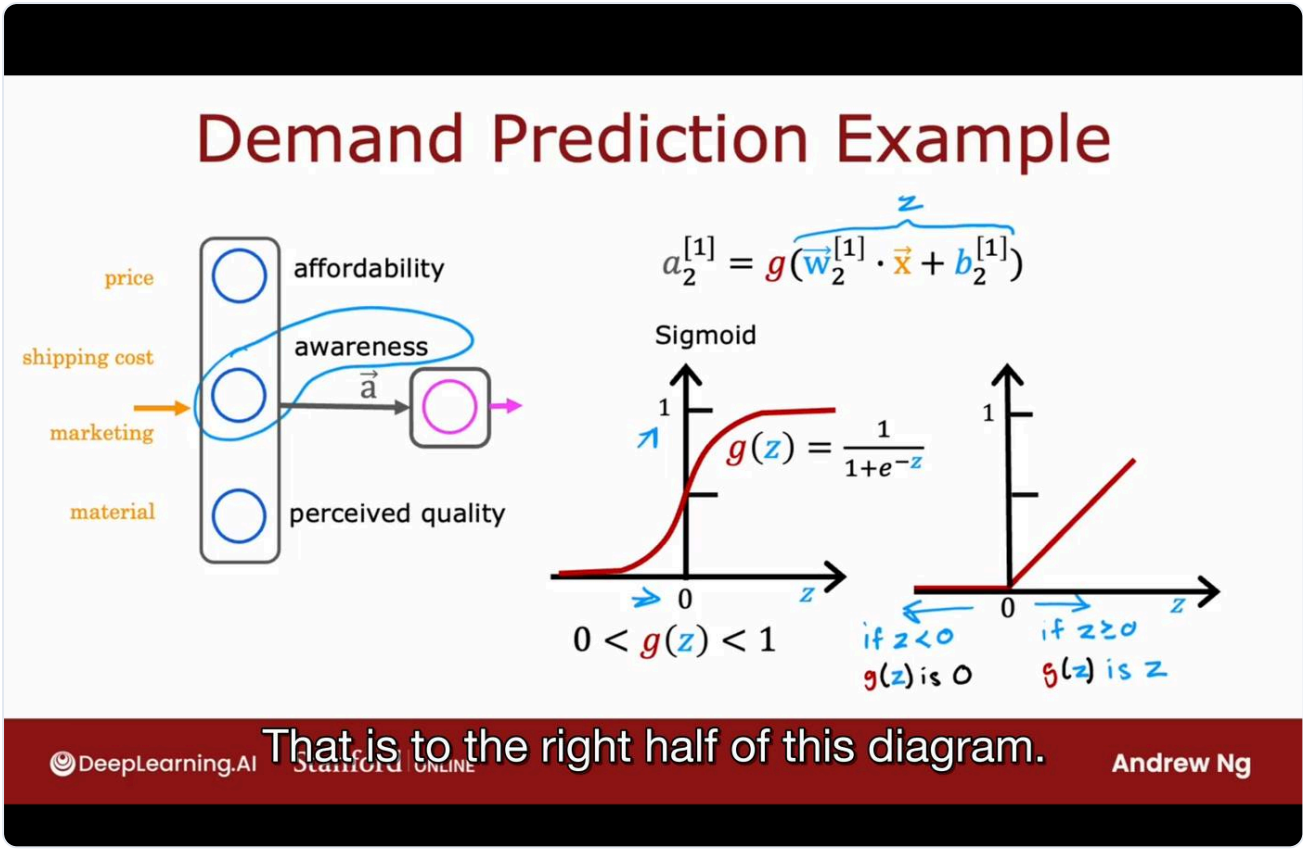


FIGURE 20.4 | Demand Prediction Example

That is to the right half of this diagram. And the mathematical equation for this is g(z) equals max(0, z). Feel free to verify for yourself that max(0, z) results in this curve that I've drawn over here. And if a 1, 2 is g(z) for this value of z, then a, the deactivation value cannot take on 0 or any non negative value. This activation function has a name.

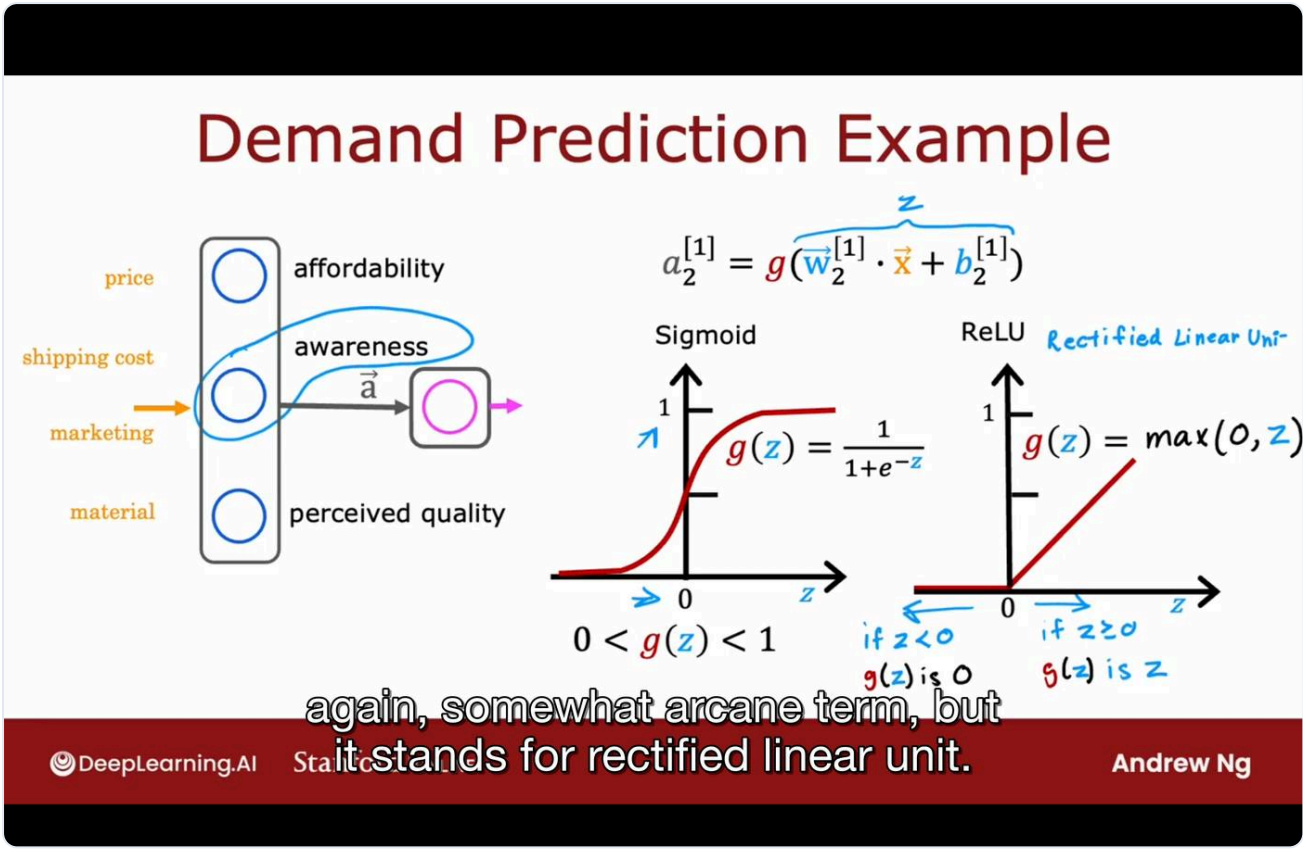


FIGURE 20.5 | Demand Prediction Example

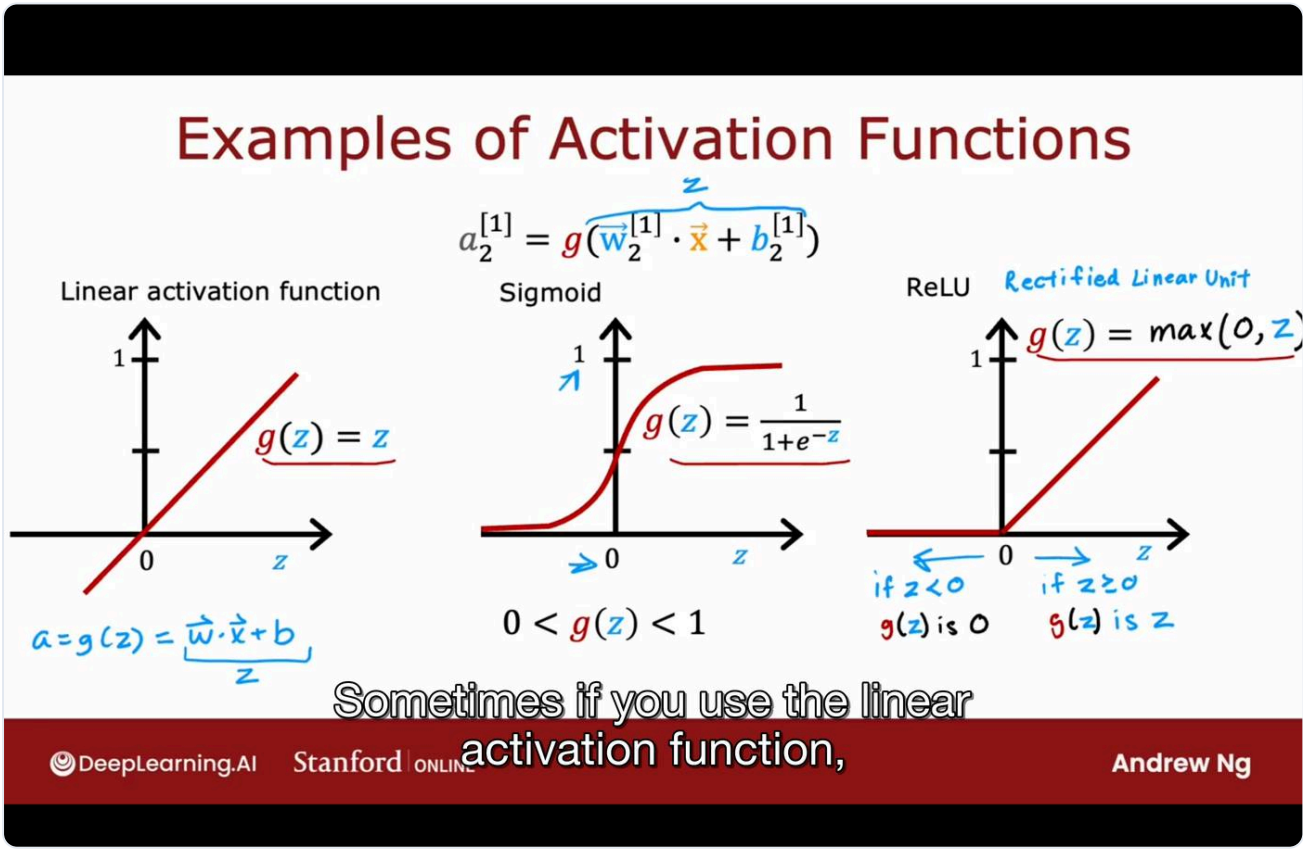


FIGURE 20.6 | Examples of Activation Functions

It goes by the name ReLU with this funny capitalization and ReLU stands for again, somewhat arcane term, but it stands for rectified linear unit. Don't worry too much about what rectified means and what linear unit means. This was just the name that the authors had given to this particular activation function when they came up with it. But most people in deep learning just say ReLU to refer to this g(z). More generally you have a choice of what to use for g(z) and sometimes we'll use a different choice than the sigmoid activation function. Here are the most commonly used activation functions. You saw the sigmoid activation function, g(z) equals this sigmoid function.

On the last slide we just looked at the ReLU or rectified linear unit g(z) equals max(0, z). There's one other activation function which is worth mentioning, which is called the linear activation function, which is just g(z) equals to z. Sometimes if you use the linear activation function, people will say we're not using any activation function because if a is g(z) where g(z) equals z, then a is just equal to this w.x plus b z. And so it's as if there was no g in there at all. So when you are using this linear activation function g(z) sometimes people say, well, we're not using any activation function. Although in this class, I will refer to using the linear activation function rather than no activation function.

But if you hear someone else use that terminology, that's what they mean. It just refers to the linear activation function. And these three are probably by far the most commonly used activation functions in neural networks. Later this week, we'll touch on the fourth one called the softmax activation function. But with these activation functions you'll be able to build a rich variety of powerful neural networks. So when building a neural network for each neuron, do you want to use the sigmoid activation function or the ReLU activation function? Or a linear activation function? How do you choose between these different activation functions? Let's take a look at that in the next video.

LECTURE 21

Hidden Layer

Let's take a look at how you can choose the activation function for different neurons in your neural network. We'll start with some guidance for how to choose it for the output layer. It turns out that depending on what the target label or the ground truth label y is, there will be one fairly natural choice for the activation function for the output layer, and we'll then go and look at the choice of the activation function also for the hidden layers of your neural network. Let's take a look. You can choose different activation functions for different neurons in your neural network, and when considering the activation function for the output layer, it turns out that there'll often be one fairly natural choice, depending on what is the target or the ground truth label y.

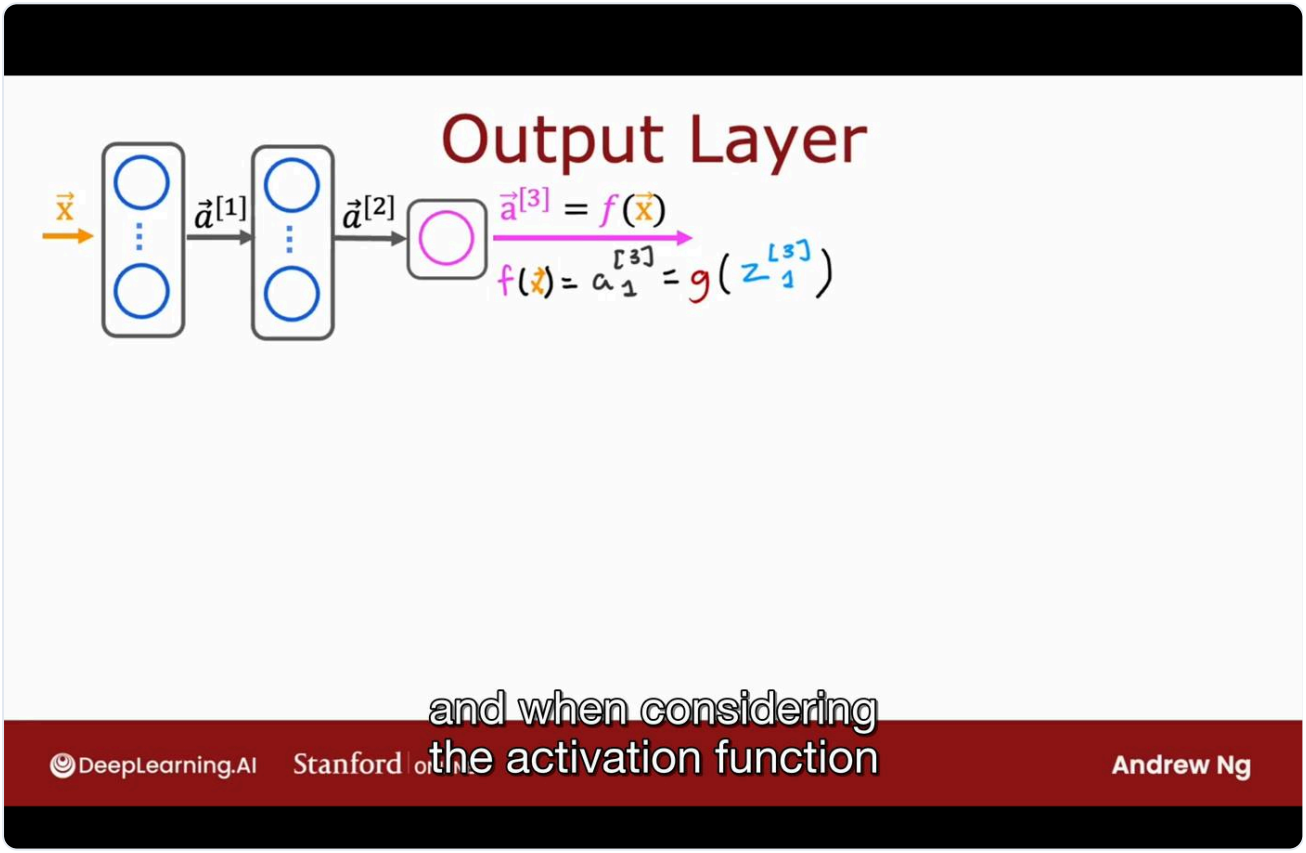


FIGURE 21.1 | Output Layer

Specifically, if you are working on a classification problem where y is either zero or one, so a binary classification problem, then the sigmoid activation function will almost always be the most natural choice, because then the neural network learns to predict the probability that y is equal to one, just like we had for logistic regression. My recommendation is, if you're working on a binary classification problem, use sigmoid at the output layer. Alternatively, if you're solving a regression problem, then you might choose a different activation function. For example, if you are trying to predict how tomorrow's stock price will change compared to today's stock price.

Well, it can go up or down, and so in this case y would be a number that can be either positive or negative, and in that case I would recommend you use the linear activation function.

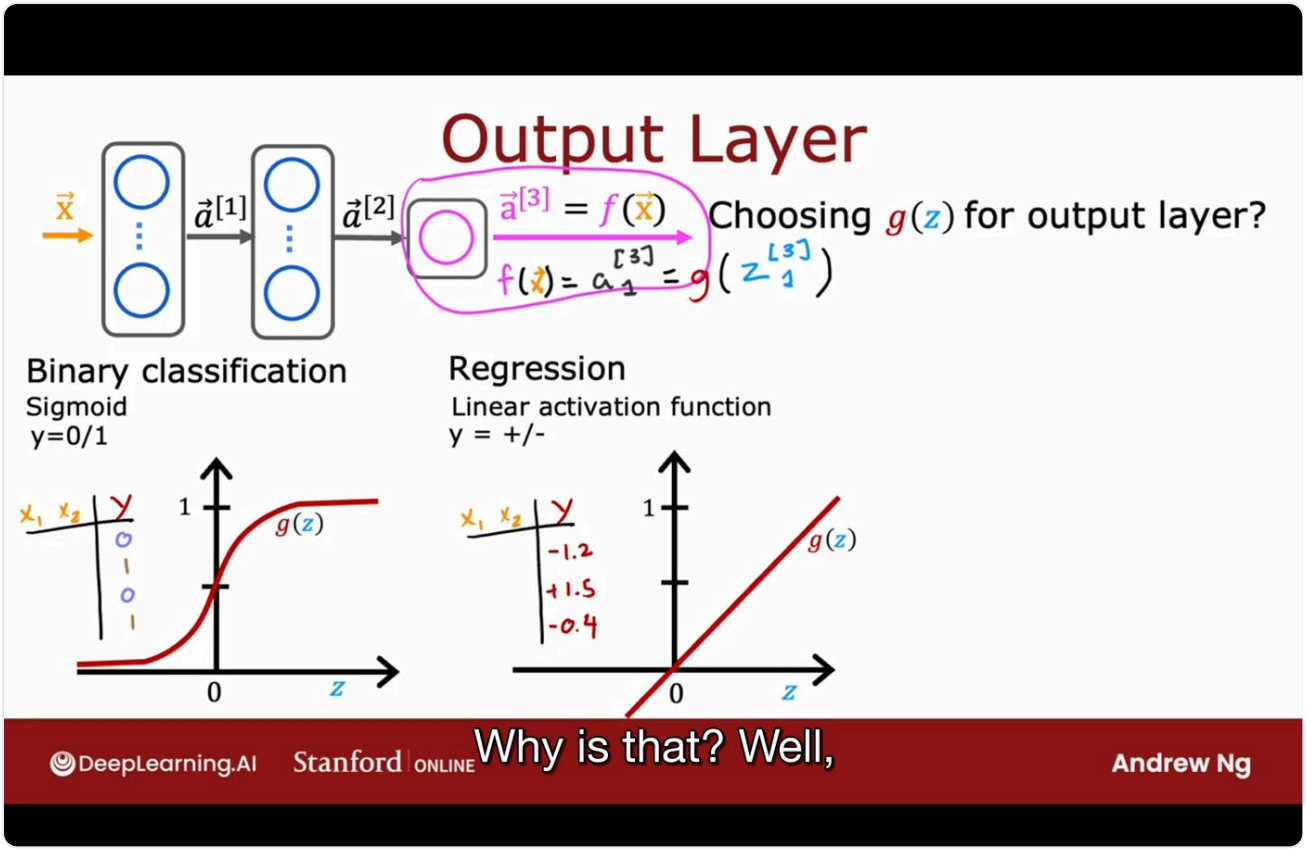


FIGURE 21.2 | Output Layer

Why is that? Well, that's because then the outputs of your neural network, f of x, which is equal to a^3 in the example above, would be g applied to z^3 and with the linear activation function, g of z can take on either positive or negative values. So y can be positive or negative, use a linear activation function.

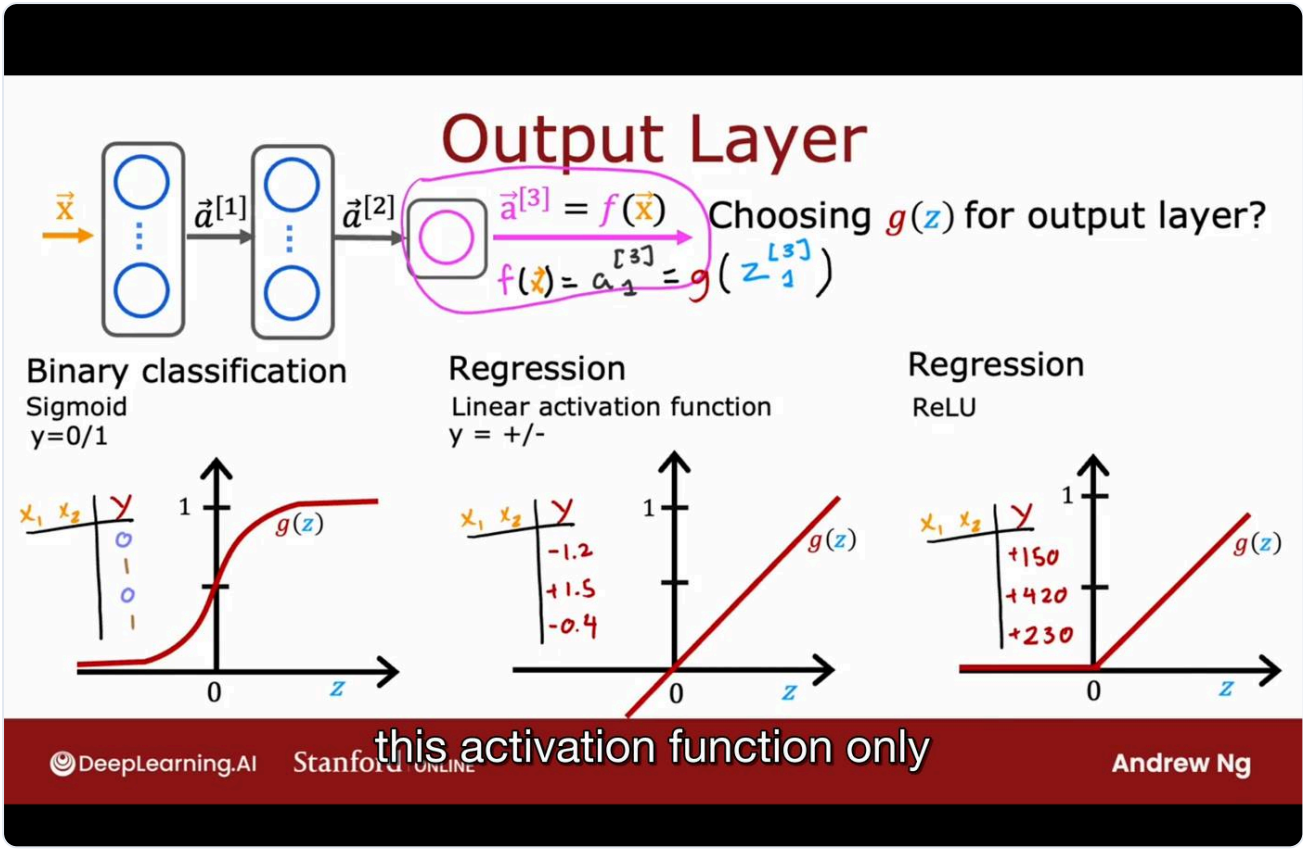


FIGURE 21.3 | Sigmoid Linear Activation Function ReLU

Finally, if y can only take on non-negative values, such as if you're predicting the price of a house, that can never be negative, then the most natural choice will be the ReLU activation function because as you see here, this activation function only takes on non-negative values, either zero or positive values. In choosing the activation function to use for your output layer, usually depending on what is the label y you're trying to predict, there'll be one fairly natural choice. In fact, the guidance on this slide is how I pretty much always choose my activation function as well for the output layer of a neural network.

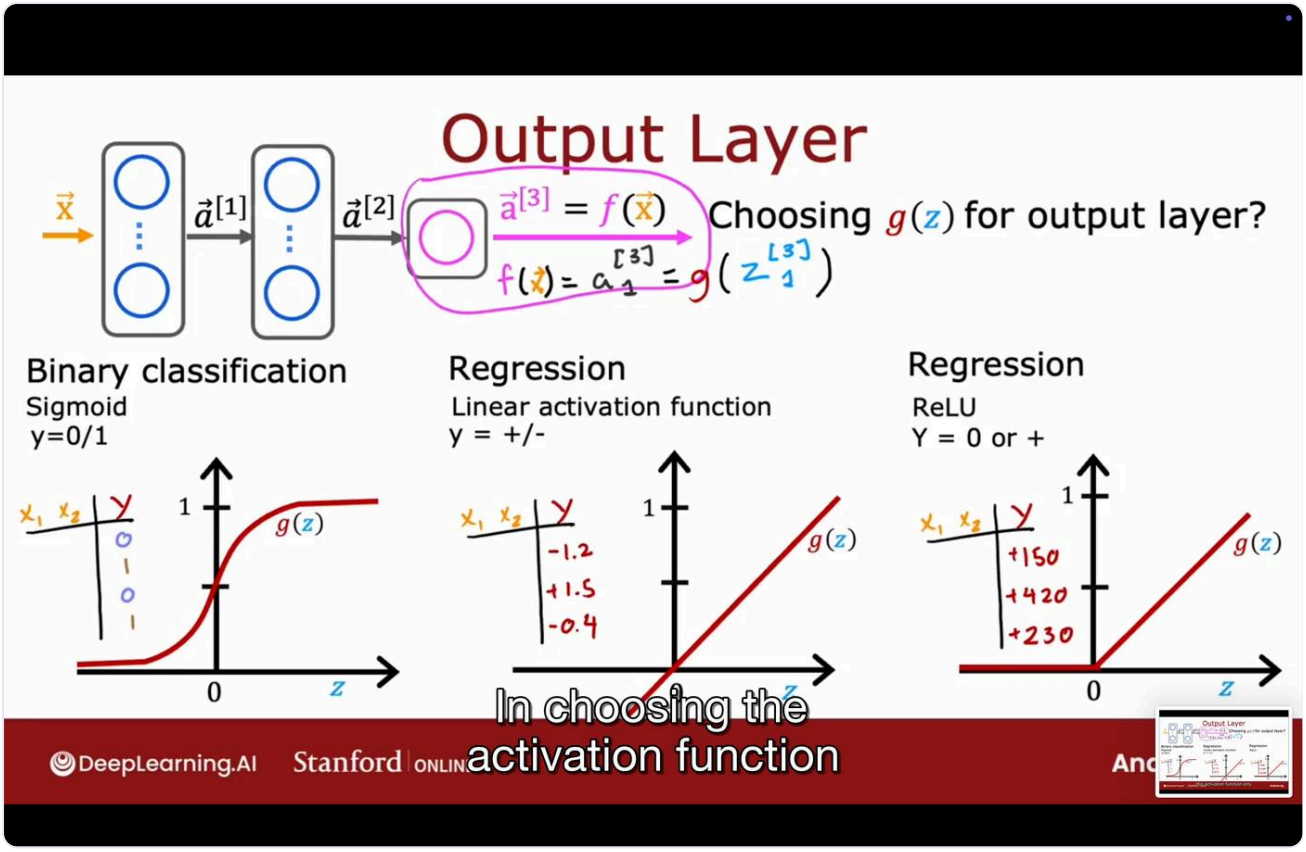


FIGURE 21.4 | Output Layer

How about the hidden layers of a neural network? It turns out that the ReLU activation function is by far the most common choice in how neural networks are trained by many practitioners today. Even though we had initially described neural networks using the sigmoid activation function, and in fact, in the early history of the development of neural networks, people use sigmoid activation functions in many places, the field has evolved to use ReLU much more often and sigmoids hardly ever.

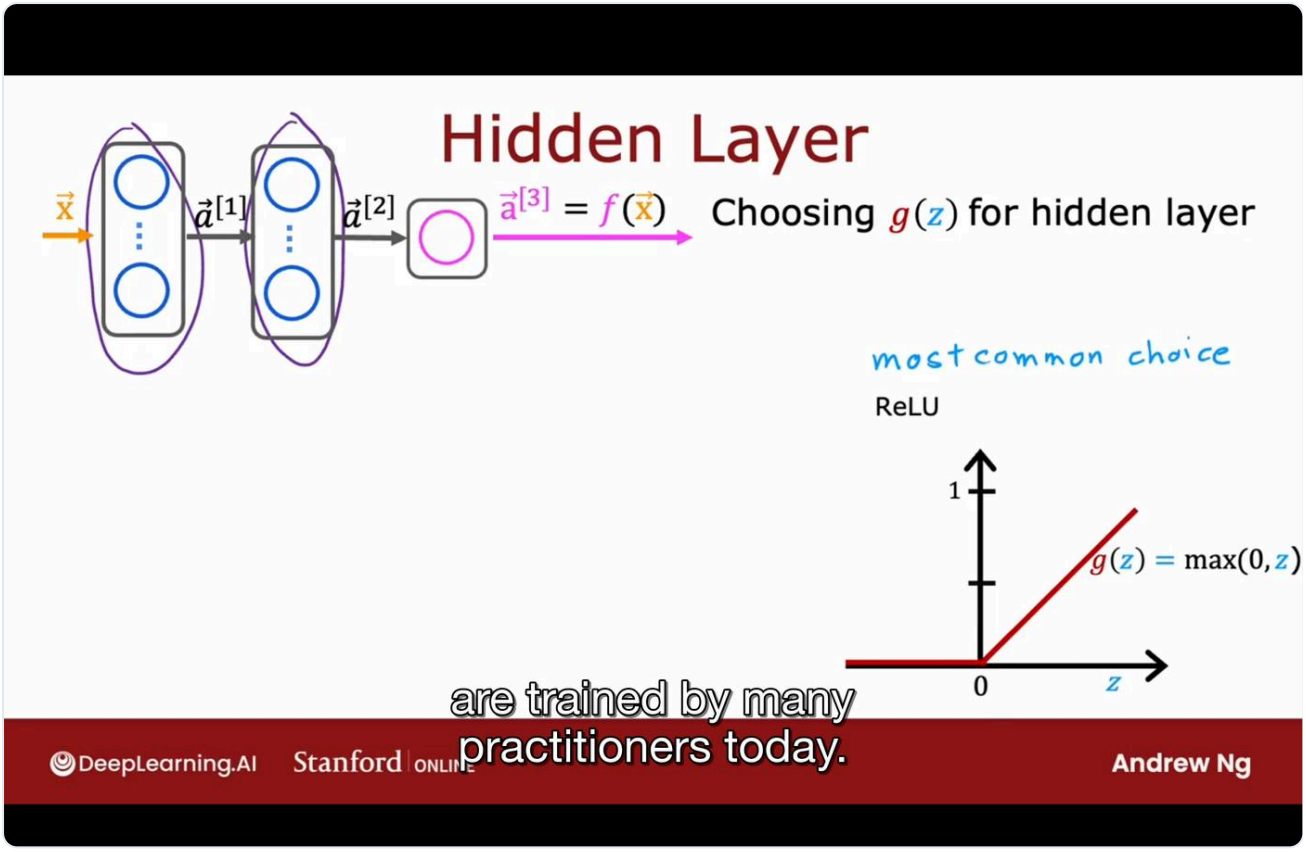


FIGURE 21.5 | Hidden Layer

Well, the one exception that you do use a sigmoid activation function in the output layer if you have a binary classification problem. So why is that? Well, there are a few reasons.

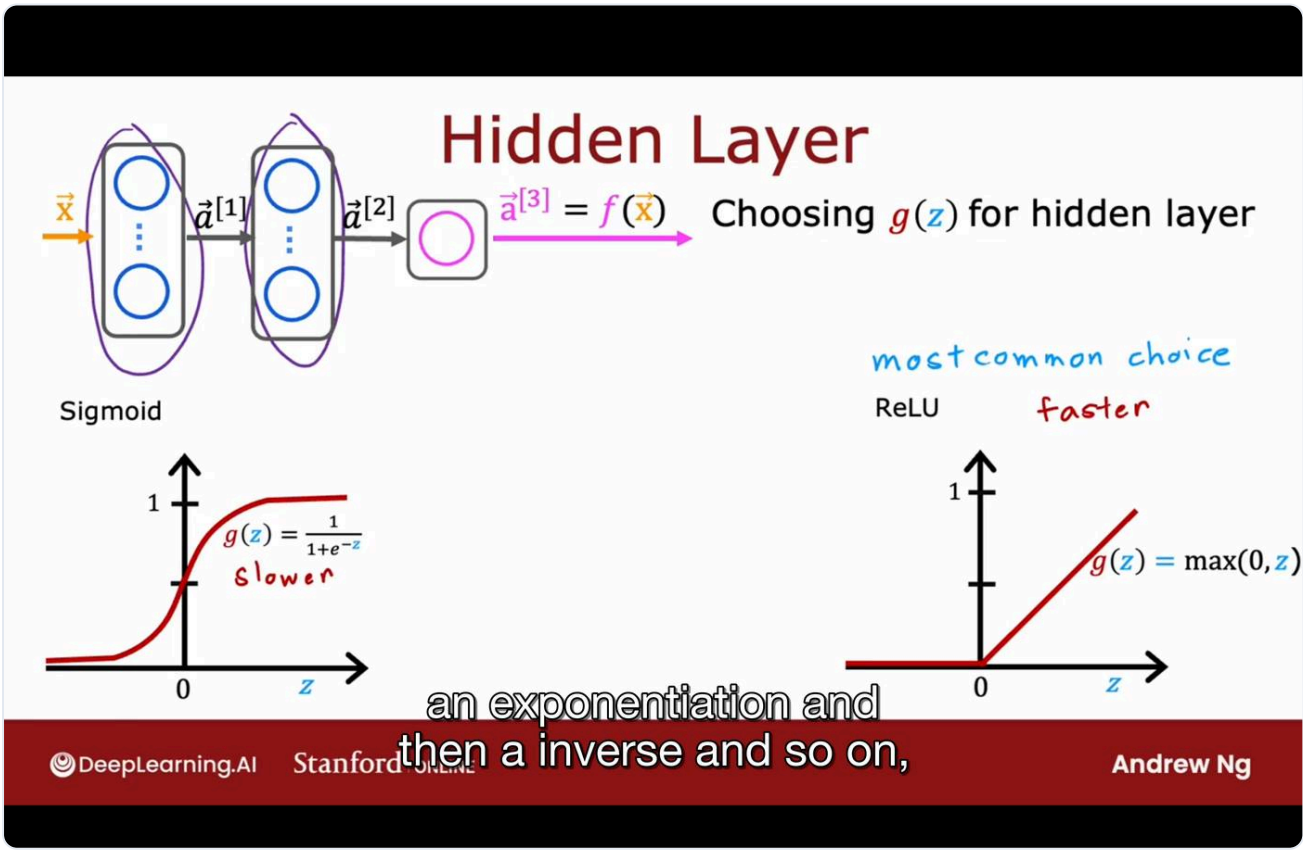


FIGURE 21.6 | Hidden Layer

First, if you compare the ReLU and the sigmoid activation functions, the ReLU is a bit faster to compute because it just requires computing max of 0, z, whereas the sigmoid requires taking an exponentiation and then a inverse and so on, and so it's a little bit less efficient.

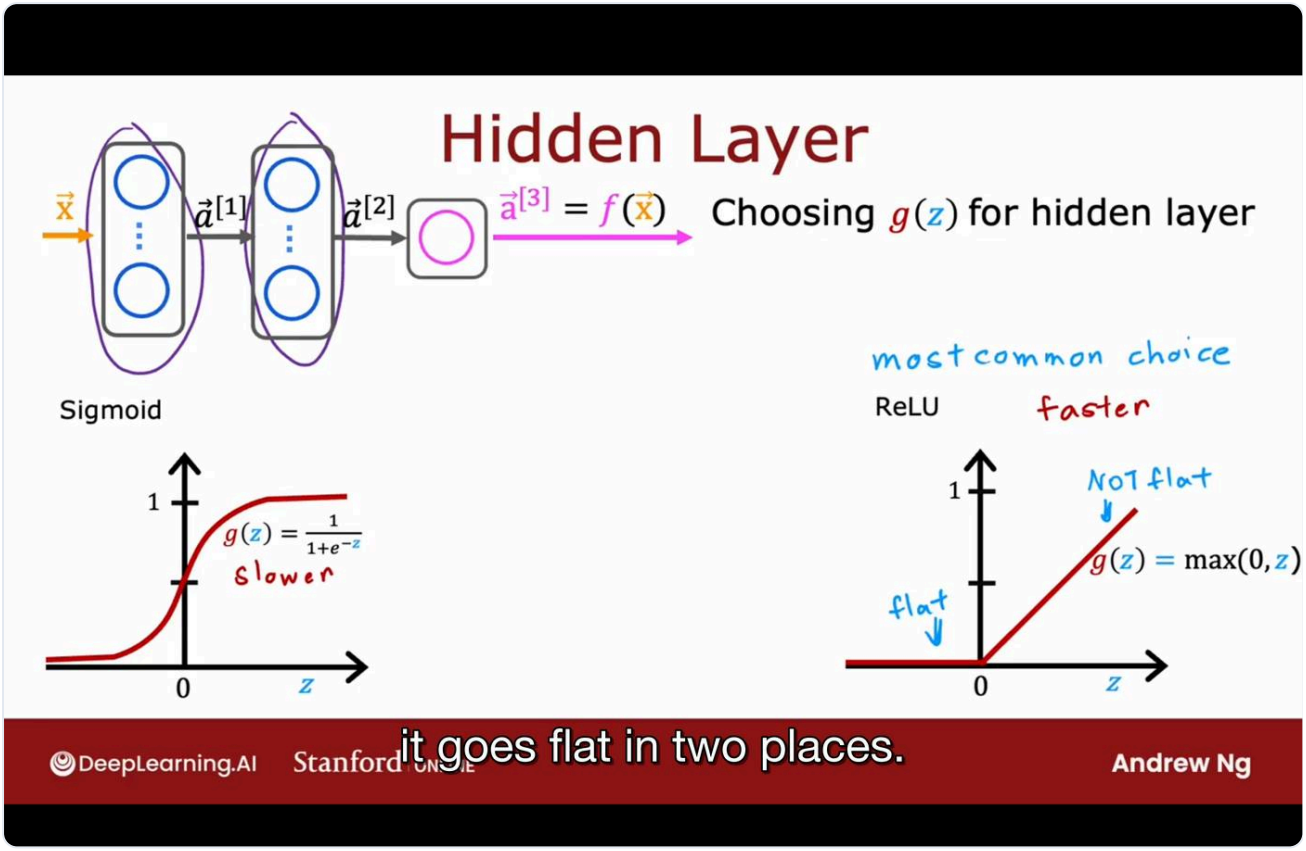


FIGURE 21.7 | Hidden Layer

But the second reason which turns out to be even more important is that the ReLU function goes flat only in one part of the graph; here on the left is completely flat, whereas the sigmoid activation function, it goes flat in two places.

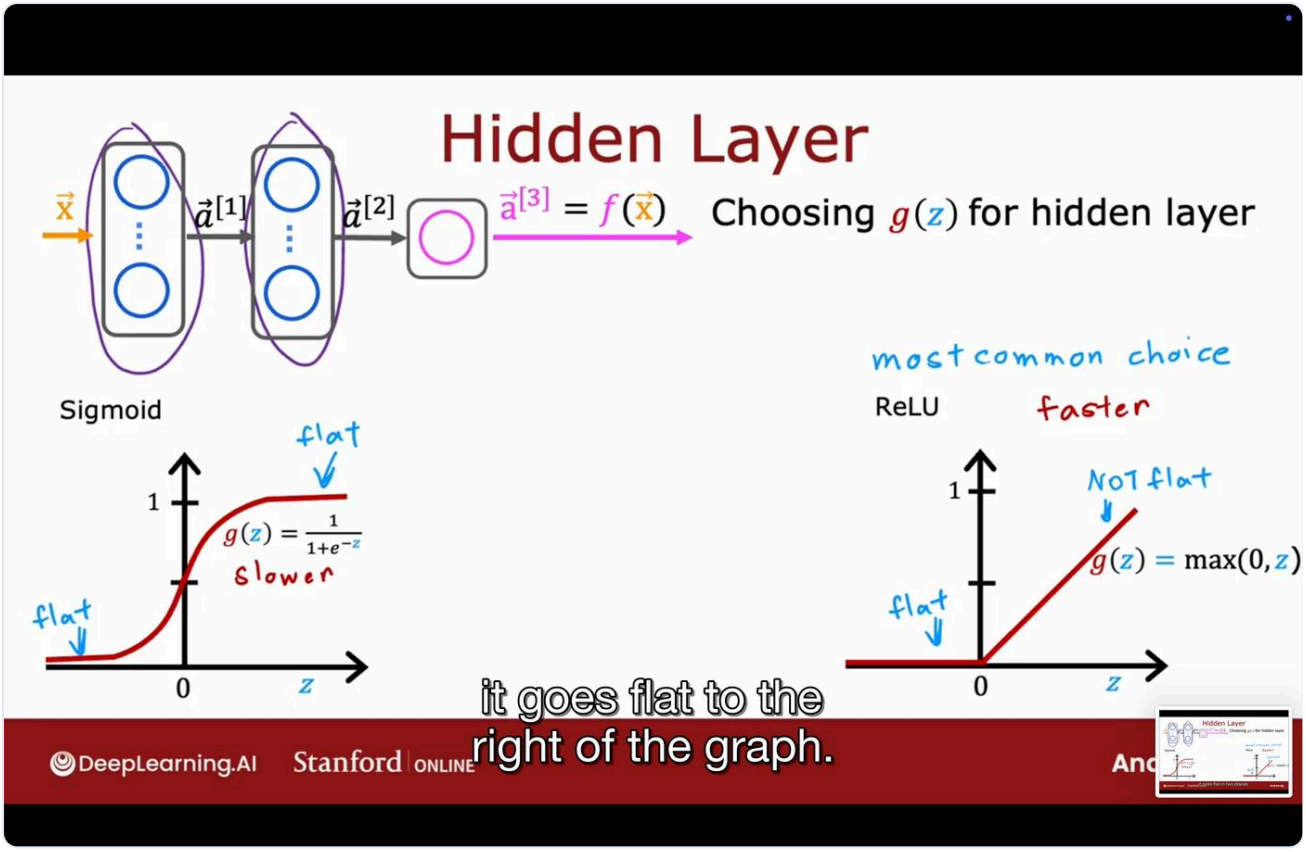


FIGURE 21.8 | Hidden Layer

It goes flat to the left of the graph and it goes flat to the right of the graph.

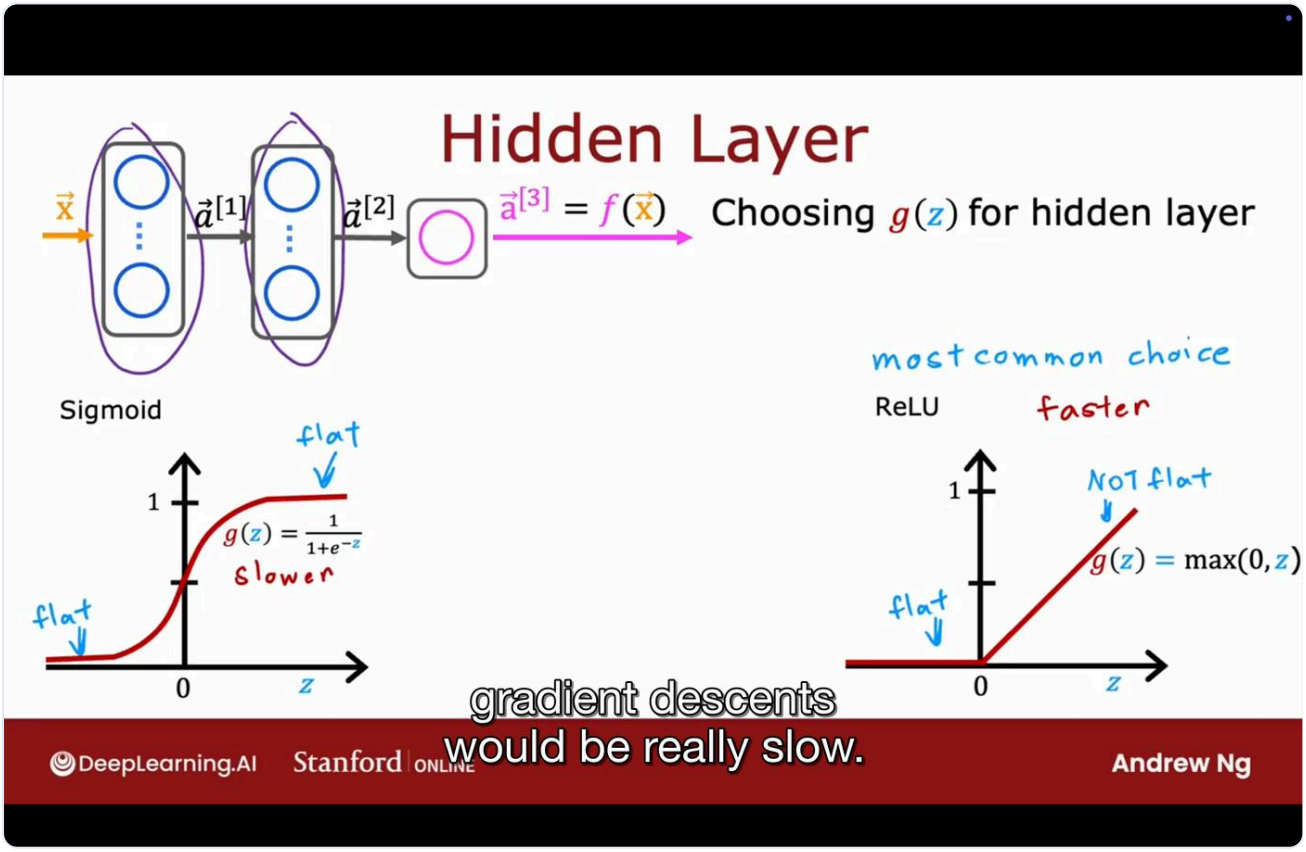


FIGURE 21.9 | Hidden Layer

If you're using gradient descent to train a neural network, then when you have a function that is fat in a lot of places, gradient descents would be really slow.

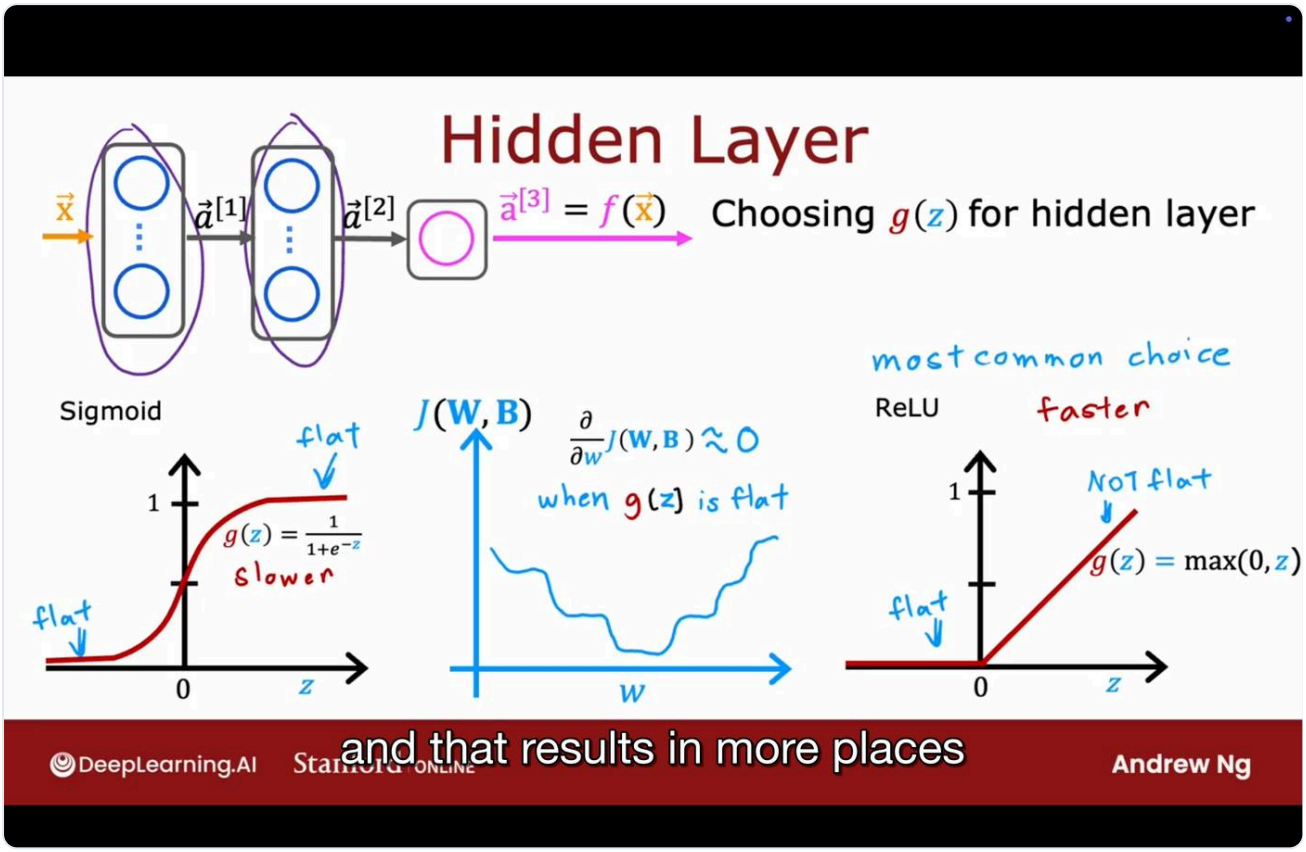


FIGURE 21.10 | Hidden Layer

I know that gradient descent optimizes the cost function J of W, B rather than optimizes the activation function, but the activation function is a piece of what goes into computing, and that results in more places in the cost function J of W, B that are flats as well and with a small gradient and it slows down learning.

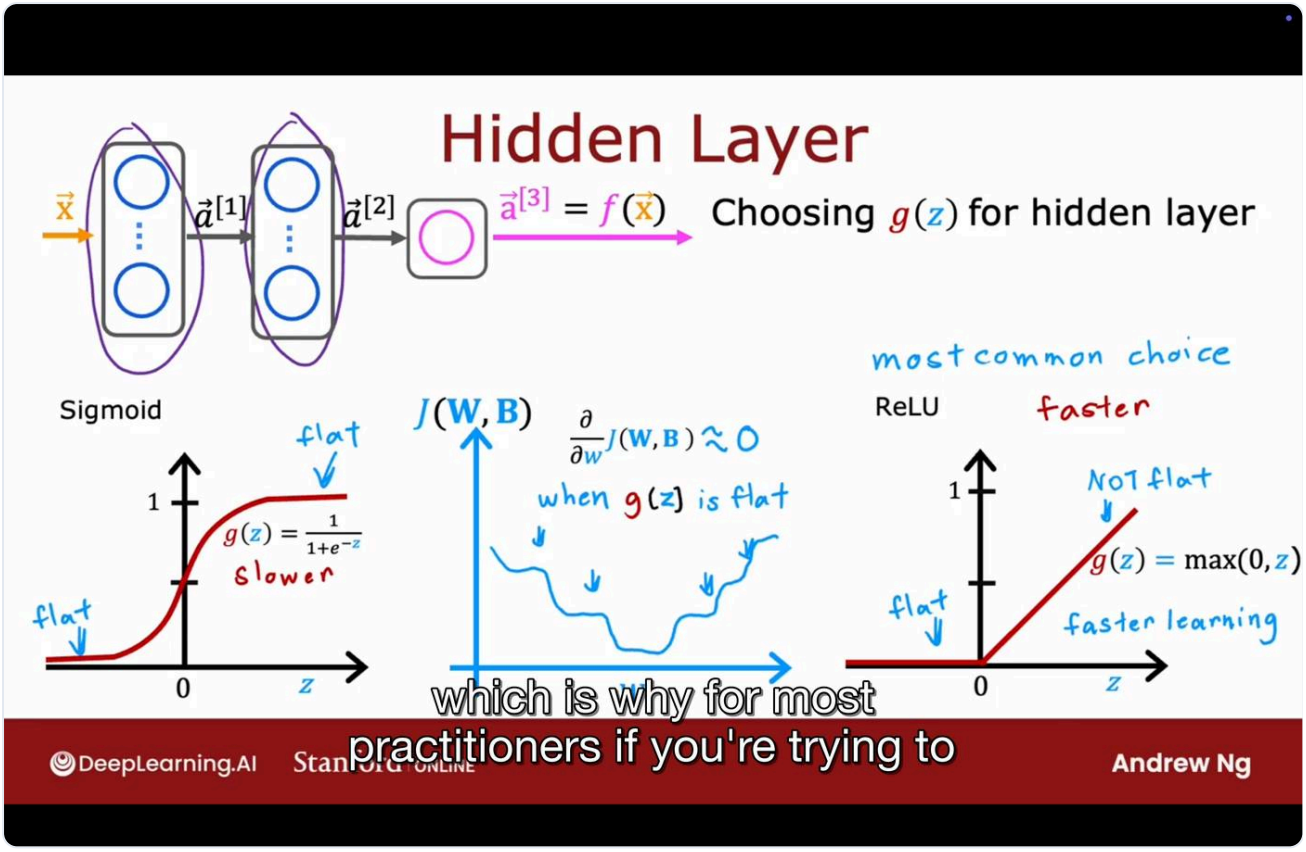


FIGURE 21.11 | Hidden Layer

I know that that was just an intuitive explanation, but researchers have found that using the ReLU activation function can cause your neural network to learn a bit faster as well, which is why for most practitioners if you're trying to decide what activation functions to use with hidden layer, the ReLU activation function has become now by far the most common choice. In fact that I'm building a neural network, this is how I choose activation functions for the hidden layers as well.

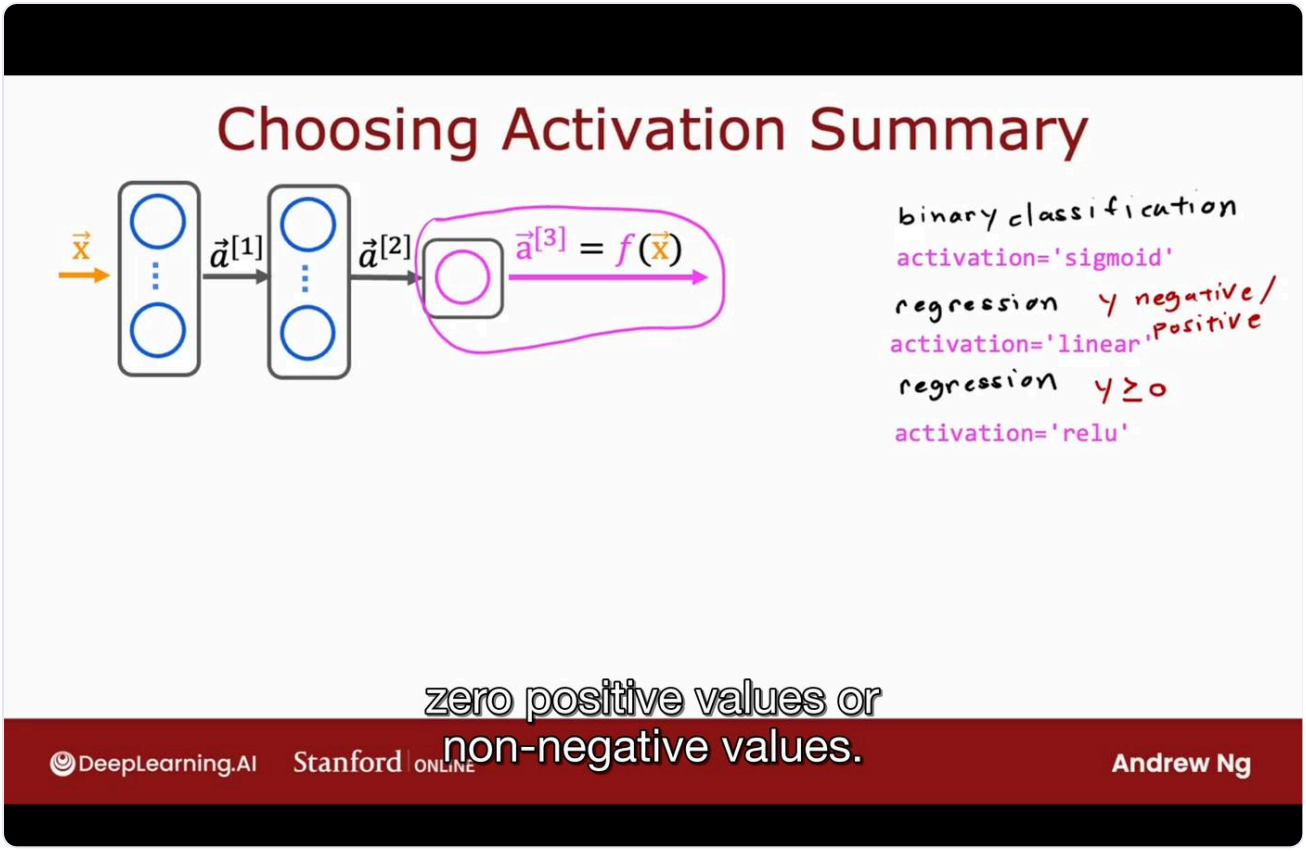


FIGURE 21.12

To summarize, here's what I recommend in terms of how you choose the activation functions for your neural network.

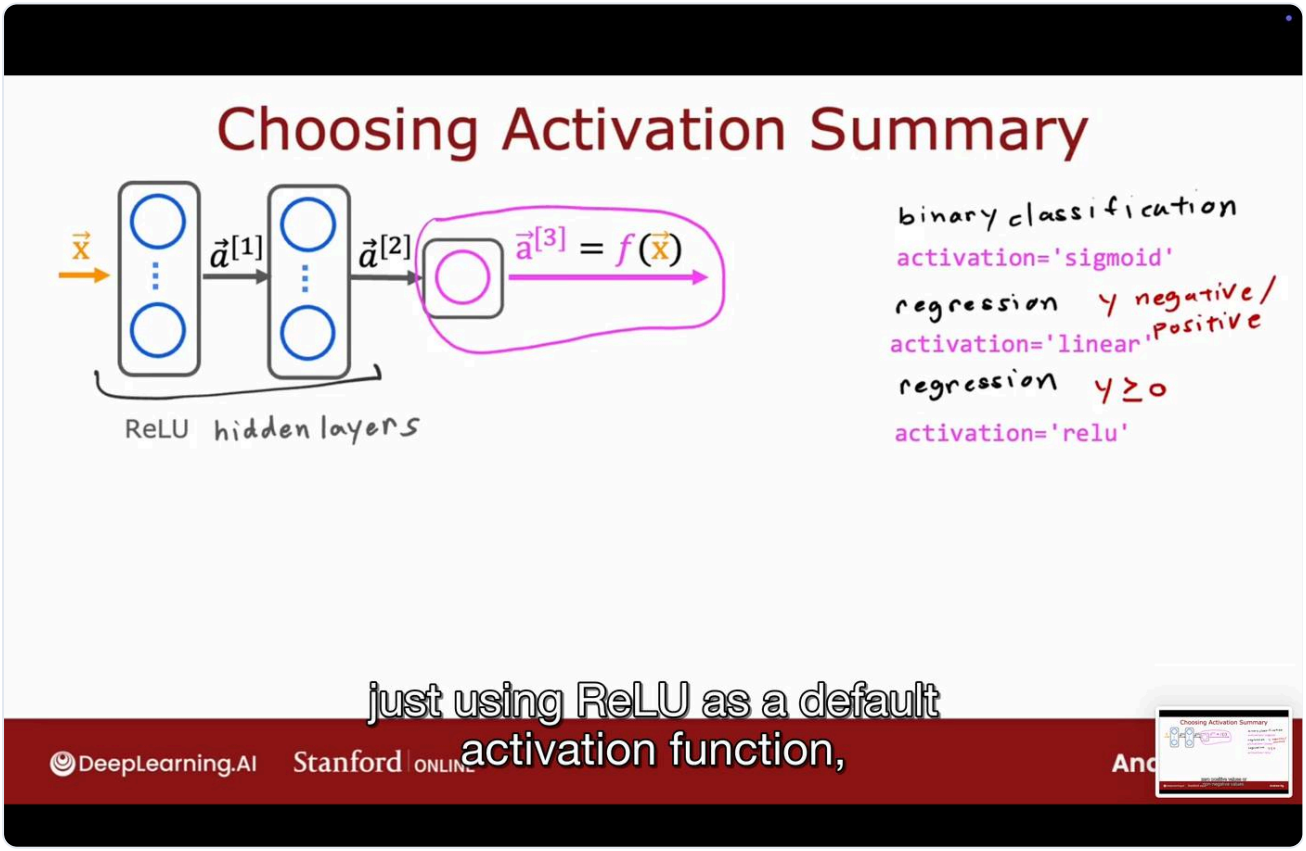


FIGURE 21.13

For the output layer, use a sigmoid, if you have a binary classification problem; linear, if y is a number that can take on positive or negative values, or use ReLU if y can take on only positive values or zero positive values or non-negative values. Then for the hidden layers I would recommend just using ReLU as a default activation function, and in TensorFlow, this is how you would implement it.

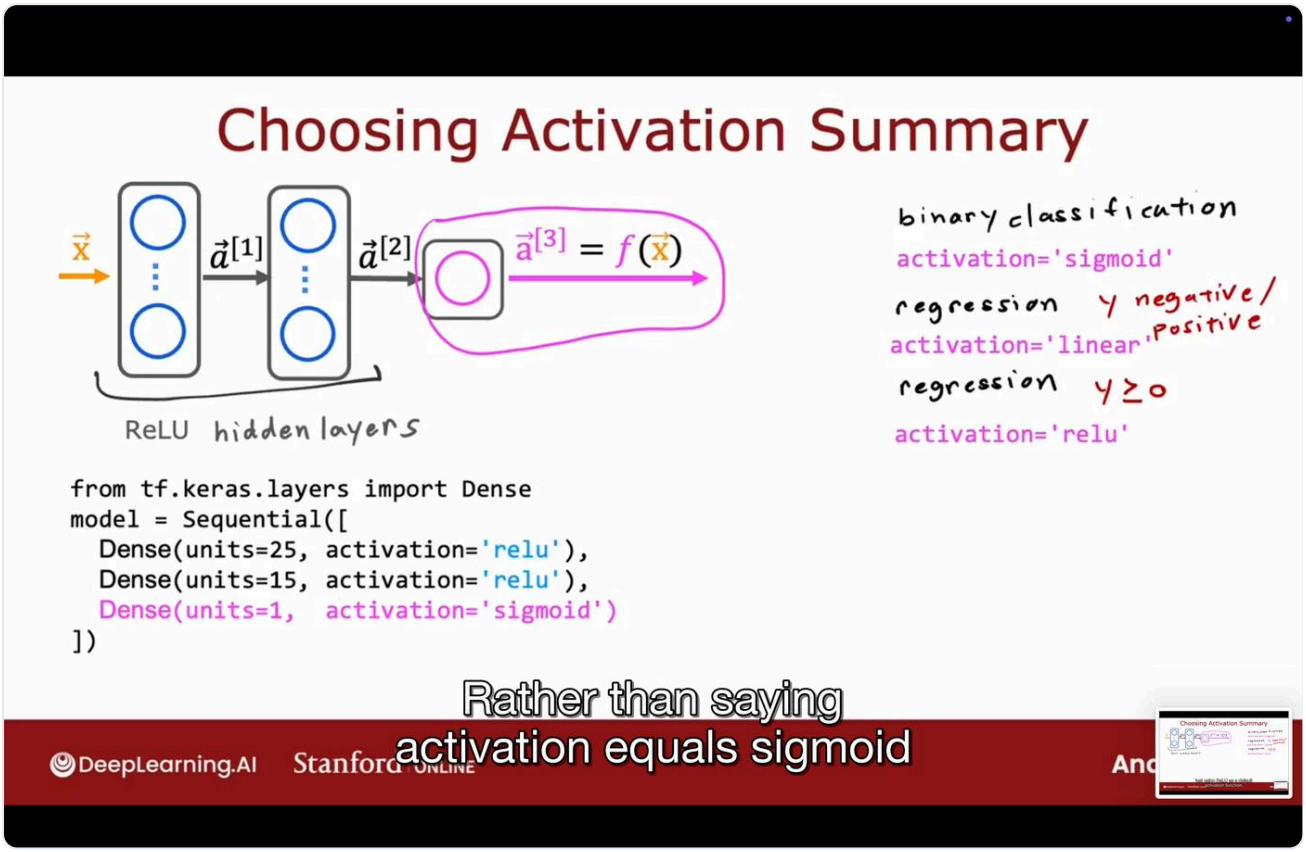


FIGURE 21.14

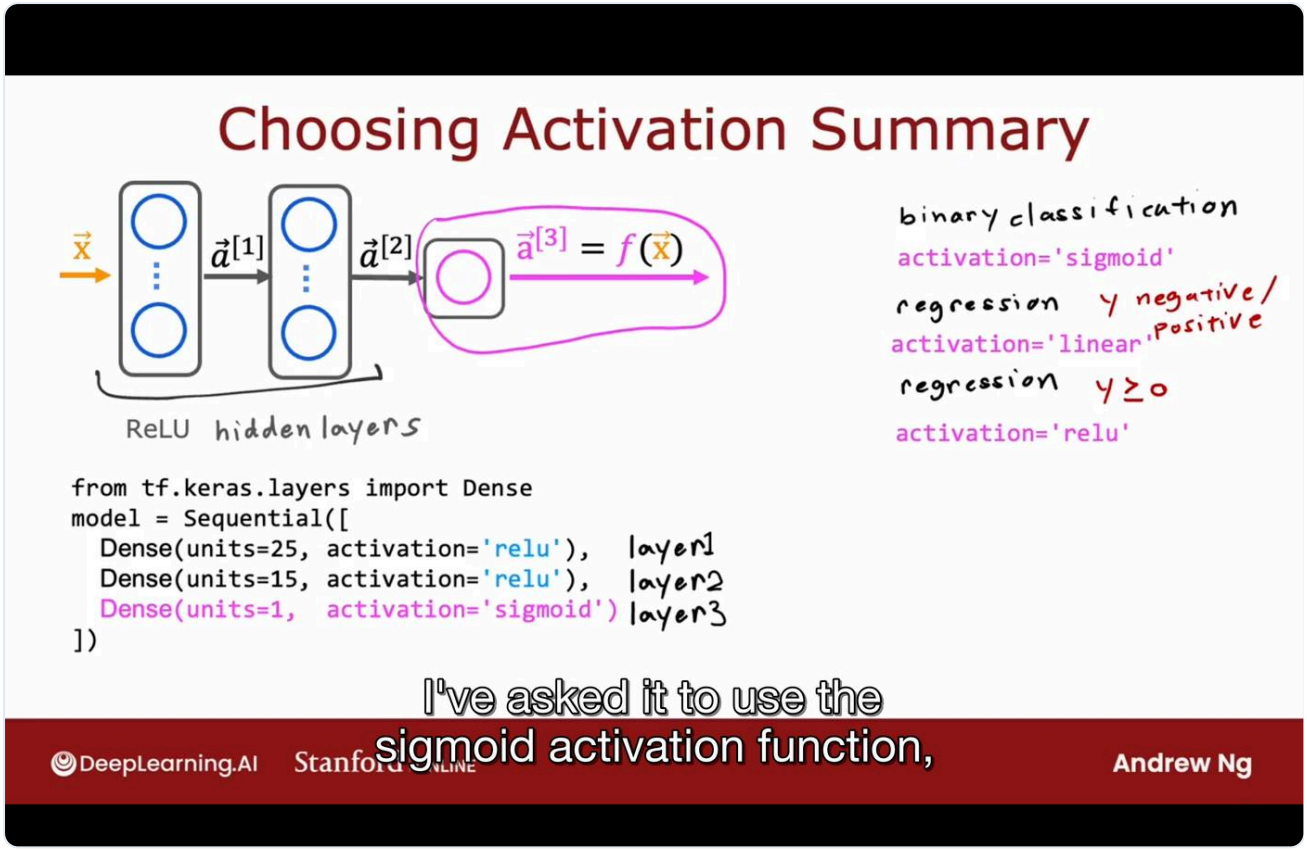


FIGURE 21.15

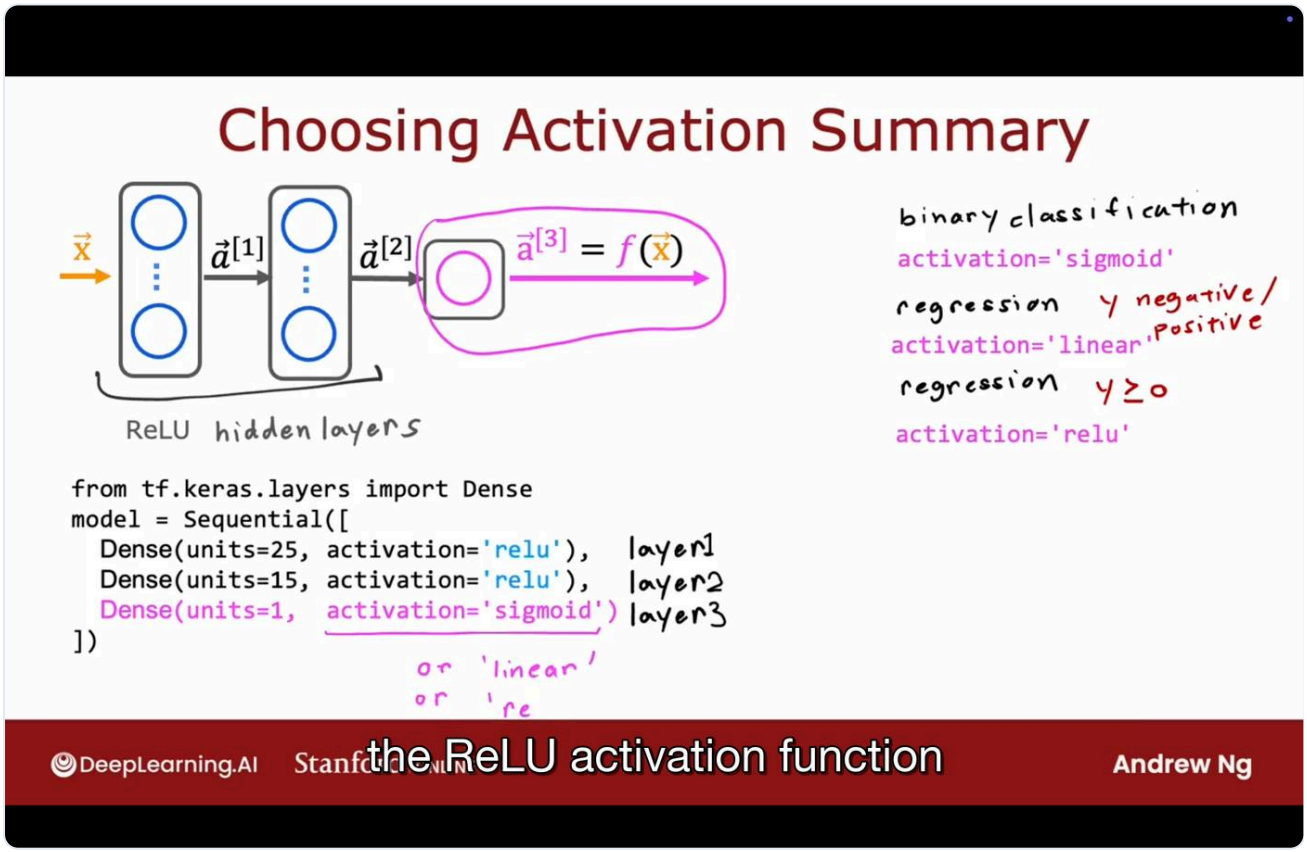


FIGURE 21.16

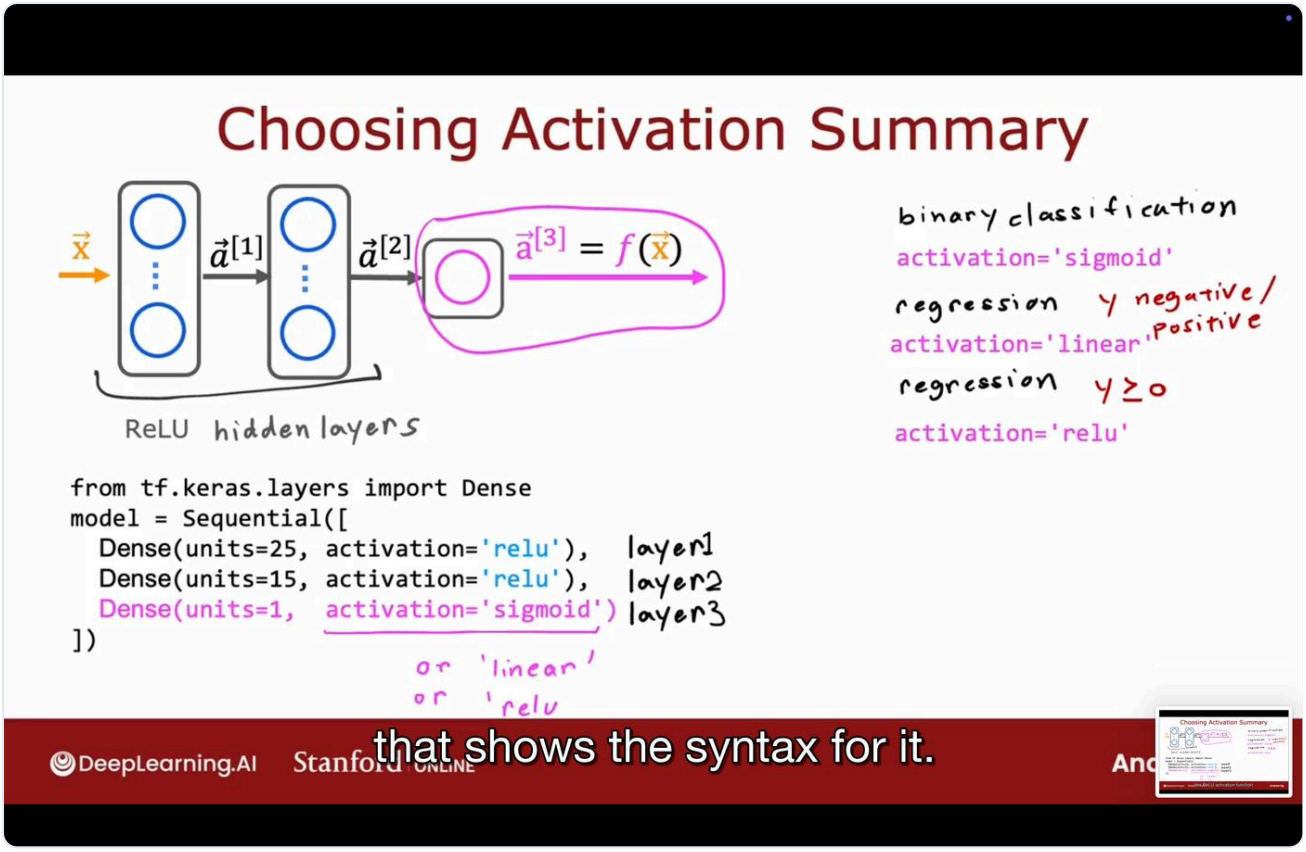


FIGURE 21.17

Rather than saying activation equals sigmoid as we had previously, for the hidden layers, that's the first hidden layer, the second hidden layer as TensorFlow to use the ReLU activation function, and then for the output layer in this example, I've asked it to use the sigmoid activation function, but if you wanted to use the linear activation function, is that, that's the syntax for it, or if you wanted to use the ReLU activation function that shows the syntax for it. With this richer set of activation functions, you'll be well-positioned to build much more powerful neural networks than just once using only the sigmoid activation function. By the way, if you look at the research literature, you sometimes hear of authors using even other activation functions, such as the tan h activation function or the LeakyReLU activation function or the swish activation function.

Every few years, researchers sometimes come up with another interesting activation function, and sometimes they do work a little bit better. For example, I've used the LeakyReLU activation function a few times in my work, and sometimes it works a little bit better than the ReLU activation function you've learned about in this video. But I think for the most part, and for the vast majority of applications what you learned about in this video would be good enough. Of course, if you want to learn more about other activation functions, feel free to look on the Internet, and there are just a small handful of cases where these other activation functions could be even more powerful as well.

With that, I hope you also enjoy practicing these ideas, these activation functions in the optional labs and in the practice labs. But this raises yet another question. Why do we even need activation functions at all? Why don't we just use the linear activation function or use no activation

function anywhere? It turns out this does not work at all. In the next video, let's take a look at why that's the case and why activation functions are so important for getting your neural networks to work.

LECTURE 22

Linear Example

Let's take a look at why neural networks need activation functions and why they just don't work if we were to use the linear activation function in every neuron in the neural network. Recall this demand prediction example. What would happen if we were to use a linear activation function for all of the nodes in this neural network? It turns out that this big neural network will become no different than just linear regression.

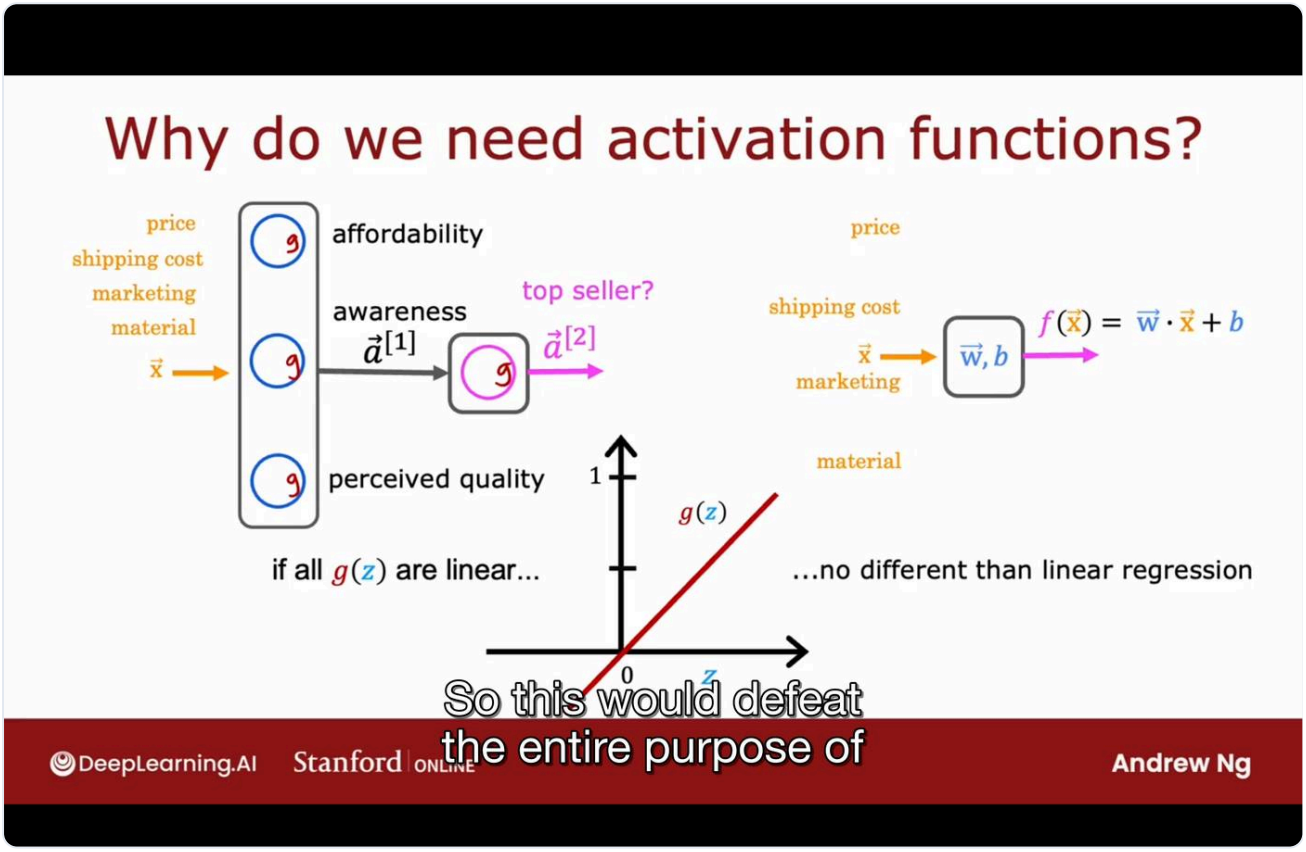


FIGURE 22.1 | Why Do We Need Activation Functions?

So this would defeat the entire purpose of using a neural network because it would then just not be able to fit anything more complex than the linear regression model that we learned about in the first course. Let's illustrate this with a simpler example. Let's look at the example of a neural network where the input x is just a number and we have one hidden unit with parameters w1 and b1 that outputs a1, which is here, just a number, and then the second layer is the output layer and it has also just one output unit with parameters w2 and b2 and then output a2, which is also just a number, just a scalar, which is the output of the neural network f of x.

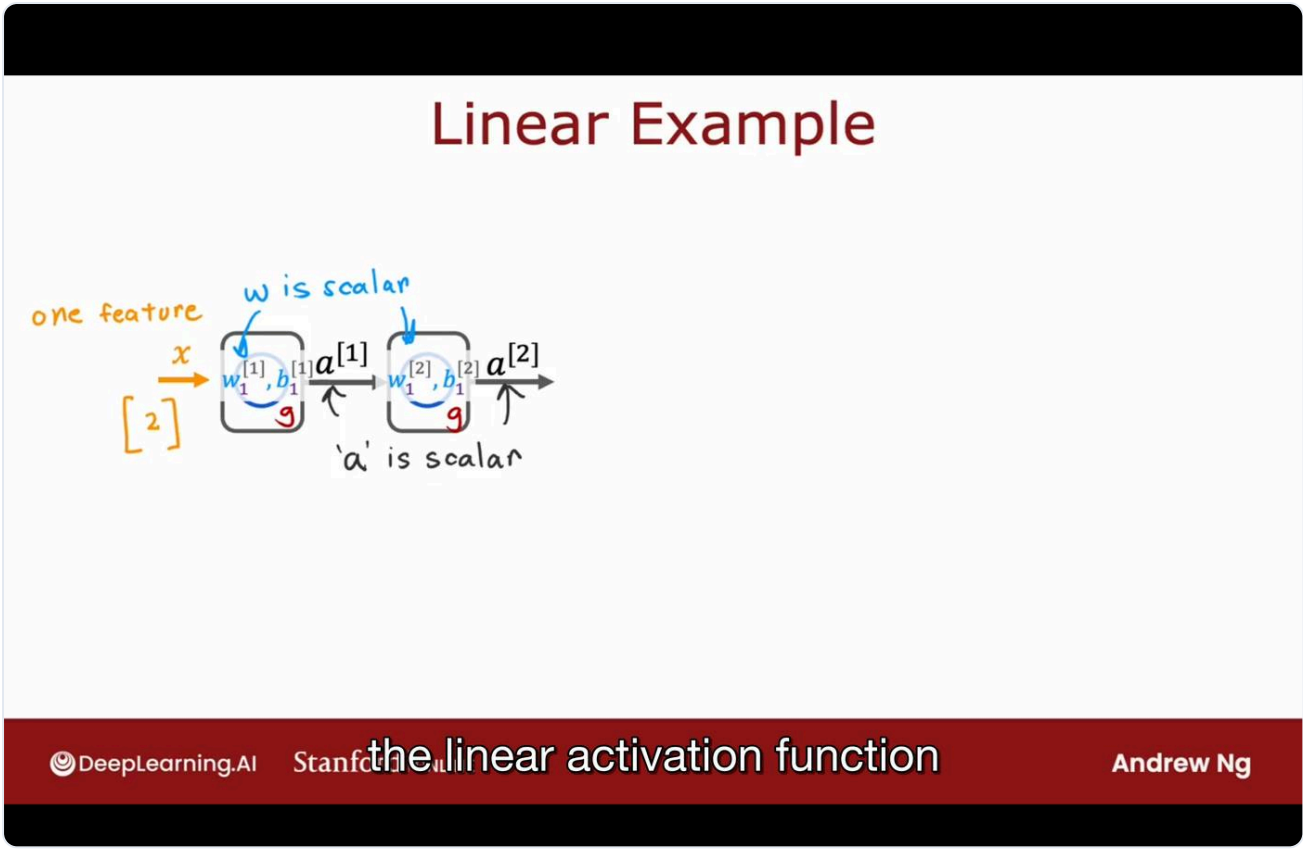


FIGURE 22.2 | Linear Example

Let's see what this neural network would do if we were to use the linear activation function g of z equals z everywhere. So to compute a1 as a function of x, the neural network will use a1 equals g of w1 times x plus b1. But g of z is equal to z.

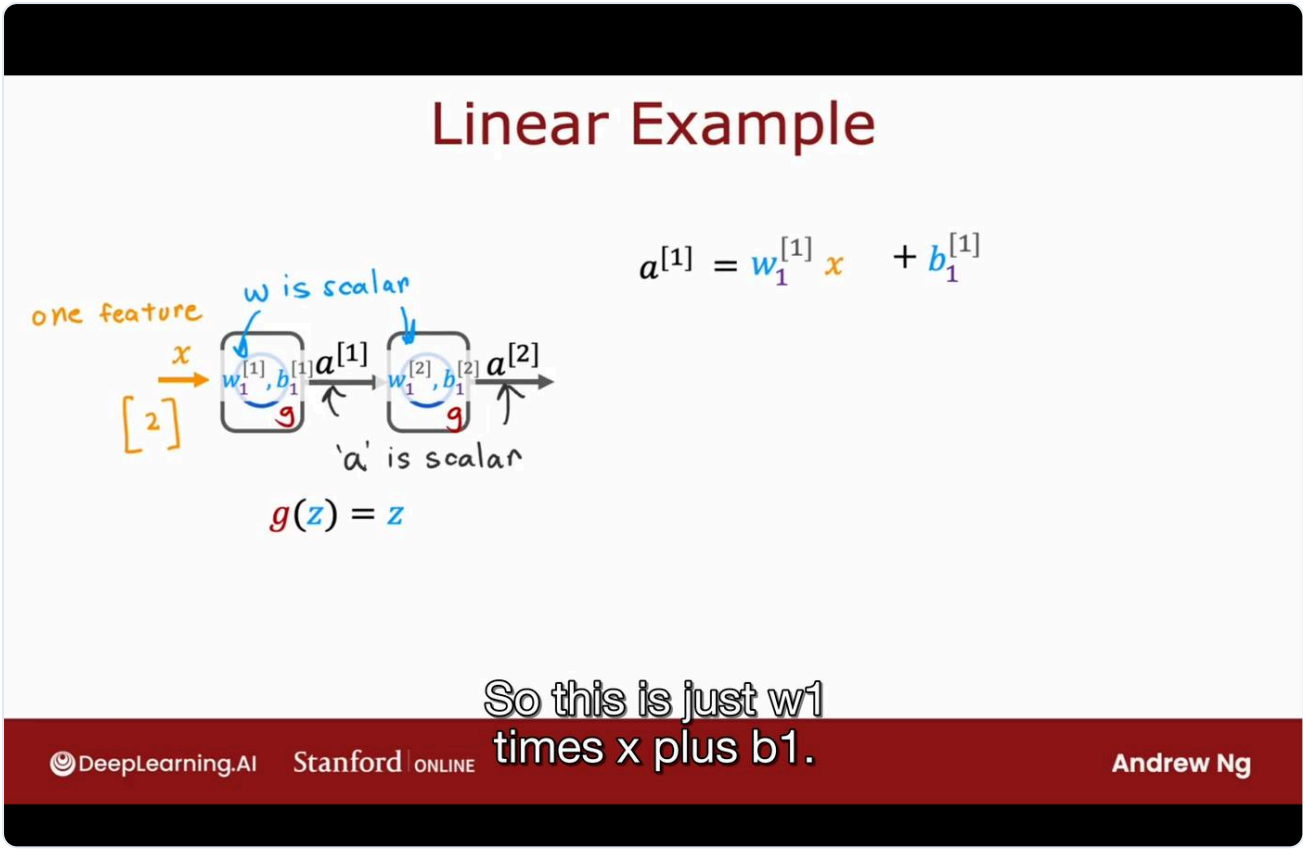


FIGURE 22.3 | Linear Example

So this is just w1 times x plus b1. Then a2 is equal to w2 times a1 plus b2, because g of z equals z.

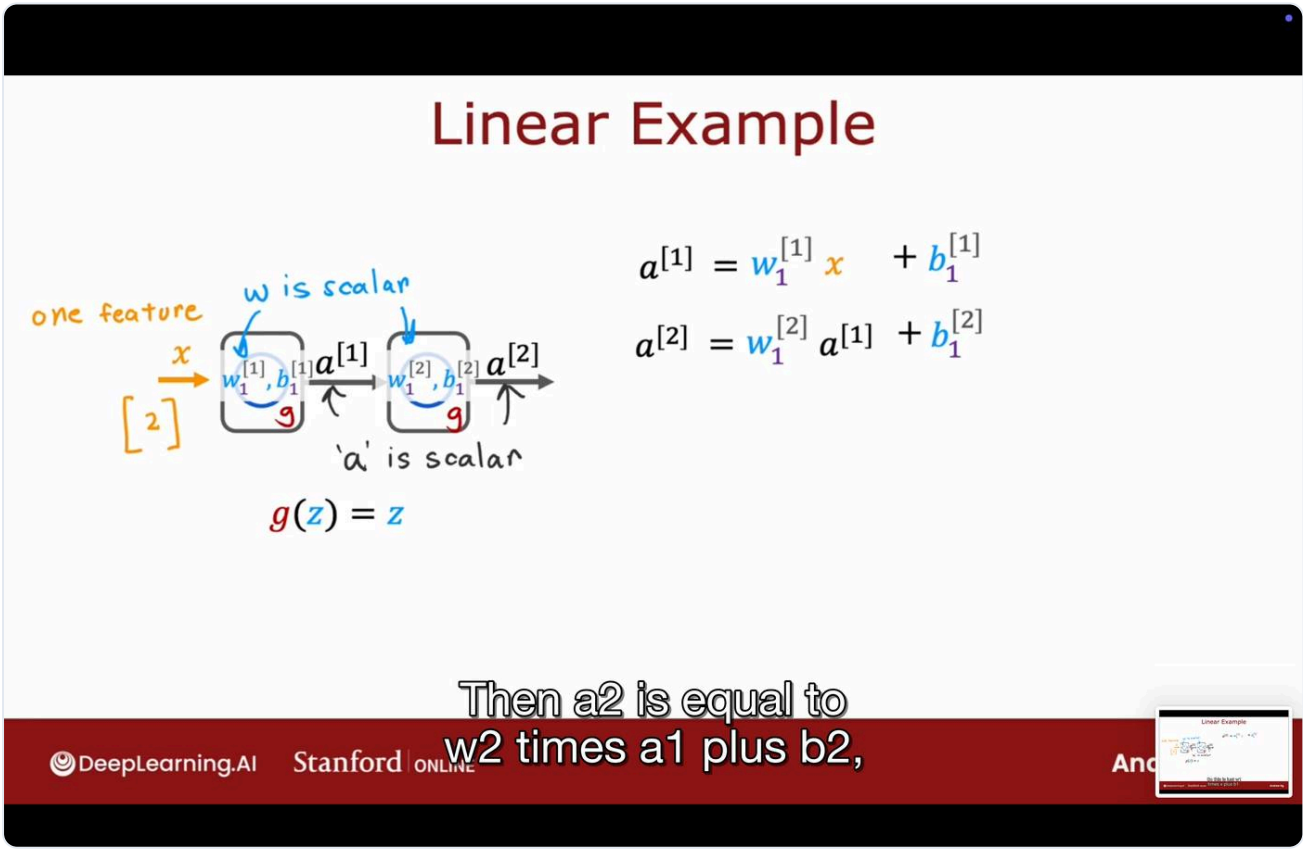


FIGURE 22.4 | Linear Example

Let me take this expression for a1 and substitute it in there. So that becomes w2 times w1 x plus b1 plus b2. If we simplify, this becomes w2, w1 times x plus w2, b1 plus b2.

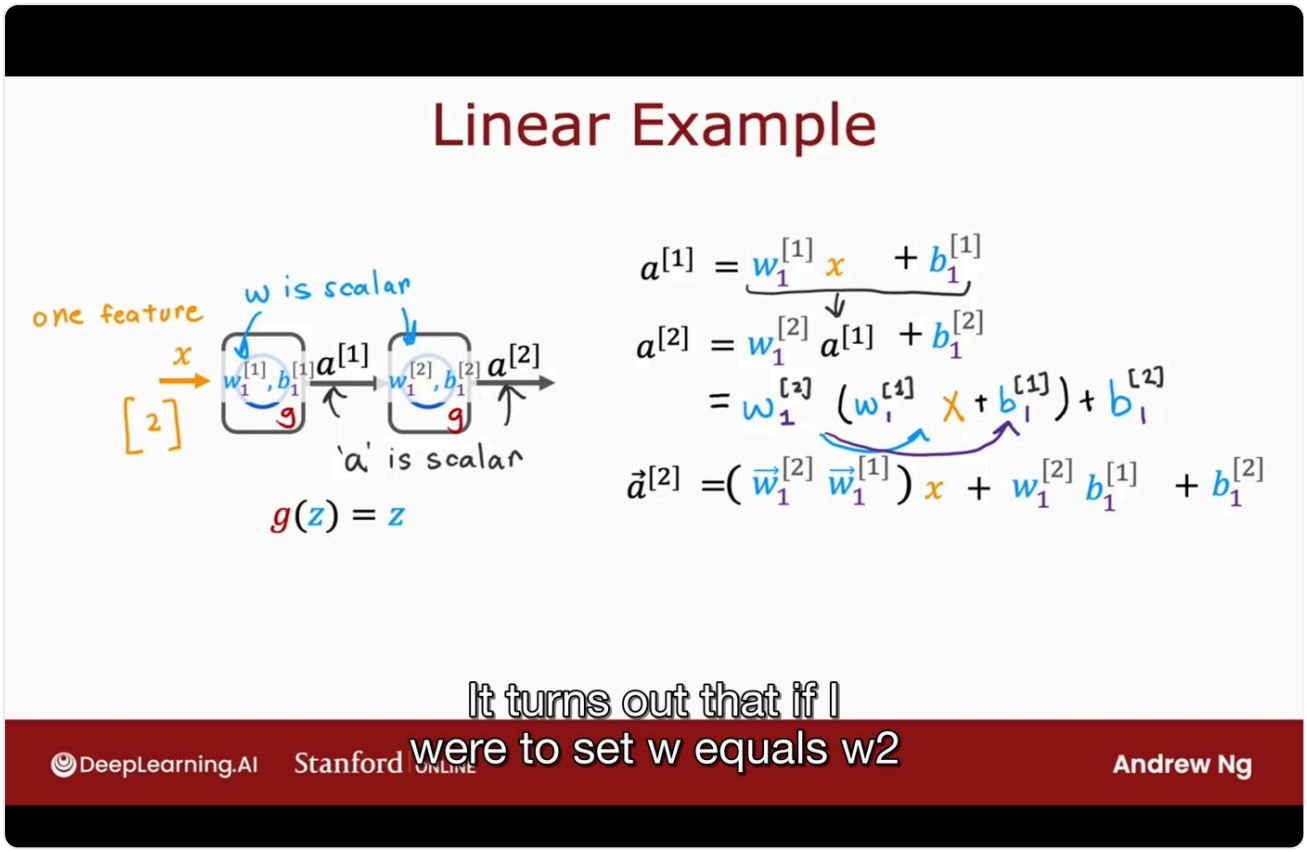


FIGURE 22.5 | Linear Example

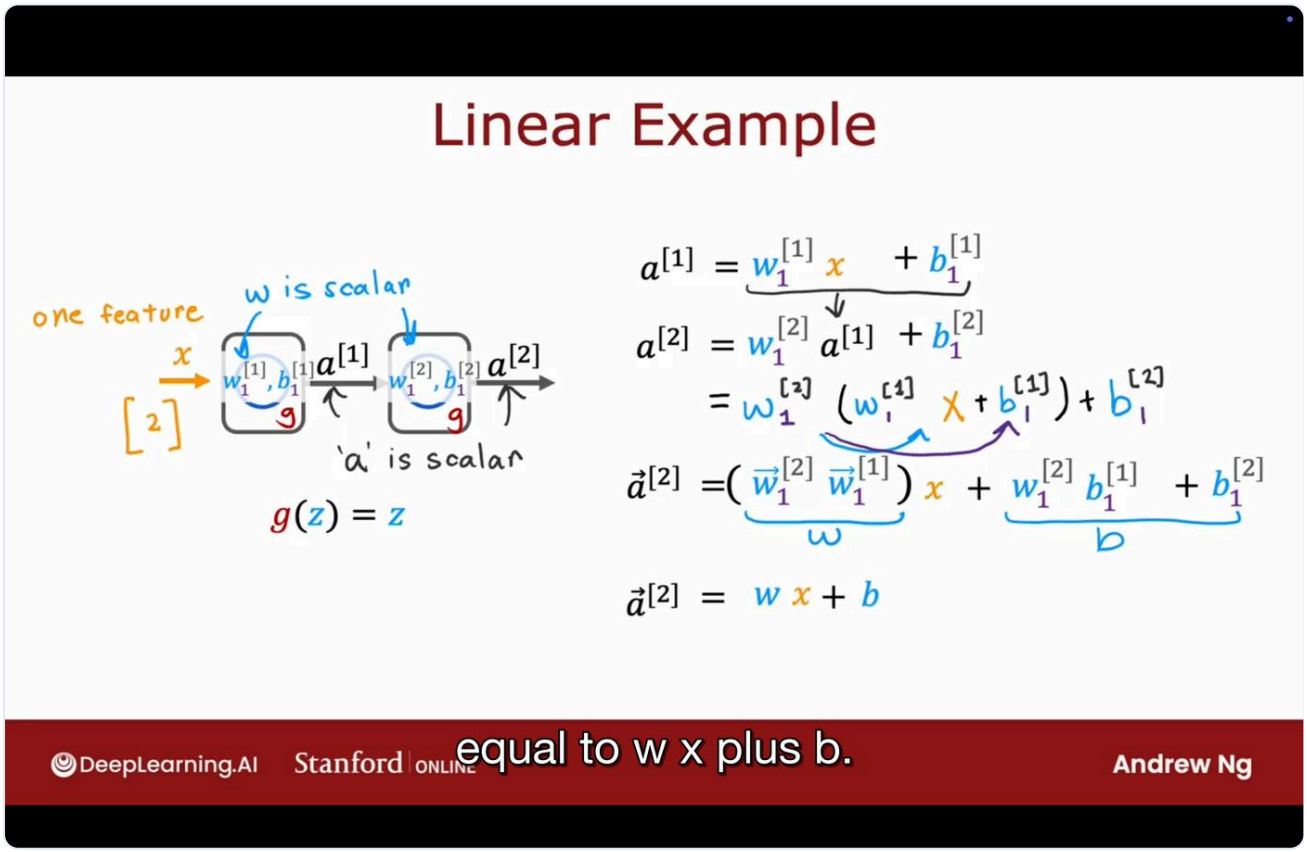


FIGURE 22.6 | Linear Example

It turns out that if I were to set w equals w2 times w1 and set b equals this quantity over here, then what we've just shown is that a2 is equal to w x plus b.

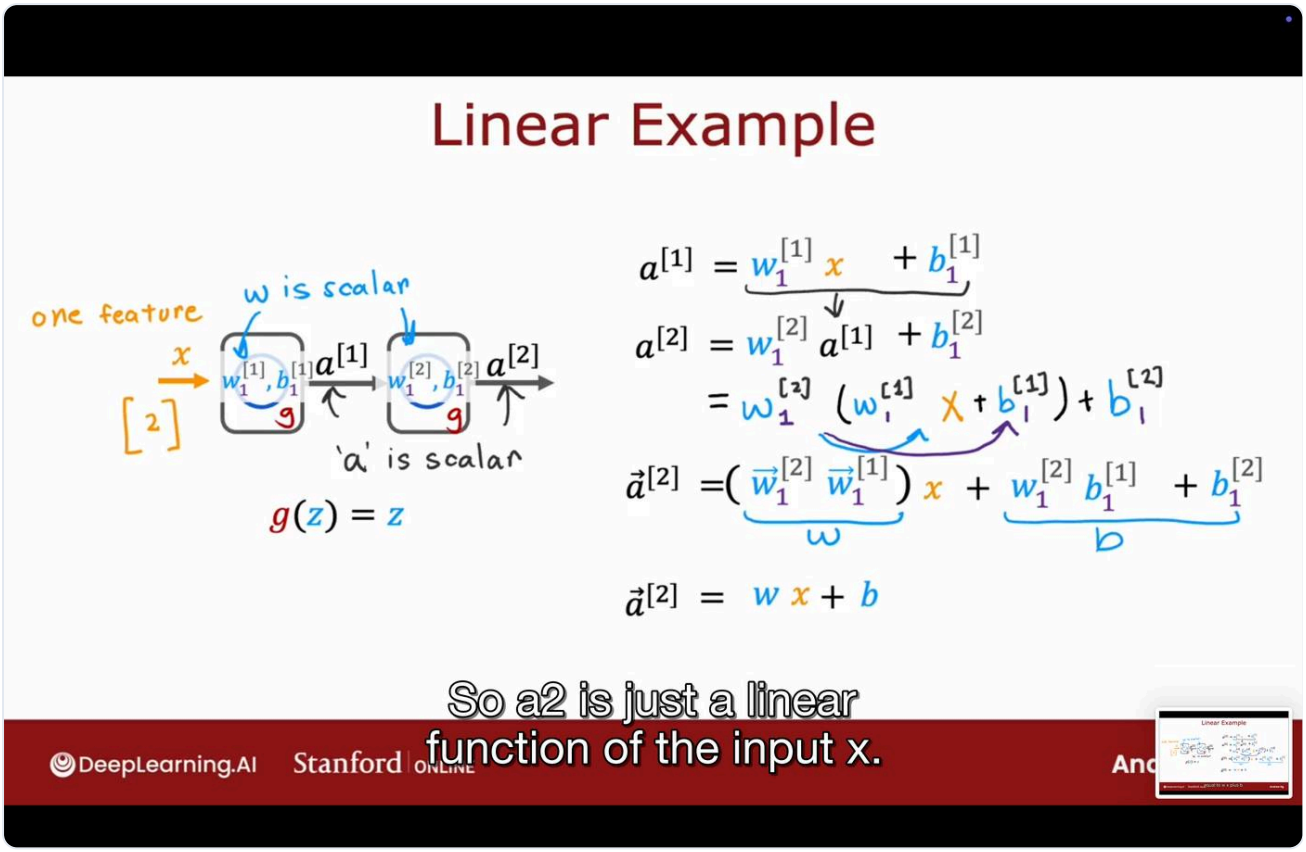


FIGURE 22.7 | Linear Example

So a2 is just a linear function of the input x.

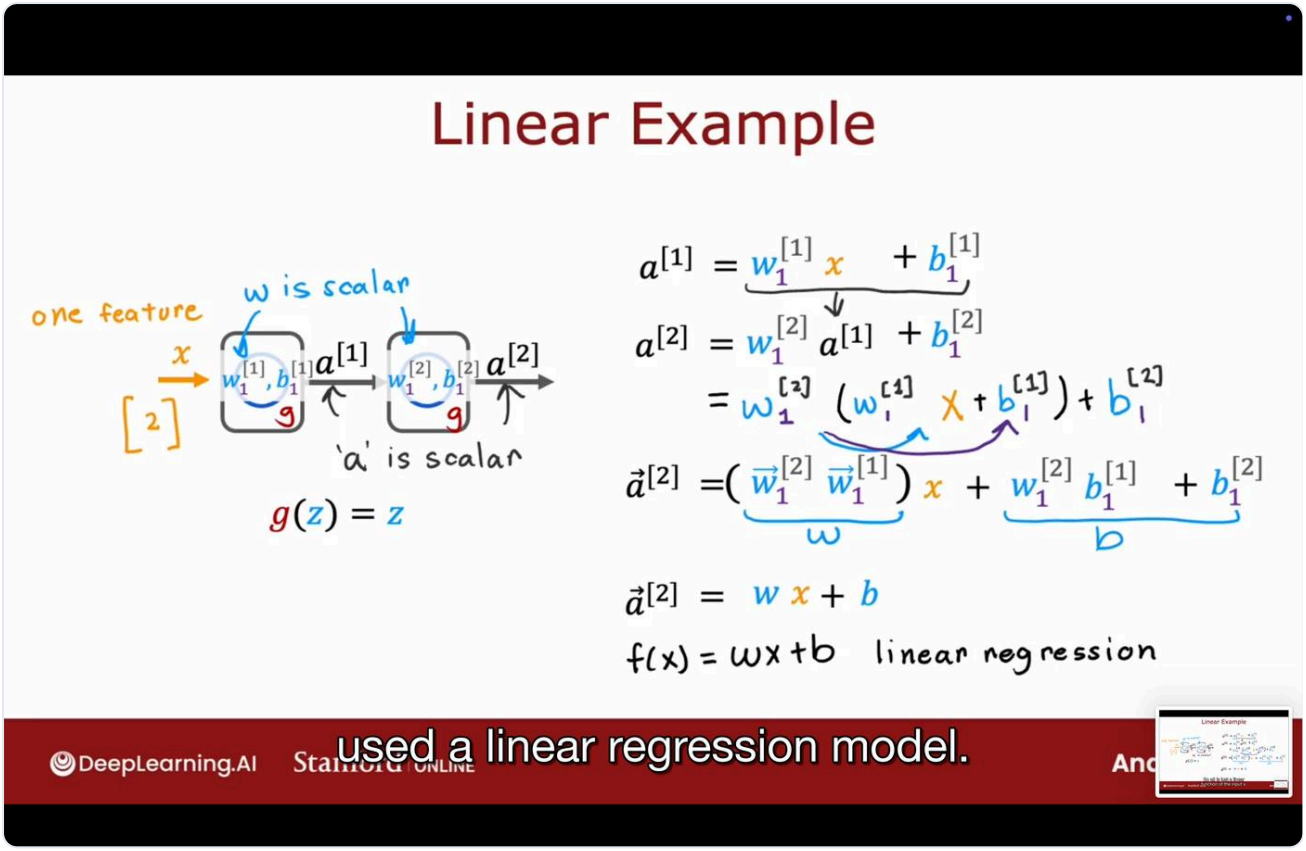


FIGURE 22.8 | Linear Example

Rather than using a neural network with one hidden layer and one output layer, we might as well have just used a linear regression model. If you're familiar with linear algebra, this result comes from the fact that a linear function of a linear function is itself a linear function. This is why having multiple layers in a neural network doesn't let the neural network compute any more complex features or learn anything more complex than just a linear function. So in the general case, if you had a neural network with multiple layers like this and say you were to use a linear activation function for all of the hidden layers and also use a linear activation function for the output layer, then it turns out this model will compute an output that is completely equivalent to linear regression.

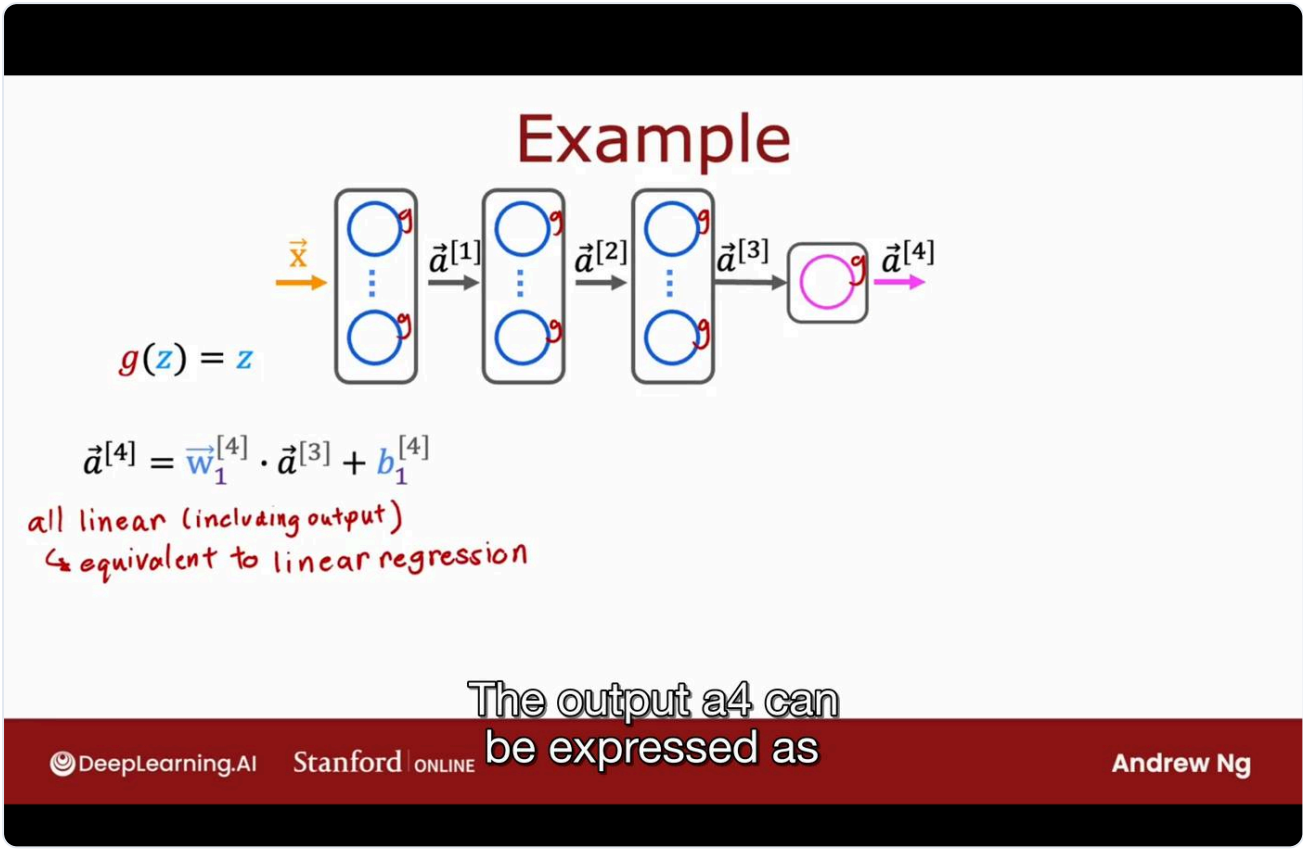


FIGURE 22.9

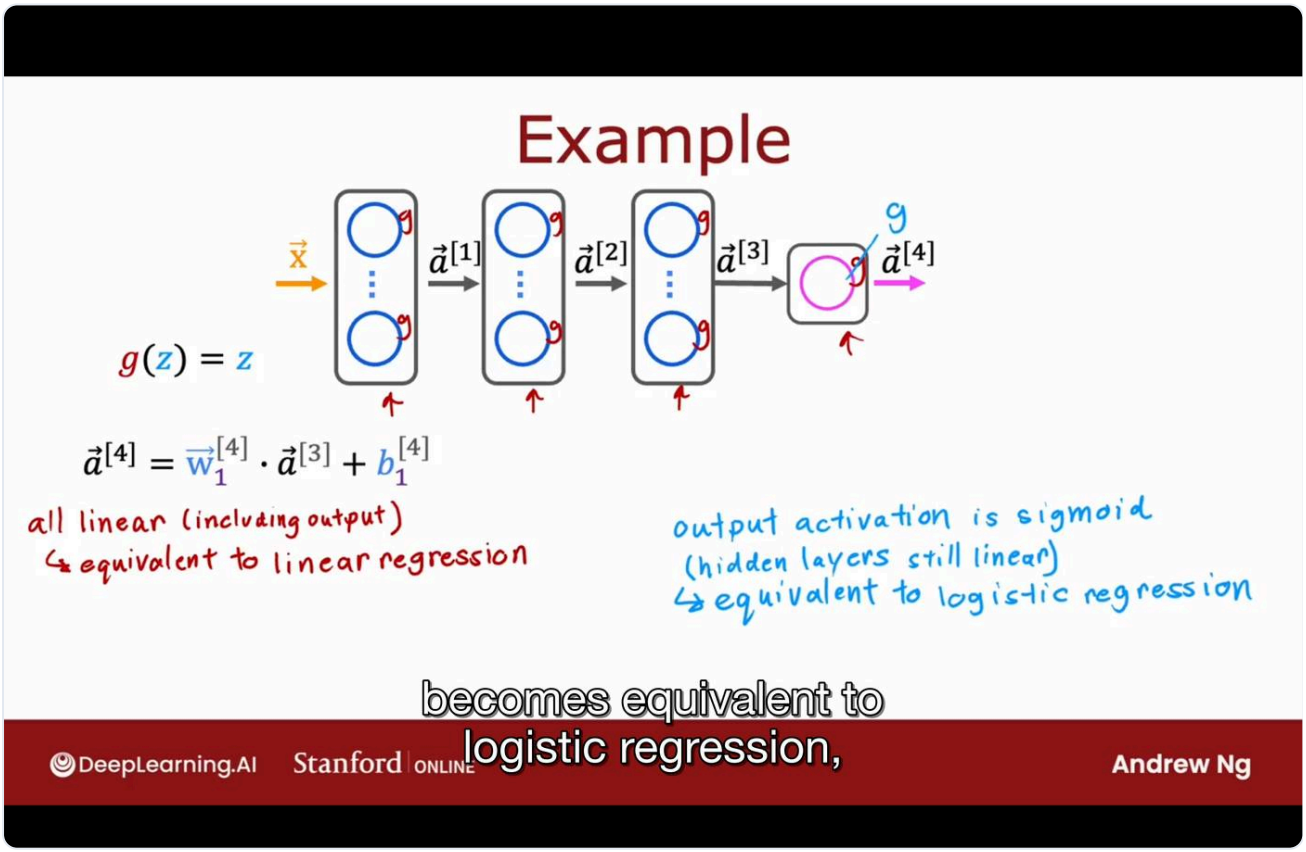


FIGURE 22.10

The output a4 can be expressed as a linear function of the input features x plus b.

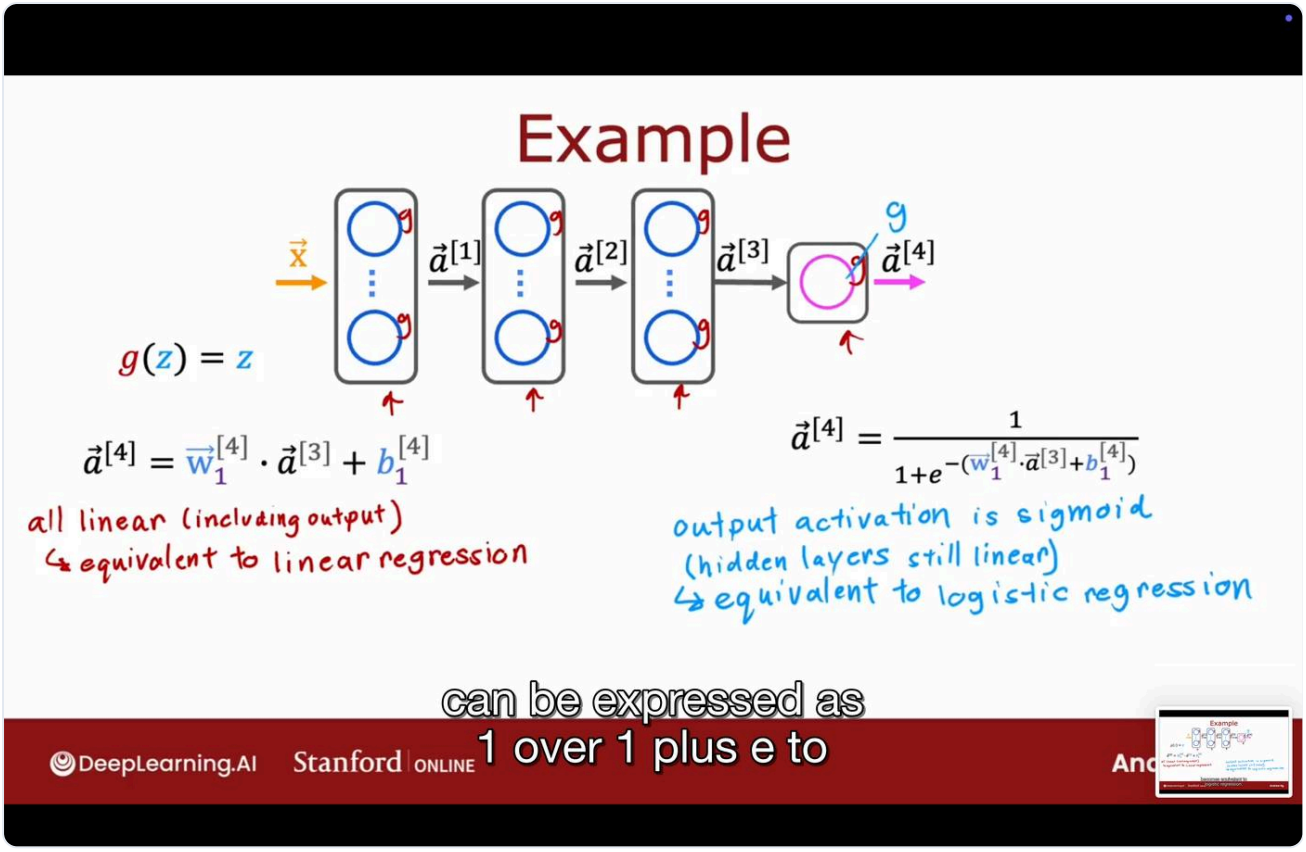


FIGURE 22.11

Or alternatively, if we were to still use a linear activation function for all the hidden layers, for these three hidden layers here, but we were to use a logistic activation function for the output layer, then it turns out you can show that this model becomes equivalent to logistic regression, and a4, in this case, can be expressed as 1 over 1 plus e to the negative wx plus b for some values of w and b. So this big neural network doesn't do anything that you can't also do with logistic regression.

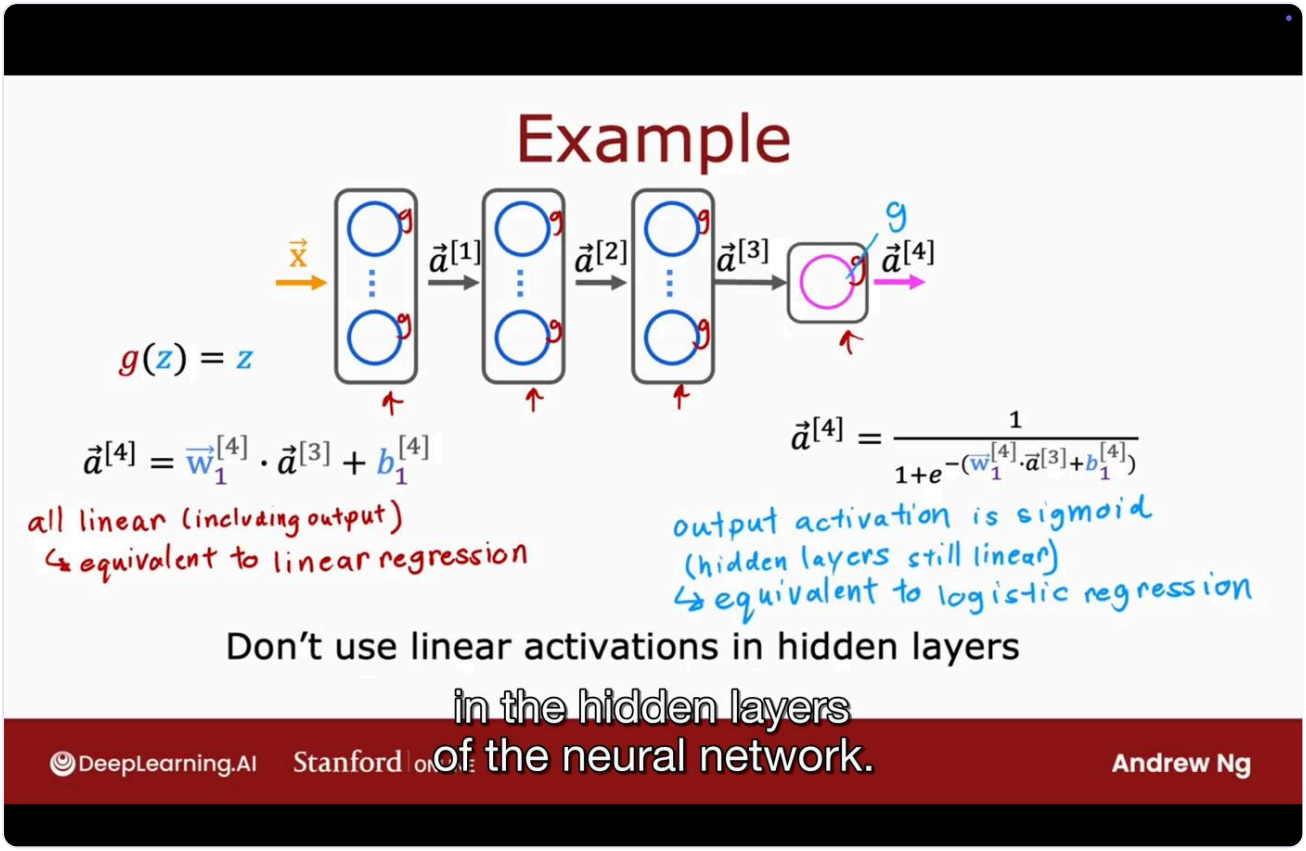


FIGURE 22.12

That's why a common rule of thumb is don't use the linear activation function in the hidden layers of the neural network.

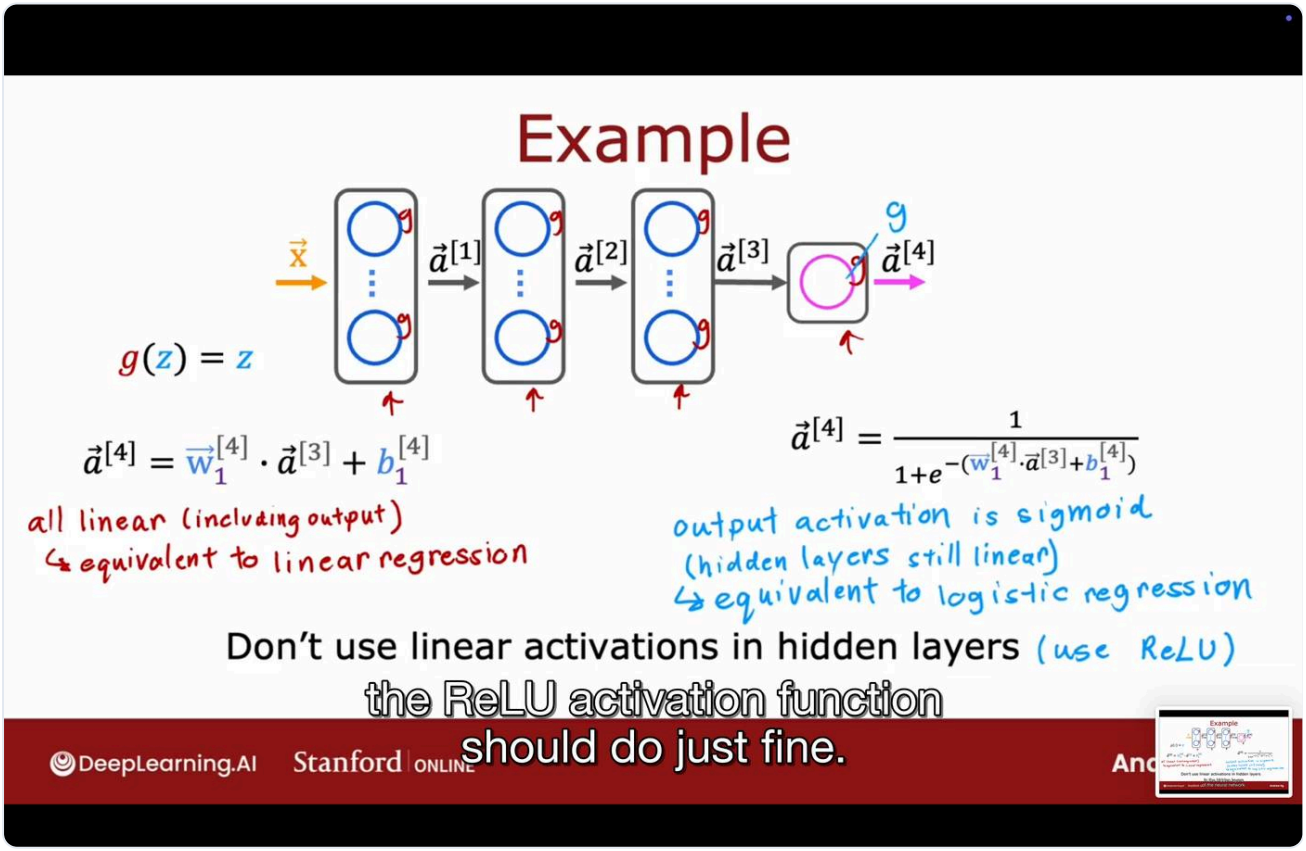


FIGURE 22.13

In fact, I recommend typically using the ReLU activation function should do just fine. So that's why a neural network needs activation functions other than just the linear activation function everywhere. So far, you've learned to build neural networks for binary classification problems where y is either zero or one. As well as for regression problems where y can take negative or positive values, or maybe just positive and non-negative values. In the next video, I'd like to share with you a generalization of what you've seen so far for classification. In particular, when y doesn't just take on two values, but may take on three or four or ten or even more categorical values.

Let's take a look at how you can build a neural network for that type of classification problem.

LECTURE 23

Multiclass Classification Example

Multiclass classification refers to classification problems where you can have more than just two possible output labels so not just zero or 1. Let's take a look at what that means. For the handwritten digit classification problems we've looked at so far, we were just trying to distinguish between the handwritten digits 0 and 1. But if you're trying to read protocols or zip codes in an envelope, well, there are actually 10 possible digits you might want to recognize. Or alternatively in the first course you saw the example if you're trying to classify whether patients may have any of three or five different possible diseases. That too would be a multiclass classification problem or one thing I've worked on a lot is visual defect inspection of parts manufacturer in the factory.

Where you might look at the picture of a pill that a pharmaceutical company has manufactured and try to figure out does it have a scratch effect or discoloration defects or a chip defect.

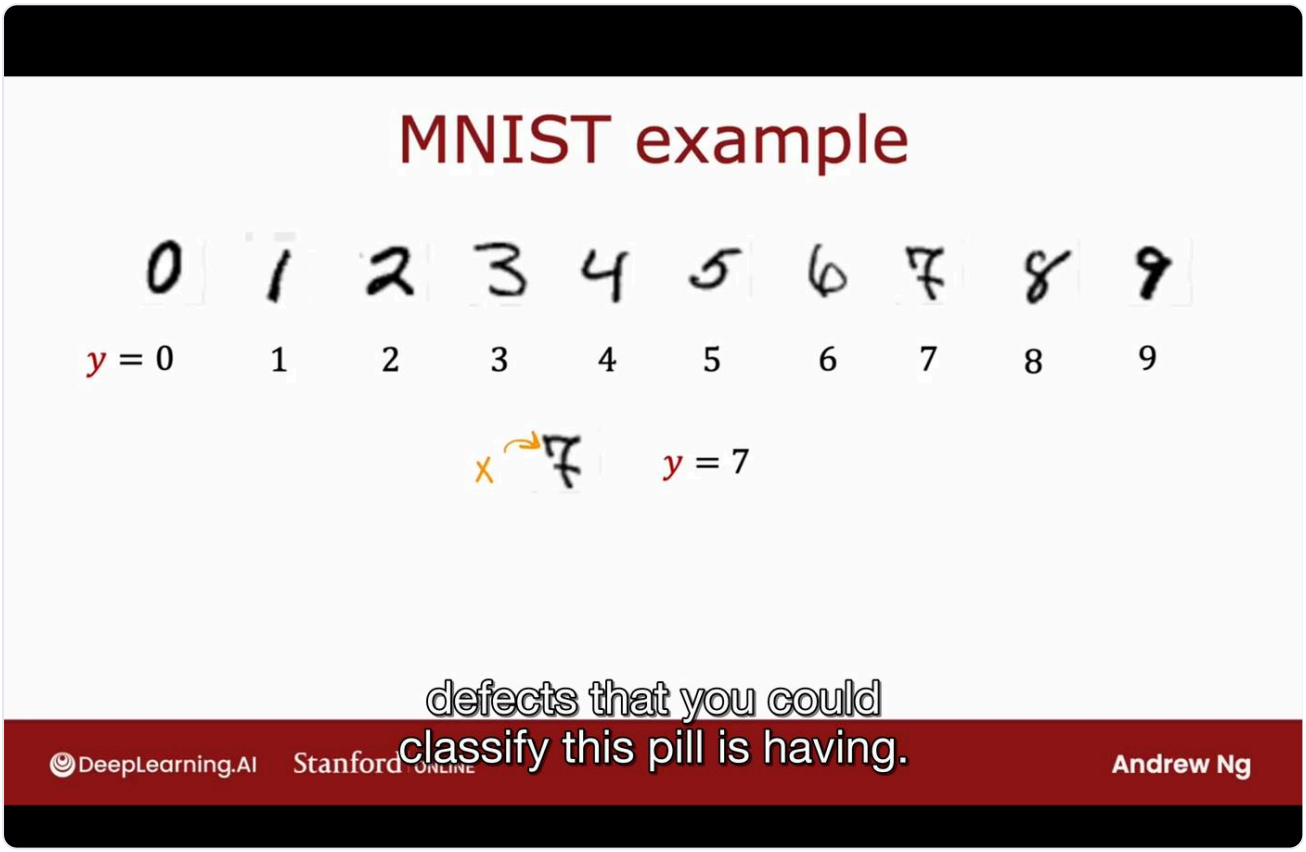


FIGURE 23.1

And this would again be multiple classes of multiple different types of defects that you could classify this pill is having.

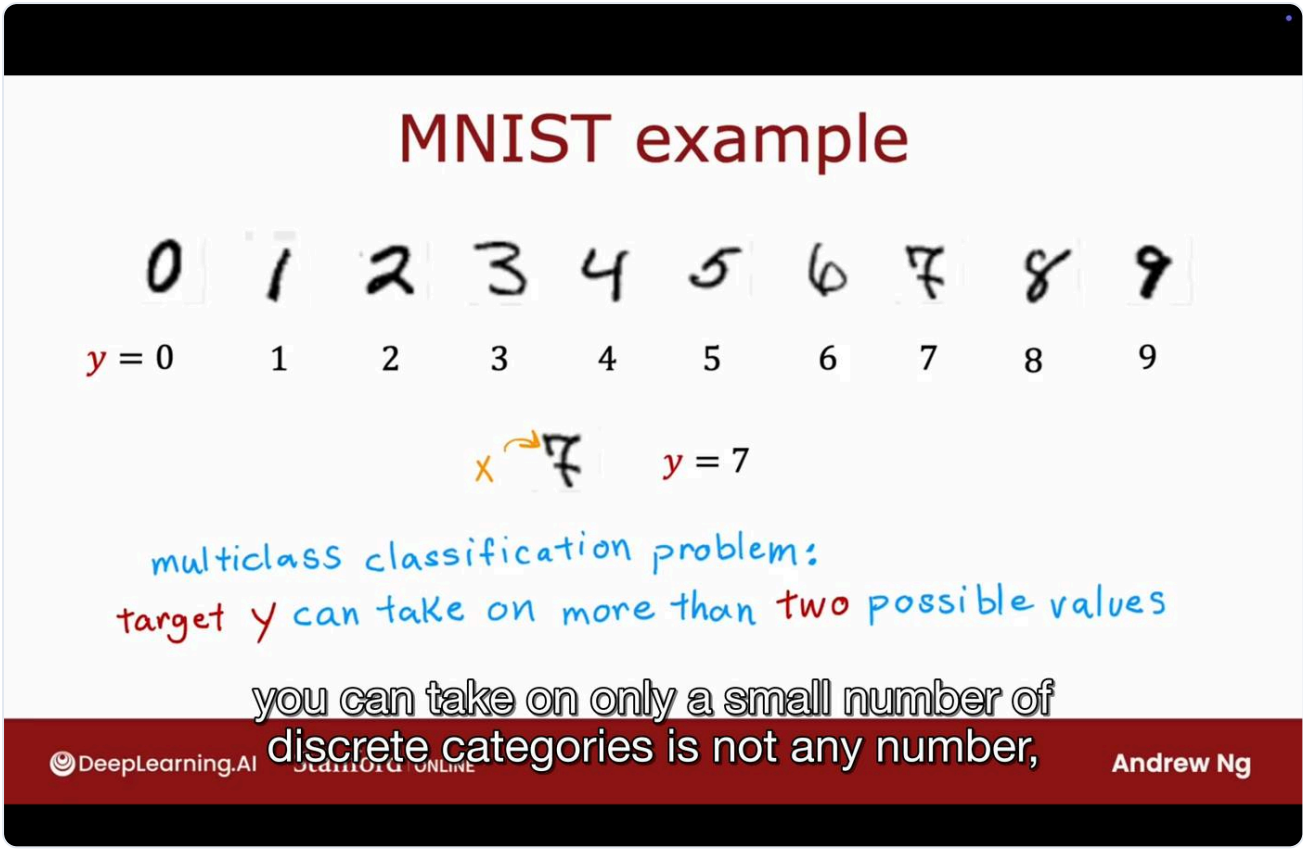


FIGURE 23.2

So a multiclass classification problem is still a classification problem in that y you can take on only a small number of discrete categories is not any number, but now y can take on more than just two possible values. So whereas previously for buying the classification, you may have had a data set like this with features x1 and x2. In which case logistic regression would fit model to estimate what the probability of y being 1, given the features x. Because y was either 01 with multiclass classification problems, you would instead have a data set that maybe looks like this. Where we have four classes where the Os represents one class, the xs represent another class.

The triangles represent the third class and the squares represent the fourth class. And instead of just estimating the chance of y being equal to 1, well, now want to estimate what's the chance that y is equal to 1, or what's the chance that y is equal to 2? Or what's the chance that y is equal to 3, or the chance of y being equal to 4?

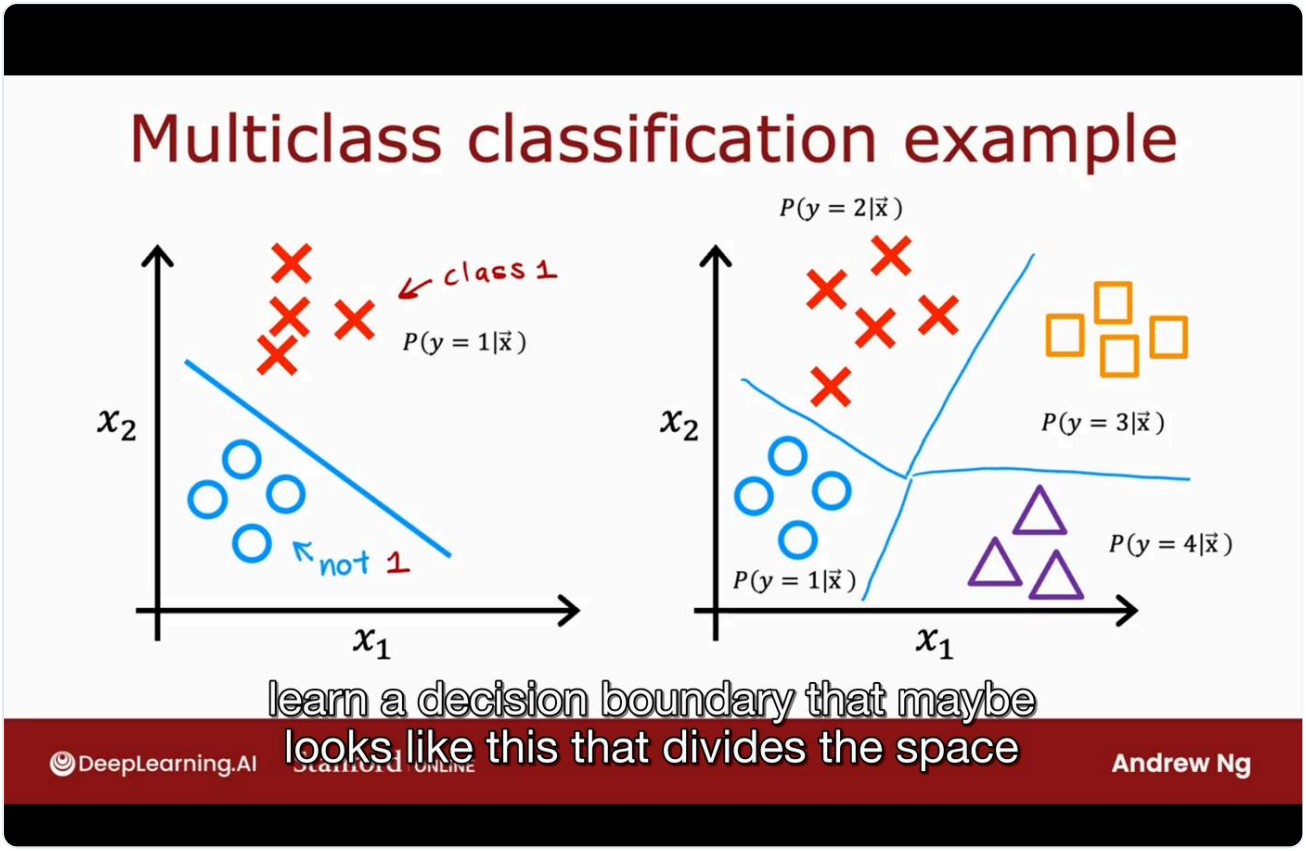


FIGURE 23.3 | Multiclass Classification Example

And it turns out that the algorithm you learned about in the next video can learn a decision boundary that maybe looks like this that divides the space exploded next to into four categories rather than just two categories. So that's the definition of the multiclass classification problem. In the next video, we'll look at the softmax regression algorithm which is a generalization of the logistic regression algorithm and using that you'll be able to carry out multiclass classification problems. And after that we'll take softmax regression and fit it into a new neural network so that you'll also be able to train a neural network to carry out multiclass classification problems.

Let's go on to the next video.

LECTURE 24

Logistic Regression

The softmax regression algorithm is a generalization of logistic regression, which is a binary classification algorithm to the multiclass classification contexts. Let's take a look at how works. Recall that logistic regression applies when y can take on two possible output values, either zero or one, and the way it computes this output is, you would first calculate z equals w.product of x plus b, and then you would compute what I'm going to call here a equals g of z which is a sigmoid function applied to z. We interpreted this as logistic regressions estimates of the probability of y being equal to 1 given those input features x.

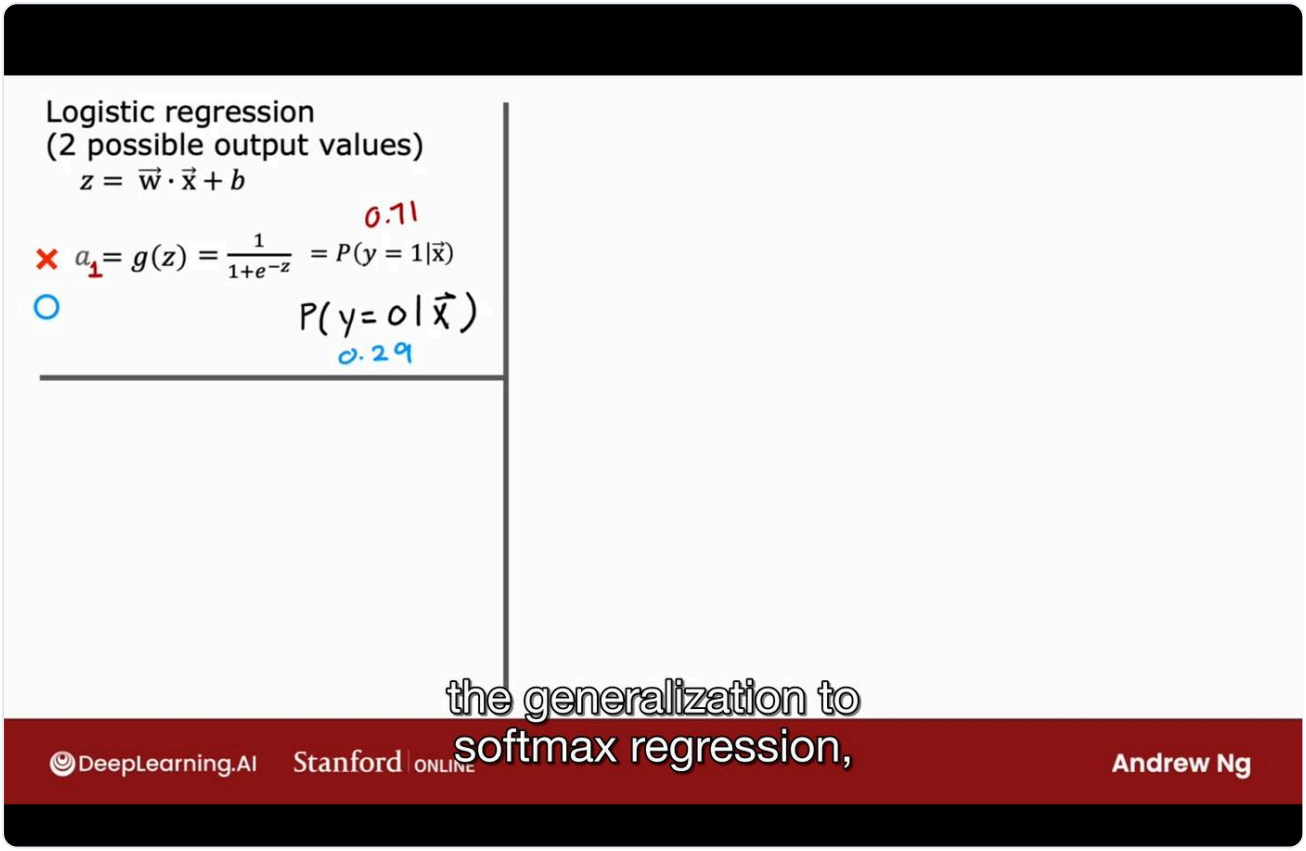


FIGURE 24.1 | Logistic Regression

Now, quick quiz question; if the probability of y equals 1 is 0.71, then what is the probability that y is equal to zero? Well, the chance of y being the one, and the chances of y being the zero, they've got to add up to one, right? So there's a 71 percent chance of it being one, there has to be a 29 percent or 0.29 chance of it being equal to zero.

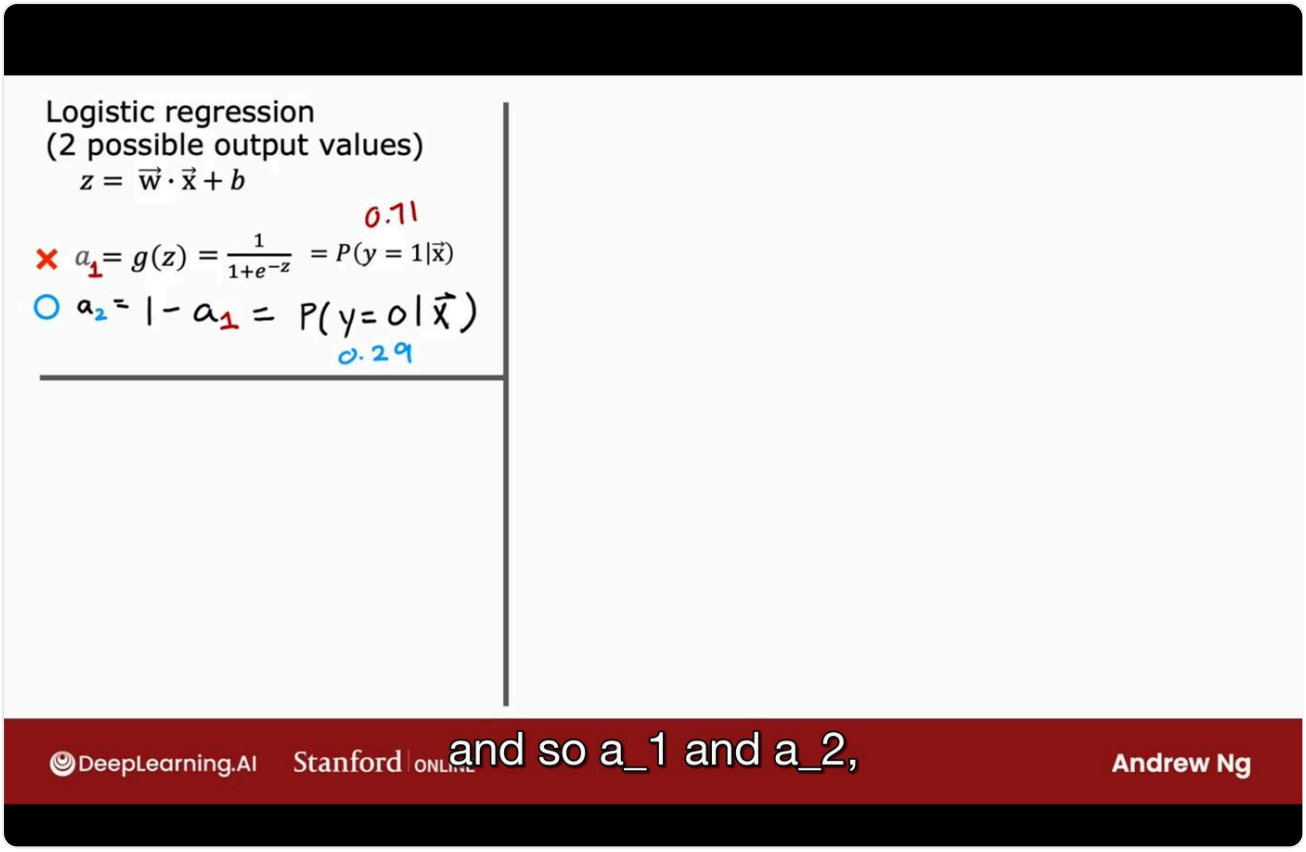


FIGURE 24.2 | Logistic Regression

To embellish logistic regression a little bit in order to set us up for the generalization to softmax regression, I'm going to think of logistic regression as actually computing two numbers: First a_1 which is this quantity that we had previously of the chance of y being equal to 1 given x, and second, I'm going to think of logistic regression as also computing a_2, which is 1 minus this which is just the chance of y being equal to zero given the input features x, and so a_1 and a_2, of course, have to add up to 1. Let's now generalize this to softmax regression, and I'm going to do this with a concrete example of when y can take on four possible outputs, so y can take on the values 1, 2, 3 or 4. Here's what softmax regression will do, it will compute z_1 as w_1.product with x plus b_1, and then z_2 equals w_2.product of x plus b_2, and so on for z_3 and z_4.

Here, w_1, w_2, w_3, w_4 as well as b_1, b_2, b_3, b_4, these are the parameters of softmax regression. Next, here's the formula for softmax regression, we'll compute a_1 equals e^z_1 divided by e^ z_1, plus e^ z_2, plus e^z_3 plus, e^ z_4, and a_1 will be interpreted as the algorithms estimate of the chance of y being equal to 1 given the input features x. Then the formula for softmax regression, we'll compute a_2 equals e^ z_2 divided by the same denominator, e^z_1 plus e^z_2, plus e^z_3, plus e^z4, and we'll interpret a_2 as the algorithms estimate of the chance that y is equal to 2 given the input features x. Similarly for a_3, where here the numerator is now e^z_3 divided by the same denominator, that's the estimated chance of y being a_3, and similarly a_4 takes on this expression.

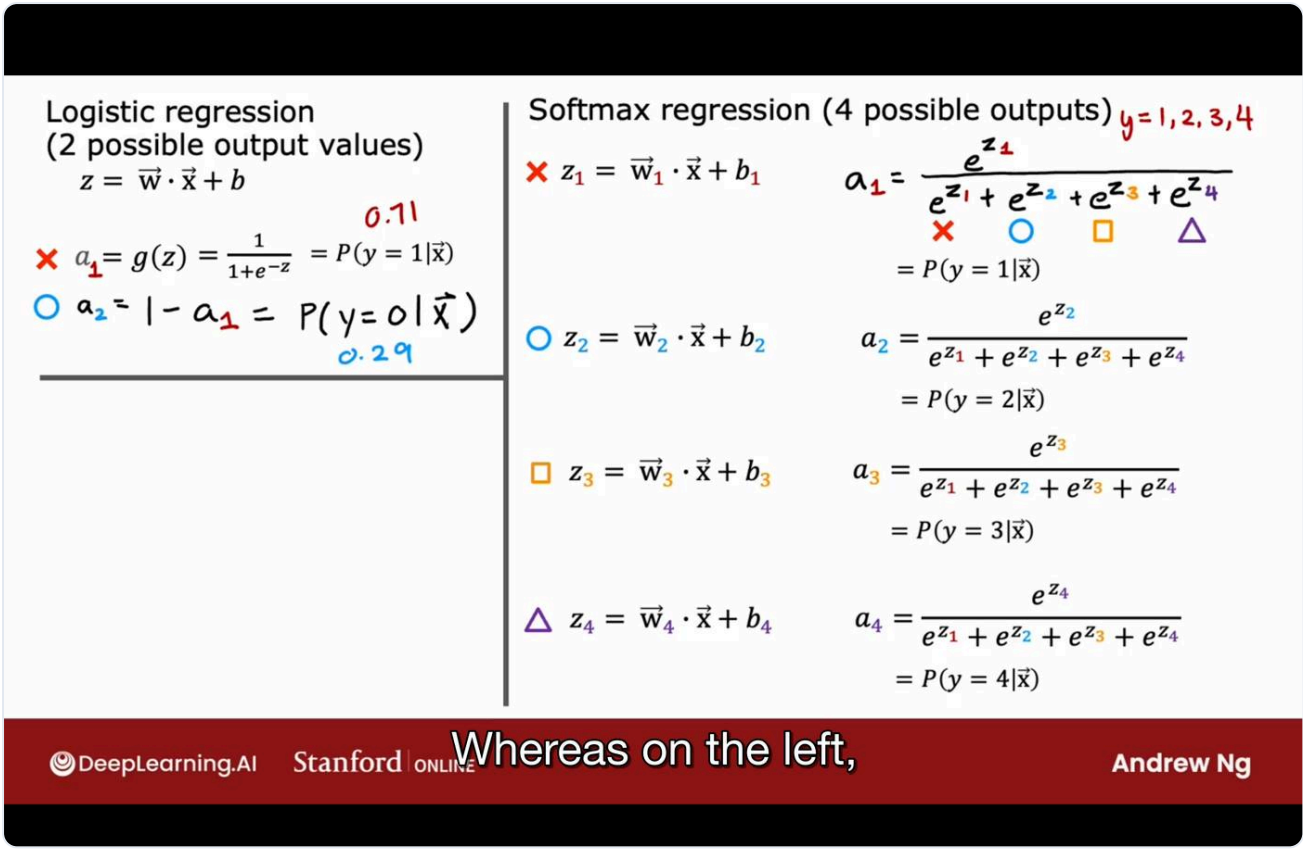


FIGURE 24.3 | Logistic Regression

Whereas on the left, we wrote down the specification for the logistic regression model, these equations on the right are our specification for the softmax regression model. It has parameters w_1 through w_4, and b_1 through b_4, and if you can learn appropriate choices to all these parameters, then this gives you a way of predicting what's the chance of y being 1, 2, 3 or 4, given a set of input features x. Quick quiz, let's see, run softmax regression on a new input x, and you find that a_1 is 0.30, a_2 is 0.20, a_3 is 0.15. What do you think a_4 will be? Why don't you take a look at this quiz and see if you can figure out the right answer?

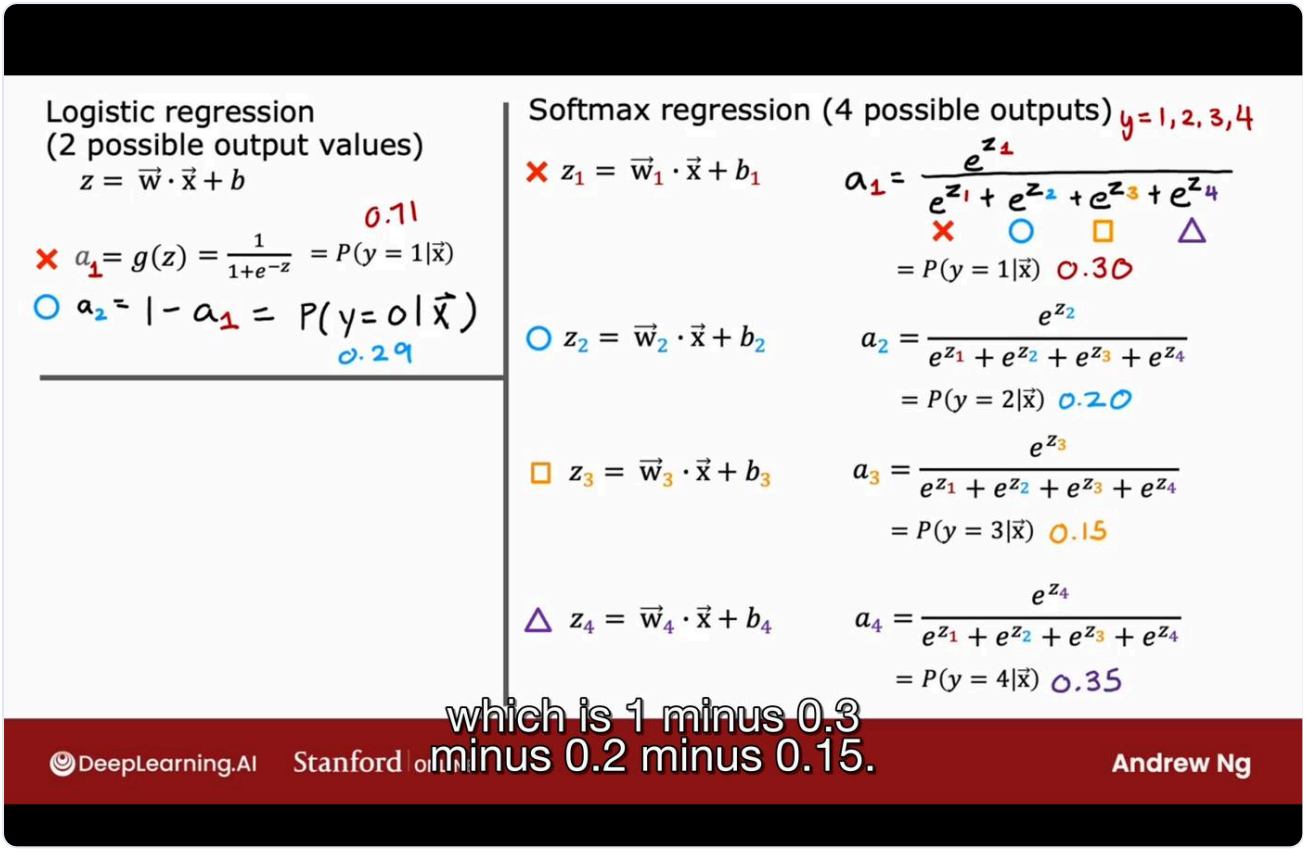


FIGURE 24.4 | Logistic Regression

You might have realized that because the chance of y take on the values of 1, 2, 3 or 4, they have to add up to one, a_4 the chance of y being with a four has to be 0.35, which is 1 minus 0.3 minus 0.2 minus 0.15. Here I wrote down the formulas for softmax regression in the case of four possible outputs, and let's now write down the formula for the general case for softmax regression. In the general case, y can take on n possible values, so y can be 1, 2, 3, and so on up to n.

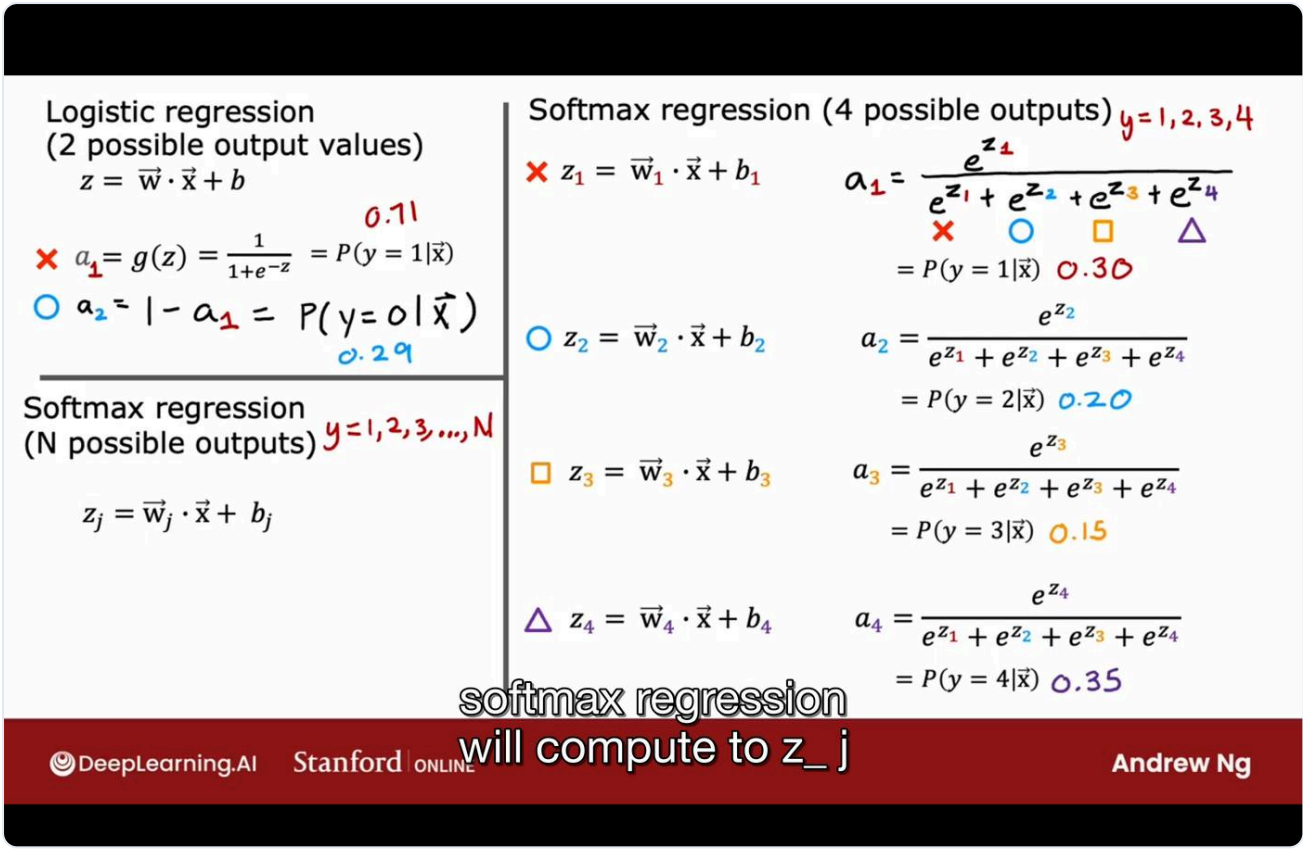


FIGURE 24.5

In that case, softmax regression will compute to z_ j equals w_ j.product with x plus b_j, where now the parameters of softmax regression are w_1, w_2 through to w_n, as well as b_1, b_2 through b_n.

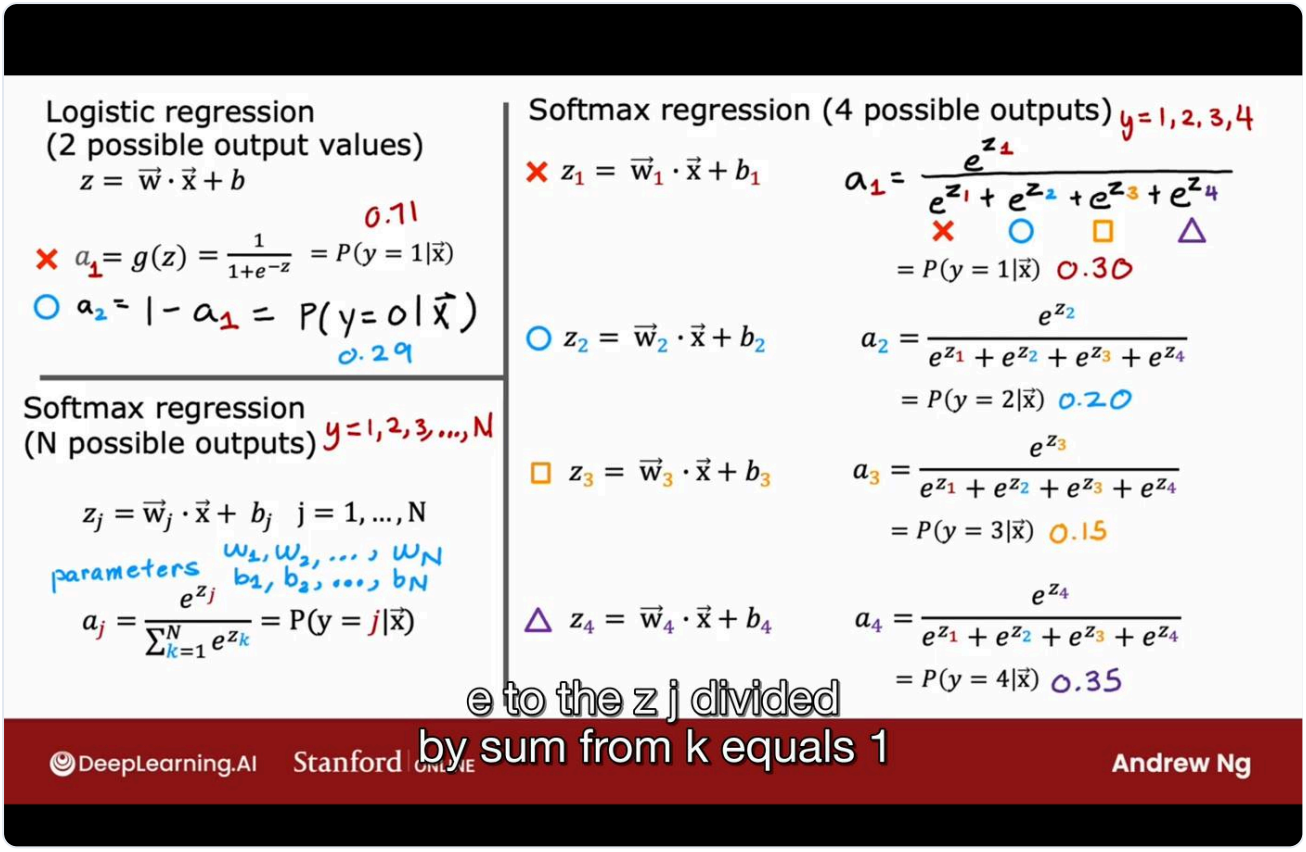


FIGURE 24.6 | Logistic Regression

Then finally, we'll compute a j equals e to the z j divided by sum from k equals 1 to n of e to the z sub k. While here I'm using another variable k to index the summation because here j refers to a specific fixed number like j equals 1. A, j is interpreted as the model's estimate that y is equal to j given the input features x.

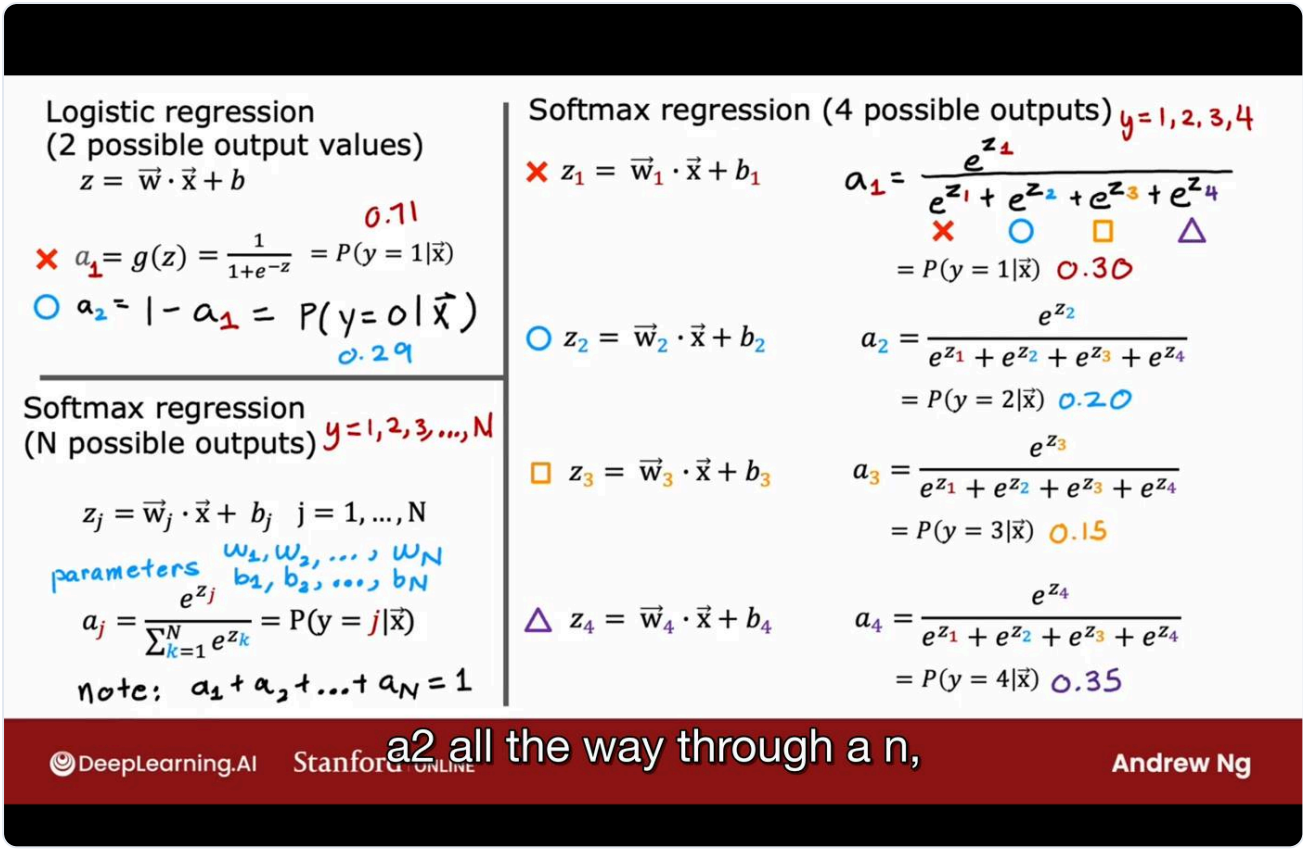


FIGURE 24.7 | Logistic Regression

Notice that by construction that this formula, if you add up a1, a2 all the way through a n, these numbers always will end up adding up to 1. We specified how you would compute the softmax regression model. I won't prove it in this video, but it turns out that if you apply softmax regression with n equals 2, so there are only two possible output classes then softmax regression ends up computing basically the same thing as logistic regression. The parameters end up being a little bit different, but it ends up reducing to logistic regression model. But that's why the softmax regression model is the generalization of logistic regression. Having defined how softmax regression computes it's outputs, let's now take a look at how to specify the cost function for softmax regression.

Recall for logistic regression, this is what we had. We said z is equal to this. Then I wrote earlier that a1 is g of z, was interpreted as a probability of y is 1.

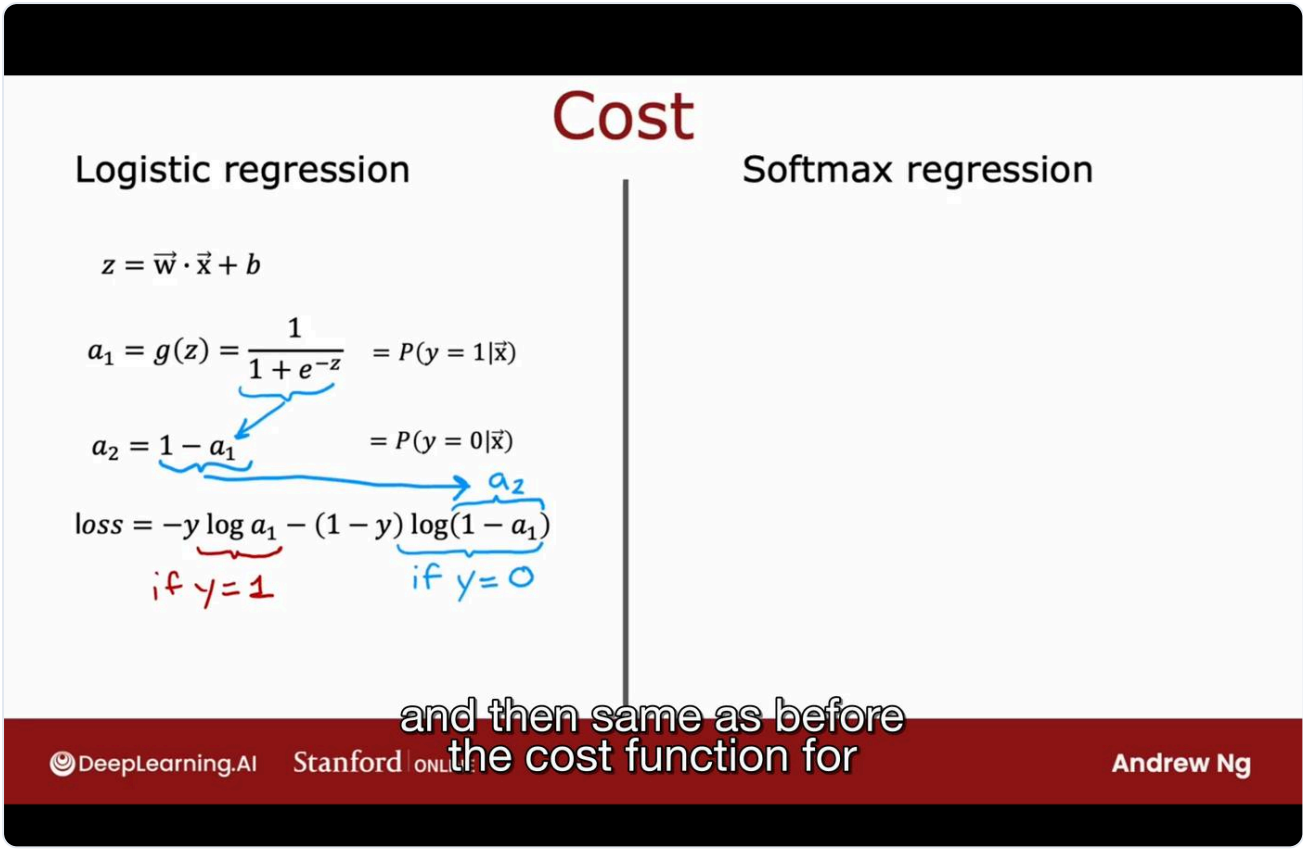


FIGURE 24.8 | Logistic Regression Softmax Regression

We also wrote a2 is the probability that y is equal to 0. Previously, we had written the loss of logistic regression as negative y log a1 minus 1 minus y log 1 minus a1. But 1 minus a1 is also equal to a2, because a2 is one minus a1 according to this expression over here. I can rewrite or simplify the loss for logistic regression little bit to be negative y log a1 minus 1 minus y log of a2. In other words, the loss if y is equal to 1 is negative log a1. If y is equal to 0, then the loss is negative log a2, and then same as before the cost function for all the parameters in the model is the average loss, average over the entire training set. That was a cost function for this regression.

Let's write down the cost function that is conventionally use the softmax regression. Recall that these are the equations we use for softmax regression.

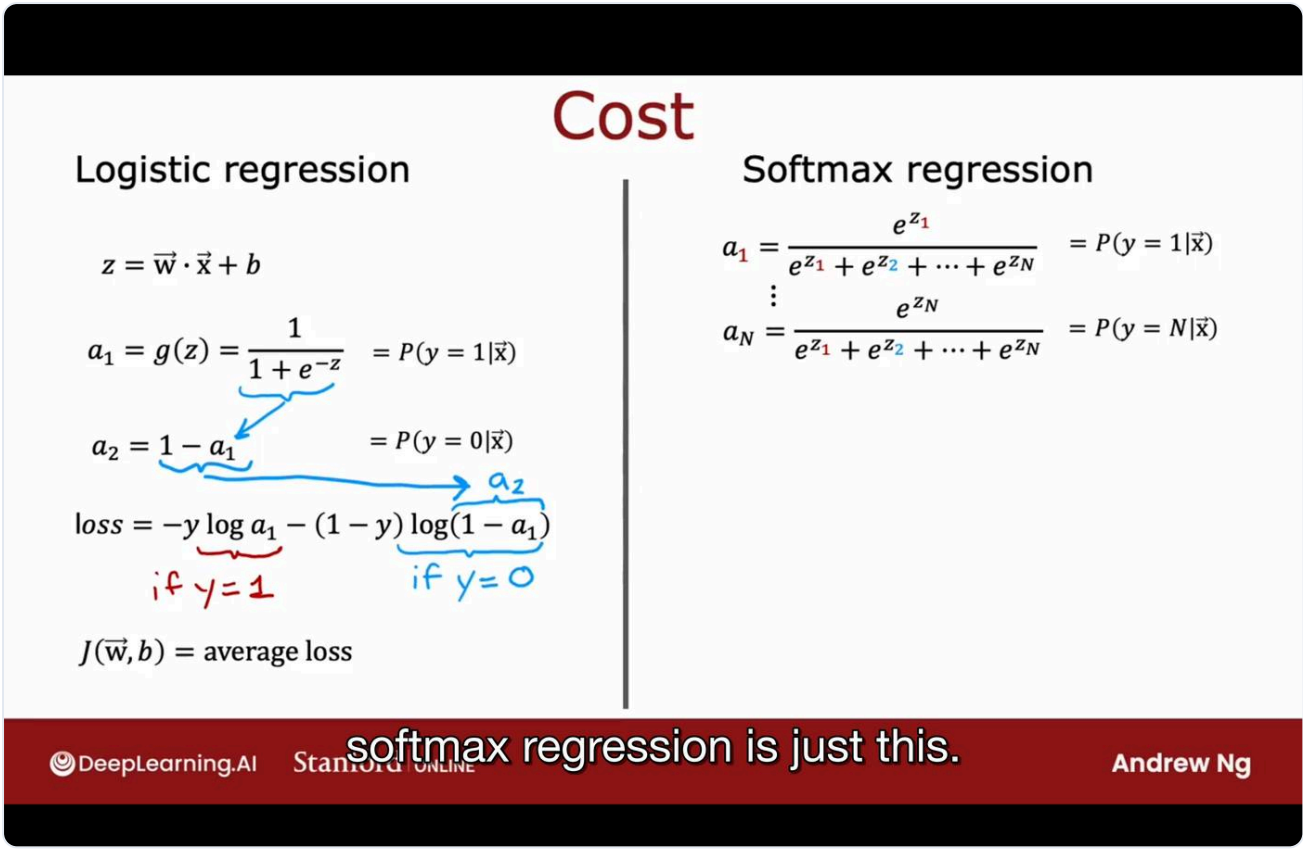


FIGURE 24.9 | Logistic Regression Softmax Regression

The loss we're going to use for softmax regression is just this. The loss for if the algorithm puts a1 through an. The ground truth label is why is if y equals 1, the loss is negative log a1.

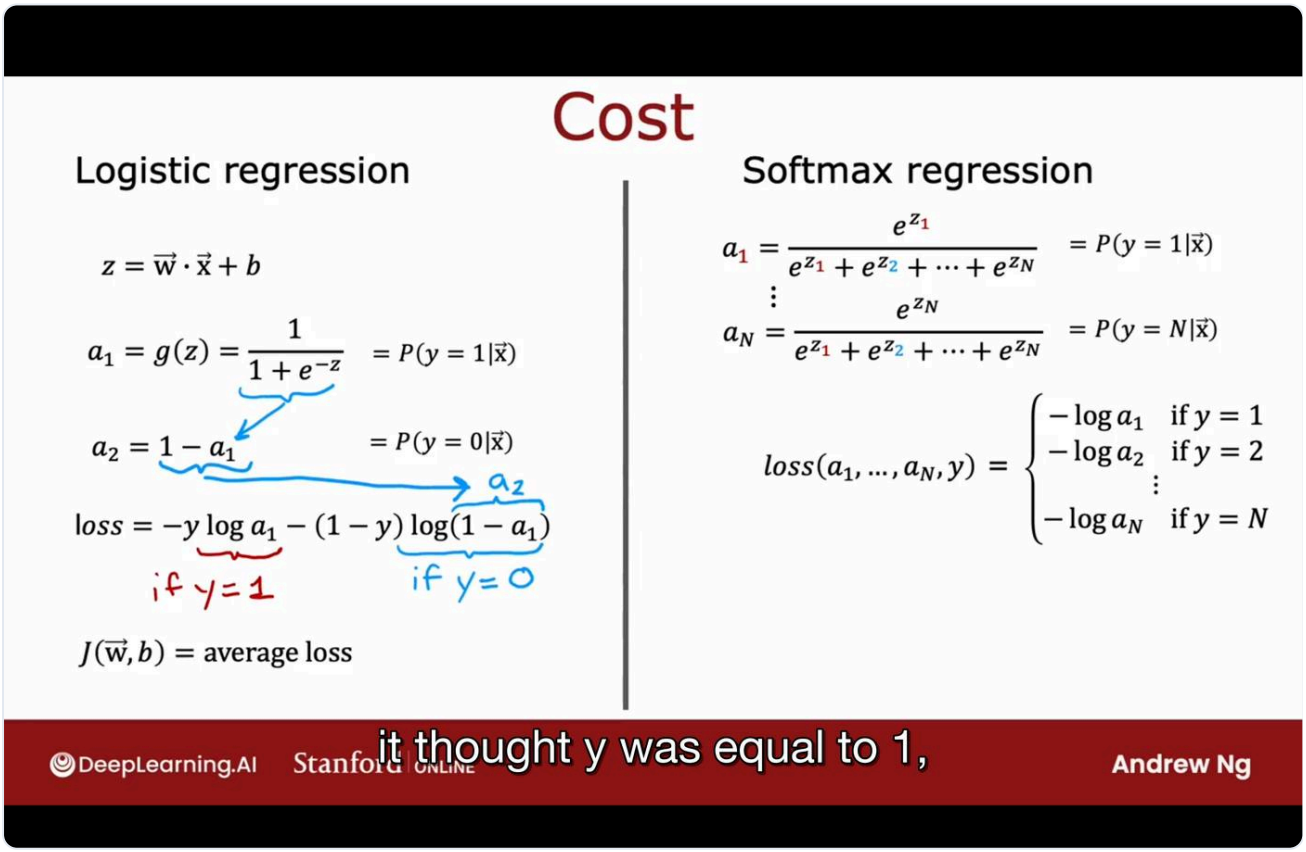


FIGURE 24.10 | Logistic Regression

Says negative log of the probability that it thought y was equal to 1, or if y is equal to 2, then I'm going to define as negative log a2. Y is equal to 2. The loss of the algorithm on this example is negative log of the probability it's thought y was equal to 2. On all the way down to if y is equal to n, then the loss is negative log of a n. To illustrate what this is doing, if y is equal to j, then the loss is negative log of a j. That's what this function looks like. Negative log of a j is a curve that looks like this.

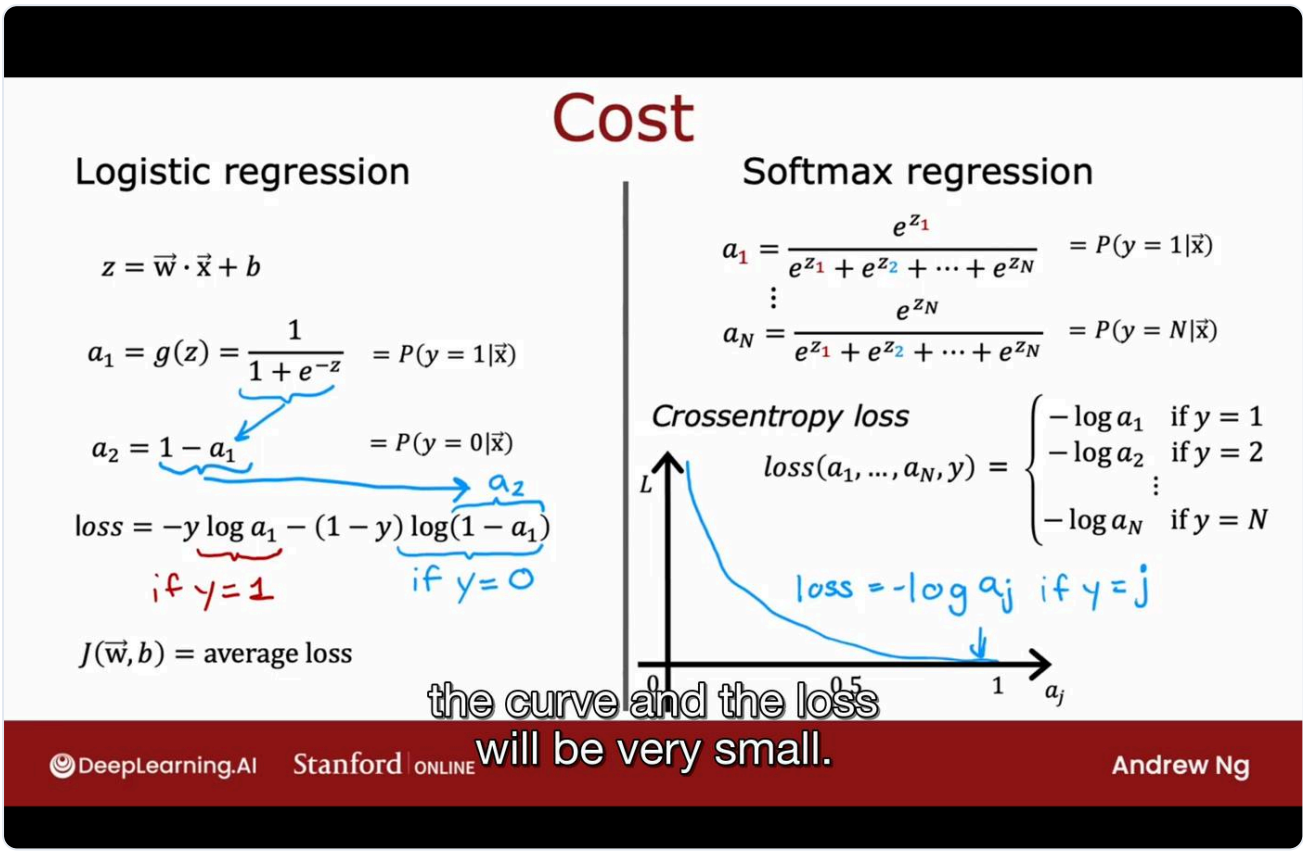


FIGURE 24.11 | Logistic Regression Softmax Regression

If a j was very close to 1, then you beyond this part of the curve and the loss will be very small. But if it thought, say, a j had only a 50% chance then the loss gets a little bit bigger.

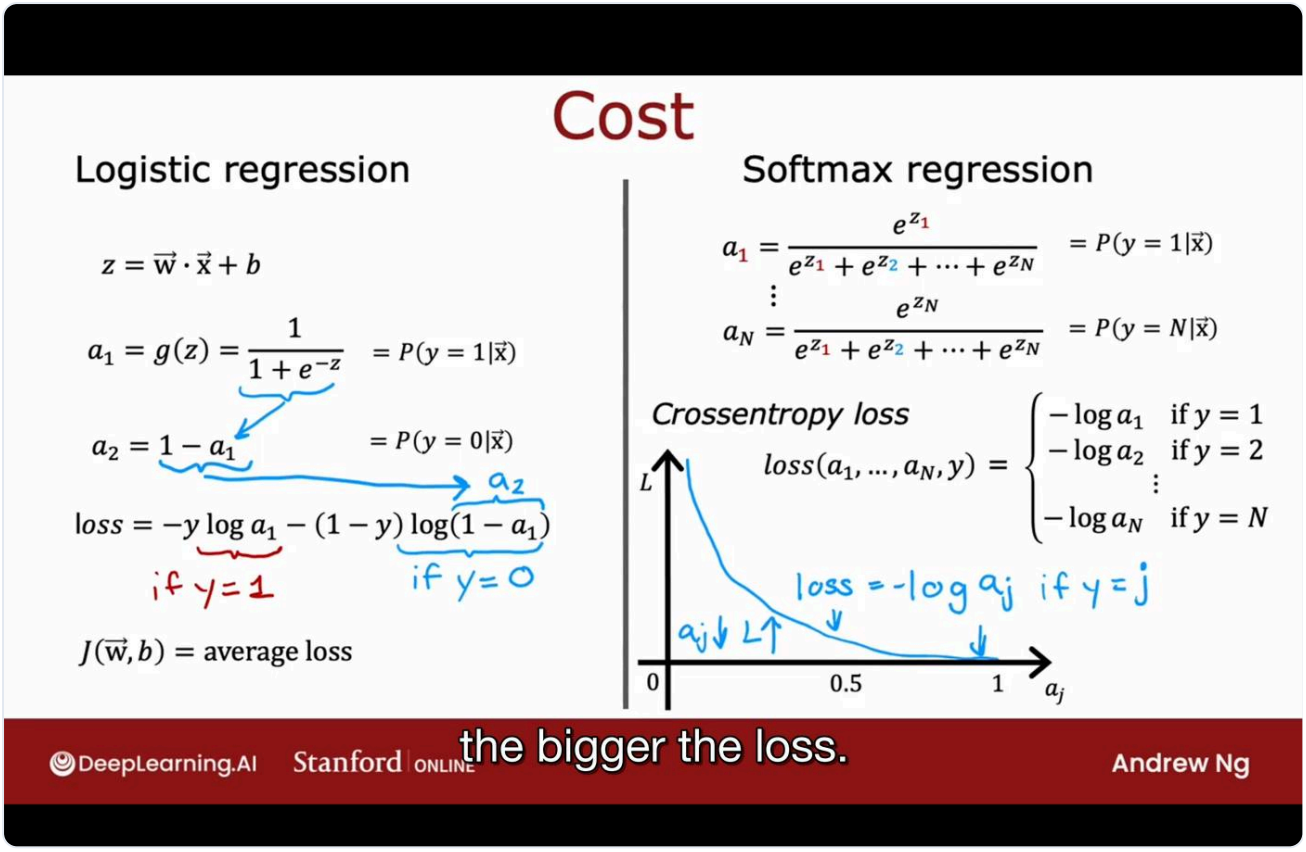


FIGURE 24.12 | Logistic Regression Softmax Regression

The smaller a j is, the bigger the loss. This incentivizes the algorithm to make a j as large as possible, as close to 1 as possible. Because whatever the actual value y was, you want the algorithm to say hopefully that the chance of y being that value was pretty large. Notice that in this loss function, y in each training example can take on only one value. You end up computing this negative log of a j only for one value of a j, which is whatever was the actual value of y equals j in that particular training example. For example, if y was equal to 2, you end up computing negative log of a2, but not any of the other negative log of a1 or the other terms here.

That's the form of the model as well as the cost function for softmax regression. If you were to train this model, you can start to build multiclass classification algorithms. What we'd like to do next is take this softmax regression model, and fit it into a new network so that you really do something even better, which is to train a new network for multi-class classification. Let's go through that in the next video.

LECTURE 25

MNIST with Softmax

In order to build a Neural Network that can carry out multi class classification, we're going to take the Softmax regression model and put it into essentially the output layer of a Neural Network. Let's take a look at how to do that. Previously, when we were doing handwritten digit recognition with just two classes. We use a new Neural Network with this architecture. If you now want to do handwritten digit classification with 10 classes, all the digits from zero to nine, then we're going to change this Neural Network to have 10 output units like so. And this new output layer will be a Softmax output layer.

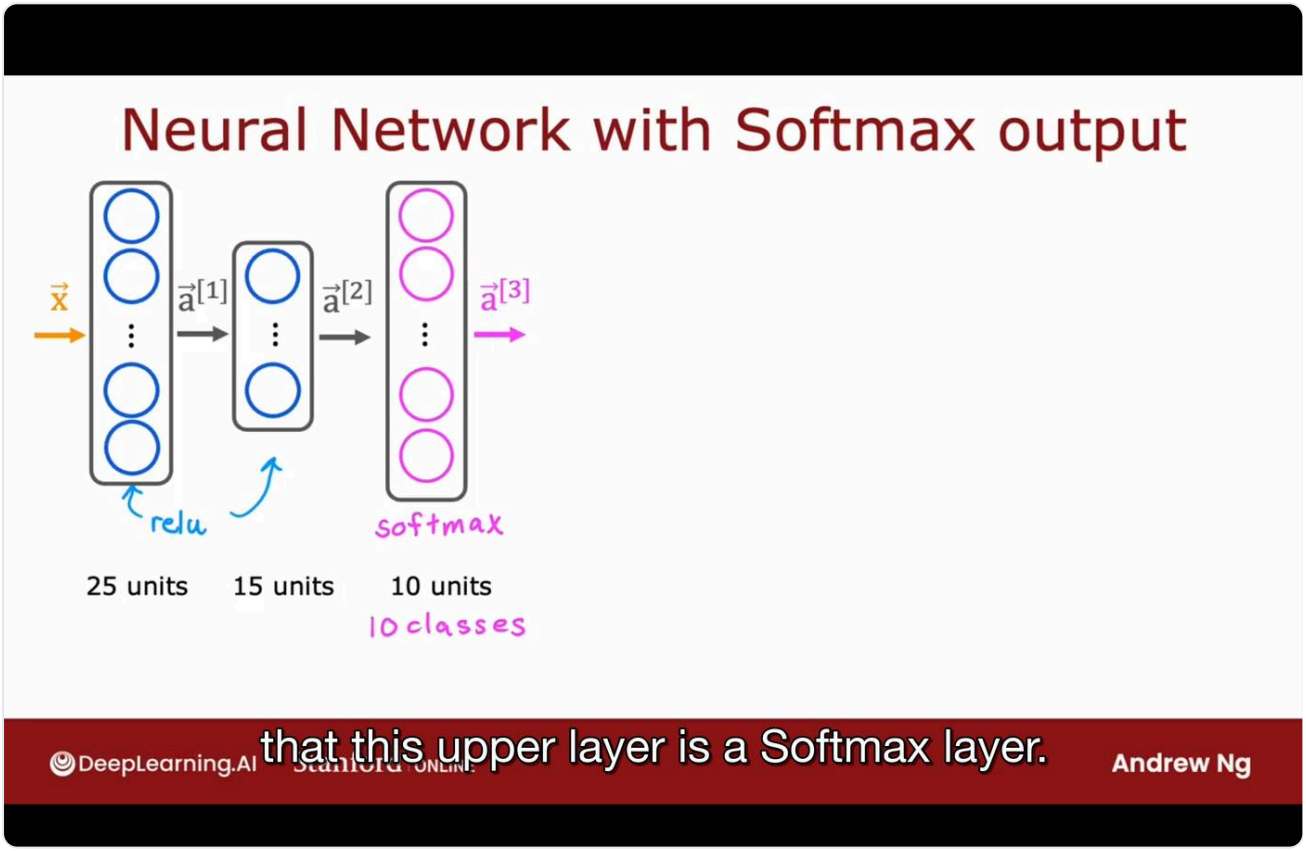


FIGURE 25.1

So sometimes we'll say this Neural Network has a Softmax output or that this upper layer is a Softmax layer. And the way forward propagation works in this Neural Network is given an input X A1 gets computed exactly the same as before. And then A2, the activations for the second hidden layer also get computed exactly the same as before. And we now have to compute the activations for this output layer, that is a3. This is how it works. If you have 10 output classes, we will compute Z1, Z2 through Z10 using these expressions. So this is actually very similar to what we had previously for the formula you're using to compute Z. Z1 is W1.product with a2, the activations from the previous layer plus b1 and so on for Z1 through Z10.

Then a1 is equal to e to the Z1 divided by e to the Z1 plus up to e to the Z10. And that's our estimate of the chance that y is equal to 1. And similarly for a2 and similarly all the way up to a10.

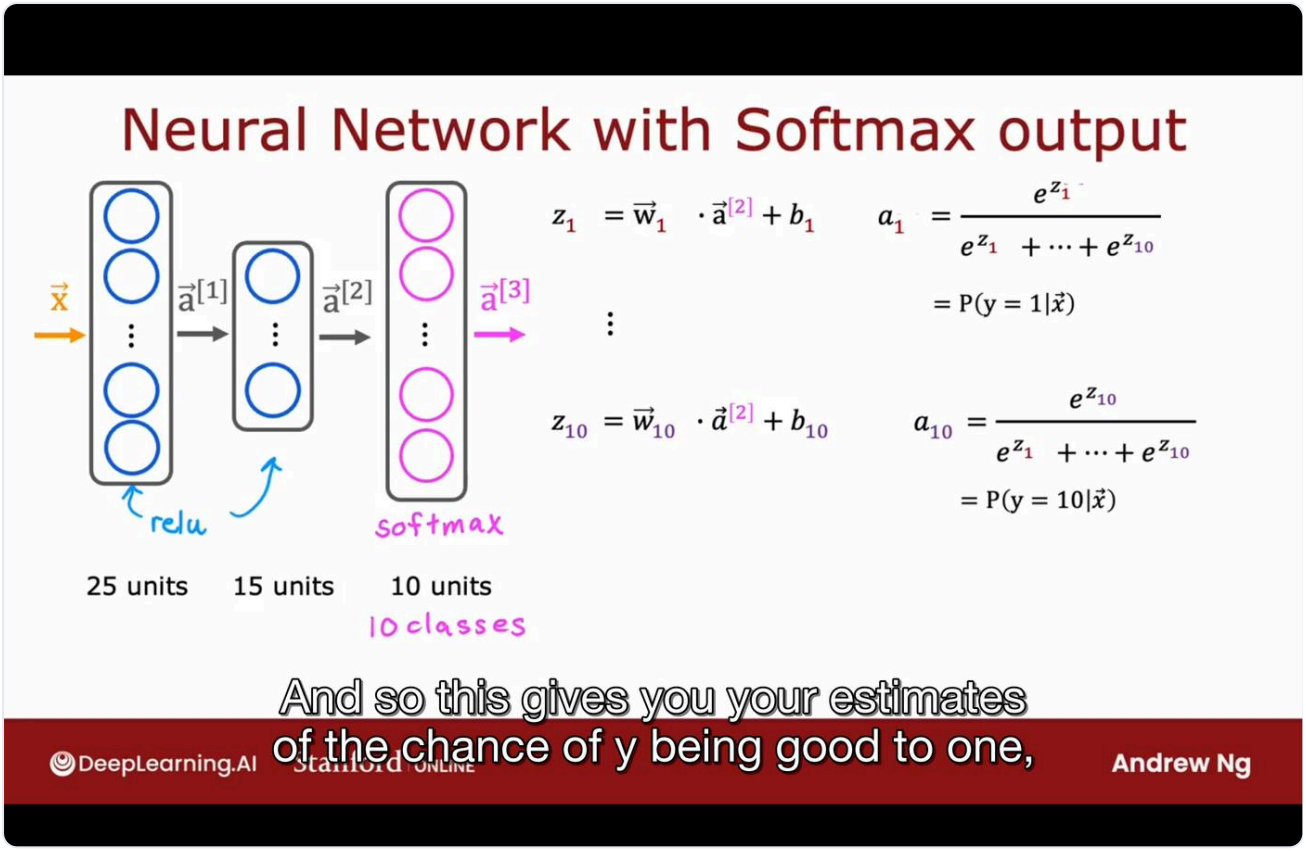


FIGURE 25.2

And so this gives you your estimates of the chance of y being good to one, two and so on up through the 10th possible label for y. And just for completeness, if you want to indicate that these are the quantities associated with layer three, technically, I should add these super strip three's there. It does make the notation a little bit more cluttered.

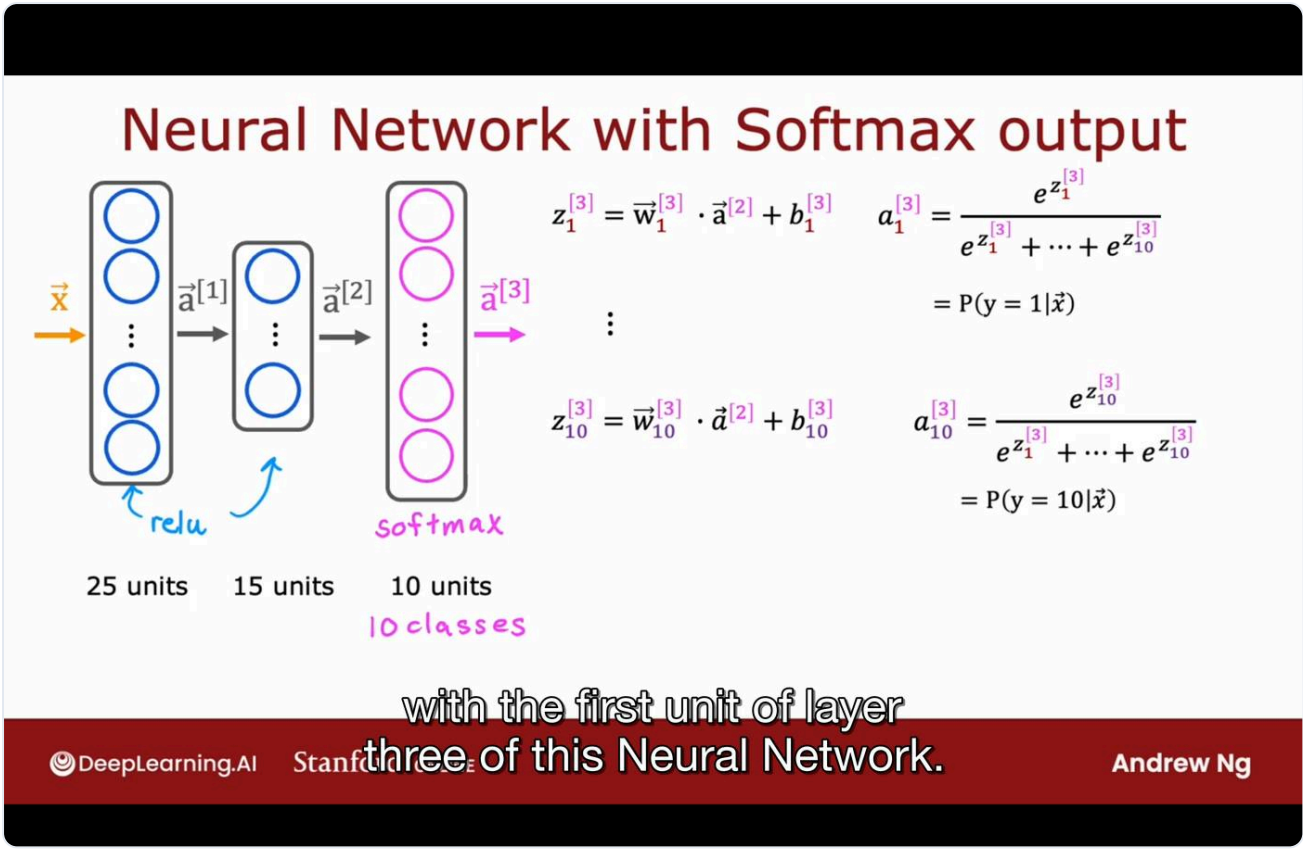


FIGURE 25.3

But this makes explicit that this is, for example, the Z(3), 1 value and this is the parameters associated with the first unit of layer three of this Neural Network. And with this, your Softmax open layer now gives you estimates of the chance of y being any of these 10 possible output labels. I do want to mention that the Softmax layer will sometimes also called the Softmax activation function, it is a little bit unusual in one respect compared to the other activation functions we've seen so far, like sigma, radial and linear, which is that when we're looking at sigmoid or value or linear activation functions, a1 was a function of Z1 and a2 was a function of Z2 and only Z2.

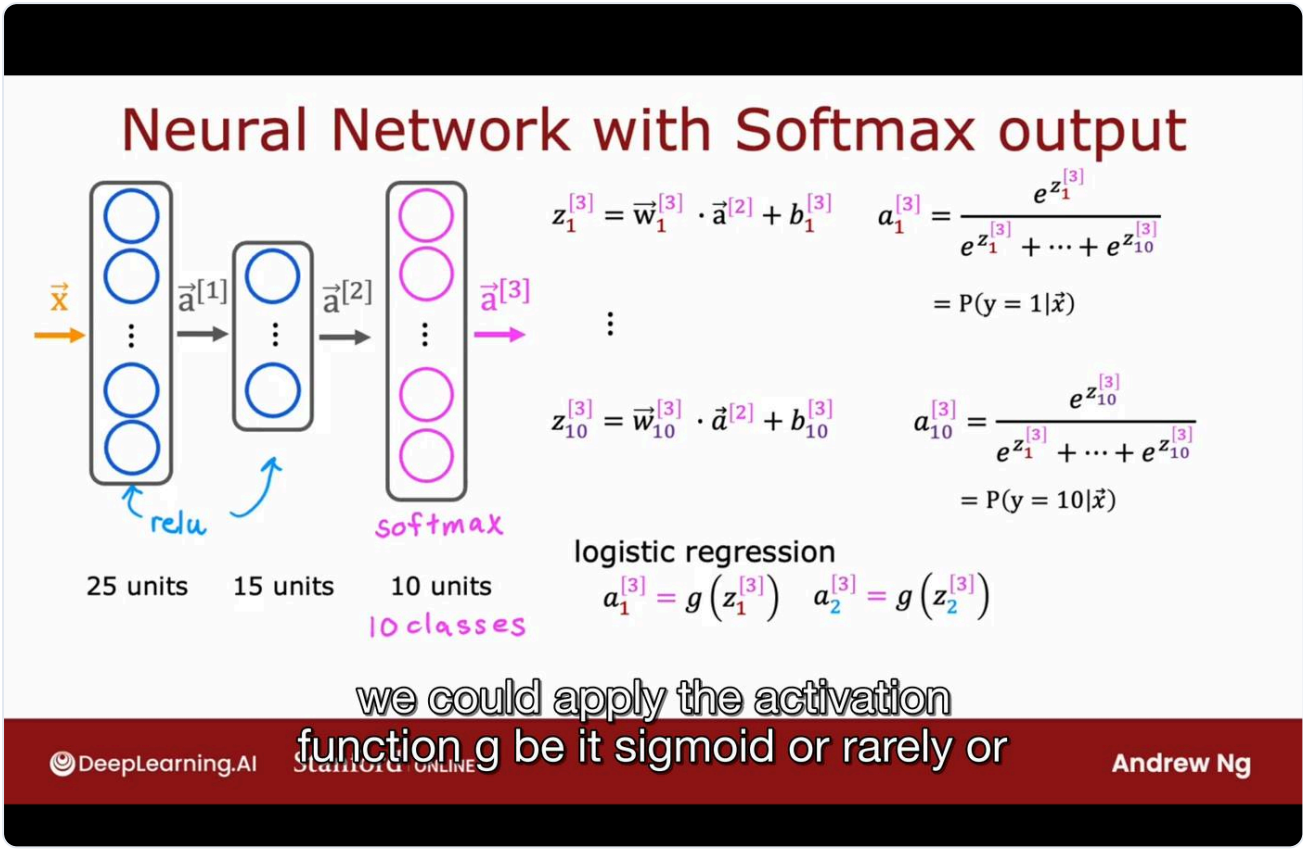


FIGURE 25.4

In other words, to obtain the activation values, we could apply the activation function g be it sigmoid or rarely or something else element wise to Z1 and Z2 and so on to get a1 and a2 and a3 and a4. But with the Softmax activation function, notice that a1 is a function of Z1 and Z2 and Z3 all the way up to Z10. So each of these activation values, it depends on all of the values of Z. And this is a property that's a bit unique to the Softmax output or the Softmax activation function or stated differently if you want to compute a1 through a10,hat is a function of Z1 all the way up to Z 10 simultaneously.

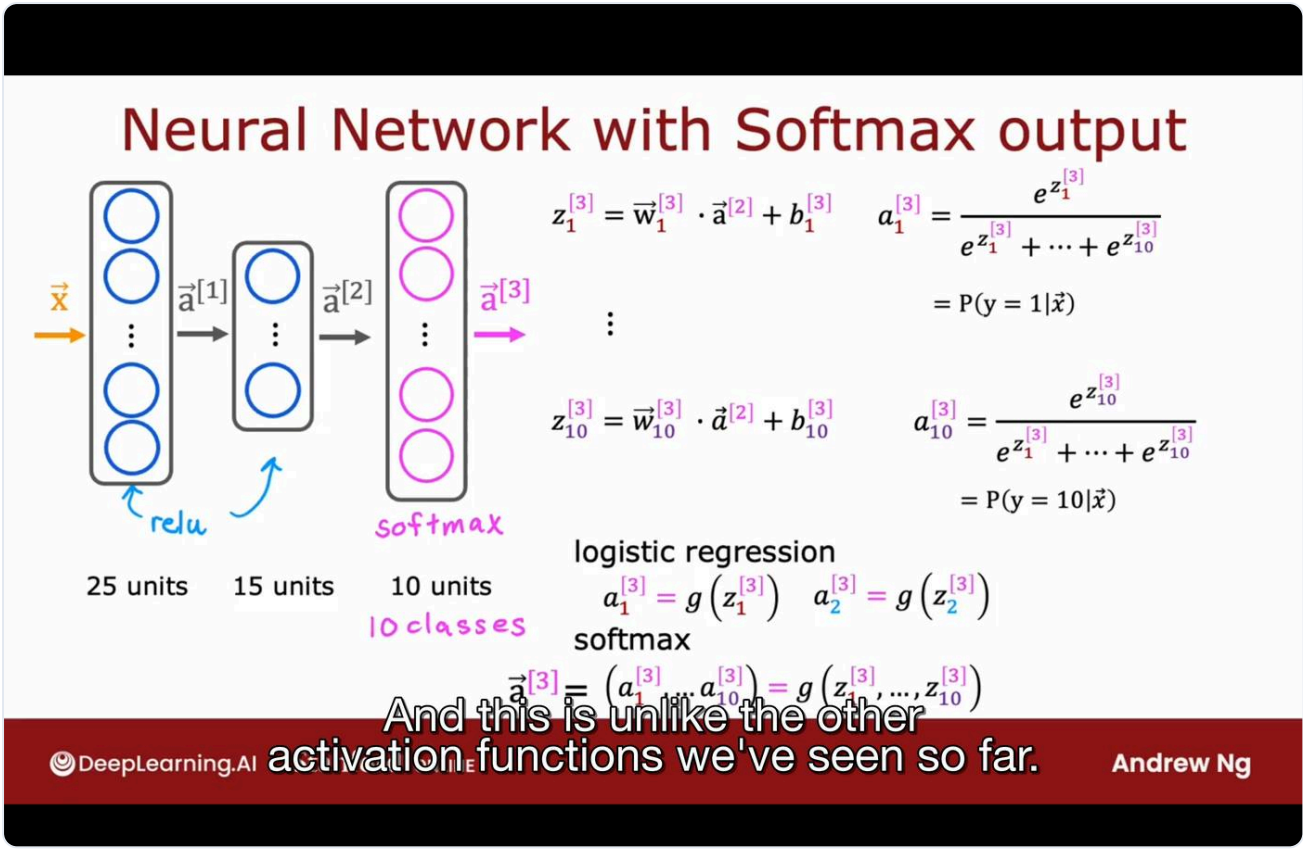


FIGURE 25.5

And this is unlike the other activation functions we've seen so far. Finally, let's look at how you would implement this in tensorflow. If you want to implement the neural network that I've shown here on this slide, this is the code to do so. Similar as before there are three steps to specifying and training the model. The first step is to tell tensorflow to sequentially string together three layers. First layer, is this 25 units with rail you activation function. Second layer, 15 units of rally activation function. And then the third layer, because there are now 10 output units, you want to output a1 through a10, so they're now 10 output units. And we'll tell tensorflow to use the Softmax activation function.

And the cost function that you saw in the last video, tensorflow calls that the SparseCategoricalCrossentropy function. So I know this name is a bit of a mouthful, whereas for logistic regression we had the BinaryCrossentropy function, here we're using the SparseCategoricalCrossentropy function. And what sparse categorical refers to is that you're still classified y into categories. So it's categorical. This takes on values from 1 to 10. And sparse refers to that y can only take on one of these 10 values. So each image is either 0 or 1 or 2 or so on up to 9. You're not going to see a picture that is simultaneously the number two and the number seven so sparse refers to that each digit is only one of these categories.

But so that's why the loss function that you saw in the last video, it's called intensive though the sparse categorical cross entropy loss function. And then the code for training the model is just the same as before.

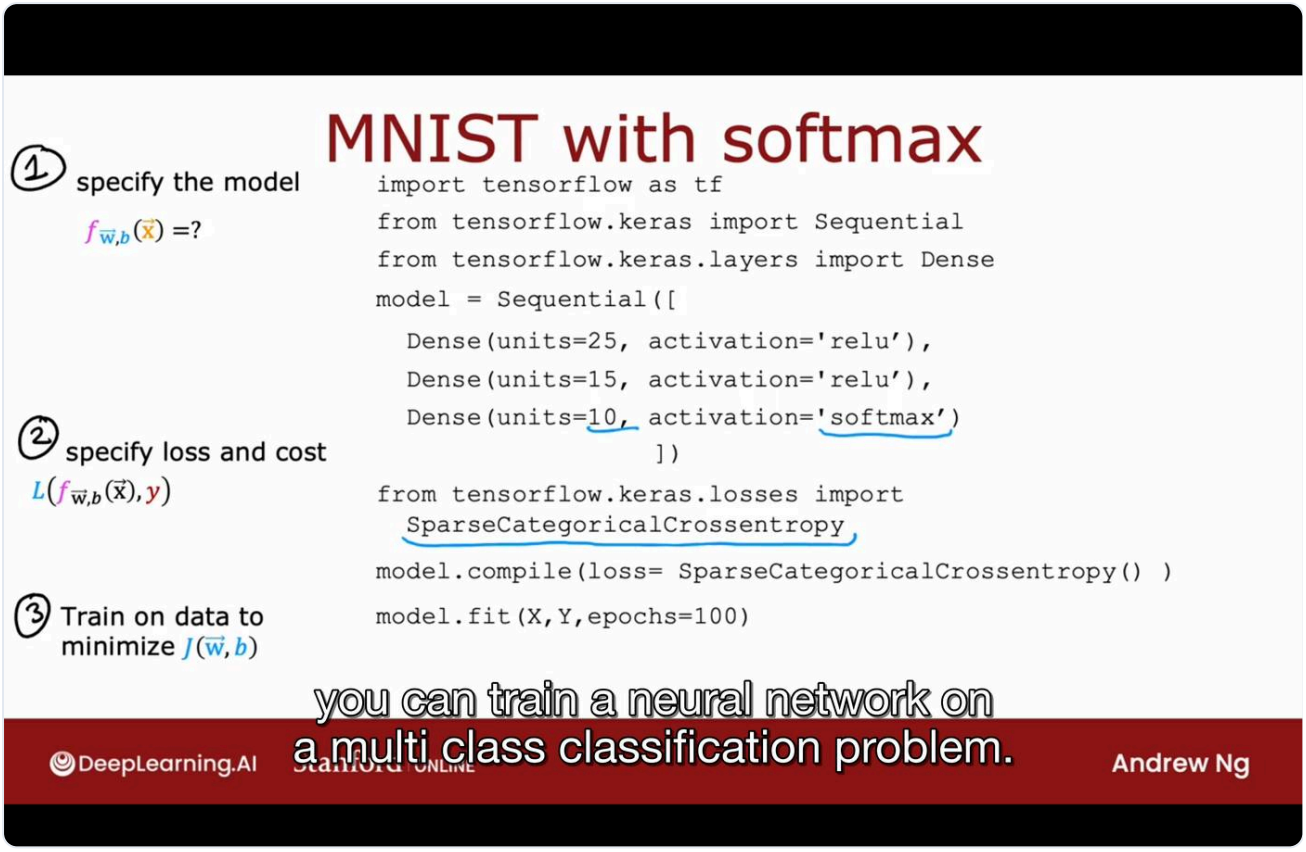


FIGURE 25.6 | MNIST with Softmax

And if you use this code, you can train a neural network on a multi class classification problem.

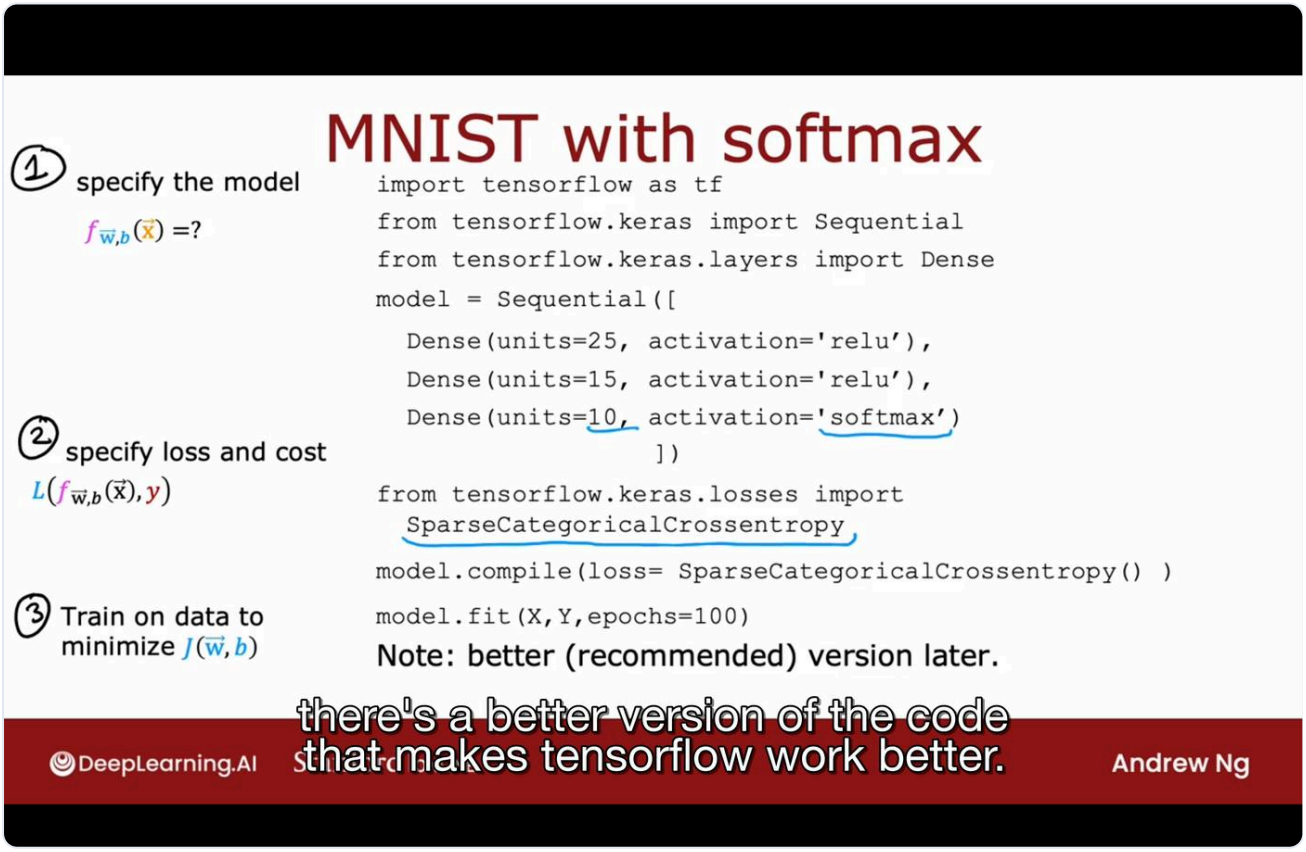


FIGURE 25.7 | MNIST with Softmax

Just one important note, if you use this code exactly as I've written here, it will work, but don't actually use this code because it turns out that in tensorflow, there's a better version of the code that makes tensorflow work better. So even though the code shown in this slide works. Don't use this code the way I've written it here, because in a later video this week you see a different version that's actually the recommended version of implementing this, that will work better.

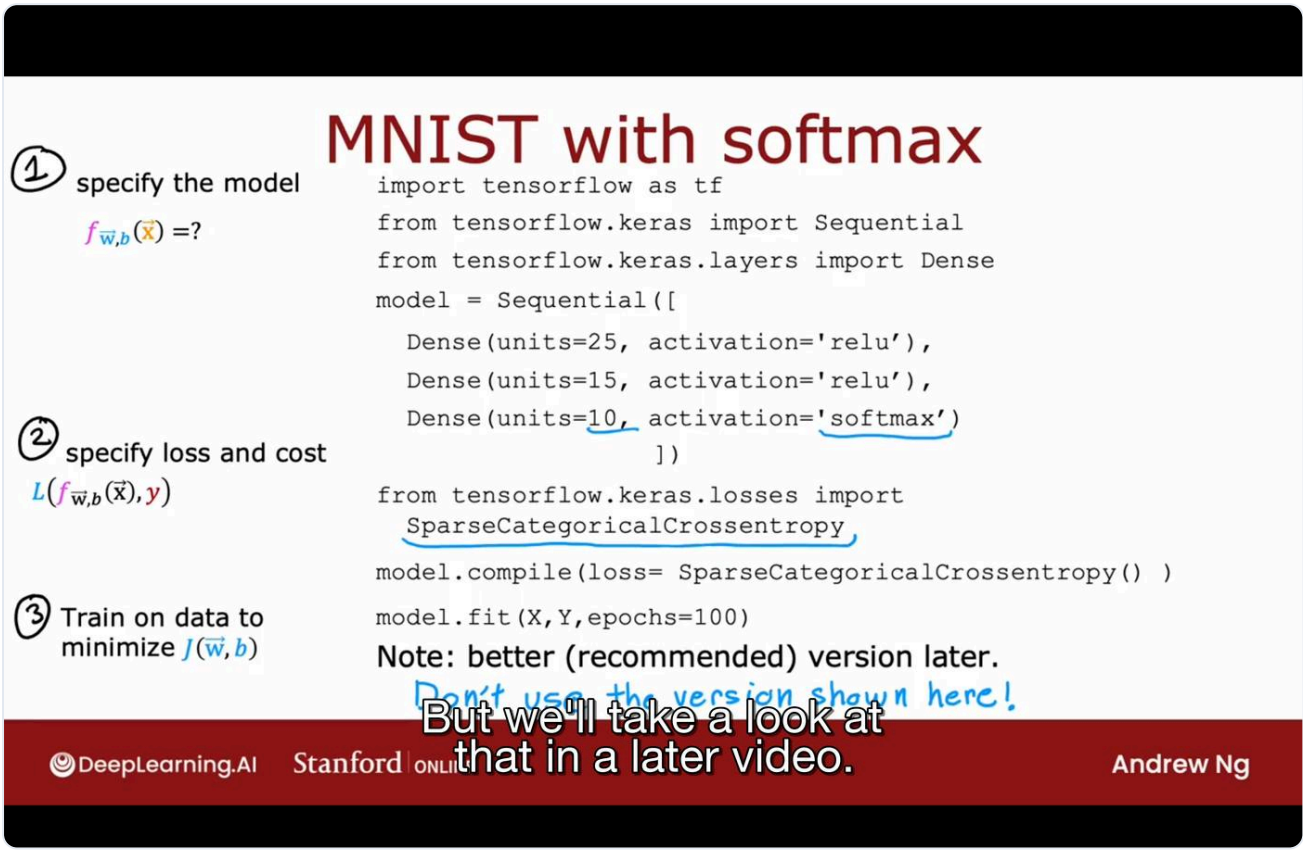


FIGURE 25.8 | MNIST with Softmax

But we'll take a look at that in a later video. So now, you know how to train a neural network with a softmax output layer with one caveat. There's a different version of the code that will make tensorflow able to compute these probabilities much more accurately. Let's take a look at that in the next video, we should also show you the actual code that I recommend you use if you're training a Softmax neural network. Let's go on to the next video.

LECTURE 26

Numerical Roundoff Errors

The implementation that you saw in the last video of a neural network with a softmax layer will work okay. But there's an even better way to implement it. Let's take a look at what can go wrong with that implementation and also how to make it better. Let me show you two different ways of computing the same quantity in a computer. Option 1, we can set x equals to 2/10,000. Option 2, we can set x equals 1 plus 1/10,000 minus 1 minus 1/10,000, which you first compute this, and then compute this, and you take the difference.

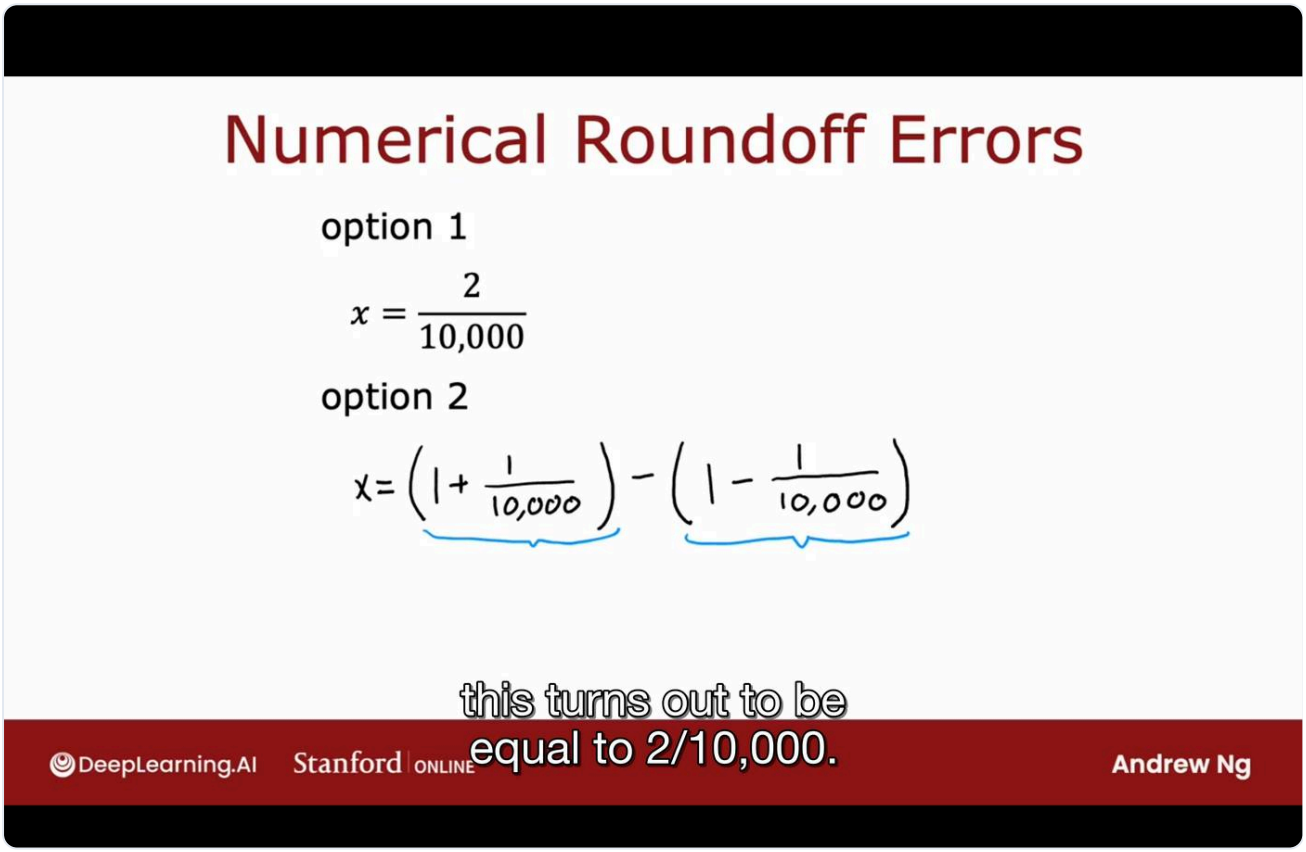


FIGURE 26.1 | Numerical Roundoff Errors

If you simplify this expression, this turns out to be equal to 2/10,000. Let me illustrate this in this notebook. First, let's set x equals 2/10,000 and print the result to a lot of decimal points of accuracy. That looks pretty good.

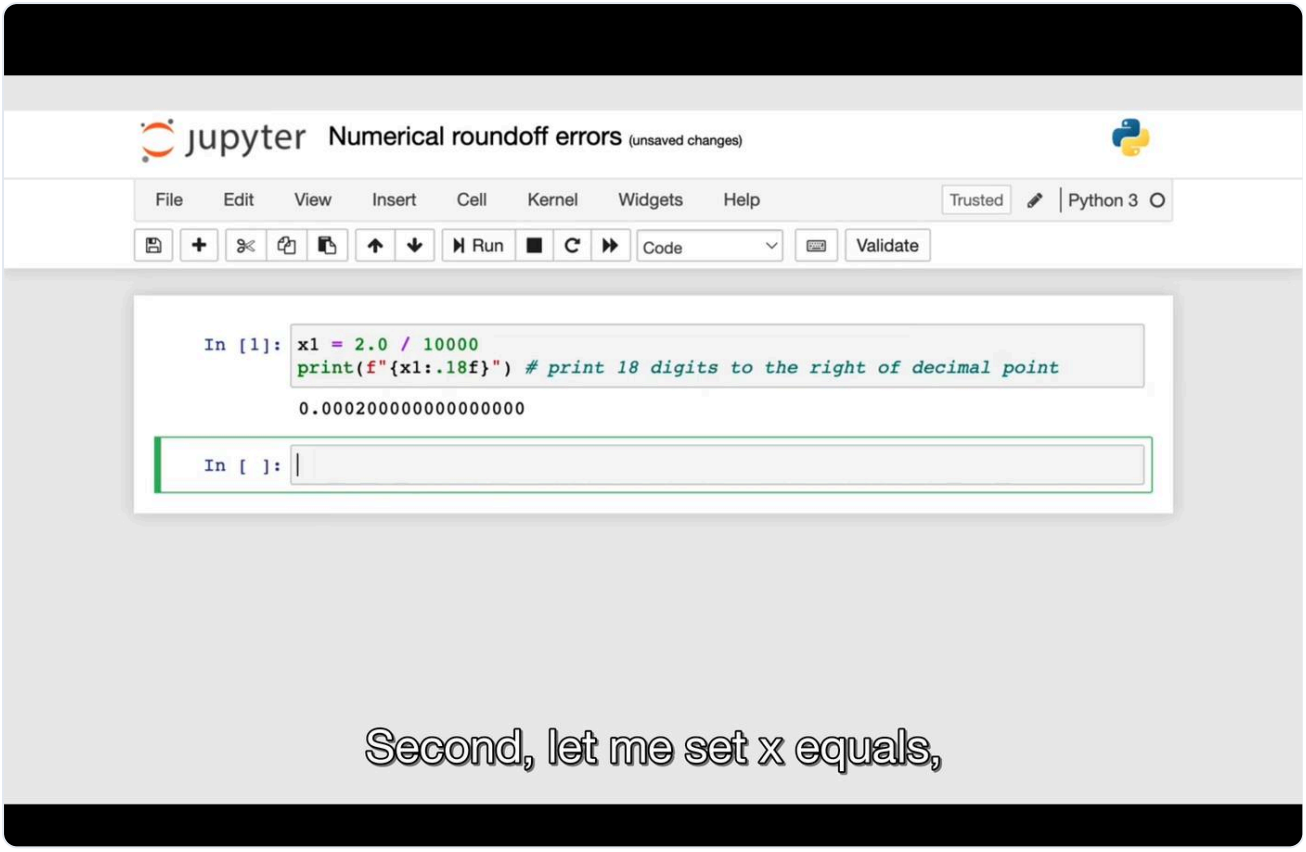


FIGURE 26.2

Second, let me set x equals, I'm going to insist on computing 1/1 plus 10,000 and then subtract from that 1 minus 1/10,000. Let's print that out.

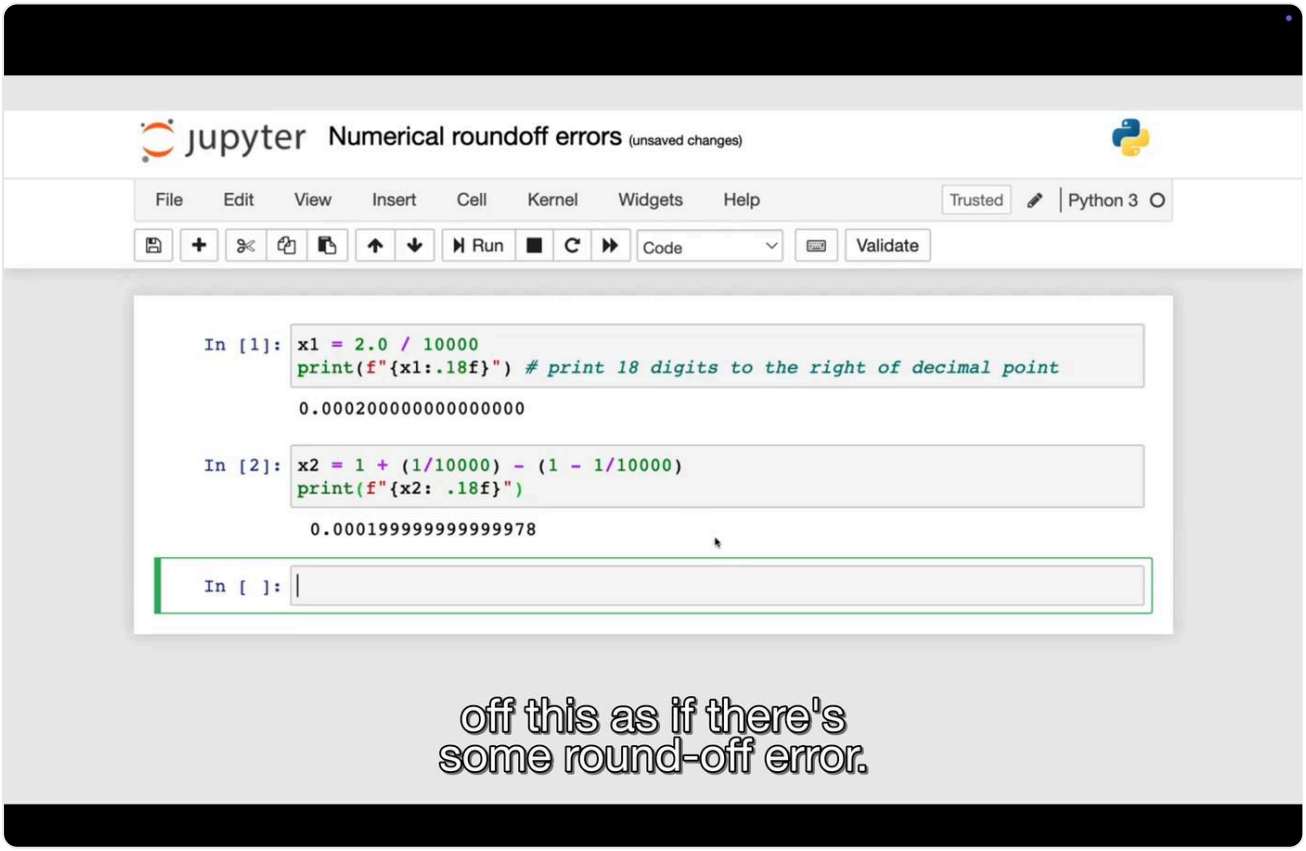


FIGURE 26.3

It just looks a little bit off this as if there's some round-off error. Because the computer has only a finite amount of memory to store each number, called a floating-point number in this case, depending on how you decide to compute the value 2/10,000, the result can have more or less numerical round-off error. It turns out that while the way we have been computing the cost function for softmax is correct, there's a different way of formulating it that reduces these numerical round-off errors, leading to more accurate computations within TensorFlow. Let me first explain this a little bit more detail using logistic regression. Then we will show how these ideas apply to improving our implementation of softmax.

First, let me illustrate these ideas using logistic regression. Then we'll move on to show how to improve your implementation of softmax as well.

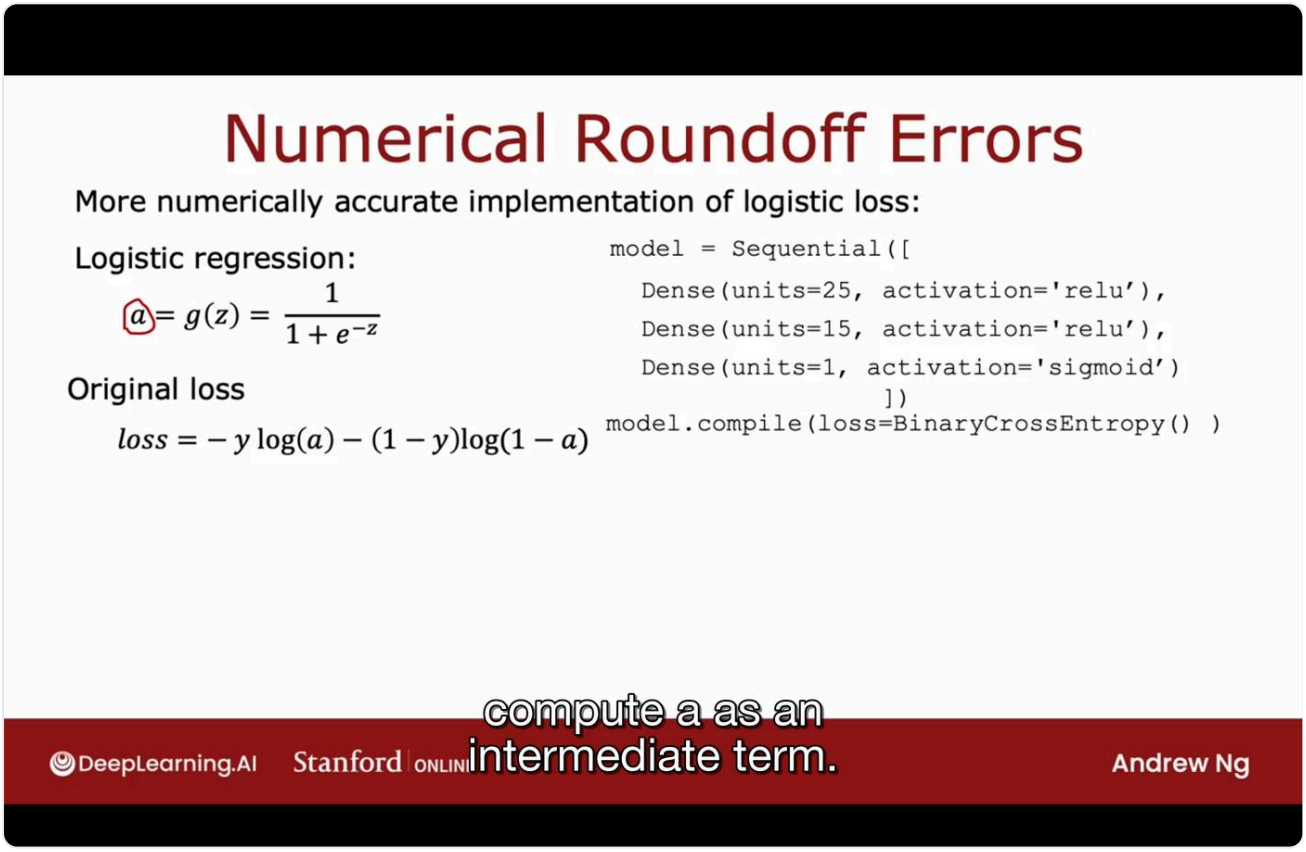


FIGURE 26.4 | Numerical Roundoff Errors

Recall that for logistic regression, if you want to compute the loss function for a given example, you would first compute this output activation a, which is g of z or 1/1 plus e to the negative z. Then you will compute the loss using this expression over here. In fact, this is what the codes would look like for a logistic output layer with this binary cross entropy loss. For logistic regression, this works okay, and usually the numerical round-off errors aren't that bad. But it turns out that if you allow TensorFlow to not have to compute a as an intermediate term. But instead, if you tell TensorFlow that the loss this expression down here. All I've done is I've taken a and expanded it into this expression down here.

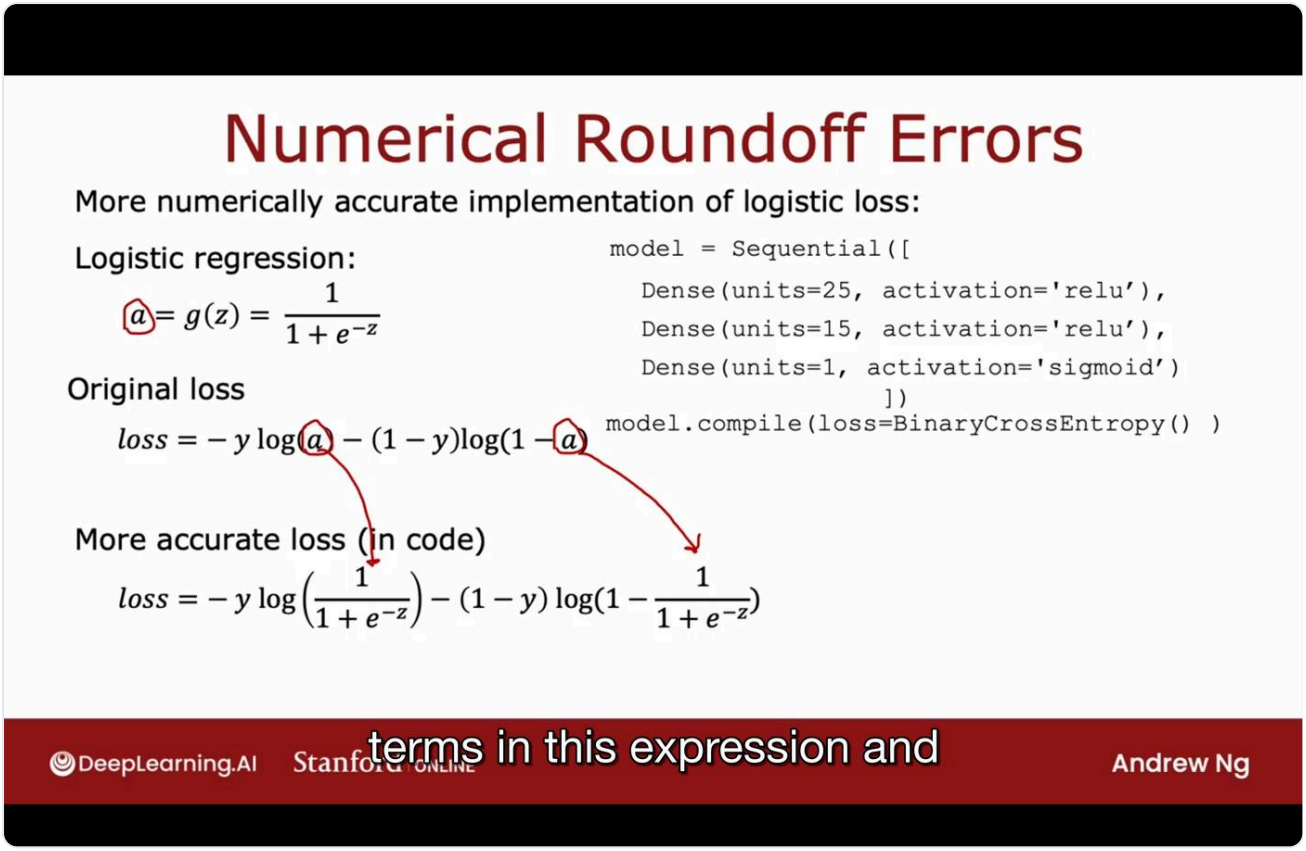


FIGURE 26.5 | Numerical Roundoff Errors

Then TensorFlow can rearrange terms in this expression and come up with a more numerically accurate way to compute this loss function. Whereas the original procedure was like insisting on computing as an intermediate value, 1 plus 1/10,000 and another intermediate value, 1 minus 1/10,000, then manipulating these two to get 2/10,000. This partial implementation was insisting on explicitly computing a as an intermediate quantity. But instead, by specifying this expression at the bottom directly as the loss function, it gives TensorFlow more flexibility in terms of how to compute this and whether or not it wants to compute a explicitly.

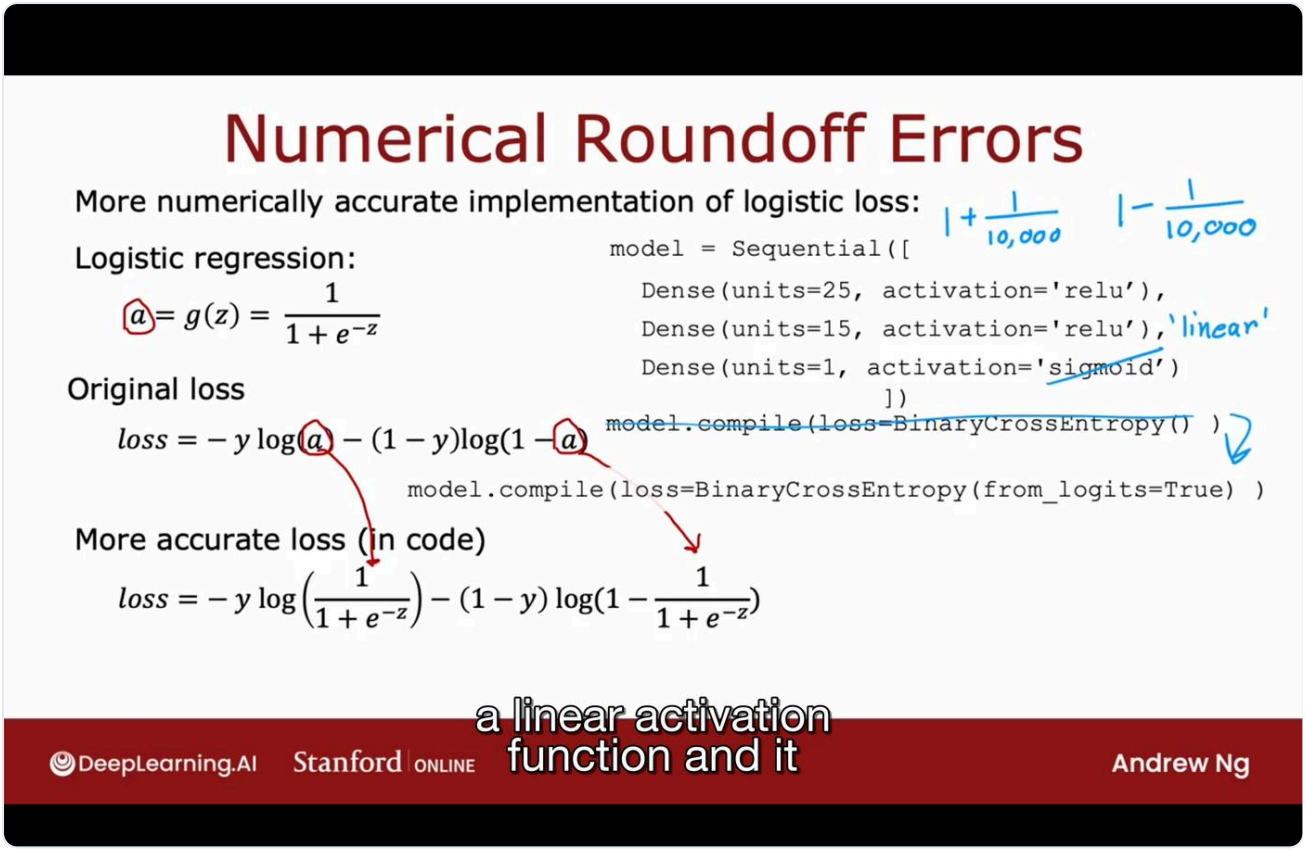


FIGURE 26.6 | Numerical Roundoff Errors

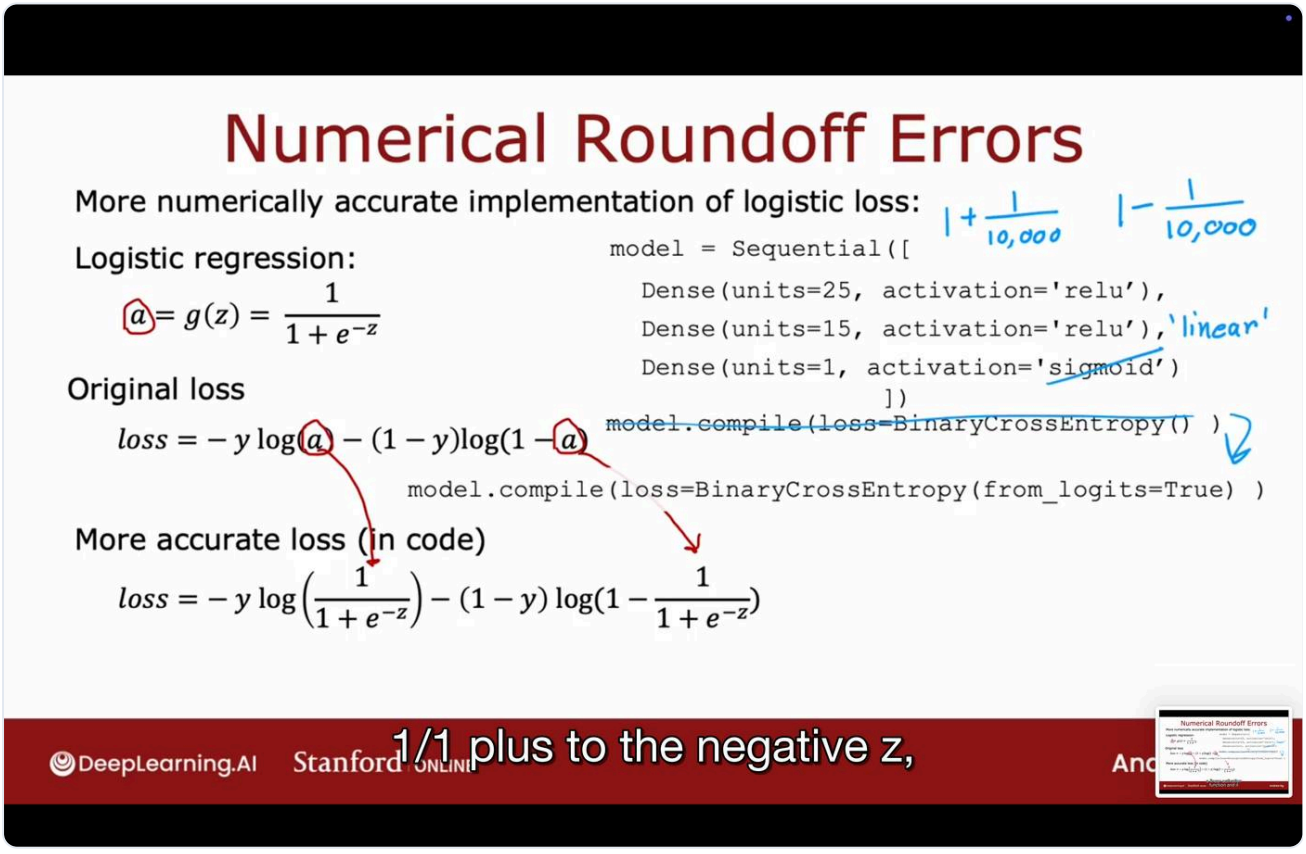


FIGURE 26.7 | Numerical Roundoff Errors

The code you can use to do this is shown here and what this does is it sets the output layer to just use a linear activation function and it puts both the activation function, 1/1 plus to the negative z, as well as this cross entropy loss into the specification of the loss function over here. That's what this from logits equals true argument causes TensorFlow to do. In case you're wondering what the logits are, it's basically this number z. TensorFlow will compute z as an intermediate value, but it can rearrange terms to make this become computed more accurately. One downside of this code is it becomes a little bit less legible.

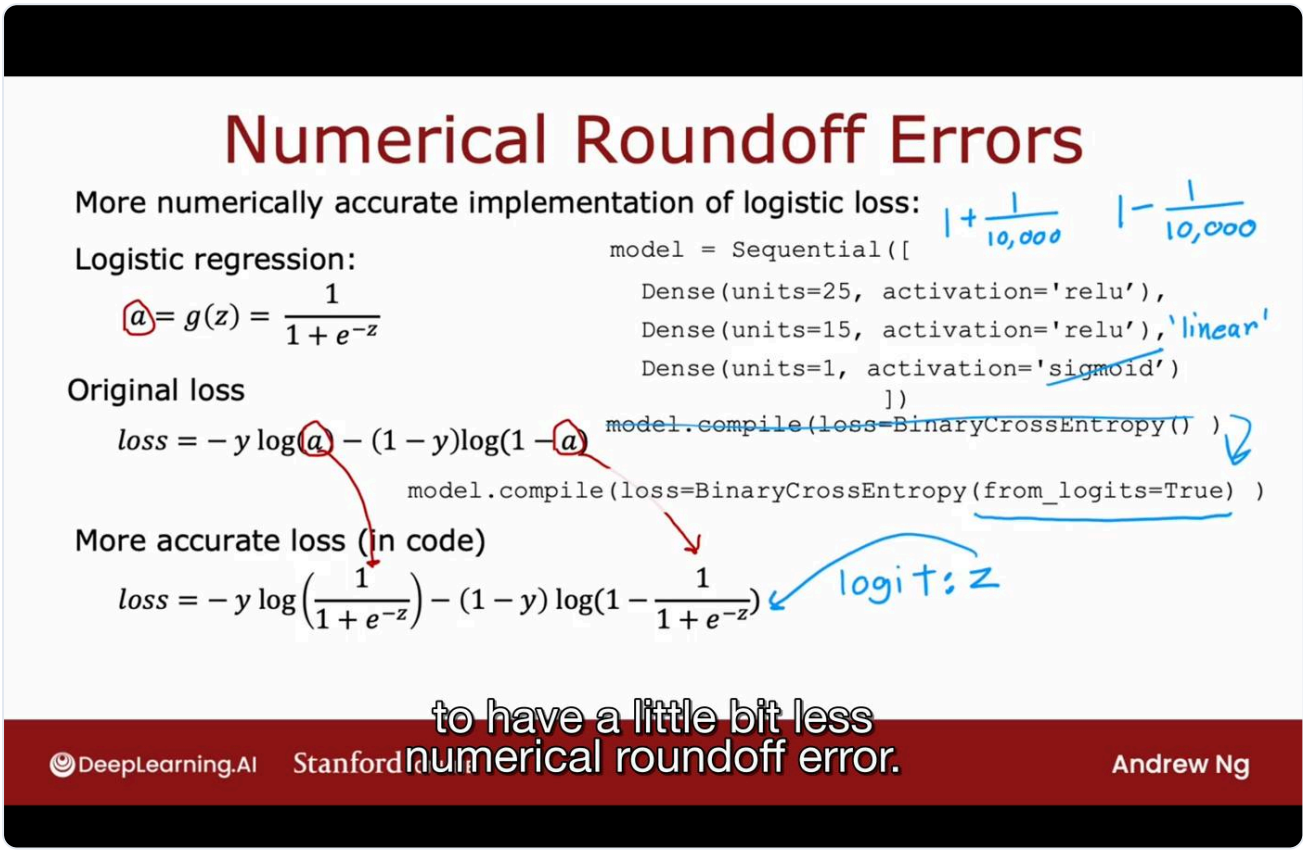


FIGURE 26.8 | Numerical Roundoff Errors

But this causes TensorFlow to have a little bit less numerical roundoff error. Now in the case of logistic regression, either of these implementations actually works okay, but the numerical roundoff errors can get worse when it comes to softmax. Now let's take this idea and apply to softmax regression. Recall what you saw in the last video was you compute the activations as follows. The activations is g of z_1, through z_10 where a_1, for example, is e to the z_1 divided by the sum of the e to the z_j's, and then the loss was this depending on what is the actual value of y is negative log of aj for one of the aj's and so this was the code that we had to do this computation in two separate steps.

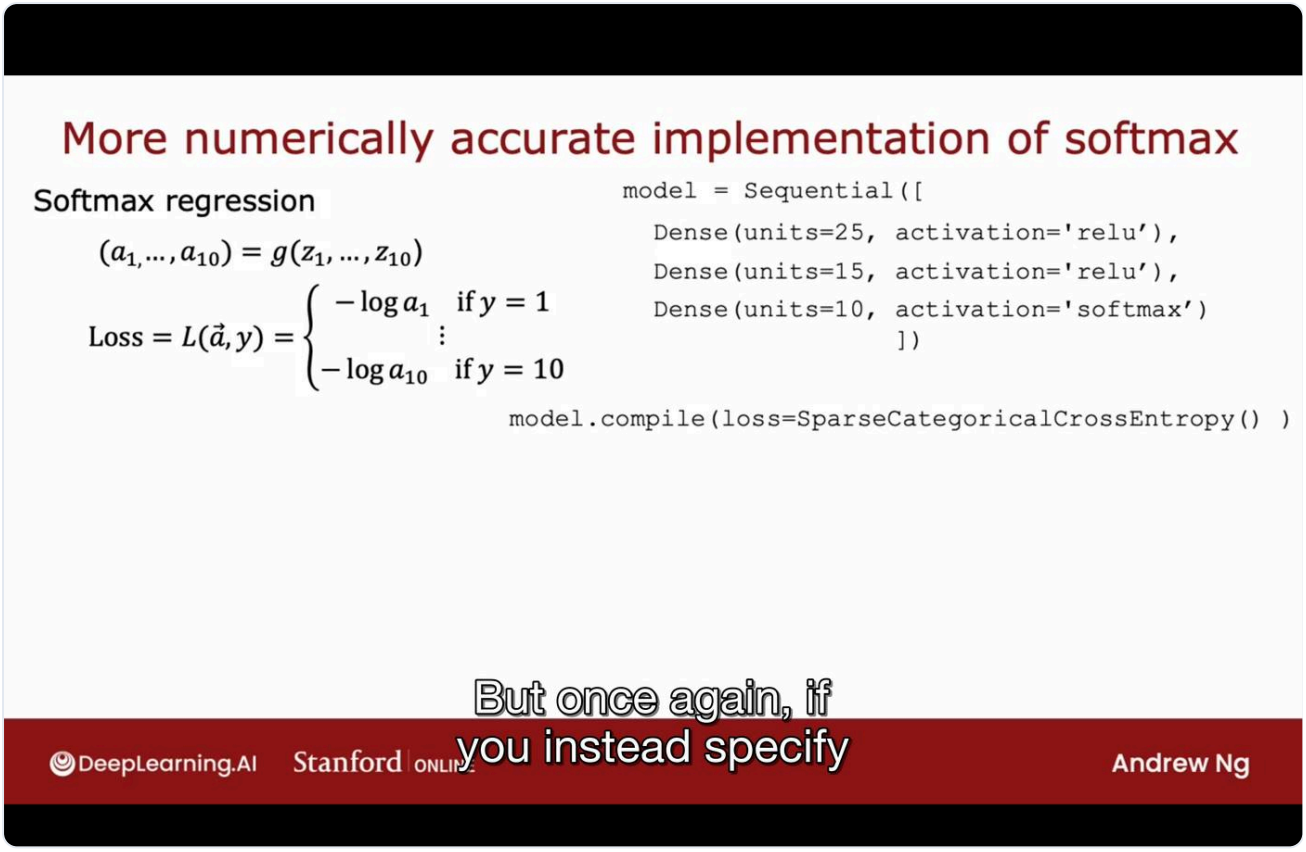


FIGURE 26.9 | More Numerically Accurate Implementation of Softmax

But once again, if you instead specify that the loss is if y is equal to 1 is negative log of this formula, and so on.

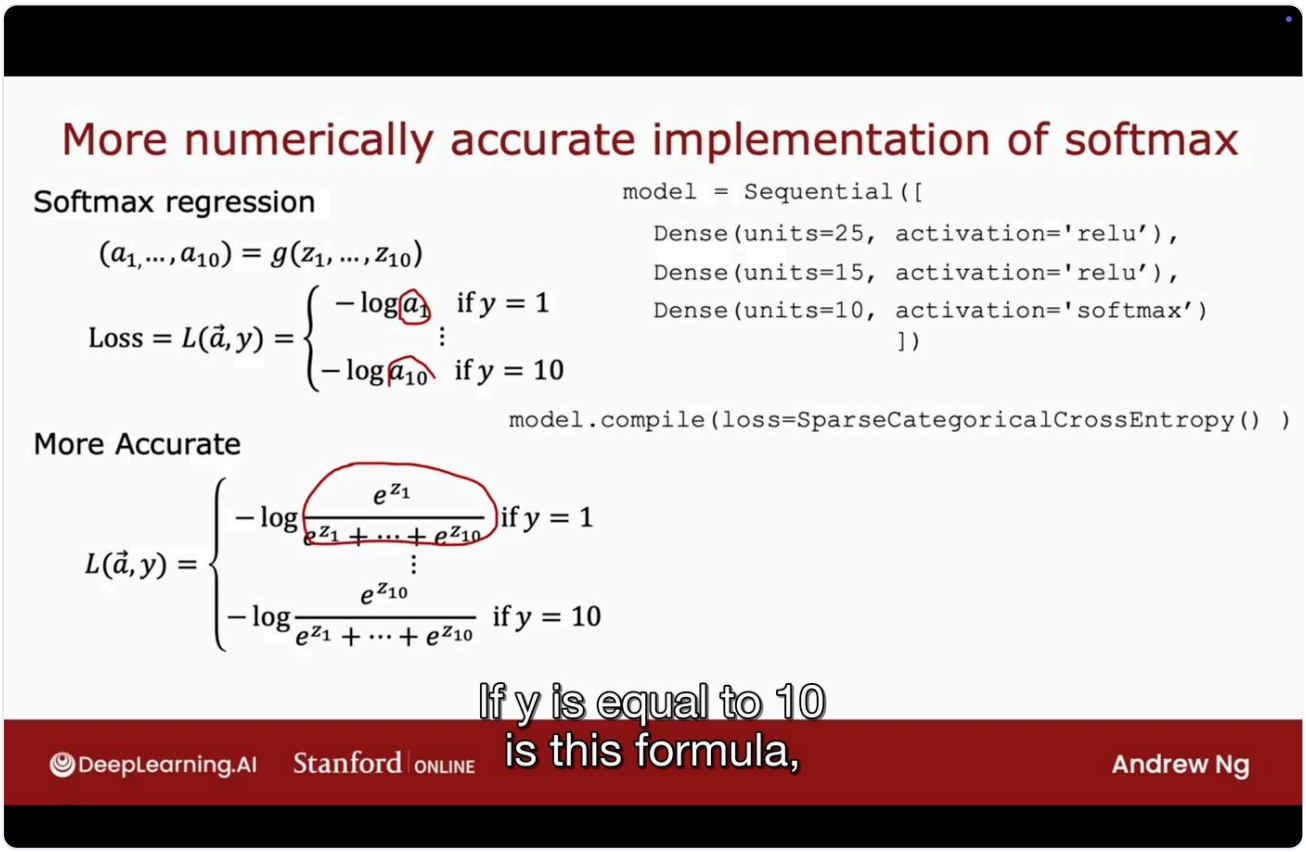


FIGURE 26.10 | More Numerically Accurate Implementation of Softmax

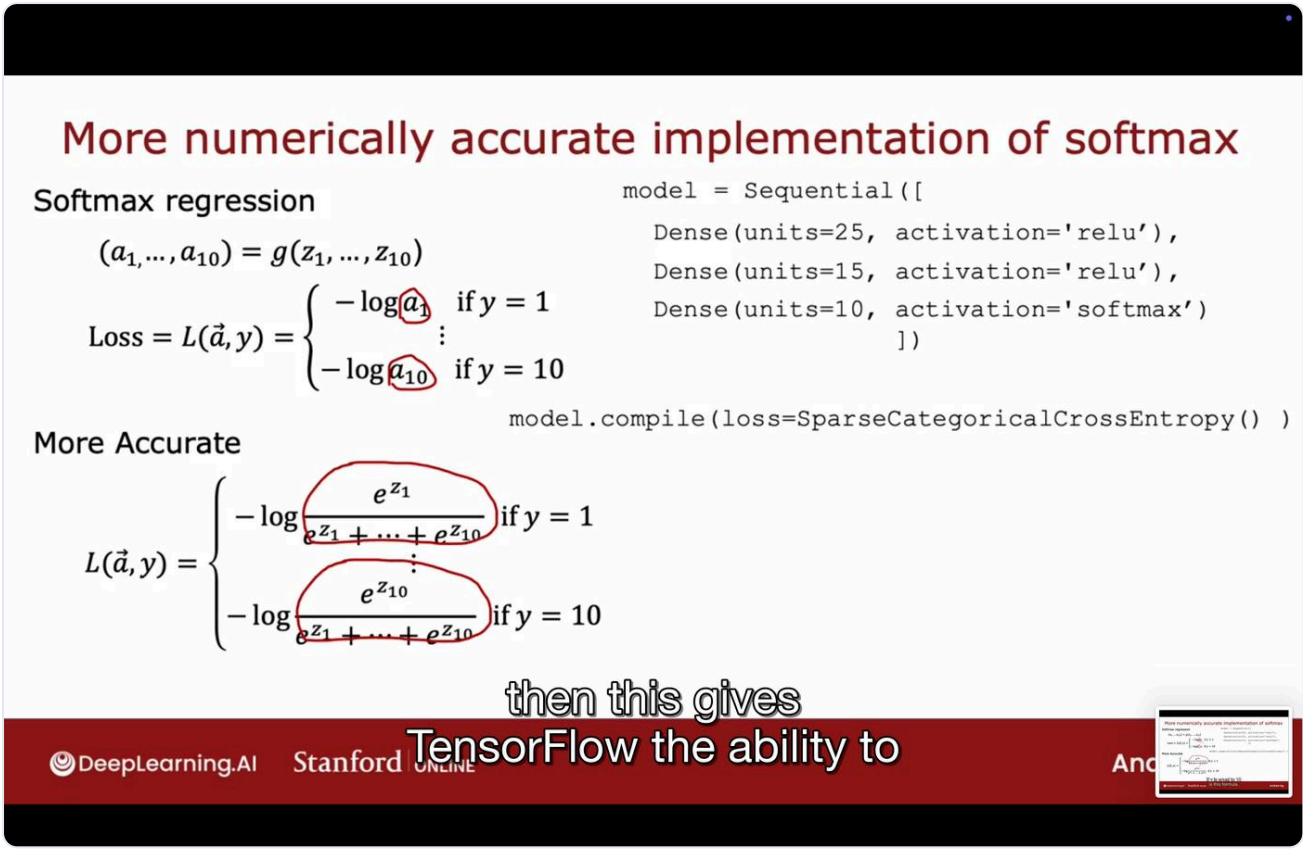


FIGURE 26.11 | More Numerically Accurate Implementation of Softmax

If y is equal to 10 is this formula, then this gives TensorFlow the ability to rearrange terms and compute this integral numerically accurate way. Just to give you some intuition for why TensorFlow might want to do this, it turns out if one of the z's really small than e to negative small number becomes very, very small or if one of the z's is a very large number, then e to the z can become a very large number and by rearranging terms, TensorFlow can avoid some of these very small or very large numbers and therefore come up with more actress computation for the loss function.

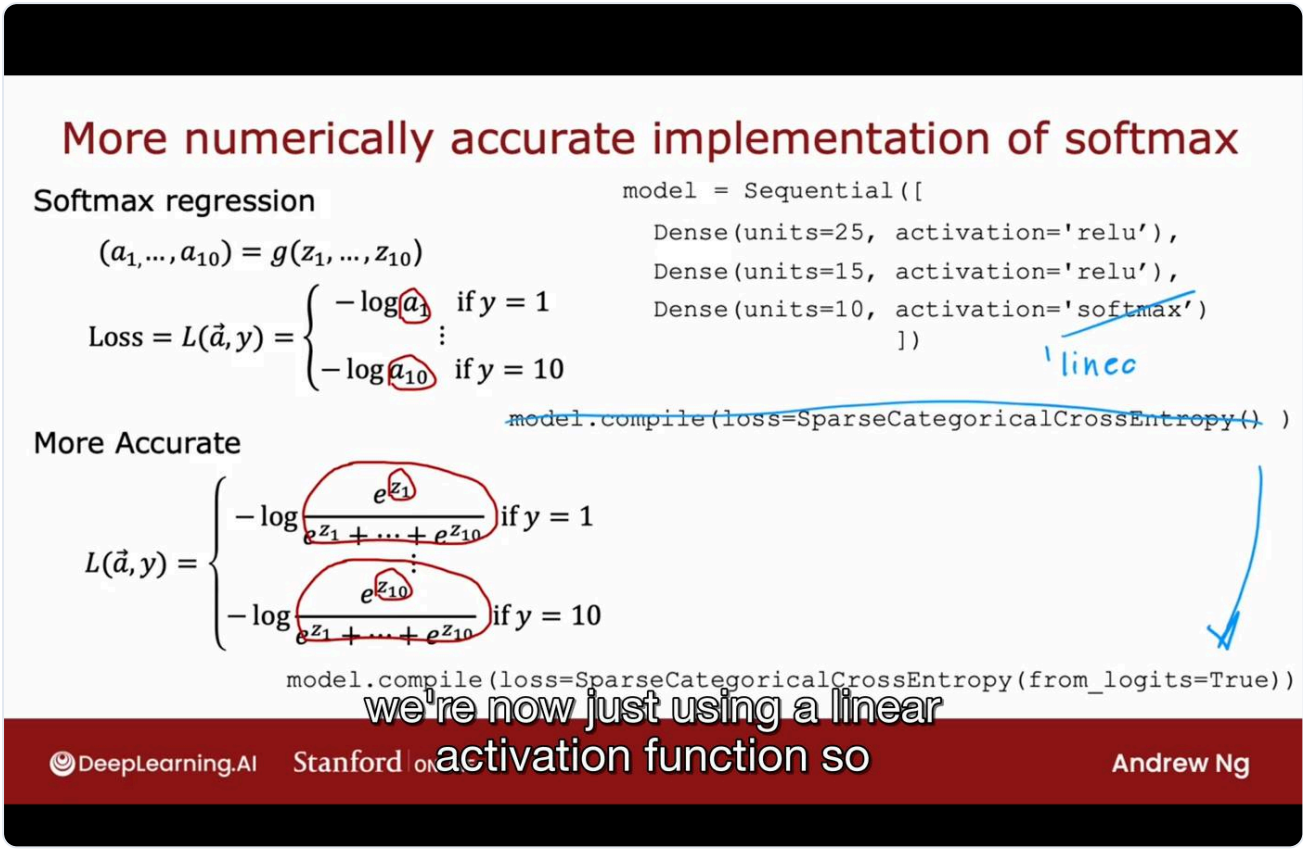


FIGURE 26.12 | More Numerically Accurate Implementation of Softmax

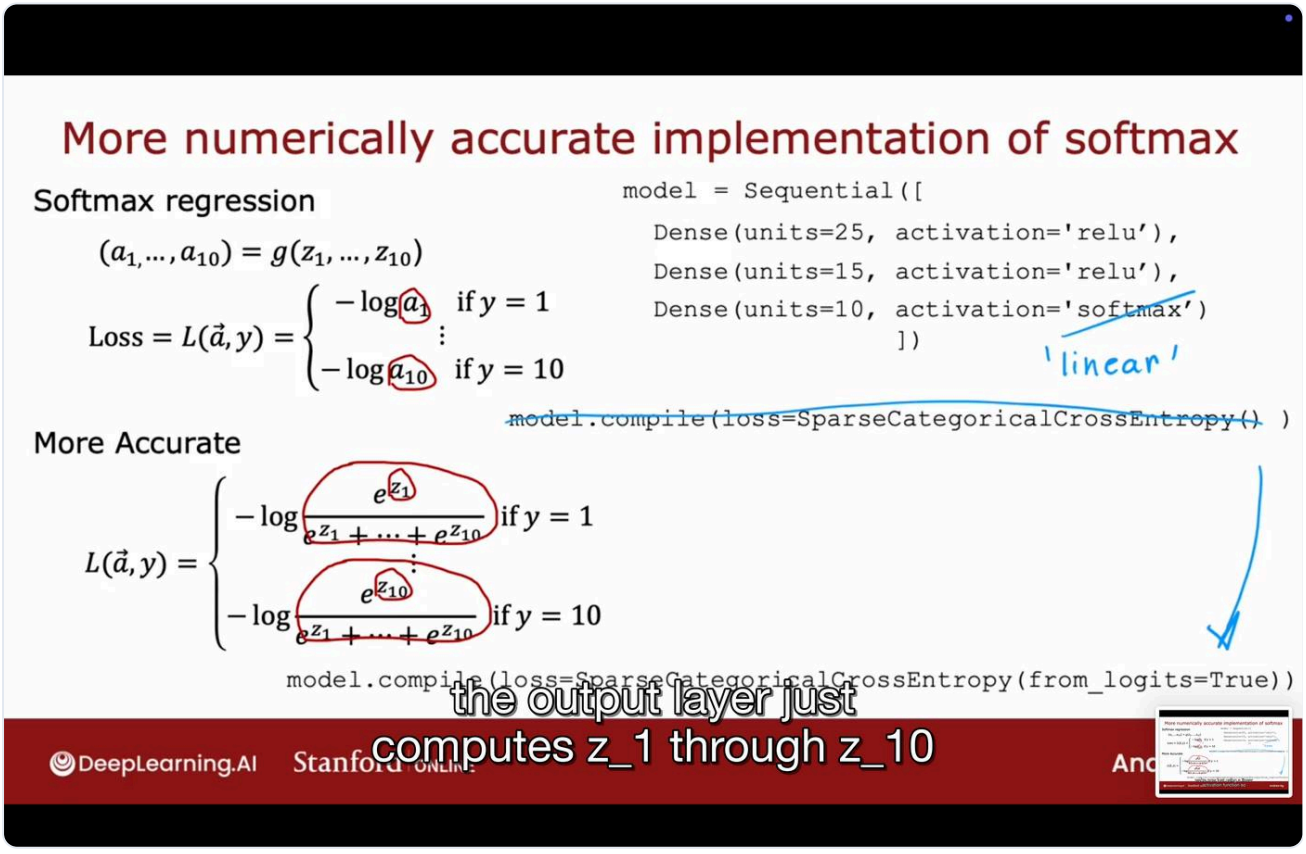


FIGURE 26.13 | More Numerically Accurate Implementation of Softmax

The code for doing this is shown here in the output layer, we're now just using a linear activation function so the output layer just computes z_1 through z_10 and this whole computation of the loss is then captured in the loss function over here, where again we have the from_logits equals true parameter. Once again, these two pieces of code do pretty much the same thing, except that the version that is recommended is more numerically accurate, although unfortunately, it is a little bit harder to read as well. If you're reading someone else's code and you see this and you're wondering what's going on is actually equivalent to the original implementation, at least in concept, except that is more numerically accurate.

The numerical roundoff errors for logistic regression aren't that bad, but it is recommended that you use this implementation down to the bottom instead, and conceptually, this code does the same thing as the first version that you had previously, except that it is a little bit more numerically accurate. Although the downside is maybe just a little bit harder to interpret as well. Now there's just one more detail, which is that we've now changed the neural network to use a linear activation function rather than a softmax activation function. The neural network's final layer no longer outputs these probabilities A_1 through A_10.

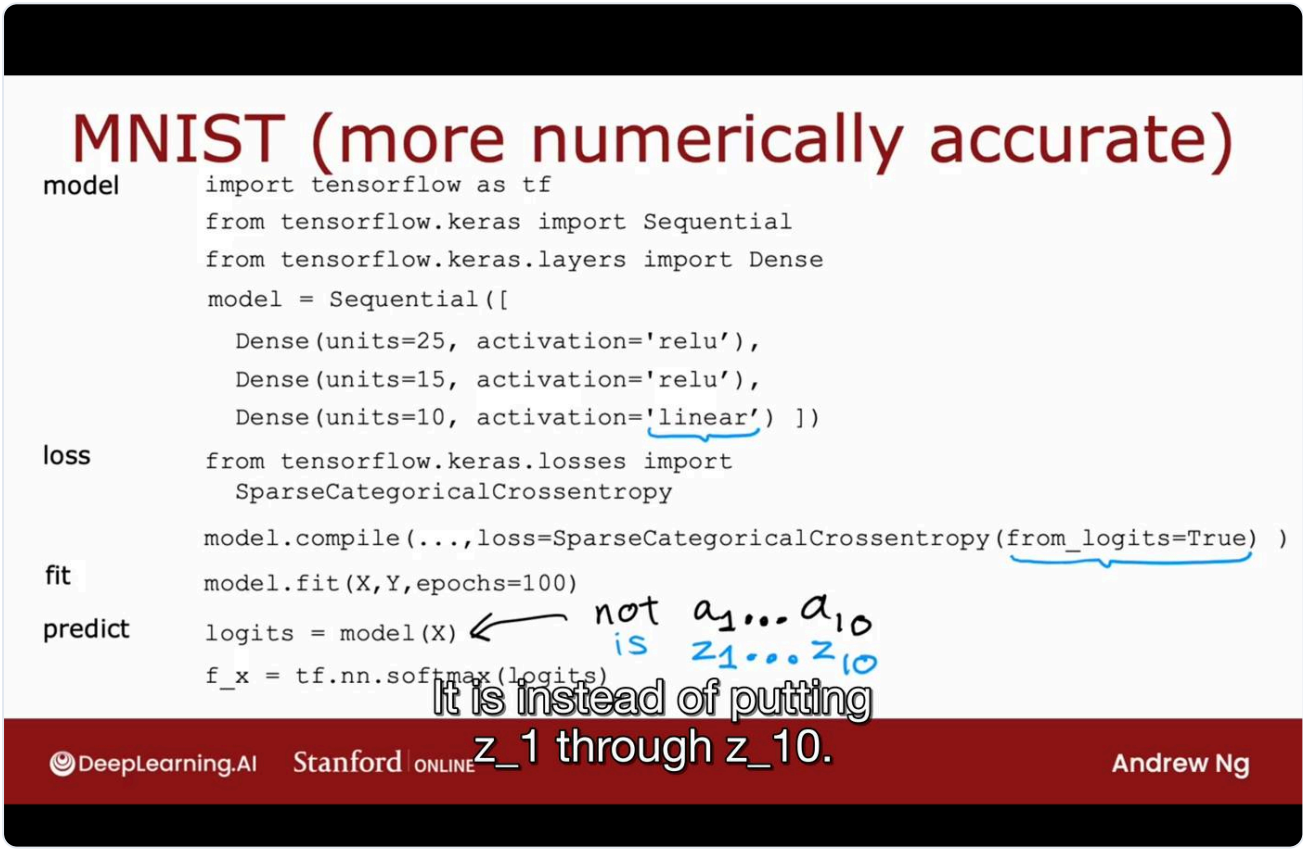


FIGURE 26.14 | MNIST (More Numerically Accurate)

It is instead of putting z_1 through z_10.

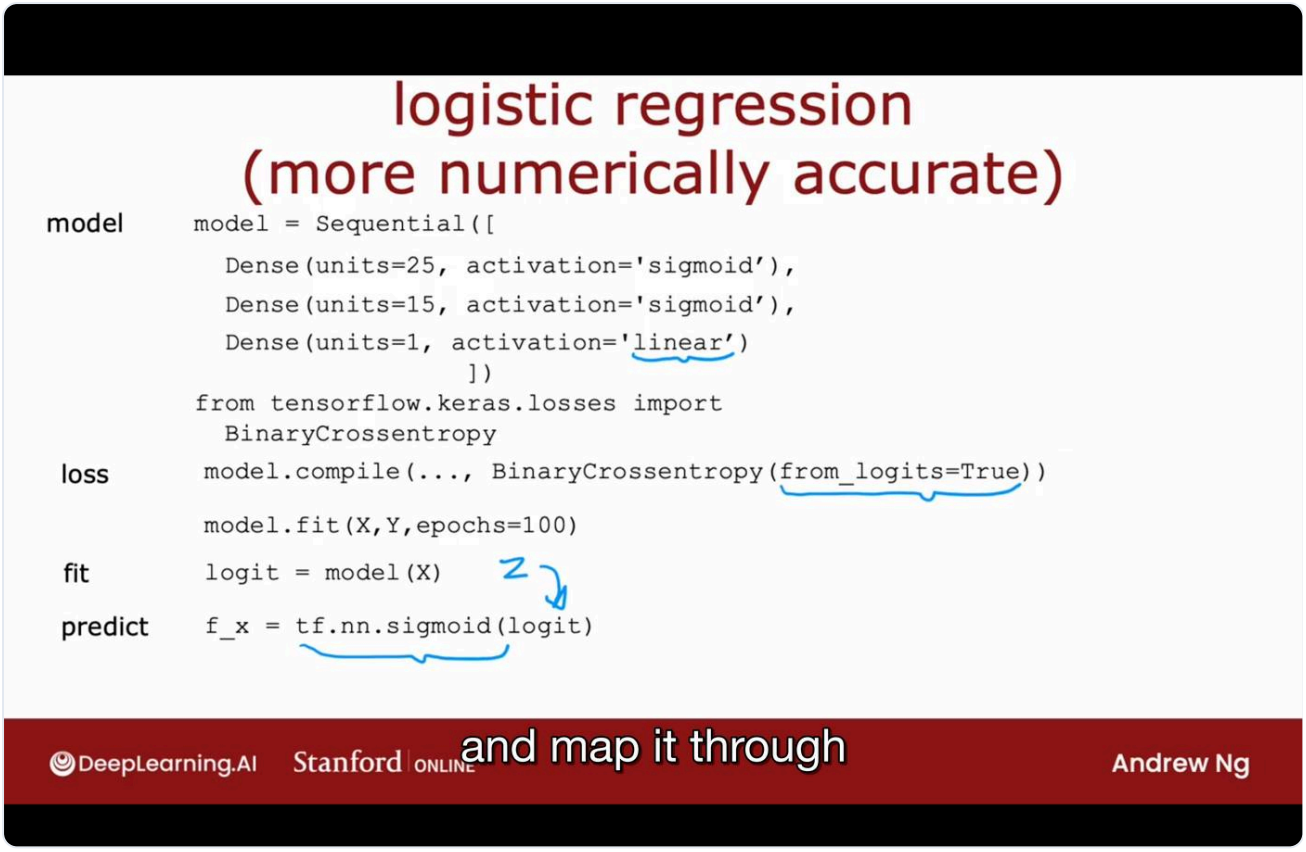


FIGURE 26.15

I didn't talk about it in the case of logistic regression, but if you were combining the output's logistic function with the loss function, then for logistic regressions, you also have to change the code this way to take the output value and map it through the logistic function in order to actually get the probability. You now know how to do multi-class classification with a softmax output layer and also how to do it in a numerically stable way. Before wrapping up multi-class classification, I want to share with you one other type of classification problem called a multi-label classification problem. Let's talk about that in the next video.

LECTURE 27

Multi-label Classification

You've learned about multi-class classification, where the output label Y can be any one of two or potentially many more than two possible categories. There's a different type of classification problem called a multi-label classification problem, which is where associate of each image, they could be multiple labels. Let me show you what I mean by that. If you're building a self-driving car or maybe a driver assistance system, then given a picture of what's in front of your car, you may want to ask a question like, is there a car or at least one car? Or is there a bus, or is there a pedestrian or are there any pedestrians?

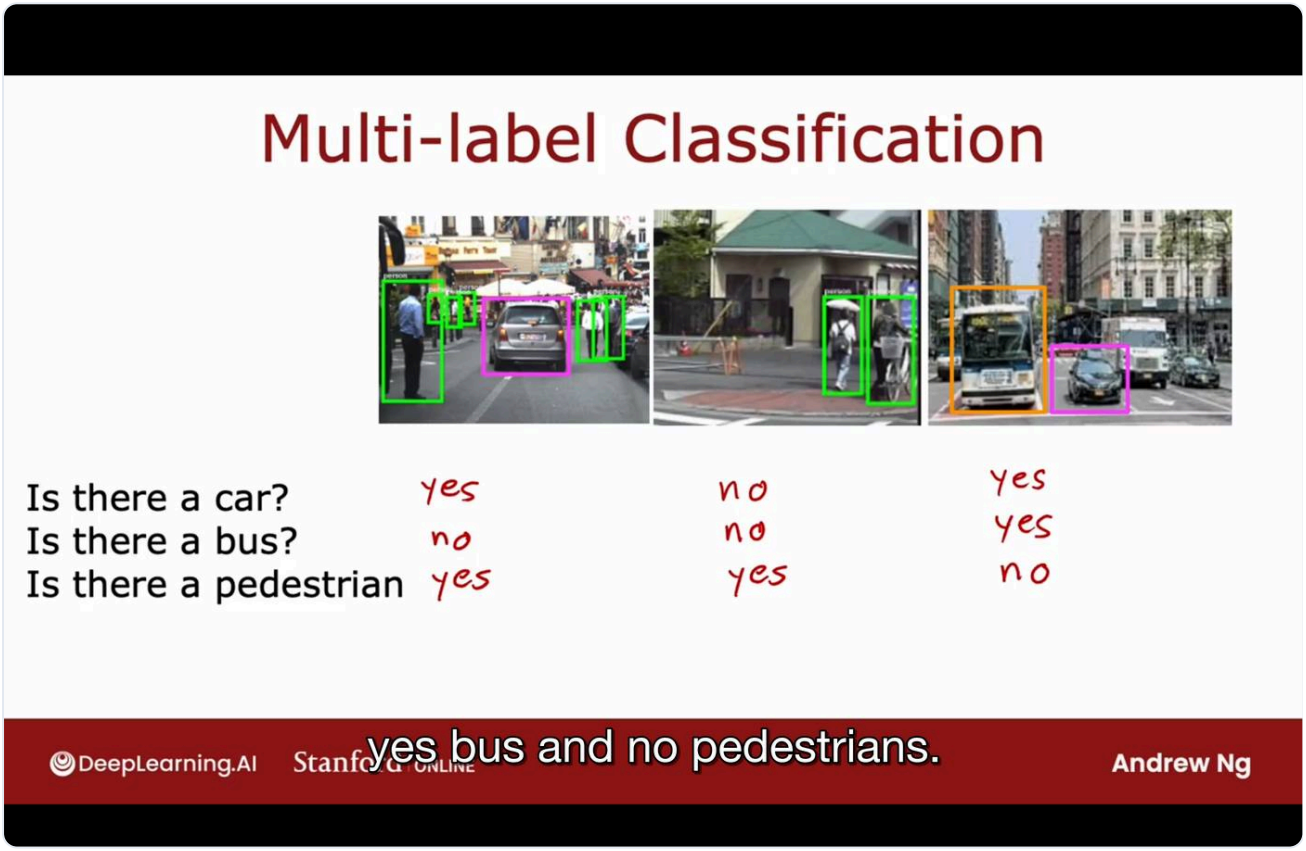


FIGURE 27.1 | Is There a Car? Yes No Oe

In this case, there is a car, there is no bus, and there is at least one pedestrian or in this second image, no cars, no buses and yes to pedestrians and yes car, yes bus and no pedestrians. These are examples of multi-label classification problems because associated with a single input, image X are three different labels corresponding to whether or not there are any cars, buses, or pedestrians in the image. In this case, the target of the Y is actually a vector of three numbers, and this is as distinct from multi-class classification, where for, say handwritten digit classification, Y was just a single number, even if that number could take on 10 different possible values.

How do you build a neural network for multi-label classification? One way to go about it is to just treat this as three completely separate machine learning problems. You could build one neural network to decide, are there any cars? The second one to detect buses and the third one to detect

pedestrians. That's actually not an unreasonable approach. Here's the first neural network to detect cars, second one to detect buses, third one to detect pedestrians. But there's another way to do this, which is to train a single neural network to simultaneously detect all three of cars, buses, and pedestrians, which is, if your neural network architecture, looks like this, there's input X.

First hidden layer offers a^1, second hidden layer offers a^2, and then the final output layer, in this case, we'll have three output neurons and we'll output a^3, which is going to be a vector of three numbers. Because we're solving three binary classification problems, so is there a car? Is there a bus? Is there a pedestrian?

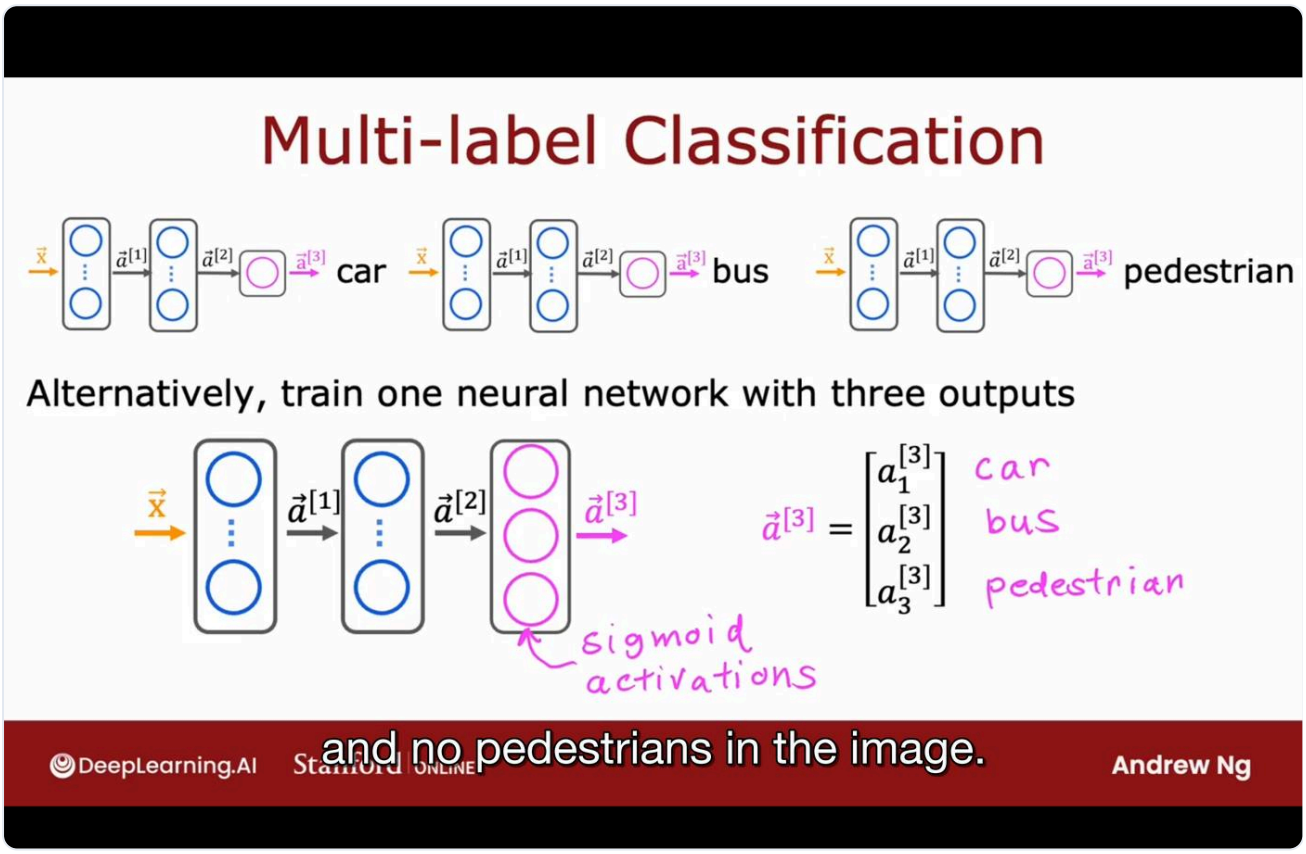


FIGURE 27.2 | Alternatively, Train One Neural Network with Three Outputs

You can use a sigmoid activation function for each of these three nodes in the output layer, and so a^3 in this case will be a_1^3, a_2^3, and a_3^3, corresponding to whether or not the learning [inaudible] as a car and no bus, and no pedestrians in the image. Multi-class classification and multi-label classification are sometimes confused with each other, and that's why in this video I want to share with you just a definition of multi-label classification problems as well, so that depending on your application, you could choose the right one for the job you want to do. So that's it for multi-label classification. I find that sometimes multi-class classification and multi-label classification are confused with other, which is why I wanted to expressively, in this video, share with you what is multi-label classification, so that depending on your application you can choose to write to for the job that you want to do.

And that wraps up the section on multi-class and multi-label classification. In the next video, we'll start to look at some more advanced neural network concepts, including an optimization algorithm that is even better than gradient descent. Let's take a look at that algorithm in the next

video, because it'll help you to get your learning algorithms to learn much faster. Let's go on to the next video.

LECTURE 28

Adam Algorithm Intuition

Gradient descent is an optimization algorithm that is widely used in machine learning, and was the foundation of many algorithms like linear regression and logistic regression and early implementations of neural networks. But it turns out that there are now some other optimization algorithms for minimizing the cost function, that are even better than gradient descent. In this video, we'll take a look at an algorithm that can help you train your neural network much faster than gradient descent. Recall that this is the expression for one step of gradient descent.

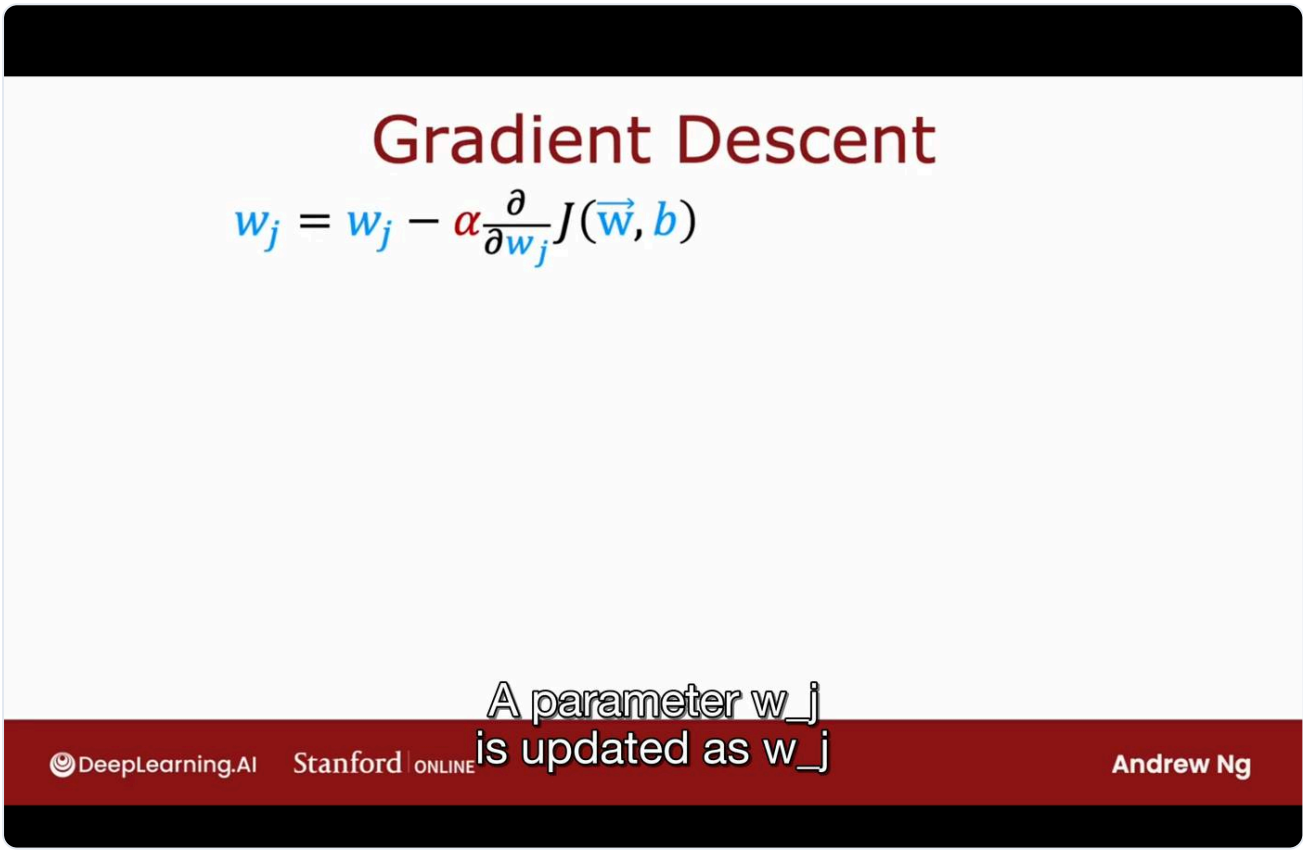


FIGURE 28.1

A parameter w_j is updated as w_j minus the learning rate Alpha times this partial derivative term. How can we make this work even better? In this example, I've plotted the cost function J using a contour plot comprising these ellipsis, and the minimum of this cost function is at the center of this ellipsis down here. Now, if you were to start gradient descent down here, one step of gradient descent, if Alpha is small, may take you a little bit in that direction.

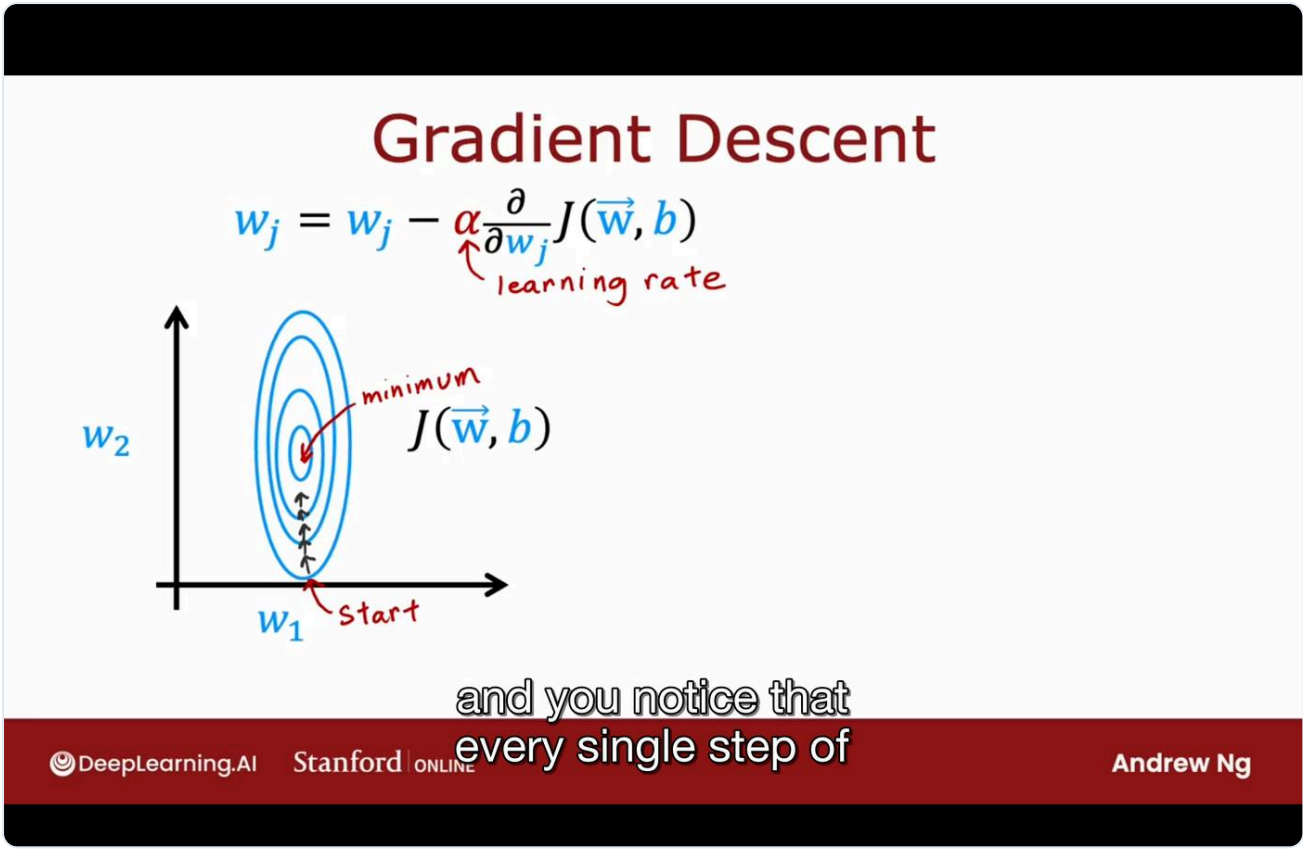


FIGURE 28.2 | Gradient Descent

Then another step, then another step, then another step, then another step, and you notice that every single step of gradient descent is pretty much going in the same direction, and if you see this to be the case, you might wonder, well, why don't we make Alpha bigger, can we have an algorithm to automatically increase Alpha? They just make it take bigger steps and get to the minimum faster.

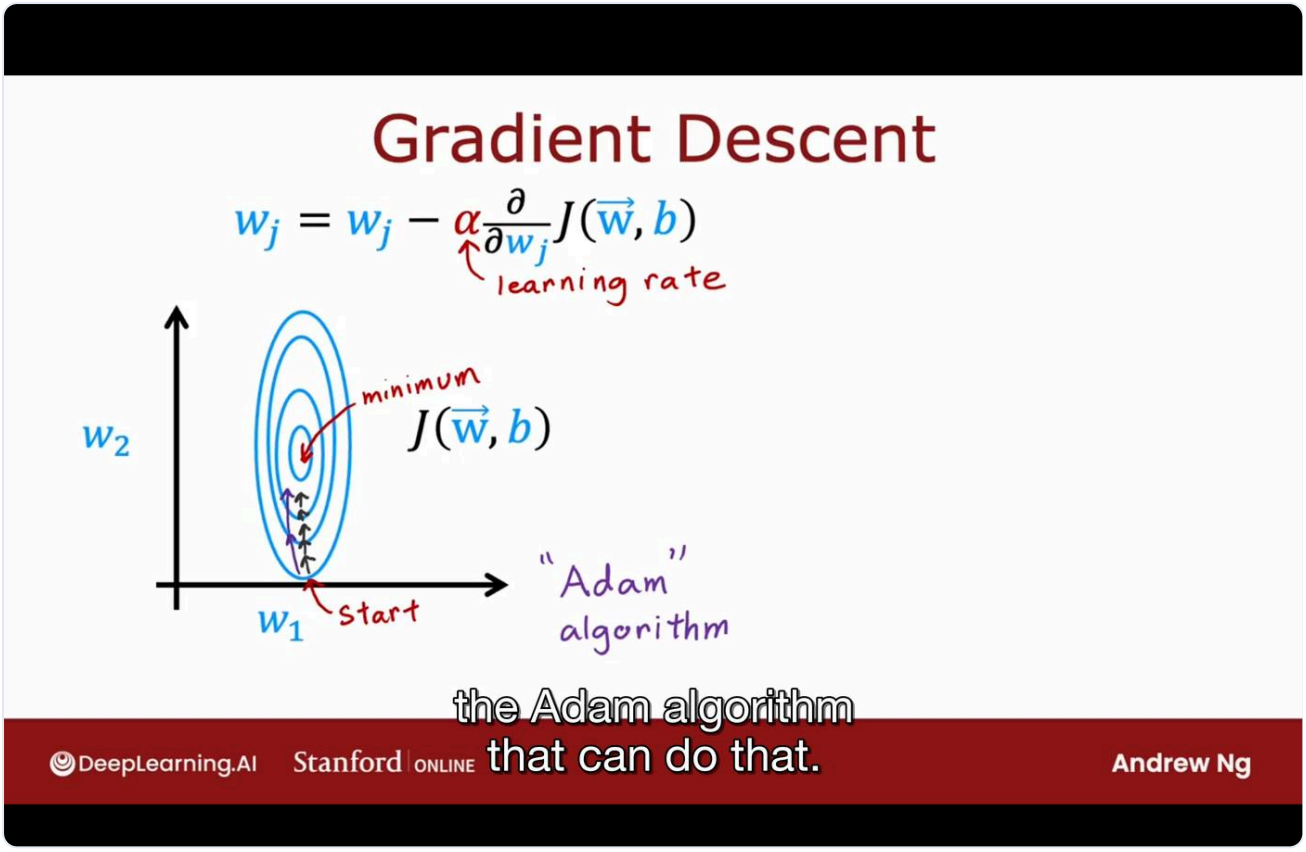


FIGURE 28.3 | Gradient Descent

There's an algorithm called the Adam algorithm that can do that. If it sees that the learning rate is too small, and we are just taking tiny little steps in a similar direction over and over, we should just make the learning rate Alpha bigger.

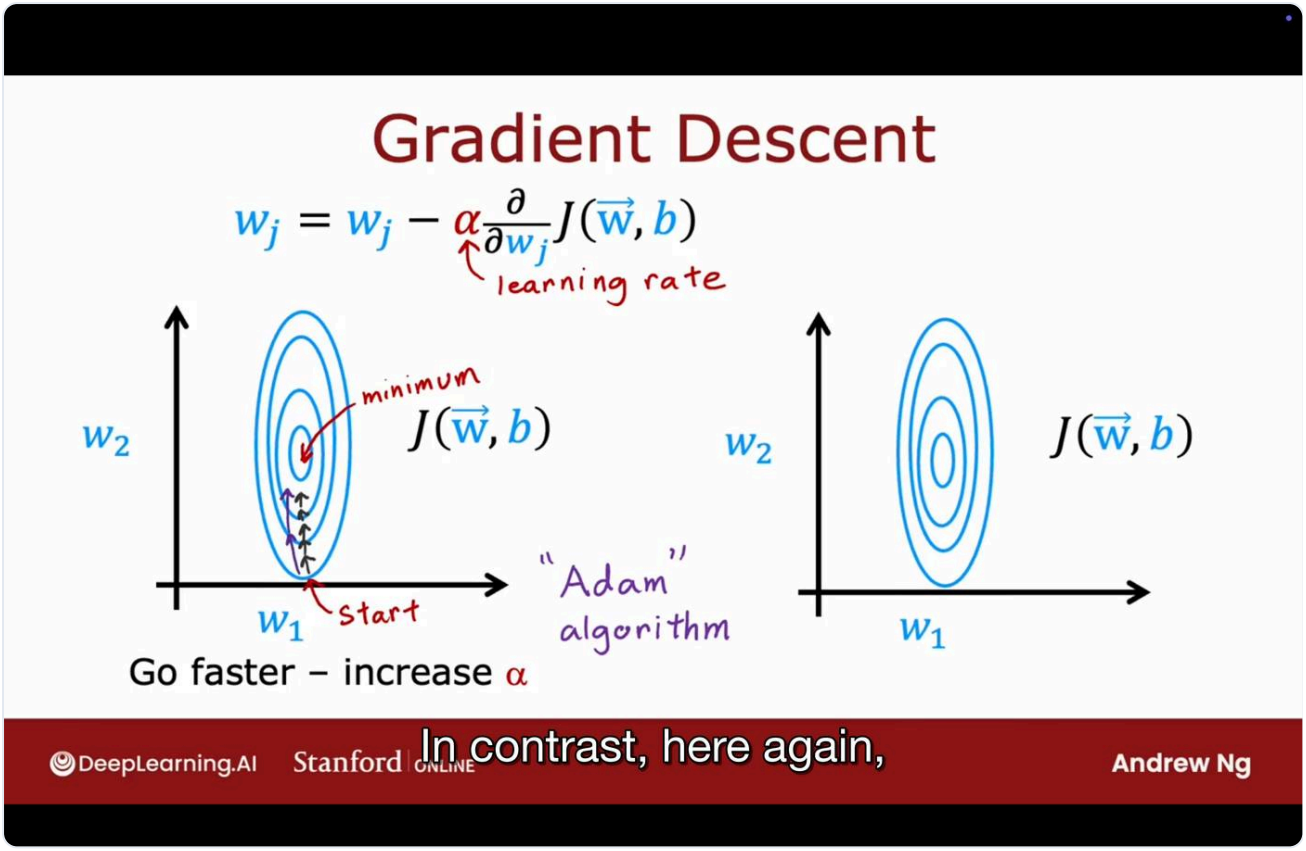


FIGURE 28.4 | Gradient Descent

In contrast, here again, is the same cost function if we were starting here and have a relatively big learning rate Alpha, then maybe one step of gradient descent takes us here, in the second step takes us here, third step, and the fourth step, and the fifth step, and the sixth step, and if you see gradient descent doing this, is oscillating back and forth. You'd be tempted to say, well, why don't we make the learning rates smaller?

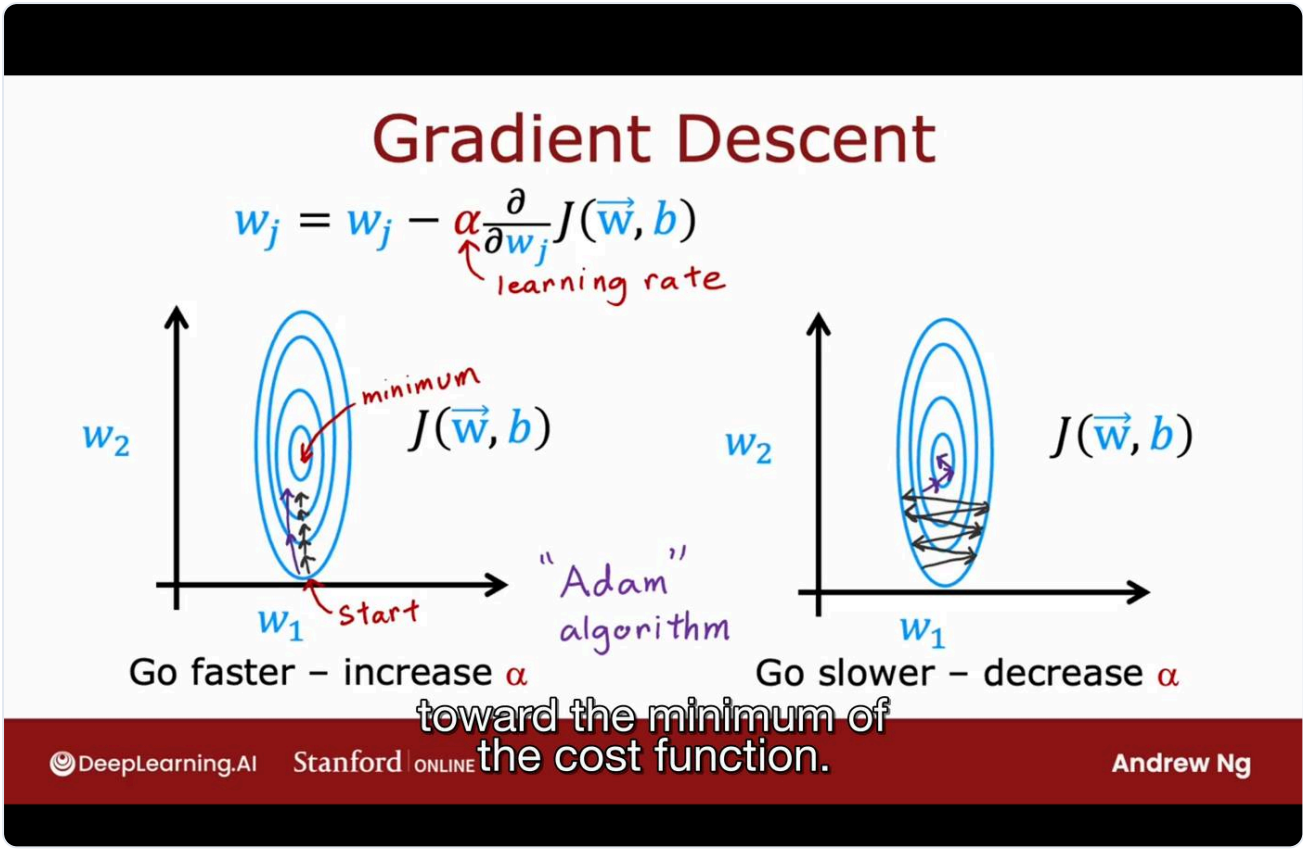


FIGURE 28.5 | Gradient Descent

The Adam algorithm can also do that automatically, and with a smaller learning rate, you can then take a more smooth path toward the minimum of the cost function. Depending on how gradient descent is proceeding, sometimes you wish you had a bigger learning rate Alpha, and sometimes you wish you had a smaller learning rate Alpha. The Adam algorithm can adjust the learning rate automatically.

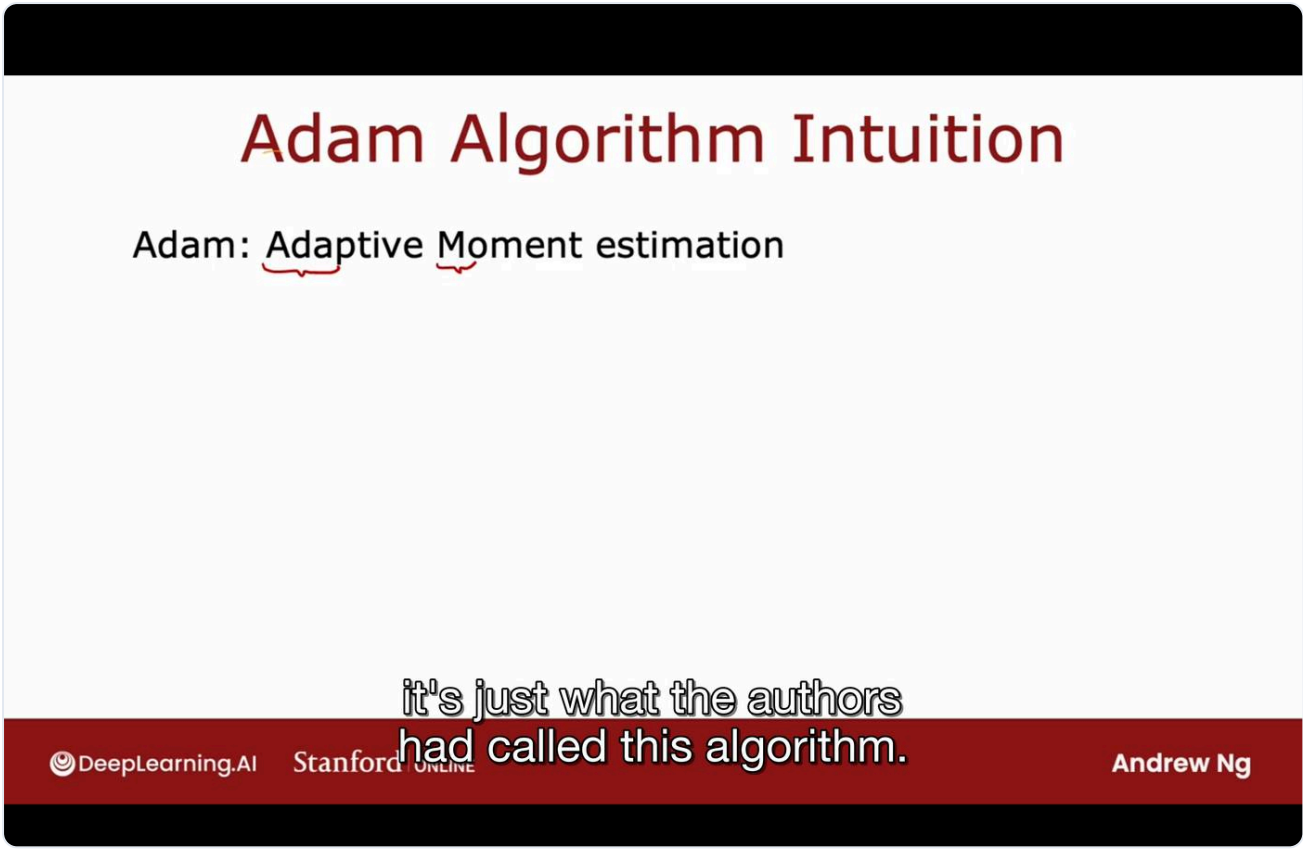


FIGURE 28.6 | Adam Algorithm Intuition

Adam stands for Adaptive Moment Estimation, or A-D-A-M, and don't worry too much about what this name means, it's just what the authors had called this algorithm. But interestingly, the Adam algorithm doesn't use a single global learning rate Alpha. It uses a different learning rates for every single parameter of your model.

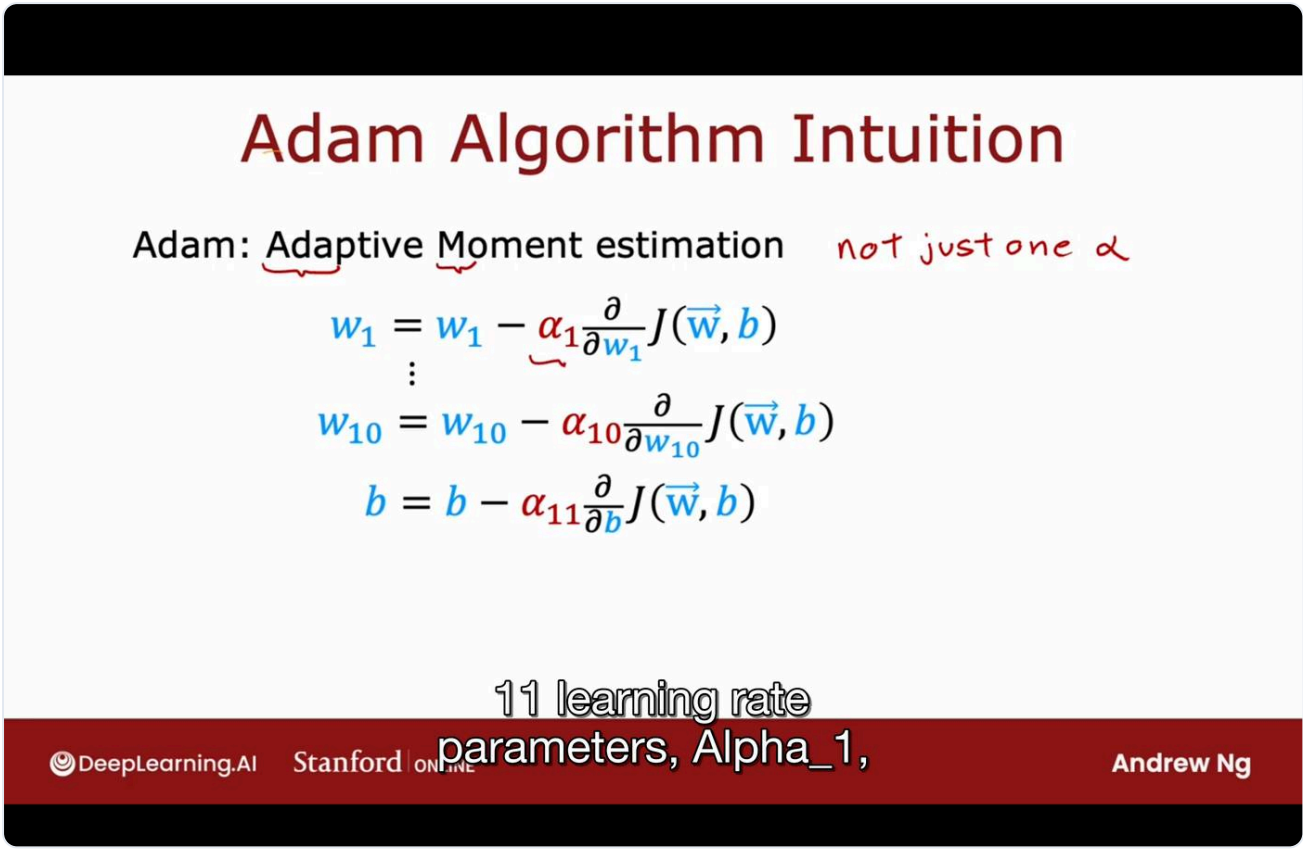


FIGURE 28.7 | Adam Algorithm Intuition

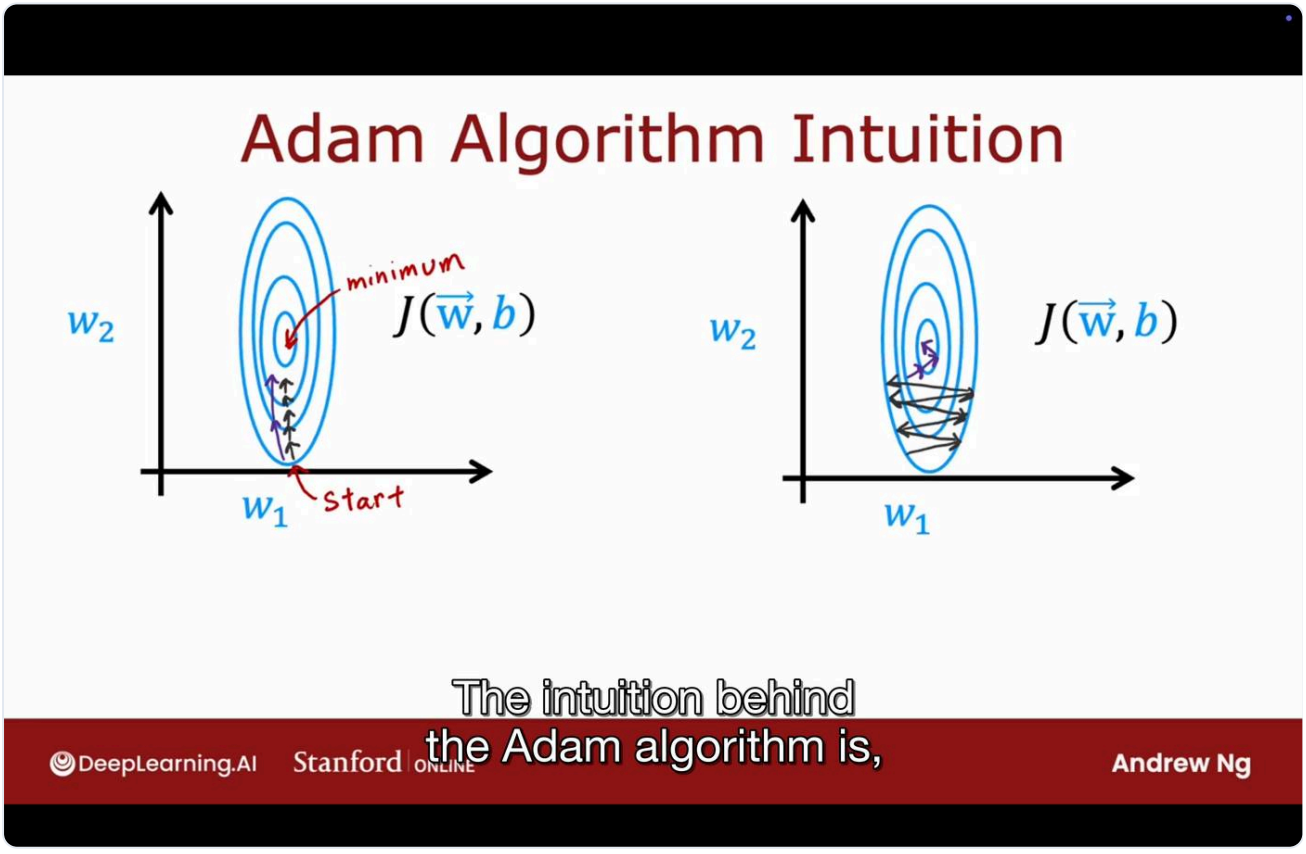


FIGURE 28.8

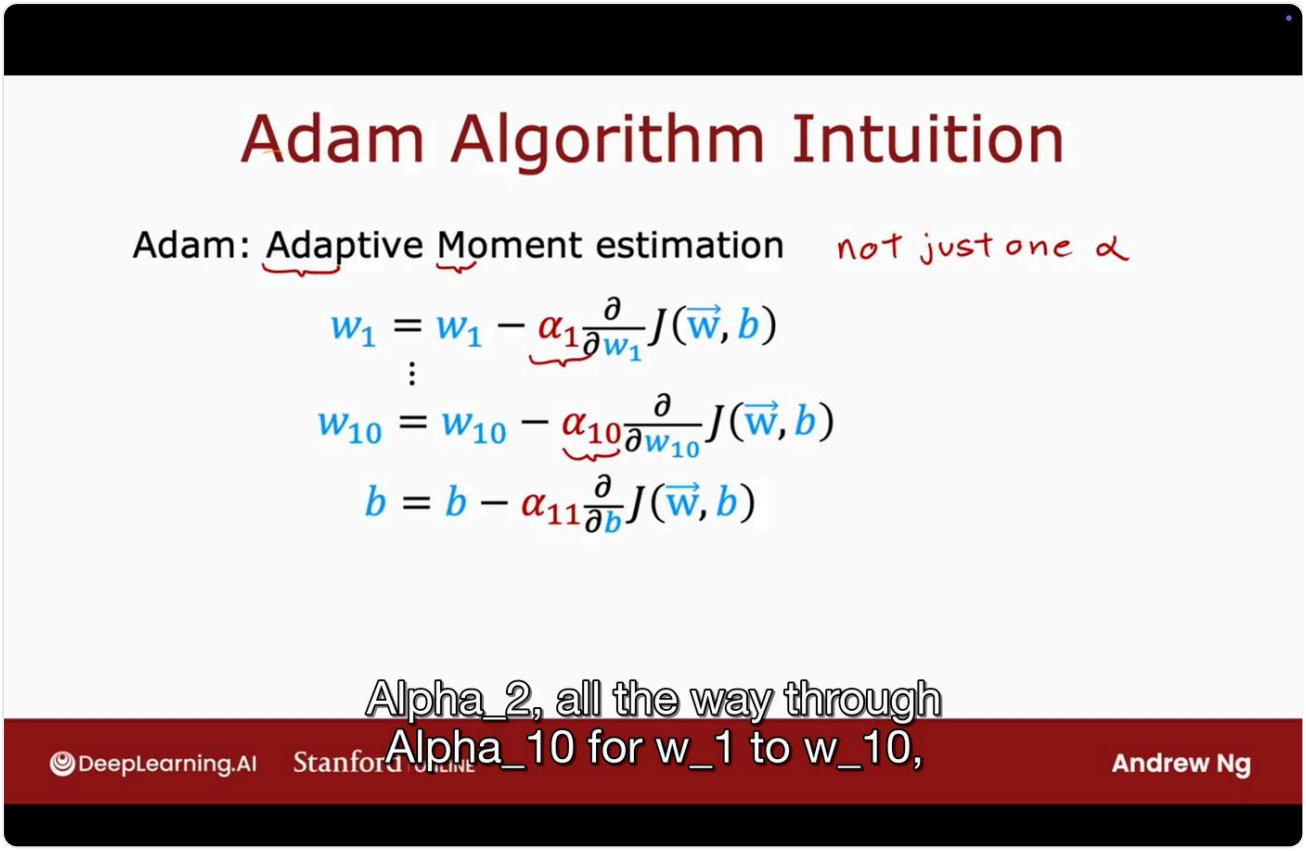


FIGURE 28.9 | Adam Algorithm Intuition

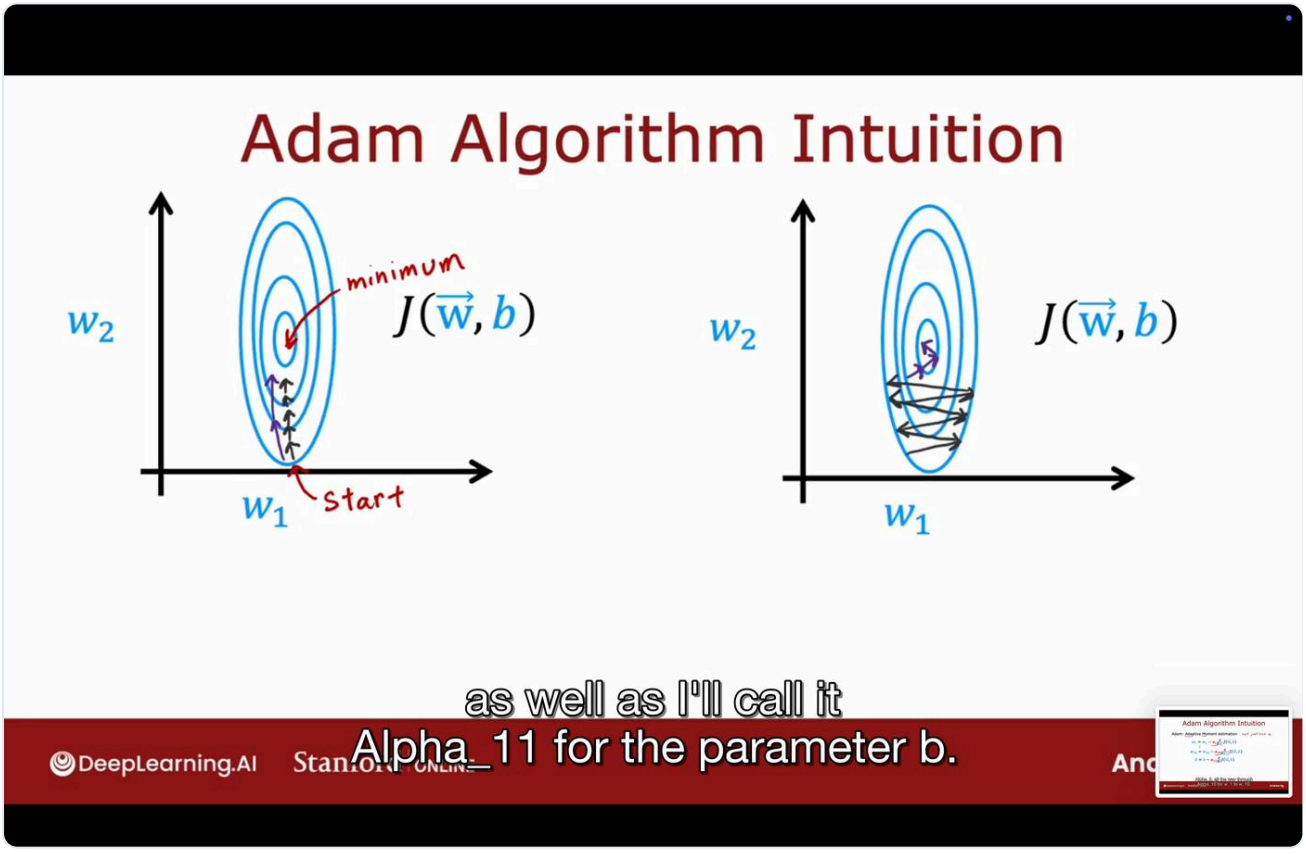


FIGURE 28.10 | Adam Algorithm Intuition

If you have parameters w_1 through w_10, as was b, then it actually has 11 learning rate parameters, Alpha_1, Alpha_2, all the way through Alpha_10 for w_1 to w_10, as well as I'll call it Alpha_11 for the parameter b.

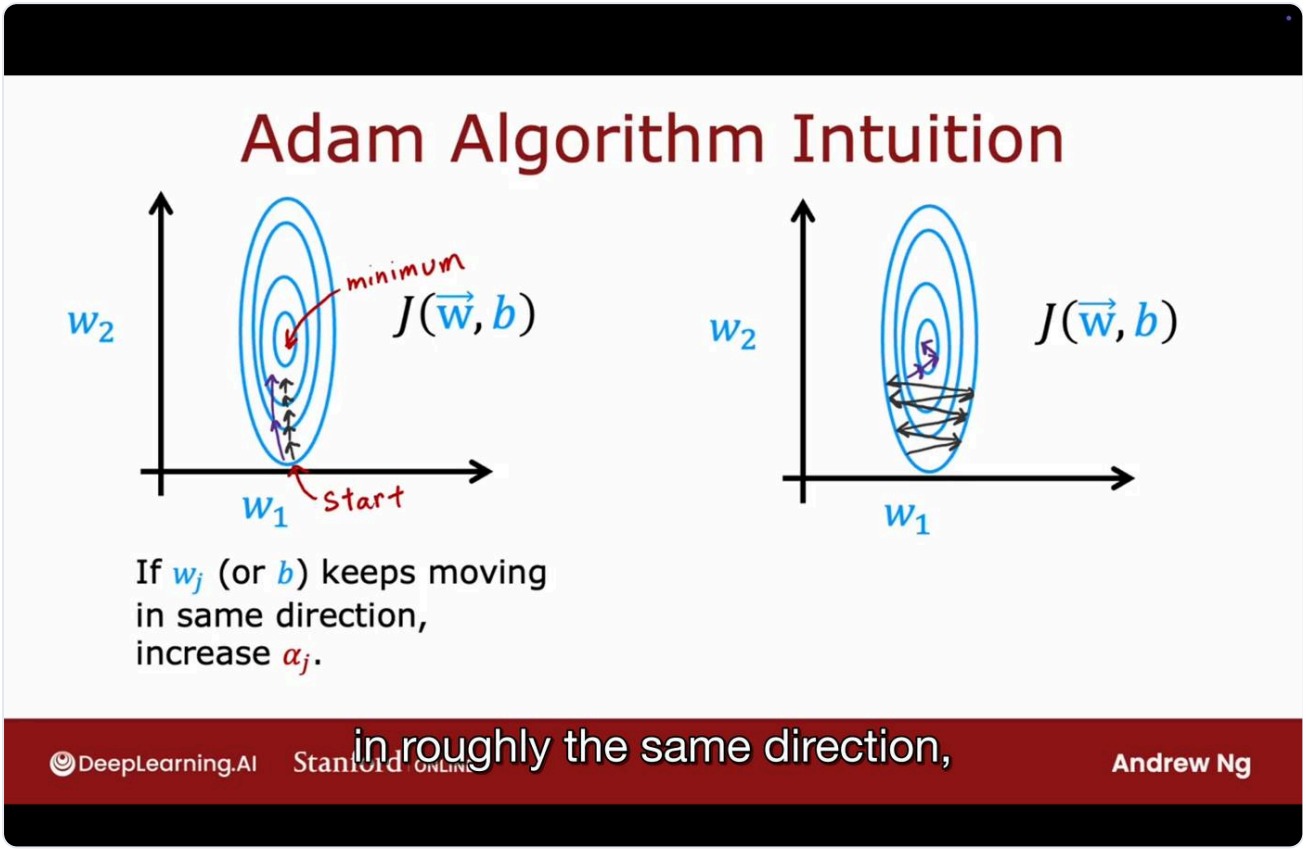


FIGURE 28.11 | Adam Algorithm Intuition

The intuition behind the Adam algorithm is, if a parameter w_j, or b seems to keep on moving in roughly the same direction. This is what we saw on the first example on the previous slide. But if it seems to keep on moving in roughly the same direction, let's increase the learning rate for that parameter. Let's go faster in that direction.

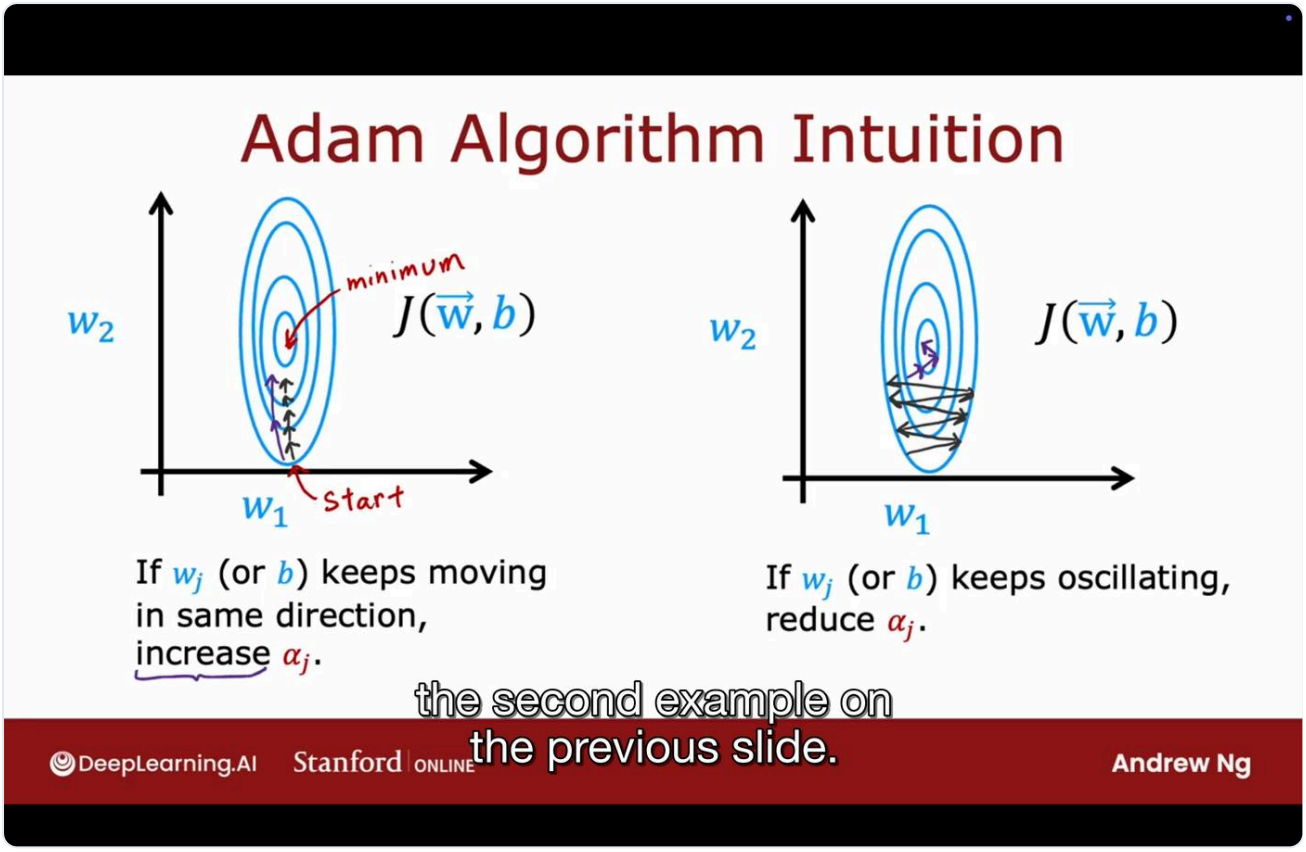


FIGURE 28.12 | Adam Algorithm Intuition

Conversely, if a parameter keeps oscillating back and forth, this is what you saw in the second example on the previous slide. Then let's not have it keep on oscillating or bouncing back and forth. Let's reduce Alpha_j for that parameter a little bit. The details of how Adam does this are a bit complicated and beyond the scope of this course, but if you take some more advanced deep learning courses later, you may learn more about the details of this Adam algorithm, but in codes this is how you implement it.



FIGURE 28.13

The model is exactly the same as before, and the way you compile the model is very similar to what we had before, except that we now add one extra argument to the compile function, which is that we specify that the optimizer you want to use is tf.keras.optimizers.Adam optimizer.

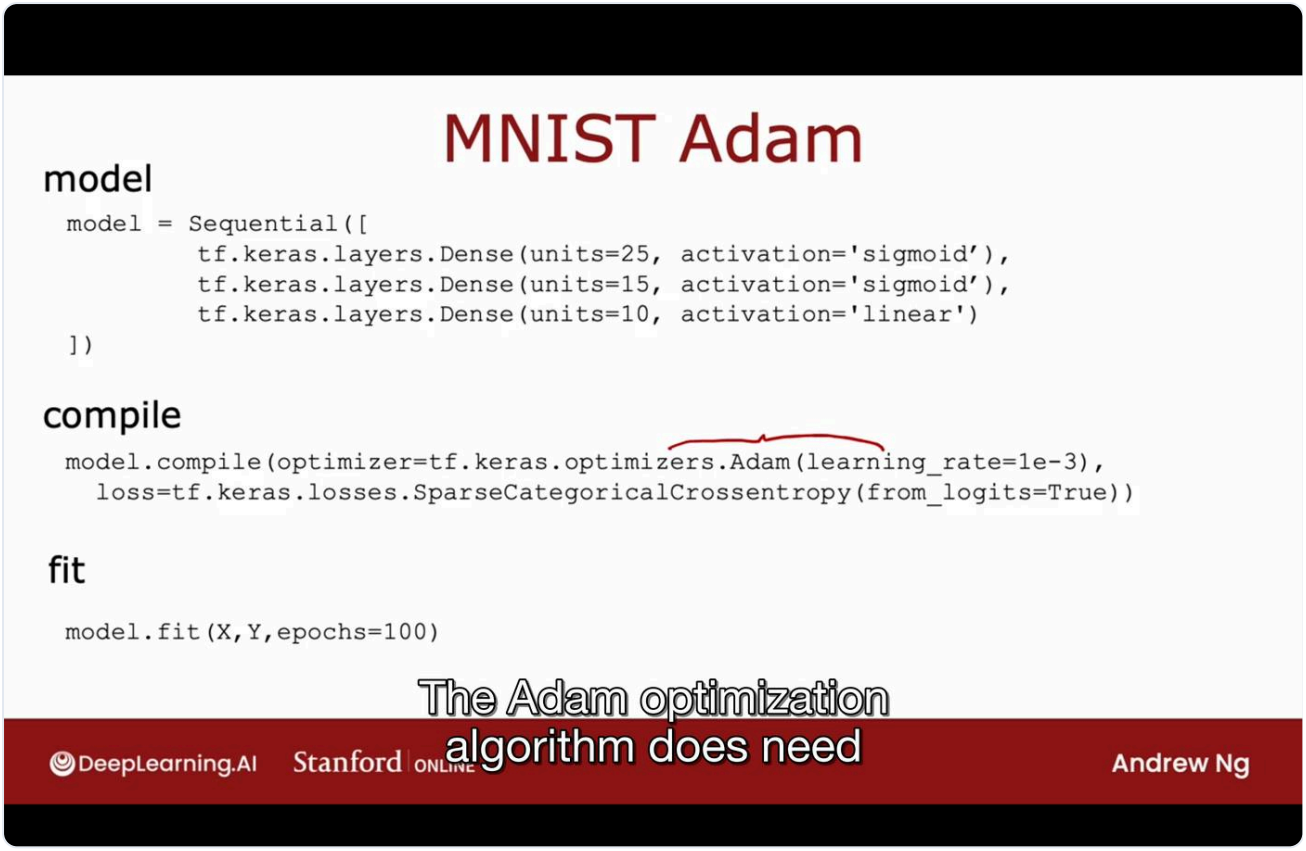


FIGURE 28.14

The Adam optimization algorithm does need some default initial learning rate Alpha, and in this example, I've set that initial learning rate to be 10^ negative 3.

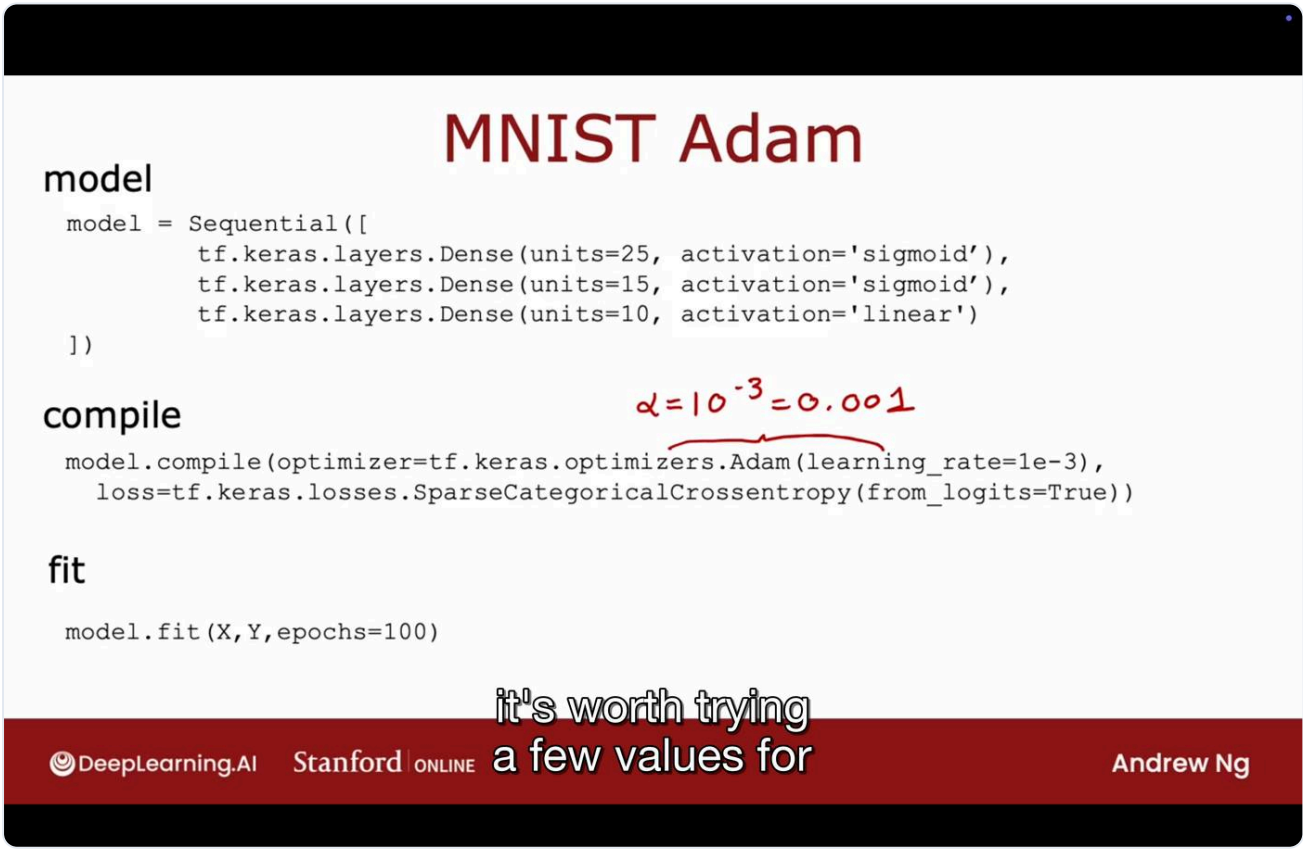


FIGURE 28.15

But when you're using the Adam algorithm in practice, it's worth trying a few values for this default global learning rate. Try some large and some smaller values to see what gives you the fastest learning performance. Compared to the original gradient descent algorithm that you had learned in the previous course though, the Adam algorithm, because it can adapt the learning rate a bit automatically, it is more robust to the exact choice of learning rate that you pick. Though there's still way tuning this parameter little bit to see if you can get somewhat faster learning. That's it for the Adam optimization algorithm. It typically works much faster than gradient descent, and it's become a de facto standard in how practitioners train their neural networks.

If you're trying to decide what learning algorithm to use, what optimization algorithm to use to train your neural network. A safe choice would be to just use the Adam optimization algorithm, and most practitioners today will use Adam rather than the optional gradient descent algorithm, and with this, I hope that your learning algorithms will be able to learn much more quickly. Now, in the next couple of videos, I'd like to touch on some more advanced concepts for neural networks, and in particular, in the next video, let's take a look at some alternative layer types.

LECTURE 29

Convolutional Neural Network

All the neural network layers with you so far have been the dense layer type in which every neuron in the layer gets its inputs all the activations from the previous layer. And it turns out that just using the dense layer type, you can actually build some pretty powerful learning algorithms. And to help you build further intuition about what neural networks can do. It turns out that there's some other types of layers as well with other properties. In this video I'd like to briefly touch on this and give you an example of a different type of neural network layer. Let's take a look to recap in the dense layer that we've been using the activation of a neuron in say the second hidden layer Is a function of every single activation value from the previous layer of a one.

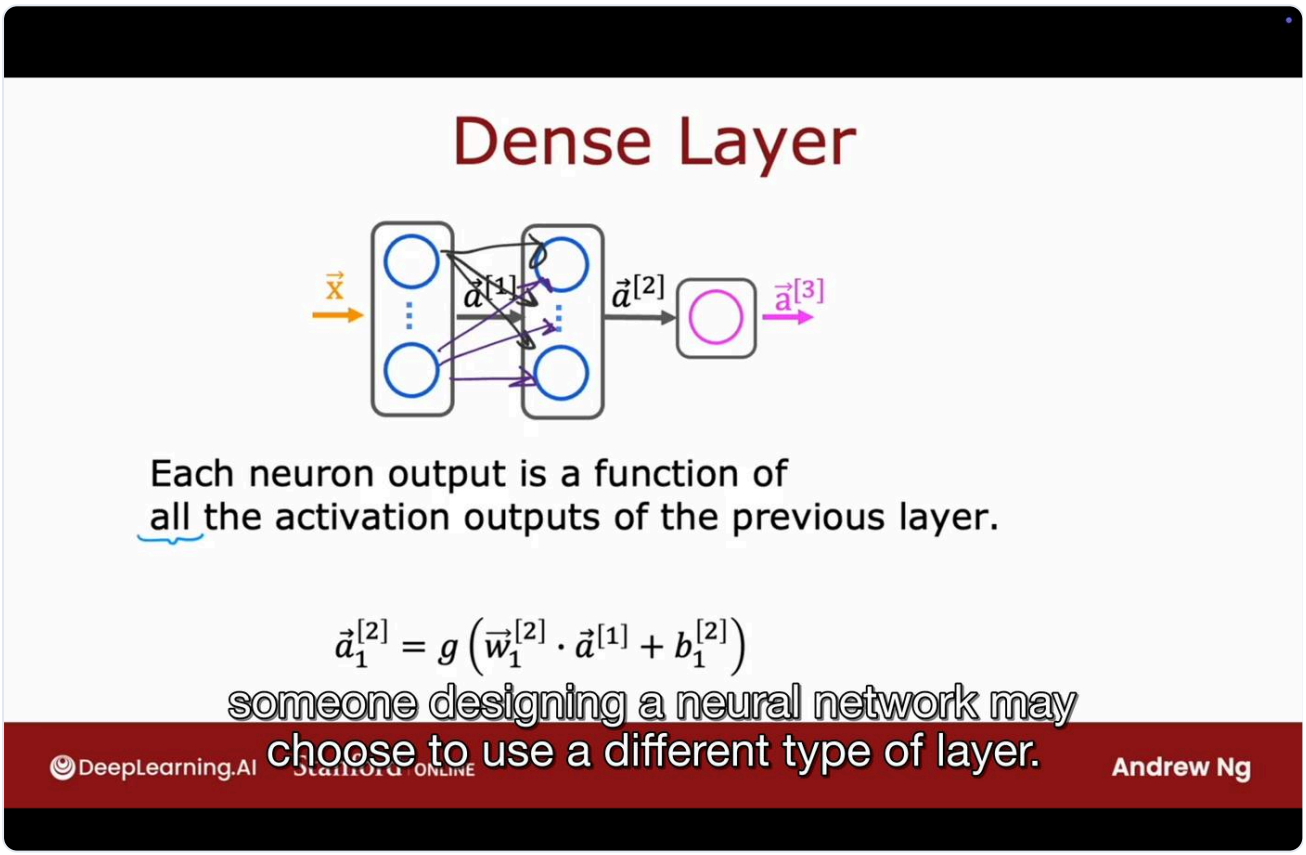


FIGURE 29.1 | Dense Layer

But it turns out that for some applications, someone designing a neural network may choose to use a different type of layer. One other layer type that you may see in some work is called a convolutional layer. Let me illustrate this with an example. So what I'm showing on the left is the input X. Which is a handwritten digit nine. And what I'm going to do is construct a hidden layer which will compute different activations as functions of this input image X. But here's something I can do for the first hidden unit, which I've drawn in blue rather than saying this neuron can look at all the pixels in this image. I might say this neuron can only look at the pixels in this little rectangular region.

Second neuron, which I'm going to illustrate in magenta is also not going to look at the entire input image X instead, it's only going to look at the pixels in a limited region of the image.

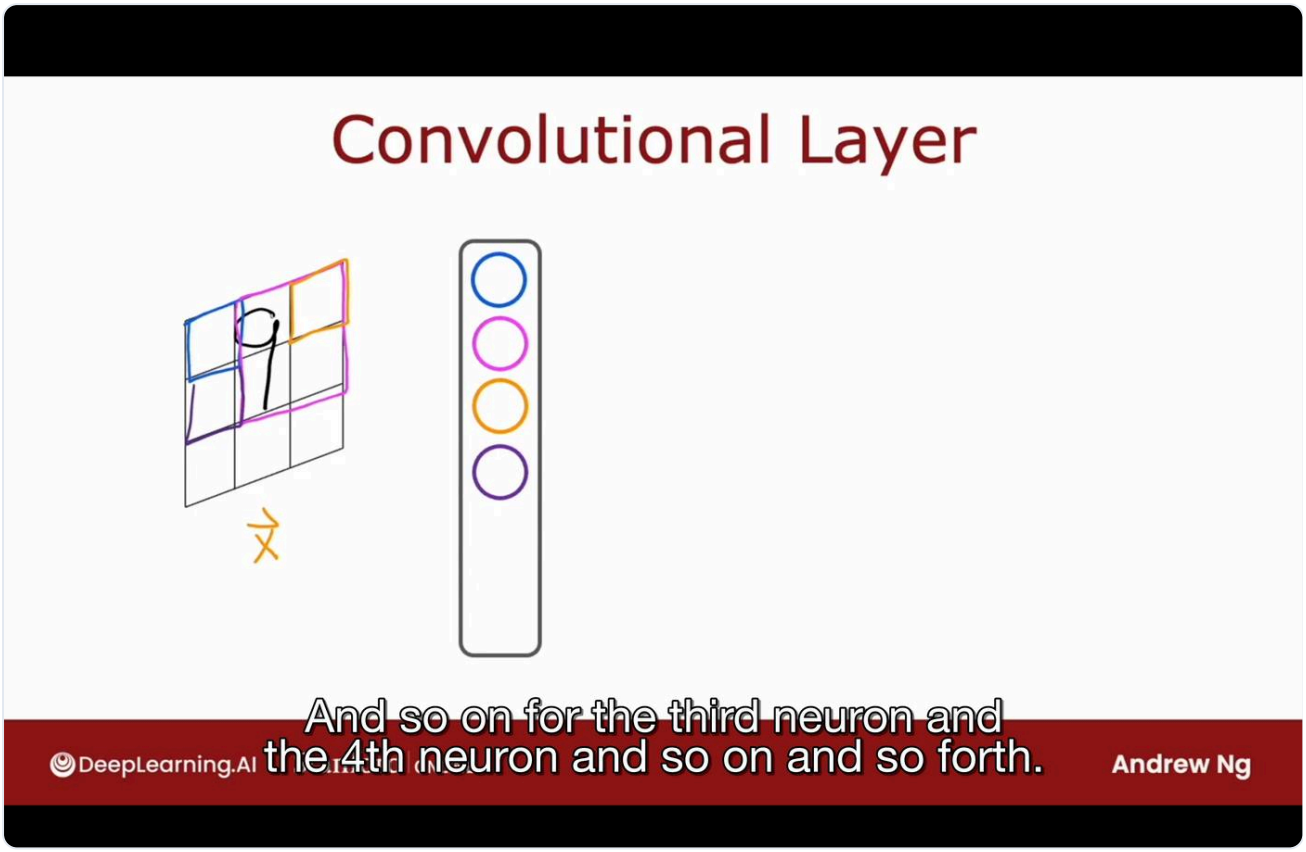


FIGURE 29.2 | Convolutional Layer

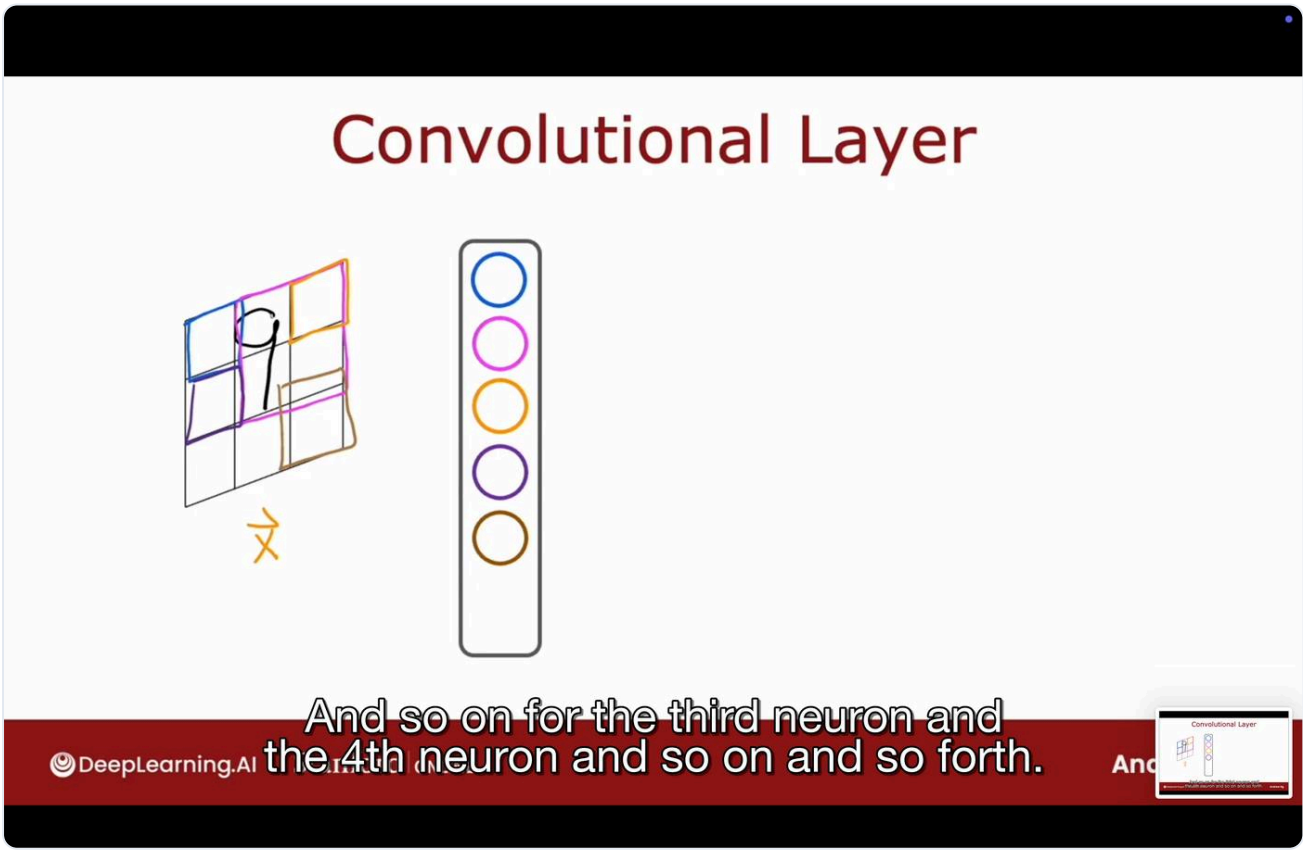


FIGURE 29.3 | Convolutional Layer

And so on for the third neuron and the 4th neuron and so on and so forth. Down to the last neuron which may be looking only at that region of the image.

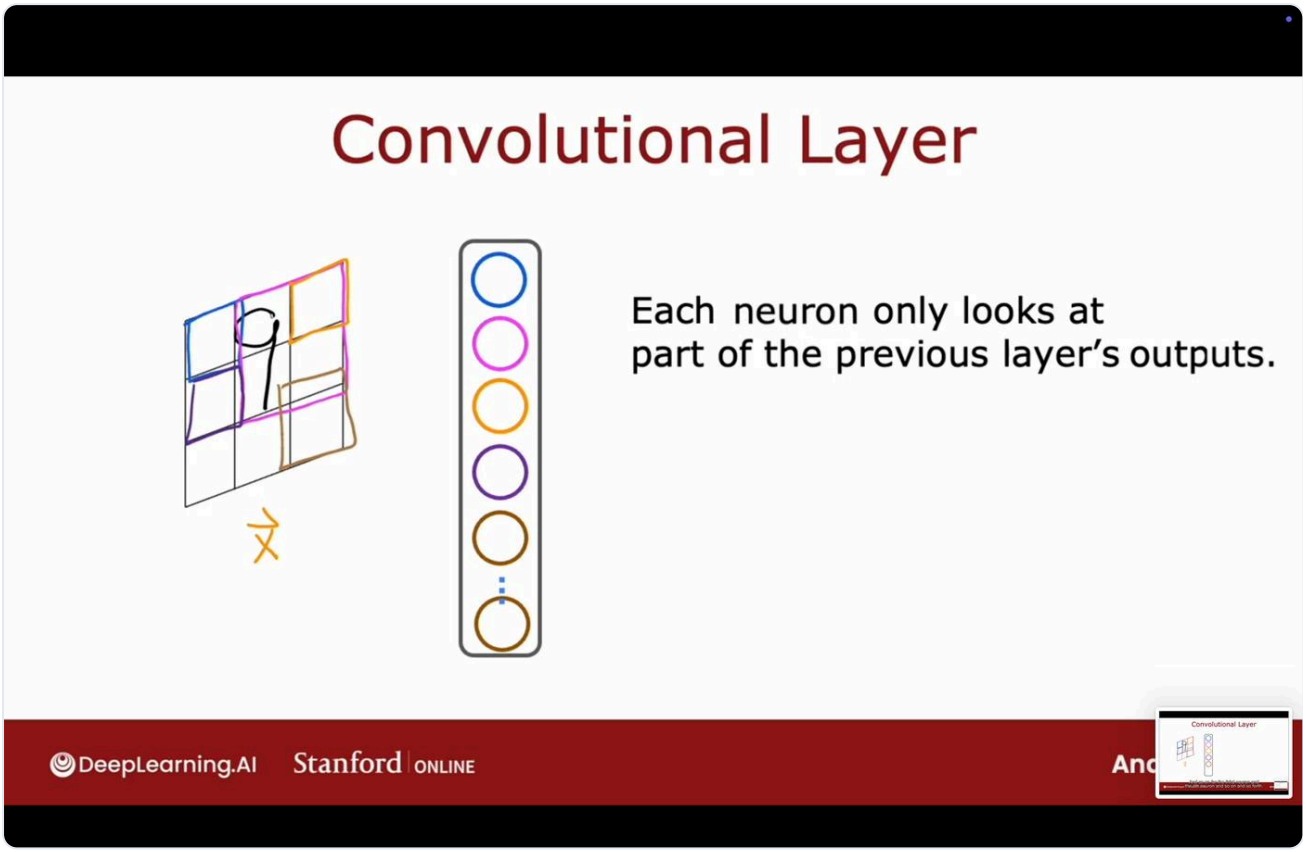


FIGURE 29.4 | Convolutional Layer

So why might you want to do this?

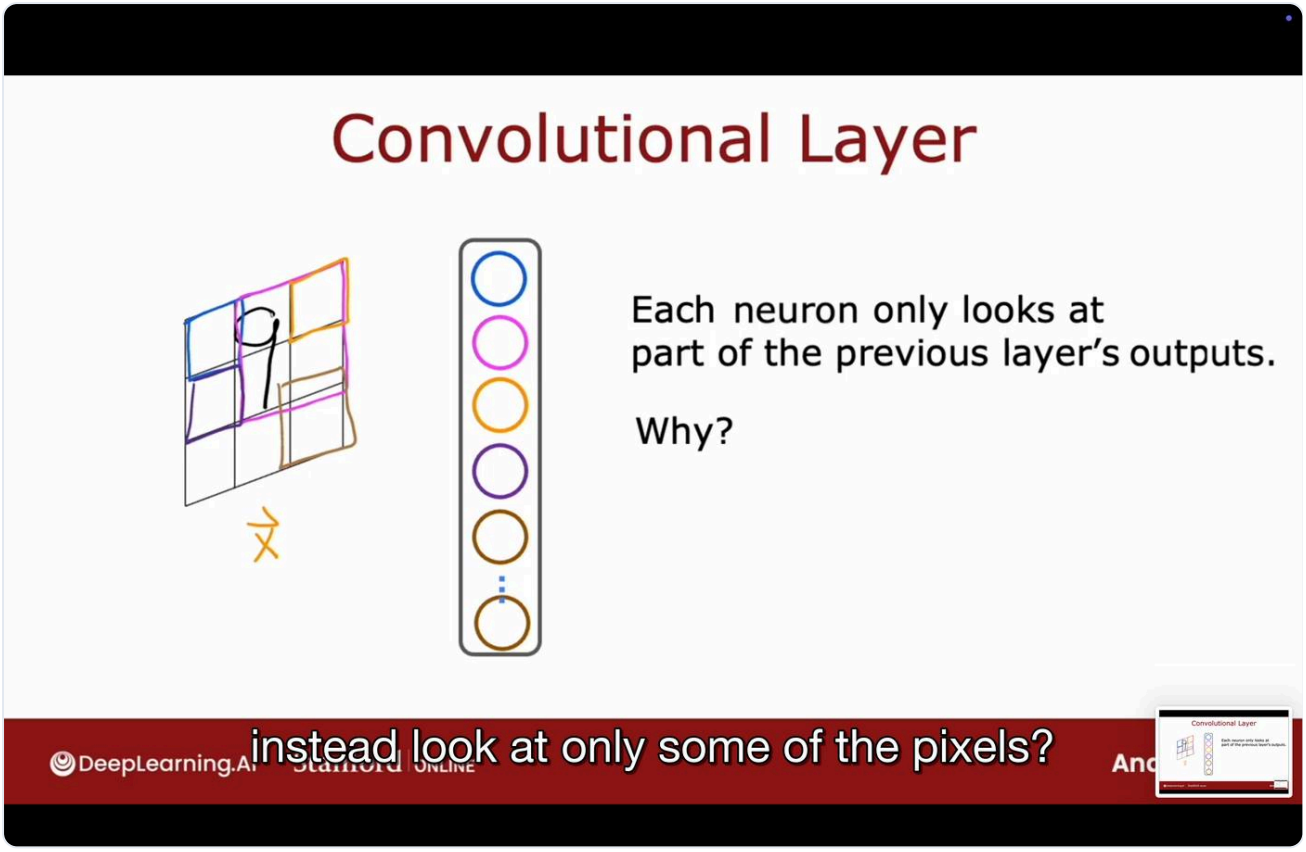


FIGURE 29.5 | Convolutional Layer

Why won't you let every neuron look at all the pixels but instead look at only some of the pixels? Well, some of the benefits are first, it speeds up computation.

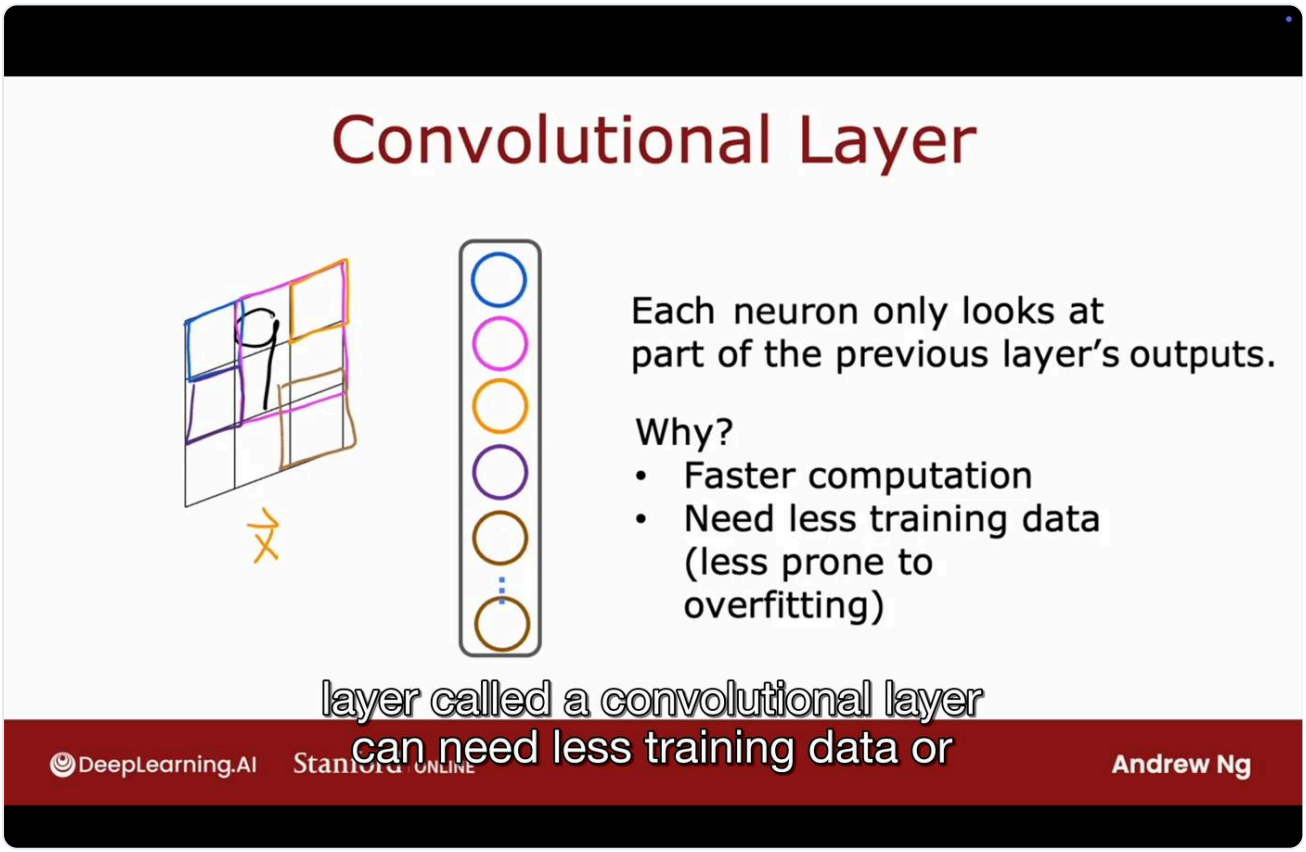


FIGURE 29.6 | Convolutional Layer

And second advantage is that a neural network that uses this type of layer called a convolutional layer can need less training data or alternatively, it can also be less prone to overfitting. You heard me talk a bit about overfitting in their previous course but this is something that will dive into greater detail on next week as well. When we talk about practical tips for using learning algorithms and this is the type of layer where each neuron only looks at a region of the input image is called a convolutional layer. It was a researcher John Macoun who had figured out a lot of the details of how to get convolutional layers to work and popularized their use.

Let me illustrate in more detail a convolutional layer. And if you have multiple convolutional layers in a neural network sometimes that's called a convolutional neural network. To illustrate the convolutional layer of convolutional neural network on this slide I'm going to use instead of a two D image input. I'm going to use a one dimensional input and the motivating example I'm going to use is classification of E K G signals or electrocardiograms. So if you put two electrodes on your chest you will record the voltages that look like this that correspond to your heartbeat. This is actually something that my stanford research group did research on.

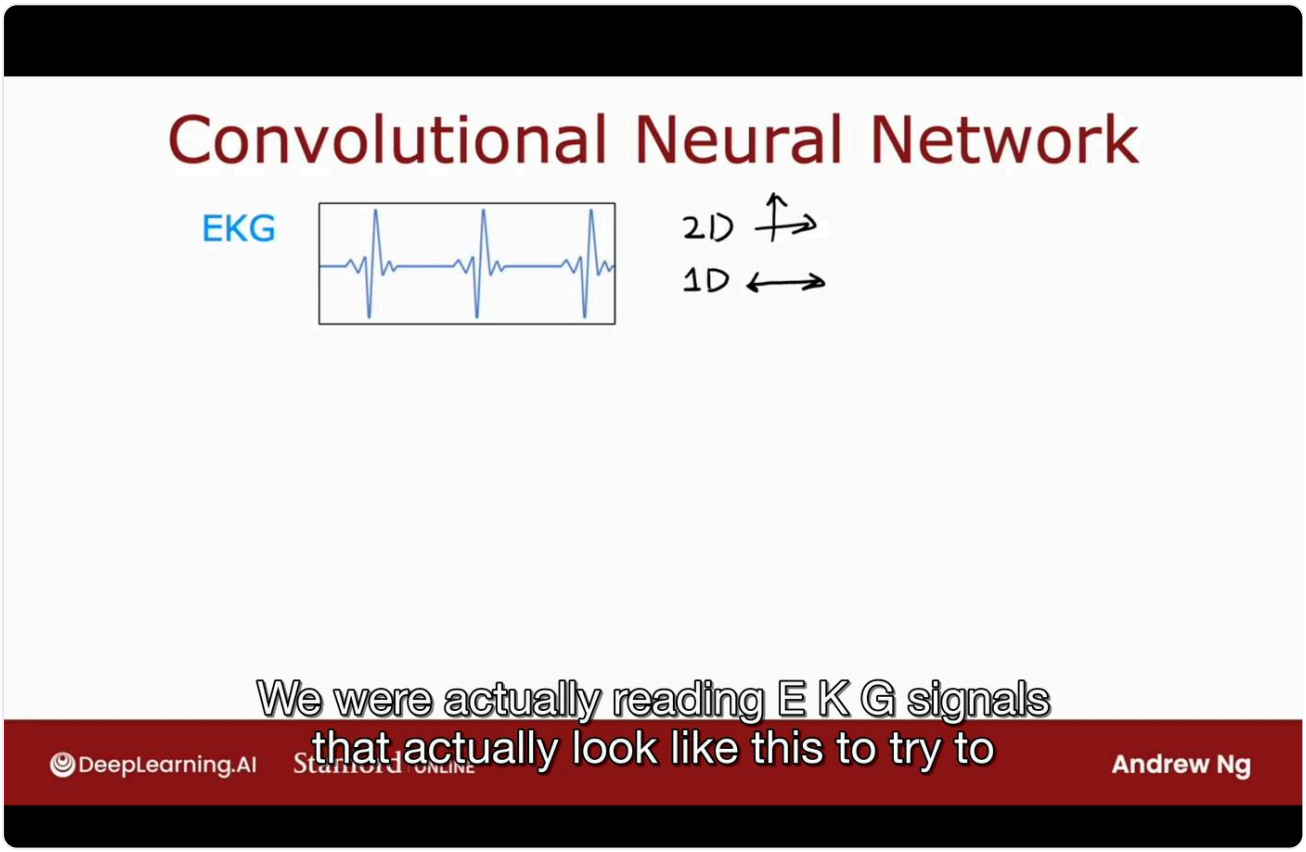


FIGURE 29.7 | Convolutional Neural Network

We were actually reading E K G signals that actually look like this to try to diagnose if patients may have a heart issue. So an E K G signal and electrocardiogram E C G in some places E K G. In some places there's just a list of numbers corresponding to the height of the surface at different points in time.

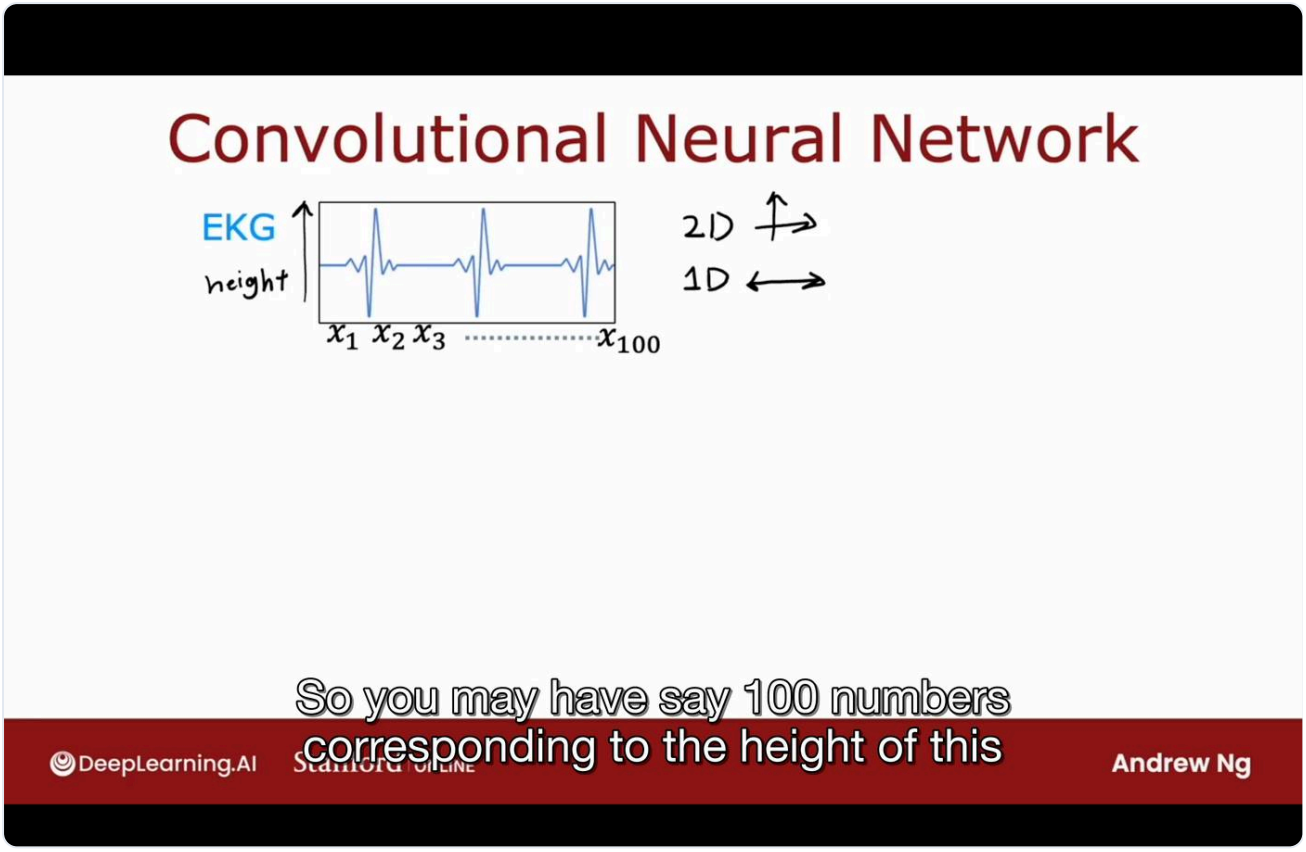


FIGURE 29.8 | Convolutional Neural Network

So you may have say 100 numbers corresponding to the height of this curve at 100 different points of time. And the learning tosses given this time series, given this E K G signal to classify say whether this patient has a heart disease or some diagnosable heart conditions. Here's what the convolutional neural network might do. So I'm going to take the E K G signal and rotated 90 degrees to lay it on the side.

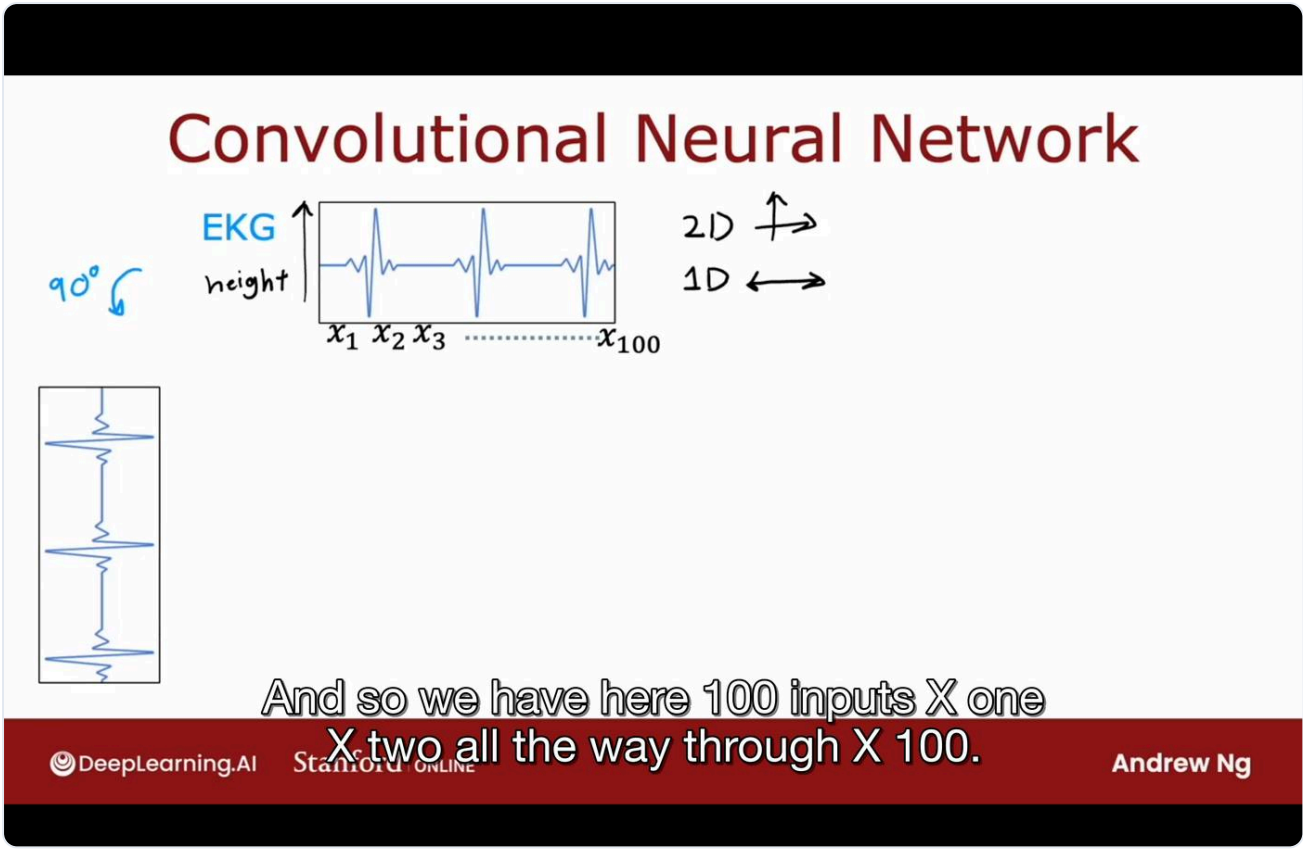


FIGURE 29.9 | Convolutional Neural Network

And so we have here 100 inputs X one X two all the way through X 100. Like.

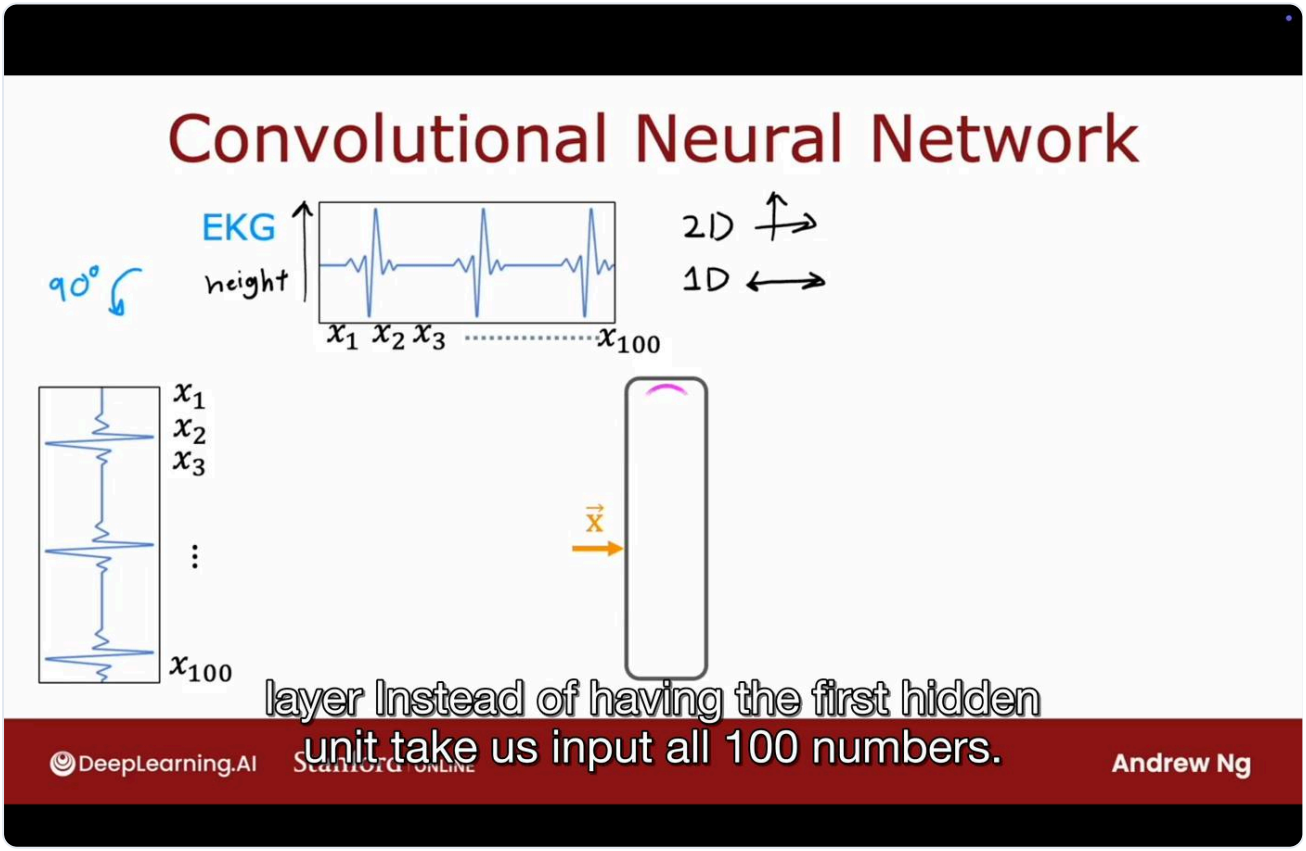


FIGURE 29.10 | Convolutional Neural Network

So and when I construct the first hidden layer Instead of having the first hidden unit take us input all 100 numbers. Let me have the first hidden unit. Look at only X one through X 20. So that corresponds to looking at just a small window of this E K G signal. The second hidden unit shown in a different color here. Well look at X 11 through X 30, so looks at a different window in this E K G signal. And the third hidden there looks at another window X21 through X 40 and so on. And the final hidden units in this example.

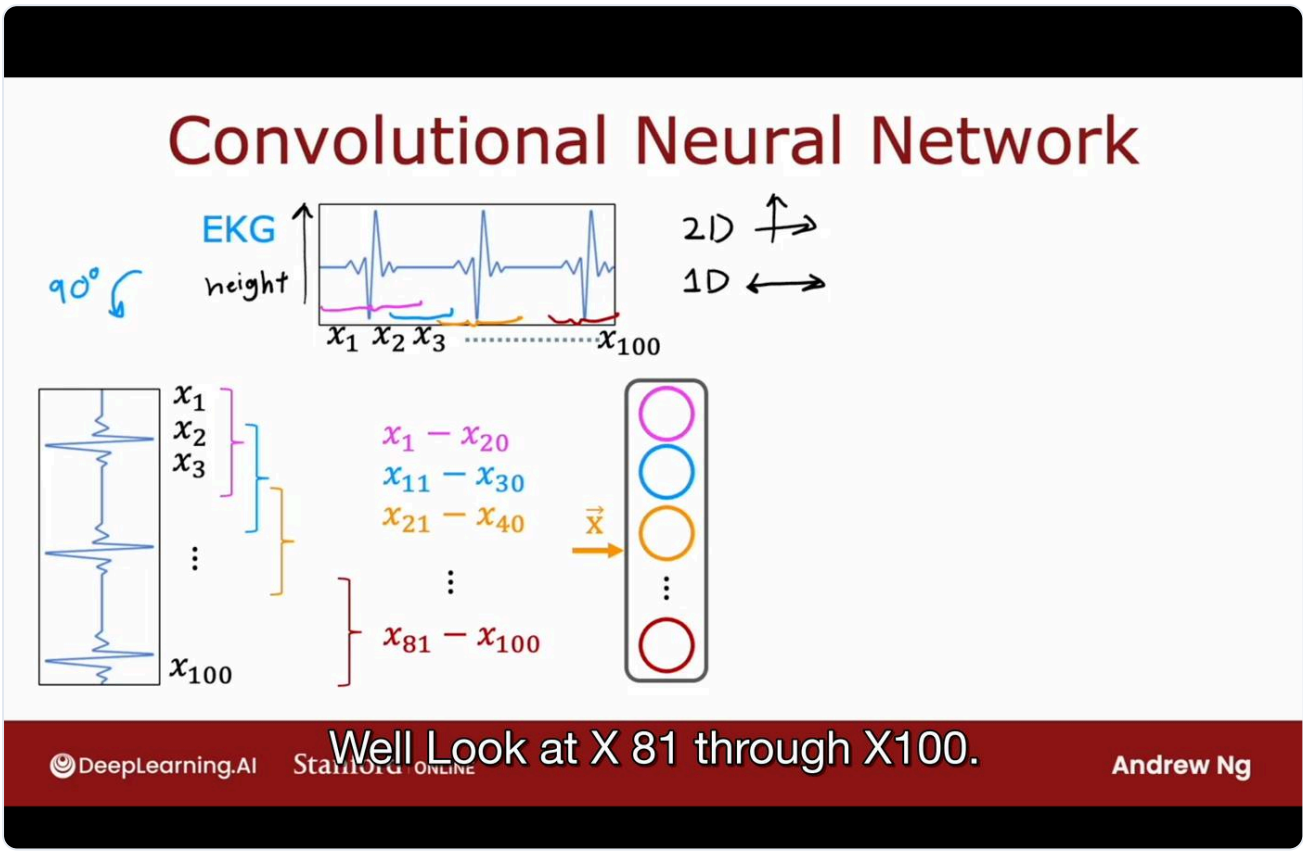


FIGURE 29.11 | Convolutional Neural Network

Well Look at X 81 through X100. So it looks like a small window towards the end of this EKG time series.

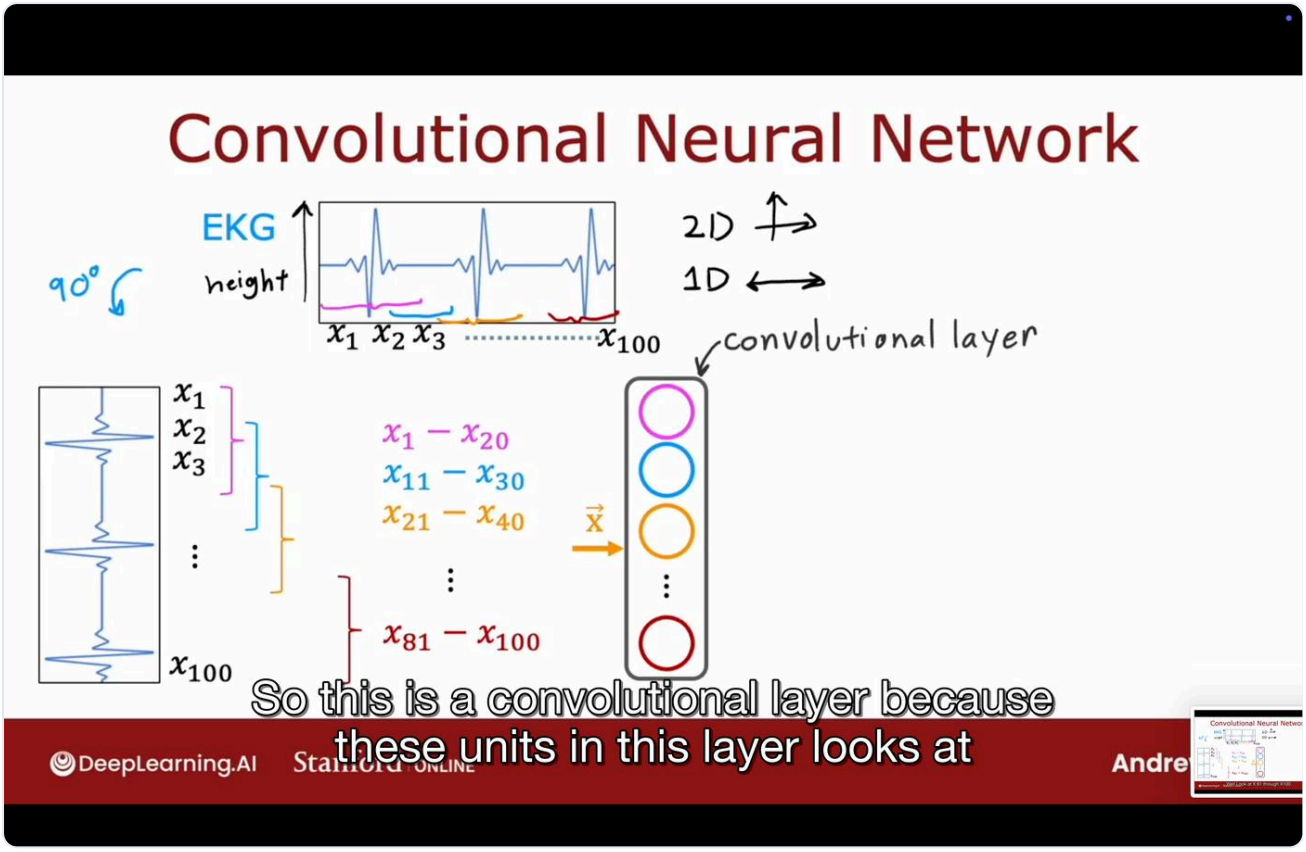


FIGURE 29.12 | Convolutional Neural Network

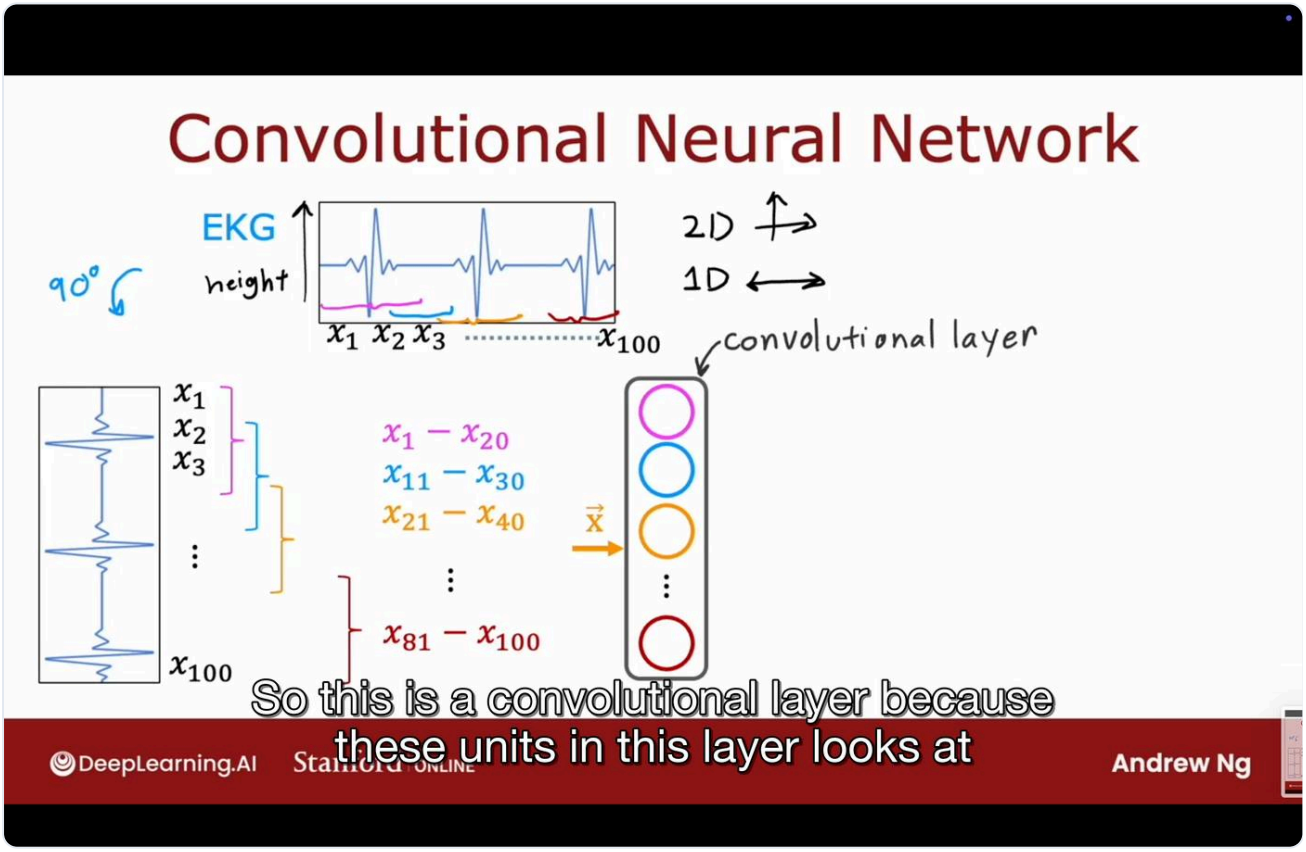


FIGURE 29.13 | Convolutional Neural Network

So this is a convolutional layer because these units in this layer looks at only a limited window of the input. Now this layer of the neural network has nine units. The next layer can also be a convolutional layer. So in the second hidden layer let me architect my first unit not to look at all nine activations from the previous layer, but to look at say just the first 5 activations from the previous layer. And then my second unit In this second hidden there may look at just another five numbers, say A3-A7.

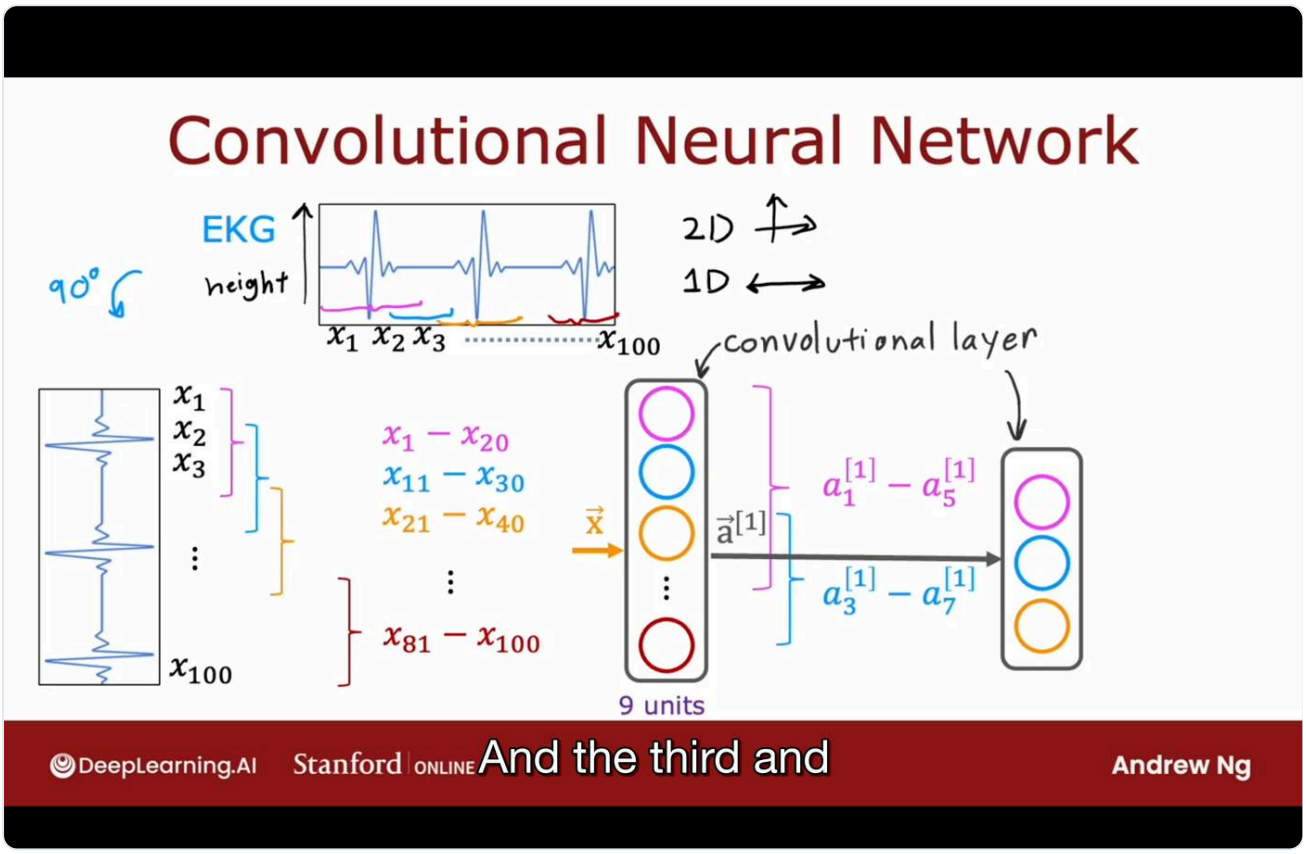


FIGURE 29.14 | Convolutional Neural Network

And the third and final hidden unit in this layer will only look at A5 through A9.

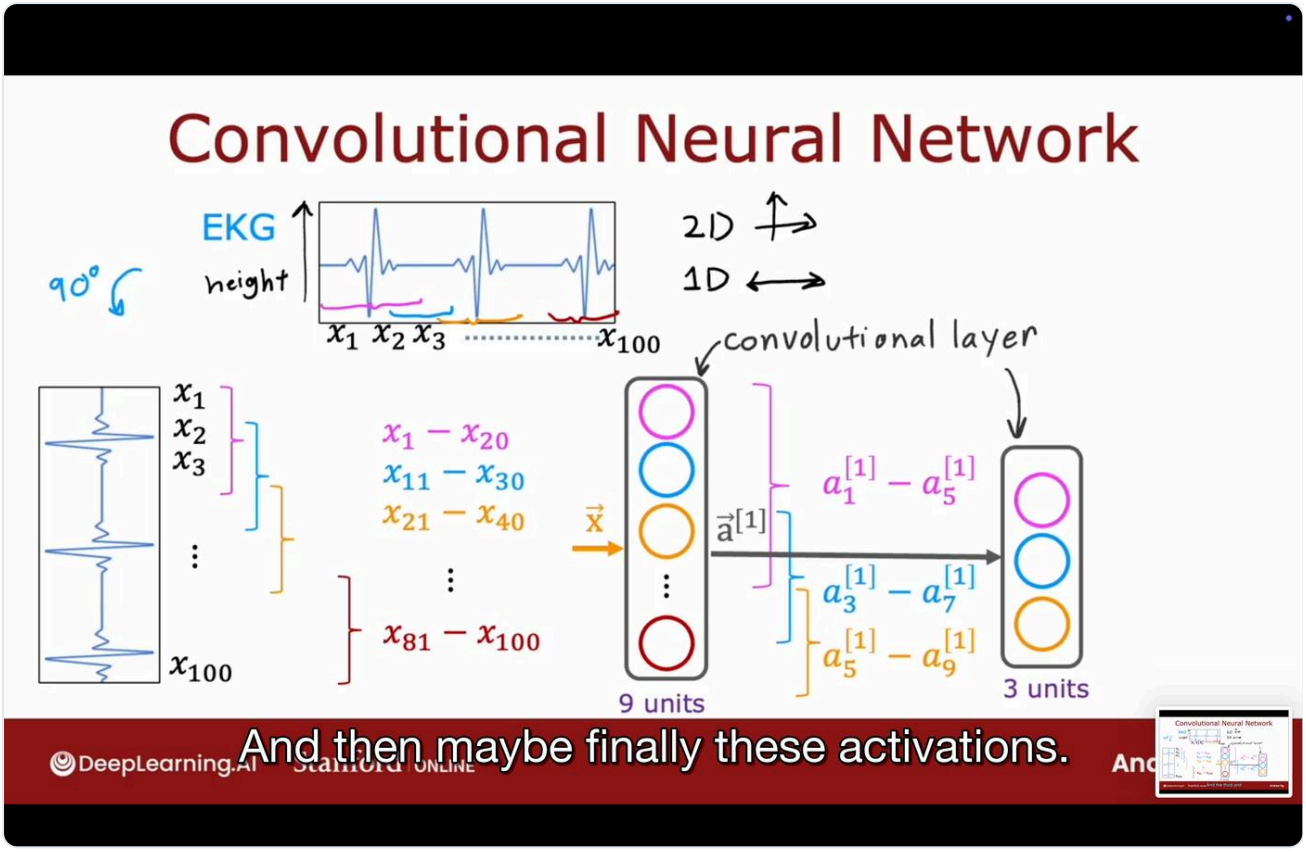


FIGURE 29.15 | Convolutional Neural Network

And then maybe finally these activations.

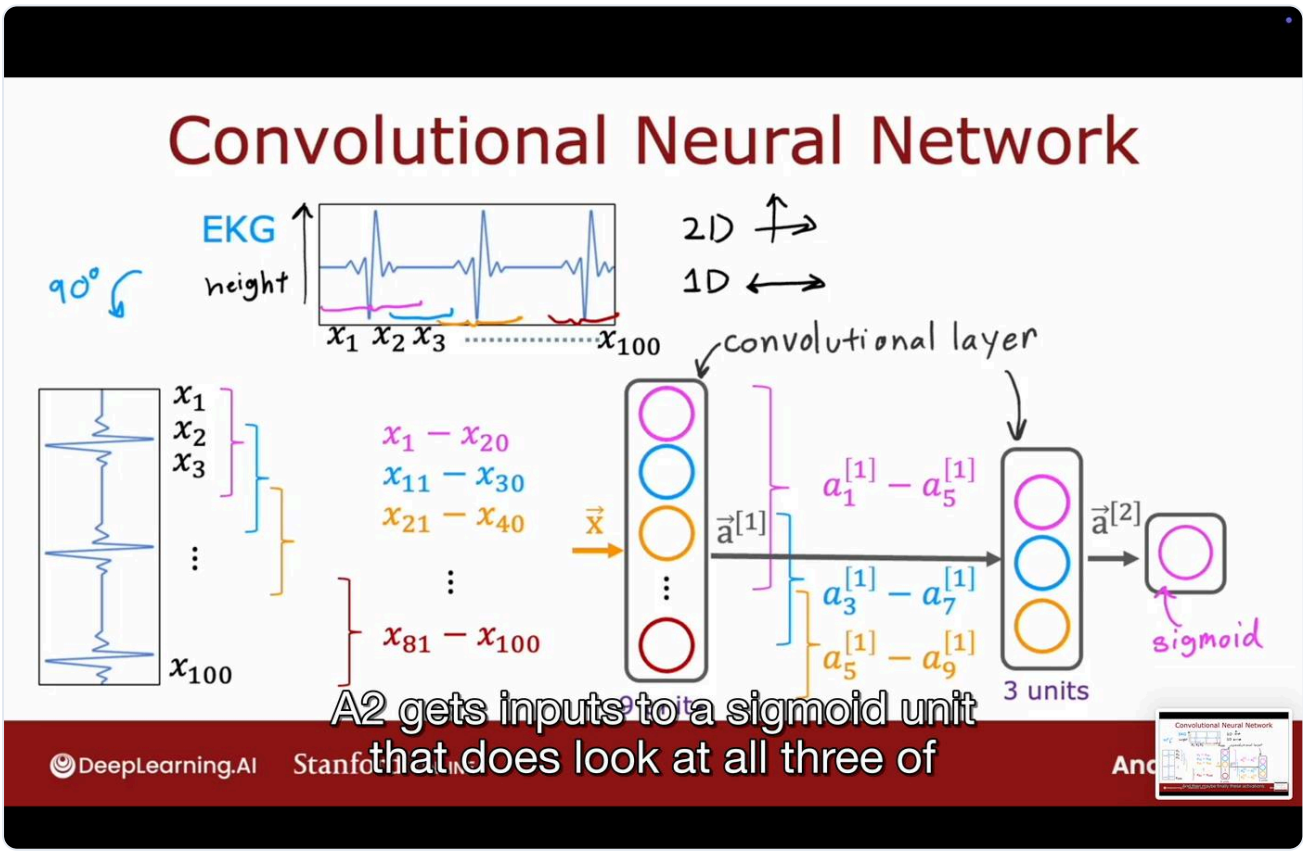


FIGURE 29.16 | Convolutional Neural Network

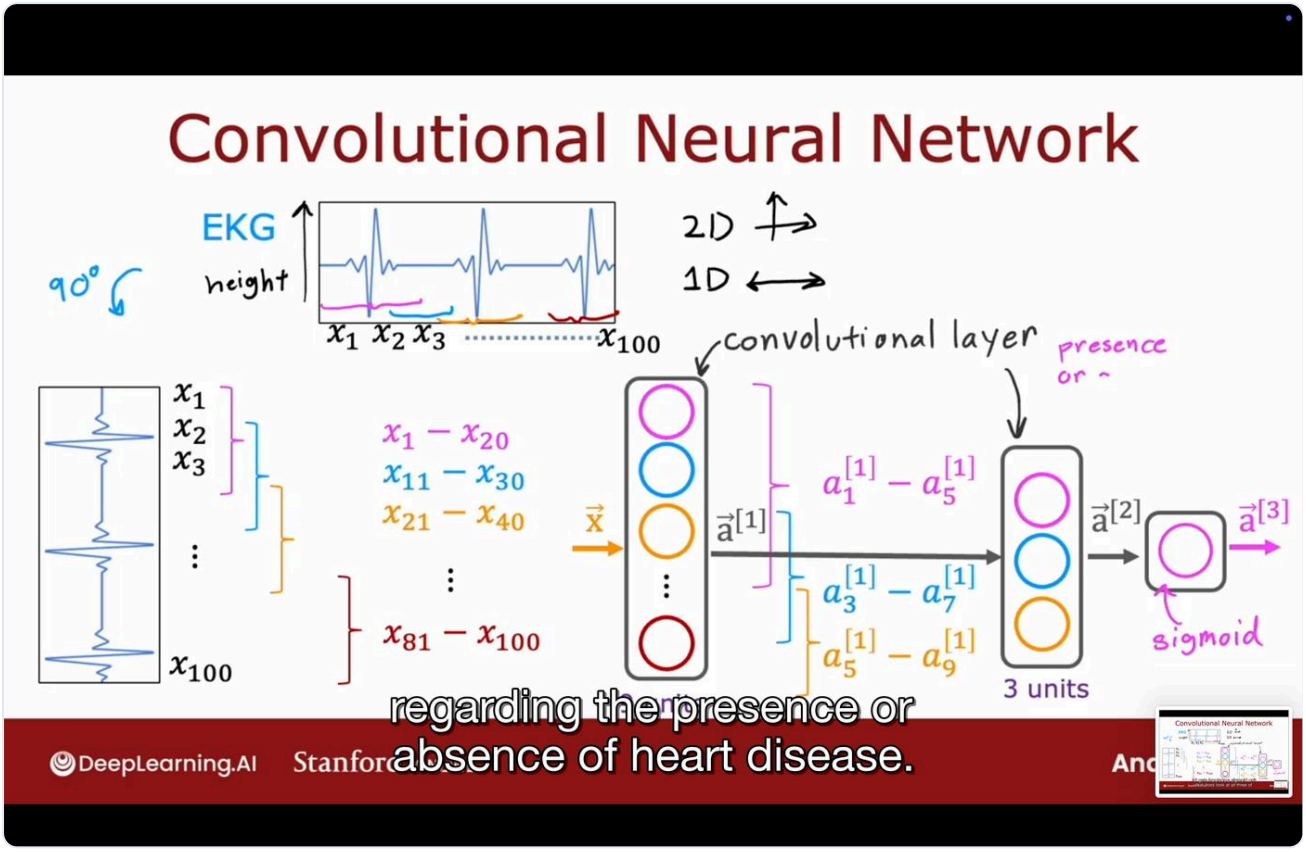


FIGURE 29.17 | Convolutional Neural Network

A2 gets inputs to a sigmoid unit that does look at all three of these values of A2 in order to make a binary classification regarding the presence or absence of heart disease.

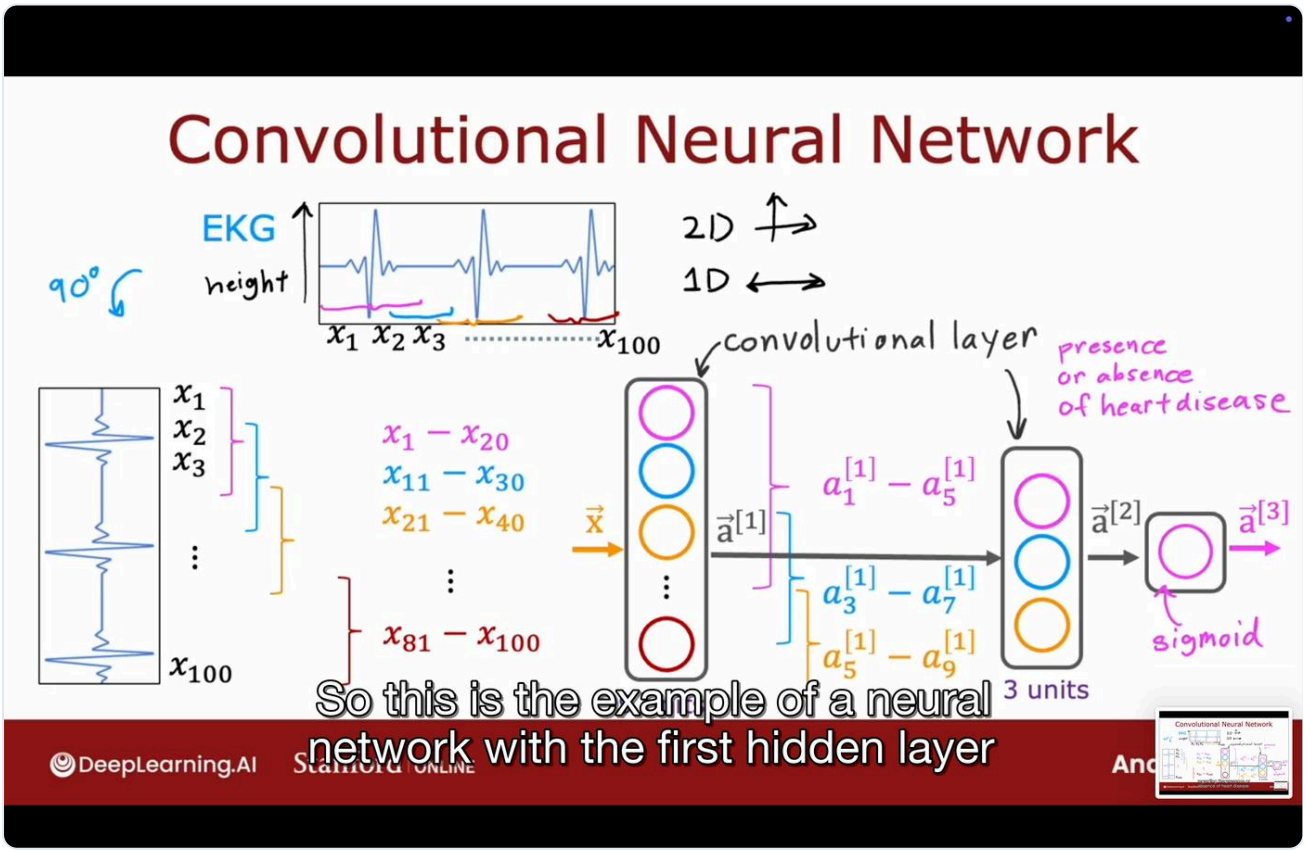


FIGURE 29.18 | Convolutional Neural Network

So this is the example of a neural network with the first hidden layer being a convolutional layer. The second hidden layer also being a convolutional layer and then the output layer being a sigmoid layer. And it turns out that with convolutional layers you have many architecture choices such as how big is the window of inputs that a single neuron should look at and how many neurons should layer have. And by choosing those architectural parameters effectively, you can build new versions of neural networks that can be even more effective than the dense layer for some applications. To recap, that's it for the convolutional layer and convolutional neural networks.

I'm not going to go deeper into convolutional networks in this class and you don't need to know anything about them to do the homework and finish this class successfully. But I hope that you find this additional intuition that neural networks can have other types of layers as well to be useful. And in fact, if you sometimes hear about the latest cutting edge architectures like a transformer model or an LS TM or an attention model. A lot of this research in neural networks even today pertains to researchers trying to invent new types of layers for neural networks. And plugging these different types of layers together as building blocks to form even more complex and hopefully more powerful neural networks.

So that's it for the required videos for this week. Thank you and congrats on sticking with me all the way through this. And I look forward to seeing you next week also where we'll start to talk about practical advice for how you can build machine learning systems. I hope that the tips you learn next week will help you become much more effective at building useful machine learning systems. So I look forward also to see you next week.

LECTURE 30

More Derivative Examples

You've seen how in TensorFlow you can specify a neural network architecture to compute the output y as a function of the input x, and also specify a cost function, and TensorFlow will then automatically use back propagation to compute derivatives and use gradient descent or Adam to train the parameters of a neural network. The backpropagation algorithm, which computes derivatives of your cost function with respect to the parameters, is a key algorithm in neural network learning. But how does it actually work? In this and in the next few optional videos, we'll try to take a look at how backpropagation computes derivatives. These videos are completely optional and they do go just a little bit into calculus.

If you're already familiar with calculus I hope you enjoy these videos, but if not, it's totally fine. We'll build up from the very basics of calculus to try to make sure you have all the intuition you need to understand how backpropagation works. Let's take a look. I'm going to use a simplified cost function, J of w equals w squared. The cost function is a function of the parameters w and say, b and for this simplified cost function let's just pretend J of w equals w squared. I'm going to ignore b for this example. Let's say the value of the parameter w is equal to 3. J of w will be equal to 9, w squared of 3 squared. Now if we were to increase w by a tiny amount, say Epsilon, which I'm going to set to 0.001.

How does the value of J of w change? If we increase w by 0.001 then w becomes 3 plus 0.001, so it's 3.001. J of w, which is w squared, which we defined above, is now this 3.001 squared, which is 9.006001. What we see is that if w goes up by 0.001, I'm going to use this up arrow here to denote w goes up by 0.001, where 0.001 is this small value Epsilon. Then J of w roughly goes up by 6 times as much, 6 times 0.001. This isn't quite exactly. It actually goes up not to 9.006 but 9.006001. But it turns out that if Epsilon where infinitesimally small, and by infinitesimally small I mean very small. Epsilon is pretty small, but it's not infinitesimally small.

If Epsilon was 0.00000, lots of zeros followed by one, then this becomes more and more accurate. In this example what we see is that if w goes up by Epsilon then J goes up roughly by 6 times Epsilon. In calculus what we would say is that the derivative of J of w with respect to w is equal to 6. All this means is if w goes up by a tiny little amount J of w goes up six times as much. What if Epsilon were to take on different value? What if Epsilon were 0.002. In this case w would be 3 plus 0.002, and w squared becomes 3.002 squared, which is 9.012004. In this case what we conclude is that if w goes up by 0.002 then J of w goes up by roughly 6 times 0.002.

It goes up roughly to 9.012, and this 0.012 is roughly 6 times 0.002. That again, is a little bit off. This is extra 0.00004 here, because Epsilon is not quite infinitesimally small. Once again, we see this six to one ratio between how much w goes up versus how much J of w goes up. That's why the derivative of J of w with respect to w is equal to six. A small Epsilon is the more accurate this becomes. By the way, feel free to pause the video and try this calculation now yourself with other

values of Epsilon. The key is that so long as Epsilon is pretty small the ratio by which J of w goes up versus the amount by which w goes up should be 6-1. Feel free to try it out yourself with other values of Epsilon, then check if this really holds true.

This leads us to an informal definition of the derivative, which is that whenever w goes up by a tiny amount Epsilon that causes J of w to go up by k times Epsilon. In our example just now k was equal to six. Then we say that the derivative of J of w with respect to w is equal to k, which was equal to 6 in the example just now. You might remember when implementing gradient descent you will repeatedly use this rule to update the parameter w J, where as usual Alpha is the learning rate. What is gradient descent do? Notice that if the derivative is small, then this updates that will make a small update to the parameter W_j, whereas if this derivative term is large, this will result in a big change to the parameter W_j.

This makes sense because this is essentially saying that if the derivative is small, this means that changing w doesn't make a big difference to the value of j and so let's not bother to make a huge change to W_j. But if the derivative is large, that means that even a tiny change the W_j can make a big difference in how much you can change or decrease the cost function j of w. In that case, let's make a bigger change to W_j, because doing so will actually make a big difference to how much we can reduce the cost function J. Let's take a look at a few more examples of derivatives. What you saw in the example just now was that if w equals 3, and j of w equals w squared equals 9, then if w goes up by Epsilon by 0.01, then j of w becomes j of 3.01 is now 9.006001.

Or in other words, j has gone up by about 0.006, which is 6 times 0.001 or 6 times Epsilon, which is why the derivative of w with respect to W is equal to 6. Let's look at what the derivative will be for other values of w, take w equals 2. In this case, j of w is, w squared is now equal to 4, and if w goes up by 0.001, then J of w becomes j of 2.001, which is equal to this 4.004001 and so j of w has gone up from four to this value over here, which is roughly four times Epsilon bigger than four, which is why now the derivative is four. Because w going up by Epsilon has caused j of w to go up four times as much.

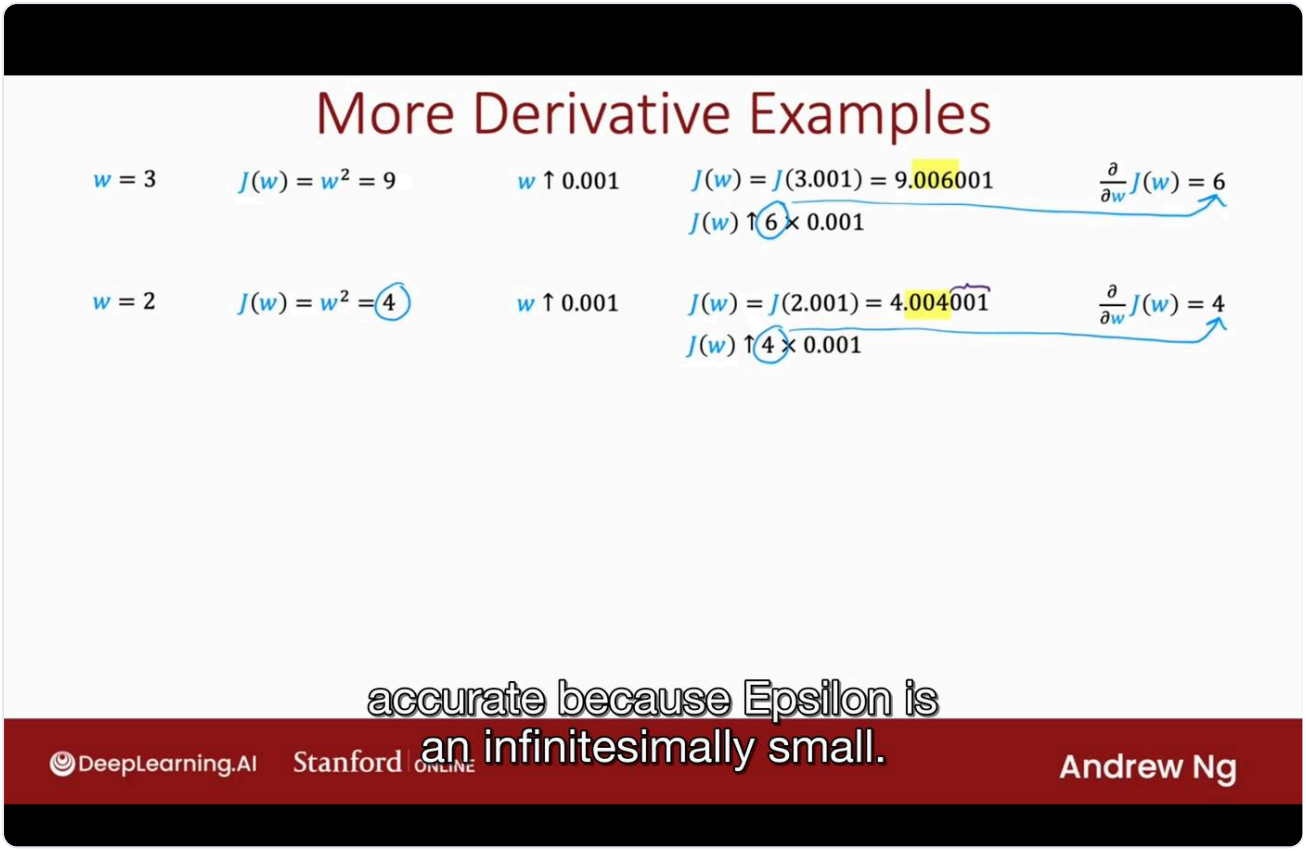


FIGURE 30.1 | More Derivative Examples

Again, there's extra 0.001 is because it's not quite accurate because Epsilon is an infinitesimally small. Or let's look at another example. What if w were equal to negative 3? J of w which is w squared, is still equal to 9 because negative 3 squared is 9. If w was to go up by Epsilon again, then you now have w equals negative 2.999, so that's j of negative 2.999. The square of negative 2.999 is equal to this 8.994001, because w is negative 3 plus 0.001. Notice here, j of w has gone down by about 0.006, which is six times Epsilon. What we have in this example is that j starts off as 9, but it has now gone down. Notice this down arrow here [inaudible] arrow by 6 times Epsilon or equivalently it has gone up by negative 6 times Epsilon.

That's why the derivative in this case is equal to negative 6.

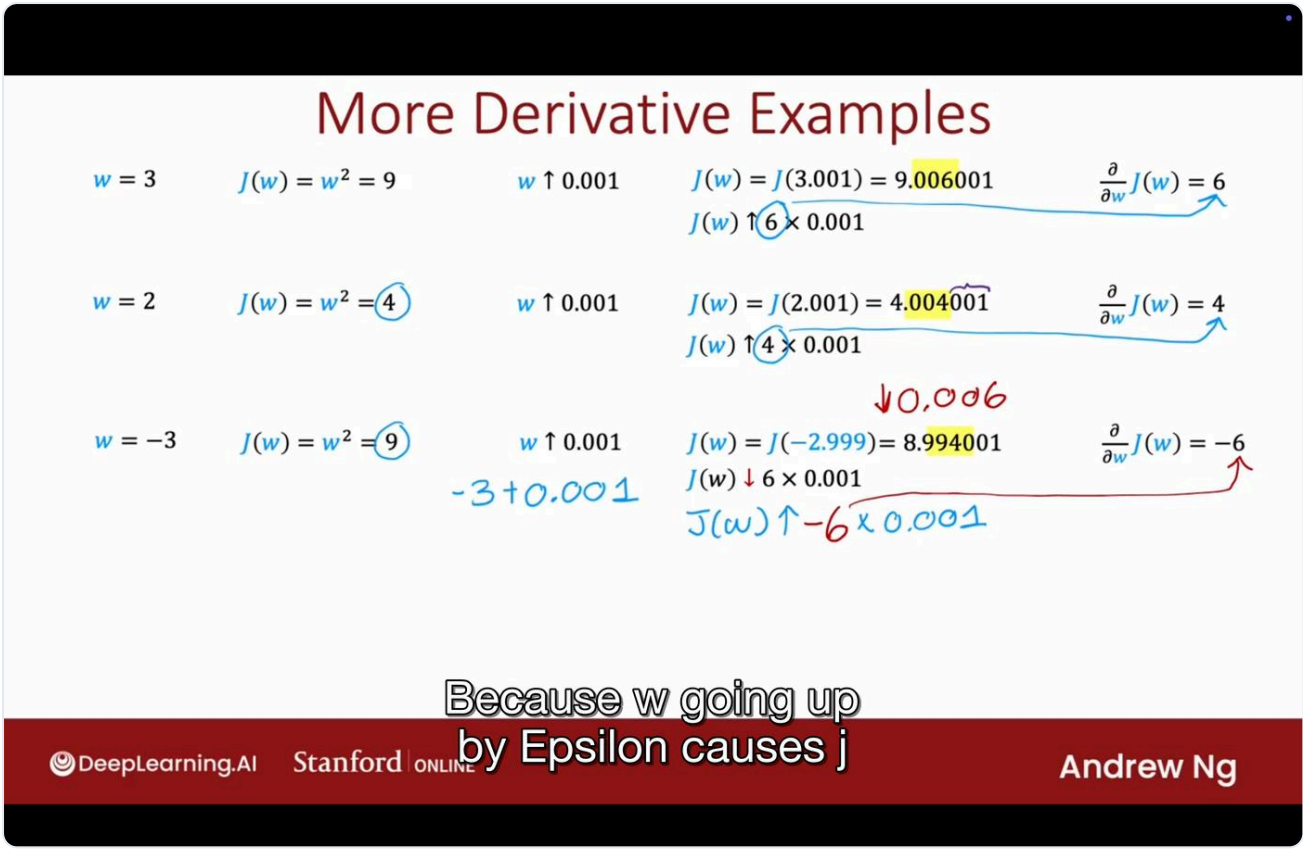


FIGURE 30.2 | More Derivative Examples

Because w going up by Epsilon causes j of w to go up by negative 6 times Epsilon when Epsilon is small. Another way to visualize this is to plot the function J of w, so that the horizontal axis is w and this is J of w, then when w is equal to 3, J of w is equal to 9. When it's negative 3 it's also equal to 9, and when it is 2, J of w is equal to 4.

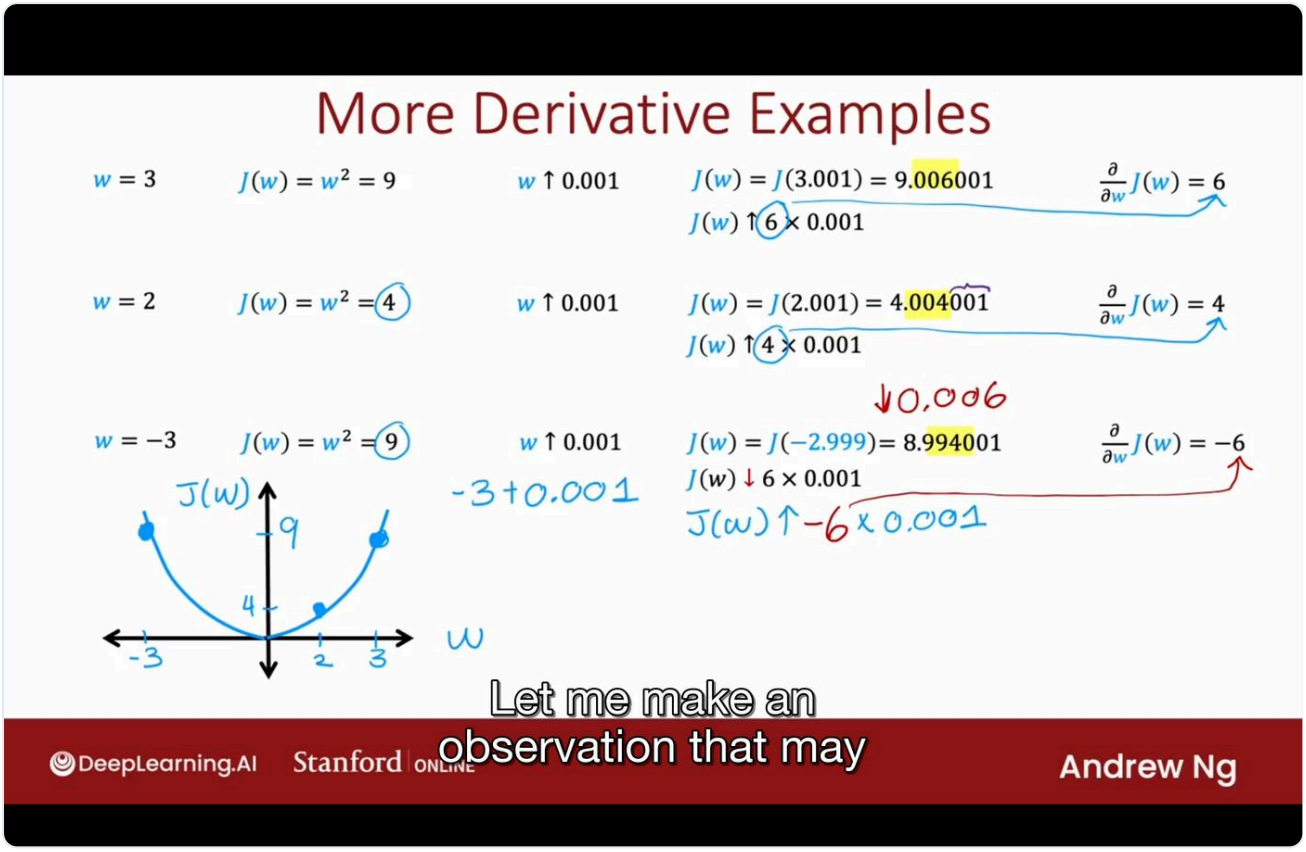


FIGURE 30.3 | More Derivative Examples

Let me make an observation that may be relevant if you've taken a calculus class before. But if you haven't, what I say in the next 60 seconds may not make sense, but don't worry about it. You will need to understand it to fully follow the rest of these videos. If you've taken a class in calculus at some point, you may recognize that the derivatives corresponds to the slope of a line that just touches the function J of w at this point, say where w equals 3. The slope of this line at this point, and the slope is this height over this width turns out to be equal to 6 when w equals 3, the slope of this line turns out to be 4 when w equals 2 and the slope of this line turns out to be negative 6 when w equals negative 3.

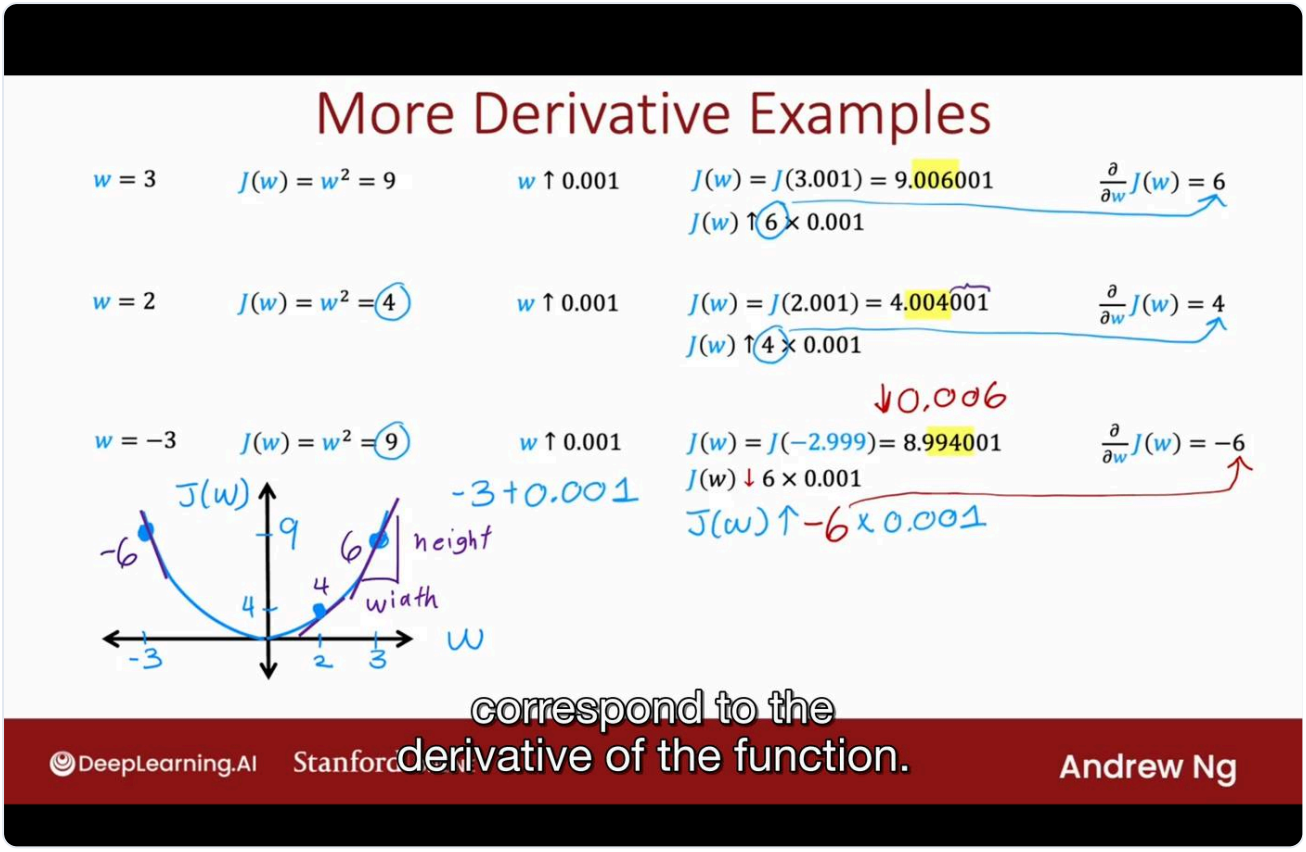


FIGURE 30.4 | More Derivative Examples

It turns out in calculus, the slope of these lines correspond to the derivative of the function. But if you haven't taken a calculus class before and haven't seen this slope concept before, don't worry about it. Now, there's one last observation I want to make before moving on, which is that you see in all three of these examples, J of w is the same function, J of w is equal to w squared. But the derivative of J of w depends on w, when w is three, the derivative is six. When w is two, the derivative is four. When w is negative 3, the derivative is negative 6. It turns out that if you are familiar with calculus, and again, it's totally fine if you're not, calculus can allow us to calculate the derivative of J of w in respect to w as 2 times w.

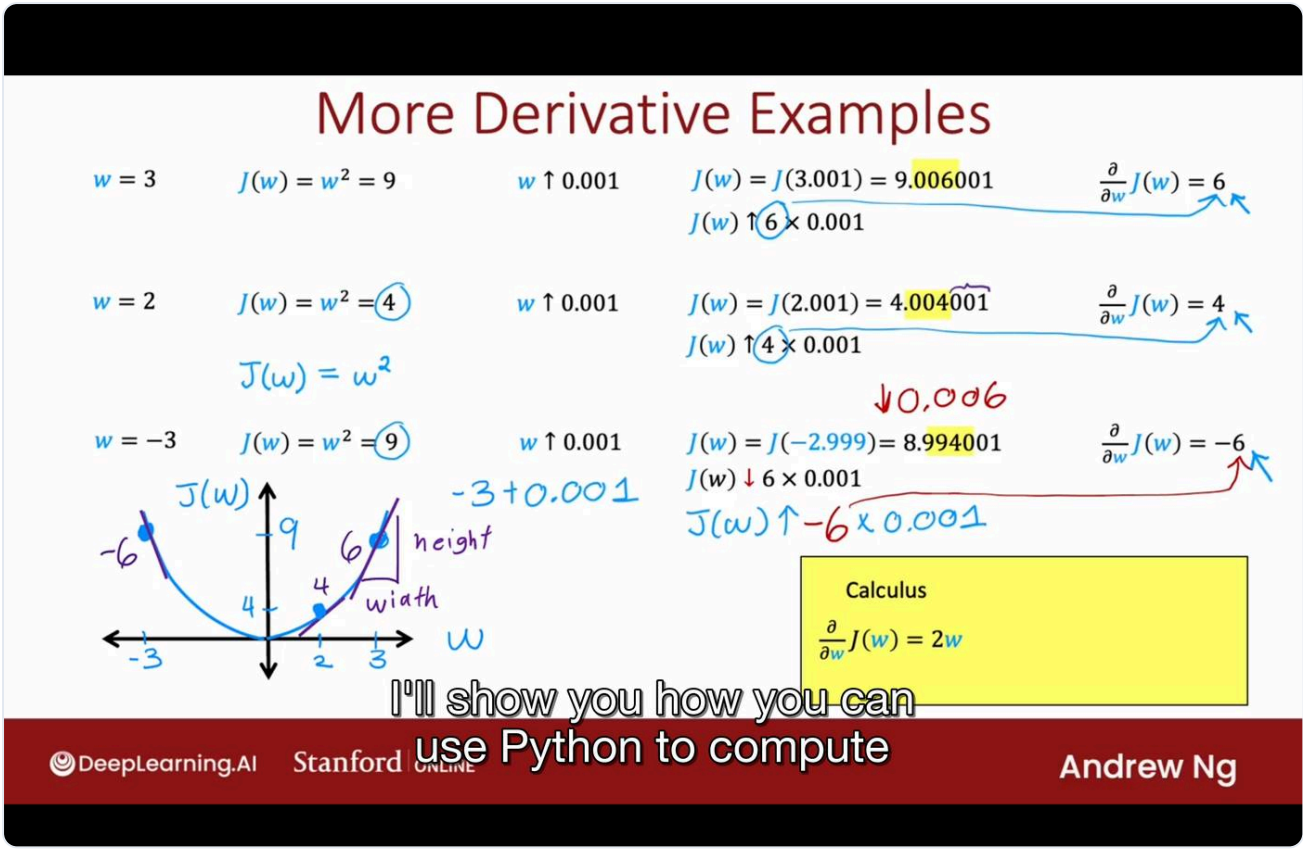


FIGURE 30.5 | More Derivative Examples

In a little bit, I'll show you how you can use Python to compute these derivatives yourself using a nifty Python package called SymPy.

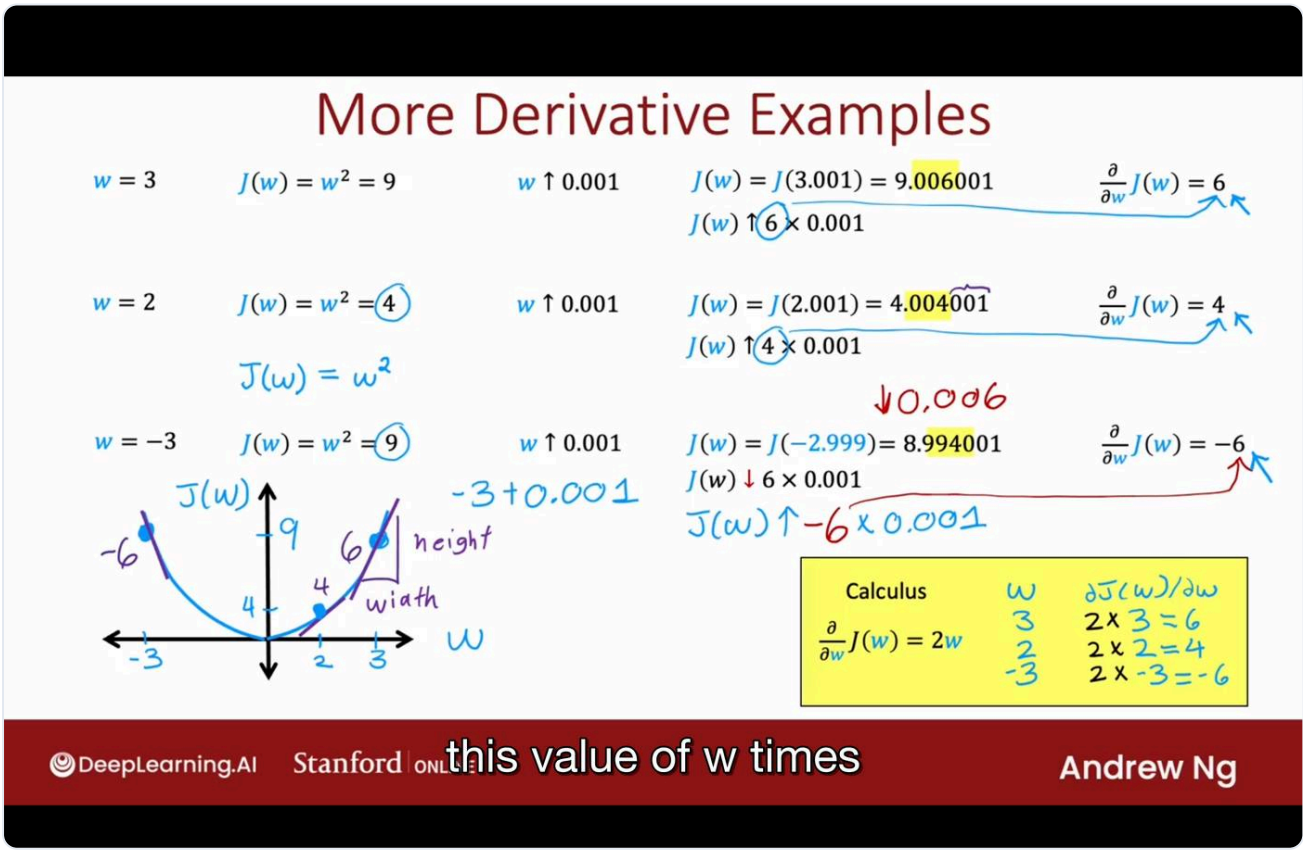


FIGURE 30.6 | More Derivative Examples

But because calculus tells us that the derivative of w squared J of w is 2w, that's why the derivative when w is three is 2 times 3 or when is two is 2 times 2, or when is negative 3 is 2 times negative 3 because this value of w times 2 turns out to give you the derivative. Let's go through just a few more examples before we wrap up. For these examples, I'm going to set w equals 2. You saw on the last slide, if J of w is w squared, then the derivative I said would be 2 times w, which was 4. If w goes up by 0.01, this being Epsilon, J of w becomes this so roughly J of w goes up by 4 times Epsilon. Let's look at a few other functions. What if J of w is equal to w cubed?

In this case, w cubed, 2 cubed would be equal to 8, or what if J of w is just equal to w? Here, w will be equal to 2. Or what if J of w was 1 over w? In this case, 1 over w, 1 over 2 would be 1/2 or 0.5. What is the derivative of J of w with respect to w when the cost function J of w is either w cubed or w or 1 over w. Let me show you how you can compute these derivatives yourself using a library and package called SymPy. Let me first import SymPy. What I'm going to do is tell SymPy that I'm going to use J and w as symbols for computing derivatives. For our first example, we had the cost function J was equal to w squared. Notice how SymPy actually renders it in this nifty font here as well.

If we were to use SymPy to take the derivative of J with respect to w, we should do as follows. You see that SymPy tells you this derivative is 2w. Let me actually choose a variable, dJ, dw, we set that to be equal to this, just type it again here. Print it out.

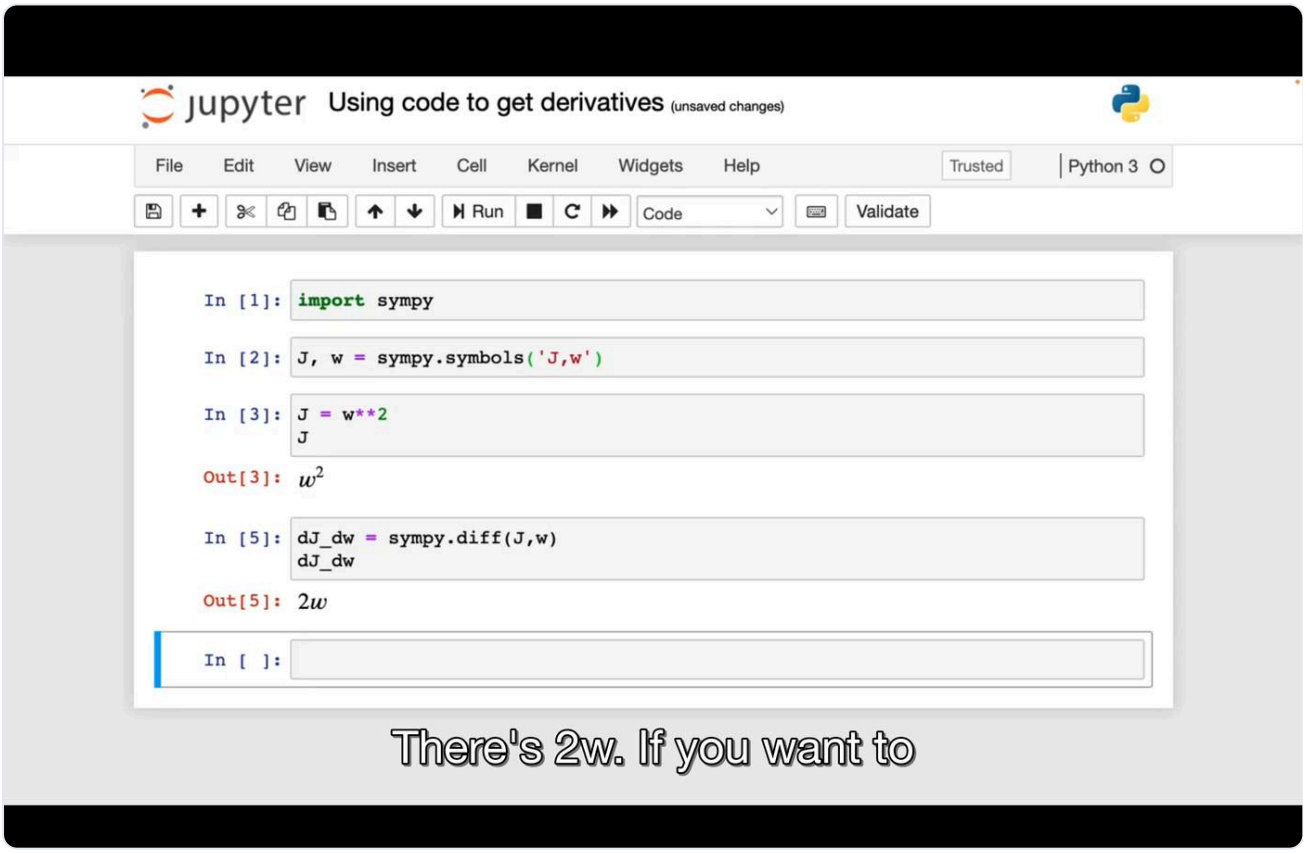


FIGURE 30.7 | In Import Sympy

There's 2w. If you want to plug in the value of w into this expression to evaluate it, you can do the derivative.subs w, 2. This means plug-in a value of w to be equal to 2 into this expression and evaluate it. That gives you the value of four, which is why when w equals to 2, we saw the deriva-

tive of J was equal to 4. Let's look at some other examples. What if J was w cubed? Then the derivative becomes 3 times w squared. It turns out from calculus, and this is what SymPy is calculating for us, if J is w cubed, then the derivative of J with respect to w is 3w squared. Depending on what w is, the value of the derivative changes as well. We can plug in if w equal to 2, you get 12 in this case.

Or what if it was J equal to w? In this case, the derivative is just equal to 1. Or the final example we have was what if J equals 1 over w? In this case, the derivative turns out to be negative 1 over w squared. This is negative 1 over 4. What I'm going to do is take the derivatives we have worked out. Remember for w squared, it was 2w, for w cubed, it was 3w squared. For w is just 1 and 1 over w it's negative 1 over w squared.

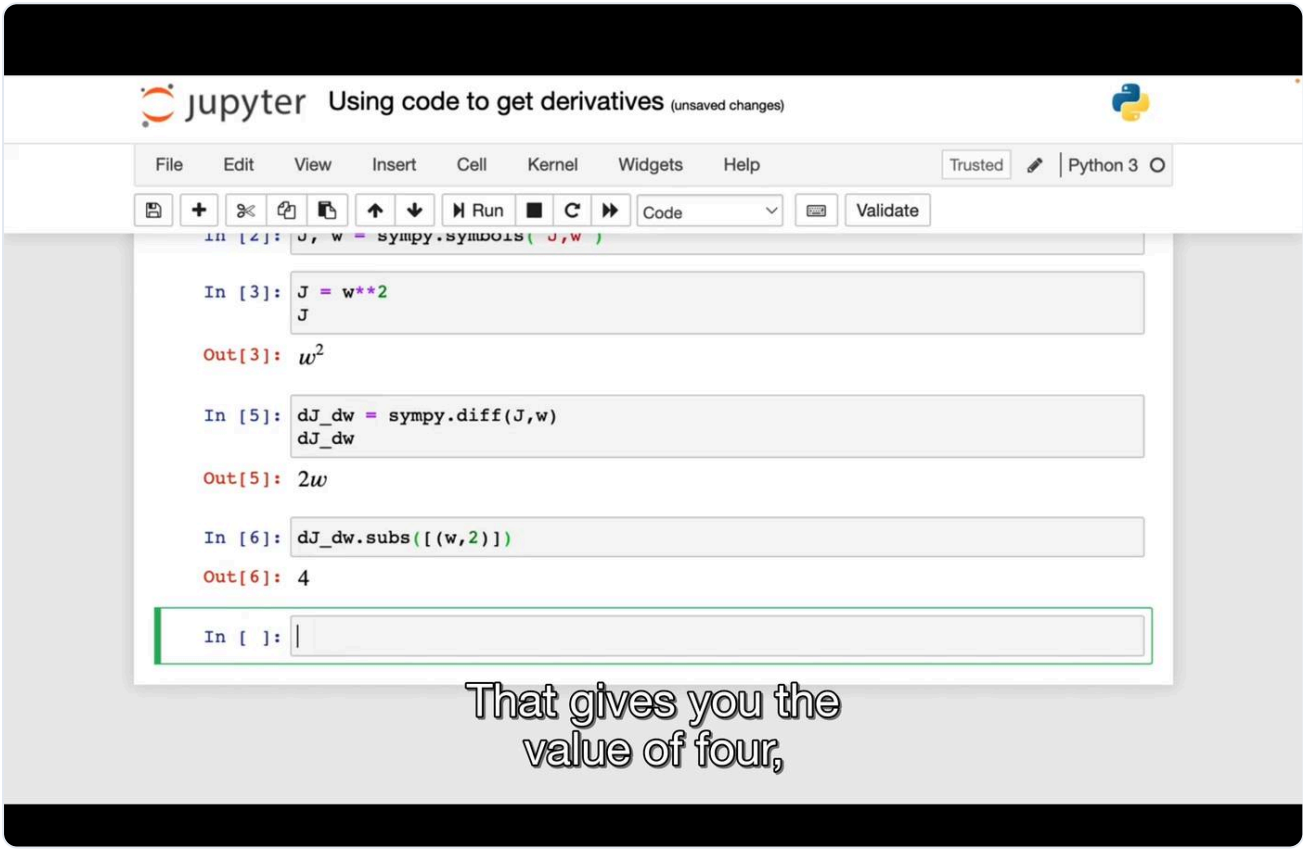


FIGURE 30.8

Let's copy this back to our other slide. What SymPy or really calculus showed us is if J of w is w cubed, the derivative is 3w squared, which is equal to 12 when w equals 2, when J of w equals w, the derivative is just equal to 1. When J of w is 1 over w is negative 1 over w squared, which is negative 1/4 when w equals 2. Let's start. We'll check if these expressions that we got from SymPy are correct. Let's try increasing w by Epsilon, in this case J of w. Feel free to pause the video and check this math on your own calculator if you want. But in this case, J of w to 0.001 cubed becomes this. So J has gone up from 8 to 8.012 roughly. It's gone up by roughly 12 times Epsilon.

Thus the derivative is indeed 12. Or if J of w equals w, then if w increase by Epsilon, then J of w, which is just w, is now 2.001. So it's gone up by 0.01, which is exactly the value of Epsilon. So J of w has gone up by 1 times Epsilon. The derivative is indeed equal to 1. Notice that here, this is actually, exactly Epsilon even though Epsilon is infinitesimally small. On our last example, if J of w

equals 1/w, if w goes up by Epsilon, then w is 1/2.001, then it turns out J of w is approximately 4.9975 with some extra digits that are truncated. But this turns out to be 0.5 minus 0.00025. J of w has started off at 0.5 and it's gone down by 0.00025.

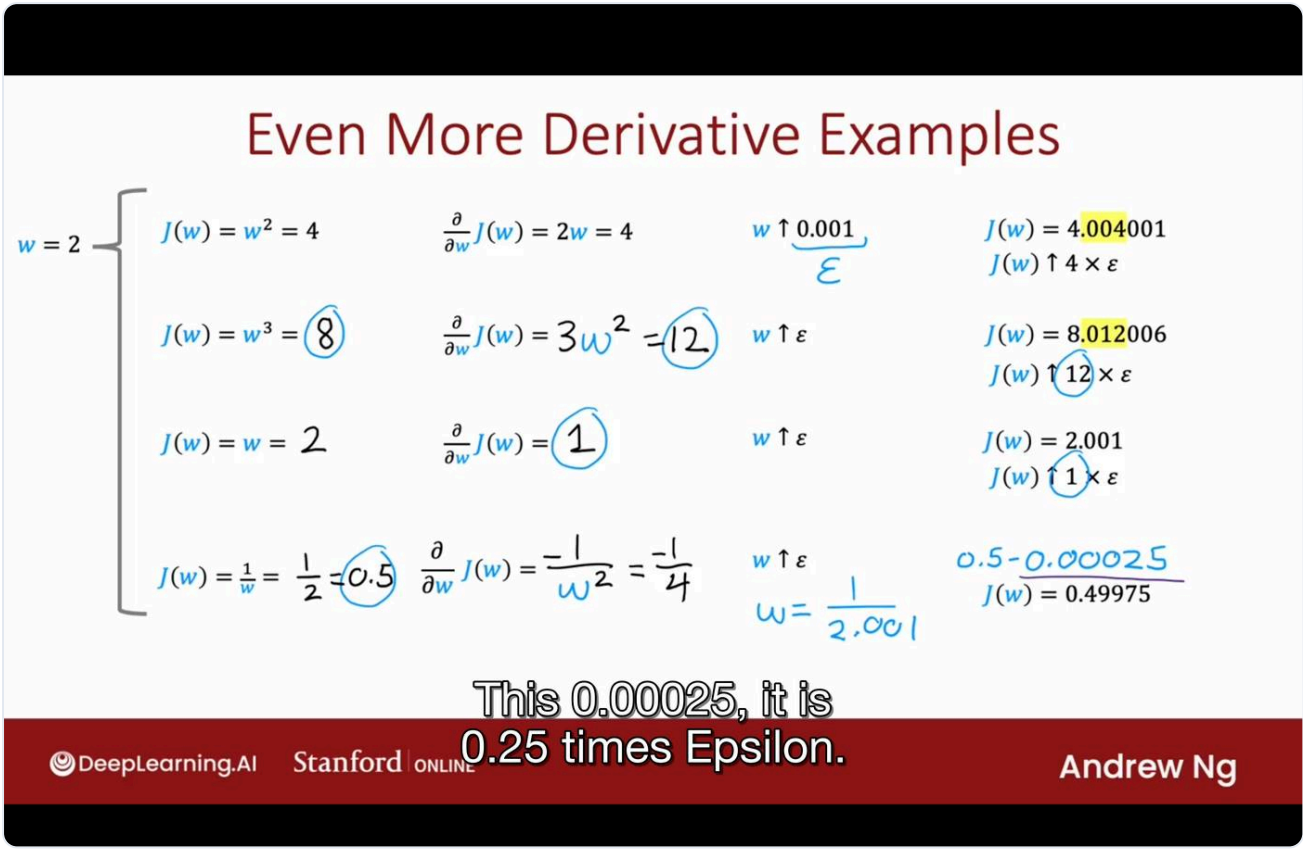


FIGURE 30.9 | Even More Derivative Examples

This 0.00025, it is 0.25 times Epsilon.

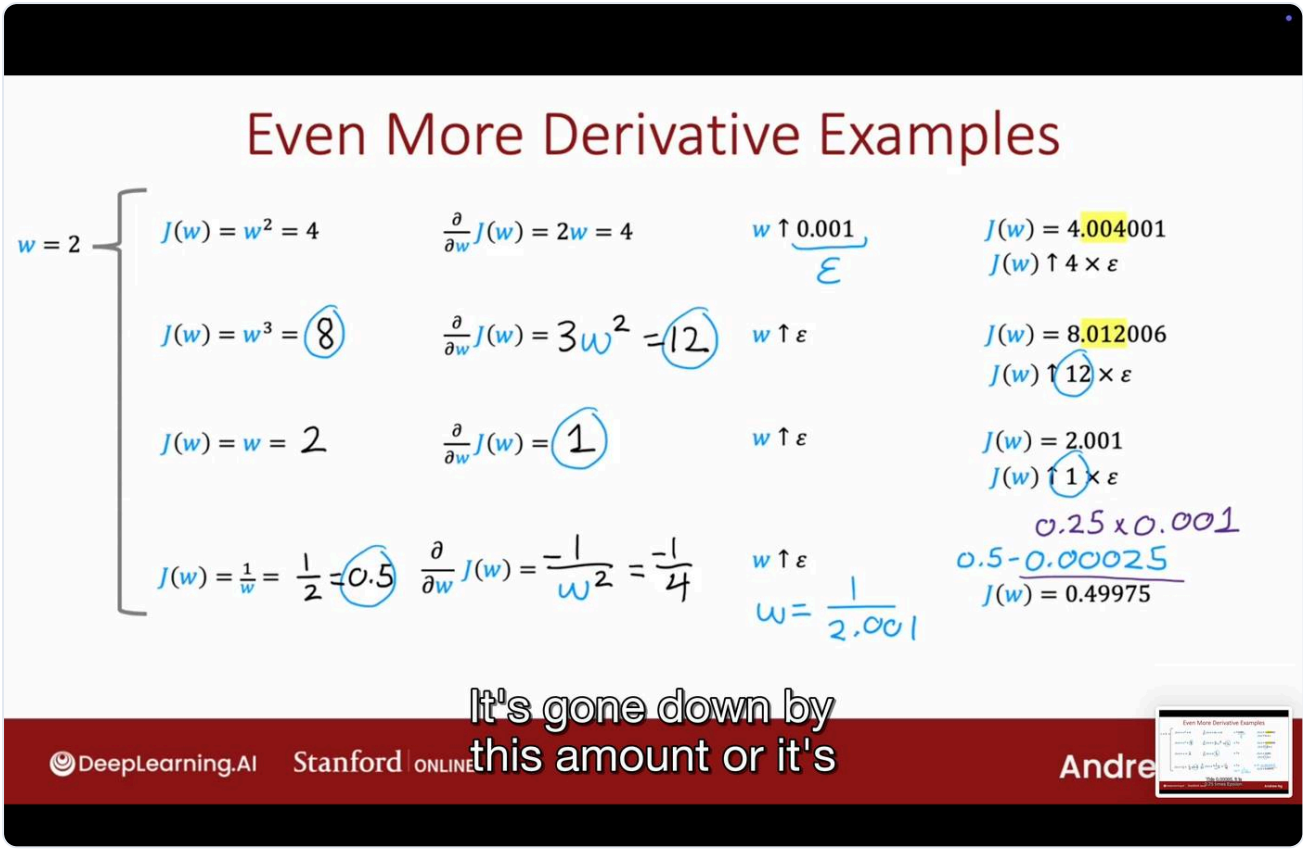


FIGURE 30.10 | Even More Derivative Examples

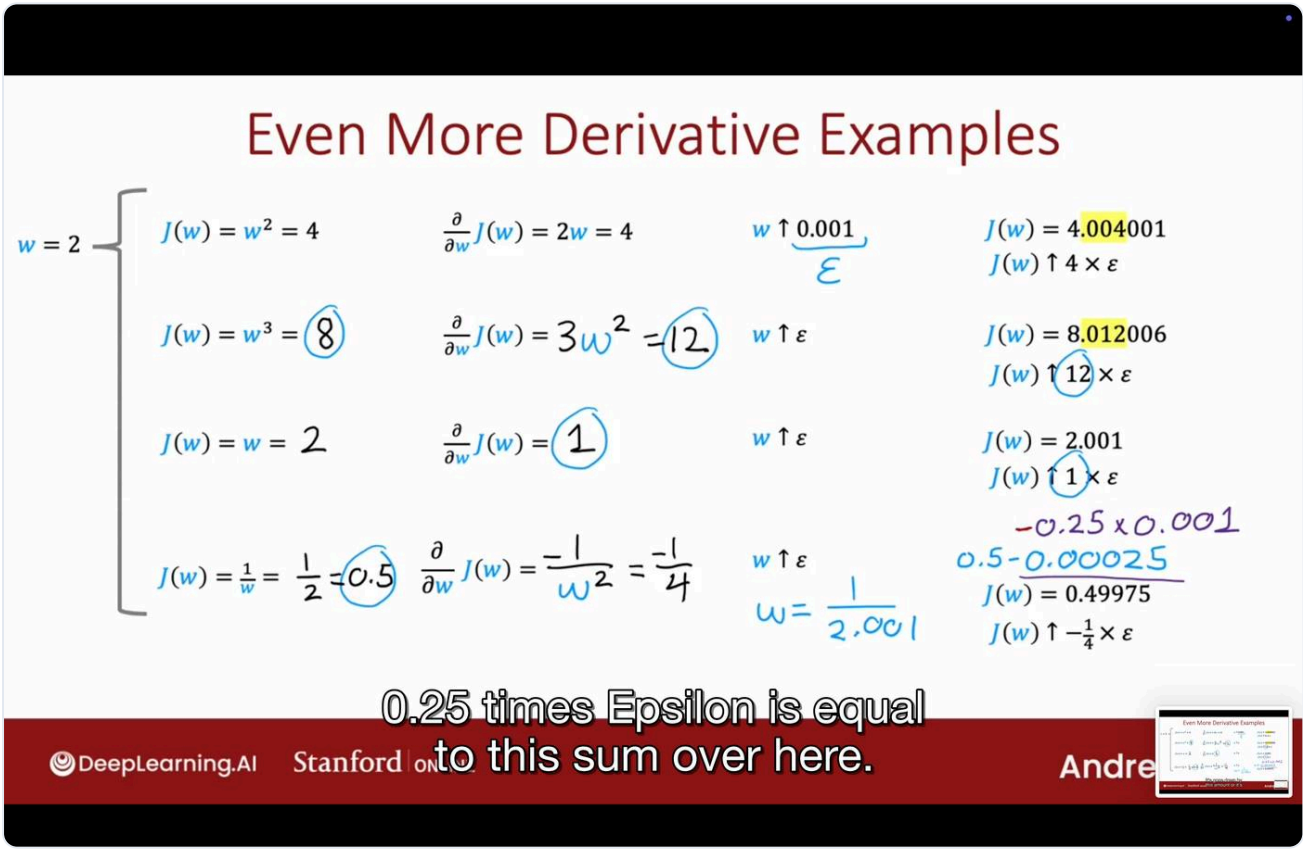


FIGURE 30.11 | Even More Derivative Examples

It's gone down by this amount or it's gone up by negative 0.25 times Epsilon because negative 0.25 times Epsilon is equal to this sum over here. We see that if w goes up by Epsilon, J of w goes up by negative 1/4 or negative 0.25 times Epsilon, which is why the derivative in this case is negative 1/4. I hope that with these examples you have a sense of what the derivative with respect to w of J of w means.

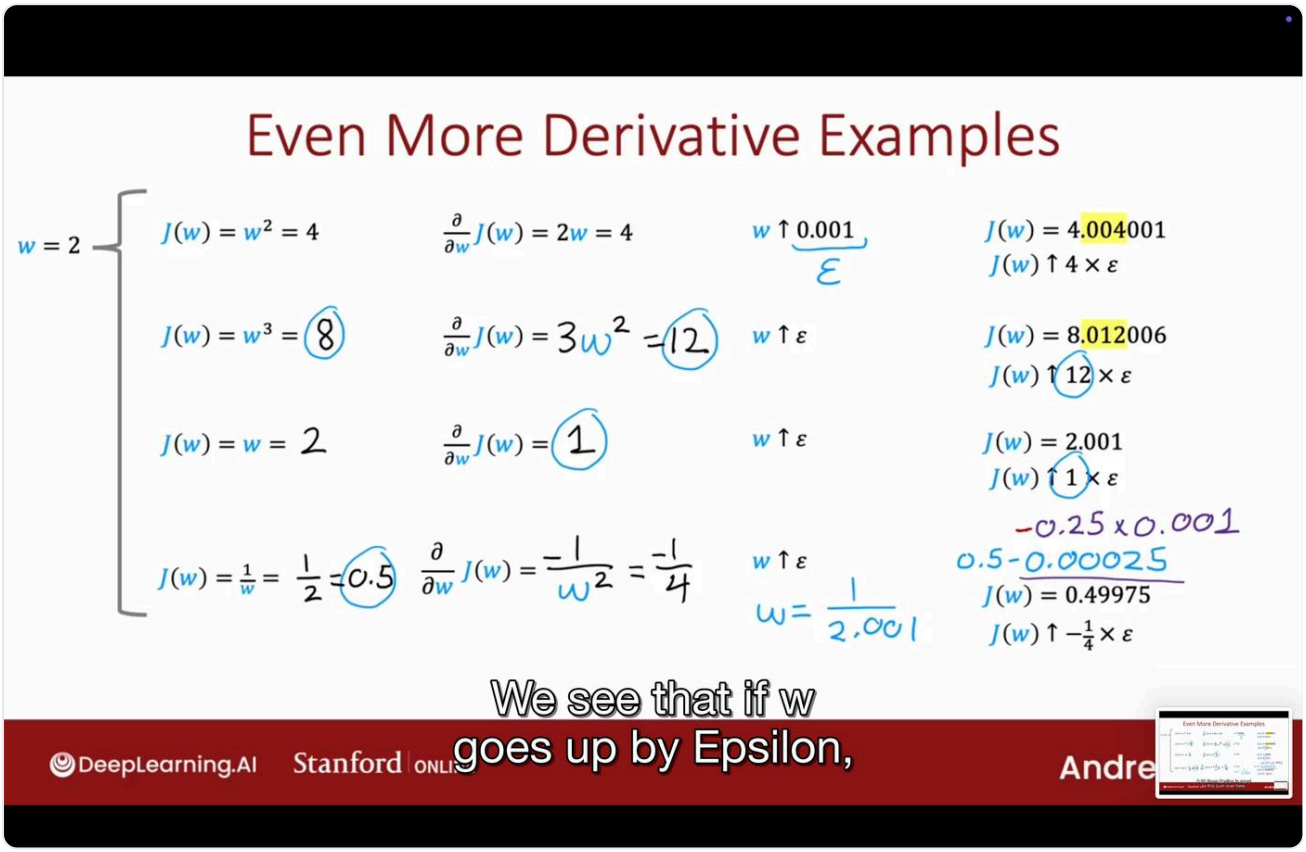


FIGURE 30.12 | Even More Derivative Examples

It just is if w goes up by Epsilon, how much does J of w goes up by some constant k times Epsilon. This constant k is the derivative. The value of k will depend both on what is the function J of w, as well as what is the value of w. Before we wrap up this video, I want to briefly touch on the notation used to write derivatives that you may see in other texts.

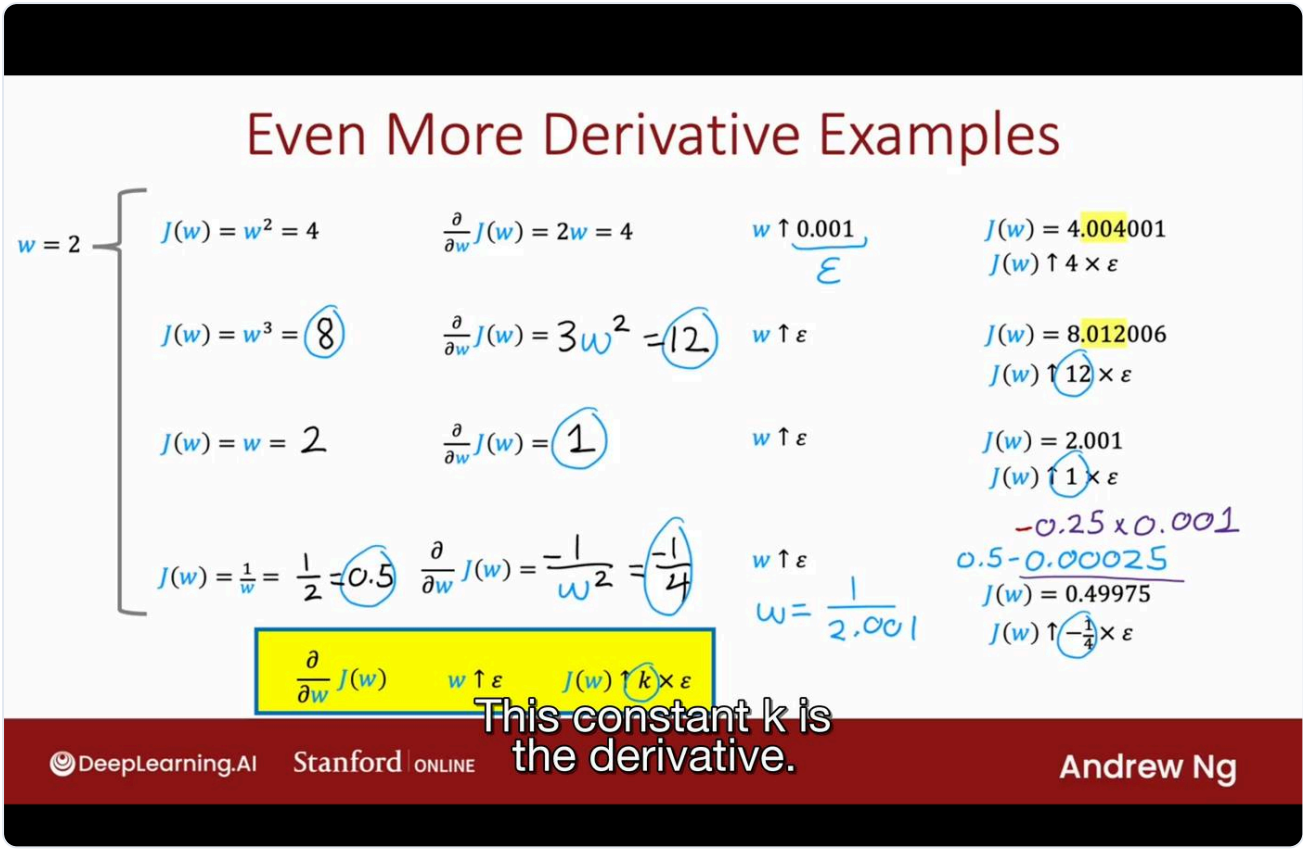


FIGURE 30.13 | Even More Derivative Examples

Which is that if J of w is a function of a single variable, say w, then mathematicians will sometimes write the derivative as d/dw of J of w. Notice here this notation is using the lowercase letter d. Whereas in contrast, if J is a function of more than one variable, then mathematicians will sometimes use this squiggly alternative d to denote the derivative of J with respect to one of the parameters w_i.

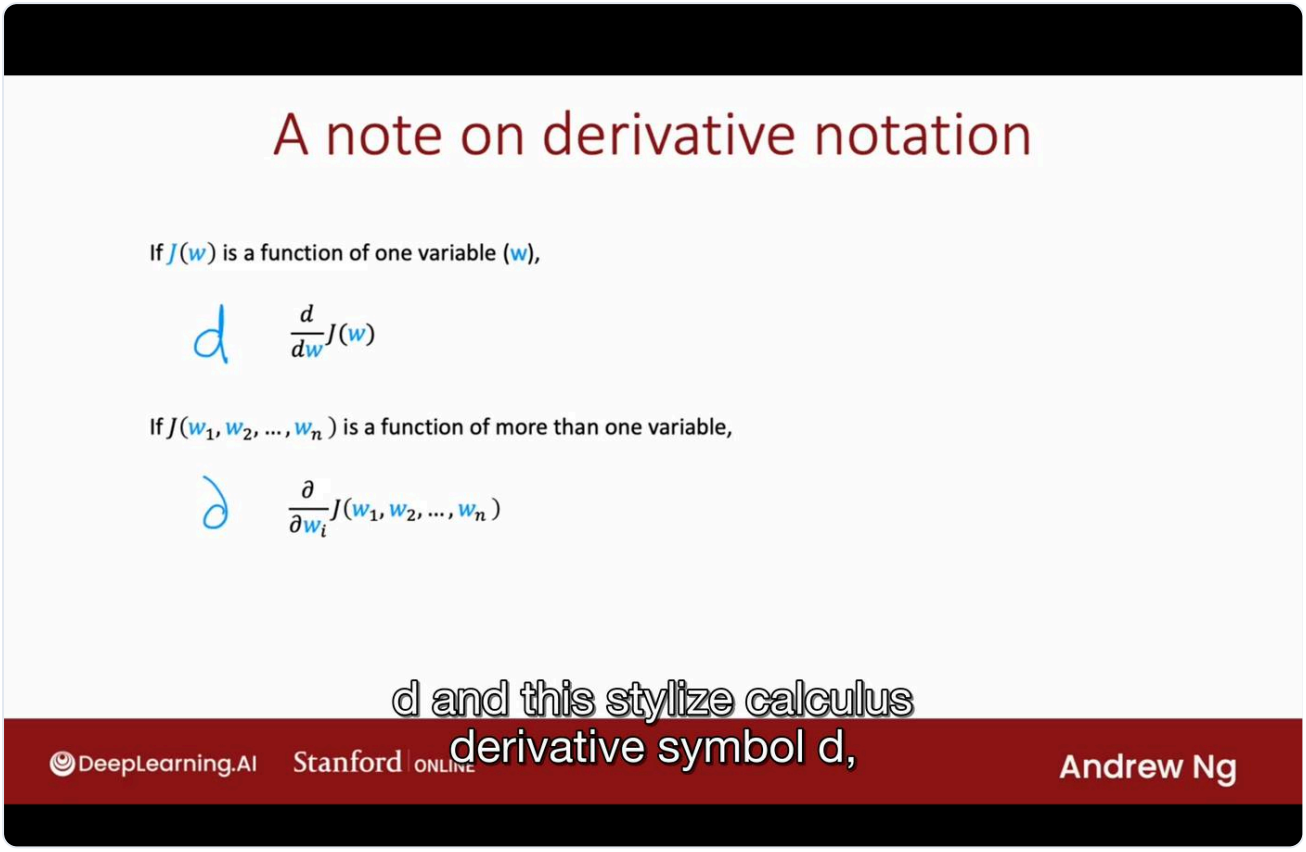


FIGURE 30.14 | A Note on Derivative Notation

To my mind, this notation distinguishing between this regular letter d and this stylize calculus derivative symbol d, it makes little sense to me to make this distinction and this notation to my mind over complicates calculus and derive this notation. But for historical reasons, calculus text will use these two different notations depending on whether J is a function of a single variable or a function of multiple variables. But I think for practical purposes, this notational convention, it tends to just over-complicate things, I think, in a way that I don't think is actually necessary. For this class, I'm just going to use this notation everywhere, even when there's just a single variable.

In fact, for most of our applications, the function J it is a function of more than one variable.

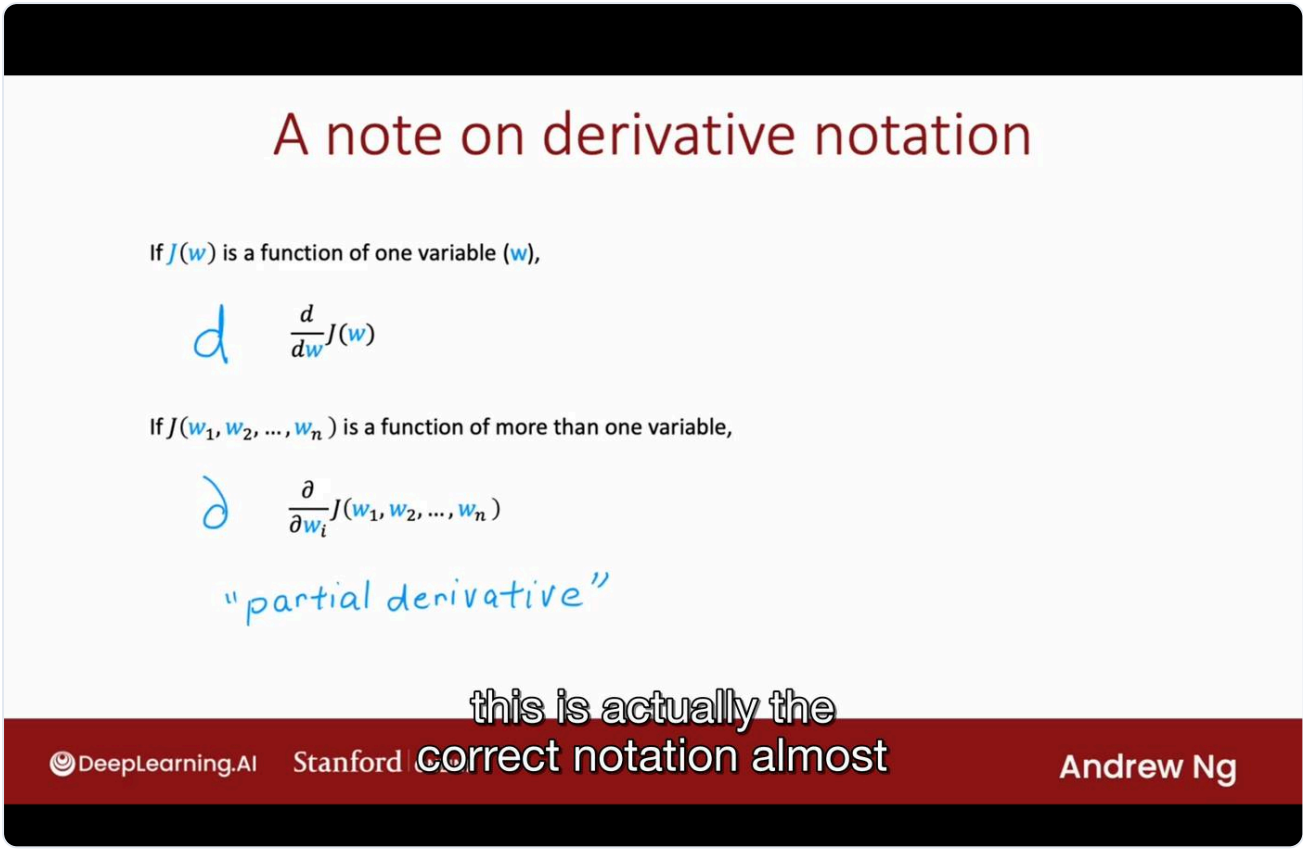


FIGURE 30.15 | A Note on Derivative Notation

So this other notation, which is sometimes called the partial derivative notation, this is actually the correct notation almost all the time because J usually has more than one variable. But I hope that using this notation throughout these lectures that simplifies the presentation and makes derivatives little bit easier to understand. In fact, this notation is the one you've been seeing in the videos leading up to now. For conciseness, instead of writing out this full expression here, sometimes you also see it shortened as derivative or partial derivative of J with respect to w_i or written like this.

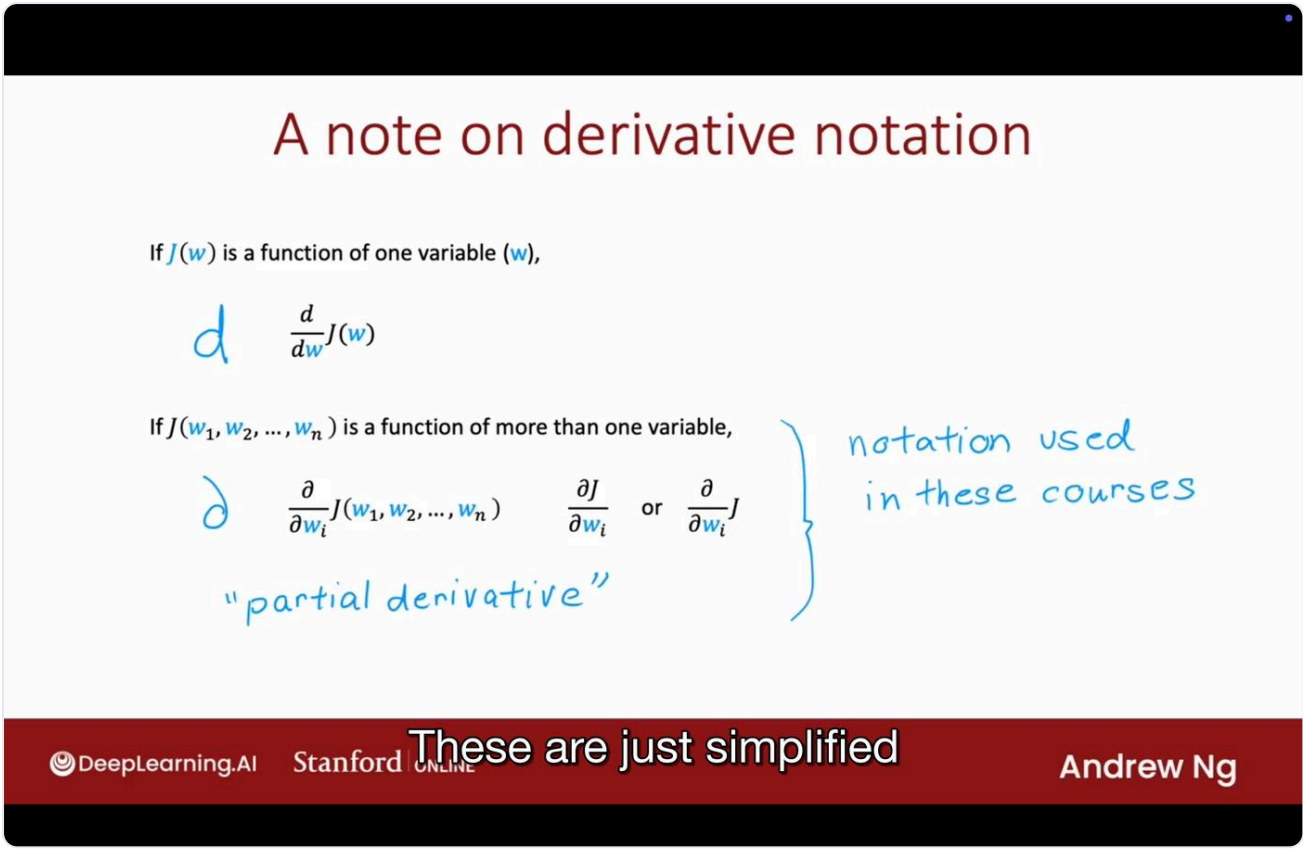


FIGURE 30.16 | A Note on Derivative Notation

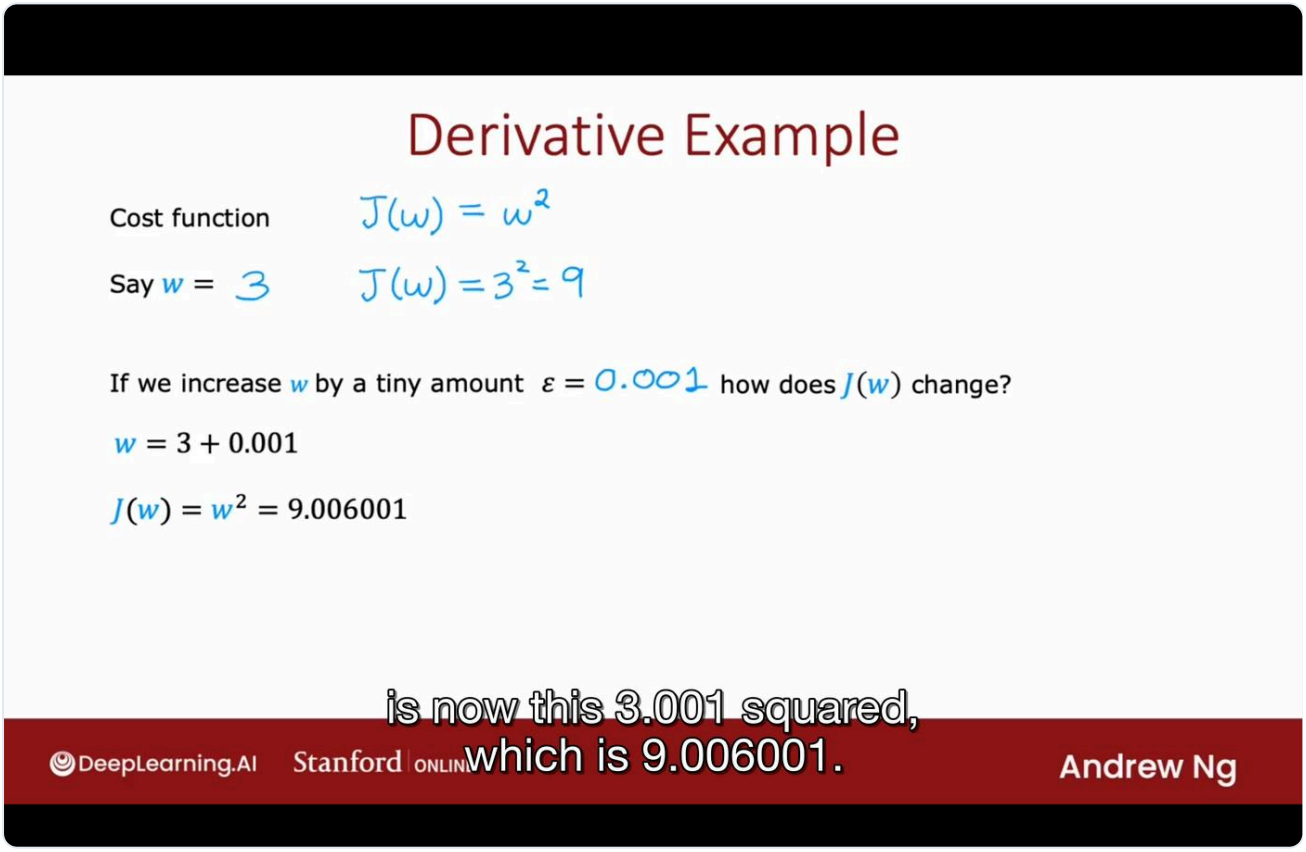


FIGURE 30.17 | Derivative Example

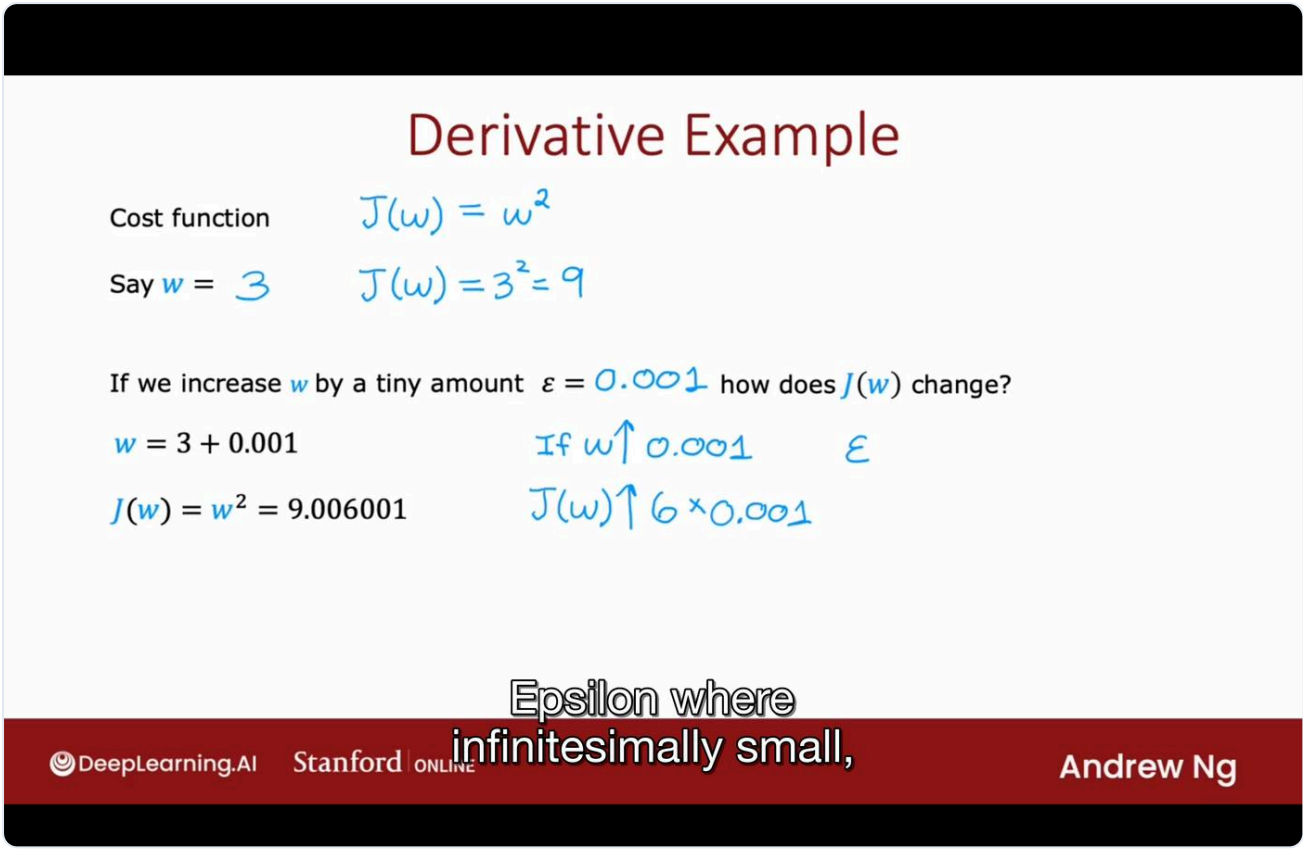


FIGURE 30.18 | Derivative Example

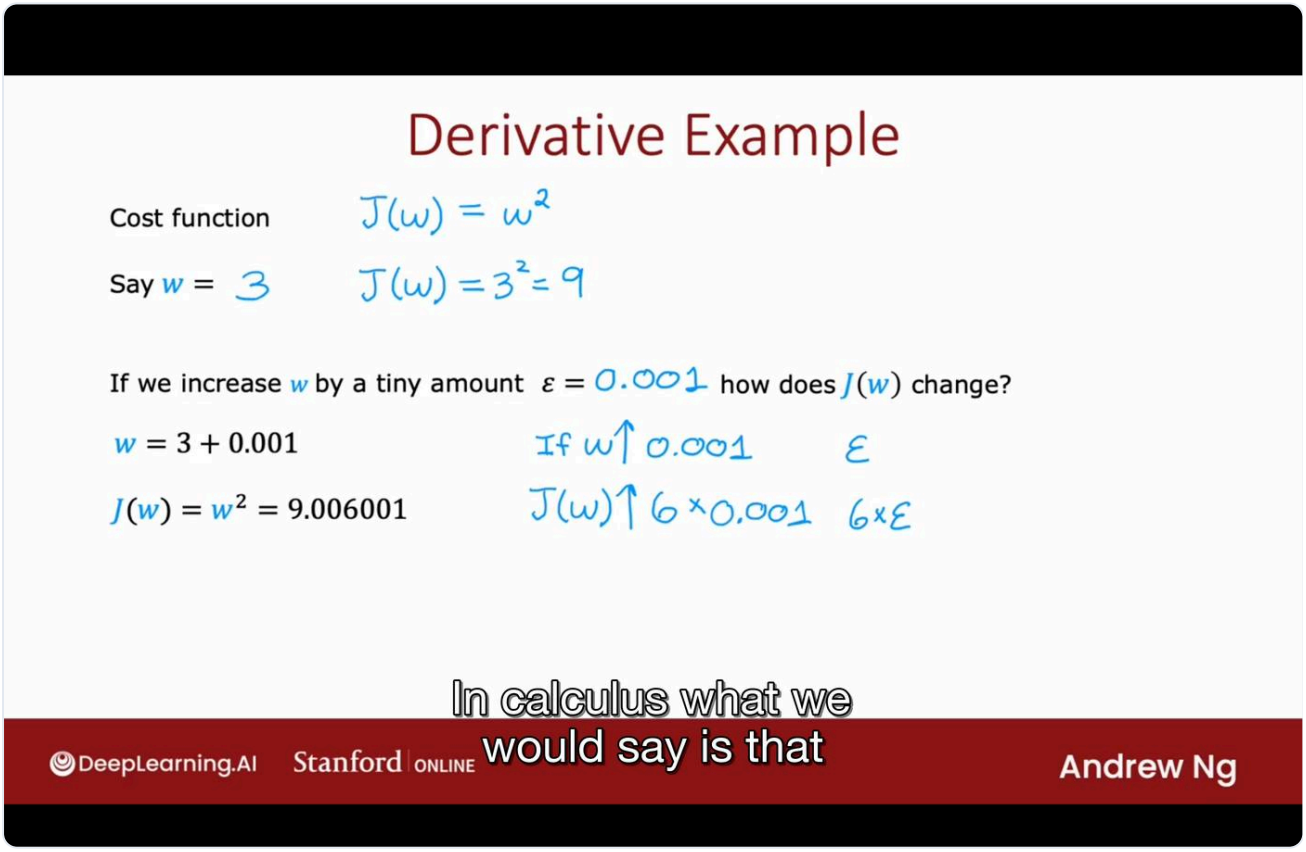


FIGURE 30.19 | Derivative Example

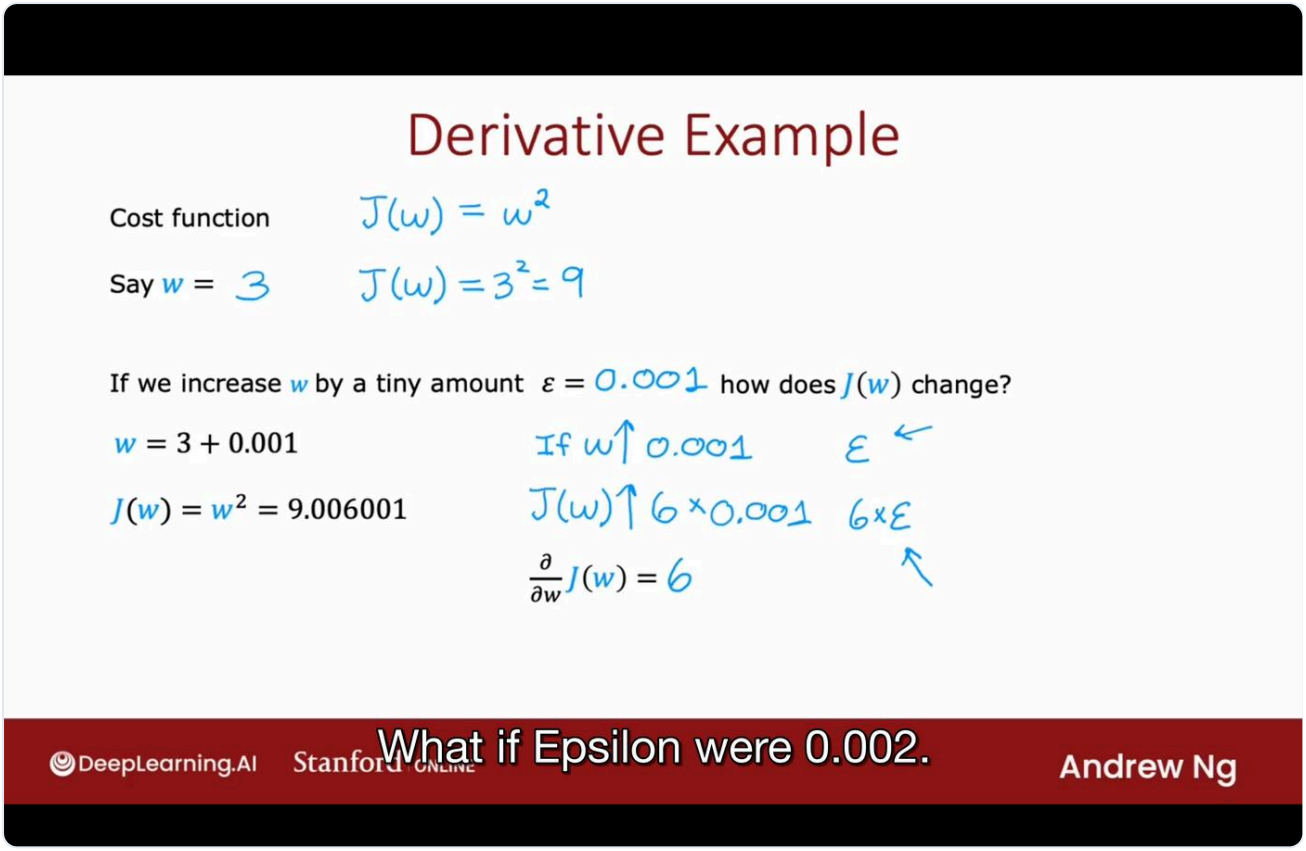


FIGURE 30.20 | Derivative Example

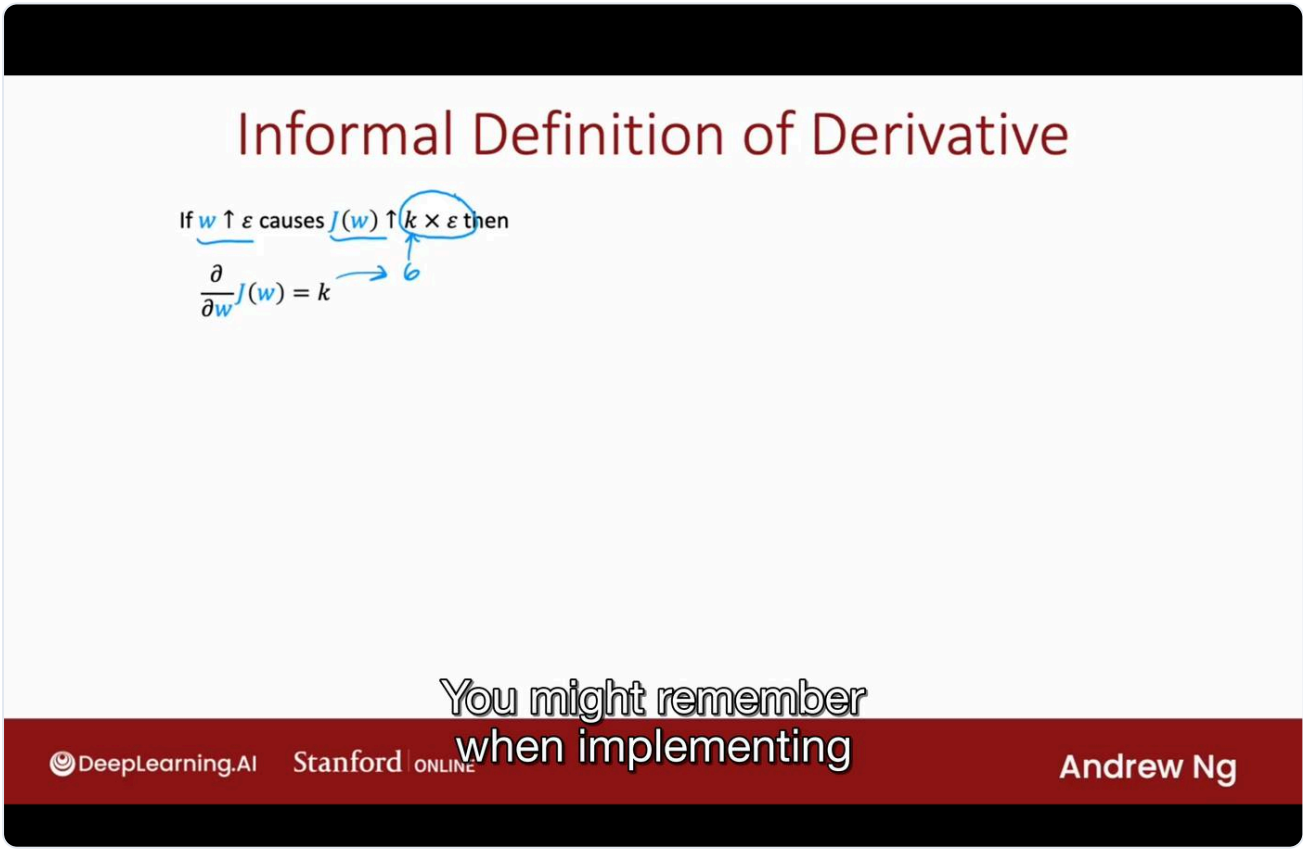


FIGURE 30.21 | Informal Definition of Derivative

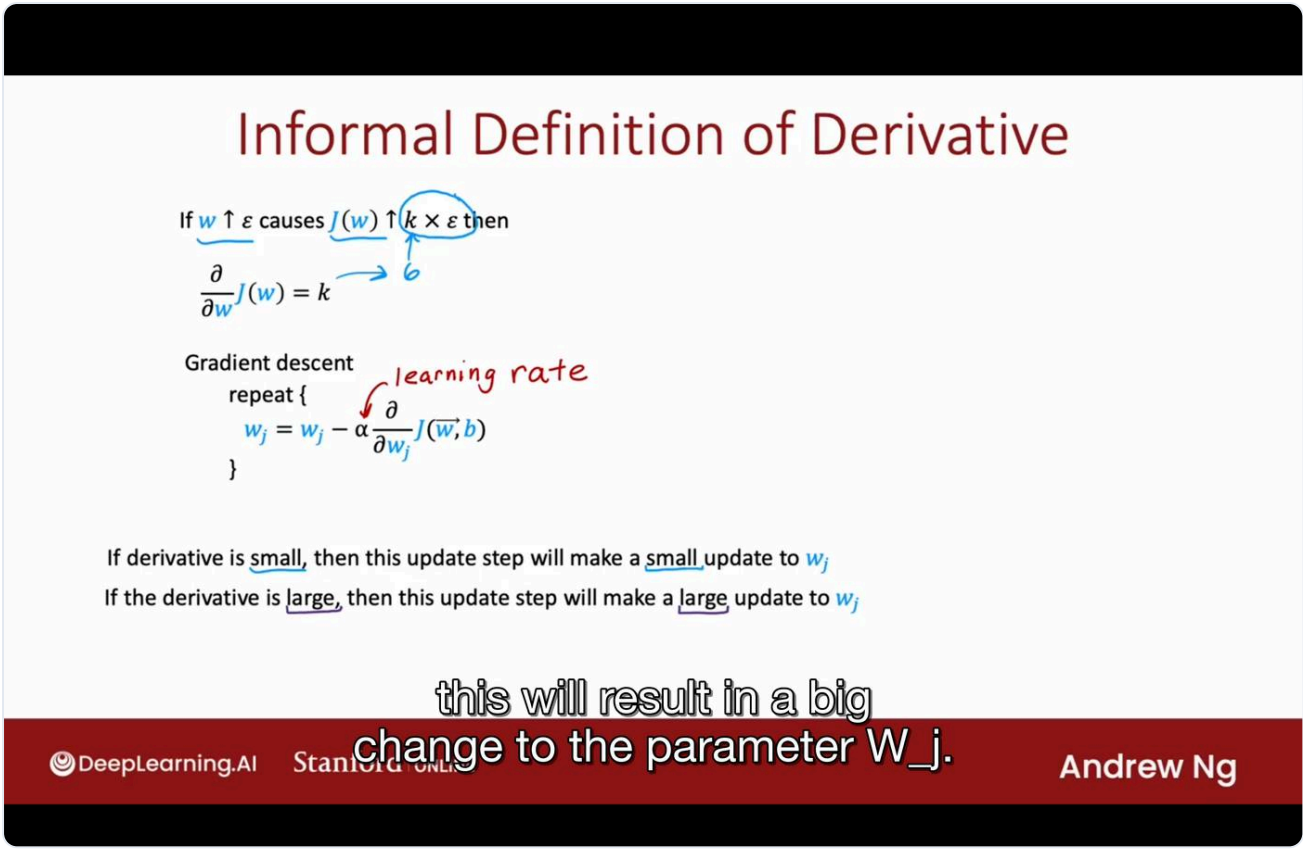


FIGURE 30.22 | Informal Definition of Derivative

These are just simplified abbreviated forms of this expression over here. I hope that gives you a sense of what are derivatives. It's just if w goes up by a little bit, by Epsilon, how much does J of w change as a consequence. Next, let's take a look at how you can compute derivatives in a neural network. To do so, we need to take a look at something called a computation graph. Let's go take a look at that in the next video.

LECTURE 31

Forward Prop

The computation graph is a key idea in deep learning, and it is also how programming frameworks like TensorFlow, automatic compute derivatives of your neural networks. Let's take a look at how it works. Let me illustrate the concept of a computation graph with a small neural network example. This neural network has just one layer, which is also the output layer, and just one unit in the output layer. It takes us inputs x, applies a linear activation function and outputs deactivation a. More specifically this output is a equals wx plus b. This basically linear regression, but expressed as a neural network with one output unit. Given the output, the cause function is then 1/2a, that is the predicted value minus the actual observed value of y.

For this small example, we're only going to have a single training example, where the training example is the input x equals negative 2.

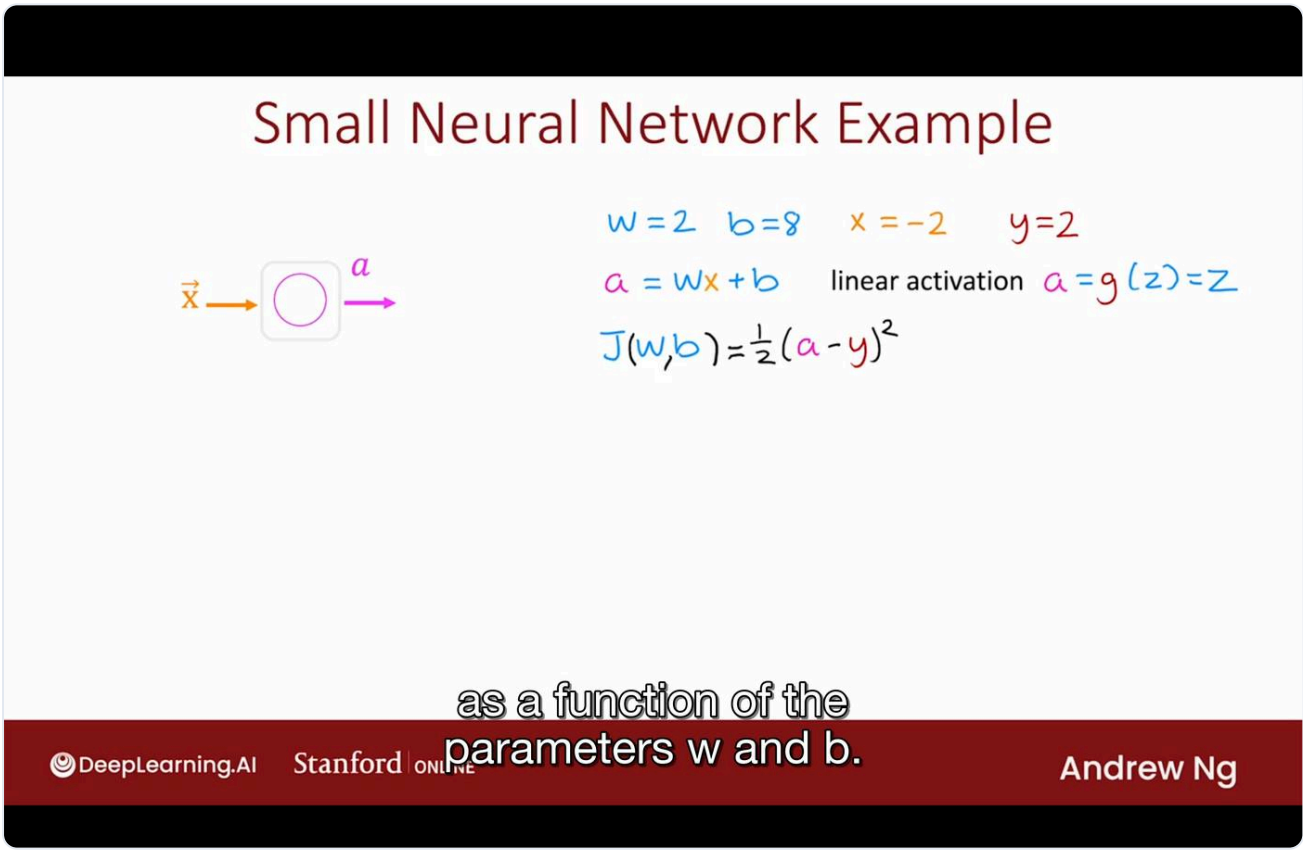


FIGURE 31.1 | Small Neural Network Example

The ground truth output value y equals 2, and the parameters of this network are; w equals 2 and b equals 8. What I'd like to do is show how the computation of the cause function J can be computed step by step using a computation graph. Just as a reminder, when learning, we like to view the cause function J as a function of the parameters w and b. Let's take the computation of J and break it down into individual steps. First, you have the parameter w that is an input to the cause

function J, and then we first need to compute w times x. Let me just call that c as follows; w is equal to 2, x is equal to negative 2, and so c would be negative 4. I'm just going to write the value here on top of this arrow to show the value that is your output on this arrow.

The next step is then to compute a, which is wx plus b.

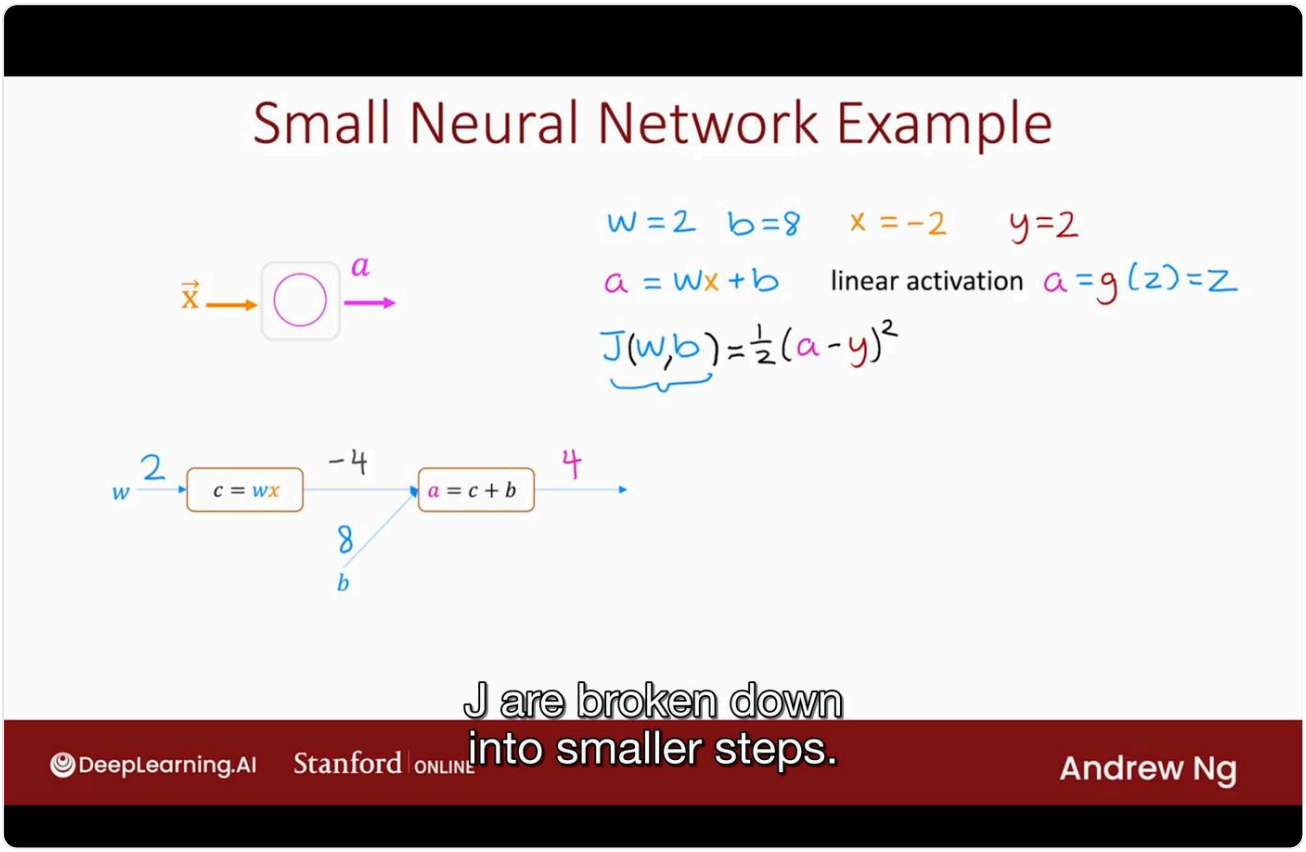


FIGURE 31.2 | Small Neural Network Example

So let me create another node here. This needs to input b, the other parameter that is input to the cause function J, and a equals wx plus b is equal to c plus b. If you add these up, that turns out to be 4. This is starting to build up a computation graph in which the steps we need to compute the cause function J are broken down into smaller steps. The next step is to then compute a minus y, which I'm going to call d. Let me have that node d, which computes a minus y. Y is equal to 2, so 4 minus 2 is 2. Then finally, J is the cause is 1/2 of a minus y squared, or 1/2 of d squared, which is just equal to 2. What we've just done is build up a computation graph.

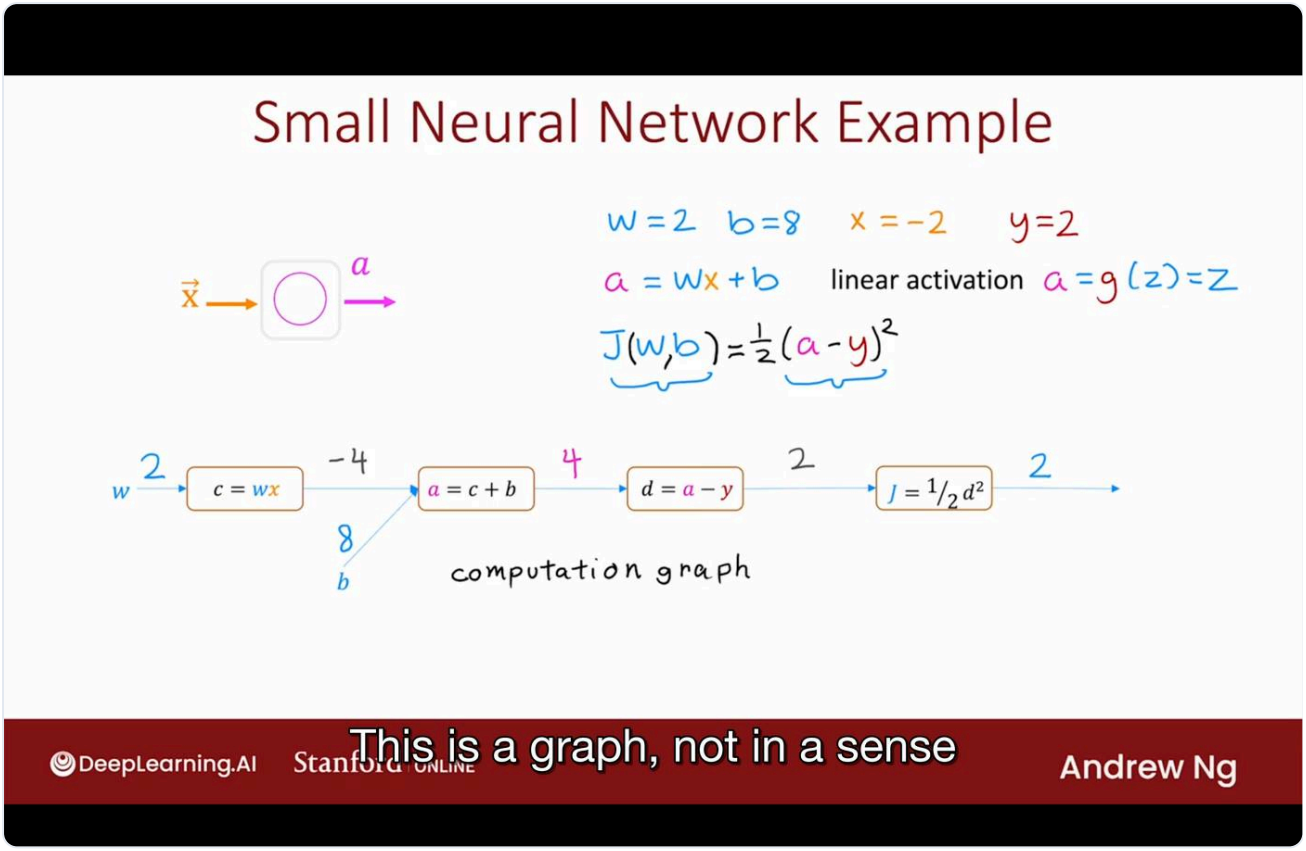


FIGURE 31.3 | Small Neural Network Example

This is a graph, not in a sense of plots with x and y axes, but this is the other sense of the word graph using computer science, which is that this is a set of nodes that is connected by edges or connected by arrows in this case. This computation graph shows the forward prop step of how we compute the output a of the neural network. But then also go further than that so also compute the value of the cause function J.

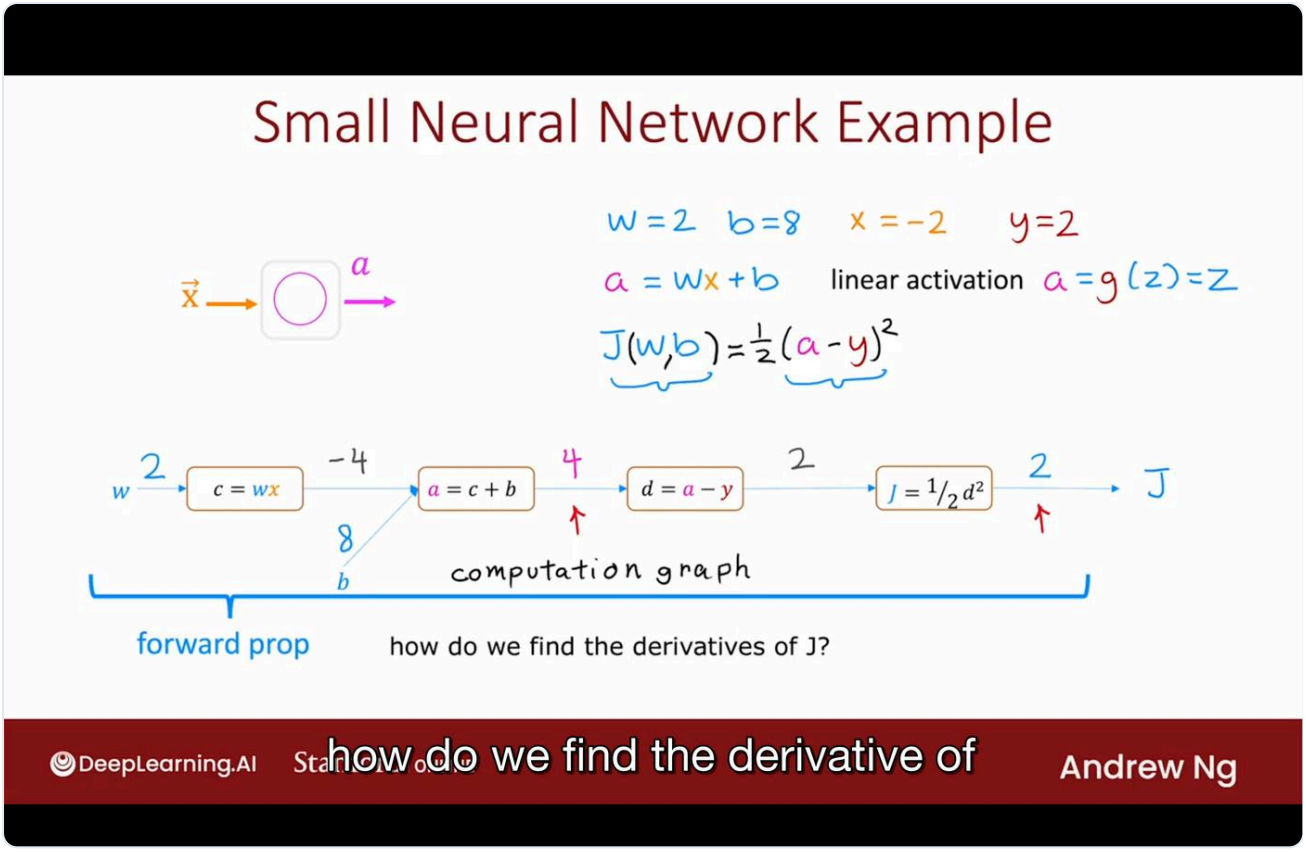


FIGURE 31.4 | Small Neural Network Example

The question now is, how do we find the derivative of J with respect to the parameters w and b?

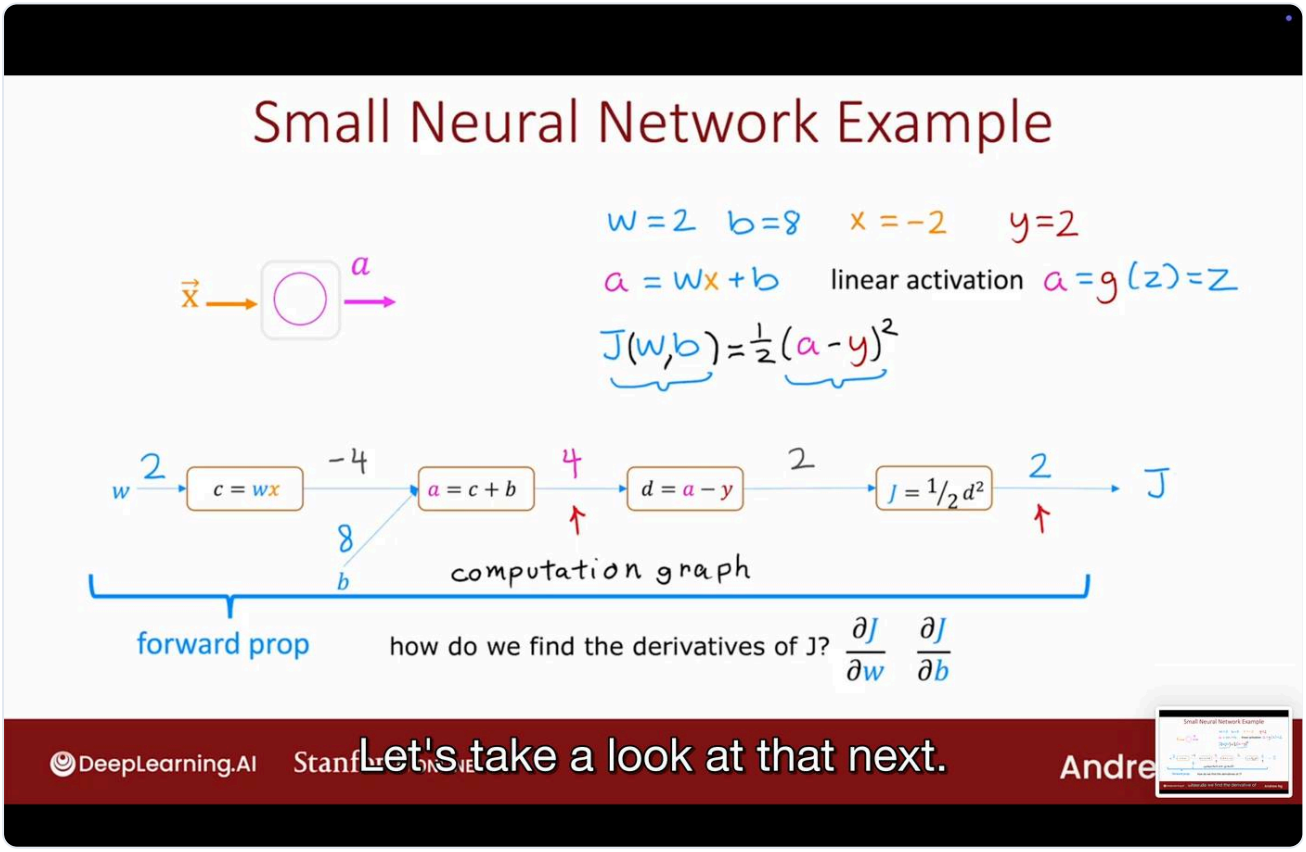


FIGURE 31.5 | Small Neural Network Example

Let's take a look at that next. Here's the computation graph from the previous slide and we've completed for a problem where we've computed that J, the cause function is equal to 2 through all these steps going from left to right for a prop in the computation graph. What we'd like to do now is compute the derivative of J with respect to w and the derivative of J with respect to b. It turns out that whereas for a prop was a left to right calculation, computing the derivatives will be a right to left calculation, which is why it's called backprop, was going backwards from right to left. The final computation nodes of this graph is this one over here, which computes J equals 1/2 of d squared.

The first step of backprop will ask if the value of d, which was the input to this node where the change a little bit. How much does the value of j change? Specifically, will ask if d were to go up by a little bit, say 0.001, and that'll be our value of Epsilon in this case, how would the value of j change? It turns out in this case if d goes from 2-2.01, then j goes from 2-2.02. So if d goes up by Epsilon, j goes up by roughly two times Epsilon. We conclude that the derivative of J with respect to this value d that is inputted this final node is equal to two.

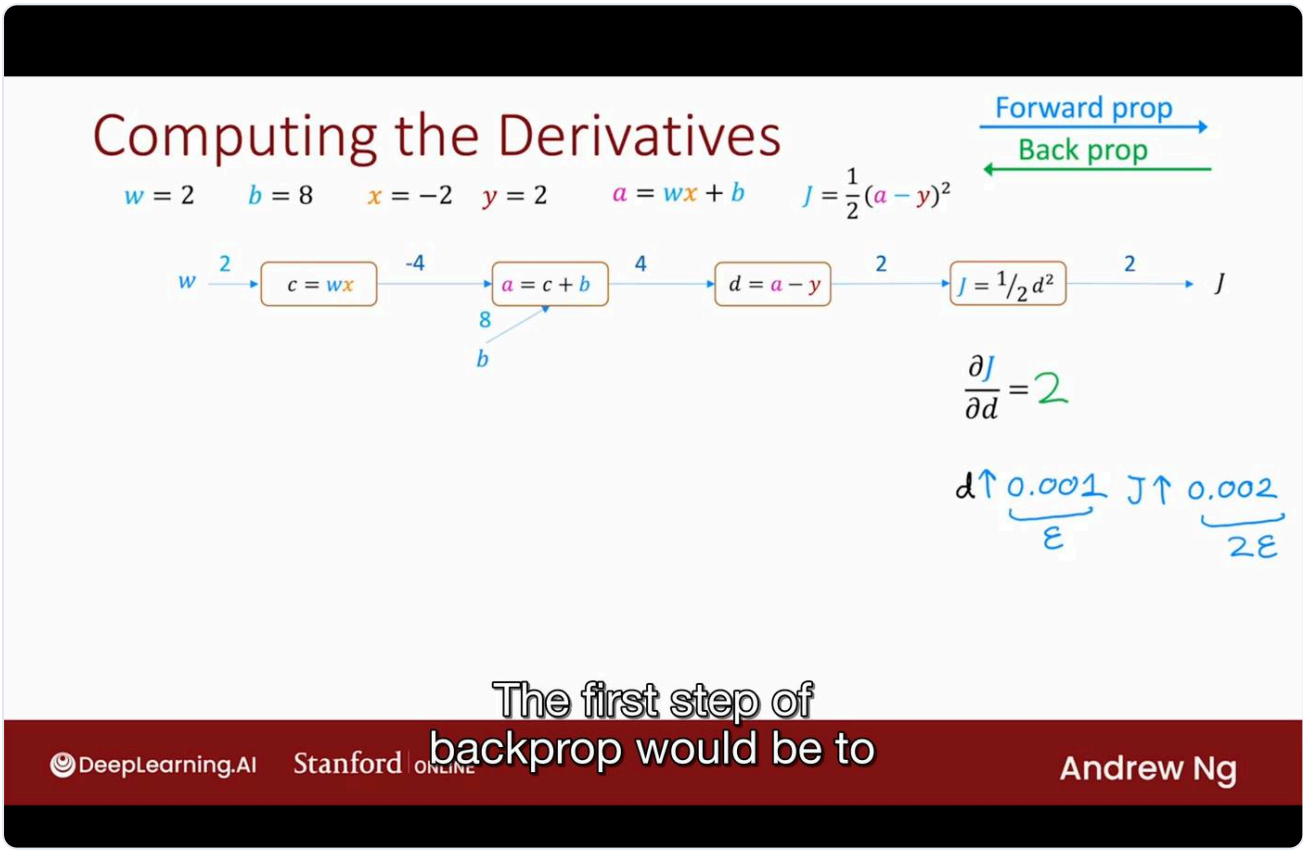


FIGURE 31.6 | Forward Prop

The first step of backprop would be to fill in this value two over here, where this value is the derivative of j with respect to this input value d. We know if d changes a little bit, j changes by twice as much because this derivative is equal to two. The next step is to look at the node before that and ask what is the derivative of j with respect to a? To answer that, we have to ask, well, if a goes up by 0.001, how does that change j? Well, we know that if a goes up by 0.001, d is just a minus y. If a becomes 4.001, d which is a minus y, becomes 4.001 minus y equals 2, so becomes 2.001, sub a goes up by 0.001, d also goes up by 0.001. But we'd already concluded previously that the d goes up by 0.001, j goes up by twice as much.

Now we know if a goes up by 0.001, d goes up by 0.001, then j goes up roughly by two times 0.001. This tells us that the derivative of j with respect to a is also equal to two. So I'm going to fill in that value over here. That this is the derivative of j with respect to a. Just as this was the derivative of j respect to d.

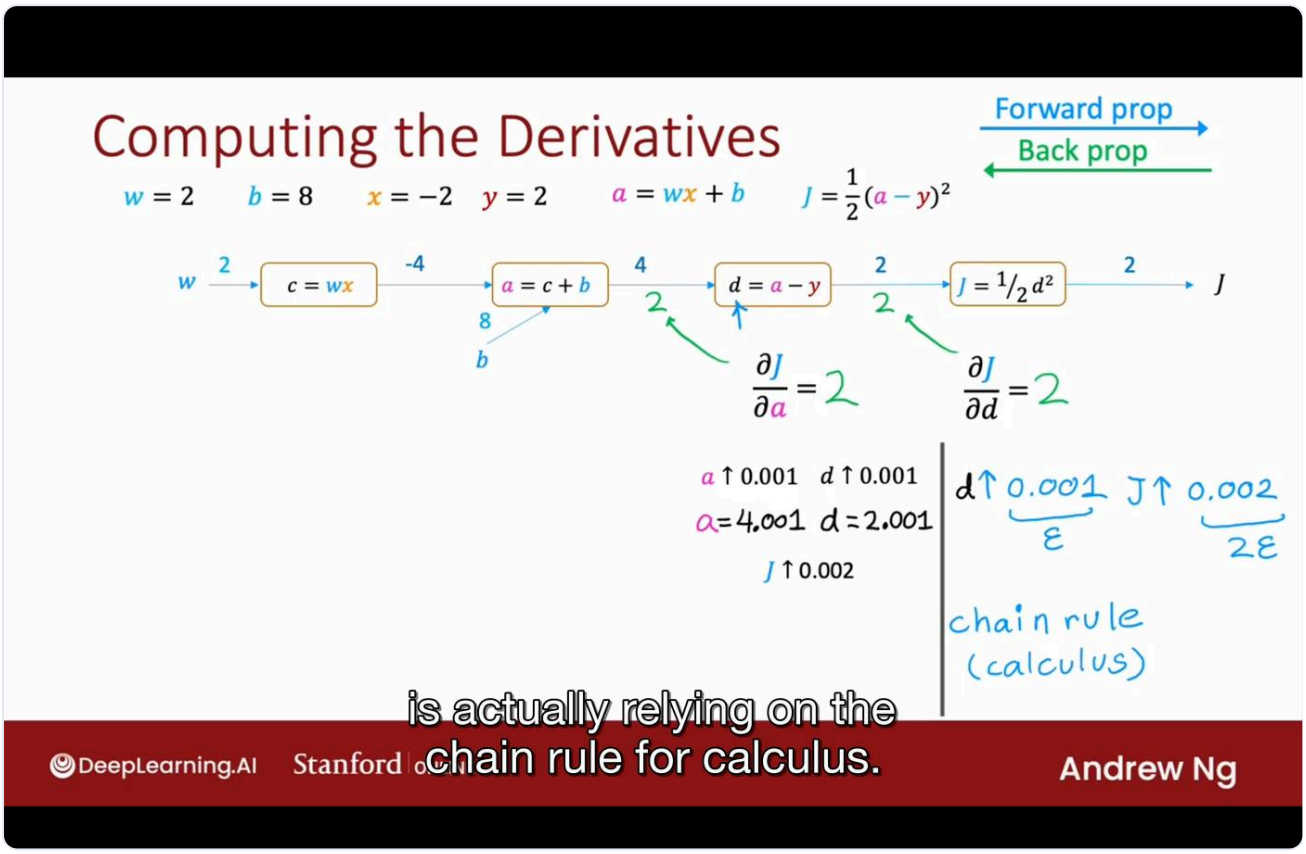


FIGURE 31.7 | Forward Prop

If you've taken a calculus class before and if you've heard of the chain rule, you might recognize that this step of computation that I just did is actually relying on the chain rule for calculus. If you're not familiar with the chain rule, don't worry about it. You won't need to know it for the rest of these videos.

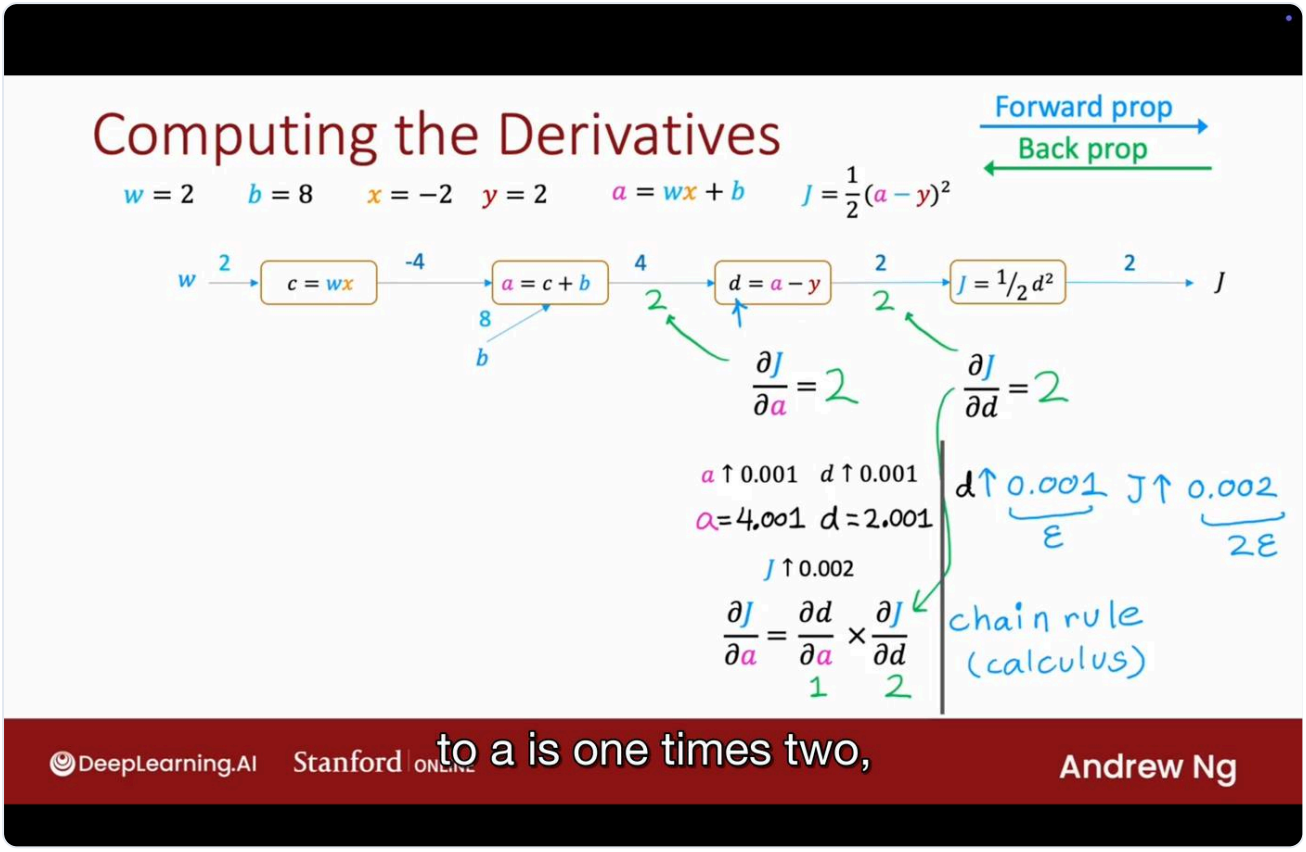


FIGURE 31.8 | Forward Prop

But if you have seen the chain rule, you might recognize that the derivative of j with respect to a is asking, how much does d change respect to a, which is derivative of d respect to a times the derivative of j with respect to d, and does little calculation on top showed that the partial of t with respect to a is one, and we'd show EZ that the derivative of J with respect to d is equal to two, which is why the derivative of J with respect to a is one times two, which is equal to two. That's the value we got. But again, if you're not familiar with the chain rule, don't worry about it. The logic that we just went through here is why we know j goes up by twice as much as a does.

That's why this derivative term is equal to two. The next step then is to keep on going right to left as we do in backprop. We'll ask, how much does a little change in c cause j to change, and how much does y change in b cause j to change? The way we figure that out is to ask, what if c goes up by Epsilon 0.001, how much does a change? Well, a is equal to c plus b. It turns out that if c ends up being negative 3.999, then a, which is negative 3.999 plus 8, becomes 4.001. If c goes up by Epsilon, a goes up by Epsilon. We know if a up by epsilon, then because the derivative of J with respect to a is two, we know that this in turn causes j to go up by two times Epsilon.

We can conclude that if c goes up by a little bit, J goes up by twice as much. We know this because we know the derivative of J with respect to a is 2. This allows us to conclude that the derivative of J with respect to c is also equal to 2. I'm going to fill in that value over here. Again, only if you're familiar with chain rule another way to write this is derivative of J respect to c, is the derivative of a respect to c. This turns out to be 1 times the derivative of J respect to a, which we have previously figured out was equal to 2, so that's why this ends up being equal to 2. By a similar calculation, the b goes up by 0.001, then a also goes up by 0.001 and J goes up by 2 times 0.001, which is why this derivative is also equal to 2.

We filled in here the derivative of J respect to b, and here the derivative of J respect to c.

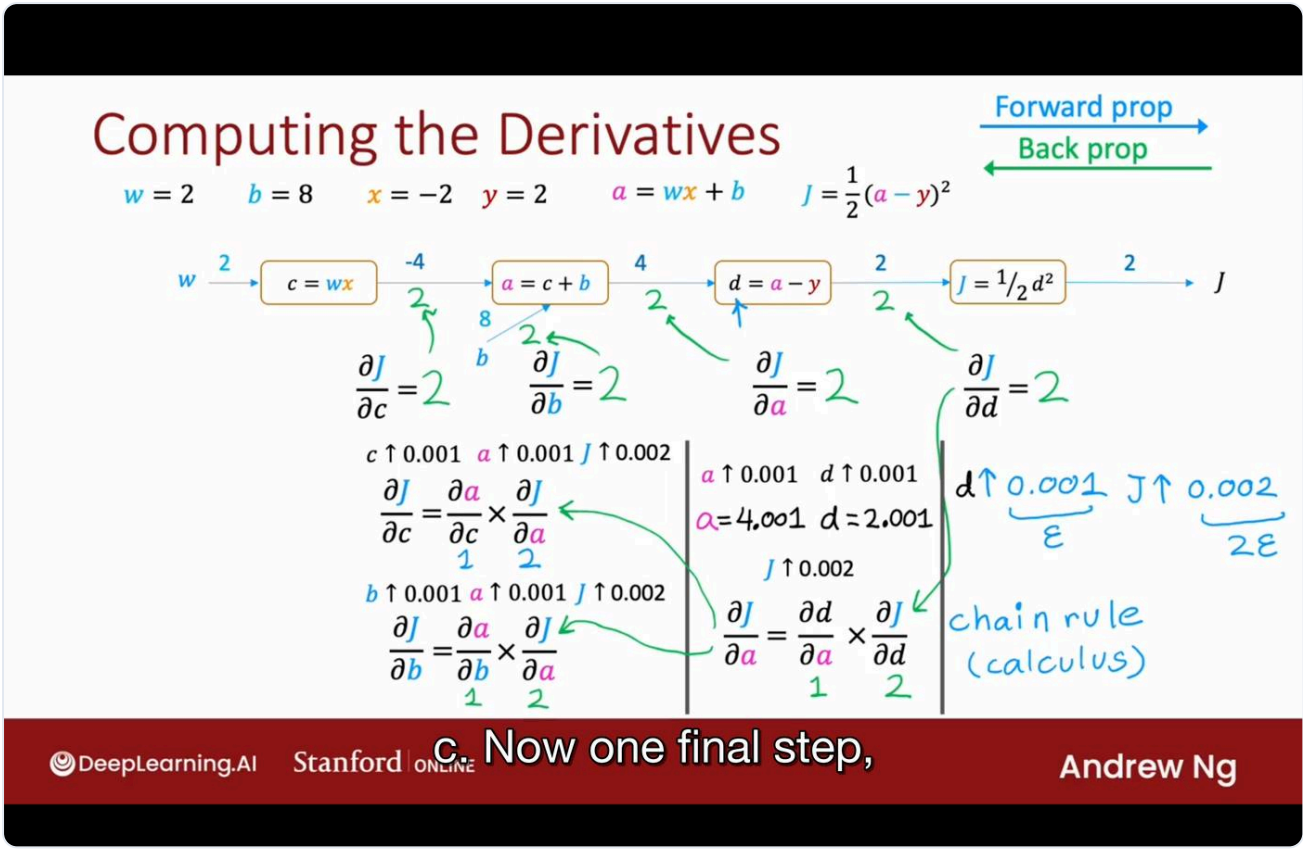


FIGURE 31.9 | Forward Prop

Now one final step, which is, what is the derivative of J with respect to w? W goes up by 0.001. What happens? C which is w times x, if w were 2.001, c which is w times x, becomes negative 2 times 2.001, so it becomes negative 4.002. If w goes up by epsilon, c goes down by 2 times 0.001, or equivalently c goes up by negative 2 times 0.001. We know that if c goes up by negative 2 times 0.001, because the derivative of J with respect to c is 2, this means that J will go up by negative 4 times 0.001, because if c goes up by a certain amount, J changes by 2 times as much, so negative 2 times this is negative 4 times this. This allows us to conclude that if w goes up by 0.001, J goes up by negative 4 times 0.001.

The derivative of J with respect to w is negative 4.

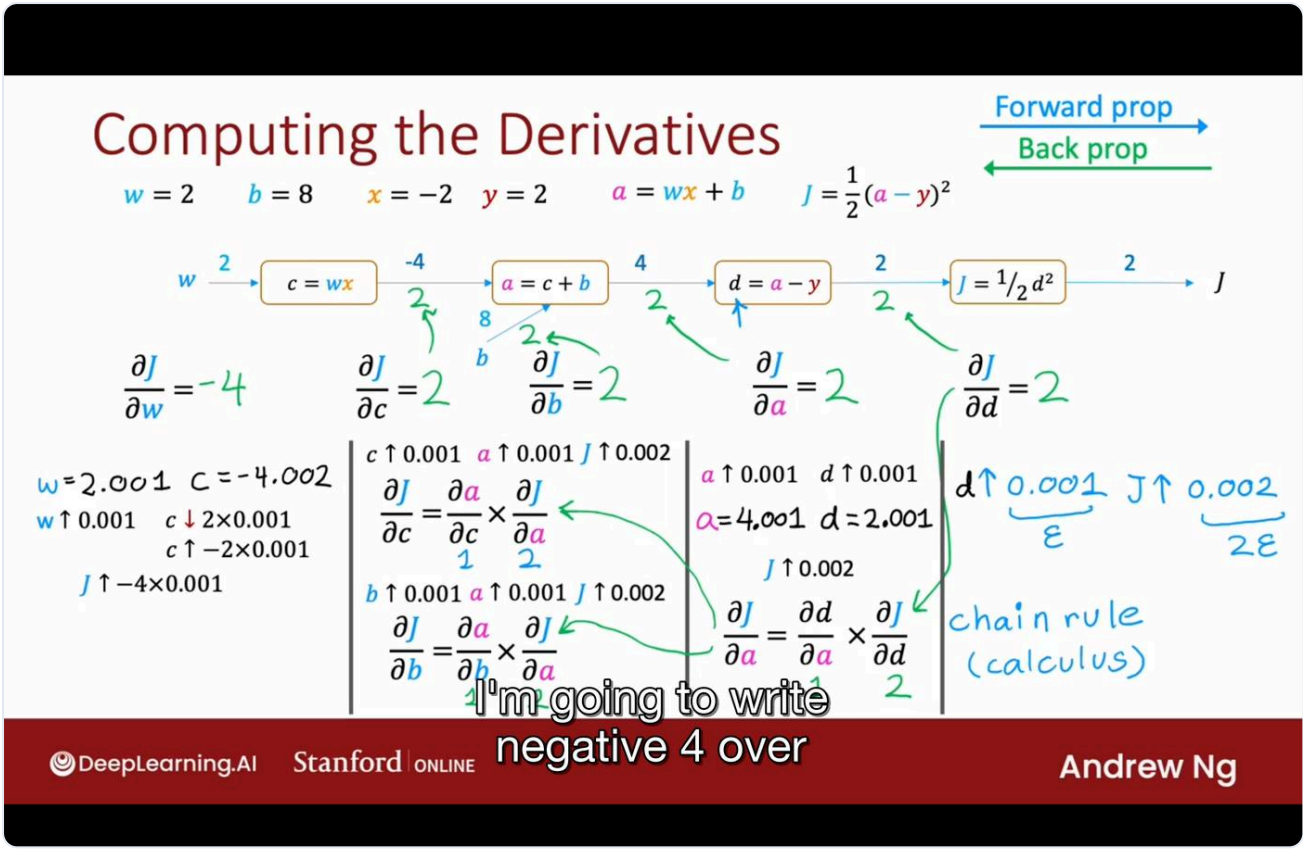


FIGURE 31.10 | Forward Prop

I'm going to write negative 4 over here because has the derivative of J with respect to w. Once again, the chain rule calculation, if you're familiar with it, is this. It is the derivative of c respect to w times derivative of J with respect to c.

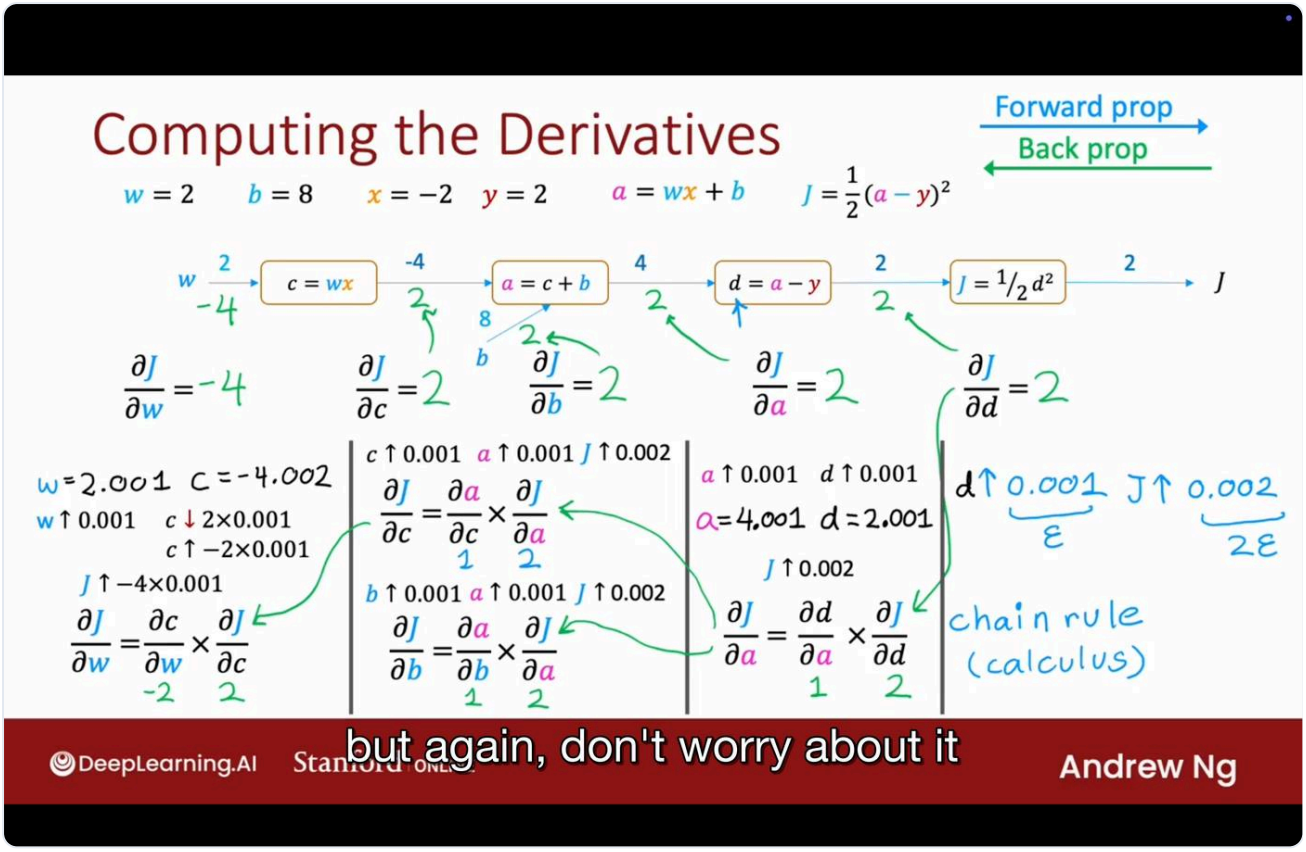


FIGURE 31.11 | Forward Prop

This is 2 and this is negative 2, which is why we end up with negative 4, but again, don't worry about it if you're not familiar with chain rule. To wrap up what we've just done this manually carry out backprop in this computation graph. Whereas forward prop was a left-to-right computation where we had w equals 2, that allowed us to compute c. Then we had b and that allows us to compute a and then d, and then J backprop went from right-to-left and we would first compute the derivative of J with respect to d and then go back to compute the derivative of J with respect to a, then the derivative of J with respect to b, derivative of J with respect to c and finally the derivative of J with respect to w.

So that's why backprop is a right-to-left computation, whereas forward prop was a left-to-right computation. In fact, let's double-check the computation that we just did. J with these values of w, b, x and y is equal to one-half times wx plus b minus y squared, which is one-half times 2 times negative 2 plus 8 minus 2 squared, which is equal to 2. Now if w were to go up by 0.001, then J becomes one-half times, w is now 2.001 times x, which is negative 2, plus b which is 8 minus y squared. If you calculate this out, this turns out to be 1.996002. Roughly J has gone from 2 down to 1.996, and then an extra 002 and J has therefore gone down by 4 times epsilon.

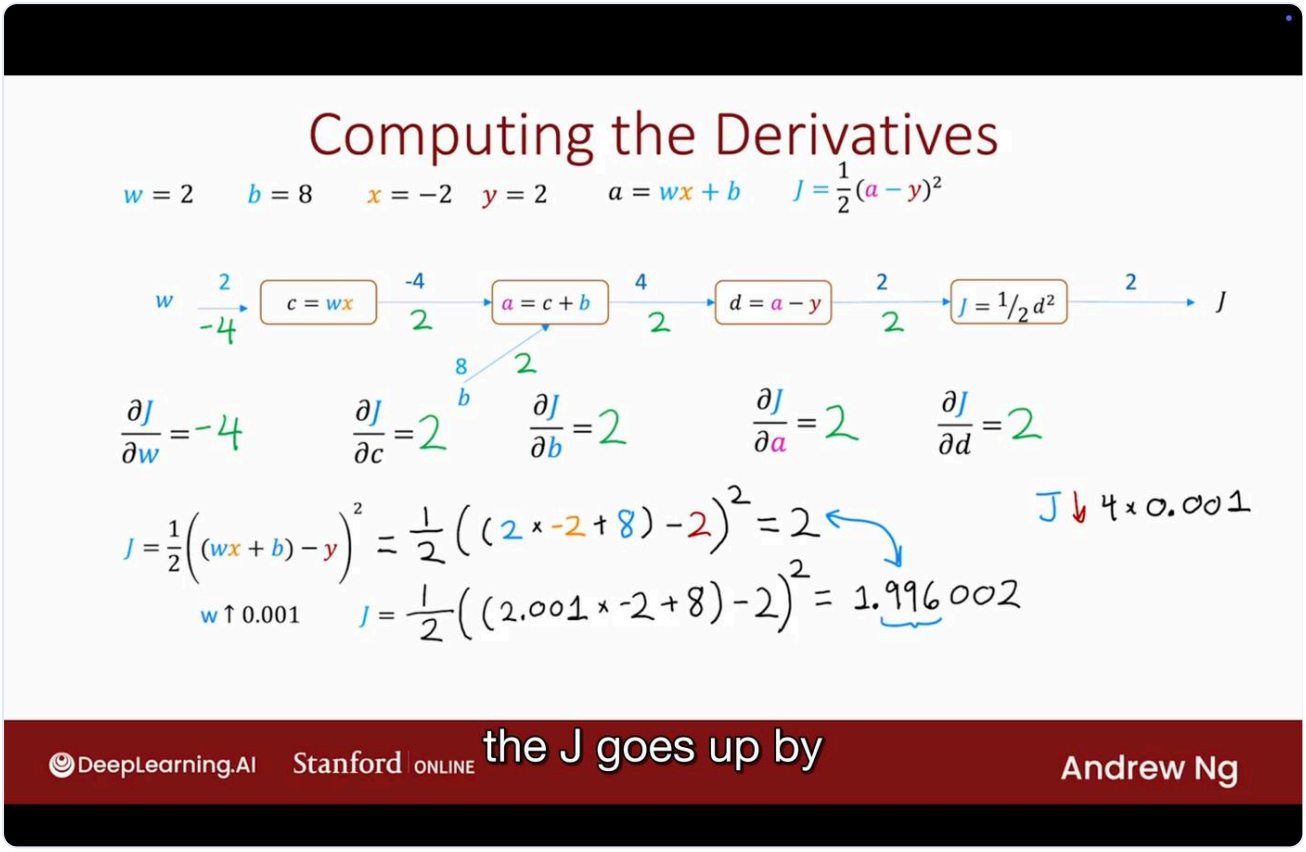


FIGURE 31.12 | Computing the Derivatives

This shows that if W goes up by Epsilon, J goes down by four times Epsilon ball equivalent the J goes up by negative four times Epsilon, which is y.

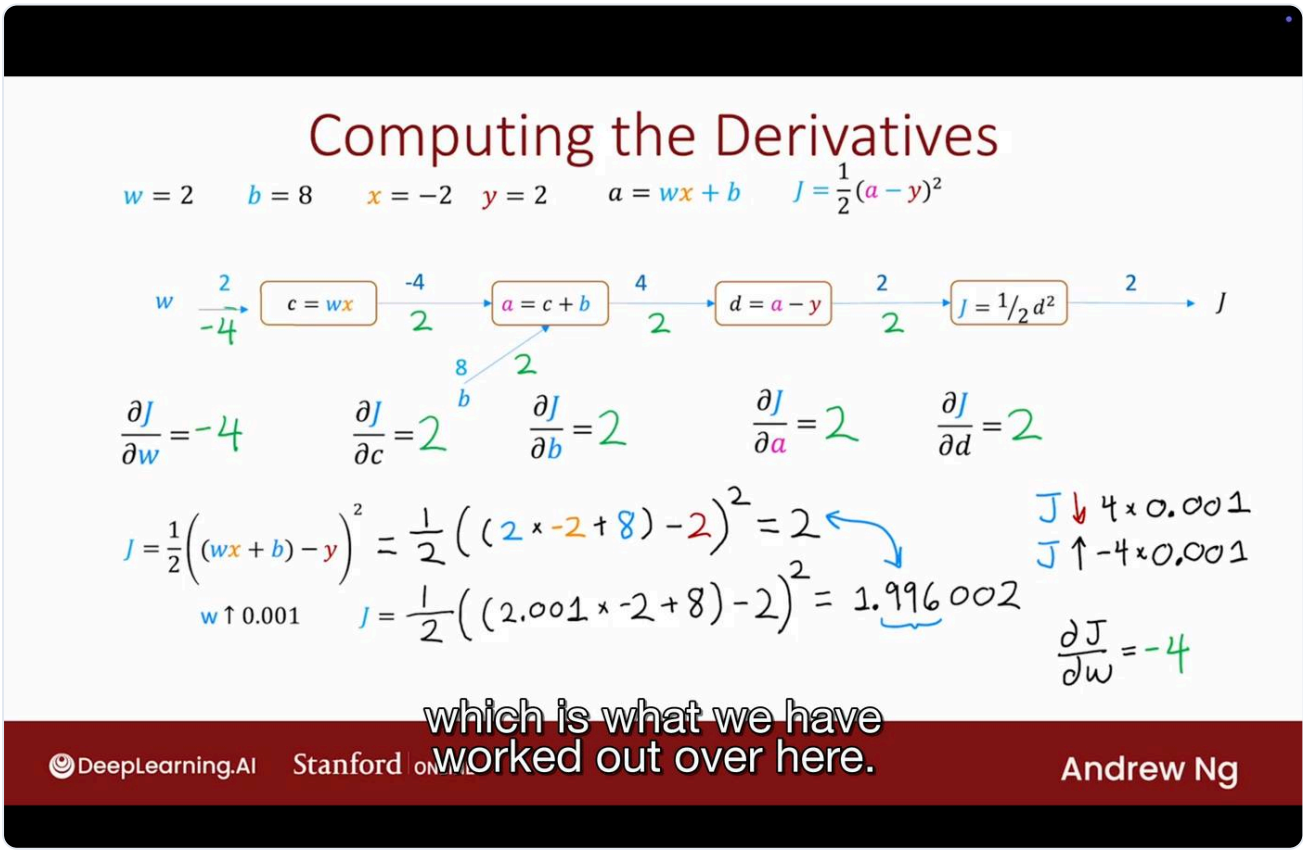


FIGURE 31.13 | Computing the Derivatives

The derivative of j with respect to w is negative 4, which is what we have worked out over here. If you want, feel free to pause the video and double-check this math yourself as well for what happens in b, the other parameter goes up by Epsilon, and hopefully you'll find that the derivative of j with respect to b is indeed two. That b goes up by Epsilon, j goes up by two times Epsilon as predicted by this derivative calculation. Why do we use the backprop algorithm to compute derivatives?

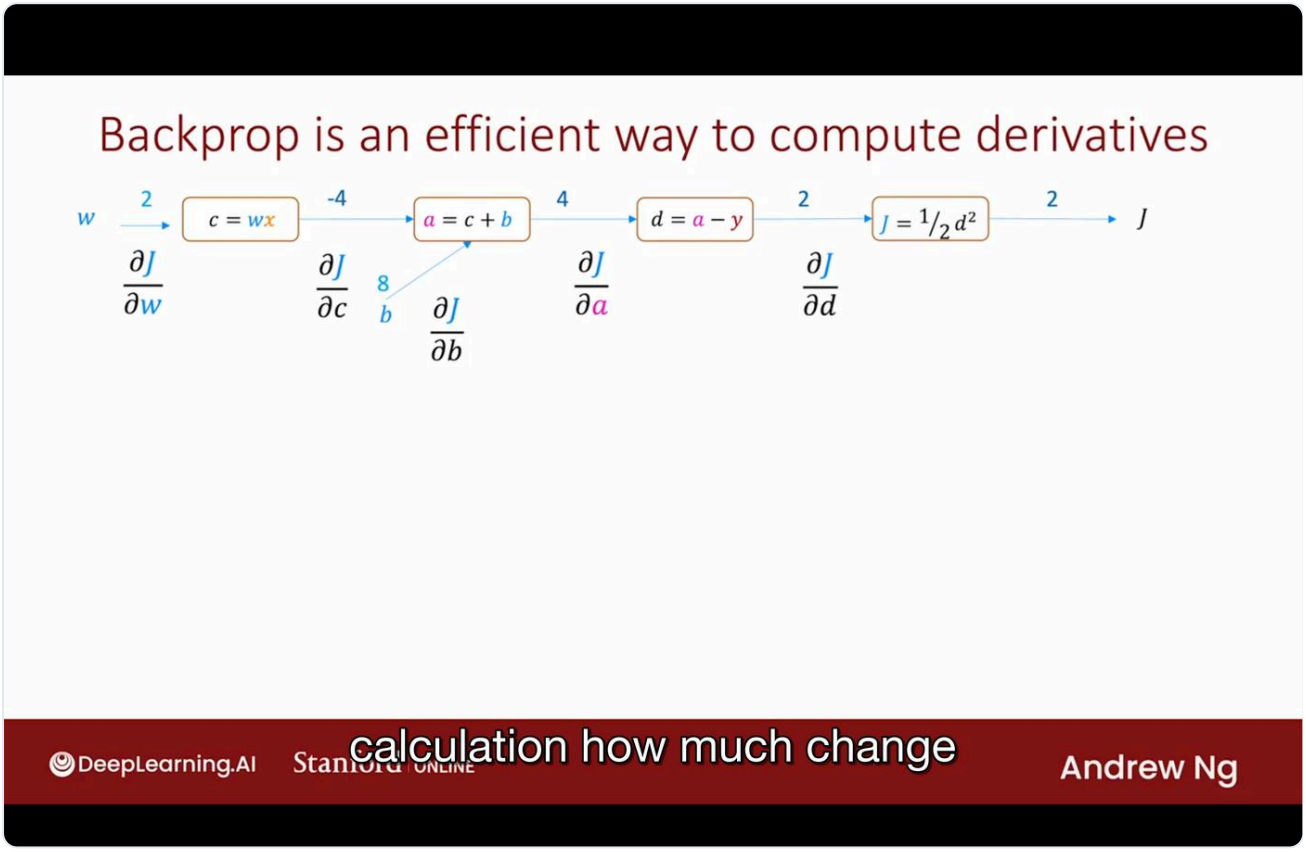


FIGURE 31.14 | Backprop Is an Efficient Way to Compute Derivatives

It turns out that backdrop is an efficient way to compute derivatives. The reason we sequence this as a right-to-left calculation is, if you were to start off and ask what is the derivative of j with respect to w? Then to know how much change in w affects change in j, if w were to go up by Epsilon, how much does j go by Epsilon? Well, the first thing we want to know is, what is the derivative of j with respect to c? Because change in w will change c this first quantity here. To know how much change in w affects j, we want to know how much does change in c affects j. But to know how much change in c affects j, the most useful thing to know to compute this would be change in c changes a.

You want to know how much this change in a effect j and so on. That's why backprop a sequence as a right-to-left calculation. Because if you do the calculation from right to left, you can find out how does change in d affect change in j. Then you can find out how much this change in a effect j and so on. Until you find the derivatives of each of these intermediate quantities, c, a, and d, as well as the parameters w and b. That you can find out with one right-to-left calculation how much change in any of these intermediate quantities, c, a, or d, as well as the input parameters w and b. How much change in any of these things will affect the final output value j.

One thing that makes backprop efficient is you notice that when we do the right-to-left calculation, we had to compute this term, the derivative of j with respect to a just once. This quantity is then used to compute both the derivative of g with respect to w and the derivative of j with respect to b. It turns out that if a computation graph has n nodes, meaning your n of these boxes and p parameters, so we have two parameters in this case.

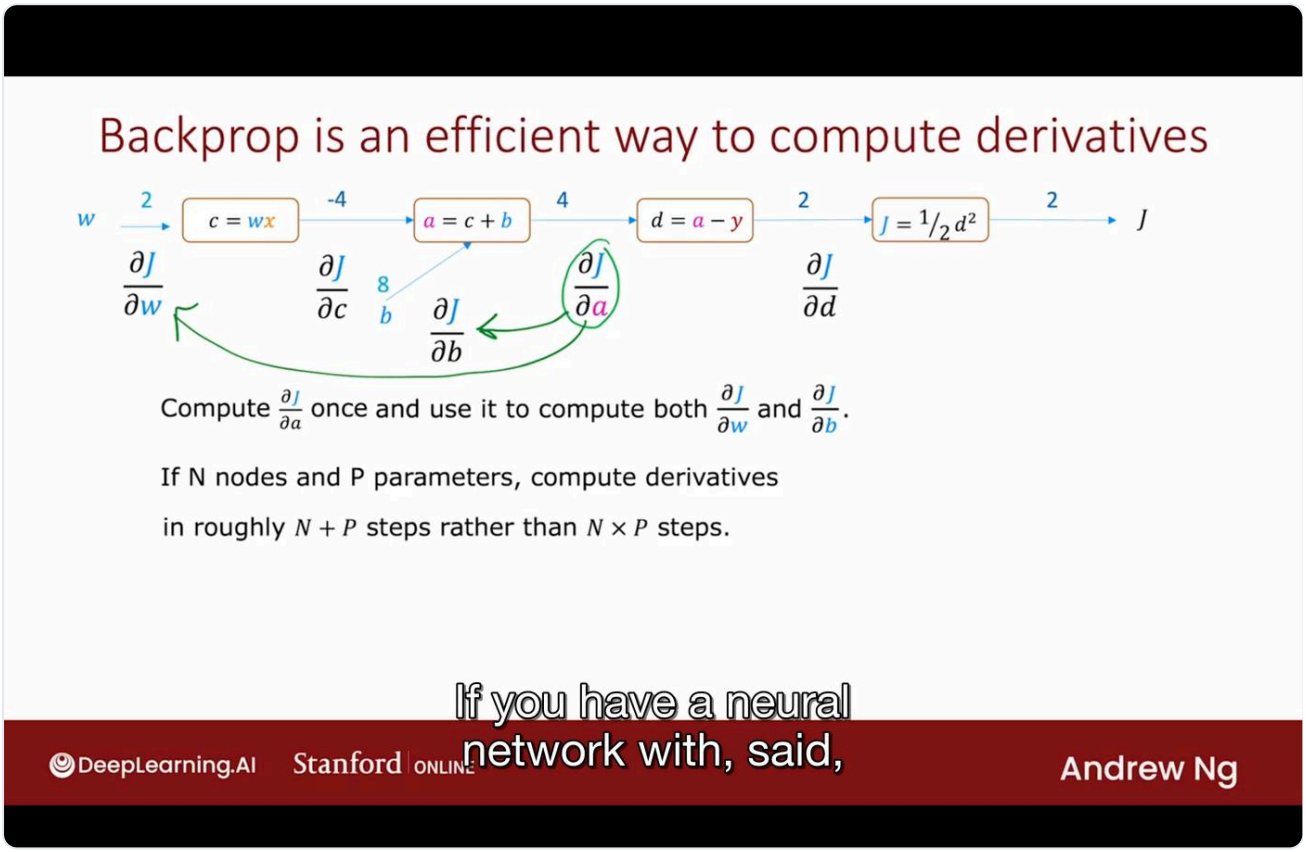


FIGURE 31.15 | Backprop Is an Efficient Way to Compute Derivatives

This procedure allows us to compute all the derivatives of j with respect to all the parameters in roughly n plus p steps, rather than n times p steps. If you have a neural network with, said, 10,000 nodes and maybe 100,000 parameters. This would not be considered even a very large neural network by modern standards. Being able to compute the derivatives and 10,000 plus 100,000 steps, which is punch and 10,000 is much better than meeting 10,000 times 100,000 steps, which would be a billion steps.

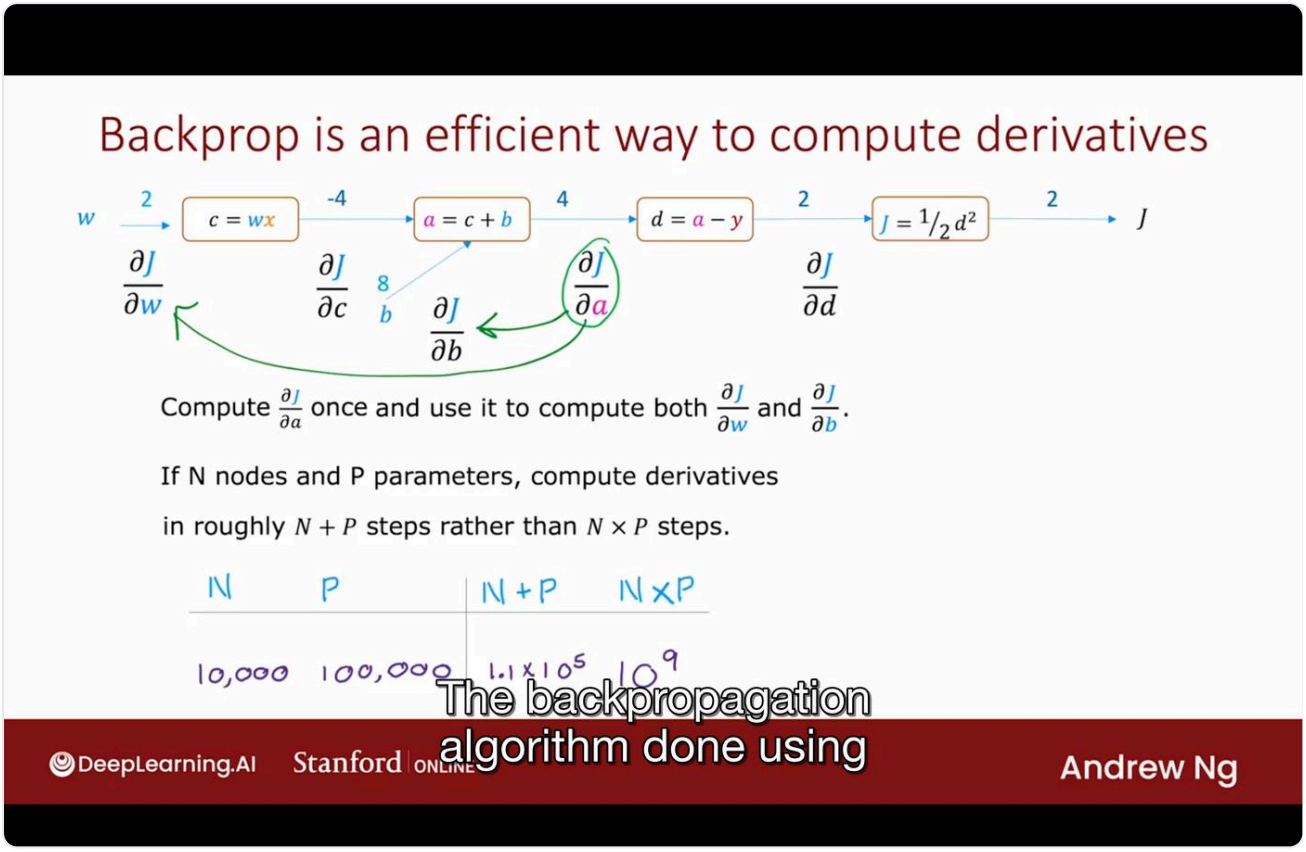


FIGURE 31.16 | Backprop Is an Efficient Way to Compute Derivatives

The backpropagation algorithm done using the computation graph gives you a very efficient way to compute all the derivatives. That's why it is such a key idea in how deep learning algorithms are implemented today. In this video, you saw how the computation graph takes all the steps of the calculation needed to compute the output of a neural network a as well as the cost function j. Takes a step-by-step computations and breaks them into the different nodes of computation graph. Then uses a left-to-right computation for a prop to compute the cost function J. Then a right-to-left or backpropagation calculation to computes all the derivatives. In this video, you saw these ideas apply to a small neural network example.

In the next video, let's take these ideas and apply them to a larger neural network. Let's go on to the next video.

LECTURE 32

Neural Network Example

In this final video on intuition for backprop, let's take a look at how the computation draft works on a larger neural network example. Here's the network we will use with a single hidden layer, with a single hidden unit that outputs a1, that feeds into the output layer that outputs the final prediction a2. To make the math more tractable, I'm going to continue to use just a single training example with inputs x = 1, y = 5. And these will be the parameters of the network. And throughout we're going to use the ReLU activation functions of g(z) = max(0, z). So for prop in your network looks like this. As usual, a1 equals g(w1 times x + b1). And so it turns out w1x + b will be positive.

So we're in the max(0, z) = z, parts of this activation function. So that's just equal to this, which is 2 times 1, that's w1 is 2 times x1 + 0, that's b1, which is equal to 2. And then similarly, a2 equals this, g(w2a1 + b2) which is w2 times a1 + b. Again, because we're in the positive part of the ReLU activation function, which is 3 x 2 + 1 = 7. Finally, we'll use the squared error cost function. So j(w, b) is 1/2(a2- y) squared = 1/2(7-5) squared, which is 1/2 of 2 squared, which is just equal to 2.

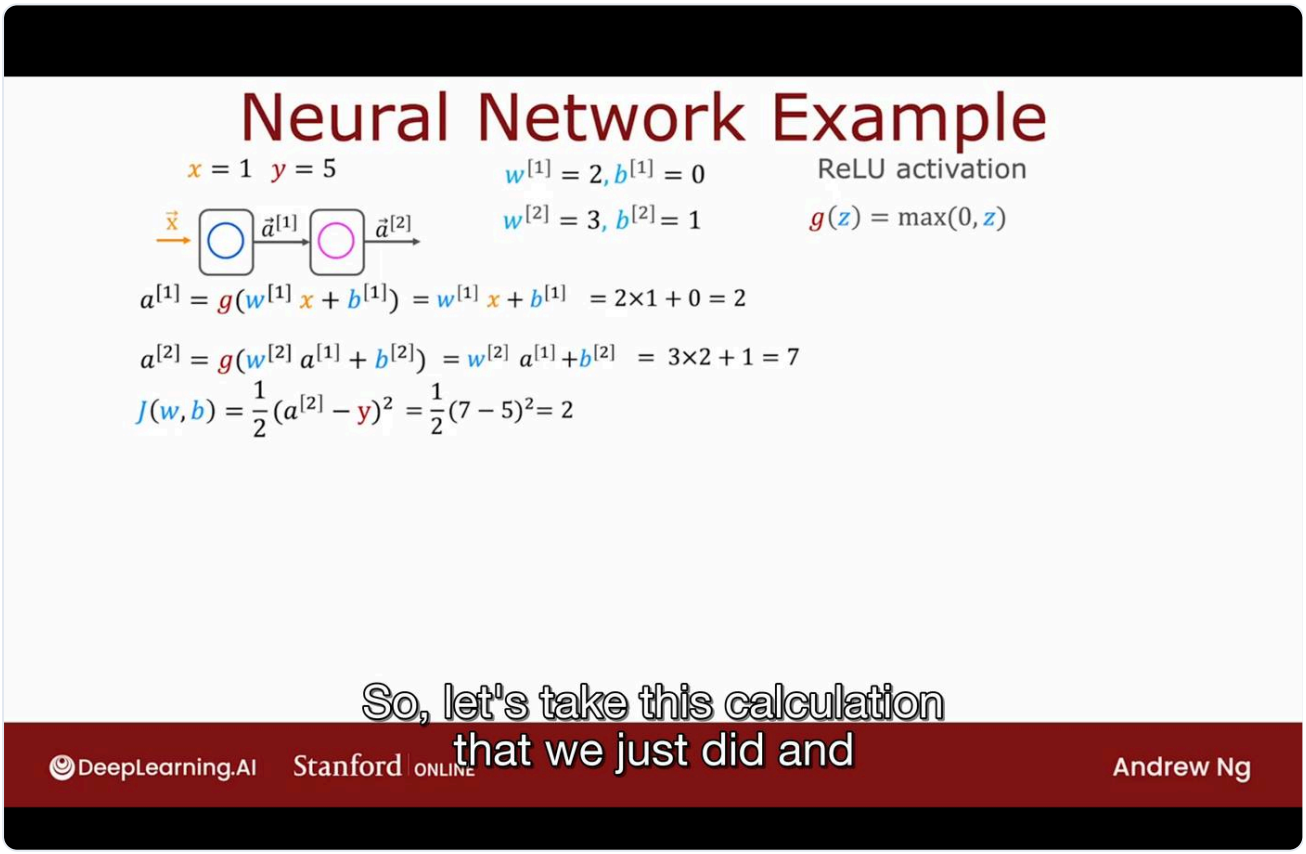


FIGURE 32.1 | Neural Network Example

So, let's take this calculation that we just did and write it down in the form of a computation graph. To carry out the computation step by step, first thing we need to do is take w1 and multiply that by x. So we have w1 that feeds into computation node that computes w1 times x. And I'm going to call this a temporary variable t1. Next, we compute z1, which is this term here, which is

t1 + b1. So we also have this input b1 over here. And finally a1 equals g(z1). We apply the activation function, and so we end up with again this value here, 2. And then next we have to compute t2, which is w2 times a1. And so, with w2, that gives us this value which is 6.

Then z2, which is this quantity, we had to b2 it and that gives us 7. And finally apply the activation function, g. We still end up with 7. And lastly, j is 1/2(a2- y) squared. And that gives us 2.

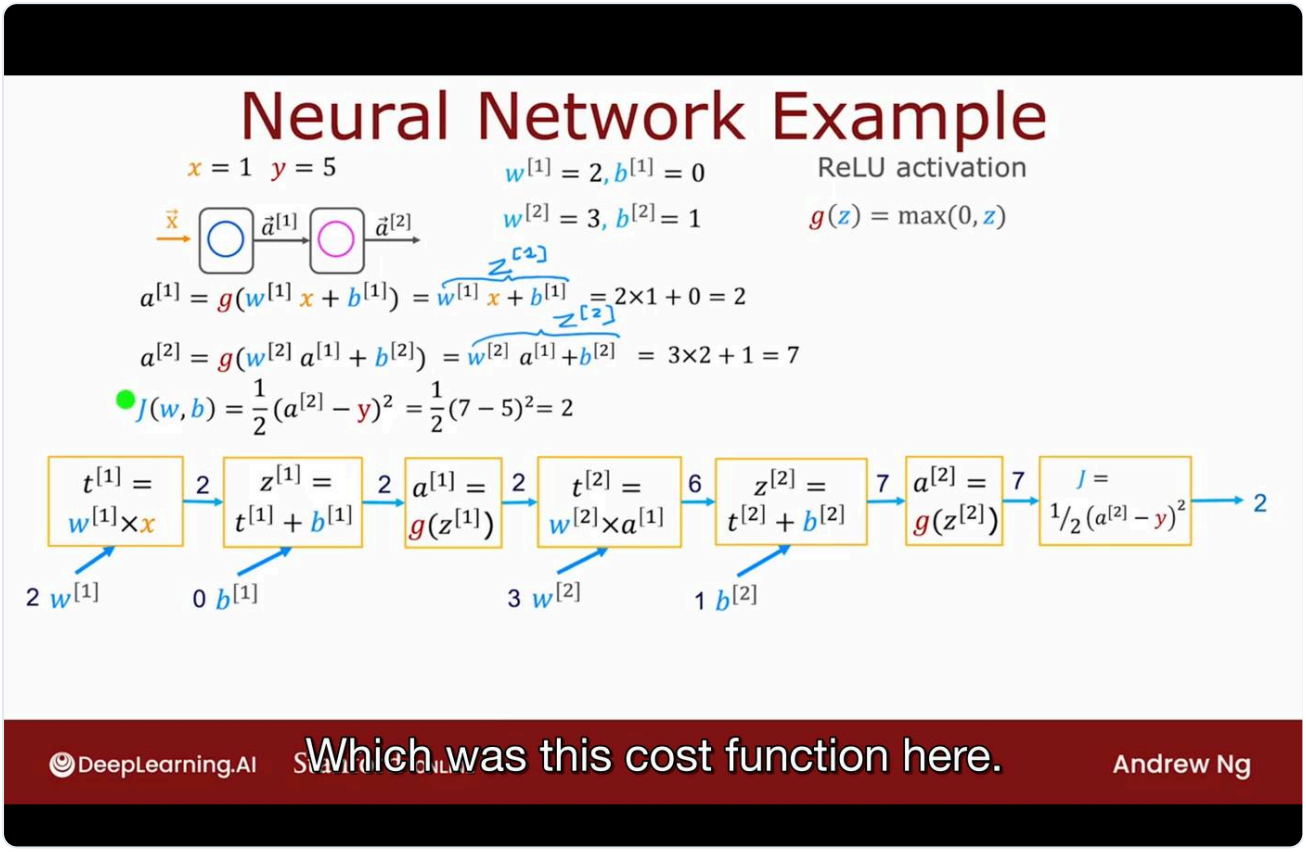


FIGURE 32.2 | Neural Network Example

Which was this cost function here. So this is how you take the step by step computations for larger neural network and write it in the computation graph. You've already seen in the last video, the mechanics of how to carry out backprop. I'm not going to go through the step by step calculations here. But if you were to carry out backprop, the first thing you do is ask, what is the derivative of the cost function j respect to a2? And it turns out if you calculate that, it turns out to be 2. So we'll fill that in here. And the next step will be asked, what's the derivative of the cost j respect to z2. And using this derivative that we computed previously, you can figure out that this turns out to be 2.

Because if z goes up by epsilon, you can show that for the current setting of all the parameters a2 will go up by epsilon. And therefore, j will go up by 2 times epsilon. So this derivative is equal to 2, and so on. Step by step.

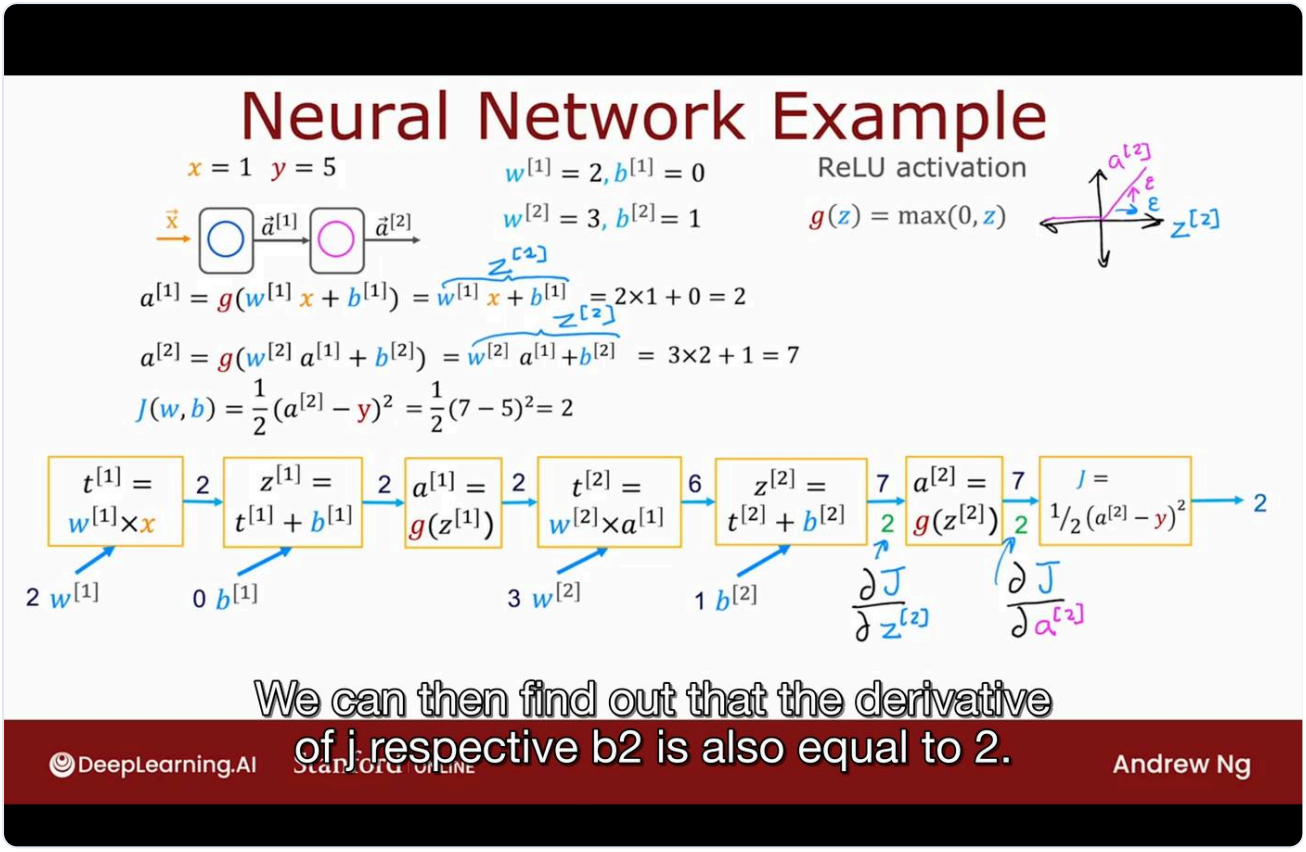


FIGURE 32.3 | Neural Network Example

We can then find out that the derivative of j respective b2 is also equal to 2.

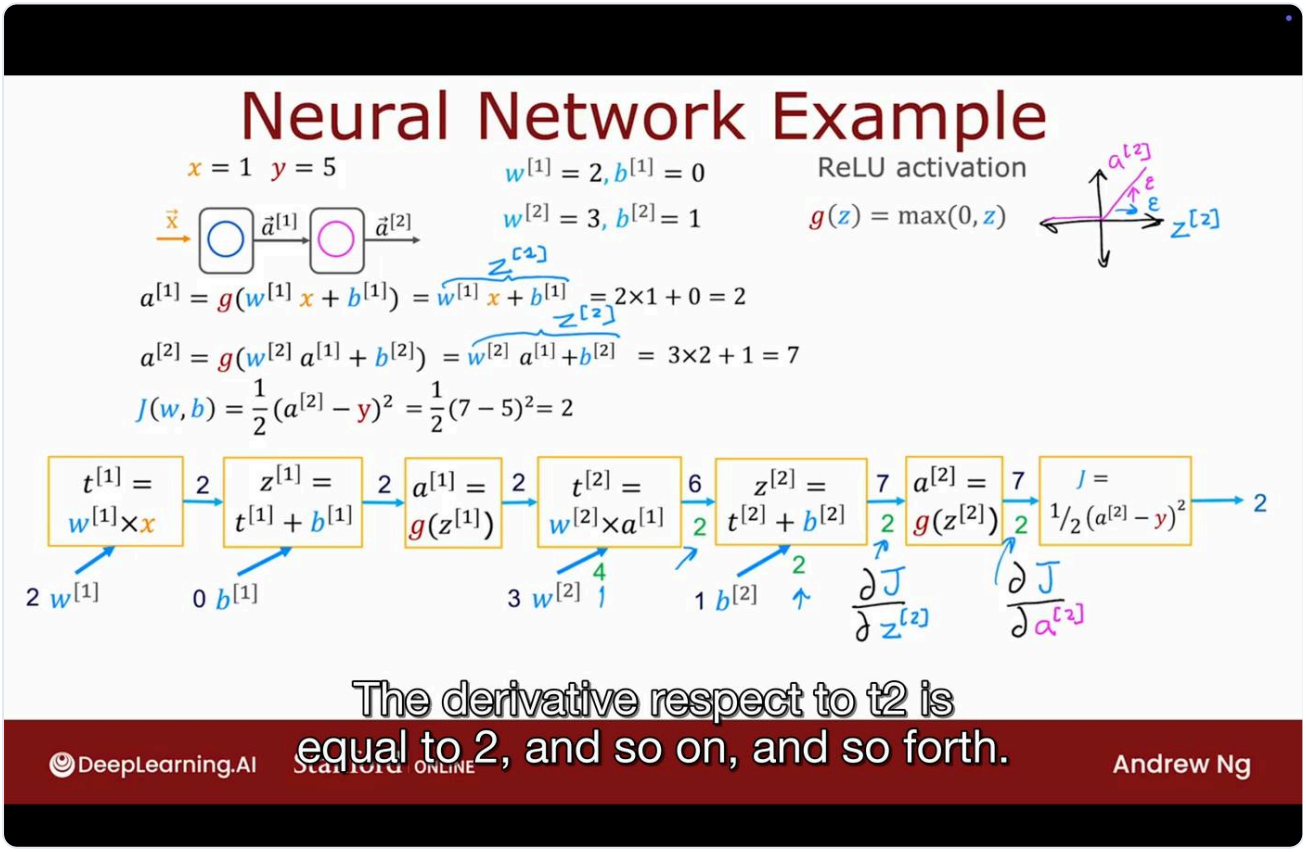


FIGURE 32.4 | Neural Network Example

The derivative respect to t2 is equal to 2, and so on, and so forth.

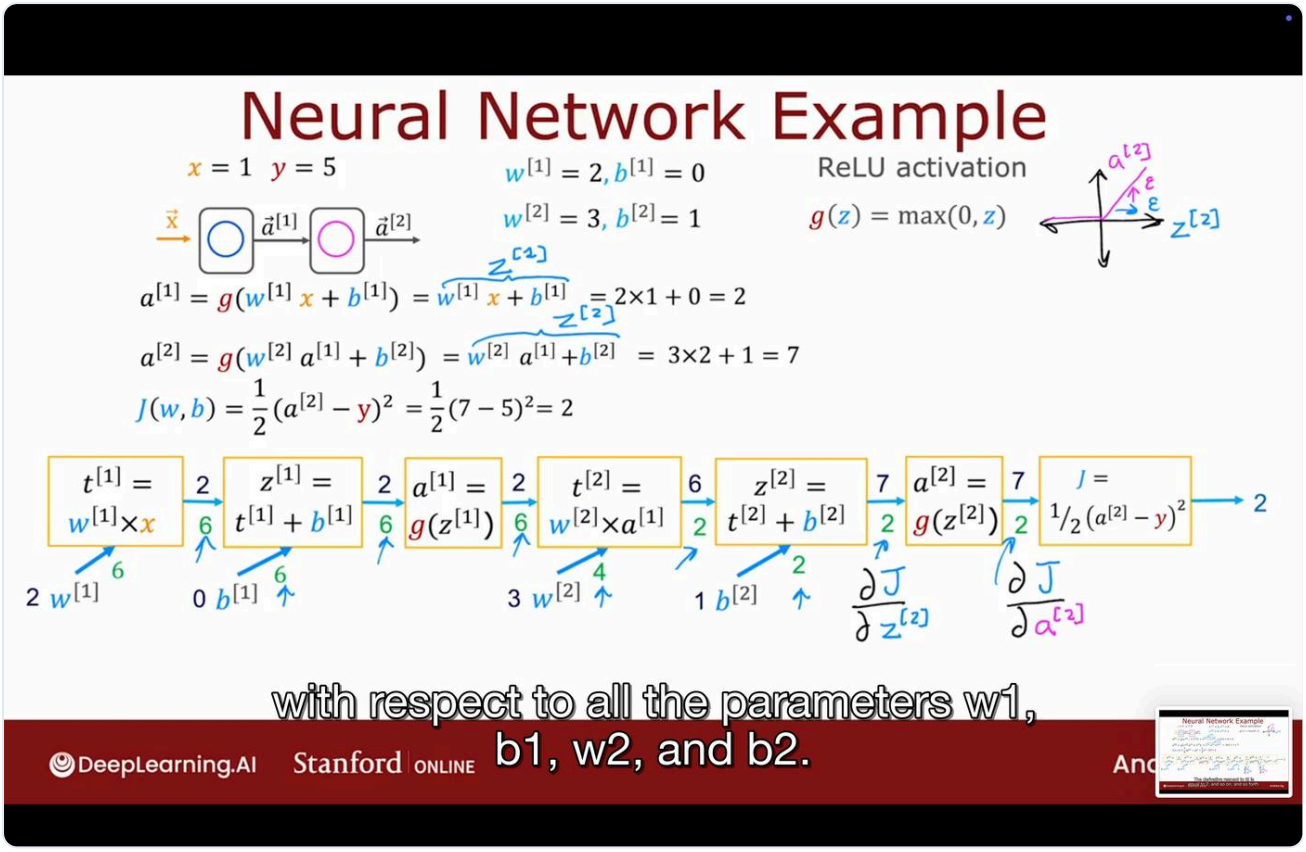


FIGURE 32.5 | Neural Network Example

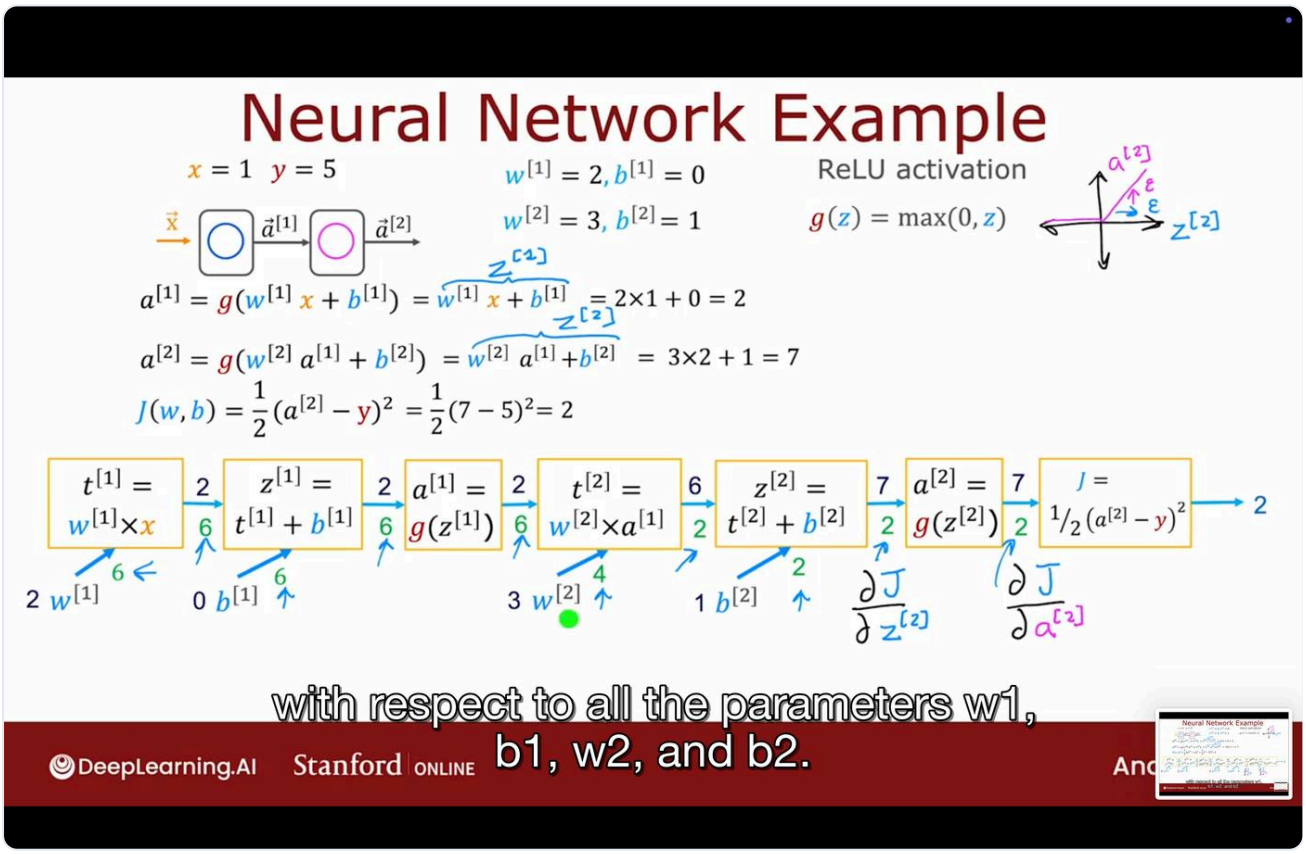


FIGURE 32.6 | Neural Network Example

Until eventually you've computed the derivative of j with respect to all the parameters w1, b1, w2, and b2. And so that's backprop. And again, I didn't go through the mechanical steps of every single step of backprop. But it's basically the process that you saw in the previous video. Let me just double check one of these examples. So we saw here that the derivative of j respect w1 is equal to 6.

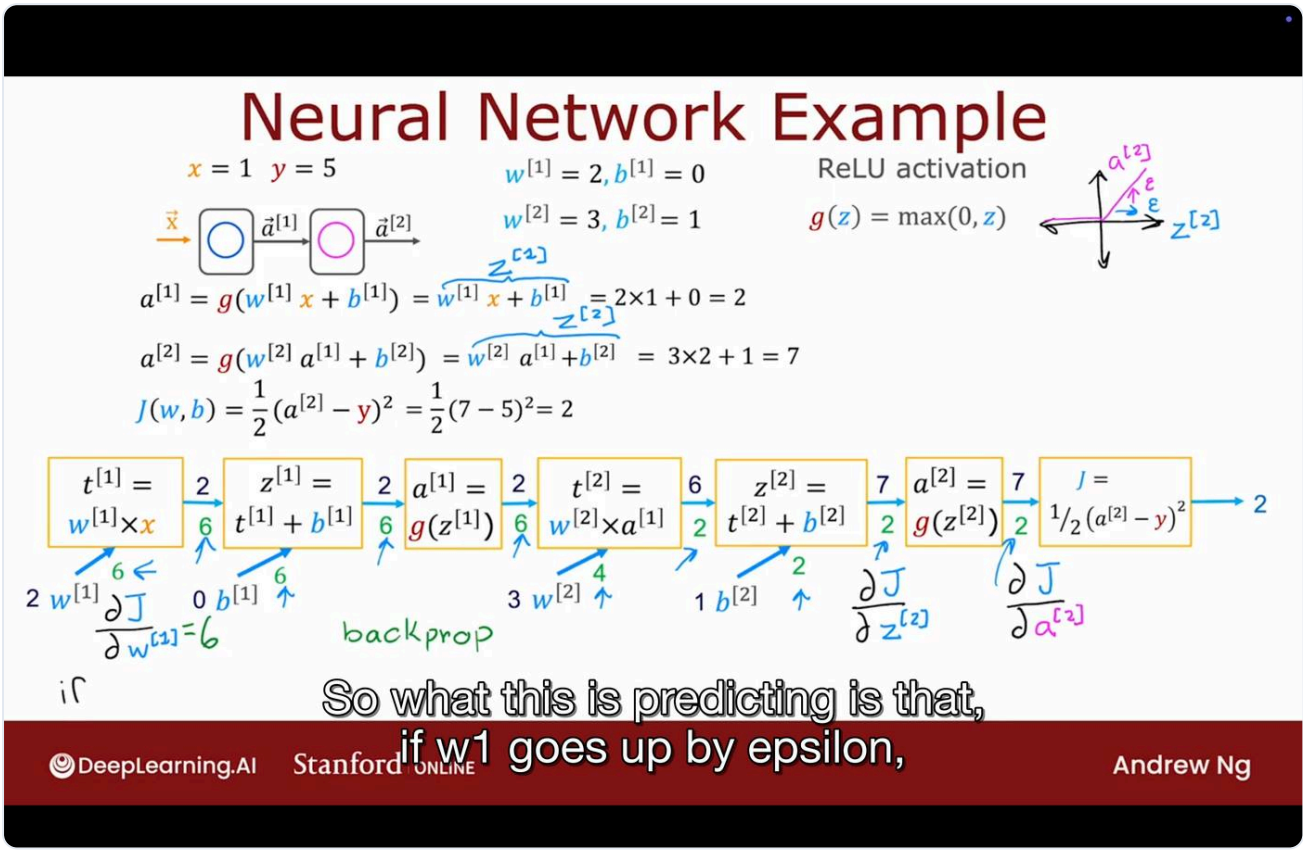


FIGURE 32.7 | Neural Network Example

So what this is predicting is that, if w1 goes up by epsilon, j should go up by roughly 6 times epsilon. Let's step through the map and see if that really is true. These are the calculations that we did, again. And so if w which was 2 were to be 2.001 goes up by epsilon, then a1 becomes, let's see, instead of 2, this is 2.001 as well. So a1 instead of 2 is now 2.001. So 3 x 2.001 + 1, this gives us 7.003. And if a2 is 7.003, then just becomes 7.003- 5 squared. And so this becomes 2.003 squared over 2, which turns out to be equal to 2.006005. So ignoring some of the extra digits, you see from this little calculation that, if w1 goes up by 0.001, j of w has gone up from 2 to 2.006 roughly.

So 6 times as much. And so the derivative of j with respect to w1 is indeed equal to 6. And so the backprop procedure gives you a very efficient way to compute all of these derivatives. Which you can then feed into the gradient descent algorithm or the Adam optimization algorithm, to then train the parameters of your neural network. And again, the reason we use background for this is, is a very efficient way to compute all the derivatives of j respect to w1, j respect to b1, j respect to w2, and j respect to b2.

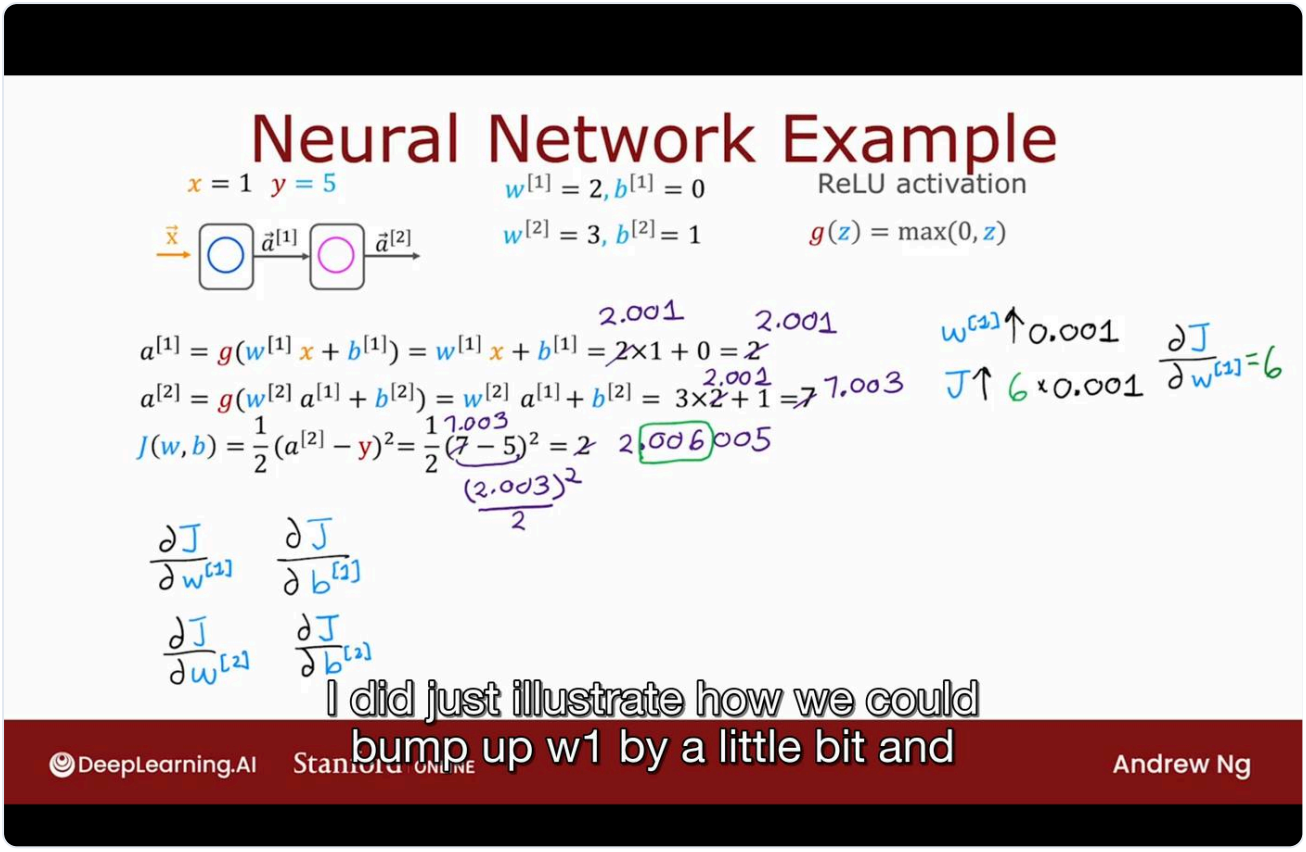


FIGURE 32.8 | Neural Network Example

I did just illustrate how we could bump up w1 by a little bit and see how much j changes. But that was a left to right calculation. And then we had to do this procedure for each parameter, one parameter at a time. If we had to increase w by 0.001 to see how that changes j. Increase b1 by a little bit to see how that changes j, and increase every parameter, one at a time by a little bit to see how that changes j. Then this becomes a very inefficient calculation. And if you had N nodes in your computation graph and P parameters, this procedure would end up taking N times P steps, which is very inefficient.

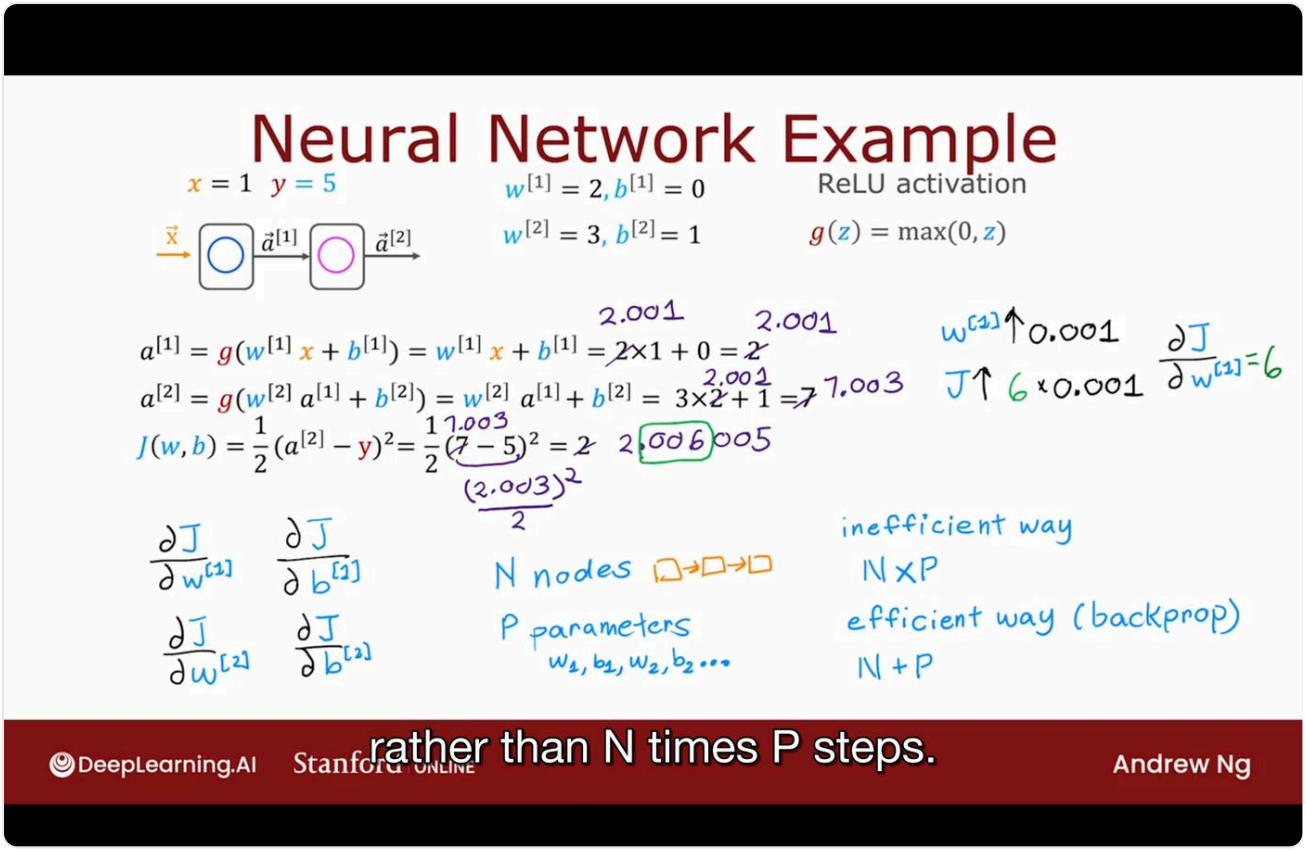


FIGURE 32.9 | Neural Network Example

Whereas we got all four of these derivatives N + P, rather than N times P steps. And this makes a huge difference in practical neural networks, where the number of nodes and the number of parameters can be really large. So, that's the end of the video for this week. Thanks for sticking with me through the end of these optional videos. And I hope that you now have an intuition for when you use a program frameworks, like tensorflow, to train a neural network. What's actually happening under the hood and how is using the computation graph to efficiently compute derivatives for you. Many years ago, before the rise of frameworks like tensorflow and pytorch, researchers used to have to manually use calculus to compute the derivatives of the neural networks that they wanted to train.

And so in modern program frameworks you can specify forwardprop and have it take care of backprop for you. Many years ago, researchers used to write down the neural network by hand, manually use calculus to compute the derivatives. And then neural implement a bunch of equations that they had laboriously derived on paper, to implement backprop. Thanks to the computation graph and these techniques for automatically carrying out derivative calculations. Is sometimes called autodiff, for automatic differentiation. This process of researchers manually using calculus to take derivatives is no longer really done. At least, I've not had to do this for many years now myself, because of autodiff.

So, many years ago, to use neural networks, the bar for the amount of calculus you have to know actually used to be higher. But because of automatic differentiation algorithms, usually based on the computation graph, you can now implement a neural network and get derivatives computed for you easier than before. So maybe with the maturing of neural networks, the amount of calcu-

lus you need to know in order to get these algorithms work, has actually gone down. And that's been encouraging for a lot of people. And so, that's it for the videos for this week. I hope you enjoy the labs and I look forward to seeing you next week.

LECTURE 33

Debugging a Learning Algorithm

Hi, and welcome back. By now you've seen a lot of different learning algorithms, including linear regression, logistic regression, even deep learning, or neural networks, and next week, you'll see decision trees as well. You now have a lot of powerful tools of machine learning, but how do you use these tools effectively? I've seen teams sometimes, say six months to build a machine learning system, that I think a more skilled team could have taken or done in just a couple of weeks. The efficiency of how quickly you can get a machine learning system to work well, will depend to a large part on how well you can repeatedly make good decisions about what to do next in the course of a machine learning project.

In this week, I hope to share with you a number of tips on how to make decisions about what to do next in machine learning project, that I hope will end up saving you a lot of time. Let's take a look at some advice on how to build machine learning systems. Let's start with an example, say you've implemented regularized linear regression to predict housing prices, so you have the usual cost function for your learning algorithm, squared error plus this regularization term.

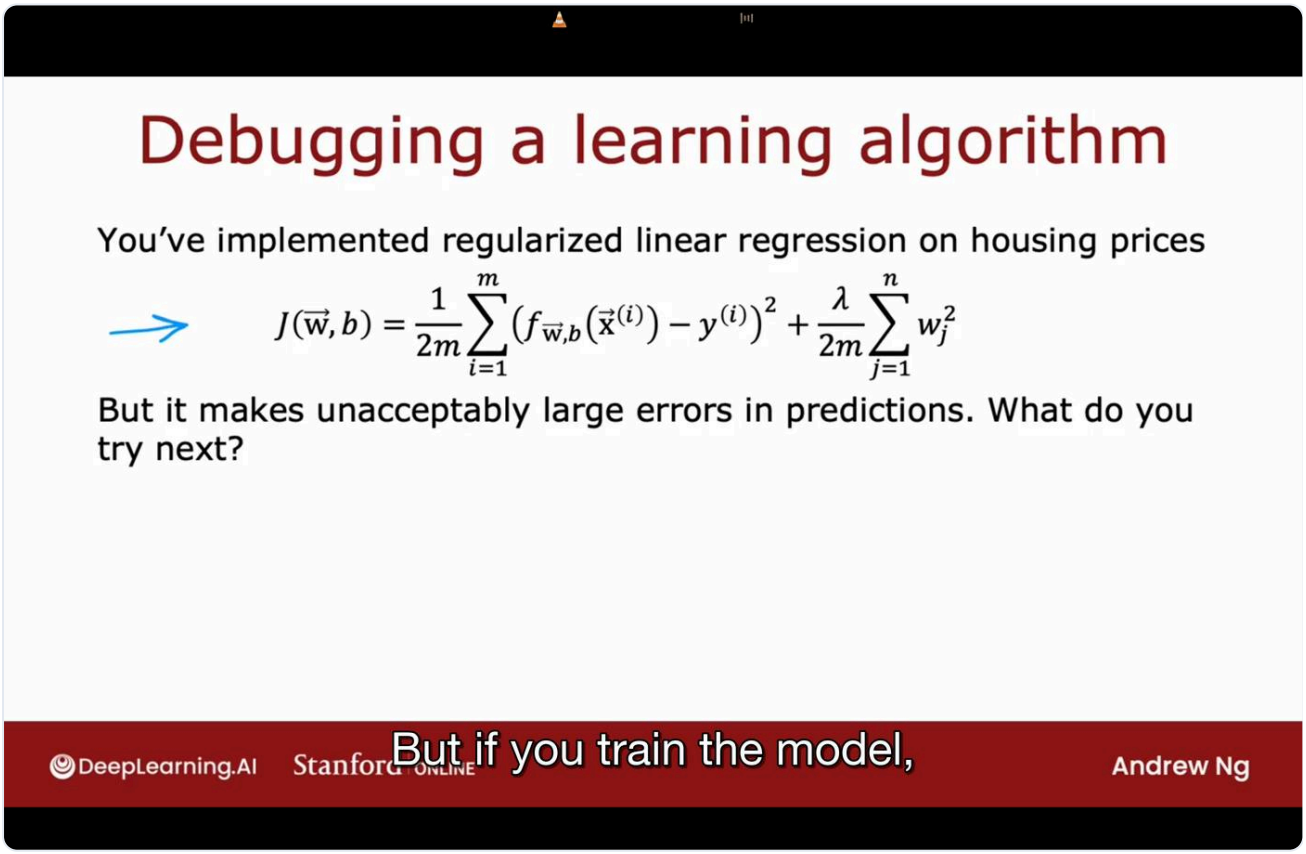


FIGURE 33.1 | Debugging a Learning Algorithm

But if you train the model, and find that it makes unacceptably large errors in it's predictions, what do you try next? When you're building a machine learning algorithm, there are usually a lot of different things you could try.

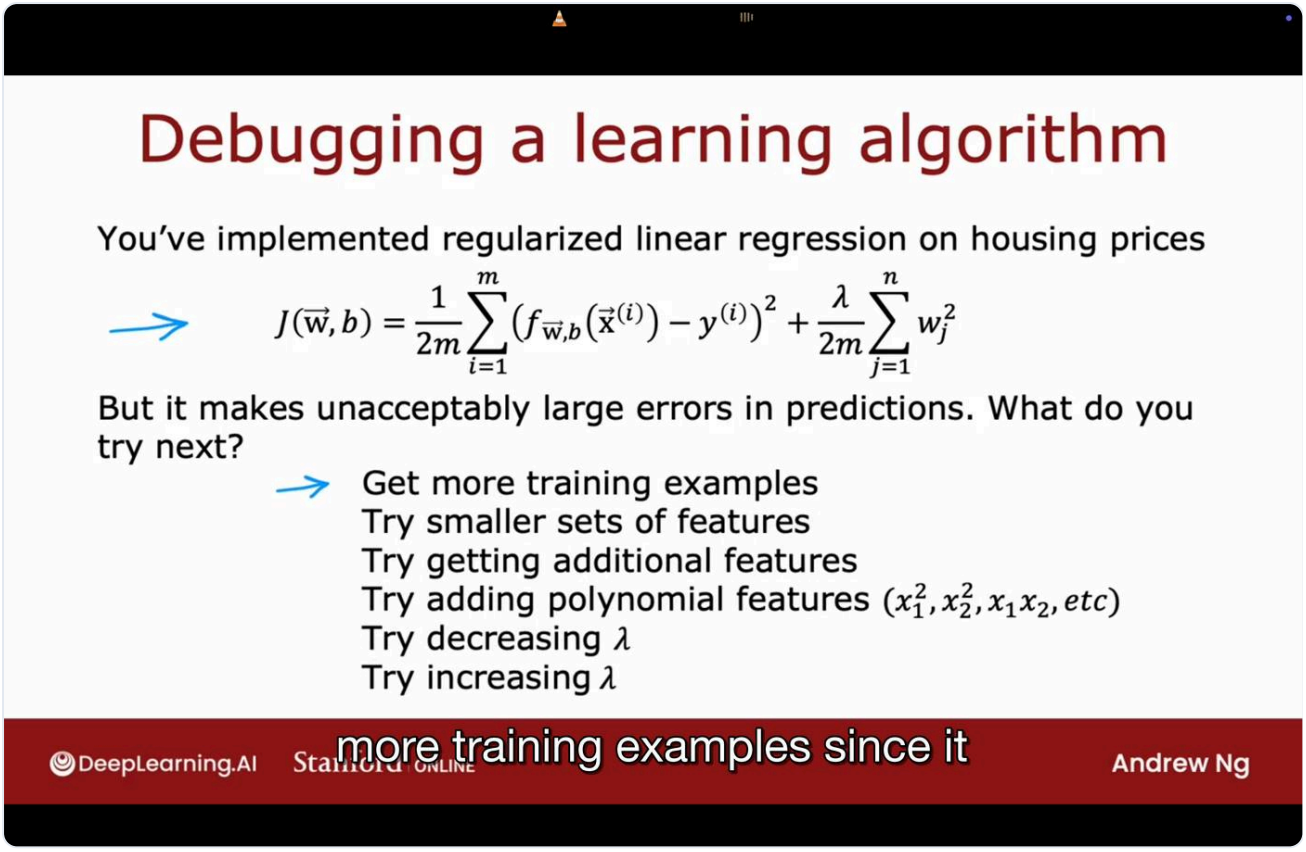


FIGURE 33.2 | Debugging a Learning Algorithm

For example, you could decide to get more training examples since it seems having more data should help, or maybe you think maybe you have too many features, so you could try a smaller set of features. Or maybe you want to get additional features, such as finally additional properties of the houses to toss into your data, and maybe that'll help you to do better. Or you might take the existing features x_1, x_2, and so on, and try adding polynomial features x_1 squared, x_2 squared, x_1, x_2, and so on. Or you might wonder if the value of Lambda is chosen well, and you might say, maybe it's too big, I want to decrease it. Or you may say, maybe it's too small, I want to try increasing.

On any given machine learning application, it will often turn out that some of these things could be fruitful, and some of these things not fruitful. The key to being effective at how you build a machine learning algorithm will be if you can find a way to make good choices about where to invest your time. For example, I have seen teams spend literally many many months collecting more training examples, thinking that more training data is going to help, but it turns out sometimes it helps a lot, and sometimes it doesn't. In this week, you'll learn about how to carry out a set of diagnostic.

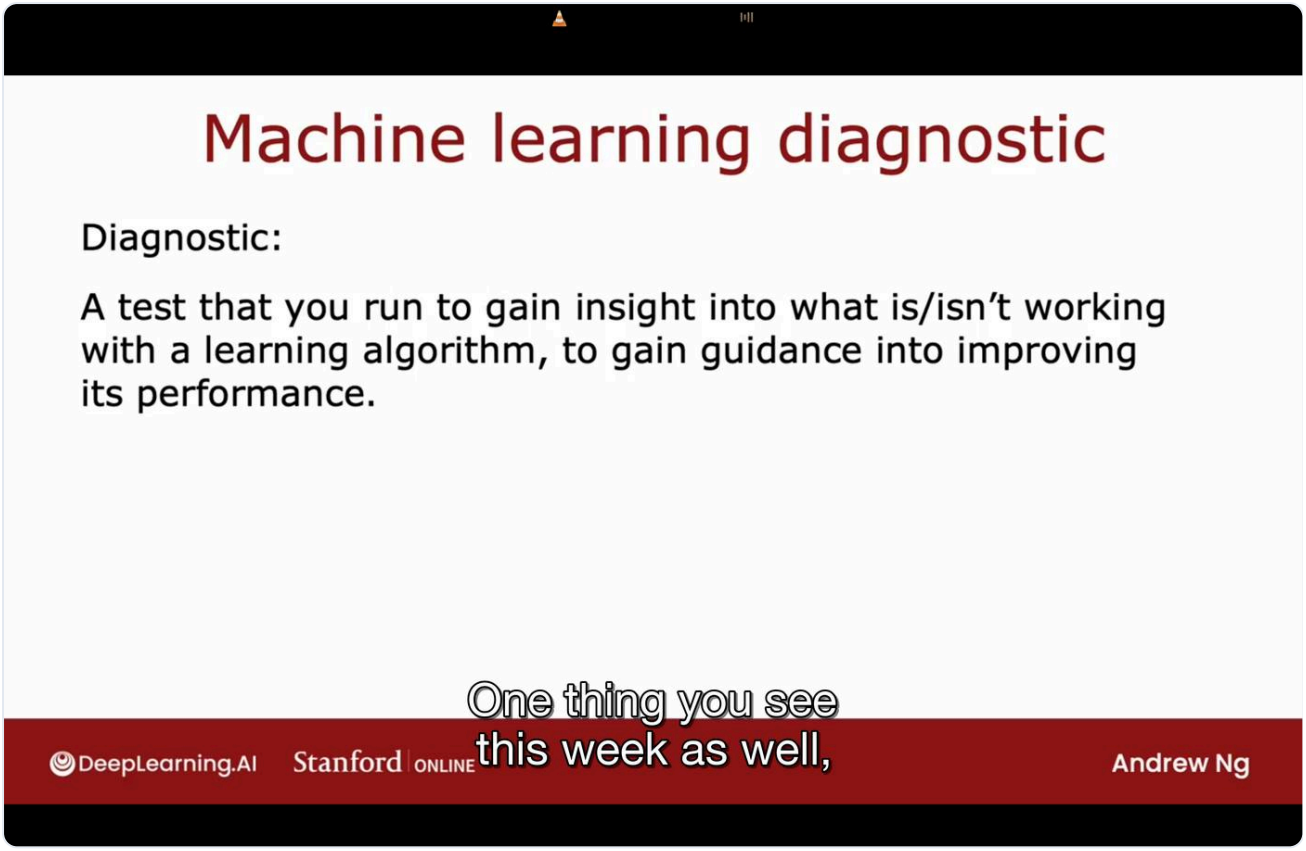


FIGURE 33.3 | Machine Learning Diagnostic

By diagnostic, I mean a test that you can run to gain insight into what is or isn't working with learning algorithm to gain guidance into improving its performance. Some of these diagnostics will tell you things like, is it worth weeks, or even months collecting more training data, because if it is, then you can then go ahead and make the investment to get more data, which will hopefully lead to improved performance, or if it isn't then running that diagnostic could have saved you months of time.

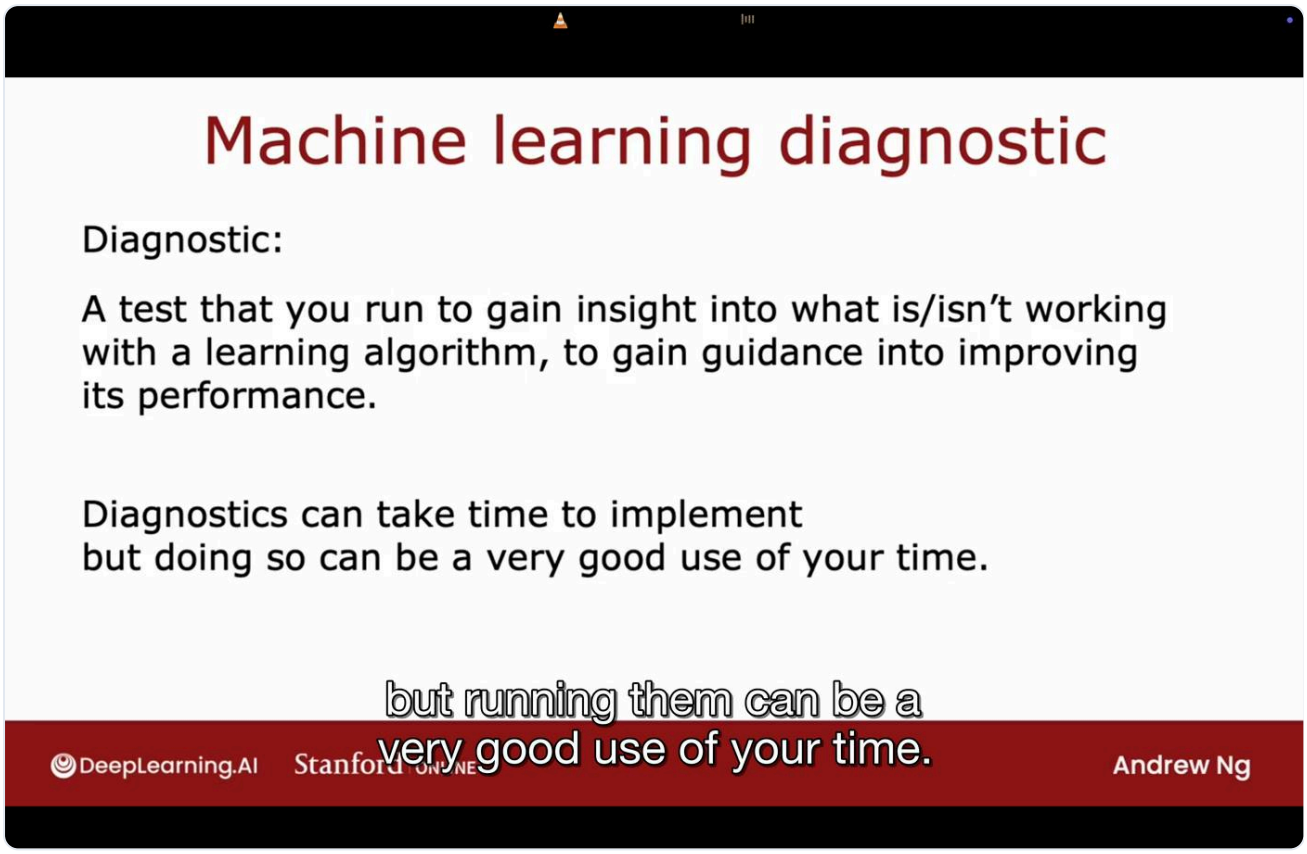


FIGURE 33.4 | Machine Learning Diagnostic

One thing you see this week as well, is that diagnostics can take time to implement, but running them can be a very good use of your time. This week we'll spend a lot of time talking about different diagnostics you can use, to give you guidance on how to improve your learning algorithm's performance. But first, let's take a look at how to evaluate the performance of your learning algorithm. Let's go do that in the next video.

LECTURE 34

Train/test Procedure for Linear Regression

Let's see, you've trained a machine learning model. How do you evaluate that model's performance? You find that having a systematic way to evaluate performance will also hope paint a clearer path for how to improve its performance. So let's take a look at how to evaluate the model. Let's take the example of learning to predict housing prices as a function of the size. Let's say you've trained the model to predict housing prices as a function of the size x. And for the model that is a fourth order polynomial. So features x, x squared, execute and x to the 4. Because we fit 1/4 order polynomial to a training set with five data points, this fits the training data really well.

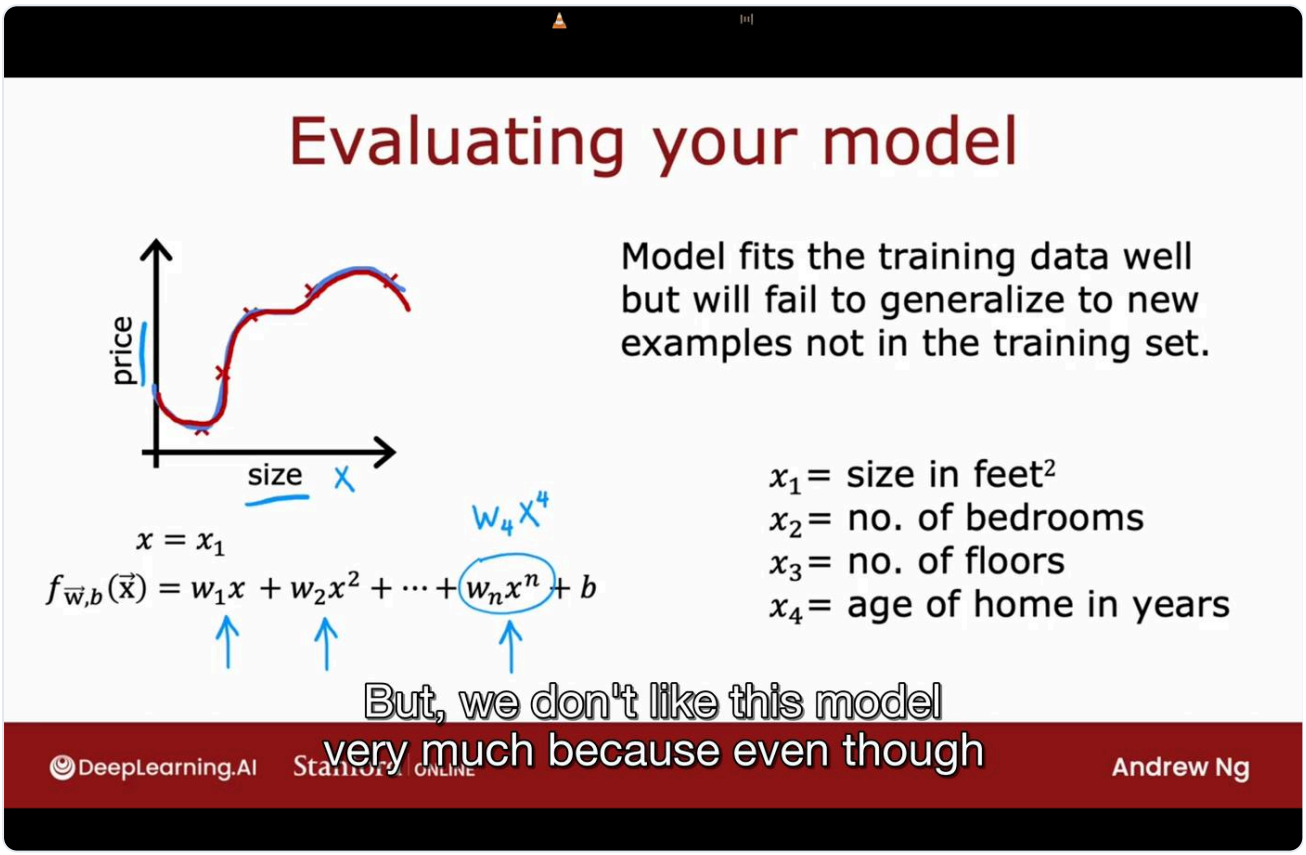


FIGURE 34.1 | Model Fits the Training Data Well

But, we don't like this model very much because even though the model fits the training data well, we think it will fail to generalize to new examples that aren't in the training set.

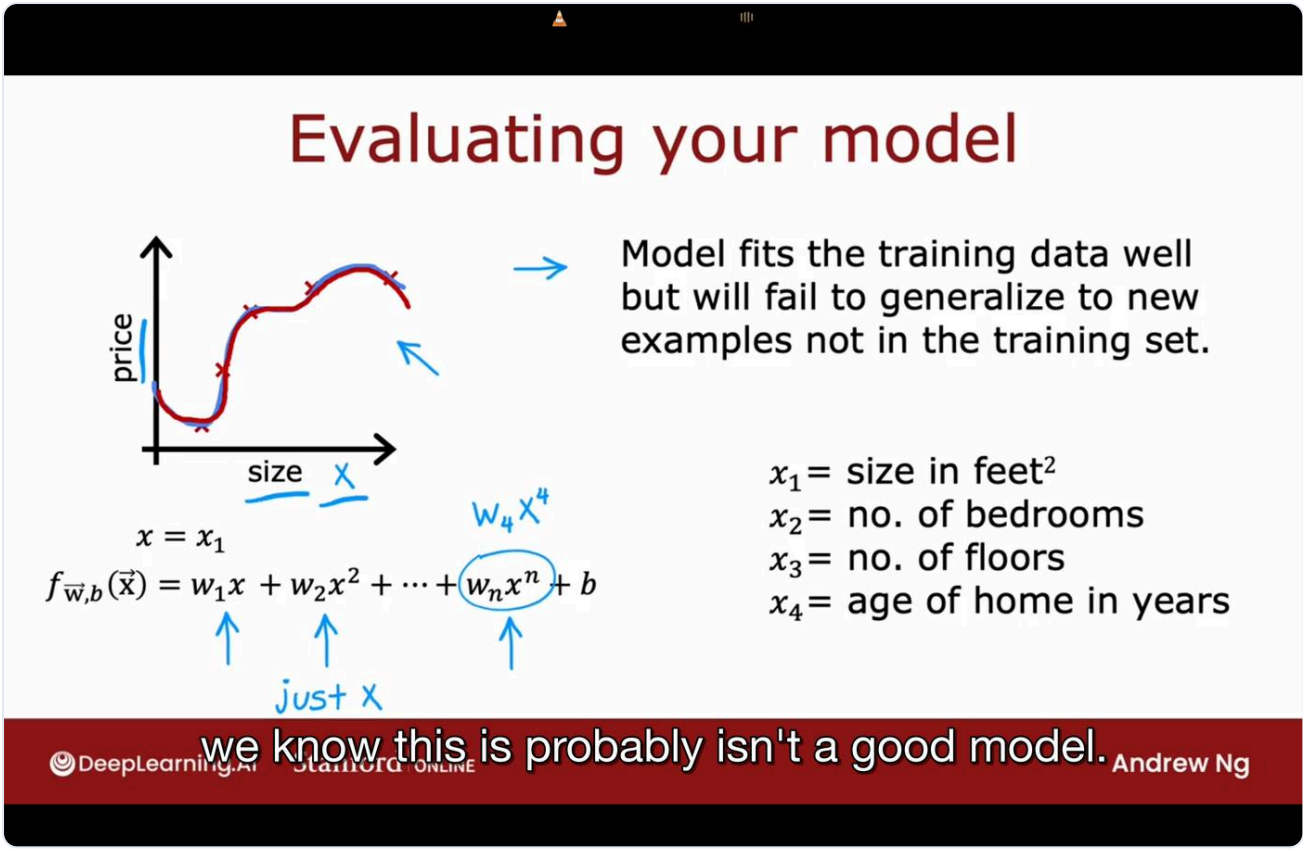


FIGURE 34.2 | S Examples Not in the Training Set

So, when you are predicting prices, just a single feature at the size of the house, you could plot the model like this and we could see that the curve is very wiggly so we know this is probably isn't a good model. But if you were fitting this model with even more features, say we had x1 the size of house, number of bedrooms, the number of floors of the house, also the age of the home in years, then it becomes much harder to plot f because f is now a function of x1 through x4. And how do you plot a four dimensional function? So in order to tell if your model is doing well, especially for applications where you have more than one or two features, which makes it difficult to plot f of x.

We need some more systematic way to evaluate how well your model is doing. Here's a technique that you can use. If you have a training set and this is a small training set with just 10 examples listed here, rather than taking all your data to train the parameters w and p of the model, you can instead split the training set into two subsets. I'm going to draw a line here, and let's put 70% of the data into the first part and I'm going to call that the training set. And the second part of the data, let's say 30% of the data, I'm going to put into it a test set.

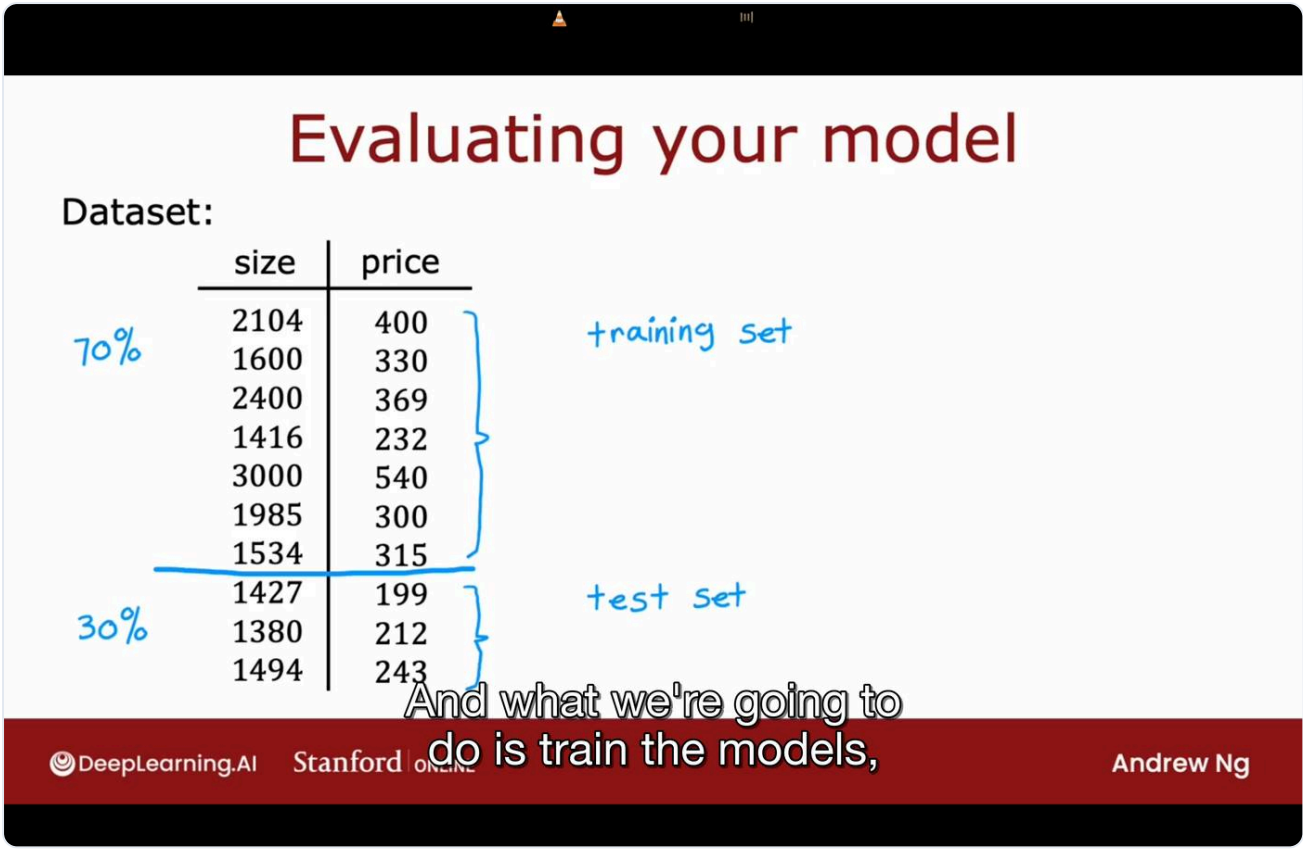


FIGURE 34.3

And what we're going to do is train the models, parameters on the training set on this first 70% or so of the data, and then we'll test its performance on this test set. In notation, I'm going to use x1, y1? Same as before, to denote the training examples through xm, ym, except that now to make explicit. So in this little example we would have seven training example. And to introduce one new piece of notation, I'm going to use m subscript train. M train is a number of training examples which in this small dataset is 7. So the subscript train just emphasizes if we're looking at the training set portion of the data.

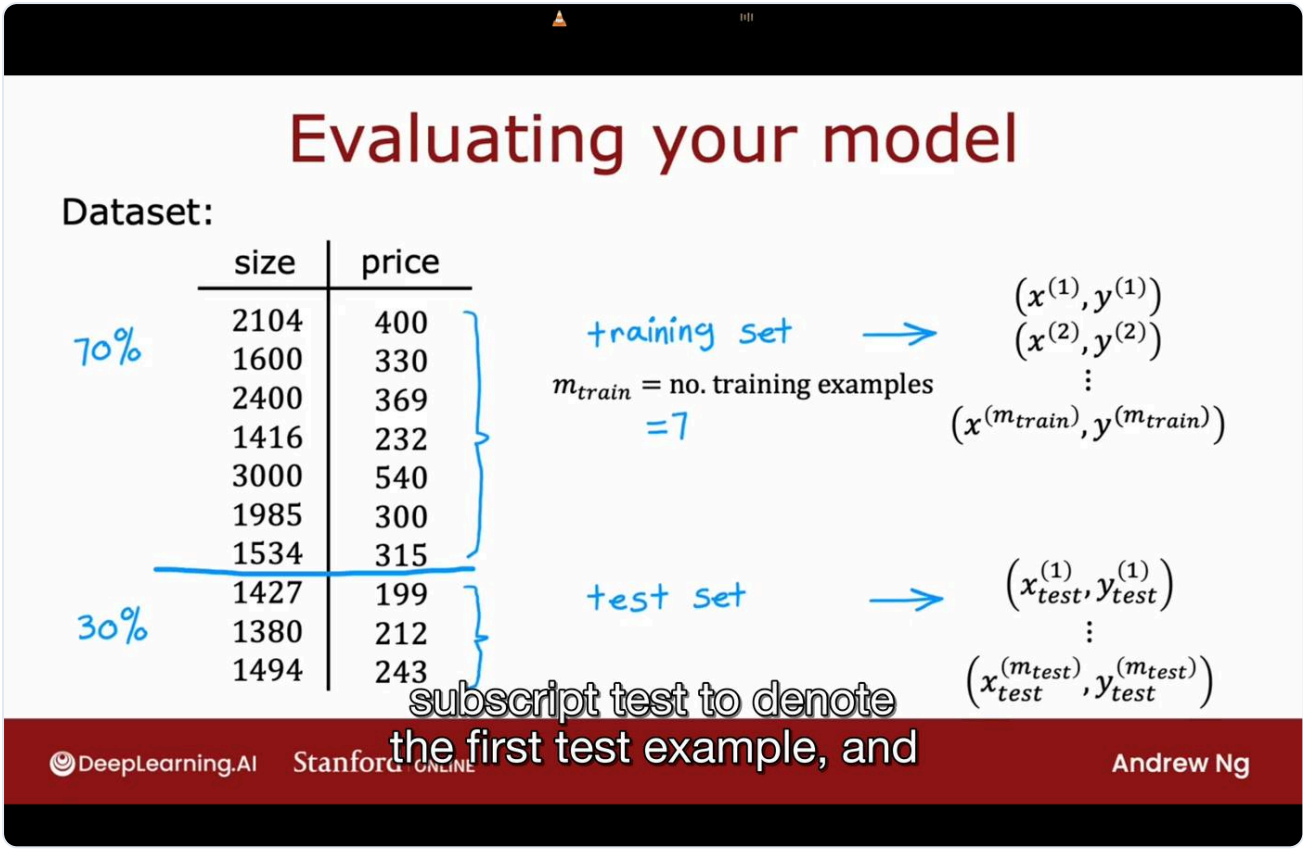


FIGURE 34.4

And for the test set, I'm going to use the notation x1 subscript test comma y1, subscript test to denote the first test example, and this goes all the way to x mtest subscript tests, y mtest subscript tests and m tests is the number of test examples, which in this case is 3. And it's not uncommon to split your dataset according to maybe a 70, 30 split or 80, 20 split with most of your data going into the training set, and then a smaller fraction going into the test set. So, in order to train a model and evaluated it, this is what it would look like if you're using linear regression with a squared error cost. Start off by fitting the parameters by minimizing the cost function j of w,b.

So this is the usual cost function minimize over w,b of this square error cost, plus regularization term lambda over 2m times some of the w,j squared.

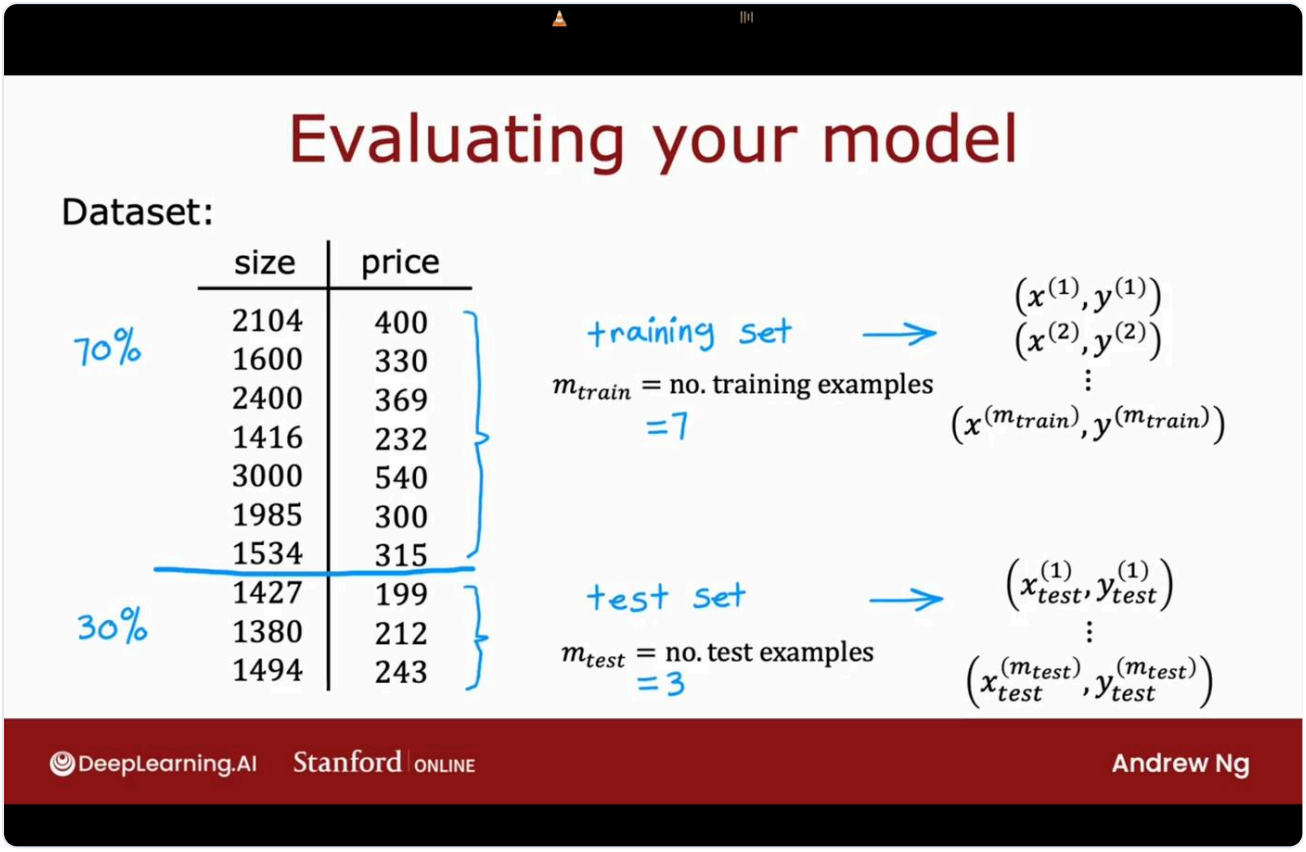


FIGURE 34.5

And then to tell how well this model is doing, you would compute J test of w,b, which is equal to the average error on the test set, and that's just equal to 1/2 times m test. That's the number of test examples. And then of some overall the examples from r equals 1, to the number of test examples of the squared era on each of the test examples like so. So it's a prediction on the i'th test example input minus the actual price of the house on the test example squared. And notice that the test error formula J test, it does not include that regularization term.

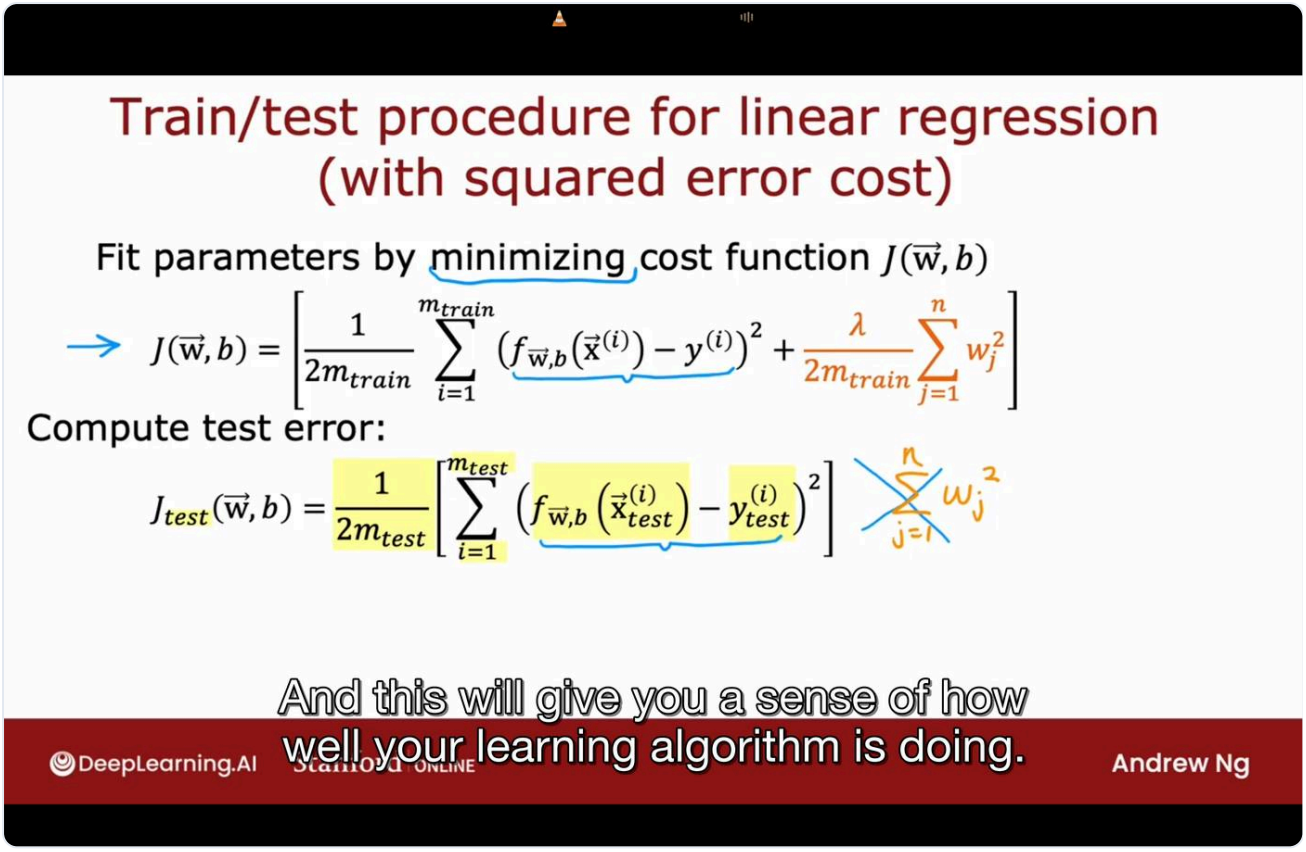


FIGURE 34.6 | Train/test Procedure for Linear Regression

And this will give you a sense of how well your learning algorithm is doing. One of the quantity that's often useful to computer as well as the training error, which is a measure of how well you're learning algorithm is doing on the training set. So let me define J train of w,b to be equal to the average over the training set. 1 over to 2m, or 1/2 m subscript train of some over your training set of this squared error term.

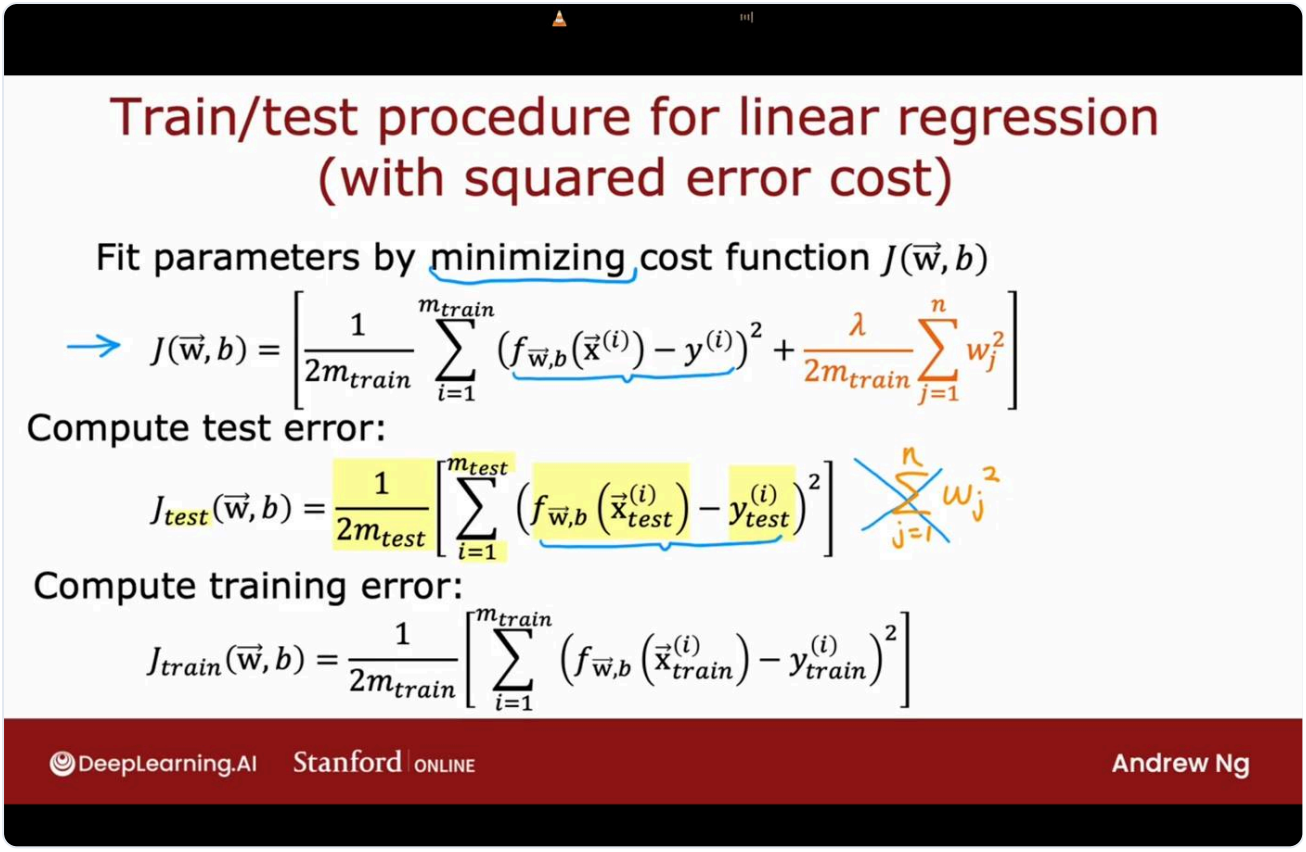


FIGURE 34.7 | Train/test Procedure for Linear Regression

And once again, this does not include the regularization term unlike the cost function that you are minimizing to fit the parameters. So, in the model like what we saw earlier in this video, J train of w,b will be low because the average era on your training examples will be zero or very close to zero. So J train will be very close to zero. But if you have a few additional examples in your test set that the album had not trained on, then those test examples, might look like these. And there's a large gap between what the album is predicting as the estimated housing price, and the actual value of those housing prices.

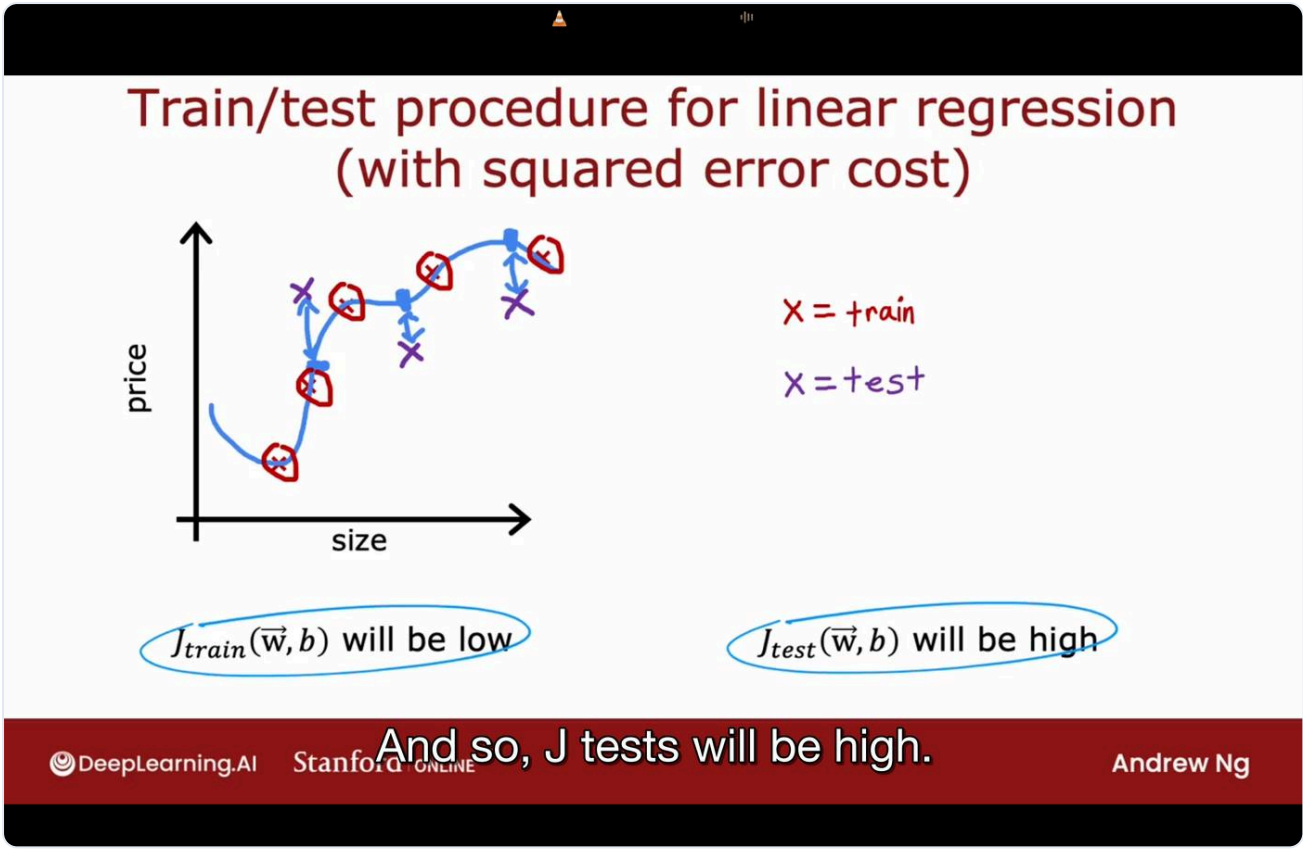


FIGURE 34.8 | Train/test Procedure for Linear Regression

And so, J tests will be high. So seeing that J test is high on this model, gives you a way to realize that even though it does great on the training set, is actually not so good at generalizing to new examples to new data points that were not in the training set. So, that was regression with squared error cost. Now, let's take a look at how you apply this procedure to a classification problem. For example, if you are classifying between handwritten digits that are either 0 or 1,, so same as before, you fit the parameters by minimizing the cost function to find the parameters w,b. For example, if you were training logistic regression, then this would be the cost function J of w,b, where this is the usual logistic loss function, and then plus also the regularization term.

And to compute the test error, J test is then the average over your test examples, that's that 30% of your data that wasn't in the training set of the logistic loss on your test set.

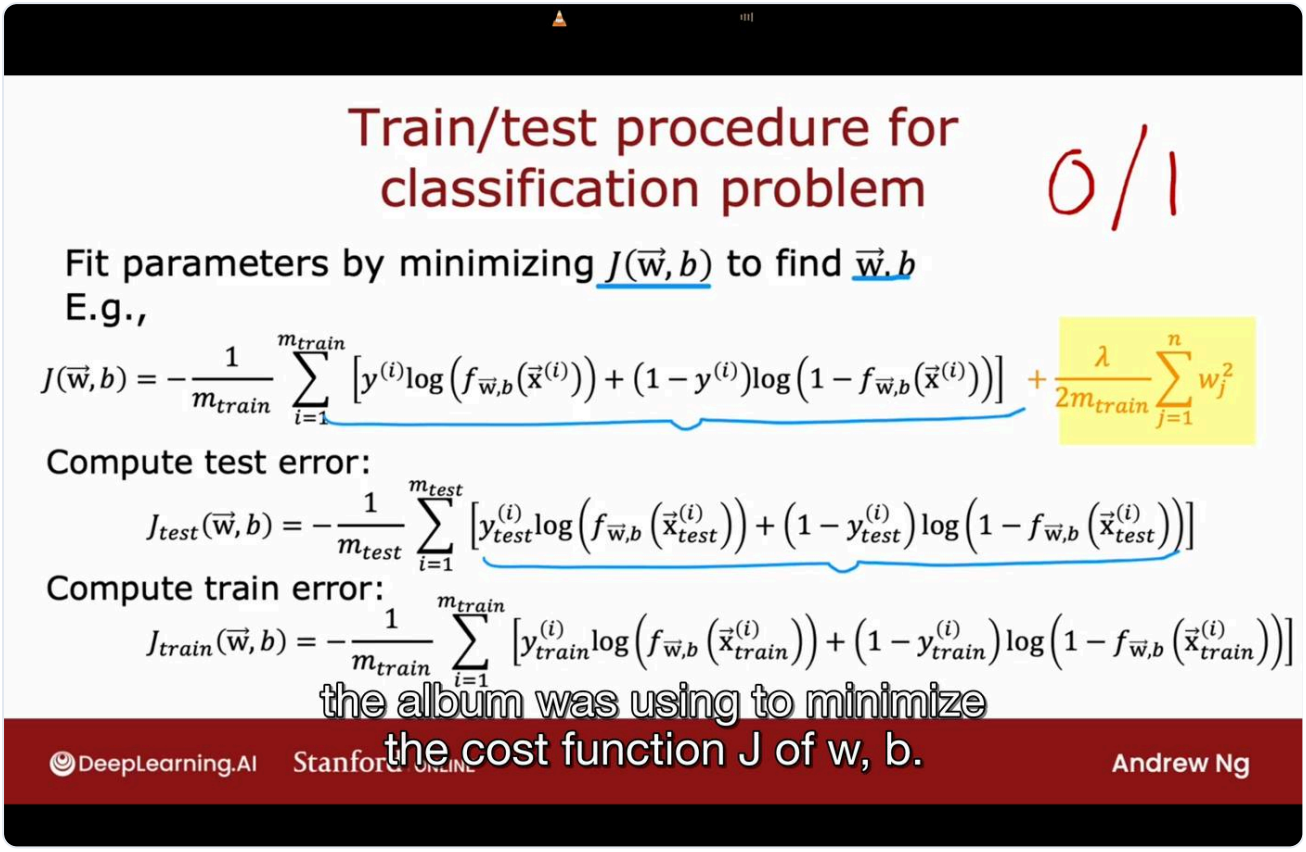


FIGURE 34.9 | Train/test Procedure for 6 /

And the training error you can also compute using this formula, is the average logistic loss on your training data that the album was using to minimize the cost function J of w, b. Well, when I described here will work, okay, for figuring out if your learning algorithm is doing well, by seeing how I was doing in terms of test error. When applying machine learning to classification problems, there's actually one other definition of J tests and J train that is maybe even more commonly used. Which is instead of using the logistic loss to compute the test error and the training error to instead measure what the fraction of the test set, and the fraction of the training set that the algorithm has misclassified.

So specifically on the test set, you can have the algorithm make a prediction 1 or 0 on every test example. So, recall y_hat we would predict us 1 if f of x is greater than equal 4.5, and zero if it's less than 0.5. And you can then count up in the test set the fraction of examples where y_hat is not equal to the actual ground truth label while in the test set. So concretely, if you are classifying handwritten digits 0, 1 binary classification loss, then J tests would be the fraction of that test set, where 0 was classified as 1 of 1, classified as 0.

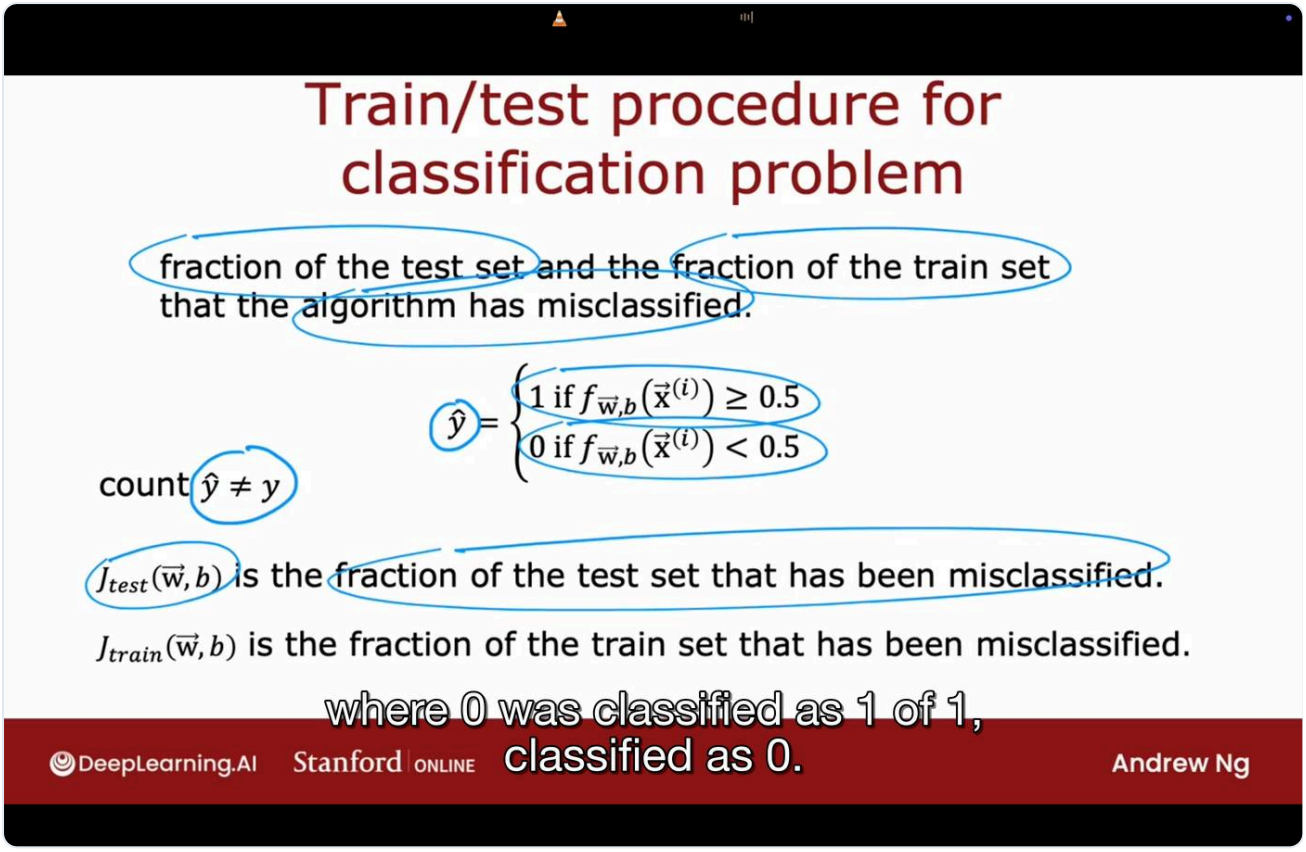


FIGURE 34.10 | Train/test Procedure for

And similarly, J train is a fraction of the training set that has been misclassified. Taking a dataset and splitting it into a training set and a separate test set gives you a way to systematically evaluate how well your learning algorithm is doing. By computing both J tests and J train, you can now measure how was doing on the test set and on the training set. This procedure is one step to what you'll be able to automatically choose what model to use for a given machine learning application. For example, if you're trying to predict housing prices, should you fit a straight line to your data, or fit a second order polynomial, or third order fourth order polynomial?

It turns out that with one further refinement to the idea you saw in this video, you'll be able to have an algorithm help you to automatically make that type of decision well. Let's take a look at how to do that in the next video.

LECTURE 35

Model Selection (Choosing a Model)

In the last video, you saw how to use the test set to evaluate the performance of a model. Let's make one further refinement to that idea in this video, which allow you to use the technique, to automatically choose a good model for your machine learning algorithm. One thing we've seen is that once the model's parameters w and b have been fit to the training set. The training error may not be a good indicator of how well the algorithm will do or how well it will generalize to new examples that were not in the training set, and in particular, for this example, the training error will be pretty much zero. That's likely much lower than the actual generalization error, and by that I mean the average error on new examples that were not in the training set.

What you saw on the last video is that J test the performance of the algorithm on examples, is not trained on, that will be a better indicator of how well the model will likely do on new data.

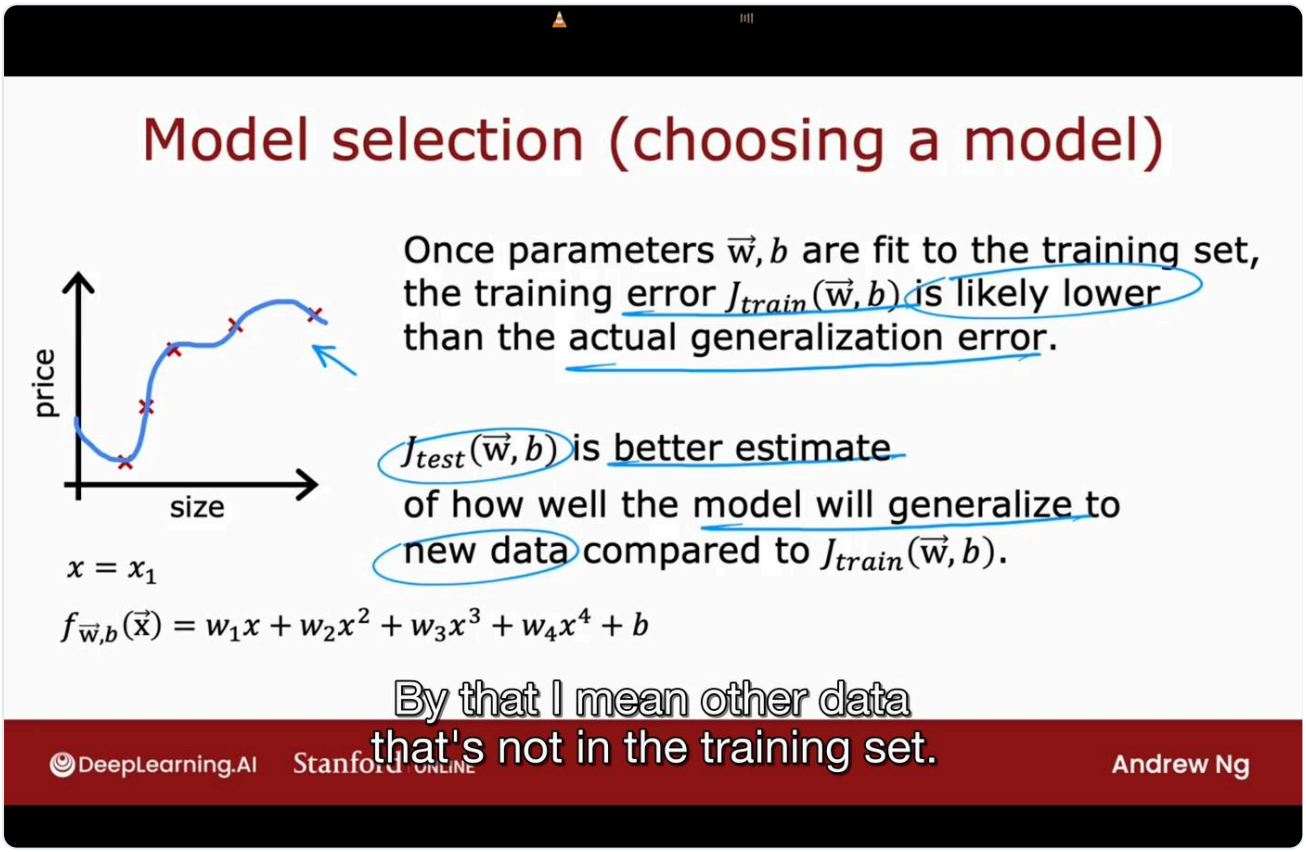


FIGURE 35.1 | Model Selection (Choosing a Model)

By that I mean other data that's not in the training set. Let's take a look at how this affects, how we might use a test set to choose a model for a given machine learning application. If a fitting a function to predict housing prices or some other regression problem, one model you might consider is to fit a linear model like this. This is a first-order polynomial and we're going to use d equals 1 on this slide to denote fitting a one or first-order polynomial. If you were to fit a model like this to your training set, you get some parameters, w and b, and you can then compute J tests

to estimate how well does this generalize to new data? On this slide, I'm going to use w^1, b^1 to denote that these are the parameters you get if you were to fit a first order polynomial, a degree one, d equals 1 polynomial.

Now, you might also consider fitting a second-order polynomial or quadratic model, so this is the model.

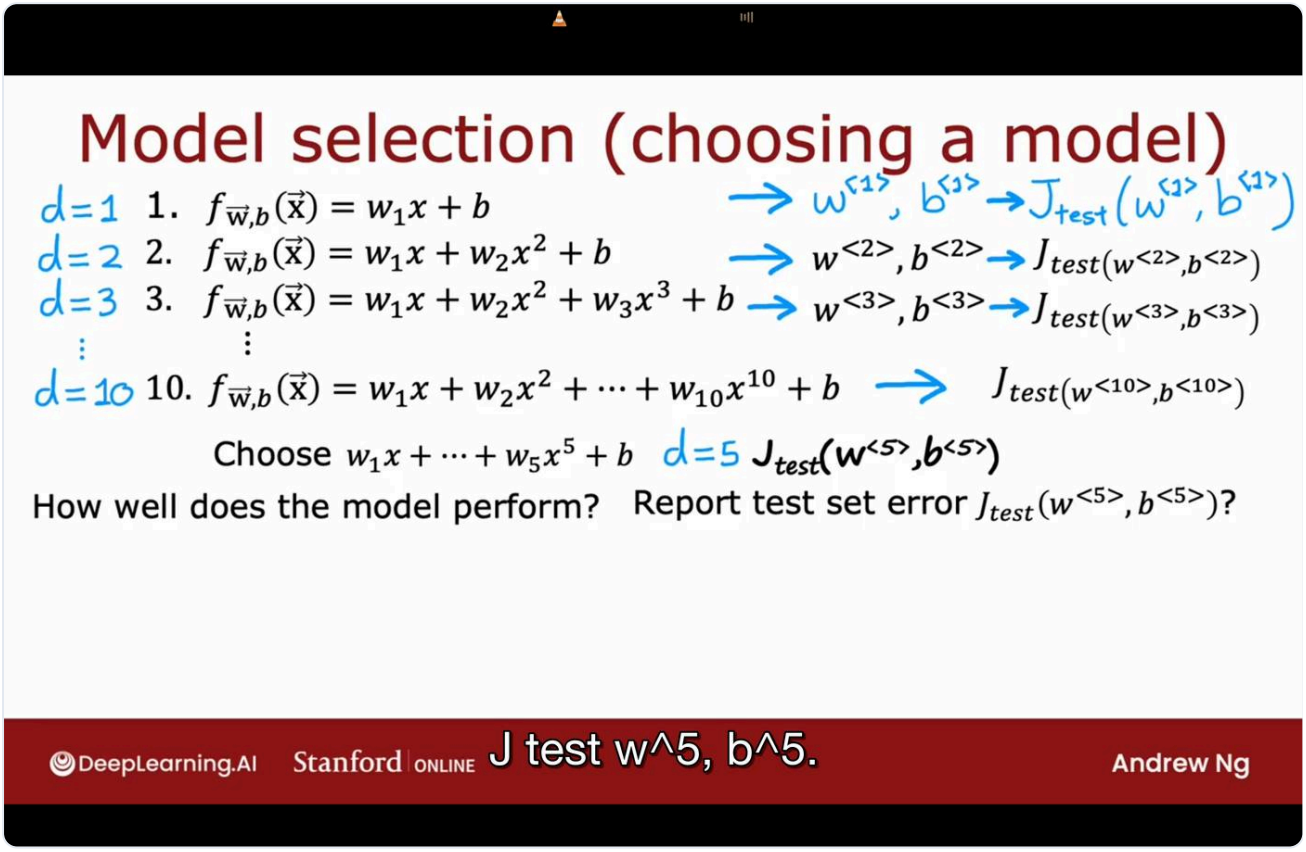


FIGURE 35.2 | Model Selection (Choosing a Model)

If you were to fit this to your training set, you would get some parameters, w^2, b^2, and you can then similarly evaluate those parameters on your test set and get J test w^2, b^2, and this will give you a sense of how well the second-order polynomial does. You can go on to try d equals 3, that's a third order or a degree three polynomial that looks like this, and fit parameters and similarly get J test. You might keep doing this until, say you try up to a 10th order polynomial and you end up with J test of w^10, b^10. That gives you a sense of how well the 10th order polynomial is doing. One procedure you could try, this turns out not to be the best procedure, but one thing you could try is, look at all of these J tests, and see which one gives you the lowest value.

Say, you find that, J test for the fifth order polynomial for w^5, b^5 turns out to be the lowest. If that's the case, then you might decide that the fifth order polynomial d equals 5 does best, and choose that model for your application. If you want to estimate how well this model performs, one thing you could do, but this turns out to be a slightly flawed procedure, is to report the test set error, J test w^5, b^5.

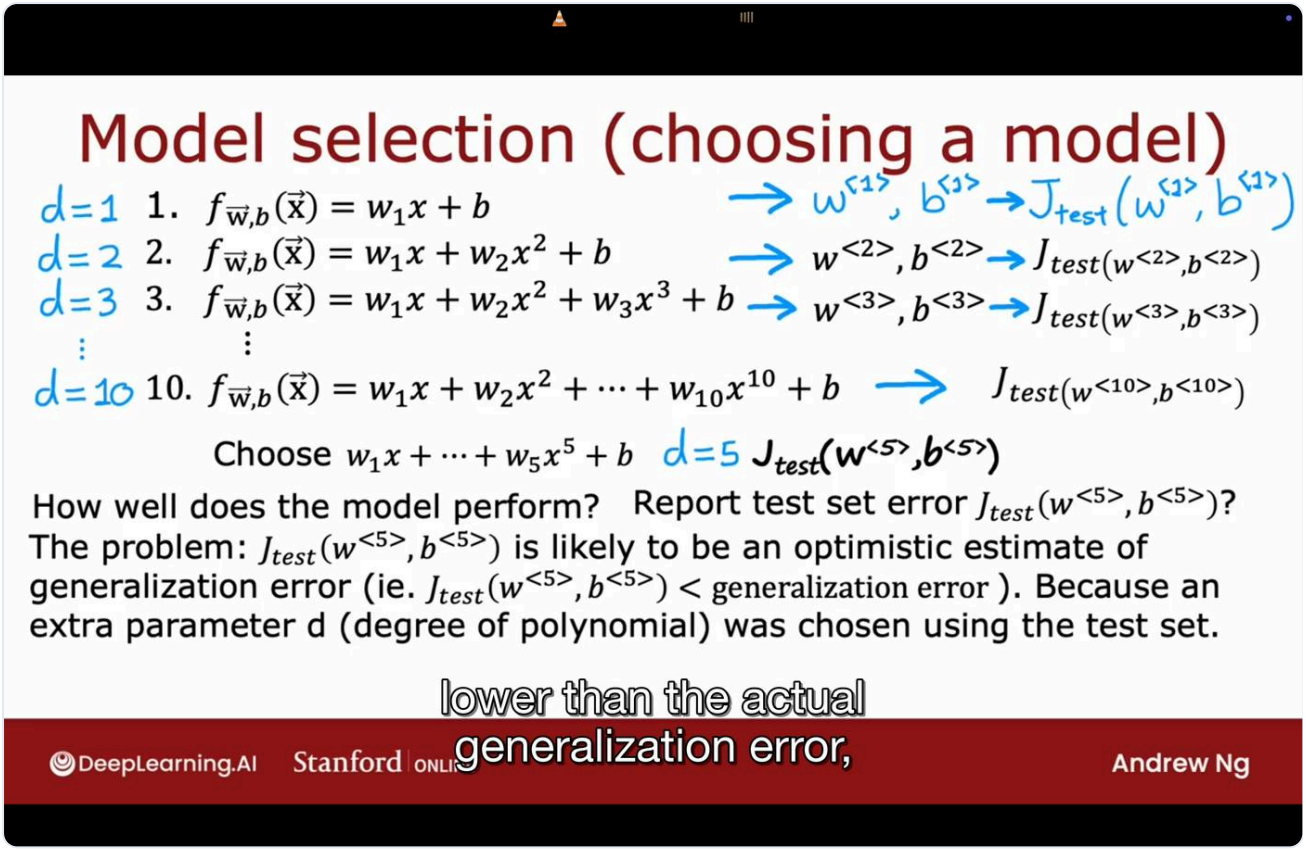


FIGURE 35.3 | Model Selection (Choosing a Model)

The reason this procedure is flawed is J test of w^5, b^5 is likely to be an optimistic estimate of the generalization error. In other words, it is likely to be lower than the actual generalization error, and the reason is, in the procedure we talked about on this slide with basic fits, one extra parameter, which is d, the degree of polynomial, and we chose this parameter using the test set.

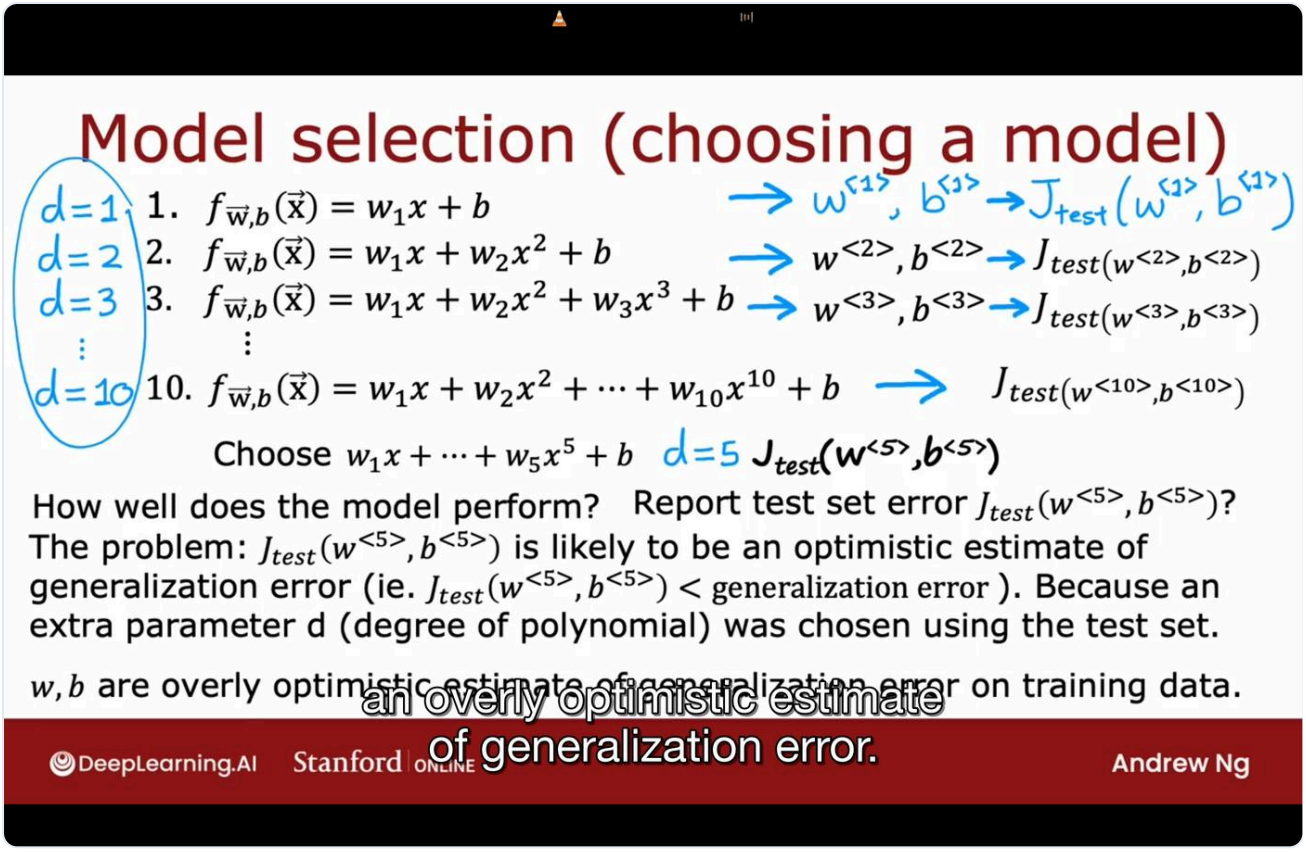


FIGURE 35.4

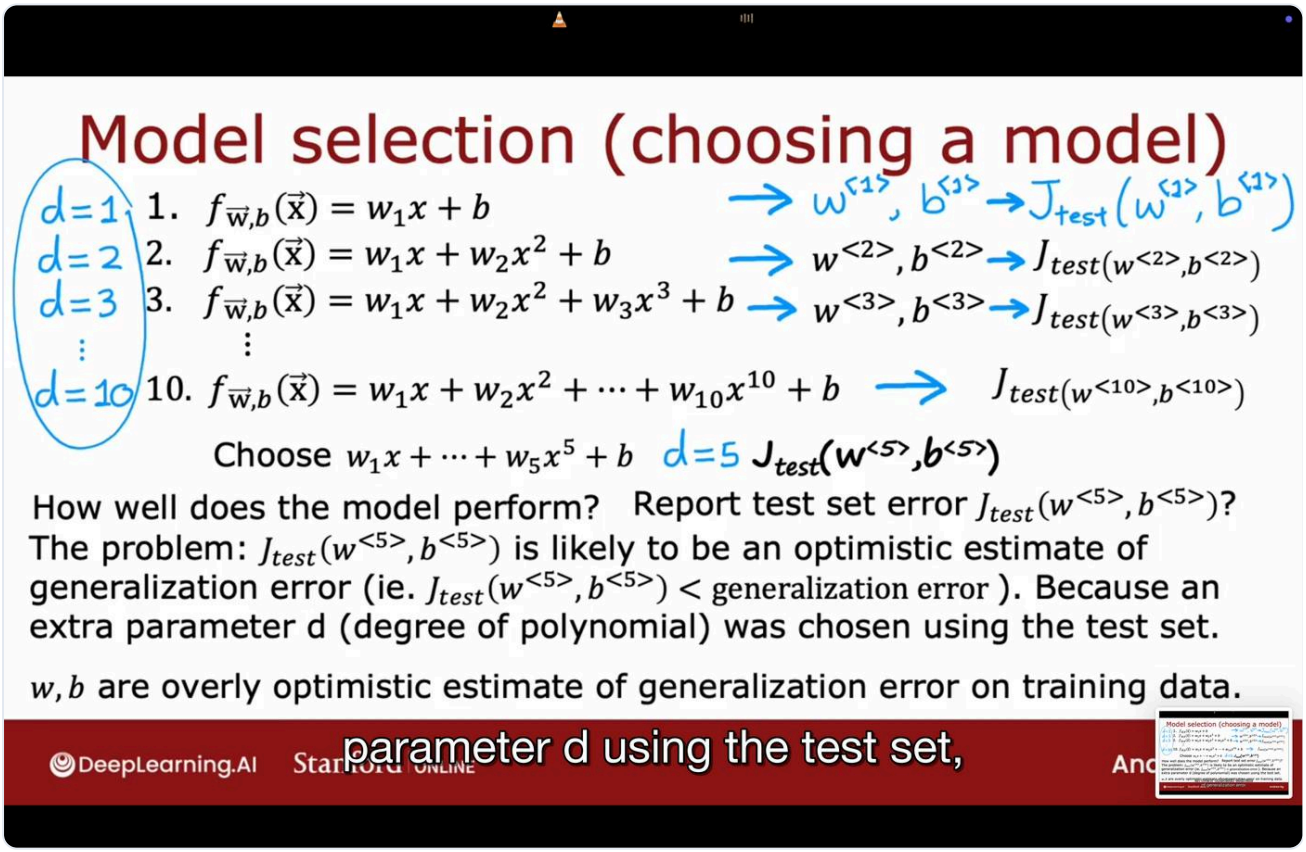


FIGURE 35.5

On the previous slide, we saw that if you were to fit w, b to the training data, then the training data would be an overly optimistic estimate of generalization error. It turns out too, that if you want to choose the parameter d using the test set, then the test set J test is now an overly optimistic, that is lower than actual estimate of the generalization error. The procedure on this particular slide is flawed and I don't recommend using this. Instead, if you want to automatically choose a model, such as decide what degree polynomial to use. Here's how you modify the training and testing procedure in order to carry out model selection. Whereby model selection, I mean choosing amongst different models, such as these 10 different models that you might contemplate using for your machine learning application.

The way we'll modify the procedure is instead of splitting your data into just two subsets, the training set and the test set, we're going to split your data into three different subsets, which we're going to call the training set, the cross-validation set, and then also the test set. Using our example from before of these 10 training examples, we might split it into putting 60 percent of the data into the training set and so the notation we'll use for the training set portion will be the same as before, except that now M train, the number of training examples will be six and we might put 20 percent of the data into the cross-validation set and a notation I'm going to use is x_cv of one, y_cv of one for the first cross-validation example. So cv stands for cross-validation, all the way down to x_cv of m_cv and y_cv of m_cv.

Where here, m_cv equals 2 in this example, is the number of cross-validation examples.

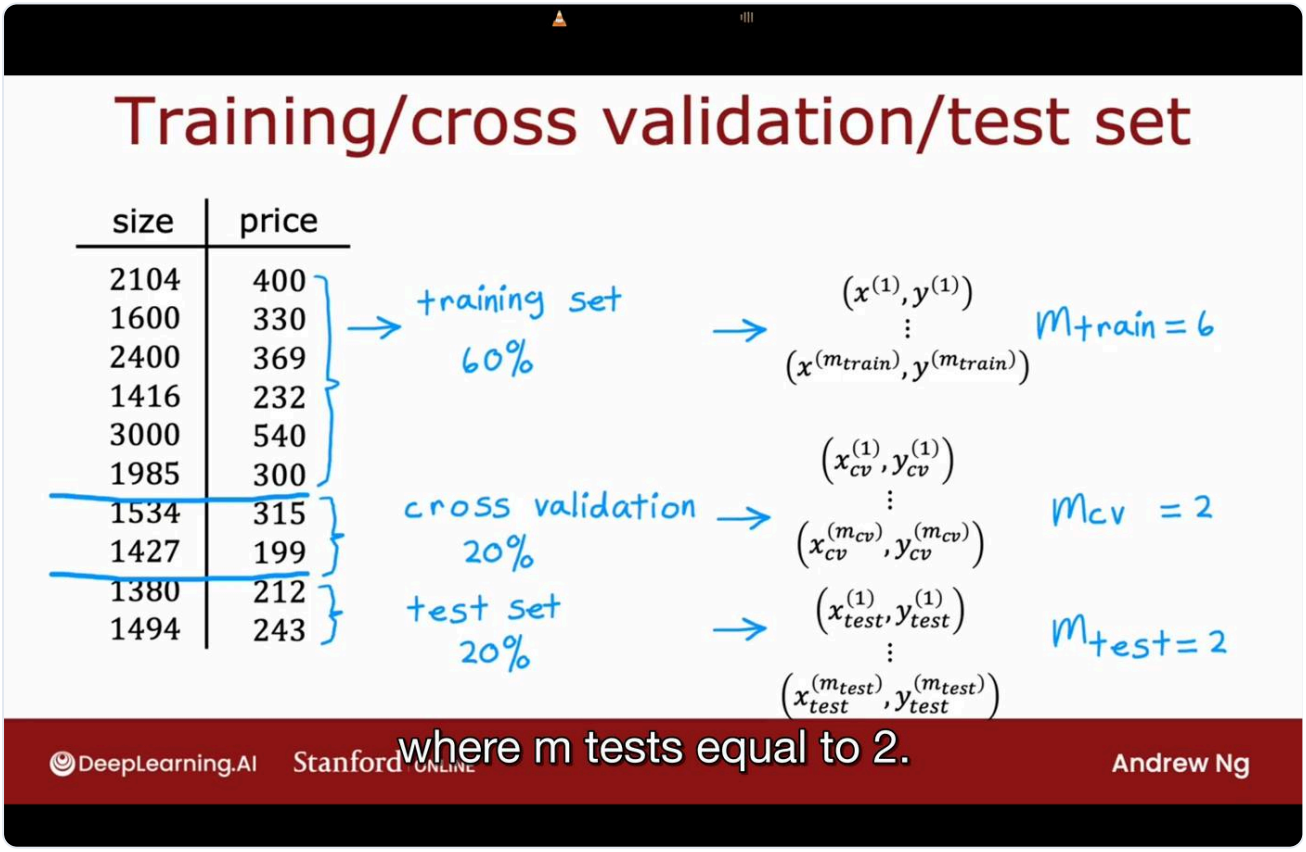


FIGURE 35.6 | Training/cross Validation/test Set

Then finally we have the test set same as before, so x1 through x m tests and y1 through y m, where m tests equal to 2. This is the number of test examples. We'll see you on the next slide how to use the cross-validation set. The way we'll modify the procedure is you've already seen the training set and the test set and we're going to introduce a new subset of the data called the cross-validation set. The name cross-validation refers to that this is an extra dataset that we're going to use to check or cross check the validity or really the accuracy of different models. I don t think it's a great name, but that is what people in machine learning have gotten to call this extra dataset.

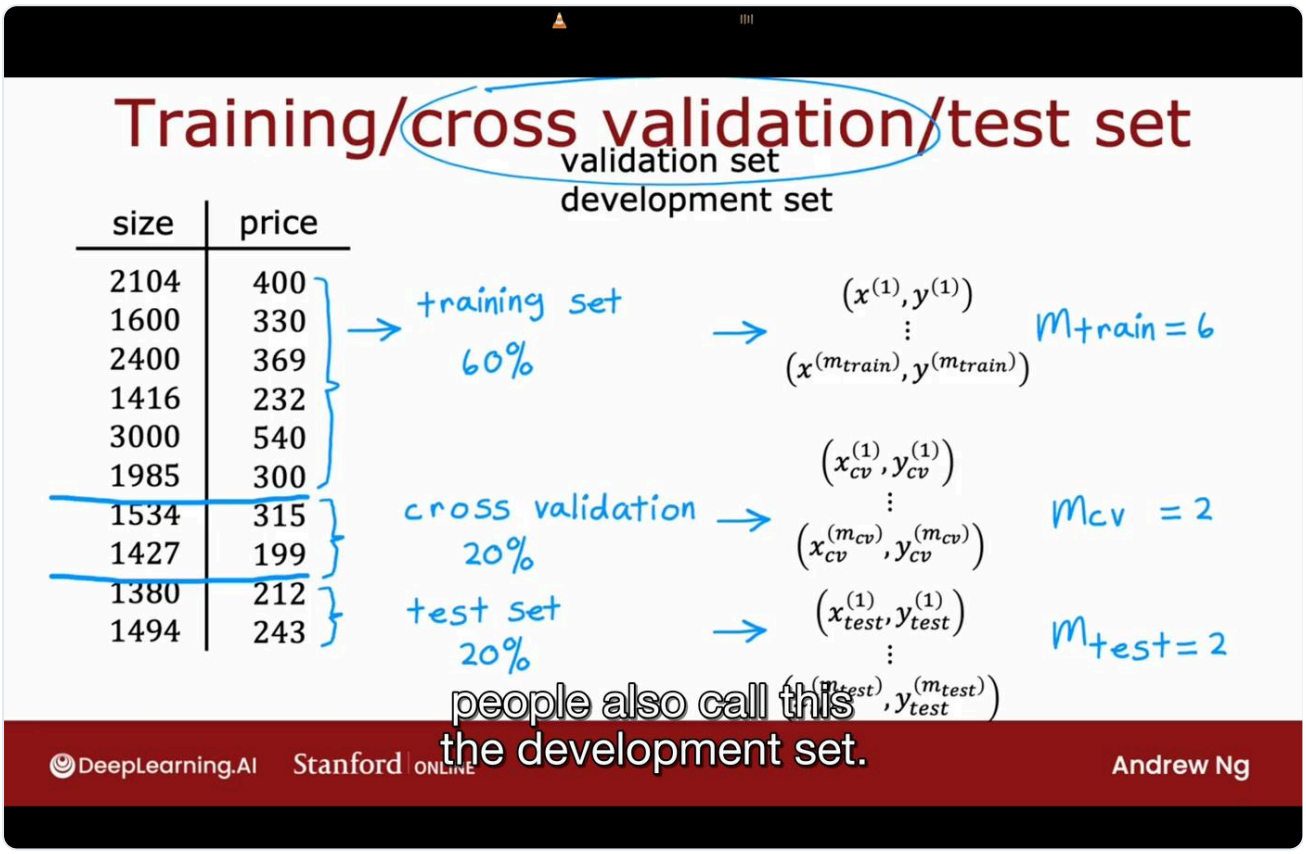


FIGURE 35.7 | Training/cross Validation/test Set

You may also hear people call this the validation set for short, it's just fewer syllables than cross-validation or in some applications, people also call this the development set. Means basically the same thing or for short. Sometimes you hear people call this the dev set, but all of these terms mean the same thing as cross-validation set. I personally use the term dev set the most often because it's the shortest, fastest way to say it but cross-validation is pretty used a little bit more often by machine learning practitioners. Onto these three subsets of the data training set, cross-validation set, and test set, you can then compute the training error, the cross-validation error, and the test error using these three formulas.

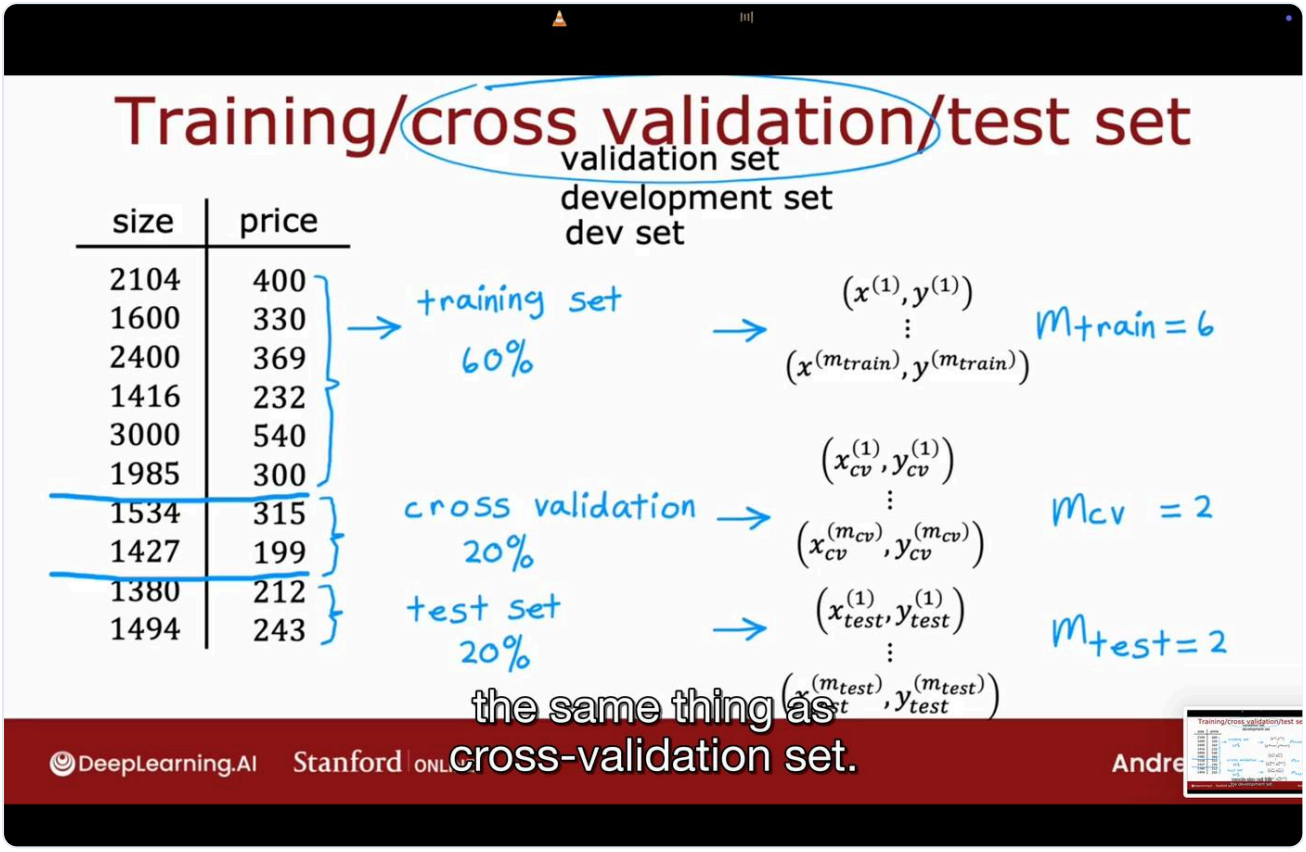


FIGURE 35.8

Whereas usual, none of these terms include the regularization term that is included in the training objective, and this new term in the middle, the cross-validation error is just the average over your m_cv cross-validation examples of the average say, squared error. This term, in addition to being called cross-validation error, is also commonly called the validation error for short, or even the development set error, or the dev error. Armed with these three measures of learning algorithm performance, this is how you can then go about carrying out model selection. You can, with the 10 models, same as earlier on this slide, with d equals 1, d equals 2, all the way up to a 10th degree or the 10th order polynomial, you can then fit the parameters w_1, b_1.

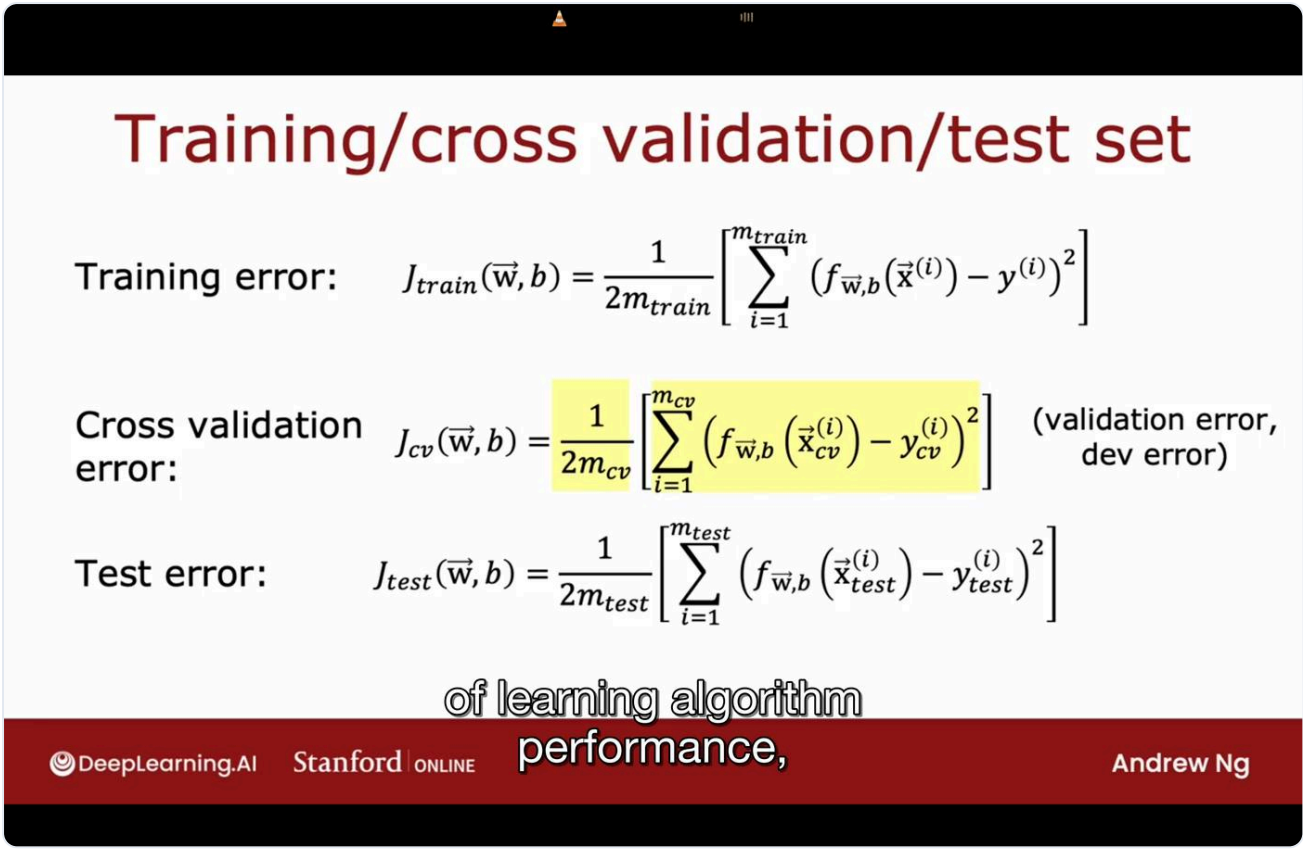


FIGURE 35.9 | Training/cross Validation/test Set

But instead of evaluating this on your test set, you will instead evaluate these parameters on your cross-validation sets and compute J_cv of w1, b1, and similarly, for the second model, we get J_cv of w2, v2, and all the way down to J_cv of w10, b10. Then, in order to choose a model, you will look at which model has the lowest cross-validation error, and concretely, let's say that J_cv of w4, b4 as low as, then what that means is you pick this fourth-order polynomial as the model you will use for this application. Finally, if you want to report out an estimate of the generalization error of how well this model will do on new data. You will do so using that third subset of your data, the test set and you report out Jtest of w4,b4.

You notice that throughout this entire procedure, you had fit these parameters using the training set.

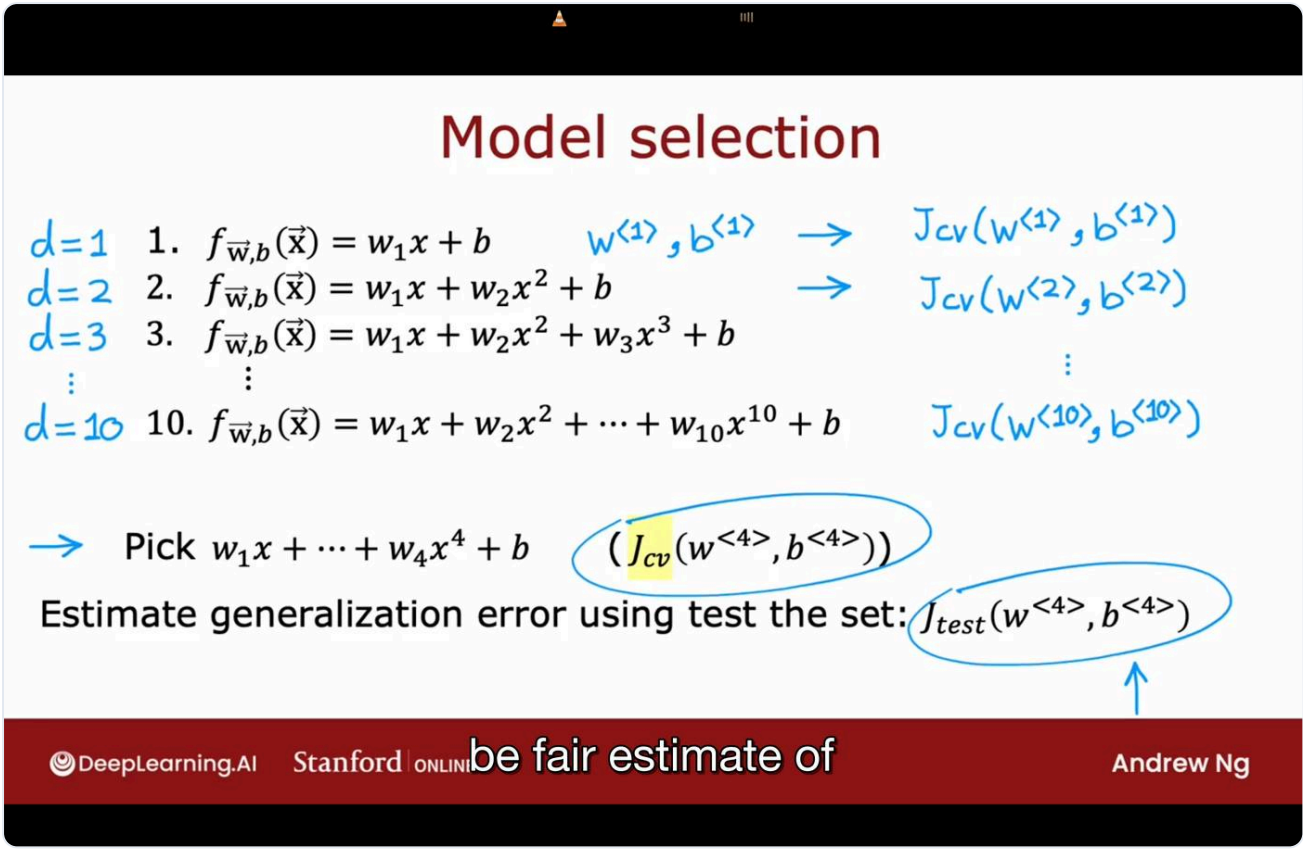


FIGURE 35.10 | Model Selection

You then chose the parameter d or chose the degree of polynomial using the cross-validation set and so up until this point, you've not fit any parameters, either w or b or d to the test set and that's why Jtest in this example will be fair estimate of the generalization error of this model thus parameters w4,b4. This gives a better procedure for model selection and it lets you automatically make a decision like what order polynomial to choose for your linear regression model. This model selection procedure also works for choosing among other types of models. For example, choosing a neural network architecture. If you are fitting a model for handwritten digit recognition, you might consider three models like this, maybe even a larger set of models than just me but here are a few different neural networks of small, somewhat larger, and then even larger.

To help you decide how many layers do the neural network have and how many hidden units per layer should you have, you can then train all three of these models and end up with parameters w1, b1 for the first model, w2, b2 for the second model, and w3,b3 for the third model. You can then evaluate the neural networks performance using Jcv, using your cross-validation set Since this is a classification problem, Jcv the most common choice would be to compute this as the fraction of cross-validation examples that the algorithm has misclassified.

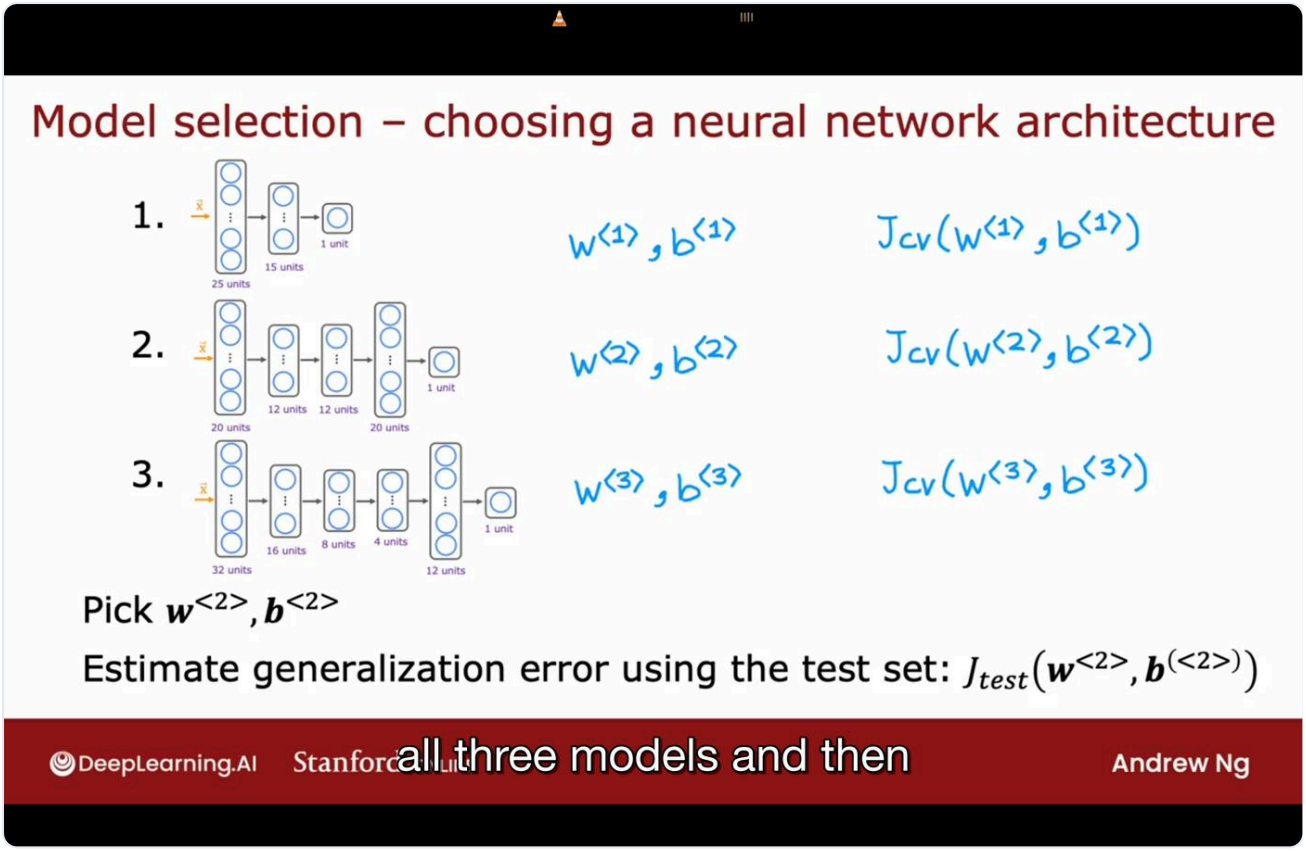


FIGURE 35.11

You would compute this using all three models and then pick the model with the lowest cross validation error.

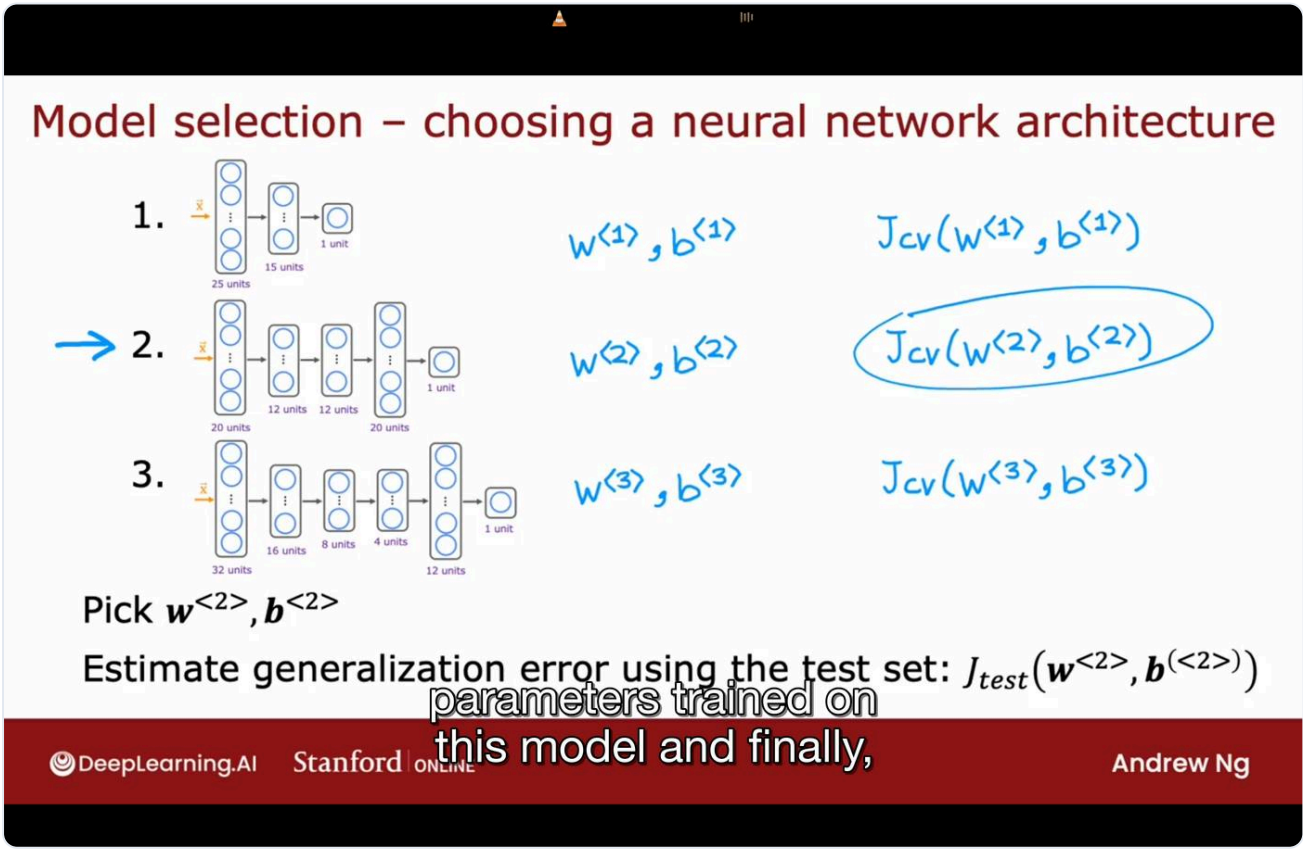


FIGURE 35.12

If in this example, this has the lowest cross validation error, you will then pick the second neural network and use parameters trained on this model and finally, if you want to report out an estimate of the generalization error, you then use the test set to estimate how well the neural network that you just chose will do. It's considered best practice in machine learning that if you have to make decisions about your model, such as fitting parameters or choosing the model architecture, such as neural network architecture or degree of polynomial if you're fitting a linear regression, to make all those decisions only using your training set and your cross-validation set, and to not look at the test set at all while you're still making decisions regarding your learning algorithm. It's only after you've come up with one model as your final model to only then evaluate it on the test set and because you haven't made any decisions using the test set, that ensures that your test set is a fair and not overly optimistic estimate of how well your model will generalize to new data.

That's model selection and this is actually a very widely used procedure. I use this all the time to automatically choose what model to use for a given machine learning application. Earlier this week, I mentioned running diagnostics to decide how to improve the performance of a learning algorithm. Now that you have a way to evaluate learning algorithms and even automatically choose a model, let's dive more deeply into examples of some diagnostics. The most powerful diagnostic that I know of and that I used for a lot of machine learning applications is one called bias and variance. Let's take a look at what that means in the next video.

LECTURE 36

Diagnosing Bias and Variance

The typical workflow of developing a machine learning system is that you have an idea and you train the model, and you almost always find that it doesn't work as well as you wish yet. When I'm training a machine learning model, it pretty much never works that well the first time. Key to the process of building machine learning system is how to decide what to do next in order to improve his performance. I've found across many different applications that looking at the bias and variance of a learning algorithm gives you very good guidance on what to try next. Let's take a look at what this means. You might remember this example from the first course on linear regression.

Where given this dataset, if you were to fit a straight line to it, it doesn't do that well. We said that this algorithm has high bias or that it underfits this dataset. If you were to fit a fourth-order polynomial, then it has high-variance or it overfits. In the middle if you fit a quadratic polynomial, then it looks pretty good. Then I said that was just right.

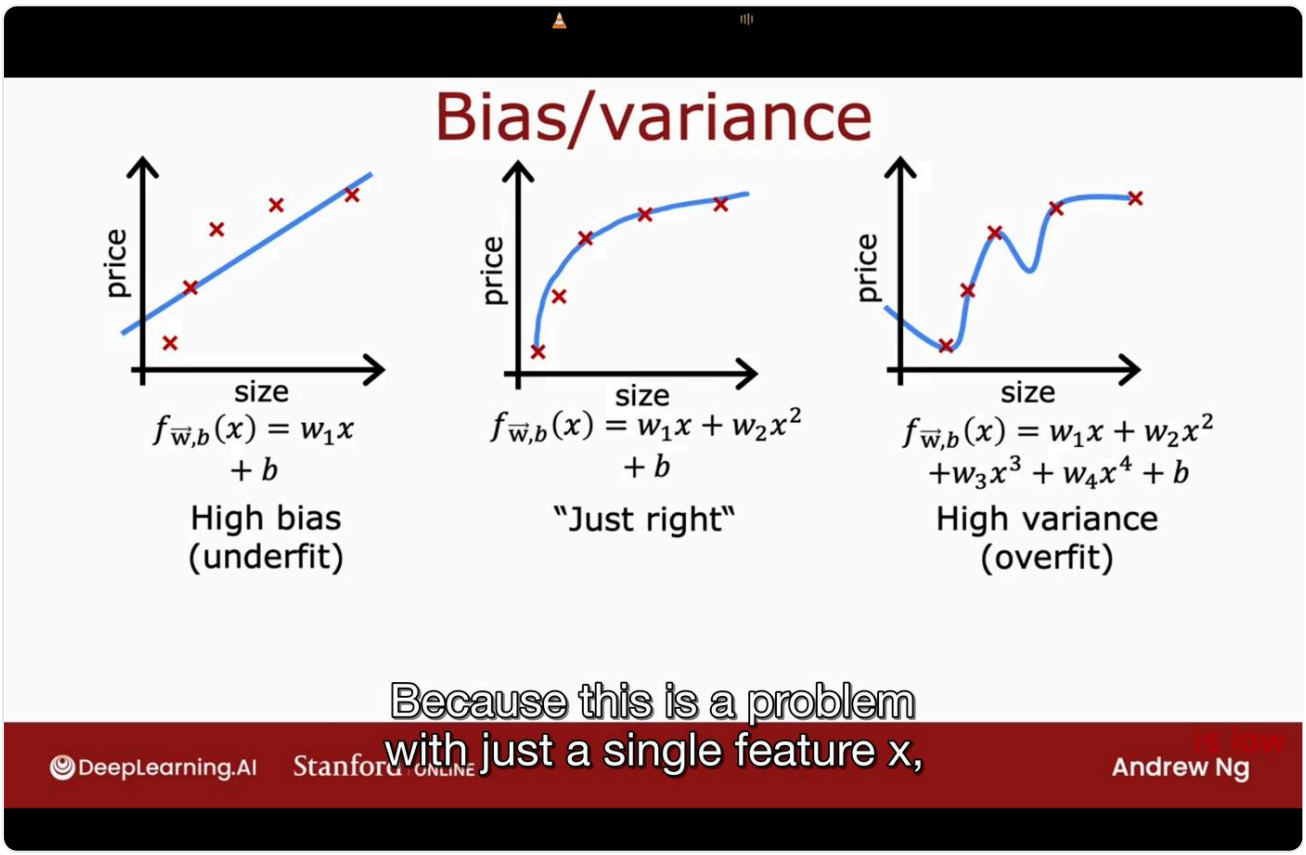


FIGURE 36.1 | Bias/variance

Because this is a problem with just a single feature x, we could plot the function f and look at it like this. But if you had more features, you can't plot f and visualize whether it's doing well as easily. Instead of trying to look at plots like this, a more systematic way to diagnose or to find out if your algorithm has high bias or high variance will be to look at the performance of your algorithm on the training set and on the cross validation set. In particular, let's look at the example

on the left. If you were to compute J_train, how well does the algorithm do on the training set? Not that well. I'd say J train here would be high because there are actually pretty large errors between the examples and the actual predictions of the model.

How about J_cv? J_cv would be if we had a few new examples, maybe examples like that, that the algorithm had not previously seen. Here the algorithm also doesn't do that well on examples that it had not previously seen, so J_cv will also be high. One characteristic of an algorithm with high bias, something that is under fitting, is that it's not even doing that well on the training set. When J_train is high, that is your strong indicator that this algorithm has high bias. Let's now look at the example on the right. If you were to compute J_train, how well is this doing on the training set? Well, it's actually doing great on the training set. Fits the training data really well.

J_train here will be low. But if you were to evaluate this model on other houses not in the training set, then you find that J_cv, the cross-validation error, will be quite high.

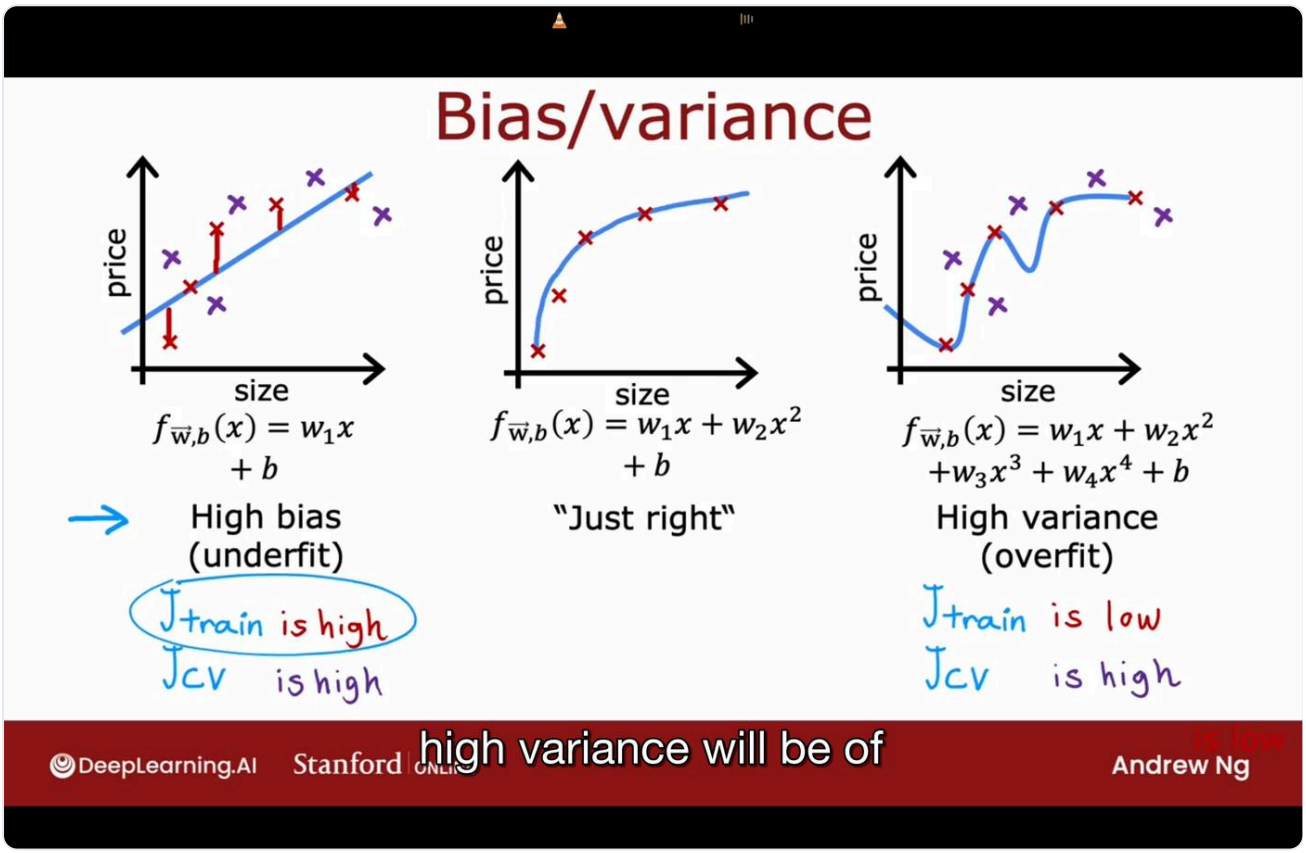


FIGURE 36.2 | Bias/variance

A characteristic signature or a characteristic Q that your algorithm has high variance will be of J_cv is much higher than J_train. In other words, it does much better on data it has seen than on data it has not seen. This turns out to be a strong indicator that your algorithm has high variance. Again, the point of what we're doing is that I'm computing J_train and J_cv and seeing if J _train is high or if J_cv is much higher than J_train. This gives you a sense, even if you can't plot to function f, of whether your algorithm has high bias or high variance. Finally, the case in the middle. If you look at J_train, it's pretty low, so this is doing quite well on the training set.

If you were to look at a few new examples, like those from, say, your cross-validation set, you find that J_cv is also a pretty low. J_train not being too high indicates this doesn't have a high bias problem and J_cv not being much worse than J_train this indicates that it doesn't have a high variance problem either. Which is why the quadratic model seems to be a pretty good one for this application. To summarize, when d equals 1 for a linear polynomial, J_train was high and J_cv was high. When d equals 4, J train was low, but J_cv is high. When d equals 2, both were pretty low.

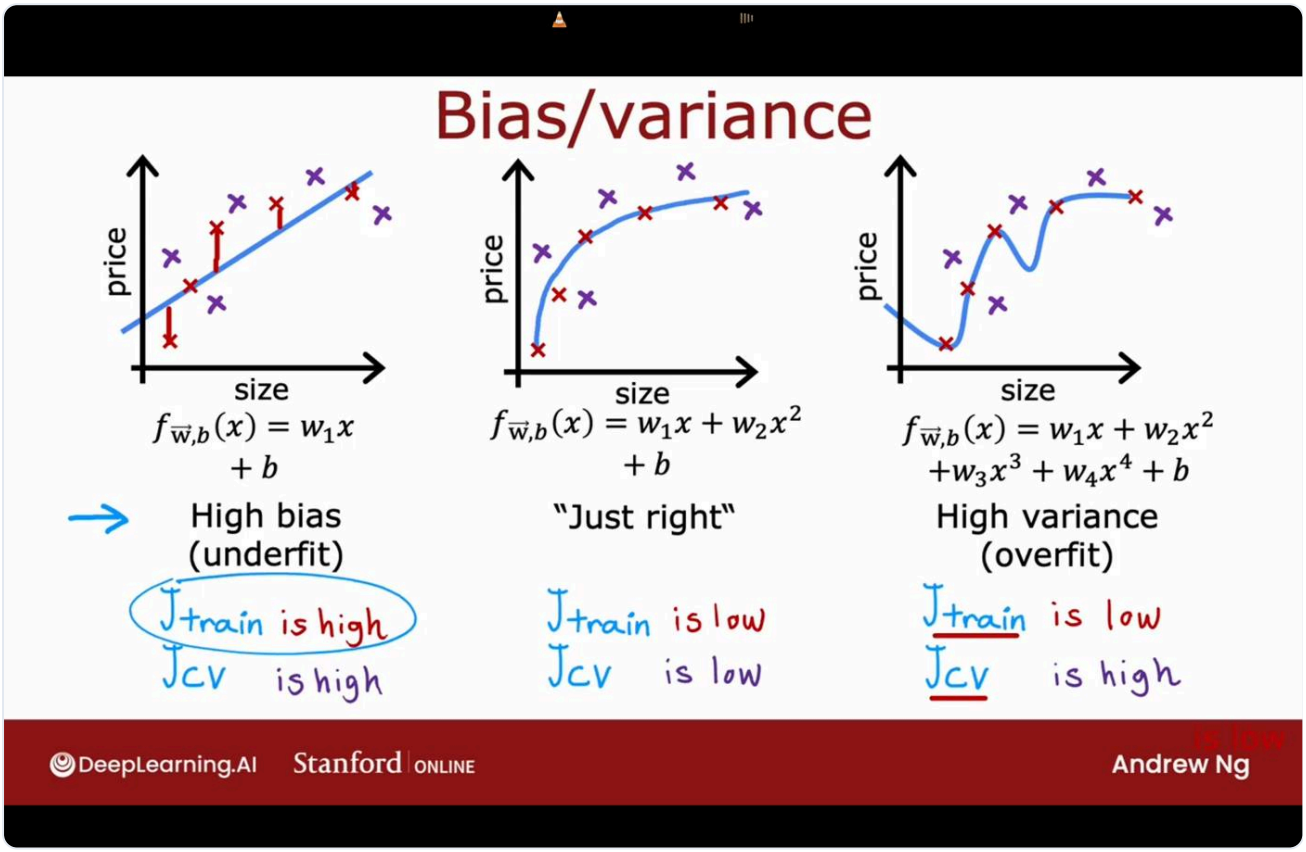


FIGURE 36.3

Let's now take a different view on bias and variance. In particular, on the next slide I'd like to show you how J_train and J_cv variance as a function of the degree of the polynomial you're fitting. Let me draw a figure where the horizontal axis, this d here, will be the degree of polynomial that we're fitting to the data. Over on the left we'll correspond to a small value of d, like d equals 1, which corresponds to fitting straight line. Over to the right we'll correspond to, say, d equals 4 or even higher values of d. We're fitting this high order polynomial. So if you were to plot J train or W, B as a function of the degree of polynomial, what you find is that as you fit a higher and higher degree polynomial, here I'm assuming we're not using regularization, but as you fit a higher and higher order polynomial, the training error will tend to go down because when you have a very simple linear function, it doesn't fit the training data that well, when you fit a quadratic function or third order polynomial or fourth-order polynomial, it fits the training data better and better.

As the degree of polynomial increases, J train will typically go down. Next, let's look at J_cv, which is how well does it do on data that it did not get to fit to? What we saw was when d equals one, when the degree of polynomial was very low, J_cv was pretty high because it underfits, so it

didn't do well on the cross validation set. Here on the right as well, when the degree of polynomial is very large, say four, it doesn't do well on the cross-validation set either, and so it's also high. But if d was in-between say, a second-order polynomial, then it actually did much better. If you were to vary the degree of polynomial, you'd actually get a curve that looks like this, which comes down and then goes back up.

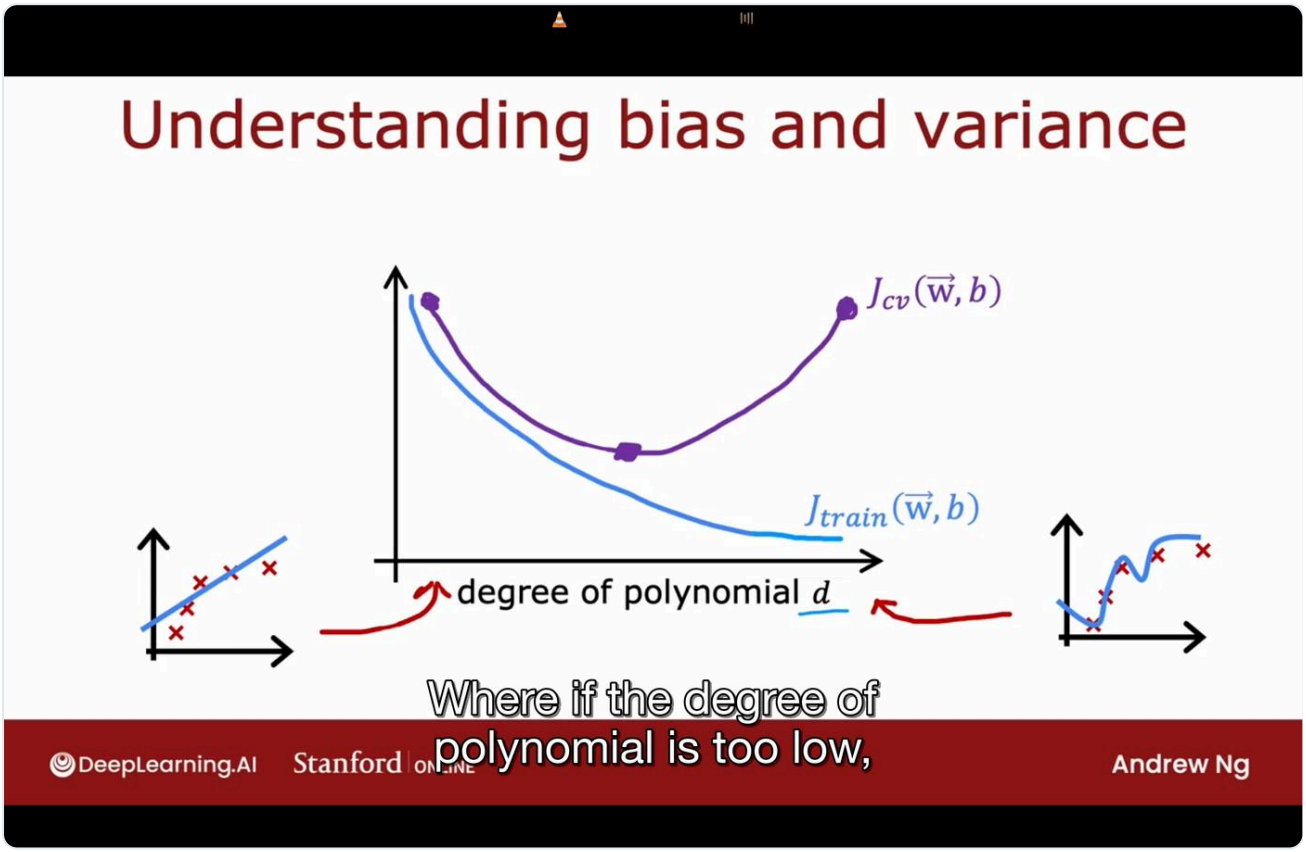


FIGURE 36.4 | Understanding Bias and Variance

Where if the degree of polynomial is too low, it underfits and so doesn't do the cross validation set, if it is too high, it overfits and also doesn't do well on the cross validation set. Is only if it's somewhere in the middle, that is just right, which is why the second-order polynomial in our example ends up with a lower cross-validation error and neither high bias nor high-variance. To summarize, how do you diagnose bias and variance in your learning algorithm? If your learning algorithm has high bias or it has undefeated data, the key indicator will be if J train is high. That corresponds to this leftmost portion of the curve, which is where J train as high.

Usually you have J train and J_cv will be close to each other. How do you diagnose if you have high variance? While the key indicator for high-variance will be if J_cv is much greater than J train does double greater than sign in math refers to a much greater than, so this is greater, and this means much greater. This rightmost portion of the plot is where J_cv is much greater than J train. Usually J train will be pretty low, but the key indicator is whether J_cv is much greater than J train. That's what happens when we had fit a very high order polynomial to this small dataset. Even though we've just seen bias in the areas, it turns out, in some cases, is possible to simultaneously have high bias and have high-variance.

You won't see this happen that much for linear regression, but it turns out that if you're training a neural network, there are some applications where unfortunately you have high bias and high variance.

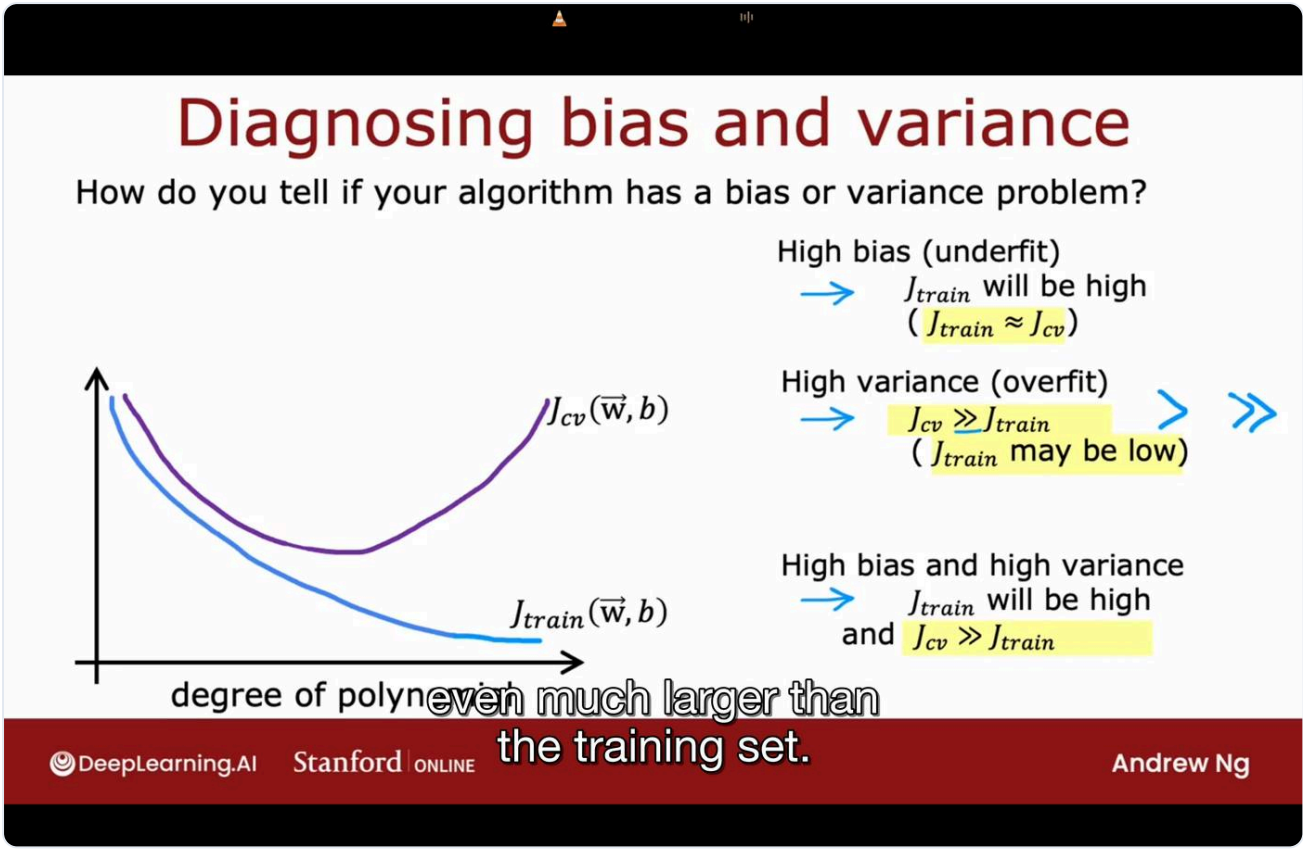


FIGURE 36.5 | Diagnosing Bias and Variance

One way to recognize that situation will be if J train is high, so you're not doing that well on the training set, but even worse, the cross-validation error is again, even much larger than the training set. The notion of high bias and high variance, it doesn't really happen for linear models applied to 1D.

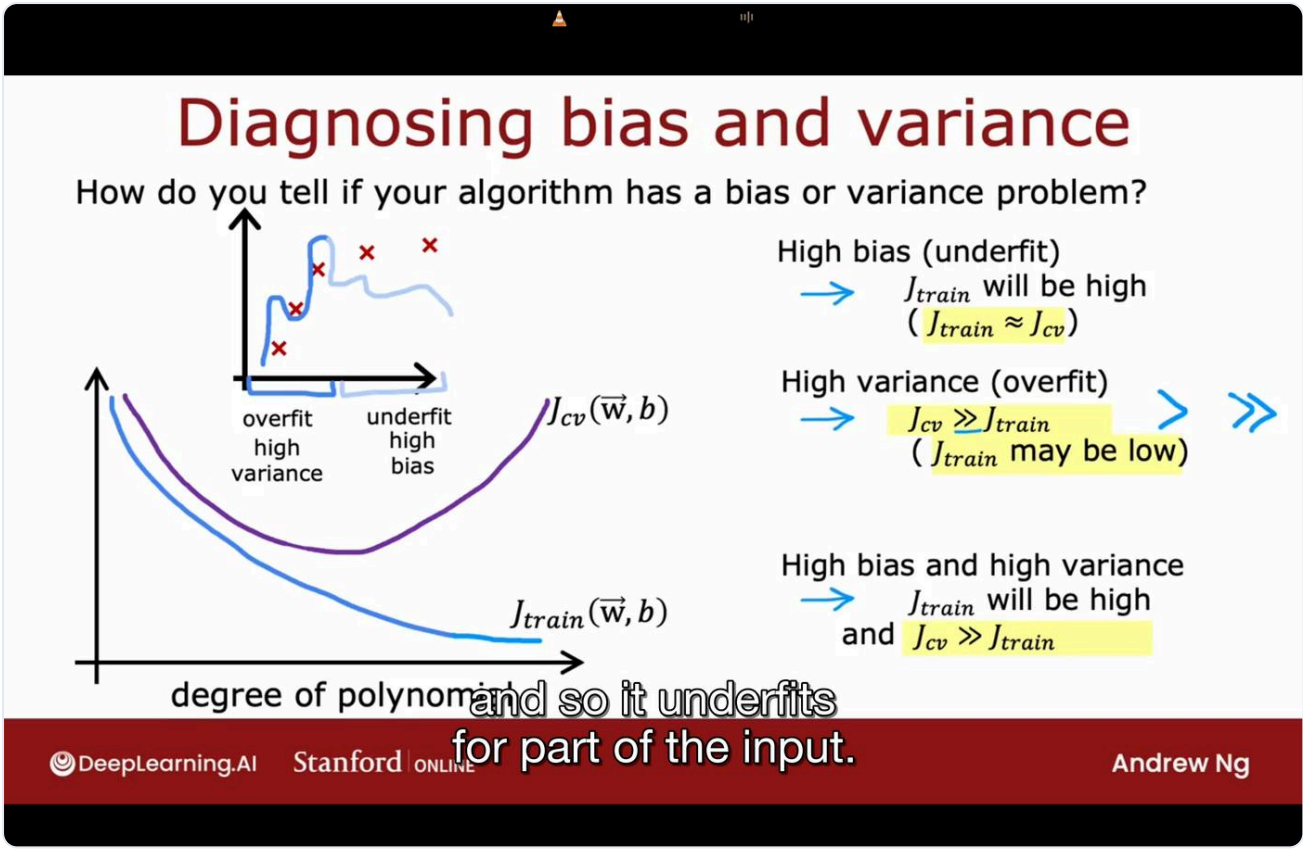


FIGURE 36.6 | Diagnosing Bias and Variance

But to give intuition about what it looks like, it would be as if for part of the input, you had a very complicated model that overfit, so it overfits to part of the inputs. But then for some reason, for other parts of the input, it doesn't even fit the training data well, and so it underfits for part of the input. In this example, which looks artificial because it's a single feature input, we fit the training set really well and we overfit in part of the input, and we don't even fit the training data well, and we underfit the part of the input. That's how in some applications you can unfortunate end up with both high bias and high variance. The indicator for that will be if the algorithm does poorly on the training set, and it even does much worse than on the training set.

For most learning applications, you probably have primarily a high bias or high variance problem rather than both at the same time. But it is possible sometimes they're both at the same time. I know that there's a lot of process, there are a lot of concepts on the slides, but the key takeaways are, high bias means is not even doing well on the training set, and high variance means, it does much worse on the cross validation set than the training set. Whenever I'm training a machine learning algorithm, I will almost always try to figure out to what extent the algorithm has a high bias or underfitting versus a high-variance when overfitting problem.

This will give good guidance, as we'll see later this week, on how you can improve the performance of the algorithm. But first, let's take a look at how regularization effects the bias and variance of a learning algorithm because that will help you better understand when you should use regularization. Let's take a look at that in the next video.

LECTURE 37

Lambda

You saw in the last video how different choices of the degree of polynomial D affects the bias in variance of your learning algorithm and therefore its overall performance. In this video, let's take a look at how regularization, specifically the choice of the regularization parameter Lambda affects the bias and variance and therefore the overall performance of the algorithm. This, it turns out, will be helpful for when you want to choose a good value of Lambda of the regularization parameter for your algorithm. Let's take a look. In this example, I'm going to use a fourth-order polynomial, but we're going to fit this model using regularization. Where here the value of Lambda is the regularization parameter that controls how much you trade-off keeping the parameters w small versus fitting the training data well.

Let's start with the example of setting Lambda to be a very large value. Say Lambda is equal to 10,000. If you were to do so, you would end up fitting a model that looks roughly like this.

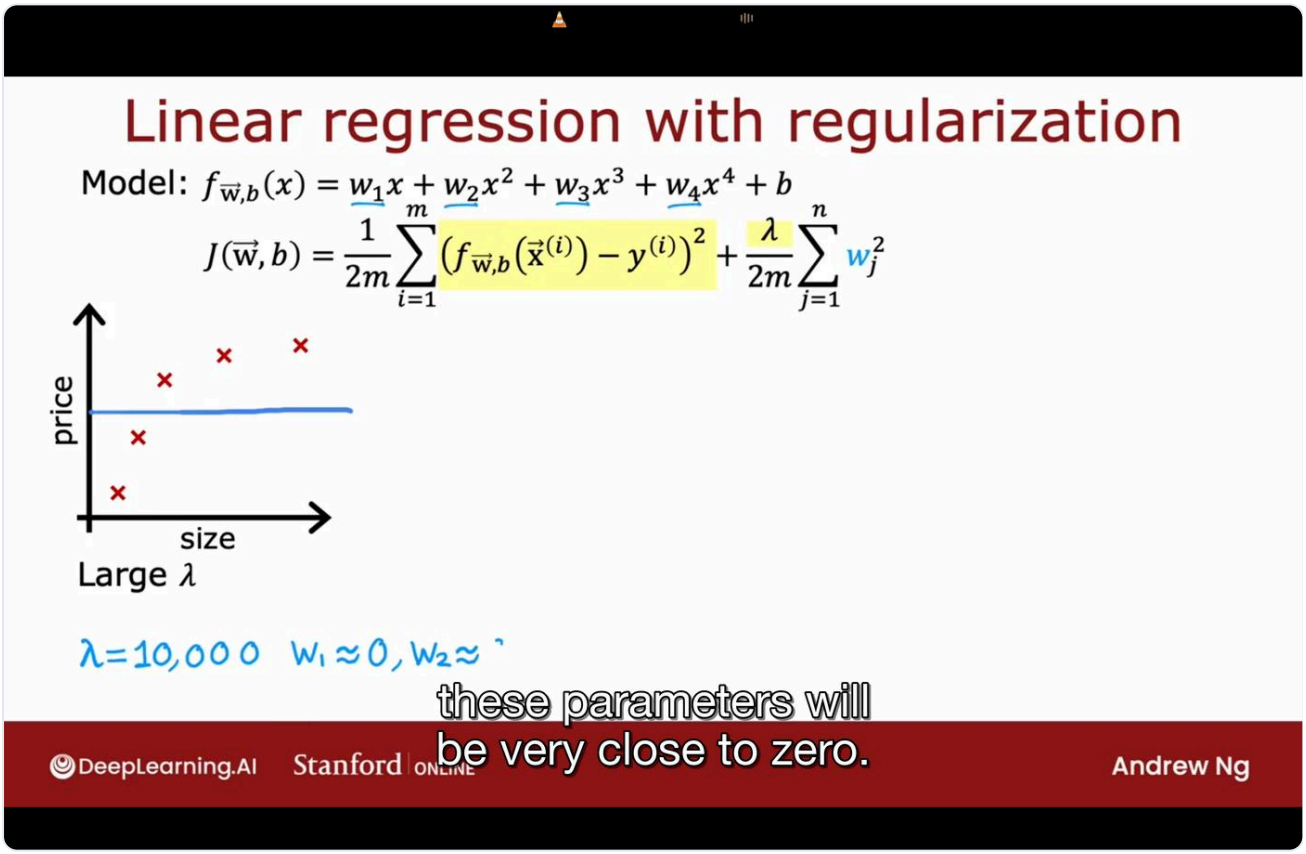


FIGURE 37.1

Because if Lambda were very large, then the algorithm is highly motivated to keep these parameters w very small and so you end up with w_1, w_2, really all of these parameters will be very close to zero. The model ends up being f of x is just approximately b a constant value, which is why you end up with a model like this.

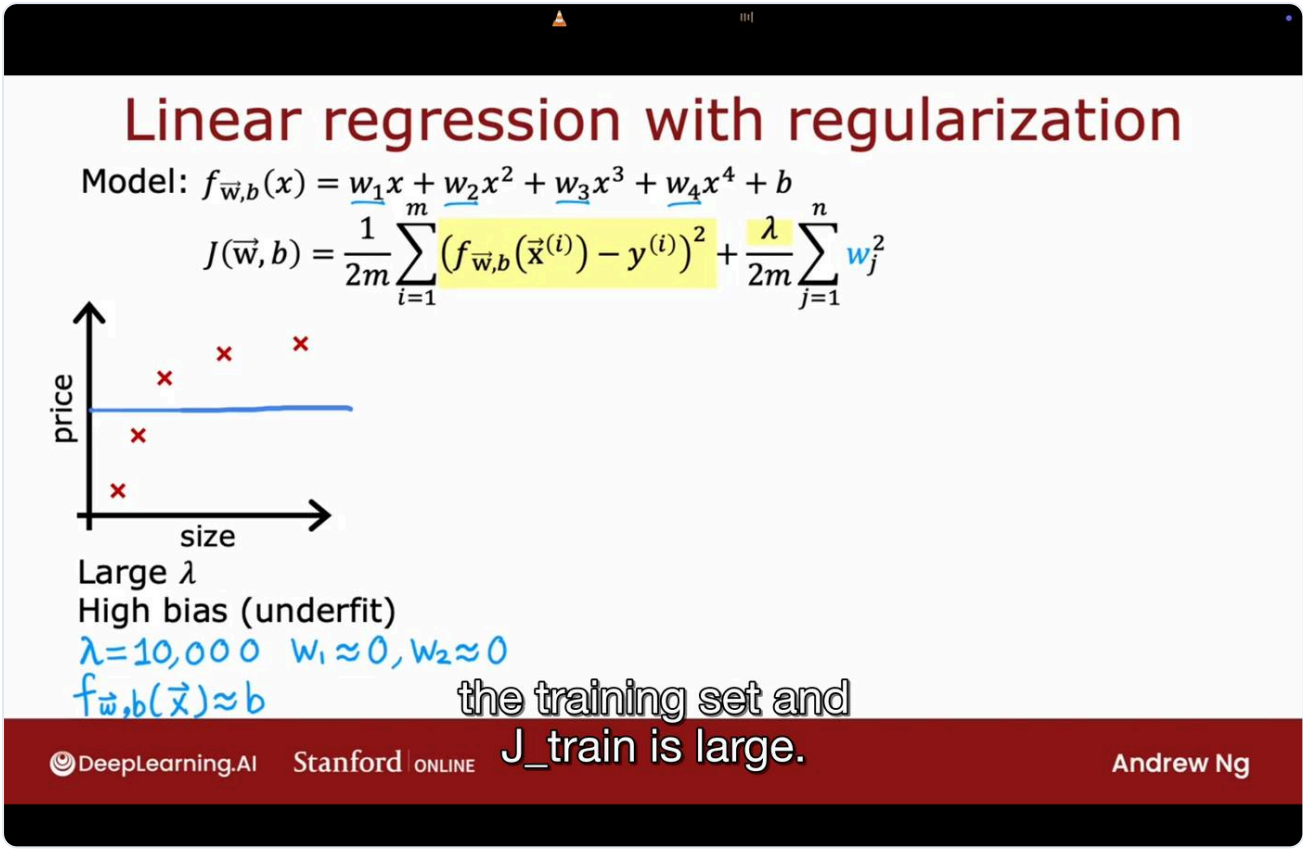


FIGURE 37.2

This model clearly has high bias and it underfits the training data because it doesn't even do well on the training set and J_train is large.

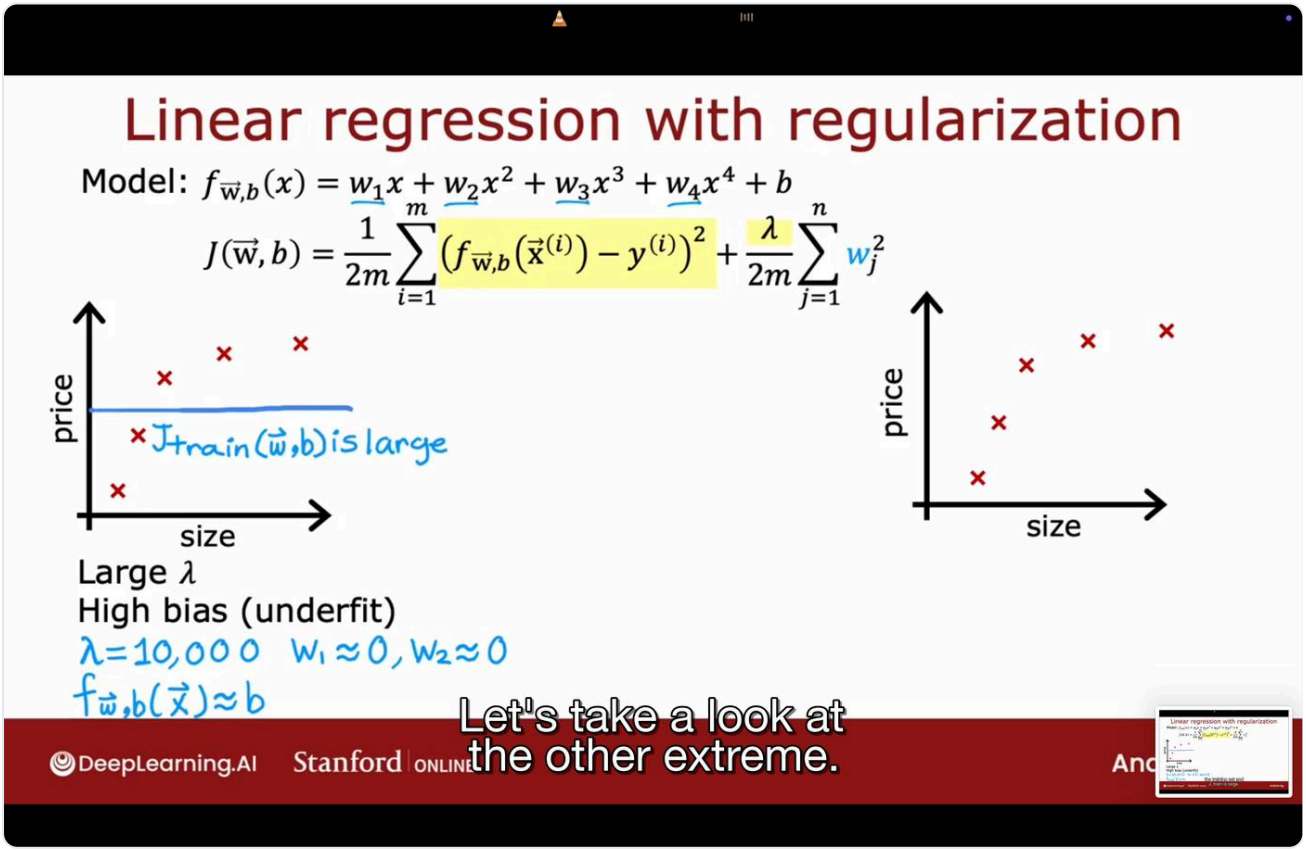


FIGURE 37.3

Let's take a look at the other extreme. Let's say you set Lambda to be a very small value. With a small value of Lambda, in fact, let's go to extreme of setting Lambda equals zero. With that choice of Lambda, there is no regularization, so we're just fitting a fourth-order polynomial with no regularization and you end up with that curve that you saw previously that overfits the data. What we saw previously was when you have a model like this, J_train is small, but J_cv is much larger than J_train or J_cv is large. This indicates we have high variance and it overfits this data.

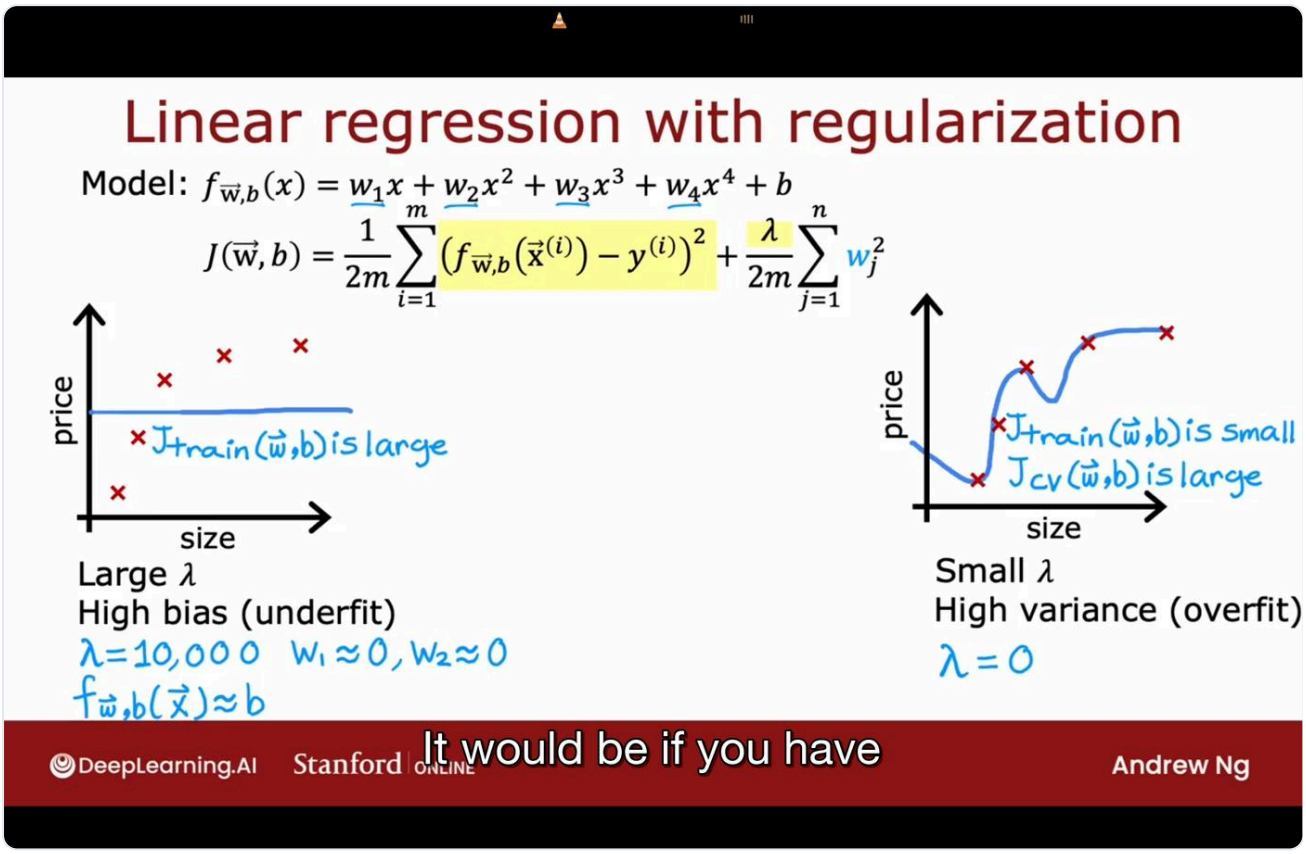


FIGURE 37.4

It would be if you have some intermediate value of Lambda, not really largely 10,000, but not so small as zero that hopefully you get a model that looks like this, that is just right and fits the data well with small J_train and small J_cv.

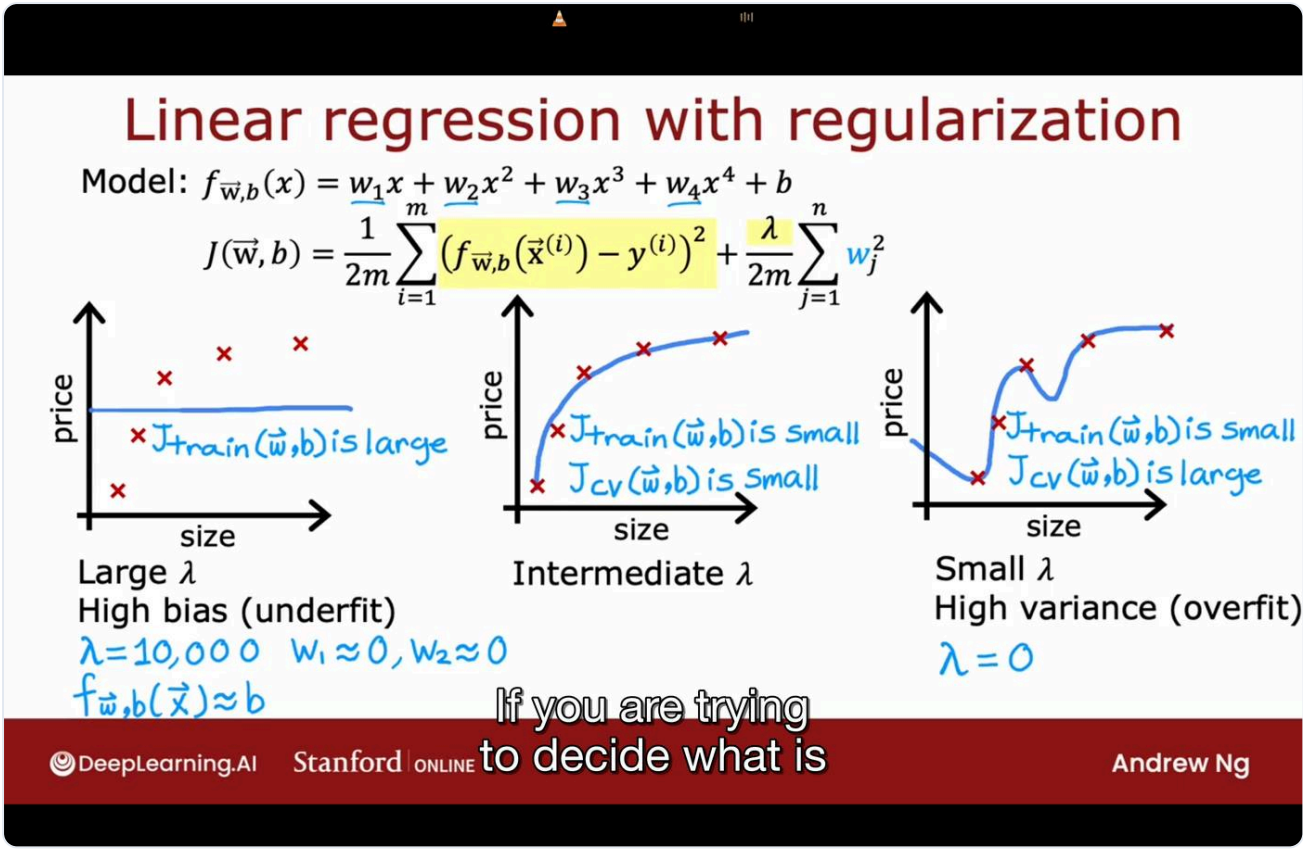


FIGURE 37.5

If you are trying to decide what is a good value of Lambda to use for the regularization parameter, cross-validation gives you a way to do so as well. Let's take a look at how we could do so. Just as a reminder, the problem we're addressing is if you're fitting a fourth-order polynomial, so that's the model and you're using regularization, how can you choose a good value of Lambda? This would be procedures similar to what you had seen for choosing the degree of polynomial D using cross-validation. Specifically, let's say we try to fit a model using Lambda equals 0. We would minimize the cost function using Lambda equals 0 and end up with some parameters w1, b1 and you can then compute the cross-validation error, J_cv of w1, b1.

Now let's try a different value of Lambda. Let's say you try Lambda equals 0.01. Then again, minimizing the cost function gives you a second set of parameters, w2, b2 and you can also see how well that does on the cross-validation set, and so on. Let's keep trying other values of Lambda and in this example, I'm going to try doubling it to Lambda equals 0.02 and so that will give you J_cv of w3, b3, and so on. Then let's double again and double again. After doubling a number of times, you end up with Lambda approximately equal to 10, and that will give you parameters w12, b12, and J_cv w12 of b12.

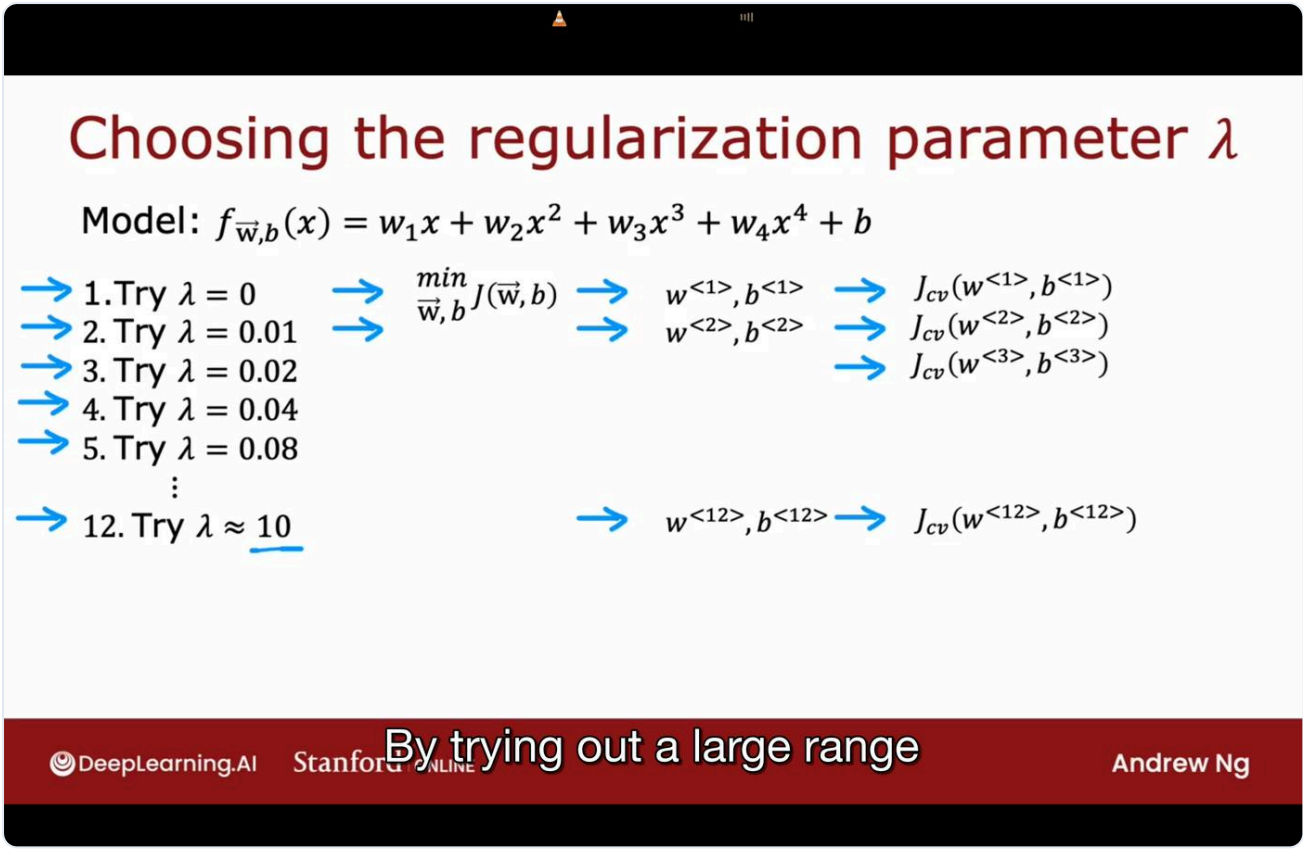


FIGURE 37.6 | Choosing the Regularization Parameter a

By trying out a large range of possible values for Lambda, fitting parameters using those different regularization parameters, and then evaluating the performance on the cross-validation set, you can then try to pick what is the best value for the regularization parameter. Quickly.

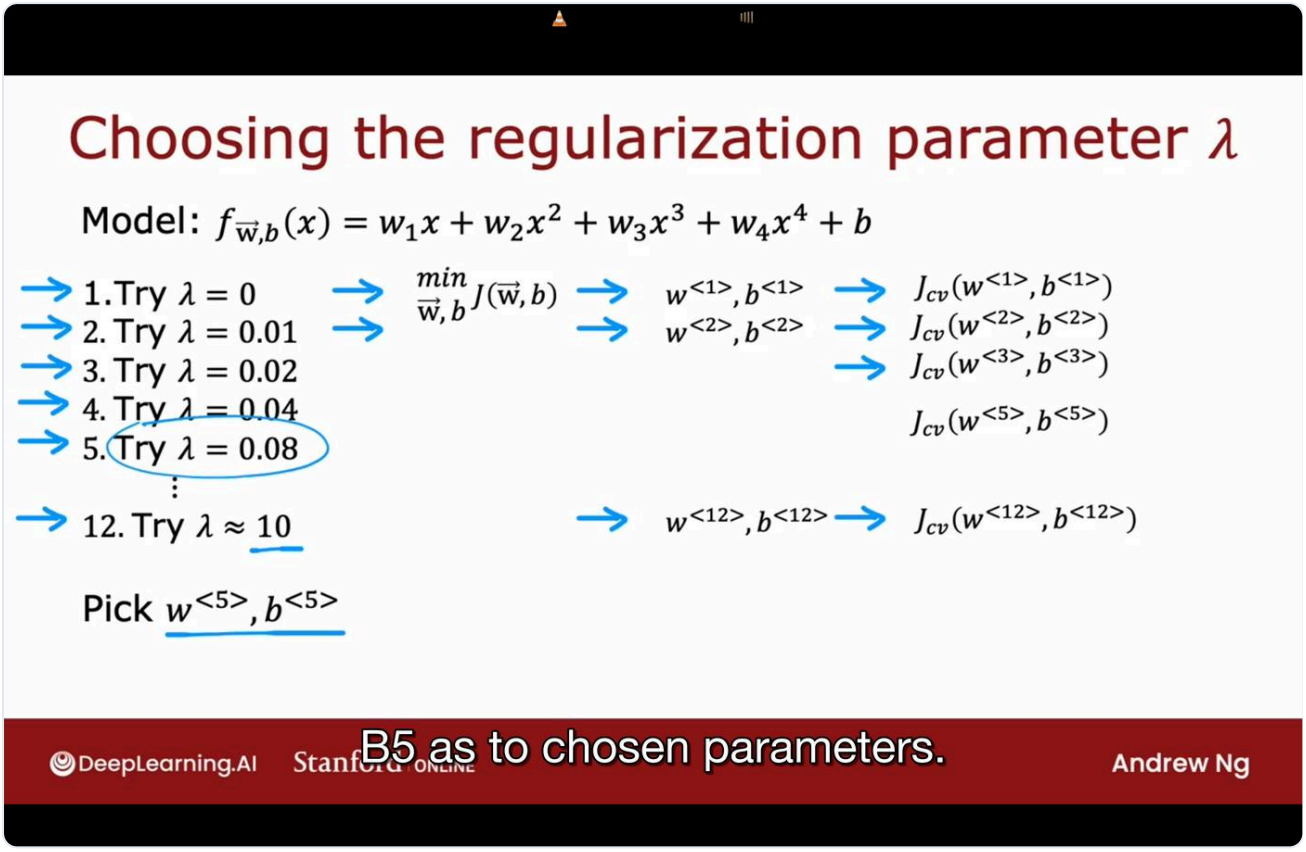


FIGURE 37.7 | Choosing the Regularization Parameter a

If in this example, you find that J_cv of W5, B5 has the lowest value of all of these different cross-validation errors, you might then decide to pick this value for Lambda, and so use W5, B5 as to chosen parameters.

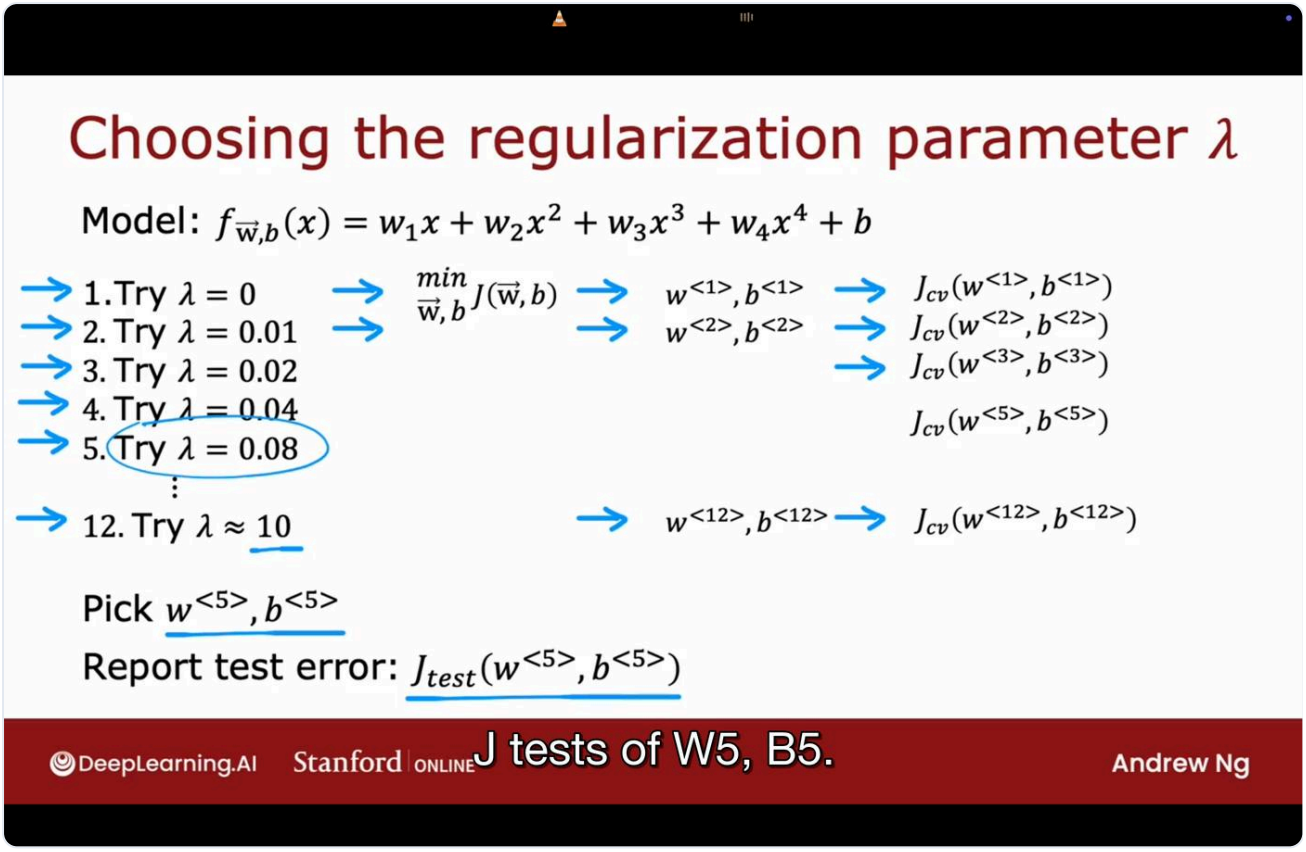


FIGURE 37.8 | Choosing the Regularization Parameter a

Finally, if you want to report out an estimate of the generalization error, you would then report out the test set error, J tests of W5, B5. To further hone intuition about what this algorithm is doing, let's take a look at how training error and cross validation error vary as a function of the parameter Lambda. In this figure, I've changed the x-axis again. Notice that the x-axis here is annotated with the value of the regularization parameter Lambda, and if we look at the extreme of Lambda equals zero here on the left, that corresponds to not using any regularization, and so that's where we wound up with this very wiggly curve. If Lambda was small or it was even zero, and in that case, we have a high variance model, and so J train is going to be small and J_cv is going to be large because it does great on the training data but does much worse on the cross validation data.

This extreme on the right were very large values of Lambda. Say Lambda equals 10,000 ends up with fitting a model that looks like that. This has high bias, it underfits the data, and it turns out J train will be high and J_cv will be high as well. In fact, if you were to look at how J train varies as a function of Lambda, you find that J train will go up like this because in the optimization cost function, the larger Lambda is, the more the algorithm is trying to keep W squared small. That is, the more weight is given to this regularization term, and thus the less attention is paid to actually do well on the training set. This term on the left is J train, so the most trying to keep the parameters small, the less good a job it does on minimizing the training error.

That's why as Lambda increases, the training error J train will tend to increase like so. Now, how about the cross-validation error? Turns out the cross-validation error will look like this.

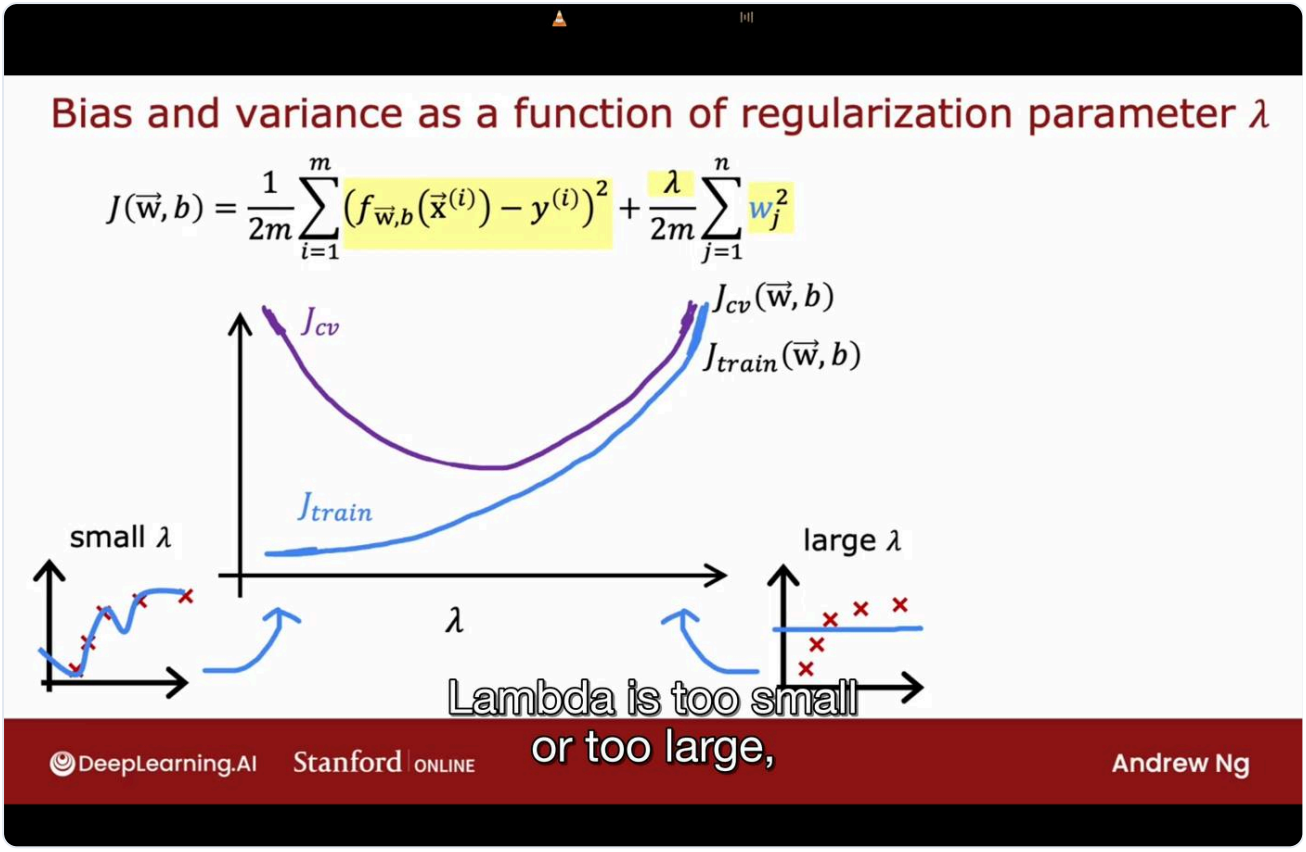


FIGURE 37.9

Because we've seen that if Lambda is too small or too large, then it doesn't do well on the cross-validation set. It either overfits here on the left or underfits here on the right. There'll be some intermediate value of Lambda that causes the algorithm to perform best. What cross-validation is doing is, it's trying out a lot of different values of Lambda. This is what we saw on the last slide; trial Lambda equals zero, Lambda equals 0.01, logic is 0,02.

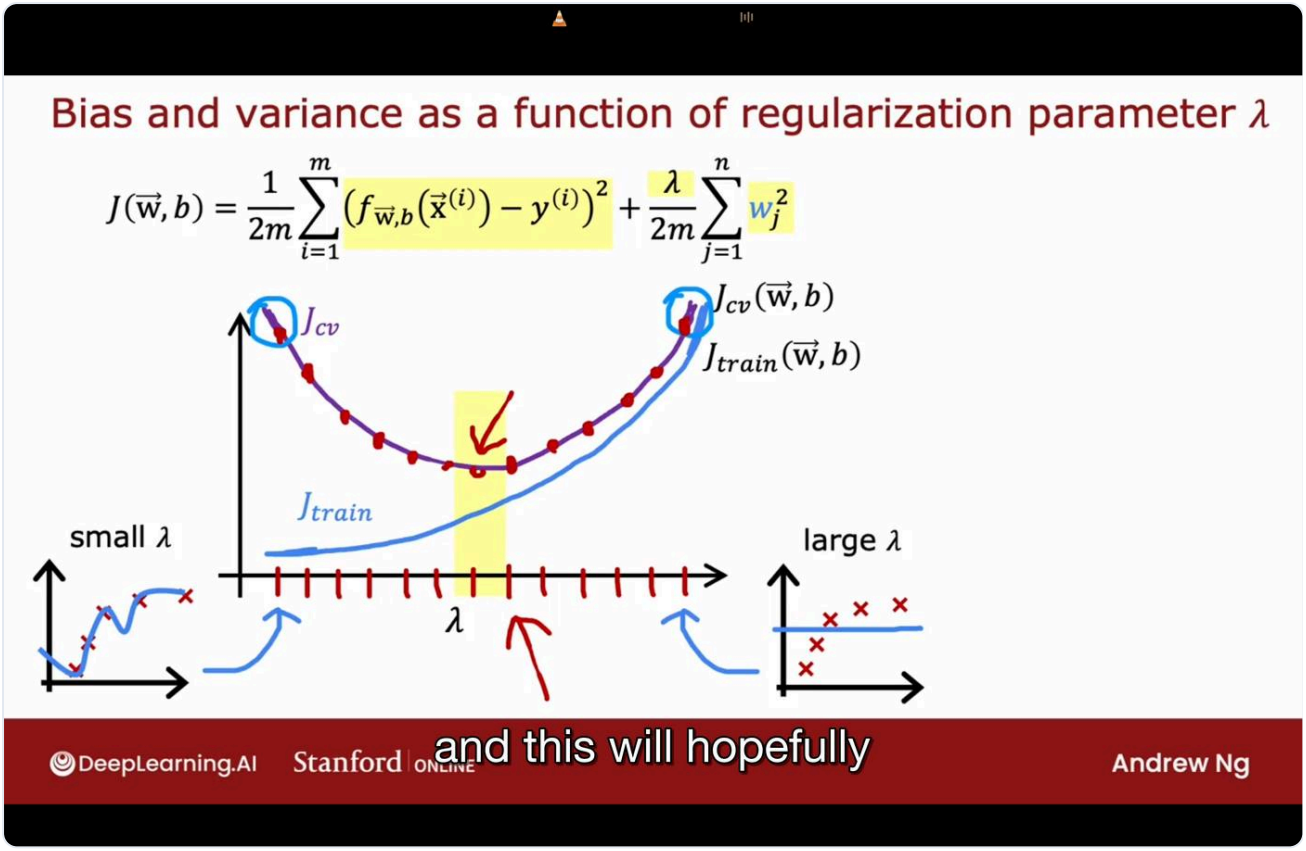


FIGURE 37.10

Try a lot of different values of Lambda and evaluate the cross-validation error in a lot of these different points, and then hopefully pick a value that has low cross validation error, and this will hopefully correspond to a good model for your application.

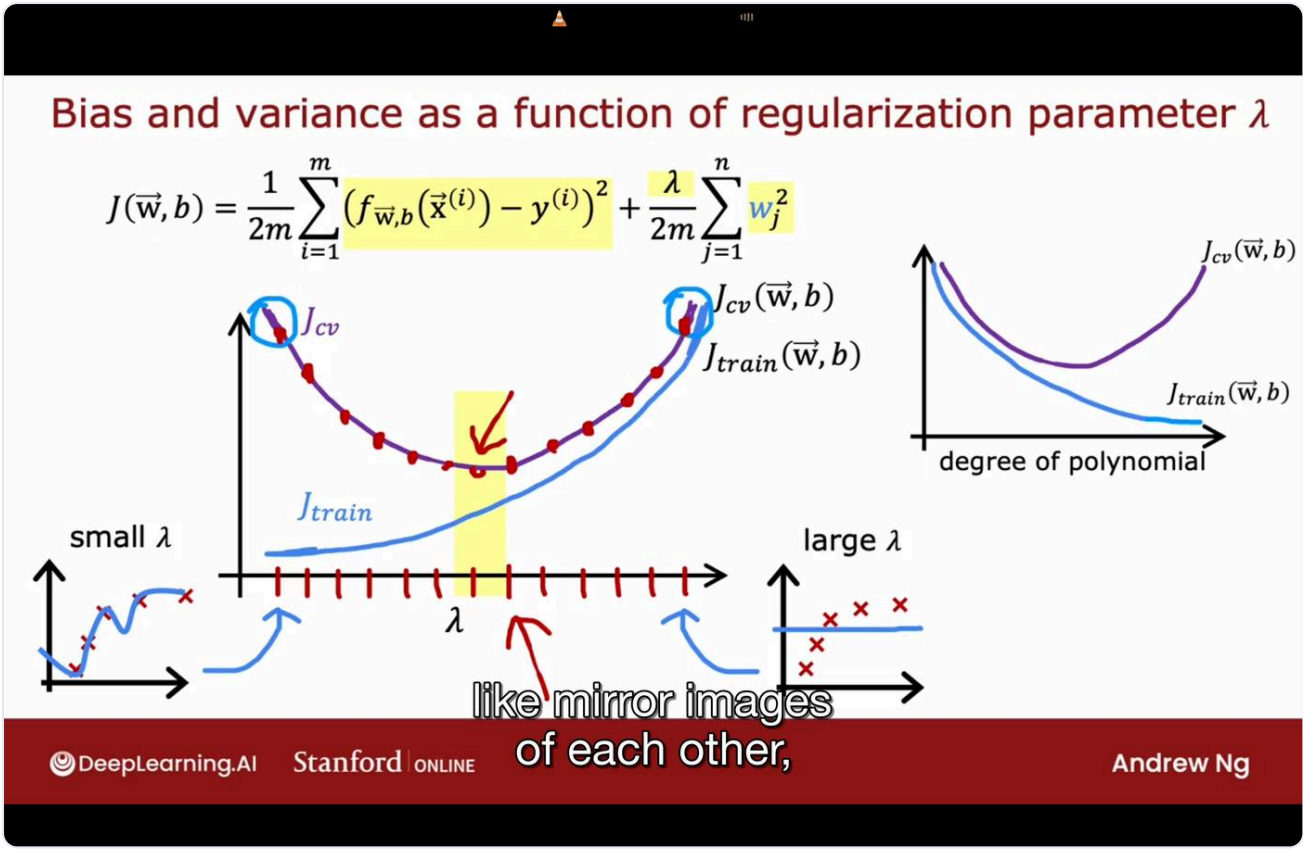


FIGURE 37.11

If you compare this diagram to the one that we had in the previous video, where the horizontal axis was the degree of polynomial, these two diagrams look a little bit not mathematically and not in any formal way, but they look a little bit like mirror images of each other, and that's because when you're fitting a degree of polynomial, the left part of this curve corresponded to underfitting and high bias, the right part corresponded to overfitting and high variance. Whereas in this one, high-variance was on the left and high bias was on the right. But that's why these two images are a little bit like mirror images of each other. But in both cases, cross-validation, evaluating different values can help you choose a good value of t or a good value of Lambda.

That's how the choice of regularization parameter Lambda affects the bias and variance and overall performance of your algorithm, and you've also seen how you can use cross-validation to make a good choice for the regularization parameter Lambda. Now, so far, we've talked about how having a high training set error, high J train is indicative of high bias and how having a high cross-validation error of J_cv, specifically if it's much higher than J train, how that's indicative of variance problem. But what does these words "high" or "much higher" actually mean? Let's take a look at that in the next video where we'll look at how you can look at the numbers J train and J_cv and judge if it's high or low, and it turns out that one further refinement of these ideas, that is, establishing a baseline level of performance we're learning algorithm will make it much easier for you to look at these numbers, J train, J_cv, and judge if they are high or low.

Let's take a look at what all this means in the next video.

LECTURE 38

Bias/variance Examples

Let's look at some concrete numbers for what J-train and JCV might be, and see how you can judge if a learning algorithm has high bias or high variance. For the examples in this video, I'm going to use as a running example the application of speech recognition which is something I've worked on multiple times over the years. Let's take a look. A lot of users doing web search on a mobile phone will use speech recognition rather than type on the tiny keyboards on our phones because speaking to a phone is often faster than typing. Typical audio that's a web search engine we get would be like this, "What is today's weather? Or like this, "Coffee shops near me.

It's the job of the speech recognition algorithms to output the transcripts whether it's today's weather or coffee shops near me. Now, if you were to train a speech recognition system and measure the training error, and the training error means what's the percentage of audio clips in your training set that the algorithm does not transcribe correctly in its entirety. Let's say the training error for this data-set is 10.8 percent meaning that it transcribes it perfectly for 89.2 percent of your training set, but makes some mistake in 10.8 percent of your training set. If you were to also measure your speech recognition algorithm's performance on a separate cross-validation set, let's say it gets 14.8 percent error.

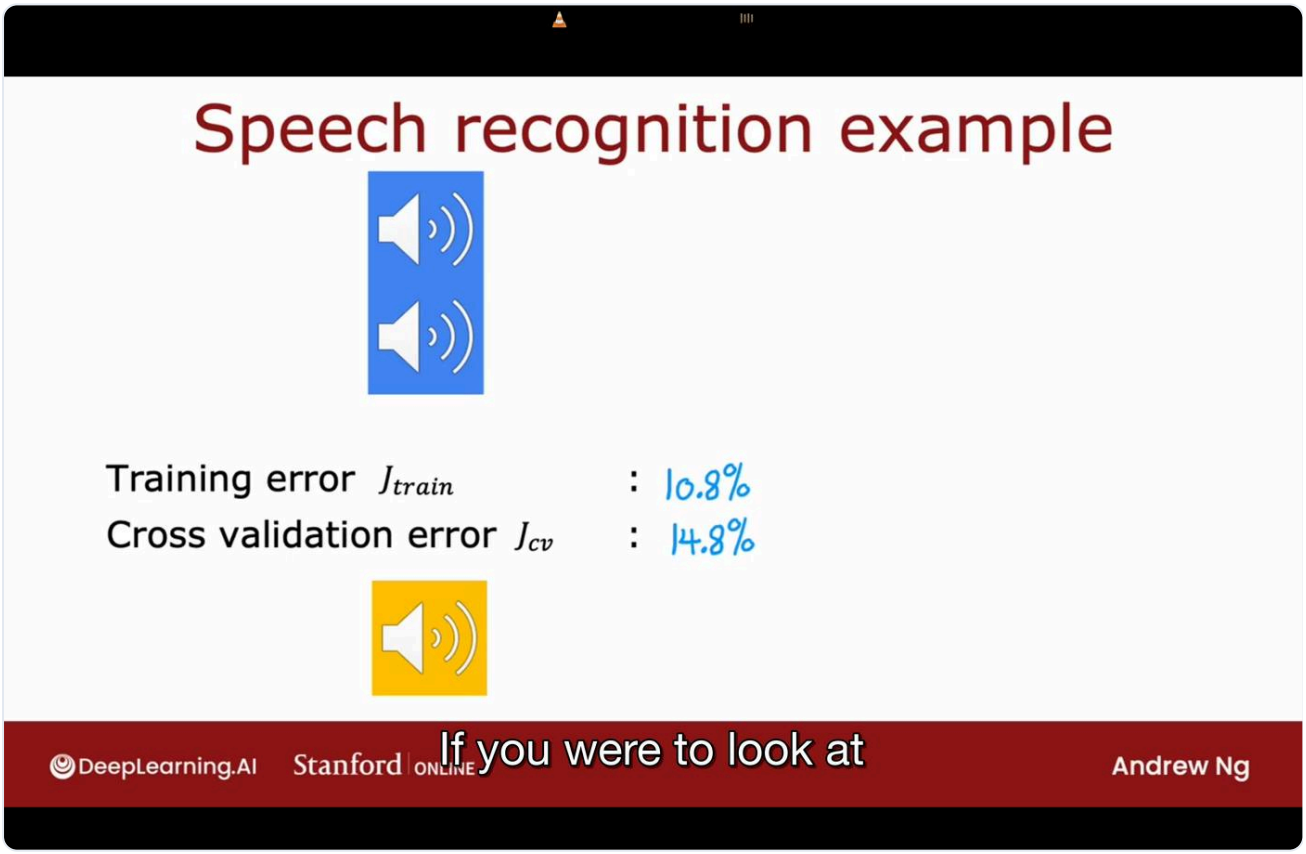


FIGURE 38.1 | Speech Recognition Example

If you were to look at these numbers it looks like the training error is really high, it got 10 percent wrong, and then the cross-validation error is higher but getting 10 percent of even your training set wrong that seems pretty high. It seems like that 10 percent error would lead you to conclude it has high bias because it's not doing well on your training set, but it turns out that when analyzing speech recognition it's useful to also measure one other thing which is what is the human level of performance? In other words, how well can even humans transcribe speech accurately from these audio clips? Concretely, let's say that you measure how well fluent speakers can transcribe audio clips and you find that human level performance achieves 10.6 percent error.

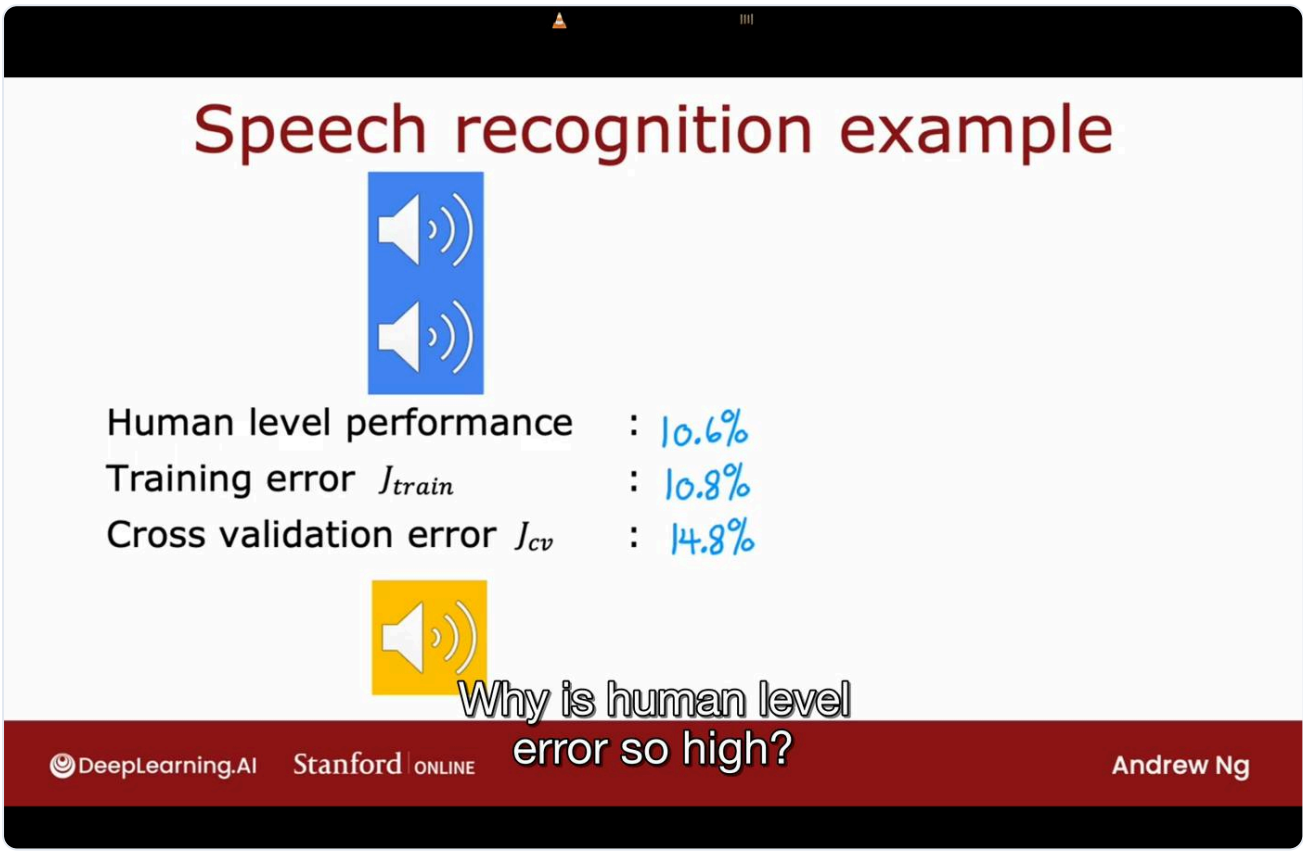


FIGURE 38.2 | Speech Recognition Example

Why is human level error so high? It turns out that for web search, there are a lot of audio clips that sound like this, "I'm going to navigate to [inaudible]. There's a lot of noisy audio where really no one can accurately transcribe what was said because of the noise in the audio. If even a human makes 10.6 percent error, then it seems difficult to expect a learning algorithm to do much better. In order to judge if the training error is high, it turns out to be more useful to see if the training error is much higher than a human level of performance, and in this example it does just 0.2 percent worse than humans. Given that humans are actually really good at recognizing speech I think if I can build a speech recognition system that achieves 10.6 percent error matching human performance I'd be pretty happy, so it's just doing a little bit worse than humans.

But in contrast, the gap or the difference between JCV and J-train is much larger.

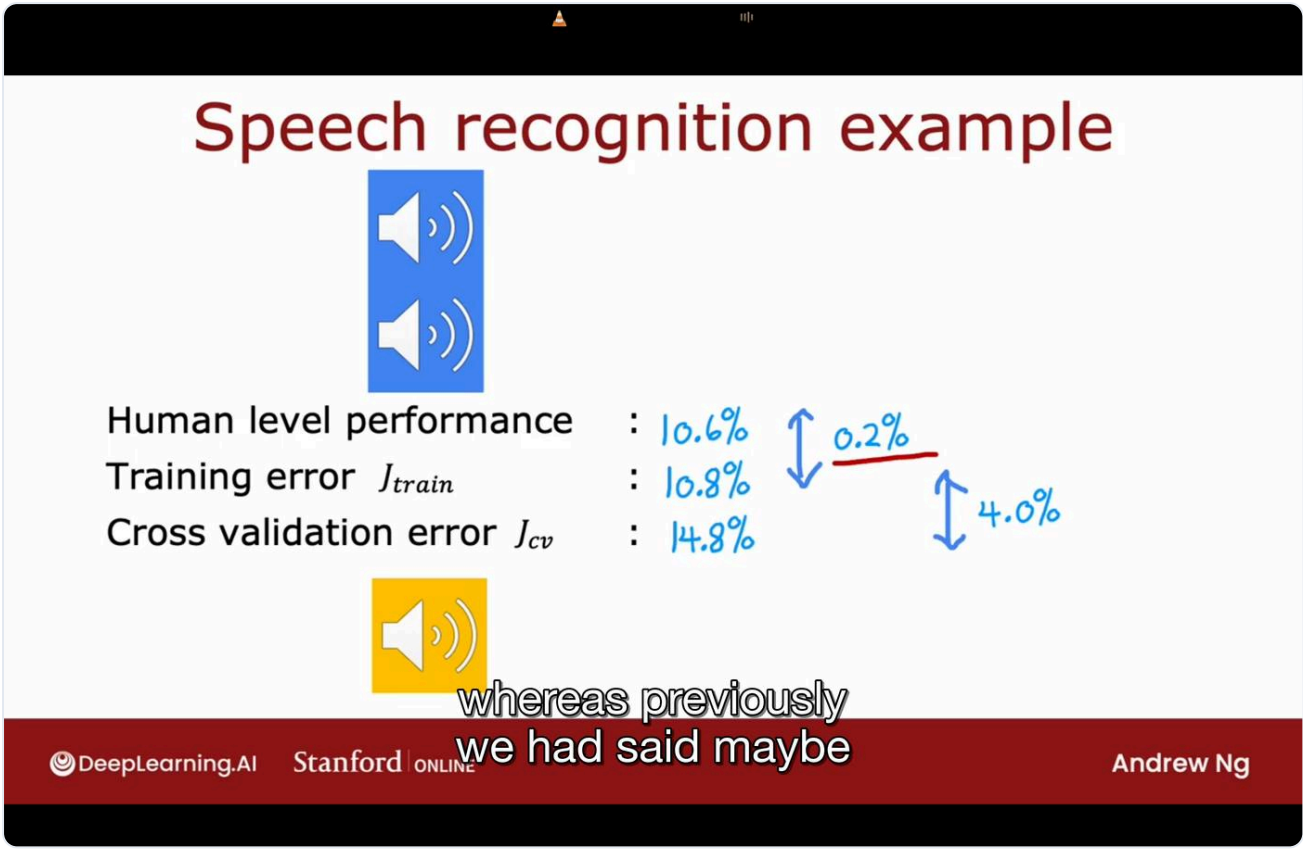


FIGURE 38.3 | Speech Recognition Example

There's actually a four percent gap there, whereas previously we had said maybe 10.8 percent error means this is high bias. When we benchmark it to human level performance, we see that the algorithm is actually doing quite well on the training set, but the bigger problem is the cross-validation error is much higher than the training error which is why I would conclude that this algorithm actually has more of a variance problem than a bias problem. It turns out when judging if the training error is high is often useful to establish a baseline level of performance, and by baseline level of performance I mean what is the level of error you can reasonably hope your learning algorithm to eventually get to.

One common way to establish a baseline level of performance is to measure how well humans can do on this task because humans are really good at understanding speech data, or processing images or understanding texts. Human level performance is often a good benchmark when you are using unstructured data, such as: audio, images, or texts. Another way to estimate a baseline level of performance is if there's some competing algorithm, maybe a previous implementation that someone else has implemented or even a competitor's algorithm to establish a baseline level of performance if you can measure that, or sometimes you might guess based on prior experience.

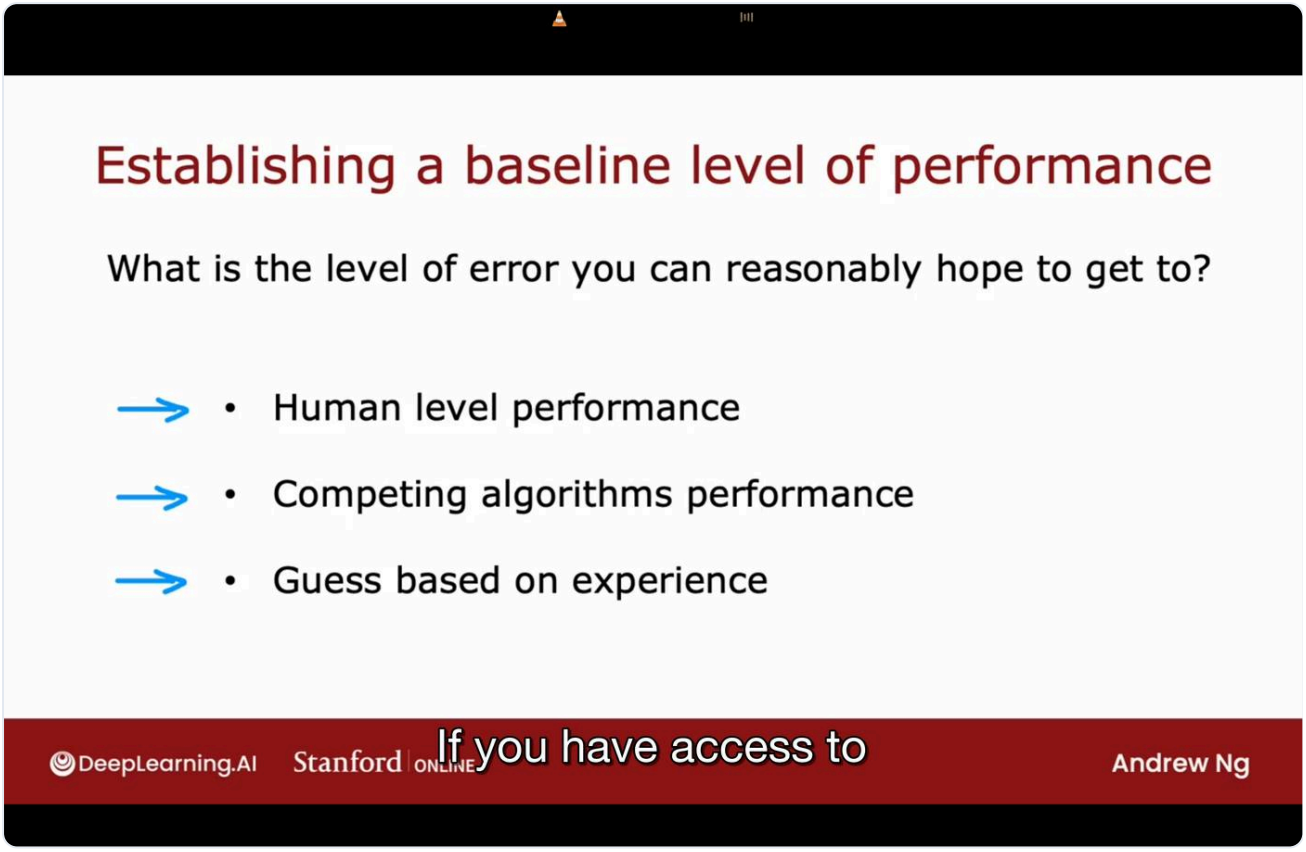


FIGURE 38.4 | Establishing a Baseline Level of Performance

If you have access to this baseline level of performance that is, what is the level of error you can reasonably hope to get to or what is the desired level of performance that you want your algorithm to get to? Then when judging if an algorithm has high bias or variance, you would look at the baseline level of performance, and the training error, and the cross-validation error. The two key quantities to measure are then: what is the difference between training error and the baseline level that you hope to get to.

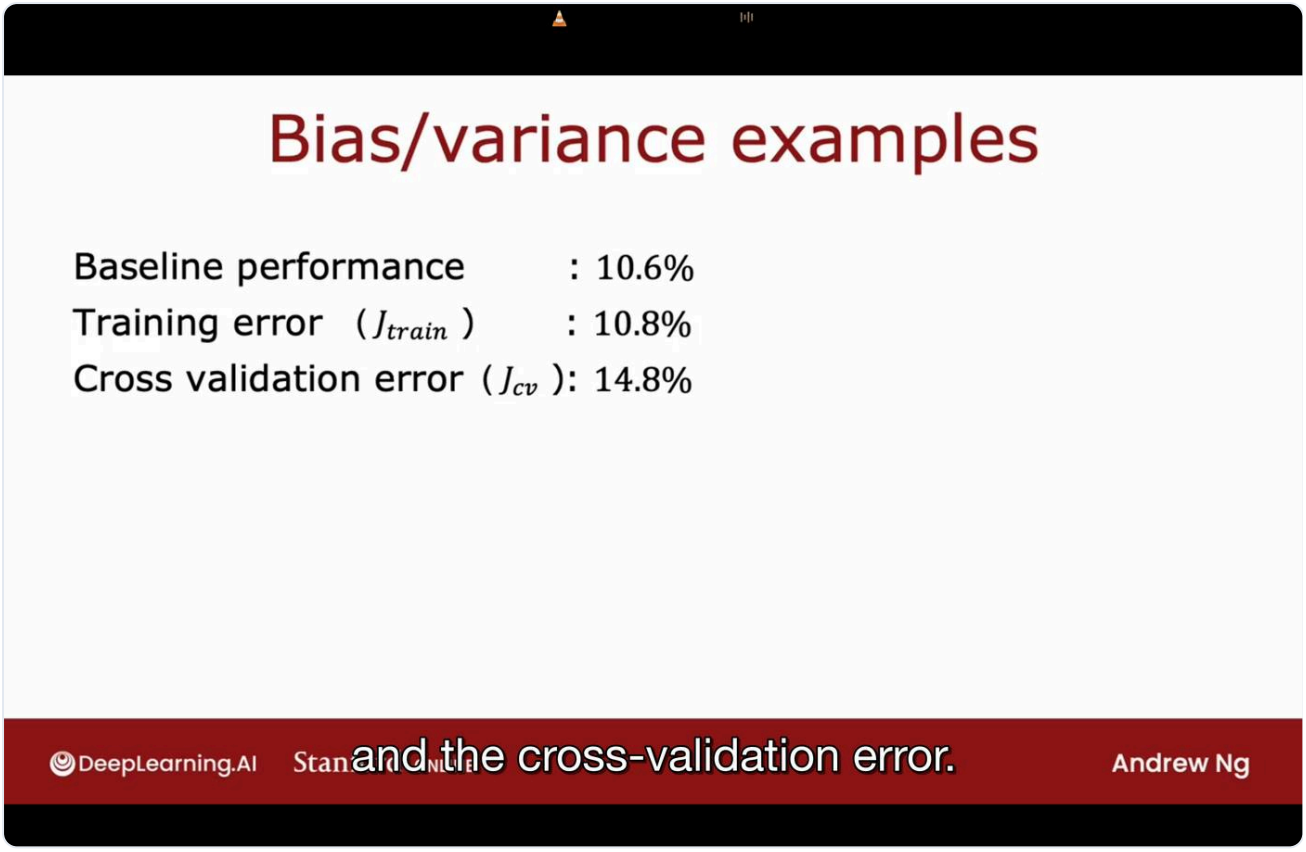


FIGURE 38.5 | Bias/variance Examples

This is 0.2, and if this is large then you would say you have a high bias problem. You will then also look at this gap between your training error and your cross-validation error, and if this is high then you will conclude you have a high variance problem.

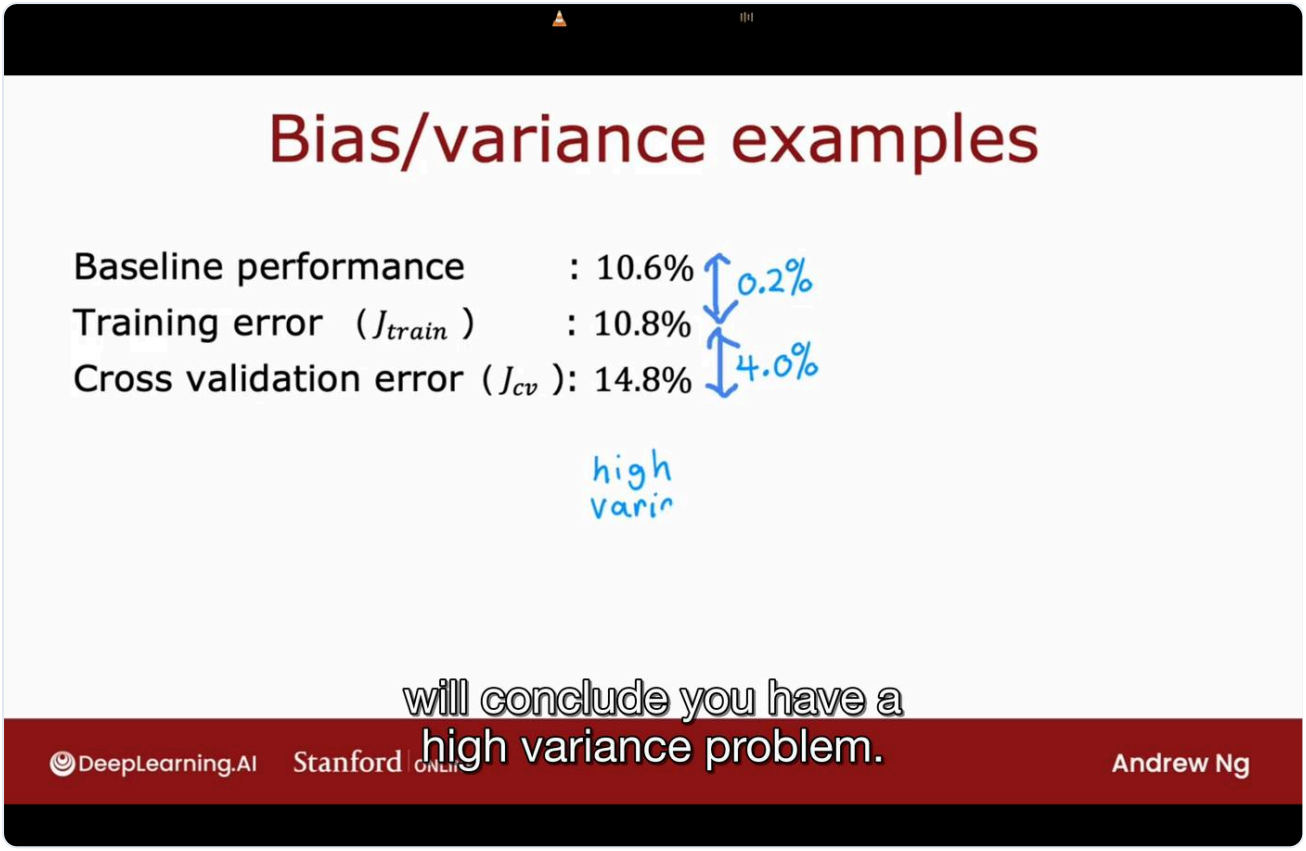


FIGURE 38.6 | Bias/variance Examples

That's why in this example we concluded we have a high variance problem, whereas let's look at the second example. If the baseline level of performance; that is human level performance, and training error, and cross validation error look like this, then this first gap is 4.4 percent and so there's actually a big gap. The training error is much higher than what humans can do and what we hope to get to whereas the cross-validation error is just a little bit bigger than the training error.

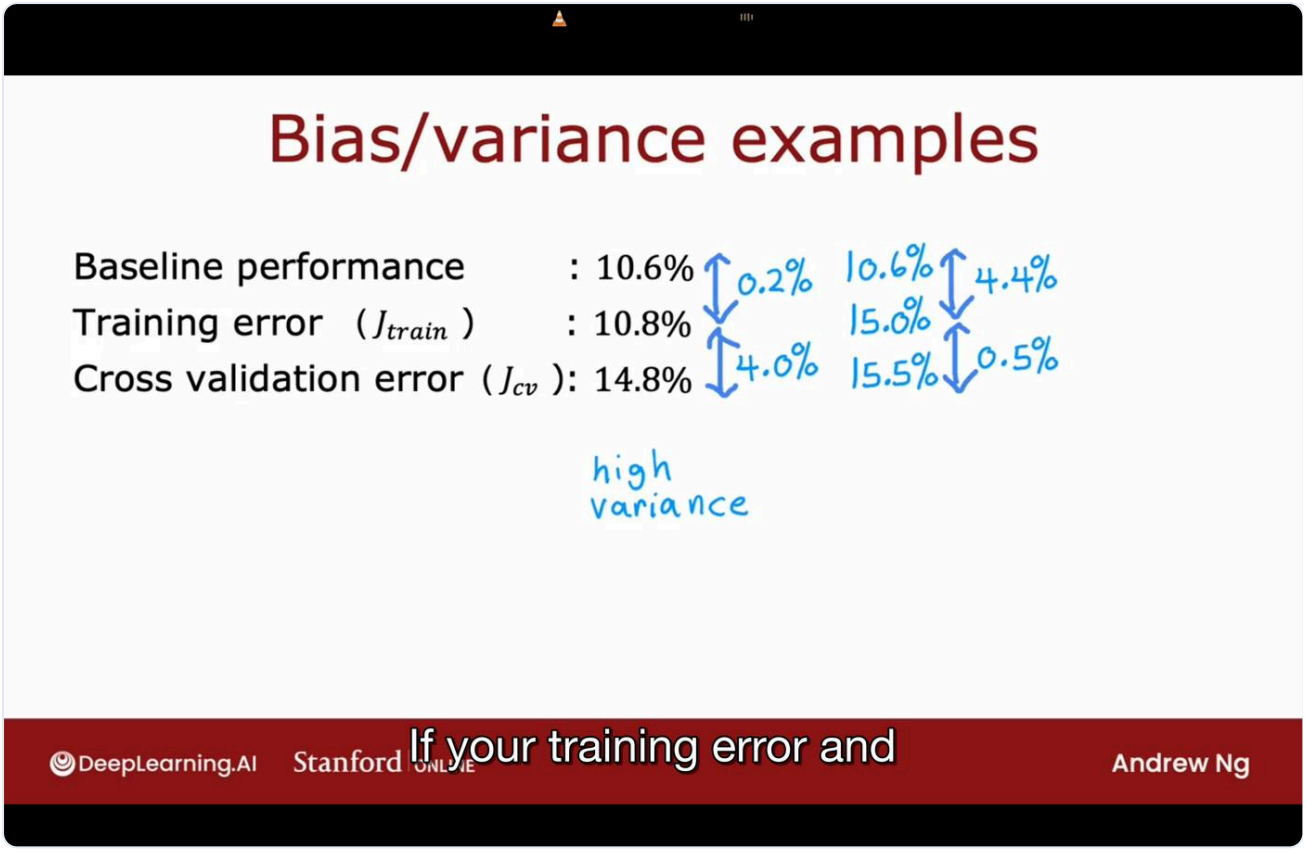


FIGURE 38.7 | Bias/variance Examples

If your training error and cross validation error look like this, I will say this algorithm has high bias.

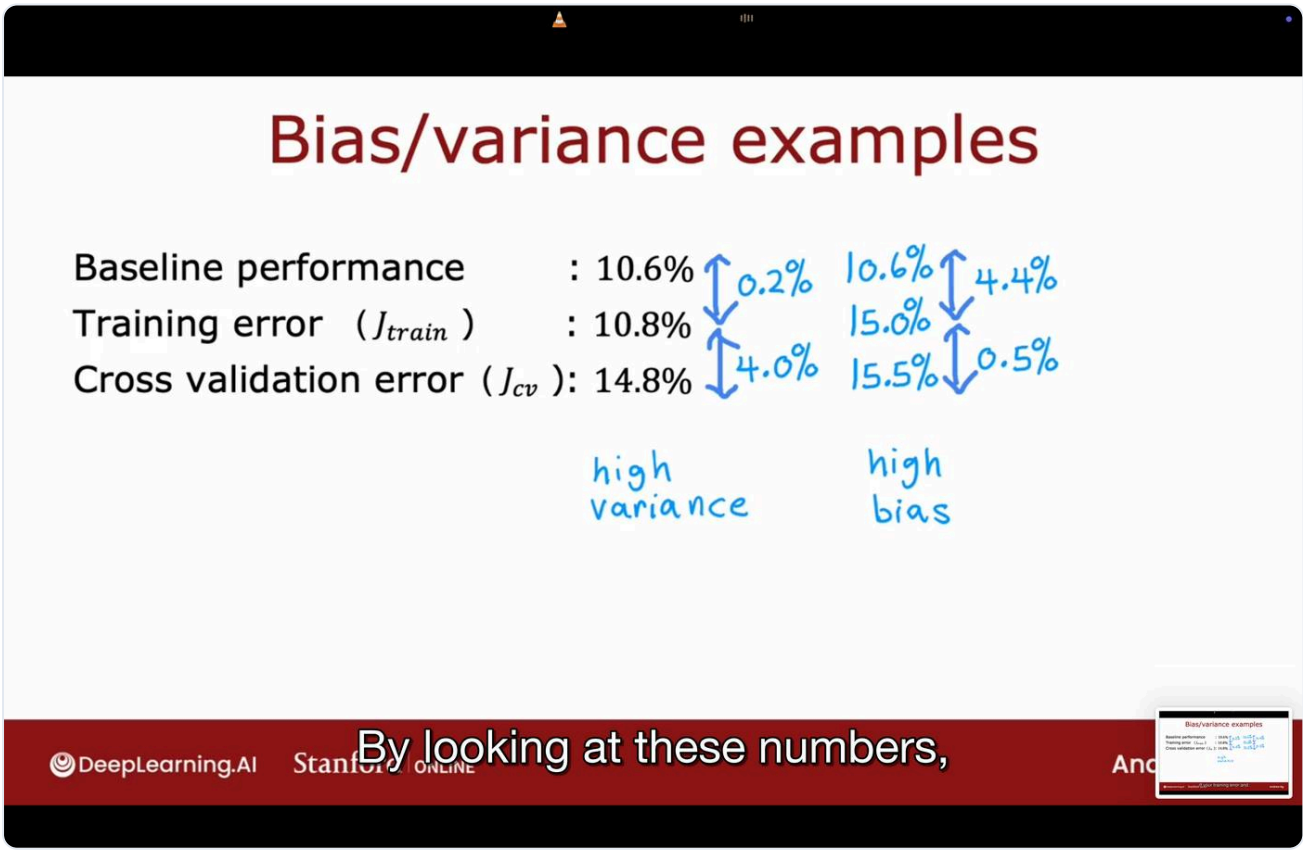


FIGURE 38.8 | Bias/variance Examples

By looking at these numbers, training error and cross validation error, you can get a sense intuitively or informally of the degree to which your algorithm has a high bias or high variance problem. Just to summarize, this gap between these first two numbers gives you a sense of whether you have a high bias problem, and the gap between these two numbers gives you a sense of whether you have a high variance problem. Sometimes the baseline level of performance could be zero percent. If your goal is to achieve perfect performance than the baseline level of performance it could be zero percent, but for some applications like the speech recognition application where some audio is just noisy then the baseline level of a performance could be much higher than zero.

The method described on this slide will give you a better read in terms of whether your algorithm suffers from bias or variance. By the way, it is possible for your algorithms to have high bias and high variance. Concretely, if you get numbers like these, then the gap between the baseline and the training error is large. That would be a 4.4 percent, and the gap between training error and cross validation error is also large. This is 4.7 percent.

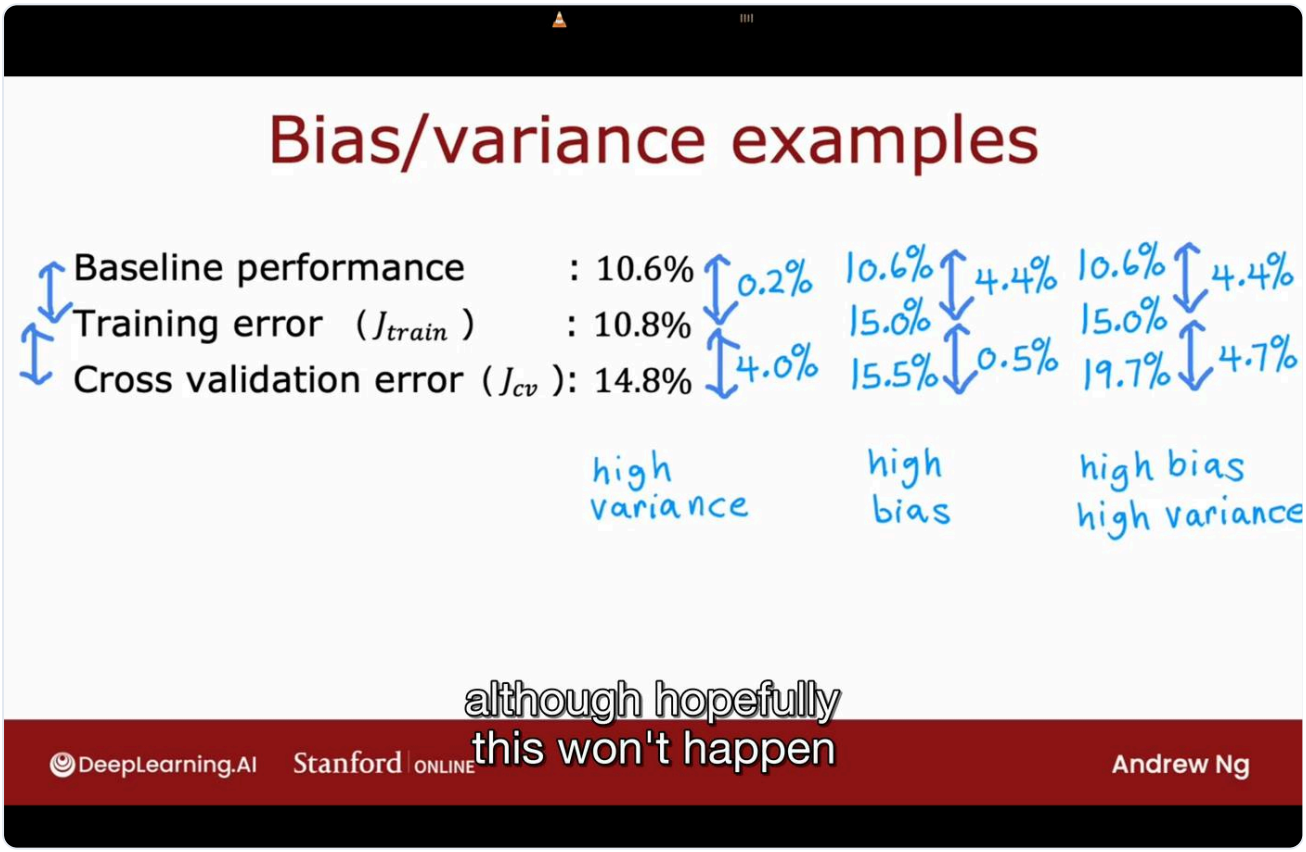


FIGURE 38.9 | Bias/variance Examples

If it looks like this you will conclude that your algorithm has high bias and high variance, although hopefully this won't happen that often for your learning applications. To summarize, we've seen that looking at whether your training error is large is a way to tell if your algorithm has high bias, but on applications where the data is sometimes just noisy and is infeasible or unrealistic to ever expect to get a zero error then it's useful to establish this baseline level of performance. Rather than just asking is my training error a lot, you can ask is my training error large relative to what I hope I can get to eventually, such as, is my training large relative to what humans can do on the task?

That gives you a more accurate read on how far away you are in terms of your training error from where you hope to get to. Then similarly, looking at whether your cross-validation error is much larger than your training error, gives you a sense of whether or not your algorithm may have a high variance problem as well. In practice, this is how I often will look at these numbers to judge if my learning algorithm has a high bias or high variance problem. Now, to further hone our intuition about how a learning algorithm is doing, there's one other thing that I found useful to think about which is the learning curve. Let's take a look at what that means in the next video.

LECTURE 39

High Bias

Learning curves are a way to help understand how your learning algorithm is doing as a function of the amount of experience it has, whereby experience, I mean, for example, the number of training examples it has. Let's take a look. Let me plot the learning curves for a model that fits a second-order polynomial quadratic function like so. I'm going to plot both J_cv, the cross-validation error, as well as J_train the training error. On this figure, the horizontal axis is going to be m_train. That is the training set size or the number of examples so the algorithm can learn from. On the vertical axis, I'm going to plot the error. By error, I mean either J_cv or J_train.

Let's start by plotting the cross-validation error. It will look something like this. That's what J_cv of (w, b) will look like. Is maybe no surprise that as m_train, the training set size gets bigger, then you learn a better model and so the cross-validation error goes down. Now, let's plot J_train of (w, b) of what the training error looks like as the training set size gets bigger. It turns out that the training error will actually look like this. That as the training set size gets bigger, the training set error actually increases. Let's take a look at why this is the case. We'll start with an example of when you have just a single training example.

Well, if you were to fit a quadratic model to this, you can fit easiest straight line or a curve and your training error will be zero. How about if you have two training examples like this? Well, you can again fit a straight line and achieve zero training error. In fact, if you have three training examples, the quadratic function can still fit this very well and get pretty much zero training error, but now, if your training set gets a little bit bigger, say you have four training examples, then it gets a little bit harder to fit all four examples perfectly. You may get a curve that looks like this, is a pretty well, but you're a little bit off in a few places here and there.

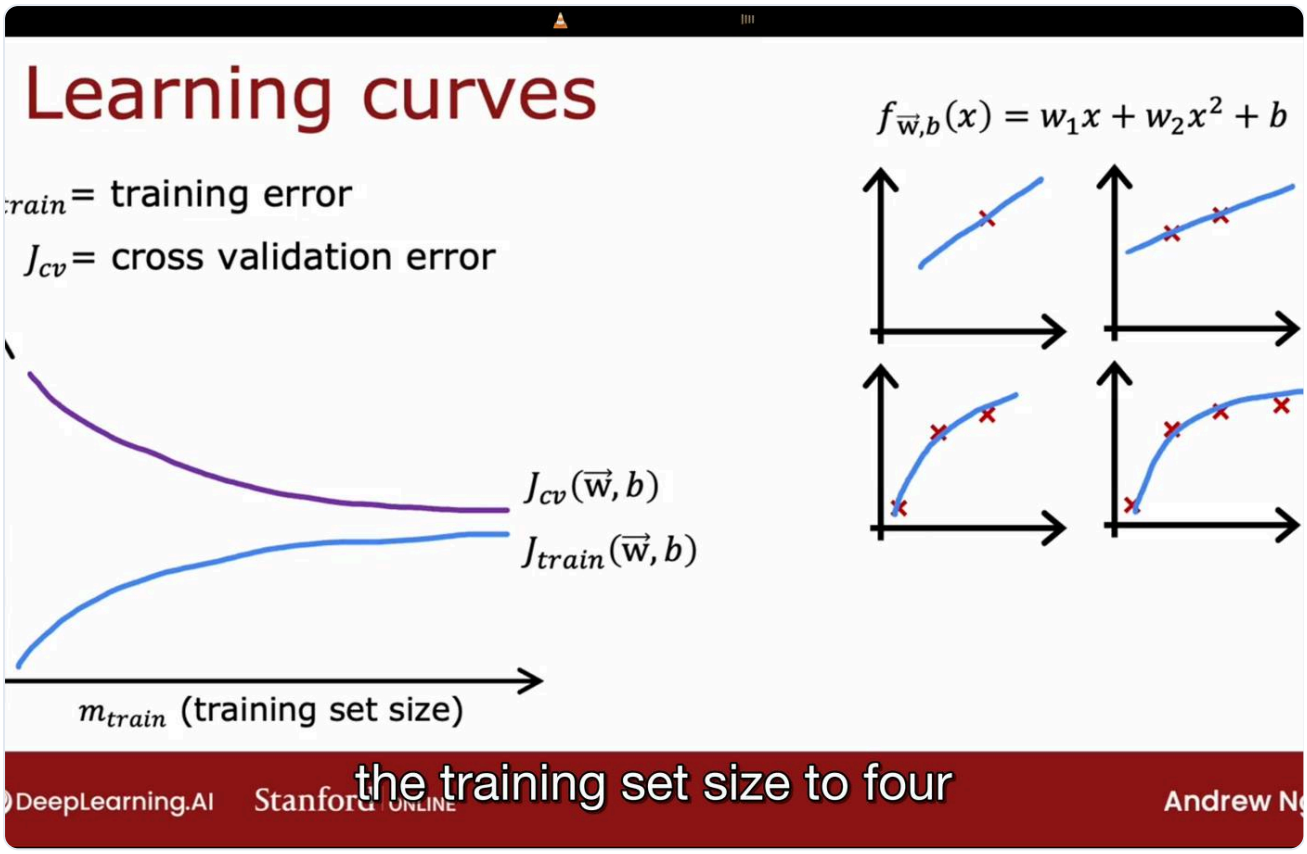


FIGURE 39.1

When you have increased, the training set size to four the training error has actually gone up a little bit. How about we have five training examples. Well again, you can fit it pretty well, but it gets even a little bit harder to fit all of them perfectly. We haven't even larger training sets it just gets harder and harder to fit every single one of your training examples perfectly.

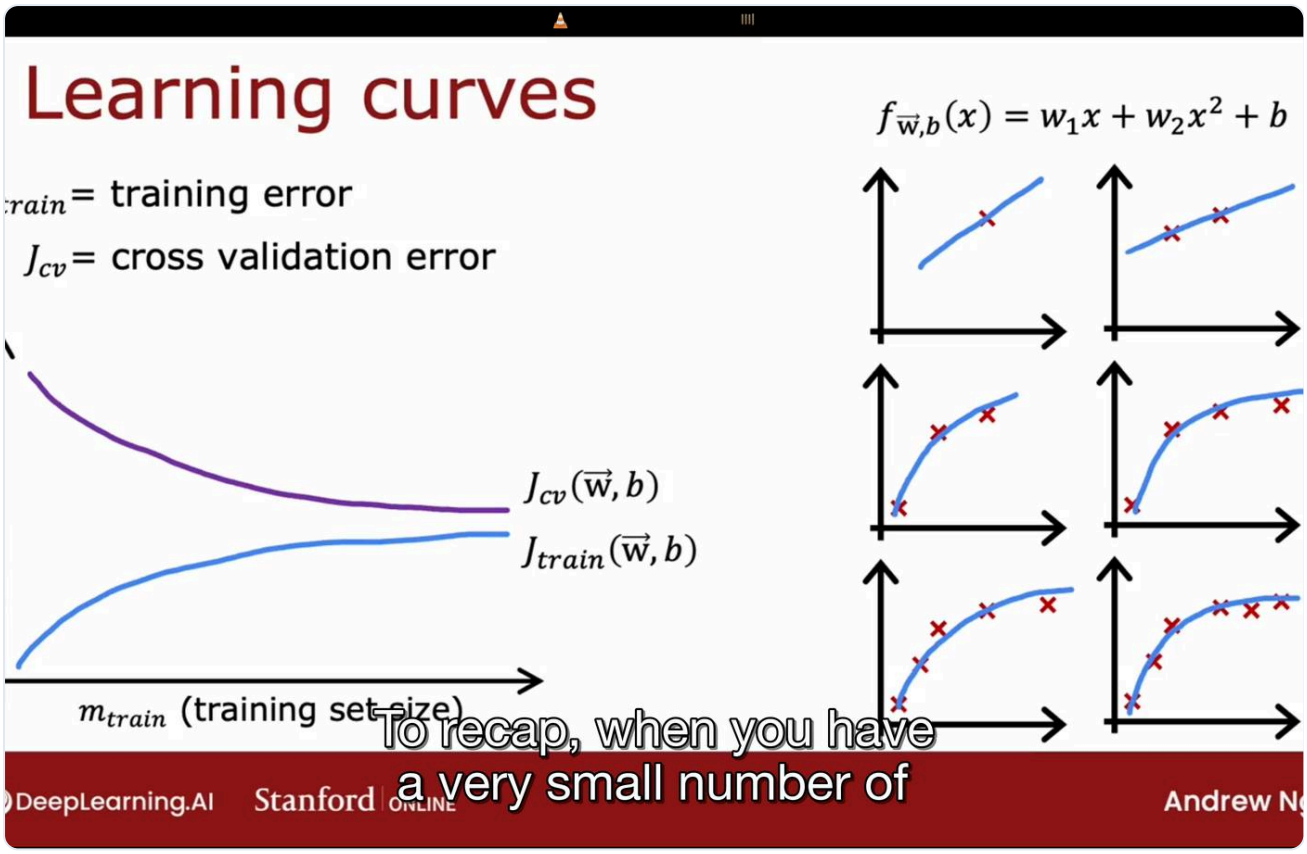


FIGURE 39.2

To recap, when you have a very small number of training examples like one or two or even three, is relatively easy to get zero or very small training error, but when you have a larger training set is harder for quadratic function to fit all the training examples perfectly. Which is why as the training set gets bigger, the training error increases because it's harder to fit all of the training examples perfectly. Notice one other thing about these curves, which is the cross-validation error, will be typically higher than the training error because you fit the parameters to the training set.

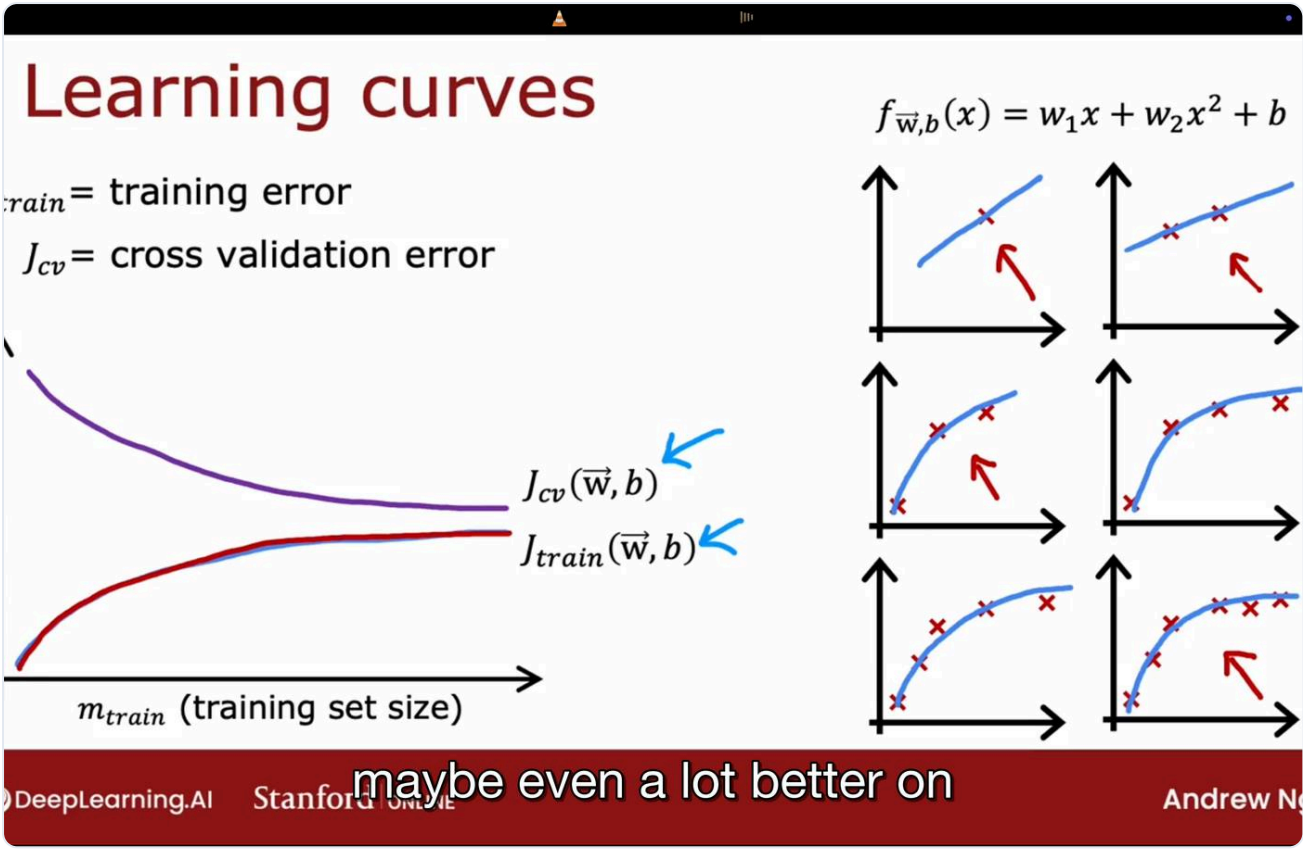


FIGURE 39.3

You expect to do at least a little bit better or when m is small, maybe even a lot better on the training set than on the trans validation set. Let's now take a look at what the learning curves will look like for an algorithm with high bias versus one with high variance. Let's start at the high bias or the underfitting case. Recall that an example of high bias would be if you're fitting a linear function, so curve that looks like this. If you were to plot the training error, then the training error will go up like so as you'd expect. In fact, this curve of training error may start to flatten out. We call it plateau, meaning flatten out after a while.

That's because as you get more and more training examples when you're fitting the simple linear function, your model doesn't actually change that much more. It's fitting a straight line and even as you get more and more and more examples, there's just not that much more to change, which is why the average training error flattens out after a while.

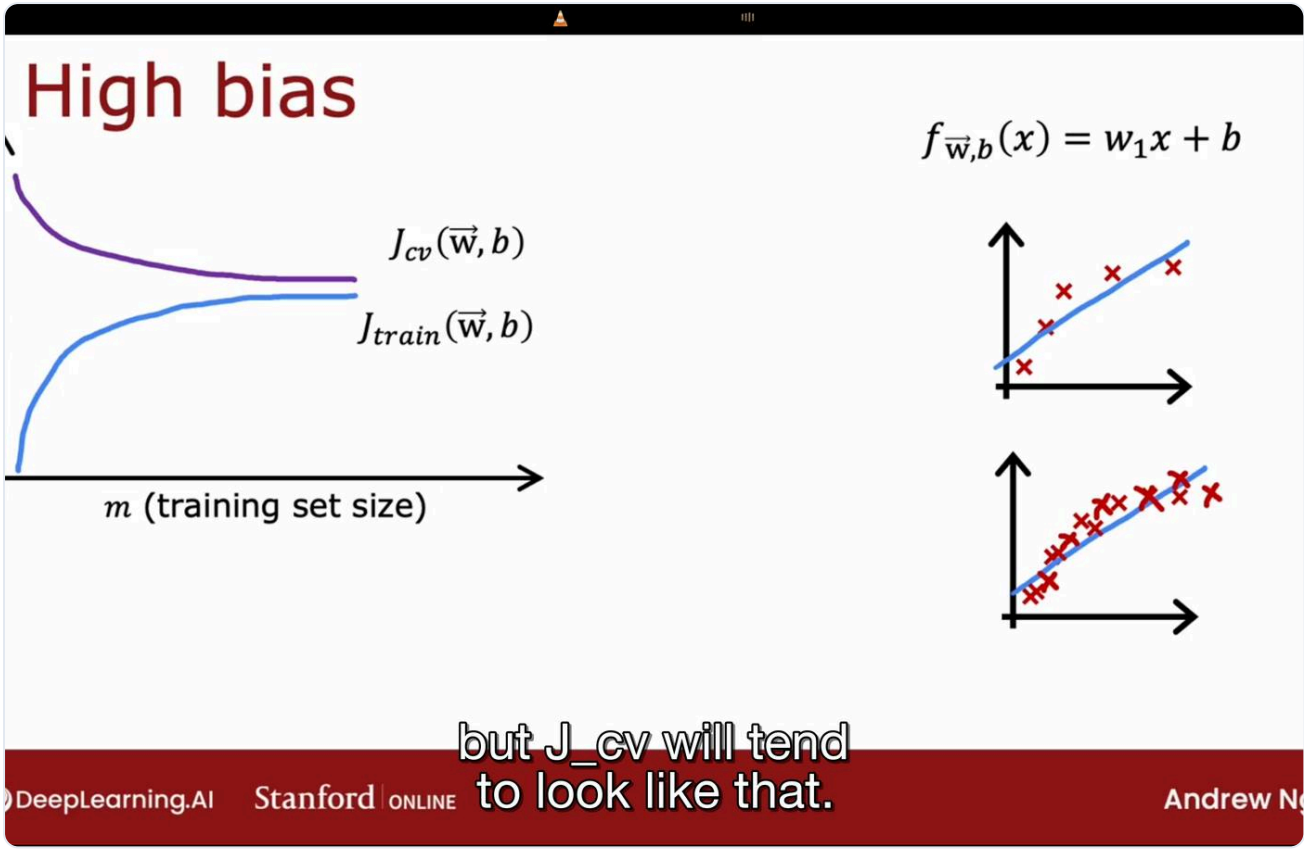


FIGURE 39.4 | High Bias

Similarly, your cross-validation error will come down and also fattened out after a while, which is why J_cv again is higher than J_train, but J_cv will tend to look like that. It's because beyond a certain point, even as you get more and more and more examples, not much is going to change about the straight line you're fitting. It's just too simple a model to be fitting into this much data. Which is why both of these curves, J_cv, and J_train tend to flatten after a while. If you had a measure of that baseline level of performance, such as human-level performance, then they'll tend to be a value that is lower than your J_train and your J_cv.

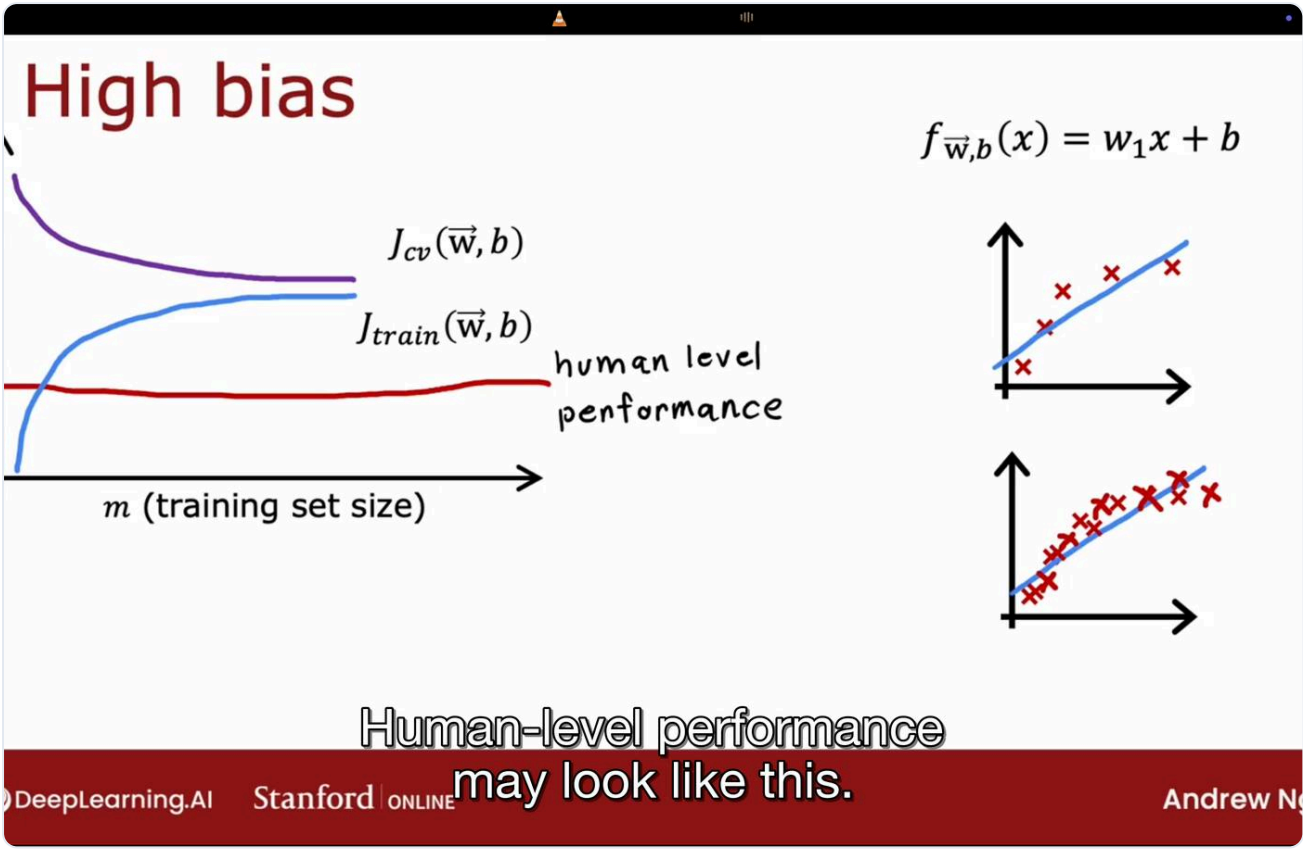


FIGURE 39.5 | High Bias

Human-level performance may look like this. There's a big gap between the baseline level of performance and J_train, which was our indicator for this algorithm having high bias. That is, one could hope to be doing much better if only we could fit a more complex function than just a straight line. Now, one interesting thing about this plot is you can ask, what do you think will happen if you could have a much bigger training set? What would it look like if we could increase even further than the right of this plot, you can go further to the right as follows?

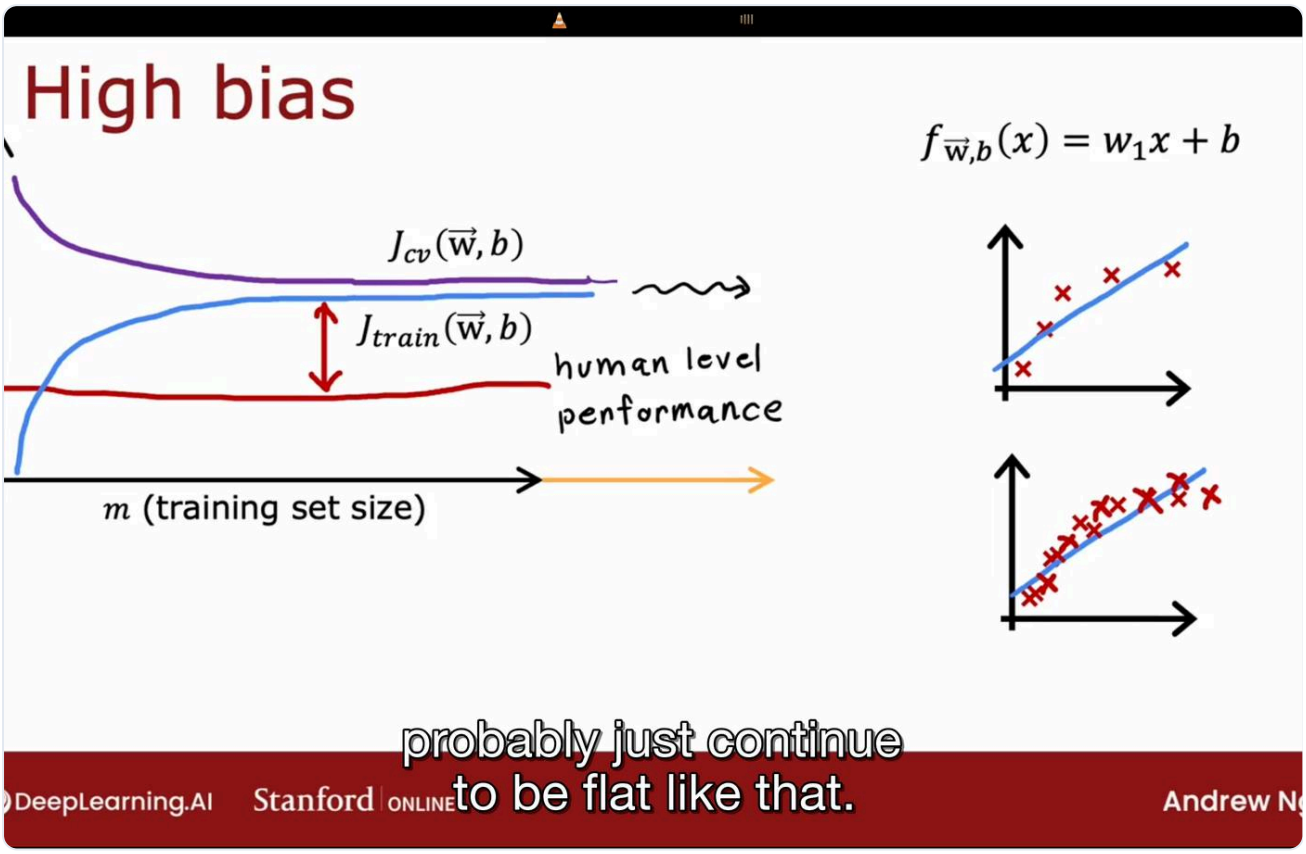


FIGURE 39.6 | High Bias

Well, you can imagine if you were to extend both of these curves to the right, they'll both flatten out and both of them will probably just continue to be flat like that.

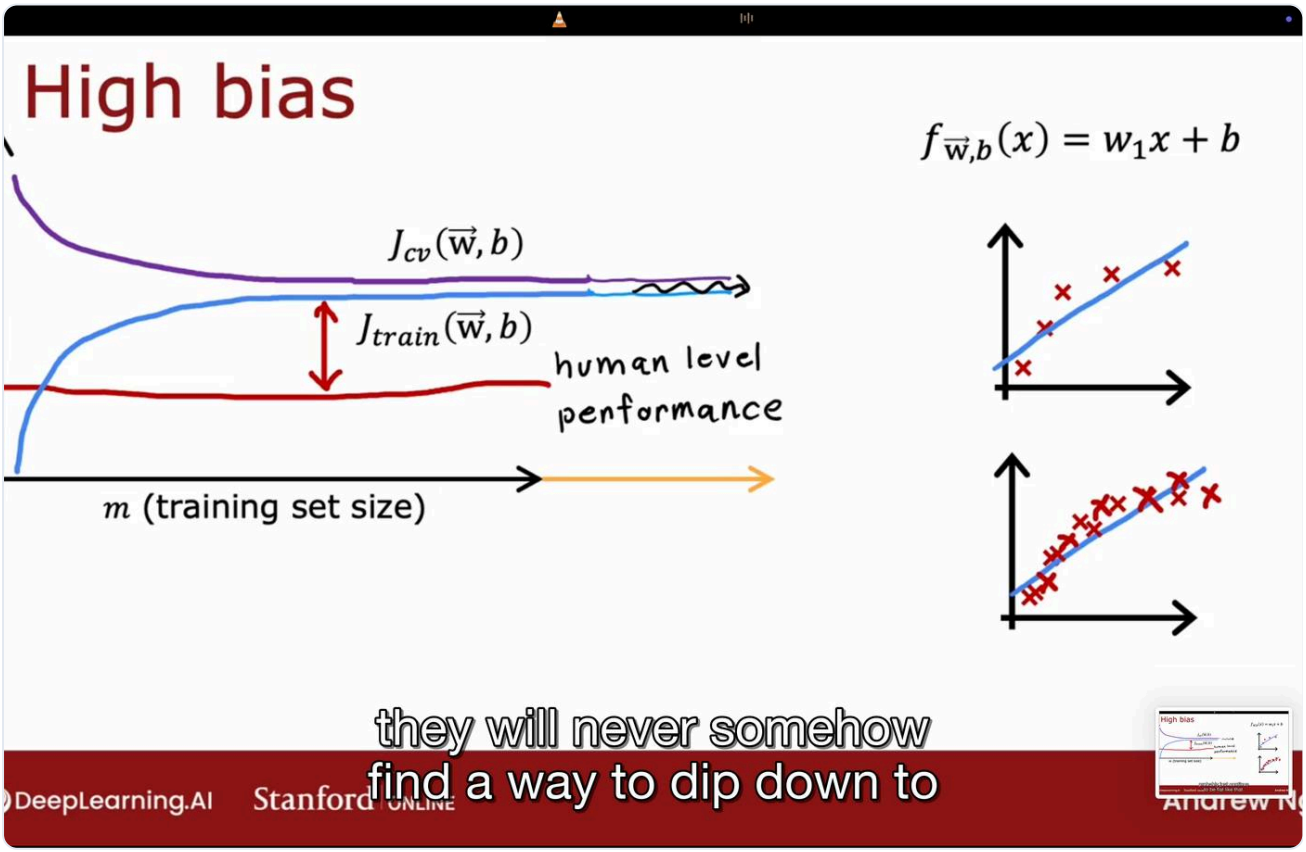


FIGURE 39.7 | High Bias

No matter how far you extend to the right of this plot, these two curves, they will never somehow find a way to dip down to this human level performance or just keep on being flat like this, pretty much forever no matter how large the training set gets.

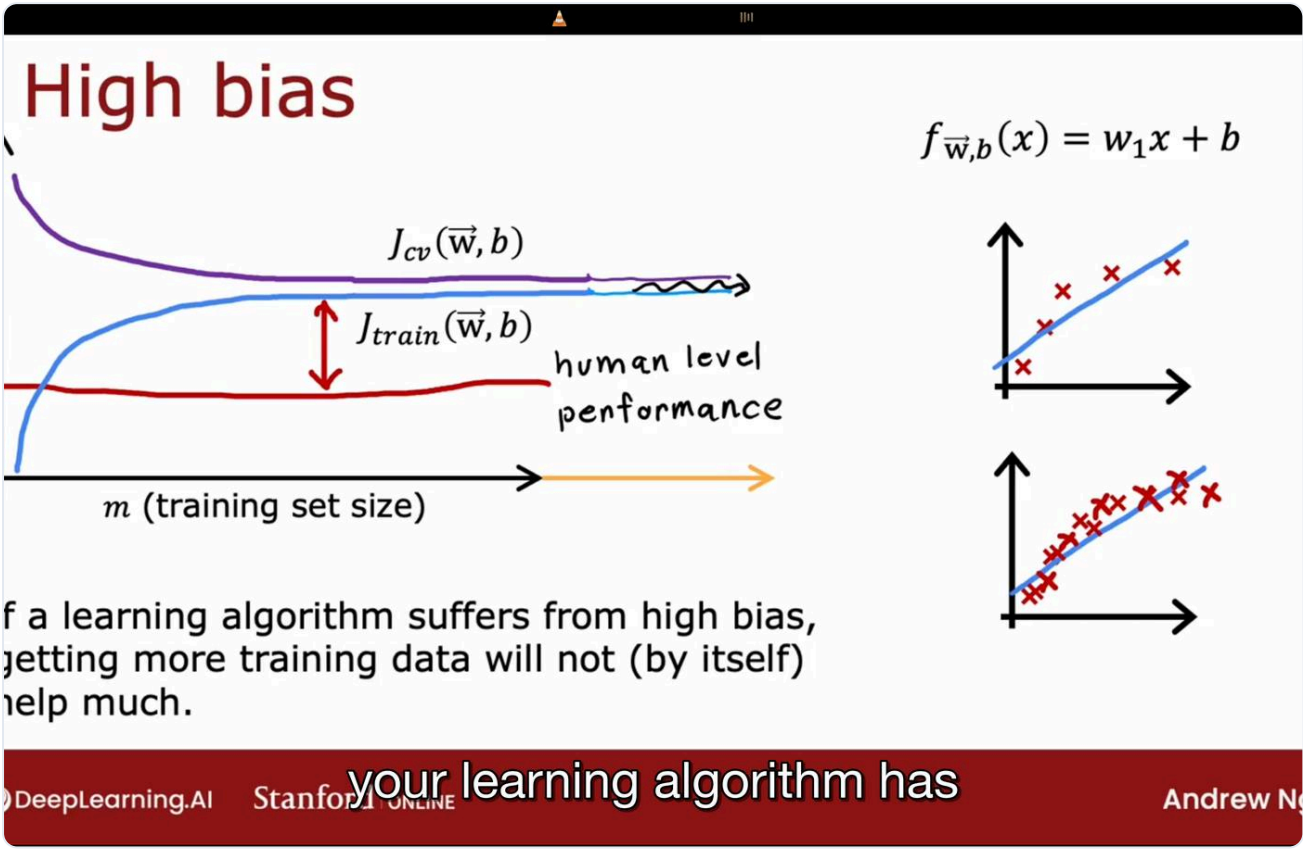


FIGURE 39.8

That gives this conclusion, maybe a little bit surprising, that if a learning algorithm has high bias, getting more training data will not by itself hope that much. I know that we're used to thinking that having more data is good, but if your algorithm has high bias, then if the only thing you do is throw more training data at it, that by itself will not ever let you bring down the error rate that much. It's because of this really, no matter how many more examples you add to this figure, the straight linear fitting just isn't going to get that much better. That's why before investing a lot of effort into collecting more training data, it's worth checking if your learning algorithm has high bias, because if it does, then you probably need to do some other things other than just throw more training data at it.

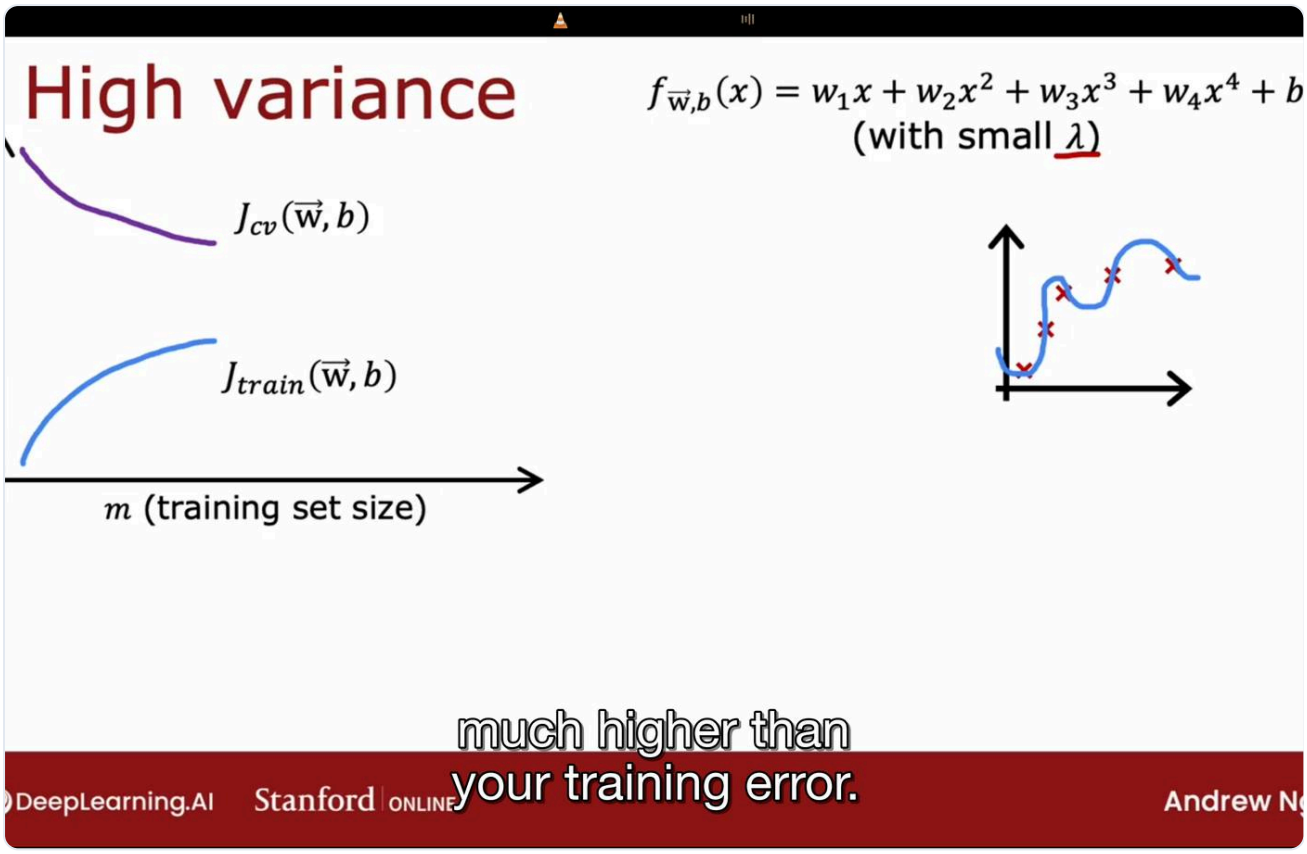


FIGURE 39.9

Let's now take a look at what the learning curve looks like for learning algorithm with high variance. You might remember that if you were to fit the fourth-order polynomial with small lambda, say, or even lambda equals zero, then you get a curve that looks like this, and even though it fits the training data very well, it doesn't generalize. Let's now look at what a learning curve might look like in this high variance scenario. J train will be going up as the training set size increases, so you get a curve that looks like this, and J cv will be much higher, so your cross-validation error is much higher than your training error.

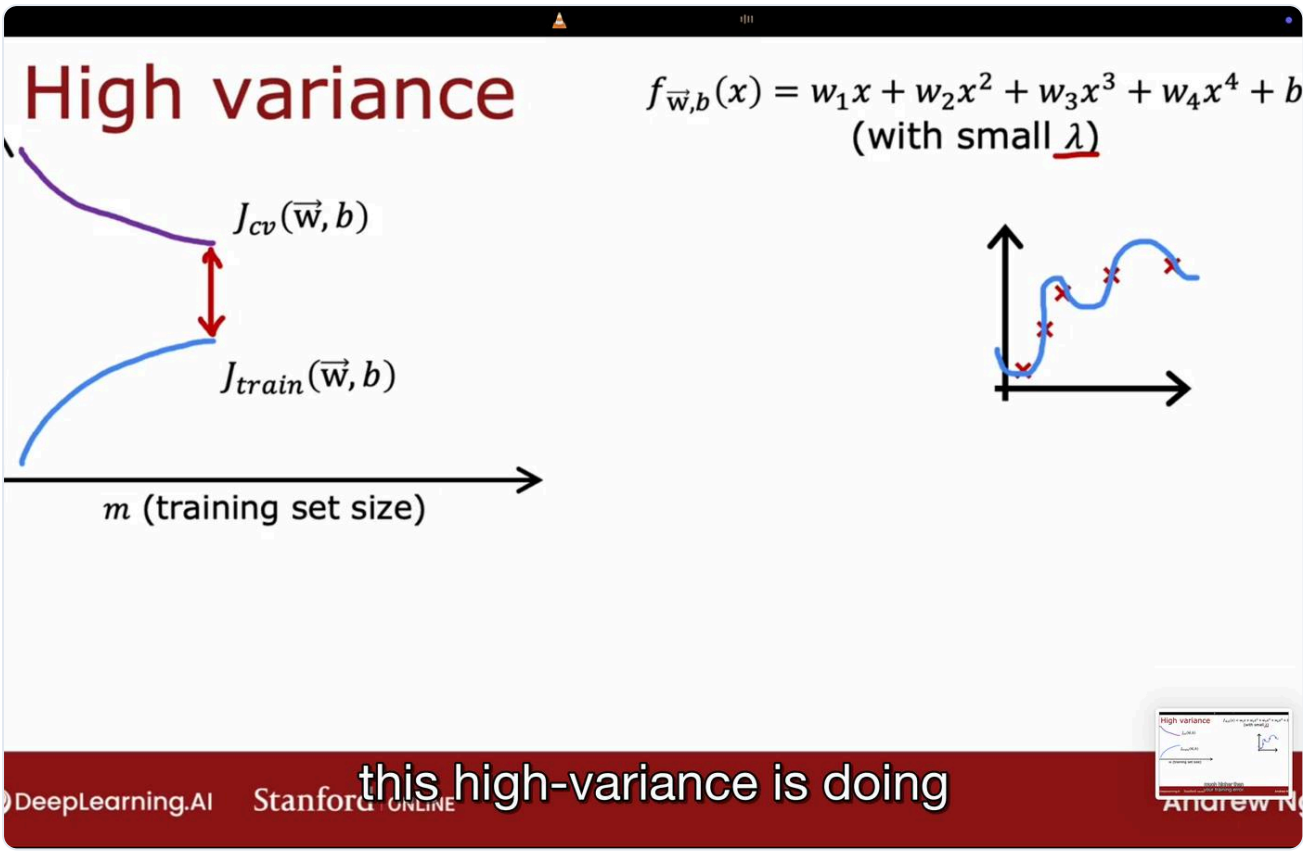


FIGURE 39.10

The fact there's a huge gap here is what I can tell you that this high-variance is doing much better on the training set than it's doing on your cross-validation set.

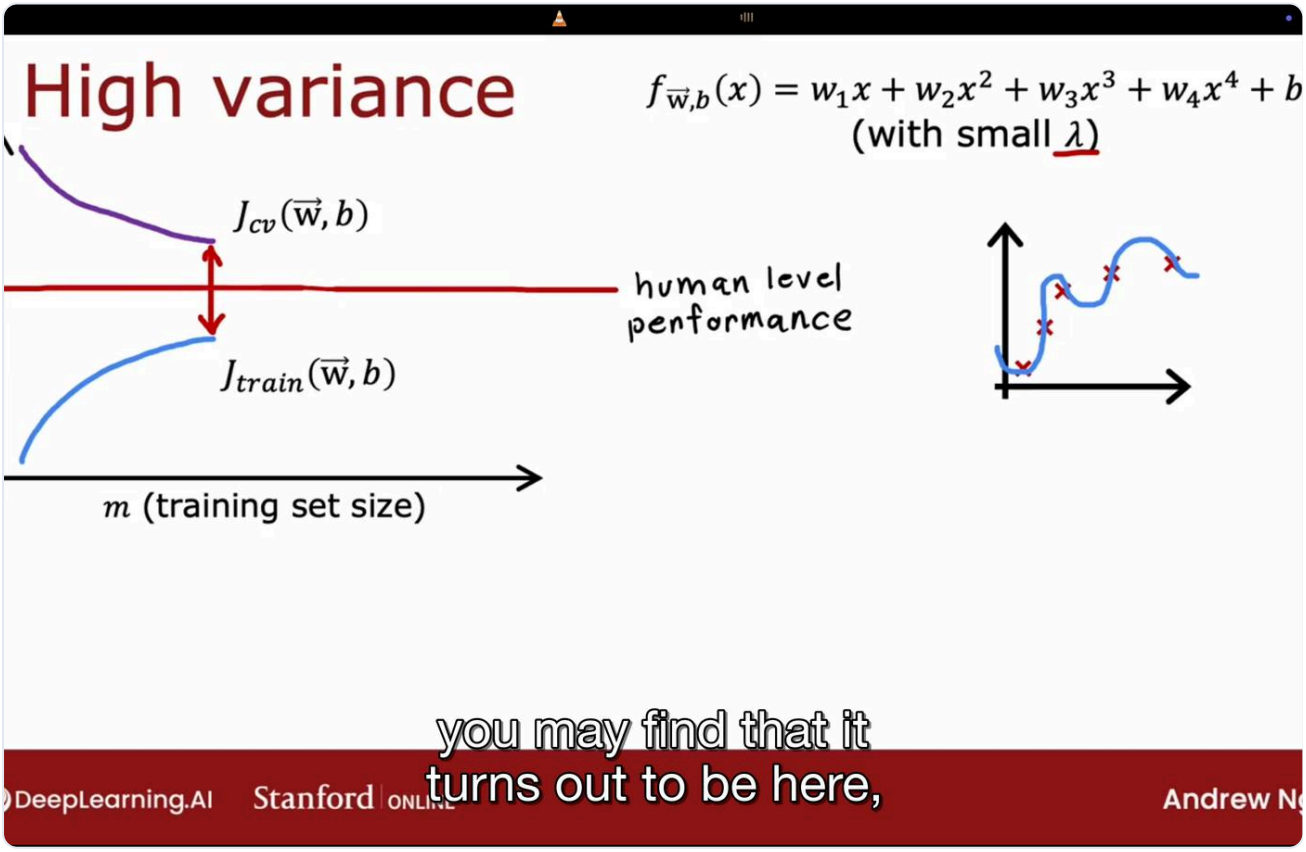


FIGURE 39.11

If you were to plot a baseline level of performance, such as human level performance, you may find that it turns out to be here, that J train can sometimes be even lower than the human level performance or maybe human level performance is a little bit lower than this. But when you're over fitting the training set, you may be able to fit the training set so well to have an unrealistically low error, such as zero error in this example over here, which is actually better than how well humans will actually be able to predict housing prices or whatever the application you're working on. But again, to signal for high variance is whether J cv is much higher than J train.

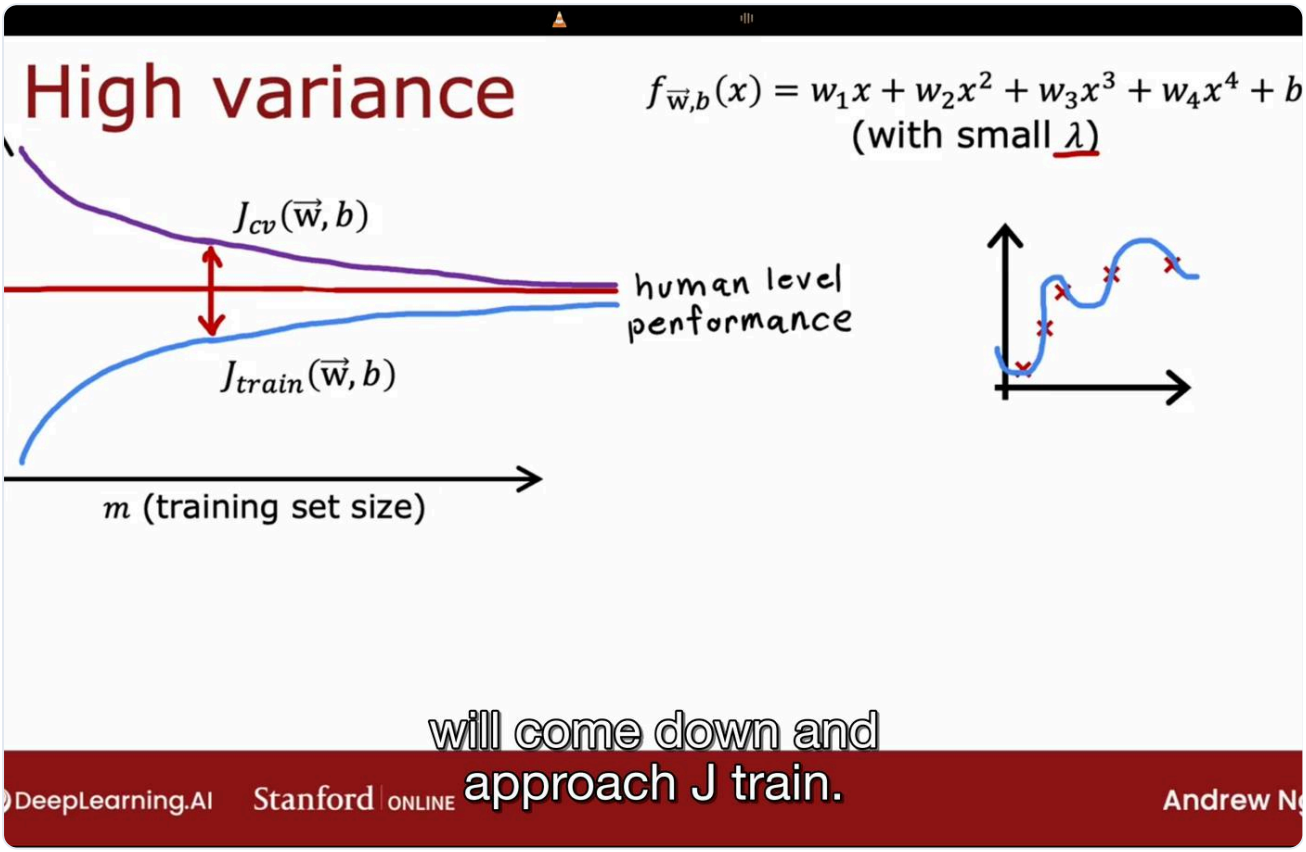


FIGURE 39.12

When you have high variance, then increasing the training set size could help a lot, and in particular, if we could extrapolate these curves to the right, increase M train, then the training error will continue to go up, but then the cross-validation error hopefully will come down and approach J train. So in this scenario, it might be possible just by increasing the training set size to lower the cross-validation error and to get your algorithm to perform better and better, and this is unlike the high bias case, where if the only thing you do is get more training data, that won't actually help you learn your algorithm performance much.

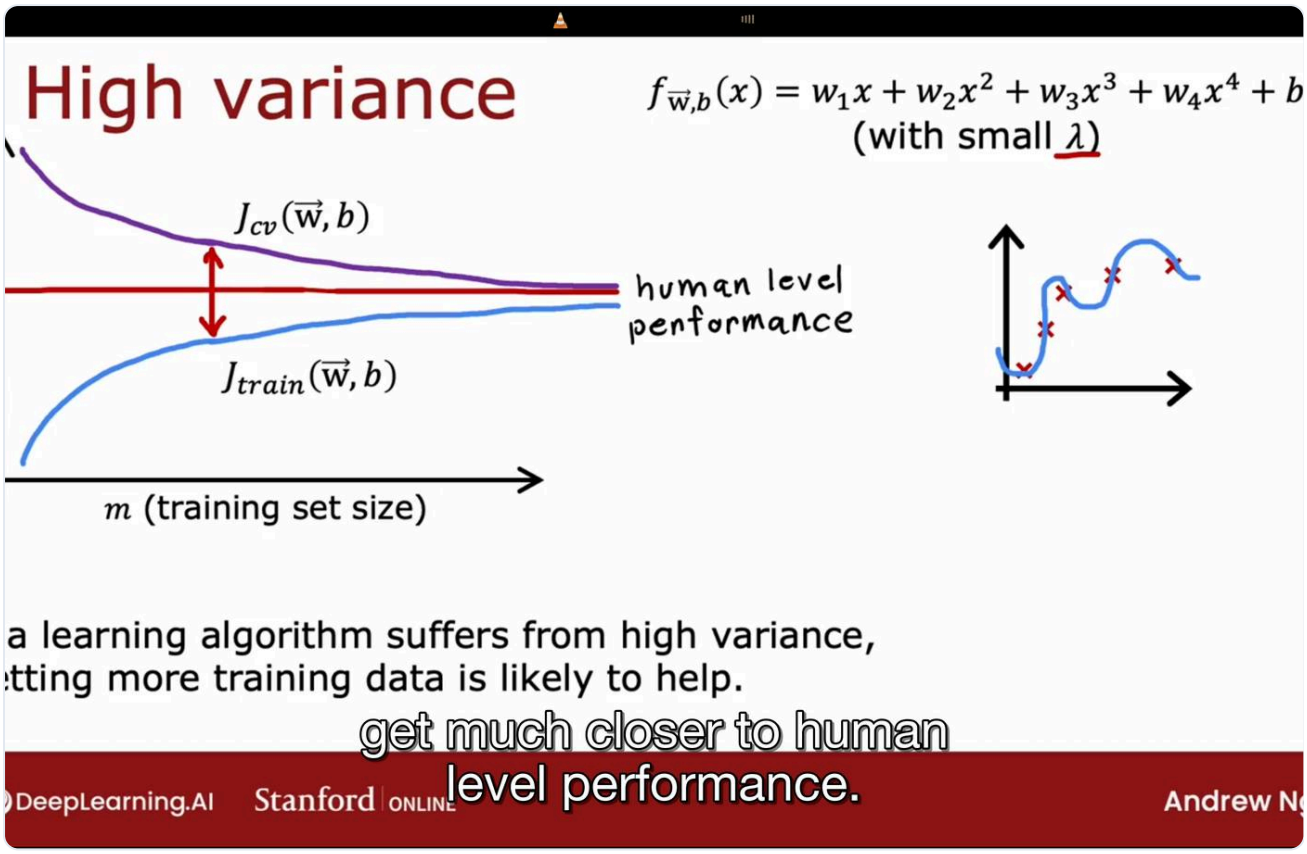


FIGURE 39.13

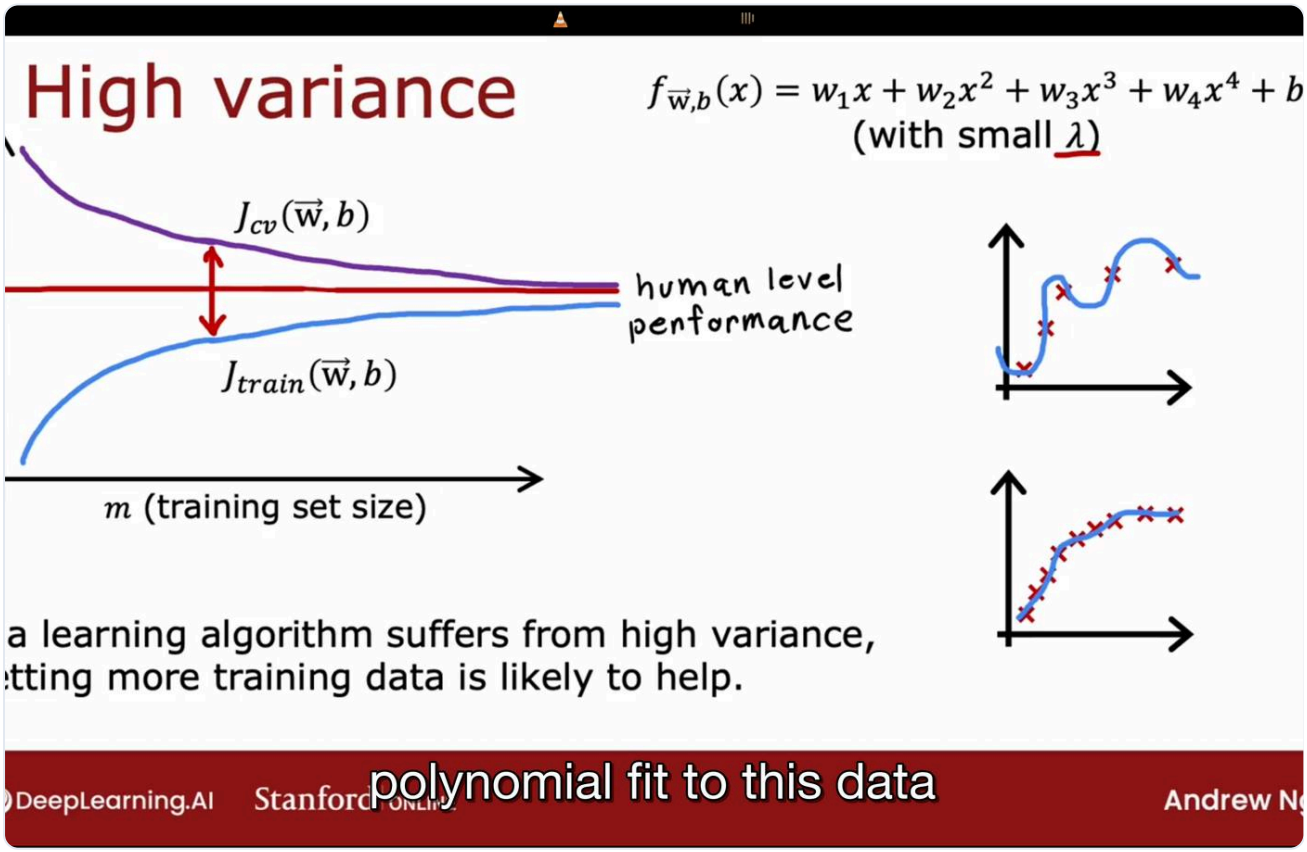


FIGURE 39.14

To summarize, if a learning algorithm suffers from high variance, then getting more training data is indeed likely to help. Because extrapolating to the right of this curve, you see that you can expect J cv to keep on coming down. In this example, just by getting more training data, allows the algorithm to go from relatively high cross-validation error to get much closer to human level performance. You can see that if you were to add a lot more training examples and continue to fill the fourth-order polynomial, then you can just get a better fourth order polynomial fit to this data than this very wiggly curve up on top. If you're building a machine learning application, you could plot the learning curves if you want, that is, you can take different subsets of your training sets, and even if you have, say, 1,000 training examples, you could train a model on just 100 training examples and look at the training error and cross-validation error, then train a model on 200 examples, holding out 800 examples and just not using them for now, and plot J train and J cv and so on the repeats and plot out what the learning curve looks like.

If we were to visualize it that way, then that could be another way for you to see if your learning curve looks more like a high bias or high variance one. One downside of the plotting learning curves like this is something I've done, but one downside is, it is computationally quite expensive to train so many different models using different size subsets of your training set, so in practice, it isn't done that often, but nonetheless, I find that having this mental visual picture in my head of what the training set looks like, sometimes that helps me to think through what I think my learning algorithm is doing and whether it has high bias or high variance. I know we've gone through a lot about bias and variance, let's go back to our earlier example of if you've trained a model for housing price prediction, how does bias and variance help you decide what to do next?

Let's go back to that earlier example, which I hope will now make a lot more sense to you. Let's do that in the next video.

LECTURE 40

Debugging a Learning Algorithm

You've seen how by looking at J train and Jcv, that is the training error and cross-validation error, or maybe even plotting a learning curve. You can try to get a sense of whether your learning algorithm has high bias or high variance. This is the procedure I routinely do when I'm training a learning algorithm more often look at the training error and cross-validation error to try to decide if my algorithm has high bias or high variance. It turns out this will help you make better decisions about what to try next in order to improve the performance of your learning algorithm. Let's look at an example. This is actually the example that you have seen earlier.

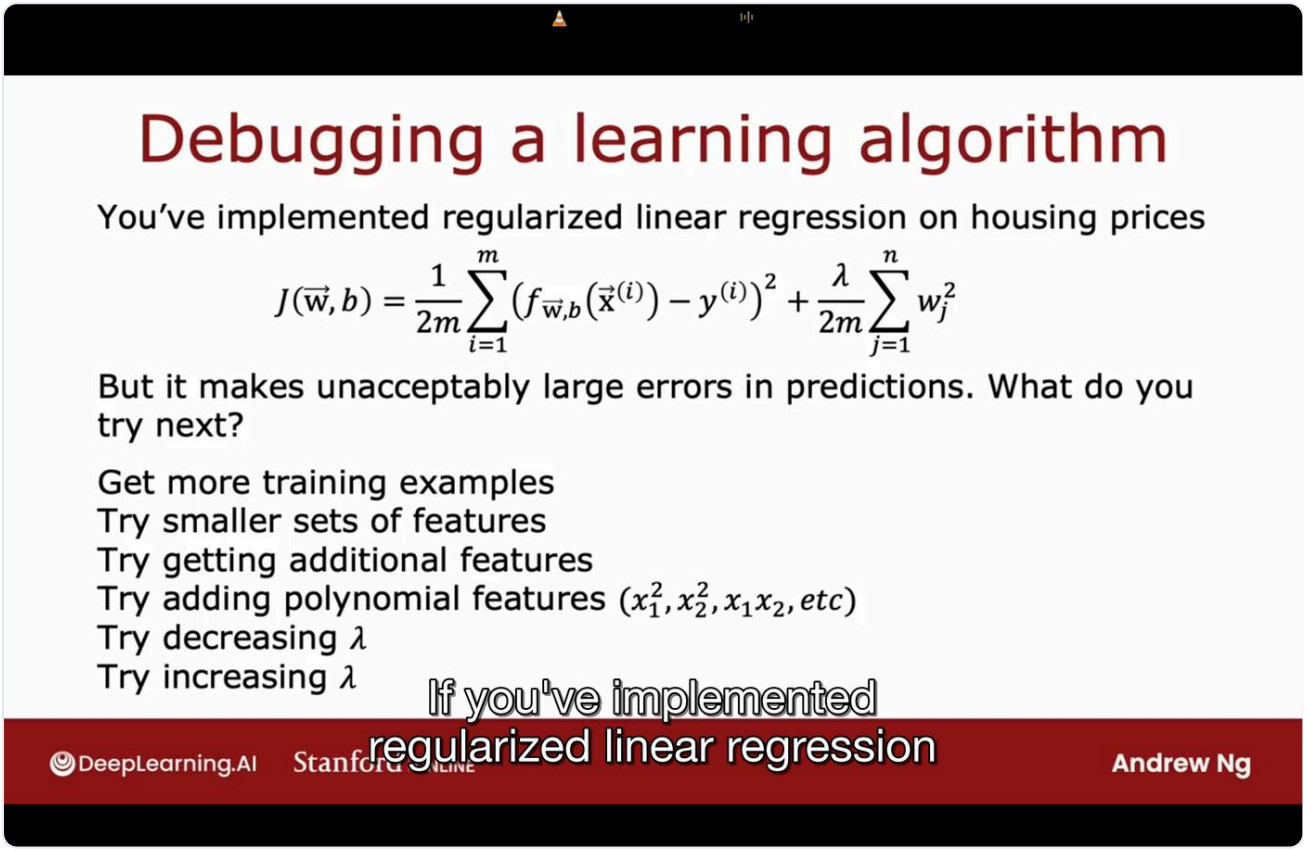


FIGURE 40.1 | Debugging a Learning Algorithm

If you've implemented regularized linear regression on predicting housing prices, but your algorithm makes unacceptably large errors in its predictions, what do you try next? These were the six ideas that we had when we had looked over this slide earlier. Getting more training examples, try small set of features, additional features, and so on. It turns out that each of these six items either helps fix a high variance or a high bias problem. In particular, if your learning algorithm has high bias, three of these techniques will be useful. If your learning algorithm has high variance than a different three of these techniques will be useful. Let's see if we can figure out which is which.

First one is get more training examples. We saw in the last video that if your algorithm has high bias, then if the only thing we do is get more training data, that by itself probably won't help that much. But in contrast, if your algorithm has high variance, say it was overfitting to a very small training set, then getting more training examples will help a lot. This first option or getting more training examples helps to fix a high variance problem. How about the other five? Do you think you can figure out which of the remaining five fix high bias or high variance problems? I'm going to go through the rest of them in this video in a minute but if you want it, you're free to pause the video and see if you can think through these five other things by yourself.

Feel free to pause the video. Just kidding, that was me pausing and not your video pausing. But seriously, if you want it, go ahead and pause the video and think through that you want or not and we'll go over these review in a minute. How about trying a smaller set of features? Sometimes if your learning algorithm has too many features, then it gives your algorithm too much flexibility to fit very complicated models. This is a little bit like if you had x, x squared, x cubed, x^4, x^5, and so on. If only you were to eliminate a few of these, then your model won't be so complex and won't have such high variance. If you suspect that your algorithm has a lot of features that are not actually relevant or helpful to predicting housing price, or if you suspect that you had even somewhat redundant features, then eliminating or reducing the number of features will help reduce the flexibility of your algorithm to overfit the data.

This is a tactic that will help you to fix high variance. Conversing, getting additional features, that's just adding additional features is the opposite of going to a smaller set of features. This will help you to fix a high bias problem. As a concrete example, if you're trying to predict the price of the house just based on the size, but it turns out that the price of house also really depends on the number of bedrooms and on the number of floors and on the age of the house, then the algorithm will never do that well unless you add in those additional features. That's a high bias problem because you just can't do that well on the training set when only the size, is only when you tell the algorithm how many bedrooms are there, how many floors are there?

What's the age of the house that it finally has enough information to even do better on the training set. Adding additional features is a way to fix a high bias problem. Adding polynomial features is a little bit like adding additional features. If you're linear functions, three-line can fit the training set that well, then adding additional polynomial features can help you do better on the training set, and helping you do better on the training set is a way to fix a high bias problem. Then decreasing Lambda means to use a lower value for the regularization parameter. That means we're going to pay less attention to this term and pay more attention to this term to try to do better on the training set.

Again, that helps you to fix a high bias problem. Finally, increasing Lambda, well that's the opposite of this, but that says you're overfitting the data. Increasing Lambda will make sense if is overfitting the training set, just putting too much attention to fit the training set, but at the expense of generalizing to new examples, and so increasing Lambda would force the algorithm to fit a smoother function, may be less wiggly function and use this to fix a high variance problem.

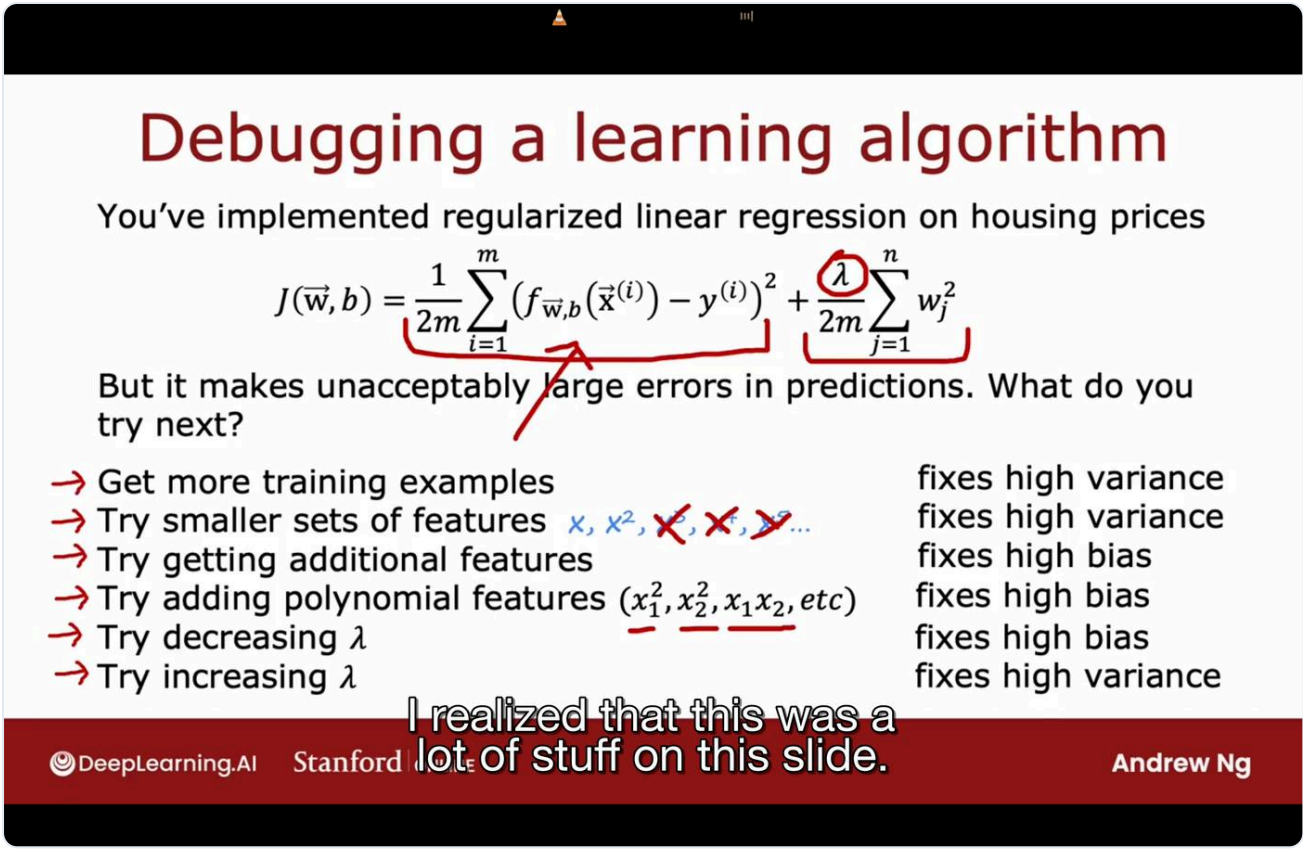


FIGURE 40.2 | Debugging a Learning Algorithm

I realized that this was a lot of stuff on this slide. But the takeaways I hope you have are, if you find that your algorithm has high variance, then the two main ways to fix that are; neither get more training data or simplify your model. By simplifying model I mean, either get a smaller set of features or increase the regularization parameter Lambda. Your algorithm has less flexibility to fit very complex, very wiggly curves. Conversely, if your algorithm has high bias, then that means is not doing well even on the training set. If that's the case, the main fixes are to make your model more powerful or to give them more flexibility to fit more complex or more wiggly functions.

Some ways to do that are to give it additional features or add these polynomial features, or to decrease the regularization parameter Lambda. Anyway, in case you're wondering if you should fix high bias by reducing the training set size, that doesn't actually help. If you reduce the training set size, you will fit the training set better, but that tends to worsen your cross-validation error and the performance of your learning algorithm, so don't randomly throw away training examples just to try to fix a high bias problem. One of my PhD students from Stanford, many years after he'd already graduated from Stanford, once said to me that while he was studying at Stanford, he learned about bias and variance and felt like he got it, he understood it.

But that subsequently, after many years of work experience in a few different companies, he realized that bias and variance is one of those concepts that takes a short time to learn, but takes a lifetime to master. Those were his exact words. Bias and variance is one of those very powerful ideas. When I'm training learning algorithms, I almost always try to figure out if it is high bias or high variance. But the way you go about addressing that systematically is something that you

will keep on getting better at through repeated practice. But you'll find that understanding these ideas will help you be much more effective at how you decide what to try next when developing a learning algorithm.

Now, I know that we did go through a lot in this video and if you feel like, boy, this is a lot of stuff here, it's okay, don't worry about it. Later this week in the practice labs and practice quizzes will have also additional opportunities to go over these ideas so that you can get additional practice. We're thinking about bias and variance of different learning algorithms. If it seems like a lot right now is okay, you get to practice these ideas later this week and hopefully deepen your understanding of them at that time. Before moving on, bias and variance also are very useful when thinking about how to train a neural network. In the next video, let's take a look at these concepts applied to neural network training.

Let's go on to the next video.

LECTURE 41

Neural Networks and Bias Variance

Was seen that high bias or high variance are both bad in the sense that they hurt the performance of your algorithm. One of the reasons that neural networks have been so successful is because your networks, together with the idea of big data or hopefully having large data sets. It's given us a new way of new ways to address both high bias and high variance. Let's take a look. You saw that if you're fitting different order polynomial is to a data set, then if you were to fit a linear model like this on the left. You have a pretty simple model that can have high bias whereas you were to fit a complex model, then you might suffer from high variance.

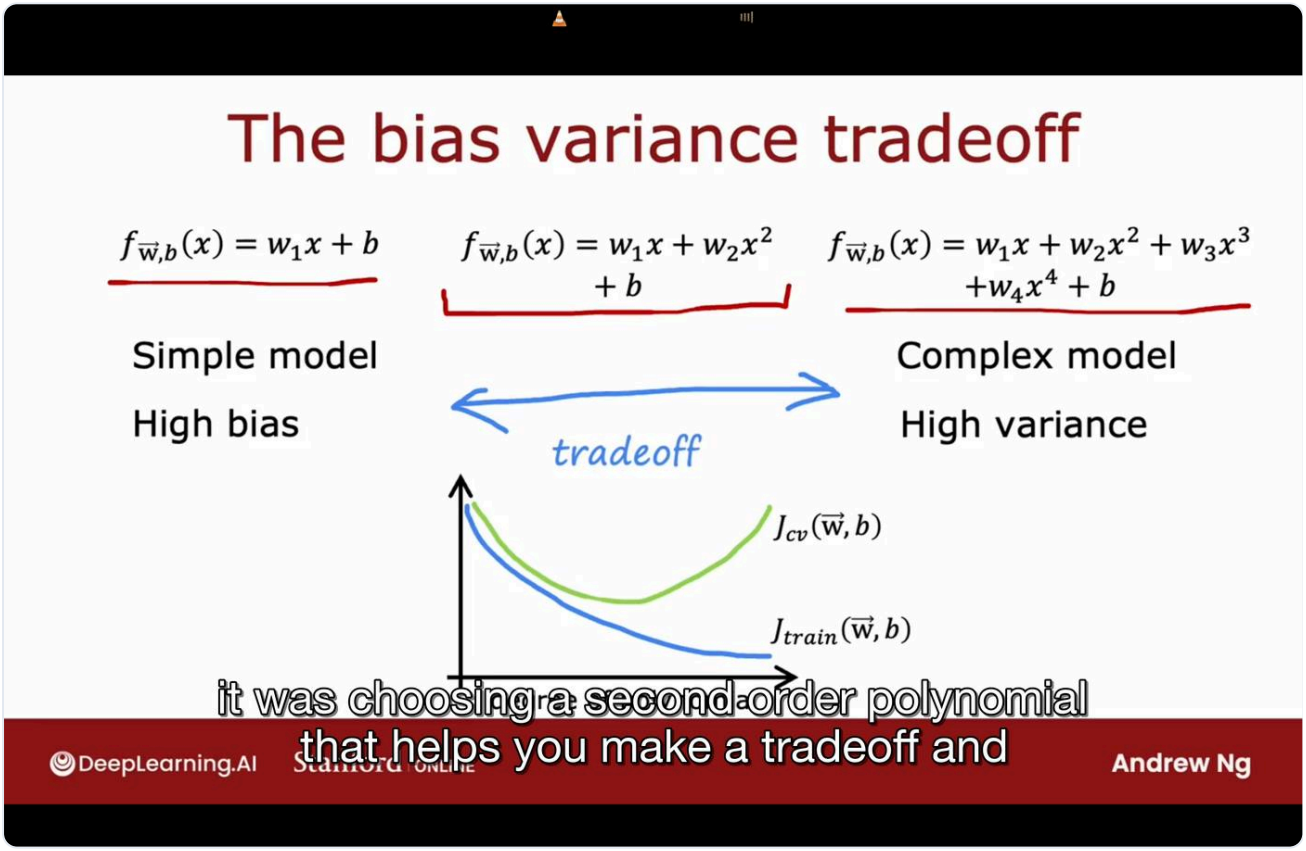


FIGURE 41.1 | The Bias Variance Tradeoff

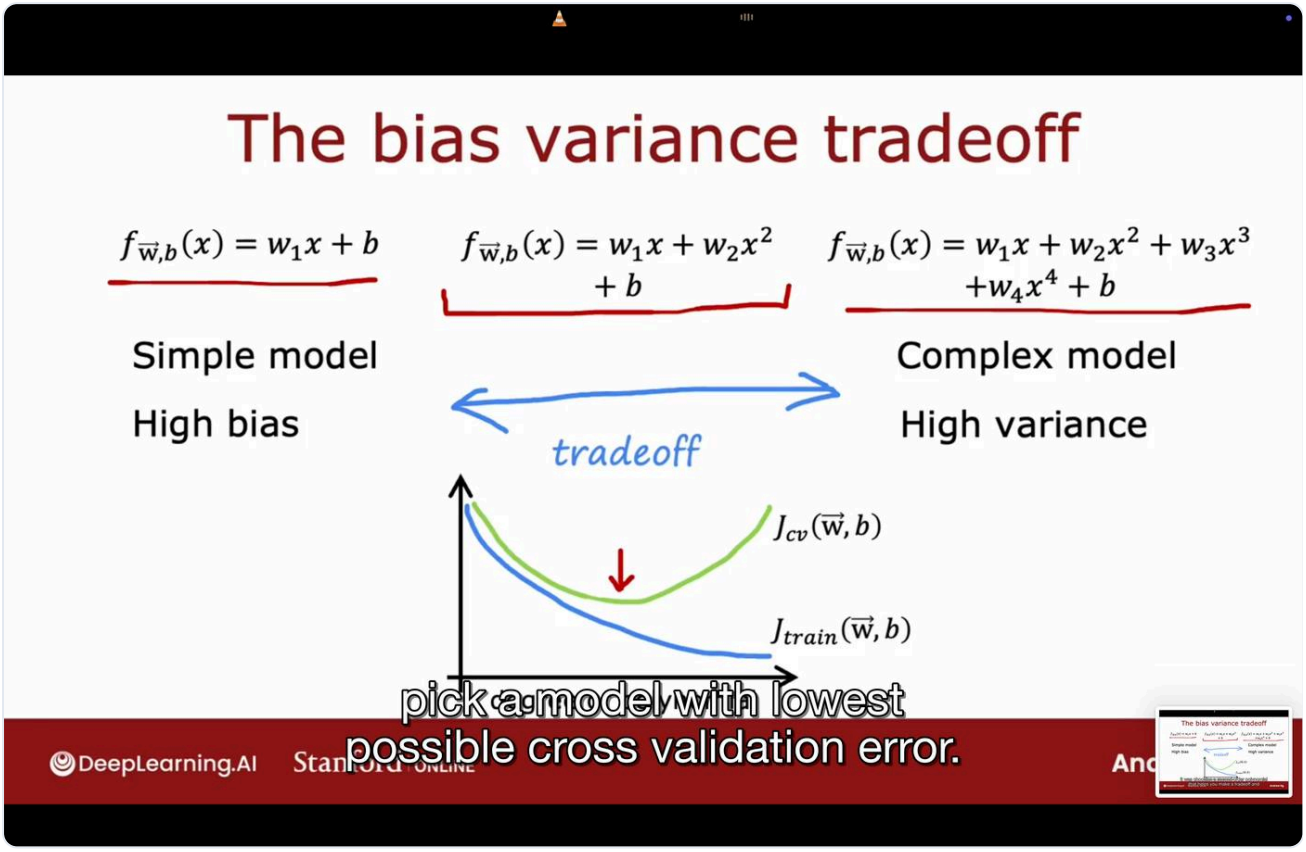


FIGURE 41.2 | The Bias Variance Tradeoff

And there's this tradeoff between bias and variance, and in our example it was choosing a second order polynomial that helps you make a tradeoff and pick a model with lowest possible cross validation error. And so before the days of neural networks, machine learning engineers talked a lot about this bias variance tradeoff in which you have to balance the complexity that is the degree of polynomial.

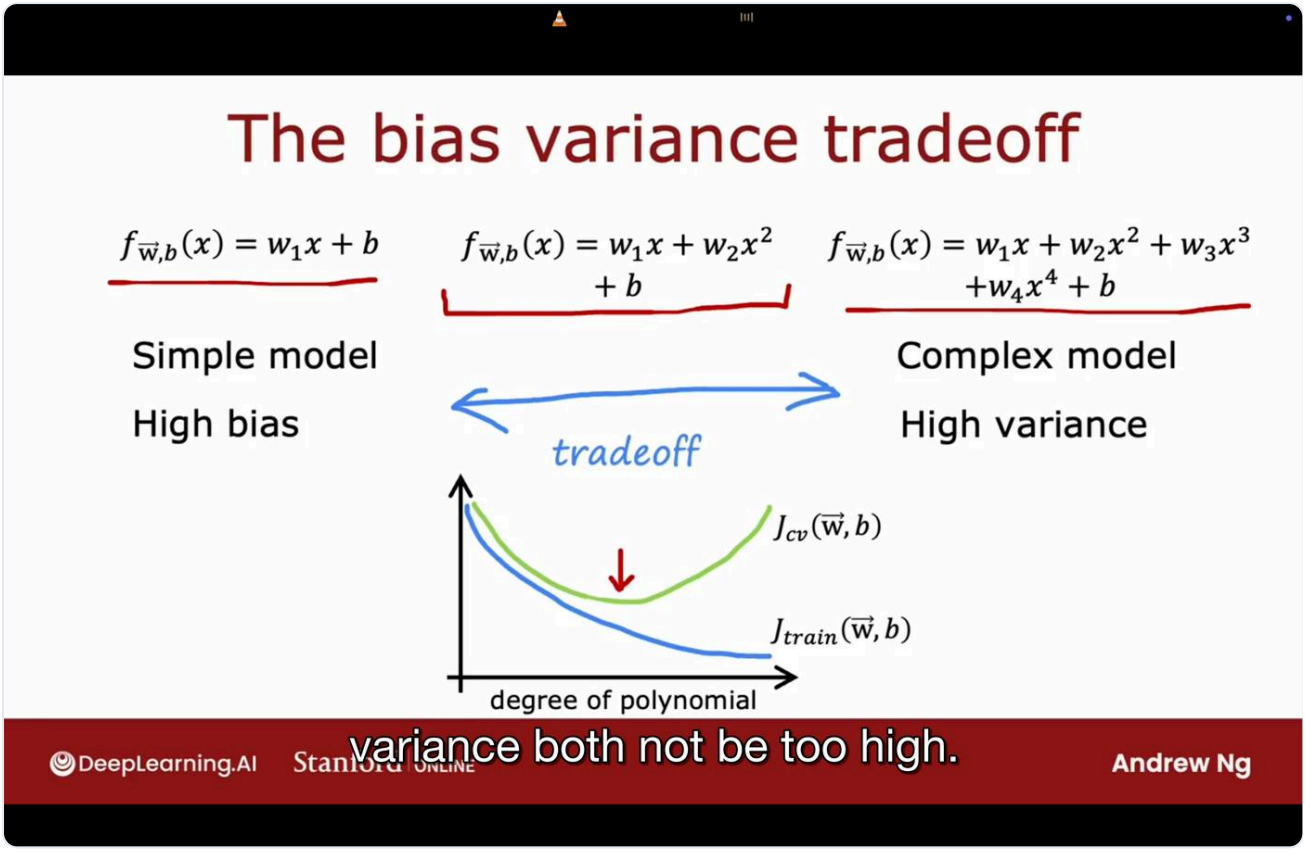


FIGURE 41.3 | The Bias Variance Tradeoff

Or the regularization parameter longer to make bias and variance both not be too high. And if you hear machine learning engineers talk about the bias variance tradeoff. This is what they're referring to where if you have too simple a model, you have high bias, too complex a model high variance. And you have to find a tradeoff between these two bad things to find probably the best possible outcome. But it turns out that neural networks offer us a way out of this dilemma of having to tradeoff bias and variance with some caveats. And it turns out that large neural networks when trained on small term moderate sized datasets are low bias machines. And what I mean by that is, if you make your neural network large enough, you can almost always fit your training set well.

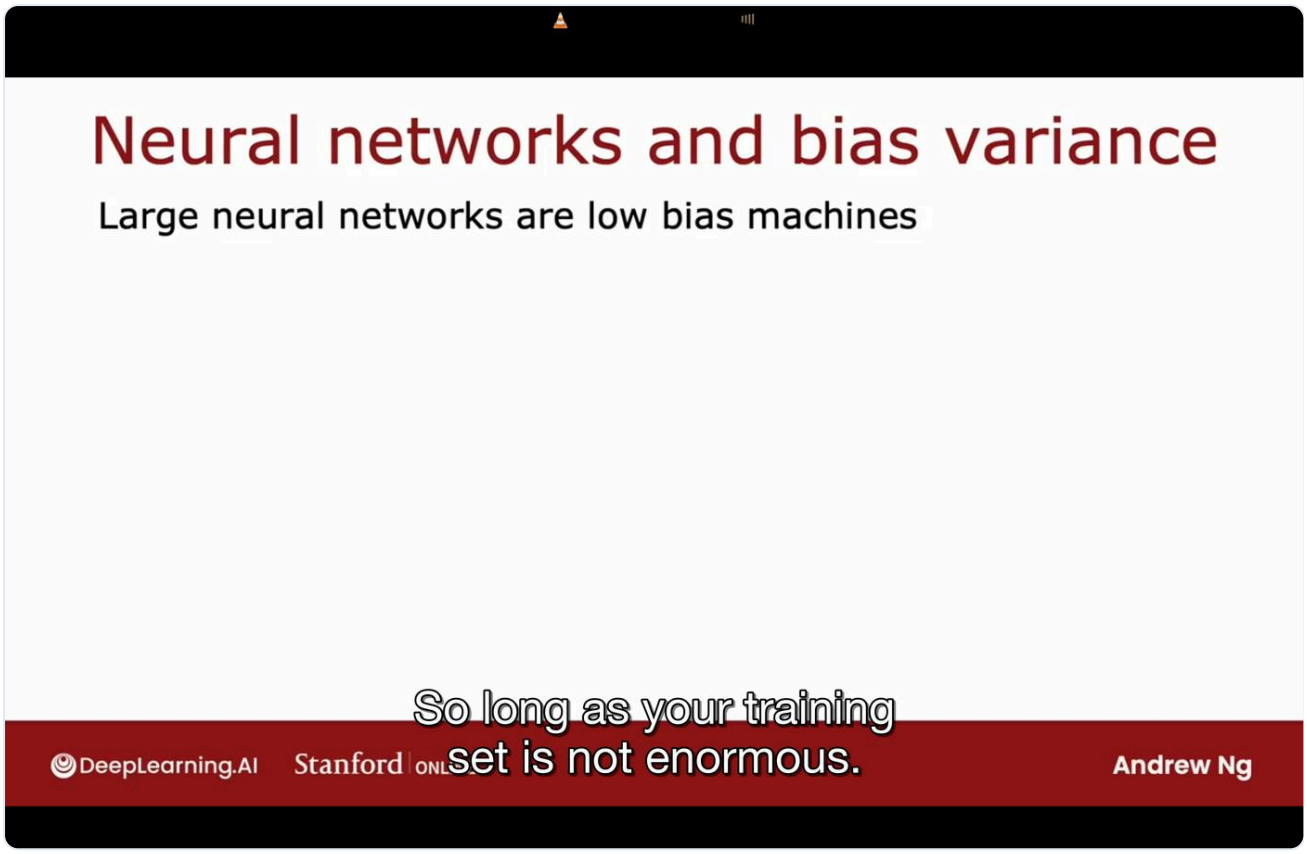


FIGURE 41.4 | Neural Networks and Bias Variance

So long as your training set is not enormous. And what this means is this gives us a new recipe to try to reduce bias or reduce variance as needed without needing to really trade off between the two of them. So let me share with you a simple recipe that isn't always applicable. But if it applies can be very powerful for getting an accurate model using a neural network which is first train your algorithm on your training set and then asked does it do well on the training set. So measure Jtrain and see if it is high and by high, I mean for example, relative to human level performance or some baseline level of performance and if it is not doing well then you have a high bias problem, high trains error.

And one way to reduce bias is to just use a bigger neural network and by bigger neural network, I mean either more hidden layers or more hidden units per layer.

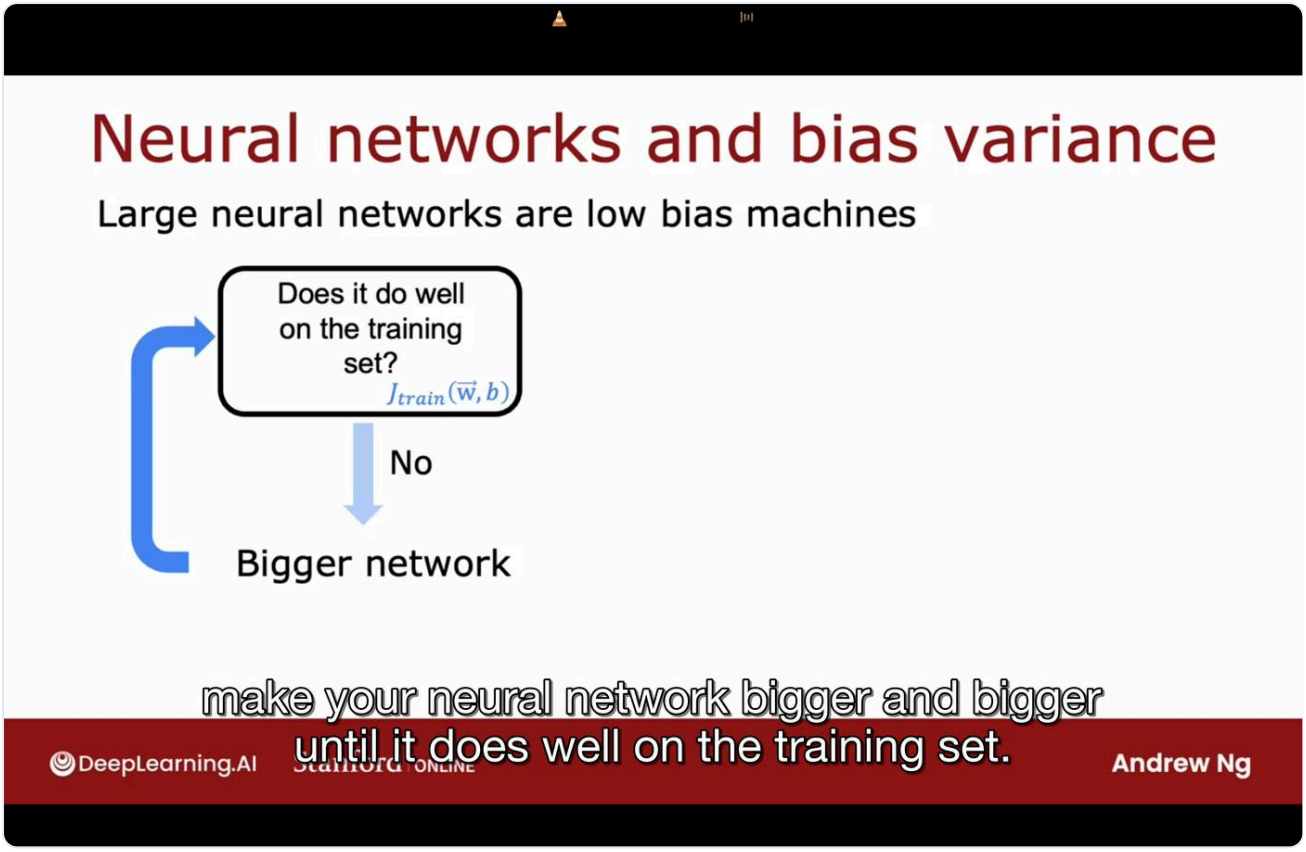


FIGURE 41.5 | Neural Networks and Bias Variance

And you can then keep on going through this loop and make your neural network bigger and bigger until it does well on the training set. Meaning that achieves the level of error in your training set that is roughly comparable to the target level of error you hope to get to, which could be human level performance. After it does well on the training set, so the answer to that question is yes. You then ask does it do well on the cross validation set? In other words, does it have high variance and if the answer is no, then you can conclude that the algorithm has high variance because it doesn't want to train set does not do on the cross validation set.

So that big gap in Jcv and Jtrain indicates you probably have a high variance problem, and if you have a high variance problem, then one way to try to fix it is to get more data.

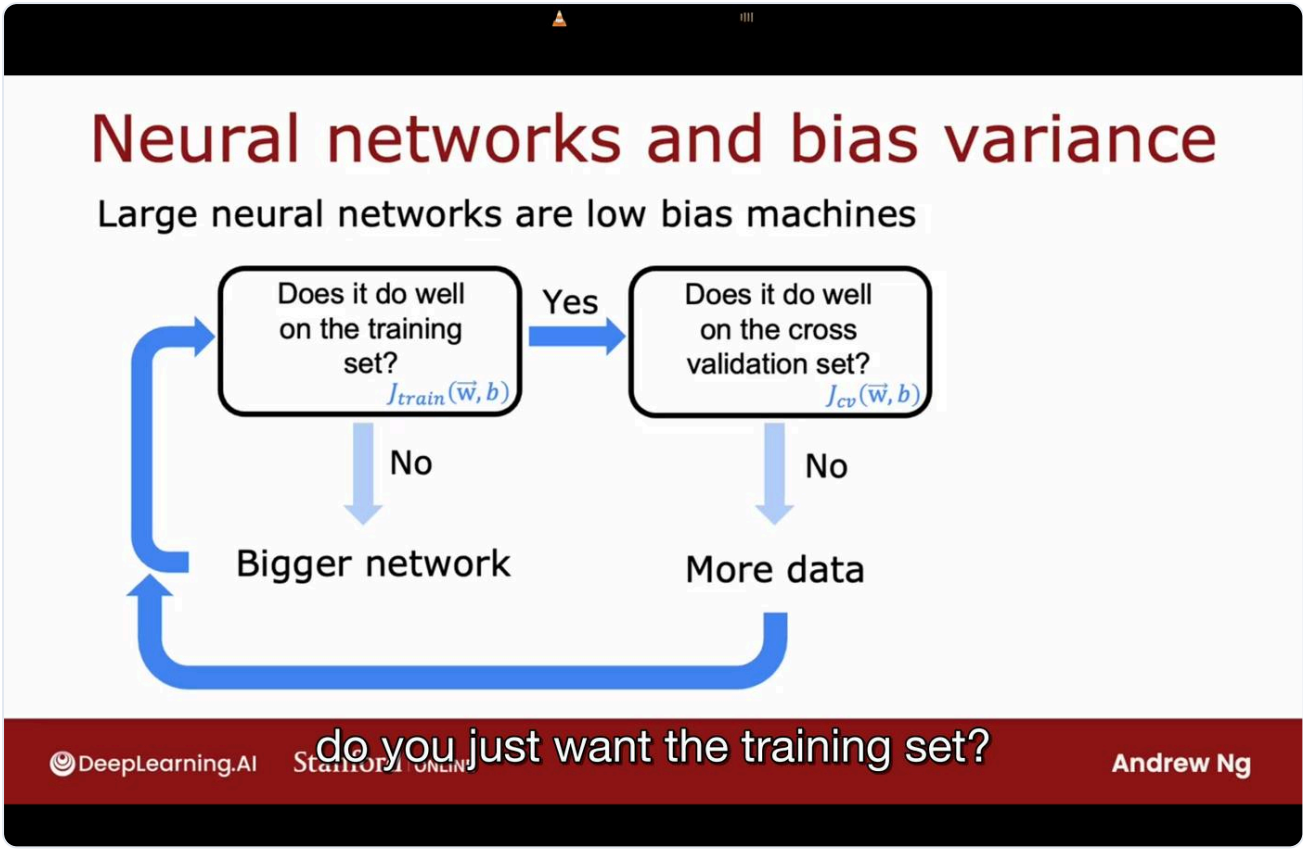


FIGURE 41.6 | Neural Networks and Bias Variance

To get more data and go back and retrain the model and just double-check, do you just want the training set? If not, have a bigger network, or it does see if it does when the cross validation set and if not get more data. And if you can keep on going around and around and around this loop until eventually it does well in the cross validation set.

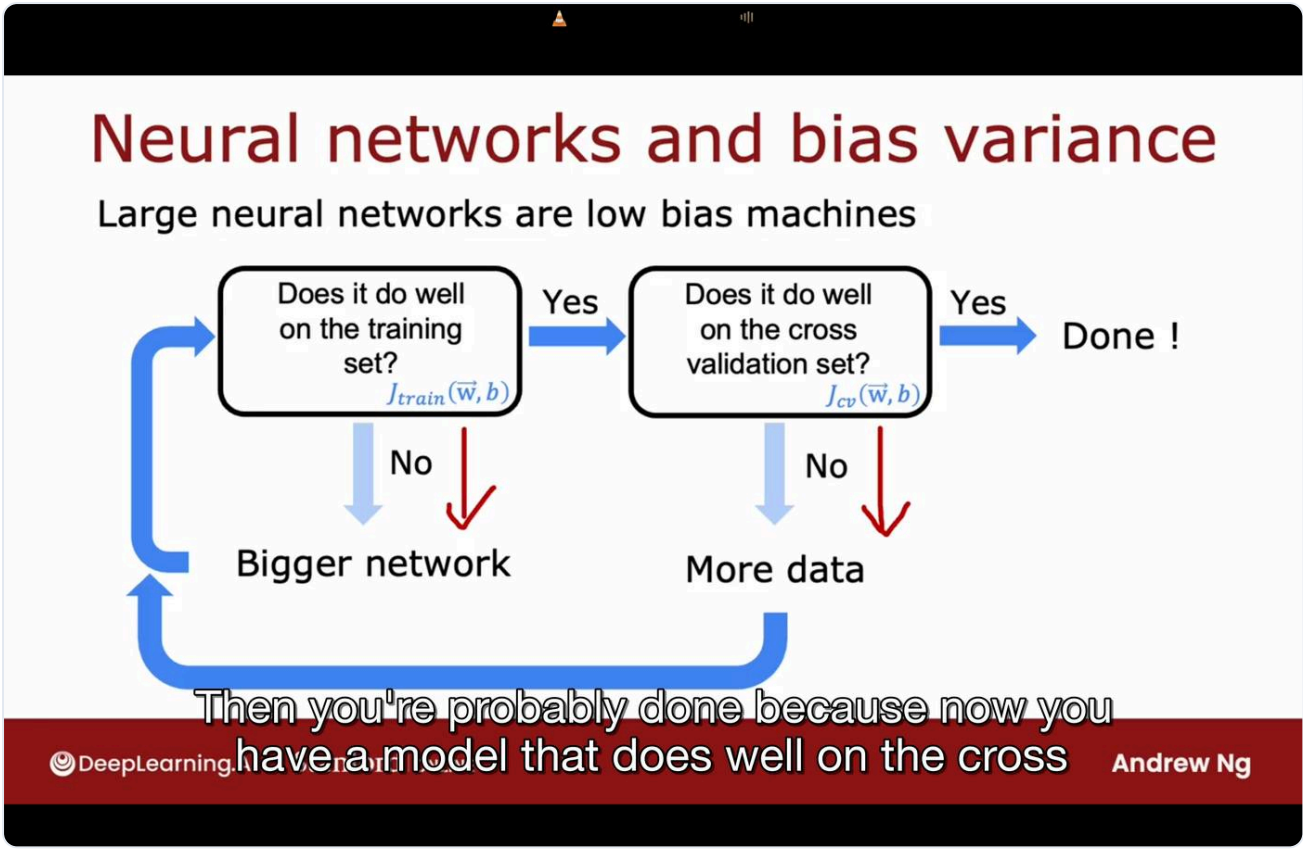


FIGURE 41.7 | Neural Networks and Bias Variance

Then you're probably done because now you have a model that does well on the cross validation set and hopefully will also generalize to new examples as well. Now, of course there are limitations of the application of this recipe training bigger neural network doesn't reduce bias but at some point it does get computationally expensive. That's why the rise of neural networks has been really assisted by the rise of very fast computers, including especially GPUs or graphics processing units. Hardware traditionally used to speed up computer graphics, but it turns out has been very useful for speeding on neural networks as well. But even with hardware accelerators beyond a certain point, the neural networks are so large, it takes so long to train, it becomes infeasible.

And then of course the other limitation is more data. Sometimes you can only get so much data, and beyond a certain point it's hard to get much more data.

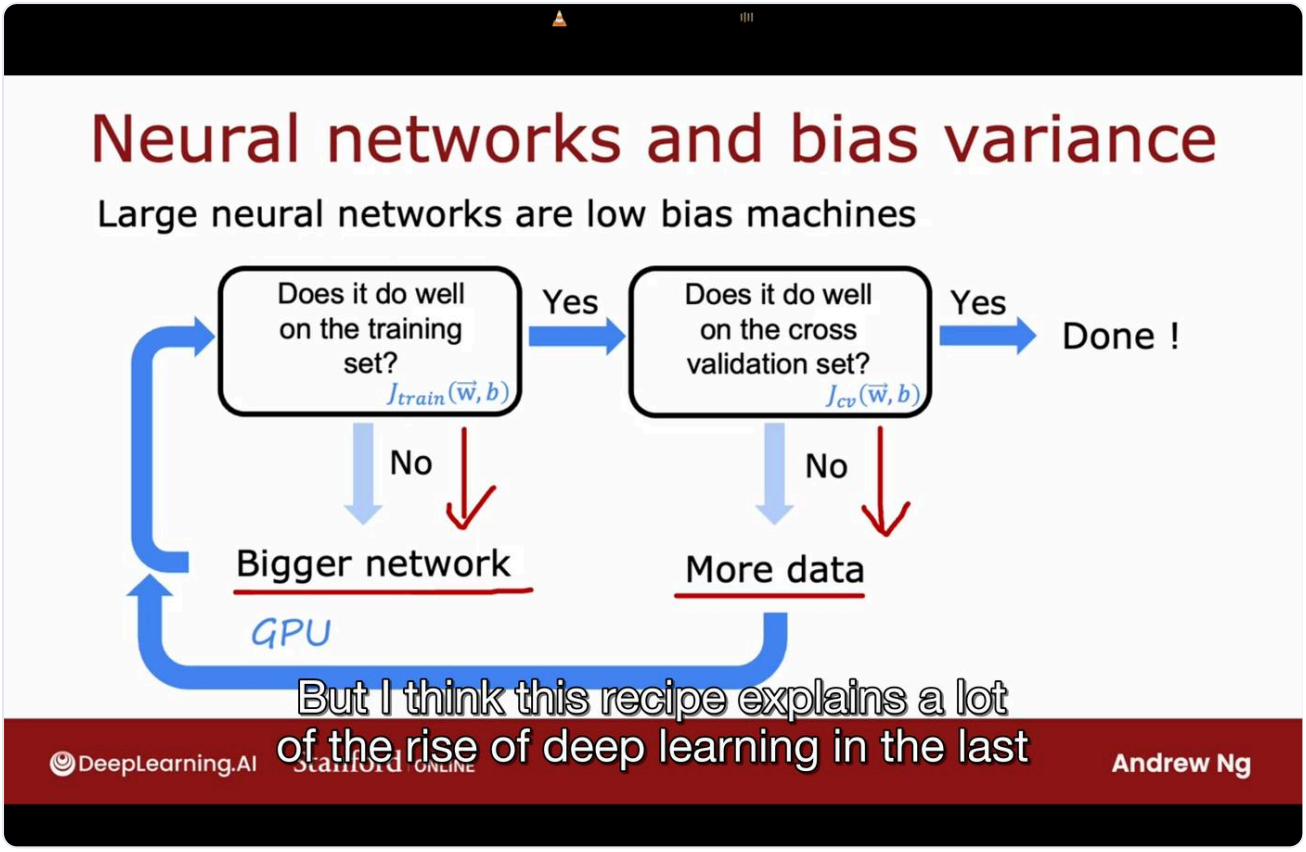


FIGURE 41.8 | Neural Networks and Bias Variance

But I think this recipe explains a lot of the rise of deep learning in the last several years, which is for applications where you do have access to a lot of data. Then being able to train large neural networks allows you to eventually get pretty good performance on a lot of applications. One thing that was implicit in this slide that may not have been obvious is that as you're developing a learning algorithm, sometimes you find that you have high bias, in which case you do things like increase your neural network. But then after you increase your neural network you may find that you have high variance, in which case you might do other things like collect more data.

And during the hours or days or weeks, you're developing a machine learning algorithm at different points, you may have high bias or high variance. And it can change but it's depending on whether your algorithm has high bias or high variance at that time. Then that can help give guidance for what you should be trying next. When you train your neural network, one thing that people have asked me before is, hey Andrew, what if my neural network is too big? Will that create a high variance problem?

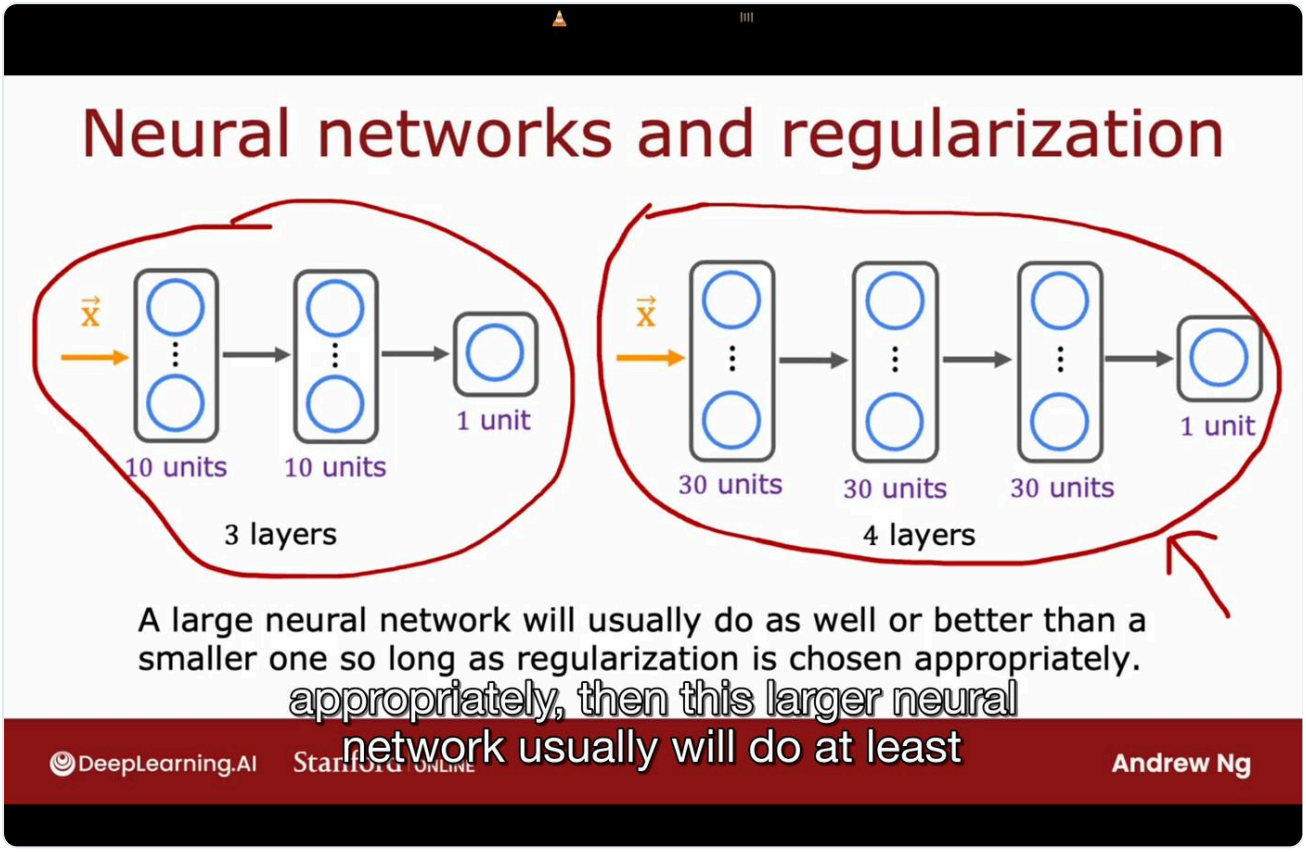


FIGURE 41.9 | Neural Networks and Regularization

It turns out that a large neural network with well-chosen regularization, well usually do as well or better than a smaller one. And so for example, if you have a small neural network like this, and you were to switch to a much larger neural network like this, you would think that the risk of overfitting goes up significantly. But it turns out that if you were to regularize this larger neural network appropriately, then this larger neural network usually will do at least as well or better than the smaller one. So long as the regularization has chosen appropriately. So another way of saying this is that it almost never hurts to go to a larger neural network so long as you regularized appropriately with one caveat, which is that when you train the larger neural network, it does become more computational e expensive.

So the main way it hurts, it will slow down your training and your inference process and very briefly to regularize a neural network. This is what you do if the cost function for your neural network is the average loss and so the loss here could be squared error or logistic loss.

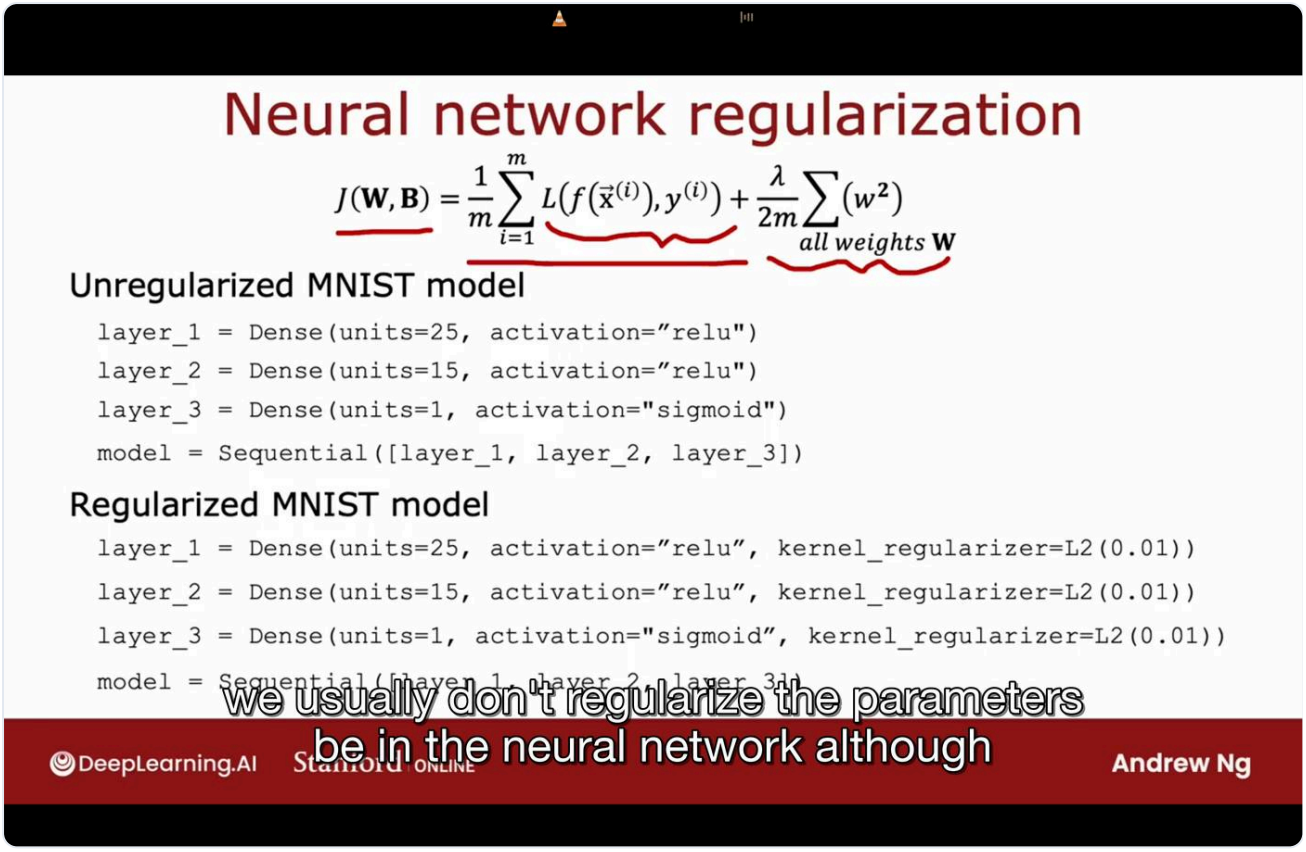


FIGURE 41.10 | Unregularized MNIST Model

Then the regularization term for a neural network looks like pretty much what you'd expect is lambda over two m times the sum of w squared where this is a sum over all weights W in the neural network and similar to regularization for linear regression and logistic regression, we usually don't regularize the parameters be in the neural network although in practice it makes very little difference whether you do so or not. And the way you would implement regularization in tensorflow is recall that this was the code for implementing an unregularized handwritten digit classification model. We create three layers like so with a number of fitting units activation And then create a sequential model with the three layers.

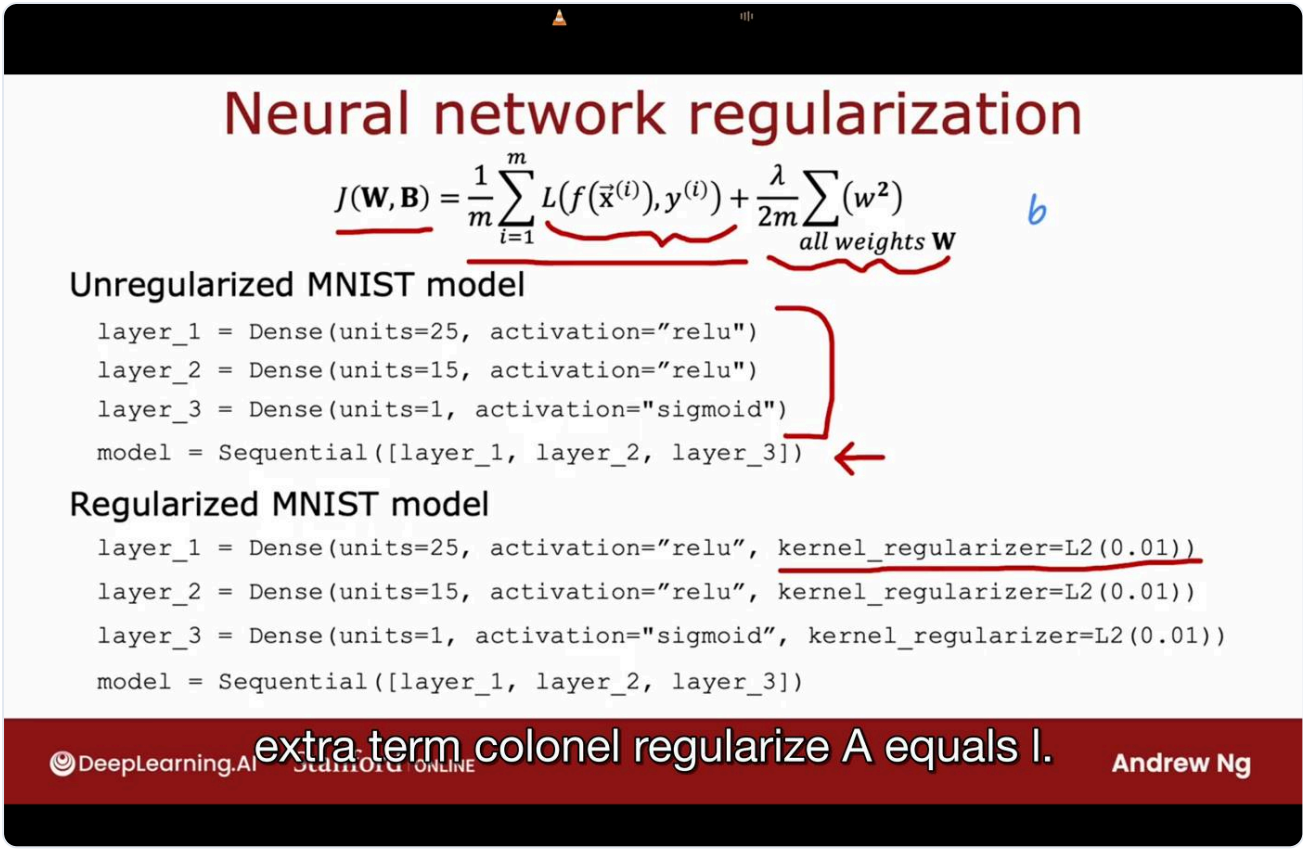


FIGURE 41.11 | Neural Network Regularization

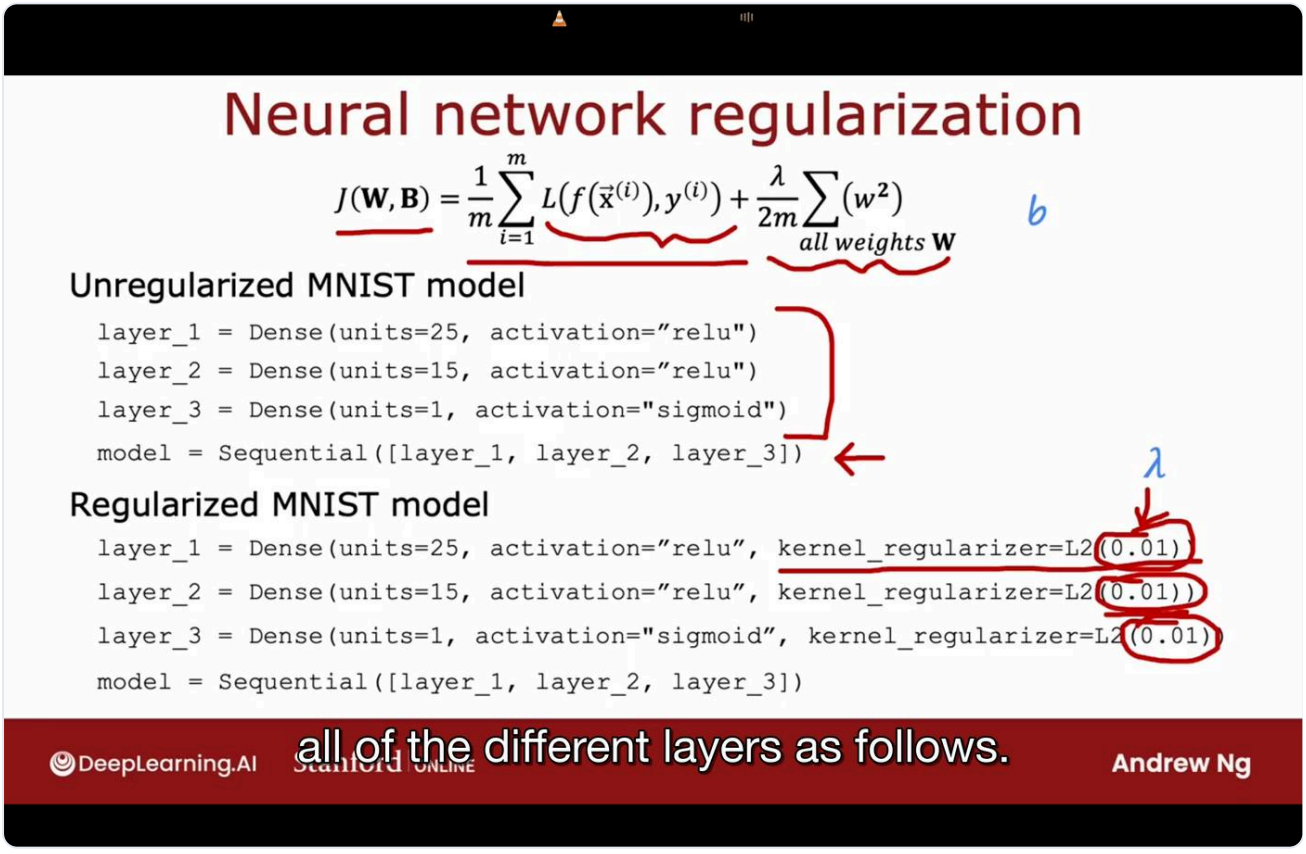


FIGURE 41.12 | Neural Network Regularization

If you want to add regularization then you would just add this extra term kernel regularize A equals l. two and then 0.01 where that's the value of longer in terms of though actually lets you choose different values of lambda for different layers although for simplicity you can choose the same value of lambda for all the weights and all of the different layers as follows. And then this will allow you to implement regularization in your neural network. So to summarize two Takeaways, I hope you have from this video are one. It hardly ever hurts to have a larger neural network so long as you regularize appropriately. one caveat being that having a larger neural network can slow down your algorithm.

So maybe that's the one way it hurts, but it shouldn't hurt your algorithm's performance for the most part and in fact it could even help it significantly. And second so long as your training set isn't too large. Then a neural network, especially large neural network is often a low bias machine. It just fits very complicated functions very well, which is why when I'm training neural networks, I find that I'm often fighting variance problems rather than bias problems, at least if the neural network is large enough. So the rise of deep learning has really changed the way that machine learning practitioners think about bias and variance. Having said that even when you're training a neural network measuring bias and variance and using that to guide what you do next is often a very helpful thing to do.

So that's it for bias and variance. Let's go on to the next video. We will take all the ideas we've learned and see how they fit in to the development process of machine learning systems. And I hope that we'll tie a lot of these pieces together to give you practical advice for how to quickly move forward in the development of your machine learning systems

LECTURE 42

Iterative Loop of ML Development

In the next few videos, I'd like to share with you what is like to go through the process of developing a machine learning system so that when you are doing so yourself, hopefully, you'd be in a position to make great decisions at many stages of the machine learning development process. Let's take a look first at the iterative loop of machine learning development. This is what developing a machine learning model will often feel like. First, you decide on what is the overall architecture of your system. That means choosing your machine learning model as well as deciding what data to use, maybe picking the hyperparameters, and so on. Then, given those decisions, you would implement and train a model.

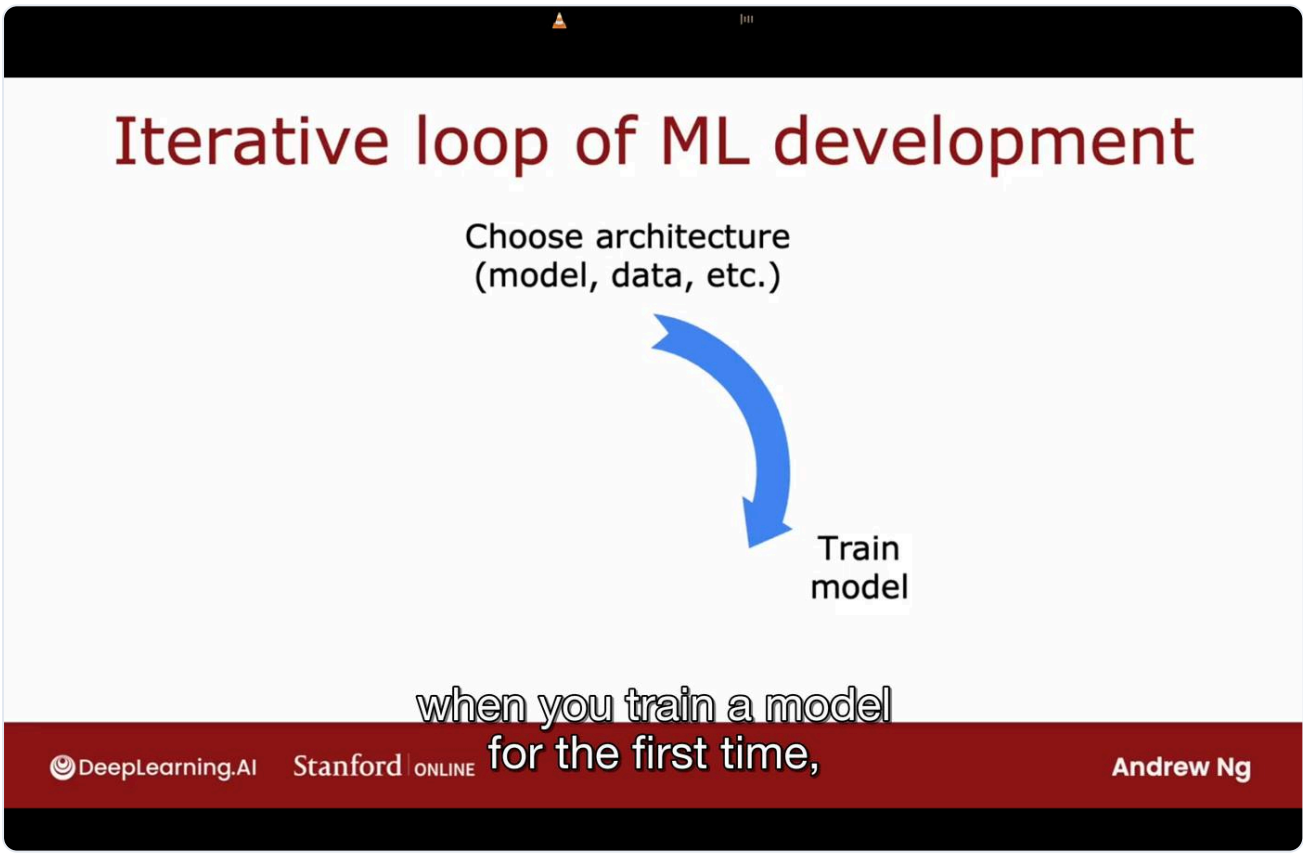


FIGURE 42.1 | Iterative Loop of ML Development

As I've mentioned before, when you train a model for the first time, it will almost never work as well as you want it to.

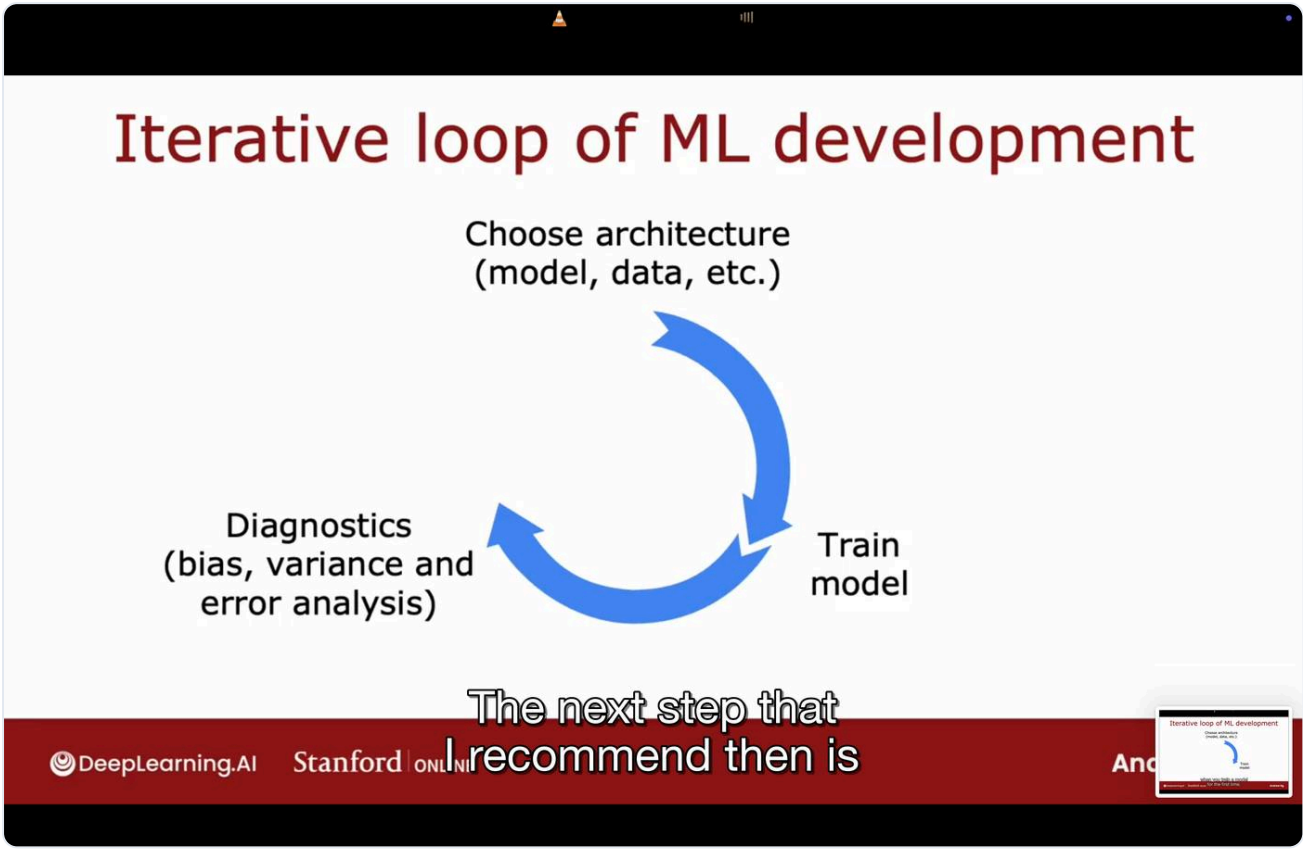


FIGURE 42.2 | Iterative Loop of ML Development

The next step that I recommend then is to implement or to look at a few diagnostics, such as looking at the bias and variance of your algorithm as well as something we'll see in the next video called error analysis.

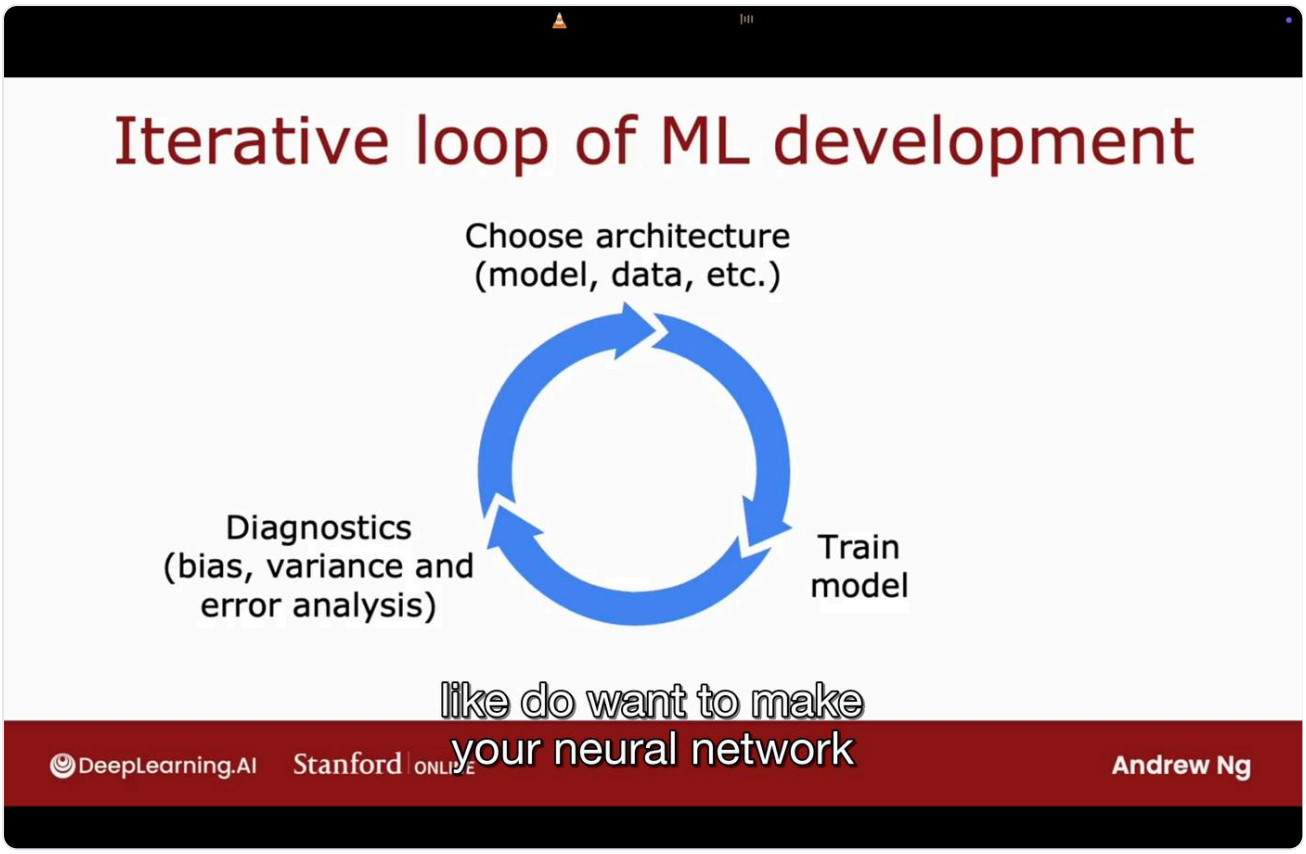


FIGURE 42.3 | Iterative Loop of ML Development

Based on the insights from the diagnostics, you can then make decisions like do want to make your neural network bigger or change the Lambda regularization parameter, or maybe add more data or add more features or subtract features.

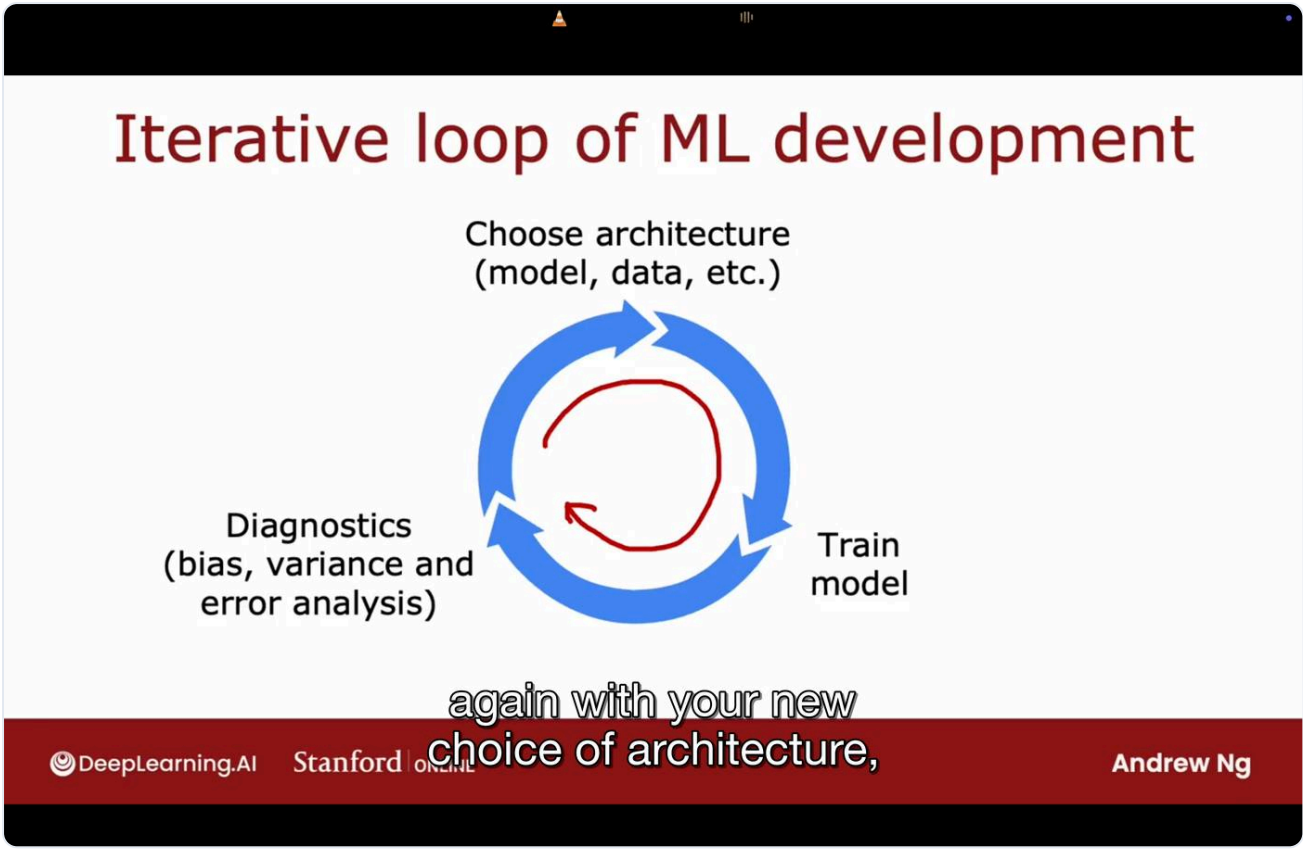


FIGURE 42.4 | Iterative Loop of ML Development

Then you go around this loop again with your new choice of architecture, and it will often take multiple iterations through this loop until you get to the performance that you want. Let's look at an example of building an email spam classifier. I think many of us passionately hate email spam and this is a problem that I worked on years ago and also was involved in starting an anti-spam conference once years ago. The example on the left is what a highly spammy email might look like. Deal of the week, by now, Rolex watches. Spammers will sometimes deliberately misspell words like these, watches, medicine, and mortgages in order to try to trip up a spam recognizer.

In contrast, this email on the right is an actual email I once got from my younger brother Alfred about getting together for Christmas.

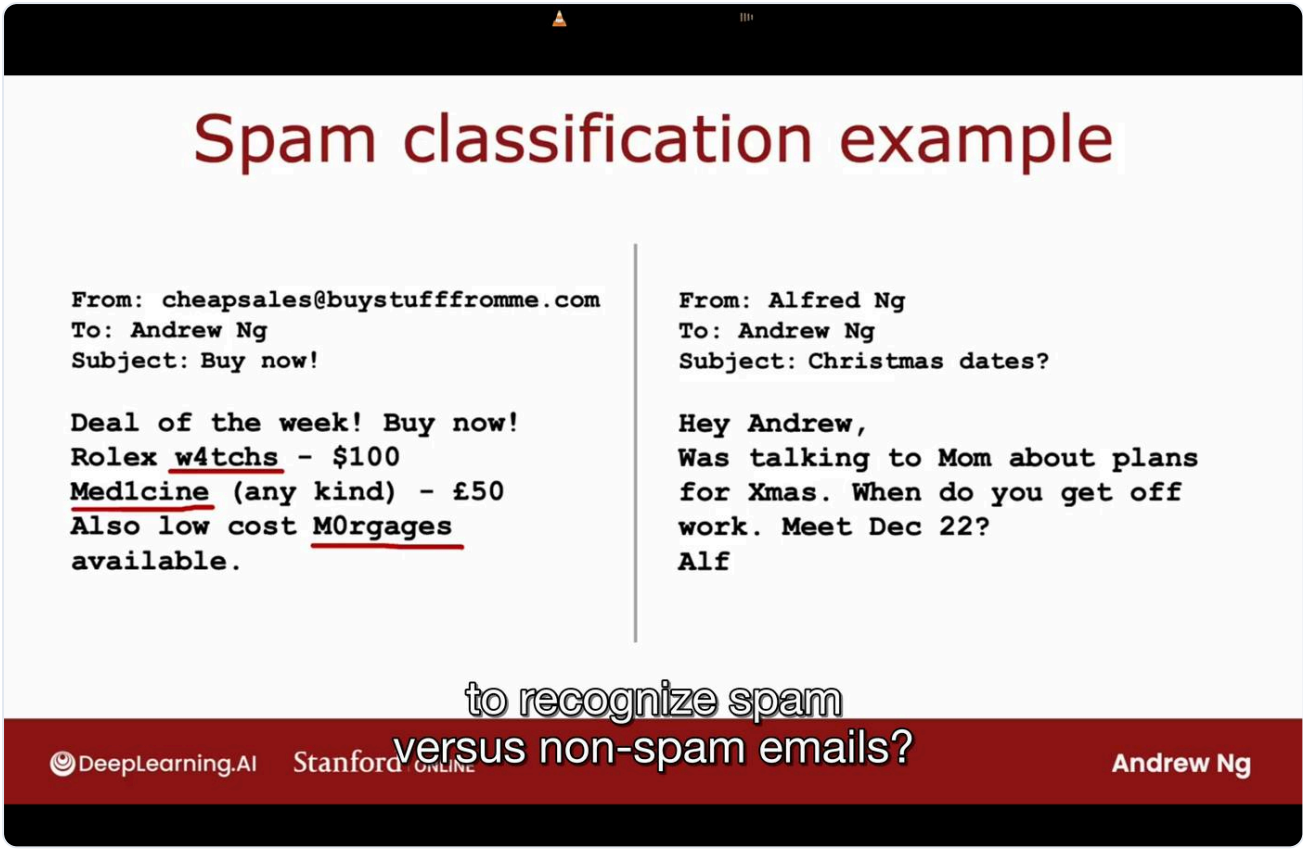


FIGURE 42.5 | Spam Classification Example

How do you build a classifier to recognize spam versus non-spam emails? One way to do so would be to train a supervised learning algorithm where the input features x will be the features of an email and the output label y will be one or zero depending on whether it's spam or non-spam. This application is an example of text classification because you're taking a text document that is an email and trying to classify it as either spam or non-spam. One way to construct the features of the email would be to say, take the top 10,000 words in the English language or in some other dictionary and use them to define features x_1, x_2 through x_10,000. For example, given this email on the right, if the list of words we have is a, Andrew buy deal discount and so on.

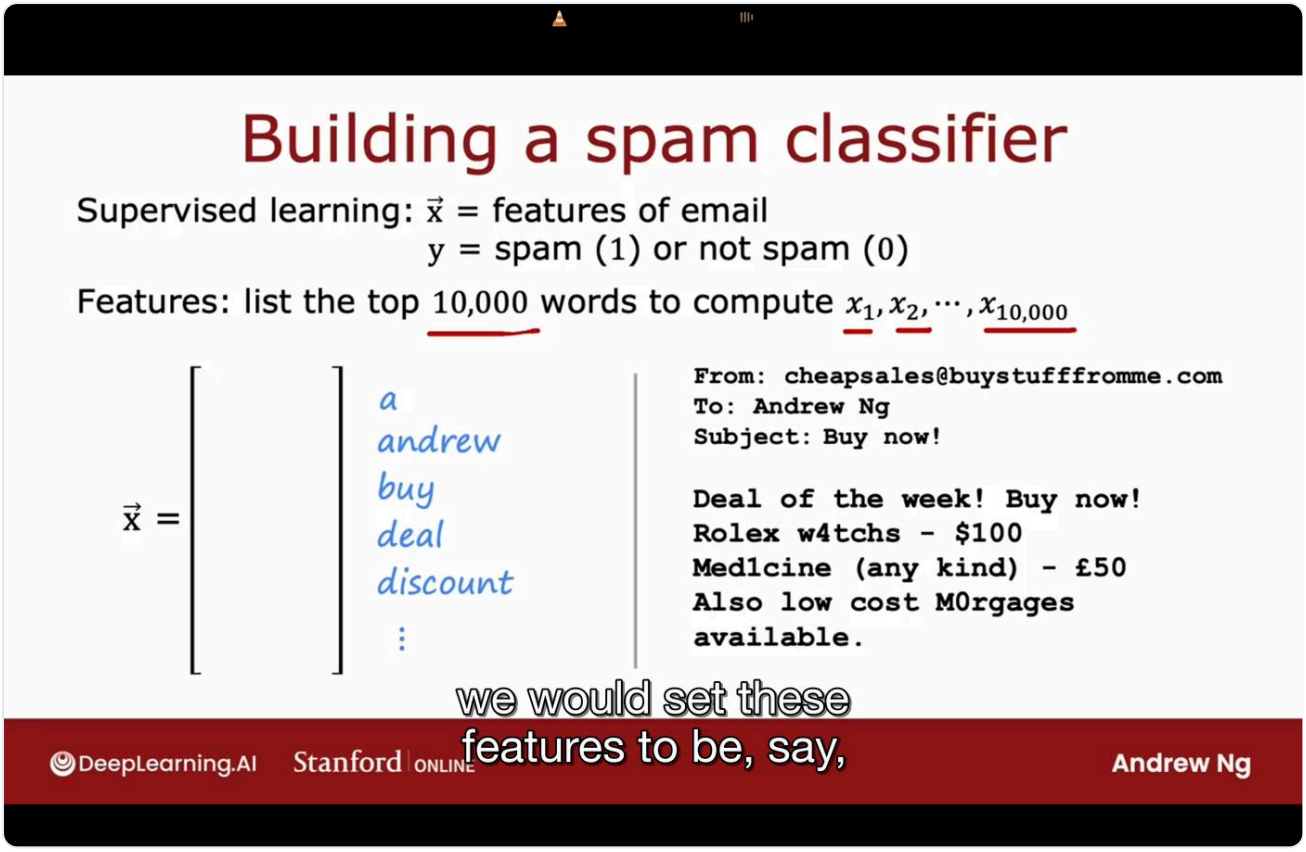


FIGURE 42.6 | Building a Spam Classifier

Then given the email on the right, we would set these features to be, say, 0 or 1, depending on whether or not that word appears. The word a does not appear. The word Andrew does appear.

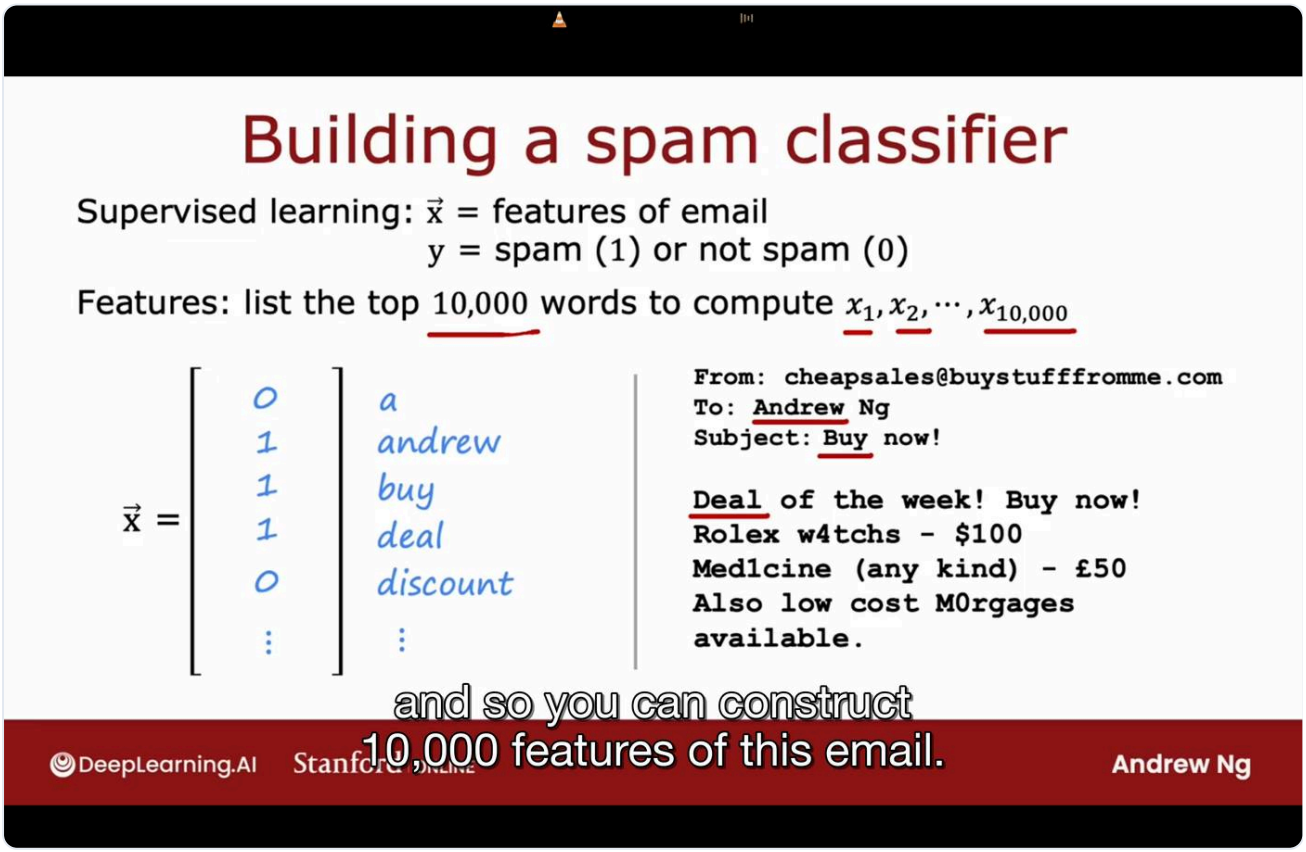


FIGURE 42.7 | Building a Spam Classifier

The word buy does appear, deal does, discount does not, and so on, and so you can construct 10,000 features of this email. There are many ways to construct a feature vector. Another way would be to let these numbers not just be 1 or 0, but actually, count the number of times a given word appears in the email. If buy appears twice, maybe you want to set this to 2, but setting into just 1 or 0. It actually works decently well. Given these features, you can then train a classification algorithm such as a logistic regression model or a neural network to predict y given these features x. After you've trained your initial model, if it doesn't work as well as you wish, you will quite likely have multiple ideas for improving the learning algorithm's performance.

For example, is always tempting to collect more data. In fact, I have friends that have worked on very large-scale honeypot projects. These are projects that create a large number of fake email addresses and tries to deliberately to get these fake email addresses into the hands of spammers so that when they send spam email to these fake emails well we know these are spam email messages and so this is a way to get a lot of spam data. Or you might decide to work on developing more sophisticated features based on the email routing. Email routing refers to the sequence of compute service. Sometimes around the world that the email has gone through all this way to reach you and emails actually have what's called email header information.

That is information that keeps track of how the email has traveled across different servers, across different networks to find its way to you. Sometimes the path that an email has traveled can help tell you if it was sent by a spammer or not. Or you might work on coming up with more sophisticated features from the email body that is the text of the email. In the features I talked about last time, discounting and discount may be treated as different words, and maybe they should be treated as the same words.

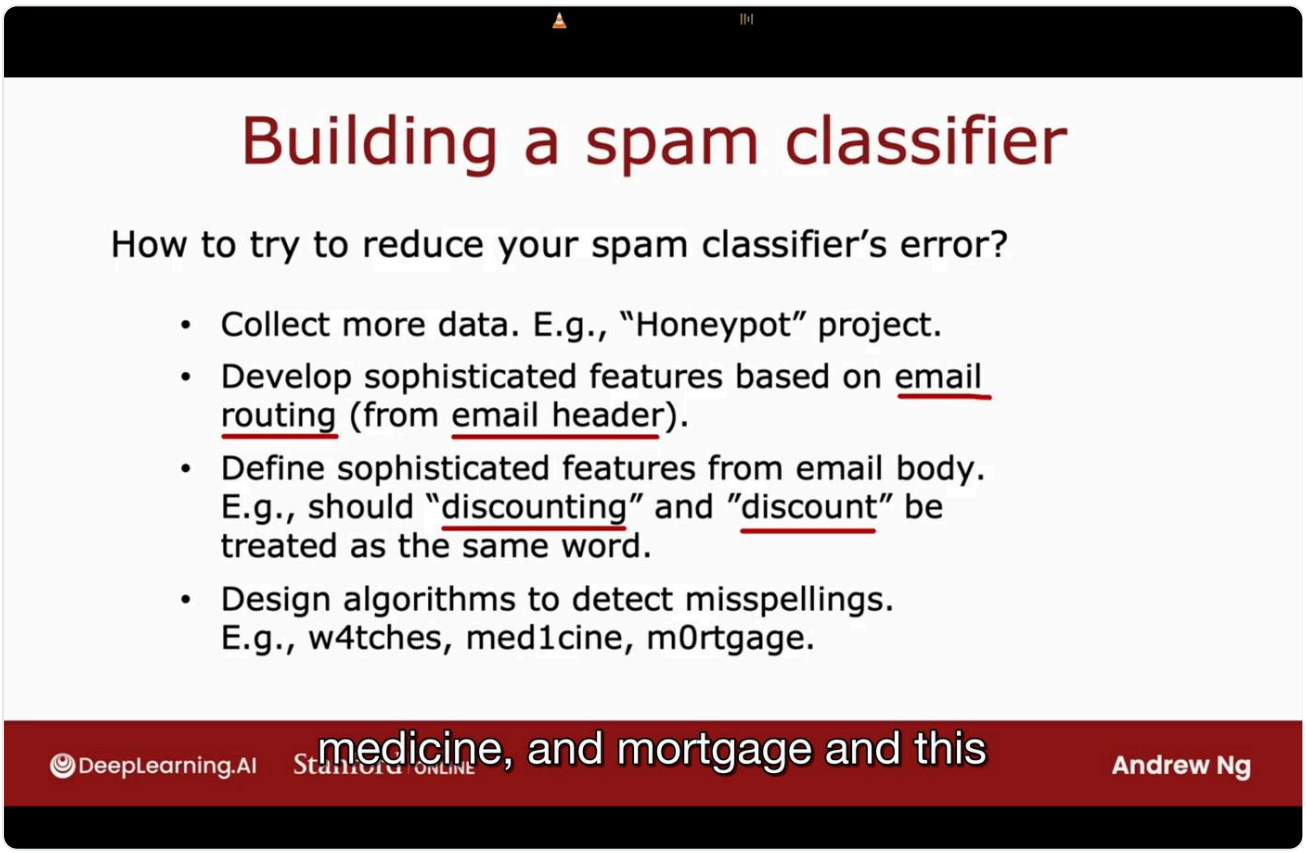


FIGURE 42.8 | Building a Spam Classifier

Or you might decide to come up with algorithms to detect misspellings or deliberate misspellings like watches, medicine, and mortgage and this too could help you decide if an email is spammy. Given all of these and possibly even more ideas, how can you decide which of these ideas are more promising to work on? Because choosing the more promising path forward can speed up your project easily 10 times compared to if you were to somehow choose some of the less promising directions. For example, we've already seen that if your algorithm has high bias rather than high variance, then spending months and months on a honeypot project may not be the most fruitful direction.

But if your algorithm has high variance, then collecting more data could help a lot. Doing the iterative loop of machinery and development, you may have many ideas for how to modify the model or the data, and it will be coming up with different diagnostics that could give you a lot of guidance on what choices for the model or data, or other parts of the architecture could be most promising to try. In the last several videos, we've already talked about bias and variance. In the next video, I'd like to start describing to you the error analysis process, which has a second key set of ideas for gaining insight about what architecture choices might be fruitful.

That's the iterative loop of machine learning development and using the example of building a spam classifier. Let's take a look at what error analysis looks like. Let's do that in the next video.

LECTURE 43

Error Analysis

In terms of the most important ways to help you run diagnostics to choose what to try next to improve your learning algorithm performance, I would say bias and variance is probably the most important idea and error analysis would probably be second on my list. Let's take a look at what this means. Concretely, let's say you have m_cv equals 500 cross validation examples and your algorithm misclassifies 100 of these 500 cross validation examples. The error analysis process just refers to manually looking through these 100 examples and trying to gain insights into where the algorithm is going wrong. Specifically, what I will often do is find a set of examples that the algorithm has misclassified examples from the cross validation set and try to group them into common teams or common properties or common traits.

For example, if you notice that quite a lot of the misclassified spam emails are pharmaceutical sales, trying to sell medicines or drugs then I will actually go through these examples and count up by hand how many emails that are misclassified are pharmaceutical spam and say there are 21 emails that are pharmaceutical spam. Or if you suspect that deliberate misspellings may be tripping over your spam classifier then I will also go through and just count up how many of these examples that it misclassified had a deliberate misspelling. Let's say I find three out of a 100. Or looking through the email routing info I find seven has unusual email routing and 18 emails trying to steal passwords or phishing emails.

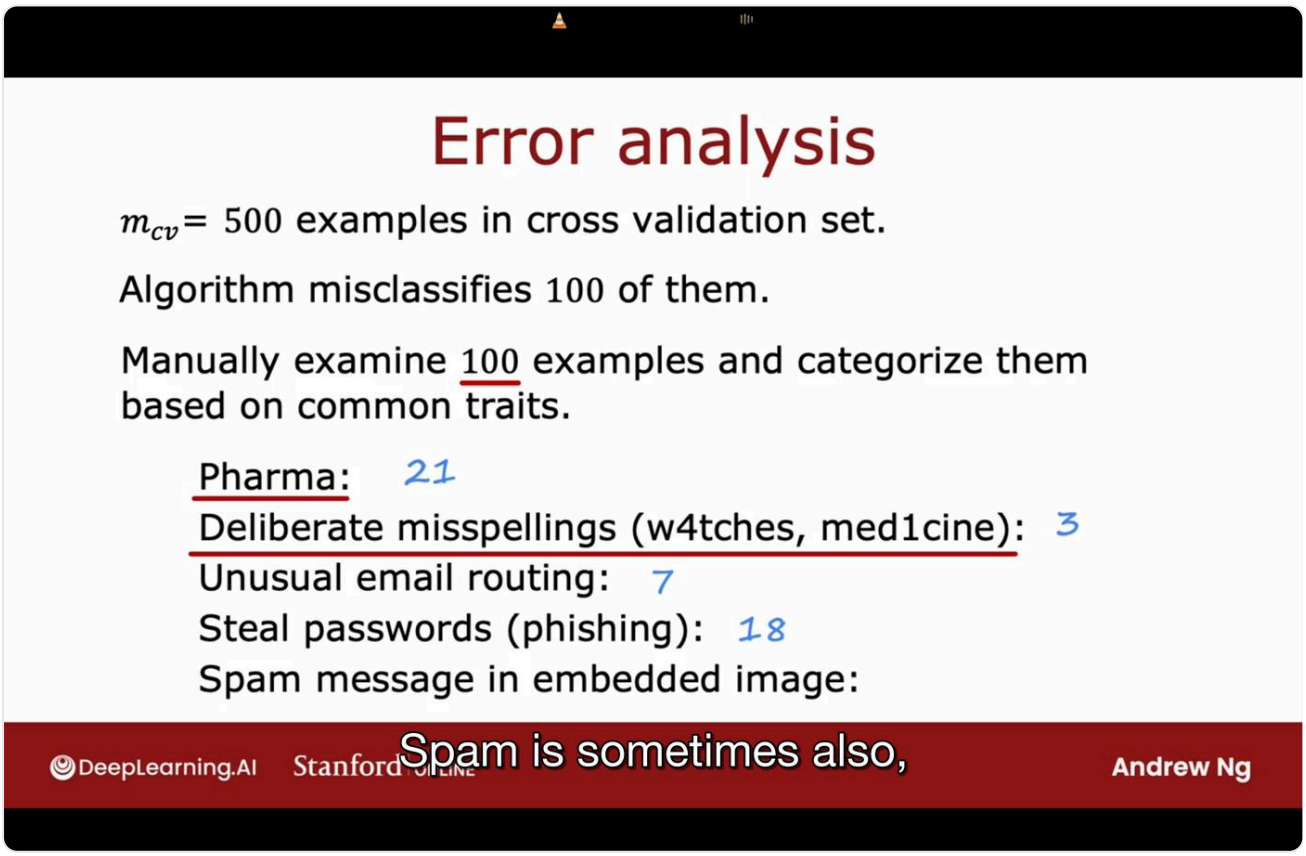


FIGURE 43.1 | Error Analysis

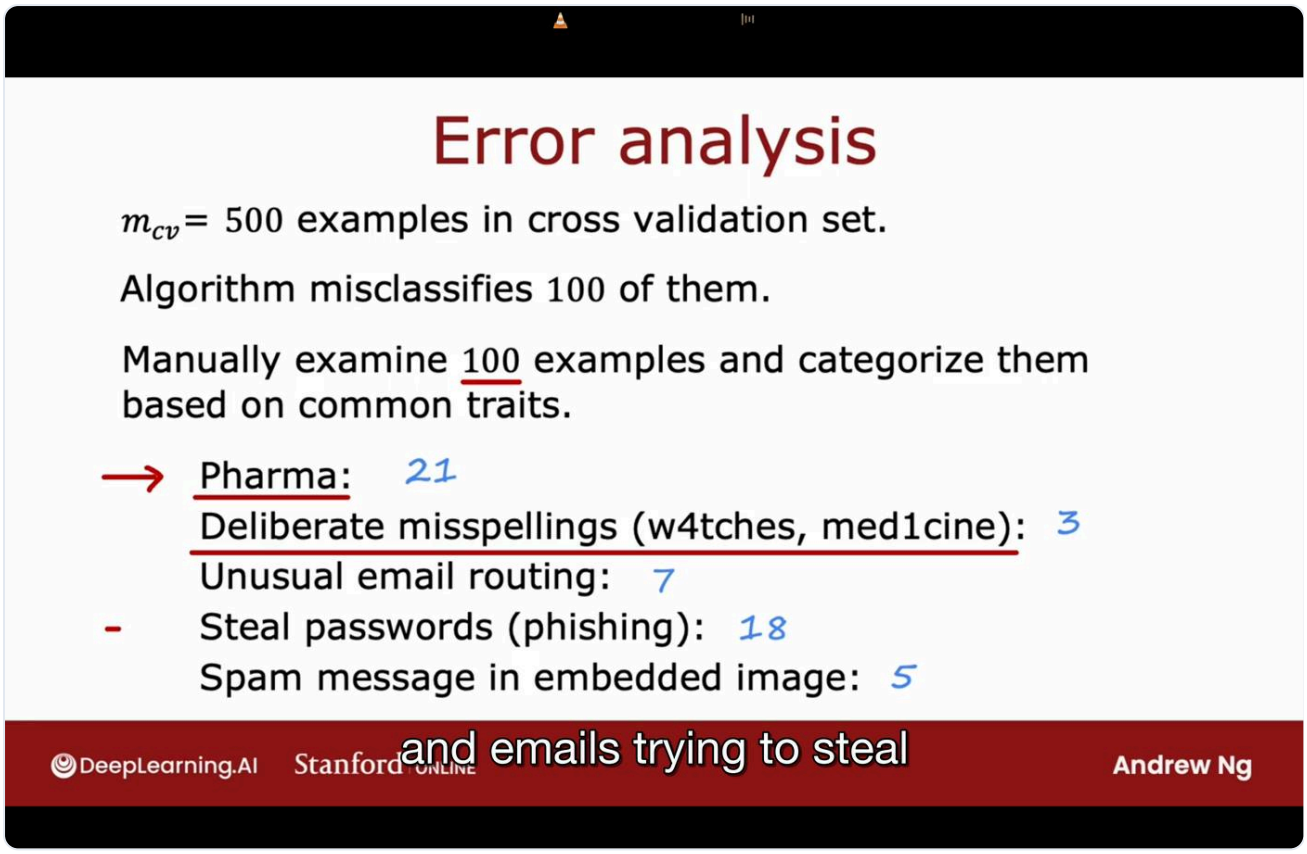


FIGURE 43.2 | Error Analysis

Spam is sometimes also, instead of writing the spam message in the email body they instead create an image and then writes to spam the message inside an image that appears in the email. This makes it a little bit harder for learning algorithm to figure out what's going on. Maybe some of those emails are these embedded image spam. If you end up with these counts then that tells you that pharmaceutical spam and emails trying to steal passwords or phishing emails seem to be huge problems whereas deliberate misspellings, well, it is a problem it is a smaller one. In particular, what this analysis tells you is that even if you were to build really sophisticated algorithms to find deliberate misspellings it will only solve three out of 100 of your misclassified examples.

The net impact seems like it may not be that large. Doesn't mean it's not worth doing? But when you're prioritizing what to do, you might therefore decide not to prioritizes this as highly. By the way, I'm telling the story because I once actually spent a lot of time building algorithms to find deliberate misspellings and spam emails only much later to realize that the net impact was actually quite small. This is one example where I wish I'd done more careful error analysis before spending a lot of time myself trying to find these deliberate misspellings. Just a couple of notes on this process.

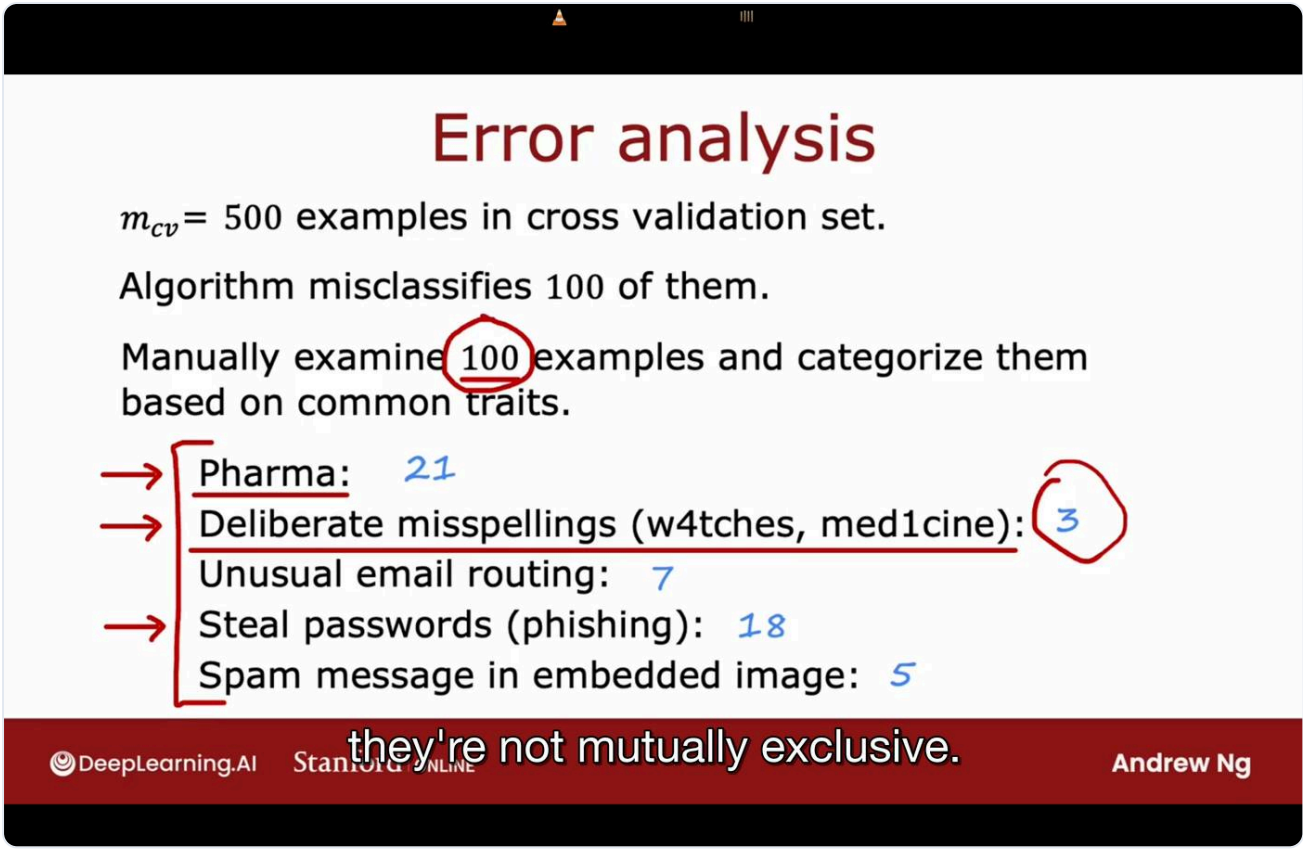


FIGURE 43.3 | Error Analysis

These categories can be overlapping or in other words they're not mutually exclusive. For example, there can be a pharmaceutical spam email that also has unusual routing or a password that has deliberate misspellings and is also trying to carry out the phishing attack. One email can be counted in multiple categories. In this example, I had said that the algorithm misclassified as 100 examples and we'll look at all 100 examples manually. If you have a larger cross validation set, say we had 5,000 cross validation examples and if the algorithm misclassified say 1,000 of them then you may not have the time depending on the team size and how much time you have to work on this project.

You may not have the time to manually look at all 1,000 examples that the algorithm misclassifies. In that case, I will often sample randomly a subset of usually around a 100, maybe a couple 100 examples because that's the amount that you can look through in a reasonable amount of time. Hopefully looking through maybe around a 100 examples will give you enough statistics about whether the most common types of errors and therefore where maybe most fruitful to focus your attention. After this analysis, if you find that a lot of errors are pharmaceutical spam emails then this might give you some ideas or inspiration for things to do next. For example, you may decide to collect more data but not more data of everything, but just try to find more data of pharmaceutical spam emails so that the learning algorithm can do a better job recognizing these pharmaceutical spam.

Or you may decide to come up with some new features that are related to say specific names of drugs or specific names of pharmaceutical products of the spammers are trying to sell in order to help your learning algorithm become better at recognizing this type of pharma spam. Then again

this might inspire you to make specific changes to the algorithm relating to detecting phishing emails. For example, you might look at the URLs in the email and write special code to come with extra features to see if it's linking to suspicious URLs.

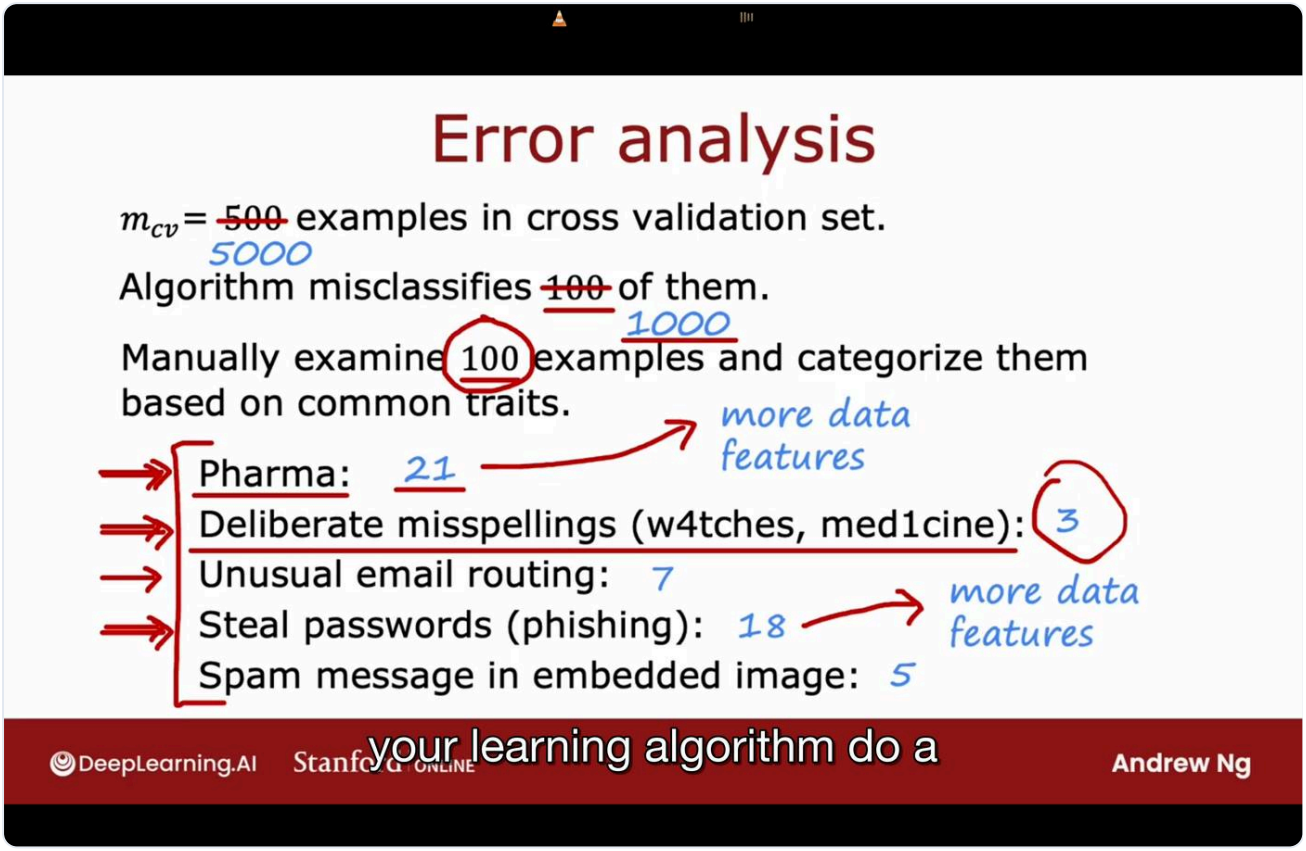


FIGURE 43.4 | Error Analysis

Or again, you might decide to get more data of phishing emails specifically in order to help your learning algorithm do a better job of recognizing them. The point of this error analysis is by manually examining a set of examples that your algorithm is misclassifying or mislabeling. Often this will create inspiration for what might be useful to try next and sometimes it can also tell you that certain types of errors are sufficiently rare that they aren't worth as much of your time to try to fix. Returning to this list, a bias variance analysis should tell you if collecting more data is helpful or not.

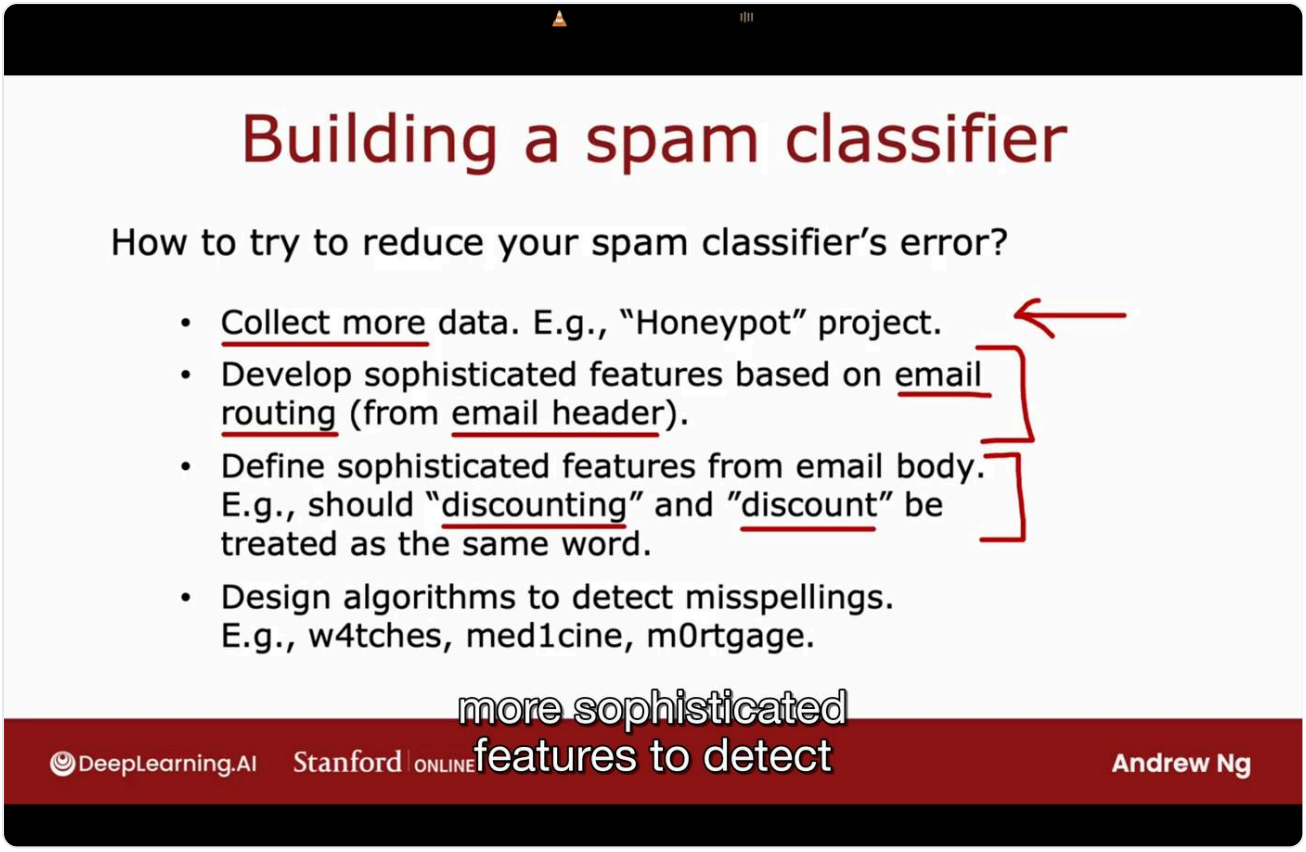


FIGURE 43.5 | Building a Spam Classifier

Based on our error analysis in the example we just went through, it looks like more sophisticated email features could help but only a bit whereas more sophisticated features to detect pharma spam or phishing emails could help a lot. These detecting misspellings would not help nearly as much.

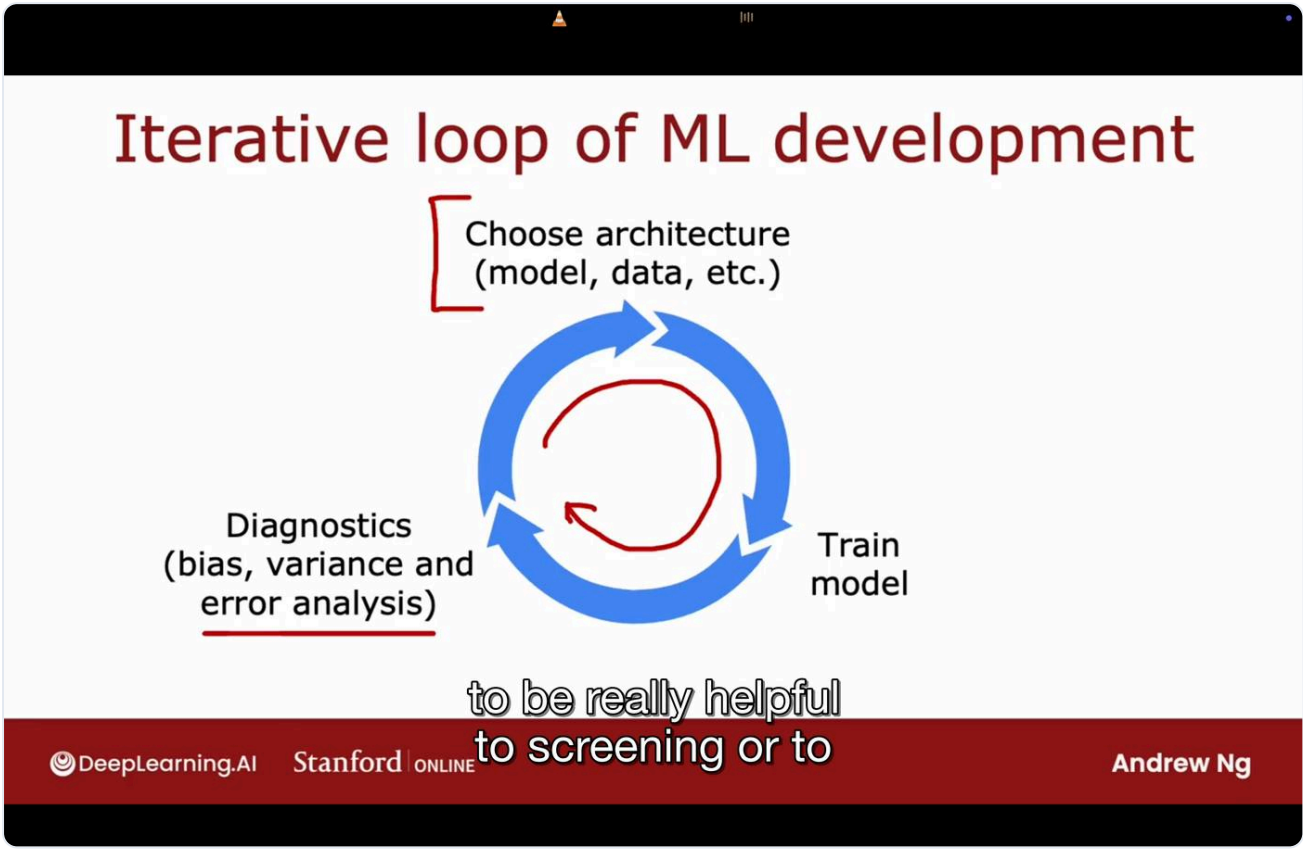


FIGURE 43.6

In general I found both the bias variance diagnostic as well as carrying out this form of error analysis to be really helpful to screening or to deciding which changes to the model are more promising to try on next. Now one limitation of error analysis is that it's much easier to do for problems that humans are good at. You can look at the email and say you think is a spam email, why did the algorithm get it wrong? Error analysis can be a bit harder for tasks that even humans aren't good at. For example, if you're trying to predict what ads someone will click on on the website. Well, I can't predict what someone will click on. Error analysis there actually tends to be more difficult.

But when you apply error analysis to problems that you can it can be extremely helpful for focusing attention on the more promising things to try. That in turn can easily save you months of otherwise fruitless work. In the next video, I'd like to dive deeper into the problem of adding data. When you train a learning algorithm, sometimes you decide there's high variance and you want to get more data for it. Some techniques they can make how you add data much more efficient. Let's take a look at that so that hopefully you'll be armed with some good ways to get more data for your learning application.

LECTURE 44

Data Augmentation by Introducing Distortions

In this video, I'd like to share with you some tips for adding data or collecting more data or sometimes even creating more data for your machine learning application. Just a heads up that this in the next few videos will seem a little bit like a grab bag of different techniques. And I apologize if it seems a little bit grab baggy and that's because machine learning applications are different. Machine learning is applied to so many different problems and for some humans are great at creating labels. And for some you can get more data and for some you can't. And that's why different applications actually sometimes call for slightly different techniques.

But I hope in this in the next few videos to share with you some of the techniques that are found to be most useful for different applications, although not every one of them will apply for every single application. But I hope many of them would be useful for many of the applications that you'll be working on as well. But let's take a look at some tips for how to add data for your application. When training machine learning algorithms, it feels like always we wish we had even more data almost all the time. And so sometimes it's tempting to let's just get more data of everything. But, trying to get more data of all types can be slow and expensive.

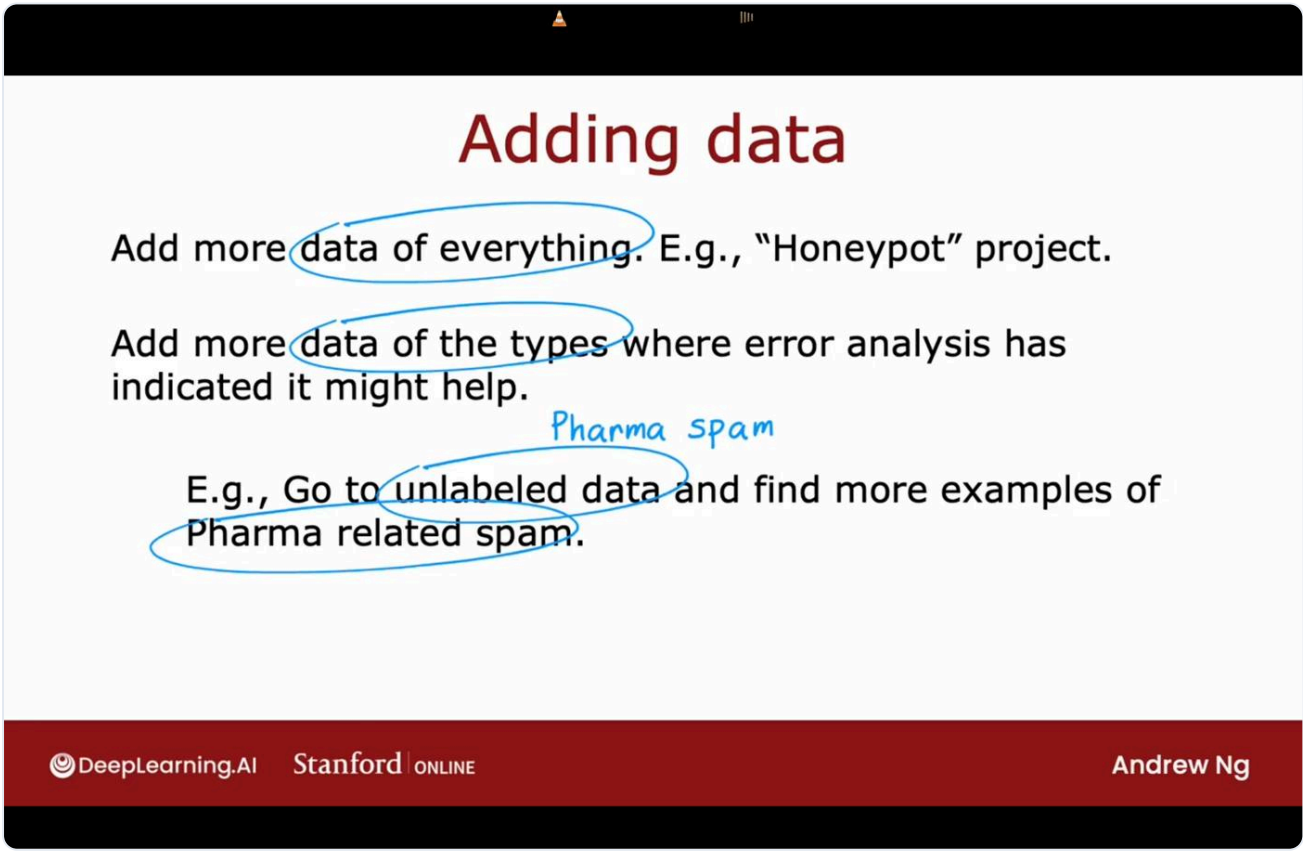


FIGURE 44.1 | Adding Data

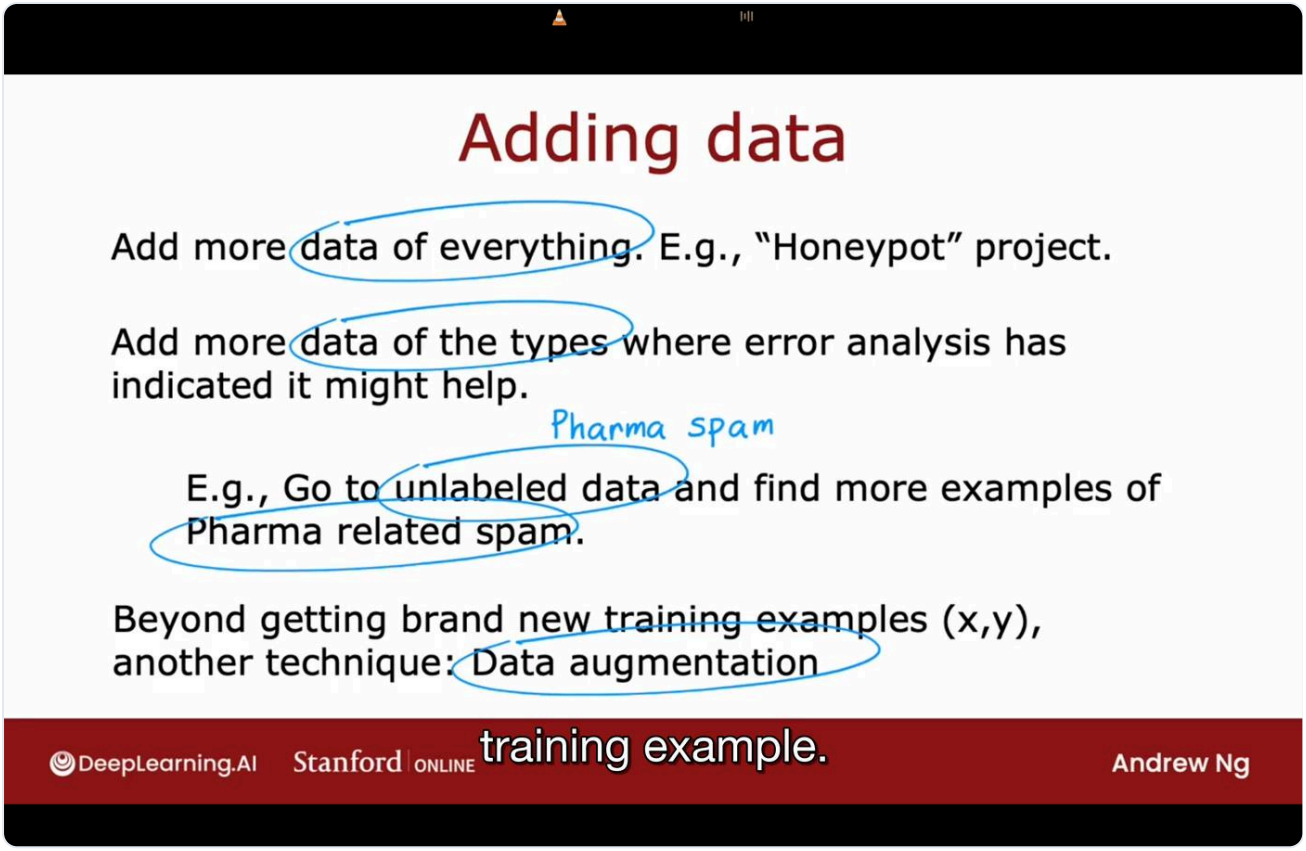


FIGURE 44.2 | Adding Data

Instead, an alternative way of adding data might be to focus on adding more data of the types where analysis has indicated it might help. In the previous slide we saw if error analysis reviewed that pharma spam was a large problem, then you may decide to have a more targeted effort not to get more data everything under the sun but to stay focused on getting more examples of pharma spam And with a more modest cost this could let you add just the emails you need to hope you're learning and get smarter on recognizing pharma spam. And so one example of how you might do this is, if you have a lot of unlabeled email data, say emails sitting around and no one has bothered to label yet as spam or non-spam You may able to ask your labors to quickly skim through the unlabeled data and find more examples specifically a pharma related spam.

And this could boost your learning algorithm performance much more than just trying to add more data of all sorts of emails. But the more general pattern I hope you take away from this is, if you have some ways to add more data of everything that's okay. Nothing wrong with that. But if error analysis has indicated that there are certain subsets of the data that the algorithm is doing particularly poorly on. And that you want to improve performance on, then getting more data of just the types where you wanted to do better. Be it more examples of pharmaceutical spam or more examples of phishing spam or something else. That could be a more efficient way to add just a little bit of data but boost your algorithms performance by quite a lot.

Beyond getting your hands on brand new training examples xy. There's another technique that's widely used especially for images and audio data that can increase your training set size significantly. This technique is called data augmentation. And what we're going to do is take an existing training example to create a new training example. For example if you're trying to recognize the letters from A to Z for an OCR optical character recognition problem. So not just the digits 0-9 but

also the letters from A to Z. Given an image like this, you might decide to create a new training example by rotating the image a bit. Or by enlarging the image a bit or by shrinking a little bit or by changing the contrast of the image.

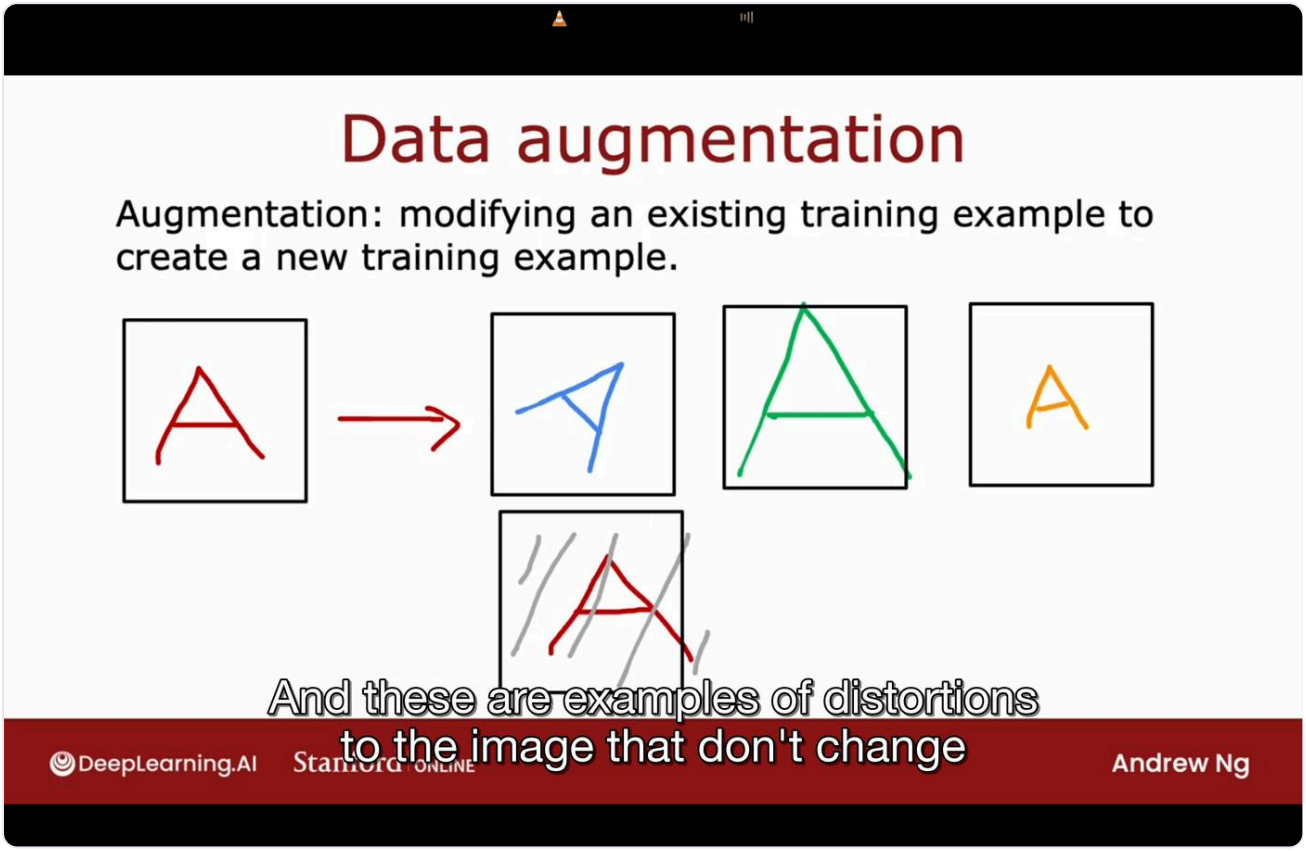


FIGURE 44.3 | Data Augmentation

And these are examples of distortions to the image that don't change the fact that this is still the letter A And for some letters but not others you can also take the mirror image of the letter and it still looks like the letter A.

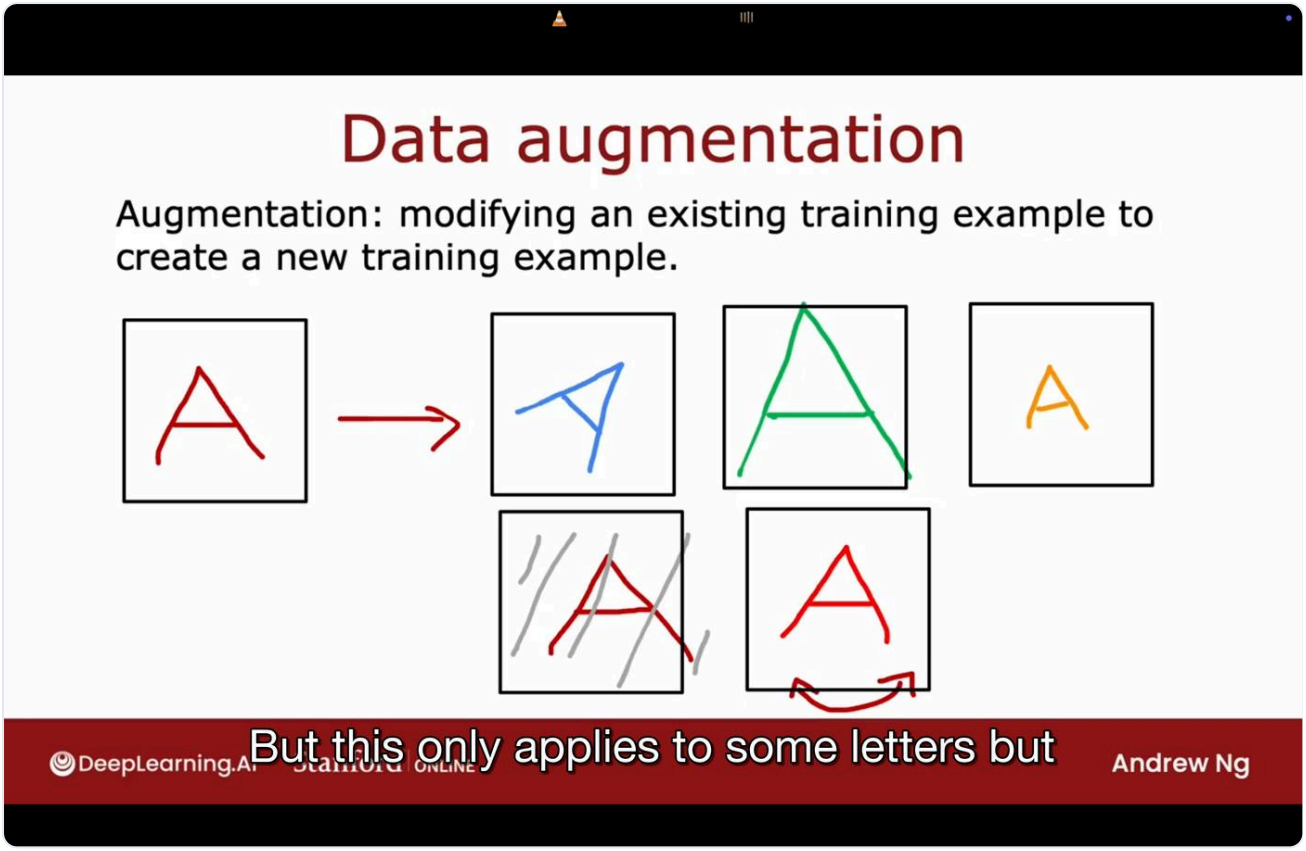


FIGURE 44.4 | Data Augmentation

But this only applies to some letters but these would be ways of taking a training example X, Y. And applying a distortion or transformation to the input X ,in order to come up with another example that has the same label. And by doing this you're telling the algorithm that the letter A rotated a bit or enlarged a bit or shrunk a little bit it is still the letter A. And creating additional examples like this holds the learning algorithm, do a better job learning how to recognize the letter A. For a more advanced example of data augmentation. You can also take the letter A and place a grid on top of it. And by introducing random warping of this grid, you can take the letter A.

And introduce warpings of the leather A to create a much richer library of examples of the letter A.

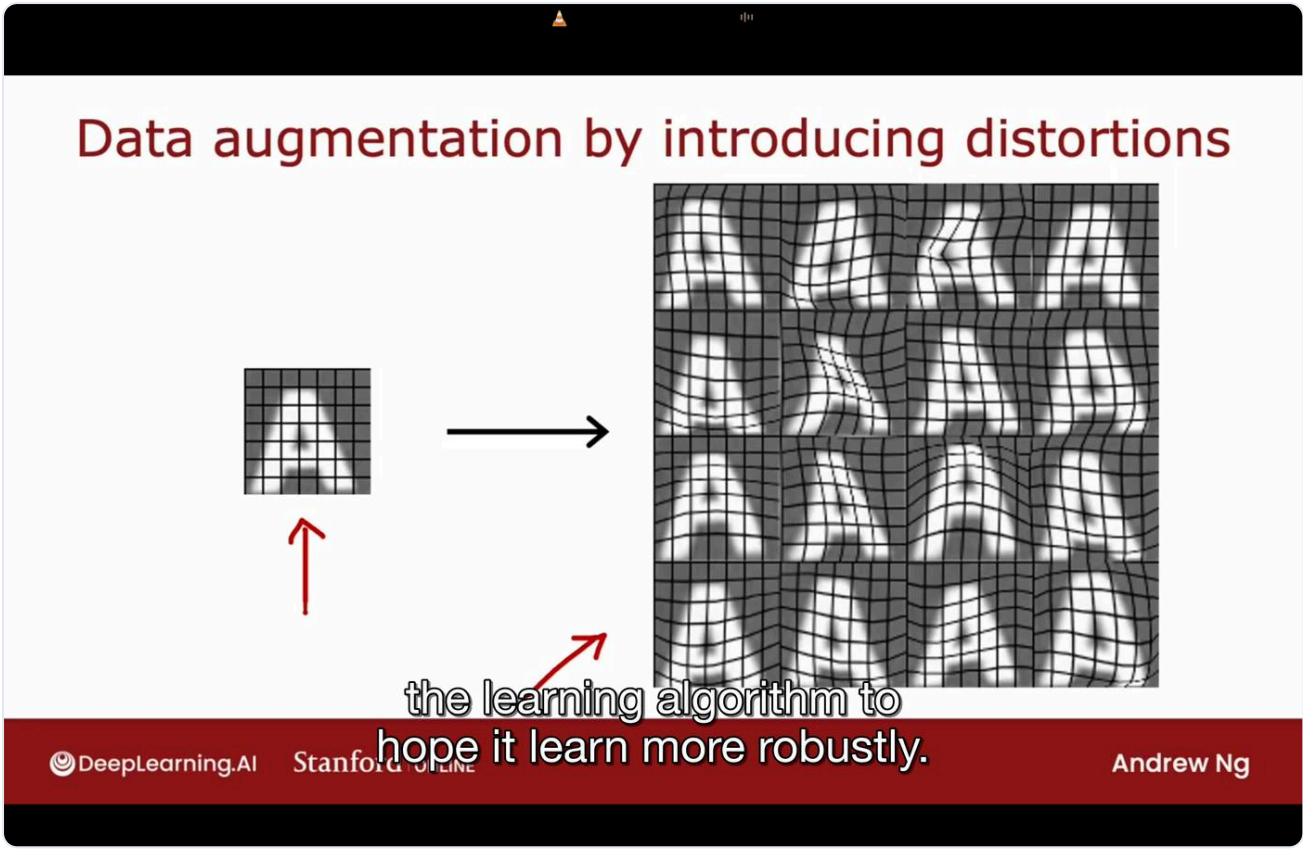


FIGURE 44.5 | Data Augmentation by Introducing Distortions

And this process of distorting these examples then has turned one image of one example into here training examples that you can feed to the learning algorithm to hope it learn more robustly. What is the letter A. This idea of data augmentation also works for speech recognition. Let's say for a voice search application, you have an original audio clip that sounds like this. >> What is today's weather. >> One way you can apply data augmentation to speech data would be to take noisy background audio like this. For example, this is what the sound of a crowd sounds like. And it turns out that if you take these two audio clips, the first one and the crowd noise and you add them together, then you end up with an audio clip that sounds like this. >> What is today's weather. >> And you just created an audio clip that sounds like someone saying what's the weather today.

But they're saying it around the noisy crowd in the background. Or in fact if you were to take a different background noise, say someone in the car, this is what background noise of a car sounds like. And you want to add the original audio clip to the car noise, then you get this. >> What is today's weather. >> And it sounds like the original audio clip, but as if the speaker was saying it from a car.



FIGURE 44.6 | Data Augmentation for Speech

And the more advanced data augmentation step would be if you make the original audio sound like you're recording it on a bad cell phone connection like this. And so we've seen how you can take one audio clip and turn it into three training examples here, one with crowd background noise, one with car background noise and one as if it was recorded on a bad cell phone connection. And the times I worked on speech recognition systems, this was actually a really critical technique for increasing artificially the size of the training data I had to build a more accurate speech recognizer. One tip for data augmentation is that the changes or the distortions you make to the data, should be representative of the types of noise or distortions in the test set.

So for example, if you take the letter a and warp it like this, this still looks like examples of letters you might see out there that you would like to recognize.

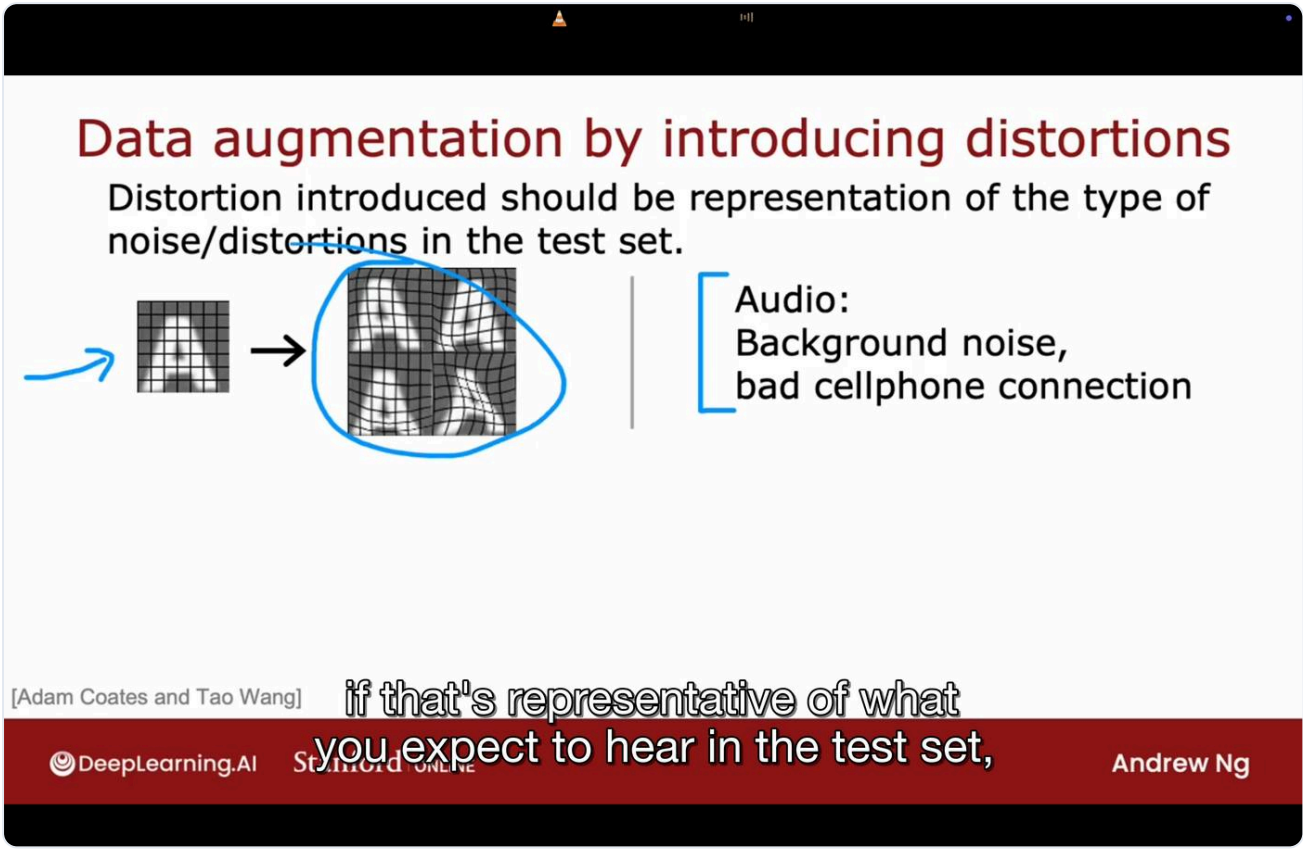


FIGURE 44.7 | Data Augmentation by Introducing Distortions

Or for audio adding background noise or bad cellphone connection if that's representative of what you expect to hear in the test set, then this will be helpful ways to carry out data augmentation on your audio data. In contrast is usually not that helpful at purely random meaningless noise to data.

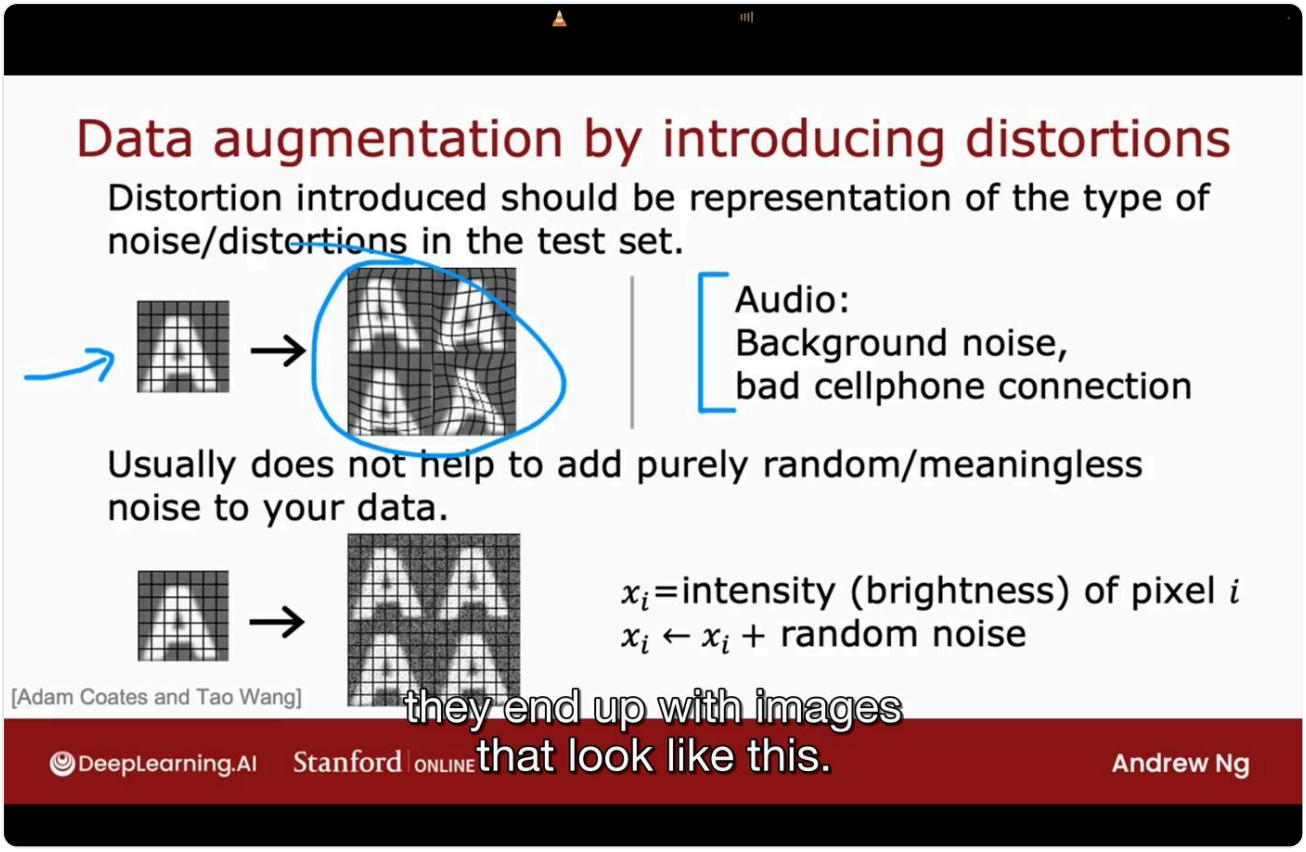


FIGURE 44.8 | Data Augmentation by Introducing Distortions

For example,, you have taken the letter A and I've added per pixel noise where if Xi is the intensity or the brightness of pixel i, if I were to just add noise to each pixel, they end up with images that look like this. But if to the extent that this isn't that representative of what you see in the test set because you don't often get images like this in the test set is actually going to be less helpful. So one way to think about data augmentation is how can you modify or warp or distort or make more noise in your data. But in a way so that what you get is still quite similar to what you have in your test set, because that's what the learning algorithm will ultimately end up doing well on.

Now, whereas data augmentation takes an existing training example and modifies it to create another training example. There's one of the techniques which is data synthesis in which you make up brand new examples from scratch.

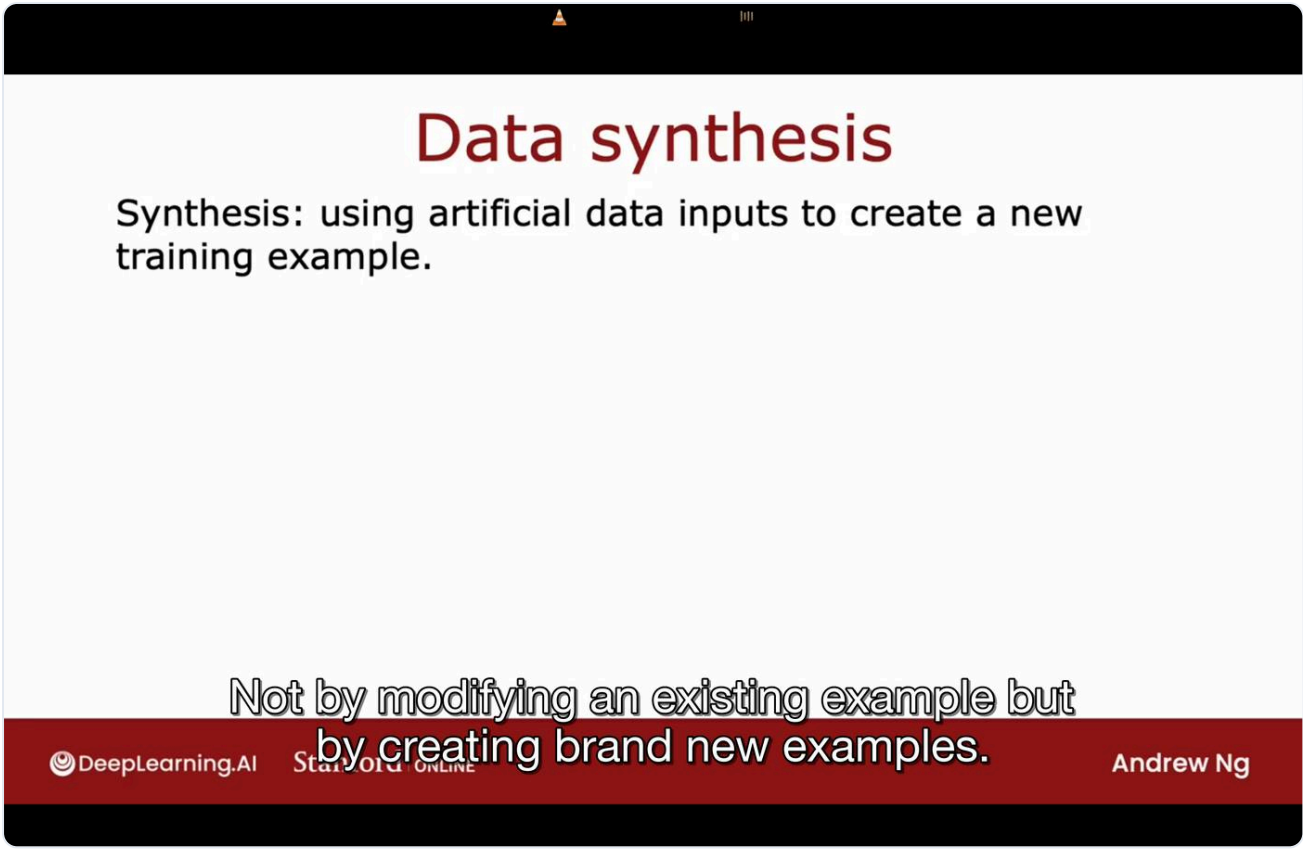


FIGURE 44.9 | Data Synthesis

Not by modifying an existing example but by creating brand new examples. So take the example of photo OCR. Photo OCR or photo optical character recognition refers to the problem of looking at an image like this and automatically having a computer read the text that appears in this image. So there's a lot of text in this image.

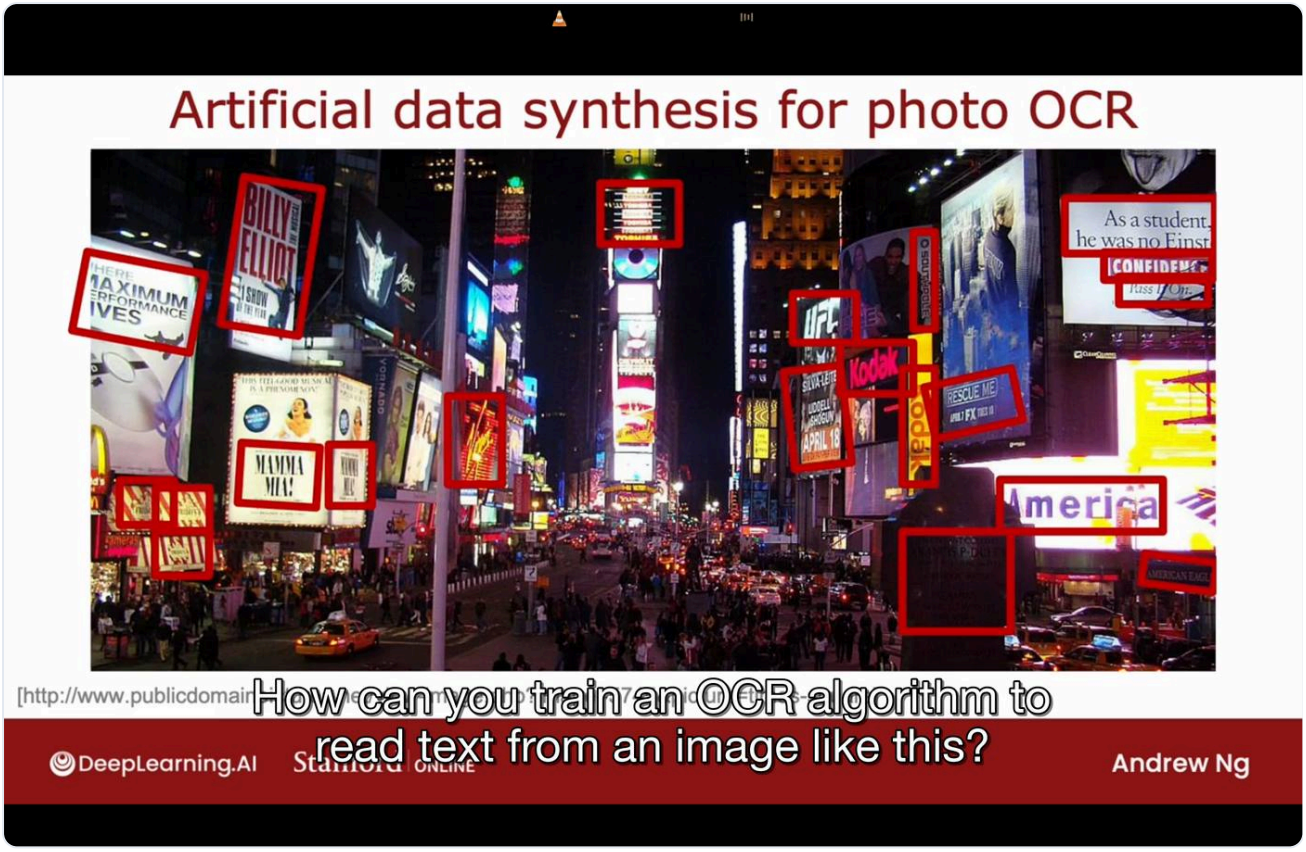


FIGURE 44.10 | Artificial Data Synthesis for Photo OCR

How can you train an OCR algorithm to read text from an image like this? Well, when you look closely at what the letters in this image looks like they actually look like this. So this is real data from a photo OCR task. And one key step with the photo OCR task is to be able to look at the little image like this, and recognize the letter at the middle. So this has T in the middle, this has the letter L in the middle, this has the letter C in the middle and so on.

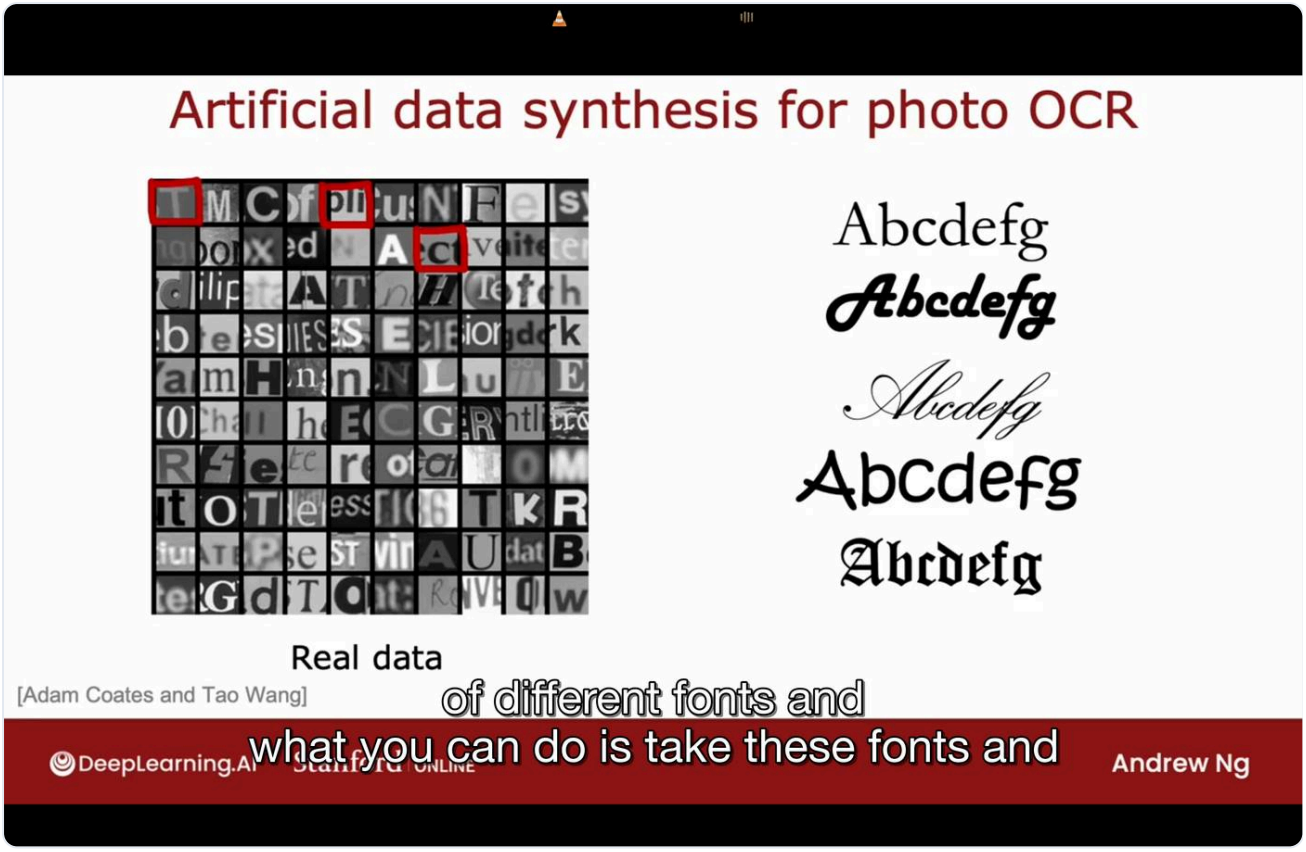


FIGURE 44.11 | Artificial Data Synthesis for Photo OCR

So one way to create artificial data for this task is if you go to your computer's text editor, you find that it has a lot of different fonts and what you can do is take these fonts and basically type of random text in your text editor.

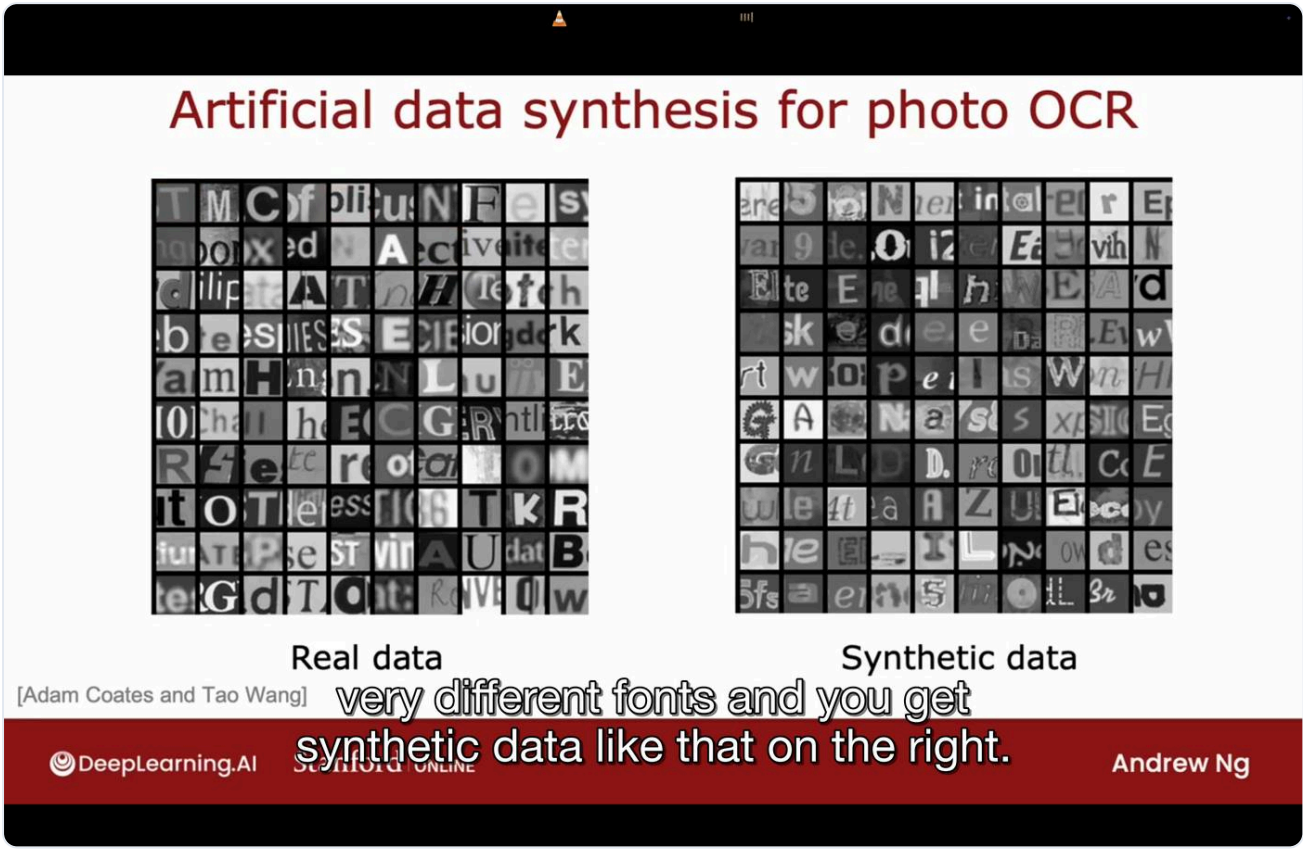


FIGURE 44.12 | Artificial Data Synthesis for Photo OCR

And screenshotted using different colors and different contrasts and very different fonts and you get synthetic data like that on the right. The images on the left were real data from real pictures taken out in the world. The images on the right are synthetically generated using fonts on the computer,, and it actually looks pretty realistic. So with synthetic data generation like this you can generate a very large number of images or examples for your photo OCR task. It can be a lot of work to write the code to generate realistic looking synthetic data for a given application. But when you spend the time to do so, it can sometimes help you generate a very large amount of data for your application and give you a huge boost to your algorithm's performance.

Synthetic data generation has been used most probably for computer vision tasks and less for other applications. Not that much for audio tasks as well.

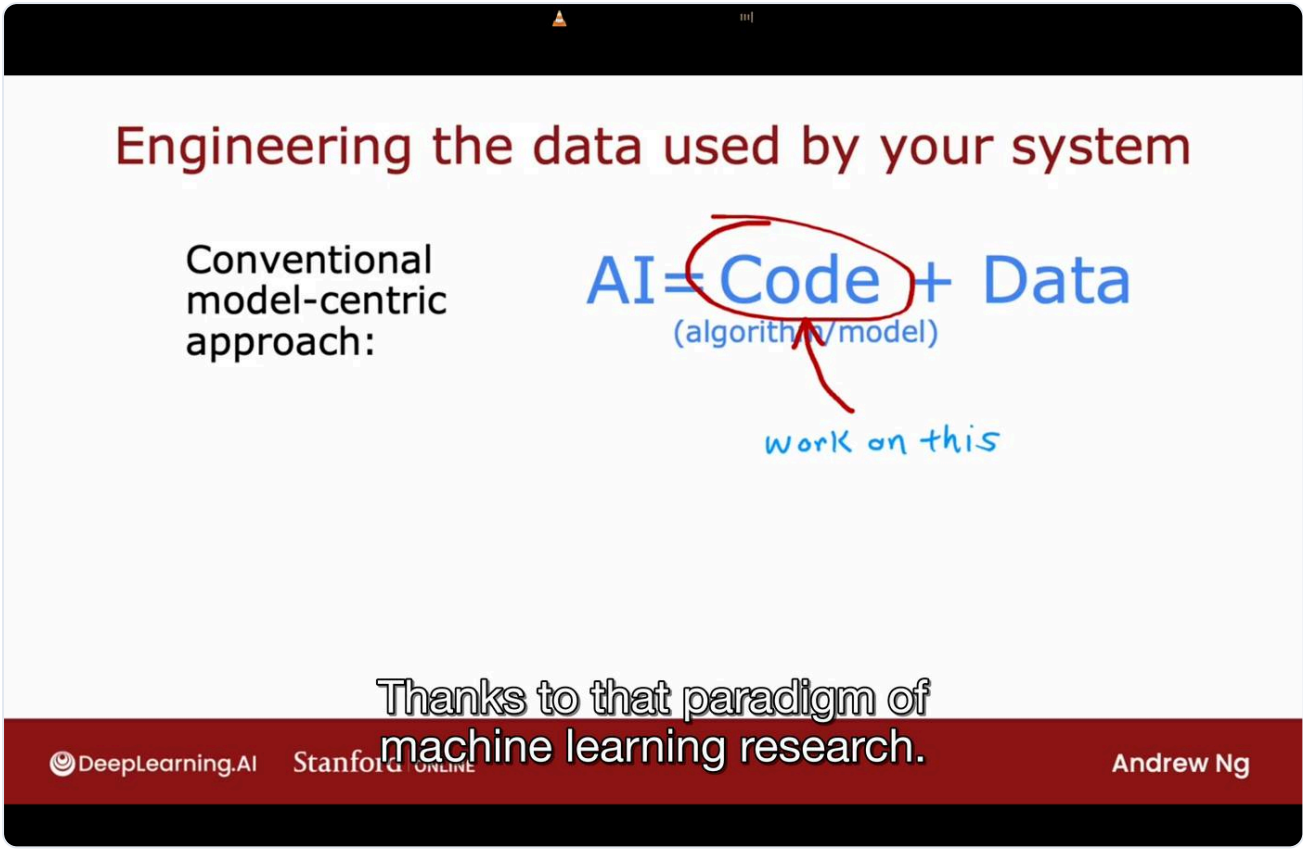


FIGURE 44.13 | Engineering the Data Used by Your System

All the techniques you've seen in this video related to finding ways to engineer the data used by your system. In the way that machine learning has developed over the last several decades, many decades. Most machine learning researchers attention was on the conventional model centric approach and here's what I mean. A machine learning system or an AI system includes both code to implement your album or your model, as well as the data that you train the algorithm model. and over the last few decades, most researchers doing machine learning research would download the data set and hold the data fixed while they focus on improving the code or the algorithm or the model.

Thanks to that paradigm of machine learning research. I find that today the algorithm we have access to such as linear regression, logistic regression, neural networks, also decision trees we should see next week. There are algorithms that already very good and will work well for many applications. And so sometimes it can be more fruitful to spend more of your time taking a data centric approach in which you focus on engineering the data used by your algorithm.

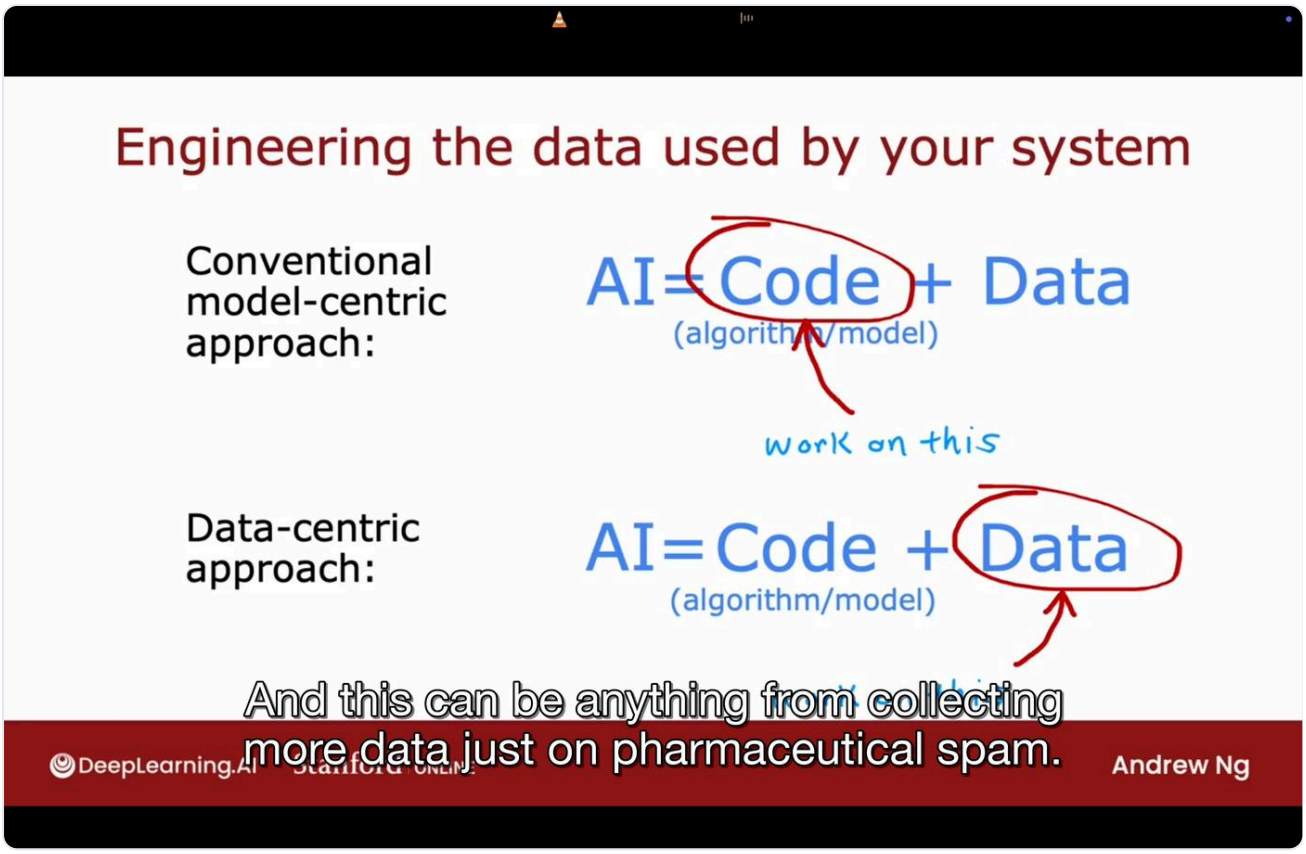


FIGURE 44.14 | Engineering the Data Used by Your System

And this can be anything from collecting more data just on pharmaceutical spam. If that's what error analysis told you to do. To using data augmentation to generate more images or more audio or using data synthesis to just create more training examples.

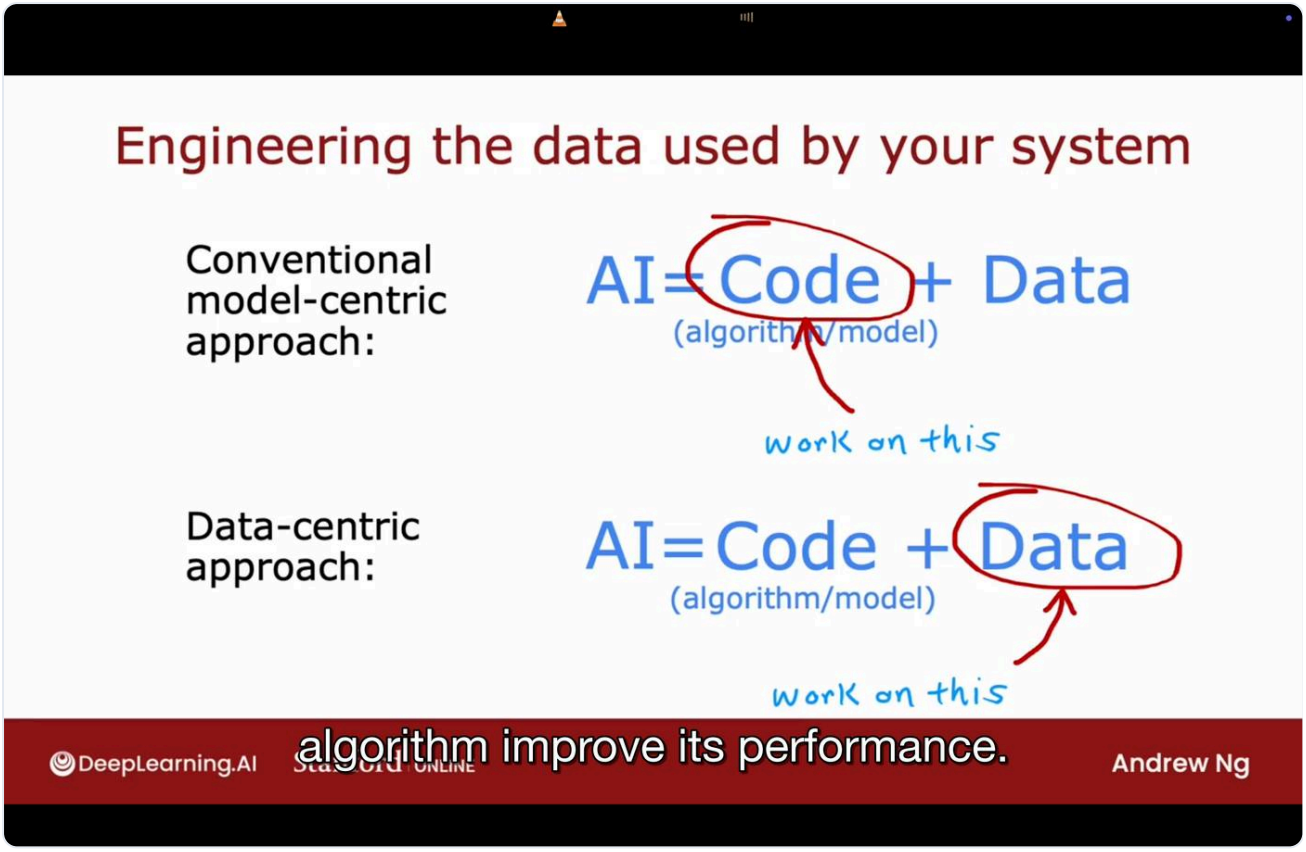


FIGURE 44.15 | Engineering the Data Used by Your System

And sometimes that focus on the data can be an efficient way to help your learning algorithm improve its performance. So I hope that this video gives you a set of tools to be efficient and effective in how you add more data to get your learning algorithm to work better. Now there are also some applications where you just don't have that much data and it's really hard to get more data. It turns out that there's a technique called transfer learning which could apply in that setting to give your learning algorithm performance a huge boost. And the key idea is to take data from a totally different barely related tasks. But using a neural network there's sometimes ways to use that data from a very different tasks to get your algorithm to do better on your application.

Doesn't apply to everything, but when it does it can be very powerful. Let's take a look in the next video and how transfer learning works.

LECTURE 45

Transfer Learning

For an application where you don't have that much data, transfer learning is a wonderful technique that lets you use data from a different task to help on your application. This is one of those techniques that I use very frequently. Let's take a look at how transfer learning works. Here's how transfer learning works. Let's say you want to recognize the handwritten digits from zero through nine but you don't have that much labeled data of these handwritten digits. Here's what you can do. Say you find a very large datasets of one million images of pictures of cats, dogs, cars, people, and so on, a thousand classes. You can then start by training a neural network on this large dataset of a million images with a thousand different classes and train the algorithm to take as input an image X, and learn to recognize any of these 1,000 different classes.

In this process, you end up learning parameters for the first layer of the neural network W^1, b^1, for the second layer W^2, b^2, and so on, W^3, b^3, W^4, b^4, and W^5, b^5 for the output layer. To apply transfer learning, what you do is then make a copy of this neural network where you would keep the parameters W^1, b^1, W^2, b^2, W^3, b^3, and W^4, b^4. But for the last layer, you would eliminate the output layer and replace it with a much smaller output layer with just 10 rather than 1,000 output units. These 10 output units will correspond to the classes zero, one, through nine that you want your neural network to recognize.

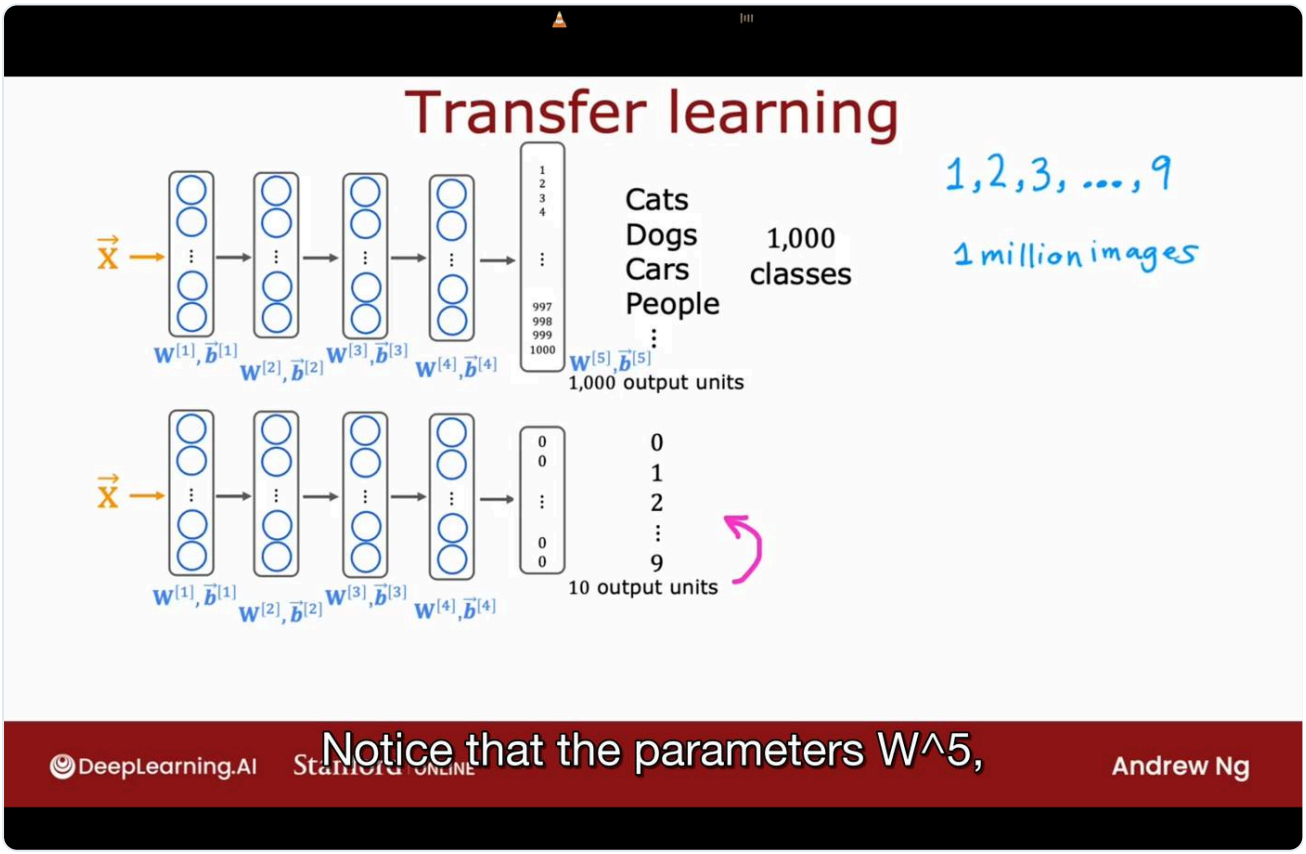


FIGURE 45.1 | Transfer Learning

Notice that the parameters W^5, b^5 they can't be copied over because the dimension of this layer has changed, so you need to come up with new parameters W^5, b^5 that you need to train from scratch rather than just copy it from the previous neural network. In transfer learning, what you can do is use the parameters from the first four layers, really all the layers except the final output layer as a starting point for the parameters and then run an optimization algorithm such as gradient descent or the Adam optimization algorithm with the parameters initialized using the values from this neural network up on top.

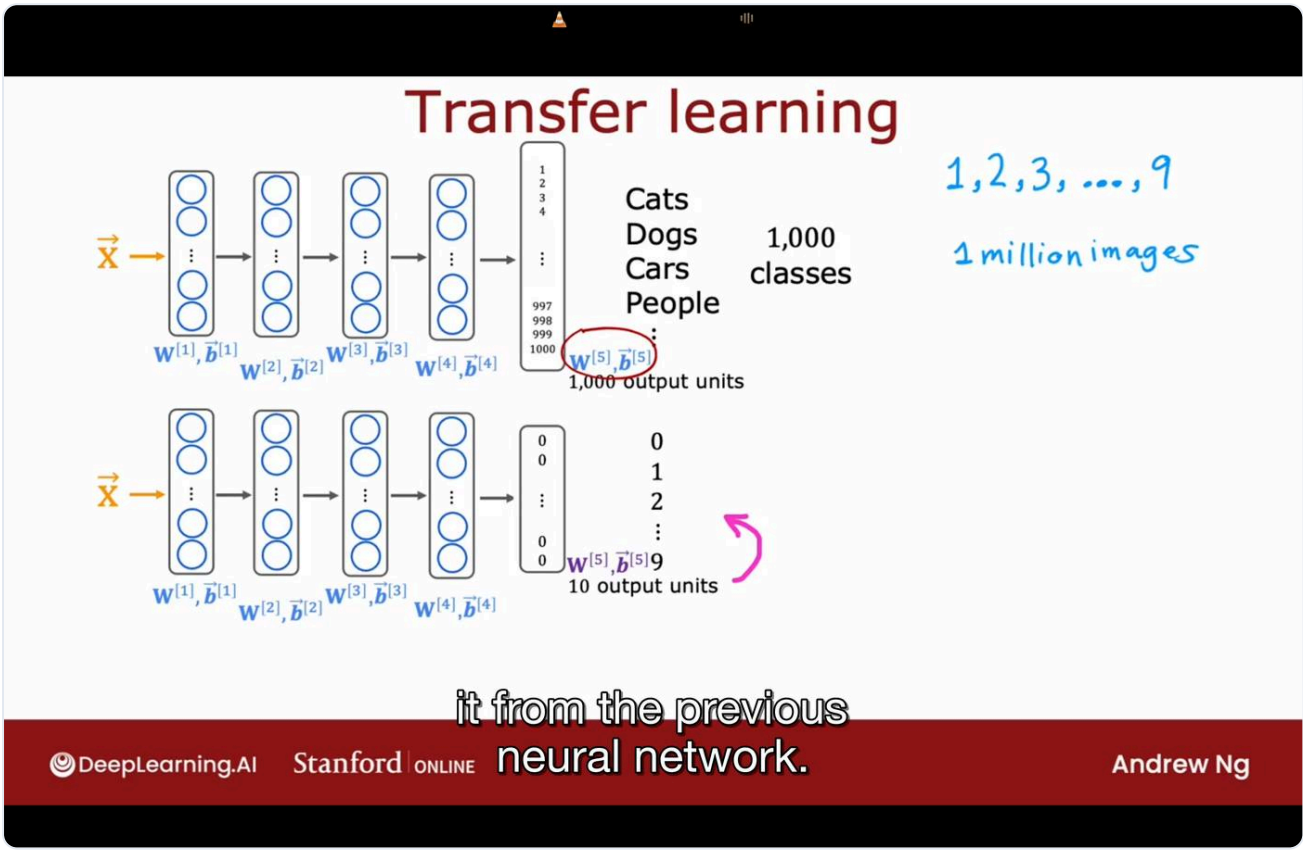


FIGURE 45.2 | Transfer Learning

In detail, there are two options for how you can train this neural networks parameters.

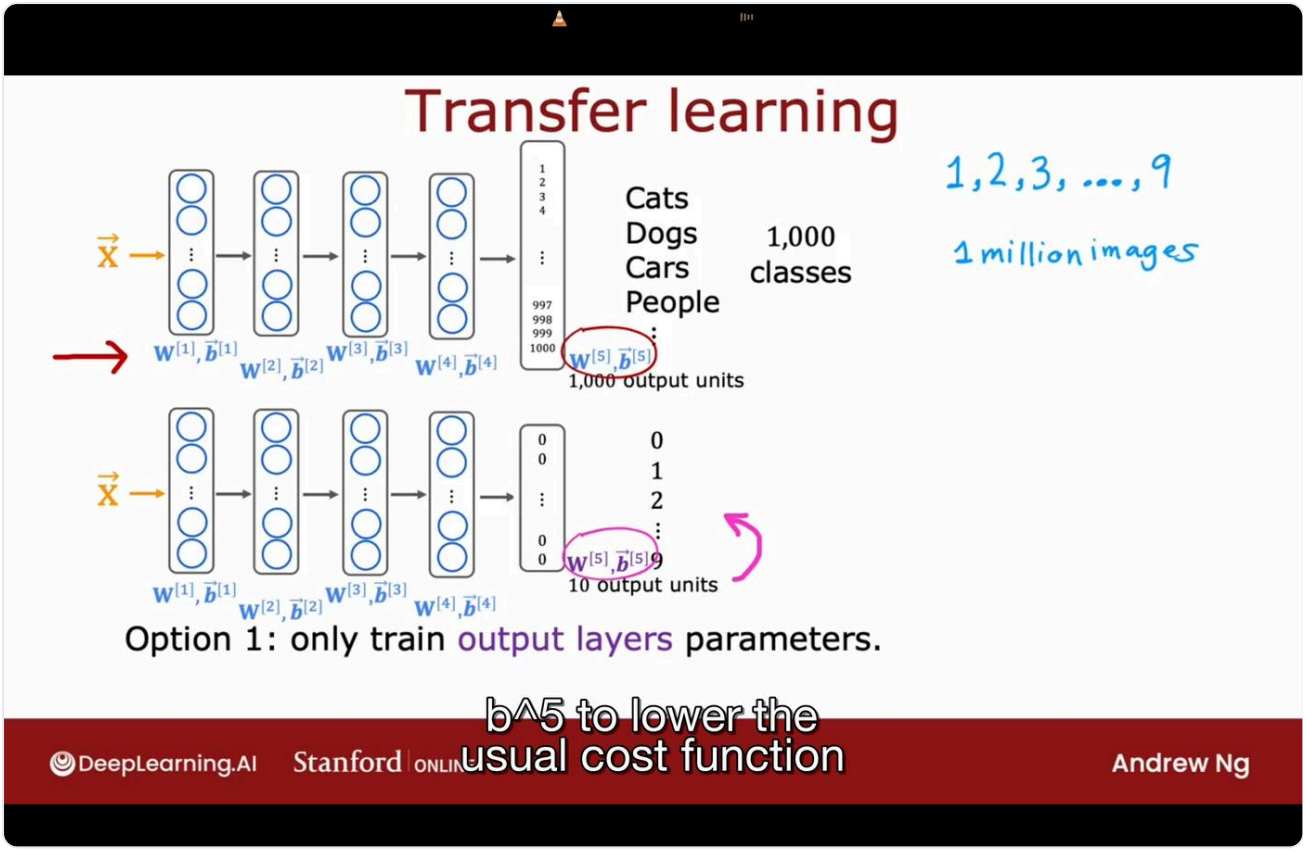


FIGURE 45.3 | Transfer Learning

Option 1 is you only train the output layers parameters. You would take the parameters W^1, b^1, W^2, b^2 through W^4, b^4 as the values from on top and just hold them fix and don't even bother to change them, and use an algorithm like Stochastic gradient descent or the Adam optimization algorithm to only update W^5, b^5 to lower the usual cost function that you use for learning to recognize these digits zero to nine from a small training set of these digits zero to nine, so this is Option 1.

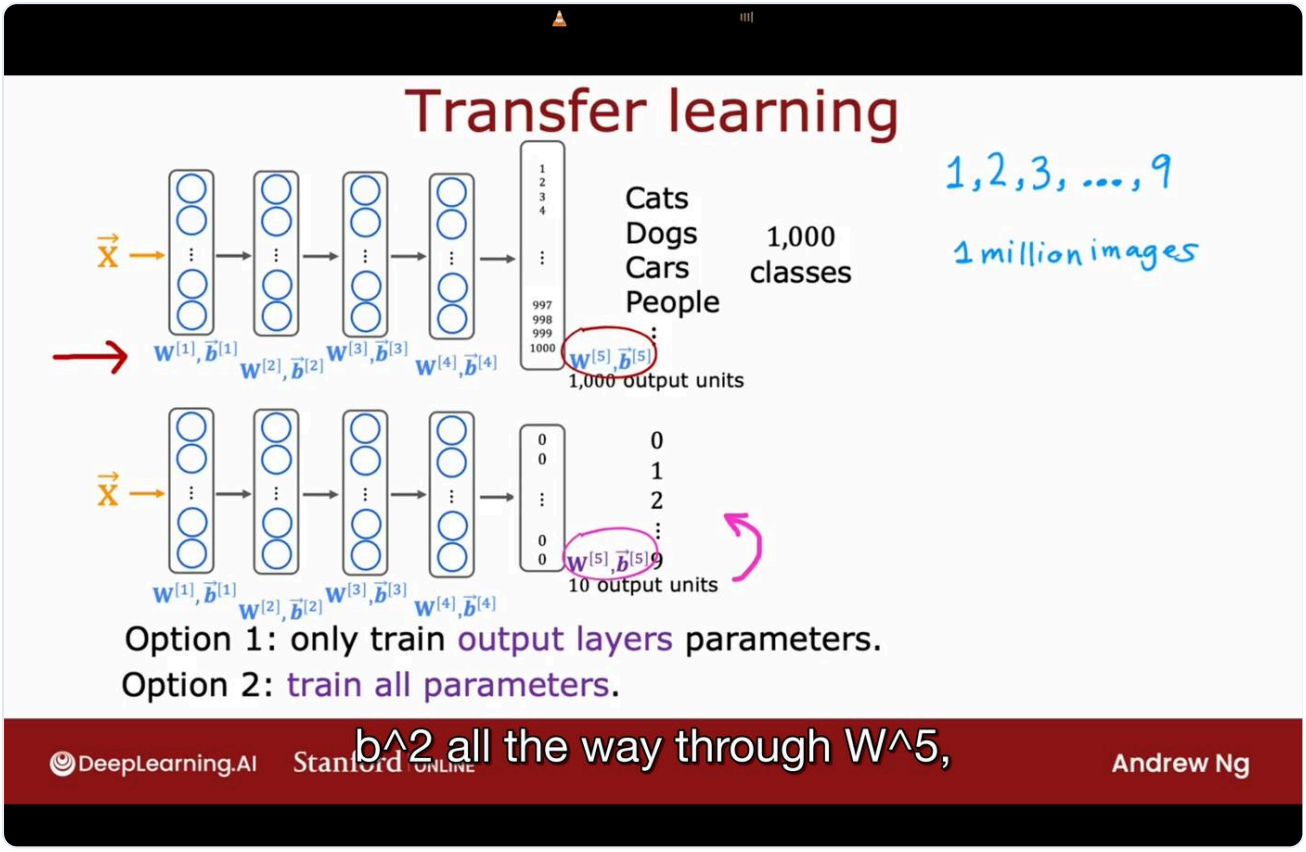


FIGURE 45.4 | Transfer Learning

Option 2 would be to train all the parameters in the network including W^1, b^1, W^2, b^2 all the way through W^5, b^5 but the first four layers parameters would be initialized using the values that you had trained on top. If you have a very small training set then Option 1 might work a little bit better, but if you have a training set that's a little bit larger then Option 2 might work a little bit better. This algorithm is called transfer learning because the intuition is by learning to recognize cats, dogs, cows, people, and so on. It will hopefully, have learned some plausible sets of parameters for the earlier layers for processing image inputs.

Then by transferring these parameters to the new neural network, the new neural network starts off with the parameters in a much better place so that we have just a little bit of further learning. Hopefully, it can end up at a pretty good model. These two steps of first training on a large dataset and then tuning the parameters further on a smaller dataset go by the name of supervised pretraining for this step on top. That's when you train the neural network on a very large dataset of say a million images of not quite the related task.

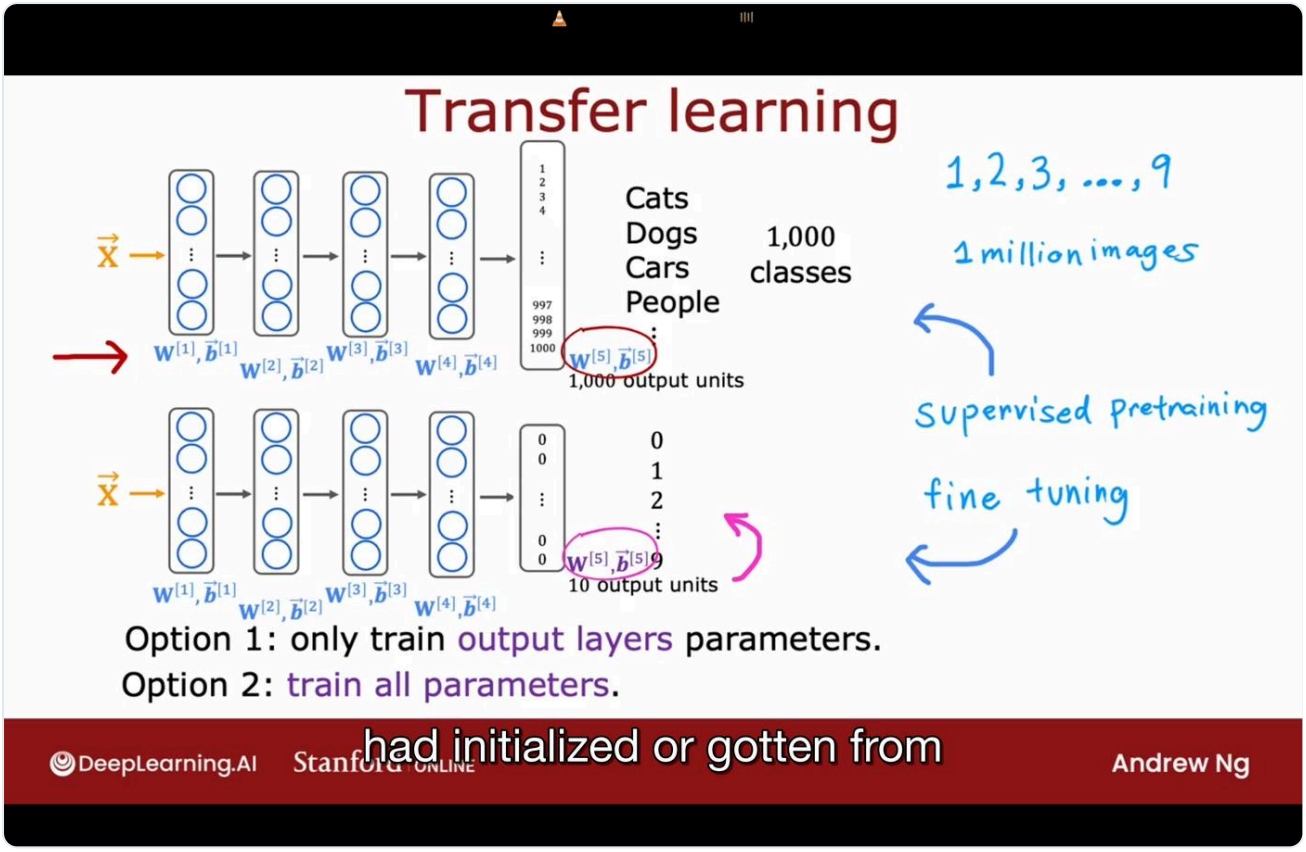


FIGURE 45.5 | Transfer Learning

Then the second step is called fine tuning where you take the parameters that you had initialized or gotten from supervised pre-training and then run gradient descent further to fine tune the weights to suit the specific application of handwritten digit recognition that you may have. If you have a small dataset, even tens or hundreds or thousands or just tens of thousands of images of the handwritten digits, being able to learn from these million images of a not quite related task can actually help your learning algorithm's performance a lot. One nice thing about transfer learning as well is maybe you don't need to be the one to carry out supervised pre-training.

For a lot of neural networks, there will already be researchers they have already trained a neural network on a large image and will have posted a trained neural networks on the Internet, freely licensed for anyone to download and use. What that means is rather than carrying out the first step yourself, you can just download the neural network that someone else may have spent weeks training and then replace the output layer with your own output layer and carry out either Option 1 or Option 2 to fine tune a neural network that someone else has already carried out supervised pre-training on, and just do a little bit of fine tuning to quickly be able to get a neural network that performs well on your task. Downloading a pre-trained model that someone else has trained and provided for free is one of those techniques where by building on each other's work on machine learning community we can all get much better results.

By the generosity of other researchers that have pre-trained and posted their neural networks online. But why does transfer learning even work? How can you possibly take parameters obtained by recognizing cats, dogs, cars, and people and use that to help you recognize something as different as handwritten digits? Here's some intuition behind it. If you are training a neural network to detect, say, different objects from images, then the first layer of a neural network may

learn to detect edges in the image. We think of these as somewhat low-level features in the image which is to detect edges. Each of these squares is a visualization of what a single neuron has learned to detect as learn to group together pixels to find edges in an image.

The next layer of the neural network then learns to group together edges to detect corners. Each of these is a visualization of what one neuron may have learned to detect, must learn to technical, simple shapes like corner like shapes like this. The next layer of the neural network may have learned to detect some are more complex, but still generic shapes like basic curves or smaller shapes like these. That's why by learning on detecting lots of different images, you're teaching the neural network to detect edges, corners, and basic shapes.

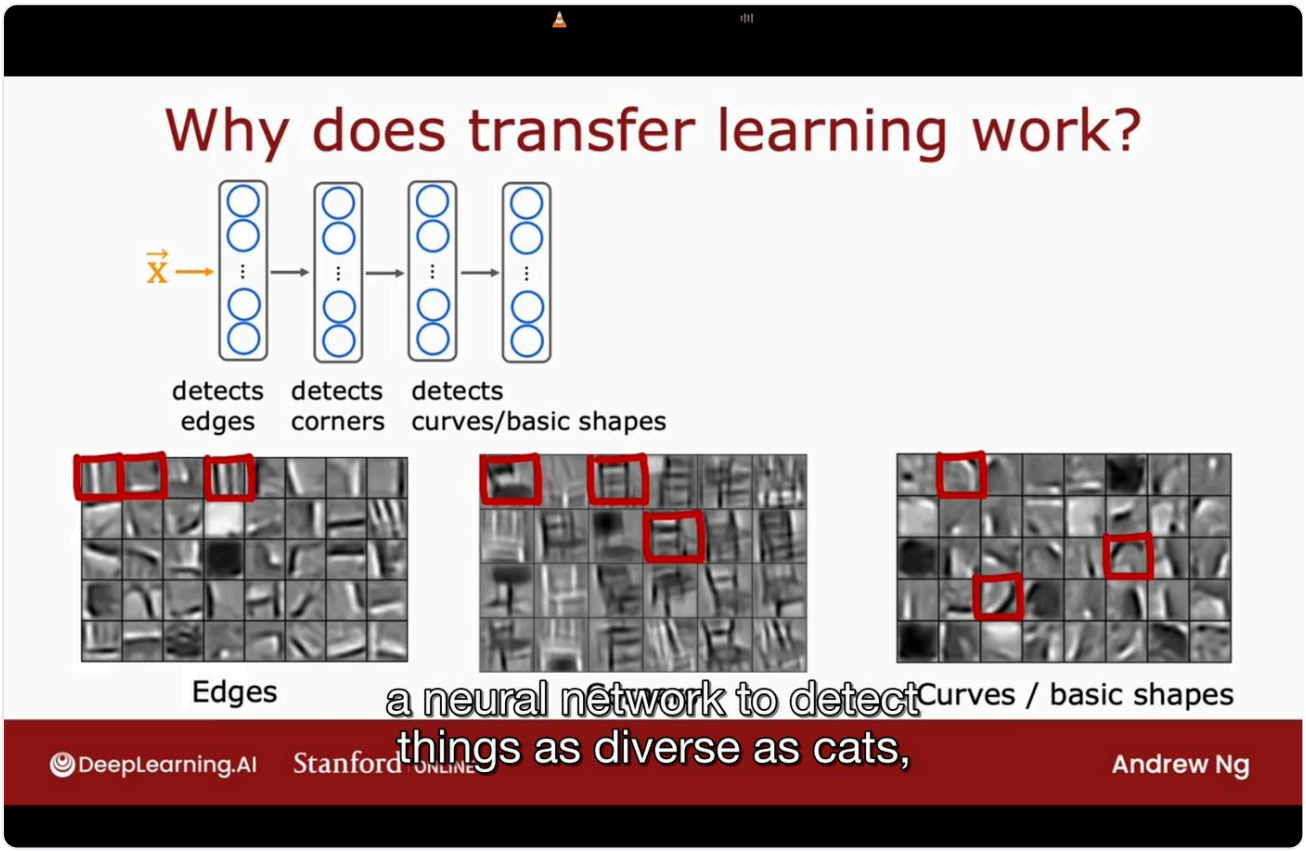


FIGURE 45.6 | Why Does Transfer Learning Work?

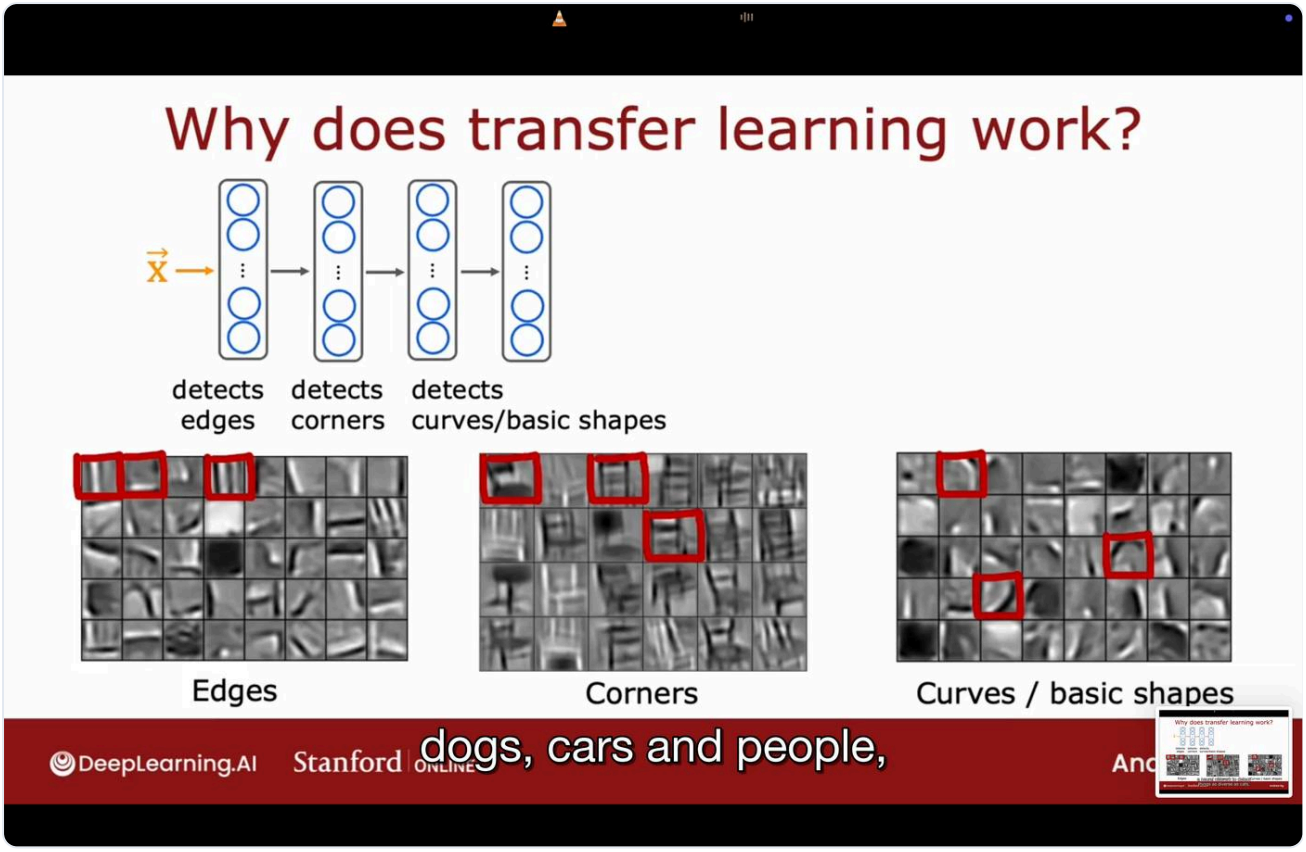


FIGURE 45.7 | Why Does Transfer Learning Work?

That's why by training a neural network to detect things as diverse as cats, dogs, cars and people, you're helping it to learn to detect these pretty generic features of images and finding edges, corners, curves, basic shapes. This is useful for many other computer vision tasks, such as recognizing handwritten digits. One restriction of pre-training though, is that the image type x has to be the same for the pre-training and fine-tuning steps. If the final task you want to solve is a computer vision tasks, then the pre-training step also has been a neural network trained on the same type of input, namely an image of the desired dimensions. Conversely, if your goal is to build a speech recognition system to process audio, then a neural network pre-trained on images probably won't do much good on audio.

Instead, you want a neural network pre-trained on audio data, there you then fine tune on your own audio dataset and the same for other types of applications. You can pre-train a neural network on text data and If your application has a save feature input x of text data, then you can fine tune that neural network on your own data. To summarize, these are the two steps for transfer learning. Step 1 is download neural network with parameters that have been pre-trained on a large dataset with the same input type as your application. That input type could be images, audio, texts, or something else, or if you don't want to download the neural network, maybe you can train your own.

But in practice, if you're using images, say, is much more common to download someone else's pre-trained neural network. Then further train or fine tune the network on your own data.

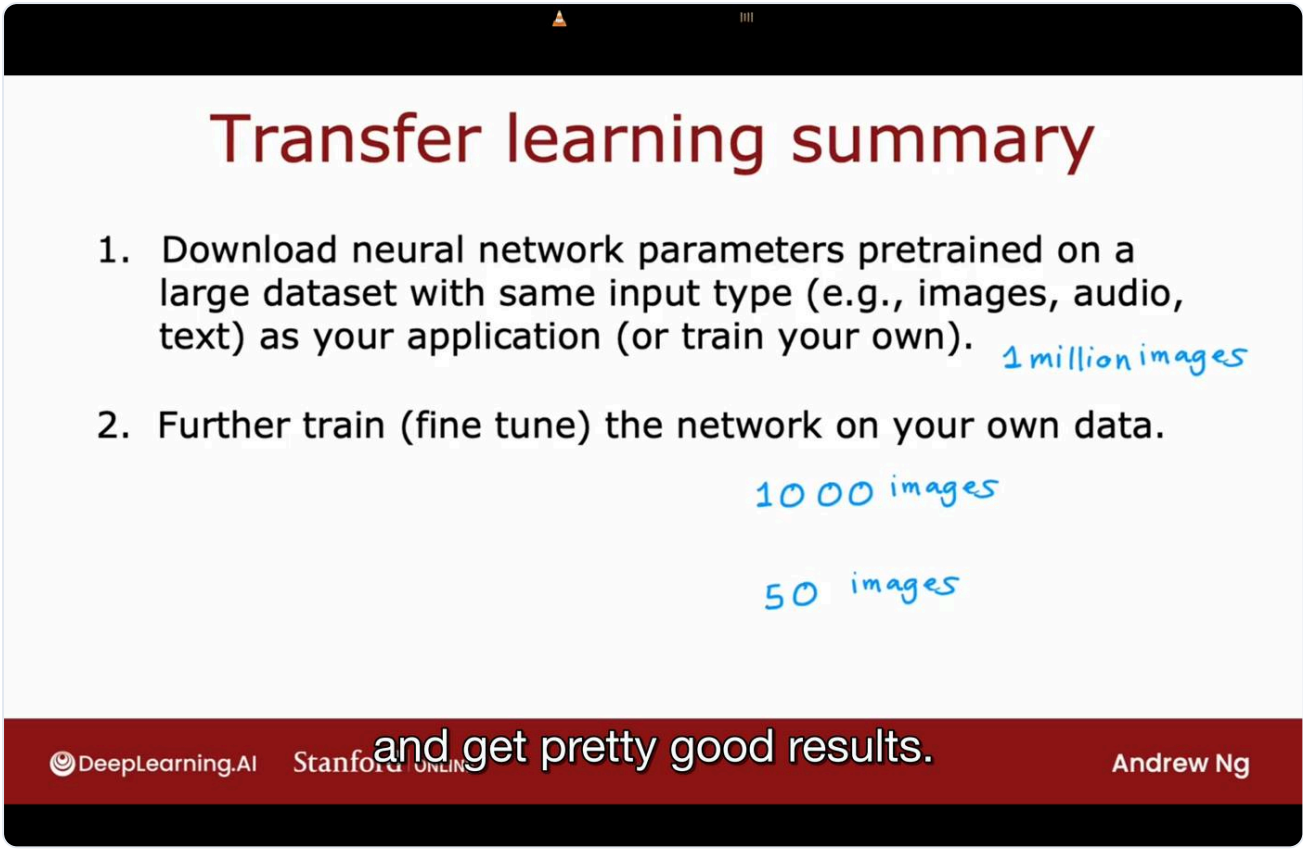


FIGURE 45.8 | 1. Download Neural Network Parameters Pretrained on a

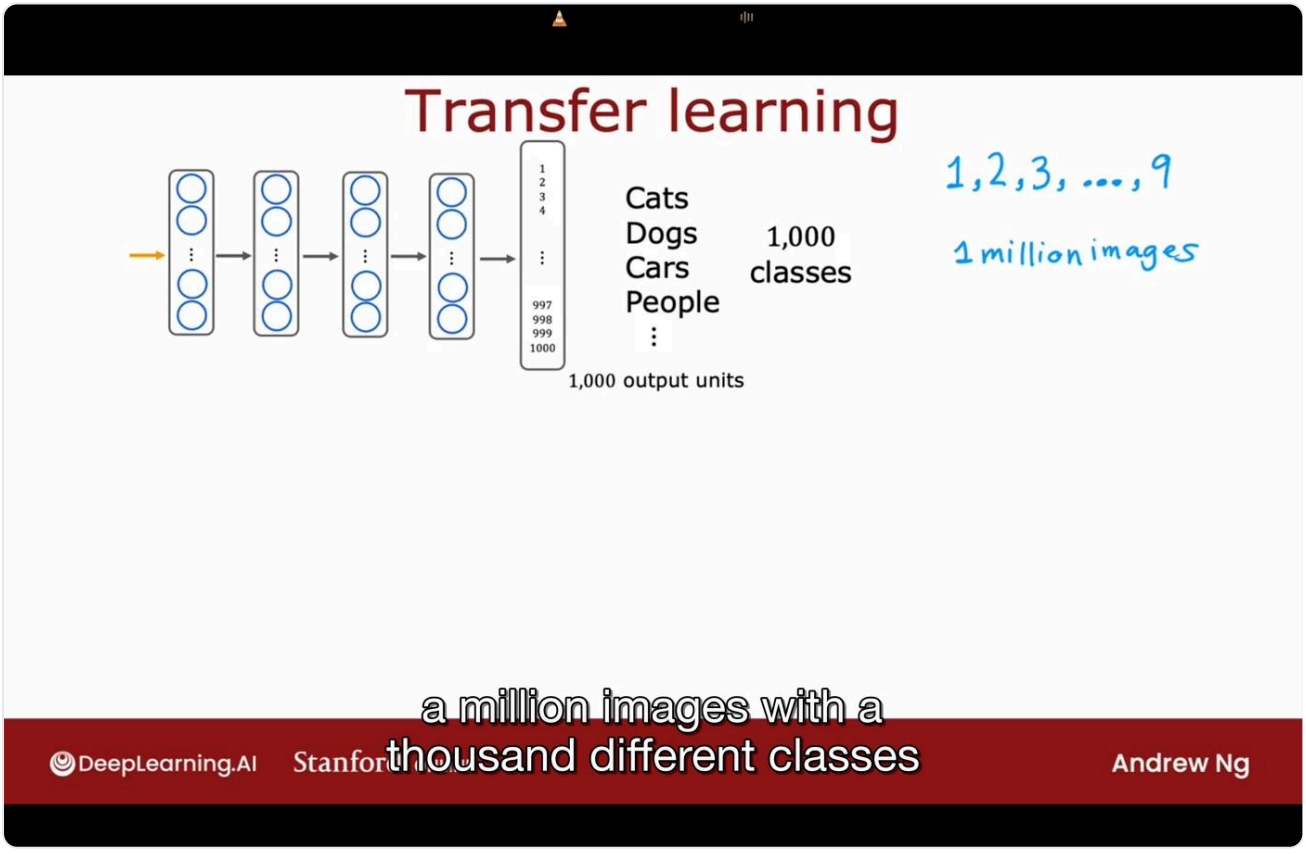


FIGURE 45.9 | Transfer Learning

I found that if you can get a neural network pre-trained on large dataset, say a million images, then sometimes you can use a much smaller dataset, maybe a thousand images, maybe even smaller, to fine tune the neural network on your own data and get pretty good results. I'd sometimes train neural networks on as few as 50 images that were quite well using this technique, when it has already been pre-trained on a much larger dataset. This technique isn't panacea., you can't get every application to work just on 50 images, but it does help a lot when the dataset you have for your application isn't that large. By the way, if you've heard of advanced techniques in the news like GPT-3 or BERTs or neural networks pre-trained on ImageNet, those are actually examples of neural networks that they have someone else's pre-trained on a very large image datasets or text dataset, they can then be fine tuned on other applications.

If you haven't heard of GPT-3, or BERTs, or ImageNet, don't worry about it, whether you have. Those have been successful applications of pre-training in the machine learning literature. One of the things I like about transfer learning is just that one of the ways that the machine learning community has shared ideas, and code, and even parameters, with each other because thanks to the researchers that have pre-trained large neural networks and posted the parameters on the internet freely for anyone else to download and use. This empowers anyone to take models, their pre-trained, to fine tune on potentially much smaller dataset. In machine learning, all of us end up often building on the work of each other and that open sharing of ideas, of codes, of trained parameters is one of the ways that the machine learning community, all of us collectively manage to do much better work than any single person by themselves can.

I hope that you joining the machine learning community will someday maybe find a way to contribute back to this community as well. That's it for pre-training. I hope you find this technique useful. In the next video, I'd like to share with you some thoughts on the full cycle of a machine learning project. When building a machine learning system, whether all the steps that are worth thinking about. Let's take a look at that in the next video.

LECTURE 46

Full Cycle of a Machine Learning Project

So far we've talked a lot about how to train a model and also talked a bit about how to get data for your machine learning application. But when I'm building a machine learning system I find that training a model is just part of the puzzle. In this video I'd like to share with you what I think of as the full cycle of a machine learning project. That is, when you're building a valuable machine learning system, what are the steps to think about and plan for? Let's take a look, let me use speech recognition as an example to illustrate the full cycle of a machine learning project. The first step of machine learning project is to scope the project. In other words, decide what is the project and what you want to work on.

For example, I once decided to work on speech recognition for voice search. That is to do web search using speaking to your mobile phone rather than typing into your mobile phone. This project scoping. After deciding what to work on you have to collect data. Decide what data you need to train your machine learning system and go and do the work to get the audio and get the transcripts of the labels for your dataset. That's data collection. After you have your initial data collection you can then start to train the model. Here you would train a speech recognition system and carry out error analysis and iteratively improve your model. Is not at all uncommon.

After you've started training the model for error analysis or for a bias-variance analysis to tell you that you might want to go back to collect more data. Maybe collect more data of everything or just collect more data of a specific type where your error analysis tells you you want to improve the performance of your learning algorithm. For example, once when working on speech I realized that my model was doing particularly poorly when there was car noise in the background. That sounded like someone was speaking in a car. My speech system perform poorly decided to get more data, actually using data augmentation to get more speech data that sounds like it was a car in order to improve the performance of my learning algorithm.

You go around this loop a few times, train the model, error analysis, go back to collect more data, maybe do this for a while until eventually you say the model is good enough to then deploy in a production environment. What that means is you make it available for users to use.

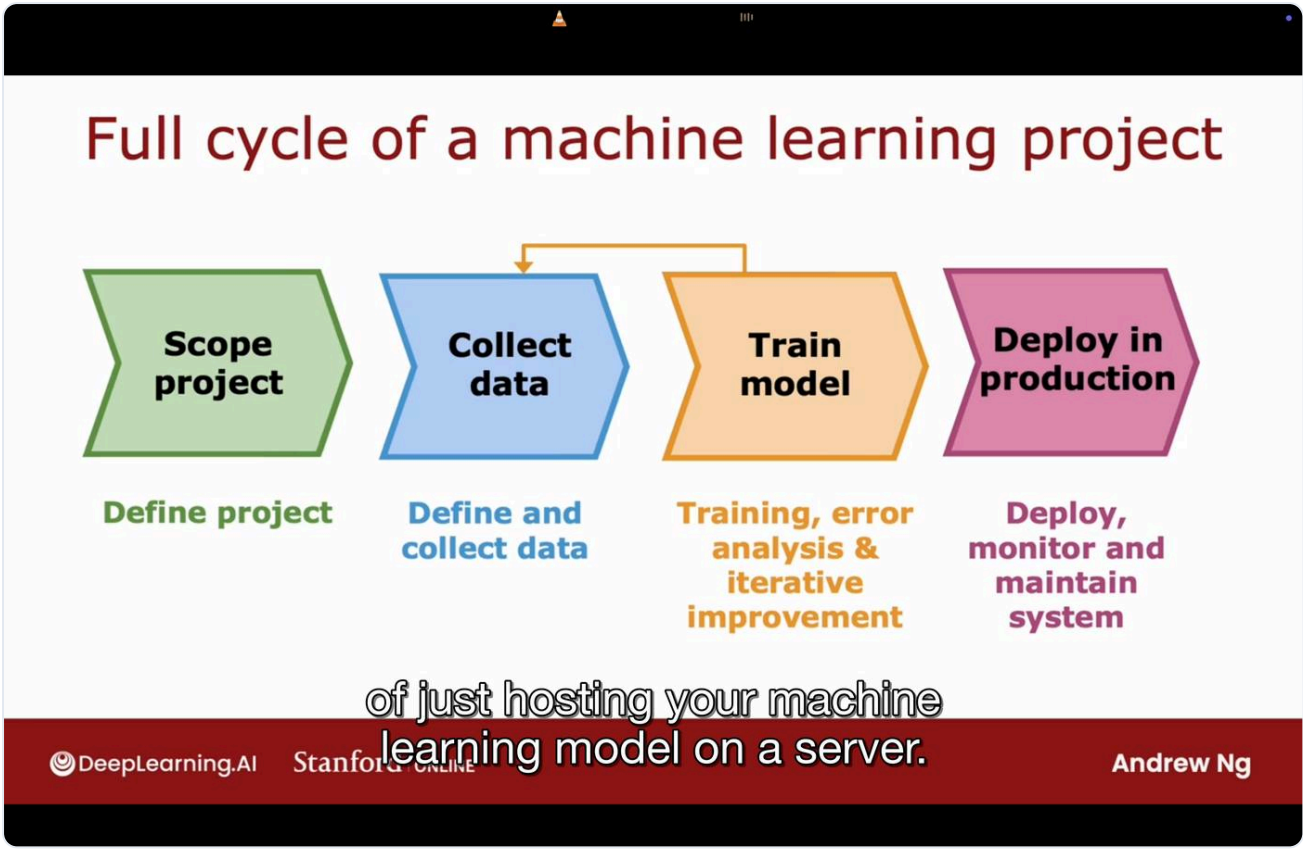


FIGURE 46.1 | Full Cycle of a Machine Learning Project

When you deploy a system you have to also make sure that you continue to monitor the performance of the system and to maintain the system in case the performance gets worse to bring us performance back up instead of just hosting your machine learning model on a server. I'll say a little bit more about why you need to maintain these machine learning systems on the next slide.

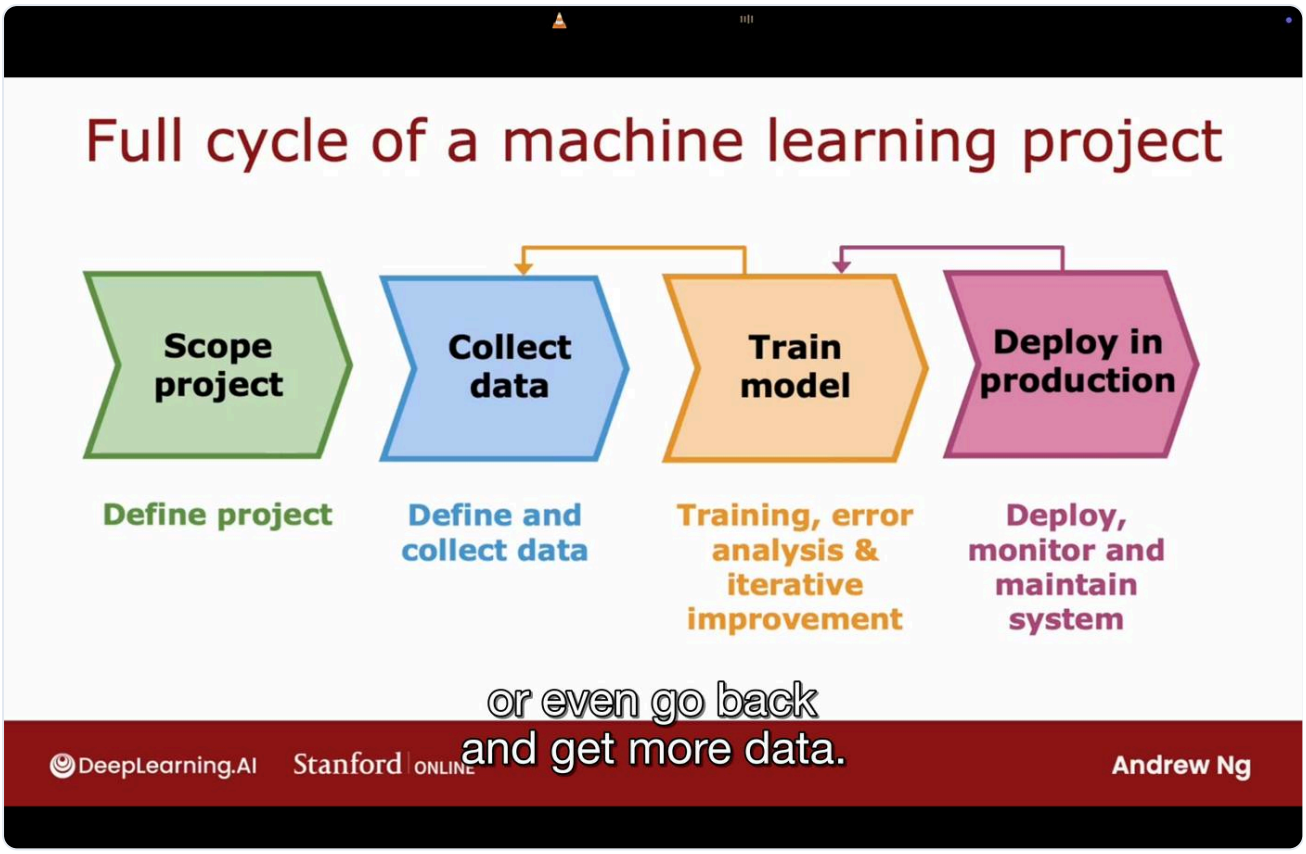


FIGURE 46.2 | Full Cycle of a Machine Learning Project

But after this deployment, sometimes you realize that is not working as well as you hoped and you go back to train the model to improve it again or even go back and get more data.

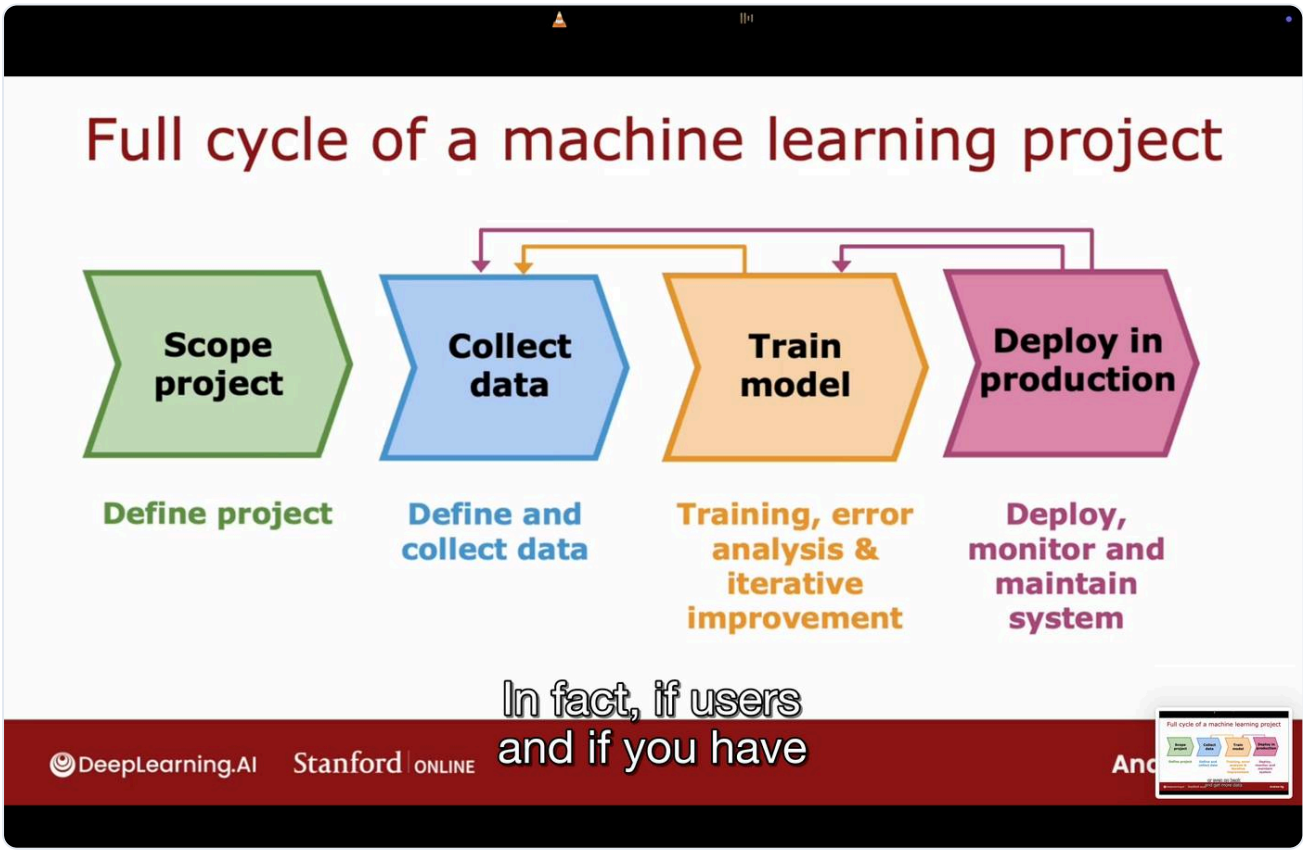


FIGURE 46.3 | Full Cycle of a Machine Learning Project

In fact, if users and if you have permission to use data from your production deployment, sometimes that data from your working speech system can give you access to even more data with which to keep on improving the performance of your system. Now, I think you have a sense of what scoping a project means and we've talked a bunch about collecting data and training models in this course. But let me share with you a little bit more detail about what deploying in production might look like. After you've trained a high performing machine learning model, say a speech recognition model, a common way to deploy the model would be to take your machine learning model and implement it in a server, which I'm going to call an inference server, whose job it is to call your machine learning model, your trained model, in order to make predictions.

Then if your team has implemented a mobile app, say a search application, then when a user talks to the mobile app, the mobile app can then make an API call to pass to your inference server the audio clip that was recorded and the inference server's job is supply the machine learning model to it and then return to it the prediction of your model, which in this case would be the text transcripts of what was said.

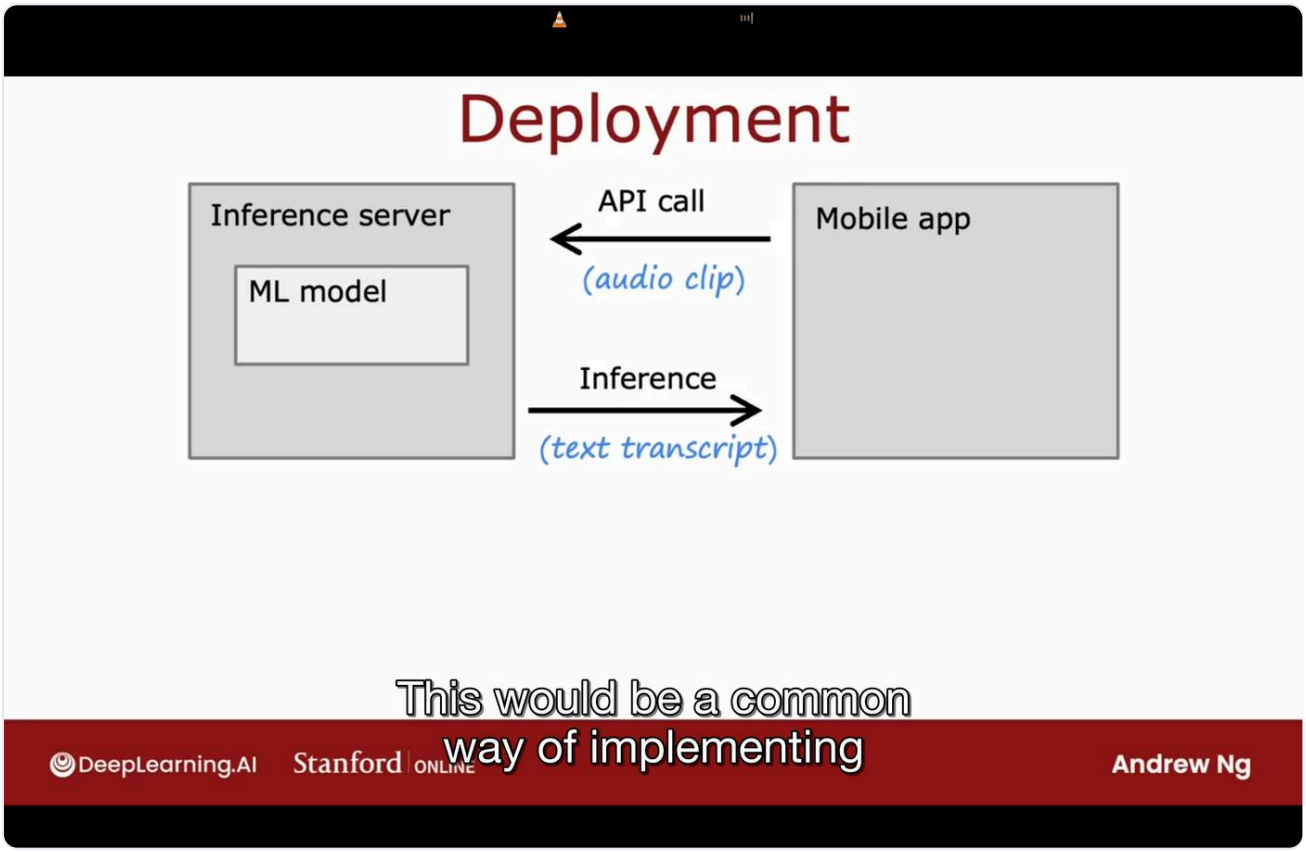


FIGURE 46.4

This would be a common way of implementing an application that calls via the API and inference server that has your model repeatedly make predictions based on the input, x. This were common pattern where depend on the application does implemented. You have an API call to give your learning algorithm the input, x, and your machine learning model within output to prediction, say y hat. To implement this some software engineering may be needed to write all the code that does all of these things. Depending on whether your application needs to serve just a few handful of users or millions of users the amounts of software engineer needed can be quite different.

I've build software that serve just a handful of users on my laptop and I've also built software that serves hundreds of millions of users requiring significant data center resources. Depending on scale application needed, software engineering may be needed to make sure that your inference server is able to make reliable and efficient predictions hopefully not too high of computational cost. Software engineering may be needed to manage scaling to a large number of users. You often want to log the data you're getting both the inputs, x, as well as the predictions, y hat, assuming that user privacy and consent allows you to store this data.

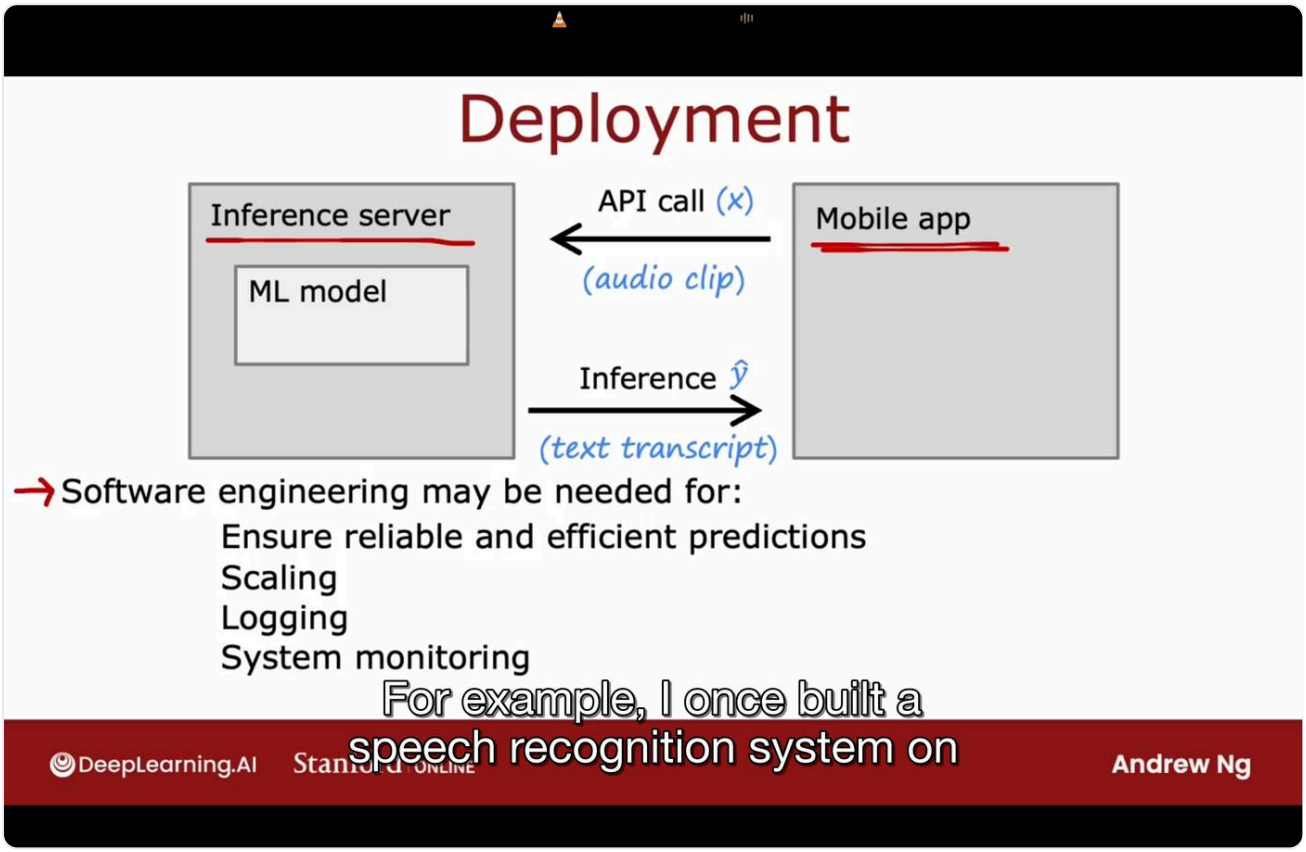


FIGURE 46.5

This data, if you can access to it, is also very useful for system monitoring. For example, I once built a speech recognition system on a certain dataset that I had but when there were new celebrities that suddenly became well-known or elections cause new politicians to become elected and people will search for these new names that were not in the training set and then my system did poorly on. It was because we were monitoring the system allowed us to figure out when the data was shifting and the algorithm was becoming less accurate. This allowed us to re-train the model and then to carry out a model update to replace the old model with a new one.

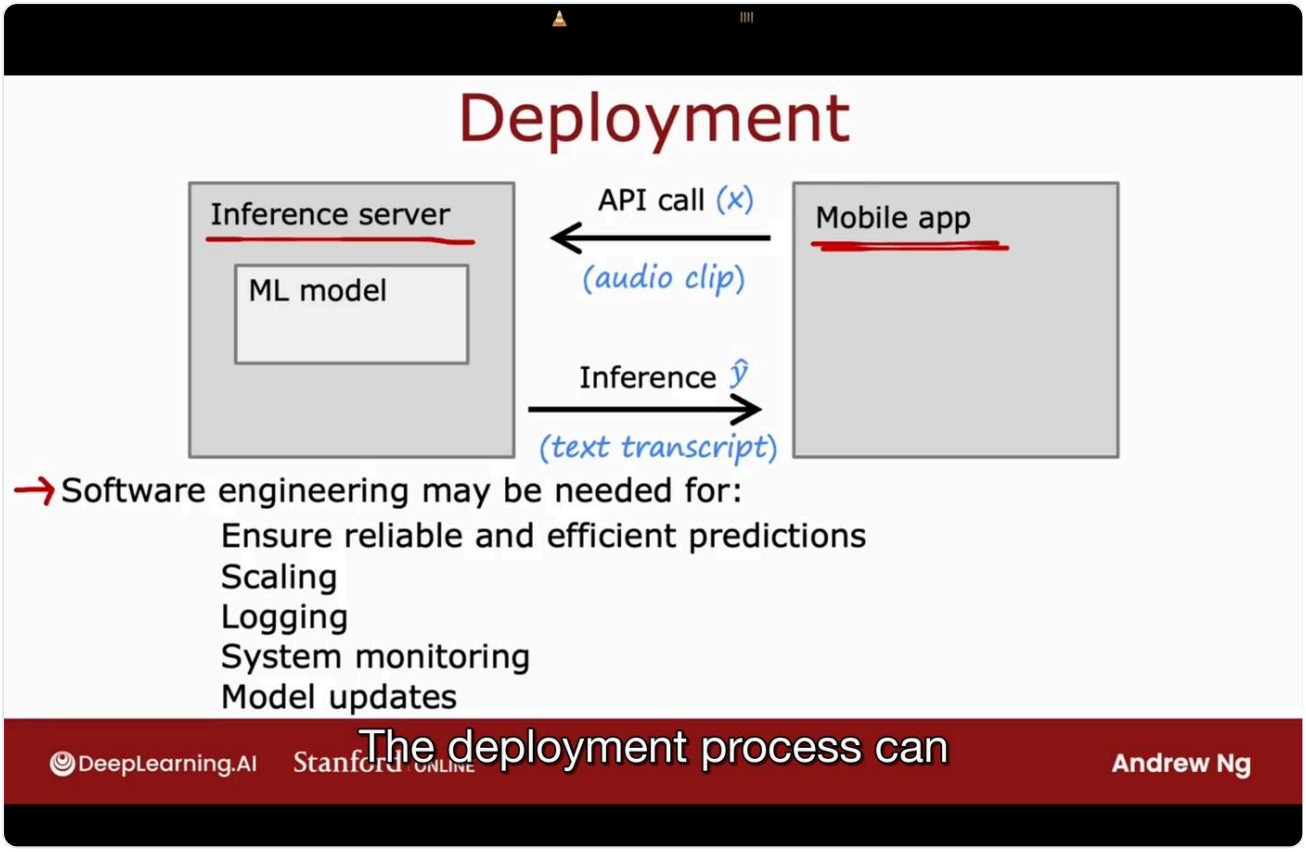


FIGURE 46.6 | Inference Server API Call (X) Mobile App

The deployment process can require some amounts of software engineering. For some applications, if you're just running it on a laptop or on a one or two servers, maybe not that much software engineering is needed. Depending on the team you're working on, it is possible that you built the machine learning model but there could be a different team responsible for deploying it. But there is a growing field in machine learning called MLOps. This stands for Machine Learning Operations.

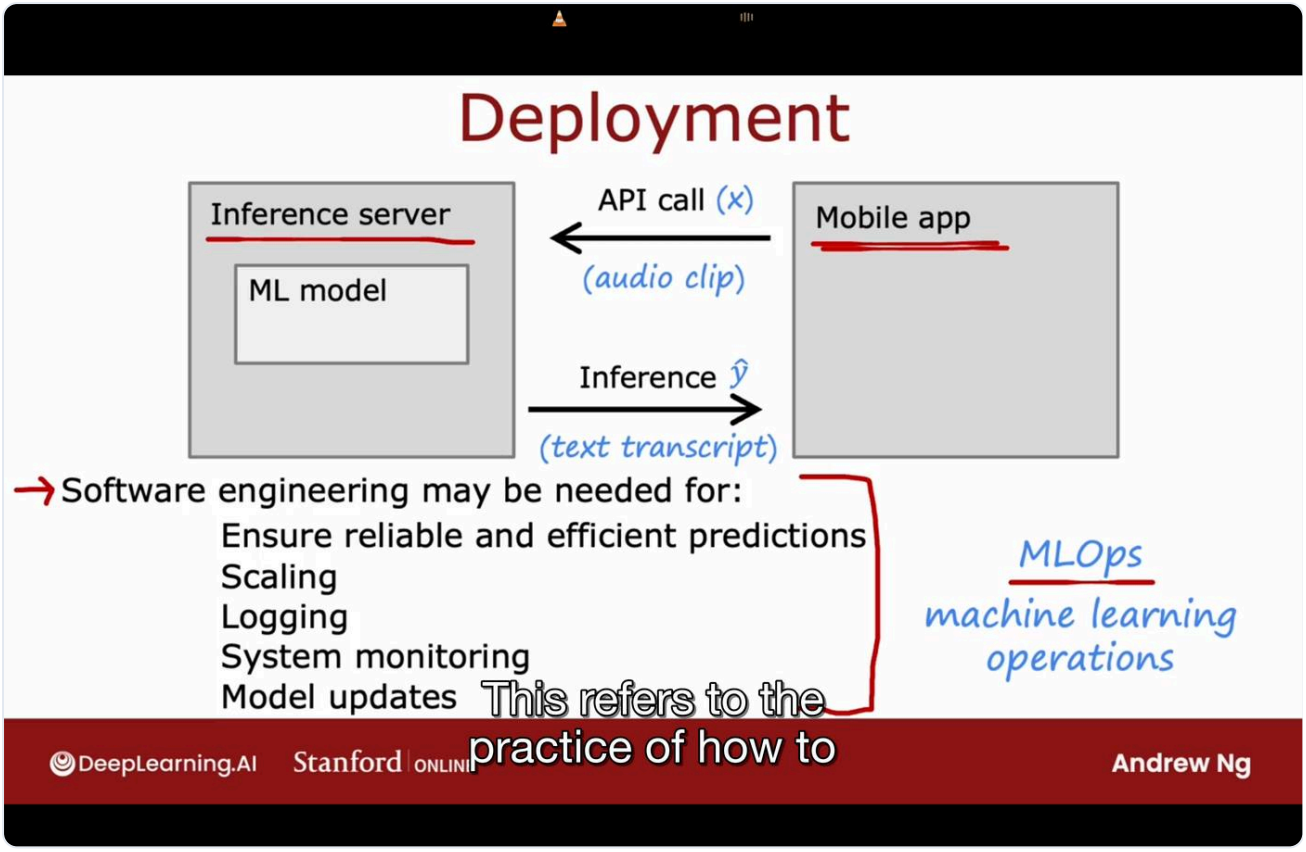


FIGURE 46.7 | API Call (X) Mobile App

This refers to the practice of how to systematically build and deploy and maintain machine learning systems. To do all of these things to make sure that your machine learning model is reliable, scales well, has good laws, is monitored, and then you have the opportunity to make updates to the model as appropriate to keep it running well. For example, if you are deploying your system to millions of people you may want to make sure you have a highly optimized implementations so that the compute cost of serving millions of people is not too expensive. In this and the last class I spent a lot of time talking about how to train a machine learning model and got this absolutely the critical piece to making sure you have a high performance system.

If you ever have to deploy system to millions of people, these are some additional steps that you probably have to address. Think about the [inaudible] at that point as well. Before moving on from the topic of the machine learning development process, there's one more set of ideas that I want to share with you that relates to the ethics of building machine learning systems. This is a crucial topic for many applications so let's take a look at this in the next video.

LECTURE 47

Adverse Use Cases

Machine learning algorithms today are affecting billions of people. You've heard me mention ethics in other videos before. And I hope that if you're building a machine learning system that affects people that you give some thought to making sure that your system is reasonably fair, reasonably free from bias. And that you're taking a ethical approach to your application. Let's take a look at some issues related to fairness, bias and ethics. Unfortunately in the history of machine learning that happened a few systems, some widely publicized, that turned out to exhibit a completely unacceptable level of bias. For example, there was a hiring tool that was once shown to discriminate against women.

The company that built the system stopped using it, but one wishes that the system had never been rolled out in the first place. Or there was also well documented example of face recognition systems that match dark skinned individuals to criminal mug shots much more often than lighter skinned individuals. And clearly this is not acceptable and we should get better as a community at just not building and deploying systems with a problem like this. In the first place, there happens systems that gave bank loan approvals in a way that was biased and discriminated against subgroups.

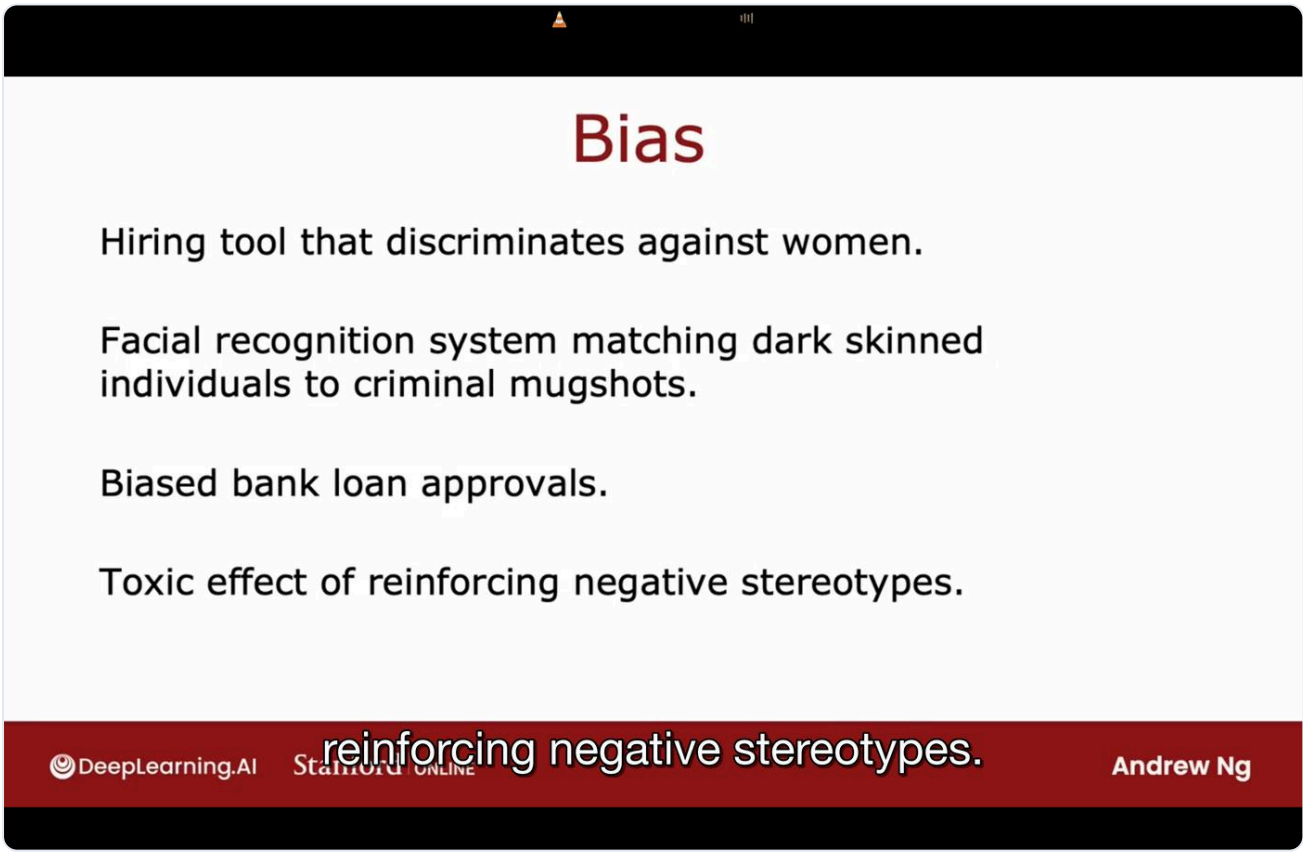


FIGURE 47.1 | Hiring Tool That Discriminates Against Women

And we also really like learning algorithms to not have the toxic effect of reinforcing negative stereotypes. For example, I have a daughter and if she searches online for certain professions and doesn't see anyone that looks like her, I would hate for that to discourage her from taking on certain professions. In addition to the issues of bias and fair treatment of individuals, there have also been adverse use cases or negative use cases of machine learning algorithms. For example, there was this widely cited and widely viewed video release with full disclosure and full transparency. By the company buzzfeed of a deepfake of former US President Barack Obama and you can actually find and watch the whole video online if you want.



FIGURE 47.2 | Adverse Use Cases

But the company that created this video did so full transparency and full disclosure. But clearly using this technology to generate fake videos without consent and without disclosure would be unethical.

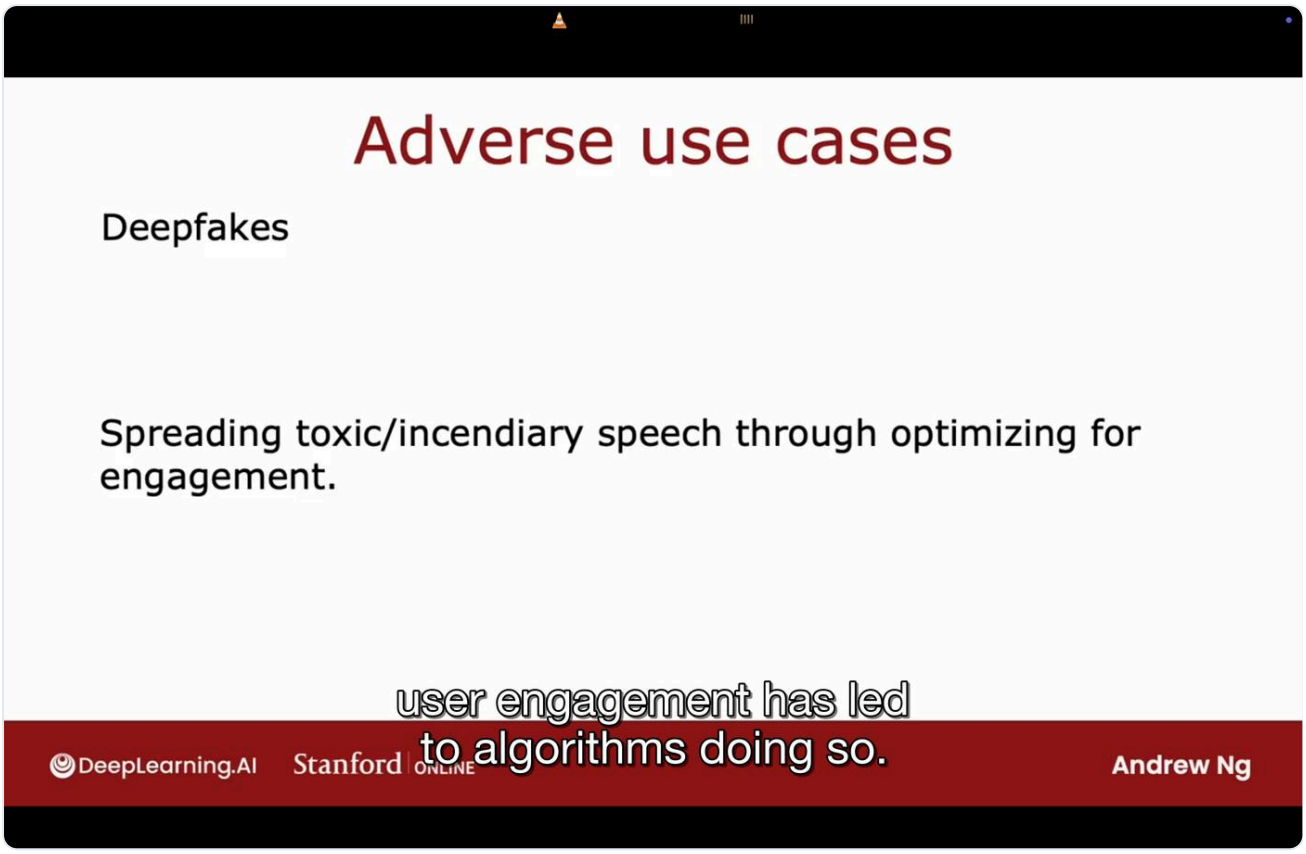


FIGURE 47.3 | Adverse Use Cases

We've also seen unfortunately social media sometimes spreading toxic or incendiary speech because optimizing for user engagement has led to algorithms doing so.

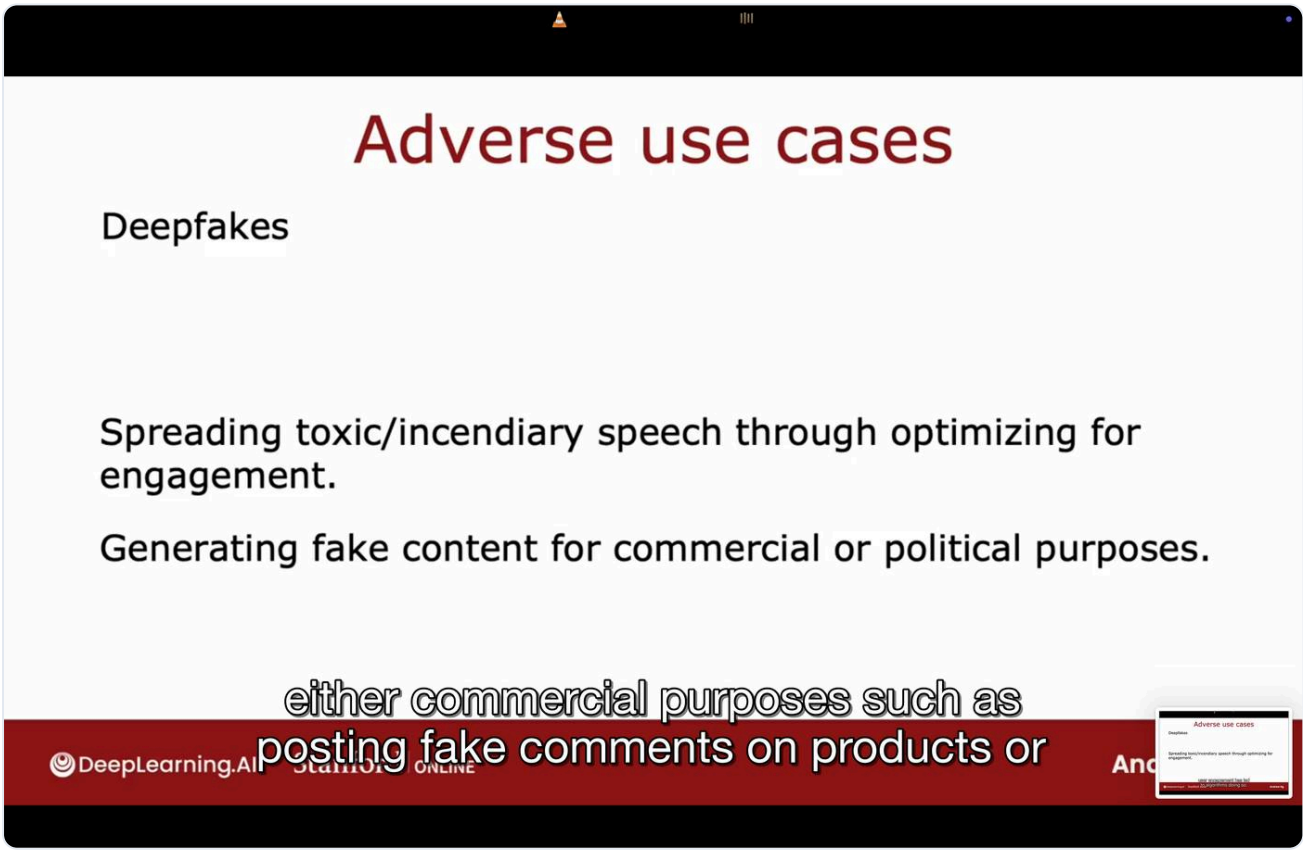


FIGURE 47.4 | Adverse Use Cases

There have been bots that were used to generate fake content for either commercial purposes such as posting fake comments on products or for political purposes.

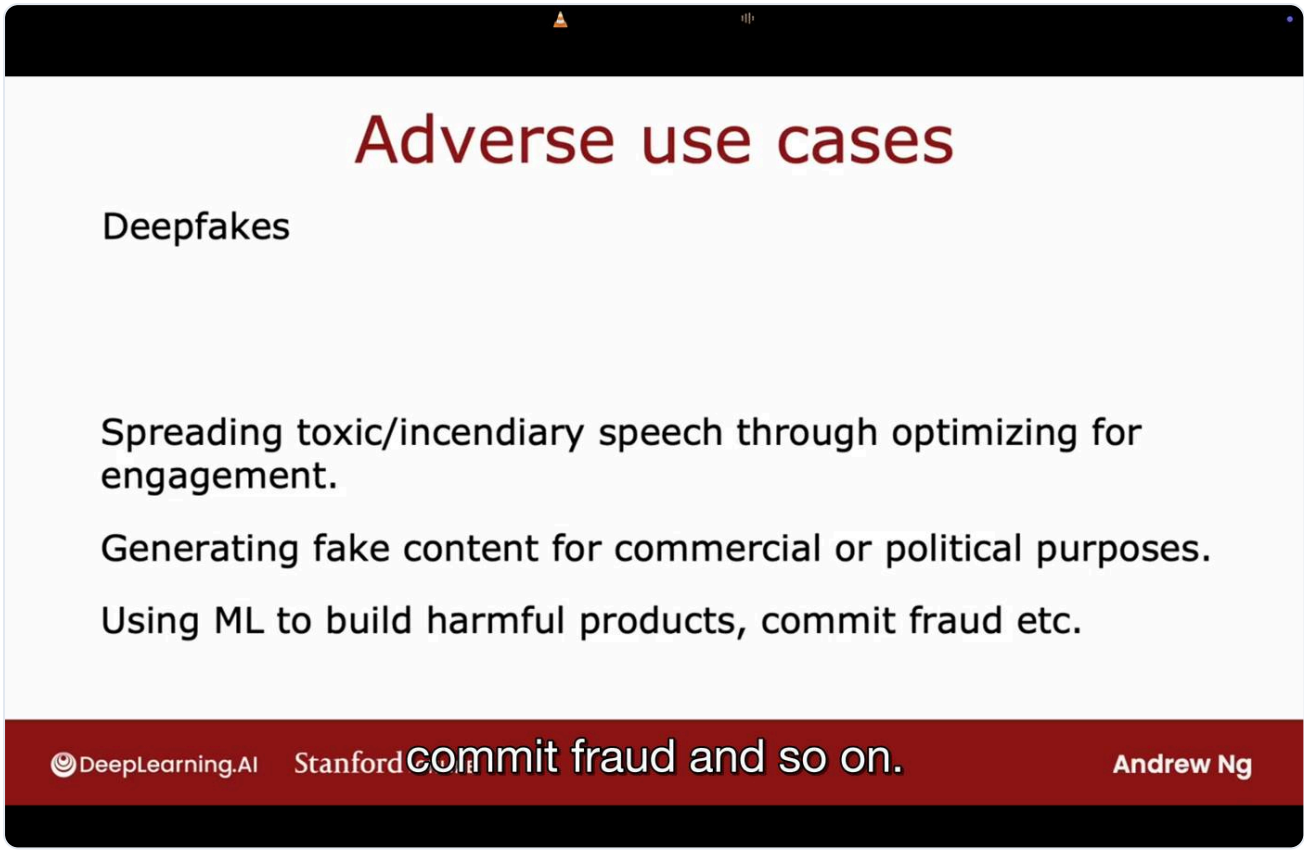


FIGURE 47.5 | Adverse Use Cases

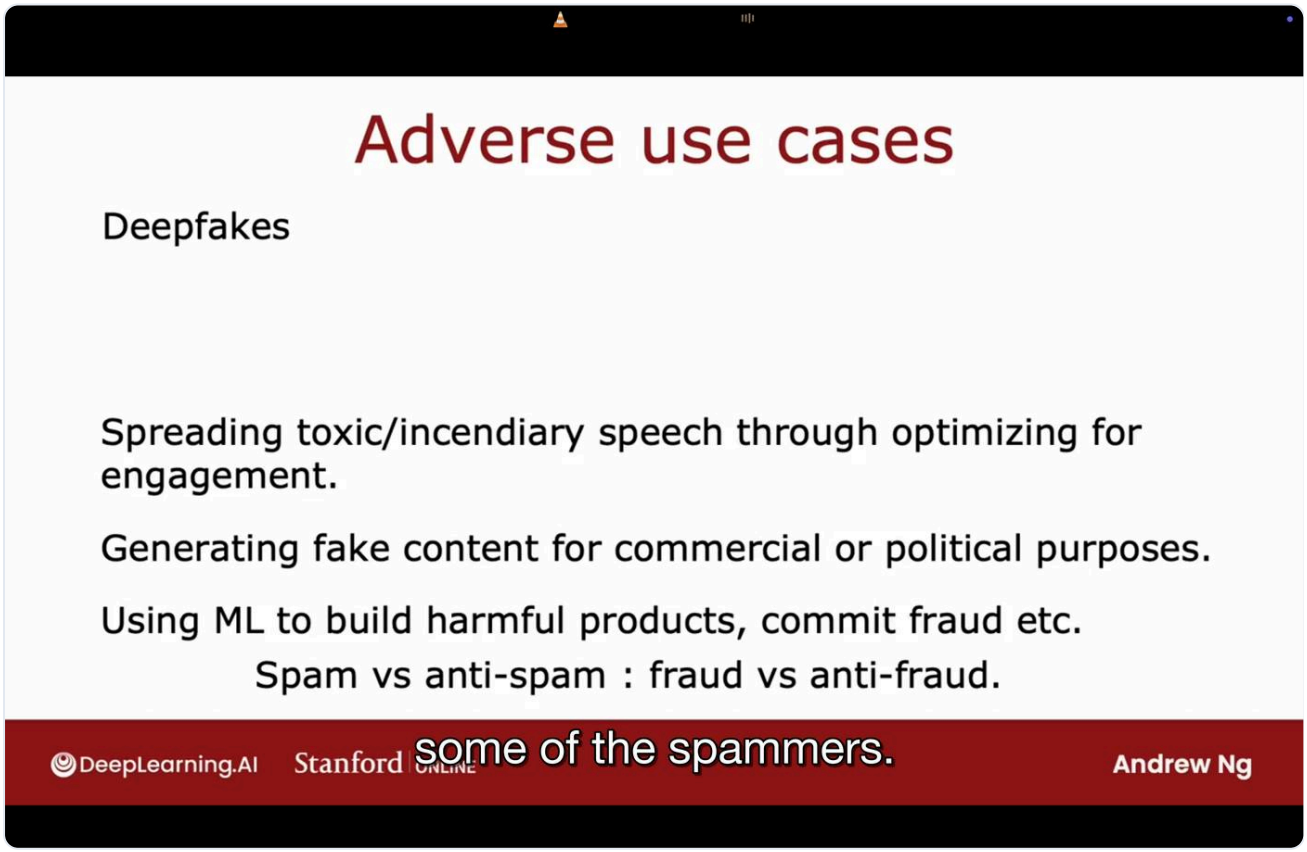


FIGURE 47.6 | Adverse Use Cases

And there are users of machine learning to build harmful products, commit fraud and so on. And in parts of the machine learning world, just as an email, there has been a battle between the spammers and the anti spam community. I am seeing today in for example, the financial industry, a battle between people trying to commit fraud and the people fighting fraud. And unfortunately machine learning is used by some of the fraudsters and some of the spammers. So for goodness sakes please don't build a machine learning system that has a negative impact on society. And if you are asked to work on an application that you consider unethical, I urge you to walk away for what it's worth.

There have been multiple times that I have looked at the project that seemed to be financially sound. You'll make money for some company. But I have killed the project just on ethical grounds because I think that even though the financial case will sound, I felt that it makes the world worse off and I just don't ever want to be involved in a project like that. Ethics is a very complicated and very rich subject that humanity has studied for at least a few 1000 years. When aI became more widespread, I actually went and read up multiple books on philosophy and multiple books on ethics because I was hoping naively it turned out to come up with if only there's a checklist of five things we could do and so as we do these five things then we can be ethical, but I failed And I don't think anyone has ever managed to come up with a simple checklist of things to do to give that level of concrete guidance about how to be ethical.

So what I hope to share with you instead is not a checklist because I wasn't even come up with one with just some general guidance and some suggestions for how to make sure the work is less bias more fair and more ethical. And I hope that some of these guidance, some which would be relatively general will help you with your work as well. So here are some suggestions for making your work more fair, less biased and more ethical when before deploying a system that could create harm. I will usually try to assemble a diverse team to brainstorm possible things that might go wrong with an emphasis on possible harm. Two vulnerable groups I found many times in my life that having a more diverse team and by diverse I mean, diversity on multiple dimensions ranging from gender to ethnicity to culture, to many other traits.

I found that having more diverse teams actually causes a team collectively to be better at coming up with ideas about things that might go wrong and it increases the odds that will recognize the problem and fix it before rolling out the system and having that cause harm to some particular group. In addition to having a diverse team carrying out brainstorming. I have also found it useful to carry out a literature search on any standards or guidelines for your industry or particular application area, for example, in the financial industry, there are starting to be established standards for what it means to be a system. So they want that decides who to approve loans to, what it means for a system like that to be reasonably fair and free from bias and those standards that still emerging in different sectors could inform your work depending on what you're working on.

After identifying possible problems. I found it useful to then audit the system against this identified dimensions of possible home. Prior to deployment, you saw in the last video, the full cycle of machine learning project. And one key step that's often a crucial line of defense against deploying something problematic is after you've trained the model. But before you deployed in production, if the team has brainstormed, then it may be biased against certain subgroups such as certain genders or certain ethnicities.

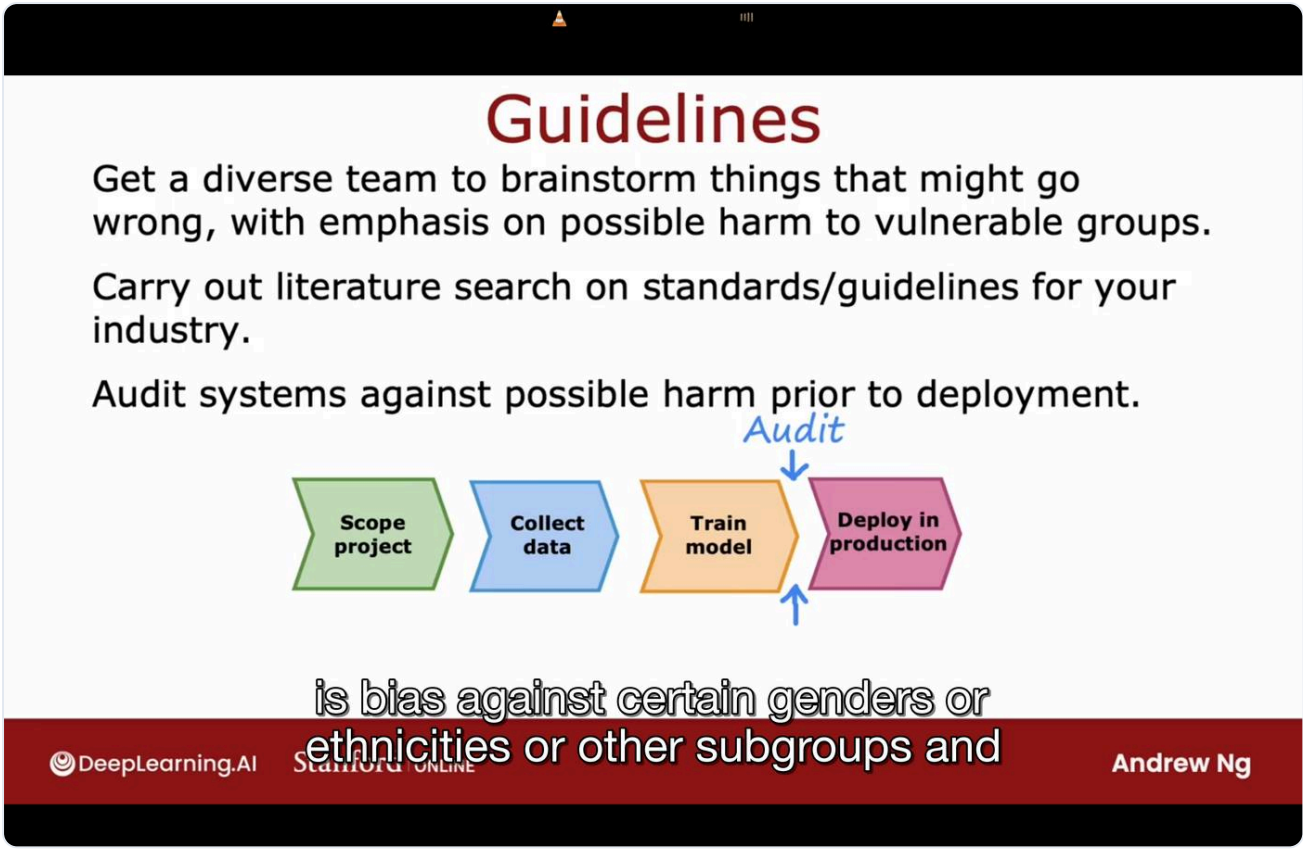


FIGURE 47.7 | Carry Out Literature Search on Standards/guidelines for Your

You can then order the system to measure the performance to see if it really is bias against certain genders or ethnicities or other subgroups and to make sure that any problems are identified and fixed. Prior to deployment. Finally, I found it useful to develop a mitigation plan if applicable.

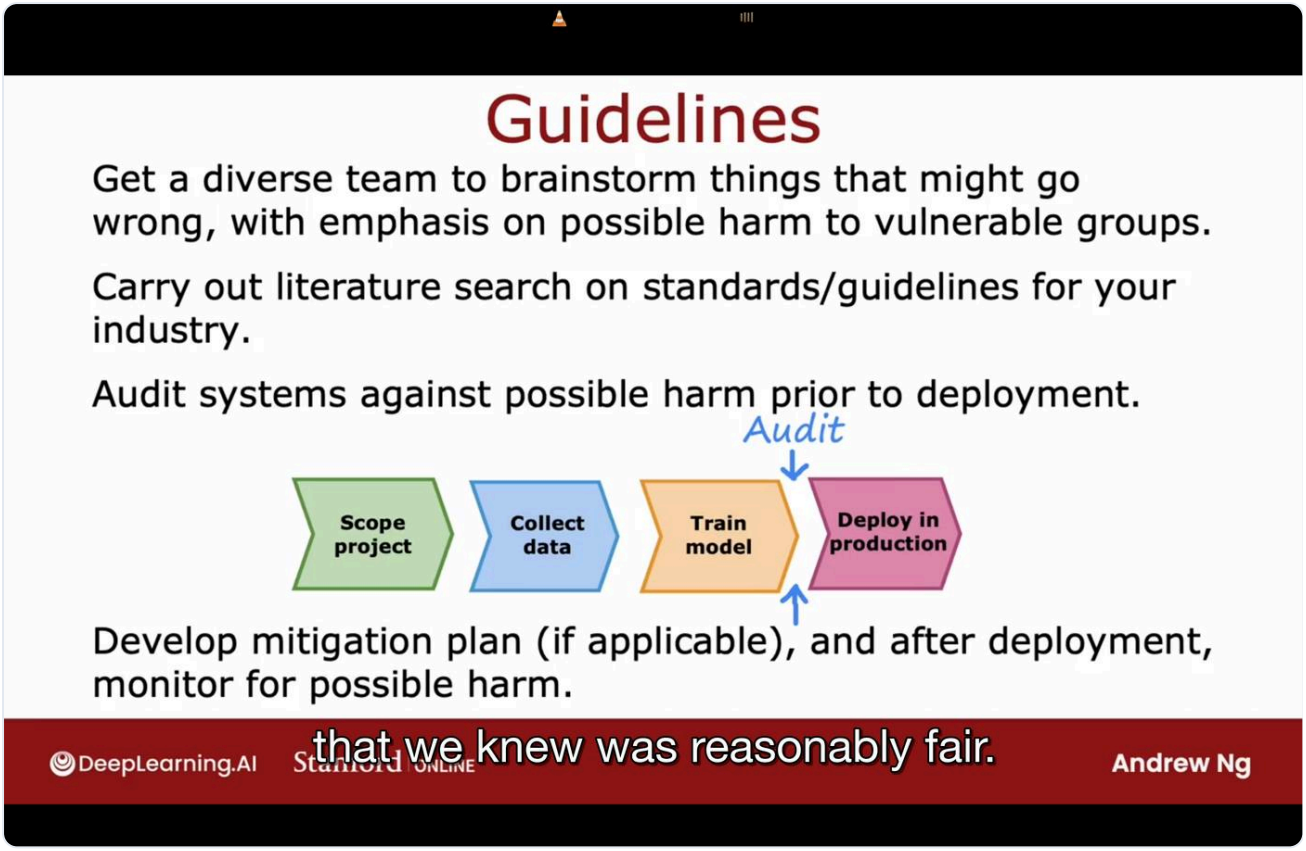


FIGURE 47.8 | Carry Out Literature Search on Standards/guidelines for Your

And one simple mitigation plan would be to roll back to the earlier system that we knew was reasonably fair. And then even after deployment to continue to monitor harm so that you can then trigger a mitigation plan and act quickly in case there is a problem that needs to be addressed. For example, all of the self driving car teams prior to rolling out self driving cars on the road had developed mitigation plans for what to do in case the car ever gets involved in an accident so that if the car was ever in an accident, there was already a mitigation plan that they could execute immediately rather than have a car got into an accident and then only scramble after the fact to figure out what to do.

I've worked on many machine learning systems and let me tell you the issues of ethics, fairness and bias issues we should take seriously. It's not something to brush off. It's not something to take lightly. Now of course, there's some projects with more serious ethical implications than others. For example, if I'm building a neural network to decide how long to roast my coffee beans, clearly, the ethical implications of that seems significantly less than if, say you're building a system to decide what loans. Bank loans are approved, which if it's bias can cause significant harm. But I hope that all of us collectively working in machine learning can keep on getting better debate these issues.

Spot problems, fix them before they cause harm so that we collectively can avoid some of the mistakes that the machine learning world had made before because this stuff matters and the systems we built can affect a lot of people. And so that's it on the process of developing a machine learning system and congratulations on getting to the end of this week's required videos. I have

just two more optional videos this week for you on addressing skewed data sets and that means data sets where the ratio of positive To negative examples is very far from 50, 50. And it turns out that some special techniques are needed to address machine learning applications like that.

So I hope to see you in the next video optional video on how to handle skewed data sets.